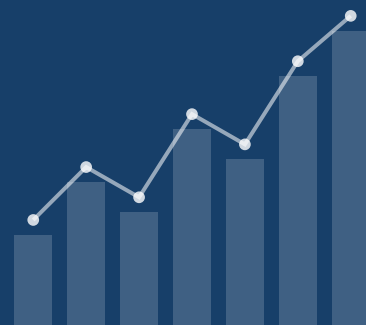


CURSO PRÁCTICO

Business Intelligence con R y RStudio

Del primer script al dashboard ejecutivo



Principiante ➤ **Intermedio** ➤ **Avanzado**

- | | |
|---------------------------------------|---------------------------------|
| ■ Fundamentos de R | ■ Bases de datos y SQL |
| ■ Manipulación con dplyr y tidyr | ■ Dashboards con Shiny |
| ■ Visualización con ggplot2 | ■ Machine learning para negocio |
| ■ Estadística y análisis exploratorio | ■ Reportes automáticos |

Código comentado ■ Ejemplos de negocio resueltos ■ Ejercicios con soluciones ■ Proyecto final

Edición 2026

Contenido

Cómo usar este libro	VIII
1. Bienvenida	VIII
2. ¿Qué es Business Intelligence?	IX
3. ¿Por qué R y RStudio para BI?	XII
4. El ciclo de trabajo de un proyecto de BI	XIV
5. Cómo está organizado el libro	XV
6. Convenciones del libro	XVII
7. Preparar tu entorno de trabajo	XXIII
8. Generar los datos del curso	XXVIII
9. Diccionario de datos	XXXII
10. El código del libro y cómo compilarlo	XL
11. Consejos para aprender	XLI
12. Cómo pedir ayuda	XLI
I Nivel principiante	1
1. Fundamentos de R y RStudio	2
1.1. Tu primera sesión en RStudio	2
1.2. R como calculadora de negocio	8
1.3. Variables: guarda tus resultados con nombre	12
1.4. Tipos de datos	16
1.5. Vectores: muchos datos a la vez	23
1.6. Trabajar con texto	31
1.7. Data frames y tibbles: tus hojas de cálculo en R	34
1.8. Listas y matrices	41
1.9. Control de flujo: decisiones y repeticiones	44
1.10. Tus propias funciones: una caja de herramientas BI	48

1.11. Paquetes: superpoderes para R	52
1.12. Importar y exportar datos	54
1.13. Cómo obtener ayuda	60
1.14. Caso práctico: tu primer análisis de ventas	63
1.15. Errores comunes y cómo resolverlos	68
1.16. Resumen del módulo	71
1.17. Ejercicios	72
2. Manipulación de datos con dplyr y tidyr	77
2.1. Preparar los datos: el 80 % del trabajo	78
2.2. Importar datos con readr	80
2.3. El pipe: encadenar pasos como una receta	86
2.4. Los verbos de dplyr	88
2.5. Funciones de ventana para KPIs	111
2.6. Fechas con lubridate	115
2.7. Texto con stringr	121
2.8. Unir tablas: los <i>joins</i>	124
2.9. Cambiar la forma de las tablas con tidyr	133
2.10. Limpieza de datos: un caso real paso a paso	140
2.11. Caso práctico integral: reporte mensual de ventas 2023	144
2.12. Si vienes de Excel o de SQL	153
2.13. Errores comunes	154
2.14. Ejercicios	158
3. Visualización de datos con ggplot2	162
3.1. Por qué visualizar	163
3.2. La gramática de gráficos	165
3.3. Elegir la gráfica correcta	170
3.4. Barras y columnas: comparar categorías	171
3.5. Líneas: la evolución en el tiempo	178
3.6. Dispersión: la relación entre dos medidas	184
3.7. Distribuciones: cómo se reparten los datos	187

3.8. Mapa de calor: dos categorías y un valor	191
3.9. Facetas: muchas gráficas pequeñas	193
3.10. Escalas y formato de negocio	195
3.11. Temas y diseño corporativo	198
3.12. Buenas y malas prácticas: antes y después	202
3.13. Exportar gráficas	206
3.14. Gráficas interactivas con plotly	207
3.15. Caso práctico: tablero de ventas 2023 para la dirección	208
3.16. Errores comunes	214
3.17. Ejercicios	217

II Nivel intermedio 220

4. Análisis exploratorio y estadística para negocios 221

4.1. ¿Qué es el análisis exploratorio (EDA)?	222
4.2. Estadística descriptiva	227
4.3. Forma de la distribución	232
4.4. Datos atípicos (outliers)	236
4.5. KPIs y métricas de desempeño	241
4.6. Análisis de Pareto y clasificación ABC	245
4.7. Correlación: ¿qué variables se mueven juntas?	249
4.8. Probabilidad y distribuciones útiles	253
4.9. Muestreo e intervalos de confianza	259
4.10. Pruebas de hipótesis para negocios	262
4.11. Regresión lineal	272
4.12. Caso práctico integral: diagnóstico comercial	279
4.13. Errores comunes	283
4.14. Resumen del módulo	288
4.15. Ejercicios	289

5. Bases de datos, SQL y ETL desde R 292

5.1. ¿Por qué bases de datos en BI?	293
---	-----

5.2. Conceptos básicos de una base de datos relacional	294
5.3. Conectarte a una base de datos con DBI y RSQLite	296
5.4. SQL desde cero con preguntas de negocio	301
5.5. Consultas parametrizadas e inyección SQL	317
5.6. dbplyr: usar dplyr contra la base de datos	321
5.7. Diseño de un data warehouse en estrella	324
5.8. ETL con R: extraer, transformar y cargar	331
5.9. Vistas e índices: el data warehouse listo para BI	339
5.10. Trabajar con datos grandes	341
5.11. Conectarte a las bases de datos de tu empresa	343
5.12. Caso práctico integral: el data warehouse de ventas	346
5.13. Errores comunes	353
5.14. Ejercicios	357
6. Dashboards interactivos con Shiny	360
6.1. ¿Qué es un dashboard de BI y qué lo hace bueno?	361
6.2. Anatomía de una app Shiny	363
6.3. Entradas y salidas	369
6.4. Reactividad: el concepto más importante	373
6.5. Organizar la pantalla: layouts	379
6.6. Caso práctico 1: el explorador de ventas	381
6.7. Dashboards profesionales con shinydashboard	388
6.8. Caso práctico 2: el dashboard ejecutivo	389
6.9. Descargas, tablas interactivas y gráficas interactivas	403
6.10. Buenas prácticas profesionales	406
6.11. Publicar tu dashboard	409
6.12. Errores comunes	410
6.13. Ejercicios	413
III Nivel avanzado	416
7. Machine learning para Business Intelligence	417

7.1. ¿Qué es machine learning para negocio?	418
7.2. Conceptos clave antes de entrenar	422
7.3. Caso 1: segmentación de clientes con RFM y k-means	428
7.4. Caso 2: regresión para predecir ventas	443
7.5. Caso 3: clasificación para predecir el abandono de clientes	452
7.6. Caso 4: pronóstico de demanda con series de tiempo	470
7.7. Caso 5: análisis de canasta (market basket)	480
7.8. Llevar el modelo a producción	487
7.9. Errores comunes	492
7.10. Resumen del módulo	495
7.11. Ejercicios	496
8. Reportes automáticos con R Markdown	499
8.1. El problema: el reporte mensual hecho a mano	500
8.2. Anatomía de un documento R Markdown	502
8.3. Opciones de los bloques de código	507
8.4. Tablas profesionales con <code>knitr::kable()</code>	509
8.5. Texto dinámico: frases que se escriben solas	513
8.6. Formatos de salida: HTML, PDF y Word	517
8.7. Reportes parametrizados	519
8.8. Automatización: un reporte por tienda, sin tocar nada	523
8.9. Reportes en Excel con writexl	531
8.10. Quarto: el sucesor de R Markdown	533
8.11. Buenas prácticas para reportes automáticos	535
8.12. Caso práctico integral: reporte ejecutivo mensual automático	536
8.13. Errores comunes	544
8.14. Ejercicios	547
IV Proyecto integrador	550
9. Proyecto final: TechRetail México	551
9.1. El caso: TechRetail México	552

9.2. Fase 1: ETL y modelo de datos	558
9.3. Fase 2: KPIs y análisis exploratorio	570
9.4. Fase 3: segmentación y predicción	585
9.5. Fase 4: el tablero ejecutivo	593
9.6. Fase 5: el reporte ejecutivo	595
9.7. Fase 6: presentar los resultados	598
9.8. El script maestro y la reproducibilidad	599
9.9. Errores comunes en un proyecto de BI	601
9.10. Checklist de entrega y portafolio	601
9.11. Retos de extensión	603
A. Soluciones de los ejercicios	606
A.1. Módulo 1. Fundamentos de R y RStudio	606
A.2. Módulo 2. Manipulación de datos con dplyr y tidyr	620
A.3. Módulo 3. Visualización de datos con ggplot2	631
A.4. Módulo 4. Análisis exploratorio y estadística	644
A.5. Módulo 5. Bases de datos, SQL y ETL	654
A.6. Módulo 6. Dashboards interactivos con Shiny	667
A.7. Módulo 7. Machine learning para BI	680
A.8. Módulo 8. Reportes automáticos	694
A.9. Módulo 9. Proyecto final: retos de extensión	706
B. Referencia rápida	718
B.1. Atajos de teclado de RStudio	719
B.2. R base esencial	720
B.3. Importar y exportar datos	725
B.4. dplyr: manipulación de datos	726
B.5. tidyr, lubridate y stringr	729
B.6. ggplot2: visualización	731
B.7. Equivalencias SQL, dplyr y Excel	734
B.8. Estadística y KPIs	736
B.9. Machine learning	740

B.10.Shiny y R Markdown	742
B.11.Errores frecuentes de R	746
B.12.Glosario de BI y analítica	749
B.13.Recursos para seguir aprendiendo	752

Cómo usar este libro

Al terminar esta introducción podrás...

- Explicar qué es Business Intelligence, qué es Business Analytics y cuáles son los cuatro tipos de analítica.
- Decidir cuándo conviene usar R y cuándo Excel, Power BI, Tableau o Python.
- Describir el ciclo de trabajo de un proyecto de BI y saber en qué módulo del libro se aprende cada paso.
- Leer los bloques de código, las salidas de R y las cajas didácticas del libro.
- Instalar R, RStudio y los paquetes del curso, y crear tu Proyecto de RStudio.
- Generar los datos del curso y conocer cada tabla, cada columna y cómo se relacionan entre sí.

1 Bienvenida

Lunes, 9:00 de la mañana. La directora comercial de una cadena de tiendas te escribe: “Necesito saber qué sucursal vendió más el último trimestre, qué productos nos están dejando sin margen y si los descuentos de verdad aumentan las ventas. Lo presento el jueves”.

Si trabajas con hojas de cálculo ya sabes lo que viene: exportar archivos de varios sistemas, copiar y pegar columnas, corregir fechas que se leyeron mal, armar tablas dinámicas, hacer gráficas una por una... y el lunes siguiente repetir todo desde cero porque llegaron datos nuevos. Este libro te enseña a hacer ese mismo trabajo con **R** y **RStudio**: escribes una sola vez las instrucciones en un archivo de código (un **script**) y, a partir de entonces, el análisis completo se repite en segundos, sin errores de copiar y pegar y con gráficas de calidad profesional.

Es un **curso práctico**. Todo gira alrededor de una cadena minorista (**retail**) ficticia con diez sucursales en México: sus ventas de 2023, sus clientes, su catálogo de productos, sus empleados y sus tiendas. Con esos datos resolverás, paso a paso, las mismas preguntas que se hacen a diario en las áreas comerciales, de finanzas, de operaciones y de marketing de cualquier empresa. Cada ejemplo sigue el mismo patrón: una pregunta de negocio, el código comentado línea por línea, la salida real de R y la interpretación (“¿qué nos dice esto?”). Después te toca a ti.

1.1 Lo que sabrás hacer al terminar

Al final del libro podrás hacer el trabajo diario de un analista de Business Intelligence:

- **Reunir datos** de archivos CSV, libros de Excel y bases de datos SQL, y **unir** tablas de ventas, clientes, productos y tiendas.
- **Limpiar y transformar** datos con scripts reproducibles: lo que hoy te toma una hora de trabajo manual se ejecutará con un clic.
- **Calcular indicadores** (ventas, ticket promedio, margen, crecimiento, participación) por tienda, mes, categoría o vendedor.
- **Crear gráficas** claras y con estilo corporativo, listas para una presentación ejecutiva.
- **Explorar datos con estadística**: detectar valores atípicos, medir relaciones y comprobar si una diferencia es real o casualidad.
- **Construir dashboards interactivos** con Shiny que tus compañeros pueden usar desde el navegador.
- **Segmentar clientes y hacer pronósticos** con técnicas de machine learning.
- **Automatizar reportes** en HTML, Word y PDF que se actualizan solos cuando llegan datos nuevos.
- **Integrarlo todo** en un proyecto final: un dashboard ejecutivo de *retail analytics*.

1.2 ¿Para quién es este libro y qué necesitas?

No necesitas saber programar. Basta con que manejes una computadora y tengas nociones de hoja de cálculo (qué es una fila, una columna, una suma). Si ya programas en otro lenguaje, avanzarás rápido por el primer módulo; si ya usas R, aprovecharás sobre todo la parte de negocio: indicadores, dashboards, reportes automáticos y modelos aplicados.

Necesitas una computadora con Windows 10/11, macOS o Linux, al menos 4 GB de memoria RAM (8 GB es más cómodo), unos 2 GB libres en disco, conexión a internet para instalar el software y, sobre todo, constancia: el plan sugerido de la sección 5.2 pide unas 8 horas por semana durante 10 semanas. Todo el software que usaremos es gratuito.

2 ¿Qué es Business Intelligence?

Concepto clave: Business Intelligence y Business Analytics

Business Intelligence (BI, inteligencia de negocios) es el conjunto de procesos, herramientas y prácticas que convierten los datos que genera una empresa —ventas, inventarios, clientes, nómina— en información confiable para tomar decisiones. Sus productos típicos son reportes, indicadores y tableros que responden *qué pasó* y *cómo vamos*.

Business Analytics (BA, analítica de negocios) va un paso más allá: usa estadística y modelos para explicar *por qué* pasó algo, anticipar *qué pasará* y recomendar *qué hacer*.

En la práctica la frontera es difusa y muchas empresas llaman “BI” a todo. En este libro aprenderás las dos cosas.

Dos términos que usarás en todo el libro. Un **KPI** (*Key Performance Indicator*, indicador clave de desempeño) es una medida numérica que dice si el negocio avanza hacia un objetivo: ventas del mes, ticket promedio, margen bruto, porcentaje de clientes que regresan. Un **dashboard** (tablero) es una pantalla que reúne varios KPI y gráficas para dar un vistazo rápido al estado del negocio.

2.1 Los cuatro tipos de analítica

Las preguntas que se hacen con datos se suelen agrupar en cuatro tipos de analítica, de menor a mayor complejidad y, en general, de menor a mayor valor para la decisión (figura 1). No es una escalera que se sube una sola vez: una empresa madura usa los cuatro tipos al mismo tiempo, y la analítica descriptiva, bien hecha, sigue siendo la base de todo.

La tabla 1 aterriza cada tipo con ejemplos de nuestra cadena de tiendas y te dice en qué módulo lo aprenderás.

Tabla 1. Los cuatro tipos de analítica aplicados a una cadena de tiendas.

Tipo	Ejemplo en nuestra cadena	Técnicas en R	Módulo
Descriptiva	Ventas totales por mes y por tienda en 2023; los 10 productos más vendidos; ticket promedio por método de pago.	Resúmenes con dplyr , gráficas con ggplot2 .	2, 3
Diagnóstica	¿Bajaron las ventas de una sucursal porque hubo menos compras o porque cada compra fue más pequeña? ¿Los clientes premium gastan más?	Comparación de grupos, correlación, pruebas de hipótesis, regresión.	4
Predictiva	Pronóstico de ventas del próximo trimestre; probabilidad de que un cliente deje de comprar.	Series de tiempo (forecast), árboles y bosques aleatorios.	7
Prescriptiva	Qué productos reabastecer primero; a qué segmento de clientes enviar la promoción.	Reglas de decisión sobre predicciones y segmentos, comparación de escenarios.	7, 9

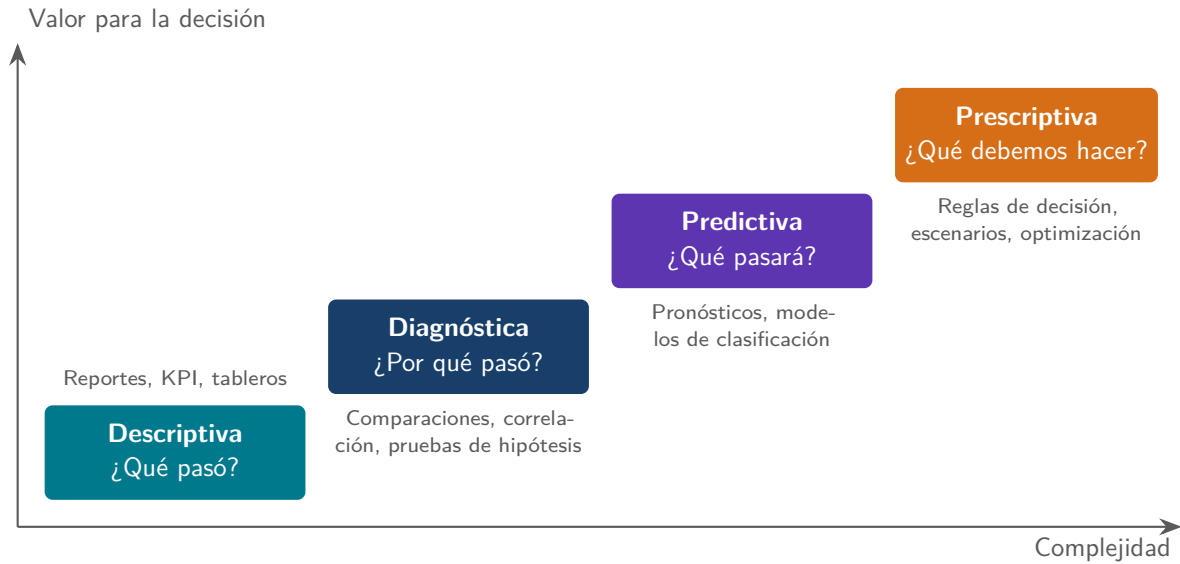


Figura 1. Los cuatro tipos de analítica. Cada escalón responde una pregunta más difícil y, por lo general, más valiosa para la decisión.

Siendo honestos: la analítica prescriptiva “completa” (optimización matemática de precios, rutas o inventarios) es un campo propio que va más allá de este libro. Aquí harás la versión que más se usa en el día a día: convertir segmentos y pronósticos en reglas de decisión claras y comparar escenarios.

2.2 Roles de datos en una empresa

Alrededor de los datos trabajan varios perfiles. Conocerlos te ayuda a entender qué parte del trabajo te toca y con quién colaboras (tabla 2).

Tabla 2. Roles de datos más comunes y la pregunta típica de cada uno.

Rol	Qué hace	Herramientas	Pregunta típica
Analista de BI	Traduce preguntas de negocio en indicadores; construye reportes y dashboards periódicos.	SQL, Excel, Power BI o Tableau, R.	¿Cómo vamos contra la meta del mes?
Analista de datos	Responde preguntas puntuales con análisis a la medida y estadística.	SQL, R o Python, Excel.	¿Qué tipo de cliente responde mejor a los descuentos?
Científico de datos	Construye modelos predictivos y diseña experimentos.	R o Python, machine learning.	¿Qué clientes tienen alta probabilidad de irse?
Ingeniero de datos	Construye y mantiene las “tuberías” que llevan los datos de los sistemas al almacén de datos.	SQL, Python, herramientas de ETL, servicios en la nube.	¿Llegaron completos y a tiempo los datos de ventas de ayer?

En la empresa: una persona, varios sombreros

En una empresa grande cada rol es un equipo distinto. En una empresa mediana o pequeña es común que una sola persona extraiga los datos, los limpie, haga el análisis y presente el dashboard. Este libro te forma principalmente como **analista de BI y de datos**, con una introducción sólida a la ciencia de datos (Módulo 7) y a la parte de ingeniería que todo analista necesita: bases de datos, SQL y procesos ETL (Módulo 5).

2.3 Preguntas de negocio que resuelve BI

BI no empieza con los datos, sino con una pregunta. La tabla 3 reúne preguntas típicas por área de la empresa y la tabla del curso con la que podrás responderlas.

Tabla 3. Preguntas de negocio por área y datos del curso que las responden.

Área	Preguntas típicas	Datos del curso
Ventas	¿Qué tiendas venden más? ¿Cómo evolucionan las ventas mes a mes? ¿Qué día de la semana vendemos más?	<code>ventas_retail</code> , <code>transacciones</code>
Clientes	¿Quiénes son los mejores clientes? ¿Los clientes premium compran más? ¿Qué segmentos existen?	<code>clientes</code> + ventas
Productos e inventario	¿Qué productos tienen margen negativo? ¿Cuáles están por debajo del stock mínimo? ¿Qué categoría aporta más?	<code>productos</code> + ventas
Tiendas	¿Cuánto vende cada tienda por metro cuadrado? ¿Las tiendas más antiguas venden más?	<code>tiendas</code> + ventas
Recursos humanos	¿Cómo se distribuyen los salarios por departamento, puesto y nivel educativo? ¿Qué vendedor vende más?	<code>empleados</code> + ventas
Finanzas	¿Cuánto dinero dejamos de cobrar por descuentos? ¿Cómo cambia el ingreso por trimestre?	<code>ventas_retail</code>
Marketing	¿Qué método de pago prefieren los clientes? ¿A qué segmento dirigimos la próxima campaña?	<code>transacciones</code> , <code>clientes</code>

Hazlo tú

Escribe tres preguntas que se hagan en tu trabajo (o en un negocio que conozcas) y clasifícalas como descriptiva, diagnóstica, predictiva o prescriptiva. Guárdalas: al terminar el libro podrás responderlas con R.

3 ¿Por qué R y RStudio para BI?

R es un lenguaje de programación gratuito y de código abierto creado específicamente para el análisis de datos y la estadística (nació en los años noventa en la Universidad de Auckland, Nueva Zelanda). Su fuerza está en sus **paquetes**: extensiones gratuitas que agregan funciones nuevas. En

el repositorio oficial, **CRAN**, hay más de 20 000. Entre ellos destaca el **tidyverse**, una colección de paquetes con una forma de escribir código muy legible, casi como una receta: “toma las ventas, filtra 2023, agrupa por tienda, suma el total”.

Concepto clave: R y RStudio no son lo mismo

R es el motor: el programa que ejecuta los cálculos. **RStudio** es el tablero del coche: un programa (un **IDE**, entorno de desarrollo integrado) que te da un editor de código, una consola, un visor de gráficas, un explorador de archivos y la ayuda, todo en una ventana. RStudio lo desarrolla la empresa Posit y necesita que R esté instalado. Tú trabajarás siempre dentro de RStudio.

3.1 Comparación honesta con otras herramientas

R no es la única herramienta de BI ni la mejor para todo. La tabla 4 resume fortalezas y limitaciones de las opciones más comunes.

Tabla 4. R frente a otras herramientas de análisis de datos.

Herramienta	Fortalezas	Limitaciones	Úsala cuando...
Excel	Está en todas las empresas; todo mundo lo sabe leer; ideal para cálculos rápidos y tablas dinámicas.	Máximo 1 048 576 filas por hoja; los procesos de copiar y pegar son difíciles de repetir y de auditar; errores de fórmula silenciosos; estadística limitada.	El análisis es pequeño y único, o necesitas entregar resultados a usuarios de negocio.
Power BI y Tableau	Dashboards interactivos arrastrando y soltando; publicación con permisos para toda la organización; muchos conectores; actualización programada.	Licencias de pago para compartir; transformaciones complejas y estadística avanzada son difíciles; lenguajes propios (DAX, cálculos) con su propia curva.	La empresa ya lo tiene y necesitas un tablero corporativo de consulta diaria para muchas personas.
Python	Lenguaje de propósito general; líder en machine learning a gran escala, deep learning e integración con otros sistemas.	Para estadística y gráficas de reporte suele requerir más código y más librerías distintas; configurar el entorno es más confuso al principio.	El modelo irá a producción dentro de una aplicación o tu equipo ya trabaja en Python.
R y RStudio	Diseñado para analizar datos; código legible con el tidyverse; gráficas de calidad con ggplot2 ; reportes reproducibles; dashboards con Shiny; gratuito.	Hay que programar (curva inicial); los datos se cargan en la memoria RAM; se usa menos para desarrollar software; publicar dashboards Shiny requiere un servidor.	Necesitas limpiar, analizar, modelar y reportar de forma reproducible y automatizable.

La gran ventaja de R para BI se resume en una palabra: **reproducible**. Un análisis es reproducible cuando cualquier persona —incluido tú dentro de seis meses— puede volver a ejecutarlo y obtener exactamente el mismo resultado. En Excel, los pasos viven en tu memoria (“primero filtré, luego

copié esta columna...”); en R, viven en un script que se puede revisar, corregir, compartir y volver a ejecutar con los datos del mes siguiente.

Consejo

R **complementa** a tus otras herramientas; no las sustituye. Un analista eficaz usa cada una para lo que hace mejor. Con R puedes exportar resultados a Excel para el área de finanzas (Módulo 1), alimentar un tablero de Power BI (que puede ejecutar scripts de R como fuente de datos) o generar el reporte en Word que pide dirección (Módulo 8). Aprender R no te obliga a abandonar nada.

4 El ciclo de trabajo de un proyecto de BI

Todo proyecto de BI, desde un reporte semanal hasta un modelo de pronóstico, sigue un ciclo parecido (figura 2). Es un ciclo porque la medición de resultados casi siempre genera una pregunta nueva. El libro está organizado para que aprendas cada paso; debajo de cada uno verás el módulo donde lo practicas.

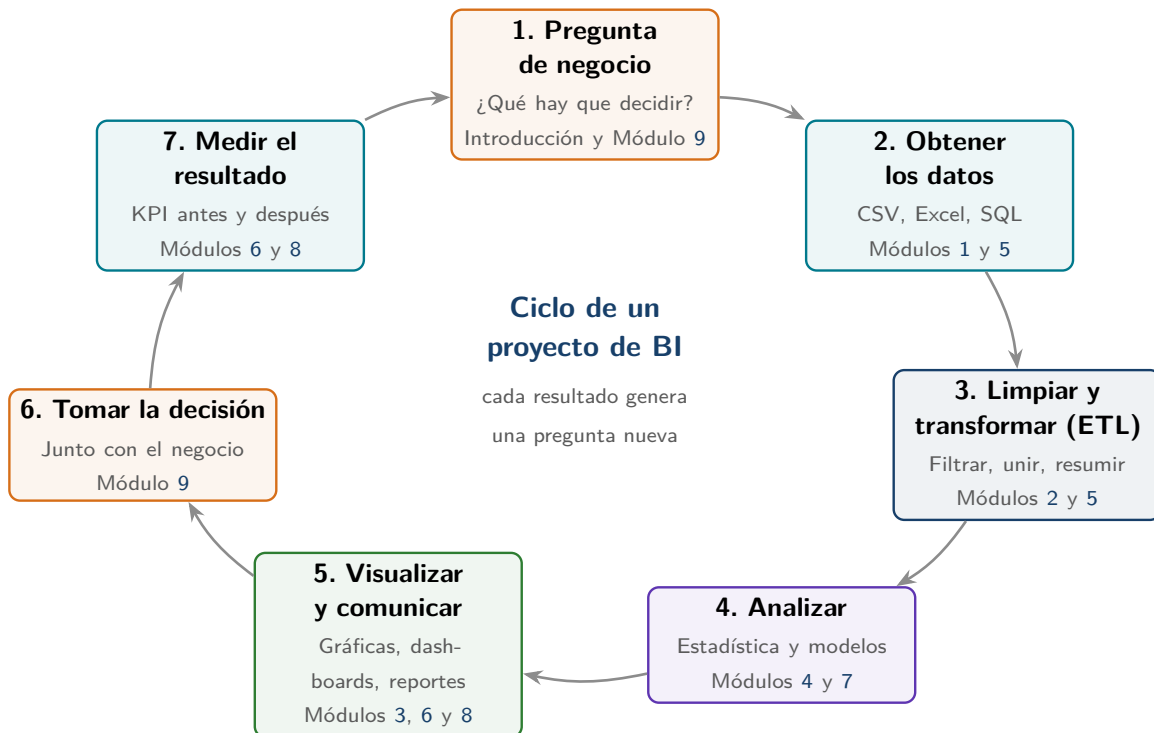


Figura 2. El ciclo de trabajo de un proyecto de BI y el módulo del libro donde se aprende cada paso.

Ejemplo práctico: el ciclo completo en una pregunta real

La directora comercial pregunta: “¿Los descuentos aumentan las unidades vendidas o solo nos quitan margen?”. Así se vería el ciclo:

1. **Pregunta:** ¿las ventas con 15 o 20 % de descuento venden más unidades que las ventas sin descuento?
2. **Datos:** `ventas_retail.csv`, columnas `descuento_pct`, `cantidad` y `total`.
3. **Limpiar:** comprobar que `total` sea igual a `subtotal` menos `descuento_monto` y que no haya valores faltantes.
4. **Analizar:** comparar las unidades promedio por nivel de descuento y probar si la diferencia es estadísticamente significativa (Módulo 4).
5. **Comunicar:** una gráfica de barras con las unidades promedio por nivel de descuento y una frase con la conclusión.
6. **Decidir:** si los descuentos no mueven las unidades, se reducen a las fechas clave.
7. **Medir:** comparar unidades y margen del trimestre siguiente contra el anterior.

Hazlo tú

Piensa en una decisión reciente de tu trabajo (un precio, una compra, una contratación). Escribe cómo habría sido cada uno de los siete pasos del ciclo. ¿Qué datos habrías necesitado? ¿Cómo habrías medido el resultado?

5 Cómo está organizado el libro

El libro tiene cuatro partes que van de principiante a avanzado, más dos apéndices (tabla 5). Cada módulo se apoya en los anteriores, así que te recomendamos seguirlos en orden, al menos la primera vez.

Tabla 5. Partes, módulos, nivel y horas estimadas de estudio y práctica.

Parte	Módulo	Tema	Nivel	Horas
I. Nivel principiante	1	Fundamentos de R y RStudio: objetos, vectores, tablas, funciones, importar y exportar CSV y Excel.	Principiante	8
	2	Manipulación de datos con dplyr y tidyr : filtrar, crear columnas, agrupar, resumir, unir y pivotar.	Principiante	10
	3	Visualización con ggplot2 : barras, líneas, dispersión, distribuciones y tema corporativo.	Principiante	8
II. Nivel intermedio	4	Análisis exploratorio y estadística: resúmenes, distribuciones, atípicos, correlación, pruebas y regresión.	Intermedio	8
	5	Bases de datos y SQL: SQLite desde R, consultas SQL, dbplyr y procesos ETL.	Intermedio	6
	6	Dashboards interactivos con Shiny y shinydashboard .	Intermedio	8
III. Nivel avanzado	7	Machine learning para BI: segmentación, clasificación, regresión y series de tiempo.	Avanzado	10
	8	Reportes automáticos con R Markdown en HTML, Word y PDF.	Avanzado	6
IV. Proyecto integrador	9	Proyecto final: dashboard ejecutivo de <i>retail analytics</i> , de los datos crudos al reporte.	Integrador	12
Apéndices	A	Soluciones completas de todos los ejercicios.	—	—
	B	Referencia rápida de funciones y atajos.	—	—
Total aproximado				76

5.1 Estructura de cada módulo

Todos los módulos siguen la misma estructura, para que sepas siempre dónde estás:

1. **Objetivos** y un escenario de negocio que motiva el tema.
2. **Secciones de contenido**: cada idea se presenta con un concepto breve, un ejemplo con código comentado, la salida real de R, su interpretación y un reto “Hazlo tú”.
3. **Casos prácticos** completos, de la pregunta a la conclusión.
4. **Errores comunes**: los mensajes de error que verás con más frecuencia y cómo resolverlos.
5. **Resumen** con lo aprendido y una tabla de funciones clave.
6. **Ejercicios** graduados por nivel (Básico, Intermedio, Avanzado y Reto). Cada ejercicio indica la página de su solución en el Apéndice A.

5.2 Plan de estudio sugerido de 10 semanas

Si estudias por tu cuenta, el plan de la tabla 6 reparte unas 80 horas (los módulos más esta introducción y la planeación del proyecto) en 10 semanas de 6 a 10 horas. Cada semana termina con

un **entregable**: algo concreto que puedes guardar en tu portafolio.

Tabla 6. Plan de estudio sugerido de 10 semanas.

Semana	Contenido	Entregable sugerido	Horas
1	Introducción y primera mitad del Módulo 1.	Entorno instalado, datos generados y un script con tus primeros cálculos.	7
2	Resto del Módulo 1 e inicio del Módulo 2.	Un script que importa los CSV del curso y exporta una tabla a Excel.	8
3	Resto del Módulo 2.	Tabla de ventas por tienda y categoría, con clientes y productos unidos.	8
4	Módulo 3.	Cinco gráficas de ventas con tu tema corporativo.	8
5	Módulo 4.	Informe exploratorio: distribuciones, atípicos y relaciones.	8
6	Módulo 5.	Base SQLite con las tablas del curso y cinco consultas SQL.	6
7	Módulo 6.	Dashboard Shiny de ventas con filtros.	8
8	Módulo 7.	Segmentación de clientes y pronóstico de ventas.	10
9	Módulo 8 y planeación del proyecto.	Reporte mensual automático en HTML y Word.	7
10	Módulo 9.	Proyecto final completo: ETL, análisis, dashboard y reporte.	10
Total aproximado			80



Consejo

Si tienes prisa por una necesidad concreta, estas rutas cortas funcionan: para **reportes y gráficas**, Módulos 1, 2, 3 y 8; para **dashboards**, añade el Módulo 6; para **modelos predictivos**, añade los Módulos 4 y 7. Los Módulos 1 y 2 son obligatorios en cualquier ruta: todo lo demás se construye sobre ellos.

6 Convenciones del libro

Antes de empezar conviene que sepas leer el libro: cómo se distingue el código que debes ejecutar, cómo se ve lo que R responde y para qué sirve cada caja de color. Si todavía no tienes R instalado, solo lee esta sección; en la sección 7 lo instalas y luego puedes volver a probar los ejemplos.

6.1 Bloques de código

El libro usa cinco tipos de bloque. El más importante es el de **código R que se ejecuta**: fondo gris claro y números de línea a la izquierda. Todo el código de estos bloques se probó de principio a fin, en orden, antes de publicar el libro.

```

1 # Ventas diarias de una sucursal durante una semana (en pesos)
2 ventas_semana <- c(1200, 950, 1830, 1100, 2400, 3100, 2800)
3 sum(ventas_semana)      # venta total de la semana
4 mean(ventas_semana)     # venta promedio por día

```

Justo debajo aparece la **salida**: lo que R imprime en la consola al ejecutar ese código. Tiene fondo blanco y una franja azul verdosa a la izquierda:

```

[1] 13380
[1] 1911.429

```

Los **números de línea** (1, 2, 3, 4 en gris) no forman parte del código: sirven para que el texto pueda decir “en la línea 2 creamos el vector” y tú sepas exactamente dónde mirar. No los escribas. Las líneas que empiezan con **#** son **comentarios**: R las ignora; están ahí para explicarte qué hace cada instrucción. En tus propios scripts, comenta igual. Si una línea es demasiado larga para el ancho de la página, el libro la parte y marca la continuación con el símbolo $\hookrightarrow$; en R escríbela en una sola línea.

El segundo tipo es el **código R que no se ejecuta automáticamente**. Se ve igual, pero con una franja naranja a la izquierda:

```

1 # Instala un paquete (se hace UNA sola vez en cada computadora)
2 install.packages("tidyverse")

```

La franja naranja significa “ejecútalo tú cuando lo necesites”: son instalaciones, aplicaciones Shiny que abren una ventana, conexiones a servidores que no existen en tu computadora, plantillas con valores ficticios que debes cambiar o código con errores intencionales para aprender a leerlos. Por eso casi nunca llevan salida debajo.

El tercer tipo es **SQL**, el lenguaje de las bases de datos, con una franja morada. Lo aprenderás en el Módulo 5:

```

-- Ventas totales por tienda, de mayor a menor
SELECT tienda_id, SUM(total) AS ventas_totales
FROM ventas
GROUP BY tienda_id
ORDER BY ventas_totales DESC;

```

Por último, los bloques de **texto literal** muestran cosas que no son código R: encabezados de R Markdown, comandos de terminal, estructuras de carpetas o archivos de configuración. Este, por ejemplo, es el encabezado de un reporte que harás en el Módulo 8:

```

1 ---
2 title: "Reporte mensual de ventas"
3 author: "Área de Inteligencia Comercial"

```

```
4 output: html_document
5 ---
```

▲ Cuidado

No copies y pegues código desde el PDF: se copian también los números de línea y a veces las comillas o los espacios cambian, lo que produce errores difíciles de ver. Escribe el código tú mismo (así aprendes más rápido) o abre el script del módulo en la carpeta `latex/codigo/` (sección 10).

6.2 Cómo leer una salida de R

En la consola de RStudio verás el símbolo `>` antes de cada instrucción: es el **prompt**, la forma en que R dice “estoy listo, escribe algo”. En el libro no lo mostramos: el bloque de código contiene exactamente lo que escribes y el bloque de salida, exactamente lo que R responde.

El `[1]` y los índices

Cada línea de salida de un vector empieza con un número entre corchetes. No es un dato: es la **posición** del primer elemento de esa línea. Se nota cuando el resultado no cabe en una sola línea:

```
1 # Números de día de un mes de 30 días
2 dias_mes <- 1:30
3 dias_mes
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
[23] 23 24 25 26 27 28 29 30
```

La primera línea empieza en el elemento 1 y la segunda en el elemento 23. En la salida de `sum(ventas_semana)`, el `[1]` solo indica que el resultado tiene un único elemento: 13380.

Tablas: data frames y tibbles

Los datos de negocio viven en tablas. En R, una tabla se llama **data frame**; el tidyverse usa una versión moderna llamada **tibble**, que se imprime de forma más informativa:

```
1 library(tibble) # paquete de tablas modernas (parte del tidyverse)
2
3 # Una tabla pequeña: ventas de tres tiendas
4 ventas_tienda <- tibble(
5   tienda = c("Centro", "Norte", "Sur"),
6   ventas = c(152300, 98750, 120400)
```

```
7 )
8 ventas_tienda
```

```
# A tibble: 3 x 2
  tienda ventas
  <chr>   <dbl>
1 Centro 152300
2 Norte  98750
3 Sur    120400
```

Así se lee:

- **# A tibble: 3 x 2:** la tabla tiene 3 filas y 2 columnas. En tu pantalla verás el signo $\times$ en lugar de la **x** y algunos elementos en color; en el libro usamos caracteres simples.
- La segunda línea tiene los nombres de las columnas y la tercera su **tipo** entre **< >** (tabla 7).
- Los números 1, 2, 3 de la izquierda son solo el número de fila.
- Si la tabla es grande, R muestra las primeras 10 filas y avisa con una línea como **# i 355 more rows** (quedan 355 filas más) o **# i 9 more variables: ...** (hay 9 columnas que no cupieron). En tu pantalla la **i** aparece como un símbolo de información. Los datos están completos: solo se recorta la impresión.
- Un número que termina en punto, como **126274.**, tiene decimales que no se muestran para ahorrar espacio.

Tabla 7. Tipos de columna que verás en las salidas de R.

Así aparece	Tipo	Ejemplo en los datos del curso
<chr>	Texto (<i>character</i>)	nombre de la ciudad, método de pago
<dbl>	Número decimal (<i>double</i>)	total de la venta, salario
<int>	Número entero (<i>integer</i>)	conteo de ventas
<lgl>	Lógico: TRUE o FALSE	¿el cliente es premium?
<date>	Fecha	fecha de la venta
<time>	Hora del día	hora de la transacción
<dtm>	Fecha y hora	momento exacto de un registro
<fct>	Factor: categoría con niveles fijos	segmento de cliente ordenado

Valores faltantes, advertencias y errores

En los datos reales faltan cosas. R marca un valor faltante con **NA** (*not available*). Mira qué pasa cuando una columna de ventas llega como texto y uno de los valores dice “N/D”:

```
1 # Ventas que llegaron como texto desde un sistema
2 ventas_texto <- c("1500", "2300", "N/D", "980")
3 ventas_numero <- as.numeric(ventas_texto) # convertir a número
```

R hace la conversión, pero responde en la consola con un aviso:

```
Warning message:
NAs introducidos por coerción
```

Veamos qué quedó y qué pasa al sumar:

```
1 ventas_numero
2 sum(ventas_numero)           # la suma con un NA da NA
3 sum(ventas_numero, na.rm = TRUE) # na.rm = TRUE ignora los NA
```

```
[1] 1500 2300  NA  980
[1] NA
[1] 4780
```

El texto “N/D” se convirtió en **NA**; la suma “normal” devuelve **NA** (si falta un dato, R no puede saber el total), y con **na.rm = TRUE** obtienes la suma de los valores disponibles, 4780.

Hay tres clases de avisos. Una **advertencia** (*warning*) significa “hice lo que pediste, pero algo raro pasó”: aquí R no pudo convertir “N/D” en número y puso **NA**. Un **error** (*error*) significa “me detuve y no hice nada”. Un **mensaje** (*message*) es solo informativo. Nunca ignores una advertencia sin entenderla: en BI suele ser la primera pista de un problema de calidad de datos.

Fíjate también en que el aviso salió mitad en inglés (*Warning message*) y mitad en español. R traduce muchos de sus mensajes según el idioma de tu sistema, así que en tu computadora pueden salir en español, en inglés o mezclados. No te preocupes: el significado es el mismo. Por último, cuando una salida es muy larga la recortamos y lo indicamos con una línea **...**

6.3 Las cajas didácticas

Además del código, el libro usa cajas de colores. Cada una tiene un propósito fijo; al verla sabrás qué tipo de contenido esperar. Al inicio de cada módulo, una caja azul de **objetivos** (como la de la primera página de esta introducción) lista lo que podrás hacer al terminarlo. Estas son las demás.

La caja **Concepto clave** contiene definiciones e ideas teóricas que necesitas para seguir adelante. Si vas a repasar un módulo en diez minutos, lee solo estas cajas:

Concepto clave: ticket promedio

El **ticket promedio** es el importe medio de cada compra: ventas totales divididas entre el número de compras (tickets). Si sube, tus clientes gastan más por visita; si baja, compran menos cosas o más baratas.

La caja **Ejemplo práctico** plantea un problema de negocio que se resuelve paso a paso. Normalmente contiene la pregunta y el código va justo después:

Ejemplo práctico: ticket promedio de la semana

Con las ventas de la semana (`ventas_semana`) y el número de tickets de cada día, ¿cuál fue el ticket promedio de la sucursal?

```
1 # Número de tickets (compras) registrados cada día de la semana
2 tickets_semana <- c(40, 31, 55, 38, 70, 88, 81)
3
4 # Ticket promedio = venta total / número total de tickets
5 ticket_promedio <- sum(ventas_semana) / sum(tickets_semana)
6 round(ticket_promedio, 2) # redondeamos a centavos
```

```
[1] 33.2
```

Interpretación: en esa semana cada cliente gastó en promedio \$33.20 por compra. Después de cada salida importante encontrarás una interpretación así: el número por sí solo no sirve de nada hasta que dices qué significa para el negocio.

La caja **En la empresa** cuenta cómo se usa una técnica en una organización real, con sus ventajas y sus problemas prácticos:

En la empresa: el reporte de los lunes

En muchas cadenas, cada lunes alguien arma a mano el reporte de ventas de la semana anterior: descarga archivos, los pega en Excel y actualiza gráficas. Con un script de R y un reporte de R Markdown (Módulo 8) ese trabajo se reduce a ejecutar un archivo, y el analista usa ese tiempo en lo que sí aporta valor: explicar qué pasó y qué hacer.

La caja **Consejo** reúne buenas prácticas profesionales, esas que no son obligatorias para que el código funcione pero que distinguen a un analista cuidadoso:

 Consejo

Nombra tus scripts con un número y un verbo que indiquen el orden en que se ejecutan: `01_importar.R`, `02_limpiar.R`, `03_analizar.R`. Dentro de un año agradecerás saber por dónde empezar.

La caja **Cuidado** advierte sobre errores comunes y cómo evitarlos:

▲ Cuidado

R distingue entre mayúsculas y minúsculas: `Ventas` y `ventas` son dos objetos distintos. Si creas `ventas` y luego escribes `Ventas`, R responderá con un error que dice que el objeto no se encontró (*object not found*).

La caja **Hazlo tú** es un reto corto para practicar en ese mismo momento. Muchas traen una pista, pero no una solución formal: la idea es que experimentes.

 Hazlo tú

Cambia los valores de `ventas_semana` por las ventas de un negocio que conozcas (o inventa unas) y vuelve a calcular el total, el promedio diario y el ticket promedio.

Al final de cada módulo, la caja **Resumen del módulo** repasa lo aprendido y suele incluir una tabla con las funciones clave:

Resumen del módulo (ejemplo)

- `sum()` suma y `mean()` promedia los elementos de un vector.
- `NA` marca un valor faltante; `na.rm = TRUE` lo ignora en los cálculos.

Después del resumen vienen los **ejercicios**, en cajas naranjas numeradas (Ejercicio 2.1, 2.2...). Cada ejercicio indica su nivel (Básico, Intermedio, Avanzado o Reto) y, en la esquina inferior derecha, la página del Apéndice A donde está su solución completa: explicación, código, salida real e interpretación. Intenta resolverlo antes de mirar la solución.

6.4 Formato del texto

Dentro de los párrafos, el formato también te da pistas: `summarise()` es código de R (funciones, objetos, argumentos); `dplyr`, el nombre de un paquete; `ventas_retail.csv`, un archivo o carpeta; **Tools > Global Options**, un menú de RStudio; `Ctrl` + `Enter`, un atajo de teclado (en Mac cambia `Ctrl` por `Cmd`); y un **término en azul** es un concepto nuevo que se define ahí mismo.

7 Preparar tu entorno de trabajo

Vas a instalar dos programas, en este orden: primero **R** (el motor) y después **RStudio** (el tablero). Luego configurarás RStudio, instalarás los paquetes del curso y crearás un Proyecto de RStudio. Calcula entre 30 y 60 minutos, la mayor parte esperando descargas. El libro se verificó con R 4.3.3; cualquier versión 4.3 o posterior sirve.

7.1 Paso 1: instalar R

R se descarga de **CRAN** (*Comprehensive R Archive Network*), el repositorio oficial: <https://cran.r-project.org>.

Windows

1. En la página de CRAN elige **Download R for Windows** y luego **base**.
2. Pulsa **Download R-4.x.x for Windows** (el número cambia con cada versión).
3. Ejecuta el instalador `.exe` y acepta las opciones por defecto.
4. Verás una página sobre **Rtools**: solo hace falta para compilar paquetes desde su código fuente; para este curso no la necesitas.

macOS

1. En CRAN elige **Download R for macOS**.
2. Descarga el instalador `.pkg` que corresponde a tu procesador: **Apple silicon** (M1, M2, M3...; archivo con `arm64` en el nombre) o **Intel** (`x86_64`). Si no sabes cuál tienes, abre el menú Apple → **Acerca de esta Mac** y mira el campo “Chip” o “Procesador”.
3. Abre el `.pkg` y sigue el asistente.

Linux (Ubuntu o Debian). Abre una terminal y ejecuta:

```
1 sudo apt update
2 sudo apt install r-base r-base-dev
3 # Bibliotecas del sistema que necesitan algunos paquetes de R
4 sudo apt install libcurl4-openssl-dev libssl-dev libxml2-dev
5 R --version
```

La versión de los repositorios de tu distribución puede estar algo atrasada; en la sección de Linux de CRAN hay instrucciones para agregar su repositorio y obtener siempre la versión más reciente.

7.2 Paso 2: instalar RStudio

1. Entra a <https://posit.co/download/rstudio-desktop/>.
2. Descarga **RStudio Desktop** (gratuito) para tu sistema. La página suele detectarlo sola.
3. Instálalo. En Windows, ejecuta el archivo `.exe`; en macOS, abre el `.dmg` y arrastra RStudio a la carpeta **Aplicaciones**; en Ubuntu, instala el archivo `.deb`.
4. Abre RStudio. Si en la consola aparece un texto que empieza con **R version 4.**, R y RStudio están conectados.

Consejo

Si no puedes instalar software en la computadora de tu trabajo, crea una cuenta gratuita en **Posit Cloud** (<https://posit.cloud>): es RStudio en el navegador. Sube ahí la carpeta del curso y sigue el libro igual. El plan gratuito tiene límites de memoria y de horas de uso, pero alcanza para practicar.

7.3 Un vistazo a RStudio

Al abrir RStudio verás cuatro paneles: el **editor** (arriba a la izquierda), donde escribes y guardas tus scripts; la **consola** (abajo a la izquierda), donde R ejecuta el código y responde; el **entorno** (arriba a la derecha), con los objetos que has creado; y un panel con pestañas para archivos, gráficas, paquetes y ayuda (abajo a la derecha). Si solo ves tres, abre un script nuevo con **File > New File > R Script**. El atajo que más usarás es `Ctrl` + `Enter` (`Cmd` + `Return` en Mac): envía a la consola

la línea donde está el cursor, o el texto seleccionado. El Módulo 1 recorre RStudio con detalle y el Apéndice B reúne los atajos de teclado.

7.4 Paso 3: configurar RStudio

Abre **Tools > Global Options** y ajusta las opciones de la tabla 8. La más importante es la primera pareja: evita que R guarde y restaure automáticamente los objetos de la sesión anterior.

Tabla 8. Configuración recomendada de RStudio (**Tools > Global Options**).

Sección	Opción	Valor	Por qué
General	Restore .RData into workspace at startup	Desmarcada	Cada sesión empieza limpia, sin objetos “fantasma” de ayer.
General	Save workspace to .RData on exit	Never	Tu trabajo debe vivir en scripts, no en la memoria.
Code > Editing	Insert spaces for Tab (Tab width 2)	Marcada	Sangría uniforme, al estilo tidyverse.
Code > Editing	Use native pipe operator, <code> ></code>	Desmarcada	El libro usa <code>%>%</code> ; así el atajo inserta ese pipe (el Módulo 2 explica los dos).
Code > Display	Show line numbers; Highlight selected line	Marcadas	Ubicas los errores por número de línea.
Code > Display	Show margin; Use rainbow parentheses	Marcadas	Líneas cortas y paréntesis de colores para ver cuál cierra a cuál.
Code > Saving	Default text encoding	UTF-8	Acentos y ñes correctos en cualquier sistema.
Appearance	Editor theme, font size	A tu gusto (12 a 14)	Trabajarás muchas horas: que sea cómodo.

7.5 Paso 4: instalar los paquetes del curso

Un **paquete** se **instala** una sola vez en cada computadora (con `install.packages()`) y se **carga** en cada sesión en que lo uses (con `library()`). Es como comprar un libro y después abrirlo cada vez que lo lees. Copia el siguiente bloque en un script nuevo y ejecútalo completo; tarda entre 10 y 30 minutos según tu conexión.

Paquetes obligatorios del curso

```

1 paquetes <- c(
2   # --- Importar, manipular, visualizar y explorar (Módulos 1 a 4) ---
3   "tidyverse",      # dplyr, ggplot2, tidyr, readr, tibble, stringr...
4   "readxl",         # leer archivos de Excel (.xlsx y .xls)
5   "writexl",        # escribir archivos de Excel
6   "lubridate",      # trabajar con fechas: meses, semanas, días
7   "scales",         # formato de ejes: $, %, separador de miles
8   "skimr",          # resumen rápido de una tabla completa
9   "corrplot",       # gráficas de matrices de correlación
10  # --- Bases de datos y SQL (Módulo 5) ---

```

```

11  "DBI",          # interfaz común para cualquier base de datos
12  "RSQLite",      # base de datos SQLite en un archivo local
13  "dbplyr",       # traducir verbos de dplyr a SQL
14  # --- Dashboards interactivos (Módulo 6) ---
15  "shiny",        # aplicaciones web con R
16  "shinydashboard", # plantilla de dashboard para Shiny
17  "DT",           # tablas interactivas: buscar, ordenar, filtrar
18  "plotly",       # gráficas interactivas
19  # --- Machine learning (Módulo 7) ---
20  "cluster",      # segmentación (clustering) y silueta
21  "factoextra",   # visualizar clusters y componentes principales
22  "forecast",     # series de tiempo y pronósticos
23  "rpart",        # árboles de decisión
24  "randomForest", # bosques aleatorios
25  "caret",        # entrenar y evaluar modelos
26  # --- Reportes automáticos (Módulo 8) ---
27  "rmarkdown",    # reportes en HTML, Word y PDF
28  "knitr"         # motor que ejecuta el código de los reportes
29 )
30
31 # Instala solo los paquetes que todavía no tienes
32 faltantes <- setdiff(paquetes, rownames(installed.packages()))
33 install.packages(faltantes)

```

Algunos módulos mencionan paquetes adicionales. No son necesarios para seguir el libro, pero puedes instalarlos si quieres probar esos ejemplos:

Paquetes opcionales

```

1  install.packages(c(
2    "arules",      # reglas de asociación (análisis de canasta de compras)
3    "rpart.plot",  # dibujos más legibles de árboles de decisión
4    "janitor",     # limpiar nombres de columnas y tablas de frecuencia
5    "openxlsx"     # Excel con formato: colores, anchos, estilos
6  ))

```

▲ Cuidado

Si durante la instalación R pregunta *Do you want to install from sources the packages which need compilation?*, responde **no**: se instalará la versión ya compilada, que funciona igual y no requiere herramientas adicionales. En Linux los paquetes siempre se compilan; si alguno falla, casi siempre es porque falta una biblioteca del sistema (el mensaje de error dice cuál) como las del paso 1.

7.6 Paso 5: crear un Proyecto de RStudio

Descarga la carpeta del curso, `curso-bi-rstudio/` (por ejemplo, descargando el repositorio como ZIP y descomprimiéndolo), y colócala en un lugar fácil de encontrar, como **Documentos**. Evita rutas con caracteres especiales o carpetas que se sincronizan constantemente con la nube, porque a veces bloquean archivos mientras R los escribe. Después:

1. En RStudio elige **File > New Project...**
2. Selecciona **Existing Directory** y, con **Browse...**, la carpeta `curso-bi-rstudio`.
3. Pulsa **Create Project**. RStudio se reinicia, crea el archivo `curso-bi-rstudio.Rproj` y muestra el nombre del proyecto en la esquina superior derecha.
4. Las próximas veces abre el curso con doble clic en `curso-bi-rstudio.Rproj` (o desde **File > Open Project**).

Concepto clave: Proyecto de RStudio y rutas relativas

Un **Proyecto de RStudio** es una carpeta con un archivo `.Rproj`. Al abrirlo, RStudio fija esa carpeta como **directorio de trabajo**: el lugar desde donde R busca y guarda archivos. Gracias a eso puedes usar **rutas relativas** como `"datasets/ventas_retail.csv"` en lugar de rutas absolutas como `"C:/Users/Ana/Documents/..."`. Tu código funcionará igual en cualquier computadora donde copies la carpeta, y cada proyecto tendrá su propio historial y sus propios archivos abiertos.

Para comprobar dónde está trabajando R, escribe en la consola:

```
1 getwd() # muestra el directorio de trabajo actual
```

La respuesta debe terminar en `curso-bi-rstudio`. Todo el código del libro supone que el directorio de trabajo es esa carpeta.

▲ Cuidado

Evita escribir `setwd("C:/Users/TuNombre/...")` al inicio de tus scripts. Funciona en tu computadora, pero falla en la de cualquier otra persona (y en la tuya, si mueves la carpeta). Con un Proyecto de RStudio no lo necesitas.

7.7 La estructura de carpetas del curso

La carpeta del curso está organizada así:

```
1 curso-bi-rstudio/
2 |-- curso-bi-rstudio.Rproj    <- lo crea RStudio (paso 5)
3 |-- README.md                <- presentación del curso
4 |-- GUIA_INSTALACION.md      <- guía de instalación resumida
```

```

5 |-- datasets/                <- datos del curso
6 |   |-- generar_datasets.R   <- script que crea todos los datos
7 |   |-- ventas_retail.csv     <- (estos archivos aparecen al
8 |   |-- clientes.csv          <- ejecutar el script)
9 |   |-- ...
10 |   `-- datos_empresa.xlsx
11 |-- latex/                  <- el libro que estás leyendo
12 |   |-- curso-bi-rstudio.tex <- archivo principal del libro
13 |   |-- preambulo.tex        <- estilos, colores y cajas
14 |   |-- capitulos/           <- un archivo .tex por módulo
15 |   |-- apendices/           <- soluciones y referencia rápida
16 |   |-- figuras/             <- gráficas del libro en PDF
17 |   |-- codigo/              <- un script .R por módulo
18 |   `-- herramientas/        <- extraer código, generar figuras
19 |-- modulo-01-fundamentos/   <- scripts complementarios de
20 |-- modulo-02-.../           <- práctica de algunos módulos
21 |-- proyecto-final/          <- material del proyecto integrador
22 `-- resultados/             <- lo crean los ejemplos que guardan
23                               archivos (CSV, Excel, gráficas)

```

Las carpetas `modulo-XX/` contienen scripts de práctica complementarios; el código completo y verificado de cada módulo del libro está en `latex/codigo/`.

8 Generar los datos del curso

Los datos del curso no se descargan: los crea un script de R. Así pesan poco, y todos obtenemos exactamente los mismos datos.

8.1 Paso 6: ejecutar el generador

Con el proyecto abierto (el directorio de trabajo debe ser `curso-bi-rstudio`), escribe en la consola:

```
1 source("datasets/generar_datasets.R") # ejecuta el script completo
```

```

[1] "✓ ventas_retail.csv creado"
[1] "✓ clientes.csv creado"
[1] "✓ productos.csv creado"
[1] "✓ empleados.csv creado"
[1] "✓ transacciones.csv creado"
[1] "✓ tiendas.csv creado"
[1] "✓ datos_empresa.xlsx creado"

```

```

=====
DATASETS GENERADOS EXITOSAMENTE
=====

```

Archivos creados:

1. ventas_retail.csv (365 filas)
2. clientes.csv (200 filas)
3. productos.csv (50 filas)
4. empleados.csv (100 filas)
5. transacciones.csv (1000 filas)
6. tiendas.csv (10 filas)
7. datos_empresa.xlsx (multi-hoja)

```
=====
¡Listo para usar en los módulos del curso!
=====
```

La función `source()` ejecuta de principio a fin todas las instrucciones de un script. El generador fija una **semilla** aleatoria con `set.seed(123)`: los datos son aleatorios, pero la semilla hace que R produzca siempre la misma secuencia de números. Por eso, con R 3.6 o posterior, tus archivos serán idénticos a los del libro y tus resultados coincidirán con las salidas impresas. Solo necesitas ejecutarlo una vez; si borras algún archivo por accidente, vuelve a ejecutarlo.

▲ Cuidado

Si aparece *cannot open file 'datasets/generar_datasets.R': No such file or directory*, R no está trabajando en la carpeta del curso. Abre el archivo `curso-bi-rstudio.Rproj` y vuelve a intentarlo. Si el error menciona **writexl**, te falta instalar los paquetes del paso 4.

8.2 Nota sobre el idioma de los días y los meses

El generador escribe el día de la semana con la función `weekdays()`, que usa el idioma configurado en tu sistema para las fechas. Puedes consultarlo así:

```
1 Sys.getlocale("LC_TIME") # idioma y región que R usa para las fechas
```

```
[1] "es_MX.UTF-8"
```

Los datos del libro se generaron con español de México (`es_MX.UTF-8`), por eso verás “lunes”, “martes”... En Windows la respuesta se parece a `"Spanish_Mexico.utf8"`. Si en tu computadora ves algo como `"English_United_States.utf8"` o `"en_US.UTF-8"`, tus días saldrán en inglés (“Monday”, “Tuesday”...) y algunas salidas del libro no coincidirán con las tuyas. Para generar los datos en español, cambia el idioma de las fechas antes de ejecutar el generador:

```
1 # Windows
2 Sys.setlocale("LC_TIME", "Spanish")
3 # macOS y Linux
4 Sys.setlocale("LC_TIME", "es_MX.UTF-8")
5
```

```
6 source("datasets/generar_datasets.R") # vuelve a generar los datos
```

El cambio dura solo esa sesión de R. En Linux, si la instrucción responde con una advertencia, el idioma no está instalado; se agrega con `sudo locale-gen es_MX.UTF-8` en la terminal. Las cifras, los identificadores y los totales no dependen del idioma: solo los nombres de días y meses.

8.3 Paso 7: verificar la instalación

El siguiente script revisa todo de una vez: la versión de R, que cada paquete del curso se pueda cargar y que existan los archivos de datos. Es un buen primer script para guardar en tu proyecto como `00_verificar_entorno.R`. Lee los comentarios: aunque todavía no conozcas `for` ni `if`, entenderás la idea.

Primero carga el tidyverse, que el script usa para leer los CSV:

```
1 library(tidyverse) # carga dplyr, ggplot2, readr y compañía
```

Al ejecutar `library(tidyverse)` verás un mensaje de bienvenida con la lista de paquetes que se cargaron y un par de avisos de “conflictos”. No es un error y no lo reproducimos en el libro (el Módulo 1 lo explica). Ahora sí, el script de verificación:

Verificación del entorno del curso

```
1 # ---- 1. Versión de R ----
2 cat("Versión de R:", as.character(getRversion()), "\n\n")
3
4 # ---- 2. Paquetes del curso: ¿se pueden cargar? ----
5 paquetes_curso <- c("tidyverse", "readxl", "writexl", "lubridate",
6                     "scales", "DBI", "RSQLite", "dbplyr", "shiny",
7                     "shinydashboard", "DT", "plotly", "cluster",
8                     "factoextra", "forecast", "rpart", "randomForest",
9                     "caret", "rmarkdown", "knitr", "skimr", "corrplot")
10 problemas <- 0 # contador de problemas
11
12 cat("Paquetes:\n")
13 for (p in paquetes_curso) { # revisa los paquetes uno por uno
14   # ¿carga bien? (suppressMessages oculta avisos técnicos de carga)
15   if (suppressMessages(requireNamespace(p, quietly = TRUE))) {
16     version <- as.character(packageVersion(p))
17     cat(sprintf(" OK      %-15s %s\n", p, version))
18   } else {
19     problemas <- problemas + 1
20     cat(sprintf(" FALTA  %-15s instálalo (paso 4)\n", p))
21   }
22 }
23
24 # ---- 3. Archivos de datos ----
```

```

25 cat("\nArchivos en datasets/:\n")
26 tablas <- c("ventas_retail", "clientes", "productos", "empleados",
27            "transacciones", "tiendas")
28 for (t in tablas) {
29   archivo_csv <- file.path("datasets", paste0(t, ".csv"))
30   if (file.exists(archivo_csv)) {
31     datos <- read_csv(archivo_csv, show_col_types = FALSE)
32     cat(sprintf(" OK      %-18s %5d filas %3d columnas\n",
33               basename(archivo_csv), nrow(datos), ncol(datos)))
34   } else {
35     problemas <- problemas + 1
36     cat(sprintf(" FALTA  %s (ejecuta el paso 6)\n", archivo_csv))
37   }
38 }
39 hojas <- readxl::excel_sheets("datasets/datos_empresa.xlsx")
40 cat(" OK      datos_empresa.xlsx con hojas:\n",
41     paste(hojas, collapse = ", "), "\n")
42
43 # ---- 4. Veredicto ----
44 if (problemas == 0) {
45   cat("\n¡Todo listo! Tu entorno está preparado para el curso.\n")
46 } else {
47   cat("\nHay", problemas, "problema(s): revisa las líneas FALTA.\n")
48 }

```

Versión de R: 4.3.3

Paquetes:

OK	tidyverse	2.0.0
OK	readxl	1.4.3
OK	writexl	1.5.0
OK	lubridate	1.9.3
...		
OK	skimr	2.1.5
OK	corrplot	0.92

Archivos en datasets/:

OK	ventas_retail.csv	365 filas	14 columnas
OK	clientes.csv	200 filas	10 columnas
OK	productos.csv	50 filas	13 columnas
OK	empleados.csv	100 filas	12 columnas
OK	transacciones.csv	1000 filas	14 columnas
OK	tiendas.csv	10 filas	8 columnas
OK	datos_empresa.xlsx con hojas:		
	Ventas, Clientes, Productos, Empleados, Tiendas		

¡Todo listo! Tu entorno está preparado para el curso.

Recortamos la lista de paquetes (la línea `...`); en tu pantalla aparecen los 22. Tus números de versión serán distintos si instalaste paquetes más recientes: no importa. Lo que importa es que no haya ninguna línea **FALTA**.

8.4 Tu primer análisis

Ya que todo funciona, date un gusto: responde tu primera pregunta de negocio. ¿Cuánto vendió la cadena en cada trimestre de 2023? No te preocupes por entender cada función; en el Módulo 2 las verás con calma. Lee el código como una receta, gracias al operador `%>%` (el **pipe**, que se lee “y luego”):

```
1 # Leer la tabla de ventas
2 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
3
4 ventas %>%                                # toma las ventas, y luego
5   group_by(trimestre) %>%                 # agrúpalas por trimestre,
6   summarise(ventas_totales = sum(total),   # suma el total vendido,
7             numero_ventas = n(),          # cuenta las ventas
8             ticket_promedio = mean(total), # y promedia su importe
9             .groups = "drop")
```

```
# A tibble: 4 x 4
  trimestre ventas_totales numero_ventas ticket_promedio
  <chr>         <dbl>         <int>         <dbl>
1 Q1             126274.             90             1403.
2 Q2             120717.             91             1327.
3 Q3             134427.             92             1461.
4 Q4             132498.             92             1440.
```

¿Qué nos dice esto? El tercer trimestre fue el mejor, con unos \$134,427, y el segundo el más flojo, con \$120,717. Como el número de ventas casi no cambia entre trimestres (de 90 a 92), la diferencia viene del **ticket promedio**: en el tercer trimestre cada venta fue de \$1,461 en promedio, frente a \$1,327 en el segundo. Ese es exactamente el tipo de lectura que harás en todo el libro: pasar de la tabla a la frase que le sirve a quien decide.

Hazlo tú

Cambia `group_by(trimestre)` por `group_by(mes)` y vuelve a ejecutar. ¿Qué mes vendió más? Luego prueba con `group_by(dia_semana)`.

9 Diccionario de datos

Un **diccionario de datos** describe cada tabla y cada columna: qué significa, de qué tipo es y qué valores puede tomar. Es el primer documento que pide un analista al llegar a una empresa, y el que

más tiempo ahorra. Vuelve a esta sección siempre que tengas dudas sobre una columna.

9.1 La empresa y sus tablas

Los datos describen el año 2023 de una cadena minorista ficticia con **10 tiendas** en 6 ciudades (CDMX, Guadalajara, Monterrey, Puebla, Querétaro y Tijuana), **50 productos** de 10 categorías, **200 clientes** y **100 empleados**. Las tablas siguen la organización típica de un almacén de datos de BI.

Concepto clave: tablas de hechos y dimensiones (esquema estrella)

Una **tabla de hechos** registra eventos del negocio con medidas numéricas: cada venta, cada transacción. Una **tabla de dimensión** describe el contexto de esos eventos: quién compró (clientes), qué se vendió (productos), dónde (tiendas) y quién atendió (empleados). Las tablas se conectan mediante **llaves**: la **llave primaria** (PK) identifica cada fila de una dimensión, y la **llave foránea** (FK) de la tabla de hechos apunta a ella. Dibujadas, las dimensiones rodean a los hechos como las puntas de una estrella: por eso se llama **esquema estrella**.

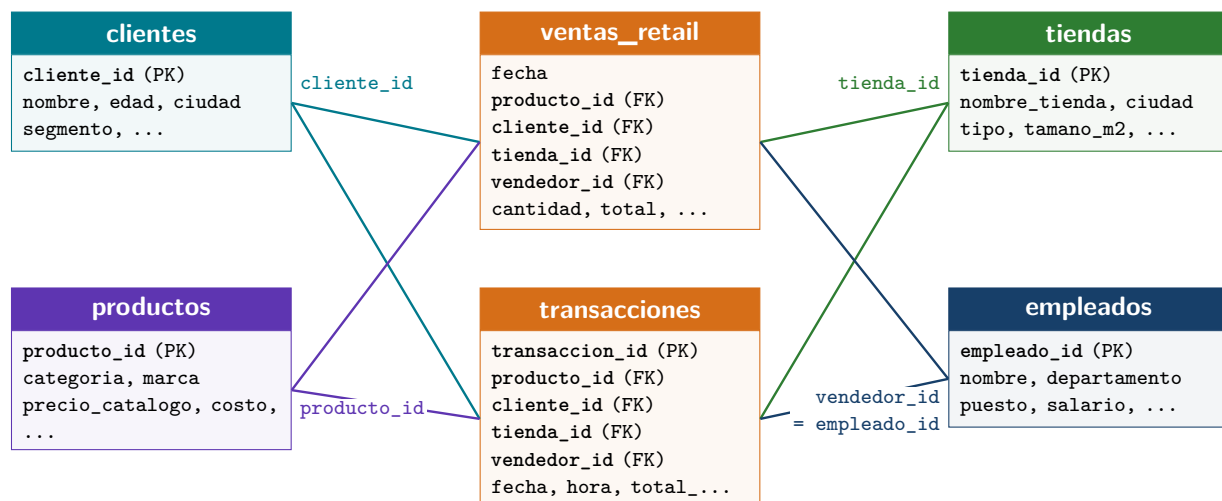


Figura 3. Relaciones entre las tablas del curso. Las dos tablas de hechos (al centro) se unen con las cuatro dimensiones mediante llaves foráneas (FK) que apuntan a la llave primaria (PK) de cada dimensión. Cada fila de una dimensión puede aparecer en muchas ventas (relación de muchos a uno).

Observa la relación más delicada: en las tablas de hechos la columna se llama **vendedor_id**, pero en **empleados.csv** se llama **empleado_id**. Cuando unas estas tablas en el Módulo 2 tendrás que indicarle a R que son la misma llave con nombres distintos.

En las tablas siguientes, el **tipo** es el que asigna **read_csv()** al leer cada archivo (tabla 7). Los identificadores y las cantidades se leen como **<dbl>** aunque sean enteros; es normal.

9.2 ventas_retail.csv: ventas diarias

365 filas y 14 columnas. Cada fila es una venta y hay exactamente una venta por cada día de 2023. Es una simplificación: una cadena real tiene cientos de tickets diarios. Sirve para practicar análisis por fecha, tienda, producto y descuento.

Tabla 9. Diccionario de `ventas_retail.csv`.

Columna	Tipo	Descripción
<code>fecha</code>	<date>	Día de la venta, del 2023-01-01 al 2023-12-31 (uno por fila).
<code>dia_semana</code>	<chr>	Nombre del día (“domingo”, “lunes”...). Depende del idioma del sistema al generar los datos.
<code>producto_id</code>	<dbl>	Producto vendido, de 1 a 50. Llave foránea hacia <code>productos</code> .
<code>cliente_id</code>	<dbl>	Cliente que compró, de 1 a 200. Llave foránea hacia <code>clientes</code> .
<code>cantidad</code>	<dbl>	Unidades vendidas, de 1 a 10.
<code>precio_unitario</code>	<dbl>	Precio cobrado por unidad, aleatorio entre \$10 y \$500, con dos decimales. No coincide con el precio de catálogo del producto.
<code>descuento_pct</code>	<dbl>	Descuento en porcentaje: 0, 5, 10, 15 o 20 (con probabilidades de 50, 20, 15, 10 y 5 %).
<code>tienda_id</code>	<dbl>	Tienda de la venta, de 1 a 10. Llave foránea hacia <code>tiendas</code> .
<code>vendedor_id</code>	<dbl>	Empleado que vendió, de 1 a 25. Llave foránea hacia <code>empleado_id</code> de <code>empleados</code> .
<code>subtotal</code>	<dbl>	$\text{cantidad} \times \text{precio_unitario}$.
<code>descuento_monto</code>	<dbl>	$\text{subtotal} \times \text{descuento_pct} / 100$.
<code>total</code>	<dbl>	$\text{subtotal} - \text{descuento_monto}$: lo que pagó el cliente.
<code>mes</code>	<chr>	Año y mes como texto: “2023-01” a “2023-12”.
<code>trimestre</code>	<chr>	Trimestre: “Q1”, “Q2”, “Q3” o “Q4”.

9.3 clientes.csv: base de clientes

200 filas y 10 columnas. Cada fila es un cliente.

Tabla 10. Diccionario de `clientes.csv`.

Columna	Tipo	Descripción
<code>cliente_id</code>	<dbl>	Identificador único, de 1 a 200. Llave primaria.
<code>nombre</code>	<chr>	Nombre y apellido, combinados al azar de 20 nombres y 18 apellidos (puede repetirse).
<code>edad</code>	<dbl>	Edad en años, de 18 a 75.
<code>genero</code>	<chr>	“M” o “F”.
<code>ciudad</code>	<chr>	Ciudad de residencia: CDMX, Guadalajara, Monterrey, Puebla, Querétaro, Tijuana, León, Mérida, Cancún o Toluca.
<code>fecha_registro</code>	<date>	Fecha de alta como cliente, entre 2020-01-01 y 2023-12-31.
<code>es_premium</code>	<lgl>	TRUE si pertenece al programa premium (alrededor de 30 % de los clientes).
<code>email</code>	<chr>	Correo en minúsculas con dominio <code>@email.com</code> . Se generó con otro nombre al azar, así que no corresponde a la columna <code>nombre</code> .
<code>telefono</code>	<chr>	Teléfono con formato “55-XXXX-XXXX”.
<code>segmento</code>	<chr>	Segmento por edad: Joven (18 a 25), Adulto (26 a 40), Maduro (41 a 60) o Senior (61 a 75).

9.4 `productos.csv`: catálogo de productos

50 filas y 13 columnas. Cada fila es un producto del catálogo.

Tabla 11. Diccionario de `productos.csv`.

Columna	Tipo	Descripción
<code>producto_id</code>	<dbl>	Identificador, de 1 a 50. Llave primaria.
<code>nombre_producto</code>	<chr>	“Producto 01” a “Producto 50”.
<code>categoria</code>	<chr>	Una de 10 categorías asignadas al azar: Electrónica, Ropa, Alimentos, Hogar, Deportes, Libros, Juguetes, Belleza, Automotriz o Mascotas.
<code>subcategoria</code>	<chr>	“Sub 1” a “Sub 5”.
<code>marca</code>	<chr>	“Marca A” a “Marca E”.
<code>precio_catalogo</code>	<dbl>	Precio de lista, aleatorio entre \$15 y \$800.
<code>costo</code>	<dbl>	Costo unitario, aleatorio entre \$10 y \$400, generado sin relación con el precio.
<code>stock_actual</code>	<dbl>	Unidades en inventario, de 0 a 500.
<code>stock_minimo</code>	<dbl>	Punto de reorden, de 10 a 50 unidades.
<code>proveedor</code>	<chr>	“Proveedor 1” a “Proveedor 4”.
<code>activo</code>	<lgl>	TRUE si el producto sigue a la venta (alrededor de 90 %).
<code>margen_pct</code>	<dbl>	Margen bruto en porcentaje: $(\text{precio_catalogo} - \text{costo}) / \text{precio_catalogo} \times 100$, con dos decimales. Puede ser negativo.
<code>estado_stock</code>	<chr>	“Sin Stock” si <code>stock_actual</code> es 0; “Stock Bajo” si es menor que <code>stock_minimo</code> ; si no, “Stock OK”.

9.5 `empleados.csv`: recursos humanos

100 filas y 12 columnas. Cada fila es un empleado.

Tabla 12. Diccionario de `empleados.csv`.

Columna	Tipo	Descripción
<code>empleado_id</code>	<dbl>	Identificador, de 1 a 100. Llave primaria; el <code>vendedor_id</code> de las ventas apunta aquí.
<code>nombre</code>	<chr>	Nombre y apellido al azar.
<code>departamento</code>	<chr>	Ventas, Marketing, IT, RRHH, Operaciones, Finanzas o Atención al Cliente.
<code>puesto</code>	<chr>	Analista, Especialista, Gerente, Coordinador o Asistente.
<code>salario_mensual</code>	<dbl>	Salario mensual en pesos, aleatorio entre \$8,000 y \$45,000.
<code>fecha_ingreso</code>	<date>	Fecha de contratación, entre 2015-01-01 y 2023-12-31.
<code>sucursal</code>	<chr>	Matriz, Sucursal Norte, Sucursal Sur, Sucursal Este o Sucursal Oeste.
<code>genero</code>	<chr>	“M” o “F”.
<code>edad</code>	<dbl>	Edad en años, de 22 a 65.
<code>nivel_educacion</code>	<chr>	Preparatoria, Licenciatura, Maestría o Doctorado (con probabilidades de 20, 50, 25 y 5 %).
<code>antiguedad_anos</code>	<dbl>	Años de antigüedad al 2024-01-01 (días transcurridos / 365, con un decimal).
<code>salario_anual</code>	<dbl>	<code>salario_mensual</code> × 12.

9.6 `transacciones.csv`: tickets de venta

1000 filas y 14 columnas. Cada fila es un ticket de venta con su hora y su método de pago. Es la tabla más parecida a la que exporta un sistema de punto de venta real. Sus datos se generaron por separado de `ventas_retail.csv`, así que las dos tablas no “cuadran” entre sí: úsalas como dos fuentes independientes.

Tabla 13. Diccionario de `transacciones.csv`.

Columna	Tipo	Descripción
<code>transaccion_id</code>	<dbl>	Identificador del ticket, de 1 a 1000. Llave primaria.
<code>fecha</code>	<date>	Día de la transacción, al azar dentro de 2023.
<code>hora</code>	<time>	Hora entre 8:00 y 20:45, en intervalos de 15 minutos. En el CSV está como texto sin cero inicial (“8:15”); <code>read_csv()</code> la reconoce como hora.
<code>cliente_id</code>	<dbl>	Cliente, de 1 a 200 (FK hacia <code>clientes</code>).
<code>producto_id</code>	<dbl>	Producto, de 1 a 50 (FK hacia <code>productos</code>).
<code>cantidad</code>	<dbl>	Unidades, de 1 a 5.
<code>precio_venta</code>	<dbl>	Precio unitario cobrado, aleatorio entre \$20 y \$600.
<code>metodo_pago</code>	<chr>	Efectivo, Tarjeta Crédito, Tarjeta Débito o Transferencia (con probabilidades de 30, 35, 25 y 10 %).
<code>tienda_id</code>	<dbl>	Tienda, de 1 a 10 (FK hacia <code>tiendas</code>).
<code>vendedor_id</code>	<dbl>	Vendedor, de 1 a 25 (FK hacia <code>empleado_id</code>).
<code>total_transaccion</code>	<dbl>	<code>cantidad</code> × <code>precio_venta</code> (esta tabla no tiene descuentos).
<code>mes</code>	<chr>	Año y mes: “2023-01” a “2023-12”.
<code>trimestre</code>	<chr>	“Q1” a “Q4”.
<code>dia_semana</code>	<chr>	Nombre del día; depende del idioma del sistema.

9.7 tiendas.csv: red de sucursales

10 filas y 8 columnas. Cada fila es una tienda.

Tabla 14. Diccionario de `tiendas.csv`.

Columna	Tipo	Descripción
<code>tienda_id</code>	<dbl>	Identificador, de 1 a 10. Llave primaria.
<code>nombre_tienda</code>	<chr>	“Sucursal Centro”, “Sucursal Norte”, . . . , “Sucursal Express”.
<code>ciudad</code>	<chr>	CDMX (4 tiendas), Guadalajara (2), Monterrey, Puebla, Tijuana y Querétaro (1 cada una).
<code>zona</code>	<chr>	Centro, Norte, Sur, Este u Oeste.
<code>tamano_m2</code>	<dbl>	Superficie de la tienda en metros cuadrados, de 300 a 800.
<code>empleados</code>	<dbl>	Número de empleados de la tienda, de 12 a 40. No es una llave hacia <code>empleados.csv</code> .
<code>fecha_apertura</code>	<date>	Fecha de apertura, entre 2009-01-15 y 2020-07-01.
<code>tipo</code>	<chr>	Principal, Estándar, Outlet, Premium o Express.

9.8 datos_empresa.xlsx: el mismo negocio en Excel

Un libro de Excel con cinco hojas, para practicar la importación de archivos de Excel en el Módulo 1: **Ventas** (las primeras 100 filas de `ventas_retail`), **Clientes** (los primeros 50), **Productos** (los 50), **Empleados** (los primeros 50) y **Tiendas** (las 10).

9.9 Datos simulados: lo que debes saber

Los datos son **simulados**: se generaron al azar con reglas simples, no provienen de una empresa real. Eso tiene una consecuencia que no vamos a esconder: algunas columnas se generaron de forma independiente y producen combinaciones que en un negocio real serían errores. Lejos de ser un problema, es una oportunidad, porque los datos reales tampoco vienen limpios. Revisar la **calidad de datos** antes de analizar es una de las tareas más importantes de un analista, y aquí la practicarás desde el primer día (tabla 15).

Tabla 15. Hallazgos de calidad de datos conocidos en los datos del curso.

Hallazgo	Por qué ocurre
Productos con margen negativo (costo mayor que precio).	<code>precio_catalogo</code> y <code>costo</code> se generaron por separado.
El precio cobrado en las ventas no coincide con el precio de catálogo.	<code>precio_unitario</code> y <code>precio_venta</code> son aleatorios.
Ventas de productos marcados como inactivos.	<code>activo</code> se asignó al azar, sin mirar las ventas.
Ventas anteriores a la fecha de registro del cliente.	<code>fecha_registro</code> llega hasta diciembre de 2023.
El correo de un cliente no corresponde a su nombre.	<code>email</code> se armó con otro nombre al azar.
Hay “vendedores” que no son del departamento de Ventas.	<code>vendedor_id</code> toma los empleados 1 a 25, sin importar su departamento.
Una sola venta por día en <code>ventas_retail</code> , y no cuadra con <code>transacciones</code> .	Son dos muestras independientes y simplificadas.
La suma de <code>empleados</code> en <code>tiendas</code> no coincide con las 100 filas de <code>empleados.csv</code> , y la sucursal “Matriz” no existe en <code>tiendas</code> .	Las dos tablas se generaron sin relación entre sí.
Hay clientes en ciudades sin tienda (León, Mérida, Cancún, Toluca).	Posible en la realidad (compran de visita), pero conviene saberlo.

No necesitas creernos: el siguiente código comprueba varios de estos hallazgos. Usa funciones que aprenderás en el Módulo 2, así que por ahora basta con que leas los comentarios. Reutiliza la tabla `ventas` que leíste en tu primer análisis.

```

1 # Leemos las tablas que vamos a revisar
2 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
3 clientes  <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
4 empleados <- read_csv("datasets/empleados.csv", show_col_types = FALSE)
5
6 # Hallazgo 1: ¿cuántos productos tienen margen negativo?
7 sum(productos$margen_pct < 0)
8
9 # Hallazgo 2: ¿cuántas ventas son de productos inactivos?
10 ventas %>%
11   left_join(productos, by = "producto_id") %>% # agrega datos del producto
12   filter(activo == FALSE) %>%                 # solo productos inactivos
13   nrow()                                       # cuenta las filas
14
15 # Hallazgo 3: ¿cuántas ventas ocurrieron ANTES del registro del cliente?
16 ventas %>%
17   left_join(clientes, by = "cliente_id") %>% # agrega datos del cliente
18   filter(fecha < fecha_registro) %>%         # compra antes del alta
19   nrow()

```

```

[1] 10
[1] 65
[1] 42

```

De los 50 productos, 10 tienen margen negativo; 65 de las 365 ventas corresponden a productos inactivos, y en 42 ventas el cliente “compró” antes de registrarse. Ahora los dos hallazgos que se ven a simple vista:

```
1 # Hallazgo 4: el nombre y el correo no corresponden a la misma persona
2 clientes %>%
3   select(cliente_id, nombre, email) %>%
4   head(3)
5
6 # Hallazgo 5: ¿de qué departamento son los "vendedores" 1 a 25?
7 empleados %>%
8   filter(empleado_id <= 25) %>%
9   count(departamento, sort = TRUE)
```

```
# A tibble: 3 x 3
  cliente_id nombre      email
  <dbl> <chr>      <chr>
1         1 Ricardo Morales juan.ramírez@email.com
2         2 Miguel Cruz   alberto.jiménez@email.com
3         3 Miguel Flores  pedro.sánchez@email.com
# A tibble: 7 x 2
  departamento      n
  <chr>          <int>
1 Atención al Cliente    10
2 RRHH                   4
3 Ventas                 4
4 Marketing              3
5 Finanzas               2
6 IT                    1
7 Operaciones            1
```

El cliente 1 se llama Ricardo Morales, pero su correo es de “juan.ramírez”. Y de los 25 empleados que aparecen como vendedores, solo 4 pertenecen al departamento de Ventas. En una empresa real, cada uno de estos hallazgos sería una conversación con el dueño del sistema de origen. En el curso los usaremos para practicar: los detectarás, decidirás cómo tratarlos y documentarás tu decisión.

▲ Cuidado

Por lo mismo, las cifras del curso no representan a ninguna empresa real. No saques conclusiones de negocio sobre el comercio en México a partir de ellas: son un campo de práctica. Lo que sí es real es el método, que aplicarás igual con los datos de tu trabajo.

📁 Hazlo tú

En el panel *Files* de RStudio haz clic en `datasets/productos.csv` y elige **View File**. Busca el producto con el margen más negativo. ¿Qué pasaría con un reporte de rentabilidad por categoría si nadie lo detectara? Pista: en la salida de arriba, R te dijo cuántos productos revisar.

10 El código del libro y cómo compilarlo

10.1 Los scripts de la carpeta `codigo/`

La carpeta `latex/codigo/` contiene un script de R por módulo (por ejemplo, `01-fundamentos.R` para el Módulo 1) con todo el código ejecutable de ese módulo, en el mismo orden que en el libro. Los bloques de franja naranja (instalaciones, apps Shiny, conexiones a servidores) aparecen comentados, solo como referencia. Para usarlos:

1. Abre tu proyecto `curso-bi-rstudio.Rproj`, para que el directorio de trabajo sea la carpeta del curso.
2. Abre el script del módulo desde el panel *Files*.
3. Ejecútalo bloque por bloque con `Ctrl` + `Enter` mientras lees el módulo. Cada bloque está marcado con un comentario `# -- Bloque N`.

Estos scripts se generan automáticamente a partir de los capítulos del libro con el programa `herramientas/extraer_codigo.py` (escrito en Python), así que siempre coinciden con lo impreso. Si modificas un capítulo y quieres regenerarlos, desde la carpeta `latex/` ejecuta en una terminal:

```
1 python3 herramientas/extraer_codigo.py
```

10.2 Compilar el libro

El libro está escrito en $\text{\LaTeX}$. Si quieres generar tu propio PDF (por ejemplo, después de añadir notas a un capítulo), tienes dos caminos:

- **Overleaf** (en línea, sin instalar nada): comprime la carpeta `latex/` en un ZIP, crea un proyecto con **New Project > Upload Project**, verifica en **Menu** que el compilador sea **pdfLaTeX** y que el documento principal sea `curso-bi-rstudio.tex`, y pulsa **Recompile**. Si el plan gratuito se queda sin tiempo de compilación, compila en tu computadora.
- **En tu computadora**, con una distribución de $\text{\LaTeX}$ como TeX Live (Windows y Linux) o MacTeX (macOS), abre una terminal en la carpeta `latex/` y ejecuta:

```
1 latexmk -pdf curso-bi-rstudio.tex
```

`latexmk` compila las veces que haga falta para que el índice y las referencias cruzadas (“Solución en la página...”) queden correctos.

11 Consejos para aprender

Aprender R es como aprender un idioma: se aprende hablándolo. Estos consejos vienen de muchas generaciones de estudiantes:

1. **Escribe el código tú mismo**, no lo copies. Teclear cada línea te obliga a leerla, y los errores que cometes al escribir te enseñarán a leer los mensajes de R.
2. **Predice antes de ejecutar**. Antes de pulsar `Ctrl` + `Enter`, pregúntate qué va a salir. Si aciertas, lo entendiste; si no, acabas de aprender algo.
3. **Lee los errores con calma**. El mensaje casi siempre dice qué pasó y en qué función. Los más comunes son un paréntesis o una comilla sin cerrar, una coma de más, un nombre mal escrito o un paquete sin cargar.
4. **Rompe el código a propósito**. Cambia un número, quita un argumento, usa otra columna. Ver qué cambia es la forma más rápida de entender qué hace cada parte.
5. **Practica con tus propios datos**. Cuando termines un módulo, repite un ejemplo con datos de tu trabajo (exportados a CSV y respetando siempre la confidencialidad). Nada motiva más que resolver un problema real.
6. **Constancia antes que intensidad**. Una hora diaria rinde mucho más que siete horas el domingo.
7. **Intenta los ejercicios antes de ver la solución**. Dale al menos 20 minutos a cada uno. Si te atorras, lee solo la primera línea de la solución y vuelve a intentarlo.
8. **Lleva tu propio acordeón**: un script con las funciones que vas aprendiendo y un ejemplo corto de cada una. El Apéndice B te da una base.
9. **Explica lo que aprendiste** a un colega. Si puedes explicarlo con palabras sencillas, lo dominas.

12 Cómo pedir ayuda

12.1 La ayuda dentro de R

Toda función de R tiene una página de ayuda con su descripción, sus argumentos y ejemplos al final. Es la primera parada cuando no recuerdas cómo se usa algo:

```
1 ?mean                # ayuda de una función (o F1 sobre su nombre)
2 help("read_csv")     # lo mismo, escrito de otra forma
3 ??"regression"       # busca un tema en todas las ayudas instaladas
4 example(mean)        # ejecuta los ejemplos de la ayuda
5 args(round)           # muestra solo los argumentos de una función
6 vignette("dplyr")     # guía larga (viñeta) de un paquete
```

La ayuda de R está en inglés. Léela primero por la sección *Examples*, al final: suele ser lo más claro.

Consejo

Cuando busques un error en internet, búscalo en inglés: hay muchas más respuestas. Si tu R muestra los mensajes en español, puedes cambiarlos a inglés en la sesión actual con `Sys.setenv(LANGUAGE = "en")`, volver a ejecutar la línea que falló y copiar el mensaje exacto en el buscador.

12.2 Recursos en línea

- **Hojas de referencia (*cheatsheets*) de Posit:** una o dos páginas por paquete con las funciones principales; varias están traducidas al español. Desde RStudio: **Help > Cheat Sheets**, o en <https://posit.co/resources/cheatsheets/>.
- **R for Data Science**, de Hadley Wickham y colaboradores, gratuito en línea (<https://r4ds.hadley.nz>). Su primera edición está traducida al español como *R para Ciencia de Datos* (<https://es.r4ds.hadley.nz>).
- **Stack Overflow**, el sitio de preguntas y respuestas de programación: en inglés con la etiqueta `r` (<https://stackoverflow.com/questions/tagged/r>) y en español en <https://es.stackoverflow.com>.
- **Posit Community**, el foro oficial de RStudio y el tidyverse: <https://forum.posit.co>.
- **TidyTuesday**: cada semana la comunidad publica un conjunto de datos nuevo para practicar y compartir análisis (<https://github.com/rfordatascience/tidytuesday>).
- **R-bloggers**: artículos y tutoriales de toda la comunidad (<https://www.r-bloggers.com>).

12.3 Comunidad de R en español

R tiene una comunidad hispanohablante muy activa y acogedora con los principiantes:

- **R-Ladies** (<https://rladies.org>): organización mundial que promueve la diversidad de género en la comunidad de R, con capítulos locales en muchas ciudades de México y Latinoamérica. Organiza reuniones y talleres gratuitos, abiertos a todas las personas interesadas.
- **Comunidad R Hispano** (<https://comunidadrhispano.es>): asociación de usuarios de R en español; organiza las Jornadas de R en España.
- **LatinR** (<https://latin-r.com>): conferencia latinoamericana sobre uso de R en investigación y desarrollo, con charlas y talleres en español y portugués.
- **Grupos locales de usuarios de R:** busca en Meetup o en redes sociales “R user group” o “useR” seguido de tu ciudad.

12.4 Cómo hacer una buena pregunta

Quien te ayuda no ve tu pantalla. Una buena pregunta incluye un **ejemplo reproducible** (*reprex*): el código mínimo que produce el problema, unos pocos datos de ejemplo (creados en el propio código, nunca datos confidenciales de tu empresa), el mensaje de error completo y lo que esperabas obtener. Muchas veces, al preparar el ejemplo mínimo, encontrarás tú mismo la solución. El paquete **reprex**, que se instala con el tidyverse, te ayuda a preparar ese ejemplo con formato listo para pegar en un foro.

Los asistentes de inteligencia artificial también pueden explicarte un mensaje de error o sugerirte código. Úsalos como a un compañero que a veces se equivoca: ejecuta lo que te propongan, comprueba que el resultado tenga sentido y asegúrate de entender cada línea antes de usarla en un reporte.

Resumen de la introducción

- **BI** convierte datos en información para decidir; **Business Analytics** añade explicación, predicción y recomendación. Hay cuatro tipos de analítica: descriptiva, diagnóstica, predictiva y prescriptiva.
- R y RStudio aportan análisis **reproducible**, estadística, gráficas de calidad, dashboards y reportes automáticos. Complementan a Excel, Power BI y Python.
- Todo proyecto de BI sigue un ciclo: pregunta, datos, ETL, análisis, comunicación, decisión y medición.
- Los bloques grises (**r**code) se ejecutan; los de franja naranja los ejecutas tú cuando haga falta; las salidas tienen franja azul verdosa. El **[1]** es un índice, no un dato.

Lista de verificación antes del Módulo 1:

- ☐ R y RStudio instalados y configurados (sin guardar **.RData**).
- ☐ Paquetes del curso instalados.
- ☐ Proyecto **curso-bi-rstudio.Rproj** creado.
- ☐ Datos generados con **source("datasets/generar_datasets.R")**.
- ☐ Script de verificación sin líneas **FALTA**.
- ☐ Tu primer análisis (ventas por trimestre) ejecutado.

¿Todo listo? Entonces abre tu proyecto y pasa al Módulo 1. Empezamos por lo más básico: cómo piensa R.

PARTE I

Nivel principiante

MÓDULO 1

Fundamentos de R y RStudio

Al terminar este módulo podrás...

- Moverte con soltura por RStudio: sus cuatro paneles, los scripts, los atajos de teclado y los Proyectos.
- Usar R como una calculadora de negocio (IVA, descuentos, márgenes, interés compuesto) y guardar resultados en variables.
- Distinguir los tipos de datos (números, texto, lógicos, fechas y factores) y convertir entre ellos sin caer en las trampas típicas.
- Trabajar con vectores, data frames, listas y matrices para responder preguntas de negocio.
- Automatizar decisiones con `if`, `ifelse()` y bucles, y crear tus propias funciones de BI.
- Instalar y cargar paquetes, importar datos desde CSV y Excel y exportar tus resultados.
- Resolver un primer análisis de ventas completo, de principio a fin, y saber qué hacer cuando R te muestra un error.

Imagina que es lunes y acabas de entrar como analista a una cadena de tiendas minoristas con diez sucursales en México. Tu jefa te comparte un archivo con las ventas de todo 2023 y te hace cuatro preguntas: *¿cuánto vendimos en el año?*, *¿cuál fue nuestro mejor y nuestro peor mes?*, *¿los días con descuento realmente venden más?* y *¿me lo puedes mandar en un Excel?* Con una hoja de cálculo podrías contestar, pero te tomaría una mañana entera de filtros, tablas dinámicas y fórmulas copiadas a mano. Y el próximo mes tendrías que repetirlo todo.

Con R vas a responder esas cuatro preguntas con unas cuantas líneas de código que puedes guardar, revisar y volver a ejecutar cada mes en segundos. Eso es lo que hace un analista de **Business Intelligence** (BI, inteligencia de negocios): convierte datos crudos en respuestas útiles para tomar decisiones, y lo hace de forma repetible.

Este módulo parte de cero. No necesitas haber programado nunca. Vamos a ir paso a paso: primero conocerás RStudio, luego aprenderás a guardar y calcular datos, después a organizarlos en tablas y, al final (sección 1.14), resolverás tú mismo el encargo de tu jefa con datos reales. Cada ejemplo tiene el código comentado, la salida exacta que imprime R y una explicación de qué significa para el negocio. Te recomiendo que escribas cada ejemplo en tu computadora en lugar de solo leerlo: programar se aprende con los dedos.

1.1 Tu primera sesión en RStudio

1.1.1 R y RStudio: el motor y el tablero

Antes de escribir código conviene distinguir dos programas que siempre aparecen juntos:

- **R** es el *lenguaje* y el motor que hace los cálculos. Puede funcionar solo, pero su ventana original es austera.
- **RStudio** es un **IDE** (entorno de desarrollo integrado): un programa que te da una interfaz cómoda para escribir, ejecutar y organizar tu código de R.

Piensa en un automóvil: R es el motor y RStudio es el tablero con volante, velocímetro y espejos. Necesitas los dos instalados, pero en el día a día solo abrirás RStudio, que a su vez usa R por dentro. Si todavía no los tienes, sigue las instrucciones de instalación de la introducción del libro (*Cómo usar este libro*) y vuelve aquí.

1.1.2 Los cuatro paneles

Al abrir RStudio verás una ventana dividida en cuatro zonas llamadas **paneles**. La figura 1.1 muestra su distribución habitual.



Figura 1.1. Los cuatro paneles de RStudio: (1) editor de scripts, (2) consola, (3) entorno e historial y (4) archivos, gráficas, paquetes y ayuda.

1. **Editor de scripts** (arriba a la izquierda). Es como un documento de Word para tu código: escribes, corriges y guardas. Si no lo ves, crea un script nuevo con **File > New File > R Script**.

2. **Consola** (abajo a la izquierda). Es donde R *ejecuta* las órdenes y responde. El símbolo **>** se llama **prompt** y significa “estoy listo, dime qué hago”.
3. **Entorno** (arriba a la derecha). La pestaña **Environment** muestra los datos que has creado (tus “objetos”); **History** guarda las órdenes que has ejecutado.
4. **Archivos, gráficas y ayuda** (abajo a la derecha). En **Files** navegas por tus carpetas, en **Plots** aparecen las gráficas, en **Packages** ves los paquetes instalados y en **Help** se abre la documentación.

Consejo

Puedes cambiar el tema de colores, el tamaño de letra y la distribución de los paneles en **Tools > Global Options > Appearance** y **Pane Layout**. Muchos analistas prefieren un tema oscuro para cansar menos la vista. No afecta en nada a tu código.

1.1.3 Consola contra script

Haz clic en la consola, escribe `2 + 2` y presiona **Enter**. R responde de inmediato:

```
1 2 + 2
```

```
[1] 4
```

El **[1]** no es parte del resultado: indica que ese es el *primer* elemento de la respuesta. Cuando R muestre respuestas largas, verás al inicio de cada renglón **[1]**, **[13]**, etc., para que sepas en qué posición vas.

La consola es cómoda para pruebas rápidas, pero tiene un problema: lo que escribes ahí se pierde al cerrar RStudio. Es como hablar por teléfono con R. El **script**, en cambio, es un archivo de texto con extensión `.R` donde escribes tus instrucciones para guardarlas y volver a ejecutarlas cuando quieras. Es como dejar la receta por escrito.

Concepto clave: Regla de oro

Escribe tu trabajo *siempre* en un script y ejecútalo desde ahí. Usa la consola solo para consultas rápidas (“¿cuánto vale esta variable?”, “¿cómo se llama esta columna?”). Un análisis que solo vive en la consola no se puede revisar, corregir ni repetir el próximo mes.

Para ejecutar código desde el script tienes tres opciones:

- **Una línea:** coloca el cursor en cualquier parte de la línea y presiona **Ctrl** + **Enter** (en Mac, **Cmd** + **Return**). El cursor baja solo a la siguiente instrucción, así que puedes ir presionando el atajo línea por línea.
- **Una selección:** marca varias líneas con el ratón y usa el mismo atajo; se ejecuta solo lo seleccionado.

- **Todo el script:** `Ctrl` + `Shift` + `Enter` (en Mac, `Cmd` + `Shift` + `Return`), o el botón **Source**.

El resultado siempre aparece en la consola, precedido de la instrucción que se ejecutó.

1.1.4 Comentarios: notas para tu yo del futuro

Todo lo que escribas después del símbolo `#` es un **comentario**: R lo ignora por completo. Los comentarios sirven para explicar *qué* hace el código y, sobre todo, *por qué*. En este libro todos los ejemplos van comentados en español.

```
1 # Esto es un comentario: R no lo ejecuta
2 120 + 95 + 140    # Ventas de tres días (también puede ir al final)
3
4 # ---- Sección: cálculo de IVA -----
5 1500 * 0.16       # IVA de una venta de $1,500
```

```
[1] 355
[1] 240
```

La línea `# -- Sección: ... --` es un truco útil: RStudio reconoce los comentarios que terminan en cuatro guiones o más como **secciones** y te permite saltar entre ellas con el menú que aparece en la parte inferior del editor. En scripts largos es una forma muy práctica de organizarte.

Hazlo tú

Crea un script nuevo (`Ctrl` + `Shift` + `N`), escribe un comentario con tu nombre y la fecha, y debajo calcula cuánto suman tus tres últimas compras del súper. Ejecuta la línea con `Ctrl` + `Enter` y observa la consola.

1.1.5 Atajos de teclado que te ahorrarán horas

No necesitas memorizarlos hoy; vuelve a esta tabla cuando la necesites. Con el tiempo los usarás sin pensar.

Tabla 1.1. Atajos de teclado más útiles de RStudio.

Acción	Windows / Linux	Mac
Ejecutar línea o selección	Ctrl + Enter	Cmd + Return
Ejecutar todo el script	Ctrl + Shift + Enter	Cmd + Shift + Return
Nuevo script	Ctrl + Shift + N	Cmd + Shift + N
Guardar el script	Ctrl + S	Cmd + S
Escribir la flecha de asignación <-	Alt + -	Option + -
Escribir el pipe %>%	Ctrl + Shift + M	Cmd + Shift + M
Comentar o descomentar líneas	Ctrl + Shift + C	Cmd + Shift + C
Autocompletar nombres	Tab	Tab
Recuperar la orden anterior (en la consola)	↑	↑
Limpiar la consola	Ctrl + L	Ctrl + L
Ir al editor / ir a la consola	Ctrl + 1 / Ctrl + 2	Ctrl + 1 / Ctrl + 2
Cancelar una orden que no termina	Esc	Esc
Reiniciar R (sesión limpia)	Ctrl + Shift + F10	Cmd + Shift + 0
Ayuda de la función bajo el cursor	F1	F1
Ver todos los atajos	Alt + Shift + K	Option + Shift + K

▲ Cuidado

Si en la consola aparece un signo **+** en lugar de **>**, R te está diciendo “tu instrucción no está completa, sigo esperando”. Casi siempre falta cerrar un paréntesis o unas comillas. Presiona **Esc** para cancelar, corrige la línea en el script y vuelve a ejecutarla.

1.1.6 Guardar tus scripts

Guarda con **Ctrl** + **S**. La primera vez RStudio te pedirá un nombre: usa nombres descriptivos, sin espacios ni acentos y con la extensión **.R**. Dos buenos ejemplos son **01_primeros_pasos.R** y **reporte_ventas_mensual.R**. Si el nombre de la pestaña del editor aparece en rojo con un asterisco, hay cambios sin guardar.

📎 Consejo

Numerar los scripts (**01_importar.R**, **02_limpiar.R**, **03_analizar.R**) deja claro el orden en que deben ejecutarse. Tu colega (o tú en seis meses) lo agradecerá.

1.1.7 Proyectos de RStudio y directorio de trabajo

Cuando le pides a R que abra un archivo, por ejemplo **datasets/ventas_retail.csv**, ¿dónde lo busca? En su **directorio de trabajo** (*working directory*): la carpeta de tu computadora que R toma como “aquí”. Puedes preguntarle cuál es con **getwd()** (*get working directory*):

```
1 getwd() # ¿En qué carpeta está trabajando R?
```

R responde con una ruta que depende de tu computadora. En Windows se verá algo como

```
"C:/Users/Ana/Documents/curso-bi-rstudio"
```

y en Mac algo como `"/Users/ana/Documents/curso-bi-rstudio"`. Lo importante es que esa carpeta sea la del curso, la que contiene `datasets/`.

La forma profesional de lograrlo son los **Proyectos de RStudio**. Un proyecto es una carpeta con un archivo `.Rproj`; al abrirlo, RStudio fija automáticamente el directorio de trabajo en esa carpeta. Es como tener un cajón por cliente: todo lo del proyecto (datos, scripts, resultados) vive junto.

Para convertir la carpeta del curso en un proyecto:

1. Ve a **File > New Project > Existing Directory**.
2. Con **Browse** elige la carpeta `curso-bi-rstudio` y presiona **Create Project**.
3. RStudio se reinicia dentro del proyecto: verás su nombre arriba a la derecha y el archivo `curso-bi-rstudio.Rproj` en el panel **Files**.

A partir de ahora, abre siempre el curso con doble clic en el archivo `.Rproj` (o desde **File > Open Project**). La estructura de carpetas con la que trabajaremos es esta:

```
1 curso-bi-rstudio/
2 |-- curso-bi-rstudio.Rproj      <- abre el proyecto con doble clic
3 |-- datasets/                  <- datos del curso
4 |   |-- generar_datasets.R
5 |   |-- ventas_retail.csv
6 |   |-- clientes.csv
7 |   |-- ...
8 |   `-- datos_empresa.xlsx
9 |-- mis_scripts/               <- aquí guarda tus scripts
10 `-- resultados/               <- aquí R guardará lo que exportes
```

Si todavía no generas los datos del curso, hazlo ahora *una sola vez* desde la consola, con el proyecto abierto:

```
1 # Crea los archivos CSV y Excel de la carpeta datasets/ (solo una vez)
2 source("datasets/generar_datasets.R")
```

La función `source()` ejecuta de principio a fin el script que le indiques. Al terminar verás un mensaje que confirma los siete archivos creados. Comprueba que R los encuentra:

```
1 file.exists("datasets/ventas_retail.csv") # ¿Existe el archivo?
2 list.files("datasets")                   # ¿Qué hay en la carpeta?
```

```
[1] TRUE
[1] "clientes.csv"      "datos_empresa.xlsx" "empleados.csv"
```

```
[4] "generar_datasets.R" "productos.csv"      "tiendas.csv"
[7] "transacciones.csv" "ventas_retail.csv"
```

Fíjate en que escribimos "datasets/ventas_retail.csv" y no la dirección completa del archivo. Eso es una **ruta relativa**: se interpreta *a partir* del directorio de trabajo, como cuando dices “la segunda puerta a la derecha”. Una **ruta absoluta** es la dirección completa, como "C:/Users/Ana/Documents/.../ventas_retail.csv", y solo funciona en la computadora de Ana.

```
1 # Cambiar el directorio de trabajo a mano (NO recomendado)
2 setwd("C:/Users/Ana/Documents/curso-bi-rstudio")      # Windows
3 setwd("/Users/ana/Documents/curso-bi-rstudio")        # Mac
```

▲ Cuidado

Verás `setwd()` en muchos tutoriales, pero evítalo dentro de tus scripts: la ruta solo existe en tu computadora, así que el script falla en cuanto se lo compartes a alguien (o cuando cambias de equipo). Con un Proyecto de RStudio y rutas relativas, tu código funciona igual en cualquier computadora.

🔗 Hazlo tú

Abre el proyecto del curso, ejecuta `getwd()` y confirma que la ruta termina en `curso-bi-rstudio`. Después crea la carpeta `mis_scripts` desde el panel **Files** (botón **New Folder**) y guarda ahí tu primer script.

1.2 R como calculadora de negocio

La forma más sencilla de usar R es como una calculadora muy potente. Los **operadores aritméticos** son los de siempre, más dos que te serán útiles con inventarios:

Tabla 1.2. Operadores aritméticos de R.

Operador	Significado	Ejemplo de negocio
+	Suma	Ventas de enero más ventas de febrero
-	Resta	Ingresos menos costos
*	Multiplicación	Precio por unidades vendidas
/	División	Ventas entre número de tickets
^	Potencia	Crecimiento compuesto $(1 + r)^n$
%/ %	División entera	Cajas completas que salen de un pedido
%%	Residuo (módulo)	Piezas sueltas que sobran

```
1 15000 + 18500      # Ventas de enero más febrero
2 485000 - 291000    # Ingresos menos costo de lo vendido
```

```

3 349 * 85           # Precio de un mouse por unidades vendidas
4 103700 / 7         # Venta promedio diaria de una semana
5 2^10              # Potencia: 2 elevado a la 10
6 125 %/% 12         # 125 piezas en cajas de 12: cajas completas
7 125 %% 12          # ... y piezas sueltas que sobran

```

```

[1] 33500
[1] 194000
[1] 29665
[1] 14814.29
[1] 1024
[1] 10
[1] 5

```

Con 125 piezas llenas 10 cajas de 12 y te sobran 5. Ese tipo de cálculo aparece a diario en almacenes y compras.

1.2.1 El orden de las operaciones importa

R respeta la **precedencia** de operadores que aprendiste en la escuela: primero paréntesis, luego potencias, después multiplicaciones y divisiones, y al final sumas y restas. Un paréntesis mal puesto cambia por completo un resultado de negocio:

```

1 # Precio de $1,000 más 16% de IVA
2 1000 + 1000 * 0.16      # Correcto: primero el IVA, luego la suma
3 (1000 + 1000) * 0.16    # Incorrecto: suma primero y luego multiplica
4 1000 * (1 + 0.16)       # Forma compacta y correcta

```

```

[1] 1160
[1] 320
[1] 1160

```

Consejo

Cuando dudes, usa paréntesis. No cuestan nada y hacen el código más fácil de leer: `(ingresos - costos) / ingresos` se entiende al instante.

1.2.2 Funciones matemáticas

Una **función** es una orden con nombre que recibe datos entre paréntesis y devuelve un resultado, igual que `=REDONDEAR(A1; 2)` en Excel. Los datos que recibe se llaman **argumentos** y se separan con comas.

```

1 round(37.77778, 1)      # Redondear a 1 decimal
2 round(16149.15)         # Sin segundo argumento redondea a entero
3 ceiling(125 / 12)       # Redondear hacia arriba: cajas necesarias
4 floor(9.99)             # Redondear hacia abajo
5 sqrt(144)              # Raíz cuadrada
6 abs(95000 - 102000)     # Valor absoluto: error del pronóstico
7 log(100)               # Logaritmo natural
8 log10(1000)            # Logaritmo base 10
9 exp(1)                 # El número e

```

```

[1] 37.8
[1] 16149
[1] 11
[1] 9
[1] 12
[1] 7000
[1] 4.60517
[1] 3
[1] 2.718282

```

Observa la diferencia entre `round()`, `ceiling()` y `floor()`. Si necesitas empacar 125 piezas en cajas de 12, no te sirven 10.42 cajas ni 10 cajas: necesitas 11, así que usas `ceiling()`. Y `abs()` te da el tamaño de un error sin importar si te pasaste o te quedaste corto: el pronóstico falló por 7,000 pesos.

1.2.3 Ejemplos de negocio

Ejemplo práctico: IVA y descuentos

Una tienda vende un producto en \$1,250 antes de impuestos. ¿Cuánto es el IVA del 16 % y cuánto paga el cliente? Si un ticket ya incluye IVA y suma \$2,320, ¿cuál era el precio sin IVA? Y si a una laptop de \$18,999 le aplicas 15 % de descuento, ¿en cuánto queda?

```

1 1250 * 0.16             # IVA de un producto de $1,250
2 1250 * 1.16             # Total que paga el cliente
3 2320 / 1.16             # Precio sin IVA de un ticket de $2,320
4 2320 - 2320 / 1.16      # IVA incluido en ese ticket
5 18999 * (1 - 0.15)      # Laptop con 15% de descuento

```

```

[1] 200
[1] 1450
[1] 2000
[1] 320
[1] 16149.15

```

El cliente paga \$1,450, de los cuales \$200 son IVA. El ticket de \$2,320 tenía una base de \$2,000 y \$320 de impuesto, y la laptop con descuento queda en \$16,149.15. Nota que para quitar el IVA *se divide* entre 1.16; restar el 16 % del total sería un error muy común (y caro).

▲ Cuidado

Los descuentos sucesivos no se suman. Un 20 % seguido de un 10 % adicional no es un 30 %: $1000 * 0.80 * 0.90$ da 720, es decir, un descuento efectivo del 28 %. Cuando diseñes promociones, calcula siempre el precio final.

Ejemplo práctico: Interés compuesto y crecimiento

Inviertes \$50,000 al 10 % anual durante 3 años. ¿Cuánto tendrás? ¿Y si la tasa es del 12 % anual pero se capitaliza cada mes? Por otro lado, las ventas de tu empresa pasaron de \$1.2 millones en 2020 a \$1.8 millones en 2023: ¿cuál fue su crecimiento anual compuesto?

```
1 50000 * (1 + 0.10)^3           # Capital después de 3 años al 10%
2 50000 * (1 + 0.12 / 12)^36      # 12% anual capitalizable cada mes
3 (1800000 / 1200000)^(1 / 3) - 1 # Crecimiento anual compuesto (TCAC)
4 log(2) / log(1.08)            # Años para duplicar ventas creciendo 8%
```

```
[1] 66550
[1] 71538.44
[1] 0.1447142
[1] 9.006468
```

La inversión llega a \$66,550 con capitalización anual y a \$71,538.44 si se capitaliza cada mes. Las ventas crecieron en promedio 14.47 % cada año: esa es la **tasa de crecimiento anual compuesta** (TCAC o *CAGR* en inglés), un indicador que verás en muchos informes financieros. Y con un crecimiento del 8 % anual, las ventas se duplican en unos 9 años.

Ejemplo práctico: Margen de un producto

Un producto se vende en \$450 y cuesta \$280. ¿Cuánto ganas por unidad, cuál es el margen sobre el precio y cuál es el *markup* (ganancia sobre el costo)? ¿Y cuál es la cantidad económica de pedido si vendes 12,000 unidades al año, cada pedido cuesta \$500 y mantener una unidad en inventario cuesta \$12 al año?

```
1 450 - 280                     # Ganancia por unidad
2 (450 - 280) / 450 * 100       # Margen sobre el precio (%)
3 (450 - 280) / 280 * 100       # Markup sobre el costo (%)
4 sqrt(2 * 12000 * 500 / 12)    # Cantidad económica de pedido (EOQ)
```

```
[1] 170
[1] 37.77778
[1] 60.71429
[1] 1000
```

Ganas \$170 por unidad: un margen de 37.8% sobre el precio o un markup de 60.7% sobre el costo. Son dos formas distintas de expresar la misma ganancia, y confundirlas es un error clásico en las juntas de precios. La fórmula de la **cantidad económica de pedido** (EOQ) dice que lo más barato es pedir 1,000 unidades cada vez.

Hazlo tú

Un cliente compra 3 artículos de \$349, \$1,299 y \$89 (precios sin IVA). Calcula en R el subtotal, el IVA y el total. Después aplica un cupón del 10% sobre el subtotal *antes* del IVA y compara. Pista: usa paréntesis.

1.3 Variables: guarda tus resultados con nombre

Hasta ahora R calcula y olvida. Para reutilizar un valor necesitas guardarlo en una **variable** (en R también se le llama **objeto**). La mejor analogía es una celda de Excel a la que le pones nombre: en lugar de recordar que las ventas de enero están en B2, las llamas **ventas_enero**.

1.3.1 Asignación con <-

Para guardar un valor se usa la flecha <-, que se lee “recibe”:

```
1 ventas_enero <- 15000      # ventas_enero "recibe" 15000
2 ventas_febrero <- 18500
3 ventas_marzo <- 22000
```

Al ejecutar estas líneas, la consola no muestra nada (asignar no imprime), pero en el panel **Environment** aparecen las tres variables con su valor. Para ver el contenido, escribe su nombre o usa **print()**:

```
1 ventas_enero      # Escribir el nombre muestra el valor
2 print(ventas_febrero) # print() hace lo mismo de forma explícita
3 total_trimestre <- ventas_enero + ventas_febrero + ventas_marzo
4 total_trimestre
5 promedio_mensual <- total_trimestre / 3
6 promedio_mensual
```

```
[1] 15000
[1] 18500
```



```
[1] 55500
[1] 18500
```

Ahora las variables funcionan como las celdas de una hoja de cálculo: si mañana corriges `ventas_marzo`, basta volver a ejecutar las líneas de abajo para que el total y el promedio se actualicen.



Consejo

El atajo `(Alt) + (-)` (en Mac, `(Option) + (-)`) escribe `<-` con espacios incluidos. También funciona `=` para asignar, pero la comunidad de R usa `<-` y así lo verás en todos los manuales. Si quieres asignar e imprimir a la vez, rodea la línea con paréntesis: `(meta <- 50000)`.

1.3.2 Reglas para nombrar variables

Un buen nombre explica qué hay dentro. R impone algunas reglas y la comunidad añade algunas convenciones:

- Deben empezar con una letra (o un punto) y pueden contener letras, números, puntos y guiones bajos: `ventas_2023` es válido, `2023_ventas` no.
- No pueden tener espacios ni guiones medios: `ventas enero` y `ventas-enero` provocan errores (el guion se interpreta como una resta).
- R distingue mayúsculas de minúsculas: `Ventas` y `ventas` son dos variables distintas.
- No uses palabras reservadas (`if`, `for`, `TRUE`, `function...`) ni nombres de funciones existentes (`c`, `mean`, `sum`, `data`).
- Evita acentos y la ñ en los nombres de variables (`anio` en lugar de `año`): funcionan, pero pueden dar problemas al abrir el script en otra computadora.

En este libro usamos el estilo **snake_case**: palabras en minúsculas unidas con guion bajo, como `ventas_netas_mensuales`. Es el estilo más común en R y el que recomienda la guía de estilo del *tidyverse*.

Tabla 1.3. Nombres de variables: buenos y malos.

Nombre	¿Sirve?	Comentario
<code>ventas_netas_2023</code>	✓ Recomendado	Descriptivo y en snake_case
<code>margen_pct</code>	✓ Recomendado	Abreviatura clara
<code>VentasNetas</code>	Funciona	Estilo camelCase, poco usado en R
<code>x, dato2</code>	Funciona	No dice qué contiene
<code>ventas netas</code>	✗ Error	Tiene un espacio
<code>2023_ventas</code>	✗ Error	Empieza con número
<code>ventas-netas</code>	✗ Error	R lo lee como “ventas menos netas”

1.3.3 Sobrescribir, listar y borrar variables

Si asignas un nuevo valor a una variable existente, el valor anterior se reemplaza sin aviso. Esto es útil para ir acumulando:

```
1 caja <- 5000           # Fondo de caja al abrir la tienda
2 caja <- caja + 1850     # Primera venta del día
3 caja <- caja + 920      # Segunda venta
4 caja <- caja - 300      # Pago a un proveedor en efectivo
5 caja
```

```
[1] 7470
```

La línea `caja <- caja + 1850` se lee: “toma el valor actual de `caja`, súmale 1,850 y guarda el resultado otra vez en `caja`”. Al final del día hay \$7,470 en caja.

Para saber qué variables existen usa `ls()`, que las lista todas (es lo mismo que ves en el panel **Environment**); con el argumento `pattern` puedes quedarte solo con las que contienen cierto texto. Para borrar una variable usa `rm()`:

```
1 ls(pattern = "ventas")  # Objetos cuyo nombre contiene "ventas"
2 rm(caja)                # Borra la variable caja
3 exists("caja")          # ¿Todavía existe?
```

```
[1] "ventas_enero"  "ventas_febrero" "ventas_marzo"
[1] FALSE
```

```
1 rm(list = ls())  # Borra TODOS los objetos del entorno (¡cuidado!)
```

Consejo

En lugar de borrar todo con `rm(list = ls())`, es mejor reiniciar R (**Session > Restart R**) y volver a ejecutar tu script desde el principio. Así compruebas que tu análisis funciona “desde cero” y no depende de algo que creaste a mano en la consola. Configura también **Tools > Global Options > General** para que RStudio *no* guarde ni restaure el archivo `.RData` al cerrar: tu script es la fuente de verdad, no la memoria de la sesión.

1.3.4 Mini caso: los KPIs de una tienda

Un **KPI** (*Key Performance Indicator*, indicador clave de desempeño) es un número que resume qué tan bien va el negocio. Vamos a calcular los KPIs básicos de una sucursal.

Ejemplo práctico: Tablero de la Sucursal Centro

En marzo la Sucursal Centro vendió \$485,000. El costo de la mercancía vendida fue de \$291,000 y los gastos de operación (renta, nómina, luz) sumaron \$98,500. La meta del mes era de \$520,000. Calcula la utilidad bruta, la utilidad de operación, los márgenes y el porcentaje de cumplimiento de la meta.

```
1 # ---- Datos de entrada -----
2 ingresos <- 485000
3 costo_ventas <- 291000
4 gastos_operacion <- 98500
5 meta_ventas <- 520000
6
7 # ---- KPIs -----
8 utilidad_bruta <- ingresos - costo_ventas
9 utilidad_operacion <- utilidad_bruta - gastos_operacion
10 margen_bruto_pct <- utilidad_bruta / ingresos * 100
11 margen_operacion_pct <- utilidad_operacion / ingresos * 100
12 cumplimiento_pct <- ingresos / meta_ventas * 100
13 faltante_meta <- meta_ventas - ingresos
14
15 # ---- Resultados -----
16 utilidad_bruta
17 utilidad_operacion
18 round(margen_bruto_pct, 1)
19 round(margen_operacion_pct, 1)
20 round(cumplimiento_pct, 1)
21 faltante_meta
```

```
[1] 194000
[1] 95500
[1] 40
[1] 19.7
[1] 93.3
[1] 35000
```

¿Qué nos dice esto? La sucursal genera \$194,000 de utilidad bruta (40 % de margen) y, después de pagar sus gastos, le quedan \$95,500 de utilidad de operación: por cada 100 pesos vendidos gana 19.7. Sin embargo, solo cumplió el 93.3 % de la meta y le faltaron \$35,000. Si el próximo mes cambian las cifras, solo actualizas las cuatro líneas de “Datos de entrada” y vuelves a ejecutar todo: ese es el poder de un script.

Hazlo tú

Crea variables con los datos de una tienda imaginaria (ingresos, costo de ventas, gastos y meta) y calcula los mismos seis KPIs. Después cambia solo los ingresos a un valor que supere la meta

y vuelve a ejecutar: ¿qué pasa con `faltante_meta`? ¿Cómo lo interpretarías?

1.4 Tipos de datos

En Excel una celda puede contener un número, un texto, una fecha o un VERDADERO/FALSO, y Excel decide qué operaciones permite. En R pasa lo mismo: cada valor tiene un **tipo de dato**, y el tipo determina qué puedes hacer con él. Sumar dos precios tiene sentido; sumar dos nombres de sucursal, no. La mayoría de los errores de un principiante vienen de un tipo equivocado, así que vale la pena conocerlos bien.

Tabla 1.4. Tipos de datos básicos de R.

Tipo (<code>class</code>)	Ejemplo	Uso en negocio
<code>numeric</code>	349.90, 12	Precios, montos, cantidades, porcentajes
<code>integer</code>	42L	Conteos (tickets, clientes); la <code>L</code> indica entero
<code>character</code>	"Sucursal Centro"	Nombres, ciudades, códigos de producto
<code>logical</code>	TRUE, FALSE	¿Es premium? ¿Cumplió la meta? ¿Tiene descuento?
<code>Date</code>	<code>as.Date("2023-03-15")</code>	Fechas de venta, de registro, de vencimiento
<code>factor</code>	<code>factor(c("CH", "M"))</code>	Categorías con valores fijos: tallas, regiones, niveles

1.4.1 Números, texto y lógicos

La función `class()` te dice de qué tipo es un objeto:

```

1 precio <- 349.90           # numeric: número con decimales
2 unidades <- 12             # también numeric, aunque sea entero
3 tickets <- 42L             # integer: la L fuerza un entero
4 sucursal <- "Sucursal Centro" # character: texto entre comillas
5 es_premium <- TRUE         # logical: verdadero o falso
6
7 class(precio)
8 class(unidades)
9 class(tickets)
10 class(sucursal)
11 class(es_premium)

```

```

[1] "numeric"
[1] "numeric"
[1] "integer"
[1] "character"
[1] "logical"

```

Tres detalles importantes:

- El texto va *siempre* entre comillas (dobles o simples). Sin comillas, R cree que **Centro** es el nombre de una variable y la busca.
- **TRUE** y **FALSE** van en mayúsculas y sin comillas. **"TRUE"** (con comillas) es un texto, no un lógico.
- En la práctica casi nunca necesitas escribir la **L**: R trabaja con **numeric** y lo convierte cuando hace falta.

Existe también `typeof()`, que muestra cómo guarda R el dato por dentro. La diferencia se entiende con las fechas, como verás en un momento.

```
1 typeof(precio)      # "double": número de doble precisión
2 typeof(tickets)
```

```
[1] "double"
[1] "integer"
```

Los valores lógicos aparecen cada vez que haces una **comparación**. Los operadores de comparación son `==` (igual a; ¡con dos signos!), `!=` (distinto de), `>`, `<`, `>=` y `<=`:

```
1 ventas_hoy <- 15600
2 meta_diaria <- 14000
3 ventas_hoy >= meta_diaria      # ¿Cumplimos la meta de hoy?
4 sucursal == "Sucursal Norte"  # ¿Es la sucursal Norte?
5 TRUE + TRUE + FALSE          # TRUE vale 1 y FALSE vale 0
```

```
[1] TRUE
[1] FALSE
[1] 2
```

Que **TRUE** valga 1 y **FALSE** valga 0 parece una curiosidad, pero es un truco muy poderoso: más adelante contarás “cuántos días se cumplió la meta” simplemente sumando comparaciones.

▲ Cuidado

`=` y `==` no son lo mismo. `x = 5` *asigna* el valor 5 a `x`; `x == 5` *pregunta* si `x` vale 5 y responde **TRUE** o **FALSE**. Confundirlos es uno de los errores más frecuentes.

1.4.2 Comprobar y convertir tipos

Las funciones `is.*()` preguntan si un objeto es de cierto tipo y las funciones `as.*()` lo convierten a otro:

```

1 is.numeric(precio)           # ¿Es número?
2 is.character(precio)        # ¿Es texto?
3 as.character(2023)          # Número -> texto
4 as.numeric("349.90")        # Texto -> número
5 as.integer(3.9)              # ¡Cuidado! Trunca, no redondea
6 as.logical(c("TRUE", "FALSE", "T", "sí"))

```

```

[1] TRUE
[1] FALSE
[1] "2023"
[1] 349.9
[1] 3
[1] TRUE FALSE TRUE NA

```

Fíjate en dos cosas: `as.integer(3.9)` devuelve 3, porque *corta* los decimales en lugar de redondear; y `as.logical()` no entiende "sí", así que devuelve **NA**. **NA** significa “no disponible” (*Not Available*): es la forma en que R marca un dato faltante o que no pudo interpretar. Lo estudiarás con calma en la sección 1.5.

1.4.3 Tres trampas típicas

Trampa 1: números con comas o signo de pesos

Cuando los datos vienen de un sistema o de un Excel mal exportado, los montos a veces llegan como texto con separador de miles o con signo de pesos. R no sabe interpretarlos:

```

1 as.numeric("1,234")          # La coma de miles confunde a R

```

```

[1] NA
Warning message:
NAs introduced by coercion

```

R devuelve **NA** y una **advertencia** (*warning*): “NAs introducidos por coerción”. Una advertencia no detiene el programa, pero te avisa que algo salió raro. Si tu R está en español verás el mensaje traducido; en este libro los mostramos en inglés, que es como los encontrarás en los foros de ayuda. La solución es quitar primero los caracteres que estorban con `gsub()` (que estudiarás en la sección 1.6):

```

1 as.numeric(gsub(",", "", "1,234"))          # Quita la coma y convierte
2 monto_texto <- "$12,850.50"
3 as.numeric(gsub("$", "", monto_texto))      # Quita $ y comas

```

```
[1] 1234
[1] 12850.5
```

Los corchetes en "`[$,]`" significan “cualquiera de estos caracteres”: R borra todos los signos `$` y todas las comas.

Trampa 2: sumar un número guardado como texto

Si un número está guardado como texto (entre comillas), R se niega a hacer aritmética con él:

```
1 precio_texto <- "100"
2 precio_texto + 1
```

```
Error in precio_texto + 1 : non-numeric argument to binary operator
```

El mensaje dice “argumento no numérico en un operador binario”: uno de los dos lados de la suma no es un número. La solución es convertirlo: `as.numeric(precio_texto) + 1`. Cuando veas este error, revisa con `class()` el tipo de tus variables.

Trampa 3: factores que parecen números

Un **factor** es el tipo que usa R para *categorías*: variables que solo pueden tomar unos cuantos valores conocidos, como tallas, regiones o niveles de cliente. Internamente guarda cada categoría como un número (1, 2, 3...) y una lista de etiquetas llamadas **niveles** (*levels*).

```
1 tallas <- c("M", "G", "CH", "M", "M", "G", "CH", "EG")
2 tallas_f <- factor(tallas)           # Niveles en orden alfabético
3 tallas_f
4 levels(tallas_f)
5 table(tallas_f)                     # Conteo por categoría
```

```
[1] M  G  CH M  M  G  CH EG
Levels: CH EG G M
[1] "CH" "EG" "G"  "M"
tallas_f
CH EG  G  M
  2  1  2  3
```

Los niveles quedaron en orden alfabético (CH, EG, G, M), que no es el orden lógico de las tallas. Para un reporte conviene fijarlo con el argumento `levels`:

```
1 tallas_f <- factor(tallas, levels = c("CH", "M", "G", "EG"))
2 table(tallas_f)                                # Ahora en el orden correcto
```

```
tallas_f
CH  M  G EG
 2  3  2  1
```

Ahora la trampa. Supón que una columna de precios quedó guardada como factor (pasaba mucho con versiones antiguas de R al importar un CSV). Si la conviertes directo a número, obtienes los *códigos internos* de los niveles, no los precios:

```
1 precios_f <- factor(c("150", "80", "150", "200"))
2 as.numeric(precios_f)                        # ¡MAL! Devuelve los códigos
3 as.numeric(as.character(precios_f))         # BIEN: primero a texto
```

```
[1] 1 3 1 2
[1] 150 80 150 200
```

El primer resultado (1 3 1 2) no tiene nada que ver con los precios: son las posiciones de cada valor en la lista de niveles "150", "200", "80". La regla es sencilla: para convertir un factor a número, pásalo primero a texto con `as.character()`.

1.4.4 Fechas

Las fechas merecen su propio tipo, `Date`, porque con ellas se hacen operaciones especiales: sumar días, calcular antigüedades, extraer el mes o el día de la semana. La función `as.Date()` convierte texto en fecha. Por defecto espera el formato internacional "año-mes-día":

```
1 fecha_pedido <- as.Date("2023-03-15")      # Formato ISO: AAAA-MM-DD
2 fecha_pedido
3 class(fecha_pedido)
4 typeof(fecha_pedido)                       # Por dentro es un número
5 as.numeric(fecha_pedido)                   # Días desde el 1-ene-1970
```

```
[1] "2023-03-15"
[1] "Date"
[1] "double"
[1] 19431
```

Aquí se ve la diferencia entre `class()` y `typeof()`: para ti y para las funciones de R es una fecha (`Date`), pero por dentro R la guarda como el número de días transcurridos desde el 1 de enero de 1970. Gracias a eso puede restar fechas o sumarle días.

Si tu fecha viene en el formato mexicano **día/mes/año**, díselo a R con el argumento **format**:

```
1 fecha_factura <- as.Date("28/02/2023", format = "%d/%m/%Y")
2 fecha_factura
3 vencimiento <- fecha_factura + 30           # Crédito a 30 días
4 vencimiento
5 fecha_pago <- as.Date("2023-04-10")
6 fecha_pago - vencimiento                   # ¿Cuántos días de atraso?
7 as.numeric(fecha_pago - vencimiento)      # Solo el número
```

```
[1] "2023-02-28"
[1] "2023-03-30"
Time difference of 11 days
[1] 11
```

La factura del 28 de febrero vencía el 30 de marzo y se pagó el 10 de abril: 11 días de atraso. La resta de fechas da un objeto especial llamado **difftime** (“diferencia de tiempo”); con **as.numeric()** te quedas solo con el número para usarlo en otros cálculos.

La fecha de hoy se obtiene con **Sys.Date()**. Su resultado, claro, depende del día en que la ejecutes:

```
1 hoy <- Sys.Date()           # Fecha de hoy según tu computadora
2 dias_desde_pedido <- as.numeric(hoy - fecha_pedido)
```

Para mostrar una fecha en otro formato se usa **format()** con los códigos de la tabla 1.5:

```
1 format(fecha_pedido, "%d/%m/%Y")           # Formato mexicano
2 format(fecha_pedido, "%A %d de %B de %Y")  # Formato largo
3 format(fecha_pedido, "%Y-%m")             # Año-mes para agrupar
4 weekdays(fecha_pedido)                    # Día de la semana
5 months(fecha_pedido)                      # Nombre del mes
```

```
[1] "15/03/2023"
[1] "miércoles 15 de marzo de 2023"
[1] "2023-03"
[1] "miércoles"
[1] "marzo"
```

Tabla 1.5. Códigos de formato de fechas más usados.

Código	Significado	Ejemplo (15 de marzo de 2023)
%d	Día con dos dígitos	15
%m	Mes con dos dígitos	03
%Y	Año con cuatro dígitos	2023
%y	Año con dos dígitos	23
%B	Nombre del mes	marzo
%b	Mes abreviado	mar
%A	Día de la semana	miércoles

▲ Cuidado

Los nombres de días y meses salen en el idioma de tu computadora. Si tu sistema está en inglés verás "Wednesday" y "March" en lugar de "miércoles" y "marzo". Tus cálculos no cambian; solo las etiquetas.

Ejemplo práctico: Frecuencia de compra de un cliente

Un cliente mayorista hizo cinco pedidos en el primer trimestre. ¿Cada cuántos días compra en promedio? Si mantiene su ritmo, ¿cuándo esperamos su siguiente pedido? Esta es la base de las alertas de “cliente en riesgo”: si pasa mucho más tiempo del habitual sin comprar, alguien de ventas debería llamarle.

```

1 pedidos <- as.Date(c("2023-01-05", "2023-01-19", "2023-02-09",
2                       "2023-02-23", "2023-03-18"))
3 diff(pedidos)                                # Días entre pedidos consecutivos
4 dias_promedio <- as.numeric(mean(diff(pedidos)))
5 dias_promedio
6 proximo_pedido <- max(pedidos) + dias_promedio
7 proximo_pedido
8 format(proximo_pedido, "%A %d/%m/%Y")

```

```

Time differences in days
[1] 14 21 14 23
[1] 18
[1] "2023-04-05"
[1] "miércoles 05/04/2023"

```

El cliente compra cada 18 días en promedio, así que esperamos su siguiente pedido alrededor del miércoles 5 de abril. La función `diff()` calcula la diferencia entre cada elemento y el anterior; la usarás mucho con ventas diarias o mensuales.

 Hazlo tú

Escribe tu fecha de nacimiento con `as.Date()` y calcula cuántos días has vivido restándola de `Sys.Date()`. ¿Qué día de la semana naciste? Pista: `format(fecha, "%A")`.

1.5 Vectores: muchos datos a la vez

Rara vez analizas un solo número. Normalmente tienes las ventas de cada día, los precios de cada producto o las edades de cada cliente. Para eso R usa **vectores**: colecciones ordenadas de valores *del mismo tipo*. Un vector es como una columna de Excel: una serie de celdas una tras otra. Y es la pieza con la que se construye todo lo demás en R.

1.5.1 Crear vectores

La función `c()` (de *combinar*) une valores en un vector:

```
1 ventas_semana <- c(12500, 9800, 11200, 13400, 18900, 22300, 15600)
2 dias <- c("lunes", "martes", "miércoles", "jueves", "viernes",
3          "sábado", "domingo")
4 ventas_semana
5 dias
6 length(ventas_semana)      # ¿Cuántos elementos tiene?
```

```
[1] 12500  9800 11200 13400 18900 22300 15600
[1] "lunes"    "martes"   "miércoles" "jueves"   "viernes"
[6] "sábado"   "domingo"
[1] 7
```

Como un vector solo admite un tipo, si mezclas números y texto R convierte *todo* a texto sin avisarte:

```
1 c(1500, "2300", 980)      # Un solo texto "contamina" el vector
```

```
[1] "1500" "2300" "980"
```

Los números ahora aparecen entre comillas: ya no puedes sumarlos. Cuando una columna numérica de un archivo trae un solo valor raro (por ejemplo, "N/D"), R suele leer la columna completa como texto. Ahora sabes por qué.

1.5.2 Secuencias y repeticiones

Escribir valores uno por uno es tedioso. Para patrones regulares existen atajos:

```

1 1:12                                # Del 1 al 12 (números de mes)
2 seq(0, 100000, by = 25000)         # De 0 a 100 mil, de 25 en 25 mil
3 seq(10, 100, length.out = 4)       # 4 valores equiespaciados
4 rep("Norte", times = 3)             # Repetir un valor
5 rep(c("Q1", "Q2"), times = 2)      # Repetir el vector completo
6 rep(c("Q1", "Q2"), each = 2)       # Repetir cada elemento
7 seq(as.Date("2023-01-01"), by = "month", length.out = 4) # Fechas

```

```

[1] 1 2 3 4 5 6 7 8 9 10 11 12
[1] 0 25000 50000 75000 100000
[1] 10 40 70 100
[1] "Norte" "Norte" "Norte"
[1] "Q1" "Q2" "Q1" "Q2"
[1] "Q1" "Q1" "Q2" "Q2"
[1] "2023-01-01" "2023-02-01" "2023-03-01" "2023-04-01"

```

La última línea es muy útil para construir calendarios: el primer día de cada mes, cada lunes, cada quincena.

1.5.3 Operaciones vectorizadas

Aquí empieza la magia de R. Las operaciones se aplican a *todos* los elementos del vector a la vez, sin que tengas que arrastrar una fórmula como en Excel. A esto se le llama **vectorización**.

Ejemplo práctico: Aplicar un descuento a todo el catálogo

El área comercial decide aplicar un 15 % de descuento a siete productos y necesita los precios nuevos, los precios con IVA y el ingreso esperado si se venden las unidades pronosticadas.

```

1 precios <- c(18999, 349, 1299, 4599, 899, 2499, 159)
2 precios * 0.85                                # 15% de descuento a los 7 de golpe
3 round(precios * 0.85 * 1.16, 2)              # Precio con descuento e IVA
4 unidades <- c(3, 40, 12, 5, 25, 8, 60)
5 ingresos <- precios * 0.85 * unidades        # Elemento por elemento
6 ingresos
7 sum(ingresos)                                # Ingreso total esperado

```

```

[1] 16149.15 296.65 1104.15 3909.15 764.15 2124.15 135.15
[1] 18733.01 344.11 1280.81 4534.61 886.41 2464.01 156.77
[1] 48447.45 11866.00 13249.80 19545.75 19103.75 16993.20 8109.00
[1] 137315

```

Cuando operas dos vectores del mismo largo, R empareja los elementos por posición: el primer precio con la primera cantidad, el segundo con la segunda, etc. El ingreso esperado de los siete productos

es de \$137,315.

¿Y si los vectores tienen distinto largo? R *recicla* el más corto: lo repite hasta completar el largo del otro. Esto se llama **reciclaje**:

```
1 c(1000, 2000, 3000, 4000) * c(1, 0.5)    # 1, 0.5, 1, 0.5
```

```
[1] 1000 1000 3000 2000
```

Si el largo del vector mayor no es múltiplo del menor, R hace el cálculo de todos modos, pero te lanza una advertencia:

```
1 c(1000, 2000, 3000) * c(1, 0.5)          # No es múltiplo: advertencia
```

```
[1] 1000 1000 3000
```

Warning message:

```
In c(1000, 2000, 3000) * c(1, 0.5) :
```

```
longer object length is not a multiple of shorter object length
```

El caso más común de reciclaje es multiplicar por un solo número (**precios * 0.85**): el 0.85 se recicla para cada precio. Si R te advierte que el largo del vector más largo no es múltiplo del más corto, casi siempre es porque te faltó o te sobró un dato: revisalo.

1.5.4 Funciones de resumen

Para describir un vector con pocos números existen las funciones de resumen. Primero pondremos nombre a cada elemento con **names()**, así los resultados serán más fáciles de leer:

```
1 names(ventas_semana) <- dias    # Cada venta con su día
2 ventas_semana
3 sum(ventas_semana)              # Total de la semana
4 mean(ventas_semana)            # Promedio diario
5 median(ventas_semana)          # Valor central
6 sd(ventas_semana)              # Desviación estándar
7 min(ventas_semana)
8 max(ventas_semana)
9 range(ventas_semana)           # Mínimo y máximo juntos
```

```
      lunes      martes miércoles   jueves   viernes   sábado   domingo
12500      9800      11200      13400      18900      22300      15600
[1] 103700
[1] 14814.29
[1] 13400
```

```
[1] 4451.752
[1] 9800
[1] 22300
[1] 9800 22300
```

La semana sumó \$103,700, con un promedio de \$14,814.29 por día. La **mediana** (\$13,400) es el valor que queda justo a la mitad al ordenar los datos: la mitad de los días vendió menos y la otra mitad más. Está por debajo del promedio porque el viernes y el sábado “jalan” el promedio hacia arriba. La **desviación estándar** (\$4,451.75) mide qué tanto se alejan los días del promedio: cuanto más grande, más irregulares son las ventas.

```
1 quantile(ventas_semana)           # Cuartiles
2 quantile(ventas_semana, probs = 0.9) # Percentil 90
3 cumsum(ventas_semana)             # Acumulado de la semana
4 diff(ventas_semana)               # Cambio respecto al día previo
5 summary(ventas_semana)            # Resumen en una línea
```

```
0%  25%  50%  75% 100%
9800 11850 13400 17250 22300
90%
20260
lunes    martes miércoles    jueves    viernes    sábado    domingo
12500    22300    33500    46900    65800    88100    103700
martes miércoles    jueves    viernes    sábado    domingo
-2700    1400    2200    5500    3400    -6700
Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
9800  11850   13400   14814  17250   22300
```

Los **cuartiles** dividen los datos ordenados en cuatro partes iguales: el 25 % de los días vendió menos de \$11,850 y el 75 % menos de \$17,250. El **acumulado** (`cumsum()`) responde preguntas como “¿en qué día superamos los \$50,000 de la semana?” (el jueves ya llevábamos \$46,900 y el viernes \$65,800). Y `diff()` muestra que la mayor subida fue del jueves al viernes (+\$5,500) y la mayor caída del sábado al domingo (\$6,700 menos). `summary()` resume casi todo en una línea.

1.5.5 Indexación: sacar elementos de un vector

Para obtener elementos concretos se usan corchetes `[ ]`. Es lo que en BI llamarías “filtrar”. Hay tres formas de indicar qué quieres.

Por posición

```
1 ventas_semana[1]           # El primer elemento (R empieza en 1)
2 ventas_semana[c(5, 6)]    # Posiciones 5 y 6
3 ventas_semana[1:5]        # De la 1 a la 5: días entre semana
4 ventas_semana[-7]         # Todos menos el séptimo
```

```
lunes
12500
viernes sábado
18900 22300
  lunes martes miércoles  jueves  viernes
  12500   9800   11200  13400   18900
lunes martes miércoles  jueves  viernes  sábado
12500   9800   11200  13400   18900  22300
```

Por nombre

```
1 ventas_semana["sábado"]
2 ventas_semana[c("sábado", "domingo")]
```

```
sábado
22300
sábado domingo
22300 15600
```

Por condición lógica

Esta es la forma más usada en análisis de datos. Primero escribes una comparación, que produce un vector de **TRUE/FALSE**, y luego la usas dentro de los corchetes: R se queda solo con los elementos marcados como **TRUE**.

```
1 meta_diaria <- 14000
2 ventas_semana > meta_diaria           # Vector lógico
3 ventas_semana[ventas_semana > meta_diaria] # Solo los días que cumplen
```

```
  lunes  martes miércoles  jueves  viernes  sábado  domingo
  FALSE  FALSE   FALSE   FALSE   TRUE   TRUE   TRUE
viernes sábado domingo
18900  22300  15600
```

Para saber *en qué posiciones* se cumple una condición usa `which()`, y para localizar el máximo o el mínimo, `which.max()` y `which.min()`:

```
1 which(ventas_semana > meta_diaria) # Posiciones que cumplen
2 which.max(ventas_semana)           # Posición del mejor día
3 names(which.max(ventas_semana))    # Solo el nombre del mejor día
4 names(which.min(ventas_semana))    # Nombre del peor día
```

```
viernes  sábado domingo
        5         6         7
sábado
        6
[1] "sábado"
[1] "martes"
```

1.5.6 Operadores lógicos

Para combinar condiciones se usan los **operadores lógicos**:

- `&` (“y”): verdadero solo si *ambas* condiciones lo son.
- `|` (“o”): verdadero si *al menos una* lo es.
- `!` (“no”): invierte el resultado.
- `%in%` (“está en”): verdadero si el valor pertenece a una lista.

```
1 fin_de_semana <- names(ventas_semana) %in% c("sábado", "domingo")
2 fin_de_semana
3 ventas_semana[fin_de_semana] # Sábado y domingo
4 ventas_semana[!fin_de_semana & ventas_semana > 12000] # Entre semana
5 ventas_semana[ventas_semana < 10000 | ventas_semana > 20000] # Extremos
```

```
[1] FALSE FALSE FALSE FALSE FALSE TRUE TRUE
sábado domingo
22300 15600
lunes  jueves viernes
12500 13400 18900
martes sábado
9800 22300
```

La tercera línea se lee: “días que *no* son fin de semana *y* vendieron más de \$12,000”. La cuarta encuentra los días atípicos: los muy bajos *o* los muy altos.

1.5.7 Contar con `sum()` y `mean()` de una condición

Recuerda que `TRUE` vale 1 y `FALSE` vale 0. Por lo tanto, sumar un vector lógico *cuenta* cuántos elementos cumplen la condición, y promediarlo da la *proporción*:

```
1 sum(ventas_semana > meta_diaria)      # ¿Cuántos días cumplieron?
2 mean(ventas_semana > meta_diaria)     # ¿Qué proporción de días?
```

```
[1] 3
[1] 0.4285714
```

Tres de siete días cumplieron la meta: el 42.9% de la semana. Este patrón, `sum(condición)` y `mean(condición)`, lo usarás constantemente: “¿cuántos clientes son premium?”, “¿qué porcentaje de pedidos llegó tarde?”.

1.5.8 Datos faltantes: `NA`

En la vida real faltan datos: el sistema de caja se cayó un día, un cliente no dio su edad, un producto no tiene costo capturado. R los marca con `NA`. Lo importante es que `NA` es “contagioso”: cualquier cálculo que lo incluya da `NA`, porque R no puede saber el resultado si le falta un dato.

```
1 ventas_tienda <- c(8200, 7650, NA, 9100, 10400, NA, 8800)
2 mean(ventas_tienda)                # NA: falta información
3 mean(ventas_tienda, na.rm = TRUE)  # Ignora los NA
4 sum(ventas_tienda, na.rm = TRUE)
5 is.na(ventas_tienda)               # ¿Dónde hay NA?
6 sum(is.na(ventas_tienda))           # ¿Cuántos NA hay?
7 which(is.na(ventas_tienda))         # ¿En qué posiciones?
```

```
[1] NA
[1] 8830
[1] 44150
[1] FALSE FALSE  TRUE FALSE FALSE  TRUE FALSE
[1] 2
[1] 3 6
```

El argumento `na.rm = TRUE` (*NA remove*) le dice a la función que ignore los faltantes. El promedio de los cinco días con datos es \$8,830. Fíjate en que es muy distinto a rellenar los faltantes con cero: eso bajaría el promedio a \$6,307 y haría creer que la tienda vendió peor.

▲ Cuidado

No reemplaces `NA` por 0 sin pensarlo. Un cero dice “no vendimos nada”; un `NA` dice “no sabemos cuánto vendimos”. Antes de decidir, averigua *por qué* falta el dato.

1.5.9 Ejemplo integrador: la semana de una tienda

Ejemplo práctico: Reporte semanal de ventas

El gerente quiere saber: ¿cuál fue el mejor día?, ¿qué días superaron la meta de \$14,000?, ¿cómo cambió la venta de un día a otro en porcentaje? y ¿cuáles fueron los tres mejores días?

```

1 # Mejor día y su venta
2 names(which.max(ventas_semana))
3 max(ventas_semana)
4
5 # Días que superaron la meta
6 names(ventas_semana)[ventas_semana > meta_diaria]
7
8 # Crecimiento porcentual respecto al día anterior
9 crecimiento_pct <- diff(ventas_semana) / head(ventas_semana, -1) * 100
10 round(crecimiento_pct, 1)
11
12 # Los tres mejores días (ordenados de mayor a menor)
13 sort(ventas_semana, decreasing = TRUE)[1:3]
```

```

[1] "sábado"
[1] 22300
[1] "viernes" "sábado" "domingo"
      martes miércoles   jueves   viernes   sábado   domingo
      -21.6      14.3    19.6    41.0    18.0   -30.0
sábado viernes domingo
22300   18900   15600
```

¿Qué nos dice esto? El sábado fue el mejor día (\$22,300) y solo viernes, sábado y domingo superaron la meta: la tienda depende del fin de semana. El crecimiento muestra un arranque débil (el martes cayó 21.6% respecto al lunes), una subida constante hasta el sábado y una caída del 30% el domingo. Una decisión razonable sería reforzar promociones de lunes a jueves en lugar de gastar más en el fin de semana, que ya funciona solo.

En el código aparecen dos funciones nuevas: `head(x, -1)` devuelve todos los elementos menos el último (para dividir cada diferencia entre el día *anterior*) y `sort()` ordena los valores; con `decreasing = TRUE` los pone de mayor a menor, y `[1:3]` se queda con los tres primeros.

Hazlo tú

Crea un vector con las ventas de los 12 meses de un negocio que conozcas (o inventa cifras) con los nombres "ene", "feb"... "dic". Calcula el total anual, el mejor mes con `names(which.max())`, cuántos meses superaron el promedio y el crecimiento porcentual de cada mes respecto al anterior.

1.6 Trabajar con texto

En BI el texto está en todas partes: nombres de clientes, ciudades, códigos de producto, y también los mensajes que tú produces (“La sucursal Norte vendió \$468,900”). R trae funciones para unir, formatear y limpiar texto. En programación, a un texto se le llama **cadena** (*string*).

1.6.1 Unir textos: `paste()` y `paste0()`

```
1 paste("Sucursal", "Norte")           # Une con un espacio
2 paste("Q", 1:4, sep = "")            # sep = "" no deja espacio
3 paste0("SKU-", 1:3)                  # paste0() = paste(sep = "")
4 paste("ventas", c("enero", "febrero"), "2023", sep = "_")
5 paste(c("CDMX", "Monterrey", "Puebla"), collapse = ", ")
```

```
[1] "Sucursal Norte"
[1] "Q1" "Q2" "Q3" "Q4"
[1] "SKU-1" "SKU-2" "SKU-3"
[1] "ventas_enero_2023" "ventas_febrero_2023"
[1] "CDMX, Monterrey, Puebla"
```

`paste()` también es vectorizada: con `1:4` produce cuatro textos a la vez, ideal para generar etiquetas de trimestres, códigos o nombres de archivo. El argumento `sep` define qué va *entre* las piezas y `collapse` junta todos los elementos de un vector en un solo texto.

1.6.2 Mensajes con formato: `sprintf()`

`sprintf()` funciona como una plantilla con huecos. Cada hueco empieza con `%` e indica qué tipo de valor va ahí: `%s` para texto, `%d` para enteros y `%.2f` para números con 2 decimales. Para escribir el propio signo de porcentaje se usa `%%`.

```
1 ventas_norte <- 468900.5
2 cumplimiento <- 97.6875
3 sprintf("La sucursal Norte vendió $%.2f", ventas_norte)
4 sprintf("Cumplimiento de meta: %.1f%%", cumplimiento)
5 sprintf("%s: %d tickets", c("Centro", "Norte"), c(1520L, 1204L))
6 sprintf("SKU-%04d", c(7, 42, 315))      # Rellena con ceros a 4 dígitos
```

```
[1] "La sucursal Norte vendió $468900.50"
[1] "Cumplimiento de meta: 97.7%"
[1] "Centro: 1520 tickets" "Norte: 1204 tickets"
[1] "SKU-0007" "SKU-0042" "SKU-0315"
```

El último ejemplo es muy práctico para construir códigos de producto o de cliente con longitud fija (SKU-0007). Nota que `sprintf()` no pone separador de miles; para eso está `format()`.

1.6.3 Números legibles: `format()`

```
1 monto <- 1234567.891
2 format(monto, big.mark = ",")           # Separador de miles
3 format(monto, big.mark = ",", nsmall = 2) # Con 2 decimales
4 montos <- c(513916.6, 1407.99, 54056.19)
5 format(montos, big.mark = ",", nsmall = 2) # Rellena con espacios
6 format(montos, big.mark = ",", nsmall = 2, trim = TRUE)
```

```
[1] "1,234,568"
[1] "1,234,567.89"
[1] "513,916.60" " 1,407.99" " 54,056.19"
[1] "513,916.60" "1,407.99"  "54,056.19"
```

El argumento `big.mark` pone la coma de miles y `nsmall` fija el mínimo de decimales. Con un vector, `format()` rellena con espacios para que todos midan lo mismo (útil para alinear columnas en la consola); con `trim = TRUE` evitas ese relleno.

1.6.4 Formatear montos como moneda

Un reporte ejecutivo debe mostrar \$1,234,567.89, no 1234567.891. Con R base puedes armarlo combinando `round()`, `format()` y `paste0()`:

```
1 paste0("$", format(round(monto, 2), big.mark = ",", nsmall = 2))
2 paste0("$", format(round(montos, 2), big.mark = ",", nsmall = 2,
3 trim = TRUE))
```

```
[1] "$1,234,567.89"
[1] "$513,916.60" "$1,407.99" "$54,056.19"
```

Funciona, pero es largo. El **scales** es un **paquete**: un complemento de R que agrega funciones nuevas (los estudiarás en la sección 1.11). La notación `scales::dollar()` significa “usa la función `dollar()` del paquete **scales**”:

```
1 scales::dollar(monto)           # Redondea a pesos
2 scales::dollar(monto, accuracy = 0.01) # Con centavos
3 scales::dollar(montos)
4 scales::dollar(15000, suffix = " MXN") # Agrega un sufijo
5 scales::percent(0.9327, accuracy = 0.1) # Proporción -> porcentaje
```

```
6 scales::comma(1234567) # Solo separador de miles
```

```
[1] "$1,234,568"
[1] "$1,234,567.89"
[1] "$513,917" "$1,408" "$54,056"
[1] "$15,000 MXN"
[1] "93.3%"
[1] "1,234,567"
```

Observa que `dollar()` decide solo si muestra centavos: cuando hay montos grandes, redondea a pesos para que la tabla sea fácil de leer; si quieres centavos siempre, usa `accuracy = 0.01`. Y `percent()` multiplica por 100 y agrega el signo: recibe una *proporción* (0.9327), no un porcentaje (93.27). Usarás mucho estas funciones en las gráficas del Módulo 3.

Consejo

Si al ejecutar `scales::dollar()` R responde que no existe el paquete, todavía no lo tienes instalado. En la sección 1.11 verás cómo instalarlo con `install.packages("scales")`.

1.6.5 Mayúsculas, longitud, extraer y reemplazar

```
1 producto <- " Laptop HP Pavilion 15 "
2 toupper(producto) # Todo a mayúsculas
3 tolower("SUCURSAL CENTRO") # Todo a minúsculas
4 trimws(producto) # Quita espacios al inicio y al final
5 nchar(trimws(producto)) # Número de caracteres
6
7 sku <- "ELEC-2023-0153" # Categoría-año-consecutivo
8 substr(sku, 1, 4) # Caracteres del 1 al 4
9 substr(sku, 6, 9) # Caracteres del 6 al 9
10 gsub("-", "", sku) # Reemplaza "-" por nada
11 gsub("Sucursal ", "", c("Sucursal Centro", "Sucursal Norte"))
12 grepl("Premium", c("Sucursal Premium", "Sucursal Outlet"))
```

```
[1] " LAPTOP HP PAVILION 15 "
[1] "sucursal centro"
[1] "Laptop HP Pavilion 15"
[1] 21
[1] "ELEC"
[1] "2023"
[1] "ELEC20230153"
[1] "Centro" "Norte"
[1] TRUE FALSE
```

- `substr(texto, inicio, fin)` extrae un pedazo por posición: perfecto para códigos con estructura fija como los SKU.
- `gsub(buscar, reemplazar, texto)` sustituye *todas* las apariciones de un patrón (`sub()` solo la primera).
- `grepl(patrón, texto)` responde **TRUE** o **FALSE**: ¿el texto contiene el patrón? Sirve para filtrar, por ejemplo, todos los productos que contienen “Premium”.

Ejemplo práctico: Limpiar nombres de ciudades

El sistema de registro de clientes permite escribir la ciudad a mano. Al contar clientes por ciudad aparecen “CDMX”, “ cdmx” y “Cdmx ” como si fueran ciudades distintas. Es un problema clásico de **calidad de datos**: el reporte sale mal aunque el cálculo sea correcto.

```
1 ciudades <- c("CDMX", " cdmx", "Cdmx ", "Monterrey", "monterrey ",
2             "Puebla")
3 table(ciudades)                                # Seis "ciudades" distintas
4 ciudades_limpias <- toupper(trimws(ciudades))
5 table(ciudades_limpias)                        # Tres ciudades reales
```

ciudades	cdmx	CDMX	Cdmx	Monterrey	monterrey	Puebla
	1	1	1	1	1	1

ciudades_limpias	CDMX	MONTERREY	PUEBLA
	3	2	1

Con dos funciones anidadas, `trimws()` para quitar espacios y `toupper()` para unificar mayúsculas, el conteo queda bien: 3 clientes en CDMX, 2 en Monterrey y 1 en Puebla. R ejecuta primero la función de adentro y le pasa el resultado a la de afuera.

Hazlo tú

Tienes los códigos `c("ROPA-2023-0012", "HOGA-2022-0450", "ROPA-2023-0131")`. Extrae la categoría (los 4 primeros caracteres) y el año de cada uno, y construye un mensaje con `sprintf()` como `"Producto 12 de la categoría ROPA (2023)"`. Pista: `as.numeric(substr(x, 11, 14))` te da el consecutivo como número.

1.7 Data frames y tibbles: tus hojas de cálculo en R

Un **data frame** es la estructura más importante para BI: una tabla con filas y columnas, igual que una hoja de Excel. Cada *columna* es un vector (todos sus valores del mismo tipo) y todas las columnas tienen el mismo largo. Cada *fila* es una observación: un producto, una venta, un cliente.

1.7.1 Crear un data frame

Ejemplo práctico: Catálogo de productos

Vas a analizar ocho productos de tres categorías. Para cada uno conoces su nombre, categoría, precio de venta, costo y unidades vendidas en el mes. Quieres saber cuáles generan más ingreso, cuáles tienen mejor margen y qué categoría deja más utilidad.

```
1 productos <- data.frame(
2   nombre = c("Laptop Pro 14", "Mouse inalámbrico", "Monitor 27",
3             "Silla ergonómica", "Escritorio", "Playera básica",
4             "Chamarra", "Tenis running"),
5   categoria = c("Electrónica", "Electrónica", "Electrónica", "Hogar",
6               "Hogar", "Ropa", "Ropa", "Ropa"),
7   precio = c(18999, 349, 5499, 3899, 4299, 199, 1299, 1899),
8   costo = c(14250, 120, 4100, 2300, 2950, 85, 690, 1050),
9   unidades = c(12, 85, 20, 15, 9, 240, 38, 44)
10 )
11 productos
```

	nombre	categoria	precio	costo	unidades
1	Laptop Pro 14	Electrónica	18999	14250	12
2	Mouse inalámbrico	Electrónica	349	120	85
3	Monitor 27	Electrónica	5499	4100	20
4	Silla ergonómica	Hogar	3899	2300	15
5	Escritorio	Hogar	4299	2950	9
6	Playera básica	Ropa	199	85	240
7	Chamarra	Ropa	1299	690	38
8	Tenis running	Ropa	1899	1050	44

Cada argumento de `data.frame()` es una columna: primero el nombre de la columna, luego `=` y el vector con sus valores. Los números de la izquierda (1 a 8) son los nombres de las filas.

1.7.2 Explorar un data frame

Cuando recibes una tabla nueva, lo primero es conocerla: ¿cuántas filas y columnas tiene?, ¿cómo se llaman las columnas?, ¿de qué tipo es cada una?

```
1 head(productos, 3)           # Primeras 3 filas (por defecto, 6)
2 nrow(productos)              # Número de filas
3 ncol(productos)              # Número de columnas
4 dim(productos)               # Filas y columnas
5 names(productos)             # Nombres de las columnas
```

```

      nombre  categoria precio costo unidades
1   Laptop Pro 14 Electrónica 18999 14250      12
2 Mouse inalámbrico Electrónica   349   120      85
3   Monitor 27 Electrónica  5499  4100      20
[1] 8
[1] 5
[1] 8 5
[1] "nombre" "categoria" "precio" "costo" "unidades"

```

```

1 str(productos)           # Estructura: tipo de cada columna
2 summary(productos[, c("precio", "costo", "unidades")])

```

```

'data.frame':  8 obs. of  5 variables:
 $ nombre   : chr  "Laptop Pro 14" "Mouse inalámbrico" "Monitor 27" "Silla ergonómica"
  ↪ ...
 $ categoria: chr  "Electrónica" "Electrónica" "Electrónica" "Hogar" ...
 $ precio   : num  18999 349 5499 3899 4299 ...
 $ costo    : num  14250 120 4100 2300 2950 ...
 $ unidades : num  12 85 20 15 9 240 38 44

      precio      costo      unidades
Min.   : 199    Min.   : 85.0    Min.   : 9.00
1st Qu.: 1062    1st Qu.: 547.5    1st Qu.: 14.25
Median : 2899    Median : 1675.0    Median : 29.00
Mean   : 4555    Mean   : 3193.1    Mean   : 57.88
3rd Qu.: 4599    3rd Qu.: 3237.5    3rd Qu.: 54.25
Max.   :18999    Max.   :14250.0    Max.   :240.00

```

`str()` (de *structure*) es probablemente la función que más usarás al empezar un análisis: te dice que hay 8 observaciones (*obs.*) y 5 variables, el tipo de cada una (`chr` = texto, `num` = número) y sus primeros valores. `summary()` da el mínimo, los cuartiles, la media y el máximo de cada columna numérica: el precio promedio es de \$4,555 pero la mediana es de \$2,899, porque la laptop sube el promedio.

```

1 View(productos)          # Abre la tabla en una pestaña, como en Excel

```

`View()` (con *V* mayúscula) abre el data frame en una pestaña del editor donde puedes ordenar y filtrar con el ratón. Lo mismo ocurre si haces clic en el nombre de la tabla dentro del panel **Environment**. Es útil para mirar, pero recuerda que lo que hagas con el ratón no queda registrado: los filtros reales se escriben en código.

1.7.3 Acceder a columnas, filas y celdas

El signo `$` extrae una columna completa como vector. Los corchetes aceptan dos posiciones separadas por coma: `[filas, columnas]`. Si dejas una posición vacía significa “todas”.


```

1 productos$precio           # Una columna (vector)
2 mean(productos$precio)     # ... y con ella calculas
3 productos[2, ]             # Fila 2, todas las columnas
4 productos[2, "precio"]     # Una celda: fila 2, col. precio
5 productos[1:3, c("nombre", "precio")] # Filas 1 a 3, dos columnas

```

```

[1] 18999  349  5499  3899  4299  199  1299  1899
[1] 4555.25
      nombre  categoria precio costo unidades
2 Mouse inalámbrico Electrónica  349  120      85
[1] 349
      nombre precio
1  Laptop Pro 14 18999
2 Mouse inalámbrico  349
3  Monitor 27  5499

```

1.7.4 Filtrar filas

Filtrar un data frame es igual que filtrar un vector: escribes una condición y la pones en la posición de las *filas*, antes de la coma.

```

1 # Productos con precio mayor a $3,000
2 productos[productos$precio > 3000, ]
3
4 # Ropa con más de 40 unidades vendidas (dos condiciones con &)
5 productos[productos$categoria == "Ropa" & productos$unidades > 40, ]
6
7 # Hogar o Ropa, mostrando solo nombre y precio
8 productos[productos$categoria %in% c("Hogar", "Ropa"),
9           c("nombre", "precio")]

```

```

      nombre  categoria precio costo unidades
1  Laptop Pro 14 Electrónica 18999 14250      12
3  Monitor 27 Electrónica  5499  4100      20
4 Silla ergonómica      Hogar  3899  2300      15
5  Escritorio      Hogar  4299  2950       9
      nombre categoria precio costo unidades
6 Playera básica      Ropa   199   85      240
8  Tenis running      Ropa  1899 1050      44
      nombre precio
4 Silla ergonómica 3899
5  Escritorio 4299
6 Playera básica 199
7  Chamarra 1299

```

```
8   Tennis running    1899
```

▲ Cuidado

No olvides la coma: `productos[productos$precio > 3000, ]`. Sin ella, R interpreta la condición como una selección de *columnas* y responde con el error `undefined columns selected`. La coma separa “qué filas” de “qué columnas”.

Una alternativa más legible es `subset()`, donde puedes escribir los nombres de las columnas sin repetir `productos$`:

```
1 subset(productos, precio < 1000, select = c(nombre, precio, unidades))
```

```
      nombre precio unidades
2 Mouse inalámbrico   349      85
6  Playera básica    199     240
```

En el Módulo 2 aprenderás `filter()` del paquete **dplyr**, que lleva esta idea todavía más lejos.

1.7.5 Ordenar

La función `order()` devuelve las *posiciones* de las filas en el orden deseado; al ponerla en la posición de las filas, reordenas la tabla:

```
1 # De menor a mayor precio
2 productos[order(productos$precio), c("nombre", "precio")]
3
4 # Los 3 productos más caros (de mayor a menor)
5 caros <- productos[order(productos$precio, decreasing = TRUE), ]
6 caros[1:3, c("nombre", "categoria", "precio")]
```

```
      nombre precio
6  Playera básica   199
2 Mouse inalámbrico  349
7      Chamarra  1299
8   Tennis running  1899
4 Silla ergonómica  3899
5      Escritorio  4299
3      Monitor 27  5499
1 Laptop Pro 14 18999
      nombre categoria precio
1 Laptop Pro 14 Electrónica 18999
3  Monitor 27 Electrónica  5499
5  Escritorio      Hogar   4299
```

1.7.6 Crear columnas calculadas

Para agregar una columna basta asignarle un vector con `$`. Como las operaciones son vectorizadas, el cálculo se hace para todas las filas a la vez:

```
1 productos$ingreso <- productos$precio * productos$unidades
2 productos$utilidad <- (productos$precio - productos$costo) *
3   productos$unidades
4 productos$margen_pct <- round((productos$precio - productos$costo) /
5   productos$precio * 100, 1)
6 productos[, c("nombre", "ingreso", "utilidad", "margen_pct")]
```

	nombre	ingreso	utilidad	margen_pct
1	Laptop Pro 14	227988	56988	25.0
2	Mouse inalámbrico	29665	19465	65.6
3	Monitor 27	109980	27980	25.4
4	Silla ergonómica	58485	23985	41.0
5	Escritorio	38691	12141	31.4
6	Playera básica	47760	27360	57.3
7	Chamarra	49362	23142	46.9
8	Tenis running	83556	37356	44.7

¿Qué nos dice esto? La laptop es el producto que más ingreso (\$227,988) y más utilidad (\$56,988) genera, pero con el margen más bajo (25 %). En cambio, el mouse y la playera, que cuestan poco, dejan márgenes de 65.6 % y 57.3 %. Un producto puede ser muy importante en ventas y poco rentable por unidad: por eso en BI siempre se miran juntas varias métricas.

1.7.7 Resumir por grupos

La pregunta más frecuente en BI es “¿cuánto por cada...?": ventas por tienda, clientes por ciudad, utilidad por categoría. En Excel harías una tabla dinámica. En R base tienes tres herramientas:

```
1 table(productos$categoria) # Conteo por grupo
2 tapply(productos$ingreso, productos$categoria, sum) # Suma por grupo
3 aggregate(ingreso ~ categoria, data = productos, FUN = sum)
```

	Electrónica	Hogar	Ropa
	3	2	3
	Electrónica	Hogar	Ropa
	367633	97176	180678
	categoria	ingreso	
1	Electrónica	367633	
2	Hogar	97176	
3	Ropa	180678	

- `table()` cuenta cuántas filas hay de cada categoría.
- `tapply(valores, grupos, función)` aplica la función a los valores de cada grupo y devuelve un vector con nombres.
- `aggregate()` hace lo mismo pero devuelve un data frame, ideal para seguir trabajando o exportarlo. La expresión `ingreso ~ categoria` es una **fórmula** y se lee “ingreso según categoría”.

`aggregate()` puede resumir varias columnas a la vez con `cbind()`. Con eso armamos una pequeña tabla dinámica de rentabilidad:

```
1 resumen_categoria <- aggregate(cbind(ingreso, utilidad) ~ categoria,
2                               data = productos, FUN = sum)
3 resumen_categoria$margen_pct <- round(resumen_categoria$utilidad /
4                                     resumen_categoria$ingreso * 100, 1)
5 resumen_categoria[order(resumen_categoria$margen_pct,
6                           decreasing = TRUE), ]
```

	categoria	ingreso	utilidad	margen_pct
3	Ropa	180678	87858	48.6
2	Hogar	97176	36126	37.2
1	Electrónica	367633	104433	28.4

¿Qué nos dice esto? Electrónica vende más del doble que Ropa (\$367,633 contra \$180,678), pero Ropa convierte casi la mitad de cada peso vendido en utilidad (48.6%) mientras Electrónica solo el 28.4%. Si el equipo comercial tiene presupuesto para una campaña, impulsar Ropa deja más utilidad por cada peso vendido.

1.7.8 Data frame o tibble

El **tibble** es una versión moderna del data frame que usa el *tidyverse*, el conjunto de paquetes que estudiarás a partir del Módulo 2. Se crea con `tibble::tibble()` y se comporta casi igual, con algunas mejoras:

```
1 productos_tb <- tibble::tibble(
2   nombre = c("Laptop Pro 14", "Mouse inalámbrico", "Monitor 27"),
3   precio = c(18999, 349, 5499),
4   unidades = c(12, 85, 20),
5   ingreso = precio * unidades      # Usa columnas recién creadas
6 )
7 productos_tb
8 tibble::as_tibble(productos)      # Convertir un data frame a tibble
```

```
# A tibble: 3 x 4
  nombre      precio unidades ingreso
  <chr>      <dbl>    <dbl>   <dbl>
1 Laptop Pro 14 18999      12 227988
2 Mouse inalámbrico 349      85 29665
3 Monitor 27 5499      20 109980

# A tibble: 8 x 8
  nombre      categoria precio costo unidades ingreso utilidad margen_pct
  <chr>      <chr>    <dbl> <dbl>    <dbl>   <dbl>    <dbl>    <dbl>
1 Laptop ~ Electrón~ 18999 14250      12 227988 56988      25
2 Mouse i~ Electrón~ 349 120      85 29665 19465     65.6
3 Monitor~ Electrón~ 5499 4100     20 109980 27980     25.4
4 Silla e~ Hogar    3899 2300     15 58485 23985     41
5 Escritor~ Hogar    4299 2950      9 38691 12141     31.4
6 Playera~ Ropa      199 85      240 47760 27360     57.3
7 Chamarra Ropa     1299 690      38 49362 23142     46.9
8 Tennis r~ Ropa     1899 1050     44 83556 37356     44.7
```

Las diferencias más visibles son:

- Al imprimirlo te dice su tamaño (8 x 8) y el tipo de cada columna debajo del nombre: `<chr>` texto, `<dbl>` número decimal, `<int>` entero, `<lgl>` lógico, `<date>` fecha.
- Solo muestra las primeras 10 filas y las columnas que caben en la pantalla; las demás las lista al final. Con tablas de un millón de filas, eso evita que la consola se inunde.
- Muestra los números con 3 cifras significativas para ahorrar espacio (los decimales se atenúan en pantalla), pero el valor guardado no cambia.
- Dentro de `tibble()` una columna puede usar otra recién creada, como hicimos con `ingreso`.
- Nunca convierte texto en factor y no tiene nombres de fila.

Todo lo que aprendiste en esta sección (`$`, corchetes, `str()`, `aggregate()`...) funciona igual con tibbles. Cuando leas archivos con `readr` o `readxl` obtendrás tibbles.

Hazlo tú

Agrega al data frame `productos` una columna `precio_con_iva` (precio por 1.16, redondeado a 2 decimales). Después muestra solo los productos de Electrónica ordenados de mayor a menor margen y calcula con `aggregate()` las unidades totales por categoría.

1.8 Listas y matrices

Los vectores y los data frames cubren el 90 % de lo que harás en BI, pero hay dos estructuras más que conviene conocer porque aparecen a menudo. La figura 1.2 resume las cuatro.

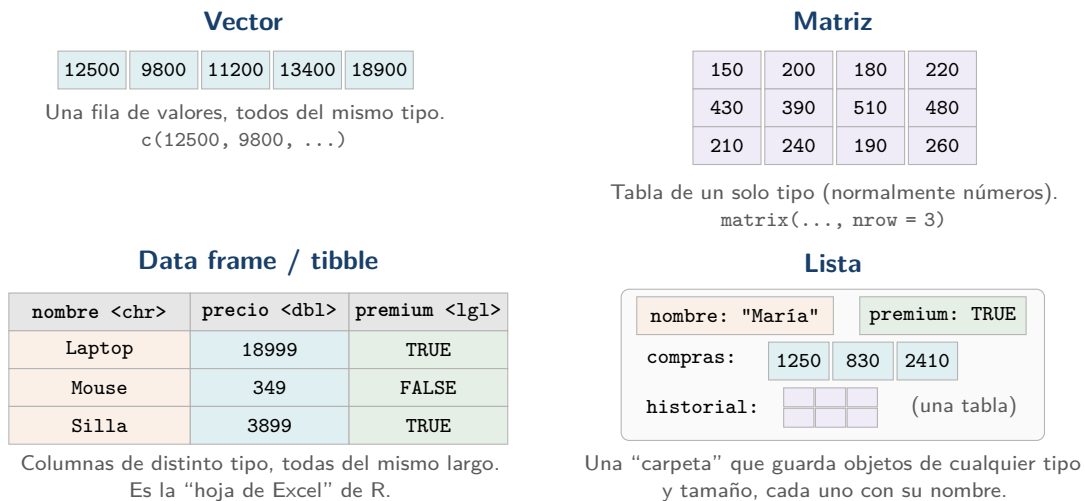


Figura 1.2. Las cuatro estructuras de datos básicas de R.

1.8.1 Matrices: tablas de un solo tipo

Una **matriz** es una tabla rectangular donde *todos* los valores son del mismo tipo, casi siempre números. Es como una tabla dinámica ya resumida: productos en las filas, meses en las columnas.

Ejemplo práctico: Ventas en unidades por producto y mes

Tienes las unidades vendidas de tres productos en los primeros cuatro meses del año. Quieres el total por producto, el total por mes y un pronóstico para el siguiente cuatrimestre con un crecimiento del 10%.

```

1 ventas_mat <- matrix(
2   c(150, 200, 180, 220,      # Laptop
3     430, 390, 510, 480,      # Mouse
4     210, 240, 190, 260),     # Monitor
5   nrow = 3, byrow = TRUE,    # 3 filas, llenando renglón por renglón
6   dimnames = list(c("Laptop", "Mouse", "Monitor"), # Filas
7                   c("ene", "feb", "mar", "abr")),   # Columnas
8 )
9
10 ventas_mat
11 ventas_mat["Mouse", "mar"] # Una celda: [fila, columna]
12 ventas_mat[, "abr"]        # Una columna completa
13 rowSums(ventas_mat)         # Total por producto
14 colSums(ventas_mat)         # Total por mes
15 round(rowMeans(ventas_mat), 1) # Promedio mensual por producto
16 ventas_mat * 1.10           # Pronóstico: +10% en todas las celdas

```

```

      ene feb mar abr
Laptop 150 200 180 220

```

```

Mouse    430 390 510 480
Monitor  210 240 190 260
[1] 510
  Laptop   Mouse Monitor
    220     480     260
  Laptop   Mouse Monitor
    750    1810     900
ene feb mar abr
790 830 880 960
  Laptop   Mouse Monitor
    187.5    452.5    225.0
      ene feb mar abr
Laptop  165 220 198 242
Mouse   473 429 561 528
Monitor 231 264 209 286

```

El mouse es el producto de mayor volumen (1,810 unidades en cuatro meses) y abril fue el mes más fuerte (960 unidades). Las funciones `rowSums()` y `colSums()` son los “totales de fila” y “totales de columna” de una tabla dinámica. Y como las matrices también son vectorizadas, `ventas_mat * 1.10` aplica el crecimiento a las doce celdas de golpe.

1.8.2 Listas: una carpeta con objetos de cualquier tipo

Una **lista** es la estructura más flexible: guarda objetos de distinto tipo y tamaño, cada uno con su nombre. Piensa en el expediente de un cliente: una carpeta con su nombre (texto), su fecha de alta (fecha), si es premium (lógico) y el historial de sus compras (un vector de montos).

```

1 cliente <- list(
2   id = 1045,
3   nombre = "María González",
4   ciudad = "Guadalajara",
5   premium = TRUE,
6   fecha_alta = as.Date("2021-06-14"),
7   compras = c(1250.00, 830.50, 2410.00, 675.25)
8 )
9 str(cliente)                # Estructura de la lista
10 cliente$nombre              # Un elemento por nombre
11 cliente[["compras"]]         # Otra forma: doble corchete
12 sum(cliente$compras)         # Total comprado
13 mean(cliente$compras)        # Ticket promedio
14 cliente$canal <- "Tienda en línea" # Agregar un elemento nuevo
15 names(cliente)

```

```

List of 6
 $ id      : num 1045
 $ nombre  : chr "María González"

```

```

$ ciudad    : chr "Guadalajara"
$ premium   : logi TRUE
$ fecha_alta: Date[1:1], format: "2021-06-14"
$ compras   : num [1:4] 1250 830 2410 675
[1] "María González"
[1] 1250.00 830.50 2410.00 675.25
[1] 5165.75
[1] 1291.438
[1] "id"      "nombre"  "ciudad"  "premium" "fecha_alta"
[6] "compras" "canal"

```

María ha comprado \$5,165.75 en cuatro visitas, con un ticket promedio de \$1,291.44. Con `$` o con `[[ ]]` sacas un elemento tal cual (aquí, el vector de compras) y puedes operar con él.

Concepto clave: ¿Por qué importan las listas?

Un data frame es, en realidad, una lista de vectores del mismo largo: por eso `productos$precio` funciona igual que `cliente$nombre`. Además, muchas funciones de R devuelven listas: los resultados de un modelo estadístico, la respuesta de una API web o la configuración de un reporte. En el Módulo 7 extraerás información de listas así.

Hazlo tú

Crea una lista `tienda` con el nombre de una sucursal, su ciudad, su fecha de apertura, su número de empleados y un vector con sus ventas de los últimos seis meses. Calcula la venta promedio y cuántos años lleva abierta la tienda. Después crea una matriz de 2 sucursales $\times$ 3 meses y obtén el total por sucursal con `rowSums()`.

1.9 Control de flujo: decisiones y repeticiones

Hasta ahora R ejecuta las instrucciones una tras otra, siempre igual. El **control de flujo** te permite decirle “haz esto *solo si* pasa tal cosa” o “repite esto para cada sucursal”. Son las mismas reglas de negocio que aplicas mentalmente todos los días.

1.9.1 Decidir con `if` y `else`

La estructura `if (condición) { ... } else { ... }` ejecuta el primer bloque si la condición es `TRUE` y el segundo si es `FALSE`. Las llaves `{ }` agrupan las instrucciones de cada caso.

Ejemplo práctico: Clasificar una venta

La política comercial dice: una venta de \$5,000 o más es “Grande” y la revisa un gerente; de \$1,000 a \$4,999 es “Mediana”; y por debajo de \$1,000 es “Chica”. Queremos que R clasifique una venta de \$3,250.


```

1 monto_venta <- 3250
2
3 if (monto_venta >= 5000) {
4   tipo_venta <- "Grande"
5 } else if (monto_venta >= 1000) {
6   tipo_venta <- "Mediana"
7 } else {
8   tipo_venta <- "Chica"
9 }
10 tipo_venta

```

```
[1] "Mediana"
```

R revisa las condiciones de arriba hacia abajo y se detiene en la primera que se cumple. Como \$3,250 no llega a 5,000 pero sí supera 1,000, la venta queda como “Mediana”. Nota la sangría (los dos espacios al inicio de las líneas internas): R no la necesita, pero hace el código mucho más fácil de leer. RStudio la pone sola; si se desordena, selecciona el código y presiona **Ctrl** + **I**.

▲ Cuidado

if evalúa *un solo* valor lógico. Si le das un vector (por ejemplo, las ventas de toda la semana), R se detiene con el error **the condition has length > 1**. Para evaluar muchos valores a la vez usa **ifelse()**, que verás enseguida.

1.9.2 Decidir para todo un vector: **ifelse()**

ifelse(condición, valor_si_TRUE, valor_si_FALSE) es la versión vectorizada de **if**: evalúa la condición elemento por elemento y devuelve un vector del mismo largo. Es el equivalente directo de la función **=SI()** de Excel arrastrada a toda una columna.

```

1 ventas_semana
2 etiqueta_meta <- ifelse(ventas_semana >= meta_diaria,
3                         "Cumplió", "No cumplió")
4 etiqueta_meta
5
6 # Anidar ifelse() para tres categorías
7 nivel_dia <- ifelse(ventas_semana >= 18000, "Alto",
8                   ifelse(ventas_semana >= 12000, "Medio", "Bajo"))
9 table(nivel_dia)

```

lunes	martes	miércoles	jueves	viernes	sábado	domingo
12500	9800	11200	13400	18900	22300	15600
lunes	martes	miércoles	jueves	viernes		
"No cumplió"	"No cumplió"	"No cumplió"	"No cumplió"	"Cumplió"		

```

      sábado      domingo
"Cumplió" "Cumplió"
nivel_dia
Alto  Bajo Medio
  2     2     3

```

Con una sola línea etiquetaste los siete días. Anidar `ifelse()` funciona para tres o cuatro categorías; con más se vuelve ilegible y es mejor usar `cut()` (que aparece en los ejercicios) o `case_when()`, que estudiarás en el Módulo 2.

1.9.3 Repetir con for

Un **bucle for** repite un bloque de código una vez por cada elemento de un vector. La variable del bucle (aquí `k`) va tomando cada valor en turno.

Ejemplo práctico: Reporte de cumplimiento por sucursal

Tienes las ventas y las metas mensuales de cinco sucursales. El director quiere un reporte de texto, una línea por sucursal, con la venta, el porcentaje de cumplimiento y una marca de alerta si quedó por debajo del 95 %.

```

1  sucursales <- c("Centro", "Norte", "Sur", "Plaza A", "Outlet")
2  ventas_mes <- c(485000, 468900, 391200, 552300, 298700)
3  metas_mes <- c(520000, 450000, 420000, 540000, 280000)
4
5  cat("REPORTE DE CUMPLIMIENTO - MARZO 2023\n")
6  for (k in seq_along(sucursales)) {
7    cumpl <- ventas_mes[k] / metas_mes[k] * 100
8    alerta <- if (cumpl < 95) " <- REVISAR" else ""
9    cat(sprintf("%-8s $%9s %6.1f%%\n", sucursales[k],
10               format(ventas_mes[k], big.mark = ","), cumpl, alerta))
11 }

```

```

REPORTE DE CUMPLIMIENTO - MARZO 2023
Centro  $ 485,000  93.3% <- REVISAR
Norte   $ 468,900 104.2%
Sur      $ 391,200  93.1% <- REVISAR
Plaza A  $ 552,300 102.3%
Outlet   $ 298,700 106.7%

```

Vamos por partes:

- `seq_along(sucursales)` produce `1 2 3 4 5`, una posición por sucursal. Es más seguro que escribir `1:5` porque se ajusta solo si agregas sucursales.
- En cada vuelta, `k` vale 1, luego 2, etc., y con `[k]` tomas el dato de esa sucursal.

- `cat()` imprime texto tal cual, sin el `[1]` ni las comillas de `print()`. El `\n` es un salto de línea.
- En `sprintf()`, `%-8s` reserva 8 espacios para el texto alineado a la izquierda y `%9s` reserva 9 alineados a la derecha: así las columnas quedan parejas.

¿Qué nos dice esto? Centro y Sur quedaron por debajo del 95 % de su meta y hay que revisarlas; Outlet es la que más superó su meta (106.7 %), aunque es la que menos vende en pesos.

1.9.4 Repetir mientras se cumpla algo: `while`

`while` repite un bloque *mientras* una condición sea verdadera. Se usa cuando no sabes de antemano cuántas vueltas darás.

Ejemplo práctico: ¿Cuántos meses para llegar a la meta?

Una tienda vende hoy \$350,000 al mes y su plan de negocio prevé crecer 3 % mensual. ¿En cuántos meses alcanzará ventas mensuales de \$450,000?

```
1 venta_actual <- 350000
2 crecimiento_mensual <- 0.03
3 meta_mensual <- 450000
4 meses <- 0
5
6 while (venta_actual < meta_mensual) {
7   venta_actual <- venta_actual * (1 + crecimiento_mensual)
8   meses <- meses + 1
9 }
10 meses
11 round(venta_actual, 2)
```

```
[1] 9
[1] 456670.6
```

En cada vuelta la venta crece 3 % y el contador de meses sube en uno. El bucle se detiene en cuanto la venta alcanza la meta: después de 9 meses la tienda vendería unos \$456,671.

▲ Cuidado

Si la condición de un `while` nunca se vuelve `FALSE` (por ejemplo, con un crecimiento de 0 %), el bucle no termina jamás: es un **bucle infinito**. Si te pasa, presiona `Esc` o el botón rojo **Stop** de la consola.

1.9.5 En R se prefiere lo vectorizado

Si vienes de Excel o de otro lenguaje, tu primer instinto será escribir bucles para todo. En R casi siempre existe una forma vectorizada más corta, más rápida y menos propensa a errores. Compara

el bucle anterior de cumplimiento con esta sola línea:

```
1 round(ventas_mes / metas_mes * 100, 1)      # Cumplimiento de las 5
2 sucursales[ventas_mes / metas_mes < 0.95]    # Las que hay que revisar
```

```
[1] 93.3 104.2 93.1 102.3 106.7
[1] "Centro" "Sur"
```

Usa **for** cuando cada vuelta haga algo que no es un simple cálculo: imprimir un reporte, leer un archivo distinto, guardar un Excel por sucursal. Para aplicar una función a cada elemento de una lista existe también la familia **apply**: **sapply()** aplica una función y simplifica el resultado a un vector, y **vapply()** hace lo mismo pero te obliga a declarar qué tipo de resultado esperas, lo que la hace más segura.

```
1 ventas_por_tienda <- list(
2   Centro = c(15200, 13900, 16800),
3   Norte = c(12100, 12900),
4   Outlet = c(8400, 9100, 7600, 9900)
5 )
6 sapply(ventas_por_tienda, sum)              # Total de cada tienda
7 sapply(ventas_por_tienda, length)          # Días con datos
8 vapply(ventas_por_tienda, mean, numeric(1)) # Promedio (seguro)
```

```
Centro Norte Outlet
45900 25000 35000
Centro Norte Outlet
3      2      4
Centro Norte Outlet
15300 12500 8750
```

numeric(1) le dice a **vapply()**: “cada resultado debe ser un número”. Si alguna tienda devolviera texto o varios valores, R te avisaría con un error en lugar de darte un resultado raro. En el Módulo 2 verás que con **dplyr** la mayoría de estos cálculos por grupo se hacen sin bucles.

Hazlo tú

Con los vectores **sucursales** y **ventas_mes** usa **ifelse()** para clasificar cada sucursal como “Grande” (ventas $\geq$ \$450,000) o “Chica”. Después escribe un **for** que imprima con **cat()** una línea como “Norte: Grande” por sucursal.

1.10 Tus propias funciones: una caja de herramientas BI

Si copias y pegas el mismo cálculo tres veces, es momento de convertirlo en una **función**. Una función propia es como una fórmula personalizada de Excel: le pones nombre una vez y la usas

siempre que la necesites. Si mañana cambia la regla de negocio, la corriges en un solo lugar.

1.10.1 Anatomía de una función

```

1  # Calcula el margen de utilidad de uno o varios productos.
2  # Argumentos:
3  #   precio: precio de venta (número o vector)
4  #   costo:  costo unitario (número o vector)
5  #   en_pct: TRUE devuelve porcentaje (0-100); FALSE, proporción (0-1)
6  # Devuelve: el margen, redondeado a 1 decimal si es porcentaje
7  calcular_margen <- function(precio, costo, en_pct = TRUE) {
8    margen <- (precio - costo) / precio
9    if (en_pct) {
10     return(round(margen * 100, 1))
11   }
12   margen
13 }
```

- `calcular_margen` es el nombre; `function()` crea la función.
- Entre paréntesis van los **argumentos**: los datos que la función necesita. `en_pct = TRUE` tiene un **valor por defecto**: si no lo indicas, vale `TRUE`.
- Entre llaves va el **cuerpo**: los cálculos.
- `return()` devuelve un resultado y termina la función. Si no lo usas, la función devuelve el resultado de la última línea (aquí, `margen`).
- Los comentarios de arriba *documentan* la función: qué hace, qué recibe y qué devuelve. Es un hábito profesional que te salvará cuando vuelvas a este código en unos meses.

Definir la función no calcula nada: solo la “guarda” en el entorno. Ahora úsala:

```

1  calcular_margen(450, 280)                # Posicional
2  calcular_margen(precio = 450, costo = 280, en_pct = FALSE)
3  calcular_margen(productos$precio, productos$costo) # Vectorizada
```

```

[1] 37.8
[1] 0.3777778
[1] 25.0 65.6 25.4 41.0 31.4 57.3 46.9 44.7
```

Como está construida con operaciones vectorizadas, la misma función sirve para un producto o para todo el catálogo. Escribir los nombres de los argumentos (`precio = 450`) es opcional, pero hace el código más claro, sobre todo cuando hay argumentos con valor por defecto.

1.10.2 Validar los datos: `stop()` y `warning()`

Una buena función se protege de datos absurdos. `stop()` detiene la función con un mensaje de error claro; `warning()` deja que continúe pero avisa que algo es sospechoso.

```

1  # ROI (retorno sobre la inversión) en porcentaje
2  calcular_roi <- function(ganancia, inversion) {
3    if (!is.numeric(ganancia) || !is.numeric(inversion)) {
4      stop("ganancia e inversion deben ser números")
5    }
6    if (any(inversion <= 0)) {
7      stop("La inversión debe ser mayor que cero")
8    }
9    (ganancia - inversion) / inversion * 100
10 }
11
12 # Variación porcentual entre un periodo y el anterior
13 variacion_porcentual <- function(actual, anterior) {
14   if (any(anterior == 0)) {
15     warning("Hay periodos anteriores en 0; su variación queda como NA")
16   }
17   resultado <- (actual - anterior) / anterior * 100
18   resultado[anterior == 0] <- NA
19   round(resultado, 1)
20 }

```

El operador `||` es un “o” para un solo valor (dentro de `if` se prefiere a `|`) y `any()` responde `TRUE` si *alguno* de los elementos cumple la condición. Veamos qué pasa con datos inválidos:

```
1 calcular_roi(5000, 0)
```

```
Error in calcular_roi(5000, 0) : La inversión debe ser mayor que cero
```

```
1 variacion_porcentual(c(120, 50), c(100, 0))
```

```

[1] 20 NA
Warning message:
In variacion_porcentual(c(120, 50), c(100, 0)) :
  Hay periodos anteriores en 0; su variación queda como NA

```

Sin la validación, dividir entre cero daría `Inf` (infinito) y ese valor absurdo se colaría en tu reporte. Con ella, el error explica exactamente qué está mal.

1.10.3 La caja de herramientas completa

Agreguemos una función para segmentar clientes según su gasto anual y usemos las cuatro juntas.

```

1 # Segmenta clientes por gasto anual:
2 # Bronce < $5,000 <= Plata < $20,000 <= Oro < $50,000 <= Platino
3 clasificar_cliente <- function(gasto_anual,
4                               cortes = c(5000, 20000, 50000)) {
5   if (any(gasto_anual < 0, na.rm = TRUE)) {
6     stop("El gasto no puede ser negativo")
7   }
8   ifelse(gasto_anual >= cortes[3], "Platino",
9         ifelse(gasto_anual >= cortes[2], "Oro",
10              ifelse(gasto_anual >= cortes[1], "Plata", "Bronce")))
11 }

```

Ejemplo práctico: Usar la caja de herramientas

Una campaña de marketing costó \$80,000 y generó \$116,000 de utilidad. Las ventas trimestrales fueron \$126,274, \$120,717, \$134,427 y \$132,498. Y seis clientes gastaron en el año \$3,200, \$18,500, \$52,000, \$7,800, \$25,000 y \$950. Calcula el ROI, la variación trimestral y el segmento de cada cliente.

```

1 calcular_roi(ganancia = 116000, inversion = 80000)
2
3 ventas_trim <- c(Q1 = 126274, Q2 = 120717, Q3 = 134427, Q4 = 132498)
4 variacion_porcentual(ventas_trim[-1], ventas_trim[-4])
5
6 gastos <- c(3200, 18500, 52000, 7800, 25000, 950)
7 segmentos <- clasificar_cliente(gastos)
8 segmentos
9 table(factor(segmentos, levels = c("Bronce", "Plata", "Oro",
10                                "Platino")))

```

```

[1] 45
   Q2   Q3   Q4
-4.4 11.4 -1.4
[1] "Bronce" "Plata" "Platino" "Plata" "Oro" "Bronce"

  Bronce  Plata   Oro Platino
      2     2     1      1

```

¿Qué nos dice esto? La campaña tuvo un ROI de 45 %: por cada peso invertido se recuperó el peso y 45 centavos más. Las ventas cayeron 4.4 % del primer al segundo trimestre, repuntaron 11.4 % en el tercero y bajaron ligeramente en el cuarto. Y la cartera tiene dos clientes Bronce, dos Plata y

un cliente en cada uno de los niveles Oro y Platino.

Observa el truco de comparar `ventas_trim[-1]` (todos menos el primero, es decir Q2 a Q4) contra `ventas_trim[-4]` (todos menos el último, Q1 a Q3): así comparas cada trimestre con el anterior sin bucles. Los nombres del resultado son los del periodo actual.

Consejo

Guarda tus funciones de uso frecuente en un script aparte, por ejemplo `funciones_bi.R`, y cárgalas al inicio de cada análisis con `source("funciones_bi.R")`. Así construyes tu propia caja de herramientas y todos tus reportes calculan los KPIs de la misma forma.

Hazlo tú

Escribe una función `precio_final(precio, descuento = 0, iva = 0.16)` que aplique primero el descuento (como proporción, $0.15 = 15\%$) y luego el IVA, y que se detenga con `stop()` si el descuento no está entre 0 y 1. Pruébala con `precio_final(18999, 0.15)`; debería dar 18,733.01.

1.11 Paquetes: superpoderes para R

R “de fábrica” (llamado **R base**) ya hace muchísimo, pero su verdadero poder está en los **paquetes**: colecciones de funciones, datos y documentación que otras personas escribieron y comparten gratis. Si R fuera un teléfono, los paquetes serían las apps. Hay paquetes para leer Excel (**readxl**), manipular datos (**dplyr**), hacer gráficas (**ggplot2**), crear tableros web (**shiny**) o entrenar modelos de *machine learning*.

1.11.1 Instalar y cargar

La mayoría de los paquetes viven en **CRAN** (*Comprehensive R Archive Network*), el repositorio oficial de R, con más de 20,000 paquetes revisados. Trabajar con un paquete tiene dos pasos, que conviene no confundir:

1. **Instalar** (una sola vez por computadora): descarga el paquete de internet. Es como descargar una app.
2. **Cargar** (en cada sesión en la que lo uses): lo activa para la sesión actual. Es como abrir la app.

```
1 # Instalar (solo una vez; necesita internet). Nota las comillas.
2 install.packages("readxl")
3 install.packages(c("tidyverse", "writexl", "scales")) # Varios a la vez
```

También puedes instalar desde el panel **Packages > Install**. No pongas `install.packages()` dentro de tus scripts de análisis: se volvería a descargar el paquete cada vez que los ejecutes. Instala desde la consola y deja en el script solo las líneas `library()`.


```

1 library(readxl) # Cargar (en cada sesión)
2 library(writexl)
3 "readxl" %in% rownames(installed.packages()) # ¿Está instalado?
4 as.character(packageVersion("readxl")) # ¿Qué versión tengo?

```

```

[1] TRUE
[1] "1.4.3"

```

Concepto clave: Instalar contra cargar

`install.packages("readxl")` va con comillas y se hace una vez. `library(readxl)` va sin comillas y se escribe al inicio de cada script que use el paquete. Si ves el error `could not find function "read_excel"`, casi siempre olvidaste el `library()`; si ves `there is no package called 'readxl'`, te falta instalarlo.

1.11.2 La notación `paquete::funcion()`

Ya usaste `scales::dollar()` y `tibble::tibble()`. Los dos signos de dos puntos permiten usar una función de un paquete *instalado* sin cargarlo completo con `library()`. Es útil cuando solo necesitas una función, y además deja claro de dónde viene:

```

1 scales::dollar(513916.6) # Sin library(scales)

```

```

[1] "$513,917"

```

1.11.3 El tidyverse y su mensaje de bienvenida

El **tidyverse** es una colección de paquetes diseñados para trabajar juntos en ciencia de datos: **readr** (leer archivos), **dplyr** (manipular tablas), **tidyr** (reorganizarlas), **ggplot2** (gráficas), **stringr** (texto), **lubridate** (fechas), **forcats** (factores), **tibble** y **purrr**. Con una sola línea los cargas todos:

```

1 library(tidyverse)

```

Al ejecutarla, la consola muestra un mensaje de bienvenida en dos partes. No es un error; conviene entenderlo:

- **Attaching core tidyverse packages:** la lista de los nueve paquetes que se cargaron, cada uno con su versión y una palomita.
- **Conflicts:** avisa de funciones con el mismo nombre en dos paquetes. Verás líneas como `dplyr::filter() masks stats::filter()`, que significan: “el paquete **stats** (que viene con

R) ya tenía una función `filter()`; a partir de ahora, cuando escribas `filter()` se usará la de `dplyr`". Lo mismo ocurre con `lag()`.

Estos conflictos son normales y esperados: la versión de `dplyr` es justo la que quieres. Si alguna vez necesitas la otra, escribe el nombre completo, por ejemplo `stats::filter()`. El mensaje aparece solo al cargar el paquete; en este libro no lo reproducimos para ahorrar espacio.

Hazlo tú

En la pestaña **Packages** de RStudio busca **readxl** y haz clic en su nombre: se abrirá su documentación en **Help**. Después ejecuta `packageVersion("dplyr")` y `"janitor"%in%rownames(installed.packages())` para saber si tienes instalado el paquete **janitor**.

1.12 Importar y exportar datos

En la vida real no escribes los datos a mano: vienen de archivos que exporta el sistema de ventas, el ERP, el CRM o un compañero. Los formatos más comunes en BI son el **CSV** (*comma-separated values*, texto plano con los valores separados por comas) y el Excel. En esta sección aprenderás a leerlos, a guardar tus resultados y a resolver los problemas que casi siempre aparecen.

1.12.1 Leer un CSV con R base: `read.csv()`

```
1 ventas_base <- read.csv("datasets/ventas_retail.csv")
2 dim(ventas_base)
3 head(ventas_base[, c("fecha", "dia_semana", "cantidad", "total")], 3)
4 class(ventas_base$fecha)
```

```
[1] 365 14
      fecha dia_semana cantidad  total
1 2023-01-01   domingo      10 3332.505
2 2023-01-02     lunes       8 2262.000
3 2023-01-03     martes       3  800.577
[1] "character"
```

`read.csv()` viene con R y no necesita paquetes. Leyó las 365 filas y 14 columnas, pero fíjate en un detalle: la columna `fecha` quedó como texto (`character`). Para usarla como fecha tendrías que convertirla con `as.Date()`.

1.12.2 Leer un CSV con readr: `read_csv()`

El paquete **readr** (parte del tidyverse) tiene `read_csv()`, con guion bajo. Es más rápida, detecta las fechas automáticamente y devuelve un tibble. La primera vez que la uses verás un mensaje:

```
1 ventas <- read_csv("datasets/ventas_retail.csv")
```

```
Rows: 365 Columns: 14
-- Column specification -----
Delimiter: ","
chr   (3): dia_semana, mes, trimestre
dbl   (10): producto_id, cliente_id, cantidad, precio_unitario, des...
date  (1): fecha

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Este mensaje de **especificación de columnas** no es un error: es **readr** contándote qué adivinó. Leyó 365 filas y 14 columnas separadas por coma (*Delimiter*); 3 columnas son texto (**chr**), 10 son números (**dbl**, de *double*) y 1 es fecha (**date**). Revisalo siempre la primera vez que leas un archivo: si una columna que debería ser número aparece como **chr**, hay algún valor raro en ella. Una vez revisado, puedes silenciarlo con **show_col_types = FALSE**, como haremos en el resto del libro:

```
1 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
2 class(ventas$fecha)
3 ventas
```

```
[1] "Date"
# A tibble: 365 x 14
  fecha      dia_semana producto_id cliente_id cantidad
  <date>    <chr>         <dbl>     <dbl>     <dbl>
1 2023-01-01 domingo         31         94         10
2 2023-01-02 lunes          15        169          8
3 2023-01-03 martes         14        136          3
4 2023-01-04 miércoles         3        167          2
5 2023-01-05 jueves         42         81         10
6 2023-01-06 viernes        50        191          8
7 2023-01-07 sábado         43         82         10
8 2023-01-08 domingo        37        106          7
9 2023-01-09 lunes         14        119          7
10 2023-01-10 martes        25         11          6
# i 355 more rows
# i 9 more variables: precio_unitario <dbl>, descuento_pct <dbl>,
#   tienda_id <dbl>, vendedor_id <dbl>, subtotal <dbl>,
#   descuento_monto <dbl>, total <dbl>, mes <chr>, trimestre <chr>
```

1.12.3 Leer Excel con readxl

Un libro de Excel puede tener varias hojas. Primero pregunta cuáles tiene con `excel_sheets()` y luego lee la que necesites con `read_excel()`, por nombre o por posición:

```
1 excel_sheets("datasets/datos_empresa.xlsx")      # ¿Qué hojas tiene?
2 tiendas <- read_excel("datasets/datos_empresa.xlsx", sheet = "Tiendas")
3 tiendas[, c("nombre_tienda", "ciudad", "tamano_m2", "empleados")]
4 productos_xl <- read_excel("datasets/datos_empresa.xlsx", sheet = 3)
5 dim(productos_xl)
```

```
[1] "Ventas"      "Clientes"    "Productos"   "Empleados"   "Tiendas"
# A tibble: 10 x 4
  nombre_tienda ciudad      tamano_m2 empleados
  <chr>         <chr>      <dbl>      <dbl>
1 Sucursal Centro CDMX        500        25
2 Sucursal Norte  Guadalajara 450         20
3 Sucursal Sur    Monterrey   600         30
4 Sucursal Este   CDMX        400         18
5 Sucursal Oeste  Puebla     550         22
6 Sucursal Plaza A CDMX        700         35
7 Sucursal Plaza B Guadalajara 480         21
8 Sucursal Outlet Tijuana     350         15
9 Sucursal Premium CDMX        800         40
10 Sucursal Express Querétaro 300         12
[1] 50 13
```

El archivo trae cinco hojas. Leímos la de tiendas por su nombre y la tercera (Productos, con 50 filas y 13 columnas) por su posición. Otros argumentos útiles de `read_excel()` son `skip = 3` (saltar las primeras filas, típicas de los reportes con título y logotipo) y `range = "B3:F40"` (leer solo un rango de celdas).

▲ Cuidado

Cierra el archivo en Excel antes de leerlo o sobrescribirlo desde R. En Windows, un archivo abierto en Excel queda bloqueado y R no podrá escribir sobre él.

1.12.4 Exportar resultados

Guardaremos todo lo que produzcamos en una carpeta `resultados/`. `dir.create()` la crea; con `showWarnings = FALSE` no se queja si ya existe.

```
1 dir.create("resultados", showWarnings = FALSE)
2
3 # Un resumen pequeño para exportar: ventas por trimestre
```

```

4 por_trimestre <- aggregate(total ~ trimestre, data = ventas_base, FUN = sum)
5 por_trimestre$total <- round(por_trimestre$total, 2)
6 por_trimestre
7
8 # CSV con R base (row.names = FALSE evita una columna extra de números)
9 write.csv(por_trimestre, "resultados/ventas_trimestre.csv",
10           row.names = FALSE)
11 # CSV con readr (nunca escribe nombres de fila)
12 write_csv(por_trimestre, "resultados/ventas_trimestre_readr.csv")
13
14 # Excel con varias hojas: una lista con nombre = una hoja por elemento
15 write_xlsx(list(Trimestres = por_trimestre,
16                 Tiendas = tiendas,
17                 Catalogo = productos),
18             "resultados/reporte_modulo1.xlsx")
19 excel_sheets("resultados/reporte_modulo1.xlsx")

```

```

      trimestre    total
1          Q1 126273.9
2          Q2 120716.9
3          Q3 134427.4
4          Q4 132498.3
[1] "Trimestres" "Tiendas"   "Catalogo"

```

`write_xlsx()` (del paquete **writexl**) crea un Excel con una hoja por cada elemento de la lista, y el nombre del elemento se usa como nombre de la hoja. Es la forma más rápida de mandarle a tu jefa varios cuadros en un solo archivo.

1.12.5 El formato nativo de R: RDS

Un CSV guarda solo texto: al volver a leerlo, R tiene que adivinar otra vez los tipos, y los factores o fechas pueden perderse. El formato **RDS** guarda *un* objeto de R exactamente como está, con sus tipos, y ocupa poco espacio. Es ideal para datos intermedios que solo usarás en R.

```

1 saveRDS(ventas, "resultados/ventas_2023.rds")      # Guardar
2 ventas_copia <- readRDS("resultados/ventas_2023.rds") # Leer
3 class(ventas_copia$fecha)                          # La fecha sigue
4 all.equal(ventas, ventas_copia)                    # ¿Son iguales?
5 file.exists(c("resultados/ventas_trimestre.csv",  # ¿Se crearon
6               "resultados/reporte_modulo1.xlsx",  # los archivos?
7               "resultados/ventas_2023.rds"))

```

```

[1] "Date"
[1] TRUE
[1] TRUE TRUE TRUE

```

 Consejo

Una regla práctica: usa CSV o Excel para *compartir* con personas u otros programas, y RDS para *guardar* tu trabajo intermedio en R. Y nunca sobrescribas los datos originales: lee de `datasets/` y escribe en `resultados/`.

1.12.6 Problemas típicos al importar

Separador punto y coma y decimales con coma

En muchos países (y en Excel configurado en español de España o Latinoamérica) los CSV usan `;` para separar columnas y `,` para los decimales. Simulemos un archivo así, como el que exportaría el sistema de un proveedor:

```
1 dir.create("datos", showWarnings = FALSE)
2 writeLines(c("producto;precio;unidades",
3             "Café molido;145,50;12",
4             "Té verde;89,90;30",
5             "Azúcar;32,00;45"),
6             "datos/proveedor.csv")
7 read.csv("datos/proveedor.csv")      # ¡Mal! Todo en una columna
```

	producto.precio.unidades
Café molido;145	50;12
Té verde;89	90;30
Azúcar;32	00;45

`read.csv()` buscó comas, encontró las de los decimales y armó una tabla sin sentido. La solución es usar las variantes “2”, pensadas para este formato:

```
1 read.csv2("datos/proveedor.csv")      # R base
2 read_csv2("datos/proveedor.csv", show_col_types = FALSE) # readr
```

```
      producto precio unidades
1 Café molido  145.5      12
2   Té verde   89.9      30
3    Azúcar   32.0      45
i Using "','" as decimal and "'.'" as grouping mark. Use `read_delim()` for more
  ↪ control.
# A tibble: 3 x 3
  producto      precio unidades
  <chr>      <dbl>     <dbl>
1 Café molido  146.         12
2 Té verde    89.9         30
```

3 Azúcar	32	45
----------	----	----

Ahora sí: tres productos, con el precio como número. `read_csv2()` te avisa qué supuso (coma como decimal y punto como separador de miles). Para combinaciones raras existe `read_delim()`, donde indicas todo a mano:

```
1 read_delim("datos/proveedor.csv", delim = ";",
2           locale = locale(decimal_mark = "."), show_col_types = FALSE)
```

```
# A tibble: 3 x 3
  producto    precio unidades
  <chr>      <dbl>    <dbl>
1 Café molido 146.        12
2 Té verde   89.9        30
3 Azúcar     32         45
```

Acentos que se ven raros: la codificación

Una **codificación** (*encoding*) es la tabla que traduce las letras a bytes. Hoy el estándar es **UTF-8**, pero los sistemas antiguos y algunas versiones de Excel en Windows guardan en **latin1** (también llamado ISO-8859-1 o Windows-1252). Si lees un archivo con la codificación equivocada, los acentos salen como símbolos extraños:

```
1 # Simulamos un archivo exportado por un sistema antiguo (latin1)
2 con <- file("datos/sistema_viejo.csv", open = "w", encoding = "latin1")
3 writeLines(c("producto,precio", "Café molido,145.5", "Té verde,89.9"), con)
4 close(con)
5
6 read_csv("datos/sistema_viejo.csv", show_col_types = FALSE)
7 read_csv("datos/sistema_viejo.csv", show_col_types = FALSE,
8           locale = locale(encoding = "latin1"))
```

```
# A tibble: 2 x 2
  producto    precio
  <chr>      <dbl>
1 "Caf\xe9 molido" 146.
2 "T\xe9 verde"    89.9
# A tibble: 2 x 2
  producto    precio
  <chr>      <dbl>
1 Café molido 146.
2 Té verde    89.9
```

En la primera lectura R no pudo interpretar la *é* y la mostró como `\xe9` (su código en latin1); en la

segunda, al indicar la codificación correcta, todo se ve bien. Con R base el argumento equivalente es `read.csv(archivo, fileEncoding = "latin1")`. Si no sabes qué codificación tiene un archivo, `guess_encoding(archivo)` de **readr** te da una pista.

Rutas en Windows: \ contra /

En Windows, el Explorador de archivos muestra las rutas con diagonal invertida (`C:\Users\ana...`). Si copias una ruta así en R, falla, porque para R la diagonal invertida es un carácter especial (`\n` es un salto de línea, `\t` un tabulador...):

```
1 ruta <- "C:\Users\ana\ventas.csv"
```

```
Error: '\U' used without hex digits in character string (<input>:1:13)
```

Tienes tres soluciones: usar diagonales normales (`/`), que R acepta también en Windows; duplicar las invertidas (`\\`); o, mejor aún, construir la ruta con `file.path()`, que pone el separador correcto por ti:

```
1 "C:/Users/ana/ventas.csv"           # Diagonal normal: funciona
2 "C:\\Users\\ana\\ventas.csv"        # Diagonal doble: también
3 file.path("datasets", "ventas_retail.csv") # Ruta relativa armada
```

```
[1] "C:/Users/ana/ventas.csv"
[1] "C:\\Users\\ana\\ventas.csv"
[1] "datasets/ventas_retail.csv"
```

Hazlo tú

Lee la hoja **"Empleados"** de `datasets/datos_empresa.xlsx`, revisa con `str()` qué tipo tiene cada columna y guarda en `resultados/empleados_ventas.csv` solo los empleados del departamento de Ventas. Después vuelve a leer ese CSV con `read_csv()` y comprueba con `nrow()` cuántos son.

1.13 Cómo obtener ayuda

Nadie memoriza todas las funciones de R, ni siquiera los expertos. Lo que distingue a un buen analista es saber *encontrar* la respuesta rápido.

1.13.1 La ayuda integrada

Cada función de R tiene una página de documentación. Estas son las formas de llegar a ella:


```

1 ?mean                                # Ayuda de una función (abre el panel Help)
2 help("read_csv")                     # Lo mismo, con el nombre como texto
3 ?? "moving average"                  # Busca un tema en TODA la documentación
4 example(mean)                        # Ejecuta los ejemplos de la ayuda
5 vignette("readxl")                   # Tutoriales largos de un paquete (viñetas)

```

También puedes colocar el cursor sobre el nombre de una función en tu script y presionar **F1**. Dos funciones más te ayudan desde la propia consola: **args()** muestra los argumentos de una función y **apropos()** busca funciones cuyo nombre contenga un texto.

```

1 args(round)                           # ¿Qué argumentos acepta round()?
2 apropos("mean")                       # Funciones cuyo nombre contiene "mean"

```

```

function (x, digits = 0)
NULL
 [1] ".colMeans"      ".rowMeans"      "colMeans"
 [4] "cummean"        "kmeans"         "mean"
 [7] "mean_cl_boot"   "mean_cl_normal" "mean_sdl"
[10] "mean_se"        "mean.Date"      "mean.default"
[13] "mean.difftime"  "mean.POSIXct"   "mean.POSIXlt"
[16] "rowMeans"       "weighted.mean"

```

1.13.2 Cómo leer una página de ayuda

Las páginas de ayuda siempre tienen las mismas secciones. Al principio intimidan; léelas en este orden:

1. **Description:** qué hace la función, en una o dos frases.
2. **Usage:** cómo se escribe, con los argumentos y sus valores por defecto. Por ejemplo, **mean(x, trim = 0, na.rm = FALSE, ...)** te dice que **na.rm** vale **FALSE** si no lo cambias: por eso los **NA** “contagian” el promedio.
3. **Arguments:** qué significa cada argumento.
4. **Value:** qué devuelve la función.
5. **Examples:** ejemplos listos para copiar. ¡Empieza por aquí si tienes prisa!

1.13.3 Cheatsheets y recursos

Las **cheatsheets** (hojas de referencia) son resúmenes de una o dos páginas con las funciones más usadas de un paquete. RStudio incluye enlaces en **Help > Cheat Sheets**, y todas están en <https://posit.co/resources/cheatsheets/>. Imprime las de RStudio, **readr** y **dplyr** y tenlas a la mano durante el curso.

1.13.4 Buscar un error en internet

Cuando R te muestre un error, sigue esta receta:

1. Lee el mensaje con calma: muchas veces dice exactamente qué pasa (“object not found”, “could not find function”).
2. Copia la parte general del mensaje, sin tus nombres de variables, y búscala entre comillas junto con la palabra “R”. Por ejemplo: `"non-numeric argument to binary operator" R`.
3. Los mejores resultados suelen estar en *Stack Overflow* (hay versión en español), en la comunidad de Posit (<https://forum.posit.co>) y en la documentación de los paquetes.

Por eso conviene conocer los mensajes en inglés: hay mucha más ayuda disponible. Si tu R muestra los mensajes en español y prefieres verlos en inglés, ejecuta `Sys.setenv(LANGUAGE = "en")` al inicio de la sesión.

1.13.5 Hacer una buena pregunta: el ejemplo reproducible

Si nada funciona, pregunta en un foro o a un colega. Una buena pregunta incluye un **ejemplo reproducible** (*reprex*): el código *mínimo* que cualquiera puede copiar y ejecutar para ver tu mismo problema. Nadie puede ayudarte con “no me sale el promedio”; todos pueden ayudarte con esto:

```
1 # Pregunta: ¿por qué mean() me da NA si mi columna tiene números?
2 ventas <- data.frame(tienda = c("A", "B", "C"),
3                       monto = c(1500, NA, 2300))
4 mean(ventas$monto)
5 #> [1] NA
6 # Esperaba 1900. Uso R 4.3.3 en Windows 11.
```

Un buen ejemplo reproducible: (1) carga los paquetes que usa, (2) incluye datos pequeños *inventados* o un fragmento de los tuyos (nunca datos confidenciales de clientes), (3) muestra el resultado que obtienes y el que esperabas. Para convertir un pedazo de tus datos en código que otros puedan pegar, usa `dput()`:

```
1 dput(productos[1:3, c("nombre", "precio")])
```

```
structure(list(nombre = c("Laptop Pro 14", "Mouse inalámbrico",
"Monitor 27"), precio = c(18999, 349, 5499)), row.names = c(NA,
3L), class = "data.frame")
```

La salida parece críptica, pero es código de R que recrea exactamente esas tres filas: quien lo pegue en su consola tendrá tus mismos datos.

Hazlo tú

Abre la ayuda de `round()` con `?round`. En la sección *Usage* encuentra qué hace la función `signif()` y pruébala con `signif(513916.6, 2)`. ¿Para qué te serviría en un reporte ejecutivo?

1.14 Caso práctico: tu primer análisis de ventas

Es momento de responder las preguntas de tu jefa del inicio del módulo. Usaremos solo R base, es decir, las herramientas que aprendiste en este módulo, sobre el archivo `ventas_retail.csv`: una venta registrada por cada día de 2023.

En la empresa: El encargo

La dirección comercial quiere un resumen de 2023 para su junta mensual: (1) venta total y ticket promedio, (2) ventas por trimestre, (3) mejor y peor mes, (4) si los días con descuento venden más que los días sin descuento y (5) los cinco días de mayor venta. Todo debe llegar en un Excel.

1.14.1 Paso 1: cargar y conocer los datos

```
1 # ---- 1. Cargar -----
2 ventas_2023 <- read.csv("datasets/ventas_retail.csv")
3 ventas_2023$fecha <- as.Date(ventas_2023$fecha) # Texto -> fecha
4 str(ventas_2023)
```

```
'data.frame': 365 obs. of 14 variables:
 $ fecha      : Date, format: "2023-01-01" ...
 $ dia_semana : chr "domingo" "lunes" "martes" "miércoles" ...
 $ producto_id : int 31 15 14 3 42 50 43 37 14 25 ...
 $ cliente_id  : int 94 169 136 167 81 191 82 106 119 11 ...
 $ cantidad    : int 10 8 3 2 10 8 10 7 7 6 ...
 $ precio_unitario: num 351 283 297 222 219 ...
 $ descuento_pct : int 5 0 10 0 0 0 15 0 0 15 ...
 $ tienda_id    : int 6 5 2 7 3 3 4 1 6 5 ...
 $ vendedor_id  : int 5 24 25 18 8 19 3 5 8 12 ...
 $ subtotal     : num 3508 2262 890 445 2188 ...
 $ descuento_monto: num 175 0 89 0 0 ...
 $ total        : num 3333 2262 801 445 2188 ...
 $ mes          : chr "2023-01" "2023-01" "2023-01" "2023-01" ...
 $ trimestre    : chr "Q1" "Q1" "Q1" "Q1" ...
```

Cada fila es una venta con su fecha, producto, cliente, cantidad, precio unitario, porcentaje de descuento, tienda y vendedor. Las columnas `subtotal`, `descuento_monto` y `total` ya vienen calculadas, y `mes` y `trimestre` sirven para agrupar. Antes de calcular, comprobemos la calidad básica:

```

1 range(ventas_2023$fecha)           # ¿Qué periodo cubre?
2 sum(is.na(ventas_2023))           # ¿Hay datos faltantes?
3 sum(duplicated(ventas_2023$fecha)) # ¿Fechas repetidas?

```

```

[1] "2023-01-01" "2023-12-31"
[1] 0
[1] 0

```

Los datos cubren el año completo, no hay valores faltantes ni días repetidos. Podemos confiar en los totales.

1.14.2 Paso 2: KPIs generales

```

1 # ---- 2. KPIs del año -----
2 venta_total <- sum(ventas_2023$total)
3 ticket_promedio <- mean(ventas_2023$total)
4 unidades_totales <- sum(ventas_2023$cantidad)
5 descuentos_otorgados <- sum(ventas_2023$descuento_monto)
6
7 scales::dollar(venta_total)
8 scales::dollar(ticket_promedio, accuracy = 0.01)
9 unidades_totales
10 scales::dollar(descuentos_otorgados)
11 summary(ventas_2023$total)

```

```

[1] "$513,917"
[1] "$1,407.99"
[1] 2057
[1] "$27,360.13"
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
20.07  448.38 1103.67 1407.99 2169.50 4950.30

```

¿Qué nos dice esto? En 2023 se vendieron \$513,917 en 2,057 unidades, con un ticket promedio de \$1,407.99. Los descuentos le costaron a la empresa \$27,360, alrededor del 5% de la venta. El ticket *mediano* (\$1,103.67) es menor que el promedio: hay algunas ventas muy grandes (hasta \$4,950) que jalen el promedio hacia arriba.

1.14.3 Paso 3: ventas por trimestre

```

1 # ---- 3. Por trimestre -----
2 venta_trimestre <- tapply(ventas_2023$total, ventas_2023$trimestre, sum)
3 round(venta_trimestre, 2)
4 round(venta_trimestre / venta_total * 100, 1)      # Participación %

```

```

      Q1      Q2      Q3      Q4
126273.9 120716.9 134427.4 132498.3
      Q1  Q2  Q3  Q4
24.6 23.5 26.2 25.8

```

Las ventas son bastante parejas a lo largo del año: cada trimestre aporta entre 23.5 % y 26.2 %. El tercer trimestre fue el más fuerte (\$134,427) y el segundo el más débil (\$120,717).

1.14.4 Paso 4: mejor y peor mes

```

1 # ---- 4. Por mes -----
2 venta_mes <- tapply(ventas_2023$total, ventas_2023$mes, sum)
3 round(venta_mes)
4 names(which.max(venta_mes))
5 names(which.min(venta_mes))
6 round(max(venta_mes) / min(venta_mes), 2)      # ¿Cuántas veces más?

```

```

2023-01 2023-02 2023-03 2023-04 2023-05 2023-06 2023-07 2023-08
  46884  42656  36733  34738  48108  37871  54056  41421
2023-09 2023-10 2023-11 2023-12
  38950  36584  50398  45516
[1] "2023-07"
[1] "2023-04"
[1] 1.56

```

Julio fue el mejor mes (\$54,056) y abril el peor (\$34,738): el mejor mes vendió 1.56 veces lo que el peor. En una junta, esto abre una pregunta de negocio: ¿qué pasó en abril? (¿Semana Santa?, ¿falta de inventario?, ¿una campaña que no funcionó?). Los datos no responden el *por qué*, pero te dicen dónde buscar.

1.14.5 Paso 5: ¿venden más los días con descuento?

```

1 # ---- 5. Con y sin descuento -----
2 ventas_2023$con_descuento <- ifelse(ventas_2023$descuento_pct > 0,
3                                     "Con descuento", "Sin descuento")
4 table(ventas_2023$con_descuento)
5 round(tapply(ventas_2023$total, ventas_2023$con_descuento, mean), 2)
6 round(tapply(ventas_2023$cantidad, ventas_2023$con_descuento, mean), 2)

```

Con descuento	Sin descuento
177	188
Con descuento	Sin descuento
1351.79	1460.90
Con descuento	Sin descuento
5.67	5.61

¿Qué nos dice esto? Hubo descuento en 177 de los 365 días. Pero los días con descuento tuvieron un ticket promedio *menor* (\$1,351.79 contra \$1,460.90) y prácticamente las mismas unidades por venta (5.67 contra 5.61). Es decir, en estos datos el descuento no se traduce en clientes que compren más piezas: solo reduce el ingreso. Es un hallazgo importante para la dirección comercial, aunque con una advertencia honesta: comparar promedios no prueba causalidad (los descuentos pudieron aplicarse a productos distintos). En el Módulo 4 aprenderás a comprobar si una diferencia así es estadísticamente significativa.

1.14.6 Paso 6: los cinco mejores días

```

1 # ---- 6. Top 5 días -----
2 orden <- order(ventas_2023$total, decreasing = TRUE)
3 top5 <- ventas_2023[orden[1:5],
4                      c("fecha", "dia_semana", "cantidad",
5                        "precio_unitario", "descuento_pct", "total")]
6 top5

```

	fecha	dia_semana	cantidad	precio_unitario	descuento_pct
205	2023-07-24	lunes	10	495.03	0
111	2023-04-21	viernes	10	492.49	0
247	2023-09-04	lunes	10	487.48	0
46	2023-02-15	miércoles	10	488.26	5
47	2023-02-16	jueves	9	485.20	0
	total				
205	4950.30				
111	4924.90				
247	4874.80				

```
46 4638.47
47 4366.80
```

Los cinco mejores días tienen algo en común: se vendieron 9 o 10 unidades de un producto caro (cerca de \$490 cada una), casi siempre sin descuento. Las ventas grandes vienen de pedidos de muchas piezas de productos de precio alto.

1.14.7 Paso 7: exportar el resumen a Excel

Armamos cada cuadro como data frame y los guardamos en un solo Excel, una hoja por cuadro:

```
1 # ---- 7. Exportar -----
2 kpis <- data.frame(
3   indicador = c("Venta total", "Ticket promedio", "Unidades vendidas",
4                 "Descuentos otorgados", "Mejor mes", "Peor mes"),
5   valor = c(scales::dollar(venta_total),
6             scales::dollar(ticket_promedio, accuracy = 0.01),
7             scales::comma(unidades_totales),
8             scales::dollar(descuentos_otorgados),
9             names(which.max(venta_mes)), names(which.min(venta_mes)))
10 )
11 tabla_trimestre <- data.frame(trimestre = names(venta_trimestre),
12                               venta = round(as.numeric(venta_trimestre), 2))
13 tabla_mes <- data.frame(mes = names(venta_mes),
14                         venta = round(as.numeric(venta_mes), 2))
15 tabla_descuento <- aggregate(total ~ con_descuento, data = ventas_2023,
16                               FUN = mean)
17
18 dir.create("resultados", showWarnings = FALSE)
19 write_xlsx(list(KPIs = kpis, Trimestres = tabla_trimestre,
20               Meses = tabla_mes, Descuentos = tabla_descuento,
21               Top5 = top5),
22            "resultados/resumen_ventas_2023.xlsx")
23 kpis
24 excel_sheets("resultados/resumen_ventas_2023.xlsx")
```

	indicador	valor
1	Venta total	\$513,917
2	Ticket promedio	\$1,407.99
3	Unidades vendidas	2,057
4	Descuentos otorgados	\$27,360.13
5	Mejor mes	2023-07
6	Peor mes	2023-04

```
[1] "KPIs"      "Trimestres" "Meses"      "Descuentos" "Top5"
```

Listo: el archivo resultados/resumen_ventas_2023.xlsx tiene cinco hojas con todo lo que pidió

la dirección. Lo mejor es que el próximo año solo cambias el archivo de entrada y vuelves a ejecutar el script: el reporte se regenera en segundos. Eso es automatizar un reporte de BI.

Hazlo tú

Agrega al caso un paso 8: calcula la venta total por día de la semana con `tapply()` y ordénala de mayor a menor con `sort()`. ¿Qué día conviene reforzar con personal? Agrégala como una hoja más del Excel.

1.15 Errores comunes y cómo resolverlos

Equivocarse es parte normal de programar; lo importante es entender qué te dice R. Los mensajes aparecen en inglés en la mayoría de las instalaciones (o en español, si tu sistema está configurado así; el significado es el mismo). La tabla 1.6 reúne los que más verás en este curso.

Tabla 1.6. Mensajes de error frecuentes, su causa y su solución.

Mensaje de R	Causa probable	Solución
<code>object 'ventas_total' not found</code>	El objeto no existe: nombre mal escrito o no ejecutaste la línea que lo crea.	Revisa mayúsculas y guiones bajos; busca el nombre en Environment ; ejecuta el script desde el principio.
<code>could not find function "read_csv"</code>	La función es de un paquete que no has cargado, o su nombre está mal escrito (Mean en vez de mean).	Ejecuta <code>library(readr)</code> (o <code>library(tidyverse)</code>) o usa <code>readr::read_csv()</code> .
<code>unexpected symbol in "ventas enero"</code>	Hay algo de más o de menos en la sintaxis: un espacio en un nombre, una coma o un paréntesis faltante.	Revisa la línea indicada: usa <code>ventas_enero</code> y cuenta comas y paréntesis.
<code>non-numeric argument to binary operator</code>	Operación aritmética con texto: <code>"100" + 1</code> .	Revisa <code>class()</code> y convierte con <code>as.numeric()</code> .
<code>cannot open file 'datos/ventas.csv': No such file or directory</code>	R no encuentra el archivo en la ruta indicada, relativa al directorio de trabajo.	Comprueba <code>getwd()</code> , abre el Proyecto de RStudio y revisa el nombre con <code>list.files()</code> .
<code>there is no package called 'janitor'</code>	El paquete no está instalado.	<code>install.packages("janitor")</code> una vez y luego <code>library(janitor)</code> .
<code>argument is not numeric or logical: returning NA</code>	(Advertencia) Pediste <code>mean()</code> de una columna de texto.	Convierte la columna a número o revisa por qué quedó como texto.
<code>the condition has length > 1 undefined columns selected</code>	Usaste <code>if</code> con un vector. Columna mal escrita o faltó la coma en <code>df[condición,]</code> .	Usa <code>ifelse()</code> o <code>any()/all()</code> . Revisa <code>names(df)</code> y la coma dentro de los corchetes.
<code>arguments imply differing number of rows: 3, 2</code>	Al crear un data frame, las columnas tienen distinto largo.	Cuenta los elementos de cada vector con <code>length()</code> .

unexpected end of inputFalta cerrar un paréntesis,
llave o comillas.En la consola aparece **+**; presiona
Esc y corrige.

Veamos los más frecuentes tal como los imprime R.

Objeto no encontrado

```
1 ventas_totales <- 1000
2 ventas_total           # Falta la "s"
```

```
Error: object 'ventas_total' not found
```

R distingue cada letra: **ventas_totales** existe, pero **ventas_total** no. Usa **Tab** para autocompletar nombres y evitar erratas.

Función no encontrada

```
1 # En una sesión nueva, sin haber cargado readr
2 total <- read_csv("datasets/ventas_retail.csv")
```

```
Error in read_csv("datasets/ventas_retail.csv") :
  could not find function "read_csv"
```

Símbolo inesperado

```
1 ventas enero <- 15000
```

```
Error: unexpected symbol in "ventas enero"
```

Texto en una operación aritmética

```
1 precio <- "100"
2 precio + 1
```

```
Error in precio + 1 : non-numeric argument to binary operator
```

Archivo que no existe

```
1 ventas <- read.csv("datos/ventas_retail.csv") # La carpeta es datasets
```

```
Error in file(file, "rt") : cannot open the connection
In addition: Warning message:
In file(file, "rt") :
  cannot open file 'datos/ventas_retail.csv': No such file or directory
```

El error principal (*cannot open the connection*) es genérico; la pista útil está en la advertencia de abajo: el archivo no está en esa ruta. Aquí la carpeta correcta era **datasets**.

Paquete no instalado

```
1 library(janitor)
```

```
Error in library(janitor) : there is no package called 'janitor'
```

Promedio de un texto

```
1 mean(c("1500", "2300")) # Números guardados como texto
```

```
[1] NA
Warning message:
In mean.default(c("1500", "2300")) :
  argument is not numeric or logical: returning NA
```

Esta es una advertencia, no un error: R no se detiene, pero el resultado es **NA**. Es peligrosa porque, si no lees la consola, un **NA** puede colarse en tu reporte.

Consejo

Ante cualquier error: (1) lee el mensaje completo, (2) identifica la línea que falló, (3) revisa nombres, tipos (`class()`, `str()`) y rutas (`getwd()`, `list.files()`), (4) reinicia R y ejecuta el script desde el principio. La mayoría de los errores se resuelven en uno de estos cuatro pasos.

 Hazlo tú

Provoca a propósito tres de estos errores en tu consola (por ejemplo, escribe mal un nombre, suma un texto y lee un archivo que no existe). Leer los mensajes cuando *sabes* qué los causó es la mejor forma de reconocerlos después.

1.16 Resumen del módulo

Resumen del módulo

- RStudio tiene cuatro paneles: editor, consola, entorno y archivos/gráficas/ayuda. Escribe siempre en scripts, ejecuta con `Ctrl` + `Enter` y trabaja dentro de un Proyecto con rutas relativas (sin `setwd()`).
- R es una calculadora potente; guardas resultados en variables con `<-` y nombres en `snake_case`.
- Cada valor tiene un tipo: `numeric`, `integer`, `character`, `logical`, `Date` y `factor`. Revisalo con `class()` y conviértelo con `as.*()`, cuidando las trampas (comas de miles, factores, texto que parece número).
- Los vectores guardan muchos valores del mismo tipo; las operaciones son vectorizadas. Se filtran con corchetes por posición, nombre o condición, y `sum(condición)` cuenta.
- `NA` marca datos faltantes; usa `na.rm = TRUE` y `is.na()`.
- Los data frames (y tibbles) son las tablas de R: se exploran con `str()` y `summary()`, se filtran con `df[condición, ]` y se resumen con `tapply()` y `aggregate()`.
- `if/else`, `ifelse()`, `for` y `while` controlan el flujo; en R se prefiere lo vectorizado.
- Tus funciones con `function()` encapsulan reglas de negocio y se validan con `stop()` y `warning()`.
- Los paquetes se instalan una vez y se cargan en cada sesión. `read_csv()`, `read_excel()`, `write_xlsx()` y `saveRDS()` mueven datos entre R y el mundo exterior.

Tabla 1.7. Funciones clave del Módulo 1.

Función	Para qué sirve
<code>getwd()</code> , <code>list.files()</code> , <code>file.exists()</code>	Directorio de trabajo y archivos disponibles
<code>round()</code> , <code>ceiling()</code> , <code>floor()</code> , <code>abs()</code> , <code>sqrt()</code> , <code>log()</code>	Funciones matemáticas
<code>ls()</code> , <code>rm()</code>	Listar y borrar objetos
<code>class()</code> , <code>typeof()</code> , <code>is.*()</code> , <code>as.*()</code>	Consultar y convertir tipos
<code>as.Date()</code> , <code>Sys.Date()</code> , <code>format()</code> , <code>weekdays()</code>	Crear, formatear y operar fechas

Función	Para qué sirve
<code>factor()</code> , <code>levels()</code> , <code>table()</code>	Categorías y conteos
<code>c()</code> , <code>seq()</code> , <code>rep()</code> , <code>length()</code>	Crear vectores
<code>sum()</code> , <code>mean()</code> , <code>median()</code> , <code>sd()</code> , <code>quantile()</code> , <code>summary()</code>	Resumir un vector
<code>cumsum()</code> , <code>diff()</code> , <code>sort()</code> , <code>order()</code>	Acumulados, cambios y ordenamiento
<code>which()</code> , <code>which.max()</code> , <code>%in%</code> , <code>is.na()</code>	Localizar elementos y faltantes
<code>paste()</code> , <code>paste0()</code> , <code>sprintf()</code>	Construir textos
<code>toupper()</code> , <code>trimws()</code> , <code>substr()</code> , <code>gsub()</code> , <code>grep()</code>	Limpiar y transformar textos
<code>scales::dollar()</code> , <code>scales::percent()</code>	Formato de moneda y porcentaje
<code>data.frame()</code> , <code>tibble::tibble()</code>	Crear tablas
<code>head()</code> , <code>str()</code> , <code>nrow()</code> , <code>names()</code>	Explorar tablas
<code>subset()</code> , <code>tapply()</code> , <code>aggregate()</code>	Filtrar y resumir por grupos
<code>matrix()</code> , <code>rowSums()</code> , <code>colSums()</code> , <code>list()</code>	Matrices y listas
<code>if</code> , <code>ifelse()</code> , <code>for</code> , <code>while</code> , <code>sapply()</code> , <code>vapply()</code>	Control de flujo
<code>function()</code> , <code>return()</code> , <code>stop()</code> , <code>warning()</code>	Funciones propias
<code>install.packages()</code> , <code>library()</code> , <code>package::funcion()</code>	Paquetes
<code>read.csv()</code> , <code>read_csv()</code> , <code>read_csv2()</code> , <code>read_excel()</code> , <code>excel_sheets()</code>	Importar datos
<code>write.csv()</code> , <code>write_csv()</code> , <code>write_xlsx()</code> , <code>saveRDS()</code> , <code>readRDS()</code>	Exportar y guardar
<code>?</code> , <code>help()</code> , <code>args()</code> , <code>apropos()</code> , <code>dput()</code>	Obtener ayuda y preguntar bien

En el Módulo 2 darás el siguiente gran salto: **dplyr** y **tidyr** convertirán muchas de las operaciones de este módulo (filtrar, ordenar, crear columnas, resumir por grupos) en un lenguaje mucho más legible y potente.

1.17 Ejercicios

Resuelve cada ejercicio en un script nuevo guardado en tu carpeta `mis_scripts/`, con el Proyecto del curso abierto. Los ejercicios van de *Básico* a *Reto*; las soluciones completas y comentadas están en el Apéndice A.

Ejercicio 1.1: Calculadora de ticketNivel: **Básico**

Un cliente compra 2 audífonos de \$899, 1 teclado de \$1,299 y 3 memorias USB de \$189 (precios sin IVA). Con variables, calcula: (a) el subtotal, (b) el IVA del 16 %, (c) el total, (d) el total si se aplica un cupón del 10 % sobre el subtotal antes de IVA y (e) cuánto ahorra el cliente con el cupón. Muestra el total final con `scales::dollar()`.

 Solución en la página 606**Ejercicio 1.2: KPIs y punto de equilibrio de una sucursal**Nivel: **Básico**

En abril la Sucursal Norte vendió \$612,400; el costo de la mercancía fue \$379,700, los gastos fijos \$158,000 y la meta del mes \$580,000. Calcula la utilidad bruta, la utilidad de operación, el margen bruto y el de operación (en %, un decimal), y el cumplimiento de la meta. Después calcula el **punto de equilibrio** en ventas: las ventas mínimas para no perder dinero, que es `gastos_fijos / margen_bruto` (con el margen como proporción). ¿Cuánto por encima del punto de equilibrio está la sucursal?

 Solución en la página 607**Ejercicio 1.3: Limpiar montos y fechas**Nivel: **Básico**

Un sistema exportó estos montos como texto: `c("$1,250.50", "$980.00", "$12,400.75", "$3,105.20")` y sus fechas de factura en formato mexicano: `c("05/01/2023", "19/01/2023", "02/02/2023", "28/02/2023")`. (a) Convierte los montos a número y súmalos. (b) Convierte las fechas a `Date`. (c) Calcula la fecha de vencimiento de cada factura a 30 días y muéstrala en formato `dd/mm/aaaa`. (d) ¿Qué día de la semana vence cada una?

 Solución en la página 608**Ejercicio 1.4: Un año de ventas en un vector**Nivel: **Básico**

Las ventas mensuales (en miles de pesos) de una tienda fueron: 420, 385, 455, 402, 478, 510, 495, 530, 468, 501, 560 y 640, de enero a diciembre. Crea un vector con nombres (`month.abb` te da los meses en inglés; usa mejor `c("ene", "feb", ...)`). Calcula: total anual, promedio, mejor y peor mes, cuántos meses superaron \$500 mil, el crecimiento porcentual de cada mes contra el anterior y el acumulado del año. ¿En qué mes el acumulado superó los 3 millones?

 Solución en la página 609**Ejercicio 1.5: Días sin datos**Nivel: **Intermedio**

Las ventas de 10 días de una tienda fueron: `c(8200, 7650, NA, 9100, 10400, NA, 8800, 12100, NA, 9650)`. (a) ¿Cuántos días no tienen dato y cuáles son? (b) Calcula la venta promedio y total con los días disponibles. (c) ¿Cuántos días con dato superaron la meta de \$9,000 y qué porcentaje representan? (d) Crea una copia donde cada `NA` se reemplace por el promedio de los días disponibles y compara el total estimado con el total observado. Explica por qué no conviene reemplazar por cero.

 Solución en la página 610

Ejercicio 1.6: Directorio de clientes

Nivel: **Intermedio**

Tienes los nombres `c(" ana LÓPEZ", "Carlos pérez ", "MARÍA gonzález")`, sus ciudades `c("cdmx", "Monterrey ", " CDMX")` y sus compras del año `c(15230.5, 8900, 42100.75)`. (a) Limpia los espacios y deja nombres y ciudades en mayúsculas. (b) Construye con `sprintf()` y `scales::dollar()` un mensaje por cliente del estilo "ANA LÓPEZ (CDMX) compró \$15,230.50". (c) Genera un código de cliente `CLI-001, CLI-002...` con `sprintf()`. (d) Cuenta cuántos clientes son de CDMX.

 Solución en la página 611

Ejercicio 1.7: Análisis de una plantilla de empleados

Nivel: **Intermedio**

Crea un data frame con 8 empleados: nombre, departamento (Ventas, Operaciones o TI), salario mensual, años de antigüedad y calificación de desempeño (1 a 5); inventa los valores o usa los de la solución. (a) Agrega una columna `bono` igual al 10 % del salario anual si la calificación es 4 o más, y 0 si no. (b) Muestra los empleados de Ventas ordenados de mayor a menor salario. (c) Con `aggregate()` calcula el salario promedio y el total de bonos por departamento. (d) ¿Qué departamento tiene el mayor costo anual de nómina (salario $\times$ 12 + bono)?

 Solución en la página 611

Ejercicio 1.8: Semáforo de metas y proyección

Nivel: **Intermedio**

Seis vendedores tuvieron estas ventas del trimestre: 185, 240, 132, 310, 198 y 260 (miles de pesos), con una meta común de 200 mil. (a) Con `ifelse()` anidado asigna un semáforo: "Verde" si cumplen 100 % o más, "Amarillo" entre 85 % y 99.9 %, "Rojo" debajo de 85 %. (b) Con un `for` imprime con `cat()` una línea por vendedor con su venta, su cumplimiento y su semáforo. (c) Con un `while` calcula cuántos trimestres necesita el vendedor con menor venta para alcanzar la meta si crece 8 % cada trimestre.

 Solución en la página 613

Ejercicio 1.9: Funciones de precios y punto de equilibrioNivel: **Avanzado**

(a) Escribe `precio_final(precio, descuento = 0, iva = 0.16)` que aplique el descuento y luego el IVA, redondee a 2 decimales y se detenga con `stop()` si el descuento no está entre 0 y 1 o si el precio es negativo. (b) Escribe `punto_equilibrio(costos_fijos, precio, costo_variable)` que devuelva las unidades mínimas a vender (redondeadas hacia arriba) y se detenga si el precio no supera al costo variable. (c) Úsalas: precio final de un catálogo de 4 productos con 20 % de descuento, y unidades de equilibrio para un negocio con \$85,000 de costos fijos, precio de \$450 y costo variable de \$280. (d) Muestra con `tryCatch()` o con un bloque comentado qué pasa si el descuento es 1.5.

 Solución en la página 614**Ejercicio 1.10: Calidad de datos del catálogo**Nivel: **Avanzado**

Lee `datasets/productos.csv` con `read_csv()`. (a) ¿Cuántos productos y columnas hay? (b) ¿Cuántos productos tienen margen negativo (costo mayor que precio de catálogo)? Muestra los 5 peores. (c) Calcula el margen promedio por categoría con `aggregate()` y ordénalo. (d) ¿Cuántos productos están en “Stock Bajo” o “Sin Stock”? (e) Exporta a `resultados/calidad_catalogo.xlsx` dos hojas: los productos con margen negativo y el resumen por categoría. Redacta dos frases de conclusión para el área de compras.

 Solución en la página 615**Ejercicio 1.11: Métodos de pago**Nivel: **Avanzado**

Con `datasets/transacciones.csv`: (a) calcula el número de transacciones, el monto total y el ticket promedio por método de pago; (b) ¿qué porcentaje del monto total se pagó con tarjeta (crédito o débito)?; (c) ¿cuál fue el mes con más ingresos y cuál el día de la semana con más transacciones?; (d) exporta el resumen por método de pago a CSV y a RDS en `resultados/`, vuelve a leer el RDS y comprueba que es idéntico.

 Solución en la página 617**Ejercicio 1.12: Reto: reporte automático por sucursal**Nivel: **Reto**

Combina `datasets/ventas_retail.csv` y `datasets/tiendas.csv`. Escribe una función `resumen_tienda(id, ventas, tiendas)` que devuelva un data frame de una fila con: nombre de la tienda, ciudad, número de ventas, venta total, ticket promedio, mejor mes y venta por metro cuadrado. Aplícala a las 10 tiendas (con un `for` o con `lapply()` + `do.call(rbind, ...)`), ordena el resultado por venta total, imprime un ranking con `cat()` y `sprintf()` y guarda todo en `resultados/ranking_tiendas.xlsx`. ¿Qué tienda vende más por metro cuadrado? ¿Coincide con la que más vende en total?

 Solución en la página [618](#)

Manipulación de datos con dplyr y tidyr

Al terminar este módulo podrás...

- Explicar qué son los *datos ordenados* (*tidy data*) y por qué facilitan todo el análisis posterior.
- Importar archivos CSV con **readr**, controlar los tipos de columna y detectar problemas de lectura.
- Encadenar pasos con el pipe (`%>%` y `|>`) y leer tu código como si fuera una receta.
- Responder preguntas de negocio con los verbos de **dplyr**: `filter()`, `select()`, `mutate()`, `arrange()`, `summarise()` y `group_by()`.
- Calcular KPIs con funciones de ventana: crecimiento mes contra mes, acumulado del año, rankings y promedios móviles.
- Manejar fechas con **lubridate** y texto con **stringr**.
- Unir tablas con los distintos tipos de *join* y detectar sus trampas (llaves duplicadas, registros huérfanos).
- Cambiar la forma de una tabla con **tidyr** (formato ancho y largo, tablas dinámicas, celdas vacías).
- Limpiar un archivo “sucio” paso a paso y construir un reporte mensual completo que se exporta a Excel.

Es lunes, 9:00 de la mañana. La directora comercial te escribe: “*¿Me mandas antes de la junta de las 12 las ventas de 2023 por mes y por categoría, cuánto crecimos contra el mes anterior, quiénes son nuestros cinco mejores clientes y qué tienda va peor? En Excel, por favor.*” Tienes los datos, pero están repartidos en varios archivos: las ventas por un lado, el catálogo de productos por otro, los clientes, las tiendas y los empleados en archivos distintos. Ninguno, por sí solo, responde la pregunta.

Si haces esto a mano en Excel, te espera una mañana de **BUSCARV**, tablas dinámicas, copiar y pegar... y el próximo mes, repetirlo todo. En este módulo aprenderás a hacerlo con **dplyr** y **tidyr**, los dos paquetes que más vas a usar en tu vida como analista. Al final del módulo (sección 2.11) construirás exactamente ese reporte con un script de R que, el mes que entra, se vuelve a ejecutar en segundos.

Este es el módulo más largo del libro, y con razón: aquí está el 80% de tu trabajo diario. No intentes memorizar todas las funciones; concéntrate en entender la *lógica* de cada verbo y teclea los ejemplos tú mismo. Todo el código usa los datos del curso (`ventas_retail.csv`, `productos.csv`, `clientes.csv`, `tiendas.csv`, `empleados.csv` y `transacciones.csv`), así que puedes reproducir cada salida exactamente como aparece en el libro.

2.1 Preparar los datos: el 80 % del trabajo

En la industria se repite mucho una cifra: un analista dedica alrededor del 80 % de su tiempo a *preparar* datos (conseguirlos, limpiarlos, unirlos y darles forma) y solo el 20 % restante a analizarlos y presentarlos. La cifra exacta varía según la empresa, pero la idea es cierta: los datos casi nunca llegan listos para el análisis. La figura 2.1 muestra el flujo de trabajo típico y qué paquete del *tidyverse* te ayuda en cada paso.

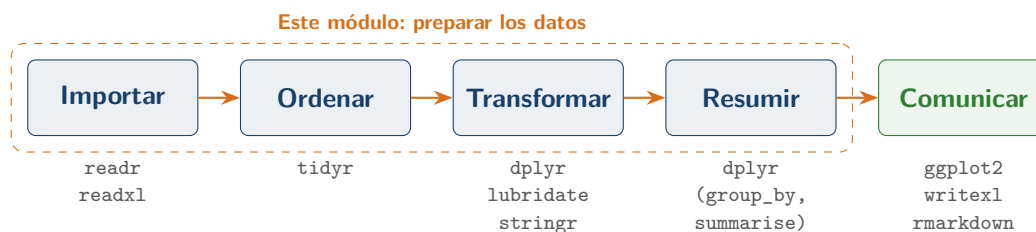


Figura 2.1. Flujo de trabajo de un análisis de datos y los paquetes que usarás en cada paso. Este módulo cubre los cuatro primeros; el Módulo 3 se dedica a comunicar con gráficas.

2.1.1 Datos ordenados (*tidy data*)

Antes de transformar datos conviene saber a qué forma queremos llegar. El estadístico Hadley Wickham, creador de buena parte de los paquetes de este módulo, propuso una forma estándar llamada **datos ordenados** (*tidy data*).

Concepto clave: Las tres reglas de los datos ordenados

Una tabla está *ordenada* cuando cumple tres reglas:

1. **Cada variable es una columna.** Fecha, sucursal, producto y monto son cuatro variables: cuatro columnas.
2. **Cada observación es una fila.** Una venta, un cliente o un mes de una sucursal ocupa una fila.
3. **Cada valor está en su propia celda.** Nada de “CDMX-Centro” en una celda si ciudad y zona son cosas distintas.

Cuando tus datos cumplen estas reglas, todas las funciones del *tidyverse* “encajan”: filtras, agrupas, unes y graficas sin pelearte con la forma de la tabla.

El ejemplo más común de datos *desordenados* en una empresa es el reporte de Excel “bonito”, pensado para leerse, no para analizarse. En la tabla 2.1 el reporte de la izquierda tiene los meses como columnas: la variable *mes* está escondida en los encabezados y la variable *ventas* está repartida en tres columnas. La versión ordenada (derecha) tiene una columna por variable.

Tabla 2.1. El mismo dato en dos formas: un reporte “ancho” típico de Excel (desordenado) y su versión ordenada (*tidy*).

Desordenado (ancho)				Ordenado (largo)		
Sucursal	Ene	Feb	Mar	sucursal	mes	ventas
Centro	12 500	13 200	11 800	Centro	Ene	12 500
Norte	9 800	10 100	11 350	Centro	Feb	13 200
Sur	15 200	14 900	16 050	Centro	Mar	11 800
3 filas; la variable “mes” vive en los encabezados.				Norte	Ene	9 800
				...		
				Sur	Mar	16 050

9 filas; cada fila es una sucursal en un mes.

¿Por qué es mejor la versión larga? Porque preguntas como “¿cuánto se vendió en total en febrero?” o “¿qué mes fue el mejor de cada sucursal?” se resuelven con un filtro o un agrupamiento, sin importar si mañana el reporte trae 3 meses o 36. En la sección 2.9 aprenderás a pasar de una forma a la otra con una sola función. Otros síntomas de datos desordenados que encontrarás en la práctica son:

- Dos datos en una celda: "Guadalajara, Jal." o "2023-Q1".
- Celdas combinadas de Excel: la región aparece solo en la primera fila del bloque y las demás quedan vacías.
- Totales y subtotales mezclados con los datos (la fila “Total general” al final).
- Varias tablas en una misma hoja, o encabezados en dos renglones.

2.1.2 El *tidyverse*: una caja de herramientas coherente

El **tidyverse** es una colección de paquetes de R diseñados para trabajar juntos con datos ordenados. Comparten la misma filosofía: funciones con nombres de verbos, el *data frame* siempre como primer argumento y resultados que también son *data frames*. La tabla 2.2 resume los que usarás en este módulo.

Tabla 2.2. Paquetes del *tidyverse* que usarás en este módulo.

Paquete	¿Para qué sirve?	Funciones estrella
readr	Leer y escribir archivos de texto (CSV)	<code>read_csv()</code> , <code>write_csv()</code>
tibble	La versión moderna del <i>data frame</i>	<code>tibble()</code> , <code>glimpse()</code>
dplyr	Transformar: filtrar, calcular, resumir, unir	<code>filter()</code> , <code>mutate()</code> , <code>left_join()</code>
tidyr	Cambiar la forma de las tablas y tratar vacíos	<code>pivot_longer()</code> , <code>fill()</code>
lubridate	Fechas y horas	<code>ymd()</code> , <code>month()</code> , <code>floor_date()</code>
stringr	Texto (cadenas de caracteres)	<code>str_detect()</code> , <code>str_squish()</code>
forcats	Variables categóricas (factores)	<code>fct_reorder()</code> , <code>fct_lump()</code>
ggplot2	Gráficas (Módulo 3)	<code>ggplot()</code> , <code>geom_col()</code>
purrr	Repetir una operación sobre muchos elementos	<code>map()</code>

Si todavía no lo tienes, instala el *tidyverse* y el paquete **writexl** (que usaremos para exportar a Excel). Solo se hace una vez por computadora:

```
1 # Se ejecuta UNA sola vez (tarda unos minutos)
2 install.packages(c("tidyverse", "writexl"))
```

Y al inicio de cada script cargas los paquetes (esto sí, cada vez):

```
1 # Carga dplyr, tidyr, readr, stringr, lubridate, ggplot2, etc.
2 library(tidyverse)
3 # writexl no es parte del tidyverse: se carga aparte
4 library(writexl)
```

Al cargar el *tidyverse* verás un mensaje de bienvenida con dos partes. La primera, *Attaching core tidyverse packages*, lista los nueve paquetes que se cargaron y sus versiones. La segunda, *Conflicts*, avisa que algunas funciones “tapan” (*mask*) a otras con el mismo nombre; por ejemplo, **dplyr::filter()** **masks stats::filter()** significa que, a partir de ahora, cuando escribas **filter()** R usará la versión de **dplyr** y no la del paquete base **stats**. No es un error: es un aviso, y es justo lo que queremos. (En la sección 2.13 verás qué pasa cuando olvidas cargar el paquete.)



Consejo

La notación **paquete::funcion()** llama a una función de un paquete específico sin ambigüedad, aunque haya otra con el mismo nombre. Por ejemplo, **dplyr::filter()** siempre es el filtro de **dplyr**. La verás en foros y en código profesional; también sirve para usar una función de un paquete instalado sin cargarlo con **library()**.

2.2 Importar datos con readr

2.2.1 Tu primera lectura con read_csv()

Un **CSV** (*comma-separated values*) es un archivo de texto donde cada renglón es una fila y las columnas se separan con comas. Es el formato más común para intercambiar datos entre sistemas: casi cualquier ERP, CRM o base de datos puede exportar a CSV. La función **read_csv()** de **readr** lo lee y devuelve un *tibble*.

Recuerda que todas las rutas del libro son relativas a la carpeta del curso (la que contiene **datasets/**). Si en RStudio abriste el proyecto del curso, ya estás ahí; si no, revisa el Módulo 1.

```
1 # Leer el archivo de ventas y guardarlo en el objeto "ventas"
2 ventas <- read_csv("datasets/ventas_retail.csv")
```

En la consola aparece este mensaje:

```
@@@SALIDA@@@
```

Este mensaje, la **especificación de columnas**, no es un error: es **readr** contándote qué hizo. Léelo de arriba abajo:

- Rows: 365 Columns: 14: se leyeron 365 filas (una venta por día de 2023) y 14 columnas. Compara siempre estos números con lo que esperabas; es tu primera prueba de calidad.
- Delimiter: ",": el separador que detectó.
- chr (3): tres columnas de texto (*character*).
- dbl (10): diez columnas numéricas (*double*, números con decimales). Los identificadores como **producto_id** también se leyeron como números.
- date (1): una columna de fecha. **readr** reconoció el formato 2023-01-01 y lo convirtió automáticamente.

readr *adivina* el tipo de cada columna mirando las primeras 1000 filas. Casi siempre acierta, pero no siempre (lo verás en un momento). Cuando ya conoces el archivo, puedes silenciar el mensaje con **show_col_types = FALSE**, que es lo que haremos en el resto del libro.

2.2.2 Mirar los datos: *tibble* y *glimpse()*

Escribe el nombre del objeto para verlo:

```
1 ventas
```

```
# A tibble: 365 x 14
  fecha      dia_semana producto_id cliente_id cantidad
<date>    <chr>          <dbl>     <dbl>     <dbl>
1 2023-01-01 domingo           31         94         10
2 2023-01-02 lunes            15        169          8
3 2023-01-03 martes           14        136          3
4 2023-01-04 miércoles         3        167          2
5 2023-01-05 jueves           42         81         10
6 2023-01-06 viernes          50        191          8
7 2023-01-07 sábado           43         82         10
8 2023-01-08 domingo          37        106          7
9 2023-01-09 lunes            14        119          7
10 2023-01-10 martes           25         11          6
# i 355 more rows
# i 9 more variables: precio_unitario <dbl>, descuento_pct <dbl>,
#   tienda_id <dbl>, vendedor_id <dbl>, subtotal <dbl>,
#   descuento_monto <dbl>, total <dbl>, mes <chr>, trimestre <chr>
```

Un **tibble** es un *data frame* con mejores modales. Fíjate en lo que hace al imprimirse:

- La primera línea dice qué es y su tamaño: A tibble: 365 x 14.
- Debajo de cada nombre de columna aparece su tipo entre < >: <date>, <chr>, <dbl>.
- Muestra solo 10 filas y las columnas que caben en la pantalla; al final te dice cuántas filas y columnas quedaron sin mostrar.
- Los números se muestran con pocas cifras significativas (por eso verás 3508. en vez de 3507.9). Es solo la impresión: el valor guardado conserva todos sus decimales.

Un *data frame* clásico, en cambio, imprime *todas* las filas y parte las columnas en bloques. Con 365 filas eso inunda la consola; por eso aquí mostramos solo tres:

```
1 # Convertimos a data.frame clásico solo para comparar la impresión
2 as.data.frame(head(ventas, 3))
```

	fecha	dia_semana	producto_id	cliente_id	cantidad
1	2023-01-01	domingo	31	94	10
2	2023-01-02	lunes	15	169	8
3	2023-01-03	martes	14	136	3

	precio_unitario	descuento_pct	tienda_id	vendedor_id	subtotal
1	350.79	5	6	5	3507.90
2	282.75	0	5	24	2262.00
3	296.51	10	2	25	889.53

	descuento_monto	total	mes	trimestre
1	175.395	3332.505	2023-01	Q1
2	0.000	2262.000	2023-01	Q1
3	88.953	800.577	2023-01	Q1

Para ver *todas* las columnas de un tibble de un vistazo, usa `glimpse()`: pone las columnas como renglones, con su tipo y sus primeros valores. Es la función que más vas a usar para “conocer” una tabla nueva.

```
1 glimpse(ventas)
```

```
Rows: 365
Columns: 14
$ fecha           <date> 2023-01-01, 2023-01-02, 2023-01-03, 2023-01~
$ dia_semana      <chr> "domingo", "lunes", "martes", "miércoles", "~
$ producto_id     <dbl> 31, 15, 14, 3, 42, 50, 43, 37, 14, 25, 26, 2~
$ cliente_id      <dbl> 94, 169, 136, 167, 81, 191, 82, 106, 119, 11~
$ cantidad        <dbl> 10, 8, 3, 2, 10, 8, 10, 7, 7, 6, 1, 4, 1, 5,~
$ precio_unitario <dbl> 350.79, 282.75, 296.51, 222.48, 218.83, 302.~
$ descuento_pct   <dbl> 5, 0, 10, 0, 0, 0, 15, 0, 0, 15, 0, 15, 0, 0~
$ tienda_id       <dbl> 6, 5, 2, 7, 3, 3, 4, 1, 6, 5, 2, 7, 10, 1, 5~
$ vendedor_id     <dbl> 5, 24, 25, 18, 8, 19, 3, 5, 8, 12, 5, 23, 25~
$ subtotal        <dbl> 3507.90, 2262.00, 889.53, 444.96, 2188.30, 2~
```

```
$ descuento_monto <dbl> 175.395, 0.000, 88.953, 0.000, 0.000, 0.000, ~
$ total           <dbl> 3332.505, 2262.000, 800.577, 444.960, 2188.3~
$ mes            <chr> "2023-01", "2023-01", "2023-01", "2023-01", ~
$ trimestre      <chr> "Q1", "Q1", "Q1", "Q1", "Q1", "Q1", "Q1", "Q~
```

Con esto ya sabes qué contiene cada venta: la **fecha**, qué producto (**producto_id**), a qué cliente, cuántas unidades, a qué precio, con qué descuento, en qué tienda y qué vendedor la hizo. Las columnas **subtotal**, **descuento_monto** y **total** son cálculos ya hechos, y **mes** y **trimestre** son etiquetas de texto.

Consejo

Si quieres ver más filas de un tibble usa `print(ventas, n = 30)`; para ver todas las columnas, `print(ventas, width = Inf)`. En RStudio también puedes abrir la tabla como hoja de cálculo con `View(ventas)` o haciendo clic en su nombre en el panel **Environment**.

2.2.3 Controlar los tipos con `col_types`

A veces la adivinanza de **readr** no es lo que necesitas. Un caso típico son los identificadores: **cliente_id** es una etiqueta, no una cantidad (sumar identificadores no tiene sentido). Con el argumento `col_types` le dices a **readr** exactamente cómo leer cada columna; las que no menciones se siguen adivinando.

```
1 # Leer los identificadores como texto y la cantidad como entero
2 ventas_tipos <- read_csv(
3   "datasets/ventas_retail.csv",
4   col_types = cols(
5     producto_id = col_character(), # texto
6     cliente_id  = col_character(), # texto
7     cantidad    = col_integer(),   # número entero
8     .default     = col_guess()     # el resto: que readr adivine
9   )
10 )
11 # Revisamos solo las columnas que nos interesan
12 glimpse(select(ventas_tipos, producto_id, cliente_id, cantidad))
```

```
Rows: 365
Columns: 3
$ producto_id <chr> "31", "15", "14", "3", "42", "50", "43", "37", "~
$ cliente_id  <chr> "94", "169", "136", "167", "81", "191", "82", "1~
$ cantidad    <int> 10, 8, 3, 2, 10, 8, 10, 7, 7, 6, 1, 4, 1, 5, 10,~
```

Ahora **producto_id** es `<chr>` y **cantidad** es `<int>` (entero). Las funciones más usadas en `cols()` son `col_character()`, `col_double()`, `col_integer()`, `col_logical()`, `col_date(format = "%d/%m/%Y")` y `col_skip()` (para no leer una columna). Si solo necesitas algunas columnas

de un archivo grande, `col_select` lee únicamente esas: `read_csv(ruta, col_select = c(fecha, total))`.

▲ Cuidado

Para unir tablas (sección 2.8), la llave debe tener el *mismo tipo* en ambas. Si lees `producto_id` como texto en ventas y como número en productos, el `join` fallará con un error `Can't join ... due to incompatible types`. En este módulo dejaremos los identificadores como números en todas las tablas para evitarlo.

2.2.4 Cuando algo sale mal: `problems()`

Imagina que la sucursal Norte te manda su inventario y alguien escribió “N/D” y “sin dato” en lugar de un número. Para simular ese archivo, lo creamos nosotros mismos con `write_lines()` (cada elemento es un renglón del archivo) en una carpeta `datos/`:

```
1 # Crear la carpeta datos/ (si ya existe, no pasa nada)
2 dir.create("datos", showWarnings = FALSE)
3 # Escribir un CSV pequeño con dos valores "raros" en existencias
4 write_lines(c("sucursal,producto,existencias",
5               "Centro,P01,120",
6               "Norte,P01,N/D",
7               "Sur,P02,85",
8               "Este,P02,sin dato"),
9             "datos/inventario.csv")
10 # Leerlo exigiendo que existencias sea numérica
11 inventario <- read_csv("datos/inventario.csv",
12                       col_types = cols(existencias = col_double()))
```

Warning: One or more parsing issues, call `problems()` on your data frame for details, e.g.:

```
dat <- vroom(...)
problems(dat)
```

readr no se detiene: lee lo que puede, pone **NA** (valor faltante) donde no pudo convertir y te avisa. La función `problems()` te dice exactamente dónde está cada problema (su columna `file` muestra la ruta completa del archivo; aquí la omitimos para ahorrar espacio):

```
1 problems(inventario) %>%
2   select(row, col, expected, actual)
```

```
# A tibble: 2 x 4
   row  col expected actual
<int> <int> <chr>    <chr>
```



```
1      3      3 a double N/D
2      5      3 a double sin dato
```

Las filas 3 y 5 del archivo (contando el encabezado como fila 1), en la columna 3, esperaban un número y encontraron N/D y *sin dato*. Como sabemos que esos textos significan “no hay dato”, la solución limpia es decirselo a **readr** con el argumento **na**:

```
1 inventario <- read_csv("datos/inventario.csv",
2                        na = c("", "NA", "N/D", "sin dato"),
3                        show_col_types = FALSE)
4 inventario
```

```
# A tibble: 4 x 3
  sucursal producto existencias
  <chr>    <chr>         <dbl>
1 Centro  P01             120
2 Norte   P01              NA
3 Sur     P02             85
4 Este    P02              NA
```

Ya no hay advertencia y **existencias** es numérica (<dbl>). Los faltantes son explícitos: nadie los confundirá con un cero.

En la empresa: CSV exportados desde Excel en español

Si tu empresa exporta los CSV desde Excel con configuración regional de México o España, puede que encuentres dos sorpresas. La primera: columnas separadas por punto y coma (;) y decimales con coma (1234,50); para eso existe **read_csv2()**. La segunda: acentos que se ven como Ã© o Ã±, porque el archivo no está en UTF-8; se corrige indicando la codificación:

```
1 reporte <- read_csv2("datos/reporte_excel.csv")      # ; y decimal ,
2 clientes_erp <- read_csv("datos/clientes_erp.csv",
3                          locale = locale(encoding = "latin1"))
```

2.2.5 Cargar todas las tablas del curso

Ahora sí, carguemos el resto de las tablas que usaremos en el módulo. Todas se leen igual; solo cambia el nombre del archivo:

```
1 productos <- read_csv("datasets/productos.csv",
2                       show_col_types = FALSE)
3 clientes <- read_csv("datasets/clientes.csv",
4                      show_col_types = FALSE)
5 tiendas <- read_csv("datasets/tiendas.csv",
```

```

6           show_col_types = FALSE)
7 empleados <- read_csv("datasets/empleados.csv",
8           show_col_types = FALSE)
9 transacciones <- read_csv("datasets/transacciones.csv",
10          show_col_types = FALSE)
11
12 # Un pequeño inventario de lo que cargamos
13 tibble(
14   tabla      = c("ventas", "productos", "clientes", "tiendas",
15   "empleados", "transacciones"),
16   filas      = c(nrow(ventas), nrow(productos), nrow(clientes),
17   nrow(tiendas), nrow(empleados), nrow(transacciones)),
18   columnas   = c(ncol(ventas), ncol(productos), ncol(clientes),
19   ncol(tiendas), ncol(empleados), ncol(transacciones))
20 )

```

```

# A tibble: 6 x 3
  tabla      filas columnas
  <chr>      <int>   <int>
1 ventas      365     14
2 productos    50     13
3 clientes    200     10
4 tiendas     10      8
5 empleados   100     12
6 transacciones 1000    14

```

En BI a esta distinción se le da nombre: **ventas** y **transacciones** son **tablas de hechos** (registran eventos: cada fila es una venta) y **productos**, **clientes**, **tiendas** y **empleados** son **tablas de dimensiones** (describen a los protagonistas de esos eventos). Las conectarás en la sección 2.8.

Hazlo tú

Lee `datasets/transacciones.csv` sin `show_col_types = FALSE` y revisa la especificación de columnas. ¿Qué tipo le asignó `readr` a la columna `hora`? Luego usa `glimpse()` sobre `clientes` y responde: ¿de qué tipo es `es_premium`?

2.3 El pipe: encadenar pasos como una receta

Supón que quieres ver las ventas de la tienda 1 en diciembre, solo con la fecha, el producto, la cantidad y el total, de la más grande a la más chica. Son tres pasos: filtrar, seleccionar columnas y ordenar. Hay tres maneras de escribirlo.

Opción 1: funciones anidadas. Cada función envuelve a la anterior. Funciona, pero se lee de adentro hacia afuera y los argumentos quedan lejos de su función:

```

1 arrange(
2   select(
3     filter(ventas, tienda_id == 1, mes == "2023-12"),
4     fecha, producto_id, cantidad, total
5   ),
6   desc(total)
7 )

```

Opción 2: objetos intermedios. Se lee mejor, pero llenas tu ambiente de objetos (`paso1`, `paso2`...) que no vuelves a usar, y es fácil equivocarte de nombre:

```

1 paso1 <- filter(ventas, tienda_id == 1, mes == "2023-12")
2 paso2 <- select(paso1, fecha, producto_id, cantidad, total)
3 paso3 <- arrange(paso2, desc(total))

```

Opción 3: el pipe. El operador `%>%`, llamado **pipe** (tubería), toma lo que está a su izquierda y lo pasa como *primer argumento* a la función de su derecha. Así, `x %>% f(y)` es lo mismo que `f(x, y)`:

```

1 ventas %>%                                # toma las ventas,
2   filter(tienda_id == 1, mes == "2023-12") %>% # y luego filtra,
3   select(fecha, producto_id, cantidad, total) %>% # y luego elige,
4   arrange(desc(total))                       # y luego ordena

```

```

# A tibble: 5 x 4
  fecha      producto_id cantidad  total
  <date>          <dbl>    <dbl> <dbl>
1 2023-12-18         18         9 2646.
2 2023-12-26         26         5 1847.
3 2023-12-12         35         3  441.
4 2023-12-24         42         1  349.
5 2023-12-25         32         1   54.0

```

Las tres opciones dan exactamente el mismo resultado, pero la tercera se lee de arriba abajo como una receta. Lee cada `%>%` como “y luego”: “toma las ventas, y luego filtra la tienda 1 en diciembre, y luego elige cuatro columnas, y luego ordena por total descendente”. La tienda 1 tuvo 5 ventas en diciembre y la mayor fue de \$2,646 el 18 de diciembre.



Consejo

El atajo de teclado para escribir el pipe en RStudio es `Ctrl` + `Shift` + `M` (en Mac, `Cmd` + `Shift` + `M`). Úsalo desde hoy: lo vas a teclear cientos de veces al día. Escribe un paso por línea y deja el `%>%` al final de la línea, nunca al principio de la siguiente.

2.3.1 El pipe nativo |>

Desde la versión 4.1, R trae su propio pipe, el **pipe nativo** |>, que no necesita cargar ningún paquete. Para el trabajo diario ambos hacen lo mismo:

```
1 # Mismo resultado que con %>%: contar las ventas de diciembre
2 ventas |>
3   filter(mes == "2023-12") |>
4   nrow()
```

```
[1] 31
```

Las diferencias que te pueden afectar son pocas:

- |> siempre exige paréntesis: `ventas |> nrow()` funciona pero `ventas |> nrow` da error, mientras que `ventas %>% nrow` sí funciona.
- Para mandar el dato a un argumento que no es el primero, %>% usa un punto (`data = .`) y |> usa un guion bajo con nombre (`data = _`, desde R 4.2).
- %>% viene del paquete **magrittr** y se carga con el *tidyverse*; |> está siempre disponible.

En este libro usamos %>% porque es el que encontrarás en la mayoría de los ejemplos en internet, pero puedes usar el que prefieras. Si quieres que el atajo **Ctrl** + **Shift** + **M** escriba |>, activa la casilla *Use native pipe operator* en **Tools > Global Options > Code**.

Hazlo tú

Reescribe con pipe la siguiente instrucción anidada y léela en voz alta usando “y luego”:

```
1 head(arrange(filter(productos, categoria == "Ropa"), precio_catalogo), 3)
```

2.4 Los verbos de dplyr

dplyr está construido alrededor de unos cuantos **verbos**: funciones cuyo nombre dice lo que hacen con la tabla. Todos siguen las mismas reglas, y una vez que las entiendes, aprender un verbo nuevo es trivial:

1. El primer argumento siempre es un *data frame* (por eso encajan tan bien con el pipe).
2. Los demás argumentos usan los nombres de las columnas *sin comillas* y sin **\$**: escribes `total`, no `ventas$total`.
3. El resultado siempre es un *data frame* nuevo. La tabla original *no se modifica*; si quieres conservar el resultado, guárdalo con `<-`.

Tabla 2.3. Los seis verbos fundamentales de **dplyr**.

Verbo	Qué hace	Actúa sobre
<code>filter()</code>	Se queda con las filas que cumplen una condición	Filas
<code>select()</code>	Elige, quita o reordena columnas	Columnas
<code>mutate()</code>	Crea columnas nuevas o modifica las existentes	Columnas
<code>arrange()</code>	Ordena las filas	Filas
<code>summarise()</code>	Resume muchas filas en una (totales, promedios...)	Filas
<code>group_by()</code>	Divide la tabla en grupos para que los demás verbos trabajen grupo por grupo	Grupos

2.4.1 `filter()`: quedarte con las filas que importan

`filter()` recibe una o más condiciones lógicas y conserva solo las filas donde la condición es **TRUE**. Las condiciones se construyen con los operadores de la tabla 2.4.

Tabla 2.4. Operadores para construir condiciones en `filter()`.

Operador	Significado	Ejemplo
<code>==</code>	Igual a (¡dos signos!)	<code>tienda_id == 3</code>
<code>!=</code>	Distinto de	<code>trimestre != "Q4"</code>
<code>>, >=</code>	Mayor, mayor o igual	<code>total >= 1000</code>
<code><, <=</code>	Menor, menor o igual	<code>cantidad < 3</code>
<code>&</code> o coma	Y: se deben cumplir ambas	<code>tienda_id == 1, cantidad > 5</code>
<code> </code>	O: basta con que se cumpla una	<code>descuento_pct == 20 cantidad == 10</code>
<code>!</code>	No (niega la condición)	<code>!activo</code>
<code>%in%</code>	Pertenece a una lista de valores	<code>tienda_id %in% c(1, 4)</code>
<code>between()</code>	Dentro de un rango (incluye extremos)	<code>between(total, 500, 900)</code>
<code>is.na()</code>	Es un valor faltante	<code>is.na(email)</code>

Ejemplo práctico: Ventas grandes

El gerente de riesgos quiere revisar las ventas de más de \$4,500, porque un ticket tan alto es poco común. ¿Cuántas son y de qué tiendas?

```
1 ventas %>%
2   filter(total > 4500) %>%           # condición numérica
3   select(fecha, tienda_id, cantidad, precio_unitario, total)
```

```
# A tibble: 4 x 5
  fecha      tienda_id cantidad precio_unitario total
  <date>      <dbl>    <dbl>         <dbl> <dbl>
1 2023-02-15         8        10          488.  4638.
2 2023-04-21         2        10          492.  4925.
3 2023-07-24         8        10          495.  4950.
4 2023-09-04         6        10          487.  4875.
```

Solo 4 ventas del año superan los \$4,500. Fíjate en que las cuatro son de 10 unidades (el máximo posible) con precios unitarios cercanos a \$490: no parecen errores sino compras al mayoreo de productos caros.

Ejemplo práctico: Varias condiciones a la vez (Y)

¿Qué ventas hizo la tienda 9 (la sucursal *Premium*) en el primer trimestre con algún descuento?

Separar condiciones con comas equivale a unir las con **&**: se deben cumplir *todas*.

```
1 ventas %>%
2   filter(tienda_id == 9,           # tienda Premium
3         trimestre == "Q1",       # y primer trimestre
4         descuento_pct > 0) %>%   # y con descuento
5   select.fecha, producto_id, descuento_pct, total)
```

```
# A tibble: 5 x 4
  fecha      producto_id descuento_pct total
  <date>      <dbl>      <dbl> <dbl>
1 2023-01-29         39          10 3650.
2 2023-02-14         25           5 1468.
3 2023-02-27         15           5  155.
4 2023-03-16         49           5 1290.
5 2023-03-18          4          10 1486.
```

Ejemplo práctico: Una u otra condición (O) y listas de valores

Mercadotecnia quiere las ventas “extremas”: las que tuvieron el descuento máximo (20%) o se llevaron 10 unidades. Y, aparte, las ventas de las cuatro tiendas de la CDMX (tiendas 1, 4, 6 y 9).

```
1 # O: basta con que se cumpla una de las dos condiciones
2 ventas %>%
3   filter(descuento_pct == 20 | cantidad == 10) %>%
4   nrow()
5
6 # %in%: la tienda está en la lista (más corto que cuatro "|")
7 ventas_cdmx <- ventas %>%
8   filter(tienda_id %in% c(1, 4, 6, 9))
9   nrow(ventas_cdmx)
```

```
[1] 54
[1] 135
```

54 ventas cumplen al menos una de las dos condiciones “extremas”, y 135 ventas (más de un tercio

del total) se hicieron en tiendas de la CDMX. Usa `%in%` siempre que compares contra una lista: es más legible y evitas errores como escribir `tienda_id == 1 | 4`, que *no* significa lo que parece.

Ejemplo práctico: Rangos de montos y de fechas

¿Cuántas ventas están en el rango medio, entre \$1,000 y \$2,000? ¿Qué se vendió durante El Buen Fin 2023 (del 17 al 20 de noviembre)?

```
1 # between() incluye ambos extremos: 1000 <= total <= 2000
2 ventas %>%
3   filter(between(total, 1000, 2000)) %>%
4   nrow()
5
6 # Con fechas: compara contra objetos Date, no contra texto
7 ventas %>%
8   filter(fecha >= as.Date("2023-11-17"),
9          fecha <= as.Date("2023-11-20")) %>%
10  select(fecha, dia_semana, producto_id, cantidad, total)
```

```
[1] 96
# A tibble: 4 x 5
  fecha      dia_semana producto_id cantidad total
<date>     <chr>          <dbl>     <dbl> <dbl>
1 2023-11-17 viernes           39         7 1731.
2 2023-11-18 sábado            30         7 2939.
3 2023-11-19 domingo           31         5  142.
4 2023-11-20 lunes            30         8 3358.
```

96 ventas caen entre \$1,000 y \$2,000. Durante El Buen Fin hubo 4 ventas (recuerda que este archivo tiene una venta por día). Como la columna `fecha` es de tipo `<date>`, la comparamos con `as.Date("2023-11-17")`; así R compara fechas reales y no texto.

Ejemplo práctico: Negaciones y valores faltantes

¿Qué productos del catálogo están inactivos? Y en una tabla de metas de vendedores donde algunas metas aún no se capturan, ¿quiénes no tienen meta?

La columna `activo` ya es lógica (`TRUE/FALSE`), así que basta con negarla con `!`:

```
1 productos %>%
2   filter(!activo) %>%           # los que NO están activos
3   select(producto_id, nombre_producto, categoria, stock_actual)
```

```
# A tibble: 7 x 4
  producto_id nombre_producto categoria stock_actual
```

	<dbl>	<chr>	<chr>	<dbl>
1	7	Producto 07	Deportes	123
2	13	Producto 13	Ropa	177
3	15	Producto 15	Mascotas	455
4	25	Producto 25	Juguetes	429
5	38	Producto 38	Alimentos	141
6	39	Producto 39	Alimentos	8
7	46	Producto 46	Automotriz	118

Hay 7 productos inactivos, y casi todos tienen todavía más de 100 unidades en almacén (el *Producto 15*, 455): inventario inmovilizado, un buen tema para la junta de compras.

Para ilustrar los faltantes creamos una tabla pequeña con `tibble()`. Cada argumento es una columna:

```
1 metas <- tibble(
2   vendedor = c("Ana", "Luis", "Rosa", "Jorge", "Elena"),
3   meta      = c(50000, NA, 42000, NA, 61000) # NA = aún no capturada
4 )
5 # filter() descarta en silencio las filas donde la condición es NA
6 metas %>% filter(meta > 45000)
7 # Para encontrar los faltantes usa is.na(), nunca meta == NA
8 metas %>% filter(is.na(meta))
```

```
# A tibble: 2 x 2
  vendedor meta
  <chr>    <dbl>
1 Ana      50000
2 Elena    61000
# A tibble: 2 x 2
  vendedor meta
  <chr>    <dbl>
1 Luis      NA
2 Jorge     NA
```

▲ Cuidado

`meta == NA` no funciona: cualquier comparación con `NA` da `NA` (“no sé”), y `filter()` descarta esas filas. Por eso Luis y Jorge no aparecieron en el primer filtro *ni aparecerían* con `meta <= 45000`. Si necesitas conservarlos, sé explícito: `filter(meta > 45000 | is.na(meta))`.

Ejemplo práctico: Filtrar por un patrón de texto

En `transacciones`, ¿cuántas se pagaron con algún tipo de tarjeta?

La función `str_detect()` de `stringr` devuelve `TRUE` si el texto contiene el patrón buscado. La veremos a fondo en la sección 2.7.


```

1 transacciones %>%
2   filter(str_detect(metodo_pago, "Tarjeta")) %>% # contiene "Tarjeta"
3   count(metodo_pago)                          # cuántas de cada una

```

```

# A tibble: 2 x 2
  metodo_pago      n
  <chr>         <int>
1 Tarjeta Crédito  350
2 Tarjeta Débito   233

```

583 de las 1000 transacciones (350 + 233) se pagaron con tarjeta. La función `count()` cuenta filas por categoría; la estudiaremos en la sección 2.4.5.

Hazlo tú

Con `productos`: (a) filtra los de la categoría “Electrónica” con precio de catálogo mayor a \$400; (b) filtra los que están en “Stock Bajo” o “Sin Stock” usando `%in%`; (c) cuenta cuántos clientes de `clientes` son premium y viven en Monterrey.

2.4.2 `select()`: elegir, quitar y renombrar columnas

Las tablas reales tienen decenas de columnas y casi nunca las necesitas todas. `select()` se queda con las que le pidas, en el orden en que las pidas.

Ejemplo práctico: Una lista de precios para el equipo de ventas

El equipo de piso solo necesita el nombre, la categoría y el precio de cada producto; el costo es confidencial y no debe aparecer.

```

1 lista_precios <- productos %>%
2   select(nombre_producto, categoria, precio_catologo) # por nombre
3   head(lista_precios, 4)

```

```

# A tibble: 4 x 3
  nombre_producto categoria precio_catologo
  <chr>          <chr>         <dbl>
1 Producto 01    Juguetes         300.
2 Producto 02    Belleza         313.
3 Producto 03    Automotriz       400.
4 Producto 04    Juguetes         744.

```

También puedes seleccionar un *rango* de columnas consecutivas con `:`, o *quitar* columnas con un signo menos:

```

1 # Rango: de fecha hasta cantidad (columnas consecutivas)
2 ventas %>% select(fecha:cantidad) %>% names()
3
4 # Quitar: todo menos las etiquetas de texto que no usaremos
5 ventas %>% select(-dia_semana, -mes, -trimestre) %>% names()

```

```

[1] "fecha"      "dia_semana" "producto_id" "cliente_id"
[5] "cantidad"
[1] "fecha"      "producto_id" "cliente_id"
[4] "cantidad"   "precio_unitario" "descuento_pct"
[7] "tienda_id"  "vendedor_id"  "subtotal"
[10] "descuento_monto" "total"

```

La función `names()` devuelve los nombres de las columnas; aquí la usamos para mostrar el resultado de forma compacta.

Funciones auxiliares de selección

Cuando las columnas siguen una convención de nombres (como `_id` al final de las llaves), las **funciones auxiliares** (*helpers*) te ahorran escribirlas una por una:

```

1 # Todas las llaves: terminan en "_id"
2 ventas %>% select(ends_with("_id")) %>% names()
3
4 # Columnas de inventario: empiezan con "stock"
5 productos %>% select(producto_id, starts_with("stock")) %>% head(3)
6
7 # Columnas que contienen "fecha" en cualquier parte del nombre
8 clientes %>% select(cliente_id, contains("fecha")) %>% head(3)
9
10 # Solo las columnas numéricas (útil antes de calcular resúmenes)
11 productos %>% select(where(is.numeric)) %>% names()

```

```

[1] "producto_id" "cliente_id"  "tienda_id"   "vendedor_id"
# A tibble: 3 x 3
  producto_id stock_actual stock_minimo
      <dbl>      <dbl>      <dbl>
1         1         374         21
2         2         371         46
3         3          22         26
# A tibble: 3 x 2
  cliente_id fecha_registro
      <dbl>   <date>
1         1 2023-10-12
2         2 2020-01-30

```

```
3          3 2020-10-17
[1] "producto_id"      "precio_catalogo" "costo"
[4] "stock_actual"     "stock_minimo"    "margen_pct"
```

`where()` recibe una función que devuelve `TRUE` o `FALSE` para cada columna: `is.numeric`, `is.character`, `is.logical`... Otras auxiliares útiles son `everything()` (“todas las demás”), `matches()` (patrón de texto) y `any_of()` (una lista de nombres que pueden no existir).

Renombrar y reacomodar: `rename()` y `relocate()`

`rename()` cambia nombres sin quitar columnas; la sintaxis es `nuevo = viejo`. `relocate()` mueve columnas de lugar sin quitar ninguna.

```
1 tiendas %>%
2   rename(tienda = nombre_tienda, m2 = tamano_m2) %>% # nuevo = viejo
3   select(tienda_id, tienda, ciudad, m2) %>%
4   head(3)
5
6 # Poner el total justo después de la fecha para leerlo más fácil
7 ventas %>%
8   relocate(total, .after = fecha) %>%
9   head(3)
```

```
# A tibble: 3 x 4
  tienda_id tienda          ciudad      m2
  <dbl> <chr>          <chr>    <dbl>
1      1 Sucursal Centro CDMX        500
2      2 Sucursal Norte Guadalajara 450
3      3 Sucursal Sur Monterrey 600
# A tibble: 3 x 14
  fecha      total dia_semana producto_id cliente_id cantidad
  <date>    <dbl> <chr>          <dbl>    <dbl>    <dbl>
1 2023-01-01 3333. domingo          31        94        10
2 2023-01-02 2262  lunes           15       169         8
3 2023-01-03 801. martes           14       136         3
# i 8 more variables: precio_unitario <dbl>, descuento_pct <dbl>,
#   tienda_id <dbl>, vendedor_id <dbl>, subtotal <dbl>,
#   descuento_monto <dbl>, mes <chr>, trimestre <chr>
```

Consejo

Puedes renombrar dentro de `select()`: `select(tienda = nombre_tienda, ciudad)` elige y renombra en un paso. Y para mandar una columna al principio sin perder las demás, usa `select(total, everything())` o `relocate(total)`.

Hazlo tú

De `empleados`, arma una tabla con `empleado_id`, el nombre renombrado como `vendedor`, y todas las columnas que empiecen con “salario”. Después usa `select(where(is.character))` para ver qué columnas de texto tiene la tabla.

2.4.3 mutate(): crear columnas calculadas

`mutate()` agrega columnas nuevas (o reemplaza existentes) calculadas a partir de las demás. Es el equivalente a escribir una fórmula en una columna de Excel y arrastrarla hacia abajo, solo que se aplica a todas las filas de una vez y queda documentada en el código.

Ejemplo práctico: IVA y total con impuestos

Los montos de `ventas` no incluyen IVA. Contabilidad necesita, para cada venta, el IVA (16 %) y el total facturado.

```
1 ventas %>%
2   mutate(
3     iva          = total * 0.16,      # 16% sobre el total
4     total_con_iva = total + iva      # usa la columna recién creada
5   ) %>%
6   select(fecha, subtotal, descuento_monto, total, iva, total_con_iva) %>%
7   head(4)
```

```
# A tibble: 4 x 6
  fecha      subtotal descuento_monto total   iva total_con_iva
<date>      <dbl>         <dbl> <dbl> <dbl>      <dbl>
1 2023-01-01   3508.           175. 3333.  533.      3866.
2 2023-01-02   2262             0 2262  362.      2624.
3 2023-01-03    890.           89.0 801.  128.      929.
4 2023-01-04    445.             0  445.  71.2     516.
```

Observa dos cosas. Dentro del mismo `mutate()` puedes usar una columna que acabas de crear (`total_con_iva` usa `iva`). Y la primera venta: subtotal de \$3,507.90 menos \$175.40 de descuento da \$3,332.50; con IVA, \$3,865.71.

Ejemplo práctico: ¿Qué tan rentable es cada producto?

Con el catálogo, calcula el margen bruto en pesos y en porcentaje, y ordena del menos al más rentable.

```
1 productos %>%
2   mutate(
3     margen_pesos = precio_catalogo - costo,
```

```

4     margen_calc = round(margen_pesos / precio_catalogo * 100, 1)
5   ) %>%
6   select(nombre_producto, precio_catalogo, costo,
7         margen_pesos, margen_calc) %>%
8   arrange(margen_calc) %>%           # del menor al mayor margen
9   head(5)

```

```

# A tibble: 5 x 5
  nombre_producto precio_catalogo costo margen_pesos margen_calc
  <chr>              <dbl> <dbl>      <dbl>      <dbl>
1 Producto 05         25.5  355.      -329.      -1292.
2 Producto 46         64.5  178.      -113.      -175.
3 Producto 32        143.   210.      -67.0      -46.7
4 Producto 26        115.   159.      -44.2      -38.3
5 Producto 29        298.   392.      -93.9      -31.5

```

En la empresa: Tu primer hallazgo de calidad de datos

¡Hay productos que cuestan más de lo que se venden! El *Producto 05* tiene un precio de catálogo de \$25.47 y un costo de \$354.63: un margen de -1292% . Puede ser un error de captura (¿el precio debía ser \$254.70?) o un producto que se vende en promoción. Como analista, tu trabajo no es ocultarlo sino *reportarlo*: en la sección 2.11 contaremos cuántos productos están en esta situación.

Condiciones: `if_else()` y `case_when()`

`if_else(condición, valor_si_verdadero, valor_si_falso)` crea una columna con dos posibles valores, como la función `SI()` de Excel:

```

1 ventas %>%
2   mutate(tipo_dia = if_else(dia_semana %in% c("sábado", "domingo"),
3                             "Fin de semana", "Entre semana")) %>%
4   count(tipo_dia)

```

```

# A tibble: 2 x 2
  tipo_dia      n
  <chr>    <int>
1 Entre semana  260
2 Fin de semana  105

```

Cuando hay más de dos opciones, anidar `if_else()` se vuelve ilegible. Para eso existe `case_when()`: una lista de reglas `condición ~ valor` que se evalúan *en orden*; a cada fila se le asigna el valor de la *primera* regla que cumple.

Ejemplo práctico: Segmentar ventas por tamaño de ticket

Mercadotecnia quiere clasificar cada venta en cuatro segmentos: Chico (menos de \$500), Mediano (\$500 a \$1,499), Grande (\$1,500 a \$2,999) y Premium (\$3,000 o más). ¿Cuántas ventas y cuánto dinero hay en cada uno?

```
1 ventas_segmentadas <- ventas %>%
2   mutate(segmento_ticket = case_when(
3     total < 500 ~ "1. Chico",      # se evalúa primero
4     total < 1500 ~ "2. Mediano",  # solo si no fue Chico
5     total < 3000 ~ "3. Grande",
6     .default    = "4. Premium"    # todo lo demás
7   ))
8
9 ventas_segmentadas %>%
10  group_by(segmento_ticket) %>%
11  summarise(ventas = n(), ingresos = sum(total))
```

```
# A tibble: 4 x 3
  segmento_ticket ventas ingresos
  <chr>          <int>    <dbl>
1 1. Chico           98   25588.
2 2. Mediano        129  125261.
3 3. Grande         101  227370.
4 4. Premium         37  135697.
```

Una lectura de negocio: los tickets Premium son solo 37 ventas (una de cada diez), pero generan \$135,697, más de cinco veces lo que dejan los 98 tickets Chicos juntos (\$25,588). El segmento que más dinero aporta es el Grande (\$227,370). La estrategia comercial debe cuidar a los tickets altos. (Adelantamos `group_by()` y `summarise()`, que veremos en detalle muy pronto.)

** Consejo**

Pon un número al inicio de cada etiqueta (“1. Chico”, “2. Mediano”...) y los grupos aparecerán en orden lógico y no alfabético. El argumento `.default` (dplyr 1.1 o posterior) asigna un valor a todo lo que no cumplió ninguna regla; en código antiguo verás `TRUE ~ "Premium"`, que hace lo mismo.

Varias columnas a la vez: `across()`

Si quieres aplicar la misma operación a varias columnas, no repitas el código: usa `across(columnas, función)`. Las columnas se eligen con la misma sintaxis de `select()`.

```
1 # Redondear a 0 decimales las tres columnas de dinero
```

```

2 ventas %>%
3   mutate(across(c(subtotal, descuento_monto, total), round)) %>%
4   select(fecha, subtotal, descuento_monto, total) %>%
5   head(3)
6
7 # Con una función "al vuelo": ~ .x indica "cada columna"
8 productos %>%
9   mutate(across(c(precio_catalogo, costo), ~ .x * 1.16,
10               .names = "{.col}_con_iva")) %>%
11   select(nombre_producto, ends_with("_con_iva")) %>%
12   head(3)

```

```

# A tibble: 3 x 4
  fecha      subtotal descuento_monto total
<date>      <dbl>      <dbl> <dbl>
1 2023-01-01    3508          175  3333
2 2023-01-02    2262           0  2262
3 2023-01-03     890          89   801
# A tibble: 3 x 3
  nombre_producto precio_catalogo_con_iva costo_con_iva
<chr>                <dbl>      <dbl>
1 Producto 01          348.        338.
2 Producto 02          363.        463.
3 Producto 03          464.        341.

```

La notación `~ .x * 1.16` es una **función anónima**: una fórmula corta donde `.x` representa cada columna. El argumento `.names` controla el nombre de las columnas nuevas (`{.col}` es el nombre original); sin él, `across()` sobrescribe las columnas originales.

Conservar solo lo calculado: `.keep` y `transmute()`

Por defecto, `mutate()` conserva todas las columnas. Con el argumento `.keep` decides qué más se queda: `"used"` (solo las columnas usadas en el cálculo) o `"none"` (solo las nuevas). La antigua función `transmute()` equivale a `.keep = "none"`.

```

1 # Solo las columnas que participaron en el cálculo, más la nueva
2 ventas %>%
3   mutate(precio_net = total / cantidad, .keep = "used") %>%
4   head(3)
5
6 # Solo las nuevas (equivalente a transmute())
7 ventas %>%
8   mutate(anio = str_sub(mes, 1, 4), ticket = round(total),
9         .keep = "none") %>%
10  head(3)

```

```
# A tibble: 3 x 3
  cantidad total precio_net
  <dbl> <dbl> <dbl>
1      10 3333.    333.
2       8 2262.    283.
3       3  801.    267.

# A tibble: 3 x 2
  anio ticket
  <chr> <dbl>
1 2023   3333
2 2023   2262
3 2023    801
```

▲ Cuidado

`mutate()` no modifica `ventas`: devuelve una copia con la columna nueva. Si ejecutas `ventas %>% mutate(iva = total * 0.16)` y luego buscas `ventas$iva`, no existirá. Para conservarla escribe `ventas <- ventas %>% mutate(...)` o, mejor, guarda el resultado en un objeto con otro nombre y deja intacta la tabla original.

🔧 Hazlo tú

En `clientes`, crea una columna `rango_edad` con `case_when()`: “18–29”, “30–44”, “45–59” y “60+”. Cuenta cuántos clientes hay en cada rango. Pista: termina con `count(rango_edad)`.

2.4.4 `arrange()` y la familia `slice_*`: ordenar y quedarse con el top

`arrange()` ordena las filas de menor a mayor (orden ascendente). Para ordenar de mayor a menor, envuelve la columna en `desc()`. Si le das varias columnas, ordena por la primera y usa las siguientes para desempatar.

Ejemplo práctico: Los productos más caros y las tiendas por ciudad

¿Cuáles son los cinco productos con mayor precio de catálogo? ¿Cómo quedan las tiendas ordenadas por ciudad y, dentro de cada ciudad, de la más grande a la más chica?

```
1 productos %>%
2   arrange(desc(precio_catologo)) %>% # de mayor a menor
3   select(nombre_producto, categoria, precio_catologo) %>%
4   head(5)
5
6 tiendas %>%
7   arrange(ciudad, desc(tamano_m2)) %>% # ciudad A-Z; luego m2 desc.
8   select(ciudad, nombre_tienda, tamano_m2)
```



```
# A tibble: 5 x 3
  nombre_producto categoria  precio_catalogo
  <chr>           <chr>          <dbl>
1 Producto 28     Mascotas         774.
2 Producto 47     Automotriz        766.
3 Producto 41     Electrónica       757.
4 Producto 04     Juguetes          744.
5 Producto 20     Belleza           742.

# A tibble: 10 x 3
  ciudad      nombre_tienda  tamano_m2
  <chr>      <chr>          <dbl>
1 CDMX       Sucursal Premium    800
2 CDMX       Sucursal Plaza A    700
3 CDMX       Sucursal Centro     500
4 CDMX       Sucursal Este       400
5 Guadalajara Sucursal Plaza B    480
6 Guadalajara Sucursal Norte    450
7 Monterrey  Sucursal Sur        600
8 Puebla     Sucursal Oeste     550
9 Querétaro  Sucursal Express    300
10 Tijuana    Sucursal Outlet     350
```

La CDMX tiene cuatro tiendas y la más grande es la *Sucursal Premium*, con 800 m². Observa que el texto se ordena alfabéticamente respetando los acentos del español (según la configuración regional de tu computadora).

Para quedarte con las primeras o las últimas filas existe la familia `slice_*`:

- `slice_head(n = 3)` y `slice_tail(n = 3)`: las primeras o las últimas 3 filas, en el orden actual.
- `slice_max(columna, n = 5)` y `slice_min(columna, n = 5)`: las 5 filas con los valores más altos o más bajos de una columna. No necesitas ordenar antes.
- `slice_sample(n = 10)`: 10 filas al azar (útil para auditorías; recuerda fijar la semilla con `set.seed()`).

```
1 # Top 3 ventas del año por monto
2 ventas %>%
3   slice_max(total, n = 3) %>%
4   select(fecha, tienda_id, total)
5
6 # Los 3 productos más baratos
7 productos %>%
8   slice_min(precio_catalogo, n = 3) %>%
9   select(nombre_producto, precio_catalogo)
10
11 # Una muestra aleatoria de 3 ventas para auditar
```

```

12 set.seed(123)
13 ventas %>%
14   slice_sample(n = 3) %>%
15   select(fecha, producto_id, total)

```

```

# A tibble: 3 x 3
  fecha      tienda_id total
<date>      <dbl> <dbl>
1 2023-07-24         8 4950.
2 2023-04-21         2 4925.
3 2023-09-04         6 4875.
# A tibble: 3 x 2
  nombre_producto precio_catalogo
<chr>              <dbl>
1 Producto 05         25.5
2 Producto 38         36.6
3 Producto 40         50.1
# A tibble: 3 x 3
  fecha      producto_id total
<date>      <dbl> <dbl>
1 2023-06-28        20 1993.
2 2023-01-14        27  982.
3 2023-07-14        14  897.

```

Top-N por grupo

La verdadera potencia de `slice_max()` aparece cuando la combinas con `group_by()`: obtienes el top-N *de cada grupo* en una sola instrucción. En Excel esto requiere fórmulas complicadas o filtros manuales grupo por grupo.

Ejemplo práctico: La mejor venta de cada trimestre y el producto más caro de cada categoría

El director quiere, para su presentación, la venta más grande de cada trimestre. Compras quiere saber cuál es el producto más caro de cada categoría.

```

1 # La venta más grande de cada trimestre
2 ventas %>%
3   group_by(trimestre) %>%
4   slice_max(total, n = 1) %>%
5   ungroup() %>%
6   select(trimestre, fecha, tienda_id, total)
7
8 # El producto más caro de cada categoría
9 productos %>%
10  group_by(categoria) %>%

```

```

11 slice_max(precio_catalogo, n = 1) %>%
12 ungroup() %>%
13 select(categoria, nombre_producto, precio_catalogo)

```

```

# A tibble: 4 x 4
  trimestre fecha      tienda_id total
  <chr>      <date>      <dbl> <dbl>
1 Q1        2023-02-15      8 4638.
2 Q2        2023-04-21      2 4925.
3 Q3        2023-07-24      8 4950.
4 Q4        2023-12-01      7 4255.
# A tibble: 10 x 3
  categoria nombre_producto precio_catalogo
  <chr>      <chr>              <dbl>
1 Alimentos Producto 33          683.
2 Automotriz Producto 47          766.
3 Belleza   Producto 20          742.
4 Deportes  Producto 19          583.
5 Electrónica Producto 41          757.
6 Hogar     Producto 10          691.
7 Juguetes  Producto 04          744.
8 Libros    Producto 35          696.
9 Mascotas  Producto 28          774.
10 Ropa     Producto 42          368.

```

`group_by()` parte la tabla en grupos y `slice_max()` se aplica dentro de cada uno. Al final, `ungroup()` quita la agrupación para que no afecte los pasos siguientes (lo explicamos en la próxima sección). Si hubiera empates, `slice_max()` devolvería todas las filas empatadas; agrega `with_ties = FALSE` si necesitas exactamente `n` filas.

Hazlo tú

Obtén las 2 ventas con más unidades (**cantidad**) de *cada* tienda. ¿Cuántas filas esperas? ¿Cuántas obtienes? Si no coinciden, explica por qué y prueba con `with_ties = FALSE`.

2.4.5 summarise() y group_by(): de miles de filas a un KPI

`summarise()` (o `summarize()`, ambas funcionan) reduce muchas filas a una sola fila de resultados. Dentro usas **funciones de resumen**: reciben un vector de muchos valores y devuelven uno solo.

Tabla 2.5. Funciones de resumen más usadas en BI.

Función	Qué calcula
<code>n()</code>	Número de filas (transacciones, registros)
<code>n_distinct(x)</code>	Número de valores distintos (clientes únicos, productos vendidos)
<code>sum(x)</code>	Suma (ingresos, unidades)
<code>mean(x), median(x)</code>	Promedio y mediana (ticket promedio, ticket típico)
<code>min(x), max(x)</code>	Mínimo y máximo (primera y última compra, venta récord)
<code>sd(x)</code>	Desviación estándar (qué tan variables son los valores)
<code>first(x), last(x)</code>	Primer y último valor según el orden actual

Ejemplo práctico: El tablero de KPIs del año

La dirección quiere los indicadores clave de 2023 en una sola tabla: ingresos, número de ventas, unidades, ticket promedio y mediano, venta mínima y máxima, y clientes únicos.

```

1 kpis_2023 <- ventas %>%
2   summarise(
3     ingresos      = sum(total),
4     num_ventas    = n(),                # cuenta filas
5     unidades      = sum(cantidad),
6     ticket_promedio = mean(total),
7     ticket_mediano = median(total),
8     venta_minima   = min(total),
9     venta_maxima   = max(total),
10    clientes_unicos = n_distinct(cliente_id) # clientes distintos
11  )
12 glimpse(kpis_2023)

```

```

Rows: 1
Columns: 8
$ ingresos      <dbl> 513916.6
$ num_ventas    <int> 365
$ unidades      <dbl> 2057
$ ticket_promedio <dbl> 1407.991
$ ticket_mediano <dbl> 1103.67
$ venta_minima   <dbl> 20.07
$ venta_maxima   <dbl> 4950.3
$ clientes_unicos <int> 172

```

Usamos `glimpse()` porque la tabla tiene una sola fila y muchas columnas. En 2023 se vendieron \$513,916.60 en 365 ventas (2,057 unidades). El ticket promedio (\$1,408) es mayor que el mediano (\$1,103.67): unas cuantas ventas muy grandes “jalan” el promedio hacia arriba, algo muy común en ventas. Por eso en BI conviene reportar ambos. Además, 172 clientes distintos compraron al menos una vez.

Resumir por grupos

Sola, `summarise()` da un total general. Precedida de `group_by()`, da un resumen *por cada grupo*. Esta combinación sigue la estrategia **dividir-aplicar-combinar** (figura 2.2): R divide la tabla según los valores de la columna de agrupación, aplica el resumen a cada pedazo y junta los resultados en una tabla nueva con una fila por grupo.

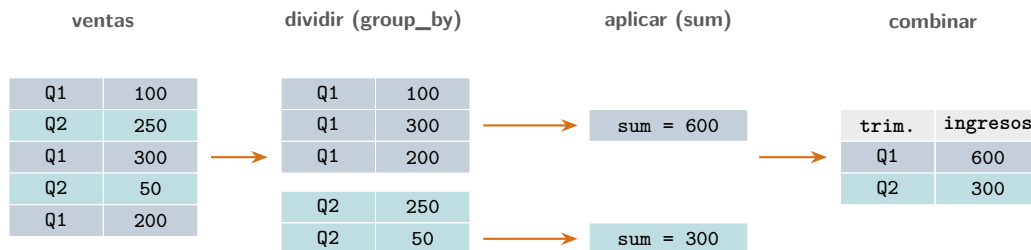


Figura 2.2. Dividir-aplicar-combinar: `group_by(trimestre) %>% summarise(ingresos = sum(total))`.

Ejemplo práctico: Ventas por trimestre

¿Cómo se comportaron las ventas en cada trimestre? ¿Cuál fue el mejor?

```
1 ventas %>%
2   group_by(trimestre) %>%           # un grupo por trimestre
3   summarise(
4     ingresos      = sum(total),
5     num_ventas    = n(),
6     ticket_promedio = mean(total)
7   )
```

```
# A tibble: 4 x 4
  trimestre ingresos num_ventas ticket_promedio
  <chr>      <dbl>      <int>      <dbl>
1 Q1        126274.         90        1403.
2 Q2        120717.         91        1327.
3 Q3        134427.         92        1461.
4 Q4        132498.         92        1440.
```

El tercer trimestre (julio a septiembre) fue el mejor, con \$134,427, y el segundo el más flojo (\$120,717). Como todos los trimestres tienen casi el mismo número de ventas (entre 90 y 92), la diferencia viene del ticket promedio: \$1,461 en el Q3 contra \$1,327 en el Q2.

Ejemplo práctico: Ranking de tiendas

¿Qué tienda vende más? Queremos ingresos, número de ventas, unidades y ticket promedio por tienda, de mayor a menor.

```

1 ventas_tienda <- ventas %>%
2   group_by(tienda_id) %>%
3   summarise(
4     ingresos      = sum(total),
5     num_ventas    = n(),
6     unidades      = sum(cantidad),
7     ticket_promedio = round(mean(total), 2)
8   ) %>%
9   arrange(desc(ingresos))           # la mejor tienda primero
10  ventas_tienda

```

```

# A tibble: 10 x 5
   tienda_id ingresos num_ventas unidades ticket_promedio
   <dbl>      <dbl>      <int>    <dbl>         <dbl>
1         8   70155.         47     246          1493.
2         9   61369.         46     260          1334.
3         7   55290.         37     209          1494.
4         2   54846.         38     198          1443.
5         3   54066.         37     222          1461.
6         6   54050.         34     219          1590.
7         5   53611.         37     229          1449.
8         1   43899.         32     169          1372.
9        10   42171.         34     195          1240.
10        4   24459.         23     110          1063.

```

La tienda 8 lidera con \$70,155 y la tienda 4 es la última con \$24,459: apenas un tercio. El problema de la tienda 4 es doble: es la que menos ventas hizo (23) y la de menor ticket promedio (\$1,063). En cambio, la tienda 6 tiene el ticket más alto (\$1,590) pero pocas ventas (34): su reto es atraer más tráfico, no vender más caro. Este tipo de lectura, que separa volumen (cuántas ventas) y valor (de cuánto cada una), es la esencia del análisis de BI.

Agrupar por varias columnas y el argumento `.groups`

Puedes agrupar por más de una columna. Observa el mensaje que aparece:

```

1 ventas_trim_tienda <- ventas %>%
2   group_by(trimestre, tienda_id) %>%
3   summarise(ingresos = sum(total))

```

```

`summarise()` has grouped output by 'trimestre'. You can override
using the `groups` argument.

```

No es un error. **dplyr** te avisa que el resultado *sigue agrupado* por `trimestre`: cada `summarise()` “pela” solo el último nivel de agrupación. Esto importa porque cualquier cálculo posterior se hará

por trimestre. Para no llevarte sorpresas, di explícitamente qué quieres con `.groups`:

```
1 ventas_trim_tienda <- ventas %>%
2   group_by(trimestre, tienda_id) %>%
3   summarise(ingresos = sum(total), .groups = "drop") # sin grupos
4 ventas_trim_tienda
```

```
# A tibble: 40 x 3
  trimestre tienda_id ingresos
  <chr>         <dbl>     <dbl>
1 Q1             1  16711.
2 Q1             2  10435.
3 Q1             3   11878.
4 Q1             4   5750.
5 Q1             5  14250.
6 Q1             6  13988.
... 
```

`.groups = "drop"` quita todos los grupos (lo más seguro y lo que usaremos casi siempre); `.groups = "keep"` los conserva todos. El resultado tiene 40 filas: 4 trimestres $\times$ 10 tiendas.

Consejo

Desde **dplyr** 1.1 existe una alternativa más corta: el argumento `.by` agrupa *solo* para esa operación y devuelve siempre una tabla sin grupos. `summarise(ingresos = sum(total), .by = c(trimestre, tienda_id))` equivale al código anterior. Verás ambas formas en código real; en este libro usamos `group_by()` porque es la más difundida.

Contar rápido con `count()`

Contar filas por grupo es tan común que tiene su propio atajo: `count(x)` equivale a `group_by(x) %>% summarise(n = n())`. Acepta `sort = TRUE` para ordenar de mayor a menor y `wt` (*weight*, peso) para *sumar* una columna en lugar de contar filas.

```
1 # ¿Cuántas transacciones por método de pago?
2 transacciones %>% count(metodo_pago, sort = TRUE)
3
4 # Con wt: suma el total en vez de contar filas
5 transacciones %>%
6   count(metodo_pago, wt = total_transaccion, sort = TRUE,
7         name = "ingresos")
8
9 # Dos columnas: combinaciones de tienda y método de pago
10 transacciones %>% count(tienda_id, metodo_pago) %>% head(5)
```

```
# A tibble: 4 x 2
  metodo_pago      n
  <chr>          <int>
1 Tarjeta Crédito  350
2 Efectivo        315
3 Tarjeta Débito   233
4 Transferencia    102
# A tibble: 4 x 2
  metodo_pago      ingresos
  <chr>          <dbl>
1 Tarjeta Crédito 351171.
2 Efectivo        303735.
3 Tarjeta Débito  220855.
4 Transferencia   84232.
# A tibble: 5 x 3
  tienda_id metodo_pago      n
  <dbl> <chr>          <int>
1      1 Efectivo        26
2      1 Tarjeta Crédito  37
3      1 Tarjeta Débito   14
4      1 Transferencia    10
5      2 Efectivo        36
```

La tarjeta de crédito es el método más frecuente (350 transacciones) y también el que más dinero mueve (\$351,171). El argumento `name` cambia el nombre de la columna de conteo, que por defecto se llama `n`.

Porcentaje del total dentro de cada grupo

Una pregunta clásica: “¿qué porcentaje representa cada cosa *dentro* de su grupo?”. La receta es: primero resúmenes con `summarise()` y después, con el resultado todavía agrupado por el nivel superior, calculas el porcentaje con `mutate()`.

Ejemplo práctico: Mezcla de métodos de pago por trimestre

Finanzas negocia las comisiones bancarias y quiere saber qué porcentaje de los ingresos de cada trimestre entró por cada método de pago.

```
1 mezcla_pago <- transacciones %>%
2   group_by(trimestre, metodo_pago) %>%
3   summarise(ingresos = sum(total_transaccion),
4             .groups = "drop_last") %>% # queda agrupado por trimestre
5   mutate(pct_trimestre = round(ingresos / sum(ingresos) * 100, 1)) %>%
6   ungroup() # quitamos el último grupo
7 mezcla_pago %>% filter(trimestre %in% c("Q1", "Q4"))
```



```
# A tibble: 8 x 4
  trimestre metodo_pago    ingresos pct_trimestre
  <chr>      <chr>          <dbl>      <dbl>
1 Q1        Efectivo        80565.      33.7
2 Q1        Tarjeta Crédito  88269.      36.9
3 Q1        Tarjeta Débito   51032.      21.3
4 Q1        Transferencia    19524.       8.2
5 Q4        Efectivo        78777.      31.9
6 Q4        Tarjeta Crédito  87412.      35.4
7 Q4        Tarjeta Débito   59178.       24
8 Q4        Transferencia    21537.       8.7
```

Con `.groups = "drop_last"` el resultado queda agrupado por `trimestre`, así que `sum(ingresos)` dentro de `mutate()` es la suma *de ese trimestre* y los porcentajes de cada trimestre suman 100. La tarjeta de débito ganó terreno: pasó de 21.3 % de los ingresos en el Q1 a 24 % en el Q4, mientras que el efectivo bajó de 33.7 % a 31.9 %. Si hubieras quitado los grupos antes del `mutate()`, `sum(ingresos)` sería el total del año y los porcentajes serían respecto al año completo: ambas preguntas son válidas, pero distintas.

Agrupar sin resumir: `group_by()` + `mutate()`

`group_by()` también funciona con `mutate()`: en lugar de reducir cada grupo a una fila, conserva todas las filas y calcula dentro de cada grupo. Es perfecto para comparar cada registro contra el promedio de su grupo.

Ejemplo práctico: ¿Qué ventas están muy por encima del promedio de su tienda?

Para detectar ventas atípicas no basta con compararlas contra el promedio general: una venta de \$3,000 puede ser normal en una tienda y excepcional en otra.

```
1 ventas_vs_tienda <- ventas %>%
2   group_by(tienda_id) %>%
3   mutate(
4     promedio_tienda = mean(total),           # promedio de SU tienda
5     veces_promedio = round(total / promedio_tienda, 2)
6   ) %>%
7   ungroup()                                # ¡no lo olvides!
8
9 ventas_vs_tienda %>%
10  select(fecha, tienda_id, total, promedio_tienda, veces_promedio) %>%
11  slice_max(veces_promedio, n = 4)
```

```
# A tibble: 4 x 5
  fecha      tienda_id total promedio_tienda veces_promedio
  <date>      <dbl> <dbl>          <dbl>          <dbl>
```

1	2023-04-21	2	4925.	1443.	3.41
2	2023-07-24	8	4950.	1493.	3.32
3	2023-02-16	9	4367.	1334.	3.27
4	2023-02-15	8	4638.	1493.	3.11

La venta del 21 de abril en la tienda 2 fue 3.41 veces el promedio de esa tienda: vale la pena confirmar que sea real. Observa que el resultado conserva las 365 filas; `mutate()` no resume, solo agrega columnas.

▲ Cuidado

Una tabla agrupada *recuerda* sus grupos. Si olvidas `ungroup()`, los pasos siguientes (un `mutate()`, un `slice_max()`, un `summarise()`) se aplicarán grupo por grupo sin que te des cuenta. Cuando imprimas una tabla y veas una línea `# Groups: tienda_id [10]` debajo del encabezado, esa tabla sigue agrupada.

Valores faltantes en los resúmenes

Si una columna tiene aunque sea un `NA`, su suma y su promedio serán `NA`: R se niega a adivinar. Usa `na.rm = TRUE` para ignorar los faltantes, pero hazlo *a conciencia* y reporta cuántos había.

```
1 metas %>%
2   summarise(
3     suma_sin_na_rm = sum(metas),           # NA "contagia"
4     suma_capturada = sum(metas, na.rm = TRUE), # ignora los NA
5     metas_faltantes = sum(is.na(metas))    # cuántos NA hay
6   )
```

```
# A tibble: 1 x 3
  suma_sin_na_rm suma_capturada metas_faltantes
      <dbl>         <dbl>         <int>
1         NA         153000             2
```

El truco `sum(is.na(x))` cuenta los faltantes: `is.na()` devuelve `TRUE/FALSE` y al sumar, cada `TRUE` vale 1.

🔗 Hazlo tú

Con `transacciones`: (a) calcula por `dia_semana` el número de transacciones, los ingresos y el ticket promedio, ordenado de mayor a menor ingreso; (b) con `count()` y `wt`, obtén los ingresos por tienda; (c) calcula qué porcentaje de las transacciones de cada tienda se pagó en efectivo.

2.5 Funciones de ventana para KPIs

Las funciones de resumen (`sum()`, `mean()`...) convierten muchos valores en uno. Las **funciones de ventana** (*window functions*) devuelven *un valor por cada fila*, pero calculado mirando a otras filas: la anterior, la siguiente, todas las previas o todo el grupo. Con ellas se calculan casi todos los KPIs de tendencia de un tablero de BI.

Tabla 2.6. Funciones de ventana más útiles en BI.

Función	Qué devuelve	KPI típico
<code>lag(x)</code>	El valor de la fila anterior	Crecimiento mes contra mes
<code>lead(x)</code>	El valor de la fila siguiente	Comparar con el periodo siguiente
<code>cumsum(x)</code>	Suma acumulada	Acumulado del año (YTD)
<code>row_number()</code>	Posición 1, 2, 3... sin empates	Numerar filas
<code>min_rank()</code>	Ranking con huecos tras empates (1, 2, 2, 4)	Ranking de tiendas
<code>dense_rank()</code>	Ranking sin huecos (1, 2, 2, 3)	Niveles de precio
<code>x / sum(x)</code>	Proporción de cada fila en el total	Participación de mercado

Primero construimos la serie mensual de ingresos: una fila por mes.

```
1 ventas_mensuales <- ventas %>%
2   group_by(mes) %>%
3   summarise(ingresos = sum(total), num_ventas = n(),
4             .groups = "drop") %>%
5   arrange(mes) # ¡las ventanas dependen del orden!
```

2.5.1 Crecimiento mes contra mes (MoM) y acumulado del año (YTD)

El **crecimiento mes contra mes** (MoM, *month over month*) mide cuánto cambió un indicador respecto al mes anterior:

$$\text{MoM} = \left(\frac{\text{ingresos del mes}}{\text{ingresos del mes anterior}} - 1 \right) \times 100.$$

El **acumulado del año** (YTD, *year to date*) es la suma de todos los meses del año hasta el mes actual.

```
1 kpis_mensuales <- ventas_mensuales %>%
2   mutate(
3     mes_anterior = lag(ingresos), # fila previa
4     mom_pct      = (ingresos / lag(ingresos) - 1) * 100,
5     ytd          = cumsum(ingresos) # acumulado
6   )
7
8 kpis_mensuales %>%
9   mutate(across(where(is.numeric), ~ round(.x, 1))) %>%
```

```
10 select(mes, ingresos, mes_anterior, mom_pct, ytd)
```

```
# A tibble: 12 x 5
  mes      ingresos mes_anterior mom_pct    ytd
  <chr>      <dbl>      <dbl>   <dbl>   <dbl>
1 2023-01    46884.         NA      NA    46884.
2 2023-02    42656.     46884.    -9    89540.
3 2023-03    36734.     42656.  -13.9 126274.
4 2023-04    34738.     36734.   -5.4 161012.
5 2023-05    48108.     34738.   38.5 209120.
6 2023-06    37871.     48108.  -21.3 246991.
7 2023-07    54056.     37871.   42.7 301047.
8 2023-08    41421.     54056.  -23.4 342468.
9 2023-09    38950.     41421.   -6   381418.
10 2023-10    36584.     38950.  -6.1 418002.
11 2023-11    50398.     36584.   37.8 468400.
12 2023-12    45516.     50398.  -9.7 513917.
```

¿Qué nos dice esto? Enero no tiene mes anterior dentro de los datos, por eso `lag()` devuelve `NA` y el crecimiento también: es correcto, no un error. El mayor crecimiento fue en julio (42.7% contra junio) y la peor caída en agosto (-23.4%). Nota el patrón de “serrucho”: a cada mes fuerte le sigue uno débil. La columna `ytd` muestra que al cierre de junio se llevaban \$246,991 (48% del año) y el año cerró en \$513,916.60.

▲ Cuidado

`lag()` toma “la fila de arriba”, no “el mes anterior”. Si la tabla no está ordenada por mes, o si falta un mes, el cálculo será incorrecto sin ningún aviso. Por eso ordenamos con `arrange(mes)` antes, y en la sección 2.9.5 verás cómo rellenar los meses faltantes con `complete()`. Para una serie con fechas desordenadas puedes usar `lag(ingresos, order_by = mes)`.

2.5.2 Participación y promedio móvil

La **participación** de cada mes es su porcentaje del total anual. El **promedio móvil** de 3 meses suaviza los altibajos: en cada mes promedia ese mes y los dos anteriores, y deja ver la tendencia de fondo. `lag(x, 2)` es el valor de dos filas atrás.

```
1 kpis_mensuales <- kpis_mensuales %>%
2   mutate(
3     participacion = ingresos / sum(ingresos) * 100,
4     prom_movil_3m = (ingresos + lag(ingresos) + lag(ingresos, 2)) / 3,
5     vs_siguiete = lead(ingresos) - ingresos      # ¿el mes que sigue?
6   )
7
8 kpis_mensuales %>%
```

```

9 mutate(across(where(is.numeric), ~ round(.x, 1))) %>%
10 select(mes, ingresos, participacion, prom_movil_3m, vs_siguiente)

```

```

# A tibble: 12 x 5
  mes      ingresos participacion prom_movil_3m vs_siguiente
<chr>      <dbl>      <dbl>      <dbl>      <dbl>
1 2023-01  46884.         9.1         NA        -4228.
2 2023-02  42656.         8.3         NA       -5923.
3 2023-03  36734.         7.1       42091.   -1996.
4 2023-04  34738.         6.8       38042.   13370.
5 2023-05  48108.         9.4       39860.  -10238.
6 2023-06  37871.         7.4       40239.   16185.
7 2023-07  54056.        10.5       46678. -12635.
8 2023-08  41421.         8.1       44449.   -2471.
9 2023-09  38950.         7.6       44809.   -2366.
10 2023-10  36584.         7.1       38985.   13814.
11 2023-11  50398.         9.8       41977.   -4882.
12 2023-12  45516.         8.9       44166.      NA

```

Cada mes aporta entre 6.8 % (abril) y 10.5 % (julio) de las ventas del año. El promedio móvil empieza en marzo (necesita tres meses) y se mueve entre \$38,042 y \$46,678: un rango mucho más estrecho que el de los ingresos mensuales, que van de \$34,738 a \$54,056. Es decir, detrás de la volatilidad mensual el negocio es bastante estable. La columna `vs_siguiente`, calculada con `lead()`, es `NA` en diciembre porque no hay mes siguiente.

2.5.3 Rankings

Hay tres funciones de ranking y se diferencian en cómo tratan los empates. Míralo con un ejemplo pequeño:

```

1 tibble(vendedor = c("Ana", "Beto", "Caro", "Dani", "Eva"),
2        ventas   = c(900, 700, 700, 500, 300)) %>%
3   mutate(
4     row_number = row_number(desc(ventas)), # 1,2,3,4,5 (sin empates)
5     min_rank   = min_rank(desc(ventas)),   # 1,2,2,4,5 (con hueco)
6     dense_rank = dense_rank(desc(ventas))  # 1,2,2,3,4 (sin hueco)
7   )

```

```

# A tibble: 5 x 5
  vendedor ventas row_number min_rank dense_rank
<chr>      <dbl>      <int>      <int>      <int>
1 Ana        900         1         1         1
2 Beto        700         2         2         2
3 Caro        700         3         2         2
4 Dani        500         4         4         3

```

5	Eva	300	5	5	4
---	-----	-----	---	---	---

Usamos `desc()` para que el valor más alto reciba el lugar 1. `min_rank()` es el ranking “deportivo” (dos segundos lugares y nadie en tercero) y es el más usado en reportes.

Ejemplo práctico: Las 3 mejores tiendas de cada trimestre

Recursos Humanos quiere premiar cada trimestre a las tres tiendas con más ingresos. ¿Cuáles son y repiten las mismas?

```
1 podio_trimestral <- ventas_trim_tienda %>%      # la calculamos antes
2   group_by(trimestre) %>%
3   mutate(lugar = min_rank(desc(ingresos))) %>% # ranking por trimestre
4   filter(lugar <= 3) %>%
5   arrange(trimestre, lugar) %>%
6   ungroup()
7 podio_trimestral
```

```
# A tibble: 12 x 4
  trimestre tienda_id ingresos lugar
  <chr>      <dbl>    <dbl> <int>
1 Q1          8    17226.     1
2 Q1          1    16711.     2
3 Q1          9    16706.     3
4 Q2          2    20947.     1
5 Q2          8    19290.     2
6 Q2          3    14997.     3
7 Q3          9    19581.     1
8 Q3          6    18015.     2
9 Q3          2    16639.     3
10 Q4          5    19978.     1
11 Q4          8    18245.     2
12 Q4          1    14823.     3
```

La tienda 8 subió al podio en tres de los cuatro trimestres: es la más consistente. Más interesante es la tienda 1: en el ranking anual quedó en octavo lugar, pero estuvo en el podio del Q1 y del Q4. Sus problemas se concentran a mitad de año, algo que el total anual escondía. Observa también que `min_rank()` dentro de `group_by(trimestre)` reinicia el ranking en cada trimestre.

Ejemplo práctico: Análisis ABC (Pareto) de productos

El principio de Pareto dice que pocos productos generan la mayor parte de las ventas. El **análisis ABC** clasifica los productos en: **A**, los que acumulan el primer 80 % de los ingresos; **B**, el siguiente 15%; y **C**, el último 5%. ¿Cuántos productos hay en cada clase?

```

1 abc_productos <- ventas %>%
2   group_by(producto_id) %>%
3   summarise(ingresos = sum(total), .groups = "drop") %>%
4   arrange(desc(ingresos)) %>%           # del mejor al peor
5   mutate(
6     pct_acumulado = cumsum(ingresos) / sum(ingresos) * 100,
7     clase = case_when(
8       pct_acumulado <= 80 ~ "A",
9       pct_acumulado <= 95 ~ "B",
10      .default           = "C"
11    )
12  )
13
14 abc_productos %>%
15   group_by(clase) %>%
16   summarise(productos = n(), ingresos = sum(ingresos),
17             .groups = "drop") %>%
18   mutate(pct_ingresos = round(ingresos / sum(ingresos) * 100, 1))

```

```

# A tibble: 3 x 4
  clase productos ingresos pct_ingresos
  <chr>      <int>    <dbl>      <dbl>
1 A             31  402751.        78.4
2 B             12   83751.        16.3
3 C              7   27415.         5.3

```

Aquí `cumsum()` se aplica después de ordenar de mayor a menor, así que `pct_acumulado` dice qué porcentaje de los ingresos suman los mejores productos hasta esa fila. El resultado es interesante porque *no* sigue el 80/20 clásico: se necesitan 31 de los 50 productos (62%) para juntar el 78.4% de los ingresos. Las ventas de esta empresa están poco concentradas, así que discontinuar productos de clase C (7 productos, 5.3% de los ingresos) tendría poco impacto en la venta. En una empresa real este análisis guía decisiones de inventario: a los productos A nunca les debe faltar existencia.

Hazlo tú

Repite el cálculo de MoM y YTD, pero con `transacciones` y la columna `total_transaccion`. ¿Cuál fue el mejor mes? ¿En qué mes el acumulado superó el medio millón de pesos? Pista: después de calcular `ytd`, usa `filter(ytd > 500000) %>% slice_head(n = 1)`.

2.6 Fechas con lubridate

Casi todo análisis de BI tiene una dimensión de tiempo: ventas por mes, comparaciones contra el año anterior, antigüedad de clientes, días sin comprar. **lubridate** (incluido en el *tidyverse* desde la versión 2.0) hace que trabajar con fechas sea sencillo.

2.6.1 Convertir texto en fechas

Las fechas llegan como texto en mil formatos: 2023-03-15, 15/03/2023, 03/15/2023 (formato de Estados Unidos)... **lubridate** tiene una función por cada *orden* de los componentes: **ymd()** (año-mes-día), **dmy()** (día-mes-año) y **mdy()** (mes-día-año). Los separadores no importan.

```
1 ymd("2023-03-15")           # año, mes, día
2 dmy("15/03/2023")           # día, mes, año (formato de México)
3 mdy("03-15-2023")           # mes, día, año (formato de EE. UU.)
4 dmy("15 de marzo de 2023")  # ¡entiende meses en español!
5 ymd(20230315)               # también números tipo AAAAMMDD
```

```
[1] "2023-03-15"
[1] "2023-03-15"
[1] "2023-03-15"
[1] "2023-03-15"
[1] "2023-03-15"
```

Las cinco dan la misma fecha. El resultado es un objeto de clase **Date**: R ya sabe que es una fecha y puede ordenarla, restarla o extraer sus partes. El texto "15/03/2023", en cambio, es solo una cadena de caracteres.

▲ Cuidado

¿03/04/2023 es 3 de abril o 4 de marzo? Depende de quién lo escribió. Antes de convertir, pregunta de qué sistema viene el archivo y revisa un valor con día mayor a 12 (como 15/03) para confirmar el orden. Si una fecha no se puede interpretar, **lubridate** devuelve **NA** con la advertencia **failed to parse**.

2.6.2 Extraer componentes de una fecha

Ejemplo práctico: Enriquecer las ventas con atributos de calendario

Para analizar estacionalidad necesitamos, por cada venta, el año, el número y el nombre del mes, el trimestre, el día de la semana y la semana del año.

```
1 ventas_cal <- ventas %>%
2   mutate(
3     anio      = year(fecha),           # 2023
4     mes_num   = month(fecha),         # 1 a 12
5     mes_nombre = month(fecha, label = TRUE), # ene, feb...
6     trim     = quarter(fecha),       # 1 a 4
7     dia_sem   = wday(fecha, label = TRUE, abbr = FALSE), # lunes...
8     semana    = week(fecha)          # 1 a 53
```



```

9   )
10  ventas_cal %>%
11    select(fecha, anio, mes_num, mes_nombre, trim, dia_sem, semana) %>%
12    head(4)

```

```

# A tibble: 4 x 7
  fecha      anio mes_num mes_nombre trim dia_sem  semana
<date>    <dbl>   <dbl> <ord>    <int> <ord>    <dbl>
1 2023-01-01  2023     1 ene      1 domingo    1
2 2023-01-02  2023     1 ene      1 lunes      1
3 2023-01-03  2023     1 ene      1 martes     1
4 2023-01-04  2023     1 ene      1 miércoles  1

```

Fíjate en los tipos: `mes_nombre` y `dia_sem` son `<ord>`, **factores ordenados**. Eso significa que R sabe que “ene” va antes que “feb” y “domingo” antes que “lunes”, así que al agrupar o graficar respetará el orden del calendario y no el alfabético (que pondría “abr” antes que “ene”). Los nombres salen en español porque la configuración regional de la computadora está en español; con `label = TRUE`, `abbr = FALSE` obtienes el nombre completo.

Ejemplo práctico: ¿Qué día de la semana vendemos más?

El gerente de operaciones quiere saber qué días necesita más personal.

```

1  ventas_cal %>%
2    group_by(dia_sem) %>%
3    summarise(ventas = n(), ingresos = sum(total),
4              ticket_promedio = mean(total), .groups = "drop")

```

```

# A tibble: 7 x 4
  dia_sem  ventas ingresos ticket_promedio
<ord>    <int>    <dbl>         <dbl>
1 domingo     53  69334.         1308.
2 lunes       52  76889.         1479.
3 martes       52  74728.         1437.
4 miércoles   52  73081.         1405.
5 jueves       52  76647.         1474.
6 viernes     52  70125.         1349.
7 sábado      52  73113.         1406.

```

Gracias al factor ordenado, los días aparecen de domingo a sábado y no en orden alfabético. El lunes es el día más fuerte (\$76,889) y el domingo el más débil (\$69,334), aunque el domingo tuvo una venta más (53, porque 2023 empezó y terminó en domingo). La diferencia entre el mejor y el peor día es de apenas 11%: no hay un día claramente “pico”, así que la plantilla de personal puede ser pareja durante la semana. Observa que el domingo tiene el ticket promedio más bajo (\$1,308).

2.6.3 Redondear fechas: `floor_date()` y `make_date()`

La columna `mes` de `ventas` es texto ("2023-01"). Funciona para agrupar, pero no es una fecha. `floor_date()` “redondea hacia abajo” cada fecha al inicio de su periodo: el primer día del mes, de la semana o del trimestre. El resultado sigue siendo una fecha, ideal para gráficas de series de tiempo (Módulo 3).

```
1 ventas %>%
2   mutate(mes_fecha = floor_date(fecha, unit = "month")) %>%
3   group_by(mes_fecha) %>%
4   summarise(ingresos = sum(total), .groups = "drop") %>%
5   head(3)
6
7 # Ventas por semana (las semanas empiezan en lunes: week_start = 1)
8 ventas %>%
9   mutate(semmana_inicio = floor_date(fecha, "week", week_start = 1)) %>%
10  count(semmana_inicio) %>%
11  head(3)
```

```
# A tibble: 3 x 2
  mes_fecha ingresos
  <date>         <dbl>
1 2023-01-01  46884.
2 2023-02-01  42656.
3 2023-03-01  36733.
# A tibble: 3 x 2
  semana_inicio  n
  <date>         <int>
1 2022-12-26      1
2 2023-01-02      7
3 2023-01-09      7
```

La primera “semana” empieza el lunes 26 de diciembre de 2022 porque el 1 de enero de 2023 fue domingo: esa semana solo tiene un día de datos. Tenlo en cuenta al reportar semanas incompletas en los extremos.

Si tienes año, mes y día en columnas separadas (muy común en reportes de sistemas antiguos), `make_date()` los junta en una fecha:

```
1 metas_mensuales <- tibble(anio = 2023, mes = 1:3,
2                             meta = c(45000, 42000, 40000)) %>%
3   mutate(fecha_mes = make_date(anio, mes, 1)) # día 1 de cada mes
4 metas_mensuales
```

```
# A tibble: 3 x 4
  anio  mes meta fecha_mes
```

```

<dbl> <int> <dbl> <date>
1  2023      1 45000 2023-01-01
2  2023      2 42000 2023-02-01
3  2023      3 40000 2023-03-01

```

2.6.4 Diferencias entre fechas: antigüedad y recencia

Restar dos fechas da un número de días (un objeto `difftime`). Hay dos formas recomendadas de convertirlo en un número utilizable:

- `as.numeric(difftime(fin, inicio, units = "days"))`: días (o semanas, horas...) como número.
- `time_length(interval(inicio, fin), "years")`: años (o meses) exactos, considerando años bisiestos y meses de distinta longitud.

Para que tus resultados sean reproducibles, usa siempre una **fecha de corte** fija en lugar de `Sys.Date()` (la fecha de hoy, que cambia cada día). Nosotros usaremos el 1 de enero de 2024, justo después del cierre del año de datos.

Ejemplo práctico: Antigüedad de los clientes

¿Cuántos años llevan registrados nuestros clientes? ¿Cuántos son clientes nuevos (menos de un año)?

```

1 fecha_corte <- as.Date("2024-01-01")      # fija: resultados reproducibles
2
3 clientes_antig <- clientes %>%
4   mutate(
5     dias_registro = as.numeric(difftime(fecha_corte, fecha_registro,
6                                         units = "days")),
7     anios_registro = time_length(interval(fecha_registro, fecha_corte),
8                                          "years"),
9     tipo_cliente  = if_else(anios_registro < 1, "Nuevo", "Recurrente")
10  )
11 clientes_antig %>%
12   select(cliente_id, fecha_registro, dias_registro, anios_registro) %>%
13   head(3)
14 clientes_antig %>% count(tipo_cliente)

```

```

# A tibble: 3 x 4
  cliente_id fecha_registro dias_registro anios_registro
  <dbl>    <date>          <dbl>          <dbl>
1         1  2023-10-12          81            0.221
2         2  2020-01-30        1432            3.92

```

```

3          3 2020-10-17          1171          3.21
# A tibble: 2 x 2
  tipo_cliente     n
  <chr>          <int>
1 Nuevo           50
2 Recurrente     150

```

El cliente 1 se registró el 12 de octubre de 2023: lleva 81 días, es decir, 0.22 años. La cuarta parte de la base (50 de 200 clientes) se registró durante 2023: una base joven, lo que obliga a cuidar la experiencia de las primeras compras.

Ejemplo práctico: Recencia: días desde la última compra

La **recencia** (cuántos días lleva un cliente sin comprar) es uno de los mejores predictores de abandono y la primera letra del análisis RFM (Recencia, Frecuencia, Monto) que harás en el Módulo 4. ¿Qué clientes llevan más tiempo sin comprar?

```

1  recencia <- ventas %>%
2    group_by(cliente_id) %>%
3    summarise(
4      primera_compra = min(fecha),
5      ultima_compra  = max(fecha),
6      compras        = n(),
7      monto           = sum(total),
8      .groups = "drop"
9    ) %>%
10   mutate(
11     dias_sin_comprar = as.numeric(fecha_corte - ultima_compra),
12     estado = case_when(
13       dias_sin_comprar <= 90 ~ "Activo",
14       dias_sin_comprar <= 180 ~ "En riesgo",
15       .default              = "Inactivo"
16     )
17   )
18  recencia %>%
19    arrange(desc(dias_sin_comprar)) %>%
20    select(cliente_id, ultima_compra, compras, dias_sin_comprar, estado) %>%
21    head(3)
22  recencia %>% count(estado)

```

```

# A tibble: 3 x 5
  cliente_id ultima_compra compras dias_sin_comprar estado
  <dbl> <date>          <int>          <dbl> <chr>
1      82 2023-01-07           1          359 Inactivo
2     113 2023-01-11           1          355 Inactivo
3      16 2023-01-21           1          345 Inactivo

```

```
# A tibble: 3 x 2
  estado      n
  <chr>    <int>
1 Activo    68
2 En riesgo 50
3 Inactivo  54
```

El cliente 82 compró una sola vez, el 7 de enero, y no ha vuelto en 359 días. De los 172 clientes que compraron en 2023, 68 están activos (compraron en los últimos 90 días), 50 están en riesgo y 54 inactivos. Esa última lista es la base de una campaña de reactivación. Observa que `fecha_corte - ultima_compra` da días directamente; `as.numeric()` lo convierte en número simple.

Hazlo tú

Con `tiendas` y la misma `fecha_corte`, calcula la antigüedad de cada tienda en años con `time_length()` y ordénalas de la más antigua a la más nueva. Luego, con `empleados`, cuenta cuántos empleados ingresaron en cada año usando `year(fecha_ingreso)`.

2.7 Texto con stringr

Nombres de clientes, ciudades, categorías, códigos de producto, correos: buena parte de los datos de negocio es texto, y el texto casi siempre viene “sucio”. Todas las funciones de **stringr** empiezan con `str_` (escribe `str_` en RStudio y pulsa `Tab` para ver la lista) y reciben el texto como primer argumento.

2.7.1 Mayúsculas, minúsculas y espacios

```
1 ciudades <- c(" cdmx", "Guadalajara ", "MONTERREY", "san luis potosí")
2
3 str_to_upper(ciudades)      # TODO EN MAYÚSCULAS
4 str_to_lower(ciudades)     # todo en minúsculas
5 str_to_title(ciudades)     # Tipo Título
6 str_trim(ciudades)         # quita espacios al inicio y al final
7 str_squish(ciudades)       # además, reduce espacios internos repetidos
```

```
[1] " CDMX"           "GUADALAJARA "    "MONTERREY"
[4] "SAN LUIS POTOSÍ"
[1] " cdmx"           "guadalajara "    "monterrey"
[4] "san luis potosí"
[1] " Cdmx"           "Guadalajara "    "Monterrey"
[4] "San Luis Potosí"
[1] "cdmx"            "Guadalajara"     "MONTERREY"
[4] "san luis potosí"
```

```
[1] "cdmx"           "Guadalajara"    "MONTERREY"
[4] "san luis potosí"
```

Para R, "CDMX", "cdmx" y " CDMX" son tres valores distintos: al agrupar aparecerían como tres ciudades. Por eso el primer paso de limpieza de cualquier columna de texto suele ser `str_squish()` seguido de `str_to_upper()` o `str_to_title()`.

2.7.2 Buscar, reemplazar y extraer

- `str_detect(x, patrón)`: ¿contiene el patrón? (TRUE/FALSE).
- `str_replace_all(x, patrón, reemplazo)`: reemplaza todas las apariciones (`str_replace()` solo la primera).
- `str_sub(x, inicio, fin)`: extrae por posición.
- `str_length(x)`: número de caracteres.

Ejemplo práctico: Una campaña para la familia García

Mercadotecnia prepara una campaña de apellidos y quiere a los clientes cuyo nombre contiene “García”. Además, para el CRM necesita el teléfono sin guiones y la clave de área (los primeros dos dígitos).

```
1 clientes %>%
2   filter(str_detect(nombre, "García")) %>%           # contiene García
3   mutate(
4     tel_limpio = str_replace_all(telefono, "-", ""), # quita guiones
5     lada       = str_sub(telefono, 1, 2)             # caracteres 1 a 2
6   ) %>%
7   select(cliente_id, nombre, telefono, tel_limpio, lada) %>%
8   head(4)
```

```
# A tibble: 4 x 5
  cliente_id nombre      telefono  tel_limpio lada
    <dbl> <chr>          <chr>      <chr>    <chr>
1      12 Laura García  55-2447-4702 5524474702 55
2      35 Isabel García  55-4055-2849 5540552849 55
3      42 Fernando García 55-3697-8875 5536978875 55
4      58 Sofia García   55-4141-3516 5541413516 55
```

`str_detect()` distingue mayúsculas de minúsculas y acentos: “garcia” no encontraría a “García”. Si no estás seguro de cómo viene escrito, normaliza antes con `str_to_lower()`.

 Consejo

Los patrones de **stringr** son **expresiones regulares** (*regex*), un minilenguaje para describir texto. No necesitas dominarlas para empezar; estos cuatro símbolos resuelven la mayoría de los casos: `^` (inicio del texto: "`^Tarjeta`"), `$` (final: "`com$`"), `|` (o: "`CDMX|Toluca`") y `[0-9]` (cualquier dígito). Si buscas un carácter especial como punto o signo de pesos de forma literal, usa `fixed(".")`.

2.7.3 Rellenar y armar códigos: `str_pad()` y `str_c()`

Ejemplo práctico: Códigos de producto para el sistema de inventarios

El nuevo sistema de inventarios exige claves con formato SKU-0001: el prefijo, un guion y el identificador con ceros a la izquierda hasta completar cuatro dígitos.

```
1 productos %>%
2   mutate(
3     sku = str_c("SKU-", str_pad(producto_id, width = 4, pad = "0")),
4     nombre_corto = str_replace(nombre_producto, "Producto", "Prod.")
5   ) %>%
6   select(producto_id, sku, nombre_producto, nombre_corto) %>%
7   slice(c(1, 9, 10, 50))           # filas 1, 9, 10 y 50
```

```
# A tibble: 4 x 4
  producto_id sku      nombre_producto nombre_corto
    <dbl> <chr>      <chr>          <chr>
1         1 SKU-0001 Producto 01      Prod. 01
2         9 SKU-0009 Producto 09      Prod. 09
3        10 SKU-0010 Producto 10      Prod. 10
4        50 SKU-0050 Producto 50      Prod. 50
```

`str_pad(x, width = 4, pad = "0")` completa con ceros a la izquierda (el lado por defecto) hasta llegar a 4 caracteres, y `str_c()` pega textos, igual que `paste0()`. `slice()` elige filas por su posición.

2.7.4 Partir texto: `str_split()` y `separate_wider_delim()`

`str_split()` parte un texto en pedazos según un separador y devuelve una lista. Dentro de una tabla es más práctico `separate_wider_delim()` de **tidyr**, que reparte los pedazos en columnas nuevas.

```
1 str_split("maria.lopez@email.com", "@")      # devuelve una lista
2
3 clientes %>%
```

```

4 select(cliente_id, nombre, email) %>%
5 separate_wider_delim(nombre, delim = " ",
6                       names = c("nombre_pila", "apellido")) %>%
7 separate_wider_delim(email, delim = "@",
8                       names = c("usuario", "dominio")) %>%
9 head(3)

```

```

[[1]]
[1] "maria.lopez" "email.com"

# A tibble: 3 x 5
  cliente_id nombre_pila apellido usuario      dominio
    <dbl> <chr>      <chr> <chr>      <chr>
1         1 Ricardo    Morales  juan.ramírez email.com
2         2 Miguel    Cruz    alberto.jiménez email.com
3         3 Miguel    Flores  pedro.sánchez email.com

```

La columna original desaparece y se reemplaza por las nuevas. Si alguna fila tuviera más o menos pedazos de los esperados (por ejemplo, un cliente con dos apellidos), la función se detendría con un error explicativo; los argumentos `too_many = "merge"` y `too_few = "align_start"` indican qué hacer en esos casos.

En la empresa: Un hallazgo en los correos

¿Notaste algo raro en la salida anterior? El cliente 1 se llama *Ricardo Morales*, pero su correo es *juan.ramírez@email.com*. En estos datos simulados el nombre y el correo se generaron por separado, así que casi nunca coinciden. En una empresa real, esta discrepancia sería un problema serio de calidad (¿el correo pertenece a otra persona?) que tendrías que reportar antes de lanzar una campaña de correo.

Hazlo tú

En `transacciones`, la columna `hora` tiene formato `hh:mm`. Conviértela a texto con `as.character()`, extrae la hora con `str_sub(..., 1, 2)`, conviértela a número y cuenta cuántas transacciones hay por hora del día. ¿Cuál es la hora pico?

2.8 Unir tablas: los *joins*

La tabla `ventas` dice que el 1 de enero se vendió el producto 31 al cliente 94 en la tienda 6. Pero ¿qué producto es el 31? ¿De qué categoría? ¿En qué ciudad está la tienda 6? Esa información vive en otras tablas. Un **join** (unión) combina dos tablas usando una o más columnas en común, llamadas **llaves** (*keys*). Es el equivalente de `BUSCARV/XLOOKUP` de Excel, pero para todas las columnas y todas las filas a la vez, y es la operación más importante de BI.

2.8.1 Tipos de join

Todos los *joins* de **dplyr** tienen la forma `tipo_join(x, y, by = "llave")`, donde **x** es la tabla “izquierda” (normalmente la de hechos) y **y** la “derecha” (normalmente la dimensión). La figura 2.3 resume qué filas conserva cada uno.

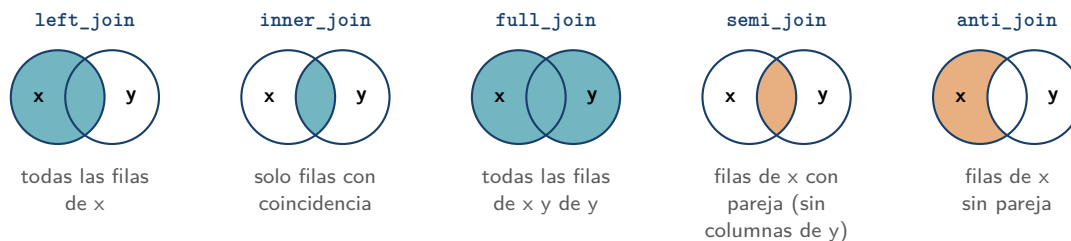


Figura 2.3. Qué conserva cada tipo de *join*. En azul, los que *agregan* columnas de **y** (*mutating joins*); en naranja, los que solo *filtran* filas de **x** (*filtering joins*).

Para verlos en acción usemos dos tablas diminutas donde es fácil seguir cada fila: tres ventas y un catálogo de tres productos. Fíjate en que la venta 3 es del producto 99, que no existe en el catálogo, y que el producto 30 del catálogo no tiene ventas.

```
1 ventas_mini <- tibble(venta = 1:3, producto_id = c(10, 20, 99),
2                       unidades = c(2, 5, 1))
3 catalogo_mini <- tibble(producto_id = c(10, 20, 30),
4                          nombre = c("Café", "Té", "Azúcar"))
5
6 left_join(ventas_mini, catalogo_mini, by = "producto_id")
7 inner_join(ventas_mini, catalogo_mini, by = "producto_id")
8 full_join(ventas_mini, catalogo_mini, by = "producto_id")
```

```
# A tibble: 3 x 4
  venta producto_id unidades nombre
<int>     <dbl>     <dbl> <chr>
1     1         10         2 Café
2     2         20         5 Té
3     3         99         1 <NA>

# A tibble: 2 x 4
  venta producto_id unidades nombre
<int>     <dbl>     <dbl> <chr>
1     1         10         2 Café
2     2         20         5 Té

# A tibble: 4 x 4
  venta producto_id unidades nombre
<int>     <dbl>     <dbl> <chr>
1     1         10         2 Café
2     2         20         5 Té
3     3         99         1 <NA>
```

4	NA	30	NA Azúcar
---	----	----	-----------

- `left_join()` conserva las 3 ventas; la del producto 99 queda con **NA** en **nombre**. Es el *join* que usarás el 90 % de las veces, porque nunca pierde filas de tu tabla de hechos.
- `inner_join()` solo conserva las 2 ventas con producto conocido. Cuidado: la venta 3 *desaparece en silencio* y tus totales bajan.
- `full_join()` conserva todo: las 3 ventas y el producto 30 sin ventas (con **NA** en **venta** y **unidades**).
- `right_join(x, y)` es un `left_join(y, x)` con las columnas en otro orden; casi no se usa.

Los *filtering joins* no agregan columnas: solo deciden qué filas de **x** se quedan.

```
1 semi_join(ventas_mini, catalogo_mini, by = "producto_id") # con pareja
2 anti_join(ventas_mini, catalogo_mini, by = "producto_id") # sin pareja
3 anti_join(catalogo_mini, ventas_mini, by = "producto_id") # sin ventas
```

```
# A tibble: 2 x 3
  venta producto_id unidades
<int>      <dbl>     <dbl>
1     1         10         2
2     2         20         5

# A tibble: 1 x 3
  venta producto_id unidades
<int>      <dbl>     <dbl>
1     3         99         1

# A tibble: 1 x 2
  producto_id nombre
      <dbl> <chr>
1         30  Azúcar
```

El último `anti_join()` invierte las tablas y responde otra pregunta: ¿qué productos del catálogo no se vendieron? El orden de las tablas importa.

2.8.2 Enriquecer las ventas con las dimensiones

Ejemplo práctico: Ventas por categoría de producto

La tabla **ventas** no tiene la categoría; **productos**, sí. ¿Cuánto vendió cada categoría?

```
1 ventas_prod <- ventas %>%
2   left_join(productos, by = "producto_id") # agrega 12 columnas
3   dim(ventas_prod)                       # mismas 365 filas
4
```

```

5 ventas_prod %>%
6   group_by(categoria) %>%
7   summarise(ingresos = sum(total), unidades = sum(cantidad),
8             productos_vendidos = n_distinct(producto_id),
9             .groups = "drop") %>%
10  arrange(desc(ingresos))

```

```

[1] 365 26
# A tibble: 10 x 4
  categoria ingresos unidades productos_vendidos
  <chr>      <dbl>    <dbl>          <int>
1 Belleza   104840.    430             9
2 Electrónica 83949.    301             7
3 Juguetes   72754.    246             6
4 Automotriz 46865.    207             5
5 Alimentos 46178.    195             6
6 Deportes  43319.    183             4
7 Hogar     41729.    146             3
8 Mascotas  26518.    117             4
9 Libros    25537.    127             4
10 Ropa      22227.    105             2

```

La tabla creció a lo ancho (de 14 a 26 columnas) pero no a lo largo: siguen siendo 365 filas, como debe ser al unir hechos con una dimensión cuya llave no se repite. Belleza lidera con \$104,840 (20% del total), en buena medida porque es la categoría con más productos en el catálogo (9). Ropa, con solo 2 productos, queda en último lugar.

Consejo

Después de cada *join*, compara el número de filas antes y después (`nrow()`). En un `left_join()` contra una dimensión, deben ser iguales. Si aumentaron, tienes llaves duplicadas en la tabla derecha (lo veremos en un momento).

Llaves con nombres distintos: `by = c(...)` y `join_by()`

En `ventas` el vendedor se identifica con `vendedor_id`, pero en `empleados` la llave se llama `empleado_id`. Hay dos formas de indicarlo:

```

1 # Forma clásica: c("nombre en x" = "nombre en y")
2 v1 <- ventas %>%
3   left_join(empleados, by = c("vendedor_id" = "empleado_id"))
4
5 # Forma moderna (dplyr 1.1+): join_by() con ==
6 v2 <- ventas %>%
7   left_join(empleados, join_by(vendedor_id == empleado_id))

```

```

8
9 identical(v1, v2)      # ¿dan exactamente lo mismo?

```

```
[1] TRUE
```

`join_by()` se lee más natural y además permite uniones por desigualdad (por ejemplo, `join_by(producto_id, fecha >= inicio)` para buscar el precio vigente en cada fecha), que son muy útiles pero rebasan este módulo.

Ejemplo práctico: ¿Quiénes son nuestros vendedores?

Recursos Humanos quiere saber de qué departamento son los empleados que aparecen como vendedores y cuánto vendió cada departamento.

```

1 v2 %>%
2   group_by(departamento) %>%
3   summarise(vendedores = n_distinct(vendedor_id),
4             ingresos = sum(total), .groups = "drop") %>%
5   arrange(desc(ingresos))

```

```

# A tibble: 7 x 3
  departamento      vendedores ingresos
  <chr>             <int>     <dbl>
1 Atención al Cliente    10  227419.
2 Ventas                 4   91570.
3 RRHH                  4   73760.
4 Marketing              3   50821.
5 Finanzas               2   42522.
6 IT                    1   20292.
7 Operaciones           1    7534.

```

En la empresa: Un hallazgo que hay que reportar

Solo 4 de los 25 vendedores pertenecen al departamento de *Ventas*; 10 son de Atención al Cliente (y generan \$227,419, el 44% de los ingresos) y hay vendedores de RRHH, Finanzas, IT y Operaciones. En estos datos simulados la tabla de empleados se generó sin relación con las ventas, pero en una empresa real este resultado indicaría un problema: o el catálogo de empleados está desactualizado, o `vendedor_id` no significa lo que creemos (quizá es el cajero, no el vendedor). Antes de pagar comisiones con estos datos, habría que aclararlo con el dueño del sistema.

Columnas con el mismo nombre: **suffix**

Si las dos tablas tienen una columna con el mismo nombre que *no* es la llave, **dplyr** conserva ambas y les agrega los sufijos **.x** y **.y**. Pasa con **ciudad**, que existe en **clientes** y en **tiendas**. Con **suffix** eliges sufijos que se entiendan.

Ejemplo práctico: ¿Nuestros clientes compran en su ciudad?

¿Qué porcentaje de las ventas se hace en una tienda de la misma ciudad donde vive el cliente?

```
1 ventas_ciudad <- ventas %>%
2   left_join(clientes, by = "cliente_id") %>%
3   left_join(tiendas, by = "tienda_id",
4             suffix = c("_cliente", "_tienda")) # en vez de .x / .y
5
6 ventas_ciudad %>%
7   select.fecha, ciudad_cliente, ciudad_tienda) %>%
8   head(3)
9
10 ventas_ciudad %>%
11   summarise(pct_misma_ciudad =
12             mean(ciudad_cliente == ciudad_tienda) * 100)
```

```
# A tibble: 3 x 3
  fecha      ciudad_cliente ciudad_tienda
<date>      <chr>          <chr>
1 2023-01-01 CDMX             CDMX
2 2023-01-02 León             Puebla
3 2023-01-03 Monterrey        Guadalajara
# A tibble: 1 x 1
  pct_misma_ciudad
              <dbl>
1                12.3
```

Solo el 12.3% de las ventas ocurre en la ciudad del cliente. De nuevo es un efecto de la simulación (clientes y tiendas se asignaron al azar), pero la técnica es la que usarías para medir, por ejemplo, cuántas compras son de turistas o de clientes que viajan. El truco **mean(condición)** da la proporción de filas donde la condición es verdadera.

2.8.3 La trampa de las llaves duplicadas

Un **left_join()** conserva todas las filas de **x...** y puede *agregar* filas. Si una llave aparece dos veces en **y**, cada fila de **x** con esa llave se duplica. A esto se le llama la **explosión de filas**, y es la causa número uno de totales inflados en los reportes.

Ejemplo práctico: La tabla de precios con historial

Compras te pasa una tabla de precios, sin avisarte que el producto 31 tiene dos registros: el precio anterior y el vigente. La unes con las ventas del 1 de enero.

```
1 precios_compras <- tibble(
2   producto_id = c(31, 31, 15),
3   vigencia     = c("2022", "2023", "2023"),
4   precio_lista = c(340, 355, 290)
5 )
6 ventas_ene <- ventas %>%
7   filter(fecha <= as.Date("2023-01-02")) %>%
8   select(fecha, producto_id, total)
9 ventas_ene
10 ventas_ene %>% left_join(precios_compras, by = "producto_id")
```

```
# A tibble: 2 x 3
  fecha      producto_id total
  <date>          <dbl> <dbl>
1 2023-01-01          31 3333.
2 2023-01-02          15 2262
# A tibble: 3 x 5
  fecha      producto_id total vigencia precio_lista
  <date>          <dbl> <dbl> <chr>          <dbl>
1 2023-01-01          31 3333. 2022          340
2 2023-01-01          31 3333. 2023          355
3 2023-01-02          15 2262 2023          290
```

¡Dos ventas se convirtieron en tres! Si ahora sumaras **total**, el 1 de enero contaría doble. **dplyr** no se queja porque, en esta dirección, cada fila de **precios_compras** encaja con una sola venta. Tres defensas:

1. **Revisa la llave antes de unir.** En una dimensión, cada llave debe aparecer una sola vez:

```
1 precios_compras %>%
2   count(producto_id) %>%
3   filter(n > 1)           # llaves repetidas: debería salir vacío
```

```
# A tibble: 1 x 2
  producto_id      n
  <dbl> <int>
1          31      2
```

2. **Declara la relación que esperas.** Con **relationship = "many-to-one"** (muchas ventas por cada producto, un producto por llave) **dplyr** se detiene con un error si la tabla derecha

tiene llaves repetidas, en lugar de duplicar filas en silencio:

```
1 ventas_ene %>%
2   left_join(precios_compras, by = "producto_id",
3             relationship = "many-to-one")
```

```
Error in `left_join()` :
! Each row in `x` must match at most 1 row in `y`.
i Row 1 of `x` matches multiple rows in `y`.
```

3. Compara filas antes y después del *join*, como vimos en el consejo anterior.

Cuando las llaves se repiten en *ambas* tablas (relación “muchos a muchos”), **dplyr** sí te avisa con una advertencia (Detected an unexpected many-to-many relationship). No la ignores: casi siempre significa que te falta una columna en la llave (por ejemplo, unir por producto *y* por fecha de vigencia).

2.8.4 anti_join() para auditar la calidad de los datos

Los *filtering joins* son la herramienta favorita del auditor de datos. Preguntas como “¿hay ventas de productos que no existen?” o “¿qué clientes nunca compraron?” se responden con una línea.

```
1 # 1. ¿Ventas con un producto que no está en el catálogo? (huérfanas)
2 ventas %>% anti_join(productos, by = "producto_id") %>% nrow()
3
4 # 2. ¿Productos del catálogo que no se vendieron en ventas?
5 productos %>% anti_join(ventas, by = "producto_id") %>% nrow()
6
7 # 3. ¿Clientes registrados que no compraron en 2023?
8 clientes_sin_compra <- clientes %>%
9   anti_join(ventas, by = "cliente_id")
10 nrow(clientes_sin_compra)
11
12 # 4. De los que sí compraron (semi_join), ¿cuántos son premium?
13 clientes %>%
14   semi_join(ventas, by = "cliente_id") %>%
15   count(es_premium)
```

```
[1] 0
[1] 0
[1] 28
# A tibble: 2 x 2
  es_premium      n
  <lg1>      <int>
1 FALSE      107
```

2 TRUE

65

La integridad referencial está bien: no hay ventas huérfanas y los 50 productos se vendieron al menos una vez. En cambio, 28 de los 200 clientes registrados (14 %) no compraron nada en 2023: otra lista para mercadotecnia. De los 172 que sí compraron, 65 (38 %) son premium.

Un *join* también sirve para comparar fechas de dos tablas. Una regla de negocio obvia es que nadie puede comprar antes de registrarse como cliente:

```
1 ventas %>%
2   left_join(select(clientes, cliente_id, fecha_registro),
3             by = "cliente_id") %>%
4   filter(fecha < fecha_registro) %>%      # compra antes del registro
5   select(fecha, cliente_id, fecha_registro, total) %>%
6   arrange(fecha_registro - fecha) %>%
7   head(3)
8
9 ventas %>%
10  left_join(select(clientes, cliente_id, fecha_registro),
11            by = "cliente_id") %>%
12  summarise(ventas_antes_registro = sum(fecha < fecha_registro))
```

```
# A tibble: 3 x 4
  fecha      cliente_id fecha_registro total
<date>      <dbl> <date>      <dbl>
1 2023-06-30      162 2023-07-04     505.
2 2023-10-11       71 2023-10-23     362.
3 2023-06-01      128 2023-06-15    1487.
# A tibble: 1 x 1
  ventas_antes_registro
      <int>
1              42
```

Las tres primeras filas son los casos más “cercaños”: compras hechas entre 4 y 14 días antes del registro. En total, 42 ventas (11.5 %) son de clientes que, según el sistema, todavía no existían. En una empresa real las explicaciones típicas son: el cliente compró como invitado y se registró después, o la fecha de registro se sobrescribió al actualizar sus datos. Sea cual sea la causa, es un hallazgo que conviene documentar. Observa el truco de unir solo las columnas que necesitas con `select()` dentro del *join*: la tabla resultante queda más limpia.

2.8.5 Apilar tablas: `bind_rows()` y `bind_cols()`

A veces no necesitas unir por una llave, sino *apilar*: cada mes el sistema te manda un archivo con las ventas de ese mes y quieres un solo archivo del año. `bind_rows()` pone tablas una debajo de otra, emparejando las columnas por nombre.


```

1 # Simulamos tres archivos mensuales
2 enero <- ventas %>% filter(mes == "2023-01")
3 febrero <- ventas %>% filter(mes == "2023-02")
4 marzo <- ventas %>% filter(mes == "2023-03") %>%
5   select(-dia_semana) # a marzo le falta una columna
6
7 primer_trimestre <- bind_rows(enero, febrero, marzo,
8                             .id = "archivo") # de qué tabla vino
9 primer_trimestre %>% count(archivo)
10 primer_trimestre %>% filter(is.na(dia_semana)) %>% nrow()

```

```

# A tibble: 3 x 2
  archivo     n
  <chr>   <int>
1 1         31
2 2         28
3 3         31
[1] 31

```

El argumento `.id` agrega una columna con el número de tabla de origen (con una lista con nombres, `list(ene = enero, ...)`, pondría el nombre). Como a marzo le faltaba `dia_semana`, `bind_rows()` no falla: llena esa columna con **NA** en sus 31 filas. Si en lugar de faltar una columna tuviera otro *tipo* (texto en un archivo y número en otro), sí fallaría; lo verás en la sección 2.13.

`bind_cols()` pega tablas lado a lado *por posición*. Úsala con mucho cuidado: no verifica que la fila 1 de una tabla corresponda a la fila 1 de la otra. Si existe una llave, usa un *join*.

Consejo

Para leer y apilar de una vez todos los CSV de una carpeta, **readr** acepta un vector de rutas: `read_csv(list.files("datos/mensuales", full.names = TRUE), id = "archivo")`. El argumento `id` guarda la ruta de origen de cada fila.

Hazlo tú

- Une `transacciones` con `tiendas` y calcula los ingresos por `zona`.
- Une `ventas` con `productos` y encuentra las 5 ventas donde `precio_unitario` está más por debajo del `costo`.
- Usa `anti_join()` para contar cuántos clientes no aparecen en `transacciones`.

2.9 Cambiar la forma de las tablas con tidyr

dplyr transforma el *contenido* de las tablas; **tidyr** cambia su *forma*. Sus dos funciones centrales son inversas una de la otra (figura 2.4):

- `pivot_wider()`: de largo a ancho. Los valores de una columna se convierten en nombres de

columnas. Es lo que hace una *tabla dinámica* de Excel.

- `pivot_longer()`: de ancho a largo. Varias columnas se “derriten” en dos: una con el nombre y otra con el valor. Es lo que necesitas para ordenar un reporte de Excel.

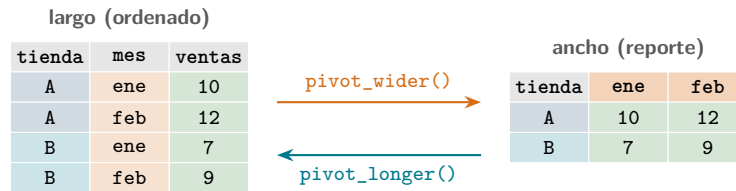


Figura 2.4. `pivot_wider()` convierte los valores de `mes` en columnas; `pivot_longer()` hace el camino inverso.

2.9.1 `pivot_wider()`: tu tabla dinámica

Ejemplo práctico: Ventas por trimestre y categoría

La directora quiere una tabla para leer de un vistazo: una fila por categoría y una columna por trimestre, con los ingresos en miles de pesos.

```
1 tabla_trim_cat <- ventas_prod %>% # ventas + productos
2   group_by(categoria, trimestre) %>%
3   summarise(ingresos = sum(total) / 1000, .groups = "drop") %>%
4   pivot_wider(
5     names_from = trimestre, # de dónde salen los nombres de columna
6     values_from = ingresos # de dónde salen los valores
7   ) %>%
8   mutate(across(where(is.numeric), ~ round(.x, 1)))
9 tabla_trim_cat
```

```
# A tibble: 10 x 5
  categoria      Q1      Q2      Q3      Q4
  <chr>      <dbl> <dbl> <dbl> <dbl>
1 Alimentos  18.6   8.6   8     10.9
2 Automotriz  8.4   12.6  10.9  14.9
3 Belleza    19.9  25.5  32.4  27
4 Deportes   14.4   9.6   8.6  10.8
5 Electrónica 21.2  18.7  24.1  19.9
6 Hogar       7.7   9.9  16.5   7.6
7 Juguetes   15.1  15.6  23.7  18.4
8 Libros      3.3   9.1   5.1   8
9 Mascotas   14.2   4.1  NA     8.2
10 Ropa       3.5   6.9   5.1   6.7
```

Primero resumimos en formato largo (una fila por categoría y trimestre) y *al final* lo pasamos a ancho para presentarlo. Esta es la regla de oro: **analiza en largo, presenta en ancho**. La tabla

deja ver patrones que en formato largo costaría encontrar: Belleza crece hasta su pico en el Q3 (\$32.4 mil), Alimentos se desploma después del Q1 (de \$18.6 mil a \$8.6 mil) y Mascotas tiene un hueco: **NA** en el Q3.

Ese **NA** aparece porque la combinación no existe: Mascotas no tuvo ninguna venta en el Q3, así que no había fila que pivotar. En un reporte de ventas lo correcto es mostrar 0; para eso está **values_fill**:

```
1 ventas_prod %>%
2   count(categoria, mes) %>%           # número de ventas
3   pivot_wider(names_from = mes, values_from = n,
4               values_fill = 0) %>%    # 0 donde no hubo ventas
5   select(categoria, `2023-01`:`2023-06`)
```

```
# A tibble: 10 x 7
  categoria `2023-01` `2023-02` `2023-03` `2023-04` `2023-05`
  <chr>      <int>      <int>      <int>      <int>      <int>
1 Alimentos      4          4          5          2          3
2 Automotriz      3          0          3          6          4
3 Belleza        5          2          4          5          6
4 Deportes       4          3          2          3          3
5 Electrónica     3          5          3          3          4
6 Hogar          2          4          1          3          2
7 Juguetes       3          5          7          1          4
8 Libros         1          1          2          2          2
9 Mascotas       4          2          3          4          2
10 Ropa          2          2          1          1          1
# i 1 more variable: `2023-06` <int>
```

Observa las comillas invertidas (*backticks*) en **‘2023-01’**: los nombres de columna que empiezan con un número o tienen guiones o espacios no son nombres “válidos” en R, y hay que escribirlos entre **‘**.

2.9.2 pivot_longer(): ordenar un reporte de Excel

Ejemplo práctico: El presupuesto que llegó en Excel

Finanzas te manda el presupuesto de ventas del primer semestre por tienda, con un mes por columna (el formato “bonito” de la tabla 2.1). Para compararlo contra las ventas reales necesitas ordenarlo.

```
1 presupuesto <- tibble(
2   tienda_id = c(1, 2, 3),
3   Ene = c(5000, 4500, 4800), Feb = c(4800, 4300, 4600),
4   Mar = c(5200, 4700, 5000), Abr = c(5100, 4600, 4900),
5   May = c(5300, 4900, 5100), Jun = c(5000, 4400, 4700)
```

```

6 )
7 presupuesto_largo <- presupuesto %>%
8   pivot_longer(
9     cols      = Ene:Jun,          # columnas a "derretir"
10    names_to   = "mes_texto",      # los nombres van a esta columna
11    values_to  = "presupuesto"    # los valores van a esta otra
12  )
13 presupuesto_largo

```

```

# A tibble: 18 x 3
  tienda_id mes_texto presupuesto
    <dbl>   <chr>         <dbl>
1         1 Ene           5000
2         1 Feb           4800
3         1 Mar           5200
4         1 Abr           5100
5         1 May           5300
6         1 Jun           5000
...

```

De 3 filas $\times$ 7 columnas pasamos a 18 filas $\times$ 3 columnas: una fila por tienda y mes. Ahora podemos convertir el mes de texto a su formato "2023-01" y unirlo con las ventas reales para calcular el cumplimiento:

```

1 meses_abrev <- c("Ene", "Feb", "Mar", "Abr", "May", "Jun")
2
3 # Ventas reales por tienda y mes
4 real_tienda_mes <- ventas %>%
5   group_by(tienda_id, mes) %>%
6   summarise(real = sum(total), .groups = "drop")
7
8 cumplimiento <- presupuesto_largo %>%
9   mutate(mes = str_c("2023-", str_pad(match(mes_texto, meses_abrev),
10                                        2, pad = "0")) %>%
11     left_join(real_tienda_mes,
12               by = c("tienda_id", "mes")) %>% # llave de 2 columnas
13   mutate(cumplimiento_pct = round(real / presupuesto * 100))
14
15 cumplimiento %>%
16   select(tienda_id, mes, presupuesto, real, cumplimiento_pct) %>%
17   filter(tienda_id == 1)

```

```

# A tibble: 6 x 5
  tienda_id mes      presupuesto real cumplimiento_pct
    <dbl>   <chr>         <dbl> <dbl>         <dbl>
1         1 2023-01           5000  7280.         146

```

2	1	2023-02	4800	6355.	132
3	1	2023-03	5200	3076.	59
4	1	2023-04	5100	2182.	43
5	1	2023-05	5300	3043.	57
6	1	2023-06	5000	802.	16

La tienda 1 arrancó el año por encima del presupuesto (146 % en enero, 132 % en febrero) y luego se desplomó hasta 16 % en junio. Es la misma tienda que vimos fuerte en el Q1 y ausente del podio a mitad de año: ahora tenemos un número para llevar a la junta. Nota dos detalles: `match()` devuelve la posición de cada mes en el vector `meses_abrev` (“Mar” → 3), y el `join` usa una llave de *dos* columnas, `c("tienda_id", "mes")`: cada fila debe coincidir en ambas.

2.9.3 Separar y unir columnas

`separate_wider_delim()` (que ya usaste con los nombres de clientes) parte una columna en varias; `unite()` hace lo contrario.

```
1 codigos <- tibble(clave_tienda = c("CDMX-Centro-001", "GDL-Norte-002",
2                               "MTY-Sur-003"),
3                     ventas = c(43899, 54846, 54066))
4 codigos_sep <- codigos %>%
5   separate_wider_delim(clave_tienda, delim = "-",
6                       names = c("ciudad", "zona", "numero"))
7 codigos_sep
8
9 codigos_sep %>%
10  unite("etiqueta", ciudad, zona, sep = " / ") # une en una columna
```

```
# A tibble: 3 x 4
  ciudad zona  numero ventas
  <chr>  <chr>  <chr>   <dbl>
1 CDMX   Centro 001     43899
2 GDL    Norte  002     54846
3 MTY    Sur   003     54066
# A tibble: 3 x 3
  etiqueta      numero ventas
  <chr>         <chr>   <dbl>
1 CDMX / Centro 001     43899
2 GDL / Norte  002     54846
3 MTY / Sur    003     54066
```

2.9.4 Datos faltantes: `fill()`, `replace_na()` y `drop_na()`

Ejemplo práctico: El reporte con celdas combinadas

En Excel, alguien combinó las celdas de la región: el nombre aparece solo en la primera fila de cada bloque. Al leer el archivo, las demás filas quedan vacías (NA). Además hay devoluciones sin capturar.

```
1 reporte_regional <- tibble(
2   region      = c("Norte", NA, NA, "Centro", NA, "Sur"),
3   tienda      = c("Monterrey", "Tijuana", "Guadalajara",
4                   "CDMX", "Puebla", "Mérida"),
5   ventas      = c(54066, 70155, 54846, 43899, 53611, NA),
6   devoluciones = c(1200, NA, 800, NA, 650, 300)
7 )
8 reporte_regional %>%
9   fill(region, .direction = "down")    # copia el valor hacia abajo
```

```
# A tibble: 6 x 4
  region tienda      ventas devoluciones
  <chr>  <chr>      <dbl>      <dbl>
1 Norte Monterrey  54066      1200
2 Norte Tijuana    70155         NA
3 Norte Guadalajara 54846         800
4 Centro CDMX      43899         NA
5 Centro Puebla    53611         650
6 Sur   Mérida      NA           300
```

`fill()` copia el último valor no vacío hacia abajo, exactamente como harías a mano en Excel. Para los demás faltantes hay que decidir según el negocio: una devolución no capturada probablemente es 0, pero una venta faltante *no* es 0 (la tienda sí vendió; solo no tenemos el dato), así que es mejor excluirla del análisis y reportarlo.

```
1 reporte_limpio <- reporte_regional %>%
2   fill(region) %>%
3   replace_na(list(devoluciones = 0)) %>%    # NA -> 0 solo aquí
4   drop_na(ventas)                          # quita filas sin ventas
5 reporte_limpio
```

```
# A tibble: 5 x 4
  region tienda      ventas devoluciones
  <chr>  <chr>      <dbl>      <dbl>
1 Norte Monterrey  54066      1200
2 Norte Tijuana    70155         0
3 Norte Guadalajara 54846         800
```

4 Centro CDMX	43899	0
5 Centro Puebla	53611	650

`replace_na()` recibe una lista con el valor de reemplazo para cada columna, y `drop_na(ventas)` elimina las filas con `NA` en `ventas` (sin argumentos, eliminaría las filas con `NA` en *cualquier* columna, lo cual suele ser demasiado agresivo).

2.9.5 Rellenar periodos sin ventas: `complete()`

Recuerda la advertencia sobre `lag()`: si falta un mes, el crecimiento “mes contra mes” se calcula contra el mes equivocado. Y los meses sin ventas no aparecen en un resumen, simplemente porque no hay filas.

Ejemplo práctico: ¿Hubo tiendas con meses en cero?

Con 10 tiendas y 12 meses deberíamos tener 120 combinaciones de tienda y mes. ¿Las tenemos?

```
1 ventas_tienda_mes <- ventas %>%
2   group_by(tienda_id, mes) %>%
3   summarise(ingresos = sum(total), .groups = "drop")
4 nrow(ventas_tienda_mes)           # ¿120?
5
6 ventas_tienda_mes_completa <- ventas_tienda_mes %>%
7   complete(tienda_id, mes,          # todas las combinaciones
8            fill = list(ingresos = 0)) # con 0 donde no hubo ventas
9 nrow(ventas_tienda_mes_completa)
10
11 ventas_tienda_mes_completa %>% filter(ingresos == 0)
```

```
[1] 118
[1] 120
# A tibble: 2 x 3
  tienda_id mes      ingresos
  <dbl> <chr>      <dbl>
1      1 2023-08          0
2      4 2023-06          0
```

Solo había 118 combinaciones: la tienda 1 no vendió nada en agosto y la tienda 4 nada en junio. `complete()` crea las filas faltantes con todas las combinaciones de los valores existentes de `tienda_id` y `mes`, y `fill` les pone 0. Ahora sí, un `lag()` por tienda compararía cada mes contra el anterior real. (Si un mes no aparece en *ninguna* tienda, `complete()` no puede inventarlo; en ese caso se le da la lista completa, por ejemplo `complete(mes = meses_del_anio)`.)

 Hazlo tú

Con `transacciones`, construye una tabla dinámica con una fila por `metodo_pago` y una columna por `trimestre`, con el número de transacciones. Después regrésala a formato largo con `pivot_longer()` y comprueba que obtienes 16 filas.

2.10 Limpieza de datos: un caso real paso a paso

Los datos del curso están bastante limpios. Los de tu empresa no lo estarán. Imagina que el área de sucursales te manda un registro de ventas capturado a mano en Excel por distintas personas. Lo reproducimos con todos los problemas típicos:

```
1 ventas_sucias <- tibble(
2   folio      = c(101, 102, 102, 103, 104, 105, 106, 107, 108, 109),
3   fecha      = c("2023-03-01", "02/03/2023", "02/03/2023", "2023-03-03",
4                 "03/03/2023", "2023-03-04", "05/03/2023", "2023-03-06",
5                 NA, "2023-03-07"),
6   ciudad     = c("CDMX", "cdmx", "cdmx", "CDMX ", "Cd. de México",
7                 " guadalajara", "Guadalajara", "GDL", "Monterrey",
8                 "monterrey"),
9   categoria  = c("Electrónica", "electronica", "electronica", "Ropa",
10                 "ROPA", "Hogar", "hogar ", "Electrónica", "Ropa", "Hogar"),
11  monto      = c("$1,234.50", "980", "980", "$2,100.00", "450.5",
12                 "$3,050", "1,120.00", "$899.99", "560", "$1,780.25"),
13  cantidad   = c(2, 1, 1, 3, 1, 4, 2, -1, 1, 2)
14 )
15 ventas_sucias
```

```
# A tibble: 10 x 6
  folio fecha      ciudad      categoria      monto      cantidad
  <dbl> <chr>      <chr>      <chr>      <chr>      <dbl>
1  101 2023-03-01 "CDMX"      "Electrónica" $1,234.50      2
2  102 02/03/2023 "cdmx"      "electronica" 980            1
3  102 02/03/2023 "cdmx"      "electronica" 980            1
4  103 2023-03-03 "CDMX "     "Ropa"        $2,100.00      3
5  104 03/03/2023 "Cd. de México" "ROPA"        450.5          1
6  105 2023-03-04 " guadalajara" "Hogar"       $3,050          4
7  106 05/03/2023 "Guadalajara" "hogar "     1,120.00      2
8  107 2023-03-06 "GDL"       "Electrónica" $899.99       -1
9  108 <NA>      "Monterrey"  "Ropa"        560            1
10 109 2023-03-07 "monterrey"  "Hogar"       $1,780.25      2
```

Haz un diagnóstico antes de corregir nada. Los problemas son:

1. El folio 102 aparece dos veces, idéntico: un **duplicado exacto** (alguien pegó la fila dos veces).
2. `fecha` es texto con **dos formatos** mezclados (AAAA-MM-DD y DD/MM/AAAA) y un faltante.

3. **ciudad** tiene **espacios sobrantes, mayúsculas inconsistentes y sinónimos**: “CDMX”, “cdmx”, “Cd. de México”; “GDL” y “Guadalajara”.
4. **categoría** tiene **errores de escritura**: sin acento, en mayúsculas, con espacios.
5. **monto** es **texto** con signo de pesos y comas de miles: no se puede sumar.
6. Hay una **cantidad negativa**, imposible en una venta.

2.10.1 Paso 1: duplicados exactos

```
1 ventas_sucias %>% count(folio) %>% filter(n > 1) # ¿folios repetidos?
2 paso1 <- ventas_sucias %>% distinct()           # quita filas idénticas
3 nrow(ventas_sucias); nrow(paso1)
```

```
# A tibble: 1 x 2
  folio     n
  <dbl> <int>
1   102     2
[1] 10
[1] 9
```

`distinct()` sin argumentos elimina las filas que son idénticas en *todas* las columnas. Si dos filas tuvieran el mismo folio pero algún dato distinto, no las quitaría: ese caso requiere investigar cuál es la correcta. (`distinct(folio, .keep_all = TRUE)` conservaría la primera aparición de cada folio.)

2.10.2 Paso 2: texto (ciudades y categorías)

```
1 paso2 <- paso1 %>%
2   mutate(
3     ciudad = str_squish(str_to_upper(ciudad)),      # " cdmx" -> "CDMX"
4     ciudad = case_when(                             # unificar sinónimos
5       ciudad %in% c("CDMX", "CD. DE MÉXICO") ~ "CDMX",
6       ciudad %in% c("GUADALAJARA", "GDL") ~ "Guadalajara",
7       ciudad == "MONTERREY" ~ "Monterrey",
8       .default = ciudad                             # por si aparece otra
9     ),
10    categoria = str_to_title(str_squish(categoria)),
11    categoria = recode(categoria, "Electronica" = "Electrónica")
12  )
13 paso2 %>% count(ciudad)
14 paso2 %>% count(categoria)
```

```
# A tibble: 3 x 2
  ciudad      n
  <chr>      <int>
1 CDMX        4
2 Guadalajara 3
3 Monterrey   2
# A tibble: 3 x 2
  categoria      n
  <chr>      <int>
1 Electrónica    3
2 Hogar          3
3 Ropa           3
```

El patrón es siempre el mismo: primero *normalizar* (quitar espacios y unificar mayúsculas) para que las variantes triviales colapsen, y después *mapear* a mano los sinónimos que quedan con `case_when()` o `recode()` (esta última cambia valores específicos con la sintaxis `"viejo" = "nuevo"`). Terminamos con 3 ciudades y 3 categorías, como debe ser.

2.10.3 Paso 3: montos y fechas

```
1 paso3 <- paso2 %>%
2   mutate(
3     monto = parse_number(monto),      # "$1,234.50" -> 1234.5
4     fecha = parse_date_time(fecha,    # prueba ambos formatos
5                               orders = c("ymd", "dmy")),
6     fecha = as_date(fecha)           # de fecha-hora a fecha
7   )
8 paso3 %>% select(folio, fecha, monto)
```

```
# A tibble: 9 x 3
  folio fecha      monto
  <dbl> <date>      <dbl>
1  101 2023-03-01  1234.
2  102 2023-03-02   980
3  103 2023-03-03  2100
4  104 2023-03-03   450.
5  105 2023-03-04  3050
6  106 2023-03-05  1120
7  107 2023-03-06   900.
8  108 NA          560
9  109 2023-03-07  1780.
```

`parse_number()` de **readr** extrae el número de un texto, ignorando símbolos de moneda y separadores de miles. Para las fechas, `parse_date_time()` de **lubridate** prueba cada formato de la lista `orders` hasta que uno funcione, y `as_date()` descarta la hora. El faltante sigue siendo **NA**: no hay forma de

inventarlo.

2.10.4 Paso 4: reglas de negocio y valores imposibles

```

1 # Revisar antes de borrar: ¿qué filas violan las reglas?
2 paso3 %>% filter(cantidad <= 0 | is.na(fecha))
3
4 ventas_limpias <- paso3 %>%
5   filter(cantidad > 0,                # cantidades imposibles fuera
6         !is.na(fecha)) %>%          # sin fecha no se puede asignar mes
7   mutate(precio_unitario = monto / cantidad)
8
9 ventas_limpias %>%
10  group_by(ciudad) %>%
11  summarise(ventas = n(), ingresos = sum(monto), .groups = "drop")

```

```

# A tibble: 2 x 6
  folio fecha      ciudad      categoria  monto cantidad
<dbl> <date>    <chr>      <chr>      <dbl>    <dbl>
1   107 2023-03-06 Guadalajara Electrónica  900.      -1
2   108 NA      Monterrey   Ropa        560       1
# A tibble: 3 x 3
  ciudad      ventas ingresos
<chr>      <int>    <dbl>
1 CDMX         4    4765
2 Guadalajara  2    4170
3 Monterrey    1    1780.

```

De 10 filas originales quedaron 7 confiables: 1 duplicado, 1 cantidad negativa y 1 venta sin fecha salieron. Ahora sí, el resumen por ciudad tiene sentido. En un proyecto real *no* borrarías esas filas en silencio: las guardarías en una tabla de “rechazos” para devolverla al área que capturó los datos.

Consejo

Escribe tu limpieza como un solo *pipeline* en un script (no a mano en Excel). El mes que entra, cuando llegue el archivo nuevo con los mismos problemas, solo vuelves a ejecutar el script. Y documenta cada decisión con un comentario: “los NA en devoluciones se tratan como 0 porque...”.

Tabla 2.7. Lista de verificación de calidad de datos.

Revisión	Pregunta	Herramienta
Dimensiones	¿Tiene las filas y columnas que esperaba?	<code>glimpse()</code> , <code>nrow()</code>
Tipos	¿Números como número, fechas como fecha?	<code>glimpse()</code> , <code>parse_number()</code>
Duplicados	¿Filas o llaves repetidas?	<code>distinct()</code> , <code>count(llave)%>% filter(n > 1)</code>
Faltantes	¿Cuántos NA hay por columna?	<code>summarise(across(everything(), ~ sum(is.na(.x))))</code>
Categorías	¿Variantes de un mismo valor?	<code>count(columna)</code> , <code>str_squish()</code>
Rangos	¿Valores imposibles (negativos, futuros)?	<code>summary()</code> , <code>filter()</code>
Integridad	¿Llaves sin pareja en la otra tabla?	<code>anti_join()</code>
Reglas de negocio	¿Precio bajo el costo, compra antes del registro?	<code>filter()</code> tras un <code>join</code>
Totales	¿Cuadrán contra el sistema de origen?	<code>summarise(sum(...))</code>

** Hazlo tú**

Aplica la revisión de faltantes de la tabla 2.7 a `ventas_sucias` y a `clientes`: `summarise(across(everything(), ~ sum(is.na(.x))))`. ¿Qué columnas tienen faltantes?

2.11 Caso práctico integral: reporte mensual de ventas 2023

Regresemos al mensaje de la directora comercial del inicio del módulo. Vas a construir, con todo lo aprendido, un reporte completo que se exporta a un archivo de Excel con varias hojas. El código de esta sección está pensado para que lo copies en un script propio (por ejemplo `reporte_mensual.R`) y lo vuelvas a ejecutar cada mes.

En la empresa: Qué se pide

1. KPIs globales del año: ingresos, ventas, unidades, ticket promedio, clientes únicos, descuento promedio y utilidad bruta.
2. Ventas por mes con crecimiento contra el mes anterior y acumulado.
3. Ventas por categoría con participación y ranking.
4. Los cinco mejores clientes.
5. El ranking de tiendas y el desempeño de los vendedores.
6. Una tabla dinámica de mes × categoría.
7. Un apartado de calidad de datos con lo que haya que aclarar.

2.11.1 Paso 1: la tabla maestra

En BI se acostumbra construir una **tabla maestra** (o tabla “plana”): los hechos unidos con todas las dimensiones que se van a usar. Unimos solo las columnas necesarias y les damos nombres claros para evitar choques (**nombre** existe en clientes y en empleados; **ciudad**, en clientes y en tiendas).

Reporte mensual: tabla maestra

```

1  maestra <- ventas %>%
2    left_join(select(productos, producto_id, nombre_producto, categoria,
3                     precio_catalogo, costo),
4              by = "producto_id") %>%
5    left_join(select(clientes, cliente_id, cliente = nombre,
6                     ciudad_cliente = ciudad, es_premium),
7              by = "cliente_id") %>%
8    left_join(select(tiendas, tienda_id, nombre_tienda,
9                     ciudad_tienda = ciudad, zona),
10             by = "tienda_id") %>%
11    left_join(select(empleados, empleado_id, vendedor = nombre),
12              join_by(vendedor_id == empleado_id)) %>%
13    mutate(
14      mes_fecha   = floor_date(fecha, "month"),
15      mes_nombre  = month(fecha, label = TRUE),
16      costo_total = costo * cantidad,          # costo de lo vendido
17      utilidad    = total - costo_total        # utilidad bruta
18    )
19
20 # Control: los joins no deben agregar ni quitar filas
21 stopifnot(nrow(maestra) == nrow(ventas))
22 dim(maestra)

```

```
[1] 365 29
```

stopifnot() detiene el script con un error si la condición es falsa. Es un **control automático**: si algún mes el catálogo trae una llave duplicada, el reporte no se generará con totales inflados, sino que se detendrá y te avisará. Dentro de **select()** renombramos al vuelo (**cliente = nombre**).

2.11.2 Paso 2: KPIs globales

Reporte mensual: KPIs

```

1  kpis <- maestra %>%
2    summarise(
3      ingresos      = sum(total),
4      ventas        = n(),
5      unidades      = sum(cantidad),
6      ticket_promedio = mean(total),

```

```

7   clientes_unicos = n_distinct(cliente_id),
8   descuento_prom  = mean(descuento_pct),
9   utilidad_bruta  = sum(utilidad),
10  margen_bruto_pct = utilidad_bruta / ingresos * 100
11  ) %>%
12  mutate(across(everything(), ~ round(.x, 2)))
13  glimpse(kpis)

```

```

Rows: 1
Columns: 8
$ ingresos      <dbl> 513916.6
$ ventas        <dbl> 365
$ unidades      <dbl> 2057
$ ticket_promedio <dbl> 1407.99
$ clientes_unicos <dbl> 172
$ descuento_prom <dbl> 4.78
$ utilidad_bruta <dbl> 150570.9
$ margen_bruto_pct <dbl> 29.3

```

En 2023 la empresa vendió \$513,916.60 en 365 ventas y 2,057 unidades, con un ticket promedio de \$1,407.99, 172 clientes distintos y un descuento promedio de 4.78 %. La utilidad bruta fue de \$150,570.90, un margen de 29.3 %. **Supuesto importante:** la utilidad se calculó con el **costo** unitario del catálogo multiplicado por la cantidad; si el costo cambió durante el año, esta cifra es aproximada. Declara siempre tus supuestos en el reporte.

2.11.3 Paso 3: tendencia mensual

Reporte mensual: tendencia

```

1  por_mes <- maestra %>%
2    group_by(mes_fecha, mes_nombre) %>%
3    summarise(ingresos = sum(total), ventas = n(),
4              clientes = n_distinct(cliente_id), .groups = "drop") %>%
5    arrange(mes_fecha) %>%
6    mutate(
7      crecimiento_mom = round((ingresos / lag(ingresos) - 1) * 100, 1),
8      acumulado_ytd   = cumsum(ingresos),
9      ingresos        = round(ingresos, 2)
10   )
11  por_mes %>% select(-mes_fecha)

```

```

# A tibble: 12 x 6
  mes_nombre ingresos ventas clientes crecimiento_mom acumulado_ytd
  <ord>         <dbl> <int>   <int>         <dbl>         <dbl>
1 ene           46884.    31     29           NA           46884.

```

2 feb	42656.	28	26	-9	89540.
3 mar	36734.	31	30	-13.9	126274.
4 abr	34738.	30	27	-5.4	161012.
5 may	48108.	31	29	38.5	209120.
6 jun	37871.	30	26	-21.3	246991.
7 jul	54056.	31	28	42.7	301047.
8 ago	41421.	31	29	-23.4	342468.
9 sep	38950.	30	29	-6	381418.
10 oct	36584.	31	27	-6.1	418003.
11 nov	50398.	30	29	37.8	468400.
12 dic	45516.	31	26	-9.7	513917.

El año tuvo tres picos (mayo, julio y noviembre, con crecimientos de 38 a 43 % contra el mes anterior) seguidos de caídas: un patrón de “serrucho” que merece investigarse (¿campanas?, ¿quincenas?, ¿promociones que adelantan compras?). El número de clientes distintos por mes es estable (entre 26 y 30), así que las variaciones vienen del monto de cada compra, no de cuántos clientes compran.

2.11.4 Paso 4: categorías

Reporte mensual: categorías

```

1 por_categoria <- maestra %>%
2   group_by(categoria) %>%
3   summarise(ingresos = sum(total), unidades = sum(cantidad),
4             utilidad = sum(utilidad), .groups = "drop") %>%
5   mutate(
6     participacion_pct = round(ingresos / sum(ingresos) * 100, 1),
7     margen_pct       = round(utilidad / ingresos * 100, 1),
8     ranking           = min_rank(desc(ingresos))
9   ) %>%
10  arrange(ranking)
11 por_categoria %>% select(ranking, categoria, ingresos,
12                          participacion_pct, margen_pct)

```

```

# A tibble: 10 x 5
  ranking categoria  ingresos participacion_pct margen_pct
  <int> <chr>          <dbl>          <dbl>          <dbl>
1     1 1 Belleza      104840.         20.4           5
2     2 2 Electrónica  83949.         16.3          49.7
3     3 3 Juguetes      72754.         14.2          38.2
4     4 4 Automotriz    46865.          9.1          35.5
5     5 5 Alimentos    46178.          9           42.4
6     6 6 Deportes     43319.          8.4          47.5
7     7 7 Hogar       41729.          8.1           2.9
8     8 8 Mascotas     26518.          5.2          42.6
9     9 9 Libros       25537.          5           16.3

```

10	10 Ropa	22227.	4.3	10.9
----	---------	--------	-----	------

Este es el hallazgo más importante del reporte: **Belleza es la categoría número uno en ingresos (20.4 %), pero su margen es de apenas 5 %**. Hogar está peor (2.9 %). En cambio, Electrónica y Deportes dejan márgenes cercanos al 50 %. Vender más Belleza casi no deja utilidad; si la dirección quiere crecer la utilidad, debe empujar Electrónica y Deportes o revisar los precios de Belleza. Sin el *join* con el catálogo (que trae el costo) este hallazgo sería invisible.

2.11.5 Paso 5: clientes, tiendas y vendedores

Reporte mensual: top clientes y tiendas

```

1 top_clientes <- maestra %>%
2   group_by(cliente_id, cliente, ciudad_cliente, es_premium) %>%
3   summarise(compras = n(), ingresos = sum(total),
4             ultima_compra = max(fecha), .groups = "drop") %>%
5   slice_max(ingresos, n = 5)
6 top_clientes %>% select(cliente, es_premium, compras, ingresos,
7                          ultima_compra)
8
9 ranking_tiendas <- maestra %>%
10  group_by(nombre_tienda, ciudad_tienda) %>%
11  summarise(ingresos = sum(total), ventas = n(),
12            ticket_promedio = mean(total), .groups = "drop") %>%
13  mutate(ranking = min_rank(desc(ingresos)),
14         vs_promedio_pct = round((ingresos / mean(ingresos) - 1) * 100,
15                                 1)) %>%
16  arrange(ranking)
17 ranking_tiendas %>% select(ranking, nombre_tienda, ingresos, ventas,
18                           vs_promedio_pct)

```

```

# A tibble: 5 x 5
  cliente      es_premium compras ingresos ultima_compra
  <chr>        <lgl>         <int>   <dbl> <date>
1 Carlos García TRUE           5    12875. 2023-11-09
2 Pedro Díaz   FALSE          5    11691. 2023-10-20
3 Diego Rivera FALSE          5    10202. 2023-12-28
4 Diego Flores FALSE          3     9180. 2023-11-05
5 Alberto González TRUE         4     8577. 2023-12-09

# A tibble: 10 x 5
  ranking nombre_tienda      ingresos ventas vs_promedio_pct
  <int> <chr>              <dbl>   <int>      <dbl>
1     1 1 Sucursal Outlet    70155.    47      36.5
2     2 2 Sucursal Premium   61369.    46      19.4
3     3 3 Sucursal Plaza B   55290.    37       7.6
4     4 4 Sucursal Norte     54846.    38       6.7

```


5	5 Sucursal Sur	54066.	37	5.2
6	6 Sucursal Plaza A	54050.	34	5.2
7	7 Sucursal Oeste	53611.	37	4.3
8	8 Sucursal Centro	43899.	32	-14.6
9	9 Sucursal Express	42171.	34	-17.9
10	10 Sucursal Este	24459.	23	-52.4

El mejor cliente, Carlos García (premium, CDMX), compró 5 veces por \$12,875. Tres de los cinco mejores clientes *no* son premium: son candidatos naturales para ofrecerles el programa. En tiendas, la Sucursal Outlet de Tijuana supera al promedio en 36.5 % y la Sucursal Este (CDMX) está 52.4 % por debajo: es la respuesta a “¿qué tienda va peor?”. La columna `vs_promedio_pct` usa `mean(ingresos)` dentro de `mutate()` sobre la tabla ya resumida, es decir, el promedio de las 10 tiendas.

Reporte mensual: vendedores

```
1 vendedores <- maestra %>%
2   group_by(vendedor_id, vendedor) %>%
3   summarise(ventas = n(), ingresos = sum(total),
4             ticket_promedio = mean(total),
5             tiendas_distintas = n_distinct(tienda_id), .groups = "drop") %>%
6   mutate(ranking = min_rank(desc(ingresos))) %>%
7   arrange(ranking)
8 bind_rows(head(vendedores, 3), tail(vendedores, 2)) %>% # mejores/peores
9   select(ranking, vendedor, ventas, ingresos, ticket_promedio)
```

```
# A tibble: 5 x 5
  ranking vendedor      ventas ingresos ticket_promedio
  <int> <chr>          <int>    <dbl>         <dbl>
1      1 1 Isabel Martínez    22  35744.         1625.
2      2 2 Juan Sánchez       23  34579.         1503.
3      3 3 Sofia Pérez        18  29500.         1639.
4     24 24 Ricardo González     9   9968.          1108.
5     25 25 Pedro Morales      7   7534.          1076.
```

La mejor vendedora, Isabel Martínez, generó \$35,744 en 22 ventas; el último lugar, Pedro Morales, \$7,534 en 7 ventas. La diferencia está sobre todo en el *número* de ventas, no en el ticket. Antes de sacar conclusiones sobre el desempeño, recuerda el hallazgo de la sección 2.8: la mayoría de estos “vendedores” ni siquiera pertenece al departamento de Ventas.

2.11.6 Paso 6: tabla dinámica mes × categoría

Reporte mensual: tabla dinámica

```
1 dinamica <- maestra %>%
2   group_by(mes_nombre, categoria) %>%
3   summarise(ingresos = round(sum(total)), .groups = "drop") %>%
4   pivot_wider(names_from = categoria, values_from = ingresos,
5               values_fill = 0) %>%
6   mutate(Total = rowSums(across(where(is.numeric)))) # total por mes
7 dinamica %>% select(mes_nombre, Belleza, Electrónica, Juguetes, Total)
```

```
# A tibble: 12 x 5
  mes_nombre Belleza Electrónica Juguetes Total
  <ord>      <dbl>      <dbl>    <dbl> <dbl>
1 ene         8606         2932     2324 46883
2 feb         5607        11414     3506 42657
3 mar         5652         6879     9248 36734
4 abr         8240         3554         584 34739
5 may         8647         7561     9302 48107
6 jun         8608         7621     5744 37871
7 jul         8829        13723     3268 54056
8 ago        17165         2512    10943 41422
9 sep         6438         7853     9471 38950
10 oct        12893         2138     2594 36584
11 nov        10045         8131    10867 50399
12 dic         4112         9631     4903 45516
```

`rowSums(across(...))` suma, fila por fila, todas las columnas numéricas: es la columna “Total general” de una tabla dinámica. En la consola mostramos solo tres categorías, pero la hoja de Excel llevará las diez.

2.11.7 Paso 7: hallazgos de calidad de datos

Un buen reporte no solo presenta números: también advierte qué tan confiables son. Calculemos los hallazgos que fueron apareciendo en el módulo.

Reporte mensual: calidad de datos

```
1 calidad <- tibble(
2   revision = c(
3     "Productos con costo > precio de catálogo",
4     "Ventas con precio unitario < costo",
5     "Ventas con precio +/-10% vs. catálogo",
6     "Ventas antes del registro del cliente",
7     "Vendedores fuera del depto. de Ventas"
8   ),
```

```

9   casos = c(
10     sum(productos$costo > productos$precio_catalogo),
11     sum(maestra$precio_unitario < maestra$costo),
12     sum(abs(maestra$precio_unitario / maestra$precio_catalogo - 1) > 0.10),
13     maestra %>%
14       left_join(select(clientes, cliente_id, fecha_registro),
15                 by = "cliente_id") %>%
16       filter(fecha < fecha_registro) %>% nrow(),
17     empleados %>%
18       filter(empledo_id %in% ventas$vendedor_id,
19              departamento != "Ventas") %>% nrow()
20   ),
21   de_un_total_de = c(nrow(productos), nrow(maestra), nrow(maestra),
22                      nrow(maestra), n_distinct(ventas$vendedor_id))
23 )
24 calidad

```

```

# A tibble: 5 x 3
  revision                                casos de_un_total_de
  <chr>                                <int>         <int>
1 Productos con costo > precio de catálogo    10             50
2 Ventas con precio unitario < costo          119            365
3 Ventas con precio +/-10% vs. catálogo       336            365
4 Ventas antes del registro del cliente        42            365
5 Vendedores fuera del depto. de Ventas        21             25

```

El resumen de calidad es contundente y debe acompañar al reporte:

- 10 de los 50 productos tienen un costo mayor que su precio de catálogo (margen negativo).
- 119 de las 365 ventas (33 %) se hicieron a un precio unitario menor que el costo: se vendió con pérdida o el costo está mal capturado.
- En 336 ventas (92 %) el precio cobrado difiere más de 10 % del precio de catálogo. O el catálogo está desactualizado, o hay descuentos que no se registran en `descuento_pct`.
- 42 ventas son anteriores al registro del cliente y 21 de los 25 vendedores no son del departamento de Ventas.

Recuerda que son datos simulados, pero en tu trabajo real cada línea de esta tabla sería una pregunta para el dueño del dato. Los KPIs de ingresos son confiables (vienen directo de las ventas); los de utilidad y margen, no tanto, mientras no se aclare el costo.

2.11.8 Paso 8: exportar a Excel

writexl escribe un archivo `.xlsx` a partir de una *lista con nombres*: cada elemento de la lista se convierte en una hoja cuyo nombre es el del elemento.

Reporte mensual: exportar

```

1  dir.create("resultados", showWarnings = FALSE)  # carpeta de salida
2
3  hojas <- list(
4    "KPIs"          = kpis,
5    "Tendencia"     = por_mes,
6    "Categorias"    = por_categoria,
7    "Top clientes"  = top_clientes,
8    "Tiendas"       = ranking_tiendas,
9    "Vendedores"    = vendedores,
10   "Mes x categoria" = dinamica %>%
11     mutate(mes_nombre = as.character(mes_nombre)),
12   "Calidad datos"  = calidad
13 )
14 write_xlsx(hojas, "resultados/reporte_ventas_2023.xlsx")
15
16 # Comprobamos que el archivo existe y qué hojas tiene
17 file.exists("resultados/reporte_ventas_2023.xlsx")
18 readxl::excel_sheets("resultados/reporte_ventas_2023.xlsx")

```

```

[1] TRUE
[1] "KPIs"          "Tendencia"     "Categorias"
[4] "Top clientes"  "Tiendas"       "Vendedores"
[7] "Mes x categoria" "Calidad datos"

```

Convertimos `mes_nombre` a texto porque Excel no conoce los factores de R. Abre el archivo en Excel: tienes ocho hojas listas para enviar. El mes que entra, cambia el archivo de datos, ejecuta el script completo y tendrás el reporte actualizado en segundos. En el Módulo 8 aprenderás a convertir este script en un reporte automático con gráficas y texto usando R Markdown.

 Consejo

Si necesitas dar formato al Excel (colores, anchos de columna, formato de moneda), **writexl** no lo permite: usa el paquete **openxlsx**. Para la mayoría de los reportes internos, un archivo limpio y bien organizado vale más que uno bonito.

 Hazlo tú

Agrega al reporte una hoja “Zonas” con los ingresos, la utilidad y el margen por `zona` de la tienda, y otra hoja “Premium” que compare el ticket promedio de clientes premium contra no premium. Vuelve a exportar el archivo y ábrelo en Excel.

2.12 Si vienes de Excel o de SQL

Si ya trabajas con Excel o SQL, la tabla 2.8 te ayudará a traducir lo que ya sabes. La lógica es la misma; cambia la sintaxis. En el Módulo 5 verás que **dplyr** incluso puede *escribir* el SQL por ti.

Tabla 2.8. Equivalencias entre **dplyr**, SQL y Excel.

dplyr / tidyr	SQL	Excel
<code>filter(total > 1000)</code>	WHERE total > 1000	Autofiltro
<code>select(fecha, total)</code>	SELECT fecha, total	Ocultar o borrar columnas
<code>mutate(iva = total * 0.16)</code>	SELECT total * 0.16 AS iva	Fórmula en columna nueva
<code>if_else(), case_when()</code>	CASE WHEN ... END	SI(), SI.CONJUNTO()
<code>arrange(desc(total))</code>	ORDER BY total DESC	Ordenar de mayor a menor
<code>slice_max(total, n = 5)</code>	ORDER BY ... LIMIT 5	Filtro “Diez mejores”
<code>group_by() + summarise()</code>	GROUP BY + SUM()	Tabla dinámica, SUMAR.SI()
<code>count(categoria)</code>	COUNT(*) ... GROUP BY	CONTAR.SI()
<code>n_distinct(cliente_id)</code>	COUNT(DISTINCT ...)	Recuento distinto (tabla dinámica)
<code>left_join(productos, by = ...)</code>	LEFT JOIN ... ON ...	BUSCARV(), BUSCARX()
<code>inner_join()</code>	INNER JOIN	—
<code>anti_join()</code>	WHERE NOT EXISTS (...)	ESERROR(BUSCARV())
<code>bind_rows()</code>	UNION ALL	Copiar y pegar abajo
<code>distinct()</code>	SELECT DISTINCT	Quitar duplicados
<code>lag(), cumsum()</code>	LAG() OVER, SUM() OVER	Referencia a la celda de arriba
<code>min_rank(desc(x))</code>	RANK() OVER (ORDER BY)	JERARQUIA()
<code>pivot_wider()</code>	PIVOT / CASE	Tabla dinámica (columnas)
<code>pivot_longer()</code>	UNPIVOT	Anular dinamización (Power Query)
<code>str_squish(), str_to_upper()</code>	TRIM(), UPPER()	ESPACIOS(), MAYUSC()
<code>fill()</code>	—	Rellenar hacia abajo (Power Query)

Como ejemplo, esta consulta SQL y el *pipeline* de **dplyr** debajo de ella producen el mismo resultado:

```
SELECT p.categoria,
       SUM(v.total) AS ingresos,
       COUNT(*)    AS ventas
FROM ventas v
LEFT JOIN productos p ON v.producto_id = p.producto_id
WHERE v.trimestre = 'Q4'
GROUP BY p.categoria
ORDER BY ingresos DESC
LIMIT 3;
```

```
1 ventas %>%
2   left_join(productos, by = "producto_id") %>%      # LEFT JOIN
3   filter(trimestre == "Q4") %>%                    # WHERE
4   group_by(categoria) %>%                          # GROUP BY
```

```

5 summarise(ingresos = sum(total), ventas = n(),      # SUM, COUNT
6           .groups = "drop") %>%
7 arrange(desc(ingresos)) %>%                      # ORDER BY DESC
8 head(3)                                           # LIMIT 3

```

```

# A tibble: 3 x 3
  categoria  ingresos ventas
  <chr>      <dbl> <int>
1 Belleza    27050.     20
2 Electrónica 19900.     12
3 Juguetes   18363.      9

```

Una diferencia importante: en SQL el orden de escritura (**SELECT** primero) no es el orden en que se ejecuta. En **dplyr** escribes los pasos en el orden en que suceden, y eso lo hace más fácil de leer y de depurar: puedes ejecutar el *pipeline* hasta cualquier línea y ver el resultado intermedio.

2.13 Errores comunes

Estos son los errores que más verás mientras aprendes. Los mensajes son los que R muestra realmente (algunos en inglés porque vienen de los paquetes, otros en español porque vienen de R base).

2.13.1 Olvidar un pipe o ponerlo al inicio de la línea

```

1 ventas %>%
2   filter(total > 4800)      # <- falta %>% al final de esta línea
3   arrange(desc(total))

```

```
Error: objeto 'total' no encontrado
```

R ejecuta las dos primeras líneas como una instrucción completa (y muestra el resultado del filtro) y luego intenta ejecutar `arrange(desc(total))` sola, sin tabla: no sabe qué es `total`. Si pones el pipe al inicio de la línea siguiente, el error es otro, porque la primera línea ya era una instrucción completa:

```

1 ventas
2   %>% filter(total > 4800)

```

```
Error: inesperado SPECIAL en " %>%"
```

Solución: el `%>%` siempre va al *final* de la línea, y la última línea del *pipeline* no lleva pipe.

2.13.2 Usar = en lugar de == en filter()

```
1 ventas %>% filter(tienda_id = 3)
```

```
Error in `filter()`:  
! We detected a named input.  
i This usually means that you've used `=` instead of `==`.  
i Did you mean `tienda_id == 3`?
```

= asigna o nombra argumentos; == compara. El mensaje de **dplyr** incluso te sugiere la corrección.

2.13.3 Olvidar cargar dplyr: el filter() equivocado

Si abres una sesión nueva y olvidas `library(tidyverse)`, `filter()` existe de todos modos: es `stats::filter()`, una función para series de tiempo que no entiende nombres de columna.

```
1 # Sesión nueva, sin library(tidyverse)  
2 ventas <- readr::read_csv("datasets/ventas_retail.csv",  
3                           show_col_types = FALSE)  
4 filter(ventas, total > 4800)
```

```
Error: objeto 'total' no encontrado
```

El mismo error puede aparecer si otro paquete cargado *después* del *tidyverse* tapa a `filter()` o `select()` (pasa, por ejemplo, con **MASS**, que tiene su propio `select()`). **Solución:** carga los paquetes al inicio del script y, ante la duda, escribe `dplyr::filter()` o `dplyr::select()`.

2.13.4 Escribir mal el nombre de una columna

```
1 ventas %>% summarise(total = sum(Total))    # "Total" con mayúscula
```

```
Error in `summarise()`:  
i In argument: `total = sum(Total)`.  
Caused by error:  
! objeto 'Total' no encontrado
```

R distingue mayúsculas de minúsculas. Revisa los nombres exactos con `names(ventas)` o `glimpse(ventas)`.

2.13.5 Nombres de columna con espacios

Los archivos de Excel suelen traer encabezados como “Ventas Totales”. Ese nombre tiene un espacio, así que no puedes escribirlo tal cual:

```
1 reporte %>% select(Ventas Totales)
```

```
Error: unexpected symbol en "reporte %>% select(Ventas Totales)"
```

Solución: escríbelo entre comillas invertidas (*backticks*), o mejor aún, renómbralo en cuanto lo leas:

```
1 reporte <- tibble(`Ventas Totales` = c(100, 200),
2                   `Sucursal ID` = c("A", "B"))
3 reporte %>%
4   rename(ventas_totales = `Ventas Totales`,      # backticks
5         sucursal_id     = `Sucursal ID`)
```

```
# A tibble: 2 x 2
  ventas_totales sucursal_id
      <dbl> <chr>
1         100 A
2         200 B
```

(El paquete **janitor** tiene la función `clean_names()`, que convierte todos los nombres a `snake_case` de una vez.)

2.13.6 `group_by()` sin `summarise()` (o sin `ungroup()`)

`group_by()` solo, no resume nada: la tabla se ve igual, con una línea extra de **Groups**. Si esperabas un total por grupo, te faltó el `summarise()`. Y si la tabla queda agrupada, los pasos siguientes se hacen por grupo:

```
1 agrupada <- ventas %>% group_by(tienda_id)
2 agrupada %>% select(fecha, tienda_id, total) %>% head(2)
3
4 # Con grupos: slice_max() da la mejor venta DE CADA tienda (10 filas)
5 agrupada %>% slice_max(total, n = 1) %>% nrow()
6 # Sin grupos: la mejor venta del año (1 fila)
7 agrupada %>% ungroup() %>% slice_max(total, n = 1) %>% nrow()
```

```
# A tibble: 2 x 3
# Groups:   tienda_id [2]
```



```

  fecha      tienda_id total
  <date>      <dbl> <dbl>
1 2023-01-01      6 3333.
2 2023-01-02      5 2262
[1] 10
[1] 1

```

Observa la línea # Groups: tienda_id [2]: la tabla sigue agrupada (en las 2 filas mostradas hay 2 tiendas distintas). El mismo `slice_max(total, n = 1)` devuelve 10 filas o 1 fila según haya grupos o no: si no te das cuenta de que la tabla está agrupada, tu “venta récord” serán diez.

2.13.7 Joins que multiplican filas

```

1 ventas_ene %>% left_join(precios_compras, by = "producto_id")
2 # ¿3 filas en lugar de 2? -> llave duplicada en la tabla derecha

```

No hay mensaje de error: ese es el peligro. Revisa siempre las filas antes y después, y usa `count(llave)%>% filter(n > 1)` o `relationship = "many-to-one"` (sección 2.8). Cuando la duplicidad está en ambas tablas, verás:

```

Warning message:
...
  Detected an unexpected many-to-many relationship between `x` and `y`.
...

```

2.13.8 NA en sumas y promedios

Una sola celda vacía convierte el total en **NA** (sección 2.4.5). Usa `na.rm = TRUE` a conciencia y cuenta los faltantes con `sum(is.na(x))`.

2.13.9 Apilar tablas con tipos distintos

```

1 a <- tibble(id = 1, monto = "100")      # monto como texto
2 b <- tibble(id = 2, monto = 200)        # monto como número
3 bind_rows(a, b)

```

```

Error in `bind_rows()`:
! Can't combine `..1$monto` <character> and `..2$monto` <double>.

```

Solución: homologa los tipos antes de apilar, por ejemplo con `mutate(monto = parse_number(monto))` en la tabla de texto.

Resumen del módulo

- Los **datos ordenados** tienen una variable por columna, una observación por fila y un valor por celda. Analiza en largo, presenta en ancho.
- `read_csv()` lee CSV y te informa los tipos que adivinó; `col_types`, `na` y `problems()` te dan el control. `glimpse()` es tu primera mirada a cualquier tabla.
- El pipe (`%>%` o `|>`) se lee “y luego” y convierte tu análisis en una receta legible.
- Los verbos de **dplyr** (`filter`, `select`, `mutate`, `arrange`, `summarise`, `group_by`) resuelven la mayoría de las preguntas de negocio. `count()` y `slice_max()` son atajos valiosos.
- Las funciones de ventana (`lag()`, `cumsum()`, `min_rank()`) calculan crecimiento, acumulados y rankings.
- **lubridate** y **stringr** limpian y enriquecen fechas y texto. Usa una fecha de corte fija para resultados reproducibles.
- Los *joins* conectan hechos con dimensiones; revisa siempre las filas antes y después, y usa `anti_join()` para auditar.
- **tidyr** cambia la forma de las tablas y rellena faltantes y periodos vacíos.
- Un reporte profesional incluye controles automáticos, supuestos explícitos y hallazgos de calidad de datos.

Tabla 2.9. Funciones clave del Módulo 2.

Tarea	Funciones
Importar y explorar	<code>read_csv()</code> , <code>cols()</code> , <code>problems()</code> , <code>glimpse()</code> , <code>tibble()</code>
Filas	<code>filter()</code> , <code>arrange()</code> , <code>desc()</code> , <code>distinct()</code> , <code>slice_max()</code> , <code>slice_min()</code> , <code>slice_head()</code> , <code>slice_sample()</code>
Columnas	<code>select()</code> , <code>rename()</code> , <code>relocate()</code> , <code>mutate()</code> , <code>across()</code> , <code>if_else()</code> , <code>case_when()</code> , <code>recode()</code>
Resumir	<code>group_by()</code> , <code>summarise()</code> , <code>count()</code> , <code>n()</code> , <code>n_distinct()</code> , <code>ungroup()</code>
Ventana	<code>lag()</code> , <code>lead()</code> , <code>cumsum()</code> , <code>row_number()</code> , <code>min_rank()</code> , <code>dense_rank()</code>
Fechas	<code>ymd()</code> , <code>dmy()</code> , <code>year()</code> , <code>month()</code> , <code>wday()</code> , <code>floor_date()</code> , <code>make_date()</code> , <code>time_length()</code> , <code>interval()</code>
Texto	<code>str_detect()</code> , <code>str_replace_all()</code> , <code>str_squish()</code> , <code>str_to_upper()</code> , <code>str_sub()</code> , <code>str_pad()</code> , <code>str_c()</code> , <code>parse_number()</code>
Unir	<code>left_join()</code> , <code>inner_join()</code> , <code>full_join()</code> , <code>semi_join()</code> , <code>anti_join()</code> , <code>join_by()</code> , <code>bind_rows()</code>
Forma	<code>pivot_wider()</code> , <code>pivot_longer()</code> , <code>separate_wider_delim()</code> , <code>unite()</code> , <code>fill()</code> , <code>replace_na()</code> , <code>drop_na()</code> , <code>complete()</code>
Exportar	<code>write_csv()</code> , <code>write_xlsx()</code>

2.14 Ejercicios

Todos los ejercicios usan los datos del curso. Empieza cada script cargando el *tidyverse* y las tablas que necesites con `read_csv()`. Las soluciones completas están en el Apéndice A.

Ejercicio 2.1: Ventas con descuento de una tiendaNivel: **Básico**

Con `ventas_retail.csv`, obtén las ventas de la tienda 3 con un descuento de 10% o más, mostrando solo `fecha`, `producto_id`, `descuento_pct` y `total`, ordenadas de mayor a menor total. ¿Cuántas son y cuánto suman?

 Solución en la página 620**Ejercicio 2.2: Productos a resurtir**Nivel: **Básico**

Con `productos.csv`, arma la lista de productos *activos* cuyo `stock_actual` está por debajo del `stock_minimo`. Muestra el nombre, la categoría, el proveedor, ambas existencias y una columna nueva `faltante` con las unidades que faltan para llegar al mínimo. Ordena de mayor a menor faltante. ¿A qué proveedor hay que llamar primero?

 Solución en la página 620**Ejercicio 2.3: Rangos de edad de los clientes**Nivel: **Básico**

Con `clientes.csv`, crea la columna `rango_edad` con `case_when()`: “18-29”, “30-44”, “45-59” y “60+”. Cuenta cuántos clientes hay en cada rango y qué porcentaje del total representan.

 Solución en la página 621**Ejercicio 2.4: Perfil de clientes por ciudad**Nivel: **Intermedio**

Con `clientes.csv`, calcula por ciudad: número de clientes, porcentaje de clientes premium y edad promedio. Ordena por número de clientes y di qué ciudad tiene la mayor proporción de premium.

 Solución en la página 622**Ejercicio 2.5: El mejor día de la semana en transacciones**Nivel: **Intermedio**

Con `transacciones.csv`, crea el día de la semana como factor ordenado con `wday(fecha, label = TRUE, abbr = FALSE)` y calcula por día: número de transacciones, ingresos y ticket promedio. Agrega la participación de cada día en los ingresos. ¿Qué día conviene reforzar con personal?

 Solución en la página 623

Ejercicio 2.6: Antigüedad y salario de los empleadosNivel: **Intermedio**

Con `empleados.csv` y la fecha de corte 1 de enero de 2024, calcula la antigüedad en años con `time_length(interval(...))` y compárala con la columna `antigüedad_anos` (¿cuál es la diferencia máxima?). Luego clasifica a los empleados en “Menos de 3 años”, “3 a 6 años” y “Más de 6 años” y calcula el salario mensual promedio de cada grupo.

 Solución en la página 623**Ejercicio 2.7: Margen real por categoría en transacciones**Nivel: **Intermedio**

Une `transacciones.csv` con `productos.csv`. Para cada transacción calcula la utilidad como $(\text{precio_venta} - \text{costo}) * \text{cantidad}$. Resume por categoría: ingresos, utilidad, margen % y número de transacciones con utilidad negativa. ¿Qué categoría deja más utilidad y cuál es la más problemática?

 Solución en la página 624**Ejercicio 2.8: Auditoría de clientes entre dos sistemas**Nivel: **Intermedio**

Usa `semi_join()` y `anti_join()` para responder: (a) ¿cuántos clientes compraron en *ambas* tablas, `ventas` y `transacciones`? (b) ¿Cuántos aparecen en `transacciones` pero no en `ventas`? (c) ¿Cuántos clientes registrados no aparecen en ninguna de las dos? Pista: para (c) puedes apilar los `cliente_id` de ambas tablas con `bind_rows()`.

 Solución en la página 625**Ejercicio 2.9: Tabla dinámica de métodos de pago por tienda**Nivel: **Avanzado**

Con `transacciones.csv` y `tiendas.csv`, construye una tabla con una fila por `nombre_tienda` y una columna por método de pago, donde cada celda sea el *porcentaje* de los ingresos de esa tienda pagado con ese método (cada fila debe sumar 100). ¿Qué tienda depende más del efectivo?

 Solución en la página 626**Ejercicio 2.10: Crecimiento mensual por tienda**Nivel: **Avanzado**

Con `ventas_retail.csv`, calcula los ingresos por tienda y mes, rellena con 0 los meses sin ventas usando `complete()` y calcula, por tienda, el crecimiento contra el mes anterior en pesos $(\text{ingresos} - \text{lag}(\text{ingresos}))$ y el acumulado del año. ¿Qué tienda tuvo la mayor caída en pesos de un mes a otro, y en qué mes?

 Solución en la página 627

Ejercicio 2.11: Limpiar un catálogo de proveedoresNivel: **Avanzado**

Un proveedor te manda su lista de precios capturada a mano. Cópiala en tu script y límpiala: quita duplicados exactos, unifica la categoría (debe quedar en tipo título y con acento), convierte el precio a número, convierte la fecha (formatos mixtos), elimina los precios imposibles (cero o negativos) y crea un SKU con formato SKU-0001. Al final, muestra el precio promedio por categoría.

```
1 lista_proveedor <- tibble(  
2   id      = c(1, 2, 2, 3, 4, 5, 6),  
3   categoria = c("electronica", "Electrónica ", "Electrónica ",  
4                 "HOGAR", "hogar", "Ropa", " ropa"),  
5   precio   = c("$1,250.00", "$899.50", "$899.50", "450", "$0",  
6                 "$320.75", "-15"),  
7   fecha    = c("2023-05-01", "02/05/2023", "02/05/2023", "2023-05-03",  
8                 "04/05/2023", "2023-05-05", "06/05/2023")  
9 )
```

 Solución en la página 628**Ejercicio 2.12: Reporte trimestral por tienda en Excel**Nivel: **Reto**

Con `transacciones.csv`, `productos.csv` y `tiendas.csv`, genera un archivo `resultados/reporte_tiendas.xlsx` con tres hojas: (1) “Resumen”: por tienda, ingresos, transacciones, ticket promedio, clientes únicos y ranking; (2) “Trimestres”: tabla dinámica tienda × trimestre con los ingresos; (3) “Top producto”: el producto con más ingresos de cada tienda con su categoría. Incluye un control con `stopifnot()` que verifique que los *joins* no cambiaron el número de filas.

 Solución en la página 629

Visualización de datos con ggplot2

Al terminar este módulo podrás...

- Explicar por qué una gráfica revela cosas que una tabla de estadísticas esconde (el cuarteto de Anscombe).
- Construir cualquier gráfica con la *gramática de gráficos* de **ggplot2**: datos, estéticas, geometrías, escalas, facetas, coordenadas y temas, capa por capa.
- Elegir la gráfica correcta según la pregunta de negocio: comparación, evolución, distribución, relación o composición.
- Crear barras, líneas, dispersión, histogramas, boxplots, mapas de calor y facetas con formato de negocio (\$, %, miles, millones).
- Aplicar reglas profesionales de diseño: un color por serie, resaltar lo importante, sin doble eje, sin arcoíris, ejes honestos.
- Construir tu propio tema corporativo `tema_empresa()` y exportar gráficas listas para PowerPoint, Word o correo.
- Armar un tablero de ventas coherente para la dirección y redactar conclusiones basadas en lo que los datos muestran.

Es jueves por la tarde y la directora general te pide “tres o cuatro láminas” con las ventas de 2023 para el consejo del lunes. En el Módulo 2 aprendiste a obtener las cifras con **dplyr**: ingresos por mes, por categoría, por tienda. El problema es que nadie en el consejo va a leer una tabla de 12 filas y 5 columnas. Lo que sí van a hacer es mirar una gráfica durante diez segundos y quedarse con una idea. Tu trabajo es que esa idea sea la correcta.

Visualizar datos no es “hacer dibujos bonitos”. Es la parte del **Business Intelligence** (BI) donde el análisis se convierte en una decisión: una gráfica bien hecha hace evidente que abril fue un mes malo, que una tienda pequeña está vendiendo más que la grande o que una categoría se desplomó en el segundo semestre. Una gráfica mal hecha (colores sin sentido, ejes recortados, pasteles con diez rebanadas) confunde o, peor, engaña.

En este módulo aprenderás **ggplot2**, el paquete de gráficas más usado en R, y, sobre todo, aprenderás a *pensar* como diseñador de visualizaciones de negocio. Cada gráfica seguirá la misma ruta: una pregunta de negocio, el código comentado línea por línea, la figura resultante, cómo leerla y un reto para que la hagas tú. Al final (sección 3.15) armarás un tablero de ventas 2023 completo para la dirección.

3.1 Por qué visualizar

3.1.1 El cuarteto de Anscombe

En 1973 el estadístico Francis Anscombe publicó cuatro pequeños conjuntos de datos, cada uno con 11 pares de valores (x, y) . Vienen incluidos en R con el nombre `anscombe`. Su gracia es que, si solo miras las estadísticas de resumen, los cuatro parecen idénticos. Empecemos cargando los paquetes que usaremos en todo el módulo.

```
1 library(tidyverse) # incluye ggplot2, dplyr, tidyr, forcats...
2 library(scales)    # formatos de ejes: dólares, comas, porcentajes
```

`ggplot2` ya viene dentro de `tidyverse`, así que no hace falta cargarlo aparte. `scales` se instala junto con `ggplot2` pero hay que cargarlo para usar sus funciones de formato (`dollar()`, `percent()`...). La tabla `anscombe` tiene las columnas `x1`, `x2`, `x3`, `x4`, `y1`, ..., `y4`; la pasamos a formato largo (una fila por punto) con `pivot_longer()`, que conociste en el Módulo 2.

```
1 anscombe_largo <- anscombe %>%
2   pivot_longer(cols = everything(),
3               # ".value" = la 1a letra (x o y) será el nombre de columna
4               names_to = c(".value", "conjunto"),
5               # el patrón separa "x1" en "x" y "1"
6               names_pattern = "(.)(.)" %>%
7   mutate(conjunto = paste("Conjunto", conjunto))
8
9 # Las estadísticas de cada conjunto
10 anscombe_largo %>%
11   group_by(conjunto) %>%
12   summarise(media_x = mean(x),
13             media_y = mean(y),
14             desv_y = sd(y),
15             correlacion = cor(x, y),
16             .groups = "drop") %>%
17   mutate(across(where(is.numeric), ~ round(.x, 2)))
```

```
# A tibble: 4 x 5
  conjunto media_x media_y desv_y correlacion
  <chr>      <dbl>   <dbl>  <dbl>      <dbl>
1 Conjunto 1      9      7.5    2.03      0.82
2 Conjunto 2      9      7.5    2.03      0.82
3 Conjunto 3      9      7.5    2.03      0.82
4 Conjunto 4      9      7.5    2.03      0.82
```

Las cuatro filas son prácticamente iguales: media de x igual a 9, media de y de 7.5, la misma desviación estándar y una correlación de 0.82 en todos los casos. Si esto fuera un reporte con solo la

tabla, concluirías que los cuatro conjuntos “se comportan igual”. Ahora grafiquémoslos.

```
1 g_anscombe <- ggplot(anscombe_largo, aes(x = x, y = y)) +
2   geom_point(color = "#2a78d6", size = 2) +           # los 11 puntos
3   geom_smooth(method = "lm", formula = y ~ x,         # recta de regresión
4               se = FALSE, color = "#eb6834", linewidth = 0.8) +
5   facet_wrap(~ conjunto, ncol = 4) +                 # un panel por conjunto
6   labs(title = "Mismas estadísticas, cuatro historias distintas",
7        subtitle = "Media, varianza, correlación y recta casi idénticas",
8        caption = "Fuente: datasets::anscombe (Anscombe, 1973)") +
9   theme_minimal() +
10  theme(panel.grid.minor = element_blank())
11 g_anscombe
```

No te preocupes todavía por cada línea del código: en la siguiente sección lo construiremos pieza por pieza. Fíjate en el resultado (figura 3.1).

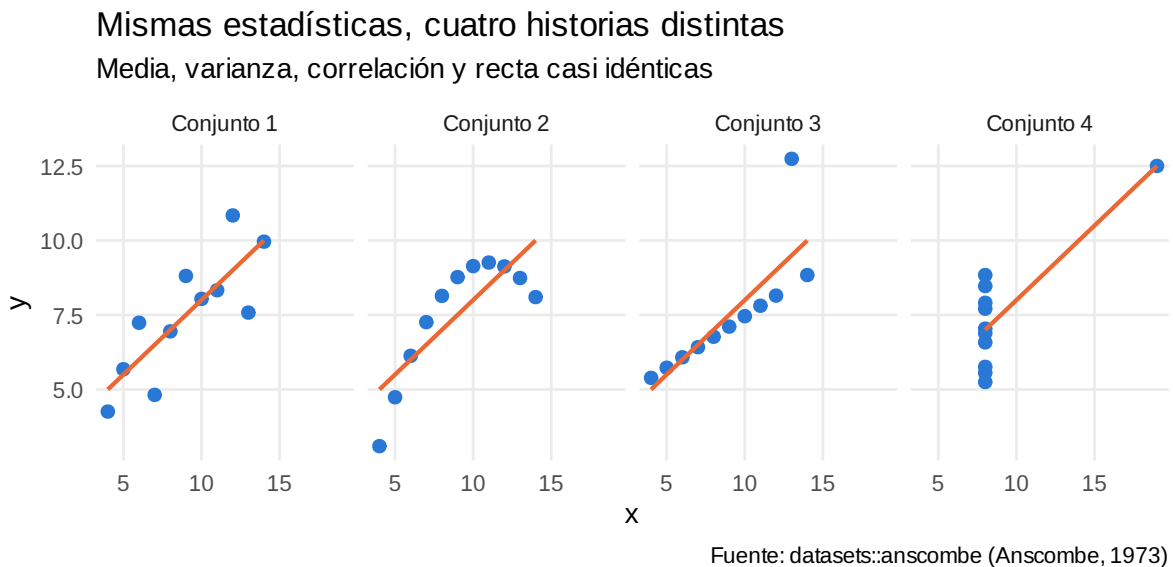


Figura 3.1. El cuarteto de Anscombe: cuatro conjuntos con estadísticas idénticas y formas completamente distintas.

Cómo leerla. El conjunto 1 es una relación lineal con ruido: la recta sí lo describe. El conjunto 2 es una curva: una recta es el modelo equivocado. En el conjunto 3 todos los puntos caen sobre una recta perfecta salvo uno, un **valor atípico** (*outlier*) que jala la recta hacia arriba. En el conjunto 4 casi todos los puntos tienen el mismo x y un solo punto extremo fabrica toda la “correlación”. La lección para un analista de BI es directa: **antes de reportar un promedio o una correlación, grafica los datos.**

En la empresa: El promedio que escondía dos tiendas

Una cadena reporta que el ticket promedio de sus tiendas es de \$950 y que es “estable”. Al graficar la distribución de tickets por tienda aparece que una sucursal tiene muchísimas compras pequeñas y unas cuantas enormes (clientes mayoristas). El promedio era el mismo, pero la operación, el riesgo y la estrategia comercial son completamente distintos. Solo la gráfica lo hizo evidente.

3.1.2 Visualización exploratoria contra explicativa

En tu trabajo harás dos tipos de gráficas muy diferentes:

- **Visualización exploratoria:** gráficas rápidas que haces *para ti* mientras analizas. No necesitan título bonito ni colores corporativos; necesitan ser rápidas. Harás decenas y tirarás casi todas. El objetivo es *descubrir*.
- **Visualización explicativa:** la gráfica que llevas a la dirección. Tiene *un* mensaje, un título que lo dice (“Belleza cayó 41 % en el segundo semestre” en vez de “Ventas por categoría”), formato de negocio en los ejes, la fuente de los datos y nada que distraiga. El objetivo es *comunicar*.

Consejo

Un buen título explicativo es una *conclusión*, no una descripción. Compara “Ingresos mensuales 2023” con “7 de 12 meses superaron la meta”. El segundo le ahorra al director el trabajo de interpretar la gráfica. El subtítulo puede decir qué se mide (“Ingresos mensuales, MXN”).

3.2 La gramática de gráficos

ggplot2 se basa en la **gramática de gráficos** (*grammar of graphics*), una idea de Leland Wilkinson que Hadley Wickham implementó en R: toda gráfica estadística se puede describir como la suma de unas cuantas piezas independientes. En vez de elegir “un tipo de gráfica” de un menú (como en Excel), *construyes* la gráfica declarando cada pieza.

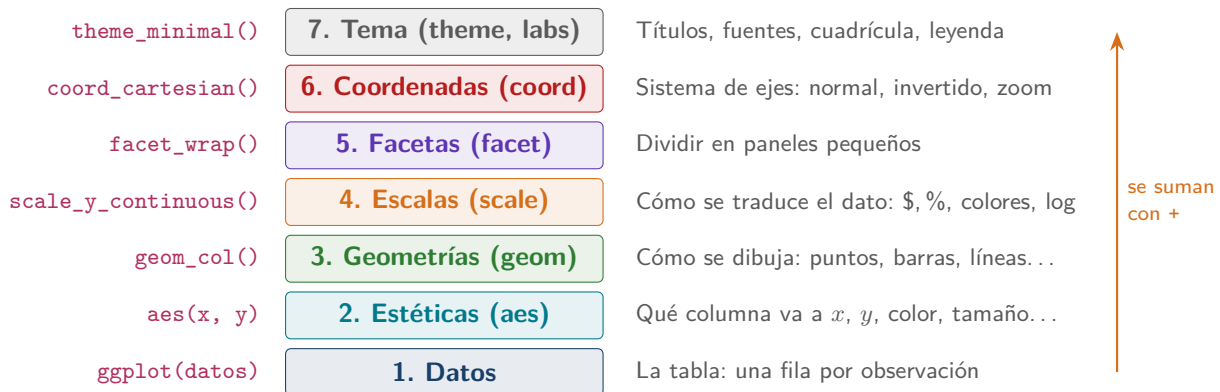


Figura 3.2. Las siete capas de la gramática de gráficos. Solo las tres primeras son obligatorias; las demás tienen valores por defecto.

Las capas se combinan con el signo `+`. Una gráfica mínima necesita solo las tres primeras: *datos*, *estéticas* y al menos una *geometría*. Todo lo demás (escalas, facetas, coordenadas, tema) tiene un valor por defecto razonable que puedes cambiar.

Concepto clave: Estética (aes)

Una **estética** es una propiedad visual que *depende de los datos*: la posición en *x*, la posición en *y*, el color, el relleno (`fill`), el tamaño, la forma, la transparencia (`alpha`). Dentro de `aes()` dices “esta columna controla esta propiedad”. Si quieres que *todas* las barras sean azules, eso no depende de los datos y va *fuera* de `aes()`: `geom_col(fill = "#2a78d6")`.

3.2.1 Los datos del módulo

Trabajaremos sobre todo con `transacciones.csv` (1,000 tickets de 2023), unida al catálogo de productos (para tener la categoría) y a las tiendas (para tener el nombre). Es el mismo *join* que hiciste en el Módulo 2.

```
1 transacciones <- read_csv("datasets/transacciones.csv",
2                             show_col_types = FALSE)
3 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
4 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
5
6 ventas_detalle <- transacciones %>%
7   left_join(select(productos, producto_id, categoria),
8             by = "producto_id") %>%
9   left_join(select(tiendas, tienda_id, nombre_tienda),
10            by = "tienda_id") %>%
11   # "Sucursal Outlet" -> "Outlet": etiquetas más cortas para las gráficas
12   mutate(tienda = str_remove(nombre_tienda, "Sucursal "))
13
14 ventas_detalle %>%
15   select(fecha, categoria, tienda, metodo_pago, total_transaccion) %>%
```

```
16 head(4)
```

```
# A tibble: 4 x 5
  fecha      categoria tienda metodo_pago total_transaccion
  <date>     <chr>     <chr>   <chr>          <dbl>
1 2023-01-13 Automotriz Express Tarjeta Crédito    1216.
2 2023-01-09 Automotriz Norte   Tarjeta Débito     232.
3 2023-02-20 Juguetes   Plaza B Tarjeta Débito     826.
4 2023-05-05 Deportes   Plaza A Tarjeta Débito     421.
```

Para la primera gráfica necesitamos los ingresos de cada mes. `floor_date(fecha, unit = "month")` (de **lubridate**) lleva cada fecha al primer día de su mes: así obtenemos una columna de tipo *fecha*, no de texto, lo que será importante para el eje *x*.

```
1 ventas_mes <- ventas_detalle %>%
2   mutate(mes = floor_date(fecha, unit = "month")) %>%
3   group_by(mes) %>%
4   summarise(ingresos = sum(total_transaccion),
5             tickets  = n(),
6             .groups  = "drop")
7 ventas_mes
```

```
# A tibble: 12 x 3
  mes      ingresos tickets
  <date>     <dbl>   <int>
1 2023-01-01 82560.     98
2 2023-02-01 65899.     79
3 2023-03-01 90931.     91
4 2023-04-01 57038.     71
5 2023-05-01 93910.     88
6 2023-06-01 88509.     84
7 2023-07-01 86456.     83
8 2023-08-01 78929.     75
9 2023-09-01 68856.     86
10 2023-10-01 70004.     74
11 2023-11-01 80507.     80
12 2023-12-01 96394.     91
```

3.2.2 Tu primer gráfico, capa por capa

Pregunta de negocio: ¿cómo evolucionaron los ingresos mes a mes en 2023? Construiremos la respuesta en cuatro pasos y guardaremos cada paso en un objeto para ver cómo cambia.

```

1 # Paso 1: datos + estéticas. Solo el "lienzo" con los ejes
2 paso1 <- ggplot(data = ventas_mes, mapping = aes(x = mes, y = ingresos))
3 paso1

```

`ggplot()` crea el lienzo. Con `aes(x = mes, y = ingresos)` le dices qué columna va en cada eje. R ya calcula los rangos (de enero a diciembre; de unos \$57,000 a \$96,000) pero no dibuja nada, porque aún no hay geometría.

```

1 # Paso 2: + una geometría de línea (color y grosor fijos, fuera de aes)
2 paso2 <- paso1 + geom_line(color = "#2a78d6", linewidth = 0.8)
3 paso2

```

Al *sumar* `geom_line()` aparece la línea. El color y el grosor (`linewidth`) son constantes, por eso van como argumentos de la geometría y no dentro de `aes()`.

```

1 # Paso 3: + puntos en cada mes y + escala del eje y en dólares
2 paso3 <- paso2 +
3   geom_point(color = "#2a78d6", size = 2) +
4   scale_y_continuous(labels = dollar)
5 paso3

```

Las capas se apilan: `geom_point()` dibuja encima de la línea. La *escala* `scale_y_continuous(labels = dollar)` no cambia los datos, solo cómo se *muestran* los números del eje: 90000 se vuelve \$90,000.

```

1 # Paso 4: + títulos y + un tema limpio
2 paso4 <- paso3 +
3   labs(title = "Ingresos mensuales 2023", x = NULL, y = "Ingresos") +
4   theme_minimal() +
5   theme(panel.grid.minor = element_blank())
6 paso4

```

`labs()` pone los textos (`x = NULL` quita el título del eje *x*, porque “mes” es obvio). `theme_minimal()` quita el fondo gris y `theme(panel.grid.minor = element_blank())` elimina las líneas secundarias de la cuadrícula, que solo agregan ruido. La figura 3.3 muestra los cuatro pasos.

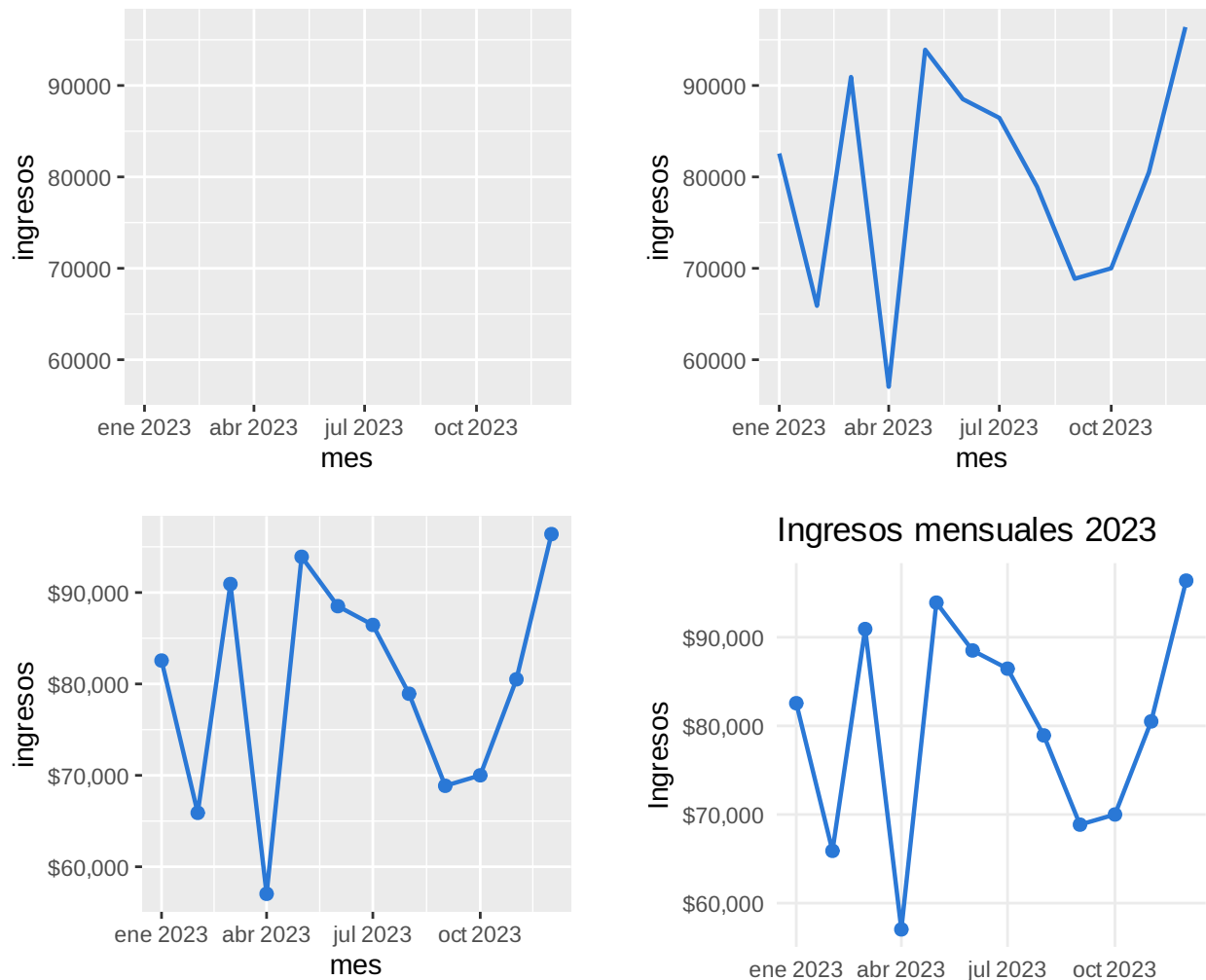


Figura 3.3. La misma gráfica en cuatro pasos: lienzo (arriba a la izquierda), línea, puntos con escala en dólares y, finalmente, títulos y tema.

Cómo leerla. Ya con el paso 4 se ve la historia del año: dos caídas fuertes (febrero y, sobre todo, abril), una meseta alta de mayo a julio, un bache en septiembre–octubre y un cierre muy fuerte en diciembre.

▲ Cuidado

El **+** va **al final** de cada línea, nunca al principio de la siguiente. Si escribes el **+** al inicio, R da por terminada la instrucción en la línea anterior y luego no sabe qué hacer con el **+** suelto. Tampoco uses **%>%** entre capas de **ggplot2**: el pipe sirve para *transformar datos*; el **+** para *añadir capas*. Verás ambos errores en la sección 3.16.

🔗 Hazlo tú

Repite los cuatro pasos pero con **y = tickets** (número de tickets por mes) y el título “Tickets por mes 2023”. Pista: en el paso 3 cambia **dollar** por **comma**. ¿Coinciden los meses bajos con

los de ingresos?

3.3 Elegir la gráfica correcta

Antes de escribir código pregúntate: *¿qué quiero que vea quien mire la gráfica?* La respuesta casi siempre cae en una de seis familias (tabla 3.1).

Tabla 3.1. ¿Qué quieres mostrar? Gráfica recomendada y qué evitar.

Quiero mostrar...	Gráfica recomendada	Evita
Comparación entre categorías (tiendas, productos)	Barras ordenadas; horizontales si los nombres son largos (<code>geom_col()</code>)	Barras de colores sin significado; pasteles; ejes que no empiezan en 0
Evolución en el tiempo (meses, semanas)	Línea (<code>geom_line()</code>); barras si son pocos periodos	Más de 4–5 líneas de colores; doble eje Y
Distribución (cómo se reparten los tickets)	Histograma, densidad, boxplot (<code>geom_histogram()</code> , <code>geom_boxplot()</code>)	Reportar solo el promedio
Relación entre dos medidas (precio y cantidad)	Dispersión (<code>geom_point()</code>) con tendencia opcional	Unir los puntos con líneas; más de 3 colores de grupo
Composición (partes de un total)	Barras 100 % o barras de participación; pastel solo con 2–3 porciones	Pastel de 10 rebanadas; dona 3D
Un solo número (ingresos del año)	Tarjeta KPI: el número grande con su contexto	Una gráfica con una sola barra

Concepto clave: Tarjeta KPI

Un **KPI** (*Key Performance Indicator*, indicador clave de desempeño) es un número que resume el desempeño: ingresos, tickets, ticket promedio. Cuando el mensaje es *un* número, la mejor visualización es una **tarjeta KPI**: el número grande, una etiqueta y, si acaso, la comparación contra la meta o el año anterior. Una gráfica con una sola barra ocupa más espacio y comunica menos.

Hazlo tú

Para cada pregunta, decide la familia de la tabla 3.1: (a) “¿qué vendedor vendió más?”, (b) “¿cuánto vendimos en total?”, (c) “¿los clientes jóvenes gastan más por ticket?”, (d) “¿cómo se

reparten los ingresos entre los métodos de pago?”.

3.4 Barras y columnas: comparar categorías

Las barras son la gráfica más útil del BI: el ojo humano compara longitudes con mucha precisión. Antes de empezar, define los *colores corporativos* que usaremos en todo el módulo. En la sección 3.11 explicaremos por qué son estos y por qué en este orden; por ahora, cópialos tal cual.

```
1 color_principal <- "#2a78d6" # azul: para una sola serie
2 color_resalte   <- "#eb6834" # naranja: para destacar UN elemento
3 color_gris      <- "#b5b3ad" # gris: para "el resto"
4 # Paleta para varias series (usar en este orden, máximo 8)
5 paleta_libro <- c("#2a78d6", "#eb6834", "#1baf7a", "#eda100",
6                  "#e87ba4", "#008300", "#4a3aa7", "#e34948")
```

3.4.1 geom_col() contra geom_bar()

ggplot2 tiene dos geometrías de barras y confundirlas es un error muy común:

- **geom_col()**: la altura de la barra es un *valor que ya calculaste* (por ejemplo, la suma de ingresos). Necesita **x** y **y**. Es la que más usarás en BI, porque normalmente resumes primero con **summarise()** y luego graficas.
- **geom_bar()**: *cuenta filas* por ti. Solo necesita **x**; la altura es el número de filas de cada categoría.

```
1 # geom_bar() cuenta cuántas filas (tickets) hay de cada método de pago
2 g_conteo <- ggplot(ventas_detalle, aes(x = metodo_pago)) +
3   geom_bar(fill = color_principal)
4
5 # Es lo mismo que contar primero y luego usar geom_col():
6 ventas_detalle %>% count(metodo_pago)
```

```
# A tibble: 4 x 2
  metodo_pago      n
  <chr>         <int>
1 Efectivo       315
2 Tarjeta Crédito 350
3 Tarjeta Débito 233
4 Transferencia  102
```

Las dos formas producen la misma gráfica de conteos. Nuestra recomendación: **resume con dplyr y grafica con geom_col()**. Así ves la tabla con las cifras exactas antes de dibujarla y puedes citarlas en tu reporte.

3.4.2 Ingresos por categoría: una serie, un color

Pregunta de negocio: ¿qué categorías generan más ingresos? Primero resumimos.

```
1 ingresos_categoria <- ventas_detalle %>%
2   group_by(categoria) %>%
3   summarise(ingresos = sum(total_transaccion),
4             tickets = n(),
5             .groups = "drop") %>%
6   mutate(participacion = ingresos / sum(ingresos)) %>% # % del total
7   arrange(desc(ingresos))
8 ingresos_categoria
```

```
# A tibble: 10 x 4
  categoria ingresos tickets participacion
  <chr>      <dbl>   <int>      <dbl>
1 Belleza    182143.    187      0.190
2 Electrónica 132672.    137      0.138
3 Juguetes   125093.    118      0.130
4 Alimentos 121543.    115      0.127
5 Automotriz 104085.    105      0.108
6 Deportes   86171.     92      0.0898
7 Hogar      63407.     64      0.0660
8 Mascotas   57668.     74      0.0601
9 Libros     57162.     70      0.0595
10 Ropa       30048.     38      0.0313
```

Ahora la gráfica. Usamos barras *horizontales* (la categoría en el eje *y*) porque los nombres de categoría se leen mucho mejor de lado que apretados bajo cada columna.

```
1 g_categorias <- ggplot(ingresos_categoria,
2   aes(x = ingresos,
3       # ordena las categorías según sus ingresos
4       y = fct_reorder(categoria, ingresos))) +
5   # UN solo color: todas las barras miden lo mismo (ingresos)
6   geom_col(fill = color_principal, width = 0.7) +
7   scale_x_continuous(labels = dollar,
8   # las barras arrancan pegadas al eje (sin hueco)
9   expand = expansion(mult = c(0, 0.05))) +
10  labs(title = "Ingresos por categoría, 2023",
11    x = "Ingresos (MXN)", y = NULL,
12    caption = "Fuente: transacciones.csv y productos.csv") +
13  theme_minimal() +
14  theme(panel.grid.minor = element_blank(),
15    # sin líneas horizontales: las barras ya marcan las filas
16    panel.grid.major.y = element_blank())
17 g_categorias
```

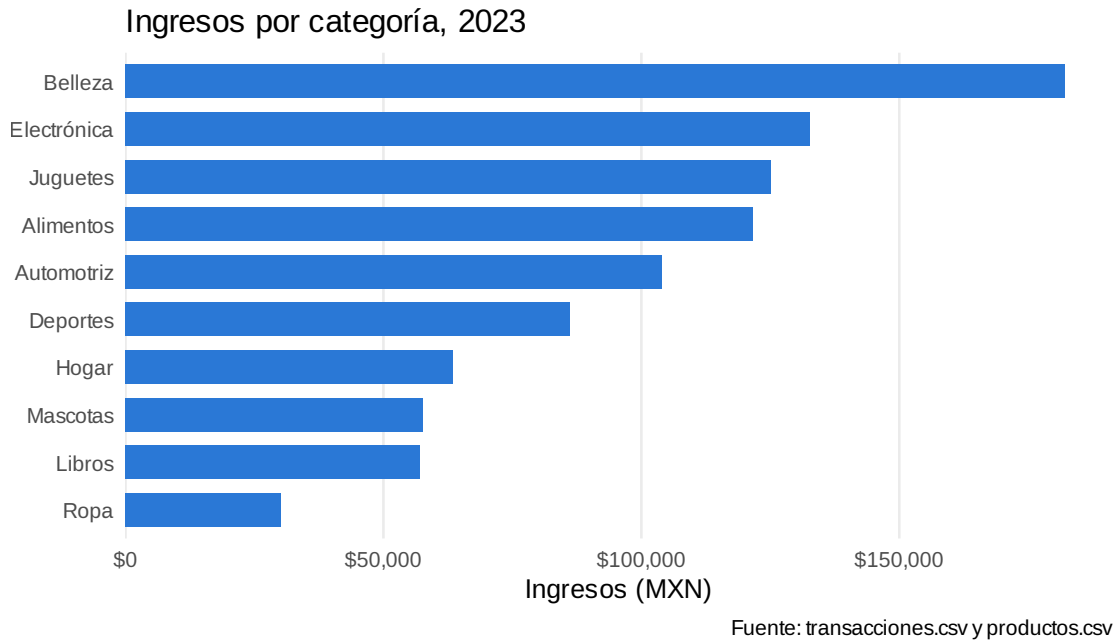



Figura 3.4. Ingresos por categoría: barras horizontales, ordenadas y de un solo color.

Cómo leerla. Belleza encabeza con claridad (\$182,143), seguida de un grupo muy parejo de cuatro categorías (Electrónica, Juguetes, Alimentos y Automotriz, entre \$104,000 y \$133,000). Ropa queda muy abajo, con apenas \$30,048. Como todas las barras tienen el mismo color, el ojo se concentra en lo único que importa: la longitud.

Concepto clave: Ordenar con `fct_reorder()` o `reorder()`

Por defecto **ggplot2** ordena el texto alfabéticamente (Alimentos, Automotriz, Belleza...), un orden que no significa nada para el negocio. `fct_reorder(categoria, ingresos)` (del paquete **forcats**, incluido en **tidyverse**) convierte la columna en un **factor** cuyos niveles siguen el orden de `ingresos`. La función base `reorder(categoria, ingresos)` hace lo mismo. Para ordenar de mayor a menor en un eje horizontal usa `.desc = TRUE`, o `reorder(categoria, -ingresos)`.

▲ Cuidado

En el material de muchos cursos verás `geom_col(aes(fill = categoria))`: cada barra de un color distinto. Si el color no significa nada nuevo (la categoría ya está escrita en el eje), solo agrega ruido, obliga a leer una leyenda inútil y hace creer que los colores codifican algo. Regla: **una sola serie = un solo color**.

3.4.3 Resaltar lo importante

Pregunta de negocio: en la presentación quieres hablar de Belleza. ¿Cómo haces que el ojo vaya directo a esa barra? Con la técnica más poderosa del diseño de gráficas: **un color de resalte para lo importante y gris para el resto**.

```

1 ingresos_categoria <- ingresos_categoria %>%
2   mutate(grupo = if_else(categoria == "Belleza", "Destacada", "Resto"))
3
4 g_resalte <- ggplot(ingresos_categoria,
5                     aes(x = ingresos,
6                         y = fct_reorder(categoria, ingresos),
7                         fill = grupo)) +      # el color SÍ significa algo
8   geom_col(width = 0.7) +
9   # etiqueta solo en la barra destacada (no un número en cada barra)
10  geom_text(data = filter(ingresos_categoria, grupo == "Destacada"),
11            aes(label = dollar(ingresos)),
12            hjust = 1.1, color = "white", fontface = "bold",
13            size = 3.5) +
14  scale_fill_manual(values = c(Destacada = color_resalte,
15                               Resto      = color_gris),
16                    guide = "none") +      # sin leyenda
17  scale_x_continuous(labels = dollar,
18                    expand = expansion(mult = c(0, 0.05))) +
19  labs(title = "Belleza aporta 19% de los ingresos del año",
20       subtitle = "Ingresos por categoría, 2023",
21       x = "Ingresos (MXN)", y = NULL) +
22  theme_minimal() +
23  theme(panel.grid.minor = element_blank(),
24        panel.grid.major.y = element_blank())
25 g_resalte

```



Figura 3.5. Un solo color de resalte y gris para el resto: el ojo va directo a Belleza.

Tres detalles del código:

- `scale_fill_manual()` asigna colores *por nombre* (`Destacada = color_resalte`), así nunca dependes del orden.
- `guide = "none"` quita la leyenda: el título ya explica qué barra se resalta y por qué.
- `geom_text()` recibe su propio `data` filtrado, de modo que solo aparece *una* etiqueta. `hjust = 1.1` la pone justo dentro del extremo de la barra.

Cómo leerla. El título ya es la conclusión: 19 % del ingreso anual viene de Belleza (fila 1 de la tabla: `participacion` de 0.190).

Hazlo tú

Resalta Ropa en lugar de Belleza y cambia el título por “Ropa aporta solo el 3 % de los ingresos”. Pista: solo cambian el `if_else()`, el título y la posición de la etiqueta (`hjust = -0.1` y `color = color_resalte`, porque la barra es muy corta para escribir dentro).

3.4.4 Barras agrupadas: comparar 2 o 3 series

Pregunta de negocio: ¿las cinco categorías principales vendieron lo mismo en el primer y en el segundo semestre? Aquí hay *dos series* (semestre 1 y semestre 2), así que ahora el color sí significa algo y necesitamos leyenda.

```
1 top5 <- ingresos_categoria %>%
2   slice_max(ingresos, n = 5) %>%      # las 5 con más ingresos
3   pull(categoria)                    # como vector de texto
4
5 ingresos_semestre <- ventas_detalle %>%
6   filter(categoria %in% top5) %>%
7   mutate(semestre = if_else(month(fecha) <= 6, "Ene-Jun", "Jul-Dic")) %>%
8   group_by(categoria, semestre) %>%
9   summarise(ingresos = sum(total_transaccion), .groups = "drop")
10
11 ingresos_semestre %>%
12   pivot_wider(names_from = semestre, values_from = ingresos) %>%
13   mutate(cambio = `Jul-Dic` / `Ene-Jun` - 1)
```

```
# A tibble: 5 x 4
  categoria `Ene-Jun` `Jul-Dic` cambio
  <chr>      <dbl>    <dbl>  <dbl>
1 Alimentos  61538.    60005. -0.0249
2 Automotriz  47343.    56742.  0.199
3 Belleza    114399.   67744. -0.408
4 Electrónica  58242.   74431.  0.278
5 Juguetes    63886.   61207. -0.0419
```

```

1 g_agrupadas <- ggplot(ingresos_semestre,
2                       aes(x = fct_reorder(categoria, ingresos,
3                                           .fun = sum, .desc = TRUE),
4                           y = ingresos, fill = semestre)) +
5   # dodge = una barra al lado de la otra (no encimadas)
6   geom_col(position = position_dodge(width = 0.8), width = 0.75) +
7   scale_fill_manual(values = paleta_libro[1:2]) + # 2 series, 2 colores
8   scale_y_continuous(labels = dollar,
9                       expand = expansion(mult = c(0, 0.05))) +
10  labs(title = "Belleza cayó 41% en el segundo semestre",
11       subtitle = "Ingresos de las 5 categorías principales, 2023",
12       x = NULL, y = "Ingresos (MXN)", fill = "Semestre") +
13  theme_minimal() +
14  theme(panel.grid.minor = element_blank(),
15        panel.grid.major.x = element_blank(),
16        legend.position = "top") # leyenda arriba
17 g_agrupadas

```

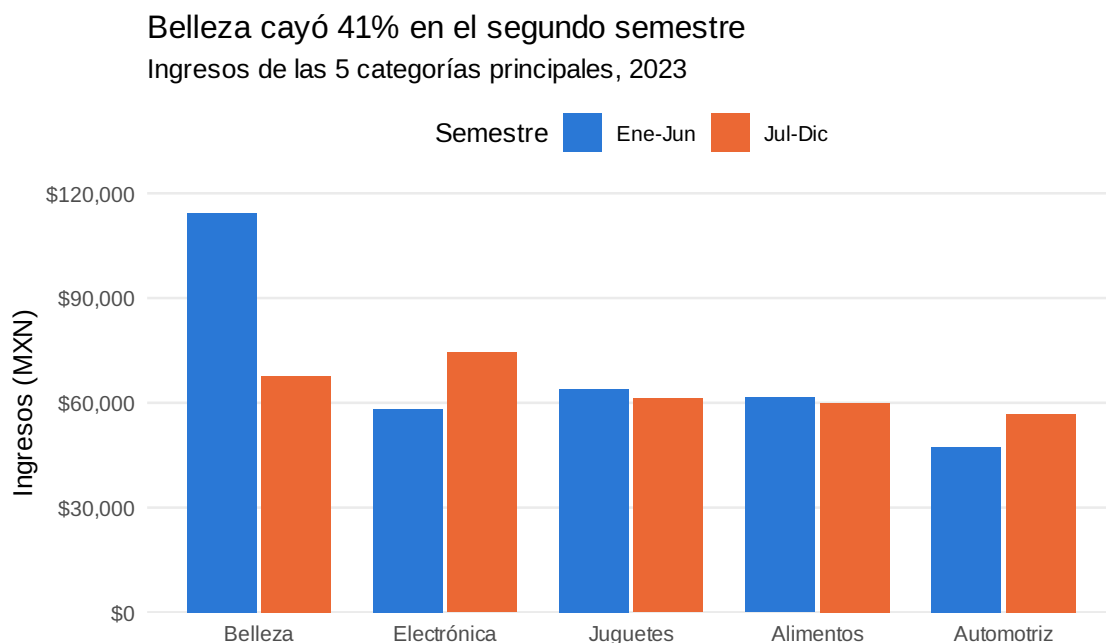


Figura 3.6. Barras agrupadas (*dodge*) para comparar dos series por categoría.

Cómo leerla. Cada categoría tiene su par de barras: azul para el primer semestre y naranja para el segundo. Belleza pasó de \$114,399 a \$67,744 (−41 %), mientras que Electrónica (+28 %) y Automotriz (+20 %) crecieron. Juguetes y Alimentos se mantuvieron casi iguales. Es una historia que la gráfica de totales anuales escondía por completo: Belleza lidera el año, pero viene en caída.

 Consejo

Las barras agrupadas funcionan bien con 2 o 3 series. Con más, el ojo ya no logra comparar; usa facetas (sección 3.9) o una gráfica de líneas.

3.4.5 Barras apiladas y barras al 100 %

Pregunta de negocio: ¿cambió la mezcla de métodos de pago a lo largo del año? Cuando lo que importa es la *composición* (qué parte del total aporta cada método) usamos barras al 100 %: `position = "fill"` apila y reescala cada barra para que sume 1.

```

1 pagos_trimestre <- ventas_detalle %>%
2   group_by(trimestre, metodo_pago) %>%
3   summarise(ingresos = sum(total_transaccion), .groups = "drop")
4
5 g_100 <- ggplot(pagos_trimestre,
6               aes(x = trimestre, y = ingresos,
7                 # el método más grande primero (abajo y en la leyenda)
8                 fill = fct_reorder(metodo_pago, ingresos,
9                                   .fun = sum, .desc = TRUE))) +
10  geom_col(position = position_fill(reverse = TRUE), width = 0.7) +
11  scale_fill_manual(values = paleta_libro[1:4]) +
12  scale_y_continuous(labels = percent) +          # 0.25 -> 25%
13  labs(title = "La mezcla de pagos casi no cambia en el año",
14       subtitle = "Participación de cada método de pago en los ingresos",
15       x = NULL, y = "% de los ingresos", fill = NULL) +
16  theme_minimal() +
17  theme(panel.grid.minor = element_blank(),
18        panel.grid.major.x = element_blank(),
19        legend.position = "top")
20 g_100

```

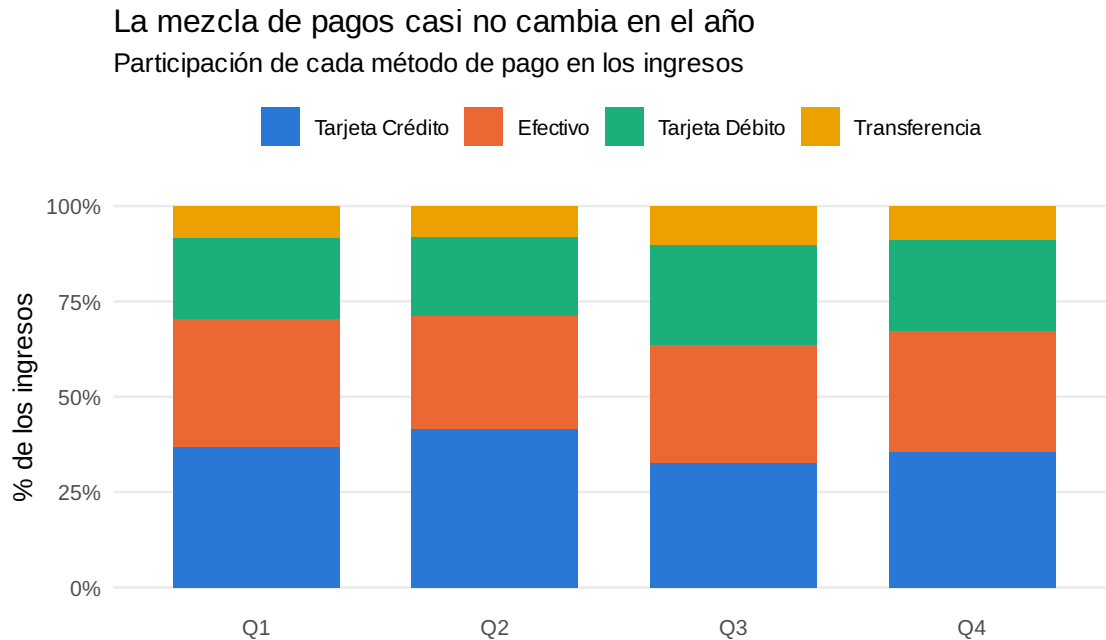


Figura 3.7. Barras al 100 %: composición de los ingresos por método de pago en cada trimestre.

Cómo leerla. En los cuatro trimestres Tarjeta de crédito ocupa algo más de un tercio (su mayor peso fue en Q2), Efectivo cerca de un tercio, Tarjeta de débito entre 20 % y 26 % y Transferencia menos del 11 %. No hay un cambio de hábito relevante. `position_fill(reverse = TRUE)` hace que el orden de las franjas (de abajo hacia arriba) coincida con el de la leyenda (de izquierda a derecha).

▲ Cuidado

En una barra apilada solo el segmento de abajo tiene una base común; los demás “flotan” y son difíciles de comparar entre barras. Úsalas con **pocas categorías** (2 a 4) y cuando te interese la composición. Si te interesa comparar el *monto* de cada método, usa barras agrupadas o facetas. Si quitas `position_fill()`, obtienes la versión apilada normal, donde la altura total es el ingreso del trimestre.

🔧 Hazlo tú

Cambia `position_fill(reverse = TRUE)` por `position_stack(reverse = TRUE)` y `labels = percent` por `labels = dollar`. ¿Qué trimestre tuvo más ingresos en total? ¿Es fácil comparar las franjas de Efectivo entre trimestres?

3.5 Líneas: la evolución en el tiempo

Cuando el eje x es el tiempo, la línea es la reina: la pendiente muestra si subimos o bajamos, y la continuidad sugiere que cada punto depende del anterior.

3.5.1 Ventas mensuales contra la meta

Pregunta de negocio: la meta comercial era vender \$80,000 al mes. ¿Cuántos meses la alcanzamos y cuál fue el mejor mes?

```
1 meta_mensual <- 80000
2 mes_record <- ventas_mes %>% slice_max(ingresos, n = 1) # mejor mes
3 sum(ventas_mes$ingresos >= meta_mensual) # meses que alcanzaron meta
```

```
[1] 7
```

```
1 g_linea <- ggplot(ventas_mes, aes(x = mes, y = ingresos)) +
2   # 1) la meta: línea horizontal punteada y gris (es referencia, no dato)
3   geom_hline(yintercept = meta_mensual, linetype = "dashed",
4             color = color_gris, linewidth = 0.6) +
5   # 2) los datos reales
6   geom_line(color = color_principal, linewidth = 0.8) +
7   geom_point(color = color_principal, size = 2) +
8   # 3) anotaciones: texto de la meta y punto + texto del récord
9   annotate("text", x = as.Date("2023-09-15"), y = meta_mensual,
10          label = "Meta mensual: $80,000", vjust = -0.6,
11          size = 3.2, color = "#52514e") +
12   annotate("point", x = mes_record$mes, y = mes_record$ingresos,
13          color = color_resalte, size = 3.5) +
14   annotate("text", x = mes_record$mes, y = mes_record$ingresos,
15          label = paste("Récord del año:",
16                        dollar(mes_record$ingresos, accuracy = 1)),
17          hjust = 1, vjust = -1.2, size = 3.2, color = color_resalte) +
18   # 4) eje x: una marca por mes, con el nombre corto del mes
19   scale_x_date(date_labels = "%b", date_breaks = "1 month") +
20   scale_y_continuous(labels = dollar,
21                     expand = expansion(mult = c(0.05, 0.15))) +
22   labs(title = "7 de 12 meses superaron la meta",
23        subtitle = "Ingresos mensuales 2023 contra la meta de $80,000",
24        x = NULL, y = "Ingresos (MXN)",
25        caption = "Fuente: transacciones.csv") +
26   theme_minimal() +
27   theme(panel.grid.minor = element_blank())
28 g_linea
```

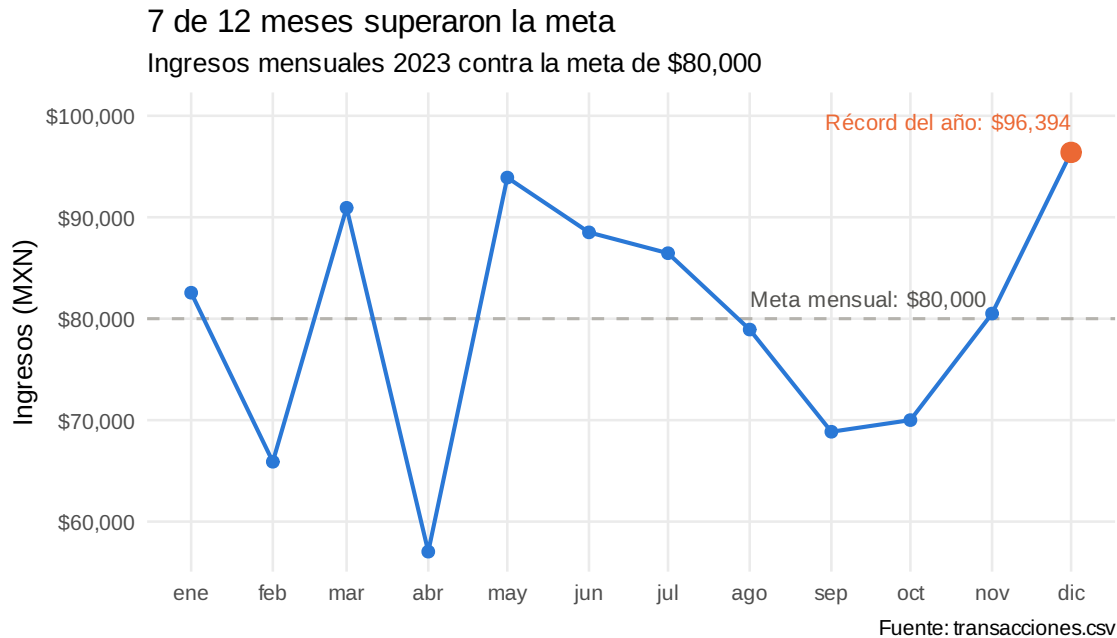


Figura 3.8. Línea de ingresos mensuales con la meta como referencia y una anotación en el mejor mes.

Lo nuevo en este código:

- `geom_hline(yintercept = ...)` dibuja una línea horizontal de referencia. Va *primero* para que quede detrás de los datos.
- `annotate()` agrega elementos sueltos que no vienen de una tabla: un texto, un punto, un rectángulo. Recibe las coordenadas en las unidades de los ejes (por eso x es una fecha).
- `scale_x_date(date_labels = "%b", date_breaks = "1 month")` pone una marca cada mes con la abreviatura del mes (`%b`: “ene”, “feb”..., en el idioma de tu sistema). Otros formatos útiles: `%B` (mes completo), `%Y` (año), `%d` (día), `"%b%Y"` (“ene 2023”).
- `expand = expansion(mult = c(0.05, 0.15))` deja 15 % de aire arriba del eje y para que quepa la anotación.

Cómo leerla. Siete meses (enero, marzo, mayo, junio, julio, noviembre y diciembre) quedaron sobre la línea punteada. Abril fue el peor mes (unos \$57,000, casi 30 % abajo de la meta) y diciembre el mejor (\$96,394). El tercer trimestre muestra una tendencia a la baja de julio a septiembre que se revierte a partir de octubre.

▲ Cuidado

Si quieres comparar los ingresos contra una meta, la meta debe ir **en el mismo eje** que los ingresos (como aquí, con `geom_hline()` o como segunda serie). Nunca uses un segundo eje Y para “hacer caber” otra medida: lo veremos en la sección 3.12.

3.5.2 Real contra meta: dos series con leyenda

En la vida real la meta cambia cada mes (diciembre suele tener una meta más alta). Construyamos una tabla de metas y grafiquemos ambas series. Para que **ggplot2** dibuje dos líneas con leyenda, los datos deben estar en **formato largo**: una columna con el nombre de la serie y otra con el valor.

```
1 metas <- tibble(
2   mes = seq(as.Date("2023-01-01"), as.Date("2023-12-01"),
3           by = "month"),
4   meta = c(75, 70, 80, 75, 80, 85, 85, 80, 75, 80, 85, 95) * 1000
5 )
6
7 real_vs_meta <- ventas_mes %>%
8   select(mes, Real = ingresos) %>%
9   left_join(rename(metas, Meta = meta), by = "mes") %>%
10  # de ancho (Real, Meta) a largo (serie, monto)
11  pivot_longer(cols = c(Real, Meta),
12              names_to = "serie", values_to = "monto") %>%
13  mutate(serie = fct_relevel(serie, "Real")) # "Real" primero
14 head(real_vs_meta, 4)
```

```
# A tibble: 4 x 3
  mes      serie monto
<date>   <fct> <dbl>
1 2023-01-01 Real  82560.
2 2023-01-01 Meta  75000
3 2023-02-01 Real  65899.
4 2023-02-01 Meta  70000
```

```
1 g_real_meta <- ggplot(real_vs_meta,
2                       aes(x = mes, y = monto,
3                           color = serie, linetype = serie)) +
4   geom_line(linewidth = 0.8) +
5   geom_point(data = filter(real_vs_meta, serie == "Real"),
6             size = 2) + # puntos solo en lo real
7   scale_color_manual(values = c(Real = color_principal,
8                                 Meta = color_gris)) +
9   scale_linetype_manual(values = c(Real = "solid", Meta = "dashed")) +
10  scale_x_date(date_labels = "%b", date_breaks = "1 month") +
11  scale_y_continuous(labels = dollar) +
12  labs(title = "Abril y octubre, los meses más lejos de su meta",
13       subtitle = "Ingresos reales contra meta mensual, 2023",
14       x = NULL, y = "Ingresos (MXN)", color = NULL,
15       linetype = NULL) +
16  theme_minimal() +
17  theme(panel.grid.minor = element_blank(),
```

```

18     legend.position = "top")
19 g_real_meta

```

Abril y octubre, los meses más lejos de su meta
Ingresos reales contra meta mensual, 2023

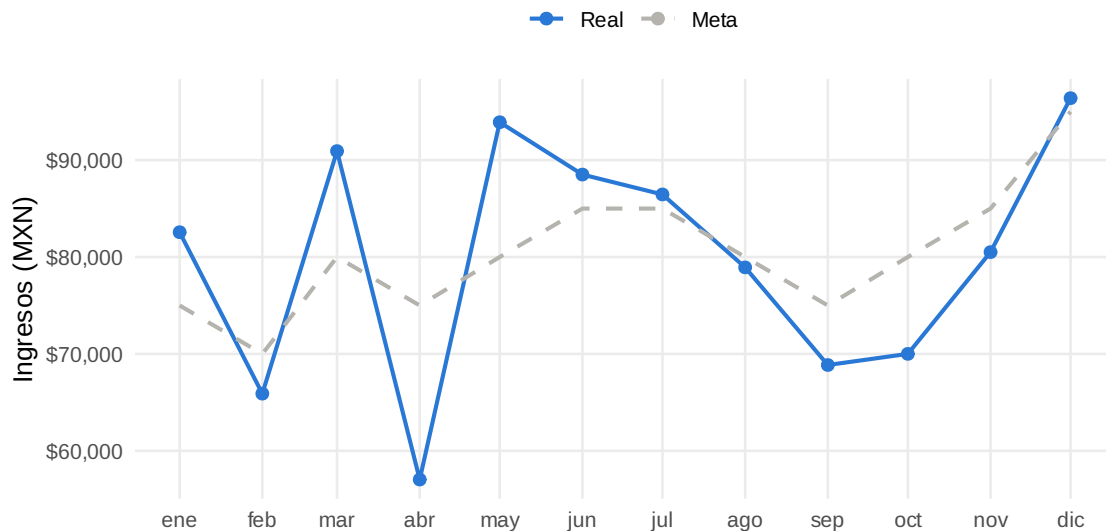


Figura 3.9. Dos series en el mismo eje: ingreso real (azul) y meta (gris punteado).

Cómo leerla. La meta es gris y punteada porque es una *referencia*; el dato protagonista es el azul. Como `color` y `linetype` usan la misma variable y el mismo título (`NULL`), **ggplot2** fusiona ambas en una sola leyenda. La brecha más grande está en abril (\$57,038 contra \$75,000) y en octubre (\$70,004 contra \$80,000).

3.5.3 Varias series: resaltar una y dejar el resto en gris

Pregunta de negocio: ¿cómo avanzó cada tienda a lo largo del año? Diez líneas de diez colores serían un “espagueti” ilegible (y la paleta no tiene diez colores). La solución profesional: **todas en gris y solo la que importa en color**. Usamos el ingreso *acumulado* (`cumsum()`) porque suaviza el ruido mensual y muestra quién va ganando la carrera.

```

1 acumulado_tienda <- ventas_detalle %>%
2   mutate(mes = floor_date(fecha, unit = "month")) %>%
3   group_by(tienda, mes) %>%
4   summarise(ingresos = sum(total_transaccion), .groups = "drop") %>%
5   arrange(tienda, mes) %>%
6   group_by(tienda) %>%
7   mutate(acumulado = cumsum(ingresos)) %>%      # suma corrida por tienda
8   ungroup() %>%
9   mutate(grupo = if_else(tienda == "Outlet", "Outlet",

```

```

10         "Otras 9 tiendas"))
11
12 g_carrera <- ggplot(acumulado_tienda,
13                   aes(x = mes, y = acumulado,
14                     group = tienda, color = grupo)) +
15   # primero las grises (quedan al fondo), luego la destacada encima
16   geom_line(data = filter(acumulado_tienda, grupo != "Outlet"),
17             linewidth = 0.6) +
18   geom_line(data = filter(acumulado_tienda, grupo == "Outlet"),
19             linewidth = 1.1) +
20   scale_color_manual(values = c("Outlet" = color_resalte,
21                                "Otras 9 tiendas" = color_gris)) +
22   scale_x_date(date_labels = "%b", date_breaks = "1 month") +
23   scale_y_continuous(labels = dollar) +
24   labs(title = "Outlet rebasó a todas las tiendas en el último trimestre",
25        subtitle = "Ingresos acumulados por tienda, 2023",
26        x = NULL, y = "Ingresos acumulados (MXN)", color = NULL) +
27   theme_minimal() +
28   theme(panel.grid.minor = element_blank(),
29         legend.position = "top")
30 g_carrera

```

Outlet rebasó a todas las tiendas en el último trimestre
Ingresos acumulados por tienda, 2023

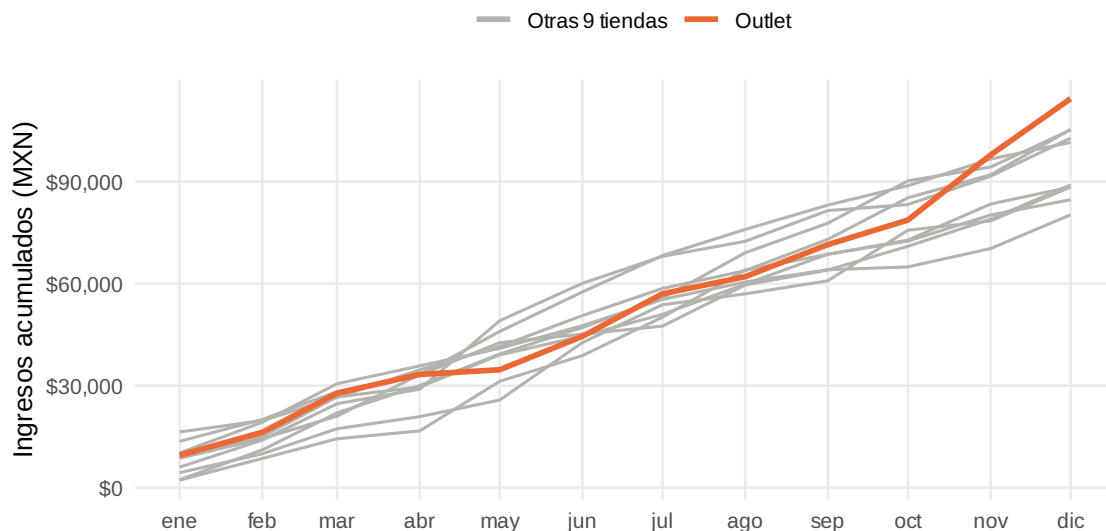


Figura 3.10. Diez series, una resaltada: la técnica de “gris y un color”.

Cómo leerla. Durante la primera mitad del año Outlet va en el montón (incluso por debajo de varias tiendas en mayo), pero en el último trimestre acelera, toma el primer lugar en noviembre y cierra el año arriba de todas. Observa el papel de `group = tienda`: le dice a **ggplot2** que dibuje una línea *por tienda*, aunque el color solo distinga dos grupos. Sin `group`, intentaría unir los puntos

de las nueve tiendas grises en una sola línea en zigzag.

Hazlo tú

Resalta la tienda Plaza B (la de menores ingresos) en lugar de Outlet. Cambia también el título por uno que diga la conclusión que ves.

3.6 Dispersión: la relación entre dos medidas

Un **diagrama de dispersión** (*scatter plot*) pone un punto por observación, con una medida en cada eje. Sirve para responder “¿cuando sube esto, sube (o baja) aquello?”.

3.6.1 Precio contra cantidad

Pregunta de negocio: ¿los clientes compran menos piezas cuando el precio es más alto? Si fuera así, subir precios reduciría el volumen.

```

1 g_precio_cantidad <- ggplot(ventas_detalle,
2                               aes(x = precio_venta, y = cantidad)) +
3   # jitter: mueve cada punto un poquito en vertical para que no se
4   # encimen (cantidad solo vale 1, 2, 3, 4 o 5); alpha = transparencia
5   geom_point(position = position_jitter(width = 0, height = 0.15,
6                                         seed = 123),
7              color = color_principal, alpha = 0.35) +
8   # recta de tendencia por mínimos cuadrados con su banda de confianza
9   geom_smooth(method = "lm", formula = y ~ x,
10              color = color_resalte, linewidth = 0.8) +
11   scale_x_continuous(labels = dollar) +
12   scale_y_continuous(breaks = 1:5) +
13   labs(title = "El precio no cambia cuántas piezas lleva el cliente",
14        subtitle = "Cada punto es un ticket de 2023 (1,000 en total)",
15        x = "Precio unitario (MXN)", y = "Piezas por ticket") +
16   theme_minimal() +
17   theme(panel.grid.minor = element_blank())
18 g_precio_cantidad
19
20 cor(ventas_detalle$precio_venta, ventas_detalle$cantidad)
```

```
[1] 0.0602253
```

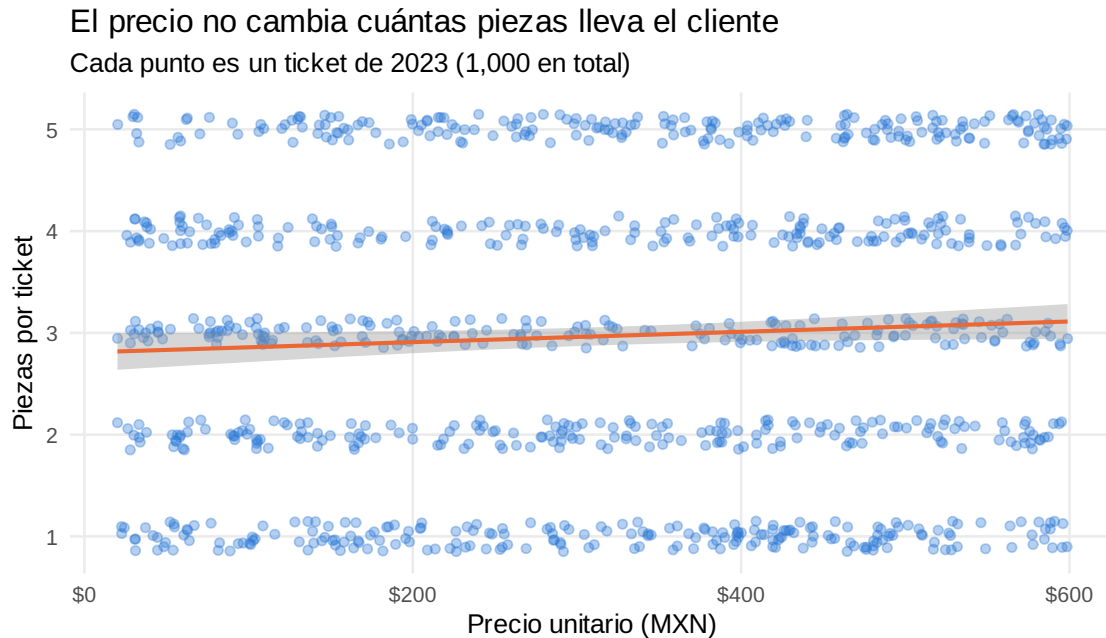


Figura 3.11. Precio contra cantidad por ticket, con transparencia, un ligero desplazamiento vertical (*jitter*) y recta de tendencia.

Cómo leerla. Los puntos llenan todo el rectángulo de manera pareja y la recta naranja es prácticamente horizontal: la correlación es de apenas 0.06. En estos datos, el precio *no* explica cuántas piezas lleva el cliente. Dos técnicas hicieron legible la gráfica:

- **Transparencia** (`alpha = 0.35`): donde se enciman muchos puntos, el color se ve más intenso. Sin transparencia, 1,000 puntos se vuelven una mancha.
- **Jitter**: como la cantidad es un número entero, todos los tickets de 3 piezas caerían en la misma línea exacta. `position_jitter(height = 0.15)` los separa un poco en vertical *solo para dibujarlos*; los datos no cambian. `seed = 123` hace que el desplazamiento sea siempre el mismo.

Concepto clave: `geom_smooth()`

`geom_smooth(method = "lm")` ajusta un modelo lineal (una recta) y la dibuja con una banda gris: el intervalo de confianza del 95 %. Escribir `formula = y ~ x` evita el mensaje informativo que **ggplot2** imprime cuando no la indicas. Con `se = FALSE` quitas la banda. Si omites `method`, dibuja una curva suavizada (*loess*), útil para explorar si la relación no es recta.

3.6.2 Color por grupo y escalas logarítmicas

Pregunta de negocio: Marketing invirtió en 120 campañas por tres canales (correo, redes sociales y radio), con presupuestos desde \$1,000 hasta \$1,000,000. ¿Qué canal devuelve más ventas por peso invertido? Como nuestros archivos no tienen campañas, las *simulamos* con una relación conocida (cada canal tiene un retorno distinto) para ver cómo se ve.

```

1  set.seed(123)
2  campanas <- tibble(
3    canal      = rep(c("Correo", "Redes", "Radio"), each = 40),
4    # inversiones entre 10^3 y 10^6: $1,000 a $1,000,000
5    inversion = round(10^runif(120, min = 3, max = 6))
6  ) %>%
7    mutate(retorno = case_when(canal == "Correo" ~ 6,
8                               canal == "Redes"  ~ 3,
9                               canal == "Radio"  ~ 1.5),
10           ventas = round(inversion * retorno *
11                           exp(rnorm(n(), mean = 0, sd = 0.35))))
12
13 # formato corto: 1500000 -> $1.5M
14 dinero_corto <- label_dollar(scale_cut = cut_short_scale())
15
16 g_campanas <- ggplot(campanas, aes(x = inversion, y = ventas,
17                                   color = canal)) + # 3 grupos
18   geom_point(alpha = 0.7) +
19   scale_color_manual(values = paleta_libro[1:3]) +
20   labs(x = "Inversión", y = "Ventas", color = NULL) +
21   theme_minimal() +
22   theme(panel.grid.minor = element_blank(),
23         legend.position = "top",
24         plot.margin = margin(5, 15, 5, 5))
25
26 g_lineal <- g_campanas +
27   scale_x_continuous(labels = dinero_corto,
28                     breaks = c(0, 500000, 1000000)) +
29   scale_y_continuous(labels = dinero_corto) +
30   labs(title = "Escala lineal")
31
32 g_log <- g_campanas +
33   geom_smooth(method = "lm", formula = y ~ x, se = FALSE,
34              linewidth = 0.8) +
35   scale_x_log10(labels = dinero_corto) + # ejes logarítmicos
36   scale_y_log10(labels = dinero_corto) +
37   labs(title = "Escala logarítmica")
38 g_lineal
39 g_log

```

Observa una técnica muy práctica: `g_campanas` es una gráfica “base” guardada en un objeto, y `g_lineal` y `g_log` le *suman* capas distintas. Así no repites código.

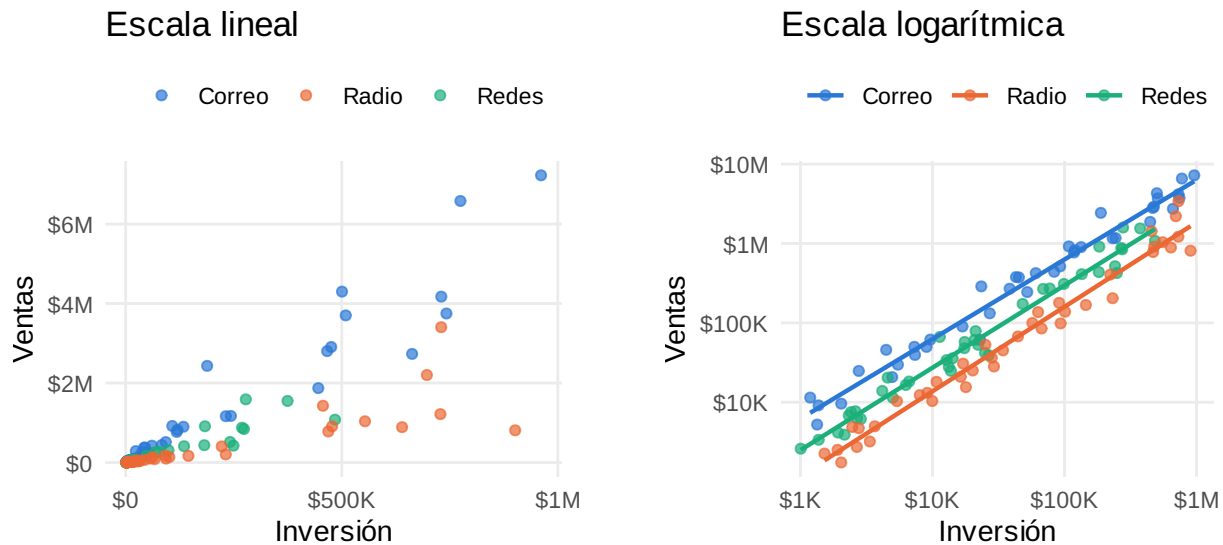


Figura 3.12. Los mismos datos en escala lineal (izquierda) y logarítmica (derecha).

Cómo leerla. En la escala lineal casi todos los puntos se amontonan en la esquina inferior izquierda: las pocas campañas millonarias dominan el espacio. En la **escala logarítmica** cada marca del eje multiplica por 10 (\$1K, \$10K, \$100K, \$1M), así que las campañas pequeñas y grandes tienen el mismo espacio. Aparecen tres franjas paralelas: para cualquier nivel de inversión, Correo (azul) queda arriba, luego Redes y abajo Radio. Correo es el canal más rentable, sin importar el tamaño de la campaña.

Consejo

Usa escala logarítmica cuando tus datos abarcan varios órdenes de magnitud (ventas por cliente, tamaño de empresas, inversión) y cuando te interesan las *proporciones* más que las diferencias absolutas. Dilo siempre en el subtítulo o en el eje: mucha gente no está acostumbrada a leerla. Y limita el color a **tres grupos como máximo** en una dispersión; con más, los puntos se vuelven confeti.

Hazlo tú

En `g_precio_cantidad` quita `position_jitter(...)` y cambia `alpha` a 1. ¿Cuántos puntos crees ver ahora? Compara con la figura 3.11.

3.7 Distribuciones: cómo se reparten los datos

El promedio del ticket es \$960, pero ¿la mayoría de los tickets andan cerca de \$960? ¿Hay muchos tickets chicos y pocos enormes? Para responder graficamos la **distribución**: cuántas observaciones caen en cada rango de valores.

3.7.1 Histograma y densidad

Pregunta de negocio: ¿de qué tamaño son los tickets típicos?

```

1 g_hist <- ggplot(ventas_detalle, aes(x = total_transaccion)) +
2   # binwidth = ancho de cada barra ($200); boundary = 0 hace que los
3   # cortes caigan en 0, 200, 400...
4   geom_histogram(binwidth = 200, boundary = 0,
5                 fill = color_principal, color = "white") +
6   scale_x_continuous(labels = dollar) +
7   labs(title = "Histograma (conteos)", x = "Importe del ticket",
8        y = "Número de tickets") +
9   theme_minimal() +
10  theme(panel.grid.minor = element_blank())
11
12 g_densidad <- ggplot(ventas_detalle, aes(x = total_transaccion)) +
13   # after_stat(density): la altura pasa de conteo a densidad para que
14   # el histograma y la curva compartan la misma escala
15   geom_histogram(aes(y = after_stat(density)), binwidth = 200,
16                 boundary = 0, fill = color_gris, color = "white") +
17   geom_density(color = color_principal, linewidth = 0.8) +
18   scale_x_continuous(labels = dollar) +
19   scale_y_continuous(labels = label_number(accuracy = 0.0001)) +
20   labs(title = "Histograma + densidad", x = "Importe del ticket",
21        y = "Densidad") +
22   theme_minimal() +
23   theme(panel.grid.minor = element_blank())
24 g_hist
25 g_densidad

```

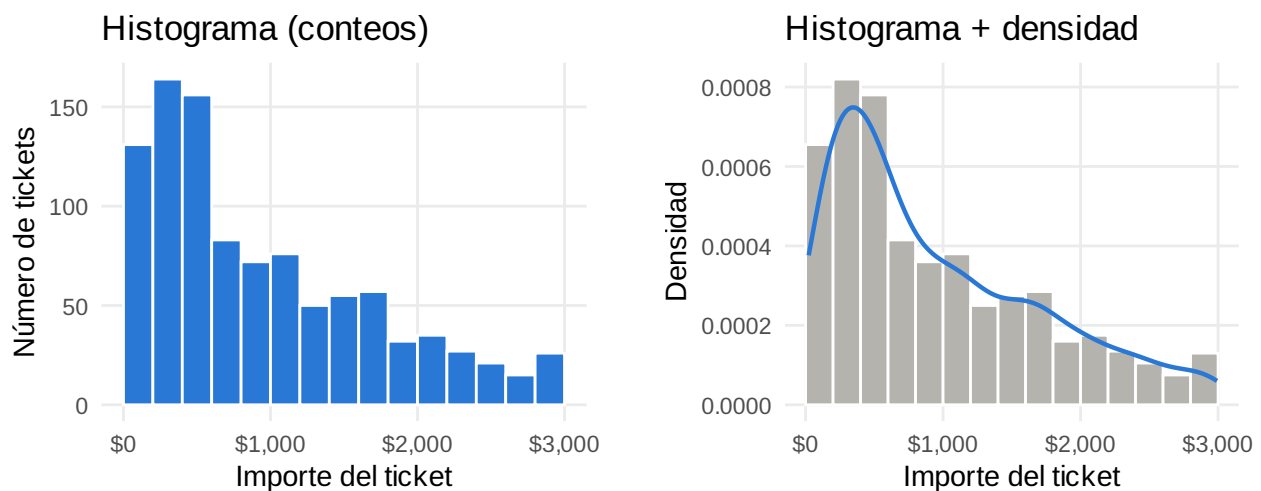


Figura 3.13. Distribución del importe por ticket: histograma de conteos (izquierda) y la misma distribución como densidad (derecha).

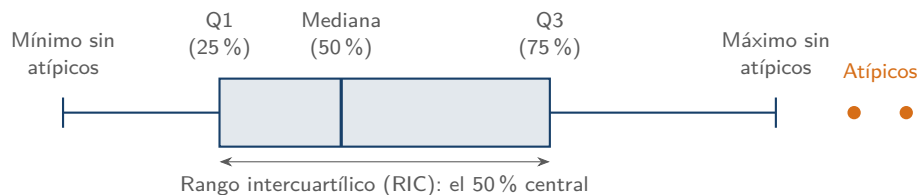
Cómo leerla. La distribución es **asimétrica a la derecha**: las barras más altas están entre \$0 y \$600 (ahí cae el 45 % de los tickets), y luego hay una “cola” larga de tickets cada vez menos frecuentes que llega hasta casi \$3,000. Por eso el promedio (\$960) es bastante mayor que la mediana (\$713): los tickets grandes jalan el promedio hacia arriba.

Concepto clave: bins, binwidth y after_stat()

Un histograma divide el eje x en intervalos (*bins*) y cuenta cuántas observaciones caen en cada uno. Puedes fijar el *número* de intervalos (`bins = 15`) o su *ancho* (`binwidth = 200`). En negocio suele ser más claro el ancho: “cada barra son \$200”. Prueba varios valores: con muy pocos intervalos pierdes la forma, con demasiados ves solo ruido. `after_stat(density)` usa una variable que **ggplot2** calcula al construir el histograma; en código antiguo verás `..density..`, que hoy está obsoleto.

3.7.2 El diagrama de caja (boxplot)

El **boxplot** resume una distribución en cinco números y marca los atípicos. Es ideal para *comparar* distribuciones entre grupos. La figura 3.14 explica cómo leerlo.



Los bigotes llegan hasta el dato más extremo que esté a menos de $1.5 \times \text{RIC}$ de la caja; lo que queda más lejos es atípico.

Figura 3.14. Anatomía de un boxplot.

Pregunta de negocio: ¿el importe de los tickets es distinto según el método de pago? Primero los números, luego la gráfica.

```
1 ventas_detalle %>%
2   group_by(metodo_pago) %>%
3   summarise(q1      = quantile(total_transaccion, 0.25),
4             mediana = median(total_transaccion),
5             q3      = quantile(total_transaccion, 0.75),
6             promedio = mean(total_transaccion),
7             .groups = "drop") %>%
8   arrange(desc(mediana))
```

```
# A tibble: 4 x 5
  metodo_pago      q1 mediana    q3 promedio
  <chr>          <dbl>   <dbl> <dbl>   <dbl>
1 Tarjeta Crédito 334.    780. 1586.   1003.
2 Tarjeta Débito  340.    732. 1415.    948.
```

3 Efectivo	383.	710.	1479.	964.
4 Transferencia	267.	608.	1121.	826.

```

1 g_caja <- ggplot(ventas_detalle,
2   aes(x = total_transaccion,
3       # ordena los métodos por su mediana
4       y = fct_reorder(metodo_pago, total_transaccion,
5                       .fun = median))) +
6   geom_boxplot(fill = "#cde2fb", color = color_principal,
7               width = 0.6, outlier.color = color_resalte) +
8   # agrega el promedio como un rombo oscuro
9   stat_summary(fun = mean, geom = "point", shape = 18, size = 3,
10               color = "#0d366b") +
11   scale_x_continuous(labels = dollar) +
12   labs(title = "Transferencia: tickets más bajos y varios atípicos",
13        subtitle = "Importe por ticket según método de pago (rombo = promedio)",
14        x = "Importe del ticket (MXN)", y = NULL) +
15   theme_minimal() +
16   theme(panel.grid.minor = element_blank(),
17         panel.grid.major.y = element_blank())
18 g_caja

```

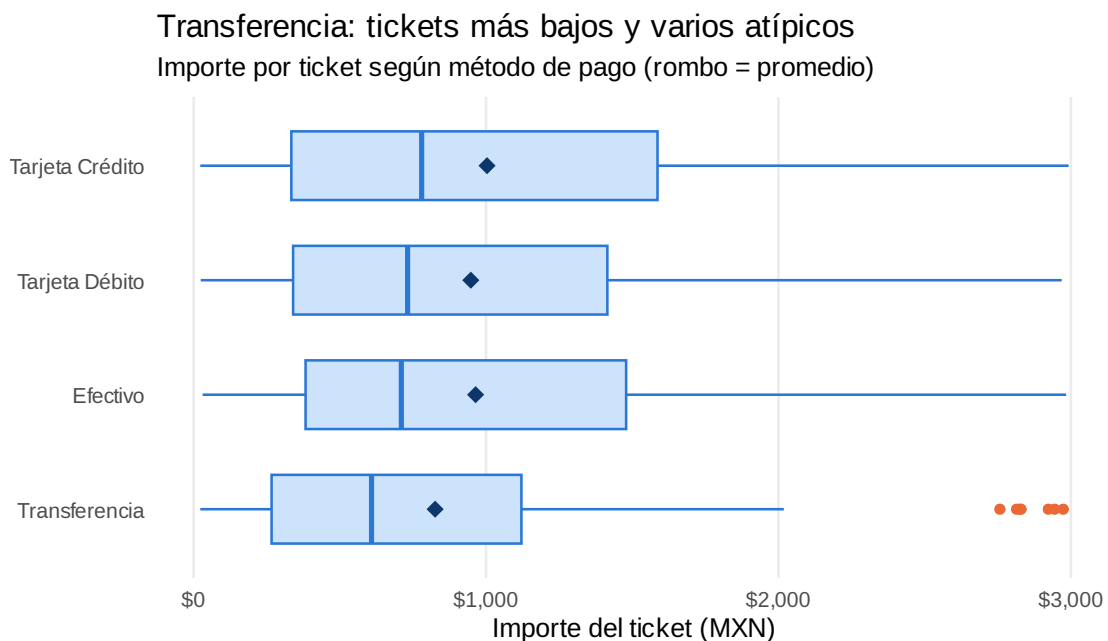


Figura 3.15. Boxplot del importe por ticket según método de pago, con el promedio marcado.

Cómo leerla. Tarjeta de crédito tiene la mediana más alta (\$780) y la caja más ancha: sus tickets son más variados. Transferencia tiene la mediana más baja (\$608) y su caja es más corta; por eso sus tickets de más de \$2,700 quedan marcados como atípicos (puntos naranjas), mientras que en

los otros métodos un ticket de ese tamaño cae dentro del bigote. En todos los métodos el promedio (rombo) queda a la derecha de la mediana: de nuevo, la asimetría que vimos en el histograma.

Consejo

Un “atípico” en un boxplot no es un error: es un dato alejado de la mayoría. En BI conviene revisarlos (¿es un cliente mayorista?, ¿un error de captura?), pero nunca borrarlos solo porque el boxplot los marca.

Variante: gráfica de violín. `geom_violin()` dibuja la densidad de cada grupo en espejo, de modo que ves la *forma* completa de la distribución y no solo cinco números. Es útil cuando sospechas que un grupo tiene dos “jorobas” (por ejemplo, clientes minoristas y mayoristas). Para una audiencia ejecutiva, el boxplot suele ser más fácil de explicar.

```
1 g_violin <- ggplot(ventas_detalle,
2                   aes(x = total_transaccion, y = metodo_pago)) +
3   geom_violin(fill = "#cde2fb", color = color_principal) +
4   scale_x_continuous(labels = dollar) +
5   theme_minimal()
```

Hazlo tú

Haz un boxplot del importe por ticket según **categoría** (10 cajas). ¿Qué categoría tiene la mediana más alta? ¿Coincide con la que más ingresos totales tiene? Pista: los ingresos totales dependen del importe *y* del número de tickets.

3.8 Mapa de calor: dos categorías y un valor

Un **mapa de calor** (*heatmap*) es una tabla donde cada celda se colorea según su valor. Funciona muy bien para cruzar dos variables categóricas u ordenadas (día × mes, tienda × categoría, hora × día).

Pregunta de negocio: ¿hay días de la semana que vendan sistemáticamente más? ¿Depende del mes? `month(fecha, label = TRUE)` y `wday(fecha, label = TRUE, week_start = 1)` (de **lubridate**) devuelven el mes y el día como *factores ordenados* (“ene” < “feb”...; “lun” < “mar”...), así que el orden correcto viene gratis.

```
1 calor <- ventas_detalle %>%
2   mutate(mes = month(fecha, label = TRUE),                # ene, feb...
3         dia = wday(fecha, label = TRUE, week_start = 1)) %>% # lun...dom
4   group_by(mes, dia) %>%
5   summarise(ingresos = sum(total_transaccion), .groups = "drop")
6
7 calor %>% slice_max(ingresos, n = 3)
8 calor %>% slice_min(ingresos, n = 1)
```

```
# A tibble: 3 x 3
  mes   dia   ingresos
<ord> <ord>   <dbl>
1 ene   lun    21287.
2 may   vie    21079.
3 dic   dom    20350.
# A tibble: 1 x 3
  mes   dia   ingresos
<ord> <ord>   <dbl>
1 abr   mar     303.
```

```
1 g_calor <- ggplot(calor, aes(x = mes, y = fct_rev(dia), fill = ingresos)) +
2   # cada celda es un rectángulo; el borde blanco separa las celdas
3   geom_tile(color = "white", linewidth = 0.6) +
4   # gradiente SECUENCIAL de un solo tono: claro = poco, oscuro = mucho
5   scale_fill_gradient(low = "#cde2fb", high = "#0d366b",
6                       labels = dollar) +
7   labs(title = "Los días fuertes cambian de un mes a otro",
8        subtitle = "Ingresos por día de la semana y mes, 2023",
9        x = NULL, y = NULL, fill = "Ingresos") +
10  theme_minimal() +
11  theme(panel.grid = element_blank(),      # sin cuadrícula
12        legend.position = "right")
13 g_calor
```

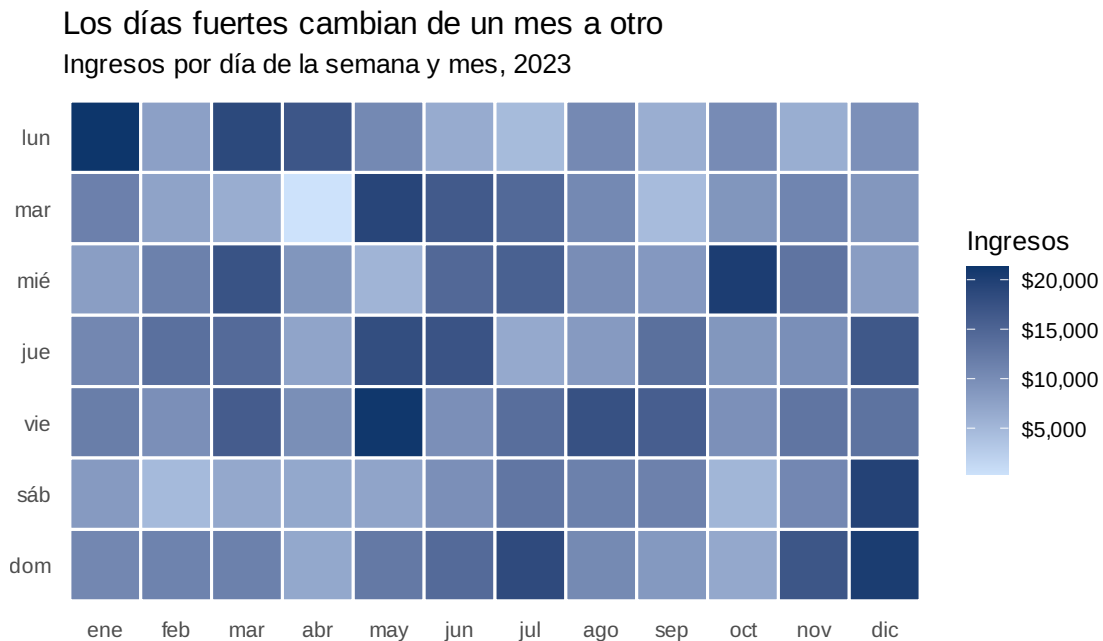


Figura 3.16. Mapa de calor de ingresos por día de la semana y mes, con un gradiente secuencial de un solo tono.

Cómo leerla. Las celdas oscuras son las combinaciones con más ingresos: los lunes de enero (\$21,287), los viernes de mayo y los domingos de diciembre. La celda más clara, casi blanca, es el martes de abril: solo \$303 en todo el mes. Pero no hay una *fila* oscura completa: ningún día de la semana es fuerte todo el año. `fct_rev(dia)` invierte el orden para que el lunes quede arriba, como en un calendario.

Concepto clave: Escala secuencial contra arcoíris

Para un valor que va de “poco” a “mucho” usa una **escala secuencial**: *un solo tono* que va de claro a oscuro. El cerebro entiende de inmediato que más oscuro es más. Una escala arcoíris (rojo, amarillo, verde, azul...) no tiene un orden natural, crea “fronteras” falsas donde cambia el tono y es ilegible para personas con daltonismo. Si tus datos tienen un centro con significado (por ejemplo, crecimiento negativo contra positivo) usa una **escala divergente**: dos tonos opuestos con gris claro en el centro, con `scale_fill_gradient2(low = ..., mid = "#f0efec", high = ..., midpoint = 0)`.

Hazlo tú

Cambia `ingresos = sum(total_transaccion)` por `tickets = n()` y ajusta `fill` y `labels = comma`. ¿Las celdas oscuras coinciden con las del mapa de ingresos?

3.9 Facetas: muchas gráficas pequeñas

Las **facetas** (*small multiples*) dividen los datos en paneles pequeños, uno por grupo, con los mismos ejes. En lugar de diez líneas de colores encimadas tienes diez gráficas limpias que el ojo compara de un vistazo.

Pregunta de negocio: ¿cómo evolucionó cada categoría mes a mes?

```
1 categoria_mes <- ventas_detalle %>%
2   mutate(mes = floor_date(fecha, unit = "month")) %>%
3   group_by(categoria, mes) %>%
4   summarise(ingresos = sum(total_transaccion), .groups = "drop") %>%
5   # si una categoría no vendió en un mes, ese mes no existe: lo creamos
6   # con ingresos = 0 para que la línea no "salte" el mes
7   complete(categoria, mes, fill = list(ingresos = 0))
8
9 g_facetas <- ggplot(categoria_mes, aes(x = mes, y = ingresos)) +
10  geom_line(color = color_principal, linewidth = 0.7) +
11  # un panel por categoría, ordenados de mayor a menor ingreso anual
12  facet_wrap(~ fct_reorder(categoria, ingresos, .fun = sum,
13                           .desc = TRUE),
14            ncol = 5) +
15  scale_x_date(date_labels = "%b", date_breaks = "4 months") +
16  scale_y_continuous(labels = label_number(scale = 1e-3)) +
17  labs(title = "Ingresos mensuales por categoría (misma escala)",
```

```

18     x = NULL, y = "Ingresos (miles de MXN)" +
19     theme_minimal() +
20     theme(panel.grid.minor = element_blank())
21 g_facetas

```

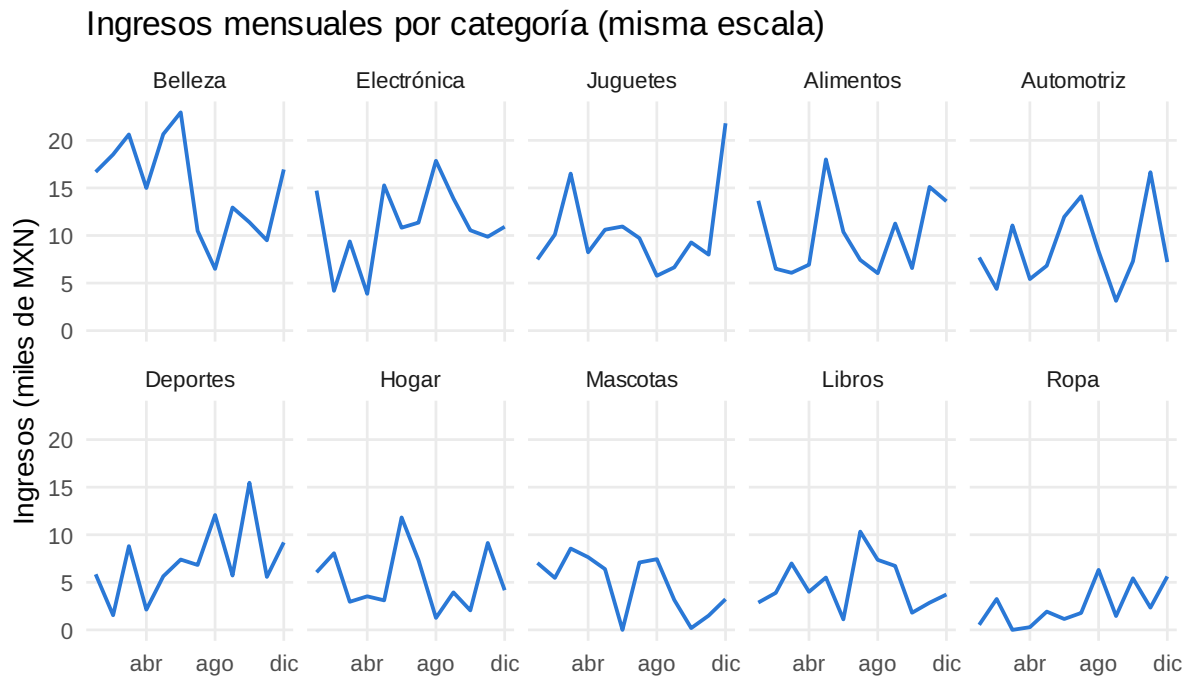


Figura 3.17. Facetas con la misma escala en todos los paneles: se compara el *tamaño* de cada categoría.

Cómo leerla. Con la misma escala en todos los paneles salta a la vista el tamaño: Belleza se mueve entre unos \$6,500 y \$23,000 al mes, mientras que Ropa nunca pasa de \$6,500. También se ve la caída de Belleza a partir de julio. `complete()` (de `tidyr`) fue necesario porque Mascotas y Ropa tuvieron un mes sin ventas; sin él, la línea uniría directamente el mes anterior con el siguiente y ocultaría el hueco.

3.9.1 ¿Escala fija o libre?

Con `scales = "free_y"` cada panel tiene su propio eje *y*.

```

1 g_facetas_libre <- g_facetas +
2   facet_wrap(~ fct_reorder(categoria, ingresos, .fun = sum,
3     .desc = TRUE),
4     ncol = 5, scales = "free_y") + # eje y propio por panel
5   labs(title = "Ingresos mensuales por categoría (escala libre)")
6 g_facetas_libre

```

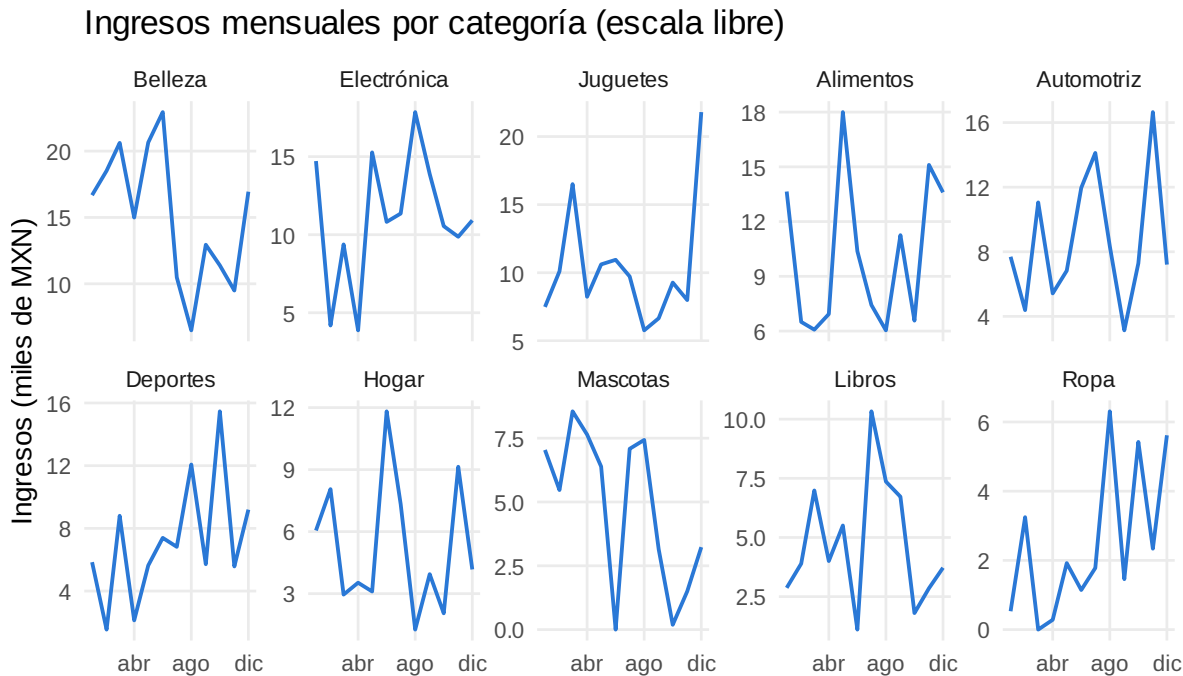


Figura 3.18. Las mismas facetas con `scales = "free_y"`: se compara la *forma* de cada serie, no su tamaño.

Cómo leerla. Ahora Ropa ocupa todo su panel y se ve su propio patrón (picos en agosto, octubre y diciembre), pero ya no puedes comparar tamaños entre paneles: el pico de Ropa (unos \$6,000) parece tan alto como el de Juguetes (más de \$20,000). La regla:

- **Escala fija** (por defecto) si la pregunta es “¿quién es más grande?”.
- **Escala libre** (`"free_y"`) si la pregunta es “¿qué patrón sigue cada uno?” (estacionalidad, tendencia). Avísalo en el título o subtítulo, porque es fácil malinterpretarla.

`facet_wrap()` acomoda los paneles en filas y columnas (`ncol`, `nrow`). Su pariente `facet_grid()` crea una cuadrícula con dos variables, una en las filas y otra en las columnas; por ejemplo, `trimestre` en las filas y `metodo_pago` en las columnas.

Hazlo tú

Haz facetas por `tienda` (10 paneles) del ingreso mensual. ¿Qué tienda tiene el mes más alto? ¿Conviene aquí escala fija o libre?

3.10 Escalas y formato de negocio

Una gráfica de negocio con el eje en “1e+05” o “0.25” no está terminada. El paquete `scales` convierte números en formatos que la dirección lee sin pensar. Cada función existe en dos versiones: `dollar(x)` formatea un número ya, y `label_dollar()` crea una *función* de formato que le pasas a un eje con `labels =`.

```

1 montos <- c(500, 25000, 750000, 1200000, 3500000)
2 dollar(montos)                                # moneda
3 comma(montos)                                # separador de miles
4 percent(c(0.19, 0.0877, 0.366), accuracy = 0.1) # porcentajes
5 label_dollar(scale = 1e-3, suffix = "K")(montos) # en miles
6 label_number(scale_cut = cut_short_scale()(montos) # K, M automáticos
7 label_dollar(scale_cut = cut_short_scale()(montos)

```

```

[1] "$500"      "$25,000"    "$750,000"   "$1,200,000" "$3,500,000"
[1] "500"       "25,000"     "750,000"    "1,200,000"  "3,500,000"
[1] "19.0%"    "8.8%"      "36.6%"
[1] "$0.50K"    "$25.00K"    "$750.00K"   "$1,200.00K" "$3,500.00K"
[1] "500"      "25K"       "750K"       "1.2M"       "3.5M"
[1] "$500"     "$25K"      "$750K"      "$1.2M"      "$3.5M"

```

Observa la cuarta línea: cuando los números son muy distintos entre sí, `label_dollar()` agrega decimales para que ninguno se vea igual (\$0.50K). En un eje, con marcas redondas, verás \$250K, \$500K; si quieres controlarlo tú, usa el argumento `accuracy` (por ejemplo, `accuracy = 1` para no mostrar decimales).

Tabla 3.2. Funciones de formato más usadas (paquete **scales**).

En el eje	Resultado
<code>labels = dollar</code>	\$1,200,000
<code>labels = comma</code>	1,200,000
<code>labels = percent</code>	0.25 → 25 %
<code>labels =</code>	25.3 %
<code>label_percent(accuracy = 0.1)</code>	
<code>labels = label_dollar(scale = 1e-3, suffix = "K")</code>	\$1,200K
<code>labels =</code>	\$1.2M, \$750K
<code>label_dollar(scale_cut = cut_short_scale())</code>	
<code>labels = label_number(big.mark = ".", decimal.mark = ",")</code>	1.200.000 (formato europeo)

▲ Cuidado

En la versión 1.3.0 de **scales**, `cut_short_scale()` a veces falla con el mensaje **NAs no son permitidos en asignaciones subscriptas** cuando formatea *algunas* combinaciones de números no redondos (por ejemplo, `c(182143, 959992.7)`) como etiquetas de texto. En los ejes funciona bien, porque las marcas son números redondos. Si te pasa en una etiqueta, usa `label_dollar(scale = 1e-3, suffix = "K", accuracy = 0.1)`, que siempre funciona.

3.10.1 Límites del eje y el error de truncar las barras

Pregunta de negocio: ¿qué tan distintas son las ventas de las tiendas? Observa qué pasa si “hacemos zoom” al eje para que las diferencias se vean más.

```

1 ingresos_tienda <- ventas_detalle %>%
2   group_by(tienda) %>%
3   summarise(ingresos = sum(total_transaccion), .groups = "drop")
4
5 g_truncado <- ggplot(ingresos_tienda,
6   aes(x = ingresos,
7     y = fct_reorder(tienda, ingresos))) +
8   geom_col(fill = color_principal, width = 0.7) +
9   coord_cartesian(xlim = c(78000, 116000)) + # el eje empieza en 78K
10  scale_x_continuous(labels = label_dollar(scale = 1e-3,
11    suffix = "K")) +
12  labs(title = "MAL: eje truncado", x = "Ingresos", y = NULL) +
13  theme_minimal() +
14  theme(panel.grid.minor = element_blank(),
15    panel.grid.major.y = element_blank())
16
17 g_honesto <- ggplot(ingresos_tienda,
18   aes(x = ingresos,
19     y = fct_reorder(tienda, ingresos))) +
20   geom_col(fill = color_principal, width = 0.7) +
21   scale_x_continuous(labels = label_dollar(scale = 1e-3,
22     suffix = "K"),
23     expand = expansion(mult = c(0, 0.1))) +
24   labs(title = "BIEN: eje desde cero", x = "Ingresos", y = NULL) +
25   theme_minimal() +
26   theme(panel.grid.minor = element_blank(),
27     panel.grid.major.y = element_blank(),
28     plot.margin = margin(5, 15, 5, 5))
29 g_truncado
30 g_honesto

```

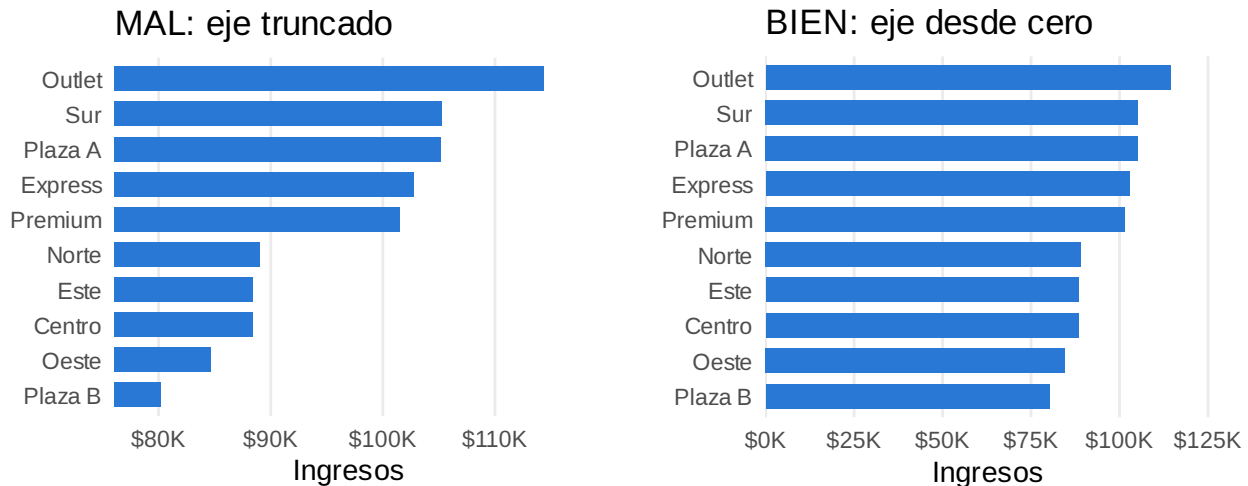


Figura 3.19. Los mismos datos con el eje truncado (izquierda) y con el eje desde cero (derecha).

Cómo leerla. En la versión truncada, la barra de Outlet parece *dieciséis veces* más larga que la de Plaza B. En realidad Outlet vendió \$114,377 y Plaza B \$80,236: 43 % más, no quince veces más. En una barra, la *longitud* es el dato; si cortas el eje, mientes con la longitud.

▲ Cuidado

Las barras siempre empiezan en cero. Si las diferencias son pequeñas y quieres mostrarlas, grafica la diferencia (por ejemplo, “% vs el promedio”) o usa puntos o líneas, donde no es obligatorio que el eje empiece en cero. Para acercar el eje sin perder datos usa `coord_cartesian(xlim = ...)`; `scale_x_continuous(limits = ...)` o `xlim()` *eliminan* los datos fuera del rango (y las barras que no caben desaparecen con un aviso `Removed 2 rows containing missing values`).

🔧 Hazlo tú

Toma `g_linea` (sección 3.5) y cambia el formato del eje *y* a miles: `labels = label_dollar(scale = 1e-3, suffix = "K")`. ¿Por qué en una gráfica de líneas no es obligatorio que el eje empiece en cero?

3.11 Temas y diseño corporativo

Hasta ahora cada gráfica repetía las mismas dos piezas de diseño: `theme_minimal()` y un `theme()` que quita la cuadrícula secundaria. En una empresa quieres que *todas* las gráficas del área se vean iguales: mismos colores, mismas fuentes, mismo lugar para la leyenda. La solución es crear tu propio **tema corporativo**: una función que guarda todas esas decisiones.

3.11.1 Los colores corporativos

En la sección 3.4 definiste cuatro objetos de color. Esta es la lógica detrás de ellos:

- `color_principal` (azul `#2a78d6`): el color de *una sola serie*. Todas las barras de un ranking, la línea de ingresos. El azul es serio, se imprime bien y lo distinguen prácticamente todas las personas.
- `color_resalte` (naranja `#eb6834`): para *destacar un elemento*. Contrasta fuertemente con el azul y con el gris.
- `color_gris` (`#b5b3ad`): para “el resto” cuando resaltas algo, y para referencias (metas, promedios).
- `paleta_libro`: ocho colores para *varias series*.

¿Por qué importa el orden de la paleta? Alrededor de 1 de cada 12 hombres tiene algún tipo de **daltonismo**, casi siempre dificultad para distinguir rojo de verde. Por eso los dos primeros colores de la paleta son azul y naranja: el par que mejor se distingue con cualquier tipo de visión. Como la mayoría de tus gráficas tendrán 2 o 3 series, usarás casi siempre los primeros colores, que son los más seguros. Por eso siempre se toman *en orden*: `paleta_libro[1:3]`, nunca colores sueltos al azar.

¿Por qué nunca más de 8 colores? Con más de 6–8 colores el lector ya no distingue los tonos y tiene que ir y venir a la leyenda: la gráfica se vuelve un juego de memoria. Si tienes más categorías, agrupa las pequeñas en “Otros” (con `fct_lump_n()`), usa facetas o resalta solo las importantes.

3.11.2 Construir `tema_empresa()` paso a paso

La función `theme()` controla cada elemento visual que no depende de los datos. Los elementos se modifican con tres funciones: `element_text()` (textos), `element_line()` (líneas de cuadrícula y ejes) y `element_blank()` (eliminar el elemento). Los más usados están en la tabla 3.3.

Tabla 3.3. Elementos de `theme()` más usados.

Elemento	Qué controla
<code>plot.title</code> , <code>plot.subtitle</code> , <code>plot.caption</code>	Título, subtítulo y nota al pie (fuente)
<code>plot.title.position = "plot"</code>	Alinear el título con el borde de la gráfica y no con el panel
<code>axis.title</code> , <code>axis.text</code>	Títulos y números de los ejes
<code>panel.grid.major</code> , <code>panel.grid.minor</code>	Líneas principales y secundarias de la cuadrícula
<code>panel.grid.major.x</code> , <code>panel.grid.major.y</code>	Solo las verticales o solo las horizontales
<code>legend.position</code>	"top", "right", "bottom", "none"
<code>plot.margin</code>	Márgenes alrededor de la gráfica

Construimos la función en tres decisiones: (1) partir de `theme_minimal()`, que ya quita el fondo

gris y los bordes; (2) dar jerarquía a los textos: título en negritas y casi negro, todo lo demás en gris oscuro para que no compita con los datos; (3) limpiar: sin cuadrícula secundaria, cuadrícula principal muy tenue, leyenda arriba y un poco de margen.

```

1 tema_empresa <- function(base_size = 11) {
2   # (1) punto de partida: tema minimalista con el tamaño de letra base
3   theme_minimal(base_size = base_size) +
4     theme(
5       # (2) jerarquía de textos
6       plot.title      = element_text(face = "bold", color = "#0b0b0b"),
7       plot.subtitle   = element_text(color = "#52514e"),
8       plot.caption    = element_text(color = "#52514e", size = rel(0.8),
9                                     hjust = 0),      # fuente a la izquierda
10      plot.title.position = "plot",    # título alineado a la izquierda
11      plot.caption.position = "plot",
12      axis.title       = element_text(color = "#52514e"),
13      axis.text        = element_text(color = "#52514e"),
14      # (3) limpieza
15      panel.grid.minor = element_blank(),
16      panel.grid.major = element_line(color = "#e6e5e1",
17                                      linewidth = 0.3),
18      legend.position = "top",
19      legend.title     = element_text(color = "#52514e"),
20      plot.margin      = margin(10, 15, 10, 10) # arriba, der., abajo, izq.
21    )
22 }

```

`rel(0.8)` significa “80 % del tamaño base”: si algún día usas `tema_empresa(base_size = 14)` para una presentación, todo crece en proporción. Ahora úsala exactamente como usarías `theme_minimal()`.

Pregunta de negocio: ¿qué tienda vendió más en 2023?

```

1 ingresos_tienda <- ingresos_tienda %>%
2   mutate(destacar = if_else(tienda == "Outlet", "Outlet", "Resto"))
3
4 g_tema <- ggplot(ingresos_tienda,
5                 aes(x = ingresos, y = fct_reorder(tienda, ingresos),
6                 fill = destacar)) +
7   geom_col(width = 0.7) +
8   scale_fill_manual(values = c(Outlet = color_resalte,
9                               Resto  = color_gris),
10                    guide = "none") +
11   scale_x_continuous(labels = dollar,
12                      expand = expansion(mult = c(0, 0.05))) +
13   labs(title = "Outlet, la tienda que más vendió en 2023",
14        subtitle = "Ingresos por tienda, en pesos",
15        x = NULL, y = NULL,
16        caption = "Fuente: transacciones.csv | Área de BI") +

```

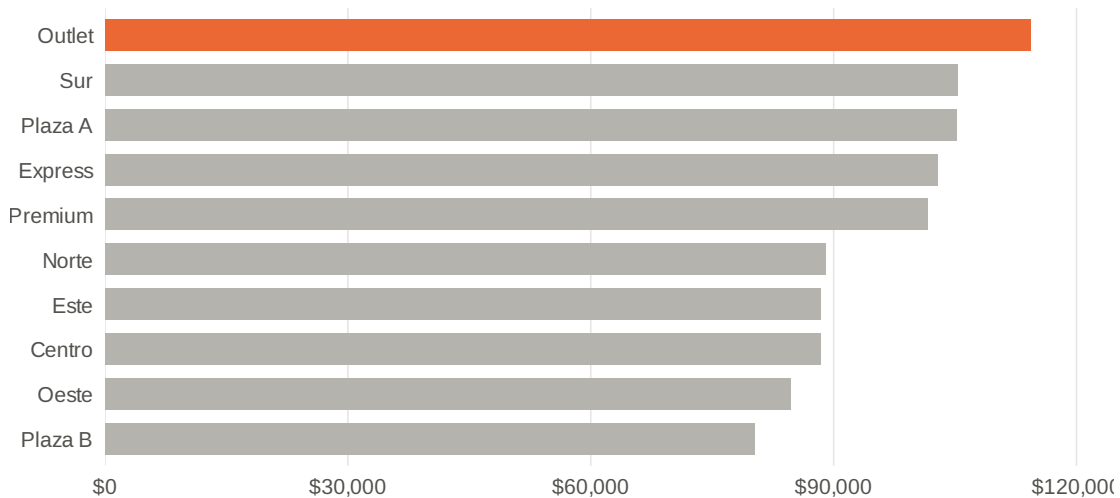
```

17 tema_empresa() +                               # nuestro tema
18 theme(panel.grid.major.y = element_blank())     # ajuste puntual
19 g_tema

```

Outlet, la tienda que más vendió en 2023

Ingresos por tienda, en pesos



Fuente: transacciones.csv | Área de BI

Figura 3.20. Una gráfica con `tema_empresa()`: título alineado a la izquierda, textos grises, cuadrícula tenue y la fuente de los datos al pie.

Observa que después de `tema_empresa()` todavía puedes sumar otro `theme()` para ajustes de una sola gráfica (aquí, quitar las líneas horizontales). Lo que va después *gana*.

En la empresa: Un archivo de estilo para todo el equipo

En un área de BI es común guardar `tema_empresa()` y los colores en un archivo `estilo_empresa.R` dentro del proyecto y empezar cada script con `source("estilo_empresa.R")`. Si mañana cambia el color de la marca, se cambia en un solo lugar y todos los reportes se actualizan. En el Módulo 8 lo usarás para tus reportes automáticos.

Hazlo tú

Crea una variante `tema_presentacion()` que llame a `tema_empresa(base_size = 16)` y ponga `legend.position = "bottom"`. Aplícala a `g_agrupadas` (¡cuidado!: esa gráfica ya tiene `theme_minimal()`; el último tema que sumes reemplaza al anterior).

3.12 Buenas y malas prácticas: antes y después

Las siguientes situaciones aparecen todos los días en reportes reales. Aprende a reconocerlas y a corregirlas.

3.12.1 Caso 1: barras multicolor sin significado

```

1 # ANTES: lo que sale "por defecto" si pones fill = tienda
2 g_antes <- ggplot(ingresos_tienda, aes(x = tienda, y = ingresos,
3                                     fill = tienda)) +
4   geom_col() +
5   labs(title = "Antes")
6
7 # DESPUÉS: un color, horizontal, ordenado, eje con formato
8 g_despues <- ggplot(ingresos_tienda,
9                     aes(x = ingresos, y = fct_reorder(tienda, ingresos))) +
10  geom_col(fill = color_principal, width = 0.7) +
11  scale_x_continuous(labels = label_dollar(scale = 1e-3, suffix = "K"),
12                    breaks = seq(0, 120000, by = 40000),
13                    expand = expansion(mult = c(0, 0.05))) +
14  labs(title = "Después", x = "Ingresos", y = NULL) +
15  tema_empresa() +
16  theme(panel.grid.major.y = element_blank())
17 g_antes
18 g_despues

```

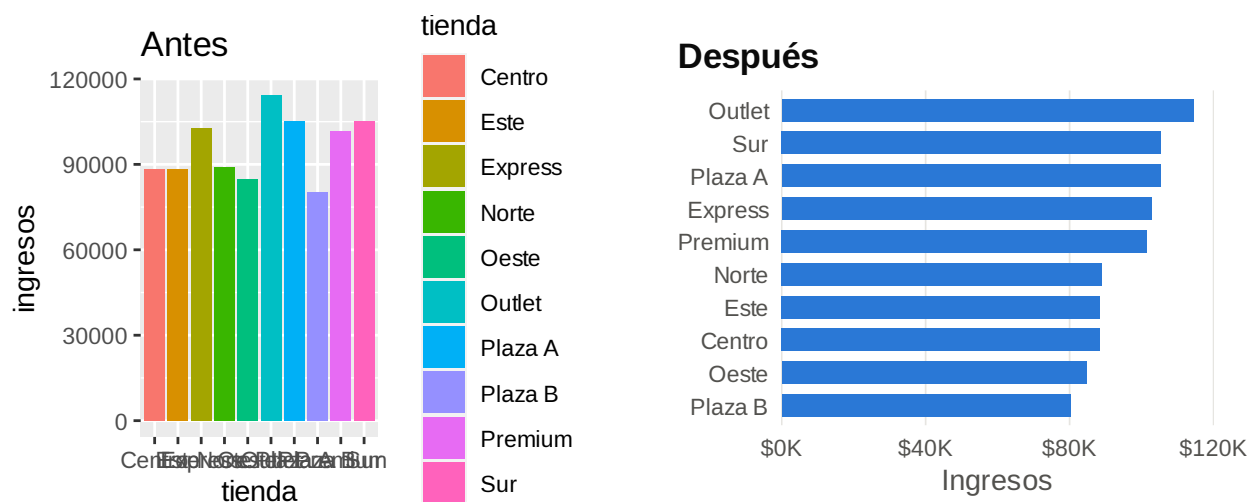


Figura 3.21. Antes: diez colores sin significado, nombres encimados, orden alfabético, eje en números crudos. Después: la misma información, legible en dos segundos.

La versión “antes” tiene cinco problemas: (1) diez colores que no significan nada y una leyenda que

repite el eje; (2) nombres encimados en el eje x ; (3) orden alfabético; (4) números del eje sin formato; (5) fondo gris que resta contraste. Todos se resuelven con lo que ya sabes.

3.12.2 Caso 2: el pastel de diez rebanadas

Imagina un pastel con las diez categorías de la tabla 3.1. En **ggplot2** se haría con una barra apilada en coordenadas polares:

```
1 # ANTES (no lo hagas): pastel de 10 porciones
2 g_pastel <- ggplot(ingresos_categoria,
3                   aes(x = "", y = ingresos, fill = categoria)) +
4   geom_col() +
5   coord_polar(theta = "y")
```

El ojo humano compara mal ángulos y áreas: ¿Juguetes (13.0%) o Alimentos (12.7%) es más grande? En un pastel es imposible decirlo, y además necesitarías diez colores. La alternativa son barras ordenadas con la participación.

```
1 g_sin_pastel <- ggplot(ingresos_categoria,
2                       aes(x = participacion,
3                           y = fct_reorder(categoria, participacion))) +
4   geom_col(fill = color_principal, width = 0.7) +
5   # aquí sí etiquetamos cada barra: son pocas y el % es el mensaje
6   geom_text(aes(label = percent(participacion, accuracy = 0.1)),
7             hjust = -0.15, size = 3.2, color = "#52514e") +
8   scale_x_continuous(labels = percent,
9                     expand = expansion(mult = c(0, 0.12))) +
10  labs(title = "Cinco categorías concentran el 69% de los ingresos",
11       subtitle = "Participación de cada categoría en los ingresos 2023",
12       x = NULL, y = NULL) +
13  tema_empresa() +
14  theme(panel.grid.major.y = element_blank())
15 g_sin_pastel
```

Cinco categorías concentran el 69% de los ingresos

Participación de cada categoría en los ingresos 2023

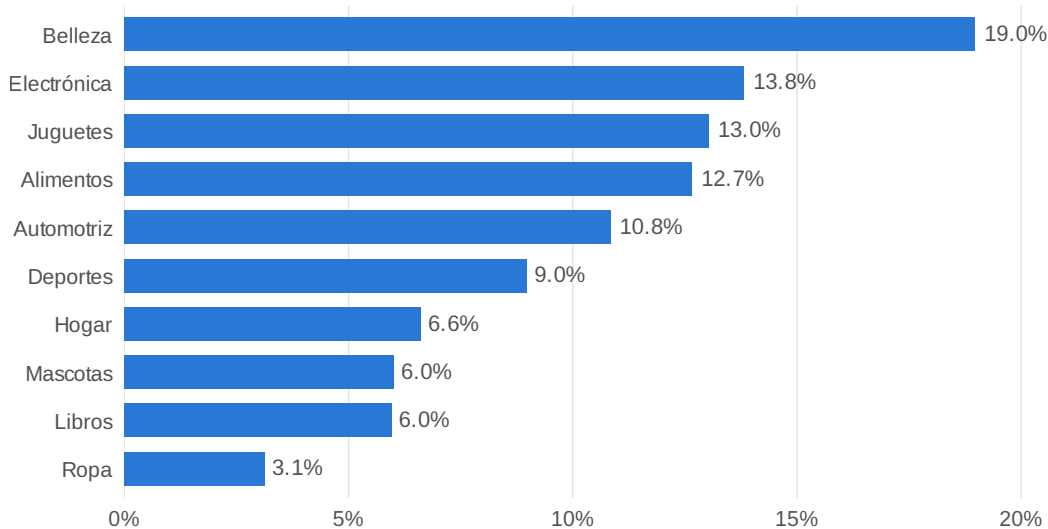


Figura 3.22. En lugar de un pastel de diez porciones: barras ordenadas con la participación.



Consejo

Un pastel solo es aceptable con 2 o 3 porciones muy distintas (“70 % de los ingresos viene de clientes recurrentes”) y nunca en 3D. Aun así, una sola barra al 100 % o una tarjeta KPI con el porcentaje suelen comunicar mejor.

3.12.3 Caso 3: el doble eje Y

Pregunta de negocio: ¿los ingresos y el número de tickets se mueven juntos? La tentación es poner ingresos (decenas de miles de pesos) en el eje izquierdo y tickets (decenas) en un eje derecho:

```
1 # ANTES (no lo hagas): segundo eje con sec_axis()
2 ggplot(ventas_mes, aes(x = mes)) +
3   geom_col(aes(y = ingresos)) +
4   geom_line(aes(y = tickets * 1000)) +           # escala inventada
5   scale_y_continuous(sec.axis = sec_axis(~ . / 1000, name = "Tickets"))
```

El problema: el factor 1000 lo eliges tú, y con él decides si las líneas “se cruzan”, “van juntas” o “se separan”. Cambia el factor y cambia la historia. El lector no puede saber qué es dato y qué es decisión de diseño. Dos alternativas honestas: dos gráficas una debajo de la otra con el mismo eje x , o un **índice base 100**: dividir cada serie entre su primer valor y multiplicar por 100. Así ambas miden “cuánto cambió respecto a enero” y comparten la misma escala.


```

1 indice <- ventas_mes %>%
2   mutate(Ingresos = ingresos / first(ingresos) * 100, # enero = 100
3         Tickets = tickets / first(tickets) * 100) %>%
4   select(mes, Ingresos, Tickets) %>%
5   pivot_longer(-mes, names_to = "medida", values_to = "indice")
6
7 g_indice <- ggplot(indice, aes(x = mes, y = indice, color = medida)) +
8   geom_hline(yintercept = 100, color = color_gris, linewidth = 0.5) +
9   geom_line(linewidth = 0.8) +
10  geom_point(size = 1.8) +
11  scale_color_manual(values = paleta_libro[1:2]) +
12  scale_x_date(date_labels = "%b", date_breaks = "1 month") +
13  labs(title = "En abril cayeron ambas; de mayo a julio subió el ticket",
14       subtitle = "Índice base 100 = enero de 2023",
15       x = NULL, y = "Índice (enero = 100)", color = NULL) +
16  tema_empresa()
17 g_indice

```

En abril cayeron ambas; de mayo a julio subió el ticket

Índice base 100 = enero de 2023

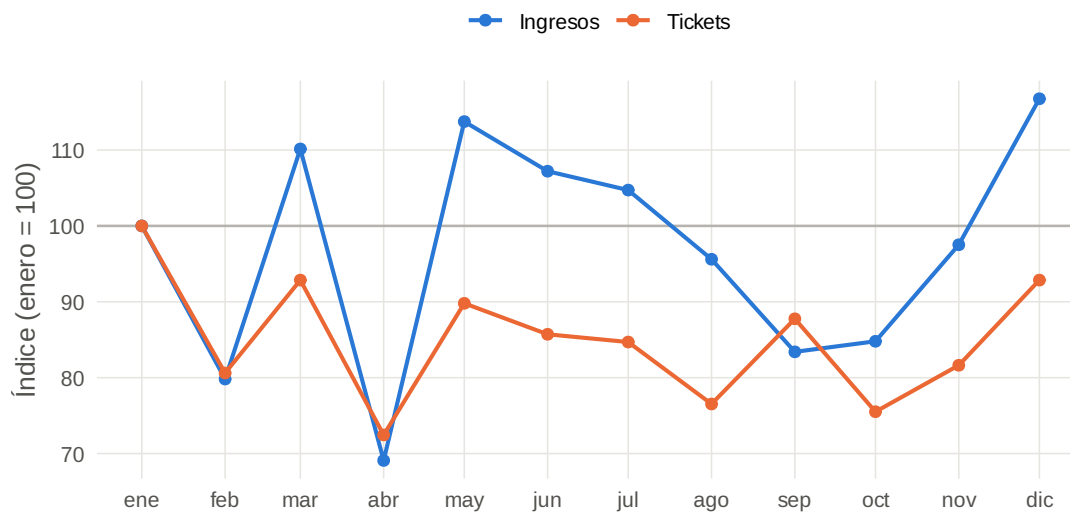


Figura 3.23. En lugar de doble eje: índice base 100 con ambas medidas en la misma escala.

Cómo leerla. Un valor de 110 significa “10 % arriba de enero”. En abril ambas medidas caen juntas (hubo menos tickets). De mayo a julio los ingresos quedan arriba de 100 mientras los tickets quedan debajo: se vendieron menos tickets pero de mayor importe. Esa conclusión es honesta porque no depende de ningún factor inventado.

3.12.4 Caso 4: el arcoíris

Imagina que en el mapa de calor de la figura 3.16 hubieras usado una escala arcoíris (`rainbow(7)`) dentro de `scale_fill_gradientn()`: las celdas de \$10,000 serían verdes y las de \$15,000 azules: el lector vería dos “grupos” que no existen y no sabría si el rojo es mucho o poco. La corrección es la que ya usaste: un gradiente secuencial de un solo tono (`scale_fill_gradient(low = "#cde2fb", high = "#0d366b")`).

3.12.5 Lista de verificación de una gráfica ejecutiva

Antes de enviar una gráfica a la dirección, revisa:

- ✓ El título dice la *conclusión*; el subtítulo dice qué se mide, el periodo y las unidades.
- ✓ Es el tipo de gráfica correcto para la pregunta (tabla 3.1).
- ✓ Las categorías están ordenadas por valor (salvo el tiempo, que va en orden cronológico).
- ✓ Una serie = un color. Para destacar: resalte + gris.
- ✓ Con 2 o más series hay leyenda (o etiquetas directas); nunca más de 8 colores.
- ✓ Las barras empiezan en cero. No hay doble eje Y.
- ✓ Los ejes tienen formato de negocio (\$, %, K, M) y unidades.
- ✓ Los escalas de color son secuenciales (un tono) o divergentes (dos tonos y gris), nunca arcoíris.
- ✓ Las etiquetas de valor están solo en lo importante.
- ✓ Sin 3D, sin sombras, sin fondo gris, cuadrícula tenue.
- ✓ La fuente de los datos aparece en el *caption*.
- ✓ Ningún texto está encimado o cortado al exportar.

Hazlo tú

Toma `g_conteo` (la de `geom_bar()` de la sección 3.4) y aplícale la lista completa: horizontal, ordenada, con título-conclusión, `tema_empresa()` y fuente.

3.13 Exportar gráficas

`ggsave()` guarda una gráfica en un archivo. El formato se deduce de la extensión (`.png`, `.pdf`, `.jpg`, `.svg`). Guardaremos siempre en una carpeta `resultados/` dentro del proyecto.

```
1 dir.create("resultados", showWarnings = FALSE) # crea la carpeta
2
3 # PNG para PowerPoint o correo: 300 puntos por pulgada (alta calidad)
4 ggsave("resultados/ingresos_categoria.png", plot = g_resalte,
5       width = 8, height = 4.5, units = "in", dpi = 300, bg = "white")
```

```

6
7 # PDF para imprimir o para LaTeX: vectorial, no se pixelee al ampliar
8 ggsave("resultados/ingresos_categoria.pdf", plot = g_resalte,
9        width = 8, height = 4.5)
10
11 file.exists(c("resultados/ingresos_categoria.png",
12              "resultados/ingresos_categoria.pdf"))

```

```
[1] TRUE TRUE
```

Los argumentos clave:

- **plot**: la gráfica a guardar. Si lo omites, guarda la *última* que se mostró, lo cual es fuente de errores: nómbrala siempre.
- **width**, **height** y **units**: el tamaño físico. Los textos conservan su tamaño en puntos, así que una gráfica guardada muy grande tendrá letras que se ven diminutas.
- **dpi**: resolución de los formatos de imagen (PNG, JPG). 300 para impresión y presentaciones; 150 basta para correo.
- **bg = "white"**: **theme_minimal()** tiene fondo *transparente*. En un PNG, eso se ve bien sobre fondo blanco pero se vuelve ilegible en una diapositiva oscura o en el modo oscuro del correo. Fija el fondo blanco al exportar.

Tabla 3.4. Tamaños recomendados para exportar con **ggsave()**.

Destino	Tamaño (pulgadas)	Formato
Diapositiva completa 16:9	10 × 5.6	PNG, dpi = 300
Media diapositiva	6 × 4.5	PNG, dpi = 300
Documento de Word (carta)	6.5 × 4	PNG 300 dpi o PDF
Cuerpo de un correo	7 × 4	PNG, dpi = 150
Reporte PDF / LaTeX	6.5 × 3.8	PDF (vectorial)

Consejo

Abre siempre el archivo exportado antes de enviarlo: lo que ves en el panel **Plots** de RStudio depende del tamaño de esa ventana, y el archivo puede verse distinto. Si los textos se enciman, aumenta **width** o reduce el texto (**base_size** del tema).

3.14 Gráficas interactivas con plotly

El paquete **plotly** convierte una gráfica de **ggplot2** en una gráfica **interactiva**: al pasar el ratón ves el valor exacto, puedes hacer zoom y ocultar series desde la leyenda. Es ideal para exploración y

para tableros web (Módulo 6).

```

1 # install.packages("plotly")      # solo la primera vez
2 library(plotly)
3
4 # Convierte cualquier ggplot en interactiva: se abre en el visor
5 # (panel Viewer) de RStudio o en el navegador
6 ggplotly(g_linea)
7
8 # Controla qué se muestra al pasar el ratón con la estética "text"
9 g_interactiva <- ggplot(ventas_mes,
10                        aes(x = mes, y = ingresos,
11                           text = paste("Mes:", format(mes, "%B"),
12                                         "<br>Ingresos:", dollar(ingresos),
13                                         "<br>Tickets:", tickets))) +
14   geom_point(color = color_principal, size = 2) +
15   tema_empresa()
16 ggplotly(g_interactiva, tooltip = "text")
17
18 # Guardar como página HTML para compartir por correo o intranet
19 htmlwidgets::saveWidget(ggplotly(g_linea),
20                          "resultados/ingresos_mensuales.html")

```

Este código no se ejecuta en el script del libro porque abre una ventana; pruébalo tú en RStudio. Dos advertencias: `ggplotly()` no traduce al 100 % todos los detalles (algunas anotaciones o subtítulos se pierden) y un archivo HTML no se puede pegar en PowerPoint como imagen. Para la presentación del consejo, exporta PNG; para el equipo que explora, HTML.

3.15 Caso práctico: tablero de ventas 2023 para la dirección

Ejemplo práctico: Las láminas del consejo

La directora general pidió “tres o cuatro láminas” con las ventas de 2023. Vas a entregar un **table-ro** (*dashboard*): una fila de tarjetas KPI y cinco gráficas coherentes, todas con `tema_empresa()` y los mismos colores, cada una con un título que dice su conclusión. Al final redactarás tres conclusiones para el consejo.

3.15.1 Paso 1: las tarjetas KPI

Lo primero que quiere ver la dirección son los números grandes del año. Aquí una gráfica no aporta nada: el mensaje es el número.

```

1 kpis <- ventas_detalle %>%
2   summarise(ingresos = sum(total_transaccion),

```

```

3      tickets = n(),
4      ticket_promedio = mean(total_transaccion),
5      clientes = n_distinct(cliente_id)) # clientes distintos
6 kpis

```

```

# A tibble: 1 x 4
  ingresos tickets ticket_promedio clientes
  <dbl>   <int>         <dbl>     <int>
1  959993.    1000           960.       198

```

Para ponerlas en la lámina podemos “dibujar” las tarjetas con **ggplot2**: un rectángulo (**geom_tile()**) y dos textos por tarjeta, sin ejes (**theme_void()**).

```

1 tarjetas <- tibble(
2   x      = 1:4,                                     # posición de cada tarjeta
3   valor  = c(dollar(kpis$ingresos, accuracy = 1),
4             comma(kpis$tickets),
5             dollar(kpis$ticket_promedio, accuracy = 1),
6             comma(kpis$clientes)),
7   titulo = c("Ingresos 2023", "Tickets", "Ticket promedio",
8             "Clientes distintos")
9 )
10
11 g_kpi <- ggplot(tarjetas, aes(x = x, y = 0)) +
12   geom_tile(width = 0.92, height = 1, fill = "#f3f7fd",
13           color = "#cde2fb") +                       # la tarjeta
14   geom_text(aes(label = valor), vjust = -0.1, size = 7,
15           fontface = "bold", color = "#0d366b") +    # el número grande
16   geom_text(aes(label = titulo), vjust = 2.2, size = 3.5,
17           color = "#52514e") +                       # la etiqueta
18   theme_void()                                       # sin ejes
19 g_kpi

```

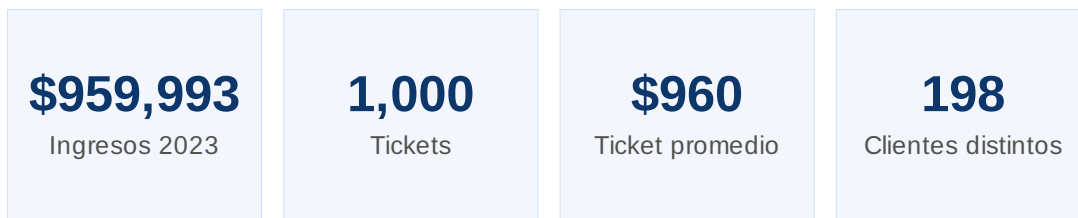


Figura 3.24. Fila de tarjetas KPI del tablero.

3.15.2 Paso 2: las cinco gráficas

Reutilizamos objetos que ya construiste (`ventas_mes`, `ingresos_categoria`, `g_tema`, `g_calor`) y les aplicamos el mismo tema. Los títulos van numerados para que en la junta se pueda decir “en la gráfica 3...”.

```

1 # 1. Evolución mensual contra la meta
2 t_mensual <- ggplot(ventas_mes, aes(x = mes, y = ingresos)) +
3   geom_hline(yintercept = meta_mensual, linetype = "dashed",
4             color = color_gris, linewidth = 0.6) +
5   geom_line(color = color_principal, linewidth = 0.8) +
6   geom_point(color = color_principal, size = 2) +
7   scale_x_date(date_labels = "%b", date_breaks = "1 month") +
8   scale_y_continuous(labels = label_dollar(scale = 1e-3, suffix = "K")) +
9   labs(title = "1. Abril fue el peor mes; diciembre, el mejor",
10        subtitle = "Ingresos mensuales (línea punteada = meta de $80K)",
11        x = NULL, y = NULL) +
12   tema_empresa()
13
14 # 2. Categorías, con Belleza resaltada
15 t_categorias <- ggplot(ingresos_categoria,
16                        aes(x = ingresos,
17                          y = fct_reorder(categoria, ingresos),
18                          fill = grupo)) +
19   geom_col(width = 0.7) +
20   scale_fill_manual(values = c(Destacada = color_resalte,
21                               Resto = color_gris),
22                    guide = "none") +
23   scale_x_continuous(labels = label_dollar(scale = 1e-3, suffix = "K"),
24                      expand = expansion(mult = c(0, 0.05))) +
25   labs(title = "2. Belleza lidera", x = NULL, y = NULL) +
26   tema_empresa() +
27   theme(panel.grid.major.y = element_blank())

```

Para la gráfica de tiendas partimos de `g_tema` y solo cambiamos la escala y los textos. Al sumar una segunda escala para el eje *x*, **ggplot2** avisa que reemplaza la anterior; es justo lo que queremos.

```

1 # 3. Tiendas: reutiliza g_tema con otra escala y otro título
2 t_tiendas <- g_tema +
3   scale_x_continuous(labels = label_dollar(scale = 1e-3, suffix = "K"),
4                     expand = expansion(mult = c(0, 0.05))) +
5   labs(title = "3. Outlet vende más", subtitle = NULL,
6        caption = NULL)

```

Scale for x is already present.
Adding another scale for x, which will replace the existing scale.

```

1 # 4. Métodos de pago: participación en los ingresos
2 ingresos_pago <- ventas_detalle %>%
3   group_by(metodo_pago) %>%
4   summarise(ingresos = sum(total_transaccion), .groups = "drop") %>%
5   mutate(participacion = ingresos / sum(ingresos))
6 ingresos_pago
7
8 t_pagos <- ggplot(ingresos_pago,
9   aes(x = participacion,
10     y = fct_reorder(metodo_pago, participacion))) +
11   geom_col(fill = color_principal, width = 0.6) +
12   geom_text(aes(label = percent(participacion, accuracy = 1)),
13     hjust = -0.2, size = 3.2, color = "#52514e") +
14   scale_x_continuous(labels = percent,
15     expand = expansion(mult = c(0, 0.2))) +
16   labs(title = "4. Seis de cada diez pesos\nllegan con tarjeta",
17     x = NULL, y = NULL) +
18   tema_empresa() +
19   theme(panel.grid.major.y = element_blank())
20
21 # 5. Mapa de calor: reutiliza g_calor, sin leyenda para ahorrar espacio
22 t_calor <- g_calor +
23   labs(title = "5. Sin un día fijo fuerte", subtitle = NULL) +
24   tema_empresa() +
25   theme(panel.grid = element_blank(), legend.position = "none",
26     axis.text.x = element_text(size = 7))

```

```

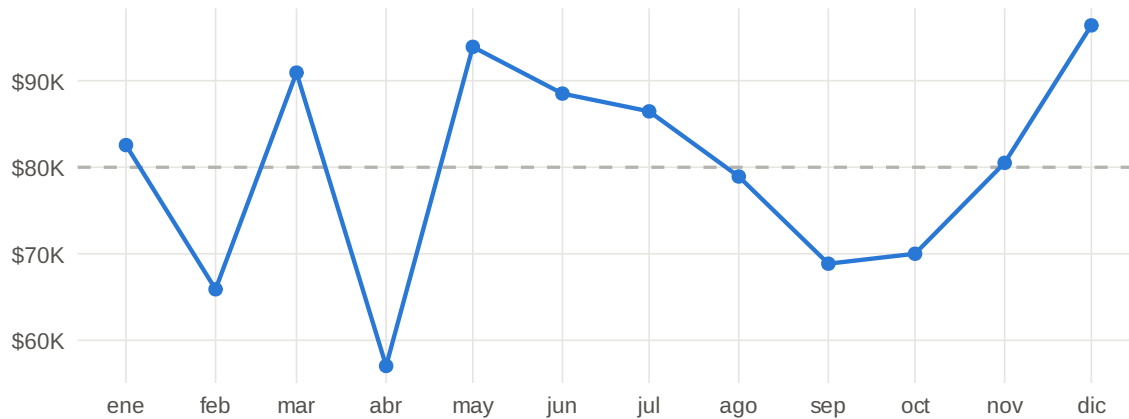
# A tibble: 4 x 3
  metodo_pago      ingresos participacion
  <chr>          <dbl>         <dbl>
1 Efectivo       303735.         0.316
2 Tarjeta Crédito 351171.         0.366
3 Tarjeta Débito  220855.         0.230
4 Transferencia   84232.         0.0877

```

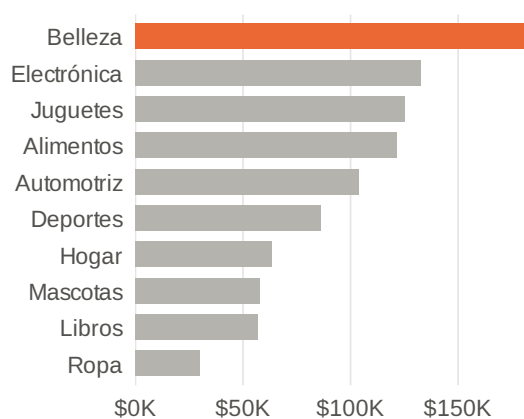
El "\n" dentro del título parte el texto en dos renglones para que quepa en una gráfica angosta.

1. Abril fue el peor mes; diciembre, el mejor

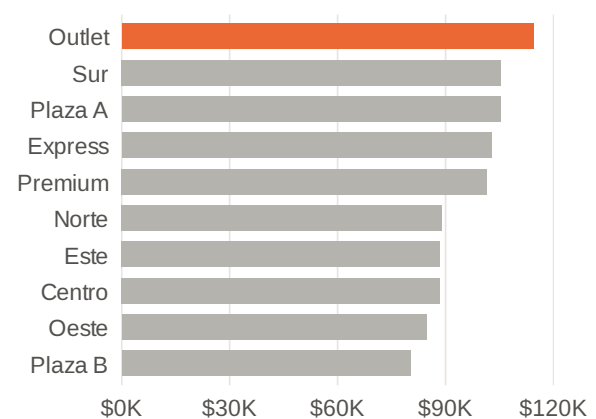
Ingresos mensuales (línea punteada = meta de \$80K)



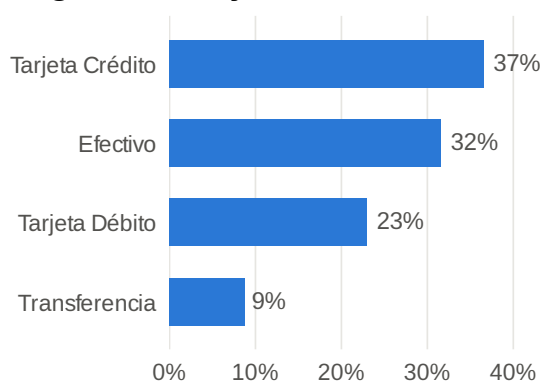
2. Belleza lidera



3. Outlet vende más



4. Seis de cada diez pesos llegan con tarjeta



5. Sin un día fijo fuerte

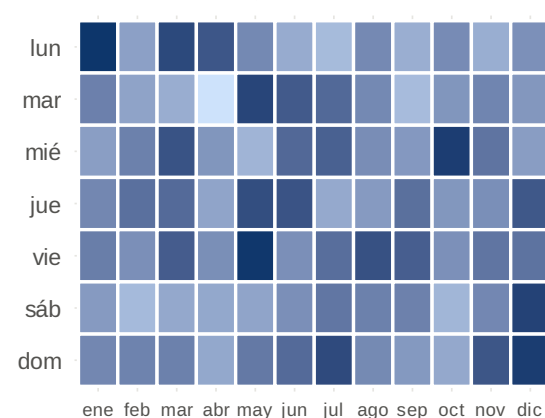


Figura 3.25. Tablero de ventas 2023: cinco gráficas con el mismo tema y colores, cada una con su conclusión en el título.

3.15.3 Paso 3: exportar todo de una vez

Guardamos las seis piezas en una subcarpeta, listas para insertarse en las diapositivas. Una lista con nombres y un ciclo `for` evitan repetir seis veces `ggsave()`.

```
1 dir.create("resultados/tablero", showWarnings = FALSE, recursive = TRUE)
2
3 piezas <- list(kpis = g_kpi, mensual = t_mensual,
4               categorias = t_categorias, tiendas = t_tiendas,
5               pagos = t_pagos, calor = t_calor)
6
7 for (nombre in names(piezas)) {
8   ggsave(file.path("resultados/tablero", paste0(nombre, ".png")),
9         plot = piezas[[nombre]], width = 6, height = 4, dpi = 300,
10        bg = "white")
11 }
12 list.files("resultados/tablero")
```

```
[1] "calor.png"      "categorias.png" "kpis.png"
[4] "mensual.png"    "pagos.png"      "tiendas.png"
```

3.15.4 Paso 4: las conclusiones para el consejo

Un tablero sin conclusiones obliga a cada director a sacar las suyas (y cada uno sacará una distinta). Estas son las que los datos *realmente* muestran:

1. **El año cerró en \$959,993 con 1,000 tickets y un ticket promedio de \$960.** Siete de doce meses superaron la meta de \$80,000; abril (\$57,038) fue el punto más bajo y diciembre (\$96,394) el más alto. Conviene investigar qué pasó en abril y en septiembre–octubre.
2. **Belleza es la categoría líder (19 % de los ingresos), pero cayó 41 % en el segundo semestre,** mientras Electrónica creció 28 %. Si la tendencia sigue, Belleza podría perder el primer lugar en 2024: es la alerta principal para el área comercial.
3. **Las diferencias entre tiendas son moderadas** (de \$80,236 en Plaza B a \$114,377 en Outlet, 43 % más). Outlet, con solo 350 m², es la que más vende: vale la pena entender su modelo. La mezcla de métodos de pago es estable (60 % con tarjeta) y no hay un día de la semana consistentemente más fuerte, así que no se justifican promociones por día fijo.

▲ Cuidado

Observa lo que *no* dijimos: “Outlet es la mejor tienda porque tiene la mejor gerencia” o “Belleza cayó por la competencia”. Las gráficas muestran *qué* pasó, no *por qué*. Las causas requieren más análisis (Módulo 4). Además, recuerda que estos datos son simulados: con 1,000 tickets repartidos en 10 tiendas y 12 meses, muchas diferencias pueden deberse al azar.

 Hazlo tú

Agrega una sexta gráfica al tablero: “6. Ticket promedio por tienda” (barras horizontales, ordenadas, un color, eje en pesos). ¿La tienda con más ingresos es también la de ticket promedio más alto?

3.16 Errores comunes

Estos son los errores que más verás al empezar con **ggplot2**. Los mensajes son los que imprime R (versión 3.4 de **ggplot2**; RStudio agrega al final una sugerencia de `rlang::last_trace()`).

1. El signo + al inicio de la línea

```
1 ggplot(ingresos_categoria, aes(x = categoria, y = ingresos))
2   + geom_col()
```

```
Error in `+.gg`:
! Cannot use `+` with a single argument
i Did you accidentally put `+` on a new line?
```

R ejecuta la primera línea completa (y dibuja un lienzo vacío). Luego encuentra `+ geom_col()` sola y no sabe a qué sumarla. **Solución:** el `+` va al *final* de la línea anterior.

2. Usar el pipe en lugar de +

```
1 ggplot(ingresos_categoria, aes(x = categoria, y = ingresos)) %>%
2   geom_col()
```

```
Error in `geom_col()`:
! `mapping` must be created by `aes()`
i Did you use `%>%` or `|>` instead of `+`?
```

El pipe pasa la gráfica como primer argumento de `geom_col()`, que espera un `aes()`. **Solución:** `%>%` para transformar datos *antes* de `ggplot()`; `+` para las capas *después*.

3. Un color constante dentro de aes()

```
1 g_rara <- ggplot(ventas_mes, aes(x = mes, y = ingresos)) +
2   geom_line(aes(color = "azul"))      # MAL: "azul" se trata como dato
3 layer_data(g_rara)$colour[1]         # ¿qué color usó realmente?
```

```
[1] "#F8766D"
```

No hay mensaje de error, pero la línea sale *rosa salmón* (**#F8766D**, el primer color por defecto de **ggplot2**) y aparece una leyenda rara con la palabra “azul”. Dentro de **aes()** el texto “azul” es un *valor de datos* (una categoría llamada “azul”), no un color. **Solución:** **geom_line(color = color_principal)**, fuera de **aes()**.

4. geom_bar() con una variable y

```
1 ggplot(ingresos_categoria, aes(x = categoria, y = ingresos)) +
2   geom_bar()
```

```
Error in `geom_bar()` :
! Problem while computing stat.
i Error occurred in the 1st layer.
Caused by error in `setup_params()` :
! `stat_count()` must only have an x or y aesthetic.
```

geom_bar() quiere *contar* filas y no acepta una altura ya calculada. **Solución:** usa **geom_col()** cuando ya tienes el valor (o **geom_bar(stat = "identity")**, que es lo mismo).

5. Discrete value supplied to continuous scale

```
1 ggplot(ingresos_categoria, aes(x = categoria, y = ingresos)) +
2   geom_col() +
3   scale_x_continuous(labels = comma)    # pero x es texto
```

```
Error in `train_continuous()` :
! Discrete value supplied to a continuous scale
```

Usaste una escala para números (**continuous**) sobre una columna de texto o factor (**discrete**). Suele pasar al voltear las barras a horizontal y olvidar cambiar **scale_y...** por **scale_x...** **Solución:** revisa qué columna está en cada eje; para texto usa **scale_x_discrete()**. El error contrario (“Continuous value supplied to a discrete scale”) aparece, por ejemplo, al usar **scale_fill_manual()**

con una columna numérica.

6. Meses como texto: orden alfabético

```
1 meses_texto <- format(ventas_mes$mes, "%b") # "ene", "feb"... (texto)
2 sort(meses_texto)                          # así los ordena ggplot2
```

```
[1] "abr" "ago" "dic" "ene" "feb" "jul" "jun" "mar" "may" "nov" "oct"
[12] "sep"
```

Si en tu tabla el mes es texto ("ene", "feb"...), **ggplot2** lo ordena alfabéticamente: abril, agosto, diciembre, enero... y tu línea de tiempo queda revuelta. **Soluciones:** (a) conserva la fecha como **Date** (con **floor_date()**) y usa **scale_x_date(date_labels = "%b")**; (b) usa **month(fecha, label = TRUE)**, que devuelve un factor ya ordenado; o (c) conviértelo a factor con los niveles en orden: **factor(mes, levels = c("ene", "feb", ...))**.

7. Una línea que no aparece

Si el eje *x* es texto o factor, **geom_line()** no sabe qué puntos unir y da el aviso:

```
`geom_line()`: Each group consists of only one observation.
i Do you need to adjust the group aesthetic?
```

Solución: agrega **aes(group = 1)** para indicar que todos los puntos forman una sola línea (o una columna, como **group = tienda**, para una línea por grupo).

Resumen del módulo

- Grafica antes de concluir: el cuarteto de Anscombe muestra que las estadísticas de resumen pueden esconder patrones muy distintos.
- Una gráfica de **ggplot2** se construye sumando capas con **+**: datos, **aes()**, geometrías, escalas, facetas, coordenadas y tema.
- Elige la gráfica por la pregunta: barras para comparar, líneas para el tiempo, histograma/-boxplot para distribuciones, dispersión para relaciones, barras 100% para composición y tarjetas KPI para un solo número.
- Reglas de diseño: una serie un color; resalte + gris; leyenda con 2+ series; barras desde cero; sin doble eje; secuencial de un tono; máximo 8 colores y en orden; títulos que dicen la conclusión.
- **tema_empresa()** y los colores corporativos hacen que todas tus gráficas se vean como un sistema coherente.
- **ggsave()** exporta con el tamaño, la resolución y el fondo correctos; **ggplotly()** las vuelve

interactivas.

Función	Para qué sirve
<code>ggplot(datos, aes(x, y))</code>	Crear el lienzo y mapear columnas
<code>geom_col()</code> / <code>geom_bar()</code>	Barras con valor calculado / barras que cuentan filas
<code>geom_line()</code> , <code>geom_point()</code>	Líneas y puntos
<code>geom_histogram()</code> , <code>geom_density()</code>	Distribución
<code>geom_boxplot()</code> , <code>geom_violin()</code>	Comparar distribuciones
<code>geom_smooth(method = "lm")</code>	Recta de tendencia
<code>geom_tile()</code>	Mapa de calor
<code>geom_hline()</code> , <code>annotate()</code>	Referencias y anotaciones
<code>geom_text()</code>	Etiquetas de valor
<code>facet_wrap()</code> , <code>facet_grid()</code>	Facetas
<code>fct_reorder()</code> , <code>fct_rev()</code>	Ordenar categorías
<code>position_dodge()</code> , <code>position_fill()</code>	Barras agrupadas / al 100 %
<code>scale_fill_manual()</code> , <code>scale_color_manual()</code>	Colores por nombre
<code>scale_fill_gradient()</code>	Escala secuencial
<code>scale_x_date()</code> , <code>scale_x_log10()</code>	Ejes de fecha y logarítmicos
<code>dollar</code> , <code>comma</code> , <code>percent</code> , <code>cut_short_scale()</code>	Formato de negocio (scales)
<code>coord_cartesian()</code>	Zoom sin eliminar datos
<code>labs()</code> , <code>theme()</code> , <code>theme_minimal()</code>	Textos y apariencia
<code>ggsave()</code> , <code>ggplotly()</code>	Exportar / interactivo

3.17 Ejercicios

Para cada ejercicio escribe un script que cargue sus propios paquetes y datos (los de la carpeta `datasets/`) y define los colores corporativos que uses. Las soluciones completas están en el Apéndice A.

Ejercicio 3.1: Tickets por tienda

Nivel: **Básico**

Con `transacciones.csv` y `tiendas.csv`, haz una gráfica de barras horizontales con el *número de tickets* por tienda, ordenada de mayor a menor, de un solo color, con título y fuente. ¿Qué tienda tuvo más tickets? ¿Coincide con la que más ingresos tuvo?

🔗 Solución en la página 631

Ejercicio 3.2: Edad de los clientes

Nivel: **Básico**

Con `clientes.csv`, haz un histograma de la edad con barras de 5 años (`binwidth = 5`, `boundary = 0`) y una línea vertical punteada en la mediana (`geom_vline()`). ¿La distribución es simétrica, asimétrica o más o menos pareja?

 Solución en la página 632

Ejercicio 3.3: Encuentra los errores

Nivel: **Básico**

El siguiente código tiene cuatro errores. Encuéntralos, explica cada uno y escribe la versión corregida (que muestre el ingreso por método de pago en azul corporativo, con el eje en pesos).

```
1 ingresos_pago <- transacciones %>%
2   group_by(metodo_pago) %>%
3   summarise(ingresos = sum(total_transaccion))
4 ggplot(ingresos_pago, aes(x = metodo_pago, y = ingresos,
5                           fill = "#2a78d6"))
6 + geom_bar() %>%
7   scale_y_continuous(labels = percent)
```

 Solución en la página 632

Ejercicio 3.4: Tickets por mes contra el promedio

Nivel: **Intermedio**

Grafica el número de tickets por mes de 2023 como línea con puntos. Agrega una línea horizontal gris en el promedio mensual y anota con `annotate()` el mes con *menos* tickets. Usa `scale_x_date()` con los nombres de los meses.

 Solución en la página 633

Ejercicio 3.5: El mejor día de la semana

Nivel: **Intermedio**

Grafica los ingresos por día de la semana (lunes a domingo, en ese orden) como columnas. Resalta en naranja el día con más ingresos, deja el resto en gris y pon la etiqueta con el monto solo en esa barra. El título debe decir la conclusión.

 Solución en la página 634

Ejercicio 3.6: Salarios por departamento

Nivel: **Intermedio**

Con `empleados.csv`, haz un boxplot horizontal del salario mensual por departamento, ordenado por la mediana, con el eje en pesos. ¿Qué departamento tiene la mediana más alta? ¿Cuál tiene

más dispersión?

 Solución en la página 636

Ejercicio 3.7: Precio contra costo: hallazgo de calidad de datos

Nivel: **Intermedio**

Con `productos.csv`, haz una dispersión de `costo` (eje x) contra `precio_catalogo` (eje y). Dibuja la recta “precio = costo” con `geom_abline()` y colorea los puntos en dos grupos: los productos con costo mayor al precio y el resto. ¿Cuántos productos venden con pérdida? ¿Qué le dirías al área de compras?

 Solución en la página 637

Ejercicio 3.8: Las tres tiendas líderes

Nivel: **Avanzado**

Grafica los ingresos *acumulados* por mes de las 10 tiendas. Las tres tiendas con más ingresos al cierre del año deben ir en color (con leyenda, colores de `paleta_libro` en orden) y las otras siete en gris, sin leyenda. Las tres líderes deben calcularse con código, no escribirse a mano.

 Solución en la página 639

Ejercicio 3.9: Tu propia función de gráficas

Nivel: **Avanzado**

Escribe una función `grafica_ranking(datos, categoria, valor, titulo)` que devuelva una gráfica de barras horizontales ordenadas, de un color, con eje en pesos y `tema_empresa()`. Pista: dentro de `aes()` usa `{{ categoria }}` para recibir nombres de columna. Úsala para graficar ingresos por método de pago y por tienda, y guarda ambas gráficas como PNG en `resultados/`.

 Solución en la página 640

Ejercicio 3.10: Reto: mini tablero de clientes

Nivel: **Reto**

Con `clientes.csv`, arma un mini tablero para el área de marketing: (a) tres KPI impresos en consola (clientes totales, % premium y edad mediana); (b) clientes por ciudad, resaltando la ciudad con más clientes; (c) % de clientes premium por segmento de edad. Usa el mismo tema en ambas gráficas, expórtalas y escribe tres conclusiones basadas en lo que muestran.

 Solución en la página 642

PARTE II

Nivel intermedio

Análisis exploratorio y estadística para negocios

Al terminar este módulo podrás...

- Aplicar un proceso ordenado de **análisis exploratorio de datos** (EDA) con una lista de verificación que puedes reutilizar en cualquier proyecto.
- Calcular e interpretar medidas de tendencia central, dispersión, percentiles y forma de una distribución, y saber cuándo el promedio engaña.
- Detectar datos atípicos con reglas objetivas y decidir qué hacer con ellos sin borrarlos a ciegas.
- Construir KPIs de desempeño (crecimiento, acumulados, promedios móviles, índices y avance contra meta) y un análisis de Pareto/ABC.
- Medir relaciones con correlaciones y distinguir correlación de causalidad.
- Usar las distribuciones normal, binomial y de Poisson para responder preguntas de probabilidad del día a día.
- Estimar con intervalos de confianza, aplicar pruebas de hipótesis (t, proporciones, ANOVA, chi-cuadrada) y ajustar regresiones lineales, explicando los resultados en lenguaje de negocio.

Imagina que trabajas como analista en una cadena de tiendas con diez sucursales en México. Es lunes y la directora comercial te escribe: *“Antes de aprobar el presupuesto de fin de año necesito entender qué está pasando con las ventas: ¿cuánto gasta un cliente típico?, ¿hay tiendas que funcionen mejor que otras?, ¿nuestros clientes premium realmente gastan más?, ¿qué productos sostienen el negocio? Y si invertimos más en publicidad, ¿cuánto deberíamos esperar vender?”*.

Con lo que aprendiste en los Módulos 2 y 3 ya puedes filtrar, agrupar, unir tablas y graficar. Pero la directora no quiere tablas sueltas: quiere *respuestas*. Para darlas necesitas dos cosas. La primera es un **método** para recorrer los datos sin perderte: eso es el análisis exploratorio. La segunda es la **estadística**, que te permite distinguir un patrón real de una simple casualidad y decir con honestidad cuánta confianza tienes en cada número.

En este módulo aprenderás ambas cosas con los datos del curso. No necesitas saber matemáticas avanzadas: cada fórmula aparece explicada en palabras, se calcula en R paso a paso y termina con la pregunta que más importa en una empresa: *¿y esto qué significa para el negocio?*

4.1 ¿Qué es el análisis exploratorio (EDA)?

Concepto clave: Análisis exploratorio de datos (EDA)

El **análisis exploratorio de datos** (EDA, por sus siglas en inglés, *Exploratory Data Analysis*) es el proceso de conocer un conjunto de datos *antes* de sacar conclusiones: revisar su estructura, su calidad, cómo se reparten los valores, cómo se relacionan las variables y qué preguntas vale la pena investigar. El término lo popularizó el estadístico John Tukey en 1977 con una idea sencilla: primero mira los datos, después decide qué prueba aplicar.

Piensa en el EDA como la consulta de un médico: antes de recetar, toma la presión, pregunta síntomas y pide análisis. Si te saltas ese paso, puedes recetar el medicamento equivocado. En BI pasa lo mismo: un dashboard construido sin EDA puede mostrar con mucha elegancia un número que está mal.

La figura 4.1 resume el proceso que seguiremos en todo el módulo. No es una receta rígida: a menudo un hallazgo en el paso 5 te obliga a volver al paso 2.

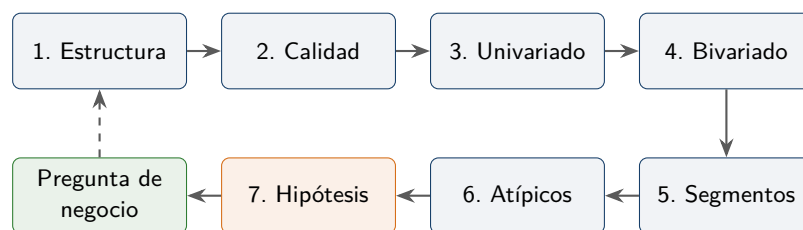


Figura 4.1. Proceso de análisis exploratorio. La flecha punteada indica que cada hipótesis genera nuevas preguntas y el ciclo se repite.

La tabla 4.1 convierte el proceso en una lista de verificación. Te recomiendo copiarla en tus proyectos: evita que se te olvide revisar algo importante.

Tabla 4.1. Lista de verificación (*checklist*) del análisis exploratorio.

Paso	Pregunta que respondes	Funciones de R
1. Estructura	¿Cuántas filas y columnas hay? ¿Qué tipo tiene cada columna? ¿Qué representa cada fila?	<code>dim()</code> , <code>glimpse()</code>
2. Calidad	¿Hay valores faltantes, duplicados, negativos imposibles o categorías mal escritas?	<code>skim()</code> , <code>summary()</code> , <code>count()</code>
3. Univariado	¿Cómo se reparte cada variable? ¿Cuál es su valor típico y cuánto varía?	<code>mean()</code> , <code>quantile()</code> , <code>geom_histogram()</code>
4. Bivariado	¿Qué variables se mueven juntas?	<code>cor()</code> , <code>geom_point()</code>
5. Segmentos	¿Cambian los resultados por tienda, categoría o tipo de cliente?	<code>group_by()</code> + <code>summarise()</code>
6. Atípicos	¿Hay valores extremos? ¿Son errores o casos valiosos?	<code>IQR()</code> , <code>geom_boxplot()</code>
7. Hipótesis	¿Qué diferencias parecen reales y cuáles pueden ser azar?	<code>t.test()</code> , <code>lm()</code>

4.1.1 Preparar el entorno

En este módulo usaremos dos paquetes nuevos: **skimr**, que resume un data frame completo con una sola función, y **corrplot**, que dibuja matrices de correlación. Instálalos una sola vez:

```
1 install.packages(c("skimr", "corrplot"))
```

Ahora carga los paquetes y los datos. Recuerda que el directorio de trabajo debe ser la carpeta del curso (la que contiene `datasets/`).

```
1 # Paquetes del módulo
2 library(tidyverse) # dplyr, ggplot2, readr, tidyr... (Módulos 2 y 3)
3 library(skimr)     # resúmenes rápidos de un data frame
4 library(corrplot)  # matrices de correlación
5 library(scales)    # formatos de negocio: dollar(), percent(), comma()
6
7 # Datos del curso
8 transacciones <- read_csv("datasets/transacciones.csv",
9                           show_col_types = FALSE)
10 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
11 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
12 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
```

También definimos los colores corporativos y el tema gráfico que construiste en el Módulo 3, para que todas las gráficas del módulo tengan el mismo estilo:

```
1 # Colores corporativos
2 color_principal <- "#2a78d6" # una sola serie
3 color_resalte <- "#eb6834"  # para destacar un elemento
4 color_gris <- "#b5b3ad"     # el resto, cuando se resalta uno
5 paleta_libro <- c("#2a78d6", "#eb6834", "#1baf7a", "#eda100",
6                  "#e87ba4", "#008300", "#4a3aa7", "#e34948")
7
8 # Tema corporativo (misma forma que en el Módulo 3)
9 tema_libro <- function(base_size = 11) {
10   theme_minimal(base_size = base_size) +
11     theme(
12       plot.title       = element_text(face = "bold", color = "#0b0b0b"),
13       plot.subtitle    = element_text(color = "#52514e"),
14       plot.caption     = element_text(color = "#52514e", size = rel(0.8)),
15       axis.title       = element_text(color = "#52514e"),
16       axis.text        = element_text(color = "#52514e"),
17       panel.grid.minor = element_blank(),
18       panel.grid.major = element_line(color = "#e6e5e1", linewidth = 0.3),
19       legend.position  = "top",
20       legend.title     = element_text(color = "#52514e")
21     )
22 }
```

```
22 }
```

4.1.2 Pasos 1 y 2: estructura y calidad

Empieza siempre por lo básico: ¿qué tamaño tiene la tabla y qué contiene cada columna?

```
1 dim(transacciones)      # filas y columnas
2 glimpse(transacciones)  # tipo y primeros valores de cada columna
```

```
[1] 1000  14
Rows: 1,000
Columns: 14
$ transaccion_id    <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ~
$ fecha             <date> 2023-01-13, 2023-01-09, 2023-02-20, 2023-~
$ hora              <time> 13:15:00, 10:45:00, 10:30:00, 10:45:00, 1~
$ cliente_id        <dbl> 155, 155, 75, 51, 173, 135, 90, 168, 13, 9~
$ producto_id       <dbl> 3, 47, 6, 17, 28, 46, 30, 48, 21, 12, 49, ~
$ cantidad          <dbl> 4, 4, 3, 4, 1, 2, 4, 2, 2, 5, 2, 3, 4, 2, ~
$ precio_venta      <dbl> 303.97, 58.07, 275.46, 105.28, 205.47, 534~
$ metodo_pago       <chr> "Tarjeta Crédito", "Tarjeta Débito", "Tarj~
$ tienda_id         <dbl> 10, 2, 7, 6, 7, 7, 8, 8, 5, 2, 7, 6, 8, 2, ~
$ vendedor_id       <dbl> 4, 22, 14, 5, 14, 22, 23, 5, 3, 14, 5, 9, ~
$ total_transaccion <dbl> 1215.88, 232.28, 826.38, 421.12, 205.47, 1~
$ mes               <chr> "2023-01", "2023-01", "2023-02", "2023-05"~
$ trimestre         <chr> "Q1", "Q1", "Q1", "Q2", "Q1", "Q1", "Q2", ~
$ dia_semana        <chr> "viernes", "lunes", "lunes", "viernes", "m~
```

Cada fila es una *transacción* (una línea de venta): un cliente compró cierta **cantidad** de un producto a un **precio_venta** en una tienda. La columna **total_transaccion** es cantidad × precio. Fíjate en que **read_csv()** reconoció **fecha** como fecha (**<date>**) y **hora** como hora (**<time>**); los identificadores (**cliente_id**, **tienda_id**...) son números, pero en realidad son *etiquetas*: promediarlos no tiene sentido.

La función **summary()** da un primer resumen numérico. Aplícala solo a las columnas de medida para que la salida sea legible:

```
1 transacciones %>%
2   select(cantidad, precio_venta, total_transaccion) %>%
3   summary()
```

	cantidad	precio_venta	total_transaccion
Min.	:1.000	Min. : 20.14	Min. : 22.65
1st Qu.:	:2.000	1st Qu.:164.11	1st Qu.: 341.11
Median	:3.000	Median :324.92	Median : 713.49

Mean	:2.969	Mean	:318.31	Mean	: 959.99
3rd Qu.	:4.000	3rd Qu.	:468.00	3rd Qu.	:1478.62
Max.	:5.000	Max.	:598.81	Max.	:2992.35

Para cada columna obtienes el mínimo (**Min.**), el primer cuartil (**1st Qu.**), la mediana (**Median**), la media (**Mean**), el tercer cuartil (**3rd Qu.**) y el máximo (**Max.**). Ya aparece una pista importante: en **total_transaccion** la media (959.99) es bastante mayor que la mediana (713.49). En la sección 4.3 verás que eso indica una distribución con “cola” hacia los valores altos.

El paquete **skimr** va más allá: con **skim()** obtienes en una sola llamada el número de faltantes (**n_missing**), el porcentaje de datos completos (**complete_rate**), la media, la desviación estándar y los percentiles de cada columna, agrupados por tipo de dato. En la consola de RStudio, **skim()** dibuja además un mini histograma con caracteres especiales; para un reporte impreso como este libro usamos su variante **skim_without_charts()**, que muestra exactamente lo mismo sin ese dibujo.

```
1 skim_without_charts(transacciones)
```

```
-- Data Summary -----
                                Values
Name                           transacciones
Number of rows                 1000
Number of columns              14
-----
Column type frequency:
  character                    4
  Date                        1
  difftime                    1
  numeric                     8
-----
Group variables                None

-- Variable type: character -----
  skim_variable n_missing complete_rate min max empty n_unique
1 metodo_pago    0             1   8  15    0         4
2 mes            0             1   7   7    0        12
3 trimestre     0             1   2   2    0         4
4 dia_semana    0             1   5   9    0         7
  whitespace
1          0
2          0
3          0
4          0

-- Variable type: Date -----
  skim_variable n_missing complete_rate min      max
1 fecha        0             1 2023-01-01 2023-12-31
  median      n_unique
```

```

1 2023-06-23      345

-- Variable type: difftime -----
  skim_variable n_missing complete_rate min      max
1 hora          0          1 28800 secs 74700 secs
  median      n_unique
1 51300 secs      52

-- Variable type: numeric -----
  skim_variable  n_missing complete_rate  mean    sd  p0  p25
1 transaccion_id      0          1 500.   289.   1  251.
2 cliente_id          0          1  97.8   57.5   1   49
3 producto_id         0          1  25.3   14.4   1   13
4 cantidad            0          1   2.97   1.45   1    2
5 precio_venta        0          1 318.   172.  20.1 164.
6 tienda_id           0          1   5.61   2.88   1    3
7 vendedor_id         0          1  13.2   7.21   1    7
8 total_transaccion   0          1 960.   760.  22.6 341.
  p50   p75  p100
1 500.  750. 1000
2  97   150   200
3  25   37.2   50
4   3    4    5
5 325.  468.  599.
6   6    8   10
7  13   19   25
8 713. 1479. 2992.

```

Ejemplo práctico: Leer un `skim()` como analista

¿Qué te dice esta salida en 30 segundos?

- **Calidad:** `n_missing` es 0 en todas las columnas y `complete_rate` es 1: no hay faltantes. En datos reales casi nunca pasa; cuando veas un `complete_rate` de 0.85, sabes que 15 % de esa columna está vacía.
- **Categorías:** `metodo_pago` tiene 4 valores distintos (`n_unique`) y `mes`, 12. Si `metodo_pago` tuviera 7 valores, sospecharías de errores de captura como “tarjeta credito” y “Tarjeta Crédito”.
- **Cobertura:** las fechas van del 1 de enero al 31 de diciembre de 2023, pero solo hay 345 fechas distintas: hubo días sin ninguna venta registrada.
- **Rangos:** `cantidad` va de 1 a 5 y el ticket (`total_transaccion`) de 22.6 a 2,992; no hay negativos ni ceros imposibles.

 Hazlo tú

Aplica `skim_without_charts()` a `clientes` y a `productos`. ¿Cuántos segmentos de edad hay? ¿Hay algún producto con `margen_pct` negativo? (Pista: mira el valor `p0`, que es el mínimo.)

4.2 Estadística descriptiva

La **estadística descriptiva** resume muchos datos en pocos números. Responde dos preguntas: ¿cuál es el valor *típico*? (tendencia central) y ¿qué tanto se alejan los datos de ese valor típico? (dispersión). Para no repetir `transacciones$total_transaccion` en cada línea, guardamos esa columna en un vector llamado `ticket`:

```
1 ticket <- transacciones$total_transaccion # 1,000 tickets de venta
2 length(ticket)
```

```
[1] 1000
```

4.2.1 Medidas de tendencia central: media, mediana y moda

La **media** (o promedio) suma todos los valores y divide entre cuántos son. Si llamamos $x_1, x_2, \dots, x_n$ a los n tickets:

$$\bar{x} = \frac{x_1 + x_2 + \dots + x_n}{n} = \frac{1}{n} \sum_{i=1}^n x_i$$

El símbolo $\sum$ (sigma) solo significa “suma todo”. La **mediana** es el valor que queda justo a la mitad cuando ordenas los datos de menor a mayor: la mitad de los tickets está por debajo y la otra mitad por encima. La **moda** es el valor que más se repite; es útil sobre todo para variables categóricas (método de pago, día de la semana).

```
1 media_ticket <- mean(ticket) # promedio
2 mediana_ticket <- median(ticket) # valor central
3 media_ticket
4 mediana_ticket
5 sum(ticket) / length(ticket) # la media "a mano": mismo resultado
```

```
[1] 959.9927
[1] 713.485
[1] 959.9927
```

R no trae una función para la moda (la función `mode()` de R hace otra cosa: dice el tipo de un objeto). La construimos nosotros con lo que aprendiste en el Módulo 1:

```

1 # Moda: el valor que más veces aparece en un vector
2 moda <- function(x) {
3   frecuencias <- table(x)                # cuenta cada valor
4   names(frecuencias)[which.max(frecuencias)] # el más frecuente
5 }
6
7 moda(transacciones$metodo_pago) # método de pago más usado
8 moda(transacciones$dia_semana)  # día con más transacciones
9 moda(transacciones$cantidad)    # unidades más comunes por ticket

```

```

[1] "Tarjeta Crédito"
[1] "viernes"
[1] "1"

```

`table()` cuenta cuántas veces aparece cada valor y `which.max()` devuelve la posición del mayor conteo. La moda siempre sale como texto (por eso "1" aparece entre comillas); si la necesitas como número usa `as.numeric(moda(x))`. Si hay empate, `which.max()` devuelve el primero en orden alfabético, así que conviene revisar la tabla completa:

```
1 table(transacciones$cantidad)
```

```

 1   2   3   4   5
219 196 190 187 208

```

En la empresa: La moda en la operación

La moda responde preguntas operativas muy concretas: ¿cuál es la talla que más se vende? (para surtir más), ¿cuál es el método de pago más usado? (para negociar comisiones con el banco), ¿a qué hora llegan más clientes? (para programar turnos). Aquí, la tarjeta de crédito es el método más frecuente y lo más común es que un ticket lleve 1 sola unidad.

4.2.2 Medidas de dispersión

Dos tiendas pueden tener el mismo ticket promedio y comportarse de forma muy distinta: en una todos gastan alrededor de 900 pesos y en la otra unos gastan 50 y otros 3,000. La **dispersión** mide esa diferencia.

- **Rango**: máximo menos mínimo. Fácil, pero depende solo de dos datos (los más extremos).
- **Varianza** (s^2): el promedio de las distancias al cuadrado respecto a la media. Elevamos al cuadrado para que las distancias negativas no cancelen a las positivas:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Se divide entre $n - 1$ y no entre n por un ajuste técnico que corrige un pequeño sesgo cuando trabajas con muestras; con muchos datos la diferencia es mínima.

- **Desviación estándar** (s): la raíz cuadrada de la varianza. Regresa a las unidades originales (pesos), por eso es la medida de dispersión más usada. Se interpreta como “cuánto se aleja, típicamente, un dato del promedio”.
- **Rango intercuartílico** (IQR): distancia entre el percentil 75 y el 25. Mide la amplitud del 50 % central de los datos y no le afectan los extremos.
- **Coeficiente de variación** (CV): la desviación estándar dividida entre la media, $CV = s/\bar{x}$. Como no tiene unidades, sirve para comparar la variabilidad de cosas distintas (tickets en pesos contra unidades vendidas).

```

1 range(ticket)                # mínimo y máximo
2 diff(range(ticket))          # rango = máximo - mínimo
3 var(ticket)                  # varianza (pesos al cuadrado)
4 sd(ticket)                   # desviación estándar (pesos)
5 IQR(ticket)                  # rango intercuartílico
6 sd(ticket) / mean(ticket)    # coeficiente de variación
7
8 # La varianza "a mano", para que veas que no hay magia:
9 sum((ticket - mean(ticket))^2) / (length(ticket) - 1)

```

```

[1] 22.65 2992.35
[1] 2969.7
[1] 578152.5
[1] 760.3634
[1] 1137.513
[1] 0.7920512
[1] 578152.5

```

¿Qué nos dice esto? Un ticket típico se aleja unos \$760 del promedio de \$960. El coeficiente de variación es 0.79: la desviación estándar equivale al 79 % de la media, lo que indica tickets *mu*y heterogéneos. Como regla práctica, un CV menor a 0.15 indica datos bastante homogéneos, entre 0.15 y 0.30 variabilidad moderada y por encima de 0.30 alta variabilidad. La varianza (578,152.5) está en “pesos al cuadrado”, una unidad sin sentido de negocio; por eso casi siempre reportarás la desviación estándar.

Hazlo tú

Calcula el coeficiente de variación de **cantidad** y de **precio_venta**. ¿Cuál de las dos varía más en términos relativos?

4.2.3 Percentiles y cuantiles

El **percentil** p es el valor que deja por debajo al $p\%$ de los datos. La mediana es el percentil 50; el primer y tercer **cuartil** son los percentiles 25 y 75. En R se calculan con `quantile()`, indicando las proporciones que quieres:

```
1 quantile(ticket, probs = c(0.10, 0.25, 0.50, 0.75, 0.90, 0.95))
```

10%	25%	50%	75%	90%	95%
159.1200	341.1125	713.4850	1478.6250	2113.6140	2501.2725

Los percentiles se leen así: el 10% de los tickets es menor a \$159.12; la mitad, menor a \$713.49; y solo el 5% supera \$2,501.27. Son perfectos para fijar umbrales de negocio: si quieres un programa de beneficios para el 10% de compras más grandes, el corte está en \$2,113.61 (percentil 90).

Ejemplo práctico: Umbral para un programa de lealtad

La gerencia de mercadotecnia quiere regalar un cupón a los tickets que estén en el 20% más alto. ¿A partir de qué monto se entrega el cupón y cuántos tickets del año lo habrían recibido?

```
1 umbral_cupon <- quantile(ticket, probs = 0.80) # percentil 80
2 umbral_cupon
3 sum(ticket > umbral_cupon) # tickets que superan el umbral
```

```
80%
1641.174
[1] 200
```

El cupón se entregaría en compras mayores a \$1,641.17 y, con los datos de 2023, lo habrían recibido 200 tickets (el 20% de 1,000, como era de esperarse). Si cada cupón cuesta 50 pesos, el costo del programa sería de unos 10,000 pesos al año.

4.2.4 Cuando la media engaña

La media tiene un defecto: **es muy sensible a los valores extremos**. Un solo dato gigante la arrastra hacia arriba; la mediana, en cambio, apenas se mueve. Veámoslo con dos ejemplos pequeños creados con `tibble()`.

Ejemplo práctico: Un distribuidor con un cliente gigante

Un distribuidor mayorista tiene 10 clientes. Nueve compran entre 20 y 30 mil pesos al mes; el décimo es una cadena nacional que compra 900 mil. El director pregunta: “¿cuánto compra un cliente típico?”.

```

1 compras_b2b <- tibble(
2   cliente = paste("Cliente", 1:10),
3   compra_mensual = c(22000, 25000, 21000, 28000, 30000,
4                     24000, 26000, 23000, 27000, 900000)
5 )
6
7 compras_b2b %>%
8   summarise(media = mean(compra_mensual),
9             mediana = median(compra_mensual))

```

```

# A tibble: 1 x 2
  media mediana
  <dbl>   <dbl>
1 112600   25500

```

Si respondes “un cliente típico compra 112,600 pesos” estarás describiendo a un cliente que no existe: nueve de diez compran menos de 31 mil. La mediana (25,500) sí describe al cliente típico. La media sigue siendo útil para otra pregunta: si multiplicas 112,600 por 10 clientes, obtienes exactamente las ventas totales. **La media sirve para totales; la mediana, para describir al caso típico.**

El segundo ejemplo es clásico en recursos humanos:

```

1 salarios <- c(rep(15000, 9), 250000) # 9 empleados y un director
2 mean(salarios)
3 median(salarios)

```

```

[1] 38500
[1] 15000

```

Anunciar que “el salario promedio en la empresa es de 38,500 pesos” es técnicamente cierto, pero engañoso: nueve de cada diez personas ganan 15,000. Por eso las estadísticas oficiales de ingresos suelen reportar la mediana.

Consejo

Reporta siempre la media *y* la mediana. Si son parecidas, los datos son bastante simétricos y cualquiera sirve. Si la media es claramente mayor que la mediana, hay valores altos que la arrastran: usa la mediana para describir el caso típico y la media solo para calcular totales.

4.2.5 Resumen estadístico por grupo

En el Módulo 2 aprendiste `group_by()` + `summarise()`. Ahora lo combinamos con todas las medidas anteriores para comparar segmentos. Pregunta de negocio: *¿el comportamiento del ticket cambia según el método de pago?*

```

1 resumen_pago <- transacciones %>%
2   group_by(metodo_pago) %>%
3   summarise(
4     n      = n(),                      # cuántos tickets
5     ventas = round(sum(total_transaccion)), # total vendido
6     media  = mean(total_transaccion),      # ticket promedio
7     mediana = median(total_transaccion),    # ticket típico
8     desv_est = sd(total_transaccion),      # dispersión
9     cv      = sd(total_transaccion) / mean(total_transaccion),
10    .groups = "drop"
11  ) %>%
12  mutate(pct_ventas = ventas / sum(ventas)) %>% # % de las ventas
13  arrange(desc(ventas))
14
15 resumen_pago

```

```

# A tibble: 4 x 8
  metodo_pago      n ventas media mediana desv_est   cv pct_ventas
  <chr>      <int> <dbl> <dbl>   <dbl>   <dbl> <dbl>   <dbl>
1 Tarjeta Crédito  350 351171 1003.   780.   782. 0.779   0.366
2 Efectivo        315 303735  964.   710.   757. 0.785   0.316
3 Tarjeta Débito  233 220855  948.   732.   738. 0.779   0.230
4 Transferencia   102  84232  826.   608.   741. 0.897   0.0877

```

Lectura de negocio: la tarjeta de crédito concentra la mayor parte de las ventas (36.6%) y tiene el ticket promedio más alto (\$1,003); la transferencia es el método menos usado y el de ticket más bajo (\$826). Los cuatro métodos tienen un CV alto (de 0.78 a 0.90): en todos conviven tickets muy pequeños y muy grandes. ¿La diferencia entre \$1,003 y \$826 es real o es azar? No lo sabemos todavía: en la sección 4.10 aprenderás a responderlo con una prueba de hipótesis.

Hazlo tú

Repite el resumen agrupando por **tienda_id**. ¿Qué tienda tiene la mediana de ticket más alta? ¿Coincide con la que tiene la media más alta?

4.3 Forma de la distribución

Los números resumen ayudan, pero nada sustituye a *ver* los datos. La **distribución** de una variable describe qué valores toma y con qué frecuencia. Su herramienta básica es el **histograma**: divide el eje en intervalos y cuenta cuántos datos caen en cada uno. Encima podemos dibujar una **curva de densidad**, una versión suavizada del histograma que deja ver la forma general sin el “ruido” de las barras.

Ejemplo práctico: ¿Cómo se reparte el ticket de compra?

La directora quiere saber si “el ticket típico” es un buen resumen o si hay grupos de clientes con compras muy distintas. Dibujamos el histograma del ticket con su curva de densidad y marcamos la media y la mediana.

```

1 p_hist <- ggplot(transacciones, aes(x = total_transaccion)) +
2   # barras: cuántos tickets caen en cada intervalo de 150 pesos
3   geom_histogram(binwidth = 150, boundary = 0,
4                 fill = color_principal, color = "white", alpha = 0.85) +
5   # densidad reescalada (x 150) para que quede en la escala de las barras
6   geom_density(aes(y = after_stat(count) * 150),
7               color = "#0b0b0b", linewidth = 0.8) +
8   # media (línea continua) y mediana (línea punteada)
9   geom_vline(xintercept = media_ticket, color = color_resalte,
10             linewidth = 0.8) +
11   geom_vline(xintercept = mediana_ticket, color = color_resalte,
12             linewidth = 0.8, linetype = "dashed") +
13   annotate("text", x = media_ticket + 50, y = 140, hjust = 0,
14           label = paste("Media:", dollar(media_ticket)),
15           color = color_resalte, size = 3.5) +
16   annotate("text", x = mediana_ticket - 50, y = 140, hjust = 1,
17           label = paste("Mediana:", dollar(mediana_ticket)),
18           color = color_resalte, size = 3.5) +
19   scale_x_continuous(labels = dollar) +
20   labs(title = "Distribución del ticket por transacción",
21        subtitle = "Muchos tickets pequeños y una cola larga a la derecha",
22        x = "Total de la transacción", y = "Número de transacciones") +
23   tema_libro()
24
25 p_hist

```

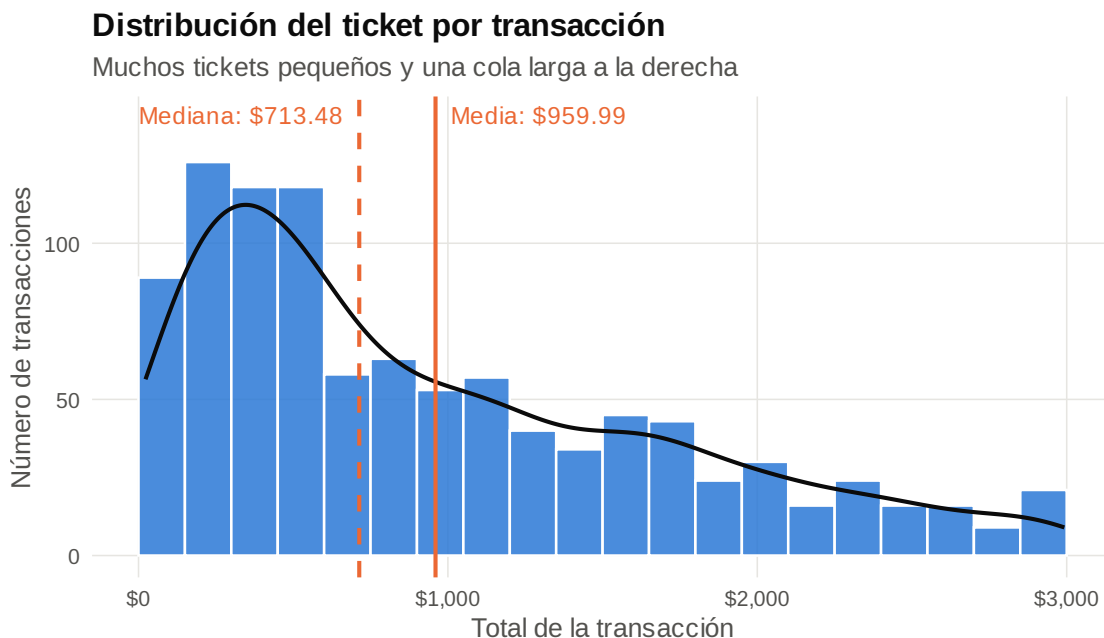


Figura 4.2. Histograma y curva de densidad del ticket. La media queda a la derecha de la mediana porque los tickets grandes la “jalan”.

La figura 4.2 confirma lo que sugerían los números: la mayoría de los tickets se concentra por debajo de 600 pesos y, a partir de ahí, las barras bajan poco a poco formando una **cola** larga hacia la derecha. A esa forma se le llama **asimetría positiva** (o sesgo a la derecha). Observa que la media queda a la derecha de la mediana: los pocos tickets grandes la arrastran.



Consejo

El ancho de las barras (**binwidth**) cambia mucho la impresión que da un histograma. Prueba siempre dos o tres anchos distintos antes de sacar conclusiones; con barras muy anchas se esconden patrones y con barras muy angostas todo parece ruido.

4.3.1 Medir la asimetría

Podemos resumir la forma con un número: el **coeficiente de asimetría**. La idea es promediar las distancias a la media *elevadas al cubo*. Al elevar al cubo se conserva el signo (las distancias negativas siguen siendo negativas) y los valores lejanos pesan muchísimo; si la cola derecha es más larga, dominan los cubos positivos. Para que no dependa de las unidades, se divide entre s^3 :

$$\text{asimetría} = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^3}{s^3}$$

```
1 # Coeficiente de asimetría (versión sencilla)
2 asimetria <- function(x) {
3   mean((x - mean(x))^3) / sd(x)^3
```

```

4 }
5
6 asimetria(ticket)           # ticket de compra
7 asimetria(transacciones$precio_venta) # precio unitario
8 asimetria(compras_b2b$compra_mensual)  # distribuidor con cliente gigante

```

```

[1] 0.8711898
[1] -0.08471191
[1] 2.276413

```

La tabla 4.2 te ayuda a interpretarlo. El ticket tiene una asimetría de 0.87 (positiva y moderada); el precio unitario está muy cerca de cero (los precios se reparten de forma pareja entre 20 y 600 pesos); y el ejemplo del distribuidor, con un cliente gigante, tiene una asimetría fuerte de 2.28.

Tabla 4.2. Cómo interpretar el coeficiente de asimetría.

Valor	Forma	Ejemplo de negocio
Entre -0.5 y 0.5	Aproximadamente simétrica; media $\approx$ mediana	Estatura de clientes, tiempos de entrega estables
Entre 0.5 y 1	Asimetría positiva moderada	Ticket de compra de este módulo
Mayor a 1	Asimetría positiva fuerte (cola derecha larga)	Ingresos de clientes, ventas por producto en catálogos grandes
Menor a -0.5	Asimetría negativa (cola izquierda)	Calificaciones de satisfacción cuando casi todos ponen 9 o 10

En la empresa: El ticket con cola larga

Una distribución con cola larga a la derecha es la norma en el comercio: hay muchas compras chicas y pocas muy grandes. Esto tiene tres consecuencias prácticas:

1. **Metas:** si fijas la meta del ticket con la media, la mayoría de los vendedores quedarán por debajo aunque trabajen bien; la mediana es una referencia más justa para el “vendedor típico”.
2. **Estrategia:** la cola derecha son los clientes que más valen. Identificarlos (percentil 90 o 95) permite diseñar atención preferente.
3. **Oportunidad:** el gran bloque de tickets bajos es el terreno para estrategias de venta cruzada (“¿le agrego...?”) que suban el ticket típico.

Hazlo tú

Dibuja el histograma de `precio_venta` con `binwidth = 50` y calcula su asimetría. ¿Se parece a la forma del ticket? ¿Por qué crees que el ticket (cantidad $\times$ precio) tiene cola y el precio no?

4.4 Datos atípicos (outliers)

Un **dato atípico** (en inglés, *outlier*) es un valor que se aleja mucho del resto. Puede ser un **error** (alguien tecleó 29,923.50 en vez de 2,992.35), un **caso legítimo pero excepcional** (una empresa que compró 50 laptops) o una **señal de fraude**. El trabajo del analista no es borrarlo: es *detectarlo, investigarlo y decidir*.

4.4.1 Regla de $1.5 \times IQR$

La regla más usada la propuso también John Tukey y es la que dibujan los diagramas de caja (*boxplots*). Se consideran atípicos los valores que caen fuera de estas “cercas”:

$$\text{límite inferior} = Q_1 - 1.5 \times IQR \qquad \text{límite superior} = Q_3 + 1.5 \times IQR$$

donde Q_1 y Q_3 son los percentiles 25 y 75, e $IQR = Q_3 - Q_1$. Como usa percentiles, la regla no se deja engañar por los propios atípicos.

```
1 q1 <- quantile(ticket, 0.25)
2 q3 <- quantile(ticket, 0.75)
3 iqr <- q3 - q1
4 limite_inferior <- unname(q1 - 1.5 * iqr) # unname() quita el "25%"
5 limite_superior <- unname(q3 + 1.5 * iqr)
6 c(limite_inferior, limite_superior)
7
8 # ¿Cuántos tickets quedan fuera de las cercas?
9 sum(ticket < limite_inferior | ticket > limite_superior)
```

```
[1] -1365.156 3184.894
[1] 0
```

Ningún ticket es atípico según esta regla: el límite superior (\$3,184.89) está por encima del ticket máximo (\$2,992.35). Tiene sentido: una transacción tiene como máximo 5 unidades de un producto de hasta 600 pesos. El límite inferior es negativo, lo cual simplemente significa que por abajo no hay forma de ser atípico. **“No hay atípicos” también es un resultado valioso:** significa que puedes usar los tickets con tranquilidad.

4.4.2 Puntuación z

La segunda regla usa la **puntuación z** (*z-score*): cuántas desviaciones estándar se aleja cada dato de la media.

$$z_i = \frac{x_i - \bar{x}}{s}$$

Un $z = 2$ significa “dos desviaciones estándar por encima del promedio”. Se suele marcar como atípico todo dato con $|z| > 3$.


```

1 z_ticket <- (ticket - mean(ticket)) / sd(ticket)
2 range(z_ticket)           # el z más bajo y el más alto
3 sum(abs(z_ticket) > 3)    # tickets con |z| > 3

```

```

[1] -1.232756  2.672876
[1] 0

```

El ticket más alto está a 2.67 desviaciones estándar de la media: tampoco hay atípicos por esta regla. Para reutilizar la regla del IQR la convertimos en función:

```

1 # Devuelve TRUE para los valores fuera de las cercas de Tukey
2 marcar_atipicos <- function(x) {
3   q1 <- quantile(x, 0.25, na.rm = TRUE)
4   q3 <- quantile(x, 0.75, na.rm = TRUE)
5   rango_iq <- q3 - q1
6   x < q1 - 1.5 * rango_iq | x > q3 + 1.5 * rango_iq
7 }

```

4.4.3 Cuando los atípicos son errores

Para ver las reglas en acción con errores de verdad, **simulamos** una situación frecuente: al exportar del punto de venta se colaron tres registros problemáticos. Agregamos a una copia de los tickets una venta con el punto decimal recorrido (29,923.50), una devolución capturada como venta negativa (−450) y un ticket en cero.

```

1 # Copia de los tickets + 3 registros con problemas (simulados)
2 tickets_revision <- transacciones %>%
3   select(transaccion_id, total_transaccion) %>%
4   bind_rows(tibble(transaccion_id = c(1001, 1002, 1003),
5                     total_transaccion = c(29923.50, -450, 0)))
6
7 tickets_revision %>%
8   mutate(
9     atipico_iqr = marcar_atipicos(total_transaccion),
10    z = (total_transaccion - mean(total_transaccion)) /
11        sd(total_transaccion),
12    atipico_z = abs(z) > 3
13  ) %>%
14  filter(atipico_iqr | atipico_z | total_transaccion <= 0)

```

```

# A tibble: 3 x 5
  transaccion_id total_transaccion atipico_iqr      z atipico_z
      <dbl>          <dbl> <lgl>      <dbl> <lgl>

```

1	1001	29924.	TRUE	24.3	TRUE
2	1002	-450	FALSE	-1.21	FALSE
3	1003	0	FALSE	-0.829	FALSE

Tres lecciones importantes:

1. Ambas reglas detectan el error de captura (29,923.50): su z es 24.3, absurdamente lejos.
2. **Ninguna regla estadística detecta el -450 ni el 0**, porque están cerca del centro de los datos. Para eso necesitas **reglas de negocio**: “un ticket de venta debe ser mayor a cero”. La estadística no sustituye al conocimiento del negocio.
3. Un solo error contamina las medidas sensibles:

```
1 sd(transacciones$total_transaccion)      # sin los registros erróneos
2 sd(tickets_revision$total_transaccion)    # con los registros erróneos
```

```
[1] 760.3634
[1] 1189.887
```

La desviación estándar pasó de 760 a 1,190 (un 56 % más), sobre todo por *un* solo dato erróneo entre 1,003. Por eso la puntuación z es menos confiable que la regla del IQR cuando hay atípicos fuertes: el propio atípico infla la s con la que se le mide.

4.4.4 Cuando los atípicos son tus mejores clientes

Ahora un caso real. Pregunta de negocio: *¿hay clientes con un gasto anual fuera de lo normal?* Calculamos el gasto de cada cliente en 2023 y aplicamos las dos reglas.

```
1 gasto_clientes <- transacciones %>%
2   group_by(cliente_id) %>%
3   summarise(compras = n(),
4             gasto    = sum(total_transaccion),
5             .groups = "drop") %>%
6   mutate(atipico = marcar_atipicos(gasto),
7          z        = (gasto - mean(gasto)) / sd(gasto))
8
9 gasto_clientes %>%
10  filter(atipico) %>%
11  arrange(desc(gasto))
```

```
# A tibble: 8 x 5
  cliente_id compras  gasto atipico      z
  <dbl>    <int>  <dbl> <lgl>    <dbl>
1         5      13 14680. TRUE     3.65
```

2	90	9	12504.	TRUE	2.85
3	183	9	11848.	TRUE	2.60
4	168	12	11676.	TRUE	2.54
5	55	13	11356.	TRUE	2.42
6	125	10	11240.	TRUE	2.38
7	162	10	11176.	TRUE	2.35
8	31	7	10980.	TRUE	2.28

La regla del IQR marca 8 clientes con gasto anual superior a unos 10,800 pesos; la regla de $|z| > 3$ solo marcaría al cliente 5. Ninguno parece un error: todos compraron entre 7 y 13 veces en el año. **Son tus mejores clientes**, no basura estadística. Borrarlos del análisis sería un error grave: son justo los que un programa de lealtad debe cuidar.

```

1 set.seed(123) # geom_jitter() mueve los puntos al azar
2 p_caja <- ggplot(gasto_clientes, aes(x = gasto, y = "")) +
3   geom_boxplot(width = 0.5, outlier.shape = NA, fill = "grey96") +
4   geom_jitter(aes(color = atipico), height = 0.15,
5               size = 1.8, alpha = 0.8) +
6   scale_color_manual(values = c(`FALSE` = color_gris,
7                                `TRUE` = color_resalte),
8                       labels = c("Dentro de las cercas",
9                                  "Atípico (regla 1.5 × IQR)"),
10                      name = NULL) +
11   scale_x_continuous(labels = dollar) +
12   labs(title = "Gasto anual por cliente en 2023",
13         subtitle = "8 clientes superan el límite superior de la caja",
14         x = "Gasto total del cliente", y = NULL) +
15   tema_libro()
16
17 p_caja

```

Gasto anual por cliente en 2023

8 clientes superan el límite superior de la caja

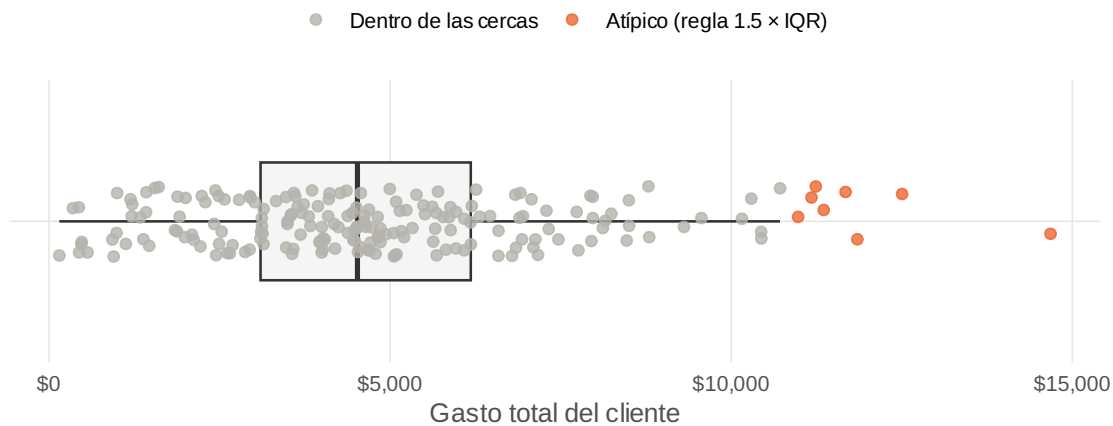


Figura 4.3. Diagrama de caja del gasto anual por cliente. Cada punto es un cliente; en naranja, los que la regla del IQR marca como atípicos.

En la figura 4.3 la caja va del percentil 25 al 75, la línea interior es la mediana y los “bigotes” llegan hasta el último dato dentro de las cercas. Usamos `outlier.shape = NA` para que `geom_boxplot()` no dibuje sus propios puntos atípicos, porque ya los dibujamos todos con `geom_jitter()`.

4.4.5 ¿Qué hacer con un atípico?

Tabla 4.3. Qué hacer con un dato atípico según su origen.

Situación	Acción	Ejemplo
Error de captura evidente	Corregir en la fuente si es posible; si no, excluir y documentarlo	29,923.50 en lugar de 2,992.35
Viola una regla de negocio	Separar y reportar al área responsable	Tickets negativos o en cero
Caso legítimo excepcional	Conservar; analizar aparte o usar medidas robustas (mediana, IQR)	Compra corporativa de 50 laptops
Cliente o producto de alto valor	Conservar y estudiar: suele ser una oportunidad	Los 8 clientes del ejemplo anterior
Posible fraude	Escalar a auditoría antes de tocarlo	Muchas devoluciones del mismo vendedor

▲ Cuidado

Nunca borres atípicos “porque estorban” para que la gráfica se vea bonita o el promedio salga mejor. Si decides excluir datos, deja constancia en tu script (un comentario y un `filter()` explícito) y menciónalo en el reporte: “se excluyeron 3 registros con errores de captura”.

Hazlo tú

Carga `ventas_retail.csv` y aplica `marcar_atipicos()` a la columna `total`. ¿Cuántas ventas quedan fuera de las cercas? Míralas con `filter()`: ¿son errores o ventas grandes legítimas?

4.5 KPIs y métricas de desempeño

Un **KPI** (*Key Performance Indicator*, indicador clave de desempeño) es una métrica que la empresa sigue de forma periódica para saber si va bien o mal. Las ventas del mes son un dato; las ventas del mes *comparadas* contra el mes anterior, contra la meta o contra el mismo mes del año pasado son un KPI. La tabla 4.4 resume los cálculos más comunes.

Tabla 4.4. KPIs de desempeño más usados y su cálculo en R.

KPI	Qué responde	Cálculo en R
Crecimiento MoM	¿Cuánto crecimos contra el mes anterior? (<i>Month over Month</i>)	<code>ventas / lag(ventas) - 1</code>
Acumulado (YTD)	¿Cuánto llevamos vendido en el año? (<i>Year to Date</i>)	<code>cumsum(ventas)</code>
Promedio móvil	¿Cuál es la tendencia sin el ruido de cada mes?	<code>(v + lag(v) + lag(v, 2)) / 3</code>
Índice base 100	¿Cuánto cambiamos respecto a un punto de partida?	<code>ventas / first(ventas) * 100</code>
Variación vs meta	¿Estamos arriba o abajo de lo planeado?	<code>ventas / meta - 1</code>

Ejemplo práctico: Tablero mensual de ventas

La meta comercial de 2023 era vender \$80,000 al mes. Construye una tabla con las ventas mensuales, su crecimiento contra el mes anterior, el acumulado del año, el promedio móvil de 3 meses, el índice con enero = 100 y la variación contra la meta.

```

1 meta_mensual <- 80000    # meta de ventas por mes
2
3 kpi_mensual <- transacciones %>%
4   group_by(mes) %>%
5   summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
6   arrange(mes) %>%      # orden cronológico
7   mutate(
8     crec_mom    = ventas / lag(ventas) - 1,          # vs mes anterior
9     acumulado   = cumsum(ventas),                    # acumulado del año
10    prom_movil   = (ventas + lag(ventas) + lag(ventas, 2)) / 3,
11    indice_100   = ventas / first(ventas) * 100,      # enero = 100
12    vs_meta      = ventas / meta_mensual - 1         # sobre/bajo la meta
13  )
14
15 kpi_mensual

```

```
# A tibble: 12 x 7
  mes      ventas  crec_mom  acumulado  prom_movil  indice_100  vs_meta
<chr>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
1 2023-01 82560.    NA        82560.    NA        100    0.0320
2 2023-02 65899.   -0.202    148459    NA        79.8  -0.176
3 2023-03 90931.    0.380    239390.   79797.    110.    0.137
4 2023-04 57038.   -0.373    296427.   71289.    69.1  -0.287
5 2023-05 93910.    0.646    390338.   80626.    114.    0.174
6 2023-06 88509.   -0.0575   478847.   79819.    107.    0.106
7 2023-07 86456.   -0.0232   565303.   89625.    105.    0.0807
8 2023-08 78929.   -0.0871   644232.   84632.    95.6  -0.0134
9 2023-09 68856.   -0.128    713088.   78080.    83.4  -0.139
10 2023-10 70004.    0.0167   783092.   72596.    84.8  -0.125
11 2023-11 80507.    0.150    863599.   73122.    97.5   0.00634
12 2023-12 96394.    0.197    959993.   82302.    117.    0.205
```

Cómo se lee cada columna, tomando marzo (2023-03) como ejemplo:

- **crec_mom** = 0.380: marzo vendió 38 % más que febrero.
- **acumulado** = 239,390: entre enero y marzo se vendieron \$239,390.
- **prom_movil** = 79,797: el promedio de enero, febrero y marzo. Los dos primeros meses quedan en **NA** porque todavía no hay tres meses para promediar. La función **lag()** “trae” el valor de la fila anterior, y **lag(ventas, 2)** el de dos filas antes.
- **indice_100** = 110: marzo vendió 10 % más que enero.
- **vs_meta** = 0.137: marzo superó la meta en 13.7 %.

Para presentarla a la dirección conviene darle formato de negocio con las funciones de **scales**:

```
1 kpi_mensual %>%
2   transmute(
3     mes,
4     ventas      = dollar(ventas, accuracy = 1),
5     crec_mom    = percent(crec_mom, accuracy = 0.1),
6     acumulado   = dollar(acumulado, accuracy = 1),
7     vs_meta     = percent(vs_meta, accuracy = 0.1)
8   )
```

```
# A tibble: 12 x 5
  mes      ventas  crec_mom  acumulado  vs_meta
<chr>    <chr>    <chr>    <chr>    <chr>
1 2023-01 $82,560 <NA>    $82,560  3.2%
2 2023-02 $65,899 -20.2%   $148,459 -17.6%
3 2023-03 $90,931 38.0%   $239,390 13.7%
4 2023-04 $57,038 -37.3%   $296,427 -28.7%
5 2023-05 $93,910 64.6%   $390,338 17.4%
```

6	2023-06	\$88,509	-5.8%	\$478,847	10.6%
7	2023-07	\$86,456	-2.3%	\$565,303	8.1%
8	2023-08	\$78,929	-8.7%	\$644,232	-1.3%
9	2023-09	\$68,856	-12.8%	\$713,088	-13.9%
10	2023-10	\$70,004	1.7%	\$783,092	-12.5%
11	2023-11	\$80,507	15.0%	\$863,599	0.6%
12	2023-12	\$96,394	19.7%	\$959,993	20.5%

¿Qué nos dice el tablero? Los resultados mes a mes brincan mucho (-20.2%, +38.0%, -37.3%, +64.6%...). Abril (\$57,038) fue el peor mes, 28.7% por debajo de la meta, y diciembre el mejor, 20.5% por encima. Contemos cuántos meses cumplieron la meta y cuánto faltó o sobró en el año:

```
1 meses_arriba <- sum(kpi_mensual$vs_meta >= 0) # meses que cumplieron
2 meses_arriba
3 sum(kpi_mensual$ventas) - meta_mensual * 12 # diferencia anual
```

```
[1] 7
[1] -7.31
```

La meta mensual se cumplió en 7 de los 12 meses y el año cerró con \$959,993: apenas \$7.31 por debajo de la meta anual de \$960,000. En el total anual la empresa está en la meta, pero con un camino muy irregular. Para ver la tendencia sin tanto ruido graficamos las ventas junto con su promedio móvil.

```
1 # Formato largo: una fila por mes y por serie (Módulo 2, pivot_longer)
2 kpi_largo <- kpi_mensual %>%
3   mutate(fecha_mes = as.Date(paste0(mes, "-01"))) %>%
4   select(fecha_mes,
5          `Ventas del mes` = ventas,
6          `Promedio móvil 3 meses` = prom_movil) %>%
7   pivot_longer(-fecha_mes, names_to = "serie", values_to = "valor")
8
9 p_kpi <- ggplot(kpi_largo, aes(x = fecha_mes, y = valor, color = serie)) +
10   geom_hline(yintercept = meta_mensual, linetype = "dashed",
11             color = "#52514e", linewidth = 0.5) +
12   annotate("text", x = as.Date("2023-09-20"), y = meta_mensual + 2500,
13           label = "Meta: $80,000", hjust = 0, size = 3.3,
14           color = "#52514e") +
15   geom_line(linewidth = 0.8, na.rm = TRUE) +
16   geom_point(size = 1.8, na.rm = TRUE) +
17   scale_color_manual(
18     values = c("Ventas del mes" = color_principal,
19               "Promedio móvil 3 meses" = color_resalte),
20     breaks = c("Ventas del mes", "Promedio móvil 3 meses"),
21     name = NULL) +
22   scale_y_continuous(labels = dollar) +
```

```

23 scale_x_date(date_breaks = "1 month", date_labels = "%b") +
24 labs(title = "Ventas mensuales 2023 contra la meta",
25       subtitle = "El promedio móvil suaviza los brinco de cada mes",
26       x = NULL, y = "Ventas") +
27 tema_libro()
28
29 p_kpi

```

Ventas mensuales 2023 contra la meta

El promedio móvil suaviza los brinco de cada mes

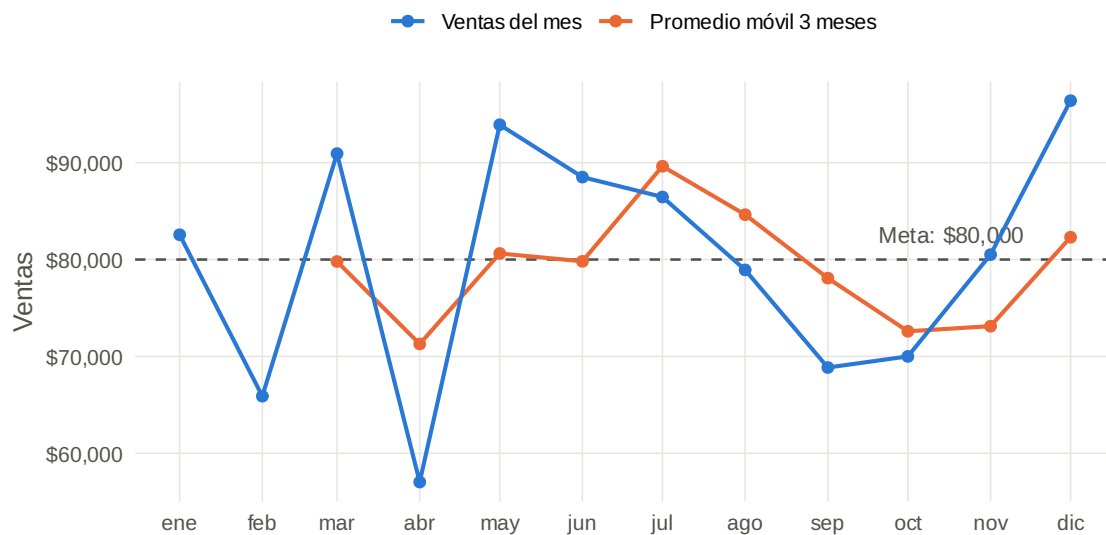


Figura 4.4. Ventas mensuales, promedio móvil de 3 meses y meta mensual.

La figura 4.4 cuenta la historia completa: las ventas mensuales (azul) suben y bajan con fuerza, mientras que el promedio móvil (naranja) muestra la tendencia de fondo: rondó la meta en primavera (con el bache de abril), la superó en julio y agosto, cayó por debajo durante el otoño y se recuperó en diciembre.

En la empresa: Por qué los directivos piden promedios móviles

Un mes malo aislado puede deberse a días festivos, a un problema de inventario o simplemente al azar. Reaccionar a cada brinco (“abril cayó 37%: ¡recorten gastos!”) lleva a decisiones erráticas. El promedio móvil filtra ese ruido: si *también* baja durante varios meses, entonces sí hay una tendencia que atender. En empresas con estacionalidad fuerte se complementa con el crecimiento contra el mismo mes del año anterior (*YoY*, *Year over Year*).

 Consejo

Para promedios móviles de muchos periodos (por ejemplo, 12 meses), escribir `lag()` doce veces es incómodo. El paquete **zoo** (que se instala junto con **forecast**) tiene `zoo::rollmean(ventas, k = 12, fill = NA, align = "right")`, que hace exactamente el mismo cálculo.

 Hazlo tú

Calcula los mismos KPIs por **trimestre** (crecimiento contra el trimestre anterior, acumulado e índice base 100). ¿Qué trimestre fue el mejor? ¿Cuál fue el crecimiento del cuarto trimestre contra el tercero?

4.6 Análisis de Pareto y clasificación ABC

El **principio de Pareto**, o regla 80/20, dice que en muchos negocios una minoría de elementos genera la mayoría de los resultados: el 20 % de los productos genera el 80 % de las ventas, el 20 % de los clientes aporta el 80 % de los ingresos. Los porcentajes exactos varían; lo importante es la idea de *concentración*. El **análisis ABC** aprovecha esa idea para clasificar productos:

- **Clase A:** los productos que, sumados de mayor a menor, acumulan el primer 80 % de los ingresos. Son vitales.
- **Clase B:** los que acumulan del 80 % al 95 %. Importantes.
- **Clase C:** el 5 % restante. Muchos productos que aportan poco.

Ejemplo práctico: ¿Qué productos sostienen el negocio?

La directora quiere saber si hay un grupo pequeño de productos “estrella” del que dependa la empresa, para asegurarse de que nunca falten en inventario.

```

1 pareto <- transacciones %>%
2   group_by(producto_id) %>%
3   summarise(ingresos = sum(total_transaccion), .groups = "drop") %>%
4   arrange(desc(ingresos)) %>%                                # de mayor a menor
5   mutate(
6     pct_ingresos = ingresos / sum(ingresos),                  # peso de cada uno
7     pct_acumulado = cumsum(pct_ingresos),                    # % acumulado
8     pct_productos = row_number() / n(),                      # % del catálogo
9     clase = case_when(
10      pct_acumulado <= 0.80 ~ "A",
11      pct_acumulado <= 0.95 ~ "B",
12      TRUE ~ "C"
13    )
14  ) %>%
15  left_join(productos %>%
16    select(producto_id, categoria, estado_stock, activo),

```

```

17     by = "producto_id")
18
19 pareto %>%
20   select(producto_id, ingresos, pct_ingresos, pct_acumulado, clase) %>%
21   head(10)

```

```

# A tibble: 10 x 5
  producto_id ingresos pct_ingresos pct_acumulado clase
    <dbl>     <dbl>      <dbl>      <dbl> <chr>
1         6  34028.      0.0354      0.0354 A
2        19  28368.      0.0296      0.0650 A
3        31  25430.      0.0265      0.0915 A
4        47  24527.      0.0255      0.117  A
5        46  24470.      0.0255      0.143  A
6        32  24333.      0.0253      0.168  A
7        14  24023.      0.0250      0.193  A
8        24  23593.      0.0246      0.217  A
9        29  22809.      0.0238      0.241  A
10       21  22083.      0.0230      0.264  A

```

El producto más vendido (el 6) aporta apenas el 3.5% de los ingresos, y los 10 primeros productos (el 20% del catálogo) suman el 26.4%. Muy lejos de un 80/20. Veamos el resumen por clase:

```

1 pareto %>%
2   group_by(clase) %>%
3   summarise(productos = n(),
4             pct_catalogo = n() / nrow(pareto),
5             ingresos = sum(ingresos),
6             pct_ingresos = sum(pct_ingresos),
7             .groups = "drop")

```

```

# A tibble: 3 x 5
  clase productos pct_catalogo ingresos pct_ingresos
  <chr>    <int>      <dbl>    <dbl>      <dbl>
1 A         36      0.72  764155.    0.796
2 B          9      0.18  137845.    0.144
3 C          5      0.1   57992.    0.0604

```

```

1 p_pareto <- ggplot(pareto, aes(x = pct_productos, y = pct_acumulado)) +
2   # diagonal: lo que pasaría si todos los productos vendieran lo mismo
3   geom_abline(slope = 1, intercept = 0, linetype = "dashed",
4             color = color_gris) +
5   geom_hline(yintercept = 0.80, color = "#52514e", linewidth = 0.4) +
6   geom_line(color = "#52514e", linewidth = 0.6) +

```

```

7   geom_point(aes(color = clase), size = 2) +
8   # punto de referencia de la regla 80/20
9   annotate("point", x = 0.2, y = 0.8, shape = 4, size = 4, stroke = 1.2) +
10  annotate("text", x = 0.22, y = 0.86, hjust = 0, size = 3.3,
11         label = "Regla 80/20 (referencia)") +
12  scale_color_manual(values = paleta_libro[1:3], name = "Clase") +
13  scale_x_continuous(labels = percent) +
14  scale_y_continuous(labels = percent, limits = c(0, 1)) +
15  labs(title = "Curva de Pareto de ingresos por producto",
16       subtitle = "Ingresos repartidos: no hay pocos productos estrella",
17       x = "% de productos (de mayor a menor ingreso)",
18       y = "% acumulado de ingresos") +
19  tema_libro()
20
21 p_pareto

```

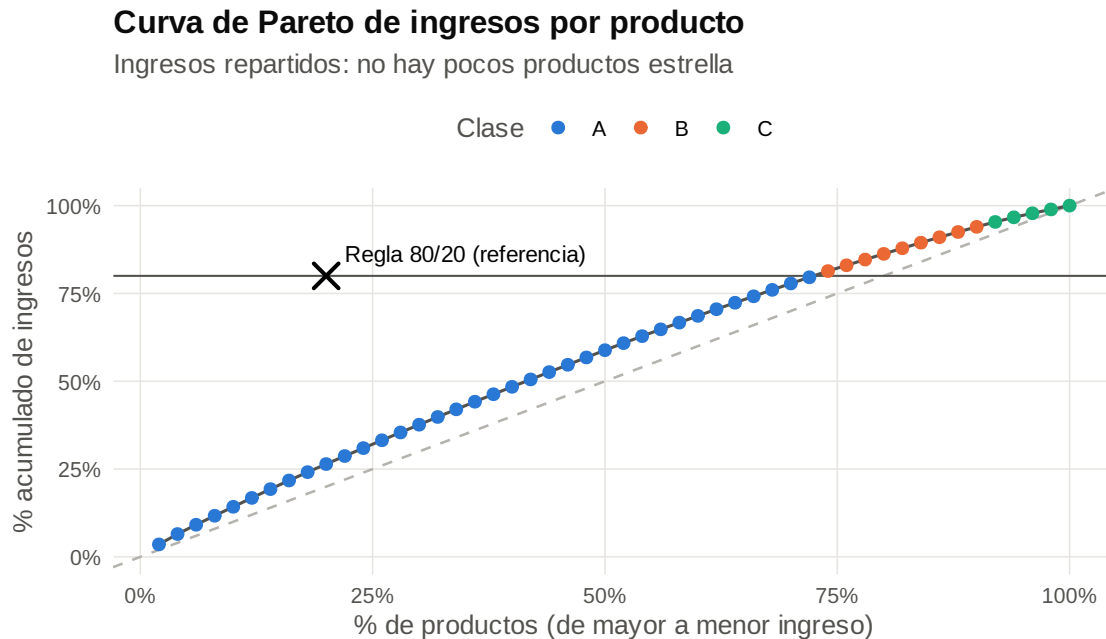


Figura 4.5. Curva de Pareto. Cuanto más se despegue la curva de la diagonal punteada, más concentrados están los ingresos.

Ejemplo práctico: Interpretar la curva de Pareto

En la figura 4.5, el eje horizontal es el porcentaje del catálogo (ordenado de mayor a menor ingreso) y el vertical, el porcentaje acumulado de ingresos. Si la empresa siguiera la regla 80/20, la curva pasaría por la cruz. Nuestra curva está muy pegada a la diagonal: los 36 productos de la clase A (el 72 % del catálogo) apenas juntan el 79.6 % de los ingresos. **Este es un hallazgo en sí mismo:** la empresa no depende de unos cuantos productos estrella; sus ingresos están muy repartidos.

¿Qué implica para el inventario? Cruzamos la clase con el estado del stock:

```
1 pareto %>% count(clase, estado_stock)
2
3 # Productos A con stock bajo: prioridad de reabastecimiento
4 pareto %>%
5   filter(clase == "A", estado_stock != "Stock OK") %>%
6   select(producto_id, categoria, ingresos, estado_stock)
```

```
# A tibble: 4 x 3
  clase estado_stock     n
  <chr> <chr>         <int>
1 A     Stock Bajo         4
2 A     Stock OK          32
3 B     Stock OK           9
4 C     Stock OK           5
# A tibble: 4 x 4
  producto_id categoria  ingresos estado_stock
  <dbl> <chr>         <dbl> <chr>
1         6 Juguetes    34028. Stock Bajo
2        33 Alimentos   21240. Stock Bajo
3         3 Automotriz   19908. Stock Bajo
4        39 Alimentos   18337. Stock Bajo
```

En la empresa: Políticas de inventario ABC

- **Clase A** (36 productos): revisión semanal de inventario, stock de seguridad alto y nunca permitir faltantes. Los 4 productos A con stock bajo (6, 33, 3 y 39) deben reabastecerse de inmediato; entre ellos está el producto 6, ¡el más vendido del año!
- **Clase B** (9 productos): revisión quincenal y stock de seguridad moderado.
- **Clase C** (5 productos): stock mínimo, pedidos bajo demanda y candidatos a revisión (¿se descontinúan?, ¿se promueven?).

Como nuestro catálogo es tan parejo, sacar productos C ahorraría poco espacio pero tampoco perdería muchas ventas: 5 productos que suman el 6.0 % de los ingresos. En catálogos reales, con miles de productos, la clase C suele tener cientos de artículos que ocupan bodega y casi no se venden: ahí el ABC libera mucho dinero.

Hazlo tú

Haz el mismo análisis de Pareto con los *clientes* en lugar de los productos. ¿Qué porcentaje de clientes genera el 80 % de los ingresos? ¿Los ingresos están más o menos concentrados que en los productos?

4.7 Correlación: ¿qué variables se mueven juntas?

La **correlación** mide qué tanto se mueven juntas dos variables numéricas. Se resume en un número entre -1 y $+1$:

- Cerca de $+1$: cuando una sube, la otra también sube (visitas a la tienda y ventas).
- Cerca de -1 : cuando una sube, la otra baja (precio y unidades vendidas).
- Cerca de 0 : no hay relación *lineal* (estatura del cliente y ticket).

Como guía práctica: $|r| < 0.3$ es una relación débil, entre 0.3 y 0.7 moderada, y mayor a 0.7 fuerte.

4.7.1 Pearson y Spearman

La **correlación de Pearson** (r) mide relaciones *lineales* (en línea recta) y usa los valores tal cual; por eso es sensible a los atípicos. Su fórmula compara cómo se desvía cada variable de su media:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}}$$

Si cuando x está por encima de su media y también lo está, los productos del numerador son positivos y r sale positiva. La **correlación de Spearman** (ρ , “ro”) hace lo mismo pero con los *lugares* (rangos) que ocupa cada dato al ordenarlos; mide si la relación es *monótona* (siempre sube o siempre baja, aunque no sea en línea recta) y casi no le afectan los atípicos.

Ejemplo práctico: ¿Qué explica el gasto de un cliente?

Construimos una tabla con una fila por cliente: número de compras, unidades, gasto total, ticket promedio, edad y antigüedad como cliente. Queremos saber qué variables se relacionan con el gasto anual.

```
1 perfil_clientes <- transacciones %>%
2   group_by(cliente_id) %>%
3   summarise(compras = n(),
4             unidades = sum(cantidad),
5             gasto = sum(total_transaccion),
6             ticket = mean(total_transaccion),
7             .groups = "drop") %>%
8   left_join(clientes, by = "cliente_id") %>%
9   mutate(antigüedad = as.numeric(as.Date("2023-12-31") - fecha_registro))
10
11 # Pearson (el método por defecto) y Spearman
12 cor(perfil_clientes$compras, perfil_clientes$gasto)
13 cor(perfil_clientes$compras, perfil_clientes$gasto, method = "spearman")
14 cor(perfil_clientes$edad, perfil_clientes$gasto)
```

```
[1] 0.7927073
[1] 0.7519843
[1] -0.07345163
```

El número de compras tiene una correlación fuerte con el gasto (0.79 con Pearson y 0.75 con Spearman): los clientes que más gastan son, sobre todo, los que compran más veces. La edad, en cambio, casi no tiene relación con el gasto (-0.07).

Para ver la diferencia entre ambos métodos, observa este ejemplo pequeño creado a mano: diez tiendas donde las ventas suben siempre con el número de visitantes, salvo que la décima tienda tuvo una venta corporativa enorme.

```
1 visitantes <- c(100, 120, 150, 170, 200, 220, 250, 270, 300, 320)
2 ventas_dia <- c(10, 12, 14, 17, 19, 22, 24, 27, 29, 300) # último: atípico
3
4 cor(visitantes, ventas_dia) # Pearson
5 cor(visitantes, ventas_dia, method = "spearman") # Spearman
```

```
[1] 0.5726456
[1] 1
```

La relación es perfectamente monótona (más visitantes, siempre más ventas), y Spearman lo detecta con $\rho = 1$. Pearson da 0.57 porque el dato atípico rompe la línea recta. **Regla práctica:** usa Pearson cuando la relación se vea lineal y no haya atípicos fuertes; usa Spearman con datos asimétricos, con atípicos o con escalas ordinales (calificaciones de 1 a 5).

4.7.2 Matriz de correlación

Cuando tienes muchas variables, `cor()` aplicado a un data frame numérico calcula todas las correlaciones a la vez: una **matriz de correlación**.

```
1 matriz_cor <- perfil_clientes %>%
2   select(compras, unidades, gasto, ticket, edad, antiguedad) %>%
3   cor()
4
5 round(matriz_cor, 2)
```

	compras	unidades	gasto	ticket	edad	antiguedad
compras	1.00	0.91	0.79	-0.09	-0.02	0.01
unidades	0.91	1.00	0.87	0.14	-0.01	0.03
gasto	0.79	0.87	1.00	0.46	-0.07	0.05
ticket	-0.09	0.14	0.46	1.00	-0.14	0.05
edad	-0.02	-0.01	-0.07	-0.14	1.00	0.01
antiguedad	0.01	0.03	0.05	0.05	0.01	1.00

La diagonal siempre vale 1 (cada variable consigo misma) y la matriz es simétrica: la correlación de **compras** con **gasto** es la misma que la de **gasto** con **compras**. Leer una tabla así es cansado; **corrplot** la convierte en una imagen:

```
1 corrplot(matriz_cor,
2           method = "color",           # celdas coloreadas
3           type = "upper",             # solo el triángulo superior
4           addCoef.col = "black",      # escribe el coeficiente
5           number.cex = 0.8,          # tamaño de los números
6           tl.col = "black",           # color de los nombres
7           tl.srt = 45,                # nombres inclinados
8           col = colorRampPalette(c("#e34948", "white", "#2a78d6"))(200),
9           diag = FALSE)              # sin la diagonal de unos
```

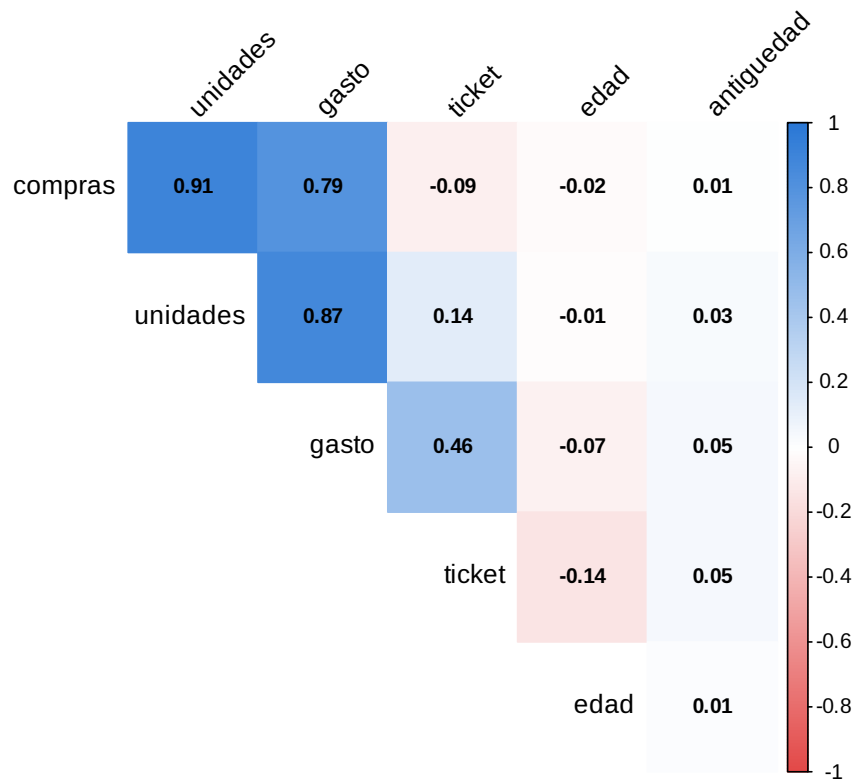


Figura 4.6. Matriz de correlación del perfil de clientes. Escala divergente: azul = correlación positiva, rojo = negativa, blanco = sin relación.

La figura 4.6 usa una paleta *divergente* (rojo, blanco, azul) porque la correlación tiene un centro natural en cero. Se lee de un vistazo: el gasto se explica por la frecuencia (**compras**, 0.79) y por las **unidades** (0.87); el **ticket** promedio tiene una relación moderada con el gasto (0.46), y la edad y la antigüedad no se relacionan con nada. Para la directora esto significa que **aumentar la frecuencia de compra** (que el cliente regrese) pesa más que la edad o la antigüedad del cliente.

4.7.3 ¿La correlación es real? `cor.test()`

Una correlación calculada con pocos datos puede salir distinta de cero por pura casualidad. La función `cor.test()` responde: *si en realidad no hubiera ninguna relación, ¿qué tan raro sería observar una correlación como esta?* (en la sección 4.10 verás esta lógica con detalle).

```
1 cor.test(perfil_clientes$edad, perfil_clientes$gasto)
```

```
Pearson's product-moment correlation

data:  perfil_clientes$edad and perfil_clientes$gasto
t = -1.0311, df = 196, p-value = 0.3038
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.21073472  0.06667273
sample estimates:
      cor 
-0.07345163
```

Lo que importa de esta salida:

- **cor = -0.073**: la correlación de la muestra.
- **p-value = 0.3038**: una probabilidad de 30 %. Si la edad no tuviera ninguna relación con el gasto, sería muy común ver una correlación como esta solo por azar. Conclusión: **no hay evidencia** de que la edad se relacione con el gasto.
- **95 percent confidence interval**: el valor real de la correlación podría estar entre -0.21 y 0.07; el intervalo incluye el cero.

Hazlo tú

Aplica `cor.test()` a `compras` y `gasto`. ¿Qué p-valor obtienes? ¿El intervalo de confianza incluye el cero? Repite con `method = "spearman"`.

4.7.4 Correlación no implica causalidad

Que dos variables se muevan juntas **no significa** que una cause a la otra. Puede haber una tercera variable, llamada **variable de confusión**, que mueve a las dos. Para verlo con claridad **simulamos** datos donde conocemos la verdad: en una cadena de tiendas, la temperatura del día aumenta tanto la venta de helados como la de ventiladores, pero los helados *no* influyen en los ventiladores.

```
1 set.seed(123)
2 dias_verano <- tibble(
3   temperatura = runif(90, min = 18, max = 38),      # grados
4   helados      = 20 + 3 * temperatura + rnorm(90, sd = 8),
```



```

5   ventiladores = 5 + 1.5 * temperatura + rnorm(90, sd = 5)
6   )
7
8   # Ventas de helados vs ventiladores: correlación alta...
9   cor(dias_verano$helados, dias_verano$ventiladores)

```

```
[1] 0.8160674
```

Una correlación de 0.82 es muy fuerte. Un análisis ingenuo diría: “pongamos los ventiladores junto a los helados, porque vender helados impulsa la venta de ventiladores”. Pero sabemos (porque así los simulamos) que la única causa común es la temperatura. En la sección 4.11 verás cómo la regresión múltiple permite “controlar” por la temperatura y comprobar que, a igual temperatura, los helados no explican los ventiladores.

En la empresa: Correlaciones engañosas en la empresa

- “Las tiendas con más empleados venden más”: tal vez las tiendas grandes tienen más empleados *y* más clientes.
- “Los clientes que usan la app gastan más”: quizá los clientes que ya gastaban mucho son los que descargan la app.
- “Cuando subimos la publicidad subieron las ventas”: ¿o se subió la publicidad justo en diciembre, cuando las ventas suben solas?

Para demostrar causalidad se necesitan experimentos controlados (como las pruebas A/B de la sección 4.10.3) o técnicas que controlen las variables de confusión.

4.8 Probabilidad y distribuciones útiles

Una **distribución de probabilidad** es un modelo que describe qué valores puede tomar algo incierto y con qué probabilidad. No necesitas memorizar fórmulas: R trae funciones con una convención de nombres muy sencilla (tabla 4.5).

Tabla 4.5. Prefijos de las funciones de distribución en R.

Prefijo	Qué calcula	Ejemplos
d	Probabilidad de un valor exacto (densidad)	<code>dbinom()</code> , <code>dpois()</code>
p	Probabilidad acumulada: $P(X \leq x)$	<code>pnorm()</code> , <code>pbinom()</code>
q	El valor que deja por debajo una probabilidad dada (cuantil)	<code>qnorm()</code>
r	Números aleatorios que siguen esa distribución	<code>rnorm()</code> , <code>rbinom()</code>

4.8.1 Distribución normal: ventas semanales

La **distribución normal** es la famosa “campana de Gauss”: simétrica, con la mayoría de los datos cerca de la media y cada vez menos hacia los extremos. Queda definida por dos números: la media (μ) y la desviación estándar (σ). Una regla útil: en una normal, cerca del 68 % de los datos cae a menos de una desviación estándar de la media y el 95 % a menos de dos.

El ticket individual no es normal (tiene cola a la derecha), pero las *sumas* de muchas ventas tienden a serlo. Esto se debe al **teorema del límite central**, uno de los resultados más importantes de la estadística: al sumar o promediar muchos datos independientes, el resultado se parece cada vez más a una normal, sin importar la forma de los datos originales. Comprobémoslo con las ventas semanales:

```
1 ventas_semanales <- transacciones %>%
2   # semanas completas de lunes a domingo (del 2 de enero al 31 de dic.)
3   filter(fecha >= as.Date("2023-01-02")) %>%
4   mutate(semmana = floor_date(fecha, unit = "week", week_start = 1)) %>%
5   group_by(semmana) %>%
6   summarise(ventas = sum(total_transaccion), .groups = "drop")
7
8 media_sem <- mean(ventas_semanales$ventas)
9 sd_sem    <- sd(ventas_semanales$ventas)
10 nrow(ventas_semanales)
11 c(media = media_sem, desv_est = sd_sem)
12 asimetria(ventas_semanales$ventas)
```

```
[1] 52
      media  desv_est
18444.936  4591.285
[1] 0.2219801
```

Tenemos 52 semanas completas con venta media de \$18,445 y desviación estándar de \$4,591. La asimetría es pequeña (0.22): la forma es casi simétrica. La figura 4.7 compara el histograma con la curva normal teórica.

```
1 p_normal <- ggplot(ventas_semanales, aes(x = ventas)) +
2   geom_histogram(aes(y = after_stat(density)), binwidth = 2000,
3     fill = color_principal, color = "white", alpha = 0.85) +
4   # curva normal con la media y desviación de los datos
5   stat_function(fun = dnorm, args = list(mean = media_sem, sd = sd_sem),
6     color = color_resalte, linewidth = 0.9) +
7   geom_vline(xintercept = 25000, linetype = "dashed", color = "#52514e") +
8   annotate("text", x = 25300, y = 9e-05, hjust = 0, size = 3.3,
9     label = "¿Más de $25,000?", color = "#52514e") +
10  scale_x_continuous(labels = dollar) +
11  scale_y_continuous(labels = NULL) +
12  labs(title = "Ventas semanales de 2023 y curva normal",
```

```

13     subtitle = "52 semanas; la curva naranja es Normal(18,445; 4,591)",
14     x = "Ventas de la semana", y = "Densidad") +
15     tema_libro()
16
17 p_normal

```

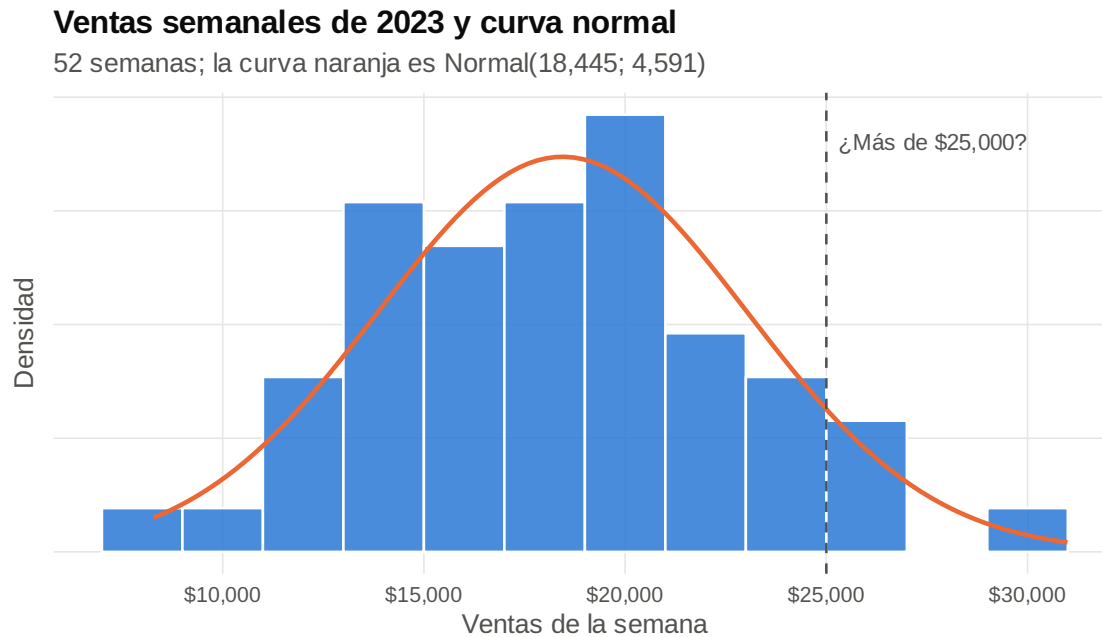


Figura 4.7. Las ventas semanales se ajustan bien a una distribución normal.

Con el modelo normal podemos responder preguntas de planeación:

Ejemplo práctico: Probabilidades con la normal

1. ¿Qué probabilidad hay de vender más de \$25,000 en una semana (lo que activaría un bono para el equipo)?
2. ¿Cuánto inventario (en pesos) debe haber para cubrir la demanda del 90 % de las semanas?
3. ¿Qué probabilidad hay de una semana “mala”, con menos de \$12,000?

```

1  # 1.  $P(\text{ventas} > 25,000) = 1 - P(\text{ventas} \leq 25,000)$ 
2  1 - pnorm(25000, mean = media_sem, sd = sd_sem)
3
4  # Comprobación con los datos reales: proporción de semanas > 25,000
5  mean(ventas_semanales$ventas > 25000)
6
7  # 2. Venta semanal que solo se supera el 10% de las veces
8  qnorm(0.90, mean = media_sem, sd = sd_sem)
9

```

```

10 # 3.  $P(\text{ventas} < 12,000)$ 
11 pnorm(12000, mean = media_sem, sd = sd_sem)

```

```

[1] 0.0766864
[1] 0.07692308
[1] 24328.9
[1] 0.08019928

```

Respuestas: (1) hay un 7.7% de probabilidad de superar \$25,000 en una semana, unas 4 semanas al año; los datos reales coinciden (7.7%, es decir, 4 de 52 semanas). (2) Con un inventario equivalente a \$24,329 se cubre la demanda del 90% de las semanas. (3) Una semana por debajo de \$12,000 tiene una probabilidad de 8.0%.

▲ Cuidado

Verifica siempre que la normal sea razonable antes de usarla (histograma, asimetría cercana a cero). Si hubieras usado la normal con los *tickets individuales*, que tienen cola larga, habrías calculado probabilidades negativas o muy equivocadas para los tickets bajos.

4.8.2 Distribución binomial: conversiones de una campaña

La **distribución binomial** cuenta cuántos “éxitos” hay en n intentos independientes, cuando cada intento tiene la misma probabilidad p de éxito. Ejemplos: cuántos de 200 correos generan una compra, cuántos de 50 clientes aceptan una oferta, cuántas de 1,000 piezas salen defectuosas. El número esperado de éxitos es $n \times p$.

Ejemplo práctico: Campaña de correo a clientes premium

La empresa enviará un correo con una oferta a 200 clientes. Por campañas anteriores, sabe que el 5% de quienes reciben el correo compran. ¿Cuántas compras debe esperar? ¿Qué probabilidad hay de conseguir exactamente 10? ¿Y de conseguir 15 o más, que es lo que el gerente prometió?

```

1 n_correos <- 200      # intentos
2 p_conv    <- 0.05     # probabilidad de conversión
3
4 n_correos * p_conv    # conversiones esperadas
5 dbinom(10, size = n_correos, prob = p_conv) #  $P(\text{exactamente } 10)$ 
6 pbinom(5, size = n_correos, prob = p_conv)  #  $P(5 \text{ o menos})$ 
7 1 - pbinom(14, size = n_correos, prob = p_conv) #  $P(15 \text{ o más})$ 

```

```

[1] 10
[1] 0.1283574
[1] 0.0623425
[1] 0.07813442

```

Lo esperado son 10 compras, pero conseguir *exactamente* 10 solo tiene un 12.8% de probabilidad: el azar mueve el resultado. Hay un 6.2% de probabilidad de un resultado decepcionante de 5 o menos, y apenas un 7.8% de llegar a 15 o más. El gerente prometió algo poco probable: para esperar 15 compras con un 5% de conversión habría que enviar 300 correos.

```
1 # ¿Cuántos correos hay que enviar para tener 90% de probabilidad
2 # de conseguir al menos 15 compras?
3 envios <- seq(200, 600, by = 10)
4 prob_15_o_mas <- 1 - pbinom(14, size = envios, prob = p_conv)
5 min(envios[prob_15_o_mas >= 0.90])
```

```
[1] 400
```

Para prometer 15 compras con un 90% de seguridad hay que enviar al menos 400 correos, el doble de lo planeado. Este tipo de cálculo evita compromisos imposibles con la dirección.

4.8.3 Distribución de Poisson: llegadas de clientes

La **distribución de Poisson** cuenta cuántos eventos ocurren en un intervalo (de tiempo o de espacio) cuando ocurren al azar a un ritmo promedio constante λ ("lambda"). Ejemplos: clientes que llegan a la caja por hora, llamadas al centro de atención por minuto, quejas por día. Una propiedad característica: en una Poisson la media y la varianza son iguales (λ).

Nuestros datos permiten comprobarlo. Contemos cuántas transacciones hubo cada uno de los 365 días de 2023 (incluidos los días con cero ventas):

```
1 tx_por_dia <- tibble(fecha = seq(as.Date("2023-01-01"),
2                               as.Date("2023-12-31"), by = "day")) %>%
3   left_join(count(transacciones, fecha), by = "fecha") %>%
4   mutate(n = replace_na(n, 0)) # días sin ventas = 0
5
6 lambda <- mean(tx_por_dia$n)
7 c(media = lambda, varianza = var(tx_por_dia$n))
8
9 # Frecuencia observada vs la que predice una Poisson(lambda)
10 tibble(transacciones_dia = 0:6) %>%
11   mutate(
12     dias_observados = map_int(transacciones_dia,
13                               ~ sum(tx_por_dia$n == .x)),
14     dias_poisson = round(365 * dpois(transacciones_dia, lambda), 1)
15   )
```

```
media varianza
2.739726 2.544709
```

```
# A tibble: 7 x 3
  transacciones_dia dias_observados dias_poisson
      <int>          <int>          <dbl>
1         0           20          23.6
2         1           69          64.6
3         2           85          88.5
4         3           78          80.8
5         4           63          55.3
6         5           32          30.3
7         6           14          13.8
```

La media (2.74) y la varianza (2.54) son muy parecidas, y los días observados con 0, 1, 2... transacciones se parecen mucho a los que predice la Poisson. `map_int()` (del paquete **purrr**, parte del **tidyverse**) aplica el conteo a cada valor de `transacciones_dia`.

Ejemplo práctico: Personal en cajas

En la sucursal Centro llegan en promedio 12 clientes por hora a las cajas en la tarde. Cada cajero atiende cómodamente a 8 clientes por hora. Con dos cajeros (capacidad de 16), ¿qué probabilidad hay de que en una hora lleguen más clientes de los que se pueden atender? ¿Y con tres cajeros (capacidad 24)?

```
1 lambda_hora <- 12
2 1 - ppois(16, lambda = lambda_hora) # P(más de 16): 2 cajeros
3 1 - ppois(24, lambda = lambda_hora) # P(más de 24): 3 cajeros
4 dpois(0, lambda = lambda_hora)     # P(ningún cliente en la hora)
```

```
[1] 0.101291
[1] 0.0006856332
[1] 6.144212e-06
```

Con dos cajeros, en un 10.1 % de las horas de la tarde habrá más clientes que capacidad (se formarán filas, aproximadamente 1 de cada 10 horas). Con tres cajeros ese riesgo baja a 0.07 %. La última línea muestra que una hora sin ningún cliente es prácticamente imposible (0.0006 %; el **e-06** significa “por 10^{-6} ”). La decisión ya no es “intuición contra intuición”: es comparar el costo de un tercer cajero contra el costo de las filas en el 10 % de las horas.

Hazlo tú

Si el ritmo sube a 15 clientes por hora en diciembre, ¿qué probabilidad hay de superar la capacidad con dos cajeros? ¿Y con tres?

4.9 Muestreo e intervalos de confianza

Casi nunca tienes todos los datos que te interesan. Los 1,000 tickets de 2023 son una **muestra** de todas las compras que tus clientes harán; una encuesta a 400 clientes es una muestra de toda la clientela (la **población**). La pregunta clave es: *si tomara otra muestra, ¿cuánto cambiaría mi resultado?*

4.9.1 Error estándar

El **error estándar** (EE) mide cuánto variaría la media si repitieras el muestreo muchas veces. Se calcula así:

$$EE = \frac{s}{\sqrt{n}}$$

Fíjate en el $\sqrt{n}$ del denominador: con más datos, el error estándar baja, pero despacio. Para reducir el error a la mitad necesitas *cuatro* veces más datos.

```
1 n_tickets <- length(ticket)
2 error_estandar <- sd(ticket) / sqrt(n_tickets)
3 error_estandar
```

```
[1] 24.0448
```

No confundas los dos conceptos: la desviación estándar (\$760) dice cuánto varían los tickets *individuales*; el error estándar (\$24) dice cuánto variaría el *promedio* de 1,000 tickets.

4.9.2 Intervalo de confianza para el ticket promedio

Un **intervalo de confianza** (IC) es un rango de valores plausibles para el verdadero valor de la población. El IC del 95 % para una media se construye como:

$$\bar{x} \pm t \times EE$$

donde t es un número que sale de la **distribución t de Student** (muy parecida a la normal) y, con muestras grandes, vale casi 2. Hagámoslo “a mano” y luego con `t.test()`:

```
1 t_critico <- qt(0.975, df = n_tickets - 1) # valor t para 95%
2 t_critico
3 media_ticket - t_critico * error_estandar # límite inferior
4 media_ticket + t_critico * error_estandar # límite superior
5
6 # Lo mismo en una línea:
7 t.test(ticket)$conf.int
```

```
[1] 1.962341
[1] 912.8086
[1] 1007.177
[1] 912.8086 1007.1768
attr(,"conf.level")
[1] 0.95
```

Concepto clave: Cómo interpretar un intervalo de confianza

Con los datos de 2023, el ticket promedio está entre \$912.81 y \$1,007.18 con una confianza del 95 %. La interpretación correcta es: *si repitiéramos este muestreo muchas veces y calculáramos un intervalo con cada muestra, el 95 % de esos intervalos contendría el verdadero ticket promedio*. En la práctica, dilo así: “estimamos un ticket promedio de \$960, con un margen de error de $\pm \$47$ ”.

No es correcto decir “el 95 % de los tickets está entre \$913 y \$1,007”: el intervalo habla del *promedio*, no de los tickets individuales (que van de \$23 a \$2,992).

En la empresa: El margen de error en las decisiones

Si el presupuesto de 2024 supone 12,000 transacciones, el ingreso esperado es $12,000 \times 960 \approx 11.5$ millones, pero con el IC podrías presentar un rango: entre $12,000 \times 913 \approx 10.95$ y $12,000 \times 1,007 \approx 12.09$ millones. Presentar un rango en vez de un número único es más honesto y ayuda a planear escenarios.

4.9.3 Bootstrap: intervalos sin fórmulas

¿Y si quieres un intervalo para la *mediana*, o para cualquier medida que no tenga una fórmula sencilla? El **bootstrap** es una técnica ingeniosa: simula “otras muestras” tomando, con reemplazo, 1,000 tickets al azar de tus propios 1,000 tickets. Algunos tickets saldrán repetidos y otros no saldrán; así cada remuestra es ligeramente distinta. Repites esto miles de veces, calculas la mediana de cada remuestra y te quedas con el 95 % central de esos resultados.

```
1 set.seed(123) # para que el resultado sea reproducible
2 medianas_boot <- replicate(5000, {
3   remuestra <- sample(ticket, size = length(ticket), replace = TRUE)
4   median(remuestra)
5 })
6
7 length(medianas_boot) # 5,000 medianas
8 quantile(medianas_boot, probs = c(0.025, 0.975)) # IC 95% (percentiles)
```

```
[1] 5000
    2.5% 97.5%
641.23 796.43
```


`replicate(5000, {...})` repite 5,000 veces el código entre llaves y guarda cada resultado. El IC del 95 % para la mediana del ticket va de unos \$641 a \$796. La figura 4.8 muestra las 5,000 medianas simuladas.

```
1 ic_boot <- quantile(mediana_boot, probs = c(0.025, 0.975))
2
3 p_boot <- ggplot(tibble(mediana = medianas_boot), aes(x = mediana)) +
4   geom_histogram(binwidth = 5, fill = color_principal, color = "white") +
5   geom_vline(xintercept = ic_boot, color = color_resalte,
6             linewidth = 0.8, linetype = "dashed") +
7   geom_vline(xintercept = mediana_ticket, color = "#0b0b0b",
8             linewidth = 0.8) +
9   scale_x_continuous(labels = dollar) +
10  labs(title = "Distribución bootstrap de la mediana del ticket",
11       subtitle = "5,000 remuestras; líneas naranjas = IC del 95%",
12       x = "Mediana del ticket en cada remuestra",
13       y = "Frecuencia") +
14  tema_libro()
15
16 p_boot
```

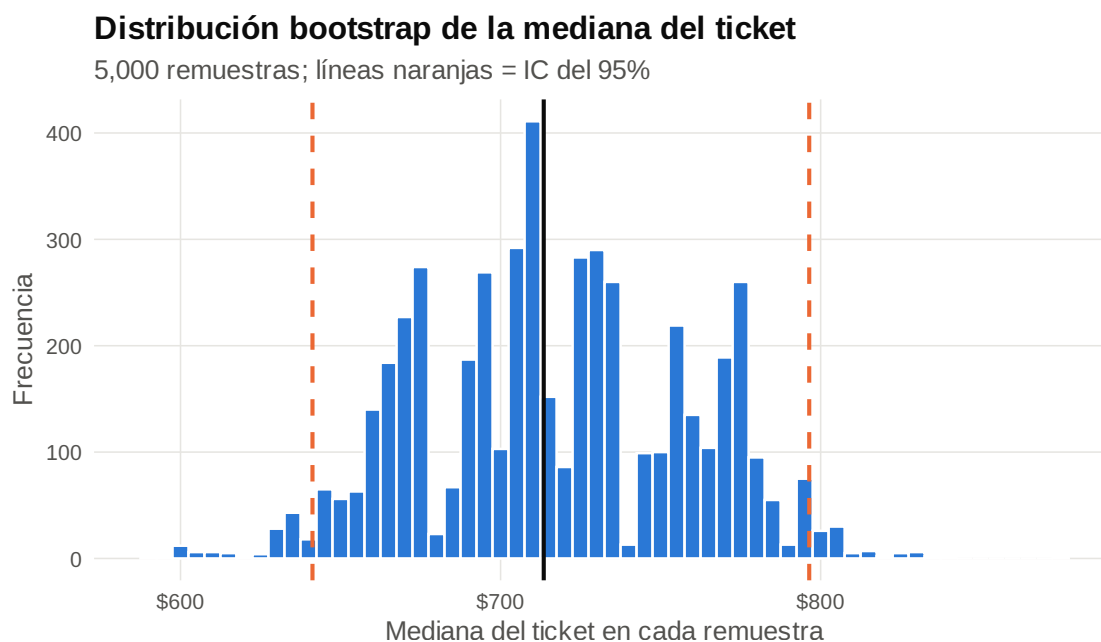


Figura 4.8. Medianas de 5,000 remuestras bootstrap. La línea negra es la mediana observada; entre las líneas naranjas queda el 95 % de las remuestras.

Hazlo tú

Usa el bootstrap para obtener un IC del 95% del *percentil 90* del ticket (cambia `median(remuestra)` por `quantile(remuestra, 0.9)`). ¿Es más ancho o más angosto que

el de la mediana? ¿Por qué crees que pasa?

4.10 Pruebas de hipótesis para negocios

Llegamos a la pregunta que aparece en todas las juntas: “¿esta diferencia es real o es casualidad?”. Los clientes premium gastan en promedio \$977 por ticket y los demás \$951. ¿Es suficiente para decir que los premium gastan más? Con otra muestra de tickets, la diferencia podría desaparecer o invertirse. Las **pruebas de hipótesis** dan un método para decidir.

4.10.1 La lógica en cinco conceptos

Concepto clave: Hipótesis nula, alternativa y p-valor

- **Hipótesis nula** (H_0): la postura “aburrida”: no hay diferencia, no hay efecto. Ejemplo: “el ticket promedio de premium y no premium es igual”.
- **Hipótesis alternativa** (H_1): lo que quieres demostrar: “el ticket promedio es distinto”.
- **p-valor**: la probabilidad de observar una diferencia *tan grande o más* que la de tus datos **si H_0 fuera cierta**. Un p-valor pequeño significa: “sería muy raro ver esto por puro azar”.
- **Nivel de significancia** (α): el umbral que fijas *antes* de ver los datos, casi siempre 0.05 (5 %). Si $p < \alpha$ rechazas H_0 y dices que el resultado es **estadísticamente significativo**. Si $p \geq \alpha$, *no* rechazas H_0 : no hay evidencia suficiente.

Una analogía útil es un juicio: H_0 es “el acusado es inocente”. Solo lo declaras culpable si la evidencia es muy fuerte (p-valor pequeño). Si la evidencia no alcanza, el veredicto es “no culpable”, que **no es lo mismo** que “inocente demostrado”. Igual en estadística: no rechazar H_0 no demuestra que no haya diferencia; solo dice que tus datos no la demuestran.

Tabla 4.6. Los dos errores posibles en una prueba de hipótesis.

	En realidad H_0 es cierta	En realidad H_0 es falsa
Rechazas H_0	Error tipo I (falso positivo): lanzas a todo el país una campaña que en realidad no funciona. Su probabilidad es α .	Acierto: detectas un efecto real.
No rechazas H_0	Acierto: no te dejas engañar por el azar.	Error tipo II (falso negativo): descartas una campaña que sí funcionaba. Se reduce con muestras más grandes.

▲ Cuidado

Significancia estadística no es lo mismo que importancia práctica. Con millones de datos, una diferencia de 50 centavos en el ticket puede ser “significativa” ($p < 0.05$) y aun así no importarle a nadie. Con pocos datos, una diferencia enorme puede no ser significativa. Reporta

siempre el *tamaño* de la diferencia (en pesos o en puntos porcentuales) junto con el p-valor, y pregúntate: ¿esta diferencia cambia alguna decisión?

4.10.2 Prueba t: ¿los clientes premium gastan más?

La **prueba t de Student** compara las medias de dos grupos. Primero unimos las transacciones con los clientes (Módulo 2) para saber quién es premium:

```
1 tx_clientes <- transacciones %>%
2   left_join(clientes, by = "cliente_id")
3
4 tx_clientes %>%
5   group_by(es_premium) %>%
6   summarise(tickets = n(),
7             media    = mean(total_transaccion),
8             mediana  = median(total_transaccion),
9             desv     = sd(total_transaccion),
10            .groups = "drop")
```

```
# A tibble: 2 x 5
  es_premium tickets media mediana desv
  <lg1>      <int> <dbl>   <dbl> <dbl>
1 FALSE         653  951.    672.  775.
2 TRUE          347  977.    803.  732.
```

Planteamos: H_0 : el ticket promedio es igual en ambos grupos; H_1 : es distinto. Nivel de significancia $\alpha = 0.05$.

```
1 prueba_premium <- t.test(total_transaccion ~ es_premium,
2                           data = tx_clientes)
3 prueba_premium
```

```
Welch Two Sample t-test

data: total_transaccion by es_premium
t = -0.52255, df = 741.31, p-value = 0.6014
alternative hypothesis: true difference in means between group FALSE and group TRUE is
  ↪ not equal to 0
95 percent confidence interval:
 -123.43528  71.53793
sample estimates:
mean in group FALSE mean in group TRUE
      950.9885      976.9372
```

Lectura línea por línea:

- **Welch Two Sample t-test**: R usa por defecto la versión de Welch, que no supone que los dos grupos tengan la misma varianza. Es la opción segura.
- **t = -0.52255**: el **estadístico t**, la diferencia de medias medida en errores estándar. Valores lejos de cero (más allá de ± 2 , aproximadamente) indican diferencias claras; -0.52 está muy cerca de cero.
- **p-value = 0.6014**: si premium y no premium gastaran lo mismo, veríamos una diferencia como esta el 60 % de las veces. Como $0.60 > 0.05$, **no rechazamos H_0** .
- **95 percent confidence interval**: la diferencia real (no premium menos premium) está entre -123 y $+72$ pesos. Incluye el cero.
- **sample estimates**: las medias de cada grupo.

Puedes extraer las piezas para usarlas en un reporte:

```
1 prueba_premium$p.value
2 prueba_premium$conf.int
```

```
[1] 0.6014434
[1] -123.43528  71.53793
attr(,"conf.level")
[1] 0.95
```

En la empresa: “No hay evidencia” es un resultado valioso

Conclusión para la directora: *con los datos de 2023 no hay evidencia de que los clientes premium tengan un ticket promedio distinto al de los demás; la diferencia observada (\$26) es compatible con el azar*. Esto no es un fracaso del análisis: es información para decidir. Si la membresía premium cuesta dinero (descuentos, envíos gratis), hay que preguntarse qué obtiene la empresa a cambio. Quizá los premium compren con más *frecuencia* aunque no gasten más por ticket: esa es la siguiente hipótesis a revisar (ejercicio 4.6).

4.10.3 Prueba A/B de proporciones: campaña A vs campaña B

Una **prueba A/B** es un experimento: divides al azar a tus clientes en dos grupos, a cada uno le muestras una versión distinta (de un correo, una página, un precio) y comparas los resultados. Como la asignación es al azar, las dos poblaciones son iguales en todo lo demás, y la diferencia se puede atribuir a la versión. Es la forma más limpia de demostrar *causalidad* en un negocio.

Ejemplo práctico: ¿Qué asunto de correo convierte más?

Mercadotecnia envió 2,000 correos con el asunto A (“Ofertas de la semana”) y 2,000 con el asunto B (“Tu cupón personal te espera”). Como queremos saber si el método funciona, **simulamos** los resultados con tasas de conversión conocidas: 5 % para A y 6.5 % para B. En la vida real no conocerías estas tasas: justamente eso es lo que la prueba intenta averiguar.

```

1 set.seed(123)
2 campana <- tibble(
3   version = rep(c("A", "B"), each = 2000),
4   # 1 = compró, 0 = no compró; tasas reales: A 5%, B 6.5%
5   convirtio = c(rbinom(2000, size = 1, prob = 0.050),
6               rbinom(2000, size = 1, prob = 0.065))
7 )
8
9 resumen_ab <- campana %>%
10   group_by(version) %>%
11   summarise(enviados = n(),
12             conversiones = sum(convirtio),
13             tasa = mean(convirtio),
14             .groups = "drop")
15 resumen_ab

```

```

# A tibble: 2 x 4
  version enviados conversiones  tasa
  <chr>      <int>      <int> <dbl>
1 A         2000         101 0.0505
2 B         2000         130 0.065

```

H_0 : las dos versiones tienen la misma tasa de conversión. H_1 : las tasas son distintas. La función `prop.test()` recibe el número de éxitos y el número de intentos de cada grupo:

```

1 prueba_ab <- prop.test(x = resumen_ab$conversiones,
2                        n = resumen_ab$enviados)
3 prueba_ab

```

```

      2-sample test for equality of proportions with continuity
      correction

data:  resumen_ab$conversiones out of resumen_ab$enviados
X-squared = 3.602, df = 1, p-value = 0.05771
alternative hypothesis: two.sided
95 percent confidence interval:
 -0.0294509801  0.0004509801
sample estimates:

```

```
prop 1 prop 2
0.0505 0.0650
```

Lectura: la versión A convirtió 5.05 % y la B 6.50 %, pero el p-valor es 0.058, apenas *mayor* a 0.05. Formalmente **no rechazamos** H_0 : con 2,000 correos por grupo, una diferencia así todavía podría deberse al azar. El intervalo de confianza de la diferencia (A menos B) va de -2.9 a $+0.05$ puntos porcentuales e incluye el cero.

Aquí hay una lección importante. Nosotros *sabemos* que B es mejor (así lo simulamos), y aun así la prueba no lo detectó: acabamos de cometer un **error tipo II**. ¿La causa? Una muestra demasiado pequeña para detectar una diferencia de 1.5 puntos. La función `power.prop.test()` calcula cuántos envíos se necesitan para detectar una diferencia con una probabilidad dada (la **potencia**, típicamente 80 %):

```
1 power.prop.test(p1 = 0.05, p2 = 0.065, power = 0.80)$n
```

```
[1] 3779.801
```

Se necesitan unos 3,780 correos *por grupo*. Repetimos el experimento (de nuevo simulado, con las mismas tasas reales) con 4,000 correos por versión:

```
1 set.seed(123)
2 campana_4000 <- tibble(
3   version = rep(c("A", "B"), each = 4000),
4   convirtio = c(rbinom(4000, size = 1, prob = 0.050),
5               rbinom(4000, size = 1, prob = 0.065))
6 )
7
8 resumen_4000 <- campana_4000 %>%
9   group_by(version) %>%
10  summarise(enviados = n(), conversiones = sum(convirtio),
11            .groups = "drop")
12
13 prop.test(x = resumen_4000$conversiones, n = resumen_4000$enviados)
```

```
2-sample test for equality of proportions with continuity
correction
```

```
data: resumen_4000$conversiones out of resumen_4000$enviados
X-squared = 5.0351, df = 1, p-value = 0.02484
alternative hypothesis: two.sided
95 percent confidence interval:
 -0.022041478 -0.001458522
sample estimates:
prop 1 prop 2
```

```
0.04975 0.06150
```

Ahora $p = 0.025 < 0.05$: rechazamos H_0 y concluimos que el asunto B convierte más (6.15 % contra 4.98 %). El intervalo (que R calcula como A menos B, por eso sale negativo) dice que B supera a A por entre 0.15 y 2.2 puntos porcentuales. Y la *significancia práctica* es clara: un punto más de conversión sobre 100,000 correos son unas 1,000 compras adicionales.

Consejo

Decide el tamaño de la prueba A/B *antes* de lanzarla (con `power.prop.test()`) y no la detengas en cuanto el p-valor baje de 0.05: si revisas el resultado cada hora y paras cuando “sale”, aumentas mucho la probabilidad de un falso positivo (error tipo I).

4.10.4 ANOVA: ¿el ticket cambia entre categorías?

Cuando quieres comparar las medias de *más de dos* grupos, no hagas muchas pruebas t de dos en dos (con 10 categorías serían 45 pruebas, y alguna saldría “significativa” por azar). Usa el **análisis de varianza** (ANOVA). H_0 : todas las medias son iguales; H_1 : al menos una es distinta. La idea es comparar cuánto varían las medias *entre* grupos con cuánto varían los datos *dentro* de cada grupo.

```
1 tx_productos <- transacciones %>%
2   left_join(productos, by = "producto_id")
3
4 ticket_categoria <- tx_productos %>%
5   group_by(categoria) %>%
6   summarise(n      = n(),
7             media = mean(total_transaccion),
8             ee     = sd(total_transaccion) / sqrt(n()),
9             .groups = "drop") %>%
10  arrange(desc(media))
11 ticket_categoria
```

```
# A tibble: 10 x 4
  categoria      n media    ee
  <chr>      <int> <dbl> <dbl>
1 Juguetes    118 1060.  77.1
2 Alimentos  115 1057.  69.9
3 Automotriz  105  991.  76.5
4 Hogar       64  991.  90.7
5 Belleza    187  974.  58.8
6 Electrónica 137  968.  61.7
7 Deportes   92  937.  78.3
8 Libros      70  817.  85.2
9 Ropa        38  791. 127.
10 Mascotas   74  779.  70.5
```

Los tickets promedio van de \$779 (Mascotas) a \$1,060 (Juguetes). ¿Es una diferencia real? Veamos las medias con su intervalo de confianza aproximado ($\text{media} \pm 2 \text{ errores estándar}$):

```

1 p_anova <- ggplot(ticket_categoria,
2                   aes(x = media, y = reorder(categoria, media))) +
3   geom_vline(xintercept = mean(ticket), linetype = "dashed",
4             color = color_gris) +
5   geom_errorbarh(aes(xmin = media - 2 * ee, xmax = media + 2 * ee),
6                 height = 0.25, color = color_principal,
7                 linewidth = 0.8) +
8   geom_point(size = 2.5, color = color_principal) +
9   scale_x_continuous(labels = dollar) +
10  labs(title = "Ticket promedio por categoría con IC del 95%",
11        subtitle = "Intervalos muy encimados; gris = promedio general",
12        x = "Ticket promedio", y = NULL) +
13  tema_libro()
14
15 p_anova

```

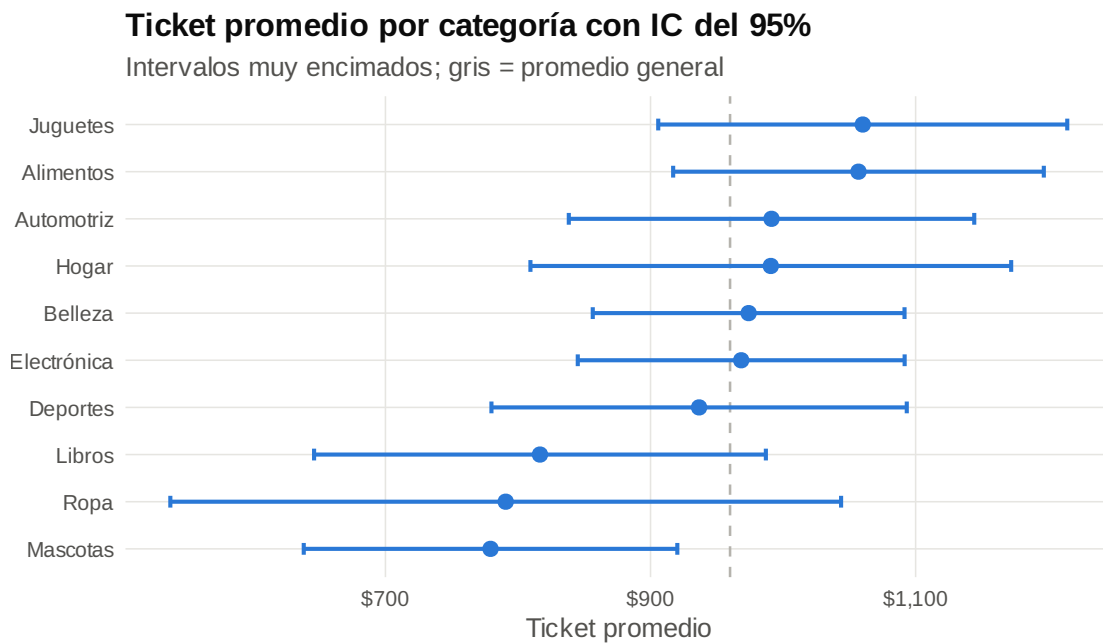


Figura 4.9. Ticket promedio por categoría. Cuando los intervalos se enciman tanto, es difícil afirmar que alguna categoría sea distinta.

```

1 modelo_anova <- aov(total_transaccion ~ categoria, data = tx_productos)
2 summary(modelo_anova)

```


	Df	Sum Sq	Mean Sq	F value	Pr(>F)
categoría	9	7466897	829655	1.441	0.166
Residuals	990	570107428	575866		

La columna clave es **Pr(>F)**, el p-valor: 0.166. Como es mayor a 0.05, **no hay evidencia** de que el ticket promedio dependa de la categoría. El **F value** (1.441) compara la variación entre categorías con la variación dentro de ellas; valores cercanos a 1 indican que las categorías no se distinguen más de lo que se distinguen los tickets entre sí. Hallazgo para la directora: las categorías difieren en *volumen* (Belleza tiene 187 tickets y Ropa 38), no en ticket promedio.

Cuando el ANOVA sí es significativo: TukeyHSD

Probemos ahora con el segmento de edad del cliente (Joven, Adulto, Maduro, Senior):

```
1 modelo_segmento <- aov(total_transaccion ~ segmento, data = tx_clientes)
2 summary(modelo_segmento)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
segmento	3	4672766	1557589	2.708	0.0441 *
Residuals	996	572901558	575202		

...					

Ahora $p = 0.044 < 0.05$: al menos un segmento tiene un ticket promedio distinto. El asterisco junto al p-valor es la forma en que R marca la significancia (una estrella: $p < 0.05$; dos: $p < 0.01$; tres: $p < 0.001$); la leyenda completa aparece en la línea **Signif. codes**, que aquí recortamos. ¿Cuál? El ANOVA no lo dice. Para eso se usa la prueba de **Tukey HSD** (*Honest Significant Difference*), que compara todos los pares de grupos ajustando los p-valores por hacer varias comparaciones:

```
1 TukeyHSD(modelo_segmento)
```

Tukey multiple comparisons of means
95% family-wise confidence level

Fit: aov(formula = total_transaccion ~ segmento, data = tx_clientes)

\$segmento		diff	lwr	upr	p adj
Joven-Adulto		62.47256	-170.39680	295.34191	0.9007953
Maduro-Adulto		-138.85507	-299.46454	21.75439	0.1172294
Senior-Adulto		-47.79558	-216.73288	121.14173	0.8858662
Maduro-Joven		-201.32763	-423.37427	20.71901	0.0912671
Senior-Joven		-110.26813	-338.41089	117.87463	0.5990701

```
Senior-Maduro    91.05950  -62.61673  244.73573  0.4229310
```

Cada fila es una comparación: **diff** es la diferencia de medias, **lwr** y **upr** su intervalo de confianza y **p adj** el p-valor ajustado. Ningún par individual queda por debajo de 0.05; el más cercano es Maduro contra Joven ($p = 0.09$): los clientes maduros gastan unos \$201 menos por ticket que los jóvenes. Esto pasa a veces: el ANOVA detecta que “algo” difiere, pero la evidencia es débil y no alcanza para señalar un par concreto. **Conclusión honesta:** hay indicios de que el ticket varía con la edad (los maduros gastan algo menos), pero la evidencia es débil; conviene confirmarlo con los datos de 2024 antes de diseñar una campaña por edad.

4.10.5 Chi-cuadrada: ¿el método de pago depende de la tienda?

Las pruebas anteriores comparan *promedios* de una variable numérica. Cuando las dos variables son *categorías*, se usa la **prueba chi-cuadrada de independencia** (χ^2). H_0 : las dos variables son independientes (la mezcla de métodos de pago es la misma en todas las tiendas); H_1 : están relacionadas.

Ejemplo práctico: Terminales de pago por sucursal

Finanzas quiere saber si debe negociar terminales bancarias distintas para cada sucursal o si todas las tiendas tienen la misma mezcla de métodos de pago.

```
1 tabla_pago <- table(transacciones$metodo_pago, transacciones$tienda_id)
2 tabla_pago
```

	1	2	3	4	5	6	7	8	9	10
Efectivo	26	36	35	29	33	43	28	31	27	27
Tarjeta Crédito	37	37	35	29	38	28	32	29	45	40
Tarjeta Débito	14	19	28	25	15	30	16	33	25	28
Transferencia	10	7	11	11	9	8	9	13	11	13

```
1 prueba_chi <- chisq.test(tabla_pago)
2 prueba_chi
3 round(prueba_chi$expected[, 1:5], 1) # conteos esperados (5 tiendas)
```

Pearson's Chi-squared test

```
data:  tabla_pago
X-squared = 28.953, df = 27, p-value = 0.3632
```

```
1 2 3 4 5
```

Efectivo	27.4	31.2	34.3	29.6	29.9
Tarjeta Crédito	30.4	34.6	38.1	32.9	33.2
Tarjeta Débito	20.3	23.1	25.4	21.9	22.1
Transferencia	8.9	10.1	11.1	9.6	9.7

El p-valor es 0.36: **no hay evidencia** de que el método de pago dependa de la tienda. Las diferencias que ves en la tabla (por ejemplo, 13 transferencias en la tienda 8 contra 7 en la tienda 2) son las esperables por azar. Los **expected** son los conteos que habría si las variables fueran totalmente independientes; la prueba compara los observados con estos. Decisión práctica: una sola negociación con el banco, con la misma configuración de terminales para todas las sucursales.

▲ Cuidado

La chi-cuadrada necesita conteos esperados de al menos 5 en casi todas las celdas. Si hay muchas celdas con esperados pequeños, R muestra la advertencia **Chi-squared approximation may be incorrect**. En ese caso agrupa categorías poco frecuentes o usa `fisher.test()`.

4.10.6 ¿Qué prueba uso?

Tabla 4.7. Guía rápida para elegir una prueba de hipótesis.

Pregunta de negocio	Tipo de datos	Función
¿El ticket promedio es distinto de \$900?	Una variable numérica vs un valor	<code>t.test(x, mu = 900)</code>
¿Premium y no premium gastan distinto?	Numérica + categórica de 2 grupos	<code>t.test(y ~ g)</code>
¿Mejoró la venta de cada tienda después del cambio?	Numérica, mismos sujetos antes y después	<code>t.test(antes, despues, paired = TRUE)</code>
¿Qué versión de la campaña convierte más?	Proporciones de 2 o más grupos	<code>prop.test()</code>
¿El ticket cambia entre categorías o tiendas?	Numérica + categórica de 3 o más grupos	<code>aov() + TukeyHSD()</code>
¿El método de pago depende de la tienda?	Dos variables categóricas	<code>chisq.test()</code>
¿El gasto se relaciona con la edad?	Dos numéricas	<code>cor.test()</code>
Datos muy asimétricos o pocos datos (alternativa sin supuesto de normalidad)	Numérica + 2 grupos / 3+ grupos	<code>wilcox.test() / kruskal.test()</code>

🔧 Hazlo tú

Usa `t.test()` para comparar el ticket de clientes hombres y mujeres (columna **genero** de **tx_clientes**). ¿El resultado es significativo? ¿Cuál es la diferencia en pesos? Guarda tu respuesta: la retomaremos en la sección de errores comunes.

4.11 Regresión lineal

La correlación dice *si* dos variables se mueven juntas; la **regresión lineal** dice *cuánto*: “por cada mil pesos extra en publicidad, las ventas suben en promedio tantos miles”. Además permite predecir y, con varias variables, separar el efecto de cada una.

4.11.1 Regresión lineal simple

La regresión simple ajusta la línea recta que mejor pasa por una nube de puntos:

$$\text{ventas} = \beta_0 + \beta_1 \times \text{publicidad} + \varepsilon$$

- β_0 (**intercepto**): las ventas esperadas con publicidad cero.
- β_1 (**pendiente**): cuánto cambian las ventas por cada unidad adicional de publicidad.
- ε (**error** o residuo): todo lo que la línea no explica.

R elige β_0 y β_1 con el método de **mínimos cuadrados**: la línea que hace más pequeña la suma de los errores al cuadrado.

Ejemplo práctico: ¿Cuánto vende cada peso de publicidad?

La directora preguntó: “si invertimos más en publicidad local, ¿cuánto deberíamos esperar vender?”. Nuestros datos del curso no tienen inversión publicitaria, así que **simulamos** 60 sucursales de una cadena con una relación conocida: ventas mensuales (miles de pesos) = $120 + 3.5 \times \text{publicidad (miles de pesos)} + 0.25 \times \text{tamaño (m}^2\text{)} + \text{ruido aleatorio}$. Así podremos comprobar si la regresión “descubre” los verdaderos coeficientes.

```
1 set.seed(123)
2 sucursales <- tibble(
3   publicidad = round(runif(60, min = 5, max = 50), 1),    # miles de $
4   tamano_m2  = round(runif(60, min = 300, max = 800))    # m2
5 ) %>%
6   mutate(ventas = round(120 + 3.5 * publicidad + 0.25 * tamano_m2 +
7                       rnorm(60, sd = 30), 1))           # miles de $
8
9 head(sucursales, 4)
10 cor(sucursales$publicidad, sucursales$ventas)
```

```
# A tibble: 4 x 3
  publicidad tamano_m2 ventas
    <dbl>      <dbl> <dbl>
1    17.9      633   352.
2    40.5      347   333.
3    23.4      492   315.
4    44.7      437   355.
```

```
[1] 0.7253112
```

La función `lm()` (*linear model*) ajusta la regresión. Se escribe con una fórmula: la variable a explicar, una tilde (`~`) y la variable explicativa.

```
1 modelo_simple <- lm(ventas ~ publicidad, data = sucursales)
2 summary(modelo_simple)
```

```
Call:
lm(formula = ventas ~ publicidad, data = sucursales)

Residuals:
    Min       1Q   Median       3Q      Max
-102.991  -26.073   -8.259   36.544  101.151

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  261.4845     12.9860   20.136 < 2e-16 ***
publicidad    3.3904      0.4225    8.024 5.59e-11 ***
---
...

Residual standard error: 43.23 on 58 degrees of freedom
Multiple R-squared:  0.5261,    Adjusted R-squared:  0.5179
F-statistic: 64.38 on 1 and 58 DF,  p-value: 5.586e-11
```

Concepto clave: Cómo leer `summary()` de una regresión

- **Residuals**: resumen de los errores (real menos predicho). La mediana cercana a cero y un mínimo y máximo parecidos en tamaño son buena señal.
- **Coefficients**, columna **Estimate**: el intercepto es 261.5 y la pendiente de **publicidad** es 3.39. Traducción: **por cada mil pesos adicionales en publicidad, las ventas mensuales suben en promedio 3.39 mil pesos**. El verdadero valor simulado era 3.5: la regresión se acercó bastante.
- **Std. Error**: la incertidumbre de cada coeficiente. Un IC aproximado del 95% es el coeficiente ± 2 errores estándar: $3.39 \pm 2 \times 0.42$, es decir, de 2.5 a 4.2.
- **t value** y **Pr(>|t|)**: prueba de hipótesis de que el coeficiente es cero (H_0 : “la publicidad no tiene relación con las ventas”). Un p-valor de **5.59e-11** (0.0000000000559) es prácticamente cero: la relación es clarísima.
- **Residual standard error**: 43.23. Las predicciones se equivocan típicamente por unos 43 mil pesos.
- **Multiple R-squared**: 0.526. El **coeficiente de determinación** (R^2) indica que la

publicidad explica el 52.6 % de las diferencias de ventas entre sucursales. El resto se debe a otras cosas (el tamaño, el azar...). **Adjusted R-squared** corrige el R^2 por el número de variables; úsalo para comparar modelos con distinto número de variables.

- **F-statistic** y su **p-value**: prueba si el modelo en conjunto explica algo. Aquí sí.
- Las estrellas (***) marcan los p-valores muy pequeños; la línea **Signif. codes** con su leyenda la recortamos.

Para usar los coeficientes en otros cálculos, extráelos con `coef()` y obtén sus intervalos de confianza con `confint()`:

```
1 coef(modelo_simple)
2 confint(modelo_simple)
```

```
(Intercept)  publicidad
261.484542    3.390407
              2.5 %    97.5 %
(Intercept) 235.490330 287.478754
publicidad   2.544602   4.236212
```

```
1 p_regresion <- ggplot(sucursales, aes(x = publicidad, y = ventas)) +
2   geom_point(color = color_principal, size = 2, alpha = 0.8) +
3   # recta de regresión con su banda de confianza del 95%
4   geom_smooth(method = "lm", formula = y ~ x, color = color_resalte,
5             fill = color_resalte, alpha = 0.15, linewidth = 0.9) +
6   annotate("text", x = 6, y = 480, hjust = 0, size = 3.5,
7             label = "ventas = 261.5 + 3.39 × publicidad\nR² = 0.53") +
8   scale_x_continuous(labels = label_dollar(suffix = "k")) +
9   scale_y_continuous(labels = label_dollar(suffix = "k")) +
10  labs(title = "Publicidad vs ventas por sucursal (datos simulados)",
11        subtitle = "Cada punto es una sucursal; banda = IC 95% de la recta",
12        x = "Inversión en publicidad (miles)",
13        y = "Ventas mensuales (miles)") +
14  tema_libro()
15
16 p_regresion
```

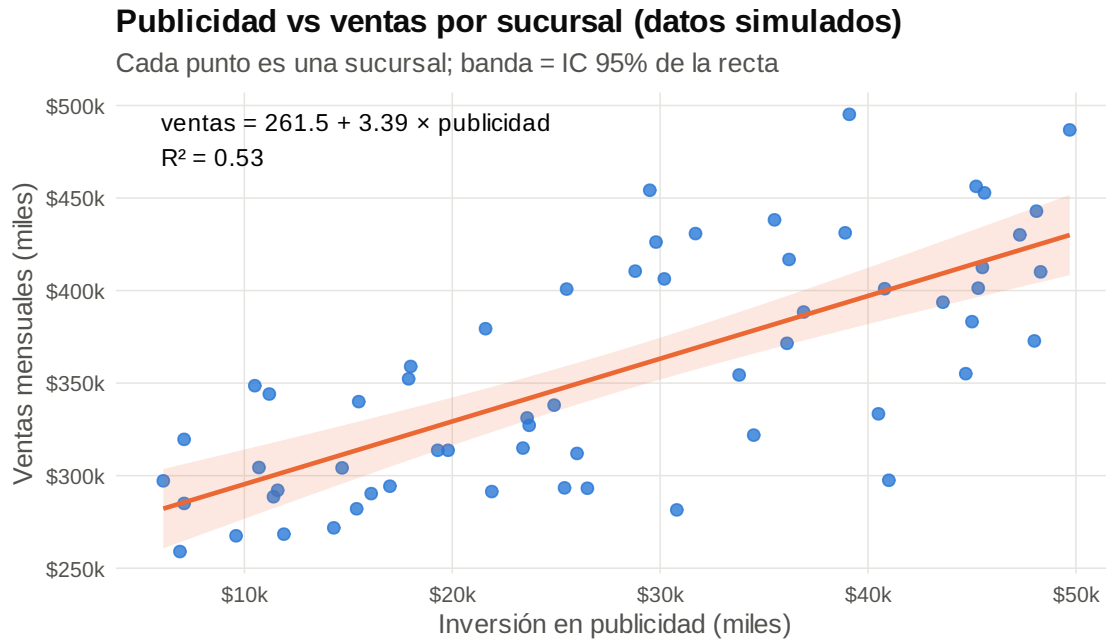


Figura 4.10. Recta de regresión de ventas sobre publicidad. La pendiente indica cuánto suben las ventas por cada mil pesos de publicidad.

4.11.2 Predecir con `predict()`

Con el modelo podemos responder la pregunta de la directora. ¿Cuánto esperamos vender en una sucursal que invierta 30 o 40 mil pesos al mes?

```
1 nuevas <- tibble(publicidad = c(30, 40))
2
3 # Intervalo de confianza: para las ventas PROMEDIO de muchas sucursales
4 predict(modelo_simple, newdata = nuevas, interval = "confidence")
5
6 # Intervalo de predicción: para UNA sucursal concreta
7 predict(modelo_simple, newdata = nuevas, interval = "prediction")
```

	fit	lwr	upr
1	363.1967	351.8641	374.5294
2	397.1008	381.8640	412.3376

	fit	lwr	upr
1	363.1967	275.9223	450.4712
2	397.1008	309.2341	484.9675

Con 30 mil pesos de publicidad esperamos ventas de 363 mil pesos al mes (**fit**). Fíjate en la diferencia entre los dos intervalos:

- **confidence**: el *promedio* de todas las sucursales que inviertan 30 mil estará entre 351 y 374

mil. Es angosto.

- **prediction:** una sucursal concreta que invierta 30 mil venderá entre 276 y 450 mil. Es mucho más ancho, porque cada sucursal tiene su propia variación.

Para presupuestar una tienda usa el intervalo de predicción; para estimar el total de la cadena, el de confianza.

▲ Cuidado

No extrapoles. Los datos van de 5 a 50 mil pesos de publicidad. Si predices con 200 mil, el modelo dará un número ($261.5 + 3.39 \times 200 = 939.6$), pero nada garantiza que la relación siga siendo una recta tan lejos: la publicidad suele tener rendimientos decrecientes (el mercado se satura). Predice solo dentro del rango de los datos que usaste.

4.11.3 Regresión lineal múltiple

En la vida real las ventas dependen de muchas cosas a la vez. La **regresión múltiple** incluye varias variables explicativas y estima el efecto de cada una *manteniendo constantes las demás*:

```
1 modelo_multiple <- lm(ventas ~ publicidad + tamano_m2, data = sucursales)
2 summary(modelo_multiple)
```

Call:

```
lm(formula = ventas ~ publicidad + tamano_m2, data = sucursales)
```

Residuals:

Min	1Q	Median	3Q	Max
-67.498	-17.754	-3.576	15.366	65.949

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	118.86100	16.83335	7.061	2.52e-09 ***
publicidad	3.64007	0.26384	13.797	< 2e-16 ***
tamano_m2	0.24322	0.02519	9.654	1.34e-13 ***

...

Residual standard error: 26.86 on 57 degrees of freedom

Multiple R-squared: 0.8202, Adjusted R-squared: 0.8138

F-statistic: 130 on 2 and 57 DF, p-value: < 2.2e-16

Ahora el modelo dice: **a igual tamaño de tienda**, cada mil pesos de publicidad se asocian con 3.64 mil pesos más de ventas; y **a igual publicidad**, cada metro cuadrado adicional suma 0.243 mil pesos (243 pesos) al mes. Ambos coeficientes están cerca de los valores reales (3.5 y 0.25) y sus p-valores son prácticamente cero. El R^2 ajustado subió de 0.518 a 0.814: añadir el tamaño mejoró mucho el modelo, y el error típico de las predicciones bajó de 43 a 27 mil pesos.

Ejemplo práctico: Controlar por una variable de confusión

¿Recuerdas los helados y los ventiladores de la sección 4.7.4? Si incluimos la temperatura en el modelo, podemos ver qué pasa con el efecto aparente de los helados.

```
1 lm(ventiladores ~ helados, data = dias_verano) %>% coef()
2 lm(ventiladores ~ helados + temperatura, data = dias_verano) %>%
3   summary() %>%
4   coef() %>%
5   round(3)
```

(Intercept)	helados			
4.1419801	0.4138884			
	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	4.627	2.916	1.587	0.116
helados	0.029	0.079	0.365	0.716
temperatura	1.408	0.272	5.180	0.000

Sin la temperatura, cada helado adicional parece “traer” 0.41 ventiladores. Al controlar por la temperatura, el coeficiente de los helados cae casi a cero (0.03) y su p-valor (0.716) indica que no es significativo; la temperatura se queda con todo el efecto (1.4 ventiladores por grado, muy cerca del 1.5 simulado). Así se desenmascara una variable de confusión.

4.11.4 Supuestos y diagnóstico básico

La regresión lineal da resultados confiables si se cumplen algunos **supuestos**:

1. **Linealidad**: la relación es aproximadamente una recta.
2. **Independencia**: cada observación es independiente (cuidado con series de tiempo, donde un mes depende del anterior).
3. **Varianza constante**: los errores tienen más o menos la misma dispersión en todo el rango.
4. **Normalidad de los residuos**: importante sobre todo con muestras pequeñas.

La herramienta básica es la gráfica de **residuos contra valores ajustados**: si el modelo es adecuado, los puntos forman una nube sin patrón alrededor de cero.

```
1 diagnostico <- tibble(
2   ajustado = fitted(modelo_multiple),      # ventas que predice el modelo
3   residuo  = residuals(modelo_multiple)   # real - predicho
4 )
5
6 p_residuos <- ggplot(diagnostico, aes(x = ajustado, y = residuo)) +
7   geom_hline(yintercept = 0, color = color_resalte, linewidth = 0.8) +
8   geom_point(color = color_principal, size = 2, alpha = 0.8) +
```

```

9  labs(title = "Residuos vs valores ajustados (modelo múltiple)",
10       subtitle = "Nube sin patrón alrededor de cero: supuestos razonables",
11       x = "Ventas ajustadas (miles)", y = "Residuo (miles)") +
12  tema_libro()
13
14  p_residuos

```

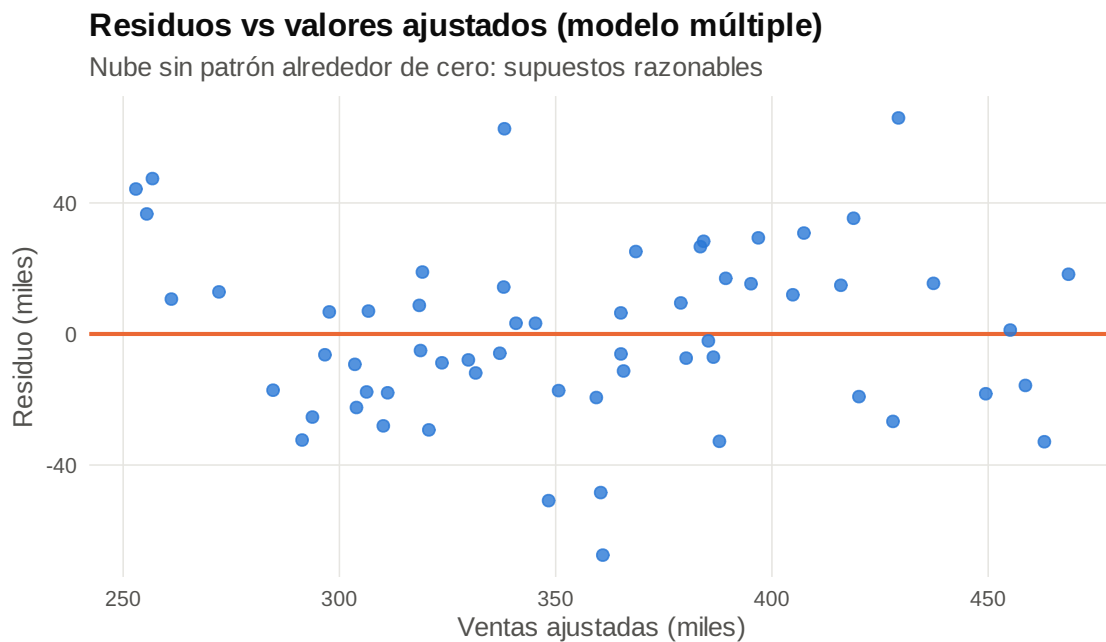


Figura 4.11. Diagnóstico de residuos. Un embudo (residuos que se abren) o una curva indicarían problemas con los supuestos.

La figura 4.11 no muestra patrones: ni curvas (que indicarían una relación no lineal) ni forma de embudo (que indicaría varianza no constante). En RStudio también puedes usar la instrucción `plot(modelo_multiple)`: dibuja las cuatro gráficas de diagnóstico estándar de R base.

Hazlo tú

Con el modelo múltiple, predice las ventas de una sucursal de 500 m² que invierta 25 mil pesos en publicidad, con su intervalo de predicción. Pista: `nuevas <- tibble(publicidad = 25, tamano_m2 = 500)`.

En la empresa: La regresión en BI

La regresión es la puerta de entrada al *machine learning* del Módulo 7. En BI se usa para: estimar la elasticidad del precio (¿cuánto bajan las unidades si subo el precio 1 %?), medir el retorno de la publicidad por canal, explicar diferencias de productividad entre tiendas y construir pronósticos sencillos. Recuerda que con datos observacionales (no experimentales) los coeficientes muestran asociaciones: para afirmar causalidad necesitas controlar las variables de

confusión relevantes o, mejor aún, un experimento.

4.12 Caso práctico integral: diagnóstico comercial

Es momento de juntar todo. La directora pidió un **diagnóstico comercial de 2023** con hallazgos y recomendaciones concretas. Seguiremos la lista de verificación de la tabla 4.1 con las cuatro tablas del negocio.

Paso 1. Integrar los datos

```

1 base <- transacciones %>%
2   left_join(clientes %>%
3     select(cliente_id, es_premium, segmento, fecha_registro),
4     by = "cliente_id") %>%
5   left_join(productos %>%
6     select(producto_id, categoria, precio_catalogo, costo),
7     by = "producto_id") %>%
8   left_join(tiendas %>%
9     select(tienda_id, nombre_tienda, tipo, tamano_m2, empleados),
10    by = "tienda_id") %>%
11   mutate(costo_total = cantidad * costo,           # costo de lo vendido
12          utilidad    = total_transaccion - costo_total) # utilidad bruta
13
14 dim(base)
15 sum(is.na(base$categoria)) + sum(is.na(base$nombre_tienda)) # sin pareja

```

```

[1] 1000  26
[1] 0

```

La base integrada tiene las mismas 1,000 filas (los `left_join()` no duplicaron nada) y todas las transacciones encontraron su producto y su tienda.

Paso 2. Calidad de datos

```

1 base %>%
2   summarise(
3     antes_registro = sum(fecha < fecha_registro), # compras "imposibles"
4     clientes_afect = n_distinct(cliente_id[fecha < fecha_registro]),
5     bajo_costo    = sum(precio_venta < costo),    # vendidas con pérdida
6     pct_bajo_costo = mean(precio_venta < costo)
7   )

```

```

8
9 # Productos cuyo costo supera su precio de catálogo
10 productos %>% filter(costo > precio_catálogo) %>% nrow()

```

```

# A tibble: 1 x 4
  antes_registro clientes_afect bajo_costo pct_bajo_costo
      <int>         <int>      <int>      <dbl>
1       114           34       252      0.252
[1] 10

```

Aparecen tres problemas que hay que reportar antes de cualquier conclusión:

- 114 transacciones (de 34 clientes) ocurrieron *antes* de la fecha de registro del cliente. O la fecha de registro se sobrescribe al actualizar datos, o las compras se asignan al cliente equivocado.
- El 25.2% de las transacciones se vendió por debajo del costo unitario.
- 10 de los 50 productos tienen un costo mayor que su precio de catálogo: aun vendiendo al precio de lista, pierden dinero.

Paso 3. Panorama general (univariado)

```

1 base %>%
2   summarise(
3     ventas      = sum(total_transaccion),
4     utilidad    = sum(utilidad),
5     margen      = sum(utilidad) / sum(total_transaccion),
6     clientes    = n_distinct(cliente_id),
7     ticket_promedio = mean(total_transaccion),
8     ticket_mediano = median(total_transaccion)
9   )

```

```

# A tibble: 1 x 6
  ventas utilidad margen clientes ticket_promedio ticket_mediano
    <dbl>    <dbl> <dbl>    <int>      <dbl>          <dbl>
1 959993. 444637. 0.463     198      960.          713.

```

Paso 4. Segmentos: productividad de las tiendas

Las ventas totales favorecen a las tiendas grandes. Una comparación más justa es la **productividad**: ventas por metro cuadrado y por empleado.

```

1 productividad <- base %>%
2   group_by(nombre_tienda, tipo, tamano_m2, empleados) %>%
3   summarise(ventas = sum(total_transaccion),
4             margen = sum(utilidad) / sum(total_transaccion),
5             .groups = "drop") %>%
6   mutate(ventas_m2 = ventas / tamano_m2,
7          ventas_empleado = ventas / empleados) %>%
8   arrange(desc(ventas_m2))
9
10 productividad %>%
11   select(nombre_tienda, tipo, ventas, margen, ventas_m2, ventas_empleado)

```

```

# A tibble: 10 x 6
  nombre_tienda tipo      ventas margen ventas_m2 ventas_empleado
  <chr>          <chr>    <dbl>  <dbl>    <dbl>      <dbl>
1 Sucursal Express Express  102791.  0.443    343.      8566.
2 Sucursal Outlet Outlet   114377.  0.542    327.      7625.
3 Sucursal Este Estándar   88420.  0.527    221.      4912.
4 Sucursal Norte Estándar   89024.  0.418    198.      4451.
5 Sucursal Centro Principal  88414.  0.523    177.      3537.
6 Sucursal Sur Estándar  105305.  0.432    176.      3510.
7 Sucursal Plaza B Estándar   80236.  0.369    167.      3821.
8 Sucursal Oeste Estándar   84672.  0.411    154.      3849.
9 Sucursal Plaza A Principal 105216.  0.457    150.      3006.
10 Sucursal Premium Premium  101538.  0.484    127.      2538.

```

```

1 # ¿El tamaño se relaciona con las ventas totales?
2 cor(productividad$tamano_m2, productividad$ventas, method = "spearman")

```

```
[1] -0.03030303
```

Las ventas de las 10 tiendas son muy parecidas (entre 80 y 114 mil pesos) y no se relacionan con el tamaño (correlación de Spearman de -0.03). Por eso las tiendas pequeñas son mucho más productivas: la sucursal Express (300 m²) vende \$343 por metro cuadrado y la Premium (800 m²) solo \$127.

Paso 5. Hipótesis: ¿qué diferencias son reales?

Juntamos en una tabla los p-valores de las pruebas que hicimos en el módulo y de una más: el ticket de las tiendas.

```

1 pruebas <- tibble(
2   pregunta = c("Ticket: premium vs no premium",

```

```

3         "Ticket: entre categorías",
4         "Ticket: entre tiendas",
5         "Método de pago vs tienda"),
6     p_valor = c(
7       t.test(total_transaccion ~ es_premium, data = base)$p.value,
8       summary(aov(total_transaccion ~ categoria,
9         data = base))[[1]]$`Pr(>F)`[1],
10      summary(aov(total_transaccion ~ nombre_tienda,
11        data = base))[[1]]$`Pr(>F)`[1],
12      chisq.test(table(base$metodo_pago, base$tienda_id))$p.value
13    )
14  ) %>%
15  mutate(significativo = p_valor < 0.05)
16
17  pruebas

```

```

# A tibble: 4 x 3
  pregunta                                p_valor significativo
  <chr>                                <dbl> <lg1>
1 Ticket: premium vs no premium      0.601 FALSE
2 Ticket: entre categorías            0.166 FALSE
3 Ticket: entre tiendas               0.863 FALSE
4 Método de pago vs tienda           0.363 FALSE

```

`summary(aov(...))` devuelve una lista; `[[1]]` toma la tabla y `$`Pr(>F)`[1]` extrae el primer p-valor (las comillas invertidas son necesarias porque el nombre de la columna tiene símbolos). Ninguna de las cuatro diferencias es significativa.

Paso 6. Hallazgos y recomendaciones

En la empresa: Diagnóstico comercial 2023: cinco hallazgos

1. **Ventas en la meta, con un camino irregular.** Se vendieron \$959,993, prácticamente la meta anual (\$960,000), pero la meta mensual solo se cumplió en 7 de 12 meses y hubo caídas fuertes en abril y en el otoño (sección 4.5).
2. **Un margen erosionado por ventas bajo costo.** El margen bruto es de 46.3 %, pero el 25.2 % de las transacciones se vendió por debajo del costo y 10 productos tienen costo mayor al precio de catálogo.
3. **Las tiendas pequeñas son las más productivas.** Express y Outlet venden más de \$320 por m²; Premium y Plaza A, menos de \$150. Las ventas no crecen con el tamaño de la tienda.
4. **El ticket no distingue segmentos.** Premium y no premium, categorías, tiendas y métodos de pago tienen tickets estadísticamente iguales. El crecimiento vendrá de la *frecuencia* (el gasto anual se correlaciona 0.79 con el número de compras), no del ticket.

5. **Ingresos repartidos, pero con productos clave sin stock.** No hay dependencia de pocos productos (el 72 % del catálogo genera el 80 % del ingreso), pero 4 productos clase A, incluido el más vendido, tienen stock bajo.

En la empresa: Tres recomendaciones

1. **Revisar precios y costos** de los 10 productos con costo mayor al precio de catálogo y establecer un piso de precio en el punto de venta que impida vender bajo costo sin autorización.
2. **Replicar el formato compacto:** antes de abrir o remodelar tiendas grandes, estudiar qué hacen Express y Outlet (surtido, ubicación, horario) y evaluar un formato más pequeño.
3. **Programa de frecuencia en lugar de membresía premium:** como los premium no gastan más por ticket, rediseñar el beneficio para premiar la recompra (cupones de regreso) y medirlo con una prueba A/B de 4,000 clientes por grupo, como en la sección 4.10.3.

Además, corregir el proceso de registro de clientes (114 compras anteriores a la fecha de registro) para que el análisis de antigüedad sea confiable.

Consejo

Observa la estructura: cada hallazgo tiene un número que lo respalda, y cada recomendación es una acción concreta que se puede medir. Evita hallazgos vagos como “las ventas se comportan de forma variable”. Una buena prueba: si la directora pregunta “¿y eso cómo lo sabes?”, debes poder señalar la salida de R exacta.

4.13 Errores comunes

4.13.1 Confundir correlación con causa

Ya lo viste con los helados y los ventiladores (sección 4.7.4). Antes de afirmar “X causa Y” pregúntate: ¿hay una tercera variable que mueva a ambas?, ¿podría ser al revés (Y causa X)?, ¿se hizo un experimento? Si no, habla de “asociación”, no de causa.

4.13.2 El p-hacking: probar hasta que algo salga

Si haces muchas pruebas, alguna saldrá significativa por puro azar: con $\alpha = 0.05$, en promedio 1 de cada 20 pruebas sin ningún efecto real da $p < 0.05$. Para verlo, **simulamos** 100 variables de puro ruido (sin ninguna relación con el ticket): cada una reparte los 1,000 tickets al azar en dos grupos, A y B. Luego las probamos una por una con `t.test()`:

```
1 set.seed(123)
2 p_valores_ruido <- replicate(100, {
```

```

3 grupo_al_azar <- sample(c("A", "B"), size = length(ticket),
4                       replace = TRUE)           # variable sin sentido
5 t.test(ticket ~ grupo_al_azar)$p.value
6 })
7 sum(p_valores_ruido < 0.05)      # "descubrimientos" falsos
8 which(p_valores_ruido < 0.05)    # ¿cuáles variables?
9 round(min(p_valores_ruido), 4)   # el p-valor más pequeño

```

```

[1] 2
[1] 42 44
[1] 0.0369

```

Aunque ninguna variable tiene relación real con el ticket, dos de las 100 salieron “significativas”. Si solo hubieras reportado esas dos, habrías anunciado un descubrimiento falso. En este módulo probamos varias segmentaciones del ticket (premium, categoría, tienda, método de pago, segmento de edad, género...) y dos salieron significativas: el segmento ($p = 0.044$) y el género (en el “Hazlo tú” de la sección 4.10). Con tantas pruebas, conviene tratarlas como *hipótesis para confirmar* con datos nuevos, no como verdades.

▲ Cuidado

P-hacking es probar muchas variantes (variables, filtros, periodos) y reportar solo la que dio $p < 0.05$. Para evitarlo: define tus hipótesis antes de ver los datos, reporta todas las pruebas que hiciste y confirma los hallazgos sorprendentes con datos nuevos. Si haces muchas comparaciones, ajusta los p-valores con `p.adjust(p, method = "holm")`.

4.13.3 Promedios de promedios

Calcular el promedio de los tickets promedio de cada tienda **no** da el ticket promedio de la empresa, porque cada tienda tiene un número distinto de transacciones. El efecto es enorme cuando los grupos son muy distintos en tamaño:

```

1 canales <- tibble(
2   canal   = c("Tienda física", "Tienda en línea"),
3   tickets = c(9500, 500),
4   ticket_promedio = c(400, 1500)
5 )
6
7 mean(canales$ticket_promedio)      # promedio de promedios: MAL
8 weighted.mean(canales$ticket_promedio,
9               w = canales$tickets)  # promedio ponderado: BIEN

```

```

[1] 950
[1] 455

```


El ticket de la empresa es de \$455, no de \$950. El promedio simple le da el mismo peso al canal en línea (500 tickets) que a la tienda física (9,500). Usa `weighted.mean()` o, mejor aún, recalcula desde los datos originales: `sum(ventas) / sum(tickets)`.

4.13.4 Ignorar los valores faltantes

En R, casi cualquier cálculo con un `NA` devuelve `NA`. Es una protección: te obliga a decidir qué hacer con los faltantes.

```
1 ventas_con_faltantes <- c(1200, 950, NA, 1100, 870)
2 mean(ventas_con_faltantes)                # NA
3 mean(ventas_con_faltantes, na.rm = TRUE)   # ignora el NA
4 sum(is.na(ventas_con_faltantes))           # ¿cuántos faltan?
```

```
[1] NA
[1] 1030
[1] 1
```

▲ Cuidado

`na.rm = TRUE` resuelve el síntoma, no el problema. Antes de usarlo cuenta los faltantes y pregúntate *por qué* faltan. Si faltan las ventas de los días en que el sistema falló (días de mucho tráfico), ignorarlas subestima el promedio. En `cor()` el argumento equivalente es `use = "complete.obs"`.

4.13.5 Comparar grupos de tamaño muy distinto

En la exploración de ciudades, los clientes de León tienen el ticket promedio más alto. Pero, ¿con cuántos tickets?

```
1 tx_clientes %>%
2   group_by(ciudad) %>%
3   summarise(tickets = n(),
4             clientes_distintos = n_distinct(cliente_id),
5             media = mean(total_transaccion),
6             ee      = sd(total_transaccion) / sqrt(n()),
7             .groups = "drop") %>%
8   arrange(desc(media)) %>%
9   head(4)
```

```
# A tibble: 4 x 5
  ciudad      tickets clientes_distintos media    ee
  <chr>         <int>           <int> <dbl> <dbl>
```

1 León	98	18	1140.	90.3
2 Toluca	116	20	992.	73.4
3 Guadalajara	118	22	977.	66.1
4 Querétaro	90	17	967.	82.6

Los grupos pequeños tienen promedios más “nerviosos” (error estándar más grande): basta un par de clientes con compras grandes para subirlos o bajarlos. Por eso los “primeros lugares” de un ranking suelen ser grupos pequeños. Reporta siempre el tamaño del grupo (`n()`) y, si puedes, el error estándar o el intervalo de confianza.

4.13.6 La paradoja de Simpson

La **paradoja de Simpson** ocurre cuando una tendencia que aparece en cada grupo se invierte al juntar los grupos. Con datos **simulados** muy pequeños: dos vendedores atienden clientes nuevos (difíciles de convencer) y clientes recurrentes (fáciles).

```

1 cierres <- tibble(
2   vendedor = rep(c("Ana", "Beto"), each = 2),
3   tipo_cliente = rep(c("Nuevo", "Recurrente"), times = 2),
4   atendidos = c(80, 20, 20, 80),
5   ventas_cerradas = c(24, 18, 4, 64)
6 )
7
8 # Tasa de cierre por tipo de cliente: Ana gana en AMBOS
9 cierres %>%
10   mutate(tasa = ventas_cerradas / atendidos) %>%
11   select(vendedor, tipo_cliente, tasa) %>%
12   pivot_wider(names_from = vendedor, values_from = tasa)
13
14 # Tasa de cierre total: ¡Beto gana!
15 cierres %>%
16   group_by(vendedor) %>%
17   summarise(tasa_total = sum(ventas_cerradas) / sum(atendidos),
18             .groups = "drop")

```

```

# A tibble: 2 x 3
  tipo_cliente  Ana  Beto
  <chr>        <dbl> <dbl>
1 Nuevo         0.3   0.2
2 Recurrente    0.9   0.8
# A tibble: 2 x 2
  vendedor tasa_total
  <chr>        <dbl>
1 Ana         0.42
2 Beto        0.68

```

Ana tiene mejor tasa de cierre con clientes nuevos (30 % contra 20 %) y con recurrentes (90 % contra 80 %), pero en el total Beto gana (68 % contra 42 %). ¿Cómo? A Ana le tocaron sobre todo clientes nuevos, los difíciles, y a Beto los recurrentes. Si pagas bonos por la tasa total, premiarías al vendedor equivocado. La lección: cuando compares grupos, revisa si la *mezcla* (de clientes, productos o regiones) es la misma; si no lo es, compara dentro de cada segmento.

Tabla 4.8. Resumen de errores comunes y cómo evitarlos.

Error	Cómo evitarlo
Usar la media con datos asimétricos	Reporta media y mediana; usa la mediana para el caso típico.
Borrar atípicos sin investigar	Clasifícalos (error, caso legítimo, oportunidad) y documenta cualquier exclusión.
Confundir correlación con causa	Busca variables de confusión; usa experimentos (A/B) o regresión múltiple.
P-hacking	Define hipótesis antes; reporta todas las pruebas; ajusta con <code>p.adjust()</code> .
Significancia sin tamaño del efecto	Reporta la diferencia en pesos o puntos porcentuales y su intervalo.
“No significativo” = “no hay diferencia”	Di “no hay evidencia suficiente”; revisa el tamaño de la muestra.
Promedio de promedios	<code>weighted.mean()</code> o recalcular con los datos originales.
Ignorar <code>NA</code>	Cuenta faltantes con <code>is.na()</code> antes de usar <code>na.rm = TRUE</code> .
Rankings con grupos pequeños	Muestra <code>n()</code> y el error estándar.
Paradoja de Simpson	Compara dentro de segmentos homogéneos.
Extrapolar una regresión	Predice solo dentro del rango de los datos.

Y tres mensajes de error reales que verás al empezar:

```

1 t.test(total_transaccion ~ metodo_pago, data = transacciones)
2 # Error in t.test.formula(...) : grouping factor must have exactly 2 levels
3 # Solución: t.test() compara 2 grupos; con 3 o más usa aov().
4
5 cor(transacciones$metodo_pago, transacciones$total_transaccion)
6 # Error in cor(...) : 'x' must be numeric
7 # Solución: cor() solo acepta columnas numéricas; para una categórica
8 # y una numérica compara grupos (t.test, aov).
9
10 chisq.test(matrix(c(3, 5, 2, 8), nrow = 2))
11 # Warning message: Chi-squared approximation may be incorrect
12 # Solución: hay conteos esperados menores a 5; usa fisher.test().

```

4.14 Resumen del módulo

Resumen del módulo

- El **análisis exploratorio** sigue un proceso: estructura, calidad, univariado, bivariado, segmentos, atípicos e hipótesis. `skim()` y `summary()` son el primer vistazo.
- La **media** sirve para totales; la **mediana**, para describir el caso típico. Con datos asimétricos (como el ticket) reporta ambas, junto con la **desviación estándar**, el **IQR**, el **coeficiente de variación** y los **percentiles**.
- Los **atípicos** se detectan con la regla de $1.5 \times \text{IQR}$ o con $|z| > 3$, pero se investigan antes de decidir: pueden ser errores, casos legítimos o tus mejores clientes. Las reglas de negocio complementan a las estadísticas.
- Los **KPIs** comparan: contra el mes anterior (MoM), contra lo acumulado, contra una base 100 y contra la meta. El **promedio móvil** muestra la tendencia sin el ruido.
- El **análisis de Pareto/ABC** mide la concentración de los ingresos y orienta las políticas de inventario.
- La **correlación** (Pearson o Spearman) mide asociación, no causalidad; cuidado con las variables de confusión.
- Las distribuciones **normal**, **binomial** y **Poisson** responden preguntas de probabilidad con las funciones `p`, `q`, `d` y `r`.
- Los **intervalos de confianza** (con fórmula o con **bootstrap**) expresan la incertidumbre de una estimación.
- Las **pruebas de hipótesis** (t, proporciones, ANOVA, chi-cuadrada) responden “¿es real o es azar?”. “No significativo” significa “no hay evidencia suficiente”, y la significancia estadística no garantiza importancia práctica.
- La **regresión lineal** cuantifica relaciones, predice con intervalos y, con varias variables, controla factores de confusión. Revisa los residuos y no extrapoles.

Tabla 4.9. Funciones clave del Módulo 4.

Función	Para qué sirve
<code>skim()</code> , <code>summary()</code>	Resumen rápido de un data frame.
<code>mean()</code> , <code>median()</code> , <code>table()</code>	Media, mediana y frecuencias (para la moda).
<code>sd()</code> , <code>var()</code> , <code>IQR()</code> , <code>range()</code>	Medidas de dispersión.
<code>quantile(x, probs)</code>	Percentiles y cuartiles.
<code>lag()</code> , <code>cumsum()</code> , <code>first()</code>	Crecimiento, acumulados e índices base 100.
<code>cor()</code> , <code>cor.test()</code> , <code>corrplot()</code>	Correlación, su prueba y su gráfica.
<code>pnorm()</code> , <code>qnorm()</code>	Probabilidades y cuantiles de la normal.
<code>dbinom()</code> , <code>pbinom()</code>	Probabilidades de la binomial.
<code>dpois()</code> , <code>ppois()</code>	Probabilidades de Poisson.
<code>replicate()</code> , <code>sample(replace = TRUE)</code>	Bootstrap y simulaciones.
<code>t.test()</code>	Prueba t e intervalo de confianza de medias.
<code>prop.test()</code> , <code>power.prop.test()</code>	Pruebas A/B de proporciones y tamaño de muestra.
<code>aov()</code> , <code>TukeyHSD()</code>	ANOVA y comparaciones por pares.
<code>chisq.test()</code>	Independencia entre dos variables categóricas.
<code>lm()</code> , <code>coef()</code> , <code>confint()</code>	Regresión lineal y sus coeficientes.
<code>predict(interval = ...)</code>	Predicciones con intervalo de confianza o de predicción.
<code>fitted()</code> , <code>residuals()</code>	Diagnóstico de la regresión.
<code>weighted.mean()</code>	Promedio ponderado (evita promedios de promedios).

4.15 Ejercicios

Todos los ejercicios usan los archivos de la carpeta `datasets/`. Empieza cada uno cargando **tidyverse** y los datos que necesites.

Ejercicio 4.1: Descriptiva del precio de venta

Nivel: **Básico**

Con `transacciones.csv`, calcula para `precio_venta`: media, mediana, desviación estándar, coeficiente de variación, rango intercuartílico y los percentiles 10 y 90. ¿La media y la mediana son parecidas? ¿Qué te dice eso sobre la forma de la distribución? Compara su coeficiente de variación con el del ticket (0.79).

 Solución en la página 644

Ejercicio 4.2: ¿Qué día se vende más?

Nivel: **Básico**

Con `transacciones.csv`, usa la función `moda()` del módulo para encontrar el día de la semana con más transacciones. Luego construye una tabla con el número de transacciones, el porcentaje del total y las ventas de cada día, ordenada de mayor a menor número de transacciones. ¿El día con más transacciones es también el de más ventas?

 Solución en la página 645

Ejercicio 4.3: Salarios: ¿media o mediana?Nivel: **Básico**

Con `empleados.csv`, calcula por `departamento`: número de empleados, media y mediana de `salario_mensual`, y la diferencia entre ambas. ¿En qué departamento la media y la mediana son más distintas? ¿Qué medida usarías para anunciar “el salario típico” de ese departamento y por qué?

 Solución en la página 646**Ejercicio 4.4: Atípicos en las ventas diarias**Nivel: **Intermedio**

Con `ventas_retail.csv`, aplica la regla de $1.5 \times \text{IQR}$ y la regla $|z| > 3$ a la columna `total`. ¿Cuántas ventas marca cada regla? Muestra las ventas atípicas con su fecha, cantidad, precio unitario y descuento. ¿Son errores o ventas legítimas? Calcula la asimetría de `total`.

 Solución en la página 647**Ejercicio 4.5: Tablero trimestral contra meta**Nivel: **Intermedio**

Con `ventas_retail.csv`, construye una tabla por `trimestre` con: ventas, crecimiento contra el trimestre anterior, acumulado, índice base 100 ($Q1 = 100$) y variación contra una meta trimestral de \$130,000. ¿Cuántos trimestres cumplieron la meta? ¿Se cumplió la meta anual?

 Solución en la página 648**Ejercicio 4.6: ¿Los premium compran más seguido?**Nivel: **Intermedio**

En la sección 4.10 vimos que los clientes premium no tienen un ticket distinto. Ahora compara la *frecuencia*: calcula el número de compras de cada cliente en 2023, únelo con `clientes.csv` y aplica una prueba t para comparar el número promedio de compras de premium y no premium. Plantea H_0 y H_1 , reporta el p-valor y la diferencia, y escribe la conclusión en una frase para la directora.

 Solución en la página 649**Ejercicio 4.7: Pareto de clientes**Nivel: **Intermedio**

Haz un análisis ABC de los *clientes* por gasto total en 2023 (A hasta 80% acumulado, B hasta 95%, C el resto). ¿Cuántos clientes hay en cada clase y qué porcentaje de la clientela representan? ¿Qué porcentaje de los ingresos aporta el 20% de clientes que más gasta? Compara con el resultado de los productos.

 Solución en la página 650

Ejercicio 4.8: Tres preguntas de probabilidadNivel: **Avanzado**

Responde con las funciones de R adecuadas:

1. Las ventas semanales siguen una Normal con media \$18,445 y desviación estándar \$4,591. ¿Qué probabilidad hay de que una semana venda entre \$15,000 y \$20,000? ¿Qué venta semanal solo se supera el 5 % de las veces?
2. El 3 % de las ventas se devuelven. En un día con 150 ventas, ¿cuál es la probabilidad de tener más de 8 devoluciones? ¿Y de no tener ninguna?
3. Al centro de atención llegan en promedio 4 llamadas cada 10 minutos. ¿Qué probabilidad hay de recibir 8 o más llamadas en 10 minutos?

 Solución en la página 651**Ejercicio 4.9: Prueba A/B y ANOVA de salarios**Nivel: **Avanzado**

1. Se probaron dos diseños de la página de pago: el diseño A tuvo 180 compras de 3,000 visitas y el B, 225 compras de 3,000 visitas. ¿Hay evidencia de que las tasas de conversión son distintas? Reporta las tasas, el p-valor y el intervalo de la diferencia.
2. Con `empleados.csv`, ¿el salario mensual promedio es distinto entre departamentos? Usa ANOVA e interpreta el resultado. ¿Tiene sentido aplicar `TukeyHSD()` en este caso?

 Solución en la página 651**Ejercicio 4.10: Reto: elasticidad del precio**Nivel: **Reto**

Simula 52 semanas de venta de un producto con `set.seed(123)`: el precio varía entre \$80 y \$120 (`runif()`) y las unidades vendidas siguen la relación $\text{unidades} = 500 - 3 \times \text{precio} + \text{ruido}$ normal con desviación estándar 15. Luego:

1. Ajusta una regresión de unidades sobre precio e interpreta la pendiente, su intervalo de confianza y el R^2 . ¿Recupera el valor real de -3 ?
2. Predice las unidades de una semana con precio de \$100 (intervalo de predicción).
3. Predice con un precio de \$200. ¿Qué pasa? Explica por qué no debes confiar en esa predicción.
4. Revisa los residuos contra los valores ajustados (basta con un resumen numérico o una gráfica).

 Solución en la página 652

Bases de datos, SQL y ETL desde R

Al terminar este módulo podrás...

- Explicar por qué las empresas guardan sus datos en bases de datos y no en archivos de Excel o CSV, y distinguir una base transaccional (OLTP) de una analítica (OLAP o *data warehouse*).
- Usar el vocabulario básico: tabla, fila, columna, tipo de dato, llave primaria, llave foránea e índice.
- Conectarte desde R a una base de datos con **DBI** y **RSQLite**, cargar tablas, consultarlas y cerrar la conexión de forma segura.
- Escribir consultas SQL desde cero para responder preguntas de negocio: filtros, agregaciones, *joins*, subconsultas, CTE y funciones de ventana.
- Proteger tus consultas contra la inyección SQL con consultas parametrizadas.
- Trabajar con tablas de la base como si fueran data frames gracias a **dbplyr**, y ver el SQL que genera.
- Diseñar y construir un *data warehouse* en estrella con tabla de hechos, dimensiones, vistas e índices.
- Programar un proceso ETL reutilizable con transacciones, carga incremental sin duplicados y bitácora de ejecución.
- Conectarte de forma segura a las bases de datos reales de tu empresa (SQL Server, PostgreSQL, MySQL) sin exponer contraseñas.

Hasta ahora has trabajado con archivos CSV que viven en tu computadora. En una empresa real las cosas son distintas. Las ventas no están en un CSV: se registran, una por una, en el sistema del punto de venta de cada sucursal, y ese sistema las guarda en una **base de datos**. El catálogo de productos lo mantiene el área de compras en otro sistema; los clientes, el CRM; los empleados, Recursos Humanos. Cuando tu jefa te pide “las ventas de ayer por sucursal”, nadie te va a mandar un archivo: te van a dar un usuario y una contraseña para que consultes tú mismo la base de datos.

Por eso este módulo es clave en tu camino como analista de BI. Aprenderás **SQL** (*Structured Query Language*), el idioma universal para pedirle datos a una base; lo usan prácticamente todas las empresas del mundo y aparece en casi cualquier vacante de analista de datos. Lo mejor: lo vas a aprender *desde R*, sin salir de RStudio, de modo que puedes combinar la potencia de SQL para extraer y agregar datos con todo lo que ya sabes de **dplyr** y **ggplot2**.

Para que puedas practicar sin instalar ningún servidor usaremos **SQLite**, un motor de base de

datos completo que guarda toda la base en un solo archivo. Construiremos, paso a paso, la base `datos/bi_retail.sqlite` con los datos del curso; primero como una copia de los CSV, después como un *data warehouse* en estrella cargado por un proceso ETL escrito en R. Todo lo que aprendas aquí (el SQL, las funciones de **DBI**, el diseño del modelo) funciona igual con SQL Server, PostgreSQL o MySQL; al final verás cómo conectarte a ellos.

5.1 ¿Por qué bases de datos en BI?

5.1.1 Los límites de Excel y los archivos CSV

Excel y los CSV son excelentes para intercambiar datos pequeños, pero se quedan cortos en cuanto una empresa crece:

- **Volumen.** Una hoja de Excel admite 1 048 576 filas. Una cadena de 10 tiendas con 2 000 tickets diarios por tienda llena ese límite en menos de dos meses. Un CSV no tiene límite formal, pero para leerlo tienes que cargarlo completo en la memoria de tu computadora.
- **Una sola versión de la verdad.** Cuando cada área guarda su propio archivo (`ventas_final.xlsx`, `ventas_final_v2_bueno.xlsx...`), en la junta cada quien llega con una cifra distinta. Una base de datos central es la **fuentes única de la verdad**: todos consultan los mismos datos.
- **Concurrencia.** Cien cajeros pueden registrar ventas al mismo tiempo en una base de datos sin pisarse; en un archivo de Excel compartido, el segundo que lo abre lo ve “de solo lectura”.
- **Integridad.** Una base de datos puede impedir que se registre una venta de un producto que no existe o con cantidad negativa. Un CSV acepta cualquier cosa.
- **Seguridad y auditoría.** En una base puedes dar permiso de solo lectura a los analistas, ocultar columnas sensibles (sueldos, teléfonos) y saber quién consultó qué.

5.1.2 Bases transaccionales (OLTP) y analíticas (OLAP)

No todas las bases de datos tienen el mismo propósito. En BI conviene distinguir dos grandes familias:

Concepto clave: OLTP frente a OLAP

Una base **OLTP** (*Online Transaction Processing*) es la que usan los sistemas del día a día: el punto de venta, el ERP, la tienda en línea. Está optimizada para registrar muchísimas operaciones pequeñas y rápidas (“inserta este ticket”, “actualiza el stock de este producto”). Una base **OLAP** (*Online Analytical Processing*), cuya forma más común es el **data warehouse** (almacén de datos), está pensada para *analizar*: pocas consultas, pero cada una recorre millones de filas para sumar, agrupar y comparar (“ventas por mes, categoría y zona de los últimos tres años”). Se alimenta de las bases OLTP mediante procesos **ETL** (*Extract, Transform, Load*: extraer, transformar, cargar).

Tabla 5.1. Diferencias entre una base transaccional y un data warehouse.

	OLTP (transaccional)	OLAP (data warehouse)
Propósito	Operar el negocio	Analizar el negocio
Usuarios	Cajeros, sistemas, clientes	Analistas, directivos, tableros
Operación típica	Insertar o actualizar una fila	Leer y agregar millones de filas
Diseño	Muy normalizado (muchas tablas)	En estrella (hechos y dimensiones)
Historia	Estado actual	Años de historia
Ejemplo	Sistema del punto de venta	Base que alimenta el tablero de ventas

En la empresa: ¿Dónde encaja el analista de BI?

Casi nunca vas a escribir directamente en la base OLTP de la empresa (y no deberías: una consulta pesada puede volver lento el punto de venta). Tu trabajo habitual es *leer* de las bases con permisos de solo lectura, construir o alimentar el *data warehouse* y, a partir de él, producir reportes, tableros y modelos. R es una herramienta excelente para las tres tareas.

5.1.3 Motores de bases de datos que encontrarás en las empresas

Un **motor de base de datos** (o **SGBD**, sistema gestor de bases de datos) es el programa que guarda los datos y responde las consultas. Todos hablan SQL, con pequeñas diferencias de “dialecto”. La tabla 5.2 resume los más comunes y el paquete de R con el que te conectas a cada uno.

Tabla 5.2. Motores de bases de datos frecuentes y cómo conectarte desde R.

Motor	Dónde lo verás	Paquete de R
SQL Server	Empresas con ecosistema Microsoft, ERPs	odbc
PostgreSQL	Startups, aplicaciones web, gobierno	RPostgres
MySQL / MariaDB	Tiendas en línea, sistemas web	RMariaDB
Oracle	Bancos, telecomunicaciones, grandes corporativos	odbc
SQLite	Aplicaciones móviles, prototipos, este curso	RSQLite
BigQuery, Snowflake, Redshift	Data warehouses en la nube	bigquery, odbc

Consejo

Lo que aprendas con SQLite te sirve para cualquier motor: las funciones de **DBI** son idénticas y el 90 % del SQL también. Las diferencias suelen estar en funciones de fecha (**strftime()** en SQLite, **DATEPART()** en SQL Server, **date_trunc()** en PostgreSQL) y en cómo limitar filas (**LIMIT** frente a **TOP**).

5.2 Conceptos básicos de una base de datos relacional

Una **base de datos relacional** organiza la información en **tablas** relacionadas entre sí. Si ya dominas los data frames, vas con ventaja: una tabla se parece mucho a un data frame.

Concepto clave: Vocabulario esencial

- **Tabla:** conjunto de datos sobre un mismo tipo de cosa (clientes, productos, ventas). Equivale a un data frame.
- **Fila** (registro): un elemento concreto, por ejemplo un cliente.
- **Columna** (campo): un atributo, por ejemplo la ciudad del cliente. Cada columna tiene un **tipo de dato**: entero (**INTEGER**), decimal (**REAL**), texto (**TEXT**), fecha, etc.
- **Llave primaria** (*primary key*, PK): columna cuyo valor identifica de forma única cada fila; no se repite ni puede estar vacía. Ejemplo: **cliente_id**.
- **Llave foránea** (*foreign key*, FK): columna de una tabla que apunta a la llave primaria de otra. En **transacciones**, la columna **cliente_id** es una llave foránea hacia **clientes**. Así se *relacionan* las tablas.
- **Índice:** estructura auxiliar (como el índice alfabético de un libro) que permite encontrar filas rápidamente sin recorrer toda la tabla.

La figura 5.1 muestra el modelo relacional de los datos del curso. La tabla central es **transacciones**: cada fila es una venta y, mediante sus llaves foráneas, sabemos a qué cliente, producto, tienda y vendedor corresponde. Observa que **vendedor_id** apunta a **empleado_id** en la tabla de empleados: la columna no tiene que llamarse igual en ambas tablas.

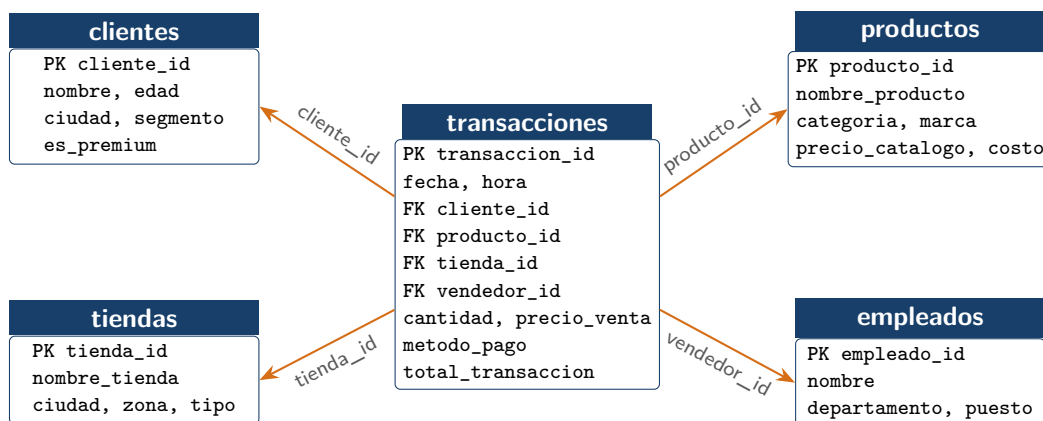


Figura 5.1. Modelo relacional de los datos del curso. Las flechas van de la llave foránea (FK) a la llave primaria (PK) de la tabla relacionada.

▲ Cuidado

En los datos del curso las llaves foráneas existen “de palabra”: los CSV no obligan a nada. Una base de datos bien diseñada *declara* esas relaciones para que el motor rechace, por ejemplo, una venta de un producto inexistente. Lo harás en la sección 5.7, cuando construyas el *data warehouse*.

Hazlo tú

Abre `ventas_retail.csv` y responde en papel: ¿cuál sería su llave primaria? (Pista: no tiene una columna `_id` propia, pero hay una venta por día.) ¿Qué columnas son llaves foráneas y hacia qué tablas apuntan?

5.3 Conectarte a una base de datos con DBI y RSQLite

En R, todas las conexiones a bases de datos siguen el mismo esquema de dos capas:

- **DBI** (*DataBase Interface*) define las funciones que tú usas: `dbConnect()`, `dbGetQuery()`, `dbWriteTable()`, etc. Son las mismas para cualquier motor.
- Un **driver** (controlador) traduce esas funciones al protocolo de un motor concreto: **RSQLite** para SQLite, **RPostgres** para PostgreSQL, **odbc** para SQL Server, etc.

Si mañana tu empresa cambia de SQLite a PostgreSQL, solo cambia la línea de `dbConnect()`; el resto de tu código sigue igual. Los paquetes que usaremos se instalan como siempre:

```
1 install.packages(c("DBI", "RSQLite", "dbplyr"))
```

5.3.1 Preparar la carpeta y crear la base

Empezamos cargando los paquetes y preparando la ruta del archivo de la base. Un detalle importante de reproducibilidad: SQLite *guarda* lo que escribes en el archivo, y ese archivo sigue ahí la próxima vez que ejecutes el script. Si lo corres dos veces, la segunda vez las tablas ya existirían y obtendrías errores del tipo “la tabla ya existe” o, peor, datos duplicados. Por eso, al inicio del módulo borramos el archivo si existe y empezamos siempre desde cero.

```
1 library(tidyverse)
2 library(DBI)           # funciones comunes para cualquier base de datos
3 library(RSQLite)       # driver (controlador) de SQLite
4 library(dbplyr)        # traduce verbos de dplyr a SQL
5 library(lubridate)     # fechas
```

Al cargar **dbplyr** verás un aviso de que “enmascara” las funciones `ident()` y `sql()` de **dplyr**; es normal y no afecta nada de lo que haremos. Ahora preparamos la carpeta y la ruta del archivo:

```
1 # Carpeta donde vivirá la base de datos
2 dir.create("datos", showWarnings = FALSE)
3 ruta_bd <- "datos/bi_retail.sqlite"
4
5 # Si la base ya existe (de una ejecución anterior), la borramos para
6 # que el script sea reproducible y siempre empiece desde cero.
```

```

7 # unlink() no marca error si el archivo no existe.
8 unlink(ruta_bd)
9 file.exists(ruta_bd)

```

```
[1] FALSE
```

Ahora abrimos la conexión. Con SQLite, si el archivo no existe, `dbConnect()` lo crea vacío:

```

1 # Abrir (y crear) la base de datos en el archivo
2 con <- dbConnect(RSQLite::SQLite(), ruta_bd)
3
4 # ¿Qué tablas tiene? Ninguna todavía
5 dbListTables(con)
6 file.exists(ruta_bd)

```

```

character(0)
[1] TRUE

```

El objeto `con` es la **conexión**: el “cable” entre tu sesión de R y la base. Todas las funciones de **DBI** reciben la conexión como primer argumento.



Consejo

SQLite también puede crear una base *temporal en memoria* con `dbConnect(RSQLite::SQLite(), ":memory:")`. Desaparece al desconectarte, así que es perfecta para pruebas rápidas y ejercicios; las soluciones de este módulo la usan mucho.

5.3.2 Cargar los CSV del curso a la base con `dbWriteTable()`

`dbWriteTable(con, "nombre", datos)` crea una tabla en la base a partir de un data frame. Vamos a cargar los seis CSV del curso. Antes de escribirlos haremos una pequeña transformación: convertir las columnas de fecha y de hora a texto con formato ISO ("2023-01-13", "13:15:00"). SQLite no tiene un tipo de dato de fecha propiamente dicho y, si le mandas un **Date** de R tal cual, lo guarda como un número. En la sección 5.4.6 verás el problema con detalle; por ahora quédate con la regla: *en SQLite, las fechas se guardan como texto 'AAAA-MM-DD'*.

```

1 # Lee un CSV del curso y deja fechas y horas como texto ISO
2 leer_para_bd <- function(archivo) {
3   read_csv(file.path("datasets", archivo), show_col_types = FALSE) %>%
4     mutate(
5       across(where(is.Date), ~ format(., "%Y-%m-%d")), # fechas
6       across(where(hms::is_hms), as.character)         # horas
7     )

```

```

8 }
9
10 # Nombre de la tabla en la base = nombre del archivo sin ".csv"
11 archivos <- c("ventas_retail", "clientes", "productos",
12              "empleados", "transacciones", "tiendas")
13
14 for (tabla in archivos) {
15   datos <- leer_para_bd(paste0(tabla, ".csv"))
16   # overwrite = TRUE: si la tabla existe, la reemplaza
17   dbWriteTable(con, tabla, datos, overwrite = TRUE)
18   cat(sprintf("%-14s %5d filas cargadas\n", tabla, nrow(datos)))
19 }
20
21 dbListTables(con)

```

```

ventas_retail    365 filas cargadas
clientes         200 filas cargadas
productos        50 filas cargadas
empleados        100 filas cargadas
transacciones    1000 filas cargadas
tiendas          10 filas cargadas
[1] "clientes"      "empleados"      "productos"      "tiendas"
[5] "transacciones" "ventas_retail"

```

En unas cuantas líneas tienes una base de datos con seis tablas. `dbListTables()` las devuelve en orden alfabético. Si abres la carpeta `datos/` verás el archivo `bi_retail.sqlite`: esa es toda la base; puedes copiarlo, respaldarlo o mandarlo por correo.

5.3.3 Explorar la base: columnas, tipos y contenido

`dbListFields()` te dice qué columnas tiene una tabla, y la instrucción especial de SQLite `PRAGMA table_info()` te muestra además el tipo de dato con el que se guardó cada una:

```

1 # Columnas de la tabla de tiendas
2 dbListFields(con, "tiendas")
3
4 # Tipo de dato de cada columna (instrucción propia de SQLite)
5 dbGetQuery(con, "PRAGMA table_info(tiendas)") %>%
6   select(name, type)

```

```

[1] "tienda_id"      "nombre_tienda"  "ciudad"
[4] "zona"           "tamano_m2"      "empleados"
[7] "fecha_apertura" "tipo"
      name type
1   tienda_id REAL

```

```

2  nombre_tienda TEXT
3      ciudad TEXT
4      zona TEXT
5      tamano_m2 REAL
6      empleados REAL
7 fecha_apertura TEXT
8      tipo TEXT

```

Observa la traducción de tipos: los números de R (**double**) se guardaron como **REAL**, el texto como **TEXT** y la fecha de apertura, gracias a nuestra conversión, también como **TEXT**. Los valores lógicos (**TRUE/FALSE**, como **es_premium** en clientes) se guardan como enteros **1/0**, porque SQLite no tiene tipo lógico.

Para traer una tabla *completa* a R usa `dbReadTable()`: `tiendas_bd <- dbReadTable(con, "tiendas")` devuelve un data frame normal con las 10 tiendas, listo para **dplyr** o **ggplot2**.

▲ Cuidado

`dbReadTable()` trae *todas* las filas y columnas. Con la tabla de tiendas (10 filas) no pasa nada, pero con la tabla de ventas de una empresa real (cientos de millones de filas) puedes congelar tu computadora. Lo profesional es pedirle a la base *solo lo que necesitas* con una consulta SQL, que es justo lo que viene.

5.3.4 Consultar y modificar: `dbGetQuery()` y `dbExecute()`

Las dos funciones que más vas a usar son:

- `dbGetQuery(con, sql)`: ejecuta una consulta que *devuelve datos* (**SELECT**) y te regresa un data frame.
- `dbExecute(con, sql)`: ejecuta una instrucción que *modifica* la base (**CREATE**, **INSERT**, **UPDATE**, **DELETE**) y te regresa el número de filas afectadas.

```

1  # Una consulta: ¿cuántas transacciones hay?
2  dbGetQuery(con, "SELECT COUNT(*) AS num_transacciones
3                  FROM transacciones")
4
5  # Una modificación: crear una tabla de metas mensuales y llenarla
6  dbExecute(con, "CREATE TABLE metas (mes TEXT, meta REAL)")
7  dbExecute(con, "INSERT INTO metas VALUES ('2023-01', 80000),
8                  ('2023-02', 75000)")
9  dbGetQuery(con, "SELECT * FROM metas")
10
11 # Borrar la tabla de prueba
12 dbExecute(con, "DROP TABLE metas")

```

```

      num_transacciones
1              1000
[1] 0
[1] 2
      mes  meta
1 2023-01 80000
2 2023-02 75000
[1] 0

```

El texto SQL puede ocupar varias líneas dentro de las comillas; a la base no le importan los saltos de línea. **CREATE TABLE** devolvió **0** (no afectó filas), el **INSERT** devolvió **2** (insertó dos filas) y el **DROP TABLE** volvió a devolver **0**.

5.3.5 Cerrar la conexión: `dbDisconnect()` y `on.exit()`

Cuando terminas, cierra la conexión con `dbDisconnect(con)`. Con SQLite no cerrarla es un descuido menor, pero en un servidor real cada conexión abierta consume recursos y muchas empresas limitan cuántas puede tener abierta cada usuario. Si escribes una *función* que abre su propia conexión, la buena práctica es usar `on.exit()`: registra una acción que se ejecutará al salir de la función, *incluso si ocurre un error a la mitad*.

```

1 # Función que abre su propia conexión y SIEMPRE la cierra
2 contar_filas <- function(ruta, tabla) {
3   con_local <- dbConnect(RSQLite::SQLite(), ruta)
4   on.exit(dbDisconnect(con_local)) # se ejecuta al salir, pase lo que pase
5   sql <- paste("SELECT COUNT(*) AS n FROM", tabla)
6   dbGetQuery(con_local, sql)$n
7 }
8
9 contar_filas(ruta_bd, "clientes")
10 contar_filas(ruta_bd, "productos")

```

```

[1] 200
[1] 50

```

Aquí usamos `paste()` para armar el SQL porque el nombre de la tabla lo escribes tú, en tu propio código. En la sección 5.5 verás por qué *nunca* debes hacer lo mismo con valores que escribe un usuario. En el resto del módulo seguiremos usando la conexión global `con`, que cerraremos al final.

Hazlo tú

Usa `dbListFields()` para ver las columnas de la tabla `empleados` y `contar_filas()` para confirmar que tiene 100 filas.

5.4 SQL desde cero con preguntas de negocio

SQL es un lenguaje *declarativo*: le dices a la base *qué* quieres, no *cómo* conseguirlo; el motor decide la forma más eficiente de obtenerlo. Una consulta tiene siempre la misma estructura, con las cláusulas en este orden (las que están entre corchetes son opcionales):

```
SELECT  columnas o cálculos      -- ¿qué quiero ver?
FROM    tabla                   -- ¿de dónde?
[JOIN   otra_tabla ON condición] -- ¿con qué la combino?
[WHERE  condición sobre filas]   -- ¿qué filas me interesan?
[GROUP BY columnas]             -- ¿por qué agrupo?
[HAVING condición sobre grupos]  -- ¿qué grupos me interesan?
[ORDER BY columnas]             -- ¿en qué orden?
[LIMIT  n]                      -- ¿cuántas filas como máximo?
```

Algunas convenciones: las palabras clave de SQL se escriben en mayúsculas por costumbre (no es obligatorio), los comentarios empiezan con `--`, el texto va entre *comillas simples* (`'CDMX'`) y cada consulta puede terminar con punto y coma. En R, toda la consulta va dentro de una cadena de texto que le pasas a `dbGetQuery()`.

Concepto clave: SQL y dplyr piensan igual

Si dominas **dplyr** (Módulo 2), ya conoces la lógica de SQL: **SELECT** es `select()/mutate()`, **WHERE** es `filter()`, **GROUP BY** + funciones de agregación es `group_by() + summarise()`, **ORDER BY** es `arrange()` y **JOIN** es `left_join()` y compañía. De hecho, **dplyr** se diseñó inspirándose en SQL. Al final del módulo tienes una tabla de equivalencias.

5.4.1 SELECT y FROM: elegir columnas

Ejemplo práctico: Directorio de sucursales

Recursos Humanos quiere un directorio con el nombre, la ciudad y la zona de cada sucursal.

```
1 dbGetQuery(con, "
2   SELECT nombre_tienda, ciudad, zona  -- columnas que quiero
3   FROM tiendas                       -- tabla de origen
4 ")
```

	nombre_tienda	ciudad	zona
1	Sucursal Centro	CDMX	Centro
2	Sucursal Norte	Guadalajara	Norte
3	Sucursal Sur	Monterrey	Norte
4	Sucursal Este	CDMX	Este
5	Sucursal Oeste	Puebla	Centro

```

6  Sucursal Plaza A          CDMX  Oeste
7  Sucursal Plaza B Guadalajara Norte
8  Sucursal Outlet          Tijuana Norte
9  Sucursal Premium          CDMX    Sur
10 Sucursal Express          Querétaro Centro

```

SELECT * trae todas las columnas; es cómodo para explorar (por ejemplo, **SELECT * FROM transacciones LIMIT 5** para ver cinco filas), pero en reportes conviene listar solo las columnas que necesitas.

5.4.2 WHERE: filtrar filas

WHERE se queda solo con las filas que cumplen una condición. Los operadores de comparación son **=** (¡un solo signo igual, no dos como en R!), **<>** o **!=** (distinto), **<**, **>**, **<=** y **>=**. Las condiciones se combinan con **AND**, **OR** y **NOT**.

Ejemplo práctico: Clientes premium del norte

Marketing prepara una campaña para los clientes premium de Monterrey y Guadalajara. ¿Cuántos son y quiénes?

```

1  # IN (...) equivale a varias condiciones unidas con OR
2  dbGetQuery(con, "
3    SELECT COUNT(*) AS clientes
4    FROM clientes
5    WHERE es_premium = 1
6          AND ciudad IN ('Monterrey', 'Guadalajara')
7  ")
8
9  dbGetQuery(con, "
10   SELECT nombre, ciudad, edad
11   FROM clientes
12   WHERE es_premium = 1
13         AND ciudad IN ('Monterrey', 'Guadalajara')
14   ORDER BY ciudad, nombre
15   LIMIT 3
16  ")

```

```

clientes
1      16
      nombre      ciudad edad
1      Carmen Reyes Guadalajara 33
2 Fernando Hernández Guadalajara 26
3      Marta González Guadalajara 38

```

Recuerda que los lógicos se guardaron como 1/0, por eso filtramos con `es_premium = 1`. La primera consulta cuenta, la segunda muestra los primeros tres ordenados por ciudad y nombre.

Ejemplo práctico: Ventas navideñas y tickets grandes

Queremos ver las transacciones entre el 22 y el 24 de diciembre y saber cuántos tickets superan los \$2 800.

```

1 # BETWEEN incluye ambos extremos; las fechas son texto ISO
2 dbGetQuery(con, "
3   SELECT transaccion_id, fecha, producto_id, total_transaccion
4   FROM transacciones
5   WHERE fecha BETWEEN '2023-12-22' AND '2023-12-24'
6   ORDER BY fecha
7 ")
8
9 dbGetQuery(con, "
10  SELECT COUNT(*) AS tickets_grandes
11  FROM transacciones
12  WHERE total_transaccion > 2800
13 ")

```

	transaccion_id	fecha	producto_id	total_transaccion
1	693	2023-12-22	17	1130.45
2	828	2023-12-22	32	1300.50
3	452	2023-12-23	31	480.65
4	558	2023-12-23	29	334.40
5	926	2023-12-23	27	2709.85
6	518	2023-12-24	3	356.36
7	796	2023-12-24	34	584.71
8	941	2023-12-24	17	1717.96

	tickets_grandes
1	26

Comparar fechas con `BETWEEN` funciona porque el texto `'AAAA-MM-DD'` se ordena alfabéticamente igual que cronológicamente (`'2023-12-22'` va antes que `'2023-12-24'`). Esa es otra razón para guardar las fechas en formato ISO.

`LIKE` busca patrones en texto: `%` representa “cualquier cantidad de caracteres” y `_` “exactamente un carácter”.

```

1 # Clientes cuyo apellido es García y cuyo nombre empieza con "Laura"
2 dbGetQuery(con, "
3   SELECT
4     SUM(nombre LIKE '%García') AS apellido_garcia,
5     SUM(nombre LIKE 'Laura%') AS nombre_laura
6   FROM clientes

```

```
7  ")
```

```
apellido_garcia nombre_laura
1              10          9
```

Aquí usamos un truco útil: una comparación en SQLite vale **1** si es verdadera y **0** si es falsa, así que **SUM()** de una condición cuenta cuántas filas la cumplen (igual que **sum()** de un vector lógico en R).

▲ Cuidado

Cuando combines **AND** y **OR**, usa paréntesis. **WHERE ciudad = 'CDMX' OR ciudad = 'Puebla' AND edad > 60** significa “todos los de CDMX, más los de Puebla mayores de 60”, porque **AND** se evalúa antes que **OR**. Si querías “mayores de 60 de CDMX o Puebla”, escribe **WHERE (ciudad = 'CDMX' OR ciudad = 'Puebla') AND edad > 60**. Y para valores vacíos no uses **= NULL**: usa **IS NULL** o **IS NOT NULL**.

5.4.3 ORDER BY y LIMIT: los primeros de la lista

Ejemplo práctico: Los cinco tickets más grandes del año

La gerencia quiere revisar las cinco ventas de mayor importe para validar que no sean errores de captura.

```
1 dbGetQuery(con, "
2   SELECT transaccion_id, fecha, cantidad, precio_venta,
3         total_transaccion
4   FROM transacciones
5   ORDER BY total_transaccion DESC  -- DESC = de mayor a menor
6   LIMIT 5
7  ")
```

	transaccion_id	fecha	cantidad	precio_venta	total_transaccion
1	176	2023-04-03	5	598.47	2992.35
2	172	2023-06-14	5	598.09	2990.45
3	835	2023-08-19	5	596.74	2983.70
4	535	2023-06-25	5	595.24	2976.20
5	451	2023-05-02	5	594.71	2973.55

ORDER BY ordena de menor a mayor por omisión (**ASC**); **DESC** invierte el orden. Los cinco tickets más grandes son de cinco unidades con precio cercano al máximo de \$600: son plausibles, no parecen errores de captura.

5.4.4 Funciones de agregación, GROUP BY y HAVING

Las **funciones de agregación** resumen muchas filas en un solo valor: `COUNT()`, `SUM()`, `AVG()`, `MIN()` y `MAX()`. `COUNT(*)` cuenta filas y `COUNT(DISTINCT columna)` cuenta valores distintos.

Ejemplo práctico: KPIs generales del año

Antes de cualquier análisis, el director quiere los números grandes de 2023: número de tickets, clientes distintos, ventas totales, ticket promedio y ticket máximo.

```
1 dbGetQuery(con, "
2   SELECT COUNT(*)                AS tickets,
3           COUNT(DISTINCT cliente_id) AS clientes,
4           ROUND(SUM(total_transaccion), 2) AS ventas,
5           ROUND(AVG(total_transaccion), 2) AS ticket_promedio,
6           MAX(total_transaccion)      AS ticket_maximo
7   FROM transacciones
8 ")
```

	tickets	clientes	ventas	ticket_promedio	ticket_maximo
1	1000	198	959992.7	959.99	2992.35

Con **AS** pones un **alias**, es decir, un nombre a la columna calculada (sin él, la columna se llamaría literalmente `ROUND(SUM(total_transaccion), 2)`). En 2023 hubo 1 000 tickets por \$959 992.7, con un ticket promedio de \$959.99. De los 200 clientes registrados, 198 compraron al menos una vez.

GROUP BY calcula las agregaciones *por grupo*, igual que `group_by()` + `summarise()`:

Ejemplo práctico: ¿Cómo pagan nuestros clientes?

Finanzas negocia comisiones con el banco y necesita saber cuántas ventas y cuánto dinero entra por cada método de pago.

```
1 dbGetQuery(con, "
2   SELECT metodo_pago,
3           COUNT(*)                AS tickets,
4           ROUND(SUM(total_transaccion), 0) AS ventas,
5           ROUND(AVG(total_transaccion), 2) AS ticket_promedio
6   FROM transacciones
7   GROUP BY metodo_pago
8   ORDER BY ventas DESC
9 ")
```

	metodo_pago	tickets	ventas	ticket_promedio
1	Tarjeta Crédito	350	351171	1003.34

2	Efectivo	315	303735	964.24
3	Tarjeta Débito	233	220855	947.88
4	Transferencia	102	84232	825.80

La tarjeta de crédito es el método más usado (350 tickets) y el que más dinero trae (\$351 171), además de tener el ticket promedio más alto (\$1 003). La transferencia es minoritaria: 102 tickets con un ticket promedio de solo \$826. Si la comisión bancaria de la tarjeta de crédito es alta, esta tabla dice cuánto dinero está en juego.

WHERE filtra *filas antes* de agrupar; para filtrar *grupos después* de agregarlos se usa **HAVING**:

Ejemplo práctico: Clientes frecuentes

El programa de lealtad quiere invitar a los clientes con 10 compras o más en el año.

```

1 dbGetQuery(con, "
2   SELECT cliente_id,
3           COUNT(*)           AS compras,
4           ROUND(SUM(total_transaccion), 2) AS gasto_total
5   FROM transacciones
6   GROUP BY cliente_id
7   HAVING COUNT(*) >= 10      -- condición sobre el grupo
8   ORDER BY compras DESC, gasto_total DESC
9 ")

```

	cliente_id	compras	gasto_total
1	5	13	14680.10
2	55	13	11355.70
3	105	13	10715.72
4	168	12	11675.62
5	125	10	11239.83
6	162	10	11176.45

Solo seis clientes compraron 10 veces o más; el cliente 5 encabeza la lista con 13 compras y \$14680. Observa que el **HAVING** usa **COUNT(*)** (el valor del grupo), algo que **WHERE** no puede hacer porque se evalúa antes de agrupar.

▲ Cuidado

Toda columna del **SELECT** que no esté dentro de una función de agregación debe aparecer en el **GROUP BY**. SQLite es permisivo y no marca error si lo olvidas (devuelve un valor cualquiera del grupo), pero SQL Server y PostgreSQL sí lo rechazan. Acostúmbrate a hacerlo bien desde el principio.

5.4.5 CASE WHEN: clasificar dentro de la consulta

CASE WHEN es el `case_when()` de SQL: evalúa condiciones en orden y devuelve el valor de la primera que se cumple.

Ejemplo práctico: Distribución de tickets por rango

El área comercial quiere saber cuántas ventas son pequeñas, medianas o grandes, y cuánto aporta cada rango.

```
1 dbGetQuery(con, "
2   SELECT CASE
3       WHEN total_transaccion < 500 THEN '1. Menos de 500'
4       WHEN total_transaccion < 1500 THEN '2. De 500 a 1,500'
5       ELSE '3. 1,500 o más'
6   END AS rango_ticket,
7   COUNT(*) AS tickets,
8   ROUND(SUM(total_transaccion), 0) AS ventas
9 FROM transacciones
10 GROUP BY rango_ticket
11 ORDER BY rango_ticket
12 ")
```

	rango_ticket	tickets	ventas
1	1. Menos de 500	377	101783
2	2. De 500 a 1,500	379	347606
3	3. 1,500 o más	244	510603

Los tickets de \$1500 o más son solo 244 de 1000 (menos de la cuarta parte), pero aportan \$510603, más de la mitad de las ventas. Los 377 tickets pequeños apenas suman \$101783. Es la clásica regla de Pareto: pocas ventas grandes explican gran parte del ingreso. Numerar las etiquetas ('1. ...') es un truco sencillo para que **ORDER BY** las deje en el orden lógico y no en orden alfabético.

Hazlo tú

Escribe una consulta que muestre, para cada ciudad, cuántos clientes hay y su edad promedio, solo para las ciudades con más de 20 clientes, ordenadas de mayor a menor número de clientes. (Pista: **GROUP BY** ciudad y **HAVING COUNT(*) > 20.**)

5.4.6 Fechas en SQLite

Las fechas son la fuente número uno de dolores de cabeza al pasar datos de R a SQLite. Veamos el problema con calma y luego la solución.

El problema: los Date de R se guardan como números

SQLite solo tiene cinco clases de almacenamiento: **NULL**, **INTEGER**, **REAL**, **TEXT** y **BLOB**; no existe un tipo fecha. Internamente, R guarda un **Date** como el número de días transcurridos desde el 1 de enero de 1970. Si escribes el data frame tal cual, eso es lo que llega a la base. Hagamos la prueba en una base temporal en memoria, leyendo el CSV de tiendas *sin* convertir las fechas:

```
1 # Base temporal para experimentar (no toca bi_retail.sqlite)
2 con_prueba <- dbConnect(RSQLite::SQLite(), ":memory:")
3
4 tiendas_r <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
5 class(tiendas_r$fecha_apertura)      # en R es una fecha
6
7 dbWriteTable(con_prueba, "tiendas", tiendas_r)
8 dbGetQuery(con_prueba, "
9     SELECT nombre_tienda, fecha_apertura,
10             typeof(fecha_apertura)      AS tipo,
11             strftime('%Y', fecha_apertura) AS anio
12     FROM tiendas
13     LIMIT 3
14 ")
```

```
[1] "Date"
      nombre_tienda fecha_apertura tipo  anio
1 Sucursal Centro      14683 real -4672
2 Sucursal Norte      15511 real -4670
3 Sucursal Sur        15227 real -4671
```

La fecha de apertura de la Sucursal Centro (15 de marzo de 2010) quedó guardada como el número **14683** (días desde 1970), de tipo **real**. Peor aún: las funciones de fecha de SQLite interpretan ese número de otra forma (como un “día juliano”, contado desde el año 4714 antes de Cristo), así que **strftime('%Y', ...)** devuelve un año absurdo, **-4672**, sin marcar ningún error. Es un error silencioso: el más peligroso de todos.

La solución: fechas como texto ISO

La práctica recomendada es convertir las fechas a texto con formato ISO **'AAAA-MM-DD'** antes de escribirlas, justo lo que hizo nuestra función **leer_para_bd()**. Las funciones de fecha de SQLite entienden ese formato de forma nativa:

```
1 tiendas_ok <- tiendas_r %>%
2   mutate(fecha_apertura = format(fecha_apertura, "%Y-%m-%d"))
3
4 dbWriteTable(con_prueba, "tiendas", tiendas_ok, overwrite = TRUE)
5 dbGetQuery(con_prueba, "
```



```

6  SELECT nombre_tienda, fecha_apertura,
7         typeof(fecha_apertura)      AS tipo,
8         strftime('%Y', fecha_apertura) AS anio
9  FROM tiendas
10 LIMIT 3
11 ")
12
13 # De regreso en R, la columna es texto: la convertimos con as.Date()
14 dbReadTable(con_prueba, "tiendas") %>%
15   mutate(fecha_apertura = as.Date(fecha_apertura)) %>%
16   pull(fecha_apertura) %>%
17   class()
18
19 dbDisconnect(con_prueba)

```

```

      nombre_tienda fecha_apertura tipo anio
1 Sucursal Centro    2010-03-15 text 2010
2 Sucursal Norte     2012-06-20 text 2012
3 Sucursal Sur       2011-09-10 text 2011
[1] "Date"

```

Ahora la fecha se ve bien, es de tipo **text** y el año es correcto. Al leerla de regreso en R llega como texto, y basta un `as.Date()` (o `ymd()` de **lubridate**) para volver a tener una fecha.

Consejo

RSQLite ofrece una alternativa: `dbConnect(RSQLite::SQLite(), ruta, extended_types = TRUE)`. Con esa opción, RSQLite guarda la fecha con el tipo declarado **DATE** y la convierte de ida y vuelta a **Date** de R automáticamente. Es cómodo si solo R va a leer la base, pero otras herramientas (Power BI, Python, un visor de SQLite) seguirán viendo el número. El texto ISO funciona con todas, por eso es la opción que usamos en el libro. El mismo cuidado aplica a las horas (**hms**) y a las fechas con hora (**POSIXct**): se guardan como segundos si no las conviertes a texto.

Funciones de fecha de SQLite

La función estrella es `strftime(formato, fecha)`, que extrae partes de una fecha: `'%Y'` año, `'%m'` mes, `'%d'` día, `'%Y-%m'` año-mes, `'%w'` día de la semana (0 = domingo). También existen `date()` para sumar o restar intervalos y `julianday()` para calcular diferencias en días.

Ejemplo práctico: Ventas por mes

La pregunta más común en BI: ¿cuánto vendimos cada mes?

```

1 ventas_mes_sql <- dbGetQuery(con, "
2   SELECT strftime('%Y-%m', fecha)      AS mes,
3         COUNT(*)                      AS tickets,
4         ROUND(SUM(total_transaccion), 2) AS ventas
5   FROM transacciones
6   GROUP BY mes
7   ORDER BY mes
8 ")
9 ventas_mes_sql

```

	mes	tickets	ventas
1	2023-01	98	82559.66
2	2023-02	79	65899.34
3	2023-03	91	90930.61
4	2023-04	71	57037.74
5	2023-05	88	93910.21
6	2023-06	84	88509.12
7	2023-07	83	86456.05
8	2023-08	75	78929.49
9	2023-09	86	68855.87
10	2023-10	74	70004.10
11	2023-11	80	80506.89
12	2023-12	91	96393.61

Diciembre es el mejor mes (\$96 394) y abril el más flojo (\$57 038, con solo 71 tickets). Aunque la tabla ya trae una columna `mes`, es buena práctica saber calcularla desde la fecha: muchas tablas reales solo tienen la fecha.

Para calcular con fechas, `date(fecha, '+1 year')` suma un intervalo (también acepta `'+7 days'`, `'-1 month'` o `'start of month'`) y la resta de dos `julianday()` da la diferencia en días; por ejemplo, `julianday('2023-12-31') - julianday(fecha_registro)` es la antigüedad de cada cliente al cierre del año.

Hazlo tú

Calcula las ventas por trimestre usando la fecha. Pista: el mes se obtiene con `CAST(strftime('%m', fecha) AS INTEGER)` y el trimestre es `(mes + 2) / 3` (en SQLite, la división entre enteros descarta los decimales).

5.4.7 JOIN: combinar tablas

Casi ninguna pregunta de negocio se responde con una sola tabla. La tabla de transacciones solo tiene `producto_id`; para saber la *categoría* hay que ir a la tabla de productos. Eso es un **JOIN**. Los dos tipos que usarás el 95 % del tiempo son:

- **INNER JOIN**: solo las filas que tienen pareja en ambas tablas (equivale a `inner_join()`).

- **LEFT JOIN**: todas las filas de la tabla de la izquierda, tengan o no pareja; si no la tienen, las columnas de la derecha quedan en **NULL** (NA en R). Equivale a `left_join()`.

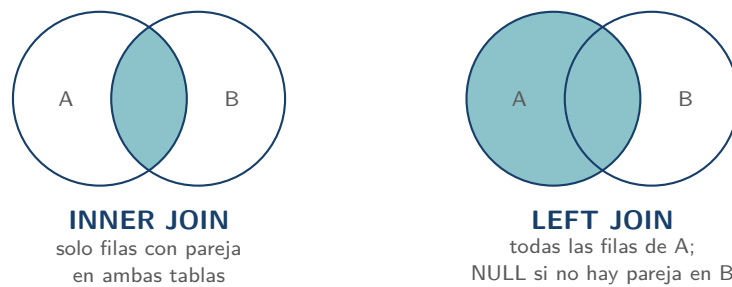


Figura 5.2. Los dos tipos de JOIN más usados. A es la tabla después de FROM y B la tabla después de JOIN.

Ejemplo práctico: Ventas por categoría de producto

El gerente de compras quiere saber qué categorías venden más para negociar mejor con los proveedores.

```

1 dbGetQuery(con, "
2   SELECT p.categoria,
3           COUNT(*)           AS tickets,
4           SUM(t.cantidad)     AS unidades,
5           ROUND(SUM(t.total_transaccion), 0) AS ventas
6   FROM transacciones AS t
7   INNER JOIN productos AS p
8       ON t.producto_id = p.producto_id -- llave de unión
9   GROUP BY p.categoria
10  ORDER BY ventas DESC
11 ")

```

	categoria	tickets	unidades	ventas
1	Belleza	187	551	182143
2	Electrónica	137	403	132672
3	Juguetes	118	356	125093
4	Alimentos	115	356	121543
5	Automotriz	105	325	104085
6	Deportes	92	280	86171
7	Hogar	64	187	63407
8	Mascotas	74	201	57668
9	Libros	70	204	57162
10	Ropa	38	106	30048

Fíjate en los alias de tabla: `transacciones AS t` permite escribir `t.cantidad` en lugar de `transacciones.cantidad`. Cuando dos tablas tienen una columna con el mismo nombre (aquí, `producto_id`), debes indicar de qué tabla viene; si no, la base responde con un error de columna

ambigua. Belleza es por mucho la categoría líder (\$182 143, 187 tickets), seguida de Electrónica; Ropa queda al final con \$30 048. Parte de la explicación está en el catálogo: Belleza tiene 9 productos y Ropa solo 2.

Ejemplo práctico: Clientes que nunca han comprado

El CRM tiene 200 clientes registrados, pero solo 198 compraron. ¿Quiénes son los que faltan? Son candidatos perfectos para una campaña de activación.

```
1 dbGetQuery(con, "
2   SELECT c.cliente_id, c.nombre, c.ciudad, c.fecha_registro
3   FROM clientes AS c
4   LEFT JOIN transacciones AS t
5       ON c.cliente_id = t.cliente_id
6   WHERE t.transaccion_id IS NULL    -- sin pareja en transacciones
7 ")
```

	cliente_id	nombre	ciudad	fecha_registro
1	171	Rosa Jiménez	Mérida	2022-08-27
2	199	Ricardo Hernández	Puebla	2023-09-16

Este patrón (`LEFT JOIN + WHERE ... IS NULL`) es el equivalente SQL de `anti_join()`: encuentra las filas de la izquierda que *no* tienen pareja. Son Rosa Jiménez, de Mérida, y Ricardo Hernández, de Puebla. Este último se registró en septiembre de 2023, así que quizá solo necesita tiempo; Rosa lleva más de un año registrada sin comprar.

▲ Cuidado

Un JOIN mal planteado puede *multiplicar* filas. Si unes por una columna que se repite en ambas tablas (por ejemplo, `ciudad` entre clientes y tiendas), cada cliente se empareja con todas las tiendas de su ciudad y tus sumas se inflan. Antes de sumar después de un JOIN, verifica con `COUNT(*)` que el número de filas es el que esperas.

5.4.8 Subconsultas

Una **subconsulta** es una consulta dentro de otra, escrita entre paréntesis. Sirve, por ejemplo, para comparar contra un valor que primero hay que calcular.

Ejemplo práctico: Productos más caros que el promedio

Queremos la lista de productos cuyo precio de catálogo supera el promedio del catálogo, con su margen.

```

1 dbGetQuery(con, "
2   SELECT nombre_producto, categoria, precio_catalogo, margen_pct
3   FROM productos
4   WHERE precio_catalogo > (SELECT AVG(precio_catalogo)
5                             FROM productos)
6   ORDER BY precio_catalogo DESC
7   LIMIT 5
8 ")

```

	nombre_producto	categoria	precio_catalogo	margen_pct
1	Producto 28	Mascotas	774.03	97.77
2	Producto 47	Automotriz	766.07	92.82
3	Producto 41	Electrónica	756.54	48.47
4	Producto 04	Juguetes	743.84	72.42
5	Producto 20	Belleza	741.63	81.42

La subconsulta (`SELECT AVG(precio_catalogo) FROM productos`) se calcula primero (da \$416.92) y la consulta externa compara cada producto contra ese valor. Una subconsulta puede ir en el **WHERE**, en el **FROM** o incluso en el **SELECT**, siempre que en este último caso devuelva un solo valor.

Una subconsulta también puede ir en el **FROM**, como si fuera una tabla temporal. Es útil para agregar en dos pasos:

```

1 # Paso 1 (subconsulta): gasto total de cada cliente
2 # Paso 2: estadísticas sobre esos totales
3 dbGetQuery(con, "
4   SELECT COUNT(*)           AS clientes,
5          ROUND(AVG(gasto), 2) AS gasto_promedio,
6          ROUND(MAX(gasto), 2) AS gasto_maximo,
7          SUM(gasto > 8000)   AS clientes_mas_de_8000
8   FROM (SELECT cliente_id, SUM(total_transaccion) AS gasto
9         FROM transacciones
10        GROUP BY cliente_id) AS gasto_cliente
11 ")

```

	clientes	gasto_promedio	gasto_maximo	clientes_mas_de_8000
1	198	4848.45	14680.1	23

Los 198 clientes con compras gastaron en promedio \$4 848 en el año; el máximo es \$14 680 (el cliente 5 que vimos antes) y 23 clientes superan los \$8 000. Cada subconsulta en el **FROM** necesita un alias (aquí, `gasto_cliente`).

5.4.9 CTE: consultas legibles con WITH

Cuando las subconsultas se anidan, el SQL se vuelve difícil de leer. Una **CTE** (*Common Table Expression*) le pone nombre a un paso intermedio con **WITH nombre AS (...)** y luego lo usas como si fuera una tabla. Es el equivalente SQL de guardar un resultado intermedio en un objeto de R.

Ejemplo práctico: Participación de cada tienda en las ventas

¿Qué porcentaje de las ventas totales aporta cada sucursal?

```
1 dbGetQuery(con, "
2   WITH ventas_tienda AS (           -- paso 1: ventas por tienda
3     SELECT tienda_id, SUM(total_transaccion) AS ventas
4     FROM transacciones
5     GROUP BY tienda_id
6   ),
7   total AS (                         -- paso 2: total de la empresa
8     SELECT SUM(ventas) AS ventas_total FROM ventas_tienda
9   )
10  SELECT ti.nombre_tienda,           -- paso 3: combinar y calcular %
11         ROUND(v.ventas, 0) AS ventas,
12         ROUND(100.0 * v.ventas / total.ventas_total, 1) AS pct
13  FROM ventas_tienda AS v
14  INNER JOIN tiendas AS ti ON v.tienda_id = ti.tienda_id
15  CROSS JOIN total
16  ORDER BY v.ventas DESC
17 ")
```

	nombre_tienda	ventas	pct
1	Sucursal Outlet	114377	11.9
2	Sucursal Sur	105305	11.0
3	Sucursal Plaza A	105216	11.0
4	Sucursal Express	102791	10.7
5	Sucursal Premium	101538	10.6
6	Sucursal Norte	89024	9.3
7	Sucursal Este	88420	9.2
8	Sucursal Centro	88414	9.2
9	Sucursal Oeste	84672	8.8
10	Sucursal Plaza B	80236	8.4

Las ventas están muy repartidas: la Sucursal Outlet lidera con 11.9% y la Plaza B cierra con 8.4%. Llama la atención que el Outlet, la tienda más pequeña después de Express (350 m²), sea la que más vende. **CROSS JOIN total** pega la única fila de **total** a cada tienda. Y observa el **100.0**: en SQL, la división entre dos enteros descarta los decimales (**5 / 2** da **2**), así que multiplicar por **100.0** obliga a usar decimales.

5.4.10 Funciones de ventana

Las **funciones de ventana** calculan un valor para cada fila mirando un grupo de filas relacionadas (la “ventana”), *sin colapsar* las filas como hace **GROUP BY**. Son el equivalente de **group_by()** + **mutate()** en dplyr. Su sintaxis es:

```
funcion() OVER (PARTITION BY grupo ORDER BY orden)
```

PARTITION BY define los grupos (como **group_by()**) y **ORDER BY** el orden dentro de cada grupo. Las más útiles en BI son **ROW_NUMBER()**, **RANK()**, **SUM() OVER** (acumulados) y **LAG()** (valor de la fila anterior). SQLite las soporta desde la versión 3.25; la que trae **RSQLite** es mucho más reciente.

Ejemplo práctico: El producto estrella de cada categoría

Para el catálogo impreso, Marketing quiere destacar el producto más vendido (en dinero) de cada categoría.

```
1 dbGetQuery(con, "
2   WITH ventas_producto AS (
3     SELECT p.categoria, p.nombre_producto,
4           SUM(t.total_transaccion) AS ventas
5     FROM transacciones AS t
6     INNER JOIN productos AS p ON t.producto_id = p.producto_id
7     GROUP BY p.categoria, p.nombre_producto
8   ),
9   ranking AS (
10    SELECT categoria, nombre_producto, ROUND(ventas, 0) AS ventas,
11          ROW_NUMBER() OVER (PARTITION BY categoria
12                             ORDER BY ventas DESC) AS posicion
13    FROM ventas_producto
14  )
15  SELECT * FROM ranking
16  WHERE posicion = 1
17  ORDER BY ventas DESC
18 ")
```

	categoria	nombre_producto	ventas	posicion
1	Juguetes	Producto 06	34028	1
2	Deportes	Producto 19	28368	1
3	Alimentos	Producto 31	25430	1
4	Automotriz	Producto 47	24527	1
5	Hogar	Producto 32	24333	1
6	Belleza	Producto 14	24023	1
7	Electrónica	Producto 21	22083	1
8	Libros	Producto 16	20501	1
9	Ropa	Producto 42	17448	1

10	Mascotas	Producto 22	16949	1
----	----------	-------------	-------	---

`ROW_NUMBER()` numera los productos de cada categoría de mayor a menor venta, y el `WHERE` externo se queda con el número 1. No se puede filtrar por una función de ventana en el mismo nivel donde se calcula; por eso va en una CTE. Como la lista está ordenada por ventas, su primera fila es también el producto más vendido de todo el catálogo: el Producto 06, de Juguetes, con \$34 028.

Ejemplo práctico: Ventas acumuladas y crecimiento mensual

La directora quiere ver, mes a mes, las ventas, el acumulado del año y el crecimiento contra el mes anterior.

```

1 dbGetQuery(con, "
2   WITH mensual AS (
3     SELECT strftime('%Y-%m', fecha) AS mes,
4            SUM(total_transaccion) AS ventas
5     FROM transacciones
6     GROUP BY mes
7   )
8   SELECT mes,
9          ROUND(ventas, 0) AS ventas,
10         ROUND(SUM(ventas) OVER (ORDER BY mes), 0) AS acumulado,
11         ROUND(LAG(ventas) OVER (ORDER BY mes), 0) AS mes_anterior,
12         ROUND(100.0 * (ventas - LAG(ventas) OVER (ORDER BY mes)) /
13              LAG(ventas) OVER (ORDER BY mes), 1) AS crec_pct
14   FROM mensual
15   ORDER BY mes
16 ")

```

	mes	ventas	acumulado	mes_anterior	crec_pct
1	2023-01	82560	82560	NA	NA
2	2023-02	65899	148459	82560	-20.2
3	2023-03	90931	239390	65899	38.0
4	2023-04	57038	296427	90931	-37.3
5	2023-05	93910	390338	57038	64.6
6	2023-06	88509	478847	93910	-5.8
7	2023-07	86456	565303	88509	-2.3
8	2023-08	78929	644232	86456	-8.7
9	2023-09	68856	713088	78929	-12.8
10	2023-10	70004	783092	68856	1.7
11	2023-11	80507	863599	70004	15.0
12	2023-12	96394	959993	80507	19.7

`SUM(ventas) OVER (ORDER BY mes)` suma todas las filas desde el inicio hasta la actual: el acumulado de diciembre, \$959 993, coincide con el total del año. `LAG(ventas)` trae el valor del mes anterior (en enero es `NA` porque no hay mes previo) y con él calculamos el crecimiento: abril cayó 37.3 % y mayo

se recuperó 64.6 %. Es exactamente lo que en dplyr harías con `cumsum()` y `lag()`.

Ejemplo práctico: Ranking de tiendas con empates

¿Qué lugar ocupa cada tienda por número de tickets? Si dos tiendas empatan, deben compartir lugar.

```
1 dbGetQuery(con, "
2   SELECT tienda_id,
3         COUNT(*)           AS tickets,
4         RANK()              OVER (ORDER BY COUNT(*) DESC) AS rank,
5         DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS dense_rank,
6         ROW_NUMBER() OVER (ORDER BY COUNT(*) DESC) AS row_number
7   FROM transacciones
8   GROUP BY tienda_id
9   ORDER BY tickets DESC
10 ")
```

	tienda_id	tickets	rank	dense_rank	row_number
1	6	109	1	1	1
2	3	109	1	1	2
3	10	108	3	2	3
4	9	108	3	2	4
5	8	106	5	3	5
6	2	99	6	4	6
7	5	95	7	5	7
8	4	94	8	6	8
9	1	87	9	7	9
10	7	85	10	8	10

Las tiendas 6 y 3 empatan con 109 tickets. `RANK()` les da el lugar 1 a ambas y salta al 3 (como en una competencia deportiva); `DENSE_RANK()` no deja huecos (1, 1, 2...) y `ROW_NUMBER()` numera sin empates, desempatando de forma arbitraria. Elige según la regla de negocio: para “premiar a los tres primeros lugares” suele usarse `RANK()`; para “quedarme exactamente con una fila por grupo”, `ROW_NUMBER()`.

Hazlo tú

Modifica la consulta del producto estrella para mostrar los *tres* productos más vendidos de cada categoría (`posicion <= 3`). ¿Cuántas filas obtienes? ¿Por qué no son exactamente 30?

5.5 Consultas parametrizadas e inyección SQL

Muy pronto querrás que tus consultas dependan de un valor que cambia: la ciudad que el usuario elige en un tablero de Shiny, el rango de fechas del reporte de este mes, el cliente que se está consultando.

La tentación es armar el texto de la consulta con `paste()`. **No lo hagas** con valores que vienen de fuera de tu código.

5.5.1 El peligro de pegar texto: inyección SQL

Ejemplo práctico: Un buscador de clientes por ciudad

Imagina un tablero donde el usuario escribe una ciudad en una caja de texto y ve cuántos clientes hay. Un usuario malicioso (o simplemente descuidado) escribe algo raro en la caja.

```
1 # Lo que escribió el usuario en la caja de texto
2 ciudad_usuario <- "Mérida" OR '1'='1'
3
4 # MAL: pegar el texto del usuario dentro del SQL
5 sql_peligroso <- paste0(
6   "SELECT COUNT(*) AS clientes FROM clientes WHERE ciudad = '",
7   ciudad_usuario, "'"
8 )
9 cat(sql_peligroso, "\n")
10 dbGetQuery(con, sql_peligroso)
```

```
SELECT COUNT(*) AS clientes FROM clientes WHERE ciudad = 'Mérida' OR '1'='1'
  clientes
1         200
```

El usuario cerró la comilla de 'Mérida' y agregó `OR '1'='1'`, una condición que siempre es verdadera. Resultado: la consulta devolvió los 200 clientes de la base en lugar de los de Mérida. A esto se le llama **inyección SQL**: el texto del usuario deja de ser un *dato* y se convierte en *código* que la base ejecuta. Con un poco más de malicia, el texto podría intentar algo destructivo:

```
-- Si el usuario escribiera:  x'; DROP TABLE clientes; --
-- la consulta armada con paste() quedaría así:
SELECT COUNT(*) FROM clientes WHERE ciudad = 'x'; DROP TABLE clientes; --'
```

Según el motor y el driver, esa segunda instrucción podría ejecutarse y borrar la tabla. La inyección SQL es una de las vulnerabilidades más explotadas en la historia de la informática; no es un riesgo teórico.

5.5.2 La solución: consultas parametrizadas

En una **consulta parametrizada** escribes el SQL con *marcadores* (?) donde irán los valores, y pasas los valores por separado en el argumento `params`. La base recibe el SQL y los datos por canales distintos, así que el valor *nunca* se interpreta como código, contenga lo que contenga.

```

1 sql_seguro <- "SELECT COUNT(*) AS clientes FROM clientes WHERE ciudad = ?"
2
3 # Con el texto malicioso: busca literalmente esa "ciudad" y no la halla
4 dbGetQuery(con, sql_seguro, params = list(ciudad_usuario))
5
6 # Con un valor legítimo funciona normalmente
7 dbGetQuery(con, sql_seguro, params = list("Mérida"))

```

```

      clientes
1          0
      clientes
1         17

```

Con el texto malicioso ahora obtenemos 0 clientes (no existe ninguna ciudad que se llame así, con todo y comillas), y con "Mérida" obtenemos la respuesta correcta. Además, los parámetros te ahorran pelearte con las comillas: no tienes que escribir ' alrededor de los textos.

También puedes usar marcadores con nombre (:nombre) y pasar una lista con nombres, lo que hace más legibles las consultas con varios parámetros:

Ejemplo práctico: Reporte de ventas por método de pago en un periodo

Finanzas quiere un reporte reutilizable: ventas de un método de pago entre dos fechas cualesquiera.

```

1 reporte_pago <- function(con, metodo, desde, hasta) {
2   dbGetQuery(con, "
3     SELECT metodo_pago,
4           COUNT(*)           AS tickets,
5           ROUND(SUM(total_transaccion), 2) AS ventas
6   FROM transacciones
7   WHERE metodo_pago = :metodo
8         AND fecha BETWEEN :desde AND :hasta
9   GROUP BY metodo_pago",
10  params = list(metodo = metodo,
11                desde = format(as.Date(desde)), # fecha como texto
12                hasta = format(as.Date(hasta))))
13 }
14
15 reporte_pago(con, "Transferencia", "2023-01-01", "2023-03-31")
16 reporte_pago(con, "Efectivo", "2023-12-01", "2023-12-31")

```

```

      metodo_pago tickets  ventas
1 Transferencia      22 19523.79
      metodo_pago tickets  ventas

```

1	Efectivo	24 25212.12
---	----------	-------------

En el primer trimestre entraron 22 transferencias por \$19 523.79, y en diciembre hubo 24 pagos en efectivo por \$25 212.12. La misma función sirve para cualquier combinación sin tocar el SQL. Observa que convertimos las fechas con `format(as.Date(...))`: así la función acepta tanto texto como objetos `Date` y siempre manda a la base el texto ISO que espera.

5.5.3 Alternativas: `glue_sql()` y `sqlInterpolate()`

A veces necesitas construir el texto SQL de forma dinámica (por ejemplo, una lista `IN (...)` de longitud variable, o el nombre de una tabla). Para eso hay dos herramientas seguras que *escapan* los valores (les ponen las comillas correctas y neutralizan las comillas internas):

- `glue::glue_sql()`, del paquete **glue** (se instala con el tidyverse): usa llaves `{}` para insertar valores; `{valor*}` expande un vector separado por comas y `{'nombre'}` (con acentos graves) inserta un nombre de tabla o columna.
- `DBI::sqlInterpolate()`: usa marcadores `?nombre`.

```
1 library(glue)
2
3 ciudades <- c("Monterrey", "Guadalajara", "Tijuana")
4 edad_min <- 40
5 sql_glue <- glue_sql("
6   SELECT ciudad, COUNT(*) AS clientes
7   FROM clientes
8   WHERE ciudad IN ({ciudades*}) AND edad >= {edad_min}
9   GROUP BY ciudad", .con = con)
10 sql_glue
11 dbGetQuery(con, sql_glue)
12
13 # El texto malicioso queda "desactivado": sus comillas se duplican
14 glue_sql("SELECT * FROM clientes WHERE ciudad = {ciudad_usuario}",
15   .con = con)
16 sqlInterpolate(con, "SELECT * FROM clientes WHERE ciudad = ?ciudad",
17   ciudad = ciudad_usuario)
```

```
<SQL> SELECT ciudad, COUNT(*) AS clientes
FROM clientes
WHERE ciudad IN ('Monterrey', 'Guadalajara', 'Tijuana') AND edad >= 40
GROUP BY ciudad
      ciudad clientes
1 Guadalajara      13
2 Monterrey        11
3 Tijuana           6
<SQL> SELECT * FROM clientes WHERE ciudad = 'Mérida' OR ''='1'
```

```
<SQL> SELECT * FROM clientes WHERE ciudad = 'Mérida' OR ''1''=''1'
```

`glue_sql()` convirtió el vector `ciudades` en `'Monterrey', 'Guadalajara', 'Tijuana'`, con sus comillas, y el número 40 sin comillas. Entre los clientes de 40 años o más, Guadalajara tiene 13, Monterrey 11 y Tijuana 6. En las dos últimas consultas, las comillas simples del texto malicioso se duplicaron (`"`), que en SQL es la forma de escribir una comilla *dentro* de un texto: todo el valor queda como un solo texto inofensivo.

▲ Cuidado

Regla de oro: **nunca** armes SQL con `paste()`, `paste0()` o `sprintf()` usando valores que escribe un usuario o que vienen de un archivo externo. Usa `params`, y si necesitas construir el texto, `glue_sql()` o `sqlInterpolate()`. Esto será especialmente importante en el Módulo 6, donde los usuarios escriben en cajas de texto de tus tableros.

🔗 Hazlo tú

Escribe una función `clientes_por_segmento(con, segmento)` que devuelva cuántos clientes premium y no premium hay en el segmento indicado (`"Joven"`, `"Adulto"`, `"Maduro"` o `"Senior"`), usando una consulta parametrizada.

5.6 dbplyr: usar dplyr contra la base de datos

¿Y si pudieras consultar la base con los mismos verbos de **dbplyr** que ya dominas, sin escribir SQL? Eso hace **dbplyr**: traduce tu código de `dplyr` a SQL, lo manda a la base y solo te trae el resultado.

5.6.1 Tablas remotas con `tbl()`

`tbl(con, "tabla")` crea una **tabla remota**: un objeto que *apunta* a la tabla dentro de la base pero no la descarga.

```
1 transacciones_bd <- tbl(con, "transacciones")
2 productos_bd     <- tbl(con, "productos")
3
4 transacciones_bd %>%
5   select(transaccion_id, fecha, producto_id, total_transaccion) %>%
6   head(3)
```

```
# Source:   SQL [3 x 4]
...
  transaccion_id fecha      producto_id total_transaccion
      <dbl> <chr>          <dbl>          <dbl>
1             1 2023-01-13             3            1216.
2             2 2023-01-09            47             232.
```

3	3 2023-02-20	6	826.
---	--------------	---	------

La primera línea indica que el origen es una consulta SQL (de 3 filas y 4 columnas, por el `head(3)`); la segunda, que omitimos, indica el motor y la ruta del archivo de la base. `dbplyr` no descargó la tabla: solo pidió a la base las tres primeras filas para mostrártelas. Los importes se ven redondeados (`1216.`) porque así imprime los números un tibble; el valor completo sigue siendo 1215.88.

5.6.2 Verbos de `dplyr`, `show_query()` y `collect()`

Ejemplo práctico: Ventas por método de pago, versión `dplyr`

Repetamos el reporte de métodos de pago, ahora con `dplyr` contra la base.

```
1 consulta_pago <- transacciones_bd %>%
2   group_by(metodo_pago) %>%
3   summarise(tickets = n(),
4             ventas = round(sum(total_transaccion, na.rm = TRUE), 0)) %>%
5   arrange(desc(ventas))
6
7 # ¿Qué SQL generó dbplyr?
8 show_query(consulta_pago)
```

```
<SQL>
SELECT
  `metodo_pago`,
  COUNT(*) AS `tickets`,
  ROUND(SUM(`total_transaccion`), 0) AS `ventas`
FROM `transacciones`
GROUP BY `metodo_pago`
ORDER BY `ventas` DESC
```

`show_query()` te enseña el SQL que `dbplyr` escribió por ti. Es una excelente forma de *aprender* SQL: escribe el código en `dplyr` y mira cómo se traduce. Nota el `na.rm = TRUE`: en SQL las funciones de agregación siempre ignoran los valores nulos, y `dbplyr` te lo recuerda con un aviso si no lo escribes.

Para traer el resultado a R como un tibble normal usa `collect()`:

```
1 resultado_pago <- collect(consulta_pago)
2 resultado_pago
3 class(resultado_pago)
```

```
# A tibble: 4 x 3
  metodo_pago tickets ventas
  <chr>         <int> <dbl>
```

```

1 Tarjeta Crédito      350 351171
2 Efectivo             315 303735
3 Tarjeta Débito      233 220855
4 Transferencia        102 84232
[1] "tbl_df"      "tbl"        "data.frame"

```

Las cifras coinciden con las que obtuvimos con SQL escrito a mano. Ahora `resultado_pago` es un tibble en la memoria de R y puedes usarlo con **ggplot2** o exportarlo a Excel.

Concepto clave: Evaluación diferida (*lazy evaluation*)

Con `dbplyr`, cada verbo *no ejecuta nada*: solo va construyendo la receta de la consulta. La base trabaja únicamente cuando pides ver el resultado (imprimirlo) o cuando llamas a `collect()`. Por eso puedes encadenar muchos pasos sobre una tabla de 100 millones de filas sin problema: todo el trabajo pesado lo hace el motor, y a R solo llega el resultado final, normalmente pequeño.

Los *joins* también se traducen. Repitamos las ventas por categoría:

```

1 ventas_categoria_dbplyr <- transacciones_bd %>%
2   inner_join(productos_bd, by = "producto_id") %>%
3   group_by(categoria) %>%
4   summarise(tickets = n(),
5             ventas = round(sum(total_transaccion, na.rm = TRUE), 0)) %>%
6   arrange(desc(ventas)) %>%
7   collect()
8
9 head(ventas_categoria_dbplyr, 4)

```

```

# A tibble: 4 x 3
  categoria tickets ventas
  <chr>      <int> <dbl>
1 Belleza      187 182143
2 Electrónica   137 132672
3 Juguetes     118 125093
4 Alimentos    115 121543

```

Los números son idénticos a los del **INNER JOIN** en SQL: Belleza lidera con \$182 143.

5.6.3 Límites de la traducción

`dbplyr` traduce muchas funciones de R (aritmética, `if_else()`, `case_when()`, `substr()`, `paste0()`, `min_rank()`, `lag()`, `cumsum()`, las funciones de agregación...), pero no todas. Por ejemplo, `month(fecha)` de **lubridate** se traduce a `strftime('%m', fecha)` en SQLite. Algunas reglas prácticas:

- Si una función no tiene traducción, `dbplyr` la pasa *tal cual* a la base, y la base marca error de

función inexistente. Por ejemplo, `floor_date()` no se traduce en SQLite.

- Algunas funciones solo se traducen parcialmente. Por ejemplo, `str_detect(x, "Tarjeta")` falla con `Only fixed patterns are supported on database backends`; la solución es `str_detect(x, fixed("Tarjeta"))`.
- Puedes escribir un fragmento de SQL literal con `sql()`: `mutate(mes = sql("strftime('%Y-%m', fecha)"))`.
- Si algo no se puede traducir, filtra y agrega primero en la base, haz `collect()` y termina el cálculo en R.

5.6.4 ¿SQL directo o dbplyr?

Ambos caminos llegan al mismo resultado. La tabla 5.3 te ayuda a elegir.

Tabla 5.3. Cuándo conviene escribir SQL y cuándo usar dbplyr.

Usa SQL directo (<code>dbGetQuery()</code>) cuando...	Usa dbplyr cuando...
Ya tienes la consulta escrita (te la pasó el área de TI o viene de otro reporte).	Quieres un solo estilo de código en todo tu proyecto.
Necesitas funciones propias del motor (<code>PRAGMA</code> , <code>EXPLAIN</code> , funciones de fecha especiales).	Estás explorando y cambias la consulta a cada rato.
Creas o modificas estructura: <code>CREATE TABLE</code> , <code>CREATE VIEW</code> , <code>INSERT</code> , índices.	Quieres combinar pasos en la base con pasos en R (<code>collect()</code> en medio).
La consulta la compartirás con personas que no usan R.	Prefieres que el mismo código funcione con varios motores sin cambiar el dialecto.

Consejo

En la práctica, los analistas combinan ambos: dbplyr para explorar y prototipar, y SQL escrito a mano para las consultas “oficiales” que se guardan como vistas en el data warehouse. Si una consulta de dbplyr va a producción, copia el resultado de `show_query()`, revísalo y guárdalo.

Hazlo tú

Con dbplyr, calcula el ticket promedio por tienda (`tienda_id`) solo para pagos en efectivo, ordénalo de mayor a menor y mira el SQL con `show_query()` antes de hacer `collect()`.

5.7 Diseño de un data warehouse en estrella

Hasta ahora la base es una copia fiel de los CSV. Funciona, pero tiene problemas típicos de los datos “crudos”: no hay llaves declaradas, los nombres de columnas son inconsistentes, faltan atributos útiles para analizar (¿la fecha cae en fin de semana?, ¿cuánto costó lo vendido?) y cualquiera podría insertar una venta de un producto inexistente. Un *data warehouse* resuelve esto con un diseño pensado para el análisis: el **esquema en estrella**.

Concepto clave: Hechos y dimensiones

- La **tabla de hechos** (*fact table*) registra los eventos del negocio que quieres medir, uno por fila, con sus **métricas** numéricas: cantidad, importe, costo, margen. Tiene muchas filas y pocas columnas, casi todas numéricas o llaves.
- Las **tablas de dimensión** describen el contexto de cada hecho: *quién* compró (cliente), *qué* (producto), *dónde* (tienda), *quién vendió* (vendedor) y *cuándo* (fecha). Tienen pocas filas y muchas columnas de texto que sirven para filtrar y agrupar.
- La **granularidad** (*grain*) es el nivel de detalle de una fila de hechos. En nuestro caso: *una fila por ticket de venta*.

Dibujado, la tabla de hechos queda al centro y las dimensiones alrededor, como las puntas de una estrella (figura 5.3).

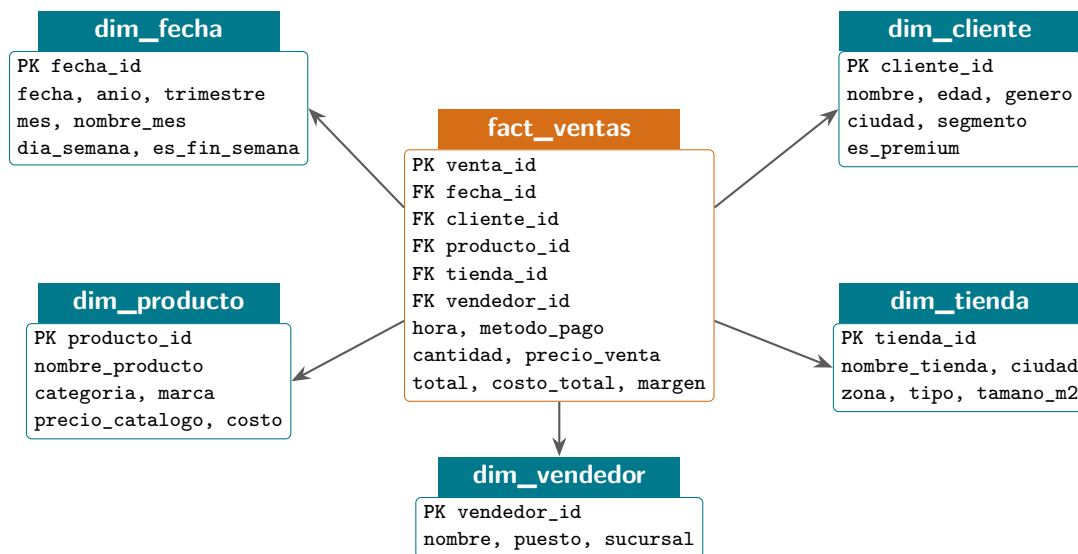


Figura 5.3. Esquema en estrella del data warehouse `bi_retail`: la tabla de hechos (naranja) al centro y cinco dimensiones alrededor.

En la empresa: ¿Por qué en estrella?

El esquema en estrella es el estándar de facto en BI porque las consultas son siempre iguales: “suma las métricas de la tabla de hechos, filtrando y agrupando por atributos de las dimensiones”. Power BI, Tableau y Qlik están optimizados para él, los usuarios de negocio lo entienden de un vistazo y los motores lo resuelven muy rápido. Si algún día trabajas con Power BI, verás que su “modelo de datos” es justamente una estrella.

5.7.1 Crear las tablas con llaves primarias y foráneas

Esta vez no dejaremos que `dbWriteTable()` adivine los tipos: escribiremos nosotros el **CREATE TABLE** de cada tabla con su llave primaria, sus llaves foráneas y algunas reglas de validación (**NOT**

NULL, **CHECK**). A este código que define estructura se le llama **DDL** (*Data Definition Language*). Lo guardamos en un vector con nombre para poder reutilizarlo.

```

1 ddl_estrella <- c(
2   dim_fecha = "
3     CREATE TABLE dim_fecha (
4       fecha_id      INTEGER PRIMARY KEY, -- AAAAMMDD, ej. 20230113
5       fecha         TEXT NOT NULL UNIQUE, -- texto ISO 'AAAA-MM-DD'
6       anio          INTEGER, trimestre INTEGER, mes INTEGER,
7       nombre_mes    TEXT, dia INTEGER, dia_semana TEXT,
8       num_dia_semana INTEGER,           -- 1 = lunes ... 7 = domingo
9       es_fin_semana INTEGER             -- 1 = sábado o domingo
10    ),
11   dim_producto = "
12     CREATE TABLE dim_producto (
13       producto_id INTEGER PRIMARY KEY, nombre_producto TEXT NOT NULL,
14       categoria TEXT, subcategoria TEXT, marca TEXT,
15       precio_catalogo REAL, costo REAL, proveedor TEXT, activo INTEGER
16    ),
17   dim_cliente = "
18     CREATE TABLE dim_cliente (
19       cliente_id INTEGER PRIMARY KEY, nombre TEXT NOT NULL,
20       edad INTEGER, genero TEXT, ciudad TEXT, segmento TEXT,
21       es_premium INTEGER, fecha_registro TEXT
22    ),
23   dim_tienda = "
24     CREATE TABLE dim_tienda (
25       tienda_id INTEGER PRIMARY KEY, nombre_tienda TEXT NOT NULL,
26       ciudad TEXT, zona TEXT, tipo TEXT, tamano_m2 REAL,
27       fecha_apertura TEXT
28    ),
29   dim_vendedor = "
30     CREATE TABLE dim_vendedor (
31       vendedor_id INTEGER PRIMARY KEY, nombre TEXT NOT NULL,
32       departamento TEXT, puesto TEXT, sucursal TEXT, fecha_ingreso TEXT
33    ),
34   fact_ventas = "
35     CREATE TABLE fact_ventas (
36       venta_id      INTEGER PRIMARY KEY,
37       fecha_id      INTEGER NOT NULL REFERENCES dim_fecha(fecha_id),
38       cliente_id    INTEGER NOT NULL REFERENCES dim_cliente(cliente_id),
39       producto_id   INTEGER NOT NULL REFERENCES dim_producto(producto_id),
40       tienda_id     INTEGER NOT NULL REFERENCES dim_tienda(tienda_id),
41       vendedor_id   INTEGER NOT NULL REFERENCES dim_vendedor(vendedor_id),
42       hora TEXT, metodo_pago TEXT,
43       cantidad      INTEGER NOT NULL CHECK (cantidad > 0),
44       precio_venta  REAL    NOT NULL CHECK (precio_venta > 0),
45       total REAL NOT NULL, costo_total REAL, margen REAL
46    ),

```

```

47 log_etl = "
48 CREATE TABLE log_etl (
49     ejecucion_id INTEGER PRIMARY KEY AUTOINCREMENT,
50     fecha_hora TEXT, archivo TEXT,
51     filas_leidas INTEGER, filas_validas INTEGER,
52     filas_nuevas INTEGER, estado TEXT, mensaje TEXT
53 )"
54 )
55
56 crear_esquema <- function(con) {
57     # SQLite trae apagada la revisión de llaves foráneas: la encendemos
58     dbExecute(con, "PRAGMA foreign_keys = ON")
59     for (sql in ddl_estrella) dbExecute(con, sql)
60     invisible(names(ddl_estrella))
61 }
62
63 crear_esquema(con)
64 dbListTables(con)

```

```

[1] "clientes"      "dim_cliente"    "dim_fecha"
[4] "dim_producto"  "dim_tienda"     "dim_vendedor"
[7] "empleados"     "fact_ventas"    "log_etl"
[10] "productos"     "sqlite_sequence" "tiendas"
[13] "transacciones" "ventas_retail"

```

Ahora la base tiene las seis tablas crudas que cargamos al principio y las nuevas tablas del *data warehouse*, además de `log_etl`, que usaremos como bitácora del proceso de carga. La tabla `sqlite_sequence` la crea SQLite automáticamente para llevar la cuenta del `AUTOINCREMENT`. Tres detalles del diseño:

- `PRAGMA foreign_keys = ON` es imprescindible en SQLite: sin él, las cláusulas `REFERENCES` se aceptan pero *no se verifican*. Hay que activarlo en cada conexión nueva.
- La llave de la fecha es un entero con forma `AAAAMMDD` (`20230113`). Es una convención muy común en data warehouses: es compacta, se ordena bien y se lee a simple vista.
- Las columnas `CHECK` (`cantidad > 0`) hacen que la base rechace datos imposibles, sin importar quién o qué programa intente cargarlos.

5.7.2 La dimensión de fechas, construida en R

Una **dimensión de fechas** (o calendario) tiene una fila por día con todos los atributos que alguien podría querer para filtrar o agrupar: año, trimestre, mes, nombre del mes, día de la semana, si es fin de semana, si es feriado... No viene de ningún sistema: la construimos nosotros, y **lubridate** lo hace muy fácil.

```

1  construir_dim_fecha <- function(desde, hasta) {
2    tibble(fecha = seq(as.Date(desde), as.Date(hasta), by = "day")) %>%
3      mutate(
4        fecha_id   = as.integer(format(fecha, "%Y%m%d")),
5        anio       = year(fecha),
6        trimestre  = quarter(fecha),
7        mes        = month(fecha),
8        nombre_mes = as.character(month(fecha, label = TRUE, abbr = FALSE)),
9        dia        = day(fecha),
10       dia_semana  = as.character(wday(fecha, label = TRUE, abbr = FALSE,
11                                       week_start = 1)),
12       num_dia_semana = wday(fecha, week_start = 1), # 1 = lunes
13       es_fin_semana = as.integer(num_dia_semana >= 6),
14       fecha       = format(fecha, "%Y-%m-%d")      # texto ISO
15     ) %>%
16     relocate(fecha_id)
17   }
18
19  dim_fecha <- construir_dim_fecha("2023-01-01", "2023-12-31")
20  nrow(dim_fecha)
21  dim_fecha %>% filter(fecha >= "2023-12-29") %>% select(-dia)

```

```

[1] 365
# A tibble: 3 x 9
  fecha_id fecha      anio trimestre  mes nombre_mes dia_semana
  <int> <chr>      <dbl>      <int> <dbl> <chr>      <chr>
1 20231229 2023-12-29 2023         4     12 diciembre viernes
2 20231230 2023-12-30 2023         4     12 diciembre sábado
3 20231231 2023-12-31 2023         4     12 diciembre domingo
# i 2 more variables: num_dia_semana <dbl>, es_fin_semana <int>

```

Tenemos 365 días con sus atributos. Los nombres de meses y días salen en español porque R usa la configuración regional de tu sistema; si en tu computadora salen en inglés, revisa `Sys.getlocale()`.

5.7.3 Las demás dimensiones

Las otras dimensiones salen de los CSV del curso con algunas transformaciones: elegimos columnas, convertimos tipos (identificadores a entero, lógicos a 0/1, fechas a texto ISO) y dejamos fuera los datos personales que el análisis no necesita (el correo y el teléfono de los clientes). Lo escribimos como una función que devuelve una lista con todas las dimensiones listas para cargar:

```

1  preparar_dimensiones <- function(carpeta = "datasets") {
2    leer <- function(archivo) {
3      read_csv(file.path(carpeta, archivo), show_col_types = FALSE)
4    }

```

```

5  empleados      <- leer("empleados.csv")
6  transacciones <- leer("transacciones.csv")
7
8  list(
9    dim_fecha = construir_dim_fecha("2023-01-01", "2023-12-31"),
10
11    dim_producto = leer("productos.csv") %>%
12      transmute(producto_id = as.integer(producto_id), nombre_producto,
13                categoria, subcategoria, marca, precio_catalogo, costo,
14                proveedor, activo = as.integer(activo)),
15
16    dim_cliente = leer("clientes.csv") %>%      # sin email ni teléfono
17      transmute(cliente_id = as.integer(cliente_id), nombre,
18                edad = as.integer(edad), genero, ciudad,
19                segmento, es_premium = as.integer(es_premium),
20                fecha_registro = format(fecha_registro, "%Y-%m-%d")),
21
22    dim_tienda = leer("tiendas.csv") %>%
23      transmute(tienda_id = as.integer(tienda_id), nombre_tienda, ciudad,
24                zona, tipo, tamano_m2,
25                fecha_apertura = format(fecha_apertura, "%Y-%m-%d")),
26
27    # Los vendedores son los empleados que aparecen como vendedor_id
28    dim_vendedor = empleados %>%
29      semi_join(transacciones, by = c("empleado_id" = "vendedor_id")) %>%
30      transmute(vendedor_id = as.integer(empleado_id), nombre,
31                departamento, puesto, sucursal,
32                fecha_ingreso = format(fecha_ingreso, "%Y-%m-%d"))
33  )
34 }
35
36 dimensiones <- preparar_dimensiones()
37 map_int(dimensiones, nrow)

```

dim_fecha	dim_producto	dim_cliente	dim_tienda	dim_vendedor
365	50	200	10	25

Ahora las cargamos. Aquí hay una diferencia importante con lo que hicimos al principio: usamos `dbAppendTable()` y no `dbWriteTable()`. `dbAppendTable()` agrega filas a una tabla que ya existe, respetando la estructura que diseñamos. En cambio, `dbWriteTable(..., overwrite = TRUE)` borraría la tabla y la volvería a crear a su manera, sin llaves ni reglas de validación.

```

1  cargar_dimensiones <- function(con, dimensiones) {
2    dbBegin(con)                                # todo o nada (ver más adelante)
3    for (tabla in names(dimensiones)) {
4      dbAppendTable(con, tabla, dimensiones[[tabla]])
5    }

```

```

6   dbCommit(con)
7   # Devolver cuántas filas quedaron en cada tabla
8   map_int(names(dimensiones), function(tabla) {
9     dbGetQuery(con, paste("SELECT COUNT(*) AS n FROM", tabla))$n
10  }) %>% set_names(names(dimensiones))
11 }
12
13 cargar_dimensiones(con, dimensiones)

```

dim_fecha	dim_producto	dim_cliente	dim_tienda	dim_vendedor
365	50	200	10	25

▲ Cuidado

Un *hallazgo de calidad de datos*: según el catálogo de empleados, de los 25 vendedores solo unos cuantos pertenecen al departamento de Ventas; los demás aparecen en Marketing, Finanzas, IT... Compruébalo:

```

1 dbGetQuery(con, "
2   SELECT departamento, COUNT(*) AS vendedores
3   FROM dim_vendedor
4   GROUP BY departamento
5   ORDER BY vendedores DESC
6 ")

```

	departamento	vendedores
1	Atención al Cliente	10
2	Ventas	4
3	RRHH	4
4	Marketing	3
5	Finanzas	2
6	Operaciones	1
7	IT	1

Solo 4 de los 25 vendedores pertenecen al departamento de Ventas; 10 están en Atención al Cliente y hay incluso uno de IT. Puede ser un error de los datos simulados, o una realidad (personal de otras áreas que apoya en caja). En una empresa lo reportarías a Recursos Humanos antes de publicar un ranking de “vendedores”. Lo importante es que un buen modelo de datos hace visibles estas inconsistencias.

La base ya aplica las reglas que diseñamos: si alguien intenta cargar un producto con un `producto_id` que ya existe, la llave primaria lo impide con el error `UNIQUE constraint failed`. Lo comprobarás en la sección de ETL y en la de errores comunes.

Hazlo tú

Intenta insertar en `dim_cliente` una fila sin nombre (`INSERT INTO dim_cliente (cliente_id) VALUES (999)`) dentro de un `tryCatch()`. ¿Qué regla del diseño lo impide?

5.8 ETL con R: extraer, transformar y cargar

Las dimensiones cambian poco; la tabla de hechos, en cambio, crece todos los días. En una empresa, cada noche un proceso toma las ventas nuevas del sistema de punto de venta y las agrega al *data warehouse*. Ese proceso es el **ETL**:

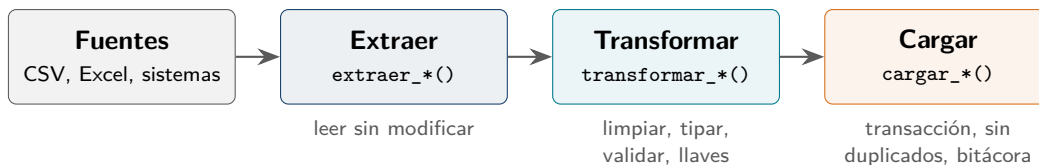


Figura 5.4. Las tres etapas de un proceso ETL y las funciones de R que escribiremos para cada una.

Escribiremos cada etapa como una función pequeña y reutilizable, y una función `ejecutar_etl()` que las orquesta. Así, cuando llegue el archivo de mañana, basta una línea para cargarlo.

5.8.1 Transacciones: todo o nada

Antes de escribir el ETL necesitas un concepto clave. Una **transacción** agrupa varias operaciones para que se apliquen *todas o ninguna*. Piensa en un pedido: se guarda el encabezado (quién compra) y después los renglones (qué compra). Si los renglones fallan, no quieres un encabezado huérfano en la base.

Ejemplo práctico: Un pedido que falla a la mitad

Probemos en una base en memoria qué pasa sin transacción y con transacción.

```

1 con_pedidos <- dbConnect(RSQLite::SQLite(), ":memory:")
2 dbExecute(con_pedidos, "PRAGMA foreign_keys = ON")
3 dbExecute(con_pedidos, "CREATE TABLE productos (id INTEGER PRIMARY KEY)")
4 dbExecute(con_pedidos, "INSERT INTO productos VALUES (1), (2)")
5 dbExecute(con_pedidos, "CREATE TABLE pedidos (pedido_id INTEGER
6                     PRIMARY KEY, cliente TEXT)")
7 dbExecute(con_pedidos, "CREATE TABLE renglones (pedido_id INTEGER,
8                     producto_id INTEGER REFERENCES productos(id))")
9
10 guardar_pedido <- function(con, id, cliente, productos) {
11   dbAppendTable(con, "pedidos", tibble(pedido_id = id,
12                                         cliente = cliente))

```

```

13 dbAppendTable(con, "renglones", tibble(pedido_id = id,
14                                           producto_id = productos))
15 }
16
17 # 1) SIN transacción: el producto 99 no existe y los renglones fallan
18 try(guardar_pedido(con_pedidos, 1, "Ana", c(1, 99)), silent = TRUE)
19 dbGetQuery(con_pedidos, "SELECT * FROM pedidos") # ¡encabezado huérfano!

```

```

[1] 0
[1] 0
[1] 2
[1] 0
[1] 0

```

	pedido_id	cliente
1	1	Ana

El encabezado del pedido 1 se guardó, pero sus renglones no: la base quedó inconsistente. Ahora con transacción. `dbBegin()` la inicia, `dbCommit()` confirma los cambios y `dbRollback()` los deshace todos:

```

1 # 2) CON transacción: si algo falla, se deshace todo
2 dbBegin(con_pedidos)
3 tryCatch({
4   guardar_pedido(con_pedidos, 2, "Luis", c(1, 99))
5   dbCommit(con_pedidos)
6 }, error = function(e) {
7   dbRollback(con_pedidos)
8   cat("Se canceló el pedido:", conditionMessage(e), "\n")
9 })
10 dbGetQuery(con_pedidos, "SELECT * FROM pedidos") # el 2 no quedó
11 dbDisconnect(con_pedidos)

```

```

Se canceló el pedido: FOREIGN KEY constraint failed

```

	pedido_id	cliente
1	1	Ana

Con la transacción, el pedido 2 no dejó rastro: la base quedó exactamente como estaba antes. **DBI** también ofrece `dbWithTransaction(con, { ... })`, que hace el `dbBegin()`, `dbCommit()` y `dbRollback()` por ti.

5.8.2 Extraer

La etapa de extracción lee los datos *sin modificarlos*. En nuestro caso, archivos CSV que el sistema de punto de venta exporta cada cierto tiempo. Forzamos la hora a texto con `col_types` para conservarla tal como viene ("13:15").


```

1 extraer_transacciones <- function(ruta) {
2   read_csv(ruta, show_col_types = FALSE,
3           col_types = cols(hora = col_character()))
4 }
5
6 # Otra fuente frecuente: una hoja de Excel
7 extraer_excel <- function(ruta, hoja) {
8   readxl::read_excel(ruta, sheet = hoja)
9 }

```

Para simular la operación real, partimos `transacciones.csv` en dos “exportaciones” del sistema: el primer semestre y otra que va de mayo a diciembre. A propósito, las dos se *traslapan* en mayo y junio, algo muy común cuando un sistema reenvía datos por seguridad. Nuestro ETL debe evitar cargar esas ventas dos veces.

```

1 # Carpeta de "archivos recibidos": se vacía para que el ejemplo dé el
2 # mismo resultado aunque ejecutes el capítulo varias veces
3 unlink("datos/entrada", recursive = TRUE)
4 dir.create("datos/entrada", recursive = TRUE, showWarnings = FALSE)
5 trans_origen <- extraer_transacciones("datasets/transacciones.csv")
6
7 trans_origen %>% filter(fecha < as.Date("2023-07-01")) %>%
8   write_csv("datos/entrada/ventas_2023_s1.csv")
9 trans_origen %>% filter(fecha >= as.Date("2023-05-01")) %>%
10   write_csv("datos/entrada/ventas_2023_s2.csv")
11
12 list.files("datos/entrada")

```

```
[1] "ventas_2023_s1.csv" "ventas_2023_s2.csv"
```

5.8.3 Transformar

La transformación convierte los datos crudos en filas válidas para `fact_ventas`: descarta registros imposibles o duplicados, calcula la llave de fecha, convierte tipos, agrega el costo desde la dimensión de productos y calcula las métricas. Nota que *recalculamos* el total (`cantidad * precio_venta`) en lugar de confiar en la columna del archivo: en un ETL las métricas derivadas se calculan con una regla única.

```

1 transformar_transacciones <- function(datos, costos) {
2   datos %>%
3     # 1. Calidad: quitar filas sin llave, imposibles o repetidas
4     filter(!is.na(transaccion_id), cantidad > 0, precio_venta > 0) %>%
5     distinct(transaccion_id, .keep_all = TRUE) %>%
6     # 2. Enriquecer con el costo unitario del producto

```

```

7   left_join(costos, by = "producto_id") %>%
8   # 3. Tipos, llaves y métricas con los nombres del data warehouse
9   transmute(
10    venta_id      = as.integer(transaccion_id),
11    fecha_id      = as.integer(format(as.Date(fecha), "%Y%m%d")),
12    cliente_id    = as.integer(cliente_id),
13    producto_id   = as.integer(producto_id),
14    tienda_id     = as.integer(tienda_id),
15    vendedor_id   = as.integer(vendedor_id),
16    hora, metodo_pago,
17    cantidad      = as.integer(cantidad),
18    precio_venta,
19    total         = round(cantidad * precio_venta, 2),
20    costo_total   = round(cantidad * costo, 2),
21    margen        = round(total - costo_total, 2)
22  )
23 }
24
25 # Prueba rápida con las primeras filas del archivo
26 costos <- dbGetQuery(con, "SELECT producto_id, costo FROM dim_producto")
27 trans_origen %>% head(3) %>% transformar_transacciones(costos) %>%
28   select(venta_id, fecha_id, cantidad, total, costo_total, margen)

```

```

# A tibble: 3 x 6
  venta_id fecha_id cantidad total costo_total margen
  <int>    <int>    <int> <dbl>    <dbl>    <dbl>
1       1  20230113         4 1216.    1177.    38.7
2       2  20230109         4  232.     220    12.3
3       3  20230220         3  826.     254.   573.

```

La venta 1 (4 piezas a \$303.97) suma \$1 215.88, le costó a la empresa \$1 177 y dejó un margen de solo \$38.7; la venta 3, en cambio, dejó \$573 de margen sobre \$826. El tibble redondea al imprimir, pero en la tabla se guardan los valores con dos decimales. Probar la transformación con unas cuantas filas antes de cargar es una buena costumbre.

5.8.4 Cargar sin duplicados

Una **carga incremental** agrega solo lo nuevo. Hay dos formas habituales de evitar duplicados:

1. **En R**, con `anti_join()`: consultas qué llaves ya existen en la base y te quedas con las filas cuya llave no está.
2. **En la base**, con `INSERT OR IGNORE`: como `venta_id` es llave primaria, SQLite ignora en silencio las filas cuya llave ya existe. (En PostgreSQL el equivalente es `INSERT ... ON CONFLICT DO NOTHING`; en SQL Server, un `MERGE` o un `WHERE NOT EXISTS`.)

```

1 # Opción 1: evitar duplicados en R con anti_join()
2 cargar_hechos <- function(con, datos) {
3   existentes <- dbGetQuery(con, "SELECT venta_id FROM fact_ventas")
4   nuevos <- anti_join(datos, existentes, by = "venta_id")
5   if (nrow(nuevos) > 0) dbAppendTable(con, "fact_ventas", nuevos)
6   nrow(nuevos) # devuelve cuántas filas se cargaron
7 }
8
9 # Opción 2: dejar que la base ignore las llaves repetidas
10 cargar_hechos_sql <- function(con, datos) {
11   dbWriteTable(con, "stg_ventas", datos, temporary = TRUE,
12               overwrite = TRUE) # tabla temporal de paso
13   n <- dbExecute(con, "INSERT OR IGNORE INTO fact_ventas
14                      SELECT * FROM stg_ventas")
15   dbExecute(con, "DROP TABLE stg_ventas")
16   n
17 }

```

La opción 1 es muy clara y funciona bien con miles o cientos de miles de filas. La opción 2 escala mejor (no trae las llaves a R) y usa una **tabla de paso** o *staging*: una tabla temporal donde se depositan los datos antes de pasarlos a la tabla definitiva. La tabla temporal solo existe mientras dura la conexión.

▲ Cuidado

En SQLite, `INSERT OR IGNORE` ignora en silencio *cualquier* fila que viole una restricción, no solo las de llave repetida: también las que no cumplen un `CHECK` o un `NOT NULL`. Una venta con cantidad negativa simplemente “desaparecería” sin error. Si quieres que esas filas detengan la carga, usa la opción 1 o un `INSERT ... SELECT ... WHERE venta_id NOT IN (SELECT venta_id FROM fact_ventas)`. En nuestro ETL, además, las reglas de calidad de `transformar_transacciones()` ya descartan esas filas antes de llegar a la base.

5.8.5 Bitácora y orquestación: `ejecutar_etl()`

Un ETL profesional deja rastro de cada ejecución: cuándo corrió, qué archivo procesó, cuántas filas leyó, cuántas cargó y si terminó bien o con error. Esa **bitácora** (*log*) es lo primero que revisas cuando alguien pregunta “¿por qué el tablero no tiene las ventas de ayer?”.

```

1 registrar_log <- function(con, archivo, leidas, validas, nuevas,
2                           estado, mensaje = NA_character_) {
3   dbAppendTable(con, "log_etl", tibble(
4     fecha_hora    = format(Sys.time(), "%Y-%m-%d %H:%M:%S"),
5     archivo       = basename(archivo),
6     filas_leidas  = leidas,  filas_validas = validas,
7     filas_nuevas  = nuevas,  estado = estado, mensaje = mensaje
8   ))

```

```

9  }
10
11  ejecutar_etl <- function(con, ruta, cargar = cargar_hechos) {
12    leidas <- validas <- NA_integer_
13    en_transaccion <- FALSE
14    estado <- tryCatch({
15      datos <- extraer_transacciones(ruta) # E
16      leidas <- nrow(datos)
17      costos <- dbGetQuery(con, "SELECT producto_id, costo
18                                FROM dim_producto")
19      limpios <- transformar_transacciones(datos, costos) # T
20      validas <- nrow(limpios)
21      dbBegin(con); en_transaccion <- TRUE
22      nuevas <- cargar(con, limpios) # L
23      registrar_log(con, ruta, leidas, validas, nuevas, "OK")
24      dbCommit(con)
25      sprintf("OK: %d leídas, %d nuevas", leidas, nuevas)
26    }, error = function(e) {
27      if (en_transaccion) dbRollback(con) # deshacer la carga a medias
28      registrar_log(con, ruta, leidas, validas, 0L, "ERROR",
29                    conditionMessage(e))
30      paste("ERROR:", conditionMessage(e))
31    })
32    cat(basename(ruta), "->", estado, "\n")
33    invisible(estado)
34  }

```

Observa cómo encajan las piezas: si cualquier paso falla, el bloque **error** deshace la transacción (si ya había empezado) y registra el error en la bitácora; la función nunca deja la base a medias. Ahora ejecutemos el ETL como lo haría el proceso nocturno:

```

1  ejecutar_etl(con, "datos/entrada/ventas_2023_s1.csv")
2  ejecutar_etl(con, "datos/entrada/ventas_2023_s2.csv")
3
4  # ¿Y si por error se vuelve a ejecutar el mismo archivo?
5  ejecutar_etl(con, "datos/entrada/ventas_2023_s2.csv")
6
7  # Lo mismo con la opción 2 (INSERT OR IGNORE)
8  ejecutar_etl(con, "datos/entrada/ventas_2023_s2.csv",
9                cargar = cargar_hechos_sql)
10
11  dbGetQuery(con, "SELECT COUNT(*) AS filas FROM fact_ventas")

```

```

ventas_2023_s1.csv -> OK: 511 leídas, 511 nuevas
ventas_2023_s2.csv -> OK: 661 leídas, 489 nuevas
ventas_2023_s2.csv -> OK: 661 leídas, 0 nuevas
ventas_2023_s2.csv -> OK: 661 leídas, 0 nuevas

```

```

filas
1 1000

```

El primer archivo cargó sus 511 ventas. El segundo leyó 661, pero solo 489 eran nuevas: las 172 de mayo y junio ya estaban y se descartaron. Las dos ejecuciones repetidas (una con `anti_join()` y otra con `INSERT OR IGNORE`) no cargaron nada. Resultado: exactamente 1000 filas, las mismas del archivo original, sin un solo duplicado.

Probemos ahora el caso de error. Llega un archivo con tres ventas nuevas, pero una tiene un producto que no existe en el catálogo (el 999):

```

1 lote_malo <- trans_origen %>%
2   slice(1:3) %>%
3   mutate(transaccion_id = 1001:1003, fecha = as.Date("2023-12-31"),
4           producto_id = c(5, 999, 7))
5 write_csv(lote_malo, "datos/entrada/ventas_2023_12_31.csv")
6
7 ejecutar_etl(con, "datos/entrada/ventas_2023_12_31.csv")
8 dbGetQuery(con, "SELECT COUNT(*) AS filas FROM fact_ventas")

```

```

ventas_2023_12_31.csv -> ERROR: FOREIGN KEY constraint failed
  filas
1 1000

```

La llave foránea hacia `dim_producto` rechazó el lote, la transacción se deshizo y la tabla de hechos sigue con 1000 filas: ni siquiera las dos ventas válidas del archivo entraron. ¿Es eso lo que queremos? En la mayoría de los data warehouses, sí: es preferible cargar el archivo completo una vez corregido que tener un día “a medias”. Veamos la bitácora (omitimos la columna de fecha y hora, que cambia en cada ejecución):

```

1 dbGetQuery(con, "
2   SELECT ejecucion_id AS id, archivo, filas_leidas AS leidas,
3           filas_validas AS validas, filas_nuevas AS nuevas, estado
4   FROM log_etl
5 ")
6 dbGetQuery(con, "SELECT mensaje FROM log_etl WHERE estado = 'ERROR'")

```

	id	archivo	leidas	validas	nuevas	estado
1	1	ventas_2023_s1.csv	511	511	511	OK
2	2	ventas_2023_s2.csv	661	661	489	OK
3	3	ventas_2023_s2.csv	661	661	0	OK
4	4	ventas_2023_s2.csv	661	661	0	OK
5	5	ventas_2023_12_31.csv	3	3	0	ERROR

	mensaje
1	FOREIGN KEY constraint failed

La bitácora cuenta la historia completa de las cargas. En un proceso programado (por ejemplo, con el Programador de tareas de Windows o **cron**, que verás en el Módulo 8), revisarías esta tabla cada mañana o, mejor, harías que el propio script te avise por correo si hay un **ERROR**.

Consejo

Tres reglas de oro de un ETL confiable: **(1)** que sea *idempotente*, es decir, que ejecutarlo dos veces con el mismo archivo deje la base igual que ejecutarlo una vez (lo logramos con la llave primaria y el **anti_join()**); **(2)** que sea *atómico*: o carga todo el lote o nada (la transacción); y **(3)** que sea *trazable*: cada ejecución deja registro (la bitácora).

5.8.6 Extraer desde Excel

Muchas fuentes llegan en Excel. El archivo `datos_empresa.xlsx` del curso tiene una hoja **Tiendas**. ¿Coincide con lo que cargamos en `dim_tienda`? Es una validación típica de ETL:

```
1 tiendas_excel <- extraer_excel("datasets/datos_empresa.xlsx", "Tiendas")
2 class(tiendas_excel$fecha_apertura)
3
4 tiendas_excel_t <- tiendas_excel %>%
5   transmute(tienda_id = as.integer(tienda_id), nombre_tienda,
6             fecha_apertura = format(as.Date(fecha_apertura), "%Y-%m-%d"))
7
8 # Filas del Excel que no coinciden exactamente con la dimensión
9 tiendas_excel_t %>%
10   anti_join(dbReadTable(con, "dim_tienda"),
11            by = c("tienda_id", "nombre_tienda", "fecha_apertura"))
```

```
[1] "POSIXct" "POSIXt"
# A tibble: 0 x 3
# i 3 variables: tienda_id <int>, nombre_tienda <chr>,
#   fecha_apertura <chr>
```

readxl lee las fechas de Excel como fecha con hora (**POSIXct**); por eso las pasamos a **Date** y luego a texto ISO antes de comparar. El **anti_join()** no devolvió ninguna fila: la hoja de Excel y la dimensión coinciden.

Hazlo tú

Crea un archivo `datos/entrada/ventas_2023_12_31_v2.csv` corrigiendo el lote malo (cambia el producto 999 por el 9) y ejecútalo con **ejecutar_etl()**. ¿Cuántas filas nuevas se cargan? ¿Qué muestra la bitácora? (Si lo haces, recuerda que la tabla de hechos tendrá 1 003 filas y los resultados de las siguientes secciones cambiarán ligeramente.)

5.9 Vistas e índices: el data warehouse listo para BI

5.9.1 Vistas para los KPIs

Una **vista** (`CREATE VIEW`) es una consulta guardada en la base con un nombre; se consulta como si fuera una tabla, pero no guarda datos: cada vez que la consultas, se ejecuta la consulta con los datos actuales. Son ideales para publicar los KPIs “oficiales”: todos (tus scripts de R, Power BI, el tablero de Shiny) leen la misma definición.

```

1 vistas <- c(
2   # Vista "ancha": cada venta con todos sus atributos descriptivos
3   "CREATE VIEW vw_ventas_detalle AS
4     SELECT f.*, d.fecha, d.anio, d.mes, d.nombre_mes, d.dia_semana,
5           d.es_fin_semana, p.nombre_producto, p.categoria, p.marca,
6           c.nombre AS cliente, c.ciudad AS ciudad_cliente, c.segmento,
7           c.es_premium, t.nombre_tienda, t.ciudad AS ciudad_tienda,
8           t.zona, t.tipo AS tipo_tienda, v.nombre AS vendedor
9     FROM fact_ventas AS f
10    JOIN dim_fecha    AS d ON f.fecha_id    = d.fecha_id
11    JOIN dim_producto AS p ON f.producto_id = p.producto_id
12    JOIN dim_cliente  AS c ON f.cliente_id  = c.cliente_id
13    JOIN dim_tienda   AS t ON f.tienda_id   = t.tienda_id
14    JOIN dim_vendedor AS v ON f.vendedor_id = v.vendedor_id",
15   # KPIs mensuales
16   "CREATE VIEW vw_kpi_mensual AS
17     SELECT d.anio, d.mes, d.nombre_mes,
18           COUNT(*) AS tickets,
19           COUNT(DISTINCT f.cliente_id) AS clientes,
20           ROUND(SUM(f.total), 2) AS ventas,
21           ROUND(SUM(f.margen), 2) AS margen,
22           ROUND(AVG(f.total), 2) AS ticket_promedio
23     FROM fact_ventas AS f
24    JOIN dim_fecha AS d ON f.fecha_id = d.fecha_id
25   GROUP BY d.anio, d.mes, d.nombre_mes"
26 )
27 for (sql in vistas) dbExecute(con, sql)
28
29 dbGetQuery(con, "SELECT * FROM vw_kpi_mensual") %>% head(3)

```

	anio	mes	nombre_mes	tickets	clientes	ventas	margen
1	2023	1	enero	98	80	82559.66	34415.12
2	2023	2	febrero	79	67	65899.34	29212.36
3	2023	3	marzo	91	72	90930.61	46572.20
			ticket_promedio				
1				842.45			
2				834.17			
3				999.24			

La vista entrega, mes por mes, tickets, clientes distintos, ventas, margen y ticket promedio (mostramos los tres primeros meses; en el caso práctico la usaremos completa). La vista ancha `vw_ventas_detalle` es útil para analistas que no quieren escribir *joins*: consultan una sola “tabla” con todo, por ejemplo `SELECT zona, SUM(total) FROM vw_ventas_detalle GROUP BY zona`.

5.9.2 Índices y planes de ejecución

Cuando filtras una tabla por una columna (`WHERE cliente_id = 5`), el motor, sin ayuda, tiene que revisar *todas* las filas: un **escaneo completo** (*full scan*). Un **índice** sobre esa columna le permite ir directo a las filas que buscas, como el índice alfabético al final de un libro. `EXPLAIN QUERY PLAN` te muestra cómo piensa resolver el motor una consulta, sin ejecutarla.

```
1 consulta_cliente <- "SELECT SUM(total) FROM fact_ventas
2                       WHERE cliente_id = 5"
3
4 # Antes del índice
5 dbGetQuery(con, paste("EXPLAIN QUERY PLAN", consulta_cliente))$detail
6
7 dbExecute(con, "CREATE INDEX idx_fact_cliente
8                ON fact_ventas(cliente_id)")
9
10 # Después del índice
11 dbGetQuery(con, paste("EXPLAIN QUERY PLAN", consulta_cliente))$detail
```

```
[1] "SCAN fact_ventas"
[1] 0
[1] "SEARCH fact_ventas USING INDEX idx_fact_cliente (cliente_id=?)"
```

Antes, el plan decía `SCAN fact_ventas`: recorrer la tabla completa. Después, `SEARCH fact_ventas USING INDEX idx_fact_cliente`: buscar directamente con el índice. Con 1 000 filas la diferencia es imperceptible, pero con 100 millones es la diferencia entre una consulta de minutos y una de milisegundos.

¿Qué columnas indexar? En un esquema en estrella, las llaves foráneas de la tabla de hechos (se usan en todos los *joins* y filtros) y las columnas por las que filtras con frecuencia. Guardamos la creación de vistas e índices en una función para reutilizarla:

```
1 indices <- c(
2   "CREATE INDEX IF NOT EXISTS idx_fact_cliente ON fact_ventas(cliente_id)",
3   "CREATE INDEX IF NOT EXISTS idx_fact_fecha ON fact_ventas(fecha_id)",
4   "CREATE INDEX IF NOT EXISTS idx_fact_producto
5     ON fact_ventas(producto_id)",
6   "CREATE INDEX IF NOT EXISTS idx_fact_tienda ON fact_ventas(tienda_id)",
7   "CREATE INDEX IF NOT EXISTS idx_fact_vendedor
8     ON fact_ventas(vendedor_id)"
```



```

9 )
10
11 crear_vistas_indices <- function(con) {
12   for (sql in c(indices, vistas)) dbExecute(con, sql)
13   invisible(TRUE)
14 }
15 for (sql in indices) dbExecute(con, sql)
16
17 dbGetQuery(con, "SELECT type, name FROM sqlite_master
18                 WHERE type IN ('index', 'view') ORDER BY type, name")

```

	type	name
1	index	idx_fact_cliente
2	index	idx_fact_fecha
3	index	idx_fact_producto
4	index	idx_fact_tienda
5	index	idx_fact_vendedor
6	index	sqlite_autoindex_dim_fecha_1
7	view	vw_kpi_mensual
8	view	vw_ventas_detalle

`sqlite_master` es el catálogo interno de SQLite: lista todas las tablas, vistas e índices. El índice `sqlite_autoindex_...` lo creó SQLite solo para la restricción **UNIQUE** de la fecha. **IF NOT EXISTS** evita el error si el índice ya existe.

▲ Cuidado

Los índices no son gratis: ocupan espacio y hacen un poco más lentas las inserciones, porque cada fila nueva también actualiza cada índice. No indexes todas las columnas “por si acaso”; indexa las que realmente usas para unir y filtrar. En cargas masivas, a veces conviene borrar los índices, cargar y volver a crearlos.

🔧 Hazlo tú

Escribe **EXPLAIN QUERY PLAN** para una consulta que sume las ventas de la tienda 3 (**WHERE tienda_id = 3**). ¿Usa el índice `idx_fact_tienda`?

5.10 Trabajar con datos grandes

Con los datos del curso todo cabe en memoria, pero en tu trabajo te topará con tablas de decenas de millones de filas. Tres principios te ahorrarán muchos problemas:

1. **Agrega en la base, no en R.** Si necesitas ventas por mes, pide a la base 12 filas, no 10 millones para agruparlas en R.
2. **Trae solo las columnas y filas que necesitas.** Nada de **SELECT *** ni `dbReadTable()` sobre

tablas grandes.

3. Si de verdad necesitas el detalle, tráelo por bloques.

Ejemplo práctico: Cuánto pesa traer datos de más

Compara el tamaño en memoria de tres formas de obtener las ventas mensuales.

```

1 # a) Todo el detalle, todas las columnas
2 todo <- dbGetQuery(con, "SELECT * FROM fact_ventas")
3 # b) Solo las columnas necesarias
4 dos_columnas <- dbGetQuery(con, "SELECT fecha_id, total FROM fact_ventas")
5 # c) Ya agregado en la base
6 agregado <- dbGetQuery(con, "
7   SELECT fecha_id / 100 AS anio_mes, SUM(total) AS ventas
8   FROM fact_ventas GROUP BY anio_mes")
9
10 tibble(forma = c("a) todo", "b) dos columnas", "c) agregado"),
11        filas = c(nrow(todo), nrow(dos_columnas), nrow(agregado)),
12        kb     = round(c(object.size(todo), object.size(dos_columnas),
13                        object.size(agregado)) / 1024, 1))

```

```

# A tibble: 3 x 3
  forma      filas kb
  <chr>      <int> <dbl>
1 a) todo      1000  79.6
2 b) dos columnas 1000  12.6
3 c) agregado     12   1

```

Traer toda la tabla ocupa 79.6 KB; solo las dos columnas necesarias, 12.6 KB (seis veces menos); y el resultado ya agregado, 1 KB con 12 filas. Con 1 000 filas da igual, pero escala: con 10 millones de filas, la opción a) ocuparía cientos de megabytes en tu computadora y la c) seguiría siendo de 12 filas. Por cierto, `fecha_id / 100` es un truco de la llave `AAAAMDD`: al dividir entre enteros, `20230113 / 100` da 202301, el año-mes.

Cuando necesitas procesar el detalle completo (por ejemplo, para exportarlo a otro sistema), usa `dbSendQuery()` + `dbFetch(n = ...)`. `dbSendQuery()` manda la consulta pero no trae datos; cada `dbFetch()` trae el siguiente bloque de `n` filas, y `dbHasCompleted()` te dice si ya no quedan más. Así la memoria solo contiene un bloque a la vez.

```

1 resultado <- dbSendQuery(con, "SELECT venta_id, metodo_pago, total
2                                FROM fact_ventas")
3 bloque <- 0
4 total_acumulado <- 0
5 while (!dbHasCompleted(resultado)) {
6   datos_bloque <- dbFetch(resultado, n = 300)    # 300 filas a la vez
7   bloque <- bloque + 1

```

```

8   total_acumulado <- total_acumulado + sum(datos_bloque$total)
9   cat(sprintf("Bloque %d: %d filas, acumulado $%s\n", bloque,
10              nrow(datos_bloque),
11              format(round(total_acumulado), big.mark = ",")))
12 }
13 dbClearResult(resultado)  # liberar el resultado en la base

```

```

Bloque 1: 300 filas, acumulado $298,877
Bloque 2: 300 filas, acumulado $584,379
Bloque 3: 300 filas, acumulado $868,613
Bloque 4: 100 filas, acumulado $959,993

```

Procesamos las 1 000 ventas en cuatro bloques (tres de 300 y uno de 100), y el acumulado final coincide con el total del año. Con 10 millones de filas usarías bloques de 100 000 y el mismo código funcionaría sin agotar la memoria. No olvides `dbClearResult()`: un resultado pendiente mantiene ocupados recursos en la base.

Consejo

Para cargas grandes en la otra dirección (de R a la base), escribe por bloques con `dbAppendTable()` dentro de una transacción: insertar un millón de filas en una sola transacción es muchísimo más rápido que en un millón de transacciones pequeñas. Y si tus datos analíticos crecen mucho, investiga **duckdb**: un motor analítico en archivo, tan fácil de usar como SQLite pero optimizado para agregar millones de filas, que funciona con **DBI** y **dbplyr** exactamente igual.

5.11 Conectarte a las bases de datos de tu empresa

Todo lo que aprendiste funciona igual con un servidor real: solo cambia `dbConnect()`. El código de esta sección es de referencia (usa servidores y credenciales ficticias), así que no se ejecuta en el libro.

5.11.1 SQL Server (y otros) con ODBC

ODBC (*Open Database Connectivity*) es un estándar para conectarse a casi cualquier base de datos mediante un *driver* instalado en el sistema operativo. Es la vía habitual para SQL Server y Oracle. Primero verifica qué drivers tienes instalados (si no aparece el de tu motor, pídelo a TI o descárgalo del sitio del fabricante):

```

1  install.packages("odbc")
2  odbc::odbcListDrivers()           # drivers ODBC instalados en tu equipo
3  odbc::odbcListDataSources()       # conexiones (DSN) ya configuradas

```

```

1  # SQL Server con autenticación de Windows (la más común en empresas)
2  con_sql <- dbConnect(
3    odbc::odbc(),
4    Driver   = "ODBC Driver 18 for SQL Server",
5    Server   = "srv-datos.miempresa.local",
6    Database = "VentasDW",
7    Trusted_Connection = "yes",
8    TrustServerCertificate = "yes"
9  )
10
11 # SQL Server con usuario y contraseña (tomados de variables de entorno)
12 con_sql <- dbConnect(
13   odbc::odbc(),
14   Driver   = "ODBC Driver 18 for SQL Server",
15   Server   = Sys.getenv("SQLSERVER_HOST"),
16   Database = "VentasDW",
17   UID      = Sys.getenv("SQLSERVER_USER"),
18   PWD      = Sys.getenv("SQLSERVER_PASS")
19 )
20
21 # Desde aquí, todo es igual: dbGetQuery(), tbl(), dbDisconnect()...
22 # Ojo con el dialecto: SQL Server usa TOP en lugar de LIMIT
23 dbGetQuery(con_sql, "SELECT TOP 10 * FROM dbo.fact_ventas")

```

5.11.2 PostgreSQL y MySQL/MariaDB

```

1  install.packages(c("RPostgres", "RMariaDB"))
2
3  # PostgreSQL
4  con_pg <- dbConnect(
5    RPostgres::Postgres(),
6    host     = Sys.getenv("PG_HOST"),
7    port     = 5432,
8    dbname   = "analitica",
9    user     = Sys.getenv("PG_USER"),
10   password = Sys.getenv("PG_PASS")
11 )
12
13 # MySQL o MariaDB
14 con_my <- dbConnect(
15   RMariaDB::MariaDB(),
16   host     = Sys.getenv("MYSQL_HOST"),
17   port     = 3306,
18   dbname   = "tienda_en_linea",
19   user     = Sys.getenv("MYSQL_USER"),
20   password = Sys.getenv("MYSQL_PASS")

```

```

21 )
22
23 dbListTables(con_pg)
24 dbDisconnect(con_pg)
25 dbDisconnect(con_my)

```

5.11.3 Credenciales: nunca en el script

Observa que en ningún ejemplo escribimos la contraseña. Un script con `password = "Ventas2024!"` termina tarde o temprano en un correo, en una carpeta compartida o en un repositorio de Git, y la contraseña queda expuesta para siempre (borrarla después no basta: Git guarda el historial). La práctica estándar es guardar las credenciales en **variables de entorno**, en el archivo `.Renviron` de tu carpeta de usuario, y leerlas con `Sys.getenv()`.

Abre el archivo con `usethis::edit_r_environ()` (o créalo a mano) y escribe una línea por variable, sin comillas ni espacios alrededor del `=`; los valores de este ejemplo son ficticios:

Archivo `.Renviron` (ejemplo ficticio)

```

1  SQLSERVER_HOST=srv-datos.miempresa.local
2  SQLSERVER_USER=analista_bi
3  SQLSERVER_PASS=cambia-esta-clave
4  PG_HOST=pg.miempresa.local
5  PG_USER=lector_bi
6  PG_PASS=otra-clave-ficticia

```

Reinicia R (**Session > Restart R**) para que lea el archivo. A partir de ahí, `Sys.getenv("SQLSERVER_USER")` devuelve `"analista_bi"` y tu script no contiene ningún secreto:

```

1  usethis::edit_r_environ()           # abre ~/.Renviron para editarlo
2  Sys.getenv("SQLSERVER_USER")        # "analista_bi"
3  nchar(Sys.getenv("SQLSERVER_PASS")) > 0 # comprobar sin imprimirla

```

▲ Cuidado

- Nunca subas `.Renviron` a Git: agrégalo a tu `.gitignore`.
- No imprimas la contraseña en la consola ni en un reporte.
- En RStudio también puedes pedir la contraseña al momento con `rstudioapi::askForPassword("Contraseña")`, útil en scripts interactivos.

En la empresa: Permisos y buenas prácticas en la empresa

- Pide a TI un usuario de **solo lectura** para analizar. Así es imposible que un `DELETE` accidental borre datos de producción.

- Consulta de preferencia una **réplica** o el *data warehouse*, no la base transaccional que usa el punto de venta.
- Evita consultas pesadas en horario pico, y usa **LIMIT**/ **TOP** mientras exploras.
- En aplicaciones Shiny con muchos usuarios no abras una conexión por cada consulta: usa el paquete **pool**, que mantiene un conjunto de conexiones abiertas y las reparte (`pool::dbPool()` en lugar de `dbConnect()`). Lo retomarás en el Módulo 6.

5.12 Caso práctico integral: el data warehouse de ventas

Ejemplo práctico: Del CSV al tablero, pasando por el data warehouse

La dirección te pide dejar lista una base analítica que cualquiera pueda consultar, y responder con ella seis preguntas para la junta mensual:

1. ¿Cómo evolucionaron las ventas y el margen mes a mes, y cuánto crecimos contra el mes anterior?
2. ¿Cuáles son los productos que más venden y cuánto margen dejan?
3. ¿Quiénes son los clientes más valiosos?
4. ¿Qué tiendas son más productivas por metro cuadrado?
5. ¿Quiénes son los dos mejores vendedores de cada zona?
6. ¿Se compra distinto en fin de semana?

5.12.1 Paso 1: construir la base desde cero

Gracias a que escribimos todo como funciones, reconstruir el *data warehouse* completo es un script corto. Cerramos la conexión actual, borramos el archivo y ejecutamos cada etapa. Esta vez la base solo contendrá el esquema en estrella (sin las tablas crudas del principio): es lo que publicarías para los usuarios de negocio.

```

1 dbDisconnect(con)
2
3 reconstruir_dw <- function(ruta_bd, archivos_ventas) {
4   unlink(ruta_bd)                                # empezar desde cero
5   con <- dbConnect(RSQLite::SQLite(), ruta_bd)
6   crear_esquema(con)                               # tablas y llaves
7   cargar_dimensiones(con, preparar_dimensiones()) # dimensiones
8   for (archivo in archivos_ventas) {               # hechos (ETL)
9     ejecutar_etl(con, archivo)
10  }
11  crear_vistas_indices(con)                         # vistas e índices

```

```

12   con
13 }
14
15 con <- reconstruir_dw("datos/bi_retail.sqlite",
16                      c("datos/entrada/ventas_2023_s1.csv",
17                        "datos/entrada/ventas_2023_s2.csv"))
18 dbListTables(con)

```

```

ventas_2023_s1.csv -> OK: 511 leídas, 511 nuevas
ventas_2023_s2.csv -> OK: 661 leídas, 489 nuevas
[1] "dim_cliente"      "dim_fecha"        "dim_producto"
[4] "dim_tienda"       "dim_vendedor"     "fact_ventas"
[7] "log_etl"          "sqlite_sequence"  "vw_kpi_mensual"
[10] "vw_ventas_detalle"

```

En segundos tienes la base completa: cinco dimensiones, la tabla de hechos con sus 1 000 ventas, la bitácora y dos vistas. Si mañana llega un archivo nuevo, solo hay que llamar a `ejecutar_etl()`.

5.12.2 Paso 2: responder las preguntas con SQL

Pregunta 1. Ventas, margen y crecimiento por mes

```

1 kpi_mensual <- dbGetQuery(con, "
2   SELECT mes, nombre_mes, tickets, ventas,
3           ROUND(100.0 * margen / ventas, 1) AS margen_pct,
4           ROUND(100.0 * (ventas - LAG(ventas) OVER (ORDER BY mes)) /
5             LAG(ventas) OVER (ORDER BY mes), 1) AS crec_pct
6   FROM vw_kpi_mensual
7   ORDER BY anio, mes
8 ")
9 kpi_mensual

```

	mes	nombre_mes	tickets	ventas	margen_pct	crec_pct
1	1	enero	98	82559.66	41.7	NA
2	2	febrero	79	65899.34	44.3	-20.2
3	3	marzo	91	90930.61	51.2	38.0
4	4	abril	71	57037.74	34.7	-37.3
5	5	mayo	88	93910.21	52.8	64.6
6	6	junio	84	88509.12	47.5	-5.8
7	7	julio	83	86456.05	47.6	-2.3
8	8	agosto	75	78929.49	42.2	-8.7
9	9	septiembre	86	68855.87	42.8	-12.8
10	10	octubre	74	70004.10	45.3	1.7
11	11	noviembre	80	80506.89	45.4	15.0

12	12	diciembre	91 96393.61	52.8	19.7
----	----	-----------	-------------	------	------

Las ventas oscilan entre \$57 038 (abril) y \$96 394 (diciembre), y el margen porcentual va de 34.7 % en abril a 52.8 % en mayo y diciembre. Abril fue malo por partida doble: vendió 37.3 % menos que marzo *y* con menor margen; mayo rebotó 64.6 % y el año cerró con dos meses de crecimiento (15.0 % en noviembre y 19.7 % en diciembre). La vista con los KPIs más una función de ventana resolvieron la pregunta en una sola consulta. ¿Podemos confiar en estas cifras? Una buena práctica es *reconciliar*: calcular lo mismo por otro camino y comprobar que coincide. Hagámoslo con **dplyr** directamente sobre el CSV original, sin pasar por la base:

```
1 kpi_dplyr <- read_csv("datasets/transacciones.csv",
2                       show_col_types = FALSE) %>%
3   group_by(mes = month(fecha)) %>%
4   summarise(tickets = n(),
5             ventas = round(sum(cantidad * precio_venta), 2),
6             .groups = "drop")
7
8 comparacion <- kpi_mensual %>%
9   select(mes, tickets, ventas) %>%
10  inner_join(kpi_dplyr, by = "mes", suffix = c("_sql", "_dplyr"))
11
12 all.equal(comparacion$ventas_sql, comparacion$ventas_dplyr)
13 all(comparacion$tickets_sql == comparacion$tickets_dplyr)
```

```
[1] TRUE
[1] TRUE
```

Las ventas y los tickets de los 12 meses coinciden exactamente: el ETL no perdió ni duplicó nada. Con los datos ya validados, la gráfica para la junta es inmediata (figura 5.5):

```
1 color_principal <- "#2a78d6"
2
3 grafica_mensual <- kpi_mensual %>%
4   mutate(nombre_mes = factor(nombre_mes, levels = nombre_mes)) %>%
5   ggplot(aes(x = nombre_mes, y = ventas, group = 1)) +
6   geom_line(color = color_principal, linewidth = 0.8) +
7   geom_point(color = color_principal, size = 2) +
8   scale_y_continuous(labels = scales::dollar,
9                     limits = c(0, NA)) +
10  labs(title = "Ventas mensuales 2023",
11       subtitle = "Fuente: vista vw_kpi_mensual del data warehouse",
12       x = NULL, y = "Ventas") +
13  theme_minimal() +
14  theme(panel.grid.minor = element_blank(),
15        axis.text.x = element_text(angle = 45, hjust = 1))
16 grafica_mensual
```

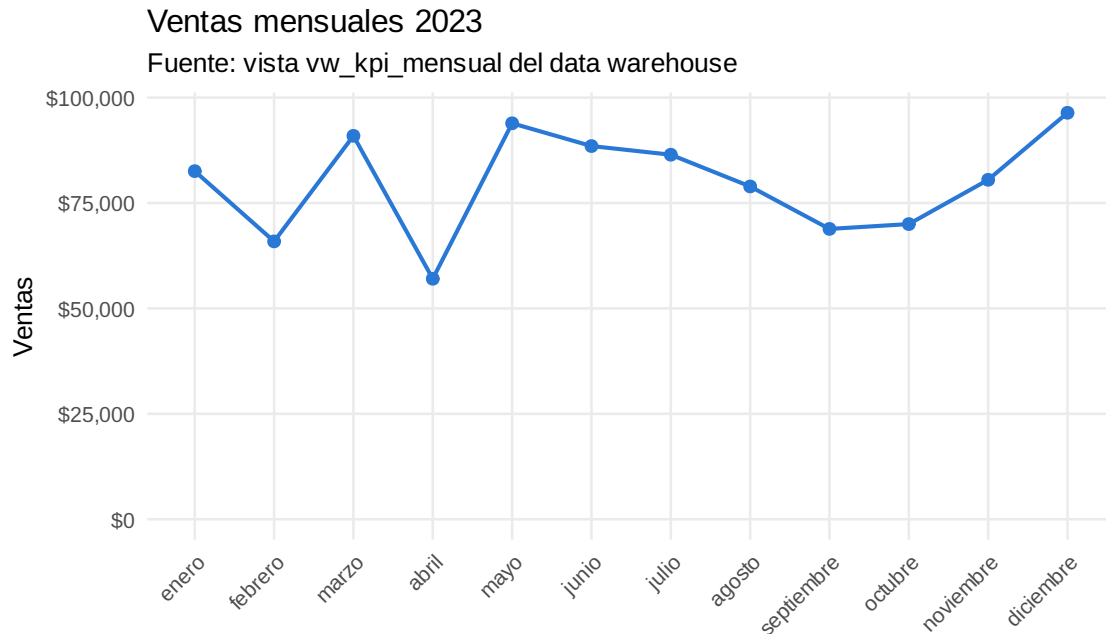



Figura 5.5. Ventas mensuales calculadas con SQL a partir del data warehouse.

Pregunta 2. Productos que más venden y su margen

```

1 dbGetQuery(con, "
2   SELECT p.nombre_producto, p.categoria,
3         SUM(f.cantidad)                AS unidades,
4         ROUND(SUM(f.total), 0)         AS ventas,
5         ROUND(SUM(f.margen), 0)        AS margen,
6         ROUND(100.0 * SUM(f.margen) / SUM(f.total), 1) AS margen_pct
7   FROM fact_ventas AS f
8   JOIN dim_producto AS p ON f.producto_id = p.producto_id
9   GROUP BY p.producto_id
10  ORDER BY ventas DESC
11  LIMIT 5
12 ")

```

	nombre_producto	categoria	unidades	ventas	margen	margen_pct
1	Producto 06	Juguetes	82	34028	27095	79.6
2	Producto 19	Deportes	85	28368	7557	26.6
3	Producto 31	Alimentos	74	25430	23622	92.9
4	Producto 47	Automotriz	82	24527	20017	81.6
5	Producto 46	Automotriz	74	24470	11333	46.3

El Producto 06 (Juguetes) lidera en ventas y además deja un margen de 79.6 %. En cambio, el Producto 19 (Deportes), segundo en ventas, apenas deja 26.6 %: vende mucho pero aporta poco. Un

producto que vende mucho no necesariamente es el más rentable.

```

1 # ¿Hay productos que se venden con pérdida?
2 dbGetQuery(con, "
3     SELECT p.nombre_producto, p.costo,
4           ROUND(AVG(f.precio_venta), 2) AS precio_venta_prom,
5           ROUND(SUM(f.margen), 0) AS margen_total
6     FROM fact_ventas AS f
7     JOIN dim_producto AS p ON f.producto_id = p.producto_id
8     GROUP BY p.producto_id
9     HAVING SUM(f.margen) < 0
10    ORDER BY margen_total
11 ")

```

	nombre_producto	costo	precio_venta_prom	margen_total
1	Producto 13	392.63	257.58	-8602
2	Producto 35	353.50	230.62	-6530
3	Producto 41	389.86	294.97	-6046
4	Producto 12	368.62	296.11	-4332
5	Producto 02	399.12	304.00	-4137
6	Producto 29	391.67	355.31	-2258
7	Producto 10	283.26	260.12	-1282

Siete productos se vendieron con pérdida en el año: su costo unitario es mayor que el precio promedio al que se vendieron. El Producto 13, por ejemplo, cuesta \$392.63 y se vendió en promedio a \$257.58, con una pérdida acumulada de \$8 602. Es el hallazgo de calidad de datos que ya conocías (precios de venta generados sin relación con el costo), pero en una empresa real sería una alerta roja para el área comercial.

Pregunta 3. Clientes más valiosos

```

1 dbGetQuery(con, "
2     SELECT c.nombre, c.ciudad, c.segmento,
3           CASE c.es_premium WHEN 1 THEN 'Sí' ELSE 'No' END AS premium,
4           COUNT(*) AS compras,
5           ROUND(SUM(f.total), 0) AS gasto,
6           ROUND(SUM(f.margen), 0) AS margen
7     FROM fact_ventas AS f
8     JOIN dim_cliente AS c ON f.cliente_id = c.cliente_id
9     GROUP BY c.cliente_id
10    ORDER BY gasto DESC
11    LIMIT 5
12 ")

```

	nombre	ciudad	segmento	premium	compras	gasto	margen
1	Pedro Cruz	Toluca	Adulto	No	13	14680	5882
2	Pedro Morales	León	Senior	No	9	12504	8669
3	Carmen Jiménez	Guadalajara	Adulto	No	9	11848	5001
4	Sofia González	Cancún	Maduro	No	12	11676	6778
5	Alberto Reyes	Guadalajara	Maduro	No	13	11356	5905

Pedro Cruz, de Toluca, es el cliente más valioso por gasto (\$14 680 en 13 compras); es el cliente 5 que ya habíamos visto. Pero si ordenamos por margen, Pedro Morales, de León, deja más (\$8 669 con solo 9 compras). Un dato curioso: ninguno de los cinco es cliente premium, una pista para revisar los criterios del programa.

Pregunta 4. Productividad de las tiendas

```

1 dbGetQuery(con, "
2   SELECT t.nombre_tienda, t.tipo, t.tamano_m2,
3         ROUND(SUM(f.total), 0)           AS ventas,
4         ROUND(SUM(f.total) / t.tamano_m2, 1) AS ventas_m2,
5         ROUND(AVG(f.total), 0)           AS ticket_prom
6   FROM fact_ventas AS f
7   JOIN dim_tienda AS t ON f.tienda_id = t.tienda_id
8   GROUP BY t.tienda_id
9   ORDER BY ventas_m2 DESC
10 ")

```

	nombre_tienda	tipo	tamano_m2	ventas	ventas_m2	ticket_prom
1	Sucursal Express	Express	300	102791	342.6	952
2	Sucursal Outlet	Outlet	350	114377	326.8	1079
3	Sucursal Este	Estándar	400	88420	221.0	941
4	Sucursal Norte	Estándar	450	89024	197.8	899
5	Sucursal Centro	Principal	500	88414	176.8	1016
6	Sucursal Sur	Estándar	600	105305	175.5	966
7	Sucursal Plaza B	Estándar	480	80236	167.2	944
8	Sucursal Oeste	Estándar	550	84672	153.9	891
9	Sucursal Plaza A	Principal	700	105216	150.3	965
10	Sucursal Premium	Premium	800	101538	126.9	940

Por metro cuadrado, las tiendas pequeñas ganan: la Sucursal Express (300 m²) vende \$342.6 por m² y el Outlet \$326.8, mientras que la Sucursal Premium, la más grande (800 m²), queda al final con \$126.9. Las ventas totales son parecidas entre tiendas, así que el tamaño no se traduce en más ventas: una conclusión valiosa para decidir el tamaño de las próximas aperturas.

Pregunta 5. Los dos mejores vendedores de cada zona

Aquí combinamos una CTE con una función de ventana particionada por zona. Un vendedor puede vender en tiendas de varias zonas, así que el ranking es “por lo que vendió en esa zona”.

```

1 dbGetQuery(con, "
2   WITH ventas_vendedor_zona AS (
3     SELECT t.zona, v.nombre AS vendedor, SUM(f.total) AS ventas
4     FROM fact_ventas AS f
5     JOIN dim_tienda AS t ON f.tienda_id = t.tienda_id
6     JOIN dim_vendedor AS v ON f.vendedor_id = v.vendedor_id
7     GROUP BY t.zona, v.vendedor_id
8   ),
9   ranking AS (
10    SELECT zona, vendedor, ROUND(ventas, 0) AS ventas,
11           RANK() OVER (PARTITION BY zona ORDER BY ventas DESC) AS lugar
12    FROM ventas_vendedor_zona
13  )
14  SELECT * FROM ranking
15  WHERE lugar <= 2
16  ORDER BY zona, lugar
17 ")

```

	zona	vendedor	ventas	lugar
1	Centro	Isabel García	21198	1
2	Centro	Sofia Rivera	17066	2
3	Este	Fernando Pérez	8881	1
4	Este	Pedro Morales	7589	2
5	Norte	Ricardo Ramírez	22238	1
6	Norte	Marta González	21909	2
7	Oeste	Isabel García	11709	1
8	Oeste	Juan Sánchez	10093	2
9	Sur	Marta Gómez	11526	1
10	Sur	Marta Rivera	9124	2

Ricardo Ramírez lidera la zona Norte (\$22 238) y Isabel García aparece como la mejor vendedora *en dos zonas* (Centro y Oeste), señal de que atiende en varias sucursales. En la zona Este, con una sola tienda, las cifras son mucho menores. Recuerda el hallazgo anterior: varios de estos “vendedores” no pertenecen al departamento de Ventas.

Pregunta 6. Fin de semana contra entre semana

```

1 dbGetQuery(con, "
2   SELECT CASE d.es_fin_semana WHEN 1 THEN 'Fin de semana'
3             ELSE 'Entre semana' END AS tipo_dia,
4             COUNT(DISTINCT d.fecha_id)           AS dias,
5             COUNT(*)                             AS tickets,
6             ROUND(1.0 * COUNT(*) / COUNT(DISTINCT d.fecha_id), 2)
7                                                     AS tickets_por_dia,
8             ROUND(AVG(f.total), 2)               AS ticket_prom
9   FROM fact_ventas AS f
10  JOIN dim_fecha AS d ON f.fecha_id = d.fecha_id
11  GROUP BY tipo_dia
12 ")

```

	tipo_dia	dias	tickets	tickets_por_dia	ticket_prom
1	Entre semana	246	724	2.94	961.49
2	Fin de semana	99	276	2.79	956.07

Hubo ventas en 246 días entre semana y 99 de fin de semana (345 días en total: 20 días del año no registraron ninguna venta). Entre semana se venden 2.94 tickets por día contra 2.79 en fin de semana, y el ticket promedio es prácticamente igual (\$961 contra \$956). No hay un patrón de fin de semana marcado. Este tipo de pregunta es trivial gracias a la dimensión de fechas: el atributo `es_fin_semana` ya está calculado y cualquier analista puede usarlo sin saber manipular fechas.

En la empresa: Qué entregarías en la junta

Con estas consultas tienes el material de un reporte ejecutivo completo, y la base `bi_retail.sqlite` queda como la fuente única de la verdad: el Módulo 6 construirá un tablero que lee de ella y el Módulo 8 un reporte automático. Lo más valioso no son las cifras de hoy, sino que el próximo mes basta ejecutar `ejecutar_etl()` con el archivo nuevo y volver a correr las consultas.

** Hazlo tú**

Agrega una séptima pregunta: ¿qué método de pago predomina en cada zona? Usa `vw_ventas_detalle` y `ROW_NUMBER()` particionado por zona.

5.13 Errores comunes

Para ver los mensajes reales sin detener el script, usamos esta pequeña función que ejecuta una expresión y, si falla, imprime el mensaje de error (o de advertencia) en lugar de detenerse:

```

1 mostrar_error <- function(expr) {
2   tryCatch(expr,
3     error    = function(e) cat("Error:", conditionMessage(e), "\n"),
4     warning  = function(w) cat("Aviso:", conditionMessage(w), "\n"))
5 }

```

Tablas, columnas y llaves

```

1 mostrar_error(dbGetQuery(con, "SELECT * FROM ventas_2024"))
2 mostrar_error(dbGetQuery(con, "SELECT totl FROM fact_ventas"))
3 mostrar_error(dbWriteTable(con, "dim_tienda", dimensiones$dim_tienda))
4 mostrar_error(dbAppendTable(con, "dim_tienda", dimensiones$dim_tienda))

```

```

Error: no such table: ventas_2024
Error: no such column: totl
Error: Table dim_tienda exists in database, and both overwrite and append are FALSE
Error: UNIQUE constraint failed: dim_tienda.tienda_id

```

- **no such table / no such column:** la tabla o la columna no existe, casi siempre por un error de dedo o porque estás conectado a otra base. Revisa con `dbListTables(con)` y `dbListFields(con, "tabla")`. Recuerda que, tras reconstruir la base, las tablas crudas (`transacciones`, `clientes`...) ya no existen.
- **La tabla ya existe:** `dbWriteTable()` se niega a pisarla. `overwrite = TRUE` la reemplaza (perdiendo llaves y reglas si la creaste con `CREATE TABLE!`) y `append = TRUE` agrega filas. Para tablas diseñadas por ti, usa `dbAppendTable()`.
- **UNIQUE constraint failed:** intentaste cargar filas cuya llave primaria ya existe (típico al correr dos veces un script). Usa carga incremental o borra primero lo que vas a recargar.

Fechas convertidas en números

Si ves fechas como 19370 o años como -4672, escribiste un `Date` de R directamente en SQLite (sección 5.4.6). Convierte las fechas a texto con `format(fecha, "%Y-%m-%d")` antes de escribirlas y con `as.Date()` al leerlas.

Comillas simples contra comillas dobles

En SQL estándar, el texto va entre comillas *simples* (`'CDMX'`) y las comillas *dobles* son para nombres de columnas o tablas. SQLite, por compatibilidad, acepta comillas dobles para texto si no encuentra una columna con ese nombre, lo que produce errores silenciosos:

```

1 dbGetQuery(con, "SELECT COUNT(*) AS n FROM dim_cliente
2               WHERE ciudad = 'CDMX'")      # correcto: 27
3 dbGetQuery(con, "SELECT COUNT(*) AS n FROM dim_cliente
4               WHERE ciudad = \"ciudad\"")    # ¡columna consigo misma!

```

```

      n
1 27
      n
1 200

```

La segunda consulta pretendía buscar clientes de una ciudad llamada “ciudad”, pero **"ciudad"** se interpretó como la *columna* ciudad, que siempre es igual a sí misma: devolvió los 200 clientes. En PostgreSQL o SQL Server, **"CDMX"** directamente marcaría error de columna inexistente. Usa siempre comillas simples para texto (y **params** para valores variables).

Olvidar cerrar conexiones y resultados

```

1 con_temporal <- dbConnect(RSQLite::SQLite(), ":memory:")
2 dbDisconnect(con_temporal)
3 mostrar_error(dbGetQuery(con_temporal, "SELECT 1"))

```

```
Error: Invalid or closed connection
```

Usar una conexión ya cerrada da **Invalid or closed connection**: vuelve a conectarte. El error contrario (nunca cerrar) no da error inmediato, pero R te avisa más tarde con un mensaje como *call dbDisconnect() when finished working with a connection*, y en un servidor acumula conexiones abiertas. Usa **dbDisconnect()** al final de cada script y **on.exit()** dentro de las funciones. Del mismo modo, cada **dbSendQuery()** necesita su **dbClearResult()**.

Tabla 5.4. Resumen de errores frecuentes con bases de datos.

Mensaje o síntoma	Causa y solución
<code>no such table: X</code>	La tabla no existe o estás en otra base. Revisa <code>dbListTables()</code> .
<code>no such column: X</code>	Error de dedo o columna de otra tabla; revisa <code>dbListFields()</code> y los alias.
<code>ambiguous column name: X</code>	La columna existe en dos tablas del JOIN: antepón el alias (<code>t.producto_id</code>).
<code>Table X exists in database...</code>	Usa <code>overwrite</code> , <code>append</code> o <code>dbAppendTable()</code> .
<code>UNIQUE constraint failed</code>	Llave repetida: carga incremental o <code>INSERT OR IGNORE</code> .
<code>FOREIGN KEY constraint failed</code>	Un valor no existe en la dimensión: carga primero las dimensiones o corrige el dato.
Fechas como números	Convierte <code>Date</code> a texto ISO antes de escribir.
<code>Invalid or closed connection</code>	La conexión se cerró: vuelve a conectarte.
Resultados “demasiado buenos”	Comillas dobles en lugar de simples, o un JOIN que multiplica filas.

Terminamos cerrando la conexión principal del módulo:

```
1 dbDisconnect(con)
```

Resumen del módulo

- Las empresas guardan sus datos en bases de datos por volumen, integridad, concurrencia y seguridad. Las OLTP operan el negocio; los *data warehouses* (OLAP) sirven para analizarlo.
- **DBI** ofrece las mismas funciones para cualquier motor: `dbConnect()`, `dbWriteTable()`, `dbGetQuery()`, `dbExecute()`, `dbDisconnect()`. Solo cambia el driver.
- En SQLite guarda las fechas como texto `'AAAA-MM-DD'`; si no, se vuelven números.
- SQL: `SELECT`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `JOIN`, subconsultas, CTE y funciones de ventana responden casi cualquier pregunta.
- Nunca pegues valores del usuario con `paste()`: usa `params`, `glue_sql()` o `sqlInterpolate()`.
- **dbplyr** traduce dplyr a SQL; `show_query()` te enseña el SQL y `collect()` trae el resultado.
- Un data warehouse en estrella tiene una tabla de hechos y dimensiones con llaves declaradas, vistas para los KPIs e índices en las llaves.
- Un ETL confiable es idempotente (sin duplicados), atómico (transacciones) y trazable (bitácora).
- Con datos grandes: agrega en la base, trae solo lo necesario y usa `dbFetch()` por bloques.
- Credenciales en `.Renviron`, nunca en el script; usuarios de solo lectura para analizar.

SQL	dplyr
SELECT col1, col2	select(col1, col2)
SELECT expr AS nueva	mutate(nueva = expr)
WHERE cond	filter(cond)
ORDER BY col DESC	arrange(desc(col))
LIMIT 5	head(5) / slice_head(n = 5)
GROUP BY g + SUM(x)	group_by(g) + summarise(sum(x))
HAVING cond	filter() después de summarise()
COUNT(DISTINCT x)	n_distinct(x)
CASE WHEN ... END	case_when()
INNER JOIN / LEFT JOIN	inner_join() / left_join()
LEFT JOIN ... WHERE b.id IS NULL	anti_join()
WITH paso AS (...)	guardar un resultado intermedio
ROW_NUMBER() OVER (PARTITION BY g ...)	group_by(g) + row_number()
SUM(x) OVER (ORDER BY t)	cumsum(x)
LAG(x) OVER (ORDER BY t)	lag(x)

5.14 Ejercicios

Resuelve cada ejercicio en un script nuevo. Para no depender de la base del capítulo, trabaja con una base en memoria (`dbConnect(RSQLite::SQLite(), ":memory:")`) y carga solo las tablas que necesites. Salvo que se indique otra cosa, responde con SQL escrito a mano mediante `dbGetQuery()`.

Ejercicio 5.1: Tu primera base de datos

Nivel: **Básico**

Crea una base en memoria, carga `productos.csv` en una tabla llamada `productos` y muestra sus tablas y columnas con `dbListTables()` y `dbListFields()`. Después, con SQL, calcula cuántos productos hay por categoría y el precio de catálogo promedio, ordenado de más a menos productos. Cierra la conexión al final.

🔗 Solución en la página 654

Ejercicio 5.2: Alerta de inventario

Nivel: **Básico**

El área de compras necesita la lista de productos *activos* cuyo `stock_actual` está por debajo del `stock_minimo`. Muestra nombre, categoría, proveedor, stock actual, stock mínimo y las piezas faltantes (`stock_minimo - stock_actual`), ordenado de mayor a menor faltante. ¿Cuántos productos están en alerta y qué proveedor aparece más?

🔗 Solución en la página 655

Ejercicio 5.3: Nómina por departamentoNivel: **Básico**

Con `empleados.csv`, calcula por departamento: número de empleados, salario mensual promedio y salario máximo. Muestra solo los departamentos con al menos 15 empleados (**HAVING**), ordenados por salario promedio descendente.

 Solución en la página 656**Ejercicio 5.4: Ventas por ciudad y productos sin ventas en diciembre**Nivel: **Intermedio**

Carga `transacciones.csv`, `tiendas.csv` y `productos.csv` (con fechas como texto ISO). (a) Con un **INNER JOIN**, calcula por ciudad de la tienda el número de tickets, las ventas y el ticket promedio. (b) Con un **LEFT JOIN**, encuentra los productos que no tuvieron ninguna venta en diciembre de 2023.

 Solución en la página 657**Ejercicio 5.5: Fechas bien guardadas**Nivel: **Intermedio**

Carga `ventas_retail.csv` dos veces en una base en memoria: una tabla `ventas_mal` con la fecha tal como la lee `read_csv()` y otra `ventas` con la fecha como texto ISO. Demuestra con `typeof()` y `strftime()` la diferencia. Luego, con la tabla correcta, calcula por trimestre (a partir de la fecha) las ventas totales, el descuento promedio y el número de días con descuento.

 Solución en la página 658**Ejercicio 5.6: Consulta segura de un cliente**Nivel: **Intermedio**

Escribe una función `ventas_cliente(con, cliente, desde, hasta)` que devuelva el número de compras y el gasto de un cliente en un periodo, usando una consulta parametrizada con marcadores con nombre. Pruébala con el cliente 5 en el primer semestre de 2023 y en el segundo. Después demuestra que el texto `"5 OR 1=1"` no logra traer las compras de todos los clientes.

 Solución en la página 659**Ejercicio 5.7: Lo mismo con dbplyr**Nivel: **Intermedio**

Con **dbplyr**, calcula para cada método de pago y trimestre el número de tickets y las ventas, a partir de la tabla `transacciones`. Muestra el SQL generado con `show_query()`, trae el resultado con `collect()` y comprueba con `all.equal()` que coincide con la misma consulta escrita a mano en SQL.

 Solución en la página 660

Ejercicio 5.8: Rankings y Pareto con funciones de ventanaNivel: **Avanzado**

(a) Encuentra los dos clientes con mayor gasto de cada ciudad (ciudad del cliente) usando `RANK()` con `PARTITION BY`; muestra solo Monterrey, Guadalajara y CDMX. (b) Construye la curva de Pareto de productos: ordena los productos por ventas y calcula con `SUM() OVER` el porcentaje acumulado. ¿Cuántos productos generan el 80 % de las ventas?

 Solución en la página 662**Ejercicio 5.9: Una estrella en miniatura**Nivel: **Avanzado**

A partir de `ventas_retail.csv`, `productos.csv` y `tiendas.csv`, crea en una base en memoria un esquema en estrella pequeño: `dim_producto`, `dim_tienda` y `fact_ventas_diarias` (llave primaria `fecha`, llaves foráneas a ambas dimensiones, `CHECK (cantidad > 0)`). Carga los datos con `dbAppendTable()`, crea una vista con las ventas por trimestre y tipo de tienda, y muestra con `EXPLAIN QUERY PLAN` el efecto de un índice sobre `tienda_id`.

 Solución en la página 663**Ejercicio 5.10: ETL incremental con bitácora**Nivel: **Reto**

Construye un ETL para `ventas_retail.csv` en una base en memoria: una tabla `fact_ventas_diarias` con llave primaria `fecha` (texto) y una tabla `log_etl`. Escribe las funciones `extraer_ventas()`, `transformar_ventas()`, `cargar_ventas()` (que inserte desde una tabla temporal solo las fechas que aún no existen) y `ejecutar_etl()` (con transacción y registro en la bitácora, también de los errores). Simula tres lotes: enero-junio, abril-septiembre (traslapado) y octubre-diciembre; ejecuta además un lote repetido y un lote con una cantidad negativa que debe ser rechazado por la regla `CHECK`. Muestra la bitácora y verifica que la tabla termina con exactamente 365 filas y el mismo total de ventas que el CSV.

 Solución en la página 665

Dashboards interactivos con Shiny

Al terminar este módulo podrás...

- Explicar qué hace bueno a un dashboard de BI (audiencia, preguntas, KPIs, filtros) y decidir cuándo conviene Shiny y cuándo Power BI o Tableau.
- Entender la anatomía de una app Shiny (`ui`, `server`, `shinyApp()`) y ejecutar las tres apps completas que acompañan este módulo.
- Usar las entradas y salidas más comunes y saber qué función `render...()` corresponde a cada `...Output()`.
- Dominar la **reactividad**: `reactive()`, `observeEvent()`, `eventReactive()`, `req()`, `validate()` e `isolate()`, y reconocer sus errores típicos.
- Organizar la pantalla con `sidebarLayout()`, `fluidRow()`, pestañas y **shinydashboard** (`valueBox()`, `box()`, menús).
- Agregar tablas interactivas con **DT**, botones de descarga en CSV y Excel, y gráficas interactivas con **plotly**.
- Aplicar buenas prácticas profesionales: separar la lógica de la interfaz, módulos, caché, pruebas con `testServer()` y publicación segura.

Llevas cinco módulos respondiendo preguntas de negocio con R. Cada vez que la directora comercial te pide algo (“¿cuánto vendimos en Monterrey en el segundo trimestre?”, “¿y solo en Electrónica?”, “¿y si quitamos el Outlet?”) tú abres RStudio, cambias un `filter()`, vuelves a correr el script y le mandas una gráfica nueva por correo. Funciona, pero no escala: tú te conviertes en el cuello de botella y ella espera horas por una respuesta que le tomaría segundos si pudiera mover los filtros por sí misma.

La solución es un **dashboard interactivo**: una página web donde la directora elige el periodo, la tienda y la categoría, y las cifras y gráficas se actualizan al instante. Con el paquete **shiny** puedes construirlo *con el mismo R que ya sabes*: tus `filter()`, `summarise()` y `ggplot()` siguen siendo el corazón; Shiny solo añade la “piel” (botones, menús, cajas) y el “sistema nervioso” (la reactividad) que conecta lo que el usuario toca con lo que R calcula.

En este módulo construirás tres apps de complejidad creciente, que vienen completas en la carpeta `latex/codigo/apps/` del curso: una calculadora de precios (tu primera app), un explorador de ventas con filtros y un dashboard ejecutivo con cinco pestañas, indicadores, tablas y botones de descarga. Al final tendrás la base del dashboard del proyecto integrador (Módulo 9).

6.1 ¿Qué es un dashboard de BI y qué lo hace bueno?

Concepto clave: Dashboard de BI

Un **dashboard** (tablero de control) es una pantalla que reúne los indicadores más importantes de un negocio para que una audiencia concreta pueda *monitorear* cómo va y *decidir* qué hacer, de un vistazo. Un **KPI** (*Key Performance Indicator*, indicador clave de desempeño) es una cifra que resume algo que importa: ventas, ticket promedio, clientes activos, margen. Un dashboard *interactivo* además deja filtrar y profundizar sin pedirle nada al analista.

Un dashboard no es “poner todas las gráficas que tengo en una pantalla”. Los buenos comparten cinco rasgos:

1. **Tiene una audiencia definida.** El director general quiere cinco cifras y una tendencia; el gerente de tienda quiere detalle de su sucursal; el analista quiere descargar datos. Un dashboard para “todos” no le sirve bien a nadie.
2. **Responde preguntas concretas.** Antes de programar, escribe las 3–5 preguntas que debe contestar (“¿vamos arriba o abajo del mes pasado?”, “¿qué tienda jala hacia abajo?”). Cada elemento de la pantalla debe responder a una de ellas.
3. **Pone los KPIs arriba.** La vista sigue un patrón en “Z”: lo primero que se lee es la esquina superior izquierda. Ahí van las cifras principales en tarjetas grandes; debajo, las tendencias; al final, el detalle (tablas).
4. **Agrupar los filtros en un solo lugar.** Todos los filtros juntos (en una barra lateral o en una fila arriba), con valores por defecto sensatos (“todo el año”, “todas las tiendas”) y que afecten a toda la página por igual.
5. **Menos es más.** Pocas gráficas, bien elegidas, con un solo color salvo que el color signifique algo, ejes con formato de negocio (\$, %, miles) y títulos que digan qué mirar. Lo que aprendiste en el Módulo 3 aplica aquí multiplicado.

La figura 6.1 resume esa jerarquía visual. La usaremos como plantilla para el dashboard ejecutivo de la sección 6.7.

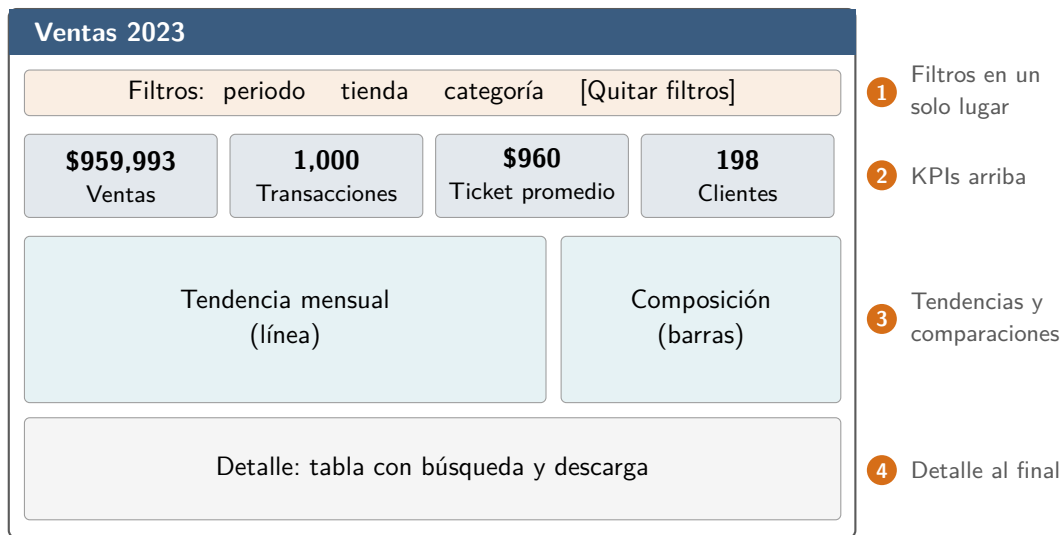


Figura 6.1. Jerarquía de un buen dashboard: filtros juntos, KPIs arriba, tendencias en medio y detalle al final.

En la empresa: Tres dashboards, tres audiencias

En una cadena de retail es común tener al menos tres vistas de los mismos datos: (1) un *dashboard ejecutivo* para la dirección, con 4–6 KPIs, la tendencia contra el año anterior y el ranking de tiendas; (2) un *dashboard operativo* para gerentes de tienda, filtrado a su sucursal, con ventas del día, inventario bajo y desempeño por vendedor; y (3) un *explorador analítico* para el equipo de BI, con muchos filtros, tablas detalladas y descarga de datos. Mismo origen, distinto diseño.

6.1.1 Shiny frente a Power BI y Tableau

Seguramente conoces herramientas como Power BI o Tableau, que permiten armar dashboards arrastrando campos con el ratón. ¿Por qué aprender Shiny? No se trata de que una herramienta sea mejor que otra, sino de elegir la adecuada para cada caso (tabla 6.1).

Tabla 6.1. Cuándo conviene cada herramienta.

Aspecto	Power BI / Tableau	Shiny (R)
Curva de aprendizaje	Baja: arrastrar y soltar	Media: hay que programar
Tiempo al primer tablero	Minutos	Una o dos horas
Lógica compleja (modelos, simulaciones, pronósticos)	Limitada o con extensiones	Todo lo que R sabe hacer: forecast , modelos de Machine Learning, optimización
Reproducibilidad y control de versiones	Archivos binarios difíciles de comparar	Código de texto: Git, revisiones, pruebas automáticas
Diseño a la medida	Dentro de lo que ofrece la herramienta	Total (HTML, CSS, JavaScript si hace falta)
Costo	Licencia por usuario	Paquete gratuito; el servidor puede ser gratuito o de pago
Ideal para	Reportes corporativos estándar, autoservicio de usuarios de negocio	Apps analíticas a la medida, prototipos rápidos para quien ya trabaja en R, herramientas con modelos

Consejo

Muchas empresas usan ambos: Power BI o Tableau para los reportes estándar que consume toda la organización, y Shiny para las herramientas analíticas que necesitan un modelo o una lógica que esas plataformas no cubren (un simulador de precios, un pronóstico de demanda, un segmentador de clientes). Saber Shiny te permite construir justo lo que las otras herramientas no pueden.

6.2 Anatomía de una app Shiny

6.2.1 Instalación

Los paquetes de este módulo se instalan una sola vez:

```
1 install.packages(c("shiny", "shinydashboard", "DT", "plotly", "writexl"))
```

A lo largo del módulo usaremos estos paquetes. Cárgalos al inicio de tu script (al cargar **shinydashboard** y **DT** verás avisos de que “enmascaran” funciones como **box()**, **renderDataTable()** o **filter()**; son normales):

```
1 library(shiny)           # apps web con R
2 library(shinydashboard)  # plantillas de dashboard
3 library(DT)              # tablas interactivas
```

```

4 library(plotly)           # gráficas interactivas
5 library(writexl)          # exportar a Excel
6 library(tidyverse)        # dplyr, ggplot2, readr...
7 library(lubridate)         # fechas
8 library(scales)           # formatos de moneda y porcentaje
9 options(width = 70)

```

6.2.2 Las tres piezas: `ui`, `server` y `shinyApp()`

Concepto clave: Estructura de toda app Shiny

Toda app Shiny tiene tres piezas:

- **ui** (*user interface*): describe lo que el usuario **ve**. Es un objeto de R que en realidad es HTML: títulos, cajas de entrada, huecos donde irán las gráficas.
- **server**: una función con los argumentos `input`, `output` y `session` que contiene la **lógica**: lee lo que el usuario eligió (`input$...`) y produce resultados (`output$...`).
- `shinyApp(ui, server)`: une ambas piezas y crea la app.

Cuando lanzas una app, R arranca un pequeño **servidor web** en tu computadora. El navegador pide la página, recibe el HTML de la **ui** y abre una conexión permanente (un **WebSocket**) con R. Cada vez que mueves un control, el navegador envía el nuevo valor; R recalcula *solo* lo que depende de ese valor y devuelve el resultado (figura 6.2).

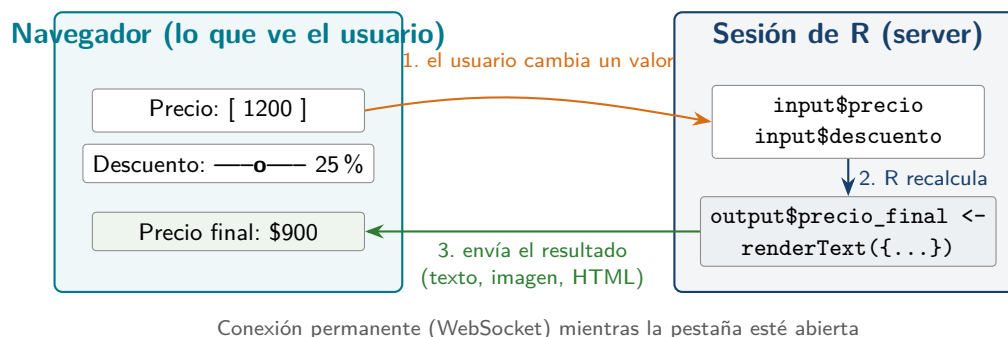


Figura 6.2. Flujo de una app Shiny: el navegador envía entradas, R recalcula y devuelve salidas.

6.2.3 Tu primera app paso a paso

Ejemplo práctico: Calculadora de precio con descuento

El área comercial quiere que los vendedores calculen el precio final de un producto con descuento sin abrir Excel. La app debe tener dos entradas (precio de lista y porcentaje de descuento) y mostrar el precio final. Está en `latex/codigo/apps/01-primer-app/app.R`.

Este es el archivo completo. Léelo con calma: son apenas 30 líneas de código útil y contienen todo lo esencial de Shiny.

latex/codigo/apps/01-primer-app/app.R

```

1  # 1. Paquete -----
2  library(shiny)          # el motor de las apps web con R
3
4  # 2. Interfaz (ui): lo que el usuario VE en el navegador -----
5  ui <- fluidPage(        # página que se adapta al ancho
6    titlePanel("Calculadora de precio con descuento"), # título
7
8    # Entrada 1: una caja para escribir un número (el precio de lista)
9    numericInput(inputId = "precio", label = "Precio de lista ($):",
10                 value = 500, min = 0, step = 10),
11
12    # Entrada 2: un deslizador para elegir el descuento
13    sliderInput(inputId = "descuento", label = "Descuento (%):",
14                min = 0, max = 50, value = 10, step = 5),
15
16    # Salida: un hueco donde el servidor escribirá el resultado
17    h3(textOutput(outputId = "precio_final"))
18 )
19
20 # 3. Servidor (server): la lógica que CORRE en R -----
21 server <- function(input, output, session) {
22   # renderText() se vuelve a ejecutar SOLO cuando cambian las entradas
23   # que usa (input$precio o input$descuento): eso es la reactividad
24   output$precio_final <- renderText({
25     final <- input$precio * (1 - input$descuento / 100)
26     paste0("Precio final: ", scales::dollar(final))
27   })
28 }
29
30 # 4. Unir interfaz y servidor: esto crea (y lanza) la app -----
31 shinyApp(ui = ui, server = server)

```

Vamos por partes:

- **Línea 2:** `library(shiny)` carga el paquete. Toda app empieza así.
- **Líneas 5–18,** la `ui: fluidPage()` crea una página que se adapta al ancho de la pantalla (de computadora o de celular). Dentro van, separados *por comas*, sus elementos: un título, dos *entradas* (*inputs*) y una *salida* (*output*).
- **El `inputId`:** cada entrada tiene un identificador único ("`precio`", "`descuento`"). Es el nombre con el que el servidor lee su valor: `input$precio`.
- **`textOutput("precio_final")`:** reserva un hueco en la página. No calcula nada; solo dice “aquí irá un texto llamado `precio_final`”. Lo envolvemos en `h3()` para que se vea grande (`h3` es un encabezado de HTML).

- **Líneas 21–28, el `server`:** es una función. Dentro asignamos `output$precio_final` con `renderText()`. El código entre llaves lee `input$precio` e `input$descuento`, calcula y devuelve un texto.
- **La magia:** no escribimos ningún “cuando el usuario mueva el deslizador, recalcula”. Shiny detecta solo que `renderText()` usó esas dos entradas y lo vuelve a ejecutar cada vez que alguna cambia. Eso es la **reactividad**.
- **Línea 31: `shinyApp()`** une las piezas. Si ejecutas el archivo, la app se abre.

La `ui` es solo un objeto de R que genera HTML. Puedes construir una entrada en la consola e imprimirla para ver lo que el navegador recibirá. Es un gran recurso para entender (y depurar) Shiny:

```
1 # Una entrada numérica, igual a la de la app, impresa en la consola
2 numericInput(inputId = "precio", label = "Precio de lista ($):",
3               value = 500, min = 0, step = 10)
```

```
<div class="form-group shiny-input-container">
  <label class="control-label" id="precio-label" for="precio">Precio de lista ($):</
    ↪ label>
  <input id="precio" type="number" class="shiny-input-number form-control" value="500"
    ↪ min="0" step="10"/>
</div>
```

Observa el `id="precio"`: el `inputId` se convierte en el identificador del elemento HTML, y por eso **debe ser único** en toda la app. Las salidas son todavía más simples, un contenedor vacío con una clase especial que Shiny sabe llenar:

```
1 textOutput(outputId = "precio_final")
2 plotOutput(outputId = "grafica", height = "300px")
```

```
<div id="precio_final" class="shiny-text-output"></div>
<div class="shiny-plot-output html-fill-item" id="grafica" style="width:100%;height
  ↪ :300px;"></div>
```

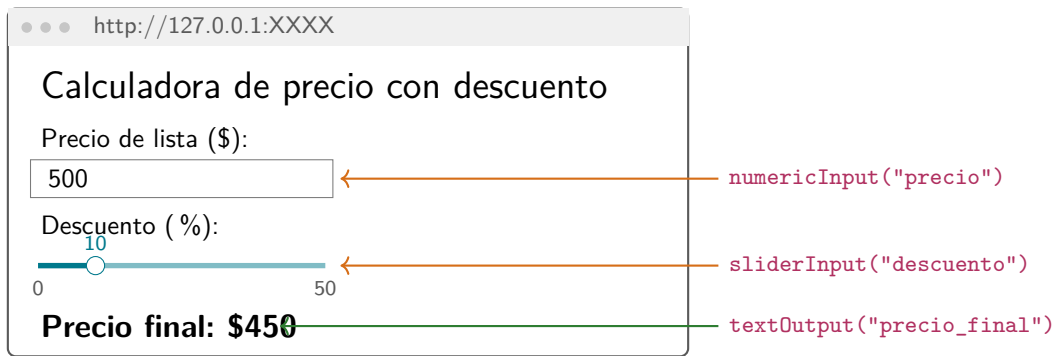


Figura 6.3. Maqueta de la app 01 tal como se ve en el navegador.

6.2.4 Cómo ejecutar (y detener) una app

Tienes tres formas de lanzar una app guardada en un archivo `app.R`:

1. Abrir `app.R` en RStudio y pulsar el botón **Run App** (arriba a la derecha del editor). Con la flechita del botón eliges si se abre en el visor de RStudio o en tu navegador.
2. Desde la consola, con el directorio de trabajo en la carpeta del curso, pasar a `runApp()` la carpeta que contiene el `app.R`:

```
1 # Directorio de trabajo: la carpeta del curso (la que contiene datasets/)
2 getwd()
3 shiny::runApp("latex/codigo/apps/01-primera-app")
4 shiny::runApp("latex/codigo/apps/02-explorador-ventas")
5 shiny::runApp("latex/codigo/apps/03-dashboard-ejecutivo")
```

3. Si defines `ui` y `server` en tu script, ejecutar `shinyApp(ui, server)` en la consola también la abre.

Mientras la app está abierta, la consola de R queda *ocupada* (verás **Listening on http://127.0.0.1:XXXX**) y no acepta otros comandos. Para detenerla pulsa **Esc** en la consola, el botón rojo **Stop** del panel de consola, o cierra la ventana de la app.

▲ Cuidado

Todo lo que escribe `print()` o `message()` dentro del `server` aparece en la *consola de R*, no en la página. Úsalo para depurar: si no sabes qué valor tiene una entrada, escribe `print(input$descuento)` dentro del `renderText()` y mira la consola mientras mueves el control.

6.2.5 El truco para que las apps encuentren datasets/

Aquí hay una trampa en la que cae casi todo el mundo. Cuando ejecutas `runApp()` con la carpeta `latex/codigo/apps/02-explorador-ventas`, Shiny **cambia el directorio de trabajo a la carpeta de la app** mientras esta corre. Por eso, dentro de la app, la ruta relativa `datasets/transacciones.csv` ya no existe y `read_csv()` falla con un mensaje como este (recortado):

```
1 Error: 'datasets/transacciones.csv' does not exist in current working
2 directory ('C:/Users/tu_usuario/cursos-bi-rstudio/latex/codigo/apps/02-explorador-
   ventas').
```

Hay varias soluciones: copiar los datos dentro de la carpeta de la app (lo normal al publicarla), usar una ruta absoluta (frágil: solo funciona en tu computadora) o, la que usan las apps de este libro, **buscar la carpeta datasets/ subiendo directorios**: primero en la carpeta actual, luego en la de arriba (`..`), luego en la de más arriba, hasta encontrarla. Así la app funciona tanto si la lanzas desde la carpeta del curso como si pulsas **Run App** con el archivo abierto.

```
1 # Busca datasets/transacciones.csv subiendo carpetas desde 'desde'
2 buscar_datasets <- function(desde = ".", max_niveles = 6) {
3   carpeta <- desde
4   for (i in 0:max_niveles) {
5     candidato <- file.path(carpeta, "datasets", "transacciones.csv")
6     if (file.exists(candidato)) {
7       return(file.path(carpeta, "datasets")) # ¡encontrada!
8     }
9     carpeta <- file.path(carpeta, "..")      # sube un nivel
10  }
11  stop("No encontré datasets/transacciones.csv. Ejecuta primero ",
12       "source('datasets/generar_datasets.R') en la carpeta del curso.",
13       call. = FALSE)
14 }
```

Pruébala. Desde la carpeta del curso la encuentra a la primera; desde la carpeta de la app 02 tiene que subir cuatro niveles:

```
1 buscar_datasets()           # desde la carpeta del curso
2 carpeta_app <- "latex/codigo/apps/02-explorador-ventas"
3 buscar_datasets(desde = carpeta_app) # como la vería la app
4 file.exists(file.path(buscar_datasets(carpeta_app), "tiendas.csv"))
```

```
[1] "../datasets"
[1] "latex/codigo/apps/02-explorador-ventas/../../../../datasets"
[1] TRUE
```

La segunda ruta se lee “desde la carpeta de la app, sube cuatro veces y entra a `datasets`”. Si la

función no encuentra los datos, se detiene con un mensaje en español que dice exactamente qué hacer, mucho más útil que el error genérico de `read_csv()`.

Hazlo tú

Ejecuta `shiny::runApp("latex/codigo/apps/01-primera-app")`, cambia el precio a 1,200 y el descuento a 25 %. ¿Qué precio final ves? (Debe ser \$900). Después detén la app con `Esc`.

6.3 Entradas y salidas

6.3.1 Las entradas más útiles en BI

Una **entrada** (*input*) es cualquier control con el que el usuario le dice algo a la app. Todas las funciones de entrada tienen la misma forma: `xxxInput(inputId, label, ...)`. El primer argumento es el identificador (lo que usarás como `input$inputId`), el segundo la etiqueta visible y el resto depende del control. La tabla 6.2 resume las que más usarás y, muy importante, *qué tipo de valor llega al servidor*.

Tabla 6.2. Entradas frecuentes en dashboards y el valor que recibe el servidor.

Función	Uso típico	Valor en <code>input\$id</code>
<code>selectInput()</code>	Elegir una tienda, un año, una métrica	Texto (" Sucursal Centro ")
<code>selectizeInput(..., multiple = TRUE)</code>	Elegir varias categorías, con buscador	Vector de texto, o NULL si no hay nada elegido
<code>dateRangeInput()</code>	Periodo de análisis	Vector de 2 fechas (Date)
<code>dateInput()</code>	Una fecha (corte, día)	Una fecha
<code>sliderInput()</code>	Top N, rango de precios, un porcentaje	Número (o 2 números si value tiene 2)
<code>numericInput()</code>	Metas, presupuestos, precios	Número (NA si el usuario lo borra)
<code>checkboxGroupInput()</code>	Varias opciones visibles a la vez (métodos de pago)	Vector de texto, o NULL
<code>radioButtons()</code>	Una opción entre pocas (ventas / unidades)	Texto
<code>checkboxInput()</code>	Sí/no ("incluir outlet")	TRUE o FALSE
<code>textInput()</code>	Buscar un cliente por nombre	Texto
<code>actionButton()</code>	"Calcular", "Actualizar", "Quitar filtros"	Contador: 0, 1, 2... (veces que se pulsó)

Cuidado

Presta atención a las entradas **múltiples**: cuando el usuario no ha elegido nada, el valor es **NULL**, no un vector vacío de texto ni "**Todas**". Si filtras con `categoria %in% input$categorias` sin revisar eso, te quedarás sin filas. La convención de este libro es: *vacío significa "todas"*, y lo comprobamos con `length(input$categorias) > 0`.

Veamos cómo se construyen, con datos reales del curso. Las opciones de los menús casi nunca se escriben a mano: se sacan de los datos, así la app no se desactualiza cuando abre una tienda nueva.

```
1 # Datos para los ejemplos de esta sección
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
5
6 # Las opciones salen de los datos, ordenadas
7 opciones_tienda <- sort(tiendas$nombre_tienda)
8 opciones_pago <- sort(unique(transacciones$metodo_pago))
9 opciones_pago
```

```
[1] "Efectivo"          "Tarjeta Crédito" "Tarjeta Débito"
[4] "Transferencia"
```

Con esos vectores armamos un “catálogo” con las entradas de la tabla 6.2. Fíjate en las comas entre elementos y en los argumentos que controlan el valor inicial:

```
1 ui_catalogo <- fluidPage(
2   selectInput("tienda", "Tienda:", choices = opciones_tienda,
3             selected = "Sucursal Centro"),
4   selectizeInput("pagos", "Métodos de pago:", choices = opciones_pago,
5                 multiple = TRUE, options = list(placeholder = "Todos")),
6   dateRangeInput("fechas", "Periodo:", start = "2023-01-01",
7                 end = "2023-12-31", language = "es", separator = " a ",
8                 format = "dd/mm/yyyy"),
9   sliderInput("top", "Top N:", min = 5, max = 20, value = 10, step = 5),
10  numericInput("meta", "Meta mensual ($):", value = 80000, step = 5000),
11  checkboxGroupInput("trimestres", "Trimestres:",
12                    choices = c("Q1", "Q2", "Q3", "Q4"),
13                    selected = c("Q1", "Q2", "Q3", "Q4"), inline = TRUE),
14  radioButtons("metrica", "Medir por:",
15              choices = c("Ventas ($)" = "ventas",
16                          "Unidades" = "unidades")),
17  actionButton("calcular", "Calcular", icon = icon("play"))
18 )
```

En `radioButtons()` usamos un vector *con nombres*: `"Ventas ($)" = "ventas"`. El nombre es lo que ve el usuario y el valor (`"ventas"`) es lo que llega a `input$metrica`. Así puedes cambiar la etiqueta sin tocar la lógica. Imprime una de las entradas para ver cómo quedan las opciones en HTML:

```
1 radioButtons("metrica", "Medir por:",
2             choices = c("Ventas ($)" = "ventas", "Unidades" = "unidades"))
```

```

<div id="metrica" class="form-group shiny-input-radiogroup shiny-input-container" role
  ↪="radiogroup" aria-labelledby="metrica-label">
  <label class="control-label" id="metrica-label" for="metrica">Medir por:</label>
  <div class="shiny-options-group">
    <div class="radio">
      <label>
        <input type="radio" name="metrica" value="ventas" checked="checked"/>
        <span>Ventas ($)</span>
      </label>
    </div>
    <div class="radio">
      <label>
        <input type="radio" name="metrica" value="unidades"/>
        <span>Unidades</span>
      </label>
    </div>
  </div>
</div>

```

El atributo `value="ventas"` es lo que viaja al servidor; el texto entre `<span>` es solo la etiqueta.

Hazlo tú

En la consola, imprime `selectInput("tienda", "Tienda:", choices = opciones_tienda)` y localiza cuál opción aparece con la palabra `selected`. ¿Qué pasa si agregas `selected = "Sucursal Sur"`?

6.3.2 Salidas y sus funciones `render`

Una **salida** siempre tiene dos mitades que deben coincidir: en la **ui**, una función `xxxOutput("id")` que reserva el lugar; en el **server**, una asignación `output$id <- renderXxx({...})` que produce el contenido. Si usas un `plotOutput()` con un `renderTable()`, o escribes el id distinto en cada lado, la salida simplemente no aparece (sin ningún error). La tabla 6.3 es la que querrás tener a mano.

Tabla 6.3. Parejas de funciones de salida (**ui**) y de render (**server**).

En la ui	En el server	Qué debe devolver el código
<code>textOutput()</code>	<code>renderText()</code>	Un texto (un KPI, un mensaje)
<code>verbatimTextOutput()</code>	<code>renderPrint()</code>	Lo que imprimiría la consola (<code>summary()</code> , <code>str()</code>)
<code>plotOutput()</code>	<code>renderPlot()</code>	Una gráfica (un objeto <code>ggplot</code>)
<code>tableOutput()</code>	<code>renderTable()</code>	Un data frame pequeño (tabla estática)
<code>DT::DTOutput()</code>	<code>DT::renderDT()</code>	Un data frame o un <code>datatable()</code> (tabla con búsqueda, orden y páginas)
<code>valueBoxOutput()</code>	<code>renderValueBox()</code>	Un <code>valueBox()</code> de shinydashboard
<code>infoBoxOutput()</code>	<code>renderInfoBox()</code>	Un <code>infoBox()</code>
<code>plotlyOutput()</code>	<code>renderPlotly()</code>	Una gráfica plotly (por ejemplo, <code>ggplotly(p)</code>)
<code>uiOutput()</code>	<code>renderUI()</code>	Elementos de interfaz creados al vuelo
<code>downloadButton()</code>	<code>downloadHandler()</code>	Un archivo para descargar (sección 6.9)

6.3.3 Probar el servidor sin navegador: `testServer()`

Como no podemos “hacer clic” desde un script, usaremos a lo largo del módulo una herramienta profesional: `testServer()`. Recibe la función `server` (o la carpeta de una app) y un bloque de código donde puedes simular al usuario con `session$setInputs()` y leer las salidas con `output$id`, todo desde la consola. Es la forma de comprobar que la lógica de tu app funciona sin abrir nada.

Ejemplo práctico: Resumen de ventas por tienda y método de pago

Queremos una mini app donde el usuario elija una tienda y, opcionalmente, uno o varios métodos de pago, y vea cuántas transacciones y cuánto se vendió. Aprovecharemos las entradas del catálogo anterior.

```

1 server_catálogo <- function(input, output, session) {
2   output$resumen <- renderText({
3     datos <- transacciones %>%
4       left_join(tiendas, by = "tienda_id") %>%
5       filter(nombre_tienda == input$tienda)
6     if (length(input$pagos) > 0) {           # vacío = todos los métodos
7       datos <- filter(datos, metodo_pago %in% input$pagos)
8     }
9     paste0(input$tienda, ": ", nrow(datos), " transacciones, ",
10           dollar(sum(datos$total_transaccion), accuracy = 1))
11   })
12 }
13
14 testServer(server_catálogo, {
15   session$setInputs(tienda = "Sucursal Centro", pagos = NULL)
16   print(output$resumen)                       # todos los métodos
17   session$setInputs(pagos = c("Efectivo", "Transferencia"))
18   print(output$resumen)                       # solo dos métodos

```



```
19 })
```

```
[1] "Sucursal Centro: 87 transacciones, $88,414"
[1] "Sucursal Centro: 36 transacciones, $30,727"
```

La Sucursal Centro tuvo 87 transacciones en el año, por \$88,414; si nos quedamos solo con efectivo y transferencia, bajan a 36. Fíjate en que la segunda llamada a `setInputs()` solo cambia **pagos**: la tienda conserva su valor, igual que en el navegador. Para ver la app en acción bastaría con:

```
1 shinyApp(ui = fluidPage(ui_catalogo, h3(textOutput("resumen"))),
2           server = server_catalogo)
```

6.4 Reactividad: el concepto más importante

Concepto clave: Reactividad

La **reactividad** es el mecanismo por el que Shiny sabe *qué* recalcular y *cuándo*. Funciona como una hoja de cálculo: si la celda C1 contiene `=A1*B1`, al cambiar A1 se recalcula C1 sola. En Shiny hay tres tipos de piezas:

- **Fuentes reactivas**: las entradas (`input$...`). Cambian cuando el usuario actúa.
- **Conductores**: expresiones intermedias creadas con `reactive()`. Calculan algo a partir de las fuentes y guardan el resultado (*caché*) hasta que una fuente cambie.
- **Consumidores (puntos finales)**: las salidas (`output$...`) y los observadores (`observe()`, `observeEvent()`). Son los que realmente “hacen” algo.

Shiny construye automáticamente un **grafo reactivo**: un mapa de qué depende de qué. La figura 6.4 muestra el de la app 02 que construiremos en la sección 6.6. Cuando el usuario cambia la tienda, Shiny marca como “desactualizado” todo lo que está río abajo de `input$tienda` (el filtrado y las tres salidas) y lo recalcula. Si en cambio mueve el deslizador `n_top`, solo se recalcula la tabla: ni el filtrado ni la gráfica se tocan.

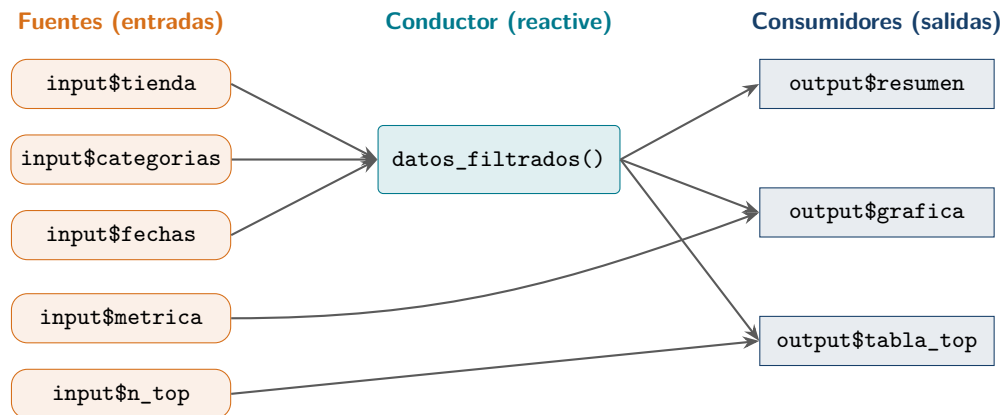


Figura 6.4. Grafo reactivo de la app 02: las entradas alimentan una expresión reactiva que comparten las tres salidas.

6.4.1 `reactive()`: calcula una vez, úsalo muchas

En un dashboard, casi todas las salidas parten de los *mismos datos filtrados*. Si cada `render...()` filtrara por su cuenta, repetirías código y harías trabajar a R varias veces. La solución es una **expresión reactiva**: `reactive({...})` guarda el resultado y solo lo recalcula si cambia alguna entrada que usó.

Ejemplo práctico: Dos KPIs a partir de un solo filtrado

Queremos mostrar el número de transacciones y el total vendido de la tienda elegida. Para *demonstrar* que el filtrado ocurre una sola vez por tienda, el reactivo deja un mensaje en la consola cada vez que se ejecuta.

```

1 server_kpis <- function(input, output, session) {
2   ventas_tienda <- reactive({                # el conductor
3     message(" (filtrando la tienda ", input$tienda, ")") # rastro
4     filter(transacciones, tienda_id == input$tienda)
5   })
6
7   output$n_ventas <- renderText(nrow(ventas_tienda()))
8   output$total    <- renderText(
9     dollar(sum(ventas_tienda()$total_transaccion), accuracy = 1))
10 }
11
12 testServer(server_kpis, {
13   session$setInputs(tienda = 3)
14   cat("Tienda 3:", output$n_ventas, "ventas,", output$total, "\n")
15   session$setInputs(tienda = 8)
16   cat("Tienda 8:", output$n_ventas, "ventas,", output$total, "\n")
17 })
  
```

```
(filtrando la tienda 3)
Tienda 3: 109 ventas, $105,305
(filtrando la tienda 8)
Tienda 8: 106 ventas, $114,377
```

Leímos dos salidas por cada tienda, pero el mensaje “filtrando” aparece una sola vez por tienda: la segunda salida reutilizó el resultado guardado. La tienda 3 (Sucursal Sur) tuvo 109 ventas por \$105,305 y la 8 (Outlet), 106 ventas por \$114,377. Con 1,000 filas no se nota; con millones de registros o una consulta a una base de datos (Módulo 5) es la diferencia entre una app ágil y una desesperante.

▲ Cuidado

Un reactivo se llama como función, con paréntesis: `ventas_tienda()`. Si escribes `ventas_tienda$total` (sin paréntesis) obtienes uno de los errores más famosos de Shiny: `object of type 'closure' is not subsettable` (en español, `objeto de tipo 'closure' no es subconjunto`). “Closure” es como R llama a las funciones: le estás pidiendo una columna a una función. Lo verás en acción en la sección 6.12.

6.4.2 `observe()` y `observeEvent()`: efectos secundarios

Las salidas *producen algo que se muestra*. A veces necesitas *hacer* algo sin mostrar nada: escribir en un registro, mostrar una notificación, guardar un archivo o actualizar otra entrada. Para eso están los **observadores**:

- `observe({...})` se ejecuta cada vez que cambia *cualquier* entrada que use dentro.
- `observeEvent(evento, {...})` se ejecuta solo cuando cambia **evento** (típicamente un botón). Es el más usado porque deja claro qué lo dispara.

Ejemplo práctico: Botón “Quitar filtros” y registro de uso

El dashboard ejecutivo tendrá un botón que regresa los filtros a sus valores iniciales, y al equipo de BI le interesa saber cuántas veces se usa. Contaremos los clics con un `reactiveVal()`, un valor reactivo que tú mismo puedes leer (`clics()`) y modificar (`clics(nuevo_valor)`).

```
1 server_limpiar <- function(input, output, session) {
2   clics <- reactiveVal(0)                                # valor reactivo propio
3
4   observeEvent(input$limpiar, {                           # solo al pulsar el botón
5     clics(clics() + 1)                                    # suma uno
6     updateSelectInput(session, "tienda", selected = "Todas")
7     message("Filtros reiniciados (clic número ", clics(), ")")
8   })
9
10  output$uso <- renderText(paste("Clics en 'Quitar filtros':", clics()))
11 }
```

```

12
13 testServer(server_limpiar, {
14   session$setInputs(limpiar = 1)           # primer clic
15   session$setInputs(limpiar = 2)           # segundo clic
16   print(output$uso)
17 })

```

```

Filtros reiniciados (clic número 1)
Filtros reiniciados (clic número 2)
[1] "Clics en 'Quitar filtros': 2"

```

`updateSelectInput()` (y sus hermanas `updateDateRangeInput()`, `updateSelectizeInput()`...) cambia desde el servidor el valor de una entrada que ya está en la página. El mensaje aparece en la consola de R, no en la app: es el lugar ideal para dejar un rastro de lo que ocurre.

6.4.3 `eventReactive()`: calcular solo al pulsar un botón

Por defecto, Shiny recalcula en cuanto cambias cualquier filtro. Si un cálculo es pesado (un modelo, un pronóstico, una consulta grande) es mejor que el usuario ajuste todos los filtros con calma y luego pulse “Calcular”. `eventReactive(evento, {...})` es un `reactive()` que *solo* se actualiza cuando ocurre el evento.

```

1 server_meta <- function(input, output, session) {
2   # Se recalcula SOLO cuando se pulsa input$calcular
3   proyeccion <- eventReactive(input$calcular, {
4     ventas_mes <- sum(transacciones$total_transaccion) / 12
5     ventas_mes * (1 + input$crecimiento / 100)
6   })
7   output$proyeccion <- renderText(
8     paste("Venta mensual proyectada:", dollar(proyeccion(), accuracy = 1)))
9 }
10
11 testServer(server_meta, {
12   session$setInputs(crecimiento = 10)      # el usuario mueve el control
13   resultado <- tryCatch(output$proyeccion,
14     error = function(e) "(vacío: aún no se pulsa)")
15   print(resultado)
16   session$setInputs(calcular = 1)          # ahora sí pulsa el botón
17   print(output$proyeccion)
18   session$setInputs(crecimiento = 20)      # cambia, pero no pulsa
19   print(output$proyeccion)                 # sigue el valor anterior
20 })

```

```

[1] "(vacío: aún no se pulsa)"
[1] "Venta mensual proyectada: $87,999"

```

```
[1] "Venta mensual proyectada: $87,999"
```

Antes del primer clic la salida queda vacía; al pulsar se calcula con 10 % (la venta mensual promedio de \$79,999 más 10 % da \$87,999); y cambiar a 20 % *sin* pulsar no altera nada. Es exactamente lo que hace el botón “Aplicar” de muchos reportes.

6.4.4 req() y validate(need()): condiciones antes de calcular

Al abrir una app, algunas entradas tardan un instante en tener valor; otras veces el usuario deja un campo vacío o elige filtros que no dejan ningún dato. Dos herramientas evitan errores rojos en la pantalla:

- **req(x)** detiene el cálculo *en silencio* si **x** está vacío (**NULL**, **NA**, **"**, **FALSE**). La salida simplemente se queda en blanco.
- **validate(need(condicion, "mensaje"))** detiene el cálculo y *muestra el mensaje* en gris en lugar de la salida. Úsalo cuando el usuario necesite saber qué pasó.

```
1 server_ticket <- function(input, output, session) {
2   output$ticket <- renderText({
3     req(input$metodo)                                # sin método: en blanco
4     datos <- filter(transacciones, metodo_pago == input$metodo,
5                     total_transaccion >= input$minimo)
6     validate(need(nrow(datos) > 0,
7                  "No hay transacciones con ese monto mínimo."))
8     paste("Ticket promedio:", dollar(mean(datos$total_transaccion)))
9   })
10 }
11
12 testServer(server_ticket, {
13   session$setInputs(metodo = "Transferencia", minimo = 0)
14   print(output$ticket)
15   session$setInputs(minimo = 5000)                  # nadie gasta tanto
16   tryCatch(output$ticket, error = function(e) print(conditionMessage(e)))
17 })
```

```
[1] "Ticket promedio: $825.80"
[1] "No hay transacciones con ese monto mínimo."
```

El ticket promedio con transferencia es de \$825.80. Con un mínimo de \$5,000 no queda ninguna transacción y, en vez de un error críptico, el usuario verá el mensaje que escribimos.

6.4.5 `isolate()`: leer sin depender

A veces quieres *leer* una entrada sin que su cambio dispare un recálculo. Por ejemplo, el título de una gráfica puede mencionar la meta capturada, pero no quieres redibujar la gráfica cada vez que el usuario teclea un dígito de la meta. `isolate(input$meta)` lee el valor actual sin crear la dependencia.

En la consola, fuera de una app, `isolate()` también sirve para “asomarte” a un valor reactivo:

```
1 precio <- reactiveVal(500)           # un valor reactivo
2 precio_con_iva <- reactive(precio() * 1.16) # depende de precio
3 isolate(precio_con_iva())
4 precio(1000)                          # cambiamos la fuente
5 isolate(precio_con_iva())              # se recalcula solo
```

```
[1] 580
[1] 1160
```

6.4.6 Dos errores típicos de reactividad

El primero: **llamar a un reactivo fuera de un contexto reactivo**. Los reactivos solo pueden leerse dentro de un `render...()`, un `reactive()`, un `observe...()` o con `isolate()`. Si lo intentas en la consola o en el cuerpo del `server`, falla:

```
1 tryCatch(precio_con_iva(),
2           error = function(e) cat(conditionMessage(e), "\n"))
```

```
Operation not allowed without an active reactive context.
* You tried to do something that can only be done from inside a reactive consumer.
```

El segundo es su pariente: **usar `input$...` directamente en el cuerpo del `server`**, fuera de cualquier `render` o `reactive`. `input` es un objeto `reactiveValues`, y sus valores tampoco se pueden leer fuera de un consumidor:

```
1 valores <- reactiveValues(n = 5)      # input funciona igual que esto
2 tryCatch(valores$n * 2,
3           error = function(e) cat(conditionMessage(e), "\n"))
```

```
Can't access reactive value 'n' outside of reactive consumer.
i Do you need to wrap inside reactive() or observe()?
```

En una app real ese error aparece al abrirla. Por ejemplo, si en el cuerpo del `server` escribes

`doble <- input$n * 2`, la consola muestra `Error in input$n: Can't access reactive value 'n' outside of reactive consumer` y la app se cierra. La solución es la que sugiere el propio mensaje: envolver el cálculo en `reactive()` (`doble <- reactive(input$n * 2)`) y usarlo como `doble()`.

Tabla 6.4. Herramientas de reactividad: cuál usar.

Función	Úsala cuando...	Devuelve
<code>reactive()</code>	Varias salidas comparten un cálculo (datos filtrados, KPIs)	Un valor que se lee con <code>()</code>
<code>eventReactive()</code>	El cálculo debe esperar a un botón	Un valor que se lee con <code>()</code>
<code>observeEvent()</code>	Hay que <i>hacer</i> algo al pulsar un botón (limpiar filtros, guardar, avisar)	Nada (efecto secundario)
<code>observe()</code>	Hay que hacer algo cada vez que cambie cualquier cosa que use	Nada
<code>reactiveVal()</code> , <code>reactiveValues()</code>	Necesitas un valor que el propio servidor modifique (contadores, historial)	Un valor reactivo
<code>req()</code>	Falta un valor y la salida debe quedar en blanco	—
<code>validate(need())</code>	Hay que explicar al usuario por qué no hay resultado	—
<code>isolate()</code>	Leer un valor sin crear dependencia	El valor

Hazlo tú

Modifica `server_ticket` para que, además del ticket promedio, muestre cuántas transacciones cumplen la condición. Pruébalo con `testServer()` usando `metodo = "Efectivo"` y `minimo = 1000`.

6.5 Organizar la pantalla: layouts

6.5.1 La cuadrícula de 12 columnas

Shiny usa por debajo **Bootstrap**, una biblioteca de diseño web que divide el ancho de la pantalla en **12 columnas**. Con `fluidRow()` creas una fila y con `column(ancho, ...)` repartes esas 12 columnas: dos columnas de 6, tres de 4, una de 3 y otra de 9, etc. (figura 6.5). En pantallas angostas (un celular) las columnas se apilan una debajo de otra automáticamente.

Bootstrap	1	2	3	4	5	6	7	8	9	10	11	12
Fila 1	column(4, ...)				column(8, ...)							
Fila 2	column(3, ...)			column(3, ...)			column(3, ...)			column(3, ...)		
Fila 3	column(12, ...): todo el ancho											

Figura 6.5. Cada `fluidRow()` reparte 12 columnas entre sus `column()`. Los anchos de una fila deben sumar 12 o menos.

```

1 fluidRow(
2   column(4, "Aquí van los filtros"),
3   column(8, "Aquí va la gráfica")
4 )

```

```

<div class="row">
  <div class="col-sm-4">Aquí van los filtros</div>
  <div class="col-sm-8">Aquí va la gráfica</div>
</div>

```

Cada `column(4, ...)` se convierte en un `<div class="col-sm-4">`: la clase le dice a Bootstrap que ocupe 4 de las 12 columnas en pantallas pequeñas o mayores (`sm`, de *small*).

6.5.2 Los layouts que más usarás

- `fluidPage()`: la página base; ocupa todo el ancho.
- `sidebarLayout(sidebarPanel(...), mainPanel(...))`: la estructura clásica “filtros a la izquierda, resultados a la derecha”. Por defecto la barra ocupa 4 columnas y el panel 8; con `width = 3` y `width = 9` das más espacio a las gráficas.
- `tabsetPanel(tabPanel("Título", ...), ...)`: pestañas dentro de una zona de la página.
- `navbarPage("Título", tabPanel(...), ...)`: una barra de navegación superior en la que cada `tabPanel()` es una “página” completa.

Este esqueleto combina todos (construir la `ui` es seguro en un script; solo `shinyApp()` lanza la app):

```

1 ui_esqueleto <- navbarPage(
2   "Ventas 2023",
3   tabPanel("Explorar",
4     sidebarLayout(
5       sidebarPanel(width = 3,
6         selectInput("tienda", "Tienda:", c("Todas", "Centro"))),

```



```

7     mainPanel(width = 9,
8       tabsetPanel(
9         tabPanel("Gráfica", plotOutput("grafica")),
10        tabPanel("Tabla", tableOutput("tabla"))
11      )
12    )
13  )
14 ),
15   tabPanel("Acerca de", p("Datos simulados del curso. "))
16 )
17 class(ui_esqueleto)

```

```
[1] "shiny.tag.list" "list"
```

La **ui** completa es una “lista de etiquetas” HTML (`shiny.tag.list`). Nada se dibuja hasta que el navegador la recibe.

▲ Cuidado

El error más frecuente al armar layouts es **olvidar una coma** entre dos elementos, o dejar una coma *de más* después del último. Una coma faltante produce **Error: unexpected symbol** al ejecutar el archivo; RStudio además marca la línea con una cruz roja. Revisa siempre la línea anterior a la que señala el error. Sangrar el código (`Ctrl` + `I` en RStudio) ayuda a ver qué está dentro de qué.

6.6 Caso práctico 1: el explorador de ventas

Ejemplo práctico: Explorador de ventas 2023

El equipo comercial quiere explorar las 1,000 transacciones de 2023 por tienda, categoría y periodo; ver la evolución mensual medida en ventas, unidades o número de transacciones; y consultar el top de productos. La app completa está en `latex/codigo/apps/02-explorador-ventas/app.R` (unas 160 líneas) y su distribución se muestra en la figura 6.6.

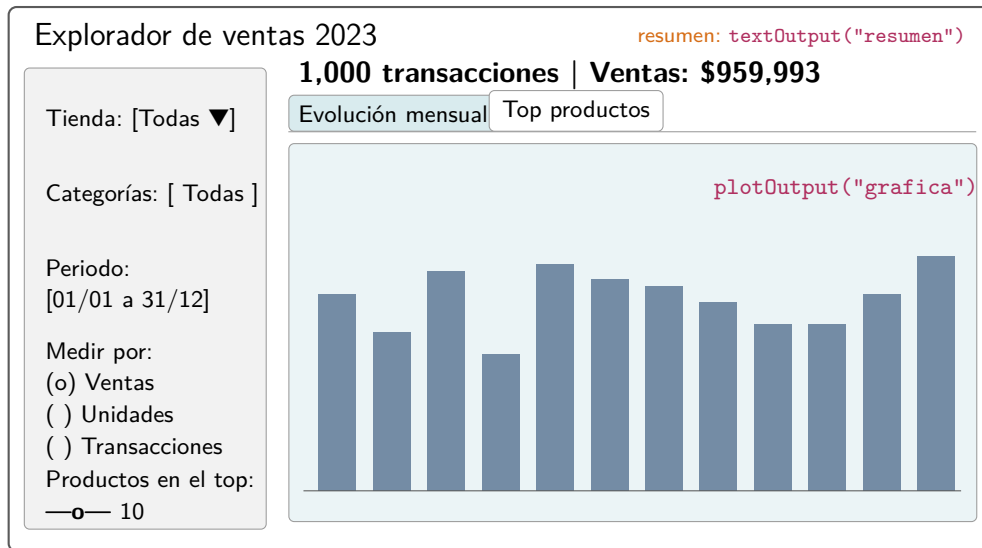


Figura 6.6. Maqueta de la app 02: filtros en la barra lateral, resumen arriba y resultados en dos pestañas.

Recorreremos el archivo en el mismo orden en que está escrito. Esa organización (paquetes → datos → funciones → interfaz → servidor) es la que te recomiendo para cualquier app.

6.6.1 Datos: se leen una sola vez, al arrancar

Todo lo que está *fuera* del **server** se ejecuta una sola vez, cuando arranca la app, y lo comparten todos los usuarios. Por eso ahí leemos y unimos los datos: si lo hiciéramos dentro del **server**, cada persona que abra la app volvería a leer los CSV. Usamos **buscar_datasets()** (sección 6.2.5) y **floor_date()** de **lubridate** para tener el primer día de cada mes (ideal para graficar por mes con un eje de fechas).

```
1 carpeta_datos <- buscar_datasets()
2 tiendas <- read_csv(file.path(carpeta_datos, "tiendas.csv"),
3                       show_col_types = FALSE)
4 productos <- read_csv(file.path(carpeta_datos, "productos.csv"),
5                        show_col_types = FALSE)
6 ventas <- read_csv(file.path(carpeta_datos, "transacciones.csv"),
7                     show_col_types = FALSE) %>%
8   left_join(select(tiendas, tienda_id, nombre_tienda), by = "tienda_id") %>%
9   left_join(select(productos, producto_id, nombre_producto, categoria),
10            by = "producto_id") %>%
11   mutate(mes = floor_date(fecha, "month")) # primer día de cada mes
12
13 # Opciones para los filtros (se calculan una vez a partir de los datos)
14 opciones_tienda <- c("Todas", sort(unique(ventas$nombre_tienda)))
15 opciones_categoria <- sort(unique(ventas$categoria))
16
17 opciones_tienda[1:4]
```

```

18 ventas %>%
19   select(fecha, mes, nombre_tienda, categoria, total_transaccion) %>%
20   head(3)

```

```

[1] "Todas"          "Sucursal Centro" "Sucursal Este"
[4] "Sucursal Express"
# A tibble: 3 x 5
  fecha      mes      nombre_tienda  categoria  total_transaccion
  <date>    <date>    <chr>         <chr>         <dbl>
1 2023-01-13 2023-01-01 Sucursal Express Automotriz      1216.
2 2023-01-09 2023-01-01 Sucursal Norte  Automotriz       232.
3 2023-02-20 2023-02-01 Sucursal Plaza B Juguetes      826.

```

Agregamos la opción **"Todas"** al principio del menú de tiendas: será el valor por defecto y significará “no filtrar”.

6.6.2 Funciones de apoyo: la lógica, separada de la interfaz

Esta es la decisión de diseño más importante del módulo: **todo el trabajo con datos va en funciones normales de R**, que reciben datos y parámetros y devuelven un data frame o una gráfica. No saben nada de **input** ni de **output**. Ventajas: las pruebas en la consola (sin abrir la app), las reutilizas en un reporte o en otra app, y el **server** queda cortito y legible.

```

1  filtrar_ventas <- function(datos, tienda = "Todas", categorias = NULL,
2                             fechas = range(datos$fecha)) {
3    resultado <- filter(datos, fecha >= fechas[1], fecha <= fechas[2])
4    if (tienda != "Todas") {                                # "Todas" = no filtrar tienda
5      resultado <- filter(resultado, nombre_tienda == tienda)
6    }
7    if (length(categorias) > 0) {                            # vacío = todas las categorías
8      resultado <- filter(resultado, categoria %in% categorias)
9    }
10   resultado
11 }
12
13 resumir_por_mes <- function(datos, metrica = "ventas") {
14   datos %>%
15     group_by(mes) %>%
16     summarise(ventas      = sum(total_transaccion),
17               unidades    = sum(cantidad),
18               transacciones = n(),
19               .groups = "drop") %>%
20   mutate(valor = .data[[metrica]]) # la columna elegida por el usuario
21 }

```

resumir_por_mes() calcula las tres métricas y copia en la columna **valor** la que eligió el usuario.

`.data[[metrica]]` es la forma de **dplyr** de usar una columna cuyo *nombre* está guardado en una variable de texto: justo lo que llega de un `radioButtons()`. Probemos ambas funciones como lo haría la app con “Sucursal Norte” y dos categorías:

```
1 norte <- filtrar_ventas(ventas, tienda = "Sucursal Norte",
2                       categorias = c("Electrónica", "Hogar"))
3 nrow(norte)
4 resumir_por_mes(norte, metrica = "unidades") %>% head(4)
```

```
[1] 21
# A tibble: 4 x 5
  mes      ventas unidades transacciones valor
<date>   <dbl>   <dbl>         <int> <dbl>
1 2023-01-01 2741.     5             1     5
2 2023-02-01  871.     2             2     2
3 2023-03-01 1728.     4             2     4
4 2023-04-01  153.     1             1     1
```

```
1 grafica_mensual <- function(resumen, metrica = "ventas") {
2   etiquetas <- c(ventas = "Ventas", unidades = "Unidades vendidas",
3                 transacciones = "Transacciones")
4   formato <- if (metrica == "ventas") label_dollar() else label_comma()
5   ggplot(resumen, aes(x = mes, y = valor)) +
6     geom_col(fill = "#2a78d6", width = 22) +      # ancho en días
7     scale_x_date(date_labels = "%b", date_breaks = "1 month") +
8     scale_y_continuous(labels = formato,
9                       expand = expansion(mult = c(0, 0.05))) +
10    labs(x = NULL, y = etiquetas[[metrica]]) +
11    theme_minimal(base_size = 13) +
12    theme(panel.grid.minor = element_blank(),
13          panel.grid.major.x = element_blank())
14 }
15
16 top_productos <- function(datos, n = 10) {
17   datos %>%
18     group_by(Producto = nombre_producto, `Categoría` = categoria) %>%
19     summarise(Unidades = sum(cantidad),
20              Ventas = sum(total_transaccion), .groups = "drop") %>%
21     slice_max(Ventas, n = n, with_ties = FALSE) %>%
22     mutate(Ventas = dollar(Ventas))      # texto con formato de moneda
23 }
```

Ahora la gráfica que verá el usuario al abrir la app, con todos los datos y la métrica por defecto (figura 6.7):

```

1 p_mensual <- resumir_por_mes(ventas, "ventas") %>%
2   grafica_mensual("ventas")
3 p_mensual

```

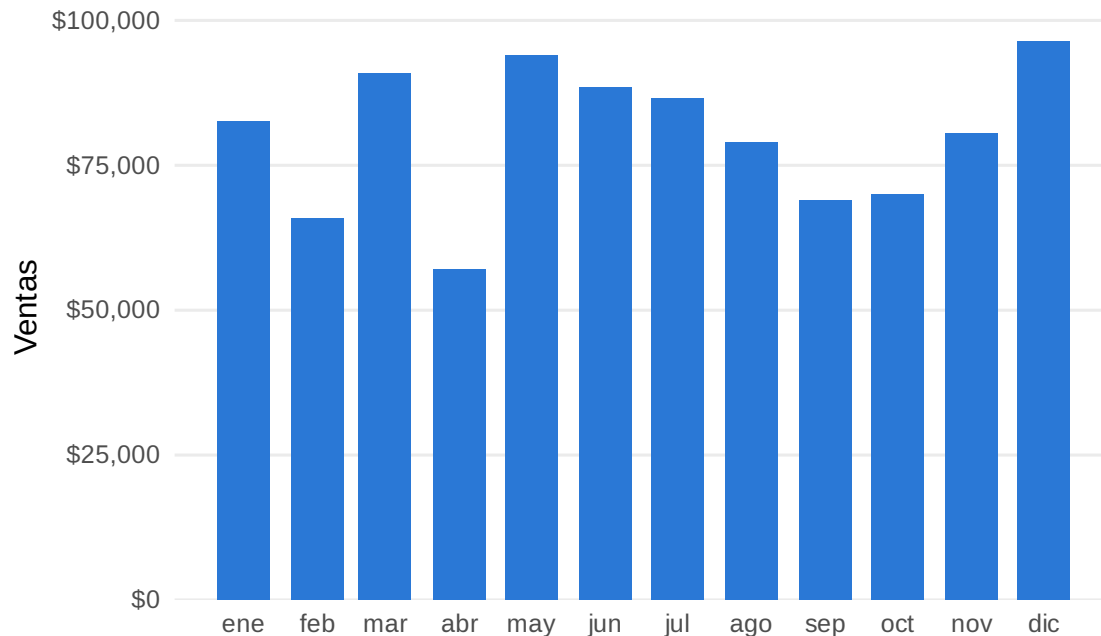


Figura 6.7. Pestaña “Evolución mensual” de la app 02 con los filtros por defecto (salida de `grafica_mensual()`).

Una sola serie, un solo color, barras que empiezan en cero y eje con formato de moneda. Se ve que abril fue el mes más flojo y diciembre el más fuerte. La tabla del top de productos, con el valor por defecto del deslizador (aquí pedimos 5 para ahorrar espacio):

```

1 top_productos(ventas, n = 5)

```

```

# A tibble: 5 x 4
  Producto Categoría Unidades Ventas
  <chr>      <chr>      <dbl> <chr>
1 Producto 06 Juguetes      82 $34,028.04
2 Producto 19 Deportes      85 $28,367.95
3 Producto 31 Alimentos      74 $25,429.70
4 Producto 47 Automotriz      82 $24,527.21
5 Producto 46 Automotriz      74 $24,470.29

```

El Producto 06 (Juguetes) lidera con \$34,028. Convertimos `Ventas` a texto con `dollar()` al final porque `renderTable()` muestra los números tal cual; así el usuario ve “\$34,028.04” en vez de “34028.04”.

6.6.3 La interfaz

Con la lógica lista, la **ui** solo acomoda piezas. Observa que todas las opciones salen de los datos (`opciones_tienda`, `min(ventas$fecha)`...) y que el `selectizeInput()` usa `placeholder = "Todas"` para que el cuadro vacío se lea como “todas las categorías”.

app.R (parte 5): interfaz

```

1 ui <- fluidPage(
2   titlePanel("Explorador de ventas 2023"),
3   sidebarLayout(
4     # Barra lateral: todos los filtros juntos
5     sidebarPanel(
6       width = 3,
7       selectInput("tienda", "Tienda:", choices = opciones_tienda),
8       selectizeInput("categorias", "Categorías:",
9         choices = opciones_categoria, multiple = TRUE,
10        options = list(placeholder = "Todas")),
11      dateRangeInput("fechas", "Periodo:",
12        start = min(ventas$fecha), end = max(ventas$fecha),
13        min = min(ventas$fecha), max = max(ventas$fecha),
14        format = "dd/mm/yyyy", language = "es",
15        separator = " a "),
16      radioButtons("metrica", "Medir por:",
17        choices = c("Ventas ($)" = "ventas",
18          "Unidades" = "unidades",
19          "Transacciones" = "transacciones")),
20      sliderInput("n_top", "Productos en el top:",
21        min = 5, max = 20, value = 10, step = 5)
22    ),
23    # Panel principal: resumen arriba y resultados en pestañas
24    mainPanel(
25      width = 9,
26      h4(textOutput("resumen")),
27      tabsetPanel(
28        tabPanel("Evolución mensual",
29          plotOutput("grafica", height = "380px")),
30        tabPanel("Top productos", tableOutput("tabla_top"))
31      )
32    )
33  )
34 )

```

6.6.4 El servidor

Gracias a las funciones, el **server** cabe en una pantalla. Tiene un único reactivo, `datos_filtrados()`, que alimenta las tres salidas (el grafo de la figura 6.4):

app.R (partes 6 y 7): servidor y lanzamiento

```

1 server <- function(input, output, session) {
2
3   # Expresión reactiva: filtra UNA vez y la reutilizan las tres salidas
4   datos_filtrados <- reactive({
5     req(input$fechas)                                # espera a que haya fechas
6     filtrar_ventas(ventas, input$tienda, input$categorias, input$fechas)
7   })
8
9   output$resumen <- renderText({
10    d <- datos_filtrados()                             # ¡con paréntesis!
11    paste0(comma(nrow(d)), " transacciones | Ventas: ",
12           dollar(sum(d$total_transaccion), accuracy = 1))
13  })
14
15  output$grafica <- renderPlot({
16    validate(need(nrow(datos_filtrados()) > 0,
17                 "No hay ventas con estos filtros. Amplía el periodo."))
18    resumir_por_mes(datos_filtrados(), input$metrica) %>%
19      grafica_mensual(input$metrica)
20  }, res = 96)
21
22  output$tabla_top <- renderTable({
23    top_productos(datos_filtrados(), n = input$n_top)
24  }, align = "lrr", digits = 0)
25 }
26
27 # 7. Lanzar la app -----
28 shinyApp(ui, server)

```

Detalles que vale la pena notar:

- `req(input$fechas)` evita calcular en el instante en que la app arranca y el calendario aún no envía su valor.
- `validate(need(...))` muestra un mensaje amable si el periodo y los filtros no dejan ninguna venta.
- `renderPlot({...}, res = 96)` dibuja la gráfica con una resolución adecuada para pantalla (textos de tamaño normal).
- `renderTable(..., align = "lrr", digits = 0)` alinea a la izquierda las dos columnas de texto y a la derecha las numéricas, y muestra las unidades sin decimales.

6.6.5 Probar la app completa con `testServer()`

`testServer()` también acepta una app completa: le pasas `shinyAppDir("carpeta")` y ejecuta su `app.R` (incluida la lectura de datos) sin abrir el navegador. Dentro del bloque puedes leer incluso los

reactivos internos, como `datos_filtrados()`:

```
1 app02 <- shinyAppDir("latex/codigo/apps/02-explorador-ventas")
2 testServer(app02, {
3   session$setInputs(tienda = "Todas", categorias = NULL,
4                     fechas = as.Date(c("2023-01-01", "2023-12-31")),
5                     metrica = "ventas", n_top = 10)
6   print(output$resumen)
7   session$setInputs(tienda = "Sucursal Centro",
8                     fechas = as.Date(c("2023-07-01", "2023-12-31")))
9   print(output$resumen)
10  print(range(datos_filtrados()$fecha))      # un reactivo interno
11 })
```

```
[1] "1,000 transacciones | Ventas: $959,993"
[1] "46 transacciones | Ventas: $49,559"
[1] "2023-07-05" "2023-12-31"
```

Con todos los filtros por defecto la app resume las 1,000 transacciones (\$959,993); al pasar a la Sucursal Centro en el segundo semestre quedan 46 transacciones por \$49,559, y el rango de fechas (del 5 de julio al 31 de diciembre) confirma que el filtro funciona. Si algo fallara (una columna mal escrita, un paquete faltante), lo verías aquí como error, antes de que lo vea tu directora.

Hazlo tú

Abre la app con `shiny::runApp("latex/codigo/apps/02-explorador-ventas")`, elige “Sucursal Outlet”, la categoría “Belleza” y la métrica “Unidades”. ¿En qué mes vendió más unidades de belleza el Outlet? Luego reduce el periodo a una semana de enero y observa el mensaje de `validate()`.

6.7 Dashboards profesionales con shinydashboard

6.7.1 Las piezas de shinydashboard

shinydashboard es un paquete que da a tu app el aspecto clásico de un tablero de BI: encabezado de color, menú lateral y cajas. Su estructura siempre es la misma:

- `dashboardPage(header, sidebar, body)`: la página completa, con tres argumentos obligatorios en ese orden.
- `dashboardHeader(title = ...)`: la barra superior.
- `dashboardSidebar(...)`: la barra lateral. Suele contener un `sidebarMenu()` con varios `menuItem()` (las “pestañas”) y, debajo, los filtros globales.
- `dashboardBody(...)`: el cuerpo. Con `tabItems(tabItem(tabName = ...), ...)` defines el

contenido de cada pestaña; el `tabName` debe coincidir con el del `menuItem()`.

- `box(title, width, status, ...)`: una caja con título que agrupa una gráfica o tabla. `width` usa la misma cuadrícula de 12 columnas; `status = "primary"` le da el color del tema.
- `valueBox(valor, subtítulo, icon, color)`: la tarjeta grande de KPI. `infoBox()` es una variante más compacta con el ícono a la izquierda.

Un `valueBox()` también es solo HTML. Imprímelo para ver su estructura: el valor va en un encabezado grande (`h3`) y el subtítulo en un párrafo.

```
1 valueBox(value = "$959,993", subtitle = "Ventas totales",
2           icon = icon("dollar-sign"), color = "blue", width = 3)
```

```
<div class="col-sm-3">
  <div class="small-box bg-blue">
    <div class="inner">
      <h3>$959,993</h3>
      <p>Ventas totales</p>
    </div>
    <div class="icon-large">
      <i class="fas fa-dollar-sign" role="presentation" aria-label="dollar-sign icon
↵ "></i>
    </div>
  </div>
</div>
```

Los íconos vienen de la colección gratuita Font Awesome: busca nombres en <https://fontawesome.com/icons> (por ejemplo `"store"`, `"users"`, `"chart-line"`) y úsalos con `icon()`.

Consejo

Resiste la tentación de pintar cada `valueBox()` de un color distinto (verde, rojo, amarillo, morado...). En un dashboard el color debe *significar* algo: usa un mismo color para todas las tarjetas y reserva el rojo o el verde para cuando un KPI esté por debajo o por encima de su meta (lo harás en el ejercicio 6.3).

6.8 Caso práctico 2: el dashboard ejecutivo

Ejemplo práctico: Dashboard ejecutivo de ventas

La dirección quiere un tablero con cinco pestañas: **Resumen** (4 KPIs y evolución mensual), **Productos**, **Tiendas**, **Clientes** y **Datos** (tabla con búsqueda y descarga). Los filtros de periodo, tiendas y categorías deben afectar a todas las pestañas, y un botón debe regresar todo a su estado inicial. La app está en `latex/codigo/apps/03-dashboard-ejecutivo/` y su

distribución aparece en la figura 6.8.

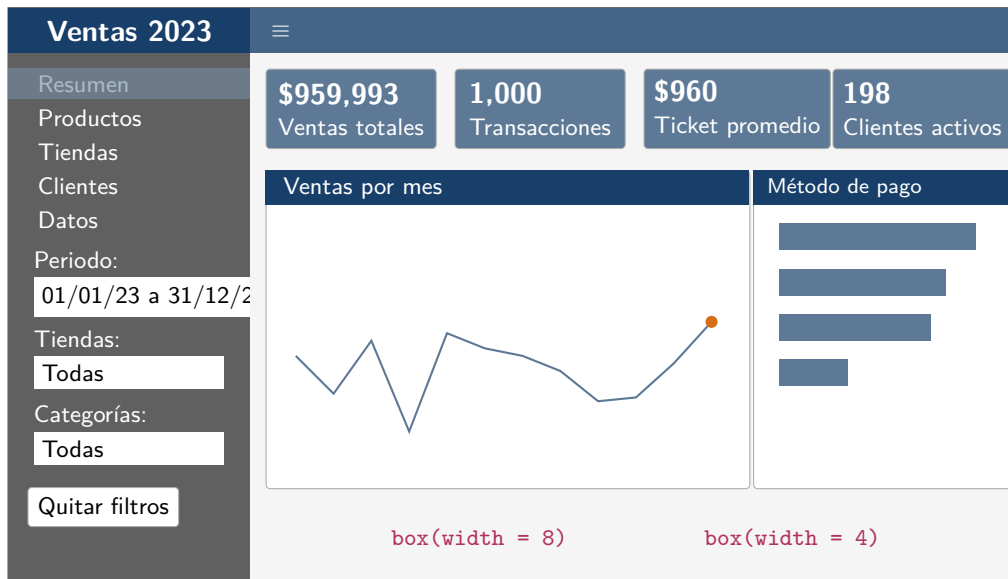


Figura 6.8. Maqueta de la pestaña “Resumen” de la app 03: menú y filtros en la barra lateral, cuatro `valueBox()` arriba y dos `box()` con gráficas.

La app tiene dos archivos:

```
1 03-dashboard-ejecutivo/
2 |-- app.R                  <- interfaz y servidor
3 +-- R/
4     +-- funciones_dashboard.R <- datos, KPIs, gráficas y tablas
```

Shiny tiene una regla muy útil: **antes de ejecutar `app.R`, carga automáticamente todos los archivos `.R` de la subcarpeta `R/`**. Así la lógica vive en su propio archivo y `app.R` se queda con lo visual. Vamos a recorrer primero las funciones (que sí podemos ejecutar y probar aquí) y después la app.

6.8.1 Cargar y unir los datos

`cargar_datos_dashboard()` lee cuatro CSV, se queda con las columnas necesarias y los une a las transacciones con los *joins* del Módulo 2. El resultado es una sola tabla “ancha” con todo lo que el dashboard necesita:

```
1 # ---- Colores y tema corporativos -----
2 color_principal <- "#2a78d6" # una sola serie
3 color_resalte   <- "#eb6834" # para destacar un elemento
4 color_gris      <- "#b5b3ad" # el resto, cuando se resalta uno
5 paleta_libro    <- c("#2a78d6", "#eb6834", "#1baf7a", "#eda100",
```

```

6         "#e87ba4", "#008300", "#4a3aa7", "#e34948")
7
8 tema_dashboard <- function(base_size = 12) {
9   theme_minimal(base_size = base_size) +
10   theme(panel.grid.minor = element_blank(),
11         legend.position = "top",
12         plot.margin      = margin(5, 15, 5, 5)) # aire a la derecha
13 }
14
15 cargar_datos_dashboard <- function(carpeta = buscar_datasets()) {
16   leer <- function(archivo) {
17     read_csv(file.path(carpeta, archivo), show_col_types = FALSE)
18   }
19   productos <- leer("productos.csv") %>%
20     select(producto_id, nombre_producto, categoria, costo)
21   tiendas <- leer("tiendas.csv") %>%
22     transmute(tienda_id, ciudad,
23              tienda = sub("Sucursal ", "", nombre_tienda)) # nombre corto
24   clientes <- leer("clientes.csv") %>%
25     select(cliente_id, nombre_cliente = nombre, segmento, es_premium)
26
27   leer("transacciones.csv") %>%
28     left_join(productos, by = "producto_id") %>%
29     left_join(tiendas,   by = "tienda_id") %>%
30     left_join(clientes,  by = "cliente_id") %>%
31     mutate(mes = floor_date(fecha, "month"),
32            segmento = factor(segmento, levels = c("Joven", "Adulto",
33                                                  "Maduro", "Senior"))) %>%
34     select(transaccion_id, fecha, mes, tienda, ciudad, categoria,
35            producto_id, nombre_producto, cliente_id, nombre_cliente,
36            segmento, es_premium, metodo_pago, cantidad, precio_venta,
37            costo, total_transaccion)
38 }
39
40 datos <- cargar_datos_dashboard()
41 glimpse(datos)

```

Rows: 1,000

Columns: 17

```

$ transaccion_id  <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ~
$ fecha           <date> 2023-01-13, 2023-01-09, 2023-02-20, 2023-~
$ mes             <date> 2023-01-01, 2023-01-01, 2023-02-01, 2023-~
$ tienda          <chr> "Express", "Norte", "Plaza B", "Plaza A", ~
$ ciudad          <chr> "Querétaro", "Guadalajara", "Guadalajara",~
$ categoria       <chr> "Automotriz", "Automotriz", "Juguetes", "D~
$ producto_id     <dbl> 3, 47, 6, 17, 28, 46, 30, 48, 21, 12, 49, ~
$ nombre_producto <chr> "Producto 03", "Producto 47", "Producto 06~
$ cliente_id      <dbl> 155, 155, 75, 51, 173, 135, 90, 168, 13, 9~

```

```

$ nombre_cliente    <chr> "Marta González", "Marta González", "Diego~
$ segmento          <fct> Adulto, Adulto, Maduro, Senior, Maduro, Ma~
$ es_premium        <lgl> TRUE, TRUE, FALSE, FALSE, FALSE, TRUE, FAL~
$ metodo_pago       <chr> "Tarjeta Crédito", "Tarjeta Débito", "Tarj~
$ cantidad          <dbl> 4, 4, 3, 4, 1, 2, 4, 2, 2, 5, 2, 3, 4, 2, ~
$ precio_venta      <dbl> 303.97, 58.07, 275.46, 105.28, 205.47, 534~
$ costo             <dbl> 294.30, 55.00, 84.55, 156.61, 17.23, 177.5~
$ total_transaccion <dbl> 1215.88, 232.28, 826.38, 421.12, 205.47, 1~

```

1,000 filas y 17 columnas: cada transacción ya trae el nombre corto de su tienda, la categoría y el costo del producto, y el segmento del cliente. (Usamos `sub("Sucursal ", "", nombre_tienda)` para mostrar “Centro” en lugar de “Sucursal Centro” y ahorrar espacio en las gráficas.) La función `buscar_datasets()` de este archivo es la misma de la sección 6.2.5.

6.8.2 Filtros globales y KPIs

```

1  # ---- Filtros globales -----
2  # Vacío (NULL o character(0)) significa "todas".
3  filtrar_datos <- function(datos, fechas, tiendas = NULL,
4                             categorias = NULL) {
5    datos <- filter(datos, fecha >= fechas[1], fecha <= fechas[2])
6    if (length(tiendas) > 0) {
7      datos <- filter(datos, tienda %in% tiendas)
8    }
9    if (length(categorias) > 0) {
10     datos <- filter(datos, categoria %in% categorias)
11    }
12    datos
13  }
14
15  calcular_kpis <- function(datos) {
16    datos %>%
17      summarise(
18        ventas          = sum(total_transaccion),
19        transacciones   = n(),
20        ticket_promedio = ventas / transacciones,
21        unidades        = sum(cantidad),
22        clientes        = n_distinct(cliente_id),
23        pct_premium     = mean(es_premium[!duplicated(cliente_id)]),
24        compras_cliente = transacciones / clientes
25      )
26  }
27
28  # Formatos para mostrar los KPIs como texto ("--" si no hay dato)
29  formato_pesos <- function(x) {
30    if_else(is.na(x), "--", dollar(x, accuracy = 1))
31  }

```

```

32 formato_numero <- function(x, decimales = 0) {
33   if_else(is.na(x), "--", comma(x, accuracy = 10^-decimales))
34 }
35 formato_pct <- function(x) {
36   if_else(is.na(x), "--", percent(x, accuracy = 0.1))
37 }

```

Dos detalles de negocio en `calcular_kpis()`: los *clientes activos* se cuentan con `n_distinct()` (un cliente que compró cinco veces cuenta una vez), y el % de clientes premium se calcula sobre clientes únicos (`!duplicated(cliente_id)`), no sobre transacciones. Las funciones de formato devuelven "--" cuando no hay dato (por ejemplo, el ticket promedio de un filtro sin ventas), en lugar de un feo NA.

```

1 anio <- as.Date(c("2023-01-01", "2023-12-31"))
2 kpis_total <- calcular_kpis(filtrar_datos(datos, anio))
3 kpis_total %>% select(ventas, transacciones, ticket_promedio, clientes)
4 formato_pesos(kpis_total$ventas)
5 formato_pct(kpis_total$pct_premium)
6
7 # Los mismos KPIs para dos tiendas y una categoría, primer semestre
8 filtrar_datos(datos, as.Date(c("2023-01-01", "2023-06-30")),
9               tiendas = c("Centro", "Norte"),
10              categorias = "Electrónica") %>%
11   calcular_kpis() %>%
12   select(ventas, transacciones, ticket_promedio, clientes)

```

```

# A tibble: 1 x 4
  ventas transacciones ticket_promedio clientes
  <dbl>         <int>         <dbl>     <int>
1 959993.         1000           960.       198
[1] "$959,993"
[1] "36.4%"
# A tibble: 1 x 4
  ventas transacciones ticket_promedio clientes
  <dbl>         <int>         <dbl>     <int>
1  6937.           10           694.        10

```

En todo 2023 hubo 1,000 transacciones por \$959,993, con un ticket promedio de \$960 y 198 de los 200 clientes activos; el 36.4% de esos clientes es premium. Con los filtros del segundo ejemplo la historia cambia por completo: solo 10 transacciones de Electrónica entre Centro y Norte en el primer semestre. Estas cifras son exactamente las que mostrarán los `valueBox()`.

6.8.3 Las gráficas de cada pestaña

Cada gráfica es una función que recibe los datos *ya filtrados* y devuelve un objeto `ggplot`. Todas usan el tema del dashboard y los colores corporativos del Módulo 3.

```

1 grafica_evolucion <- function(datos) {
2   por_mes <- datos %>%
3     group_by(mes) %>%
4     summarise(ventas = sum(total_transaccion), .groups = "drop")
5   mejor_mes <- slice_max(por_mes, ventas, n = 1, with_ties = FALSE)
6   ggplot(por_mes, aes(x = mes, y = ventas)) +
7     geom_line(color = color_principal, linewidth = 0.8) +
8     geom_point(color = color_principal, size = 2) +
9     geom_point(data = mejor_mes, color = color_resalte, size = 3.5) +
10    geom_text(data = mejor_mes, aes(label = dollar(ventas, accuracy = 1)),
11              vjust = -1.2, size = 3.5, color = "gray25") +
12    scale_x_date(date_labels = "%b", date_breaks = "1 month") +
13    scale_y_continuous(labels = label_dollar(), limits = c(0, NA),
14                      expand = expansion(mult = c(0, 0.15))) +
15    labs(x = NULL, y = "Ventas") +
16    tema_dashboard()
17 }
18
19 p_evolucion <- grafica_evolucion(datos)
20 p_evolucion

```

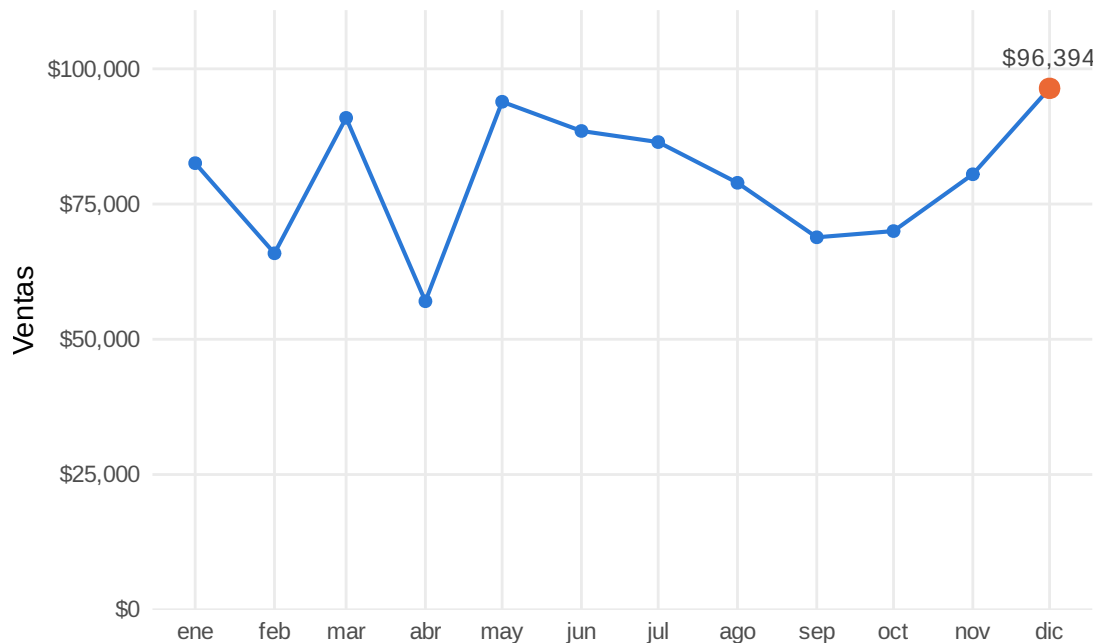


Figura 6.9. Pestaña Resumen: ventas por mes con el mejor mes resaltado (`grafica_evolucion()`).

La línea usa un solo color y solo el mejor mes lleva color de resalte y etiqueta (diciembre, con \$96,394): el ojo va directo a lo importante. Como el eje Y empieza en cero, la caída de abril se ve en su verdadera magnitud.

```

1 grafica_barras <- function(resumen, etiqueta_y = NULL) {
2   # resumen: columnas 'nombre' y 'ventas'; barras horizontales ordenadas
3   ggplot(resumen, aes(x = ventas, y = reorder(nombre, ventas))) +
4     geom_col(fill = color_principal, width = 0.7) +
5     scale_x_continuous(labels = label_dollar(),
6                        expand = expansion(mult = c(0, 0.05))) +
7     labs(x = "Ventas", y = etiqueta_y) +
8     tema_dashboard()
9 }
10
11 grafica_top_productos <- function(datos, n = 10) {
12   datos %>%
13     group_by(nombre = nombre_producto) %>%
14     summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
15     slice_max(ventas, n = n, with_ties = FALSE) %>%
16     grafica_barras()
17 }
18
19 p_top <- grafica_top_productos(datos)
20 p_top

```

`grafica_barras()` es una función “genérica” que reutilizan la gráfica de productos y la de categorías (`grafica_categorias()`, en el archivo, junto con `grafica_metodo_pago()` de la pestaña Resumen): ambas resumen a dos columnas, `nombre` y `ventas`, y le pasan el resultado. Escribir la gráfica una sola vez garantiza que las dos pestañas se vean iguales (figura 6.10).

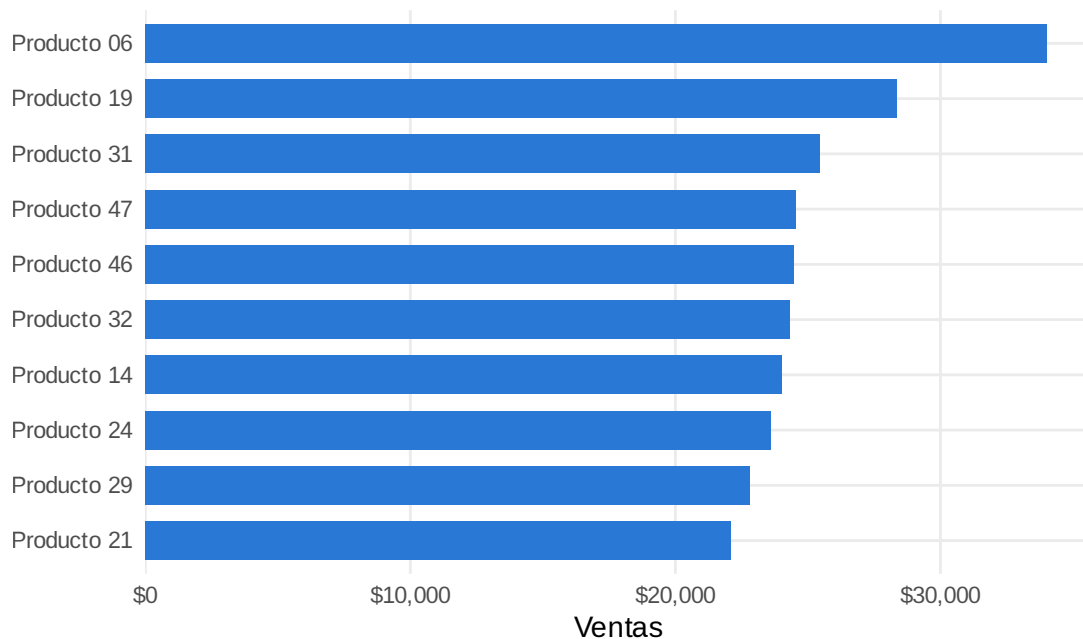


Figura 6.10. Pestaña Productos: los 10 productos con más ventas (`grafica_top_productos()`).

```

1 grafica_tiendas <- function(datos) {
2   por_tienda <- datos %>%
3     group_by(tienda) %>%
4     summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
5     mutate(es_mejor = ventas == max(ventas)) # la tienda líder
6   ggplot(por_tienda, aes(x = ventas, y = reorder(tienda, ventas),
7     fill = es_mejor)) +
8     geom_col(width = 0.7) +
9     geom_text(data = filter(por_tienda, es_mejor),
10      aes(label = dollar(ventas, accuracy = 1)),
11      hjust = 1.1, color = "white", size = 3.5) +
12     scale_fill_manual(values = c(`TRUE` = color_resalte,
13      `FALSE` = color_gris), guide = "none") +
14     scale_x_continuous(labels = label_dollar(),
15      expand = expansion(mult = c(0, 0.05))) +
16     labs(x = "Ventas", y = NULL) +
17     tema_dashboard()
18 }
19
20 p_tiendas <- grafica_tiendas(datos)
21 p_tiendas

```

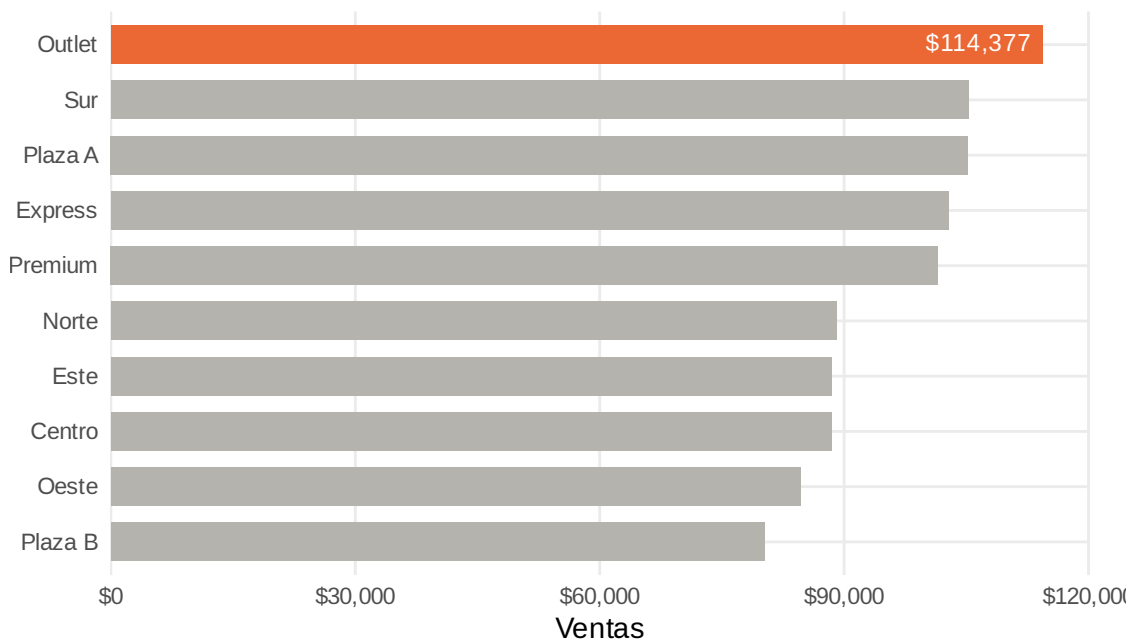


Figura 6.11. Pestaña Tiendas: la tienda líder en color de resalte y el resto en gris (`grafica_tiendas()`).

La figura 6.11 aplica la técnica de “resaltar uno y agrisar el resto”: la pregunta es “¿quién lidera?” y la respuesta salta a la vista. Un hallazgo interesante: el *Outlet*, una de las tiendas más pequeñas (350 m²), es la que más vende en estos datos simulados, con \$114,377.


```

1 grafica_segmentos <- function(datos) {
2   datos %>%
3     mutate(tipo = if_else(es_premium, "Premium", "Regular")) %>%
4     group_by(segmento, tipo) %>%
5     summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
6     ggplot(aes(x = segmento, y = ventas, fill = tipo)) +
7     geom_col(position = position_dodge(width = 0.8), width = 0.7) +
8     scale_fill_manual(values = c(Premium = paleta_libro[2],
9                                   Regular = paleta_libro[1])) +
10    scale_y_continuous(labels = label_dollar(),
11                       expand = expansion(mult = c(0, 0.05))) +
12    labs(x = "Segmento de edad", y = "Ventas", fill = "Tipo de cliente") +
13    tema_dashboard()
14 }
15
16 p_segmentos <- grafica_segmentos(datos)
17 p_segmentos

```

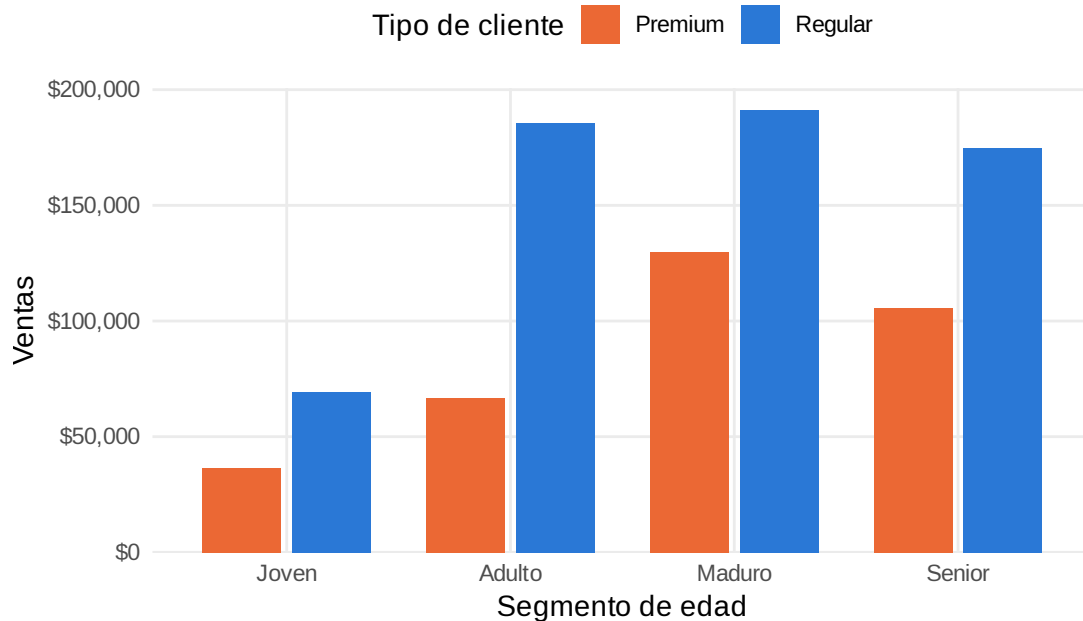


Figura 6.12. Pestaña Clientes: ventas por segmento de edad y tipo de cliente (`grafica_segmentos()`).

Aquí sí hay dos series (Premium y Regular), así que usamos dos colores de la paleta y leyenda. En todos los segmentos los clientes regulares aportan más ventas que los premium, lo cual es lógico porque son cerca del 64 % de los clientes.

6.8.4 Tablas y detalle

```

1  # ---- Tablas -----
2  tabla_tiendas <- function(datos) {
3    datos %>%
4      group_by(Tienda = tienda) %>%
5      summarise(Ventas = sum(total_transaccion),
6                Transacciones = n(),
7                Ticket = Ventas / Transacciones, .groups = "drop") %>%
8      arrange(desc(Ventas)) %>%
9      mutate(Ventas = formato_pesos(Ventas), Ticket = formato_pesos(Ticket))
10 }
11
12 tabla_top_clientes <- function(datos, n = 10) {
13   datos %>%
14     group_by(Cliente = nombre_cliente, Segmento = segmento) %>%
15     summarise(Compras = n(), Ventas = sum(total_transaccion),
16               .groups = "drop") %>%
17     slice_max(Ventas, n = n, with_ties = FALSE) %>%
18     mutate(Ventas = formato_pesos(Ventas))
19 }
20
21 # Datos listos para la tabla interactiva y para las descargas
22 datos_detalle <- function(datos) {
23   datos %>%
24     arrange(fecha, transaccion_id) %>%
25     select(Fecha = fecha, Tienda = tienda, `Categoría` = categoria,
26            Producto = nombre_producto, Cliente = nombre_cliente,
27            Cantidad = cantidad, Precio = precio_venta,
28            Total = total_transaccion, `Método de pago` = metodo_pago)
29 }
30
31 tabla_tiendas(datos) %>% head(4)
32 tabla_top_clientes(datos, n = 3)

```

```

# A tibble: 4 x 4
  Tienda Ventas Transacciones Ticket
<chr>   <chr>         <int> <chr>
1 Outlet $114,377         106 $1,079
2 Sur    $105,305         109 $966
3 Plaza A $105,216         109 $965
4 Express $102,791         108 $952
# A tibble: 3 x 4
  Cliente      Segmento Compras Ventas
<chr>        <fct>      <int> <chr>
1 Luis Hernández Maduro      18 $15,744
2 Pedro Cruz    Adulto      13 $14,680

```

3 Miguel Flores	Adulto	11 \$13,742
-----------------	--------	-------------

El Outlet no solo vende más: también tiene el ticket promedio más alto (\$1,079). Las funciones que preparan la tabla interactiva (`tabla_detalle_dt()`) las veremos en la sección 6.9.2.

6.8.5 La interfaz del dashboard

Con la lógica probada, pasamos a `app.R`. Primero paquetes y datos (fuera del `server`, una sola vez) y luego encabezado y barra lateral:

app.R (partes 1 a 3)

```

1  # 1. Paquetes -----
2  library(shiny)
3  library(shinydashboard)  # dashboardPage(), box(), valueBox()...
4  library(dplyr)
5  library(readr)
6  library(ggplot2)
7  library(lubridate)
8  library(scales)
9  library(DT)              # tablas interactivas
10 library(writexl)         # descarga en Excel
11
12 # 2. Datos: se leen UNA vez, fuera del server -----
13 datos <- cargar_datos_dashboard()
14 fecha_min <- min(datos$fecha)
15 fecha_max <- max(datos$fecha)
16 opciones_tiendas <- sort(unique(datos$tienda))
17 opciones_categorias <- sort(unique(datos$categoria))
18
19 # 3. Interfaz: encabezado y barra lateral -----
20 encabezado <- dashboardHeader(title = "Ventas 2023")
21
22 barra_lateral <- dashboardSidebar(
23   sidebarMenu(
24     id = "menu",
25     menuItem("Resumen", tabName = "resumen", icon = icon("gauge")),
26     menuItem("Productos", tabName = "productos", icon = icon("box")),
27     menuItem("Tiendas", tabName = "tiendas", icon = icon("store")),
28     menuItem("Clientes", tabName = "clientes", icon = icon("users")),
29     menuItem("Datos", tabName = "datos", icon = icon("table"))
30   ),
31   # Filtros globales: afectan a TODAS las pestañas
32   dateRangeInput("fechas", "Periodo:", start = fecha_min, end = fecha_max,
33     min = fecha_min, max = fecha_max, format = "dd/mm/yyyy",
34     language = "es", separator = " a "),
35   selectizeInput("tiendas", "Tiendas:", choices = opciones_tiendas,
36     multiple = TRUE, options = list(placeholder = "Todas")),

```

```

37   selectizeInput("categorias", "Categorías:",
38                 choices = opciones_categorias, multiple = TRUE,
39                 options = list(placeholder = "Todas")),
40   actionButton("limpiar", "Quitar filtros", icon = icon("rotate-left"))
41 )

```

Los filtros van *después* del `sidebarMenu()`, dentro de `dashboardSidebar()`: así quedan visibles en todas las pestañas, lo que deja claro que son globales. Los `selectizeInput()` múltiples usan el marcador “Todas” para indicar que vacío significa sin filtro.

El cuerpo tiene un `tabItem()` por cada `menuItem()`. Cada fila es un `fluidRow()` y cada gráfica vive en una `box()`:

app.R (parte 4): cuerpo (extracto)

```

1  # 4. Interfaz: cuerpo con una tabItem por pestaña del menú -----
2  cuerpo <- dashboardBody(
3    tabItems(
4      tabItem(tabName = "resumen",
5        fluidRow(
6          # fila de KPIs arriba
7          valueBoxOutput("kpi_ventas", width = 3),
8          valueBoxOutput("kpi_transacciones", width = 3),
9          valueBoxOutput("kpi_ticket", width = 3),
10         valueBoxOutput("kpi_clientes", width = 3)
11       ),
12       fluidRow(
13         box(title = "Ventas por mes", width = 8, status = "primary",
14           solidHeader = TRUE, plotOutput("graf_evolucion", height = 300)),
15         box(title = "Ventas por método de pago", width = 4,
16           status = "primary", solidHeader = TRUE,
17           plotOutput("graf_pago", height = 300))
18       )
19     ),
20     # ... pestañas "productos", "tiendas" y "clientes": misma idea,
21     # ... una fluidRow() con dos box() cada una (ver app.R)
22     tabItem(tabName = "datos",
23       fluidRow(
24         box(title = "Detalle de transacciones", width = 12,
25           status = "primary",
26           downloadButton("descargar_csv", "Descargar CSV"),
27           downloadButton("descargar_excel", "Descargar Excel"),
28           br(), br(),
29           DTOutput("tabla_detalle"))
30       )
31     )
32 )
33
34 ui <- dashboardPage(encabezado, barra_lateral, cuerpo, skin = "blue")

```

Observa la fila de KPIs: cuatro `valueBoxOutput()` de ancho 3 suman exactamente 12 columnas. En la pestaña Clientes, los `infoBoxOutput()` tienen el ancho por defecto (4), así que caben tres por fila.

6.8.6 El servidor del dashboard

app.R (parte 5): reactivos, botón y KPIs

```

1  # 5. Servidor: datos filtrados, KPIs y botón para limpiar filtros -----
2  server <- function(input, output, session) {
3
4    # La pieza central: TODO lo demás depende de datos_filtrados()
5    datos_filtrados <- reactive({
6      req(input$fechas)
7      filtrar_datos(datos, input$fechas, input$tiendas, input$categorias)
8    }) %>%
9      bindCache(input$fechas, input$tiendas, input$categorias)
10
11    kpis <- reactive(calcular_kpis(datos_filtrados()))
12
13    # Igual que datos_filtrados(), pero si los filtros no dejan ninguna
14    # venta, las salidas que lo usan muestran un mensaje amable
15    datos_validos <- reactive({
16      validate(need(nrow(datos_filtrados()) > 0,
17                    "No hay ventas con estos filtros."))
18      datos_filtrados()
19    })
20
21    observeEvent(input$limpiar, {
22      updateDateRangeInput(session, "fechas", start = fecha_min,
23                           end = fecha_max)
24      updateSelectizeInput(session, "tiendas", selected = character(0))
25      updateSelectizeInput(session, "categorias", selected = character(0))
26    })
27
28    output$kpi_ventas <- renderValueBox({
29      valueBox(formato_pesos(kpis()$ventas), "Ventas totales",
30                icon = icon("dollar-sign"), color = "blue")
31    })
32    output$kpi_transacciones <- renderValueBox({
33      valueBox(formato_numero(kpis()$transacciones), "Transacciones",
34                icon = icon("receipt"), color = "blue")
35    })
36    output$kpi_ticket <- renderValueBox({
37      valueBox(formato_pesos(kpis()$ticket_promedio), "Ticket promedio",
38                icon = icon("cart-shopping"), color = "blue")
39    })
40    output$kpi_clientes <- renderValueBox({
41      icon = icon("users"), color = "blue")

```

```
42    })
```

La arquitectura reactiva es la de la figura 6.4, un poco más grande: `datos_filtrados()` depende de los tres filtros; de él cuelgan `kpis()` (que alimenta los `valueBox()` y los `infoBox()`) y `datos_validos()` (que alimenta las gráficas y muestra un mensaje si no hay ventas). El botón “Quitar filtros” es un `observeEvent()` que devuelve cada entrada a su valor inicial con las funciones `update...()`. Sobre `bindCache()` hablaremos en la sección 6.10.

app.R (partes 6 a 8): salidas, descargas y lanzamiento (extracto)

```
1  # 6. Servidor: gráficas y tablas de cada pestaña -----
2  output$graf_evolucion <- renderPlot({
3    grafica_evolucion(datos_validos())
4  }, res = 96)
5  output$graf_pago <- renderPlot({
6    grafica_metodo_pago(datos_validos())
7  }, res = 96)
8  # ... las demás gráficas siguen el mismo patrón de una línea ...
9  output$tabla_tiendas <- renderTable(tabla_tiendas(datos_filtrados()))
10 output$info_premium <- renderInfoBox({
11   infoBox("Clientes premium", formato_pct(kpis())$pct_premium),
12   icon = icon("star"), color = "blue")
13 })
14 # ... infoBox de clientes y frecuencia, gráfica de segmentos ...
15 output$tabla_detalle <- renderDT(tabla_detalle_dt(datos_filtrados()))
16
17 # 7. Servidor: descargas (respetan los filtros activos) -----
18 output$descargar_csv <- downloadHandler(
19   filename = function() paste0("ventas_filtradas_", Sys.Date(), ".csv"),
20   content = function(file) {
21     write_excel_csv(datos_detalle(datos_filtrados()), file)
22   }
23 )
24 output$descargar_excel <- downloadHandler(
25   filename = function() paste0("ventas_filtradas_", Sys.Date(), ".xlsx"),
26   content = function(file) {
27     write_xlsx(datos_detalle(datos_filtrados()), file)
28   }
29 )
30 }
31
32 # 8. Lanzar la app -----
33 shinyApp(ui, server)
```

Cada `renderPlot()` tiene una sola línea: llama a su función con los datos válidos. Si mañana cambias el diseño de una gráfica, lo haces en `R/funciones_dashboard.R` sin tocar `app.R`.

6.8.7 Probar el dashboard completo

```

1 app03 <- shinyAppDir("latex/codigo/apps/03-dashboard-ejecutivo")
2 testServer(app03, {
3   session$setInputs(fechas = as.Date(c("2023-01-01", "2023-12-31")),
4                     tiendas = NULL, categorias = NULL)
5   cat("Ventas:", kpis()$ventas, "| Clientes:", kpis()$clientes, "\n")
6
7   # El usuario elige dos tiendas y una categoría
8   session$setInputs(tiendas = c("Centro", "Norte"),
9                     categorias = "Electrónica")
10  cat("Ventas:", round(kpis()$ventas, 2),
11      "| Transacciones:", kpis()$transacciones, "\n")
12
13  # Filtros sin ventas: las gráficas muestran el mensaje de validate()
14  session$setInputs(tiendas = "Express", categorias = "Ropa",
15                    fechas = as.Date(c("2023-01-01", "2023-01-31")))
16  tryCatch(output$graf_evolucion,
17            error = function(e) cat("Gráfica:", conditionMessage(e), "\n"))
18  cat("Ticket en el valueBox:", formato_pesos(kpis()$ticket_promedio), "\n")
19 })

```

```

Ventas: 959992.7 | Clientes: 198
Ventas: 32911.76 | Transacciones: 31
Gráfica: No hay ventas con estos filtros.
Ticket en el valueBox: --

```

La prueba confirma los tres escenarios: todo el año (\$959,992.70 y 198 clientes), un filtro combinado (31 transacciones) y un filtro sin ventas, donde la gráfica muestra el mensaje amable y la tarjeta del ticket muestra “--” en lugar de NA.

Hazlo tú

Ejecuta `shiny::runApp("latex/codigo/apps/03-dashboard-ejecutivo")`. Recorre las cinco pestañas; luego elige solo la tienda “Outlet” y comprueba que *todas* las pestañas cambian. Pulsa “Quitar filtros”. ¿Qué método de pago concentra más ventas en el Outlet?

6.9 Descargas, tablas interactivas y gráficas interactivas

6.9.1 Botones de descarga con `downloadHandler()`

Lo primero que pide cualquier usuario de un dashboard es “¿me lo puedes pasar en Excel?”. Un botón de descarga tiene, como toda salida, dos mitades: `downloadButton("id", "Texto")` en la `ui` y `output$id <- downloadHandler(filename, content)` en el `server`:

- **filename**: una *función* que devuelve el nombre del archivo. Incluir la fecha (`Sys.Date()`) evita que el usuario sobrescriba descargas anteriores.
- **content**: una *función* que recibe una ruta temporal (`file`) y escribe ahí el archivo. Shiny se encarga de enviarlo al navegador.

En la app 03 (parte 7 del servidor) hay dos botones, uno para CSV y otro para Excel, y ambos descargan `datos_detalle(datos_filtrados())`: **respetan los filtros activos**, que es lo que el usuario espera. Para el CSV usamos `write_excel_csv()` de **readr** en lugar de `write_csv()`: agrega una marca invisible al inicio del archivo para que Excel muestre bien los acentos (“Categoría”, “Tarjeta Crédito”). Para Excel usamos `write_xlsx()` de **writexl**.

Como **content** es una función normal, puedes probar lo que hará escribiendo tú mismo los archivos:

```
1 dir.create("resultados", showWarnings = FALSE)
2 detalle <- datos_detalle(filtrar_datos(datos, anio, tiendas = "Outlet"))
3 write_excel_csv(detalle, "resultados/ventas_outlet.csv")
4 write_xlsx(detalle, "resultados/ventas_outlet.xlsx")
5 dim(detalle)
6 names(detalle)
```

```
[1] 106    9
[1] "Fecha"      "Tienda"      "Categoría"
[4] "Producto"   "Cliente"     "Cantidad"
[7] "Precio"     "Total"       "Método de pago"
```

Las 106 transacciones del Outlet, con 9 columnas de nombres legibles, listas para quien las pida. En la prueba con `testServer()` también puedes leer `output$descargar_csv`: devuelve la ruta del archivo generado, que puedes abrir con `read_csv()` para comprobar su contenido.

6.9.2 Tablas interactivas con DT

`renderTable()` está bien para tablas pequeñas (un top 10). Para mostrar cientos o miles de filas necesitas búsqueda, orden por columna y paginación: eso hace **DT**, un puente a la biblioteca DataTables de JavaScript. La pareja es `DTOutput()` / `renderDT()`, y dentro se usa `datatable()`:

```
1 # Textos de la tabla DT en español (sin depender de internet)
2 idioma_dt <- list(
3   search = "Buscar:", lengthMenu = "Mostrar _MENU_ filas",
4   info = "Filas _START_ a _END_ de _TOTAL_",
5   infoEmpty = "Sin resultados", zeroRecords = "No hay coincidencias",
6   paginate = list(previous = "Anterior", `next` = "Siguiente")
7 )
8
9 tabla_detalle_dt <- function(datos) {
10   datatable(datos_detalle(datos), rownames = FALSE,
```



```

11     options = list(pageLength = 10, language = idioma_dt)) %>%
12     formatCurrency(c("Precio", "Total"), currency = "$", digits = 2)
13 }
14
15 tabla_dt <- tabla_detalle_dt(datos)
16 class(tabla_dt)

```

```
[1] "datatables" "htmlwidget"
```

El resultado es un **htmlwidget**: un objeto de R que se dibuja con JavaScript en el navegador (en RStudio, escribe `tabla_dt` en la consola y lo verás en el visor). Las opciones más útiles:

- `formatCurrency(columnas, currency = "$")`: formato de moneda sin convertir los números a texto, así que ordenar por esa columna sigue funcionando bien. También existen `formatPercentage()`, `formatRound()` y `formatDate()`.
- `options = list(pageLength = 10)`: filas por página.
- `language = idioma_dt`: traduce “Search”, “Next”, etc. Lo definimos a mano para no depender de internet.
- `filter = "top"`: agrega una caja de búsqueda encima de cada columna.
- `rownames = FALSE`: oculta la columna de números de fila.

▲ Cuidado

shiny también tiene funciones llamadas `dataTableOutput()` y `renderDataTable()`, antiguas y distintas a las de **DT**. Por eso al cargar **DT** aparece el aviso de que las “enmascara”. Usa siempre `DT::DTOutput()` y `DT::renderDT()` y evitarás confusiones.

6.9.3 Gráficas interactivas con plotly

Con **plotly** una gráfica de **ggplot2** gana tooltips (al pasar el ratón ves el valor exacto), zoom y la opción de ocultar series. La forma más rápida es `ggplotly()`, que convierte un **ggplot** ya hecho (ya cargamos **plotly** al inicio del módulo):

```

1 graf_interactiva <- ggplotly(p_tiendas)
2 class(graf_interactiva)

```

```
[1] "plotly"      "htmlwidget"
```

En la app basta con cambiar la pareja de funciones:

```

1 # En la ui, en lugar de plotOutput("graf_tiendas"):
2 plotly::plotlyOutput("graf_tiendas", height = 380)
3
4 # En el server, en lugar de renderPlot():
5 output$graf_tiendas <- plotly::renderPlotly({
6   ggplotly(grafica_tiendas(datos_validos()))
7 })

```

Consejo

No conviertas todo a plotly “porque se ve moderno”. La interactividad ayuda en gráficas con muchos puntos o cuando el usuario necesita el valor exacto; en un KPI o un ranking de 10 barras solo añade ruido y tiempo de carga. Las etiquetas de `geom_text()` y algunos detalles de estilo no siempre se traducen igual; revisa el resultado.

6.10 Buenas prácticas profesionales

6.10.1 Separa la lógica de la interfaz

Ya lo aplicaste en las dos apps: las funciones de datos, KPIs y gráficas no tocan `input` ni `output`. Regla práctica: *si un bloque dentro de un `render...()` tiene más de 3 o 4 líneas, conviértelo en una función* y guárdala en R/. Ganas pruebas sencillas, reutilización (las mismas funciones sirven para el reporte automático del Módulo 8) y un servidor que se lee como un índice.

6.10.2 Lee los datos una sola vez y precalcula

Todo lo que está fuera del `server` se ejecuta al arrancar y lo comparten todos los usuarios; lo que está dentro se ejecuta *por cada usuario*. Por eso:

- Lee y une los datos fuera del `server` (como en ambas apps).
- Si los datos son grandes, prepáralos antes con un script aparte y guárdalos ya resumidos (por ejemplo en `.rds` con `saveRDS()`, que se lee mucho más rápido que un CSV), o consúltalos en una base de datos agregando en SQL (Módulo 5).
- Usa `reactive()` para no repetir cálculos entre salidas.

6.10.3 Caché con `bindCache()`

`reactive()` recuerda *el último* resultado. `bindCache()` va más allá: recuerda los resultados para *cada combinación* de valores que le indiques, y además los comparte entre usuarios. En la app 03:

```

1 datos_filtrados <- reactive({
2   req(input$fechas)
3   filtrar_datos(datos, input$fechas, input$tiendas, input$categorias)
4 }) %>%
5   bindCache(input$fechas, input$tiendas, input$categorias)

```

Si la directora mira “Outlet”, luego “Centro” y vuelve a “Outlet”, la tercera vez el resultado sale de la caché sin recalcular; y si un gerente abre la app con los mismos filtros, también lo aprovecha. También funciona con `renderPlot()`: `renderPlot(...) %>% bindCache(input$tiendas)` guarda las imágenes ya dibujadas. La regla de oro: en `bindCache()` pon *todas* las entradas de las que depende el cálculo; si olvidas una, mostrarás resultados viejos.

6.10.4 Módulos de Shiny: componentes reutilizables

Cuando un mismo bloque (una tarjeta de KPI, un selector de periodo, una gráfica con su tabla) se repite varias veces, copiar y pegar lleva a errores: hay que inventar ids distintos (`kpi1`, `kpi2...`) y cambiar cada copia a mano. Un **módulo de Shiny** es un par de funciones (una de `ui` y otra de `server`) que encapsulan ese bloque. El truco está en el **espacio de nombres** (*namespace*): `NS(id)` antepone el id del módulo a cada id interno, de modo que dos copias nunca chocan.

Ejemplo práctico: Módulo “tarjeta KPI”

Queremos una tarjeta reutilizable que reciba un valor reactivo, un título, un ícono y una función de formato, y muestre el `valueBox()`.

```

1 # Parte visual del módulo: todos los ids pasan por ns()
2 tarjeta_kpi_ui <- function(id, ancho = 3) {
3   ns <- NS(id)
4   valueBoxOutput(ns("caja"), width = ancho)
5 }
6
7 # Parte lógica: recibe un reactivo 'valor' desde fuera
8 tarjeta_kpi_server <- function(id, valor, titulo, icono,
9                               formato = formato_pesos) {
10  moduleServer(id, function(input, output, session) {
11    output$caja <- renderValueBox({
12      valueBox(formato(valor()), titulo, icon = icon(icono),
13              color = "blue")
14    })
15  })
16 }
17
18 tarjeta_kpi_ui("ventas")           # el id interno se vuelve "ventas-caja"

```

```
<div class="shiny-html-output col-sm-3" id="ventas-caja"></div>
```

El `id="ventas-caja"` muestra el namespace en acción. Con el módulo, la fila de KPIs del dashboard quedaría así:

```
1 # ui
2 fluidRow(
3   tarjeta_kpi_ui("ventas"),
4   tarjeta_kpi_ui("ticket"),
5   tarjeta_kpi_ui("clientes")
6 )
7 # server
8 tarjeta_kpi_server("ventas", reactive(kpis())$ventas),
9   "Ventas totales", "dollar-sign")
10 tarjeta_kpi_server("ticket", reactive(kpis())$ticket_promedio),
11   "Ticket promedio", "cart-shopping")
12 tarjeta_kpi_server("clientes", reactive(kpis())$clientes),
13   "Clientes activos", "users", formato = formato_numero)
```

Los módulos también se prueban con `testServer()`; los argumentos del módulo se pasan con `args`:

```
1 testServer(tarjeta_kpi_server,
2   args = list(valor = reactive(kpis_total$clientes),
3     titulo = "Clientes activos", icono = "users",
4     formato = formato_numero), {
5   cat(as.character(output$caja$html))
6 })
```

```
<div class="small-box bg-blue">
  <div class="inner">
    <h3>198</h3>
    <p>Clientes activos</p>
  </div>
  <div class="icon-large">
    <i class="fas fa-users" role="presentation" aria-label="users icon"></i>
  </div>
</div>
```

6.10.5 Prueba tu app con `testServer()`

Ya lo usaste en todo el módulo. En un proyecto real, guarda esas pruebas en una carpeta `tests/` y ejecútalas antes de publicar cada cambio (el paquete **testthat** las organiza con `expect_equal()`). Para probar además la parte visual (clics reales en un navegador) existe el paquete **shinytest2**.

6.10.6 Estructura de carpetas de un proyecto Shiny

Esta es una organización que escala bien de una app pequeña a una grande:

```

1 dashboard-ventas/
2 |-- app.R                <- solo ui, server y shinyApp()
3 |-- R/                   <- Shiny carga estos archivos solo
4 |   |-- funciones_datos.R <- cargar, limpiar, filtrar
5 |   |-- funciones_graficas.R <- una función por gráfica
6 |   +-- mod_tarjeta_kpi.R  <- módulos (ui + server)
7 |-- datos/               <- datos ya preparados (.rds, .csv)
8 |-- www/                 <- logo, CSS, imágenes
9 |-- tests/
10 |   +-- testthat/
11 |       +-- test-kpis.R   <- pruebas con testServer()
12 +-- preparar_datos.R     <- script que genera datos/ (no se publica)

```

La carpeta `www/` es especial: lo que pongas ahí (por ejemplo `logo.png`) queda disponible en la página como `img(src = "logo.png")`.

6.11 Publicar tu dashboard

Mientras la app corre en tu computadora, solo tú la ves. Para que la use tu equipo, debe correr en un **servidor**. Opciones:

- **shinyapps.io**: el servicio en la nube de Posit (la empresa detrás de RStudio). Tiene un plan gratuito con horas limitadas al mes, ideal para prototipos y portafolio.
- **Posit Connect**: servidor empresarial de pago que se instala en la infraestructura de la empresa; incluye autenticación con las cuentas corporativas, control de acceso por app y publicación de reportes R Markdown y Quarto.
- **Shiny Server** (versión gratuita, de código abierto): lo instala el área de TI en un servidor Linux propio. No incluye autenticación; hay que ponerla con otras herramientas.

Para publicar en shinyapps.io, crea una cuenta en <https://www.shinyapps.io>, copia tu *token* (menú **Account > Tokens**) y ejecuta una sola vez:

```

1 install.packages("rsconnect")
2 rsconnect::setAccountInfo(name = "tu_usuario",
3                           token = "TOKEN_DE_EJEMPLO",
4                           secret = "SECRETO_DE_EJEMPLO")

```

Antes de publicar, **copia los datos dentro de la carpeta de la app**: en el servidor no existe la carpeta del curso, así que `buscar_datasets()` no encontraría nada. Basta con crear `datasets/` dentro de la carpeta de la app (la función busca primero ahí, en el nivel 0). Luego:

```
1 rsconnect::deployApp("latex/codigo/apps/03-dashboard-ejecutivo",
2                       appName = "dashboard-ventas")
```

En uno o dos minutos obtienes una dirección del tipo https://tu_usuario.shinyapps.io/dashboard-ventas/. Cada vez que cambies la app, vuelve a ejecutar `deployApp()`. Si falla al arrancar en el servidor, revisa el registro con `rsconnect::showLogs()`: casi siempre es un paquete no instalado o un archivo de datos que no se subió.

▲ Cuidado

Seguridad. Una app en shinyapps.io con plan gratuito es *pública*: cualquiera con la dirección la ve. Nunca publiques así datos de clientes (nombres, correos, teléfonos), salarios o cifras confidenciales. Para datos internos usa un servidor con autenticación (Posit Connect, planes de pago de shinyapps.io o un servidor de la empresa), publica solo datos agregados o anonimizados, y jamás escribas contraseñas de bases de datos en el código: guárdalas en variables de entorno (`.Renviron`), como viste en el Módulo 5.

6.11.1 Alternativas sin servidor: flexdashboard y Quarto dashboards

Si tu tablero no necesita recalcular nada al vuelo (por ejemplo, un reporte semanal fijo), no necesitas Shiny ni un servidor: **flexdashboard** (con R Markdown) y los *dashboards de Quarto* generan un archivo HTML con aspecto de tablero, gráficas **plotly** y tablas **DT** interactivas, que puedes enviar por correo o subir a una intranet. El encabezado de un dashboard de Quarto se ve así:

```
1 ---
2 title: "Ventas 2023"
3 format: dashboard
4 ---
```

Verás cómo automatizar este tipo de reportes en el Módulo 8. Regla rápida: si el usuario necesita *filtrar y recalcular*, Shiny; si solo necesita *consultar* un resultado fijo con algo de interactividad, un HTML estático basta.

6.12 Errores comunes

1. Usar un reactivo sin paréntesis. Es el error más famoso. En esta mini app, `datos` es un reactivo y lo usamos como si fuera un data frame:

```
1 server_error <- function(input, output, session) {
2   datos <- reactive(filter(transacciones, tienda_id == input$tienda))
3   output$total <- renderText(sum(datos$total_transaccion)) # ¡error!
4 }
5 testServer(server_error, {
```

```

6  session$setInputs(tienda = 1)
7  tryCatch(output$total,
8           error = function(e) cat("Error:", conditionMessage(e), "\n"))
9  })

```

Error: objeto de tipo 'closure' no es subconjunto

Con R en inglés el mensaje es `object of type 'closure' is not subsettable`. La corrección: `sum(datos())$total_transaccion`. Ojo: `nrow(datos)` sin paréntesis *no* da error, devuelve `NULL` y la salida aparece vacía, lo que es aún más desconcertante.

2. Olvidar una coma entre elementos de la ui. R no puede ni leer el archivo:

```

1  codigo_ui <- 'fluidPage(
2    titlePanel("Ventas")
3    sliderInput("n", "Top N:", 5, 20, 10)
4  )'
5  tryCatch(parse(text = codigo_ui),
6           error = function(e) cat(conditionMessage(e), "\n"))

```

```

<text>:3:3: unexpected symbol
2:   titlePanel("Ventas")
3:   sliderInput
   ^

```

El error señala la línea 3, pero la coma falta al final de la línea 2. Con una coma *de más* después del último elemento el mensaje suele ser `argument is missing, with no default`.

Tabla 6.5. Otros errores frecuentes en Shiny y cómo resolverlos.

Síntoma o mensaje	Causa y solución
<code>Operation not allowed without an active reactive context</code>	Llamaste a un reactivo (<code>datos()</code>) en la consola o en el cuerpo del <code>server</code> . Muévelo dentro de un <code>render...()</code> , <code>reactive()</code> u <code>observe()</code> .
<code>Can't access reactive value 'x' outside of reactive consumer</code>	Usaste <code>input\$x</code> fuera de un contexto reactivo. Envuélvelo en <code>reactive()</code> .
<code>'datasets/...csv' does not exist in current working directory</code>	La app corre con su carpeta como directorio de trabajo. Usa <code>buscar_datasets()</code> o copia los datos a la carpeta de la app.
Una salida no aparece, sin error	El id de <code>xxxOutput("id")</code> no coincide con <code>output\$id</code> , o la pareja no corresponde (<code>plotOutput</code> con <code>renderTable</code>). Revisa la tabla 6.3.
Dos controles o salidas dejan de funcionar (en la consola del navegador, <code>F12</code> , aparece <code>Duplicate binding for ID ...</code>)	Usaste el mismo id dos veces. Los ids deben ser únicos en toda la app; con módulos, usa <code>ns()</code> .
La gráfica muestra un error rojo al abrir la app	Una entrada aún no tiene valor. Agrega <code>req(input\$...)</code> al inicio del <code>render</code> .
Un filtro múltiple vacío deja la app sin datos	<code>input\$x</code> vale <code>NULL</code> ; revisa <code>length(input\$x) > 0</code> antes de filtrar.
<code>there is no package called 'DT'</code> al publicar	El servidor no tiene el paquete: cárgalo con <code>library()</code> en <code>app.R</code> (no con <code>require()</code> dentro de funciones) para que <code>rsconnect</code> lo detecte.
La app funciona en tu computadora pero no en el servidor	Rutas absolutas (<code>C:/Users/...</code>), datos no subidos o paquetes faltantes. Revisa <code>rsconnect::showLogs()</code> .

 Consejo

Para depurar, pon `browser()` dentro de un `render...()` o `reactive()`: al ejecutarse, la app se pausa y la consola de RStudio te deja inspeccionar `input$...` y los objetos en ese momento. Escribe `c` para continuar. Quítalo antes de publicar.

Resumen del módulo

- Un buen dashboard tiene una audiencia y unas preguntas definidas, KPIs arriba, filtros juntos y pocas gráficas bien diseñadas. Shiny conviene cuando necesitas lógica a la medida en R; Power BI o Tableau, para reportes estándar de autoservicio.
- Toda app tiene `ui` (lo que se ve), `server` (la lógica) y `shinyApp()`. Cada salida es una pareja `xxxOutput()` + `renderXxx()` con el mismo id.
- La reactividad recalcula solo lo que depende de lo que cambió. `reactive()` comparte cálculos; `observeEvent()` hace acciones; `eventReactive()` espera a un botón; `req()` y `validate()` evitan errores; `isolate()` lee sin depender. Los reactivos se llaman con paréntesis.

- Organiza la pantalla con la cuadrícula de 12 columnas, `sidebarLayout()`, pestañas o **shinydashboard** (`box()`, `valueBox()`, `menuItem()`).
- Separa la lógica en funciones (carpeta R/), lee los datos una vez, usa `bindCache()` y módulos, y prueba con `testServer()`.
- Publica en shinyapps.io, Posit Connect o Shiny Server, sin exponer datos sensibles.

Función	Para qué sirve
<code>fluidPage()</code> , <code>sidebarLayout()</code>	Página y layout con barra lateral
<code>fluidRow()</code> , <code>column()</code>	Cuadrícula de 12 columnas
<code>tabsetPanel()</code> , <code>navbarPage()</code>	Pestañas y barra de navegación
<code>selectInput()</code> , <code>selectizeInput()</code>	Menús de una o varias opciones
<code>dateRangeInput()</code> , <code>sliderInput()</code>	Periodos y rangos
<code>actionButton()</code> , <code>downloadButton()</code>	Botones de acción y de descarga
<code>renderPlot()</code> , <code>renderText()</code> , <code>renderTable()</code>	Salidas básicas
<code>DT::renderDT()</code> , <code>plotly::renderPlotly()</code>	Tabla y gráfica interactivas
<code>reactive()</code> , <code>eventReactive()</code>	Cálculos reactivos compartidos
<code>observeEvent()</code> , <code>update...Input()</code>	Acciones y cambios de entradas
<code>req()</code> , <code>validate(need())</code>	Condiciones antes de calcular
<code>dashboardPage()</code> , <code>box()</code> , <code>valueBox()</code>	Estructura de shinydashboard
<code>downloadHandler()</code>	Contenido de una descarga
<code>bindCache()</code>	Caché de resultados
<code>NS()</code> , <code>moduleServer()</code>	Módulos reutilizables
<code>testServer()</code>	Probar el servidor sin navegador
<code>runApp()</code> , <code>rsconnect::deployApp()</code>	Ejecutar y publicar

6.13 Ejercicios

Para los ejercicios que modifican una app, copia primero su carpeta (por ejemplo, `latex/codigo/apps/02-explorador-ventas` a `mis-apps/02-mi-explorador`) y trabaja sobre la copia. Además de abrirla en el navegador, comprueba la lógica con `testServer()`.

Ejercicio 6.1: Calculadora con cantidad e IVANivel: **Básico**

Amplía la app 01: agrega un `numericInput()` para la *cantidad* de piezas (valor inicial 1) y un `checkboxInput()` “Incluir IVA (16%)”. La app debe mostrar el total a pagar (precio $\times$ cantidad, menos el descuento y, si se marca la casilla, más IVA). Comprueba con `testServer()` que 3 piezas de \$500 con 10 % de descuento e IVA dan \$1,566.

 Solución en la página 667**Ejercicio 6.2: Filtro por método de pago**Nivel: **Básico**

En el explorador de ventas (app 02) agrega un `checkboxGroupInput()` con los cuatro métodos de pago, todos marcados al inicio, y haz que filtre todas las salidas. ¿Cuántas transacciones y cuántas ventas quedan si solo dejas “Efectivo”?

 Solución en la página 668**Ejercicio 6.3: Un KPI nuevo: margen bruto**Nivel: **Intermedio**

En el dashboard ejecutivo (app 03) agrega en la pestaña Resumen una quinta fila con un `valueBox()` de *margen bruto* (ventas $-$ costo $\times$ cantidad) y otro con el *% de margen*. La caja del margen debe ser verde si es positivo y roja si es negativo. Escribe primero la función `calcular_margen(datos)` y pruébala: ¿qué margen tiene la categoría Belleza? (Recuerda el hallazgo de calidad de datos del curso: hay productos cuyo costo supera su precio).

 Solución en la página 669**Ejercicio 6.4: Aplicar filtros solo al pulsar un botón**Nivel: **Intermedio**

Escribe una app con un `selectizeInput()` de tiendas (múltiple), un `dateRangeInput()` y un botón “Aplicar”. Las ventas totales y el número de transacciones deben actualizarse *solo* al pulsar el botón (usa `eventReactive()`). Demuestra con `testServer()` que cambiar los filtros sin pulsar no altera el resultado.

 Solución en la página 671**Ejercicio 6.5: Una pestaña nueva: días de la semana**Nivel: **Intermedio**

Agrega al dashboard ejecutivo una pestaña “Días” (con su `menuItem()` y su `tabItem()`) con una gráfica de barras de ventas por día de la semana (lunes a domingo, en ese orden) que resalte el mejor día. Respeta los filtros globales. ¿Qué día vende más en todo 2023?

 Solución en la página 672

Ejercicio 6.6: Descargar el top de productosNivel: **Intermedio**

En la pestaña “Top productos” de la app 02 agrega dos botones: “Descargar CSV” y “Descargar Excel”. El archivo debe contener exactamente la tabla que se está viendo (con el número de productos del deslizador y los filtros activos) y llamarse `top_productos_AAAA-MM-DD`. Prueba el `downloadHandler()` con `testServer()`.

 Solución en la página 674**Ejercicio 6.7: Un módulo de ranking reutilizable**Nivel: **Avanzado**

Crea un módulo `ranking_ui(id, titulo)` / `ranking_server(id, datos, columna)` que muestre dentro de una `box()` un `sliderInput()` “Mostrar top” (3 a 10) y una gráfica de barras horizontales con las ventas por la columna indicada. Úsalo dos veces en la misma app: para productos ("`nombre_producto`") y para ciudades ("`ciudad`"), y verifica con `testServer()` que cada copia responde a su propio deslizador.

 Solución en la página 675**Ejercicio 6.8: Reto: dashboard de Recursos Humanos**Nivel: **Reto**

Construye desde cero, con **shinydashboard**, un dashboard para el área de Recursos Humanos con `empleados.csv`: filtros de departamento y sucursal (múltiples); cuatro `valueBox()` (plantilla, salario mensual promedio, antigüedad promedio y % con maestría o doctorado); una gráfica de salario promedio por puesto; una tabla **DT** con formato de moneda y un botón para descargar la lista en Excel. Separa la lógica en funciones y pruébalas.

 Solución en la página 677

PARTE III

Nivel avanzado

Machine learning para Business Intelligence

Al terminar este módulo podrás...

- Explicar con palabras de negocio qué es el *machine learning*, qué problemas resuelve y cuáles no.
- Distinguir aprendizaje supervisado (regresión y clasificación) y no supervisado (clustering y reglas de asociación), y elegir el algoritmo adecuado para cada pregunta de negocio.
- Aplicar el flujo de trabajo completo: separar datos de entrenamiento y prueba, entrenar, evaluar con las métricas correctas y evitar el sobreajuste y la fuga de datos.
- Segmentar clientes con RFM y *k-means*, y darle un nombre de negocio a cada segmento.
- Predecir ventas con regresión lineal y *random forest*, y medir el error con MAE, RMSE, MAPE y R^2 .
- Predecir qué clientes van a abandonar (*churn*) con regresión logística, árboles de decisión y *random forest*; leer una matriz de confusión, una curva ROC y elegir el umbral según el costo.
- Pronosticar la demanda mensual con modelos de series de tiempo (ingenuo, ETS y ARIMA) y comunicar la incertidumbre.
- Descubrir qué productos se compran juntos (análisis de canasta) para campañas de venta cruzada.
- Guardar un modelo, usarlo para puntuar clientes nuevos, vigilar que siga funcionando y usarlo de forma ética.

Es lunes por la mañana y la directora comercial te escribe tres preguntas: “¿Qué clientes están a punto de irse y a quiénes llamamos primero?”, “¿Cuánto vamos a vender en diciembre para pedir el inventario a tiempo?” y “¿A qué clientes les mandamos la campaña VIP?”. Hasta ahora, con lo que aprendiste en los módulos anteriores, sabes responder muy bien *qué pasó*: cuánto se vendió, en qué tienda, qué producto fue el más rentable. Estas tres preguntas son distintas: piden *anticipar* lo que va a pasar y *decidir* qué hacer. Ahí entra el **machine learning** (aprendizaje automático).

En este módulo vas a construir, paso a paso y con código comentado, los modelos que más se usan en Business Intelligence: segmentación de clientes, predicción de ventas, predicción de abandono, pronóstico de demanda y análisis de canasta. No necesitas matemáticas avanzadas: te explicaremos cada modelo con analogías de negocio, y nos concentraremos en lo que de verdad importa en una

empresa: **hacer la pregunta correcta, evaluar honestamente el modelo y convertir el resultado en una decisión.**

Concepto clave: Las cuatro preguntas de la analítica

- **Descriptiva:** ¿qué pasó? (reportes, tableros; Módulos 2 y 3).
- **Diagnóstica:** ¿por qué pasó? (análisis exploratorio, Módulo 4).
- **Predictiva:** ¿qué va a pasar? (machine learning, este módulo).
- **Prescriptiva:** ¿qué debemos hacer? (combinar la predicción con costos y reglas de negocio; también lo verás aquí, por ejemplo al elegir a qué clientes llamar).

Una advertencia honesta desde el principio: los datos del curso (por ejemplo, `ventas_retail.csv` o `transacciones.csv`) se generaron *al azar*. Sirven perfectamente para la segmentación de clientes, porque ahí solo queremos *describir* grupos, pero no tienen patrones que un modelo predictivo pueda aprender. Por eso, para predicción de ventas, abandono de clientes, series de tiempo y canastas vamos a **simular datos con relaciones conocidas** (como si fueran los datos de un cliente real). Tiene una ventaja didáctica enorme: como sabemos la “verdad” con la que se generaron, podremos comprobar si el modelo la descubre. Y en la sección 7.1.4 verás, con los datos del curso, la primera gran lección del machine learning: **sin señal no hay modelo**.

7.1 ¿Qué es machine learning para negocio?

Piensa en la gerente de una sucursal con 15 años de experiencia. Con solo ver a un cliente “sabe” si va a volver: se fija en cuánto tiempo lleva comprando, si se ha quejado, si viene seguido. Nadie le dio una fórmula; la aprendió viendo miles de casos. El machine learning hace lo mismo, pero con datos: revisa miles de casos históricos, encuentra el patrón y lo aplica a casos nuevos, de forma consistente y a escala (a 100,000 clientes en segundos).

Concepto clave: Machine learning

El **machine learning** es un conjunto de técnicas con las que la computadora **aprende patrones a partir de datos históricos** para hacer predicciones o encontrar grupos, sin que alguien programe las reglas a mano.

- Un **modelo** es una “receta” que convierte datos de entrada (antigüedad, quejas, uso...) en una salida (probabilidad de abandono).
- **Entrenar** un modelo es ajustar esa receta usando datos históricos donde ya conocemos la respuesta.
- **Predecir** (o *puntuar*) es aplicar la receta a datos nuevos, donde todavía no conocemos la respuesta.

7.1.1 Supervisado y no supervisado

Hay dos grandes familias de problemas, y distinguirlas es el primer paso para elegir la herramienta correcta:

- **Aprendizaje supervisado:** tienes ejemplos históricos *con la respuesta* (este cliente se fue, este no; esta tienda vendió \$850,000). El modelo aprende a predecir esa respuesta.
 - **Regresión:** la respuesta es un **número** (ventas, demanda, precio, días de retraso).
 - **Clasificación:** la respuesta es una **categoría** (se va / se queda; fraude / no fraude; compra / no compra).
- **Aprendizaje no supervisado:** *no* hay una respuesta que predecir; buscas estructura oculta en los datos.
 - **Clustering** (agrupamiento): formar grupos de clientes, tiendas o productos parecidos entre sí.
 - **Reglas de asociación:** descubrir qué cosas ocurren juntas (quien compra laptop también compra mouse).

Los **pronósticos de series de tiempo** son un caso especial de regresión: predicen un número futuro usando la historia de la propia variable (tendencia y estacionalidad). La tabla 7.1 traduce preguntas de negocio típicas a tipo de modelo y algoritmo.

Tabla 7.1. De la pregunta de negocio al algoritmo.

Pregunta de negocio	Tipo de problema	Algoritmos típicos
¿Cuánto venderá cada tienda el próximo mes?	Regresión	Regresión lineal, random forest
¿Qué clientes cancelarán su suscripción?	Clasificación	Regresión logística, árbol, random forest
¿Esta transacción es fraudulenta?	Clasificación (desbalanceada)	Regresión logística, random forest
¿Cuántas unidades pediremos al proveedor en diciembre?	Serie de tiempo	Ingenuo estacional, ETS, ARIMA
¿Qué grupos de clientes tenemos?	Clustering	RFM + k-means, clustering jerárquico
¿Qué productos se compran juntos?	Reglas de asociación	Soporte–confianza–lift, Apriori
¿Qué sucursales se parecen entre sí?	Clustering	k-means con variables escaladas
¿Aprobamos este crédito a un cliente?	Clasificación	Regresión logística (fácil de explicar)
¿Cuánto gastará un cliente en su vida con nosotros?	Regresión	Regresión lineal, random forest

7.1.2 El flujo de trabajo de un proyecto de machine learning

Un proyecto de machine learning no es “correr un algoritmo”. Es un ciclo que empieza y termina en el negocio (figura 7.1).

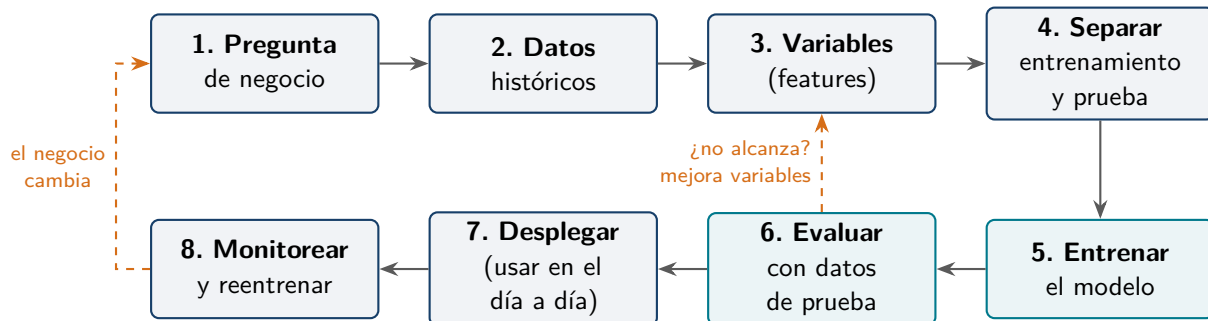


Figura 7.1. Flujo de trabajo de machine learning. Las flechas punteadas indican que el proceso es un ciclo, no una línea recta.

1. **Pregunta de negocio.** “¿Qué clientes se van a ir en los próximos 3 meses?” es una buena pregunta; “haz algo con IA” no lo es. Define también qué decisión tomarás con la respuesta.
2. **Datos históricos.** Casos pasados donde ya conoces el resultado (clientes que se fueron y que se quedaron).
3. **Variables (features).** Las características que el modelo usará: antigüedad, número de quejas, uso del servicio... Aquí se gana o se pierde la mayor parte del valor.
4. **Separar entrenamiento y prueba.** Guardas una parte de los datos que el modelo *nunca* ve al aprender, para evaluarlo honestamente.
5. **Entrenar** uno o varios algoritmos.
6. **Evaluar** con métricas que tengan sentido para el negocio.
7. **Desplegar:** poner el modelo a trabajar (un CSV semanal para el call center, una columna en el tablero, una API).
8. **Monitorear y reentrenar:** los clientes, los precios y la competencia cambian; un modelo que no se vigila se degrada.

En la empresa: El 80 % del trabajo no es el algoritmo

En proyectos reales, la mayor parte del tiempo se va en los pasos 1 a 4: entender la pregunta, conseguir y limpiar datos, y construir buenas variables (todo lo que aprendiste en los Módulos 2 a 4). Entrenar el modelo son unas cuantas líneas de código. Un modelo sencillo con buenas variables casi siempre le gana a un modelo sofisticado con variables pobres.

7.1.3 Paquetes que usaremos

Este módulo usa, además del **tidyverse**, estos paquetes: **cluster** y **factoextra** (clustering), **rpart** (árboles de decisión), **randomForest** (bosques aleatorios), **caret** (marco común para entrenar y

comparar modelos) y **forecast** (series de tiempo). Si no los tienes, instálalos una sola vez:

```
1 install.packages(c("cluster", "factoextra", "rpart", "randomForest",
2                   "caret", "forecast"))
```

Y cárgalos al inicio de tu script:

```
1 library(tidyverse)      # dplyr, ggplot2, readr, tidyr, purrr...
2 library(cluster)        # silueta y otras medidas de clustering
3 library(factoextra)     # gráficas de clustering (devuelven ggplot)
4 library(rpart)          # árboles de decisión
5 library(randomForest)   # bosques aleatorios
6 library(caret)          # entrenar y comparar modelos con un mismo formato
7 library(forecast)       # series de tiempo: ETS, ARIMA, pronósticos
8
9 # Comillas simples normales en las salidas de R (en lugar de tipográficas)
10 options(useFancyQuotes = FALSE)
11
12 # Colores corporativos del libro (Módulo 3)
13 color_principal <- "#2a78d6"
14 color_resalte   <- "#eb6834"
15 color_gris      <- "#b5b3ad"
16 paleta_libro <- c("#2a78d6", "#eb6834", "#1baf7a", "#eda100",
17                  "#e87ba4", "#008300", "#4a3aa7", "#e34948")
```

Al cargarlos verás algunos mensajes normales, no errores: **caret** avisa que carga **lattice**, **forecast** menciona un método de **quantmod**, y **randomForest** avisa que “enmascara” las funciones **combine()** de **dplyr** y **margin()** de **ggplot2**. Eso significa que, si necesitas esas dos funciones, debes escribirlas con su paquete: **dplyr::combine()** o **ggplot2::margin()**.

7.1.4 Sin señal no hay modelo

Antes de entrenar cualquier cosa, veamos la lección más importante del módulo. Un modelo solo puede aprender un patrón *si el patrón existe en los datos*. Intentemos explicar el monto de cada compra de **transacciones.csv** con la tienda, el método de pago y el día de la semana, usando una regresión lineal (la viste en el Módulo 4).

```
1 # Transacciones del curso (1,000 compras de 2023)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4
5 # ¿El monto depende de la tienda, el método de pago o el día?
6 modelo_sin_senal <- lm(
7   total_transaccion ~ factor(tienda_id) + metodo_pago + dia_semana,
8   data = transacciones
9 )
```

```

10
11 # R2: proporción de la variación del monto que explica el modelo (0 a 1)
12 summary(modelo_sin_senal)$r.squared
13 summary(modelo_sin_senal)$adj.r.squared # R2 ajustado

```

```

[1] 0.01406052
[1] -0.00403011

```

El modelo explica apenas 1.4 % de la variación del monto, y el **R² ajustado** (que penaliza agregar variables inútiles) es incluso negativo: el modelo es *peor* que simplemente predecir el promedio. No es un error de código: el generador de datos eligió tienda, método de pago y monto de forma independiente, al azar. No hay nada que aprender.

▲ Cuidado

Cuidado con la “señal” falsa. Si agregas **cantidad** al modelo anterior, el R² sube a 0.45. ¿Descubrimos algo? No: en este archivo **total_transaccion** se *calcula* como **cantidad** × **precio_venta**, así que es aritmética, no un patrón de comportamiento. Usar como predictora una variable que es parte del cálculo de la respuesta es una forma de **fuga de datos** (sección 7.2.5).

Por eso, en las secciones de predicción simularemos datos donde *sí* existe una relación (por ejemplo, las quejas aumentan la probabilidad de abandono). En tu empresa, los datos reales tendrán señal (y ruido); tu trabajo será encontrarla y medir cuánta hay.

🔧 Hazlo tú

Ajusta `lm(total_transaccion ~ cantidad, data = transacciones)` y revisa su R². Luego piensa: si mañana quisieras predecir el monto de una compra *antes* de que ocurra, ¿conocerías la cantidad? ¿Te sirve ese modelo?

7.2 Conceptos clave antes de entrenar

7.2.1 Variables predictoras y variable objetivo

En aprendizaje supervisado cada fila es una **observación** (un cliente, una tienda, un mes) y las columnas se dividen en dos tipos:

- La **variable objetivo** (*target*, respuesta, *y*): lo que quieres predecir. Ejemplo: **churn** (si el cliente se fue).
- Las **variables predictoras** (*features*, *X*): la información que usarás para predecir. Ejemplo: antigüedad, quejas, uso, plan contratado.

Tabla 7.2. Ejemplo de datos para predecir abandono: cuatro predictoras y una variable objetivo.

Cliente	Antigüedad	Quejas	Uso mensual	Plan	¿Se fue? (<i>y</i>)
C0001	36	0	11	Premium	No
C0002	4	3	5	Básico	Sí
C0003	18	1	8	Estándar	No
<i>variables predictoras (X)</i>					<i>objetivo</i>

La regla de oro: **una predictora solo es válida si la conocerás en el momento de predecir**. Si quieres saber hoy quién se irá el próximo mes, no puedes usar “motivo de cancelación”, porque ese dato solo existe cuando el cliente ya se fue.

7.2.2 Entrenamiento y prueba

Imagina que preparas a un vendedor con 100 casos resueltos y luego lo evalúas con *esos mismos* 100 casos: sacará 10, pero no sabrás si aprendió o solo memorizó. Con los modelos pasa igual. Por eso separamos los datos:

- **Conjunto de entrenamiento** (*train*, 70–80 %): con él aprende el modelo.
- **Conjunto de prueba** (*test*, 20–30 %): se guarda bajo llave y solo se usa al final para medir qué tan bien predice casos que nunca vio.

Veámoslo con un ejemplo que usaremos en las siguientes secciones: 60 sucursales, cada una con su inversión mensual en publicidad y sus ventas del mes (ambas en miles de pesos). Lo simulamos con una relación conocida: $ventas = 200 + 6 \times publicidad + ruido$.

```

1 set.seed(123) # fija el azar: cualquiera que corra esto obtiene lo mismo
2 sucursales <- tibble(
3   sucursal    = 1:60,
4   publicidad  = round(runif(60, min = 5, max = 50), 1), # miles de $
5   ventas      = round(200 + 6 * publicidad + rnorm(60, 0, 25), 1)
6 )
7
8 # Elegimos al azar 45 filas (75 %) para entrenar
9 set.seed(123)
10 filas_ent <- sample(nrow(sucursales), size = 45)
11
12 suc_entrenamiento <- sucursales[filas_ent, ] # 45 sucursales
13 suc_prueba       <- sucursales[-filas_ent, ] # las 15 restantes
14
15 nrow(suc_entrenamiento)
16 nrow(suc_prueba)

```

[1] 45

[1] 15

`sample()` desordena al azar los números de fila y toma 45; el signo menos en `sucursales[-filas_ent, ]` significa “todas las filas *excepto* esas”. Gracias a `set.seed(123)`, cada vez que ejecutes el código obtendrás exactamente la misma división, algo indispensable para que tus resultados sean reproducibles (y para que coincidan con los del libro).

Consejo

Para problemas de clasificación con una clase poco frecuente (por ejemplo, 20% de clientes que se van), usa `caret::createDataPartition()`: hace una división **estratificada**, es decir, mantiene la misma proporción de cada clase en entrenamiento y prueba. Lo usaremos en la sección de churn.

7.2.3 Sobreajuste: cuando el modelo memoriza

El **sobreajuste** (*overfitting*) ocurre cuando un modelo es tan flexible que aprende de memoria los datos de entrenamiento, incluido su ruido, y falla con datos nuevos. Es como el estudiante que memoriza las respuestas del examen de práctica y reprueba el examen real.

Para verlo, supongamos que solo tuviéramos 14 sucursales para entrenar y ajustemos polinomios cada vez más flexibles (grado 1 es una recta; grado 10 es una curva con muchas ondas). Medimos el error con el **RMSE** (raíz del error cuadrático medio: el tamaño típico del error, en las mismas unidades que las ventas; lo verás a detalle en la sección 7.4.3).

```
1 # Función auxiliar: error típico de una predicción
2 rmse <- function(real, predicho) sqrt(mean((real - predicho)^2))
3
4 pocas_sucursales <- sucursales %>% slice(1:14)      # entrenamiento chico
5 resto_sucursales <- sucursales %>% slice(15:60)    # 46 casos "nuevos"
6
7 errores_por_grado <- map_dfr(1:10, function(g) {
8   modelo <- lm(ventas ~ poly(publicidad, g), data = pocas_sucursales)
9   tibble(
10     grado = g,
11     error_entrenamiento = rmse(pocas_sucursales$ventas, fitted(modelo)),
12     error_prueba = rmse(resto_sucursales$ventas,
13                          predict(modelo, resto_sucursales))
14   )
15 })
16
17 errores_por_grado %>% mutate(across(-grado, ~ round(.x, 1)))
```

```
# A tibble: 10 x 3
  grado error_entrenamiento error_prueba
```

	<int>	<dbl>	<dbl>
1	1	20.3	22
2	2	20.3	22.2
3	3	20.2	22.1
4	4	20.2	22
5	5	19.8	26.3
6	6	17.6	56.6
7	7	17.4	35.6
8	8	15.9	669.
9	9	13.1	2945.
10	10	10	14455.

Lee la tabla por columnas. El **error de entrenamiento** baja siempre que el modelo se vuelve más flexible: la curva se acerca cada vez más a los 14 puntos. Pero el **error de prueba**, el que importa, empieza a subir en el grado 5 y se dispara a partir del grado 8 (¡más de 14,000 en el grado 10!); el modelo memorizó el ruido. La recta (grado 1), que coincide con la forma real de los datos, es la que mejor generaliza. La figura 7.2 lo muestra visualmente.

```

1 malla <- tibble(publicidad = seq(min(pocas_sucursales$publicidad),
2                               max(pocas_sucursales$publicidad),
3                               length.out = 300))
4 recta <- lm(ventas ~ publicidad, data = pocas_sucursales)
5 memoria <- lm(ventas ~ poly(publicidad, 10), data = pocas_sucursales)
6
7 curvas <- bind_rows(
8   malla %>% mutate(modelo = "Grado 1: generaliza",
9                     ventas = predict(recta, malla)),
10  malla %>% mutate(modelo = "Grado 10: memoriza",
11                    ventas = predict(memoria, malla))
12 )
13
14 grafico_sobreaajuste <- ggplot() +
15   geom_point(data = resto_sucursales, aes(publicidad, ventas),
16             color = color_gris, alpha = 0.8) +
17   geom_point(data = pocas_sucursales, aes(publicidad, ventas),
18             color = "black", size = 2.2) +
19   geom_line(data = curvas, aes(publicidad, ventas, color = modelo),
20            linewidth = 0.8) +
21   scale_color_manual(values = c(color_principal, color_resalte)) +
22   coord_cartesian(ylim = c(150, 600)) +
23   labs(title = "Un modelo demasiado flexible memoriza el ruido",
24        subtitle = paste("Negro: 14 sucursales de entrenamiento;",
25                          "gris: sucursales nuevas"),
26        x = "Publicidad (miles de $)", y = "Ventas (miles de $)",
27        color = NULL) +
28   theme_minimal() +
29   theme(panel.grid.minor = element_blank(), legend.position = "top")
30 grafico_sobreaajuste

```

Un modelo demasiado flexible memoriza el ruido

Negro: 14 sucursales de entrenamiento; gris: sucursales nuevas

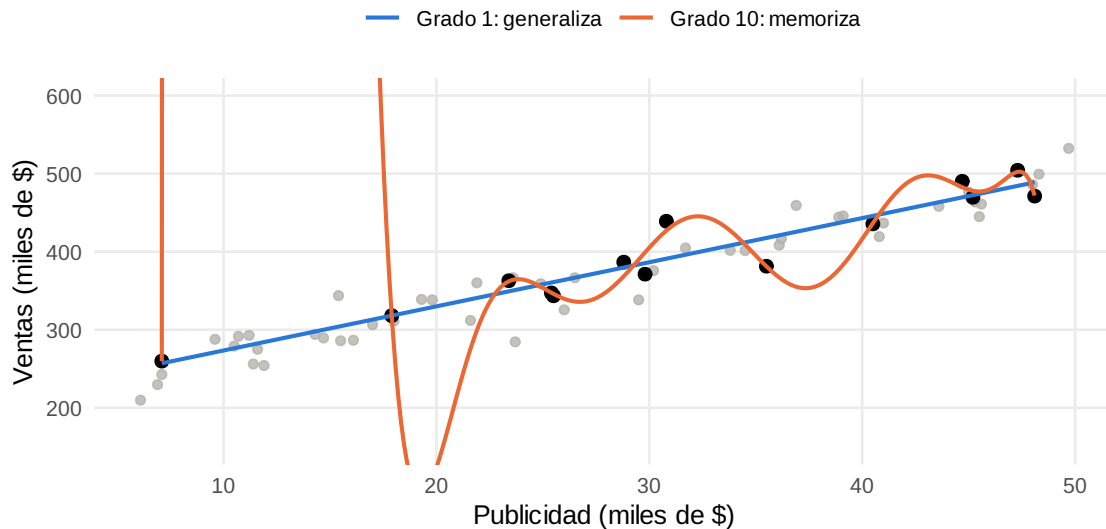


Figura 7.2. La recta (azul) captura la relación real; el polinomio de grado 10 (naranja) pasa cerca de cada punto de entrenamiento, pero hace predicciones absurdas para sucursales nuevas.

Concepto clave: Sesgo y varianza, en una frase

Un modelo demasiado **simple** no captura el patrón (*subajuste*); uno demasiado **complejo** captura hasta el ruido (*sobreajuste*). El objetivo es el punto medio, y la única forma honesta de encontrarlo es medir el error en datos que el modelo no usó para aprender.

7.2.4 Validación cruzada

Con una sola división entrenamiento/prueba, el resultado depende un poco de la suerte: quizá justo quedaron casos fáciles en la prueba. La **validación cruzada** de k pliegues (*k-fold cross-validation*) lo resuelve: parte los datos en k grupos (típicamente 5 o 10), entrena k veces dejando fuera un grupo distinto cada vez, y promedia los k errores (figura 7.3).

Ronda 1	validación	entrena	entrena	entrena	entrena	error ₁
Ronda 2	entrena	validación	entrena	entrena	entrena	error ₂
Ronda 3	entrena	entrena	validación	entrena	entrena	error ₃
Ronda 4	entrena	entrena	entrena	validación	entrena	error ₄
Ronda 5	entrena	entrena	entrena	entrena	validación	error ₅

promedio

Figura 7.3. Validación cruzada de 5 pliegues: cada sucursal se usa 4 veces para entrenar y exactamente una vez para validar.

Hagámoslo a mano una vez, para entender lo que después hará **caret** automáticamente. Comparamos la recta con polinomios de grado 3, 6 y 10 usando las 60 sucursales:

```

1 set.seed(123)
2 # Asigna cada sucursal a uno de 5 pliegues, al azar y en partes iguales
3 pliegue <- sample(rep(1:5, length.out = nrow(sucursales)))
4 table(pliegue)
5
6 error_cv <- function(grado) {
7   errores <- map_dbl(1:5, function(k) {
8     entrena <- sucursales[pliegue != k, ] # 4 pliegues para aprender
9     valida <- sucursales[pliegue == k, ] # 1 pliegue para evaluar
10    modelo <- lm(ventas ~ poly(publicidad, grado), data = entrena)
11    rmse(valida$ventas, predict(modelo, valida))
12  })
13  mean(errores) # promedio de los 5 errores
14 }
15
16 tibble(grado = c(1, 3, 6, 10)) %>%
17   mutate(rmse_cv = map_dbl(grado, error_cv))

```

```

pliegue
 1  2  3  4  5
12 12 12 12 12
# A tibble: 4 x 2
  grado rmse_cv
  <dbl>   <dbl>
1     1    21.5
2     3    22.2
3     6    22.6
4    10    29.0

```

Primero, `table(pliegue)` confirma que cada pliegue tiene 12 sucursales. Luego, la recta tiene el menor error de validación cruzada (21.5 mil pesos), y el error crece con la complejidad hasta 29.0 con el grado 10. La validación cruzada llega a la misma conclusión que la figura, pero usando las 60 sucursales y sin depender de una sola división afortunada. En la práctica la usarás para **elegir entre modelos o ajustar sus parámetros**, y reservarás el conjunto de prueba para la evaluación final.

7.2.5 Fuga de datos: el error más caro

La **fuga de datos** (*data leakage*) ocurre cuando, al entrenar, el modelo tiene acceso a información que **no estará disponible en el momento real de predecir**. El resultado es un modelo espectacular en el laboratorio y inútil en la operación. Es el error más común y más caro del machine learning, porque no produce ningún mensaje de error: produce *optimismo*.

En la empresa: Un modelo de abandono “perfecto”

Un equipo entrena un modelo de churn y obtiene 98 % de exactitud. Entre las predictoras incluyó `dias_desde_ultima_llamada_a_cancelaciones`. Pero los clientes llaman al área de cancelaciones *cuando ya decidieron irse*: esa variable es una consecuencia del abandono, no una causa observable con anticipación. En producción, el modelo avisa cuando el cliente ya está cancelando, demasiado tarde para retenerlo.

Formas típicas de fuga en BI:

- **Variables del futuro:** motivo de cancelación, fecha de baja, monto de reembolso, “estatus final” del pedido.
- **Variables calculadas con la respuesta:** como el `total_transaccion` que ya contiene a `cantidad`.
- **Preprocesar con todos los datos:** escalar, imputar o calcular promedios usando también los datos de prueba antes de separarlos.
- **Series de tiempo mezcladas:** separar al azar meses de una serie (entrenas con marzo de 2025 para “predecir” febrero de 2025). En series de tiempo, la prueba siempre son los periodos más recientes.

En la sección de errores comunes (sección 7.9) verás con código cómo una sola variable con fuga convierte un modelo mediocre en uno aparentemente perfecto.

** Hazlo tú**

Piensa en tu empresa: escribe tres variables que *parecerían* excelentes para predecir si un pedido llegará tarde, pero que en realidad solo se conocen después de la entrega.

7.3 Caso 1: segmentación de clientes con RFM y k-means

Ejemplo práctico: ¿A quién le mandamos la campaña VIP?

El área de marketing tiene presupuesto para tres campañas distintas y quiere dejar de mandar el mismo correo a los 200 clientes. La pregunta es: *¿qué grupos de clientes con comportamiento de compra parecido tenemos y qué estrategia conviene para cada uno?* Es un problema **no supervisado**: no hay una respuesta correcta que predecir; queremos descubrir grupos. Usaremos los datos reales del curso: `transacciones.csv` y `clientes.csv`.

7.3.1 Paso 1: calcular Recencia, Frecuencia y Monto

Concepto clave: RFM

El análisis **RFM** resume el comportamiento de cada cliente en tres números, muy usados en retail y marketing directo:

- **Recencia (R):** días desde su última compra. *Menos* es mejor: quien compró hace poco tiene más probabilidad de volver.
- **Frecuencia (F):** número de compras en el periodo. *Más* es mejor.
- **Monto (M):** dinero total gastado en el periodo. *Más* es mejor.

La recencia se mide respecto a una **fecha de corte** fija (por ejemplo, el día en que haces el análisis). Fíjala en el código; si usas `Sys.Date()`, tus resultados cambiarán cada día.

Las transacciones cubren todo 2023, así que usaremos como fecha de corte el 1 de enero de 2024:

```
1 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
2
3 fecha_corte <- as.Date("2024-01-01") # fecha fija del análisis
4
5 rfm <- transacciones %>%
6   group_by(cliente_id) %>%
7   summarise(
8     ultima_compra = max(fecha),
9     recencia      = as.numeric(fecha_corte - ultima_compra), # días
10    frecuencia    = n(), # compras
11    monto         = sum(total_transaccion), # pesos
12    .groups = "drop"
13  )
14
15 rfm
```

```
# A tibble: 198 x 5
  cliente_id ultima_compra recencia frecuencia monto
  <dbl> <date>          <dbl>      <int> <dbl>
1         1 2023-12-01         31         7 4859.
2         2 2023-12-02         30         5 7950.
3         3 2023-11-04         58         8 9562
4         4 2023-12-24          8         5 7759.
5         5 2023-12-31          1        13 14680.
6         6 2023-12-31          1         6 6087.
7         7 2023-12-13         19         5 1552.
8         8 2023-12-05         27         5 3555.
9         9 2023-12-23          9         6 5071.
10        10 2023-07-09        176         4 2487.
# i 188 more rows
```

Cada fila es ahora un cliente. El cliente 1, por ejemplo, compró por última vez el 1 de diciembre (31 días antes del corte), hizo 7 compras y gastó \$4,859. Hay 198 filas, pero `clientes.csv` tiene 200 clientes. ¿Quiénes faltan?

```
1 # Clientes registrados que no aparecen en las transacciones
2 clientes %>%
3   anti_join(rfm, by = "cliente_id") %>%
4   select(cliente_id, nombre, fecha_registro)
```

```
# A tibble: 2 x 3
  cliente_id nombre          fecha_registro
  <dbl> <chr>                <date>
1      171 Rosa Jiménez      2022-08-27
2      199 Ricardo Hernández 2023-09-16
```

Dos clientes no compraron nada en 2023. No entran al clustering (no tienen recencia ni frecuencia), pero no los olvides: en la práctica formarían su propio grupo, “registrados sin compra”, con una estrategia de activación.

```
1 rfm %>%
2   select(recencia, frecuencia, monto) %>%
3   summary()
```

	recencia	frecuencia	monto
Min.	: 1.00	Min. : 1.000	Min. : 152.6
1st Qu.:	21.00	1st Qu.: 3.000	1st Qu.: 3101.4
Median :	48.00	Median : 5.000	Median : 4521.8
Mean :	69.19	Mean : 5.051	Mean : 4848.4
3rd Qu.:	98.75	3rd Qu.: 6.000	3rd Qu.: 6183.0
Max.	:314.00	Max. :13.000	Max. :14680.1

Observa las escalas: la recencia va de 1 a 314 días, la frecuencia de 1 a 13 compras y el monto de 153 a 14,680 pesos. Esto es crucial para el siguiente paso.

7.3.2 Paso 2: escalar las variables

Concepto clave: Por qué escalar antes de k-means

k-means agrupa clientes según la *distancia* entre ellos. Si una variable se mide en miles (monto) y otra en unidades (frecuencia), la distancia la domina casi por completo la de números grandes: una diferencia de 2 compras “pesaría” lo mismo que una diferencia de 2 pesos. **Escalar** (estandarizar) convierte cada variable a media 0 y desviación estándar 1, para que las tres pesen lo mismo. En R se hace con `scale()`.

```

1 rfm_escalado <- rfm %>%
2   select(recencia, frecuencia, monto) %>%
3   scale()                                # (x - media) / desviación estándar
4
5 round(colMeans(rfm_escalado), 2)          # medias: todas 0
6 round(apply(rfm_escalado, 2, sd), 2)     # desviaciones: todas 1
7 head(round(rfm_escalado, 2), 3)

```

```

      recencia frecuencia      monto
      0           0           0
      recencia frecuencia      monto
      1           1           1
      recencia frecuencia monto
[1,]    -0.59         0.85  0.00
[2,]    -0.60        -0.02  1.15
[3,]    -0.17         1.28  1.75

```

Un valor escalado se lee como “cuántas desviaciones estándar está este cliente por arriba o por debajo del promedio”. El primer cliente tiene una recencia de -0.59 : compró más recientemente que el cliente promedio. `scale()` guarda las medias y desviaciones usadas como atributos del resultado; los necesitaremos después para asignar clientes nuevos.

7.3.3 Paso 3: ¿cuántos grupos? Método del codo y silueta

k-means necesita que le digas cuántos grupos (k) formar. Dos herramientas ayudan a decidir:

- **Método del codo:** para cada k calcula la **suma de cuadrados dentro de los grupos** (WSS, qué tan dispersos están los clientes respecto al centro de su grupo). Siempre baja al aumentar k ; buscamos el “codo”, el punto a partir del cual agregar grupos ya casi no mejora.
- **Silueta:** mide, para cada cliente, qué tan cerca está de su propio grupo comparado con el grupo vecino (de -1 a 1 ; más alto es mejor). Buscamos el k con la silueta promedio más alta.

```

1 set.seed(123)
2 codo <- tibble(k = 1:10) %>%
3   mutate(wss = map_dbl(k, function(k) {
4     kmeans(rfm_escalado, centers = k, nstart = 25)$tot.withinss
5   }))
6
7 codo %>%
8   mutate(mejora_pct = round(100 * (lag(wss) - wss) / lag(wss), 1),
9          wss = round(wss, 1))

```

```
# A tibble: 10 x 3
      k    wss mejora_pct
  <int> <dbl>    <dbl>
1     1  591         NA
2     2  344.      41.9
3     3  239      30.5
4     4  175.      26.9
5     5  146.      16.3
6     6  127.       13
7     7  114      10.5
8     8  102.      10.3
9     9   92.3       9.7
10    10   85.1       7.8
```

La columna `mejora_pct` dice cuánto baja la dispersión al agregar un grupo más. Pasar de 1 a 2 grupos reduce la WSS 41.9%; de 2 a 3, 30.5%; de 3 a 4, 26.9%; y a partir de ahí las mejoras son cada vez menores. Veámoslo en una gráfica, resaltando $k = 4$:

```
1 grafico_codo <- ggplot(codo, aes(x = k, y = wss)) +
2   geom_line(color = color_principal, linewidth = 0.8) +
3   geom_point(color = color_principal, size = 2) +
4   geom_point(data = filter(codo, k == 4), color = color_resalte,
5     size = 4) +
6   annotate("text", x = 4.3, y = codo$wss[4] + 60, hjust = 0,
7     label = "k = 4: las mejoras\nempiezan a ser pequeñas",
8     color = color_resalte, size = 3.5) +
9   scale_x_continuous(breaks = 1:10) +
10  labs(title = "Método del codo",
11    subtitle = "Dispersión dentro de los grupos según k",
12    x = "Número de grupos (k)",
13    y = "Suma de cuadrados dentro (WSS)") +
14  theme_minimal() +
15  theme(panel.grid.minor = element_blank())
16 grafico_codo
```

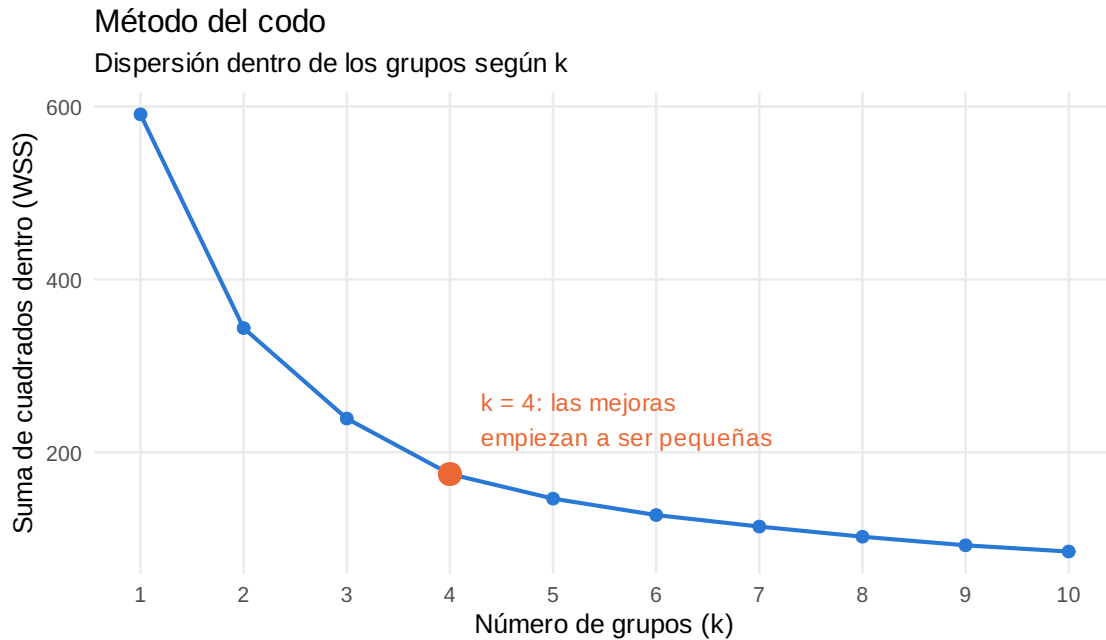


Figura 7.4. Método del codo para el RFM de los clientes del curso. La curva se suaviza entre $k = 3$ y $k = 4$.

Ahora la silueta. La función `fviz_nbclust()` de **factoextra** hace todo el cálculo y devuelve una gráfica de **ggplot2**, que puedes personalizar con `labs()` y `theme()` como cualquier otra:

```
1 set.seed(123)
2 grafico_silueta <- fviz_nbclust(rfm_escalado, kmeans,
3                               method = "silhouette", k.max = 10,
4                               nstart = 25, linecolor = color_principal) +
5   labs(title = "Silueta promedio según el número de grupos",
6         subtitle = "Más alto = grupos más separados",
7         x = "Número de grupos (k)", y = "Silueta promedio") +
8   theme_minimal() +
9   theme(panel.grid.minor = element_blank())
10 grafico_silueta
11
12 # Los valores calculados vienen dentro del objeto ggplot
13 grafico_silueta$data %>%
14   mutate(y = round(y, 3)) %>%
15   filter(clusters %in% 2:6)
```

clusters	y
1	2 0.345
2	3 0.337
3	4 0.328
4	5 0.325
5	6 0.292

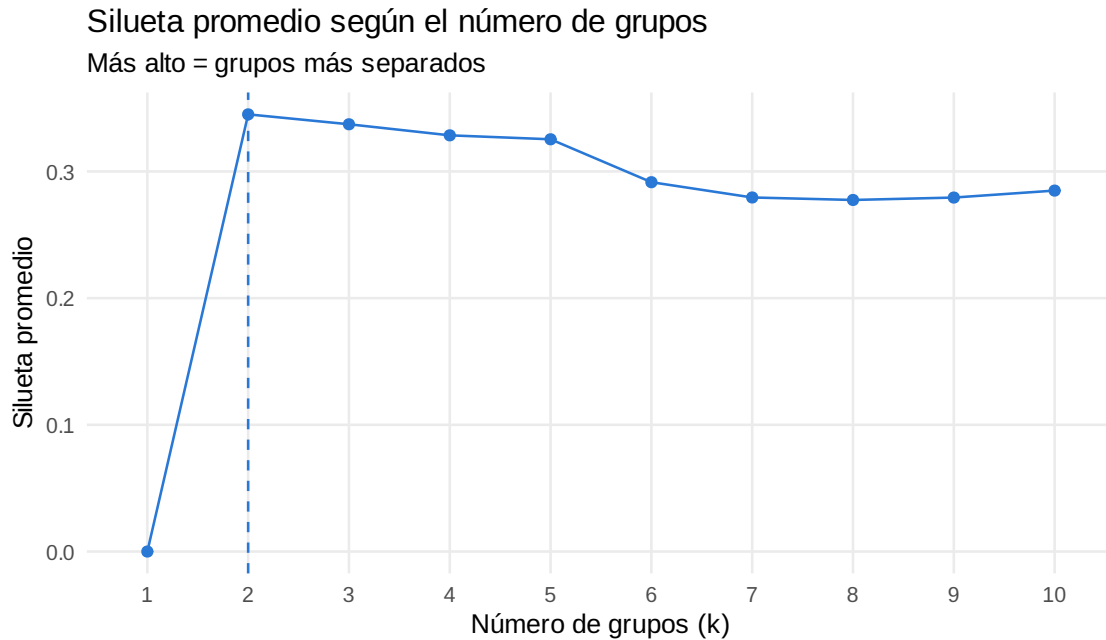


Figura 7.5. Silueta promedio para k de 1 a 10. La línea punteada marca el máximo ($k = 2$), aunque las diferencias entre 2 y 5 grupos son mínimas.

La silueta más alta es para $k = 2$ (0.345), pero $k = 3, 4$ y 5 están prácticamente empatados (0.337, 0.328 y 0.325). Una silueta de alrededor de 0.33 indica una estructura *débil*: los clientes no forman “islas” naturales, sino un continuo que k-means parte en regiones. Es lo esperable con datos generados al azar, y muy común con clientes reales.

Consejo

Cuando las métricas no dan un ganador claro, decide con criterio de negocio. Dos grupos (“buenos” y “malos”) no alcanzan para diseñar tres campañas; diez grupos son imposibles de operar. Aquí elegimos $k = 4$: el codo lo respalda, la silueta casi no empeora y el área de marketing puede manejar cuatro estrategias. Las métricas orientan; la decisión es tuya.

7.3.4 Paso 4: entrenar k-means

Concepto clave: Cómo trabaja k-means

1. Coloca k centros al azar.
2. Asigna cada cliente al centro más cercano.
3. Mueve cada centro al promedio de sus clientes.
4. Repite 2 y 3 hasta que los grupos ya no cambian.

Como el resultado depende de los centros iniciales, el argumento `nstart = 25` prueba 25 arranques distintos y se queda con el mejor (el de menor WSS). Usa siempre `set.seed()` y un

`nstart` de 10 o más.

```
1 set.seed(123)
2 modelo_kmeans <- kmeans(rfm_escalado, centers = 4, nstart = 25)
3
4 modelo_kmeans$size                # clientes por grupo
5 round(modelo_kmeans$centers, 2)    # centros (en escala estandarizada)
6
7 # Proporción de la variación total explicada por los grupos
8 round(modelo_kmeans$betweenss / modelo_kmeans$totss, 3)
```

```
[1] 22 87 60 29
   recencia frecuencia monto
1   -0.53         1.81  1.97
2   -0.42         0.36  0.27
3   -0.13        -0.71 -0.73
4    1.95        -0.98 -0.80
[1] 0.704
```

Los centros están en escala estandarizada, así que se leen como “desviaciones respecto al promedio”. El grupo 1 tiene frecuencia +1.81 y monto +1.97: compra mucho más que el promedio. El grupo 4 tiene recencia +1.95: hace mucho que no compra. Los cuatro grupos explican el 70.4 % de la variación total de los datos.

▲ Cuidado

Los números de grupo (1, 2, 3, 4) son etiquetas **arbitrarias**: si cambias la semilla, el orden de los datos o la versión de R, el grupo de “mejores clientes” puede aparecer como 3 en lugar de 1. Nunca escribas en tu código `cluster == 1 ~ "VIP"`. Nombra los grupos a partir de su perfil calculado, como haremos enseguida.

Revisemos también la calidad de cada grupo con la silueta individual:

```
1 silueta <- silhouette(modelo_kmeans$cluster, dist(rfm_escalado))
2
3 # Silueta promedio por grupo y cuántos clientes quedaron "mal ubicados"
4 round(summary(silueta)$clus.avg.widths, 3)
5 sum(silueta[, "sil_width"] < 0)
```

```
      1      2      3      4
0.375 0.321 0.298 0.379
[1] 0
```

Todos los grupos tienen silueta positiva; los más compactos son el 4 (0.379) y el 1 (0.375), y el menos definido es el 3 (0.298). Ningún cliente tiene silueta negativa (el 0 de la última línea): nadie quedó

más cerca del grupo vecino que del suyo. Si hubiera clientes con silueta negativa, serían clientes “frontera” que conviene revisar a mano.

7.3.5 Paso 5: perfilar y nombrar los segmentos

Un segmento no sirve hasta que alguien de negocio puede describirlo en una frase. Calculamos el perfil de cada grupo *en las unidades originales* (días, compras, pesos) y su peso en clientes e ingresos:

```
1 rfm_seg <- rfm %>%
2   mutate(cluster = modelo_kmeans$cluster)
3
4 perfil <- rfm_seg %>%
5   group_by(cluster) %>%
6   summarise(
7     clientes      = n(),
8     recencia_prom = mean(recencia),
9     frecuencia_prom = mean(frecuencia),
10    monto_prom    = mean(monto),
11    ingresos      = sum(monto),
12    .groups = "drop"
13  ) %>%
14    mutate(pct_clientes = 100 * clientes / sum(clientes),
15           pct_ingresos = 100 * ingresos / sum(ingresos))
16
17 perfil %>%
18   select(cluster, clientes, recencia_prom, frecuencia_prom, monto_prom,
19          pct_ingresos) %>%
20   mutate(across(where(is.double), ~ round(.x, 1)))
```

```
# A tibble: 4 x 6
  cluster clientes recencia_prom frecuencia_prom monto_prom
  <int>    <int>      <dbl>          <dbl>      <dbl>
1     1      22      34.5            9.2     10159.
2     2      87      41.8            5.9      5577.
3     3      60      60.5            3.4      2879.
4     4      29     196.            2.8      2708.
# i 1 more variable: pct_ingresos <dbl>
```

Ahora asignamos los nombres **con una regla basada en el perfil**, no en el número del grupo. La regla de negocio es: el grupo con la recencia promedio más alta son los “Dormidos”; los demás se ordenan por monto promedio de mayor a menor y se llaman “Campeones”, “Leales” y “Ocasionales”.

```
1 nombres_segmentos <- perfil %>%
2   mutate(es_dormido = recencia_prom == max(recencia_prom)) %>%
3   arrange(es_dormido, desc(monto_prom)) %>% # dormidos al final
4   mutate(segmento = c("Campeones", "Leales", "Ocasionales",
```



```

5         "Dormidos")) %>%
6   select(cluster, segmento)
7
8   orden_segmentos <- c("Campeones", "Leales", "Ocasionales", "Dormidos")
9
10  rfm_seg <- rfm_seg %>%
11    left_join(nombres_segmentos, by = "cluster") %>%
12    mutate(segmento = factor(segmento, levels = orden_segmentos))
13
14  perfil_segmentos <- perfil %>%
15    left_join(nombres_segmentos, by = "cluster") %>%
16    mutate(segmento = factor(segmento, levels = orden_segmentos)) %>%
17    arrange(segmento) %>%
18    transmute(segmento, clientes,
19              recencia_dias = round(recencia_prom),
20              compras      = round(frecuencia_prom, 1),
21              monto_prom   = round(monto_prom),
22              pct_ingresos  = round(pct_ingresos, 1))
23  perfil_segmentos

```

```

# A tibble: 4 x 6
  segmento  clientes recencia_dias compras monto_prom pct_ingresos
  <fct>      <int>      <dbl>   <dbl>   <dbl>      <dbl>
1 Campeones    22         35     9.2    10159      23.3
2 Leales       87         42     5.9     5577      50.5
3 Ocasionales  60         60     3.4     2879       18
4 Dormidos    29        195     2.8     2708       8.2

```

Ahora sí, cada grupo cuenta una historia:

- **Campeones** (22 clientes, 11 % de la base): compraron hace unos 35 días, 9.2 veces en el año y gastaron en promedio \$10,159. Son apenas uno de cada nueve clientes, pero generan el 23.3 % de los ingresos.
- **Leales** (87 clientes): el grueso de la base. Compran seguido (5.9 veces) y aportan la mitad de los ingresos (50.5 %).
- **Ocasionales** (60): activos (60 días de recencia), pero compran poco y gastan \$2,879 en promedio.
- **Dormidos** (29): su última compra fue hace 195 días en promedio, más de seis meses. Aportan solo 8.2 % de los ingresos.

Observa que la regla de nombres no menciona ningún número de grupo: si mañana k-means numera distinto, cada cliente seguirá recibiendo el nombre correcto.

Hazlo tú

Cambia la semilla a `set.seed(2024)` antes de `kmeans()` y vuelve a correr los pasos 4 y 5. ¿Cambian los números de grupo? ¿Cambian los nombres asignados a cada cliente? (Deberían mantenerse, porque los nombres dependen del perfil y no del número.)

7.3.6 Paso 6: visualizar los segmentos

`fviz_cluster()` dibuja los grupos en dos dimensiones. Como tenemos tres variables, primero las resume en dos “componentes principales” (combinaciones de R, F y M que conservan la mayor parte de la información); los porcentajes de los ejes indican cuánta información conserva cada una.

```
1 grafico_clusters <- fviz_cluster(modelo_kmeans, data = rfm_escalado,
2                                geom = "point", ellipse.type = "convex",
3                                palette = paleta_libro[1:4],
4                                ggtheme = theme_minimal()) +
5  labs(title = "Grupos de k-means sobre el RFM escalado",
6       subtitle = "Proyección en los dos componentes principales",
7       color = "Grupo", fill = "Grupo", shape = "Grupo")
8 grafico_clusters
```

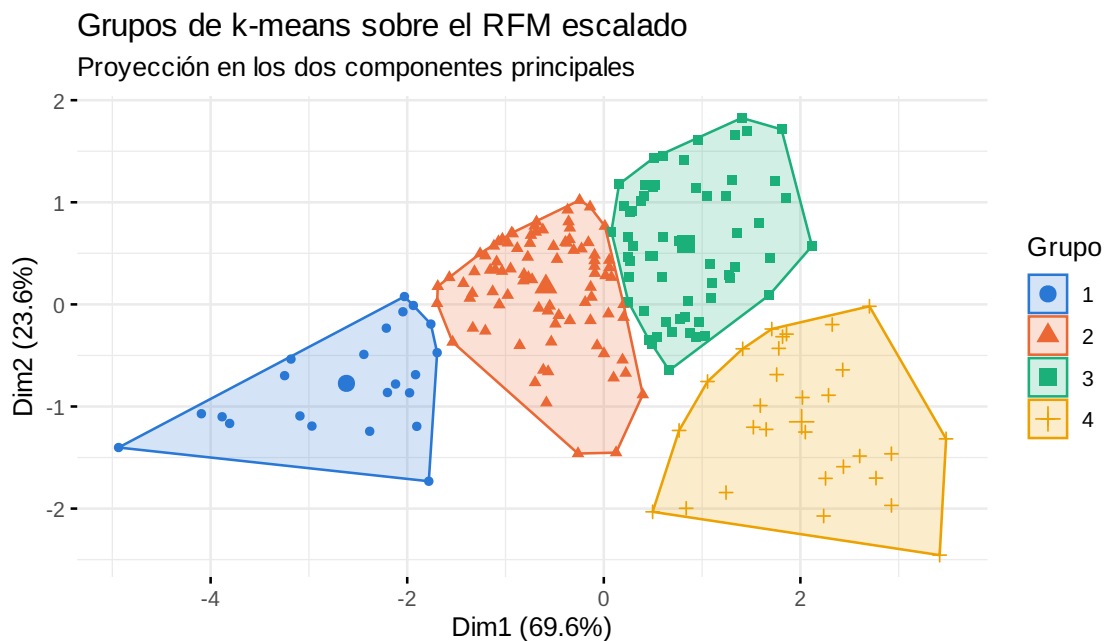


Figura 7.6. Los cuatro grupos de k-means proyectados en dos componentes principales, que juntos conservan 93.2% de la información de R, F y M.

La gráfica de componentes principales es útil para ver si los grupos se separan, pero sus ejes son difíciles de explicar a la dirección. Para una presentación, grafica las variables originales con los

nombres de negocio:

```
1 grafico_segmentos <- ggplot(rfm_seg,
2                             aes(recencia, monto, color = segmento)) +
3   geom_point(size = 2, alpha = 0.8) +
4   scale_color_manual(values = paleta_libro[1:4]) +
5   scale_y_continuous(labels = scales::dollar) +
6   labs(title = "Segmentos de clientes según recencia y monto",
7         subtitle = "Clientes del curso, compras de 2023",
8         x = "Días desde la última compra (recencia)",
9         y = "Monto total comprado", color = NULL) +
10  theme_minimal() +
11  theme(panel.grid.minor = element_blank(), legend.position = "top")
12 grafico_segmentos
```

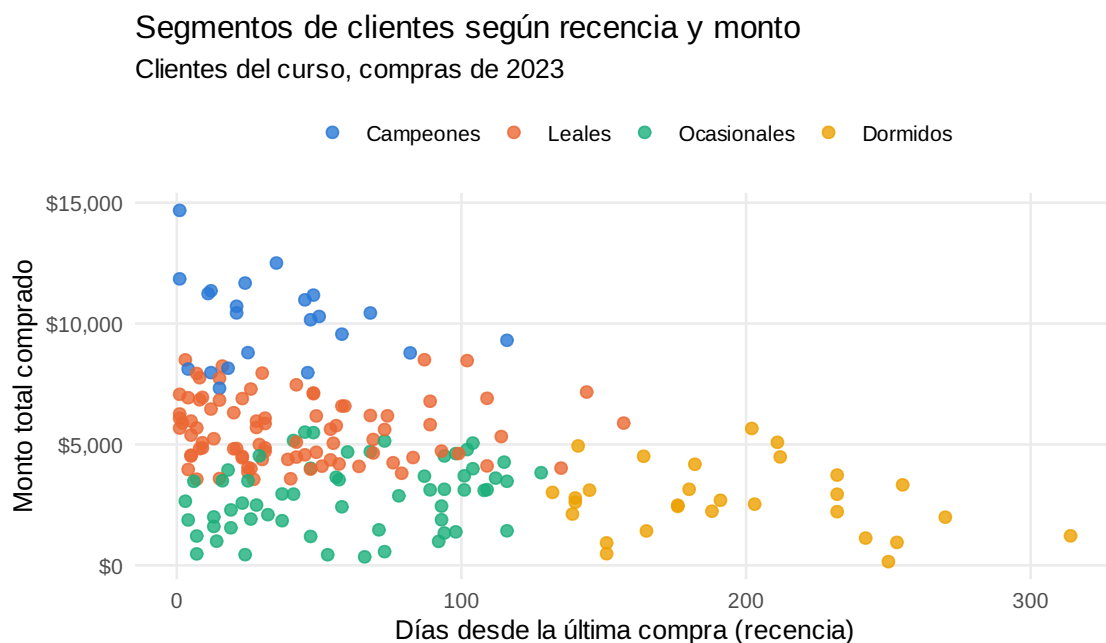


Figura 7.7. Segmentos RFM en unidades de negocio: los Dormidos se separan por recencia; Campeones, Leales y Ocasionales, por monto.

7.3.7 Paso 7: de segmentos a estrategias

La segmentación solo genera valor cuando cada segmento tiene una acción asociada. La tabla 7.3 es un ejemplo de cómo se presentaría a marketing.

Tabla 7.3. Estrategia sugerida por segmento RFM.

Segmento	Quiénes son	Estrategia
Campeones	Compran seguido, gastan el doble del promedio y compraron hace poco.	Programa VIP, acceso anticipado a lanzamientos, atención preferente. No regalar descuentos: ya compran sin ellos.
Leales	Compradores regulares con gasto por encima del promedio.	Venta cruzada y <i>up-selling</i> (sección 7.7); programa de puntos para convertirlos en Campeones.
Ocasionales	Compran poco y gastan poco, pero siguen activos.	Campañas de frecuencia: cupones con fecha de vencimiento, recordatorios, paquetes de entrada.
Dormidos	No compran desde hace más de seis meses en promedio.	Campaña de reactivación con oferta fuerte y encuesta de salida; si no responden, reducir la inversión en ellos.

Para enriquecer el perfil, puedes unir los segmentos con los datos demográficos de `clientes.csv`. Por ejemplo, ¿los Campeones son más “premium” o más jóvenes?

```

1 rfm_seg %>%
2   # de clientes.csv solo tomamos lo que necesitamos (ya trae una
3   # columna "segmento" por edad que chocaría con la nuestra)
4   left_join(select(clientes, cliente_id, edad, es_premium),
5             by = "cliente_id") %>%
6   group_by(segmento) %>%
7   summarise(clientes = n(),
8             edad_prom = round(mean(edad), 1),
9             pct_premium = round(100 * mean(es_premium), 1),
10            .groups = "drop")

```

```

# A tibble: 4 x 4
  segmento  clientes edad_prom pct_premium
  <fct>      <int>    <dbl>     <dbl>
1 Campeones     22     49.2      31.8
2 Leales        87     47.4      34.5
3 Ocasionales   60     49.2       45
4 Dormidos     29     51.3      27.6

```

La edad promedio es casi igual en los cuatro segmentos (47 a 51 años) y el porcentaje de clientes *premium* no sigue ningún patrón lógico: los Ocasionales tienen más premium (45 %) que los Campeones (31.8 %). En estos datos simulados, la membresía premium y la edad no tienen relación con el comportamiento de compra. En una empresa real este cruce sería un hallazgo importante: si pagas beneficios premium a clientes que compran poco, el programa necesita revisión.

7.3.8 Paso 8: asignar un segmento a clientes nuevos

El modelo ya está entrenado. Cuando un cliente acumula historial, no necesitas volver a correr k-means: basta con escalar sus valores **con las mismas medias y desviaciones del entrenamiento** y asignarlo al centro más cercano.

```

1  asignar_segmento <- function(nuevos, modelo, datos_escalados, nombres) {
2    medias <- attr(datos_escalados, "scaled:center")
3    desv_est <- attr(datos_escalados, "scaled:scale")
4    # Escalamos con los parámetros del entrenamiento (no con los nuevos)
5    nuevos_esc <- scale(nuevos[, names(medias)], center = medias,
6                        scale = desv_est)
7    # Para cada cliente: el centro con menor distancia al cuadrado
8    grupo <- apply(nuevos_esc, 1, function(fila) {
9      which.min(colSums((t(modelo$centers) - fila)^2))
10   })
11   nuevos %>%
12     mutate(cluster = grupo) %>%
13     left_join(nombres, by = "cluster")
14 }
15
16 nuevos_clientes <- tibble(
17   cliente = c("Ana", "Beto", "Carla"),
18   recencia = c(10, 250, 45),
19   frecuencia = c(11, 2, 4),
20   monto = c(9500, 1800, 3500)
21 )
22
23 asignar_segmento(nuevos_clientes, modelo_kmeans, rfm_escalado,
24                 nombres_segmentos)

```

```

# A tibble: 3 x 6
  cliente recencia frecuencia monto cluster segmento
  <chr>      <dbl>      <dbl> <dbl>   <int> <chr>
1 Ana         10         11  9500     1 Campeones
2 Beto        250         2  1800     4 Dormidos
3 Carla        45         4  3500     3 Ocasionales

```

Ana (compró hace 10 días, 11 compras, \$9,500) es Campeona; Beto, que no compra desde hace 250 días, cae en Dormidos; y Carla, con un comportamiento modesto pero reciente, en Ocasionales. Este tipo de función es la que después se programa para que corra cada semana (Módulo 8).

7.3.9 Alternativa sencilla: RFM por reglas con quintiles

No siempre necesitas k-means. Muchas empresas usan **RFM por reglas**: a cada cliente se le asigna una calificación de 1 a 5 en cada dimensión según su quintil (el 20 % con mejor valor recibe 5) y

luego se definen segmentos con reglas fáciles de explicar. En R, `ntile()` de **dplyr** hace el trabajo.

```

1 rfm_reglas <- rfm %>%
2   mutate(
3     r_score = ntile(desc(recencia), 5),    # 5 = compró más recientemente
4     f_score = ntile(frecuencia, 5),        # 5 = compra más seguido
5     m_score = ntile(monto, 5),             # 5 = gasta más
6     codigo_rfm = paste0(r_score, f_score, m_score),
7     segmento_reglas = case_when(
8       r_score >= 4 & f_score >= 4 & m_score >= 4 ~ "Campeones",
9       r_score >= 3 & f_score >= 3 ~ "Leales",
10      r_score <= 2 & f_score >= 3 ~ "En riesgo",
11      r_score <= 2 ~ "Dormidos",
12      TRUE ~ "Ocasionales"
13    )
14  )
15
16 rfm_reglas %>%
17   select(cliente_id, recencia, frecuencia, monto, codigo_rfm,
18          segmento_reglas) %>%
19   head(5)
20
21 rfm_reglas %>% count(segmento_reglas, sort = TRUE)

```

```

# A tibble: 5 x 6
  cliente_id recencia frecuencia monto codigo_rfm segmento_reglas
  <dbl>      <dbl>      <int> <dbl> <chr>      <chr>
1         1         31         7  4859. 443      Leales
2         2         30         5  7950. 435      Leales
3         3         58         8  9562 355      Leales
4         4          8         5  7759. 535      Leales
5         5          1        13 14680. 555      Campeones
# A tibble: 5 x 2
  segmento_reglas      n
  <chr>          <int>
1 Leales          56
2 Dormidos        50
3 Campeones       32
4 En riesgo       30
5 Ocasionales     30

```

El cliente 1 tiene código 443 (recencia y frecuencia altas, monto medio): no alcanza el 4 en monto para ser Campeón y cae en “Leales”. Ahora un segmento nuevo aparece: “En riesgo”, clientes que *solían* comprar seguido pero llevan tiempo sin volver; k-means los había repartido entre otros grupos. ¿Qué tanto coinciden ambos métodos?

```
1 table(Reglas = rfm_reglas$segmento_reglas, kmeans = rfm_seg$segmento)
```

Reglas	kmeans			
	Campeones	Leales	Ocasionales	Dormidos
Campeones	12	20	0	0
Dormidos	0	1	23	26
En riesgo	3	19	5	3
Leales	7	44	5	0
Ocasionales	0	3	27	0

Las filas son los segmentos por reglas y las columnas los de k-means. La coincidencia es buena en los extremos: 27 de los 30 Ocasionales por reglas son Ocasionales en k-means y 26 de los 29 Dormidos de k-means son Dormidos por reglas. Los 30 clientes “En riesgo” se reparten entre Leales (19) y otros grupos: k-means los mezcló con clientes activos porque su monto y frecuencia pesan más que su recencia. Ningún método es “el correcto”; son dos formas de mirar a los mismos clientes.

En la empresa: ¿Reglas o k-means?

Las reglas RFM son transparentes (“Campeón = calificación 4 o 5 en todo”), estables en el tiempo y fáciles de programar incluso en SQL o Excel. k-means descubre grupos sin que tú fijas los cortes y admite más variables (margen, categorías compradas, devoluciones), pero sus grupos pueden moverse al reentrenar. Muchas empresas empiezan con reglas y pasan a clustering cuando necesitan más variables.

Hazlo tú

Agrega al RFM una cuarta variable: el número de tiendas distintas en las que compró cada cliente (`n_distinct(tienda_id)`). Escala, vuelve a correr k-means con $k = 4$ y compara los perfiles. ¿Aparece algún grupo nuevo con sentido de negocio?

7.4 Caso 2: regresión para predecir ventas

Ejemplo práctico: ¿Cuánto venderá una tienda nueva?

Una cadena con 400 tiendas planea abrir sucursales y quiere estimar sus ventas mensuales antes de firmar el contrato de renta. Para cada tienda existente conoce su zona, tamaño en m^2 , tráfico peatonal mensual, inversión en publicidad local y si participa en el programa de promociones. La pregunta: *¿cuánto venderá una tienda con ciertas características y qué palancas mueven más las ventas?* Es un problema de **regresión**: la respuesta es un número.

7.4.1 Datos simulados con una relación conocida

Como explicamos al inicio, los datos del curso no tienen esta información (`tiendas.csv` solo tiene 10 tiendas y sin publicidad), así que simulamos 400 tiendas. Las ventas mensuales (en miles de pesos) se generan con esta “verdad” que el modelo tendrá que descubrir:

$$\text{ventas} = 120 + 3.2 \text{ publicidad} + 0.25 \text{ m}^2 + 0.04 \text{ tráfico} + 60 \text{ promoción} + \text{efecto zona} + \text{ruido}$$

con efecto zona de +40 en el Norte, −30 en el Sur y 0 en el Centro. El tráfico depende en parte del tamaño (las tiendas grandes atraen más gente), como en la vida real.

```

1  set.seed(123)
2  n_tiendas <- 400
3  tiendas_sim <- tibble(
4    tienda      = sprintf("T%03d", 1:n_tiendas),
5    zona        = sample(c("Centro", "Norte", "Sur"), n_tiendas,
6                        replace = TRUE),
7    tamano_m2   = round(runif(n_tiendas, 150, 1200)),
8    publicidad  = round(runif(n_tiendas, 10, 120), 1), # miles de $/mes
9    promocion   = rbinom(n_tiendas, 1, 0.35),          # 1 = participa
10   trafico     = round(3000 + 5 * tamano_m2 +          # personas/mes
11                     rnorm(n_tiendas, 0, 1500))
12 ) %>%
13   mutate(
14     efecto_zona = case_when(zona == "Norte" ~ 40,
15                             zona == "Sur"   ~ -30,
16                             TRUE           ~ 0),
17     ventas = round(120 + 3.2 * publicidad + 0.25 * tamano_m2 +
18                   0.04 * trafico + 60 * promocion + efecto_zona +
19                   rnorm(n_tiendas, 0, 50), 1) # miles de $/mes
20 ) %>%
21   select(-efecto_zona) # el modelo no debe ver la "respuesta secreta"
22
23 head(tiendas_sim, 4)

```

```

# A tibble: 4 x 7
  tienda zona  tamano_m2 publicidad promocion trafico ventas
  <chr>  <chr>    <dbl>    <dbl>    <int>   <dbl>  <dbl>
1 T001   Sur      508      17.9      0     5980   517.
2 T002   Sur      638      10.4      0     3903   408.
3 T003   Sur      237      15.7      0     5900   400.
4 T004   Norte    1053     105.      1    10671  1263.

```

Separamos 80 % para entrenar y 20 % para probar:


```

1 set.seed(123)
2 filas_ent <- sample(nrow(tiendas_sim), size = 0.8 * nrow(tiendas_sim))
3 tiendas_ent <- tiendas_sim[filas_ent, ] # 320 tiendas
4 tiendas_prueba <- tiendas_sim[-filas_ent, ] # 80 tiendas
5 c(entrenamiento = nrow(tiendas_ent), prueba = nrow(tiendas_prueba))

```

entrenamiento	prueba
320	80

7.4.2 Regresión lineal múltiple

La **regresión lineal múltiple** (función `lm()`, que ya usaste en el Módulo 4 con una sola variable) estima cuánto cambia la respuesta por cada unidad de cada predictora, *manteniendo las demás constantes*. La fórmula `ventas ~ publicidad + tamano_m2 + ...` se lee “ventas en función de publicidad, tamaño, ...”.

```

1 modelo_lm <- lm(ventas ~ publicidad + tamano_m2 + trafico + promocion +
2                 zona, data = tiendas_ent)
3 summary(modelo_lm)

```

```

Call:
lm(formula = ventas ~ publicidad + tamano_m2 + trafico + promocion +
    zona, data = tiendas_ent)

Residuals:
    Min       1Q   Median       3Q      Max
-120.181  -38.036   -1.857   35.175  141.550

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 107.296043  11.071131   9.692  < 2e-16 ***
publicidad   3.259854   0.091005  35.821  < 2e-16 ***
tamano_m2    0.270330   0.013935  19.399  < 2e-16 ***
trafico      0.038681   0.001903  20.330  < 2e-16 ***
promocion    57.226212   6.042430   9.471  < 2e-16 ***
zonaNorte    41.704523   6.880492   6.061 3.88e-09 ***
zonaSur     -29.067746   6.992714  -4.157 4.17e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 50.65 on 313 degrees of freedom
Multiple R-squared:  0.937,    Adjusted R-squared:  0.9358
F-statistic: 776 on 6 and 313 DF,  p-value: < 2.2e-16

```

Concepto clave: Cómo leer la tabla de coeficientes

- **Estimate:** el efecto estimado. Para una variable numérica, es el cambio en ventas por cada unidad adicional; para una categoría (**zonaNorte**), es la diferencia respecto a la categoría base (Centro, la primera en orden alfabético).
- **Pr(>|t|)** (valor p): si es muy pequeño (menor a 0.05), el efecto difícilmente se debe al azar. Los asteriscos lo resumen.
- **Residual standard error:** el tamaño típico del error del modelo, en unidades de la respuesta.
- **R-squared:** proporción de la variación de las ventas que el modelo explica.

Traducido al lenguaje de negocio (recuerda que ventas y publicidad están en miles de pesos):

- **Publicidad:** cada \$1,000 adicionales de publicidad se asocian con \$3,260 más de ventas mensuales. El valor real era 3.2: el modelo lo descubrió.
- **Tamaño:** cada m² adicional aporta unos \$270 al mes (real: 250).
- **Tráfico:** cada 100 visitantes más, \$3,900 más de ventas (0.039×100 , real: 0.04).
- **Promoción:** participar suma unos \$57,000 al mes (real: 60).
- **Zona:** una tienda del Norte vende \$41,700 más que una equivalente del Centro; una del Sur, \$29,100 menos.
- El modelo explica el 93.7 % de la variación de las ventas ($R^2 = 0.937$), y el error típico es de unos \$50,000, justo el tamaño del ruido que simulamos.

▲ Cuidado

Un coeficiente de regresión mide **asociación, no causalidad**. En datos simulados sabemos que la publicidad *causa* ventas, porque así lo programamos. En datos reales, quizá las tiendas que ya venden mucho reciben más presupuesto de publicidad (la causalidad va al revés). Para afirmar “si invierto \$1,000 más en publicidad venderé \$3,260 más” necesitas un experimento (por ejemplo, aumentar la publicidad en tiendas elegidas al azar).

✎ Consejo

Para saber qué variable “pesa” más, no compares los coeficientes directamente: están en unidades distintas (pesos, m², personas). Compara el efecto de un cambio *realista*: pasar de 40 a 80 mil pesos de publicidad suma $40 \times 3.26 \approx 130$ mil pesos; pasar de 400 a 800 m² suma $400 \times 0.27 \approx 108$ mil pesos directamente, más el tráfico extra que atrae.

7.4.3 Evaluar el modelo: MAE, RMSE, MAPE y R²

El resumen del modelo describe el ajuste en los datos de entrenamiento. La evaluación honesta se hace con las 80 tiendas de prueba. Las cuatro métricas de regresión más usadas son:

Tabla 7.4. Métricas de error para regresión.

Métrica	Cálculo	Cómo se lee
MAE (error absoluto medio)	promedio de $ \text{real} - \text{predicho} $	“En promedio nos equivocamos por \$X”. La más fácil de explicar.
RMSE (raíz del error cuadrático medio)	$\sqrt{\text{promedio de } (\text{real} - \text{predicho})^2}$	Como el MAE, pero castiga más los errores grandes. Útil si un error grande es muy costoso.
MAPE (error porcentual absoluto medio)	promedio de $ \text{real} - \text{predicho} / \text{real} \times 100$	“Nos equivocamos X% en promedio”. Comparable entre negocios de distinto tamaño; falla si hay valores reales cercanos a cero.
R ² (coeficiente de determinación)	$1 - \frac{\sum(\text{real} - \text{pred.})^2}{\sum(\text{real} - \text{prom.})^2}$	Proporción de la variación explicada: 1 es perfecto, 0 es igual a predecir siempre el promedio, negativo es peor que el promedio.

Calculémoslas a mano con una función reutilizable:

```

1  metricas_regresion <- function(real, predicho) {
2    error <- real - predicho
3    tibble(
4      MAE = mean(abs(error)),
5      RMSE = sqrt(mean(error^2)),
6      MAPE = mean(abs(error) / real) * 100,
7      R2 = 1 - sum(error^2) / sum((real - mean(real))^2)
8    )
9  }
10
11 tiendas_prueba <- tiendas_prueba %>%
12   mutate(pred_lm = predict(modelo_lm, newdata = tiendas_prueba))
13
14 metricas_regresion(tiendas_prueba$ventas, tiendas_prueba$pred_lm) %>%
15   mutate(across(everything(), ~ round(.x, 3)))

```

```

# A tibble: 1 x 4
  MAE  RMSE  MAPE   R2
<dbl> <dbl> <dbl> <dbl>
1  37.2  45.1  5.04  0.947

```

En tiendas que el modelo nunca vio, nos equivocamos en promedio por \$37,200 al mes (MAE = 37.2 miles), lo que equivale a un error de 5 % (MAPE). El RMSE (45.1) es algo mayor que el MAE porque penaliza más los errores grandes; si fuera mucho mayor, indicaría que hay algunas tiendas con errores enormes. El R² en prueba (0.947) es tan bueno como en entrenamiento (0.937): no hay señales de sobreajuste.

 Consejo

Compara siempre contra una **línea base** (*baseline*): el modelo más tonto posible. Aquí, predecir para toda tienda nueva el promedio de ventas del entrenamiento. Si tu modelo no le gana claramente a la línea base, no aporta valor.

```
1 prediccion_ingenua <- mean(tiendas_ent$ventas) # siempre el promedio
2 metricas_regresion(tiendas_prueba$ventas,
3                   rep(prediccion_ingenua, nrow(tiendas_prueba))) %>%
4   mutate(across(everything(), ~ round(.x, 3)))
```

```
# A tibble: 1 x 4
  MAE  RMSE  MAPE    R2
<dbl> <dbl> <dbl> <dbl>
1 154.  196.  24.7 -0.003
```

Predecir siempre el promedio tiene un MAE de 154 mil pesos y un MAPE de 24.7%; su R^2 es prácticamente cero (por definición, el promedio no explica nada). Nuestro modelo reduce el error promedio en más de tres cuartas partes: de 154 a 37 mil pesos.

7.4.4 Gráfico real contra predicho

La mejor forma de mostrar la calidad de un modelo de regresión a la dirección es un diagrama de dispersión de valores reales contra predichos: si el modelo fuera perfecto, todos los puntos caerían sobre la diagonal.

```
1 grafico_real_pred <- ggplot(tiendas_prueba,
2                             aes(x = pred_lm, y = ventas)) +
3   geom_abline(slope = 1, intercept = 0, color = color_gris,
4               linewidth = 0.8, linetype = "dashed") +
5   geom_point(color = color_principal, size = 2, alpha = 0.8) +
6   scale_x_continuous(labels = scales::dollar_format(suffix = "k")) +
7   scale_y_continuous(labels = scales::dollar_format(suffix = "k")) +
8   labs(title = "Ventas reales contra ventas predichas",
9         subtitle = "80 tiendas de prueba; diagonal = predicción perfecta",
10        x = "Ventas predichas (miles)", y = "Ventas reales (miles)") +
11   theme_minimal() +
12   theme(panel.grid.minor = element_blank())
13 grafico_real_pred
```

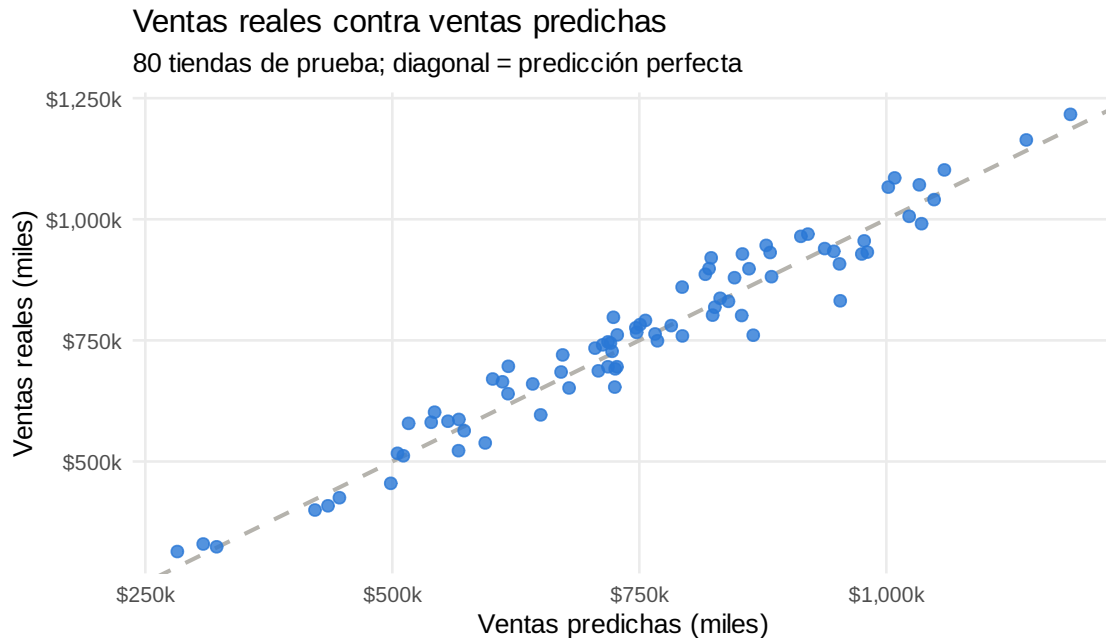


Figura 7.8. Ventas reales contra predichas por la regresión lineal en las tiendas de prueba. Los puntos se agrupan alrededor de la diagonal, sin patrones sistemáticos.

Busca dos cosas en esta gráfica: que los puntos estén cerca de la diagonal y que el error no dependa del tamaño (por ejemplo, que el modelo no subestime sistemáticamente las tiendas más grandes). Aquí ambas se cumplen.

7.4.5 Usar el modelo: estimar las ventas de tiendas candidatas

```

1  candidatas <- tibble(
2    tienda      = c("Local Plaza Norte", "Local Centro Histórico",
3                    "Local Sur Express"),
4    zona        = c("Norte", "Centro", "Sur"),
5    tamano_m2   = c(900, 400, 250),
6    publicidad  = c(80, 50, 30),
7    promocion   = c(1, 0, 1),
8    trafico     = c(9000, 6500, 4000)
9  )
10
11 predict(modelo_lm, newdata = candidatas, interval = "prediction") %>%
12   as_tibble() %>%
13   mutate(tienda = candidatas$tienda, .before = 1) %>%
14   mutate(across(where(is.numeric), ~ round(.x)))

```

```

# A tibble: 3 x 4
  tienda          fit   lwr   upr

```

	<chr>	<dbl>	<dbl>	<dbl>
1	Local Plaza Norte	1058	958	1159
2	Local Centro Histórico	630	529	730
3	Local Sur Express	456	354	557

`interval = "prediction"` agrega un **intervalo de predicción** del 95 %: `fit` es la estimación puntual, y `lwr` y `upr` los límites inferior y superior. Para el local de Plaza Norte, el modelo estima ventas de \$1,058 mil al mes, y con 95 % de confianza entre \$958 mil y \$1,159 mil. Ese rango es lo que debes llevar a la junta: si la renta solo es rentable con ventas mayores a \$1,100 mil, el local es una apuesta arriesgada aunque la estimación puntual se vea bien. El local Sur Express, con ventas esperadas de \$456 mil, probablemente no justifica la inversión.

7.4.6 Comparación con random forest

Concepto clave: Árboles de decisión y random forest

Un **árbol de decisión** hace preguntas sucesivas del tipo “¿tamaño mayor a 700 m²?” y en cada hoja predice el promedio de las tiendas que llegan ahí. Un **random forest** (bosque aleatorio) entrena cientos de árboles, cada uno con una muestra distinta de tiendas y de variables, y promedia sus predicciones. Captura relaciones no lineales e interacciones sin que tengas que especificarlas, pero es una “caja negra” más difícil de explicar.

```
1 set.seed(123)
2 modelo_rf_ventas <- randomForest(
3   ventas ~ publicidad + tamano_m2 + trafico + promocion + zona,
4   data = tiendas_ent %>% mutate(zona = factor(zona)),
5   ntree = 300,           # número de árboles
6   importance = TRUE     # calcular importancia de variables
7 )
8 modelo_rf_ventas
```

```
Call:
...
      Type of random forest: regression
      Number of trees: 300
No. of variables tried at each split: 1

      Mean of squared residuals: 7887.08
      % Var explained: 80.2
```

El bosque reporta un error **OOB** (*out-of-bag*): como cada árbol se entrena con una muestra, las tiendas que quedaron fuera de esa muestra sirven para evaluarlo, una especie de validación cruzada gratuita. Comparemos ambos modelos en el conjunto de prueba:

```

1 tiendas_prueba <- tiendas_prueba %>%
2   mutate(pred_rf = predict(modelo_rf_ventas,
3                             newdata = tiendas_prueba %>%
4                               mutate(zona = factor(zona))))
5
6 bind_rows(
7   metricas_regresion(tiendas_prueba$ventas, tiendas_prueba$pred_lm) %>%
8     mutate(modelo = "Regresión lineal", .before = 1),
9   metricas_regresion(tiendas_prueba$ventas, tiendas_prueba$pred_rf) %>%
10     mutate(modelo = "Random forest", .before = 1)
11 ) %>%
12   mutate(across(where(is.numeric), ~ round(.x, 3)))

```

```

# A tibble: 2 x 5
  modelo      MAE  RMSE  MAPE    R2
  <chr>      <dbl> <dbl> <dbl> <dbl>
1 Regresión lineal  37.2  45.1  5.04  0.947
2 Random forest    67.7  85.3 10.5   0.809

```

La regresión lineal gana con claridad: MAE de 37.2 contra 67.7 del random forest, y R^2 de 0.947 contra 0.809. El bosque explica el 80.2 % de la varianza según su estimación OOB, muy parecido a lo que obtuvo en la prueba (0.809): el OOB es un buen adelanto del desempeño real.

En la empresa: El modelo más sofisticado no siempre gana

En esta simulación la relación es lineal por construcción, así que la regresión lineal es el modelo “correcto” y le gana al bosque aleatorio. En datos reales con umbrales y combinaciones (por ejemplo, la promoción solo funciona en tiendas grandes), el bosque suele ganar. Por eso la práctica profesional es: **probar un modelo simple y uno flexible, compararlos en datos de prueba y quedarte con el más sencillo que cumpla**. Además, la regresión lineal te da coeficientes que la dirección entiende.

Hazlo tú

Quita **trafico** del modelo lineal y vuelve a evaluarlo. ¿Cuánto empeora el MAE? ¿Cómo cambia el coeficiente de **tamano_m2**? (Pista: al no estar el tráfico, el tamaño absorbe parte de su efecto, porque ambas variables están relacionadas.)

7.5 Caso 3: clasificación para predecir el abandono de clientes

Ejemplo práctico: ¿A quién llama primero el equipo de retención?

Una empresa de suscripción (piensa en un club de compras con membresía mensual, un servicio de internet o un gimnasio) pierde cada mes a una parte de sus clientes. El equipo de retención puede llamar a 150 clientes por semana y ofrecerles un descuento. La pregunta: *¿qué clientes tienen mayor probabilidad de irse y, por lo tanto, a quiénes conviene llamar primero?* Es un problema de **clasificación**: la respuesta es “se va” o “se queda”.

El **churn** (abandono, deserción o cancelación) es la proporción de clientes que dejan de serlo en un periodo. Retener a un cliente suele costar mucho menos que conseguir uno nuevo, por eso los modelos de churn son de los más rentables en BI.

7.5.1 Datos simulados de clientes

Simulamos 2,000 clientes con variables típicas de un sistema de suscripciones. El abandono se genera con reglas de negocio realistas: aumenta con las quejas y los pagos tardíos, es mayor en los primeros meses, y disminuye con la antigüedad, el uso del servicio y los planes más completos. El **género y la edad no influyen** (los incluimos para una discusión de ética más adelante).

```

1  set.seed(123)
2  n_clientes <- 2000
3  clientes_churn <- tibble(
4    cliente_id      = sprintf("C%04d", 1:n_clientes),
5    antigüedad_meses = sample(1:72, n_clientes, replace = TRUE),
6    plan            = sample(c("Básico", "Estándar", "Premium"),
7                            n_clientes, replace = TRUE,
8                            prob = c(0.5, 0.3, 0.2)),
9    uso_mensual      = rpois(n_clientes, 8),      # visitas/usos al mes
10   quejas_6m        = rpois(n_clientes, 0.8),    # quejas últimos 6 meses
11   pagos_tardios     = rpois(n_clientes, 0.4),    # pagos atrasados en el año
12   genero           = sample(c("F", "M"), n_clientes, replace = TRUE),
13   edad             = sample(18:70, n_clientes, replace = TRUE)
14 ) %>%
15   mutate(
16     # "Verdad" oculta: puntaje de riesgo (el modelo no la verá)
17     efecto_plan = case_when(plan == "Básico" ~ 0,
18                             plan == "Estándar" ~ -0.6,
19                             TRUE ~ -1.3),
20     riesgo = 2.0 - 0.05 * antigüedad_meses +
21             1.0 * (antigüedad_meses <= 6) + 1.0 * quejas_6m -
22             0.35 * uso_mensual + 0.8 * pagos_tardios + efecto_plan,
23     prob_real = plogis(riesgo),                  # convierte a probabilidad
24     churn = if_else(runif(n_clientes) < prob_real, "Si", "No"),
25     churn = factor(churn, levels = c("No", "Si")),

```



```

26   plan = factor(plan, levels = c("Básico", "Estándar", "Premium")),
27   cuota_mensual = case_when(plan == "Básico" ~ 199,
28                             plan == "Estándar" ~ 349,
29                             TRUE ~ 599)
30 ) %>%
31 select(-efecto_plan, -riesgo, -prob_real)
32
33 glimpse(clientes_churn)
34 clientes_churn %>% count(churn) %>% mutate(pct = round(100 * n / sum(n), 1))

```

```

Rows: 2,000
Columns: 10
$ cliente_id      <chr> "C0001", "C0002", "C0003", "C0004", "C0005"~
$ antiguedad_meses <int> 31, 51, 14, 67, 42, 50, 43, 14, 25, 69, 57,~
$ plan            <fct> Estándar, Premium, Premium, Estándar, Básic~
$ uso_mensual     <int> 9, 8, 5, 14, 11, 14, 8, 10, 10, 8, 8, 9, 8,~
$ quejas_6m       <int> 1, 0, 0, 2, 1, 1, 1, 0, 0, 1, 0, 0, 3, 0, 0~
$ pagos_tardios   <int> 0, 3, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1~
$ genero          <chr> "M", "F", "M", "M", "F", "M", "M", "F", "M"~
$ edad           <int> 58, 44, 27, 57, 19, 43, 48, 36, 67, 18, 40,~
$ churn           <fct> No, No, No, No, No, No, No, No, No, No, No,~
$ cuota_mensual   <dbl> 349, 599, 599, 349, 199, 199, 349, 199, 349~
# A tibble: 2 x 3
  churn      n  pct
  <fct> <int> <dbl>
1 No     1545  77.2
2 Si      455  22.8

```

`glimpse()` muestra las 10 columnas: 7 predictoras, el identificador, la cuota mensual (que usaremos para calcular el valor del cliente) y la variable objetivo `churn`. De los 2,000 clientes, 455 (22.8 %) se fueron. Es una clase **desbalanceada**: los que se quedan son más de tres veces los que se van. Tenlo presente al leer la exactitud.

Consejo

Convierte la variable objetivo a `factor` con los niveles en un orden explícito: `levels = c("No", "Si")`. Así R sabe que “Si” es la clase de interés (la segunda), y `glm()` modelará la probabilidad de “Si”. Evitamos la tilde en “Si” para no tener problemas con nombres de columnas que generan algunas funciones.

Antes de modelar, una mirada exploratoria rápida (como en el Módulo 4): ¿cómo cambia la tasa de abandono con las quejas?

```

1 clientes_churn %>%
2   mutate(quejas = if_else(quejas_6m >= 3, "3 o más",
3                           as.character(quejas_6m))) %>%

```

```

4  group_by(quejas) %>%
5  summarise(clientes = n(),
6            tasa_churn = round(100 * mean(churn == "Si"), 1),
7            .groups = "drop")

```

```

# A tibble: 4 x 3
  quejas  clientes tasa_churn
  <chr>    <int>    <dbl>
1 0         903      13.4
2 1         747      22.6
3 2         270      42.6
4 3 o más     80      62.5

```

La relación es clarísima: sin quejas, solo 13.4% abandona; con 3 o más quejas, 62.5%. Este tipo de tabla es la mejor forma de verificar, antes de modelar, que hay señal en los datos.

Dividimos en entrenamiento (70%) y prueba (30%) de forma **estratificada** con la función `createDataPartition()`, para que ambos conjuntos tengan la misma proporción de abandono:

```

1  set.seed(123)
2  filas_ent <- createDataPartition(clientes_churn$churn, p = 0.7,
3                                list = FALSE)
4  churn_ent   <- clientes_churn[filas_ent, ]
5  churn_prueba <- clientes_churn[-filas_ent, ]
6
7  # Misma proporción de "Si" en ambos conjuntos
8  round(prop.table(table(churn_ent$churn)), 3)
9  round(prop.table(table(churn_prueba$churn)), 3)

```

```

      No    Si
0.772 0.228

```

```

      No    Si
0.773 0.227

```

7.5.2 Regresión logística

Concepto clave: Regresión logística

La **regresión logística** es la “regresión lineal de la clasificación”. En lugar de predecir un número cualquiera, predice una **probabilidad** entre 0 y 1 (por ejemplo, 0.73 = 73% de probabilidad de abandono). En R se ajusta con `glm(..., family = binomial)`. Es rápida, estable y, sobre todo, **explicable**: cada coeficiente dice cómo cambia el riesgo.

```

1 formula_churn <- churn ~ antiguedad_meses + plan + uso_mensual +
2   quejas_6m + pagos_tardios + genero + edad
3
4 modelo_log <- glm(formula_churn, data = churn_ent, family = binomial)
5 summary(modelo_log)

```

Call:

```
glm(formula = formula_churn, family = binomial, data = churn_ent)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	2.114728	0.385699	5.483	4.19e-08 ***
antiguedad_meses	-0.054049	0.004438	-12.178	< 2e-16 ***
planEstándar	-0.448662	0.175005	-2.564	0.0104 *
planPremium	-1.247963	0.234692	-5.317	1.05e-07 ***
uso_mensual	-0.310629	0.032282	-9.622	< 2e-16 ***
quejas_6m	1.048309	0.091879	11.410	< 2e-16 ***
pagos_tardios	0.528275	0.121604	4.344	1.40e-05 ***
generoM	0.019595	0.154767	0.127	0.8992
edad	-0.003247	0.005179	-0.627	0.5306

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1503.2 on 1400 degrees of freedom
 Residual deviance: 1063.0 on 1392 degrees of freedom
 AIC: 1081

Number of Fisher Scoring iterations: 5

Los coeficientes de la logística están en una escala poco intuitiva (*log-odds*). Para explicarlos al negocio, conviértelos en **razones de momios** (*odds ratios*) con `exp()`:

```

1 tibble(variable = names(coef(modelo_log)),
2   odds_ratio = round(exp(coef(modelo_log)), 3)) %>%
3   mutate(efecto = case_when(
4     odds_ratio > 1 ~ paste0("+", round(100 * (odds_ratio - 1)), "% riesgo"),
5     TRUE ~ paste0(round(100 * (odds_ratio - 1)), "% riesgo")
6   )) %>%
7   filter(variable != "(Intercept)")

```

```

# A tibble: 8 x 3
  variable      odds_ratio efecto
  <chr>         <dbl> <chr>
1 antiguedad_meses 0.947 -5% riesgo

```

2	planEstándar	0.638	-36% riesgo
3	planPremium	0.287	-71% riesgo
4	uso_mensual	0.733	-27% riesgo
5	quejas_6m	2.85	+185% riesgo
6	pagos_tardios	1.70	+70% riesgo
7	generoM	1.02	+2% riesgo
8	edad	0.997	0% riesgo

Concepto clave: Cómo leer una razón de momios

Los *momios* (*odds*) son la probabilidad de que algo pase dividida entre la probabilidad de que no pase. Una razón de momios de 1 significa “no cambia el riesgo”; mayor a 1, lo aumenta; menor a 1, lo reduce. Se interpreta *por cada unidad* de la variable y manteniendo las demás constantes.

Traducido al negocio:

- **Quejas:** cada queja adicional multiplica los momios de abandono por 2.85 (casi los triplica). Es la palanca más fuerte: resolver bien una queja vale mucho.
- **Pagos tardíos:** cada pago atrasado aumenta los momios 70 %. Es una señal temprana que cobranza ya tiene y puede compartir.
- **Uso:** cada uso adicional al mes reduce los momios 27 %. Los clientes que usan el servicio se quedan.
- **Antigüedad:** cada mes adicional reduce los momios 5 %; en un año, eso se acumula ($0.947^{12} \approx 0.52$, casi la mitad).
- **Plan:** frente al Básico, el Premium reduce los momios 71 % y el Estándar 36 %.
- **Género y edad:** razones cercanas a 1 y valores p altos (0.90 y 0.53 en el resumen): no influyen, tal como los simulamos.

Observa también que el modelo no “descubrió” el riesgo extra de los primeros 6 meses, porque no le dimos una variable que lo represente; la antigüedad lineal lo absorbe en parte. Crear variables como `cliente_nuevo = antigüedad_meses <= 6` es el tipo de mejora que se logra entendiendo el negocio (ejercicio 7.6).

7.5.3 Predicciones y matriz de confusión

El modelo entrega probabilidades. Para convertirlas en una decisión (llamar o no llamar) necesitamos un **umbral**: si la probabilidad es mayor o igual al umbral, clasificamos como “Si”. Empecemos con el umbral por defecto, 0.5:

```
1 churn_prueba <- churn_prueba %>%
2   mutate(prob_log = predict(modelo_log, newdata = churn_prueba,
3                             type = "response"), # probabilidad de "Si"
```

```

4     pred_log = factor(if_else(prob_log >= 0.5, "Si", "No"),
5                       levels = c("No", "Si"))
6
7 churn_prueba %>%
8   select(cliente_id, antiguedad_meses, quejas_6m, prob_log, pred_log,
9          churn) %>%
10  mutate(prob_log = round(prob_log, 3)) %>%
11  head(5)
12
13 # Matriz de confusión: filas = predicción, columnas = realidad
14 matriz <- table(Prediccion = churn_prueba$pred_log,
15                Real = churn_prueba$churn)
16 matriz

```

```

# A tibble: 5 x 6
  cliente_id antiguedad_meses quejas_6m prob_log pred_log churn
  <chr>          <int>      <int>    <dbl> <fct>    <fct>
1 C0003             14         0    0.181 No      No
2 C0007             43         1    0.154 No      No
3 C0012              9         0    0.119 No      No
4 C0014            26         0    0.224 No      No
5 C0015              7         0    0.416 No      Si

      Real
Prediccion No  Si
      No 434  69
      Si  29  67

```

Concepto clave: Matriz de confusión

Compara lo que el modelo predijo con lo que realmente pasó:

	Real: No se fue	Real: Sí se fue
Predicción: No	Verdadero negativo (VN)	Falso negativo (FN)
Predicción: Sí	Falso positivo (FP)	Verdadero positivo (VP)

- **Falso positivo:** llamas a un cliente que no se iba a ir (desperdicias un descuento).
- **Falso negativo:** no llamas a un cliente que sí se va (lo pierdes). Suele ser el error más caro.

En nuestros 599 clientes de prueba: el modelo acertó 434 clientes que se quedaron (VN) y 67 que se fueron (VP). Se equivocó con 29 clientes que marcó como “se va” y se quedaron (FP) y, lo más grave, dejó pasar a 69 clientes que se fueron sin que los detectara (FN). Mira también el cliente C0015 en la tabla de arriba: tenía 41.6 % de probabilidad y sí se fue; con el umbral de 0.5 no lo habríamos llamado.

7.5.4 Métricas de clasificación

De la matriz de confusión salen todas las métricas. Las calculamos a mano para entender qué significa cada una:

```
1 metricas_clasificacion <- function(prediccion, real, positivo = "Si") {
2   vp <- sum(prediccion == positivo & real == positivo)
3   vn <- sum(prediccion != positivo & real != positivo)
4   fp <- sum(prediccion == positivo & real != positivo)
5   fn <- sum(prediccion != positivo & real == positivo)
6   precision <- vp / (vp + fp)
7   sensibilidad <- vp / (vp + fn)
8   tibble(
9     exactitud = (vp + vn) / (vp + vn + fp + fn),
10    precision = precision,
11    sensibilidad = sensibilidad,
12    especificidad = vn / (vn + fp),
13    F1 = 2 * precision * sensibilidad /
14        (precision + sensibilidad)
15  )
16 }
17
18 metricas_clasificacion(churn_prueba$pred_log, churn_prueba$churn) %>%
19   mutate(across(everything(), ~ round(.x, 3)))
```

```
# A tibble: 1 x 5
  exactitud precision sensibilidad especificidad    F1
  <dbl>      <dbl>      <dbl>      <dbl> <dbl>
1    0.836    0.698    0.493    0.937 0.578
```

Tabla 7.5. Métricas de clasificación y la pregunta de negocio que responden.

Métrica	Fórmula	Pregunta que responde
Exactitud (<i>accuracy</i>)	$(VP + VN)/total$	¿Qué proporción de clientes clasificamos bien? Engañosa si una clase es rara.
Precisión (<i>precision</i>)	$VP/(VP + FP)$	De los clientes que marcamos como “se va”, ¿cuántos realmente se iban? (eficiencia de las llamadas).
Sensibilidad (<i>recall</i>)	$VP/(VP + FN)$	De los clientes que se fueron, ¿a cuántos detectamos? (cobertura).
Especificidad	$VN/(VN + FP)$	De los que se quedaron, ¿a cuántos dejamos tranquilos?
F1	$2 \cdot \frac{prec. \cdot sens.}{prec. + sens.}$	Equilibrio entre precisión y sensibilidad en un solo número.

La exactitud es de 83.6 %, que suena bien, pero la sensibilidad es de solo 49.3 %: con el umbral de 0.5 detectamos a menos de la mitad de los clientes que se van. La precisión de 69.8 % dice que de cada 10 llamadas, 7 serían a clientes realmente en riesgo. Recuerda que predecir “nadie se va” ya

daría 77.3% de exactitud (la proporción de “No” en la prueba), así que la exactitud por sí sola dice poco (sección 7.9).

7.5.5 Curva ROC y AUC

Cada umbral produce una matriz de confusión distinta. La **curva ROC** muestra todos los umbrales a la vez: en el eje horizontal, la tasa de falsos positivos (1 – especificidad); en el vertical, la sensibilidad. El **AUC** (área bajo la curva) resume la curva en un número: la probabilidad de que, al tomar un cliente que se fue y uno que se quedó al azar, el modelo le asigne mayor riesgo al que se fue. Un AUC de 0.5 es adivinar con una moneda; 1 es perfecto; en churn, de 0.75 a 0.85 ya es un buen modelo.

Calculamos la curva a mano, probando 101 umbrales:

```
1 curva_roc <- function(prob, real, positivo = "Si") {
2   map_dfr(seq(0, 1, by = 0.01), function(u) {
3     pred <- prob >= u
4     tibble(umbral = u,
5            sensibilidad = sum(pred & real == positivo) /
6                          sum(real == positivo),
7            tasa_fp = sum(pred & real != positivo) / sum(real != positivo))
8   })
9 }
10
11 # AUC como probabilidad de ordenar bien un par (positivo, negativo)
12 calcular_auc <- function(prob, real, positivo = "Si") {
13   p_pos <- prob[real == positivo]
14   p_neg <- prob[real != positivo]
15   mean(outer(p_pos, p_neg, ">") + 0.5 * outer(p_pos, p_neg, "=="))
16 }
17
18 roc_log <- curva_roc(churn_prueba$prob_log, churn_prueba$churn)
19 auc_log <- calcular_auc(churn_prueba$prob_log, churn_prueba$churn)
20 round(auc_log, 3)
21 # round() evita problemas de precisión al comparar decimales
22 roc_log %>% filter(round(umbral, 2) %in% c(0.1, 0.2, 0.3, 0.5, 0.7))
```

```
[1] 0.86
# A tibble: 5 x 3
  umbral sensibilidad tasa_fp
  <dbl>         <dbl>   <dbl>
1    0.1         0.941  0.460
2    0.2         0.801  0.298
3    0.3         0.691  0.173
4    0.5         0.493  0.0626
5    0.7         0.331  0.0151
```

El AUC es de 0.86: si tomas al azar un cliente que se fue y uno que se quedó, en 86% de los casos el

modelo le asigna más riesgo al que se fue. La tabla muestra el dilema del umbral: con 0.5 detectamos 49.3 % de los que se van molestando solo a 6.3 % de los que se quedan; con 0.2 detectamos 80.1 %, pero molestamos a 29.8 %; con 0.1, 94.1 % y 46 %. En el otro extremo, con 0.7 casi no molestamos a nadie (1.5 %), pero solo detectamos a un tercio (33.1 %) de los que se van.

```

1 grafico_roc <- ggplot(roc_log, aes(x = tasa_fp, y = sensibilidad)) +
2   geom_abline(slope = 1, intercept = 0, color = color_gris,
3     linetype = "dashed", linewidth = 0.8) +
4   geom_path(color = color_principal, linewidth = 0.8) +
5   geom_point(data = filter(roc_log, umbral %in% c(0.2, 0.5)),
6     color = color_resalte, size = 3) +
7   geom_text(data = filter(roc_log, umbral %in% c(0.2, 0.5)),
8     aes(label = paste("umbral", umbral)), color = color_resalte,
9     hjust = -0.2, vjust = 1.2, size = 3.5) +
10  annotate("text", x = 0.75, y = 0.25,
11    label = paste("AUC =", round(auc_log, 3)), size = 4.5) +
12  scale_x_continuous(labels = scales::percent) +
13  scale_y_continuous(labels = scales::percent) +
14  coord_equal() +
15  labs(title = "Curva ROC de la regresión logística",
16    subtitle = "Clientes de prueba; la diagonal equivale a adivinar",
17    x = "Tasa de falsos positivos (1 - especificidad)",
18    y = "Sensibilidad") +
19  theme_minimal() +
20  theme(panel.grid.minor = element_blank())
21 grafico_roc

```

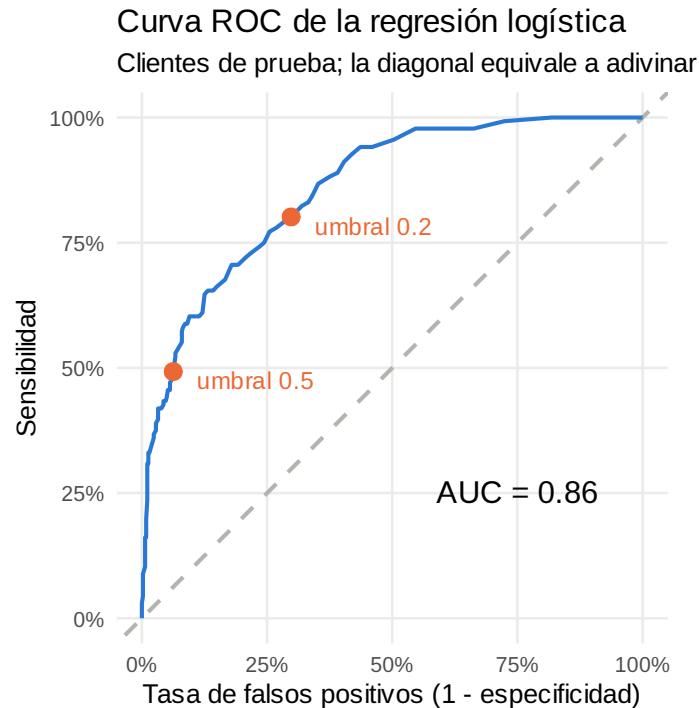



Figura 7.9. Curva ROC del modelo de abandono. Bajar el umbral de 0.5 a 0.2 sube mucho la sensibilidad a cambio de más falsos positivos.

7.5.6 Elegir el umbral según el costo de negocio

El umbral de 0.5 no tiene nada de especial. La decisión correcta depende de cuánto cuesta cada error. Supongamos estos valores (en pesos):

Tabla 7.6. Supuestos de costos para la campaña de retención.

Resultado	Qué pasa	Costo
Verdadero positivo (VP)	Llamamos y ofrecemos descuento; la mitad de ellos se queda y la otra mitad se va de todos modos.	$300 + 0.5 \times 3,000 = 1,800$
Falso positivo (FP)	Damos un descuento a quien no se iba a ir.	300
Falso negativo (FN)	No llamamos y el cliente se va: perdemos su margen de un año.	3,000
Verdadero negativo (VN)	No hacemos nada y el cliente se queda.	0

Con esos costos, calculamos el costo total de la campaña para cada umbral y buscamos el mínimo:

```

1 costo_vp <- 300 + 0.5 * 3000 # descuento + clientes que igual se van
2 costo_fp <- 300              # descuento desperdiciado
3 costo_fn <- 3000             # cliente perdido sin intentar retenerlo
4

```

```

5 costos_umbral <- map_dfr(seq(0.05, 0.95, by = 0.05), function(u) {
6   pred <- churn_prueba$prob_log >= u
7   real <- churn_prueba$churn == "Si"
8   tibble(umbral = u,
9          llamadas = sum(pred),
10         vp = sum(pred & real), fp = sum(pred & !real),
11         fn = sum(!pred & real),
12         costo_total = vp * costo_vp + fp * costo_fp + fn * costo_fn)
13 })
14
15 costos_umbral %>% arrange(costo_total) %>% head(3)
16 costos_umbral %>% filter(umbral %in% c(0.5, 0.95))

```

```

# A tibble: 3 x 6
  umbral llamadas    vp    fp    fn costo_total
  <dbl>    <int> <int> <int> <int>    <dbl>
1  0.15      294   120   174    16    316200
2  0.1       341   128   213     8    318300
3  0.2       247   109   138    27    318600
# A tibble: 2 x 6
  umbral llamadas    vp    fp    fn costo_total
  <dbl>    <int> <int> <int> <int>    <dbl>
1  0.5         96    67    29    69    336300
2  0.95         4     4     0   132    403200

```

Con estos costos, el mejor umbral es 0.15: llamaríamos a 294 clientes de prueba, detectaríamos a 120 de los 136 que se van y el costo total sería \$316,200. Con el umbral por defecto de 0.5 el costo sube a \$336,300, y casi sin llamar a nadie (umbral 0.95) a \$403,200. Elegir bien el umbral ahorra \$20,100 en solo 599 clientes; escalado a una cartera de 100,000 clientes, son más de 3 millones de pesos.

```

1 grafico_costos <- ggplot(costos_umbral, aes(umbral, costo_total)) +
2   geom_line(color = color_principal, linewidth = 0.8) +
3   geom_point(color = color_principal, size = 1.5) +
4   geom_point(data = slice_min(costos_umbral, costo_total, n = 1),
5             color = color_resalte, size = 4) +
6   scale_y_continuous(labels = scales::dollar) +
7   labs(title = "Costo total de la campaña según el umbral",
8        subtitle = "Clientes de prueba; en naranja, el umbral óptimo",
9        x = "Umbral de probabilidad para llamar al cliente",
10       y = "Costo total") +
11   theme_minimal() +
12   theme(panel.grid.minor = element_blank())
13 grafico_costos

```

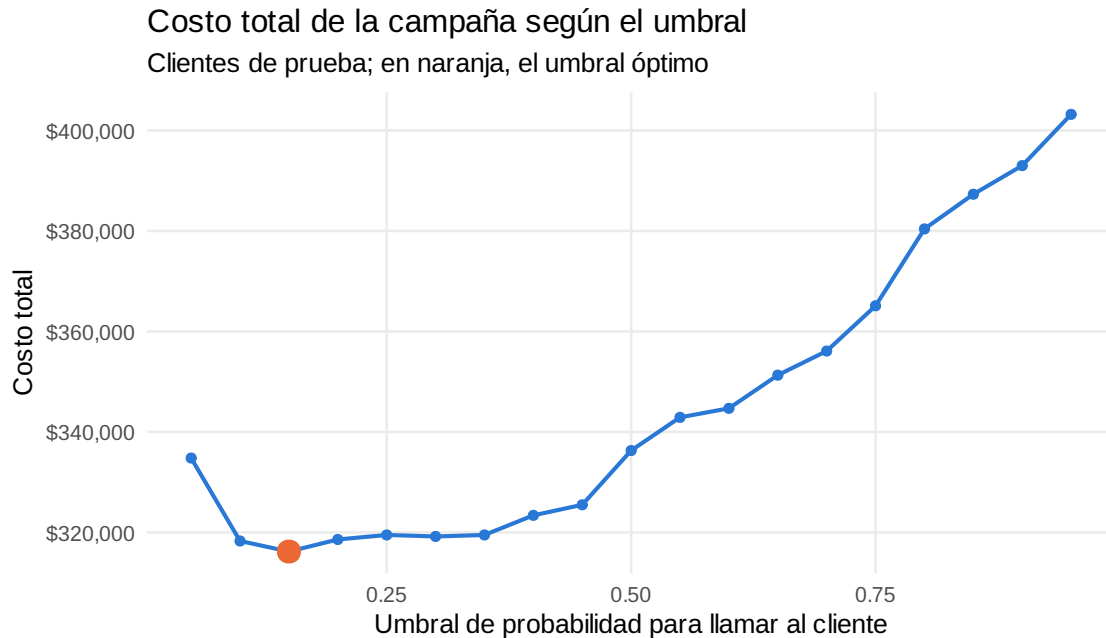


Figura 7.10. Costo total de la campaña de retención para cada umbral. Con errores de costos tan distintos, conviene un umbral mucho más bajo que 0.5.

En la empresa: El umbral es una decisión de negocio, no técnica

Cuando el costo de perder un cliente es mucho mayor que el de un descuento, el umbral óptimo baja: conviene llamar a más gente aunque muchos no se fueran a ir. Si el equipo de retención tiene capacidad limitada (150 llamadas por semana), el umbral lo fija la capacidad: llamas a los 150 de mayor probabilidad. Lo importante es que el modelo *ordena* bien a los clientes por riesgo, y eso lo mide el AUC.

Hazlo tú

Repite el cálculo de costos suponiendo que el descuento cuesta \$800 en lugar de \$300. ¿Sube o baja el umbral óptimo? ¿Por qué?

7.5.7 Árbol de decisión con `rpart`

Un **árbol de decisión** produce reglas del tipo “si antigüedad < 20 meses y uso < 8, entonces riesgo alto”. Su gran ventaja es que cualquier gerente las entiende. Para que el árbol sea legible limitamos su complejidad con el parámetro `cp` (*complexity parameter*): un corte solo se hace si mejora el ajuste al menos en esa proporción.

```
1 set.seed(123)
2 modelo_arbol <- rpart(formula_churn, data = churn_ent, method = "class",
3                       control = rpart.control(cp = 0.02))
4 modelo_arbol
```

```
n= 1401

node), split, n, loss, yval, (yprob)
      * denotes terminal node

1) root 1401 319 No (0.7723055 0.2276945)
  2) antigüedad_meses>=20.5 1014 151 No (0.8510848 0.1489152) *
  3) antigüedad_meses< 20.5 387 168 No (0.5658915 0.4341085)
    6) uso_mensual>=7.5 211 63 No (0.7014218 0.2985782) *
    7) uso_mensual< 7.5 176 71 Si (0.4034091 0.5965909)
      14) quejas_6m< 1.5 134 64 Si (0.4776119 0.5223881)
        28) plan=Estándar,Premium 66 25 No (0.6212121 0.3787879) *
        29) plan=Básico 68 23 Si (0.3382353 0.6617647) *
      15) quejas_6m>=1.5 42 7 Si (0.1666667 0.8333333) *
```

Cada línea es un nodo: número, regla, clientes que llegan, clientes mal clasificados, clase predicha y proporciones (No, Si). Las hojas se marcan con *. Léidas como reglas de negocio:

1. **Antigüedad $\geq$ 21 meses** (1,014 clientes): riesgo bajo, 14.9 % abandona.
2. **Antigüedad < 21 meses y uso $\geq$ 8** (211): riesgo medio, 29.9 %.
3. **Antigüedad < 21, uso < 8 y 2 o más quejas** (42): riesgo muy alto, 83.3 %.
4. **Antigüedad < 21, uso < 8, menos de 2 quejas y plan Básico** (68): riesgo alto, 66.2 %.
5. Lo mismo pero con plan Estándar o Premium (66): 37.9 %.

Estas reglas se pueden pegar tal cual en una presentación o programar en SQL. Coinciden con lo que dijo la logística: antigüedad, uso, quejas y plan.

Para dibujar el árbol, el paquete **rpart.plot** produce diagramas muy claros. No viene con R; si lo instalas, bastan dos líneas:

```
1 install.packages("rpart.plot")
2 library(rpart.plot)
3 rpart.plot(modelo_arbol, type = 4, extra = 104,
4             box.palette = "BuOr", main = "Árbol de abandono")
```

Sin paquetes extra, `plot(modelo_arbol); text(modelo_arbol)` dibuja una versión sencilla con gráficos base. ¿Qué variables usa más el árbol?

```
1 round(modelo_arbol$variable.importance, 1)
2
3 churn_prueba <- churn_prueba %>%
4   mutate(prob_arbol = predict(modelo_arbol, newdata = churn_prueba,
5                               type = "prob")[, "Si"])
6 round(calcular_auc(churn_prueba$prob_arbol, churn_prueba$churn), 3)
```

```

antigüedad_meses      uso_mensual      quejas_6m      plan
                46.1                17.4                7.0                5.4
                edad                genero      pagos_tardios
                0.6                0.4                0.2
[1] 0.674

```

La antigüedad es, por mucho, la variable más importante para el árbol, seguida por el uso. Pero su AUC en prueba es de solo 0.674, bastante peor que el 0.86 de la logística: con apenas 5 hojas, el árbol asigna la misma probabilidad a cientos de clientes y no puede ordenarlos con detalle. Es el precio de la simplicidad.

7.5.8 Random forest

```

1 set.seed(123)
2 modelo_rf <- randomForest(formula_churn, data = churn_ent,
3                           ntree = 300, importance = TRUE)
4 modelo_rf

```

```

Call:
...
      Type of random forest: classification
      Number of trees: 300
No. of variables tried at each split: 2

      OOB estimate of  error rate: 19.49%
Confusion matrix:
      No  Si class.error
No 1008  74  0.06839187
Si  199 120  0.62382445

```

El error OOB es de 19.5 %, pero la matriz de confusión OOB revela el mismo problema de antes: el bosque clasifica bien a los que se quedan (6.8 % de error) y mal a los que se van (62.4 % de error), porque usa el umbral de 0.5 por defecto. Como vimos, para churn conviene trabajar con las probabilidades y elegir el umbral.

```

1 importancia_rf <- importance(modelo_rf) %>%
2   as.data.frame() %>%
3   rownames_to_column("variable") %>%
4   as_tibble() %>%
5   select(variable, MeanDecreaseAccuracy) %>%
6   arrange(desc(MeanDecreaseAccuracy))
7 importancia_rf %>% mutate(MeanDecreaseAccuracy =
8                           round(MeanDecreaseAccuracy, 1))
9

```

```

10 grafico_importancia <- ggplot(importancia_rf,
11     aes(x = MeanDecreaseAccuracy,
12         y = reorder(variable, MeanDecreaseAccuracy))) +
13     geom_col(fill = color_principal, width = 0.7) +
14     labs(title = "¿Qué variables usa el bosque para predecir el abandono?",
15         subtitle = "Pérdida de exactitud al desordenar cada variable",
16         x = "Importancia (disminución media de exactitud)", y = NULL) +
17     theme_minimal() +
18     theme(panel.grid.minor = element_blank(),
19         panel.grid.major.y = element_blank())
20 grafico_importancia

```

```

# A tibble: 7 x 2
  variable      MeanDecreaseAccuracy
  <chr>          <dbl>
1 antigüedad_meses      32.6
2 quejas_6m             31.3
3 uso_mensual           23
4 plan                  6.9
5 pagos_tardios         4.7
6 genero                3
7 edad                 2.4

```

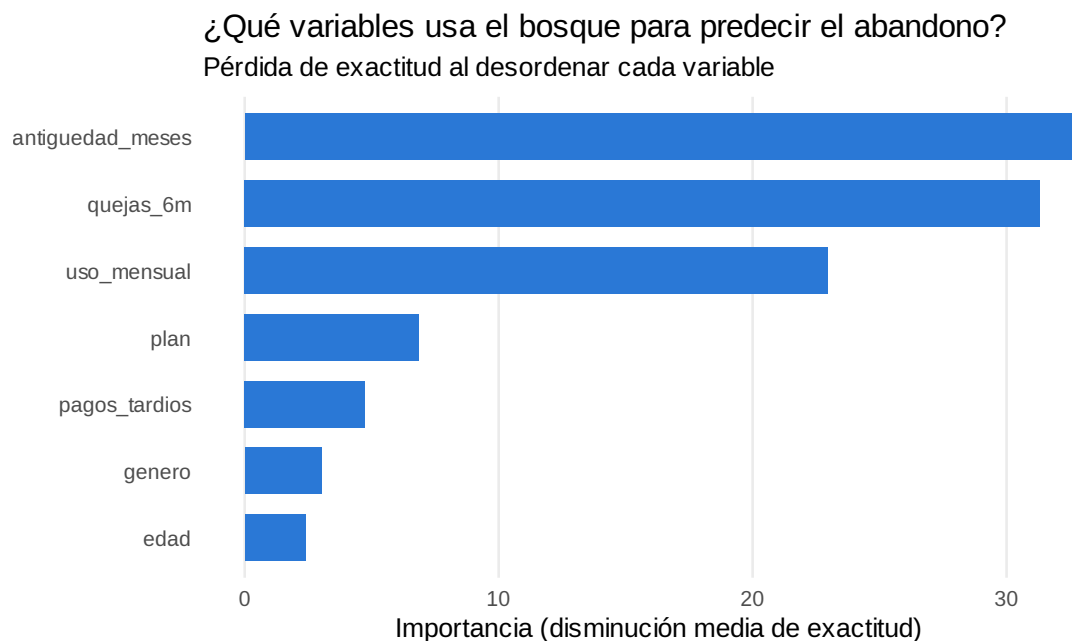


Figura 7.11. Importancia de variables del random forest. Antigüedad, quejas y uso dominan; género y edad no aportan.

Concepto clave: Importancia por permutación

MeanDecreaseAccuracy responde: “si desordeno al azar los valores de esta variable (rompiendo su relación con el abandono), ¿cuánto empeora el modelo?”. Si casi no empeora, la variable no aportaba. Es una forma de abrir un poco la caja negra.

El bosque coincide con los demás modelos: antigüedad (32.6), quejas (31.3) y uso (23.0) son las variables clave; plan y pagos tardíos aportan algo; género y edad aportan muy poco. Que distintos algoritmos coincidan en las variables importantes da confianza en el hallazgo.

7.5.9 Comparar modelos con caret

Hasta ahora cada algoritmo tenía su propia sintaxis. El paquete **caret** ofrece una interfaz común, **train()**, para más de 200 algoritmos, y se encarga de la validación cruzada y del ajuste de parámetros. Comparemos los tres modelos con validación cruzada de 5 pliegues usando el AUC (que **caret** llama **ROC**) como criterio.

```

1  # caret toma como clase positiva el PRIMER nivel del factor
2  churn_ent_caret <- churn_ent %>%
3    mutate(churn = factor(churn, levels = c("Si", "No")))
4
5  control_cv <- trainControl(
6    method = "cv", number = 5,           # validación cruzada de 5 pliegues
7    classProbs = TRUE,                  # calcular probabilidades
8    summaryFunction = twoClassSummary  # reportar ROC, Sens y Spec
9  )
10
11 set.seed(123)
12 cv_log <- train(formula_churn, data = churn_ent_caret, method = "glm",
13                family = binomial, trControl = control_cv, metric = "ROC")
14 set.seed(123)
15 cv_arbol <- train(formula_churn, data = churn_ent_caret,
16                  method = "rpart", trControl = control_cv,
17                  metric = "ROC", tuneLength = 5) # prueba 5 valores de cp
18 set.seed(123)
19 cv_rf <- train(formula_churn, data = churn_ent_caret, method = "rf",
20               trControl = control_cv, metric = "ROC", ntree = 200,
21               tuneGrid = data.frame(mtry = 2:4)) # variables por corte
22
23 cv_rf

```

Random Forest

1401 samples
 7 predictor
 2 classes: 'Si', 'No'

No pre-processing
 Resampling: Cross-Validated (5 fold)
 Summary of sample sizes: 1121, 1121, 1120, 1122, 1120
 Resampling results across tuning parameters:

mtry	ROC	Sens	Spec
2	0.8187057	0.3446925	0.9454600
3	0.8115210	0.4198413	0.9149727
4	0.8048380	0.4293155	0.9094342

ROC was used to select the optimal model using the largest value.
 The final value used for the model was mtry = 2.

`train()` probó tres valores de `mtry` (cuántas variables considera el bosque en cada corte) y eligió el de mayor AUC promedio en los 5 pliegues. Ahora juntamos los tres modelos en una tabla:

```
1 comparacion <- resamples(list(Logistica = cv_log, Arbol = cv_arbol,
2                               RandomForest = cv_rf))
3
4 summary(comparacion)$statistics$ROC[, c("Min.", "Mean", "Max.")] %>%
5   round(3)
```

	Min.	Mean	Max.
Logistica	0.828	0.843	0.882
Arbol	0.657	0.691	0.735
RandomForest	0.795	0.819	0.842

Y confirmamos con los datos de prueba, que ningún modelo ha visto:

```
1 churn_prueba <- churn_prueba %>%
2   mutate(prob_rf = predict(modelo_rf, newdata = churn_prueba,
3                             type = "prob")[, "Si"])
4
5 tibble(
6   modelo = c("Regresión logística", "Árbol de decisión",
7             "Random forest"),
8   auc_prueba = c(calcular_auc(churn_prueba$prob_log, churn_prueba$churn),
9                  calcular_auc(churn_prueba$prob_arbol, churn_prueba$churn),
10                 calcular_auc(churn_prueba$prob_rf, churn_prueba$churn))
11 ) %>%
12   mutate(auc_prueba = round(auc_prueba, 3)) %>%
13   arrange(desc(auc_prueba))
```



```
# A tibble: 3 x 2
  modelo          auc_prueba
  <chr>          <dbl>
1 Regresión logística 0.86
2 Random forest      0.846
3 Árbol de decisión  0.674
```

Validación cruzada y prueba cuentan la misma historia: la regresión logística gana (AUC de 0.843 en validación cruzada y 0.86 en prueba), seguida muy de cerca por el random forest (0.819 y 0.846), mientras que el árbol sencillo queda muy atrás (0.691 y 0.674). Como en la regresión de ventas, la relación simulada es casi lineal y el modelo simple gana. Con la logística tenemos, además, odds ratios que el negocio entiende: es el modelo que llevaríamos a producción.

7.5.10 Del modelo a la acción: lista priorizada de clientes en riesgo

El entregable final para el equipo de retención no es un AUC: es una **lista de clientes ordenada por prioridad**. Una buena práctica es ordenar por **pérdida esperada**: probabilidad de abandono $\times$ valor del cliente (aquí, su cuota de un año). Así, un cliente Premium con 50 % de riesgo puede ir antes que uno Básico con 70 %.

```
1 lista_retencion <- churn_prueba %>%
2   mutate(valor_anual = cuota_mensual * 12,
3          perdida_esperada = prob_log * valor_anual) %>%
4   arrange(desc(perdida_esperada)) %>%
5   mutate(prioridad = row_number()) %>%
6   select(prioridad, cliente_id, plan, quejas_6m, prob_log,
7          perdida_esperada)
8
9 lista_retencion %>%
10  mutate(prob_log = round(prob_log, 2),
11         perdida_esperada = round(perdida_esperada)) %>%
12  head(8)
```

```
# A tibble: 8 x 6
  prioridad cliente_id plan      quejas_6m prob_log perdida_esperada
  <int>   <chr>    <fct>    <int>    <dbl>         <dbl>
1         1    C0259  Premium      2      0.88          6313
2         2    C1336  Premium      2      0.87          6261
3         3    C0609  Premium      1      0.67          4838
4         4    C1058  Premium      1      0.66          4768
5         5    C1912  Premium      2      0.57          4085
6         6    C1072  Estándar     3      0.97          4049
7         7    C1354  Premium      2      0.56          4029
8         8    C0795  Premium      2      0.55          3973
```

Los primeros lugares los ocupan clientes Premium con probabilidad alta: su cuota de \$599 hace que perderlos cueste más. El cliente C1072 tiene la mayor probabilidad de la tabla (0.97), pero como es Estándar queda en el sexto lugar. Esta lista, exportada a CSV cada semana, es lo que realmente usa el call center (sección 7.8).

Hazlo tú

Agrega a la lista una columna `motivo_principal` con `case_when()`: “Quejas” si `quejas_6m` ≥ 2 , “Cliente nuevo” si `antiguedad_meses` ≤ 6 , “Poco uso” si `uso_mensual` < 5 y “Otro” en los demás casos. El equipo de retención podrá adaptar su guion según el motivo.

7.6 Caso 4: pronóstico de demanda con series de tiempo

Ejemplo práctico: ¿Cuántas unidades pedimos para diciembre?

El área de compras de la categoría de electrónica debe colocar sus pedidos con dos meses de anticipación. Si pide de menos en noviembre y diciembre (Buen Fin y Navidad) pierde ventas; si pide de más, se queda con inventario que en enero nadie quiere. La pregunta: *¿cuántas unidades venderemos cada mes del próximo año, y con qué margen de error?*

Una **serie de tiempo** es una secuencia de mediciones ordenadas en el tiempo a intervalos regulares (ventas diarias, demanda mensual). Sus componentes típicos son:

- **Tendencia:** la dirección de largo plazo (el negocio crece).
- **Estacionalidad:** patrones que se repiten cada año (picos en noviembre y diciembre, caída en enero y febrero).
- **Ruido** (componente irregular): lo que queda, variaciones que no siguen un patrón.

7.6.1 Datos: cinco años de ventas mensuales

Las ventas del curso solo cubren 2023 y son aleatorias, así que simulamos 5 años (2021–2025) de unidades vendidas por mes con una tendencia creciente (unas 15 unidades más cada mes), una estacionalidad típica del retail mexicano (picos de 15% y 32% sobre lo normal en noviembre y diciembre) y 3% de ruido.

```
1 set.seed(123)
2 meses <- seq(as.Date("2021-01-01"), as.Date("2025-12-01"), by = "month")
3 n_meses <- length(meses) # 60 meses
4
5 # Factor estacional de cada mes (1 = mes normal)
6 factor_mes <- c(0.88, 0.85, 0.95, 0.97, 1.04, 0.96, # ene-jun
7               0.97, 1.00, 0.93, 0.98, 1.15, 1.32) # jul-dic
8 factor_mes <- factor_mes / mean(factor_mes) # promedio = 1
9
```

```

10 ventas_mensuales <- tibble(
11   mes = meses,
12   tendencia = 1500 + 15 * (1:n_meses),
13   unidades = round(tendencia * factor_mes[month(meses)] *
14                   (1 + rnorm(n_meses, 0, 0.03))) # ruido de 3 %
15 ) %>%
16   select(-tendencia)
17
18 # Objeto ts: la estructura de series de tiempo de R
19 serie <- ts(ventas_mensuales$unidades, start = c(2021, 1),
20           frequency = 12) # 12 = datos mensuales
21 serie

```

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
2021	1311	1292	1536	1516	1644	1605	1578	1559	1489	1595	1985	2242
2022	1510	1458	1611	1778	1852	1599	1768	1774	1634	1782	2056	2402
2023	1619	1525	1855	1871	1944	1942	1930	1962	1905	2022	2386	2748
2024	1838	1756	1963	2014	2154	2032	2002	2301	2096	2074	2505	2889
2025	2013	1908	2168	2210	2384	2309	2240	2446	2088	2363	2753	3189

El objeto `ts` guarda los valores junto con su calendario: inicio (`start = c(2021, 1)`, enero de 2021) y frecuencia (12 observaciones por año; usa 4 para trimestres, 52 para semanas). Por eso R imprime la serie como una tabla año por mes (los nombres de los meses siempre salen en inglés en este formato). Ya a simple vista se nota el crecimiento de un año a otro y los picos de diciembre.

```

1 grafico_serie <- autoplot(serie, color = color_principal,
2                           linewidth = 0.8) +
3   scale_y_continuous(labels = scales::comma) +
4   labs(title = "Unidades vendidas por mes, electrónica",
5        subtitle = "2021-2025 (datos simulados)",
6        x = NULL, y = "Unidades") +
7   theme_minimal() +
8   theme(panel.grid.minor = element_blank())
9 grafico_serie

```



Figura 7.12. Serie mensual de unidades vendidas: tendencia creciente y picos cada fin de año.

`autoplot()` de **forecast** convierte objetos de series de tiempo en gráficas de **ggplot2**, así que puedes agregarles capas, escalas y temas como a cualquier otra.

7.6.2 Descomposición: separar tendencia, estacionalidad y ruido

```
1 descomposicion <- decompose(serie, type = "multiplicative")
2
3 # Índices estacionales: cuánto se aleja cada mes de un mes "normal"
4 nombres_mes <- c("ene", "feb", "mar", "abr", "may", "jun",
5                 "jul", "ago", "sep", "oct", "nov", "dic")
6 tibble(mes = nombres_mes, indice = round(descomposicion$figure, 3)) %>%
7   arrange(desc(indice)) %>%
8   head(4)
9
10 grafico_descomp <- autoplot(descomposicion) +
11   labs(title = "Descomposición multiplicativa de la serie",
12        x = NULL) +
13   theme_minimal() +
14   theme(panel.grid.minor = element_blank())
15 grafico_descomp
```

```
# A tibble: 4 x 2
  mes   indice
<chr> <dbl>
```

1 dic	1.32
2 nov	1.16
3 may	1.03
4 ago	1

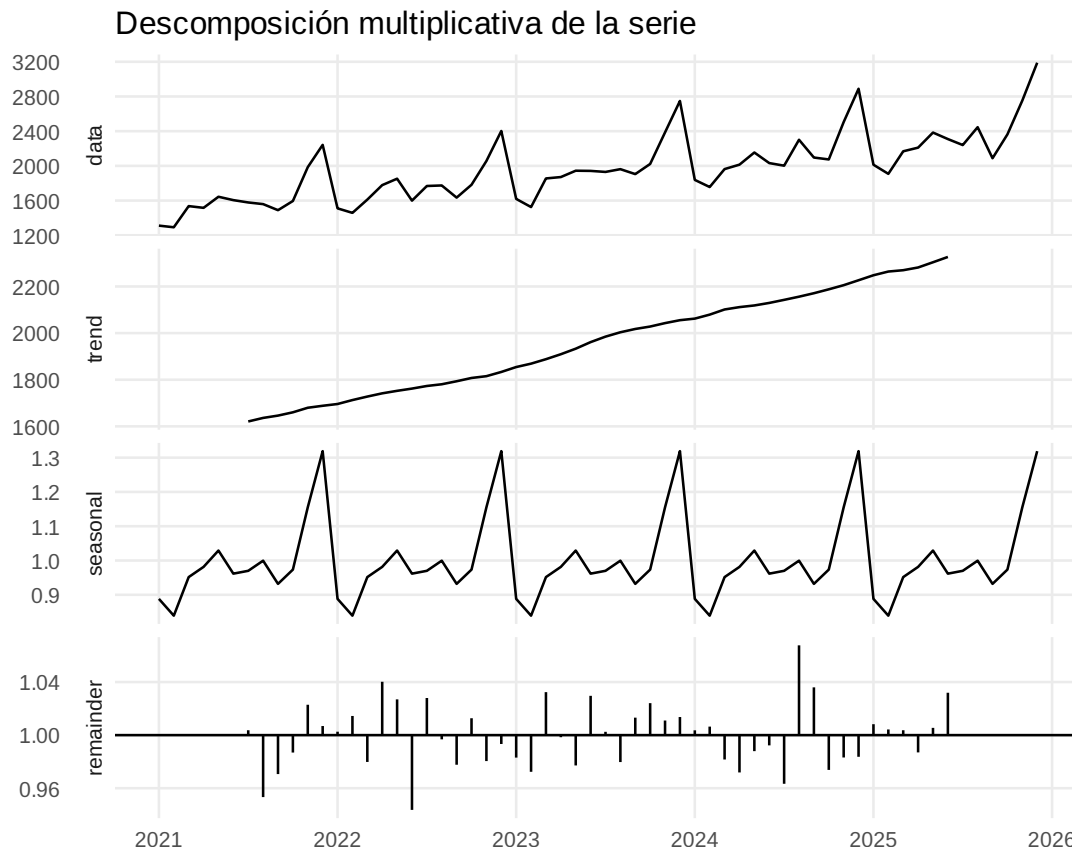


Figura 7.13. Descomposición de la serie: datos originales, tendencia, componente estacional y residuo (*remainder*).

Usamos el modelo *multiplicativo* porque la estacionalidad es un porcentaje (diciembre vende “32 % más”, no “400 unidades más”), y ese porcentaje se mantiene aunque el negocio crezca. Los índices estacionales recuperan casi exactamente lo que simulamos: diciembre vende 32 % más que un mes normal (índice 1.32) y noviembre 16 % más (1.16). Esos índices, por sí solos, ya son información valiosa para planear personal, inventario y presupuesto de publicidad.

Hazlo tú

Usa `stl(serie, s.window = "periodic")` en lugar de `decompose()` y grafica el resultado con `autoplot()`. STL es un método de descomposición más robusto ante valores atípicos (trabaja de forma aditiva; para una versión multiplicativa aplica `log()` a la serie).

7.6.3 Entrenamiento y prueba en series de tiempo

En series de tiempo **no** se separan los datos al azar: el pasado entrena y el futuro evalúa. Guardamos los últimos 12 meses (todo 2025) como prueba, que es exactamente lo que queremos saber hacer: pronosticar un año.

```
1 serie_ent    <- window(serie, end = c(2024, 12))    # 2021-2024: 48 meses
2 serie_prueba <- window(serie, start = c(2025, 1))    # 2025: 12 meses
3 c(length(serie_ent), length(serie_prueba))
```

```
[1] 48 12
```

7.6.4 Cuatro modelos de pronóstico

Concepto clave: Modelos de pronóstico

- **Ingenuo** (`naive()`): el próximo mes será igual al último. Parece tonto, pero es la línea base obligatoria.
- **Ingenuo estacional** (`snaive()`): cada mes será igual al mismo mes del año anterior (diciembre 2025 = diciembre 2024).
- **ETS** (`ets()`, *Error, Trend, Seasonal*): suavizamiento exponencial; da más peso a los datos recientes y modela explícitamente nivel, tendencia y estacionalidad. Elige solo la mejor combinación.
- **ARIMA** (`auto.arima()`): modela la relación de cada mes con los meses anteriores y con sus propios errores pasados; `auto.arima()` busca la mejor especificación automáticamente.

```
1 h <- length(serie_prueba)    # horizonte: 12 meses
2
3 pron_naive <- naive(serie_ent, h = h)
4 pron_snaive <- snaive(serie_ent, h = h)
5 modelo_ets <- ets(serie_ent)
6 pron_ets   <- forecast(modelo_ets, h = h)
7 modelo_arima <- auto.arima(serie_ent)
8 pron_arima <- forecast(modelo_arima, h = h)
9
10 modelo_ets    # ¿qué combinación eligió ets()?
11 modelo_arima
```

```
ETS(M,A,M)
```

```
Call:
```

```

ets(y = serie_ent)

Smoothing parameters:
  alpha = 0.0199
  beta  = 4e-04
  gamma = 1e-04

Initial states:
  l = 1527.3166
  b = 14.5543
  s = 1.3231 1.1584 0.9739 0.9307 1.0039 0.9739
      0.9589 1.0257 0.9811 0.9589 0.8368 0.8747

sigma: 0.0339

      AIC      AICc      BIC
597.8241 618.2241 629.6346
Series: serie_ent
ARIMA(0,0,0)(1,1,0)[12] with drift

Coefficients:
      sar1      drift
    -0.5378  14.9348
s.e.    0.1418   0.7466

sigma^2 = 5463: log likelihood = -207
AIC=420.01  AICc=420.76  BIC=424.76

```

`ets()` eligió un modelo **ETS(M,A,M)**: error multiplicativo, tendencia aditiva (*A*, crece una cantidad fija cada mes: **b = 14.55**, muy cerca de las 15 unidades que simulamos) y estacionalidad multiplicativa (*M*). `auto.arima()` eligió un modelo que compara cada mes con el mismo mes del año anterior (la parte **(1,1,0)[12]**) más una deriva (*drift*) de 14.9 unidades por mes: otra forma de describir la misma tendencia. No necesitas memorizar la notación; lo importante es que ambos modelos capturaron tendencia y estacionalidad sin que se las indicáramos.

7.6.5 Comparar los modelos con `accuracy()`

`accuracy()` calcula las métricas de error del pronóstico contra los valores reales. Nos interesa la fila **Test set** (error en 2025, que los modelos no vieron):

```

1 accuracy(pron_ets, serie_prueba) # salida completa de un modelo
2
3 comparacion_ts <- list(Ingenue = pron_naive,
4                        `Ingenue estacional` = pron_snaive,
5                        ETS = pron_ets, ARIMA = pron_arima) %>%
6   map_dfr(function(p) {
7     acc <- accuracy(p, serie_prueba)["Test set", ]

```

```

8   tibble(MAE = acc["MAE"], RMSE = acc["RMSE"], MAPE = acc["MAPE"])
9   }, .id = "modelo") %>%
10  mutate(across(where(is.numeric), ~ round(.x, 1))) %>%
11  arrange(MAPE)
12  comparacion_ts

```

```

          ME      RMSE      MAE      MPE      MAPE
Training set -7.685706 52.06332 43.34896 -0.5184226 2.336622
Test set      27.712908 60.48803 48.44893  1.1527656 2.130479
          MASE      ACF1 Theil's U
Training set 0.2483390 0.1141467      NA
Test set      0.2775559 -0.5886038 0.2412182
# A tibble: 4 x 4
  modelo      MAE  RMSE  MAPE
  <chr>      <dbl> <dbl> <dbl>
1 ARIMA      46.8  68.3    2
2 ETS        48.4  60.5   2.1
3 Ingenio estacional 205.  219.   8.7
4 Ingenio     600.  642  27.3

```

La salida completa de `accuracy()` incluye más métricas (`ME` es el error promedio con signo: 27.7 indica que ETS se quedó ligeramente corto en 2025; `MASE` compara contra el ingenuo). En la tabla resumida, ETS y ARIMA prácticamente empatan con errores de apenas 2 % (MAPE) en 2025: ARIMA tiene un MAE algo menor (46.8 contra 48.4) y ETS un RMSE menor (60.5 contra 68.3), es decir, menos errores grandes. El ingenuo estacional se equivoca 8.7 % porque ignora el crecimiento, y el ingenuo simple, 27.3 %, porque ignora la estacionalidad: diciembre de 2024 no se parece a enero de 2025. Elegimos ETS por su menor RMSE: en inventario, los errores grandes son los que duelen.

```

1  grafico_comparacion <- autoplot(window(serie, start = c(2023, 1)),
2                                color = "black", linewidth = 0.6) +
3  autolayer(pron_snaive, series = "Ingenio estacional", PI = FALSE) +
4  autolayer(pron_ets, series = "ETS", PI = FALSE) +
5  autolayer(pron_arima, series = "ARIMA", PI = FALSE) +
6  scale_color_manual(values = paleta_libro[c(2, 1, 3)]) +
7  scale_y_continuous(labels = scales::comma) +
8  labs(title = "Pronósticos de 2025 contra la realidad",
9       subtitle = "Negro: ventas reales; entrenamiento hasta dic-2024",
10      x = NULL, y = "Unidades", color = NULL) +
11  theme_minimal() +
12  theme(panel.grid.minor = element_blank(), legend.position = "top")
13  grafico_comparacion

```


Pronósticos de 2025 contra la realidad

Negro: ventas reales; entrenamiento hasta dic-2024

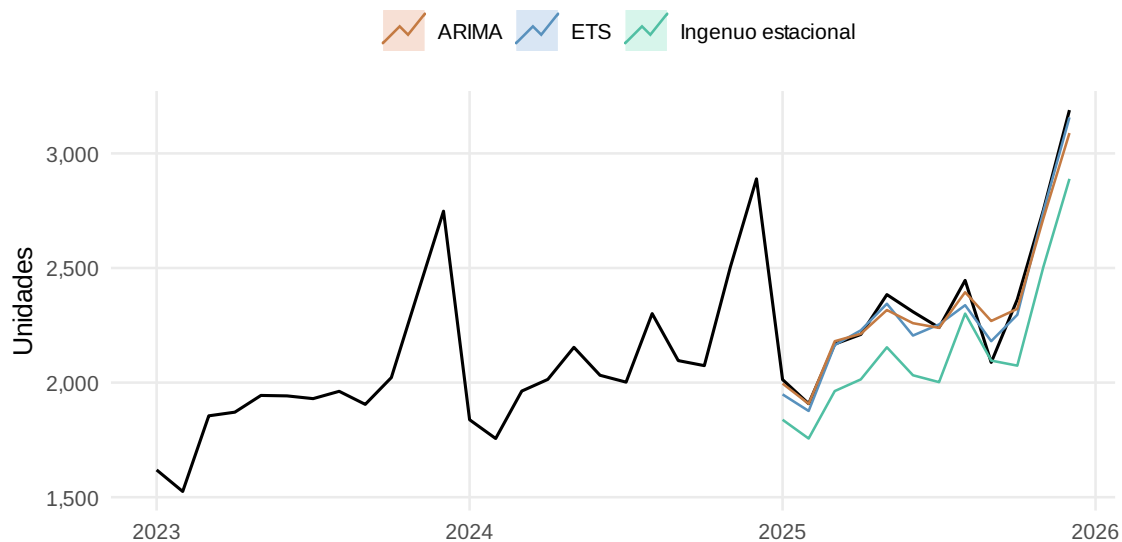


Figura 7.14. Pronósticos de 2025 de tres modelos contra las ventas reales. ETS y ARIMA siguen de cerca la realidad; el ingenio estacional se queda corto porque ignora el crecimiento.

7.6.6 Pronóstico final con intervalos

Una vez elegido el modelo, se **reentrena con todos los datos** (incluido 2025) para pronosticar 2026:

```
1 modelo_final <- ets(serie)
2 pronostico_2026 <- forecast(modelo_final, h = 12, level = c(80, 95))
3 pronostico_2026
```

	Point Forecast	Lo 80	Hi 80	Lo 95	Hi 95
Jan 2026	2125.663	2040.343	2210.984	1995.177	2256.150
Feb 2026	2037.570	1955.751	2119.389	1912.439	2162.702
Mar 2026	2333.729	2239.979	2427.480	2190.350	2477.108
Apr 2026	2400.133	2303.674	2496.591	2252.612	2547.653
May 2026	2547.002	2444.599	2649.406	2390.390	2703.615
Jun 2026	2411.232	2314.247	2508.217	2262.906	2559.557
Jul 2026	2420.791	2323.381	2518.200	2271.816	2569.766
Aug 2026	2548.945	2446.337	2651.553	2392.020	2705.871
Sep 2026	2349.700	2255.074	2444.327	2204.982	2494.419
Oct 2026	2488.212	2387.967	2588.457	2334.901	2641.523
Nov 2026	2975.194	2855.281	3095.107	2791.803	3158.585
Dec 2026	3411.931	3274.360	3549.501	3201.535	3622.327

Concepto clave: Intervalos de predicción

Point Forecast es la estimación central. **Lo 80** y **Hi 80** forman el intervalo del 80 %: esperamos que 8 de cada 10 meses la venta real caiga dentro de ese rango. El intervalo del 95 % es más ancho porque exige más seguridad. Los intervalos se ensanchan a medida que el pronóstico se aleja: es más difícil predecir diciembre que enero.

```
1 grafico_pronostico <- autoplot(pronostico_2026, color = color_principal,
2                               fcol = color_resalte, flwd = 0.8) +
3   scale_y_continuous(labels = scales::comma) +
4   labs(title = "Pronóstico de unidades para 2026",
5         subtitle = "Modelo ETS; bandas de 80 % y 95 %",
6         x = NULL, y = "Unidades") +
7   theme_minimal() +
8   theme(panel.grid.minor = element_blank())
9 grafico_pronostico
```

Pronóstico de unidades para 2026

Modelo ETS; bandas de 80 % y 95 %

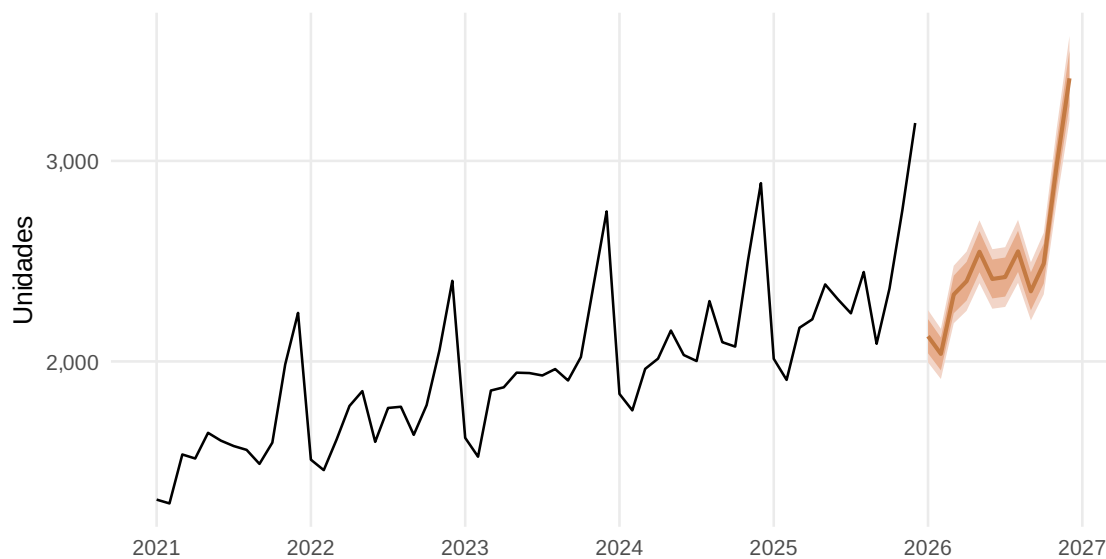


Figura 7.15. Pronóstico de 12 meses con intervalos de predicción del 80 % (banda oscura) y 95 % (banda clara).

7.6.7 Comunicar la incertidumbre a la dirección

Un pronóstico sin rango es una promesa que no se va a cumplir exactamente. Presenta siempre tres escenarios en lenguaje de negocio:

```

1 tabla_direccion <- tibble(
2   mes           = format(seq(as.Date("2026-01-01"), by = "month",
3                             length.out = 12), "%b %Y"),
4   pesimista     = round(pronostico_2026$lower[, "80%"]),
5   esperado      = round(pronostico_2026$mean),
6   optimista     = round(pronostico_2026$upper[, "80%"])
7 )
8 tabla_direccion %>% slice(c(1, 6, 11, 12))
9 sum(tabla_direccion$esperado) # unidades esperadas en todo 2026

```

```

# A tibble: 4 x 4
  mes      pesimista esperado optimista
<chr>      <dbl>      <dbl>      <dbl>
1 ene 2026      2040      2126      2211
2 jun 2026      2314      2411      2508
3 nov 2026      2855      2975      3095
4 dic 2026      3274      3412      3550
[1] 30051

```

Para la dirección: en diciembre de 2026 esperamos vender 3,412 unidades; en un escenario pesimista (límite inferior del 80 %), 3,274, y en uno optimista, 3,550. En todo el año se esperan 30,051 unidades. Observa que `format(..., "%b%Y")` sí respeta el idioma del sistema (“ene 2026”).

En la empresa: Cómo presentar un pronóstico

- Da un rango y un escenario central: “esperamos vender 3,412 unidades en diciembre; con 80 % de confianza, entre 3,274 y 3,550”.
- Explica los supuestos: “suponemos que el crecimiento y la estacionalidad de los últimos cinco años se mantienen”.
- Di qué lo podría romper: una devaluación, un competidor nuevo, una promoción extraordinaria, una pandemia.
- Compara con el pronóstico anterior y con lo que realmente pasó: la credibilidad se gana mostrando el historial de aciertos y errores.

7.6.8 Del pronóstico al inventario: stock de seguridad

El pronóstico central no basta para decidir cuánto pedir: si pides exactamente lo esperado, la mitad de los meses te faltará mercancía. El **stock de seguridad** es el inventario extra que cubre la incertidumbre. Una fórmula sencilla y muy usada es:

$$\text{stock de seguridad} = z \times \sigma_{\text{error}} \times \sqrt{L}$$

donde z depende del **nivel de servicio** deseado (1.28 para 90 %, 1.65 para 95 %, 2.33 para 99 %),

σ_{error} es la desviación estándar del error de pronóstico (usaremos el RMSE del modelo ETS en 2025) y L es el tiempo de reabastecimiento en meses.

```

1 nivel_servicio <- 0.95
2 z <- qnorm(nivel_servicio)                # 1.645
3 error_tipico <- comparacion_ts %>%
4   filter(modelo == "ETS") %>% pull(RMSE)   # error típico en 2025
5 tiempo_entrega <- 2                      # meses de anticipación
6
7 stock_seguridad <- z * error_tipico * sqrt(tiempo_entrega)
8
9 tabla_direccion %>%
10  slice(11:12) %>%
11  mutate(stock_seguridad = round(stock_seguridad),
12         pedido_sugerido = esperado + stock_seguridad) %>%
13  select(mes, esperado, stock_seguridad, pedido_sugerido)

```

```

# A tibble: 2 x 4
  mes      esperado stock_seguridad pedido_sugerido
<chr>      <dbl>      <dbl>      <dbl>
1 nov 2026    2975          141          3116
2 dic 2026    3412          141          3553

```

Con un nivel de servicio de 95 % ($z = 1.645$), un error típico de 60.5 unidades y dos meses de anticipación, el stock de seguridad es de 141 unidades. El pedido sugerido para diciembre es de 3,553 unidades: el pronóstico central más el colchón. Subir el nivel de servicio a 99 % aumenta el colchón (y el costo de inventario); es otra decisión de negocio que el modelo te ayuda a cuantificar.

▲ Cuidado

Un modelo de series de tiempo asume que **el futuro se parece al pasado**. No sabe que el próximo año abrirás 20 tiendas, que un proveedor quebró o que habrá una promoción extraordinaria. Pronostica a 12 meses; más allá, la incertidumbre crece tanto que el número deja de ser útil para decisiones operativas. Y revisa el pronóstico cada mes con los datos nuevos.

7.7 Caso 5: análisis de canasta (market basket)

Ejemplo práctico: ¿Qué le ofrecemos al cliente que compra una laptop?

Una tienda de electrónica y papelería quiere diseñar promociones de venta cruzada (*cross-selling*): “lleva el mouse con 20 % de descuento”, colocar productos juntos en el anaquel o recomendar productos en la tienda en línea. La pregunta: *¿qué productos se compran juntos con más frecuencia de la que esperaríamos por casualidad?*

El **análisis de canasta** (*market basket analysis*) busca **reglas de asociación** del tipo {Laptop} →

{Mouse}: “quien compra una laptop tiende a comprar un mouse”. Es aprendizaje no supervisado: no hay nada que predecir, solo patrones que descubrir en los tickets de compra.

7.7.1 Soporte, confianza y lift

Tres medidas describen una regla $X \rightarrow Y$:

- **Soporte:** proporción de tickets que contienen X y Y . Mide qué tan frecuente es la combinación.

$$\text{soporte}(X \rightarrow Y) = \frac{\text{tickets con } X \text{ y } Y}{\text{total de tickets}}$$

- **Confianza:** de los tickets que tienen X , qué proporción también tiene Y . Es una probabilidad condicional.

$$\text{confianza}(X \rightarrow Y) = \frac{\text{tickets con } X \text{ y } Y}{\text{tickets con } X}$$

- **Lift:** cuántas veces más probable es comprar Y cuando se compra X , comparado con comprar Y en general.

$$\text{lift}(X \rightarrow Y) = \frac{\text{confianza}(X \rightarrow Y)}{\text{soporte}(Y)}$$

Lift = 1: no hay relación; lift > 1: se compran juntos más de lo esperado; lift < 1: se “evitan”.

Ejemplo a mano. En 1,000 tickets, 100 tienen laptop, 200 tienen mouse y 60 tienen ambos.

- Soporte de {Laptop, Mouse}: $60/1,000 = 0.06$ (6 % de los tickets).
- Confianza de Laptop $\rightarrow$ Mouse: $60/100 = 0.60$ (60 % de quienes compran laptop llevan mouse).
- Soporte de Mouse: $200/1,000 = 0.20$ (20 % de todos los tickets llevan mouse).
- Lift: $0.60/0.20 = 3$. Quien compra laptop tiene **tres veces** más probabilidad de llevar mouse que un cliente cualquiera.

▲ Cuidado

La confianza sola engaña. Si el 30 % de todos los tickets lleva papel, una regla “Laptop $\rightarrow$ Papel” con confianza de 30 % parece buena, pero su lift es 1: el papel se vende igual con o sin laptop. Por eso ordenamos las reglas por lift y exigimos un soporte mínimo (las reglas con muy pocos tickets son casualidades).

7.7.2 ¿Sirven los datos del curso?

Antes de simular, revisemos si `transacciones.csv` permite formar canastas. Consideremos una canasta como todos los productos que un cliente compró el mismo día:

```
1 canastas_curso <- transacciones %>%
2   group_by(cliente_id, fecha) %>%
3   summarise(productos = n_distinct(producto_id), .groups = "drop")
```

```

4
5 canastas_curso %>% count(productos, name = "canastas")

```

```

# A tibble: 2 x 2
  productos canastas
    <int>     <int>
1       1       974
2       2        13

```

De 987 canastas, solo 13 tienen dos productos; ninguna tiene más. Con los datos del curso no hay canastas que analizar (cada transacción es prácticamente una compra de un solo producto). Otra vez: sin señal no hay modelo. Simulamos, por lo tanto, 3,000 tickets de una tienda con asociaciones conocidas.

7.7.3 Tickets simulados con asociaciones conocidas

Ocho productos se compran de forma independiente, y cuatro dependen de otro: el mouse y la funda son mucho más probables si el ticket incluye una laptop, el cable HDMI si incluye un monitor y la tinta si incluye una impresora.

```

1 set.seed(123)
2 n_tickets <- 3000
3 tickets_ancho <- tibble(
4   ticket_id = 1:n_tickets,
5   Laptop    = rbinom(n_tickets, 1, 0.08), # 1 = el ticket lo incluye
6   Monitor   = rbinom(n_tickets, 1, 0.07),
7   Impresora = rbinom(n_tickets, 1, 0.05),
8   Audifonos = rbinom(n_tickets, 1, 0.15),
9   USB       = rbinom(n_tickets, 1, 0.18),
10  Papel     = rbinom(n_tickets, 1, 0.20),
11  Teclado   = rbinom(n_tickets, 1, 0.08)
12 ) %>%
13 mutate(
14   # Asociaciones "reales" que el análisis debe descubrir
15   Mouse = if_else(Laptop == 1, rbinom(n(), 1, 0.60),
16                   rbinom(n(), 1, 0.08)),
17   Funda = if_else(Laptop == 1, rbinom(n(), 1, 0.40),
18                   rbinom(n(), 1, 0.02)),
19   `Cable HDMI` = if_else(Monitor == 1, rbinom(n(), 1, 0.50),
20                           rbinom(n(), 1, 0.04)),
21   Tinta = if_else(Impresora == 1, rbinom(n(), 1, 0.70),
22                   rbinom(n(), 1, 0.06))
23 )
24
25 # Formato largo: una fila por ticket y producto (como en un sistema real)
26 tickets <- tickets_ancho %>%

```

```

27 pivot_longer(-ticket_id, names_to = "producto", values_to = "compro") %>%
28 filter(compro == 1) %>%
29 select(-compro)
30
31 head(tickets, 5)
32 n_total <- n_distinct(tickets$ticket_id) # tickets con al menos 1 producto
33 n_total

```

```

# A tibble: 5 x 2
  ticket_id producto
  <int> <chr>
1         1 Teclado
2         2 Monitor
3         2 Audifonos
4         3 USB
5         4 USB
[1] 1975

```

El formato largo (una fila por ticket y producto) es exactamente como vendrían los datos de un sistema de punto de venta. Hay 1,975 tickets con al menos un producto (los tickets vacíos de la simulación se descartan).

7.7.4 Calcular reglas de pares con dplyr

Primero el soporte de cada producto:

```

1 soporte_producto <- tickets %>%
2   count(producto, name = "tickets_x") %>%
3   mutate(soporte_x = tickets_x / n_total) %>%
4   arrange(desc(soporte_x))
5 soporte_producto %>% mutate(soporte_x = round(soporte_x, 3))

```

```

# A tibble: 11 x 3
  producto tickets_x soporte_x
  <chr>         <int>      <dbl>
1 Papel           596      0.302
2 USB             557      0.282
3 Audifonos       442      0.224
4 Mouse           386      0.195
5 Tinta           276      0.14
6 Laptop          225      0.114
7 Cable HDMI      215      0.109
8 Teclado         210      0.106
9 Monitor         205      0.104
10 Impresora      149      0.075

```

11 Funda 144 0.073

Ahora el truco: un **self-join** (unir la tabla consigo misma por **ticket_id**) genera todos los pares de productos que aparecen en el mismo ticket. Luego contamos cuántos tickets tiene cada par y calculamos las tres medidas:

```
1 reglas <- tickets %>%
2   inner_join(tickets, by = "ticket_id", suffix = c("_x", "_y"),
3             relationship = "many-to-many") %>%
4   filter(producto_x != producto_y) %>% # sin pares consigo mismo
5   count(producto_x, producto_y, name = "tickets_xy") %>%
6   left_join(soporte_producto, by = c("producto_x" = "producto")) %>%
7   left_join(soporte_producto %>%
8     select(producto_y = producto, soporte_y = soporte_x),
9     by = "producto_y") %>%
10  mutate(soporte = tickets_xy / n_total,
11         confianza = tickets_xy / tickets_x,
12         lift = confianza / soporte_y)
13
14 mejores_reglas <- reglas %>%
15   filter(soporte >= 0.02) %>% # soporte mínimo: 2 %
16   arrange(desc(lift)) %>%
17   transmute(regla = paste(producto_x, "->", producto_y),
18             tickets_xy, soporte = round(soporte, 3),
19             confianza = round(confianza, 3), lift = round(lift, 2))
20 mejores_reglas %>% head(10)
```

```
# A tibble: 10 x 5
  regla tickets_xy soporte confianza lift
  <chr>      <int>   <dbl>    <dbl> <dbl>
1 Funda -> Laptop      82  0.042    0.569  5
2 Laptop -> Funda      82  0.042    0.364  5
3 Impresora -> Tinta  104  0.053    0.698  4.99
4 Tinta -> Impresora  104  0.053    0.377  4.99
5 Cable HDMI -> Monitor 105  0.053    0.488  4.71
6 Monitor -> Cable HDMI 105  0.053    0.512  4.71
7 Laptop -> Mouse     136  0.069    0.604  3.09
8 Mouse -> Laptop     136  0.069    0.352  3.09
9 Funda -> Mouse       61  0.031    0.424  2.17
10 Mouse -> Funda      61  0.031    0.158  2.17
```

El análisis descubrió exactamente las cuatro asociaciones que simulamos, y en ambas direcciones:

- **Impresora** → **Tinta**: confianza de 69.8 % (simulamos 70 %) y lift de 4.99. Casi 7 de cada 10 compradores de impresora llevan tinta.
- **Monitor** → **Cable HDMI**: confianza de 51.2 % (simulamos 50 %), lift de 4.71.

- **Laptop → Mouse:** confianza de 60.4 % (simulamos 60 %), lift de 3.09, y es la regla con más tickets (136).
- **Laptop → Funda:** lift de 5.0, el más alto.

Observa que el lift es simétrico (Laptop → Funda y Funda → Laptop tienen el mismo), pero la confianza no: 56.9 % de quienes compran funda llevan laptop, pero solo 36.4 % de quienes compran laptop llevan funda. Y aparece una regla que *no* simulamos directamente: Funda → Mouse (lift 2.17). No es que la funda “cause” el mouse: ambos se compran con laptop. Las reglas de asociación describen coocurrencias, no causas.

```
1 grafico_lift <- mejores_reglas %>%
2   filter(lift > 1.5) %>%
3   ggplot(aes(x = lift, y = reorder(regla, lift))) +
4   geom_col(fill = color_principal, width = 0.7) +
5   geom_vline(xintercept = 1, color = color_gris, linetype = "dashed") +
6   labs(title = "Reglas de asociación con mayor lift",
7        subtitle = "Soporte mínimo 2 %; la línea punteada es lift = 1",
8        x = "Lift (veces más probable que al azar)", y = NULL) +
9   theme_minimal() +
10  theme(panel.grid.minor = element_blank(),
11        panel.grid.major.y = element_blank())
12 grafico_lift
```

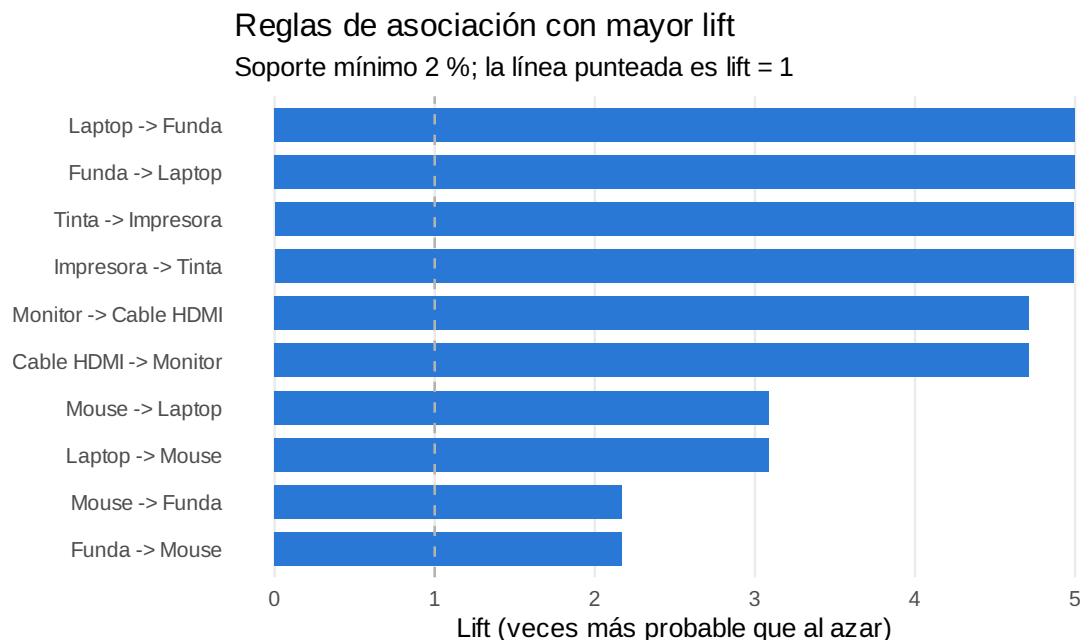


Figura 7.16. Reglas de asociación ordenadas por lift. Las diez reglas corresponden a los cuatro pares que simulamos, en ambas direcciones, más la relación indirecta Funda–Mouse.

¿Y las reglas con lift menor a 1? También dan información:

```
1 mejores_reglas %>% arrange(lift) %>% head(3)
```

```
# A tibble: 3 x 5
  regla tickets_xy soporte confianza lift
  <chr>      <int>   <dbl>    <dbl> <dbl>
1 Papel -> USB      100  0.051    0.168 0.59
2 USB -> Papel      100  0.051    0.18 0.59
3 Audifonos -> Mouse  52  0.026    0.118 0.6
```

Papel y USB aparecen juntos *menos* de lo esperado (lift 0.59), aunque los simulamos de forma independiente. Es un efecto de cómo se forman los datos: al quedarnos solo con tickets que tienen al menos un producto, los productos frecuentes “compiten” por aparecer. En datos reales, un lift claramente menor a 1 suele indicar **productos sustitutos** (dos marcas de la misma tinta, por ejemplo): no tiene sentido promocionarlos juntos. La lección es que ningún número se interpreta sin entender cómo se generaron los datos.

7.7.5 Con el paquete **arules**

Para canastas con cientos de productos y reglas de más de dos productos ($\{Laptop, Mouse\} \rightarrow \{Funda\}$), el estándar en R es el paquete **arules**, que implementa el algoritmo **Apriori**. No está instalado en este entorno, pero así se usaría con los mismos datos:

```
1 install.packages("arules")
2 library(arules)
3
4 # Convertir a formato de transacciones: una lista de productos por ticket
5 lista_tickets <- split(tickets$producto, tickets$ticket_id)
6 trans <- as(lista_tickets, "transactions")
7 summary(trans)
8
9 reglas_apriori <- apriori(trans,
10                           parameter = list(supp = 0.02, conf = 0.3,
11                                              minlen = 2))
12 inspect(head(sort(reglas_apriori, by = "lift"), 10))
```

7.7.6 De las reglas a las recomendaciones

En la empresa: Acciones de venta cruzada

- **Paquetes:** “Kit oficina en casa” con Impresora + Tinta, o “Kit estudiante” con Laptop + Mouse + Funda, con un pequeño descuento sobre la suma.
- **Ubicación en tienda:** los cables HDMI junto a los monitores; las tintas a la vista desde

el pasillo de impresoras.

- **Recomendaciones en línea:** al agregar una laptop al carrito, mostrar mouse y funda (las reglas con mayor confianza desde Laptop).
- **No descontar lo que ya se vende junto:** si el 70 % de quienes compran impresora ya lleva tinta, un descuento en tinta regala margen. Mejor ofrecer el descuento al 30 % restante (por ejemplo, en el correo de seguimiento).
- **Medir:** lanza la promoción en algunas tiendas y compara con otras similares sin promoción antes de extenderla.

Hazlo tú

Calcula las reglas cuyo antecedente es "Laptop" (`filter(producto_x == "Laptop")`) y ordénalas por confianza. ¿Qué producto recomendarías primero en el carrito de compras? ¿Coincide con el orden por lift?

7.8 Llevar el modelo a producción

Un modelo que vive solo en tu computadora no genera valor. **Desplegar** (o “poner en producción”) un modelo significa que se ejecuta de forma periódica y confiable sobre datos nuevos, y que sus resultados llegan a quien toma decisiones. En BI, lo más común es empezar con algo sencillo: un script que cada lunes puntúa a los clientes y deja un CSV (o una tabla en la base de datos, Módulo 5) que alimenta el tablero y el call center.

7.8.1 Guardar y cargar el modelo

No necesitas reentrenar cada vez: guarda el objeto del modelo en un archivo `.rds` con `saveRDS()` y cárgalo con `readRDS()`.

```
1 dir.create("resultados", showWarnings = FALSE)
2
3 saveRDS(modelo_log, "resultados/modelo_churn.rds")
4
5 # ... días después, en otro script o en otra computadora:
6 modelo_cargado <- readRDS("resultados/modelo_churn.rds")
7
8 # Comprobamos que predice exactamente lo mismo
9 all.equal(predict(modelo_cargado, churn_prueba, type = "response"),
10           predict(modelo_log, churn_prueba, type = "response"))
```

```
[1] TRUE
```

 Consejo

Guarda junto al modelo la información para reproducirlo: fecha de entrenamiento, periodo de datos, métricas en prueba y versión de R y de los paquetes. Un nombre como `modelo_churn_2025-06_v2.rds` y una nota en un archivo de texto te ahorrarán muchos dolores de cabeza.

7.8.2 Una función para puntuar clientes nuevos

Encapsula la predicción en una función que **valide** los datos de entrada, calcule la probabilidad y la traduzca a una acción:

```

1 predecir_churn <- function(nuevos_datos, modelo, umbral = 0.15) {
2   # 1. Validar que vengan todas las columnas que el modelo necesita
3   necesarias <- c("antiguedad_meses", "plan", "uso_mensual", "quejas_6m",
4                 "pagos_tardios", "genero", "edad")
5   faltan <- setdiff(necesarias, names(nuevos_datos))
6   if (length(faltan) > 0) {
7     stop("Faltan columnas: ", paste(faltan, collapse = ", "))
8   }
9   # 2. Mismos niveles de factor que en el entrenamiento
10  nuevos_datos <- nuevos_datos %>%
11    mutate(plan = factor(plan, levels = c("Básico", "Estándar",
12                                          "Premium")))
13  # 3. Probabilidad y acción sugerida
14  nuevos_datos %>%
15    mutate(
16      prob_churn = predict(modelo, newdata = nuevos_datos,
17                          type = "response"),
18      nivel_riesgo = cut(prob_churn, breaks = c(0, umbral, 0.5, 1),
19                        labels = c("Bajo", "Medio", "Alto"),
20                        include.lowest = TRUE),
21      accion = if_else(prob_churn >= umbral, "Llamar", "Sin acción")
22    )
23 }
```

Simulamos un lote de 500 clientes activos de esta semana (en la vida real vendría de la base de datos) y lo puntuamos:

```

1 set.seed(123)
2 lote_semana <- tibble(
3   cliente_id      = sprintf("N%04d", 1:500),
4   antiguedad_meses = sample(1:72, 500, replace = TRUE),
5   plan            = sample(c("Básico", "Estándar", "Premium"), 500,
6                           replace = TRUE, prob = c(0.5, 0.3, 0.2)),
7   uso_mensual     = rpois(500, 8),
8   quejas_6m       = rpois(500, 0.8),
```

```

9   pagos_tardios    = rpois(500, 0.4),
10  genero            = sample(c("F", "M"), 500, replace = TRUE),
11  edad              = sample(18:70, 500, replace = TRUE)
12 )
13
14 puntuados <- predecir_churn(lote_semana, modelo_cargado)
15 puntuados %>% count(nivel_riesgo, accion)
16
17 # Exportar la lista para el equipo de retención
18 puntuados %>%
19   filter(accion == "Llamar") %>%
20   arrange(desc(prob_churn)) %>%
21   mutate(prob_churn = round(prob_churn, 3)) %>%
22   select(cliente_id, plan, quejas_6m, prob_churn, nivel_riesgo) %>%
23   write_csv("resultados/clientes_a_llamar.csv")
24
25 file.exists("resultados/clientes_a_llamar.csv")

```

```

# A tibble: 3 x 3
  nivel_riesgo accion      n
  <fct>        <chr>    <int>
1 Bajo        Sin acción  275
2 Medio       Llamar     142
3 Alto        Llamar      83
[1] TRUE

```

De los 500 clientes, 225 superan el umbral de 0.15 y se llaman: 83 de riesgo alto (probabilidad de 0.5 o más) y 142 de riesgo medio; 275 no requieren acción. `file.exists()` confirma que el archivo quedó en `resultados/`. Si el equipo solo puede hacer 150 llamadas, empieza por los de riesgo alto y sigue en orden de probabilidad (o de pérdida esperada).

¿Qué pasa si alguien le pasa a la función un archivo incompleto? En lugar de un error críptico de R, recibe un mensaje claro:

```

1 predecir_churn(select(lote_semana, -quejas_6m), modelo_cargado)
2 #> Error in predecir_churn(select(lote_semana, -quejas_6m), modelo_cargado):
3 #> Faltan columnas: quejas_6m

```

7.8.3 Monitoreo y reentrenamiento

Los modelos se degradan con el tiempo porque el mundo cambia. Se llama **deriva de datos** (*data drift*) a cuando los datos nuevos ya no se parecen a los de entrenamiento (por ejemplo, llega una ola de clientes nuevos por una campaña) y **deriva de concepto** (*concept drift*) a cuando cambia la relación entre variables y respuesta (un competidor lanza un plan más barato y ahora se van también los clientes antiguos).

Un monitoreo básico compara cada semana las variables del lote con las del entrenamiento:

```

1  comparar_distribucion <- function(entrenamiento, nuevo, variables) {
2    map_dfr(variables, function(v) {
3      tibble(variable = v,
4              media_entrenamiento = mean(entrenamiento[[v]]),
5              media_nuevo = mean(nuevo[[v]]))
6    }) %>%
7      mutate(cambio_pct = round(100 * (media_nuevo - media_entrenamiento) /
8                                media_entrenamiento, 1),
9            alerta = if_else(abs(cambio_pct) > 20, "REVISAR", "ok"))
10   }
11
12  comparar_distribucion(churn_ent, lote_semana,
13                        c("antiguedad_meses", "uso_mensual", "quejas_6m",
14                          "pagos_tardios")) %>%
15    mutate(across(starts_with("media"), ~ round(.x, 2)))

```

```

# A tibble: 4 x 5
  variable      media_entrenamiento media_nuevo cambio_pct alerta
  <chr>          <dbl>          <dbl>    <dbl> <chr>
1 antiguedad_meses      36.7          36.8      0.3 ok
2 uso_mensual           7.97           8.22      3.2 ok
3 quejas_6m             0.78           0.79      2.4 ok
4 pagos_tardios         0.41           0.39     -4 ok

```

En este lote todas las variables se parecen a las del entrenamiento (cambios de 4 % o menos), así que no hay alerta. Si un mes la media de quejas saltara de 0.78 a 1.5 (por ejemplo, por una falla en el servicio), el modelo seguiría funcionando, pero la tasa de “Llamar” se dispararía y convendría revisar tanto el modelo como la operación. El umbral de 20 % es arbitrario: ajústalo a tu negocio. Existen medidas más formales, como el índice de estabilidad poblacional (PSI), pero la idea es la misma.

Buenas prácticas de monitoreo y reentrenamiento:

- **Mide el desempeño real:** tres meses después, compara las predicciones con quién realmente se fue. Si el AUC cae de 0.86 a 0.70, es hora de reentrenar.
- **Reentrena con calendario** (por ejemplo, cada trimestre) y **por alerta** (cuando el monitoreo detecta cambios).
- **Compara el modelo nuevo contra el actual** en los mismos datos de prueba antes de reemplazarlo.
- **Automatiza:** programa el script con el Programador de tareas de Windows, **cron** o RStudio Connect, y envía un reporte automático (Módulo 8).

7.8.4 Ética, sesgos y explicabilidad

Un modelo toma decisiones que afectan a personas: a quién se le ofrece un descuento, a quién se le aprueba un crédito, a quién se contrata. Tienes la responsabilidad de que sea justo y explicable.

- **Variables sensibles:** género, edad, estado civil, origen o código postal (que puede reflejar nivel socioeconómico) pueden llevar a discriminar. En nuestro modelo, género y edad no aportaban nada (odds ratios de 1.02 y 0.997, importancia mínima): lo correcto es **quitarlas**. En decisiones como crédito o contratación, su uso puede ser además ilegal.
- **Variables “proxy”:** aunque quites el género, otra variable muy correlacionada (por ejemplo, la categoría de productos que compra) puede reintroducirlo. Revisa las tasas de predicción por grupo.
- **Sesgo histórico:** si en el pasado solo se ofrecían créditos a cierto perfil, el modelo aprenderá que “solo ese perfil paga bien”. El modelo repite los sesgos de los datos.
- **Explicabilidad:** prefiere modelos interpretables (logística, árboles) cuando la decisión debe justificarse ante un cliente o un regulador. Con modelos de caja negra, usa la importancia de variables y explica los casos individuales.
- **Privacidad:** usa solo los datos necesarios, respeta los avisos de privacidad y la legislación de protección de datos personales.

Revisemos si el modelo trata distinto a hombres y mujeres del lote:

```
1 puntuados %>%
2   group_by(genero) %>%
3   summarise(clientes = n(),
4             prob_media = round(mean(prob_churn), 3),
5             pct_llamar = round(100 * mean(accion == "Llamar"), 1),
6             .groups = "drop")
```

```
# A tibble: 2 x 4
  genero clientes prob_media pct_llamar
  <chr>    <int>      <dbl>    <dbl>
1 F         258      0.223      40.7
2 M         242      0.238      49.6
```

La probabilidad media es casi igual (0.223 contra 0.238), pero se llamaría a 49.6% de los hombres y a 40.7% de las mujeres. Como el coeficiente de género es prácticamente nulo, la diferencia viene de las otras variables de este lote en particular (por azar, los hombres tienen algo más de quejas o menos uso) y del efecto pequeño pero no nulo que el modelo le asignó a **generoM**. Aun así, este tipo de tabla es la que debes revisar y documentar: si la diferencia fuera grande y se debiera a la variable sensible, habría que quitarla y volver a evaluar.

 Hazlo tú

Reentrena la regresión logística sin **genero** ni **edad** y calcula su AUC en **churn_prueba**. ¿Perdiste capacidad predictiva? Si la respuesta es “casi nada”, ya tienes un argumento para usar el modelo más justo y más simple.

7.9 Errores comunes

7.9.1 Fuga de datos: el modelo demasiado bueno

Agreguemos a los clientes de churn una variable que el área de atención registra: si el cliente llamó al área de cancelaciones. Como la mayoría de los que se van llaman para cancelar, parece una variable maravillosa...

```

1 set.seed(123)
2 churn_fuga <- clientes_churn %>%
3   mutate(llamo_cancelaciones = if_else(
4     churn == "Si", rbinom(n(), 1, 0.85),    # casi todos los que se van
5     rbinom(n(), 1, 0.03)))                 # muy pocos de los que se quedan
6
7 fuga_ent    <- churn_fuga[filas_ent, ]    # misma división que antes
8 fuga_prueba <- churn_fuga[-filas_ent, ]
9
10 modelo_fuga <- glm(update(formula_churn, . ~ . + llamo_cancelaciones),
11                     data = fuga_ent, family = binomial)
12 prob_fuga <- predict(modelo_fuga, fuga_prueba, type = "response")
13 round(calcular_auc(prob_fuga, fuga_prueba$churn), 3)

```

```
[1] 0.964
```

El AUC salta de 0.86 a 0.964. En el laboratorio parece un modelo extraordinario. Pero en la operación no sirve: cuando sabemos que el cliente llamó a cancelaciones, ya decidió irse. La variable es una consecuencia del abandono, no una señal anticipada.

 Cuidado

Pregunta obligatoria para cada variable: **¿conoceré este dato en el momento en que tenga que predecir?** Si un modelo tiene resultados “demasiado buenos para ser verdad” (AUC mayor a 0.95 en un problema de comportamiento humano), sospecha primero de fuga de datos.

7.9.2 Evaluar con los datos de entrenamiento

Evaluar un modelo con los mismos datos con que se entrenó da resultados optimistas, sobre todo en modelos flexibles. El random forest es un caso extremo:


```

1 prob_rf_ent <- predict(modelo_rf, newdata = churn_ent,
2                       type = "prob")[, "Si"]
3 c(auc_entrenamiento = calcular_auc(prob_rf_ent, churn_ent$churn),
4   auc_prueba = calcular_auc(churn_prueba$prob_rf, churn_prueba$churn)) %>%
5   round(3)

```

```

auc_entrenamiento    auc_prueba
              1.000              0.846

```

En entrenamiento, el bosque tiene un AUC perfecto (1.000): cada árbol crece hasta memorizar a sus clientes. En prueba baja a 0.846. Si hubieras reportado el 1.000, habrías prometido algo imposible. Por eso existen el conjunto de prueba, la validación cruzada y el error OOB.

7.9.3 Clases desbalanceadas y la exactitud engañosa

Un banco revisa 2,000 transacciones con tarjeta, de las cuales 5 % son fraude. Un “modelo” que dice *siempre* “no es fraude”:

```

1 set.seed(123)
2 fraude <- tibble(
3   es_fraude = factor(if_else(runif(2000) < 0.05, "Si", "No"),
4                     levels = c("No", "Si"))
5 )
6 prediccion_perezosa <- factor(rep("No", 2000), levels = c("No", "Si"))
7
8 metricas_clasificacion(prediccion_perezosa, fraude$es_fraude) %>%
9   mutate(across(everything(), ~ round(.x, 3)))

```

```

# A tibble: 1 x 5
  exactitud precision sensibilidad especificidad    F1
    <dbl>      <dbl>      <dbl>      <dbl> <dbl>
1    0.947      NaN          0          1    NaN

```

¡94.7% de exactitud sin detectar un solo fraude! La sensibilidad es 0 y la precisión y F1 son **NaN** (*Not a Number*: división entre cero, porque el modelo nunca predijo “Si”). Un modelo inútil con una exactitud altísima: por eso, con clases desbalanceadas, la exactitud es la métrica menos informativa.

Qué hacer con clases desbalanceadas:

- Reporta sensibilidad, precisión, F1 y AUC, no solo exactitud.
- Compara siempre con la línea base “predecir la clase mayoritaria”.
- Ajusta el umbral según costos (sección 7.5.6).
- Usa divisiones estratificadas (`createDataPartition()`).

- Si hace falta, rebalancea el entrenamiento: **caret** lo permite con `trainControl(sampling = "down")` o `"up"`.

7.9.4 No escalar antes de k-means

```

1 set.seed(123)
2 kmeans_sin_escalar <- kmeans(rfm %>% select(recencia, frecuencia, monto),
3                               centers = 4, nstart = 25)
4
5 rfm %>%
6   mutate(grupo = kmeans_sin_escalar$cluster) %>%
7   group_by(grupo) %>%
8   summarise(clientes = n(), recencia = round(mean(recencia)),
9             frecuencia = round(mean(frecuencia), 1),
10            monto = round(mean(monto)), .groups = "drop") %>%
11   arrange(monto)

```

```

# A tibble: 4 x 5
  grupo clientes recencia frecuencia monto
  <int>   <int>   <dbl>      <dbl> <dbl>
1     3     56    106        2.9  1946
2     4     83     66         5  4491
3     1     44     39        6.2  7089
4     2     15     37        9.7 11092

```

Sin escalar, los grupos quedan ordenados casi perfectamente por monto (de \$1,946 a \$11,092): el monto, que se mide en miles, domina la distancia. La recencia apenas participa: ya no aparece un grupo de Dormidos con 195 días de recencia como en la segmentación escalada, y los clientes que dejaron de comprar quedan mezclados con los demás según lo que gastaron. Escalar no es un detalle técnico: cambia los segmentos y, por lo tanto, las estrategias.

7.9.5 Otros errores frecuentes

Tabla 7.7. Otros errores comunes en proyectos de machine learning.

Error	Cómo evitarlo
Usar un código como número (<code>tienda_id</code> , código postal)	<code>lm(ventas ~ tienda_id)</code> estima un solo coeficiente, como si la tienda 10 fuera “diez veces” la tienda 1. Conviértelo con <code>factor(tienda_id)</code> , como hicimos en la sección 7.1.4.
Niveles de factor distintos entre entrenamiento y datos nuevos	Fija los niveles con <code>factor(x, levels = ...)</code> (como en <code>predecir_churn()</code>). Un nivel nuevo (un plan “Familiar”) produce el error <code>factor plan has new levels Familiar</code> .
Extrapolar	Un modelo entrenado con tiendas de 150 a 1,200 m ² no sabe nada de una de 5,000 m ² . Un pronóstico a 5 años con 5 años de historia es una apuesta, no una predicción.
Separar series de tiempo al azar	El conjunto de prueba siempre son los periodos más recientes (<code>window()</code>).
Olvidar <code>set.seed()</code>	Sin semilla, k-means, la división de datos y el random forest dan resultados distintos cada vez y nadie puede reproducir tu análisis.
Confundir correlación con causalidad	Un coeficiente dice “se asocia con”, no “provoca”. Para medir el efecto de una acción, diseña un experimento (grupo de control).
Nombrar clusters por número	Asigna nombres a partir del perfil calculado (sección 7.3).

7.10 Resumen del módulo

Resumen del módulo

- El machine learning aprende patrones de datos históricos. Supervisado: regresión (números) y clasificación (categorías). No supervisado: clustering y reglas de asociación. Las series de tiempo pronostican con la historia de la propia variable.
- Sin señal no hay modelo: primero verifica que exista relación (exploración, línea base).
- Evalúa siempre en datos que el modelo no vio (prueba, validación cruzada); vigila el sobreajuste y la fuga de datos.
- Segmentación: RFM + escalar + k-means (codo, silueta, `nstart`) + perfil + nombres basados en el perfil + estrategia por segmento. Alternativa sencilla: quintiles con `ntile()`.
- Regresión: `lm()` explica y predice; mide con MAE, RMSE, MAPE y R²; compara con una línea base y con random forest.
- Clasificación: logística (odds ratios), árbol (reglas), random forest (importancia); matriz de confusión, sensibilidad, precisión, AUC; el umbral se elige con costos de negocio.
- Series de tiempo: `ts()`, descomposición, prueba con los últimos meses, `naive/snaive` como línea base, `ets()` y `auto.arima()`; comunica rangos y calcula stock de seguridad.
- Canasta: soporte, confianza y lift con un self-join; ordena por lift con soporte mínimo.

- Producción: `saveRDS()`, función de puntuación con validación, exportar resultados, monitorear deriva, reentrenar y cuidar la ética.

Tabla 7.8. Guía rápida: del problema a la función de R y la métrica.

Problema	Algoritmo	Función de R	Métrica
Segmentar clientes	k-means	<code>scale()</code> , <code>kmeans()</code> , <code>fviz_nbclust()</code>	Codo (WSS), silueta
Segmentar con reglas	Quintiles RFM	<code>ntile()</code> , <code>case_when()</code>	Tamaño y valor de segmentos
Predecir un número	Regresión lineal	<code>lm()</code> , <code>predict()</code>	MAE, RMSE, MAPE, R ²
Predecir un número (no lineal)	Random forest	<code>randomForest()</code>	MAE, RMSE, OOB
Predecir una categoría	Regresión logística	<code>glm(family = binomial)</code>	AUC, sensibilidad, precisión, F1
Reglas explicables	Árbol de decisión	<code>rpart()</code>	AUC, matriz de confusión
Comparar modelos	Validación cruzada	<code>trainControl()</code> , <code>train()</code> , <code>resamples()</code>	ROC (AUC) promedio
Pronosticar demanda	Ingenuo, ETS, ARIMA	<code>snaive()</code> , <code>ets()</code> , <code>auto.arima()</code> , <code>forecast()</code>	<code>accuracy()</code> : MAE, RMSE, MAPE
Productos que se compran juntos	Reglas de asociación	self-join con <code>inner_join()</code> ; <code>arules::apriori()</code>	Soporte, confianza, lift
Usar el modelo	Despliegue	<code>saveRDS()</code> , <code>readRDS()</code> , <code>write_csv()</code>	Desempeño real y deriva

7.11 Ejercicios

Para los ejercicios con datos simulados, copia el código de simulación de la sección correspondiente (con la misma semilla) al inicio de tu script. Todos los ejercicios suponen que tienes cargados los paquetes de este módulo.

Ejercicio 7.1: Supervisado o no supervisado

Nivel: **Básico**

Para cada situación, indica si es un problema de regresión, clasificación, clustering, reglas de asociación o series de tiempo, y propón un algoritmo: (a) estimar el ticket promedio de un cliente nuevo; (b) detectar si un correo de cliente es una queja; (c) agrupar las sucursales por su mezcla de ventas; (d) predecir la demanda semanal de pan de muerto; (e) decidir qué producto recomendar a quien compra una cafetera. Después, con `ventas_retail.csv`, ajusta `lm(total ~ descuento_pct + dia_semana)` y reporta su R² ajustado. ¿Hay señal?

📖 Solución en la página 680

Ejercicio 7.2: Métricas de regresión a manoNivel: **Básico**

Un gerente pronosticó las ventas (miles de pesos) de 6 sucursales: reales `c(120, 95, 150, 80, 110, 130)` y pronosticadas `c(110, 100, 140, 90, 115, 120)`. Calcula MAE, RMSE, MAPE y R^2 escribiendo las fórmulas en R (sin paquetes). ¿Cuál es el error promedio en pesos y en porcentaje?

 Solución en la página 681**Ejercicio 7.3: Matriz de confusión e interpretación**Nivel: **Básico**

Un modelo de fraude se evaluó con 1,000 transacciones: detectó 40 de los 50 fraudes reales y marcó como fraude 60 transacciones legítimas. Construye la matriz de confusión como un `matrix()` de R y calcula exactitud, precisión, sensibilidad, especificidad y F1. Si cada fraude no detectado cuesta \$5,000 y cada revisión innecesaria \$100, ¿cuál es el costo de los errores?

 Solución en la página 682**Ejercicio 7.4: Segmentación RFM por reglas**Nivel: **Intermedio**

Con `transacciones.csv` calcula el RFM de cada cliente (fecha de corte 2024-01-01) y asigna calificaciones de 1 a 5 con `ntile()`. Calcula un puntaje total `r + f + m` y clasifica en “Oro” (13 a 15), “Plata” (9 a 12) y “Bronce” (3 a 8). Reporta, por nivel, número de clientes, monto promedio y porcentaje de los ingresos totales.

 Solución en la página 682**Ejercicio 7.5: k-means con otras variables**Nivel: **Intermedio**

Con `transacciones.csv`, construye por cliente: monto total, ticket promedio (`mean(total_transaccion)`), número de tiendas distintas y número de productos distintos. Escala las cuatro variables, elige k con la silueta (entre 2 y 6), entrena k-means con `nstart = 25` y nombra los grupos a partir de su perfil (sin escribir números de cluster a mano).

 Solución en la página 683**Ejercicio 7.6: Mejorar el modelo de churn con una variable nueva**Nivel: **Intermedio**

Con los datos simulados de churn (misma semilla y misma división de la sección 7.5), crea la variable `cliente_nuevo = antiguedad_meses <= 6`, agrégala a la regresión logística y compara el AUC en prueba con el del modelo original. Interpreta la razón de momios de la nueva variable. Quita también `genero` y `edad`: ¿cambia el AUC?

 Solución en la página 685

Ejercicio 7.7: Umbral con otros costosNivel: **Intermedio**

Con las probabilidades de la regresión logística en `churn_prueba`, encuentra el umbral que minimiza el costo total si el descuento cuesta \$800 (en lugar de \$300), un cliente perdido cuesta \$3,000 y la mitad de los clientes contactados que iban a irse se retiene. ¿Cuántos clientes se llamarían? Compara con el umbral óptimo del texto (0.15) y explica la diferencia.

 Solución en la página 687**Ejercicio 7.8: Pronóstico trimestral**Nivel: **Avanzado**

A partir de la serie mensual simulada de la sección 7.6, construye una serie *trimestral* (suma de cada trimestre, frecuencia 4). Deja como prueba los 4 trimestres de 2025, compara `snaive()`, `ets()` y `auto.arima()` con `accuracy()` y pronostica los 4 trimestres de 2026 con el mejor modelo, reentrenado con toda la serie.

 Solución en la página 688**Ejercicio 7.9: Canasta con reglas de tres productos**Nivel: **Avanzado**

Con los tickets simulados de la sección 7.7, calcula el soporte, la confianza y el lift de la regla {Laptop, Mouse} → {Funda} usando `dplyr`. Compárala con Laptop → Funda. ¿Saber que el cliente ya lleva mouse cambia la probabilidad de que lleve funda?

 Solución en la página 690**Ejercicio 7.10: Reto: sistema de puntuación de churn completo**Nivel: **Reto**

Simula (con `set.seed(2024)`) 3,000 clientes con las mismas variables y reglas de la sección 7.5. Entrena con `caret` una regresión logística y un random forest (validación cruzada de 5 pliegues, AUC como métrica), elige el mejor según el AUC en un 30 % de prueba, guárdalo con `saveRDS()`, escribe una función `puntuar()` que devuelva los 100 clientes con mayor pérdida esperada (probabilidad × cuota anual) de un lote nuevo de 1,000 clientes y exporta la lista a `resultados/top100_churn.csv`.

 Solución en la página 691

Reportes automáticos con R Markdown

Al terminar este módulo podrás...

- Explicar por qué un **reporte reproducible** ahorra horas y evita errores frente al reporte hecho a mano en Excel o PowerPoint.
- Crear un documento R Markdown (`.Rmd`) con encabezado YAML, texto en Markdown, bloques de código y código en línea, y convertirlo a HTML, PDF y Word con el botón **Knit**.
- Controlar qué se ve en el reporte con las opciones de los bloques (`echo`, `include`, `fig.cap`, `results = "asis"`, etc.).
- Presentar tablas profesionales con `knitr::kable()` y escribir **texto dinámico** que cambia solo según los datos.
- Construir **reportes parametrizados** (uno por tienda, por mes, por región) y generarlos todos con un script que registra lo que salió bien y lo que falló.
- Programar la ejecución automática (cron, Programador de tareas de Windows), enviar los reportes por correo y exportar reportes a Excel con varias hojas.
- Dar el salto a Quarto y aplicar buenas prácticas de reproducibilidad.

Es el primer lunes del mes. Andrea, analista de la cadena TechRetail México, llega a la oficina sabiendo lo que le espera: abrir el sistema, exportar las ventas del mes a Excel, armar diez tablas dinámicas (una por sucursal), copiar los totales a una presentación de PowerPoint, pegar las gráficas, escribir a mano frases como “las ventas crecieron 12 % respecto al mes anterior” y, por último, enviar diez correos a diez gerentes. Le toma casi dos días. Y el mes pasado, al copiar y pegar, el gerente de la Sucursal Norte recibió las cifras de la Sucursal Sur. Nadie lo notó hasta la junta de resultados.

Ahora imagina otra versión de ese lunes: Andrea llega, abre su correo y ya tiene la confirmación de que a las 7:00 de la mañana un script de R generó los diez reportes (con tablas, gráficas y conclusiones escritas en español a partir de los datos) y se los envió a cada gerente. Andrea dedica su mañana a lo que de verdad aporta valor: investigar por qué la Sucursal Centro cayó tanto en diciembre.

Esa segunda versión es lo que vas a construir en este módulo. Usarás todo lo que aprendiste de manipulación de datos (Módulo 2) y de visualización (Módulo 3), y le agregarás una pieza nueva: **R Markdown**, la herramienta que convierte tu análisis en un documento terminado que se vuelve a generar con un clic, o sin ningún clic.

8.1 El problema: el reporte mensual hecho a mano

Casi todas las empresas tienen un reporte que se hace “cada mes, igual que el mes pasado”: el reporte de ventas, el de cartera vencida, el de inventario, el de rotación de personal. Cuando se hace a mano, el proceso suele verse así:

1. Exportar datos de un sistema a Excel.
2. Limpiar, filtrar y resumir con fórmulas o tablas dinámicas.
3. Copiar los resultados a Word o PowerPoint.
4. Rehacer las gráficas o pegarlas como imagen.
5. Escribir los comentarios (“crecimos”, “caímos”, “destaca”).
6. Revisar, exportar a PDF y enviar.

Cada paso manual es una oportunidad de error: una fila que no se seleccionó, una fórmula que apunta a la celda equivocada, una gráfica del mes pasado que nadie actualizó, un porcentaje escrito a mano que ya no corresponde a la tabla. Y el costo en tiempo se multiplica: si el reporte es por tienda, son diez veces el mismo trabajo.

Tabla 8.1. Reporte manual frente a reporte reproducible

Aspecto	Manual (Excel + PowerPoint)	Reproducible (R Markdown)
Tiempo por reporte	Horas, cada mes	Minutos la primera vez; segundos después
Errores de copiar y pegar	Frecuentes y difíciles de detectar	No hay copiar y pegar: los números salen del código
Diez versiones (una por tienda)	Diez veces el trabajo	Un bucle de diez vueltas
Texto de conclusiones	Se escribe a mano	Se genera con reglas (<code>if/else</code>)
Auditoría (“¿de dónde salió este número?”)	Difícil	El código lo documenta
Cambiar el formato (PDF, Word, HTML)	Rehacer	Cambiar una línea

Concepto clave: Reporte reproducible

Un **reporte reproducible** es un documento que se genera *completamente* a partir de los datos y de un código, sin pasos manuales. Si los datos cambian (llega un mes nuevo, se corrige una venta), basta con volver a generar el documento para que todos los números, tablas, gráficas y frases se actualicen. Cualquier persona con los mismos datos y el mismo código obtiene exactamente el mismo reporte.

8.1.1 Qué es R Markdown y cómo funciona

R Markdown es un formato de archivo (extensión `.Rmd`) que mezcla tres cosas en un solo documento:

- **Texto** escrito en **Markdown**, un lenguaje muy sencillo para dar formato (títulos, negritas, listas) con símbolos como `#` o `**`.
- **Código R** en bloques llamados **chunks**, cuyos resultados (tablas, gráficas, números) se insertan en el documento.
- Un **encabezado** en formato **YAML** con el título, el autor y el formato de salida.

Cuando presionas el botón **Knit** (“tejer”) en RStudio, ocurren dos pasos, que resume la figura 8.1. Primero, el paquete **knitr** ejecuta cada bloque de código y “teje” sus resultados con el texto, generando un archivo Markdown puro (`.md`). Después, **pandoc**, un convertidor universal de documentos que viene incluido con RStudio, transforma ese `.md` en el formato final: una página HTML, un documento de Word o, pasando por LaTeX, un PDF. El paquete **rmarkdown** coordina todo el proceso con su función `render()`.

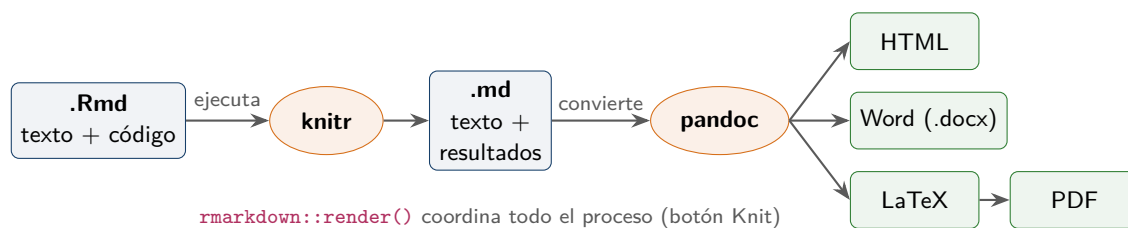


Figura 8.1. Cómo se convierte un archivo R Markdown en un reporte terminado.

Lo importante de este flujo es que **el reporte nunca se edita a mano**: si algo está mal, corriges el `.Rmd` (o los datos) y vuelves a tejer. El documento final es un producto desechable; lo valioso es el `.Rmd`.

Para que veas cómo trabaja **knitr** sin salir de la consola, puedes pedirle que teja un pequeño texto con `knitr::knit(text = ...)`. Cada elemento del vector es una línea del documento:

```

1 library(tidyverse) # dplyr, readr, ggplot2, tidyr, purrr...
2 library(knitr)      # knit() y kable()
3 library(scales)     # dollar(), percent(), comma()
4
5 # Un mini documento R Markdown guardado en un vector de texto
6 rmd_mini <- c(
7   "Este año vendimos `r format(1234567, big.mark = ',')` pesos.",
8   "",
9   "`````{r suma}`,",
10  "sum(1:10)",
11  "`````",
12 )
13 # knitr ejecuta el código y devuelve Markdown puro (lo que ve pandoc)
14 cat(knit(text = rmd_mini, quiet = TRUE))

```

```
Este año vendimos 1,234,567 pesos.
```

```

```r
sum(1:10)
```

```
[1] 55
```

```

Observa lo que pasó: la expresión `'r format(...)'` de la primera línea desapareció y en su lugar quedó el número ya formateado (`1,234,567`). El bloque de código se conservó (para que se vea en el reporte) y debajo apareció su resultado, precedido de `##`. Eso es exactamente lo que knitr hace con un reporte completo; después pandoc le da formato bonito.

Consejo

Para seguir este módulo necesitas los paquetes **rmarkdown** y **knitr**. RStudio los instala automáticamente la primera vez que creas un archivo R Markdown, pero también puedes instalarlos tú. Para los PDF necesitarás además LaTeX (sección 8.6).

```
1 install.packages(c("rmarkdown", "knitr", "rmarkdown", "readxl"))
```

8.2 Anatomía de un documento R Markdown

8.2.1 Crear tu primer .Rmd en RStudio

1. Ve a **File > New File > R Markdown...**
2. Escribe un título (por ejemplo, “Mi primer reporte de ventas”) y tu nombre, deja seleccionado **HTML** y presiona **OK**.
3. RStudio abre un documento de ejemplo. Guárdalo con **Ctrl** + **S** dentro de la carpeta del curso (la que contiene `datasets/`), por ejemplo como `reportes/01-primer-reporte.Rmd`.
4. Presiona el botón **Knit** (el ovillo de estambre azul) o el atajo **Ctrl** + **Shift** + **K**. En unos segundos se abre la vista previa del HTML y el archivo `01-primer-reporte.html` aparece junto al `.Rmd`.

En la carpeta `codigo/reportes/` del libro encontrarás los archivos completos de este módulo, listos para copiar a tu carpeta `reportes/`: `01-primer-reporte.Rmd`, `02-reporte-ejecutivo.Rmd`, `generar_reportes.R` y `reporte_excel.R`. La estructura de carpetas que usaremos es:

```

1 curso-bi-rstudio/      <- directorio de trabajo (proyecto RStudio)
2 |-- datasets/          <- datos de entrada (CSV)
3 |   |-- transacciones.csv

```

```

4 | |-- tiendas.csv
5 | `-- ...
6 |-- reportes/                <- plantillas .Rmd y scripts
7 | |-- 01-primer-reporte.Rmd
8 | |-- 02-reporte-ejecutivo.Rmd
9 | |-- generar_reportes.R
10 | `-- reporte_excel.R
11 |-- resultados/             <- salidas (se pueden borrar y regenerar)
12 | |-- reporte_ventas.xlsx
13 | `-- reportes/
14 |     |-- 2023-12_T01_sucursal-centro_2024-01-05.html
15 |     `-- ...

```

Un archivo `.Rmd` tiene tres tipos de contenido, que muestra la figura 8.2. Vamos a recorrer el archivo `01-primer-reporte.Rmd` por partes.

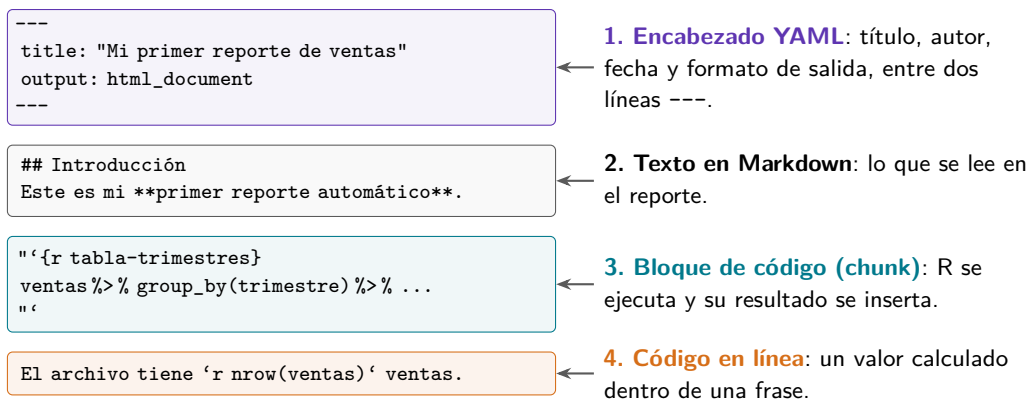


Figura 8.2. Las cuatro piezas de un documento R Markdown.

8.2.2 El encabezado YAML

El documento empieza con un bloque entre dos líneas de tres guiones. Es el **encabezado YAML** (*YAML Ain't Markup Language*), un formato de configuración del tipo **clave: valor**:

```

1  ---
2  title: "Mi primer reporte de ventas"
3  author: "Tu nombre"
4  date: "`r format(Sys.Date(), '%d/%m/%Y')`"
5  output: html_document
6  ---

```

- **title**, **author** y **date** aparecen en la portada del reporte. Observa que la fecha es código R: cada vez que tejes, se escribe la fecha de ese día.
- **output** indica el formato: `html_document`, `pdf_document` o `word_document`.
- Los textos con caracteres especiales (dos puntos, comillas, acentos al inicio) van entre comillas.

▲ Cuidado

En YAML **la sangría importa**: las opciones de un formato se escriben debajo de él con dos espacios de sangría, y nunca con tabuladores. Un espacio de más o de menos rompe el documento (sección 8.13).

8.2.3 Texto en Markdown

Después del encabezado escribes el texto del reporte. Markdown usa símbolos sencillos para dar formato; la tabla 8.2 resume los que usarás el 95 % del tiempo.

Tabla 8.2. Sintaxis básica de Markdown

| Quieres... | Escribes | Notas |
|-------------------------|--------------------------------------|--|
| Título de nivel 1, 2, 3 | # Título, ## Sección, ### Subsección | Deja un espacio después de # y una línea en blanco antes |
| Negritas | **texto** | También <code>__texto__</code> |
| Cursivas | <i>*texto*</i> | También <code>_texto_</code> |
| Código en línea | <code>'read_csv()'</code> | Letra de máquina de escribir |
| Lista con viñetas | - elemento | Una línea en blanco antes de la lista |
| Lista numerada | 1. elemento | La numeración se ajusta sola |
| Enlace | [texto](https://...) | |
| Imagen | ![pie](logo.png) | Ruta relativa al .Rmd |
| Línea horizontal | --- | En una línea sola |
| Tabla | A B y -- | Mejor generarlas con <code>kable()</code> |
| Salto de párrafo | Una línea en blanco | Un solo salto de línea no separa párrafos |

Un ejemplo de texto con varios de estos elementos:

```

1  ## Introducción
2
3  Este es mi primer reporte automático. Analiza las ventas de 2023 de la
4  cadena TechRetail México. Puntos clave:
5
6  - Los datos vienen de `ventas_retail.csv`.
7  - Se actualiza cada vez que presiono Knit.
8
9  | Trimestre | Meta          |
10 |-----|-----:|
11 | Q1        | $1,000,000 |

```

8.2.4 Bloques de código (chunks)

Un **chunk** empieza con tres acentos graves y `{r}`, y termina con tres acentos graves. Opcionalmente lleva un nombre (**cargar-datos**) y opciones separadas por comas. En RStudio lo insertas con **Ctrl** + **Alt** + **I** y lo ejecutas, sin tejer, con la flechita verde de su esquina.

```
1  ```{r cargar-datos}
2  library(dplyr)
3  library(readr)
4  library(ggplot2)
5
6  # El .Rmd se ejecuta en SU carpeta. Buscamos datasets/ ahí mismo
7  # o en la carpeta superior (si el .Rmd está en curso/reportes/).
8  carpeta <- if (dir.exists("datasets")) "datasets" else "../datasets"
9  ventas <- read_csv(file.path(carpetas, "ventas_retail.csv"),
10                    show_col_types = FALSE)
11  ```
```

Fíjate en la búsqueda de la carpeta de datos. Cuando tejes, R usa como directorio de trabajo **la carpeta donde está guardado el .Rmd**, no la de tu proyecto. Si el .Rmd vive en `reportes/`, la ruta `datasets/ventas_retail.csv` no existe desde ahí: hay que subir un nivel (`../datasets`). Volveremos a esto en la sección 8.7.4, porque es el error número uno de quien empieza con R Markdown.

8.2.5 Código en línea

Para meter un número calculado *dentro de una frase* se usa el **código en línea**: un acento grave, la letra **r**, un espacio, la expresión y otro acento grave.

```
1  El archivo tiene `r nrow(ventas)` ventas y el total vendido en el año
2  fue de **$`r format(round(sum(ventas$total)), big.mark = ",")`**.
```

Antes de ponerlo en el reporte, conviene probar la expresión en la consola. Carguemos los datos igual que lo hace el reporte (con el directorio de trabajo en la carpeta del curso):

```
1  ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
2
3  nrow(ventas)                                # número de ventas
4  format(round(sum(ventas$total)), big.mark = ",") # total con comas
```

```
[1] 365
[1] "513,917"
```

En el reporte, la frase se leerá: “El archivo tiene 365 ventas y el total vendido en el año fue de

\$513,917". Si mañana llegan más datos, la frase se corrige sola.

8.2.6 El documento completo y el botón Knit

Juntando las piezas, este es el resto de `01-primer-reporte.Rmd` (después del encabezado, el texto y el bloque de carga):

```

1  ```{r setup, include=FALSE}
2  # Opciones para todos los bloques: mostrar código, ocultar mensajes
3  knitr::opts_chunk$set(echo = TRUE, message = FALSE, warning = FALSE,
4                        fig.width = 7, fig.height = 3.5)
5  ```
6
7  ## Ventas por trimestre
8
9  ```{r tabla-trimestres}
10 ventas %>%
11   group_by(trimestre) %>%
12   summarise(ventas = round(sum(total)), n_ventas = n()) %>%
13   knitr::kable(col.names = c("Trimestre", "Ventas ($)", "Núm. de ventas"),
14               format.args = list(big.mark = ","))
15 ```
16
17 ## Tendencia mensual
18
19 ```{r grafica-mensual, fig.cap = "Ventas totales por mes, 2023"}
20 ventas %>%
21   group_by(mes) %>%
22   summarise(ventas = sum(total)) %>%
23   ggplot(aes(x = mes, y = ventas, group = 1)) +
24   geom_line(color = "#2a78d6", linewidth = 0.8) +
25   geom_point(color = "#2a78d6") +
26   scale_y_continuous(labels = scales::dollar) +
27   labs(x = NULL, y = "Ventas") +
28   theme_minimal() +
29   theme(panel.grid.minor = element_blank(),
30         axis.text.x = element_text(angle = 45, hjust = 1))
31 ```

```

Al presionar **Knit**, en el panel **Render** de RStudio verás cómo knitr procesa cada bloque (1/9, 2/9...) y al final la línea **Output created: 01-primer-reporte.html**. El HTML resultante es un solo archivo autocontenido (las gráficas van incrustadas): lo puedes enviar por correo y se abre en cualquier navegador, sin necesidad de R.

Hazlo tú

Crea `reportes/01-primer-reporte.Rmd` con el contenido anterior y tejelo. Después agrega una sección `## Ventas por día de la semana` con una tabla que use `dia_semana`, y vuelve a tejer. ¿Cuánto tardó en actualizarse todo el documento?

8.3 Opciones de los bloques de código

Un reporte para el director no debe mostrar código, mensajes de carga de paquetes ni advertencias; un tutorial para el equipo de análisis sí debe mostrar el código. Eso se controla con las **opciones de chunk**, que se escriben dentro de las llaves después del nombre:

```
1 ```{r grafica-ventas, echo = FALSE, fig.cap = "Ventas por mes",
2   fig.width = 7, fig.height = 3.5, out.width = "90%"}
```

La tabla 8.3 reúne las opciones que más usarás.

Tabla 8.3. Opciones de chunk más usadas en reportes de negocio

| Opción | Valor típico | Qué hace |
|------------------------------------|--------------------|--|
| <code>echo</code> | <code>FALSE</code> | Oculto el código; muestra el resultado |
| <code>eval</code> | <code>FALSE</code> | Muestra el código pero no lo ejecuta |
| <code>include</code> | <code>FALSE</code> | Ejecuta el código pero no muestra nada (ni código ni resultado). Ideal para cargar datos |
| <code>message</code> | <code>FALSE</code> | Oculto mensajes (p. ej., al cargar paquetes) |
| <code>warning</code> | <code>FALSE</code> | Oculto advertencias |
| <code>error</code> | <code>TRUE</code> | Muestra el error y sigue tejiendo (por defecto, un error detiene todo) |
| <code>fig.cap</code> | "Título" | Pie de la figura (acepta una variable de R) |
| <code>fig.width, fig.height</code> | 7, 3.5 | Tamaño de la gráfica en pulgadas |
| <code>out.width</code> | "90%" | Ancho con el que se muestra en el documento |
| <code>fig.align</code> | "center" | Alineación de la figura |
| <code>results</code> | "asis" | El texto que imprime el código se trata como Markdown, no como salida de consola |
| <code>results</code> | "hide" | Oculto la salida impresa |
| <code>cache</code> | <code>TRUE</code> | Guarda el resultado y no vuelve a ejecutar el bloque si su código no cambió |

Veamos la diferencia entre `echo = FALSE` e `include = FALSE` tejiendo un pequeño documento en la consola:

```
1 doc_opciones <- c(
2   "```{r a, echo = FALSE}",
3   "sum(1:10)",
4   "```",
```

```

5   "",
6   "`r b, include = FALSE}",
7   "total <- sum(1:10)",
8   "`r",
9   "",
10  "El total es `r total`."
11 )
12 cat(knit(text = doc_opciones, quiet = TRUE))

```

```

```
[1] 55
```

El total es 55.

```

Con `echo = FALSE` el código desapareció pero quedó su resultado (`## [1] 55`). Con `include = FALSE` no quedó nada visible, pero el objeto `total` sí se creó, y por eso la frase final puede usarlo. Ese es el patrón de un reporte ejecutivo: bloques invisibles que calculan, y texto que muestra.

8.3.1 El chunk de configuración global

Repetir `echo = FALSE`, `message = FALSE` en cada bloque es tedioso. Lo habitual es un primer bloque, llamado `setup` o *configuración*, con `include = FALSE`, que fija las opciones *por defecto* para todo el documento con `knitr::opts_chunk$set()`:

```

1  ```{r configuracion, include=FALSE}
2  knitr::opts_chunk$set(
3    echo = FALSE,      # un reporte ejecutivo NO muestra código
4    message = FALSE,   # sin mensajes de carga de paquetes
5    warning = FALSE,   # sin advertencias
6    fig.width = 7, fig.height = 3.4, out.width = "90%",
7    fig.align = "center"
8  )
9  library(dplyr)
10 library(ggplot2)
11 ```

```

Cualquier bloque puede cambiar una opción para sí mismo: si en un reporte con `echo = FALSE` quieres mostrar un bloque, le pones `echo = TRUE` solo a ese.

8.3.2 `results = "asis"`: cuando el código escribe el reporte

Por defecto, lo que tu código imprime con `cat()` o `print()` aparece en un recuadro gris, como salida de consola. Con `results = "asis"` (“tal cual”), ese texto se inserta como parte del documento, así que puedes *generar Markdown desde R*: listas, títulos o párrafos que dependen de los datos.

```
1 doc_asis <- c(
2   "```{r lista, echo = FALSE, results = 'asis'}",
3   "tiendas_top <- c('Sucursal Outlet', 'Sucursal Sur')",
4   "cat(paste0('- **', tiendas_top, '**'), sep = '\\n')",
5   "```"
6 )
7 cat(knit(text = doc_asis, quiet = TRUE))
```

```
- **Sucursal Outlet**
- **Sucursal Sur**
```

El resultado ya no lleva `##` ni recuadro: es una lista de Markdown que pandoc mostrará con viñetas y negritas. En el caso práctico usaremos esta técnica para escribir las conclusiones automáticas del reporte.

8.3.3 `cache`: no recalcular lo que no cambió

Si un bloque tarda mucho (leer millones de filas, entrenar un modelo del Módulo 7), `cache = TRUE` guarda su resultado en una carpeta `*_cache/` y en el siguiente Knit lo reutiliza si el código del bloque no cambió. Úsalo con cuidado: si cambian los *datos* pero no el código, knitr no se entera y te mostrará resultados viejos. En reportes mensuales automáticos, donde lo que cambia son precisamente los datos, lo más seguro es no usar caché en los bloques que leen datos.

Hazlo tú

En tu `01-primer-reporte.Rmd` cambia el bloque `setup` para que `echo = FALSE` y vuelve a tejer. Luego agrega `fig.cap` con un título que incluya el año y cambia `out.width` a `"70%"`. Compara ambas versiones.

8.4 Tablas profesionales con `knitr::kable()`

Si imprimes un tibble en un reporte, se ve como salida de consola: letra de máquina, `<dbl>` y `<chr>` en los encabezados, números sin separador de miles. La función `kable()` del paquete **knitr** convierte un data frame en una tabla con formato de documento (HTML, LaTeX o Word, según la salida).

Ejemplo práctico: Ventas de diciembre por tienda

El director quiere una tabla con las ventas de diciembre de 2023 de cada tienda: nombre, ciudad, número de transacciones, ventas y ticket promedio, ordenada de mayor a menor. Preparamos los datos con **dplyr** y después le damos formato con **kable()**.

```
1 # Datos que usaremos en el resto del módulo
2 transacciones <- read_csv("datasets/transacciones.csv",
3                             show_col_types = FALSE)
4 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
5 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
6
7 # Resumen de diciembre por tienda
8 ventas_dic <- transacciones %>%
9   filter(mes == "2023-12") %>%
10  group_by(tienda_id) %>%
11  summarise(transacciones = n(),
12            ventas = sum(total_transaccion),
13            ticket = mean(total_transaccion), .groups = "drop") %>%
14  left_join(tiendas %>% select(tienda_id, nombre_tienda, ciudad),
15            by = "tienda_id") %>%
16  select(nombre_tienda, ciudad, transacciones, ventas, ticket) %>%
17  arrange(desc(ventas))
18
19 ventas_dic
```

```
# A tibble: 10 x 5
  nombre_tienda ciudad transacciones ventas ticket
  <chr>         <chr>          <int>  <dbl>  <dbl>
1 Sucursal Outlet Tijuana             12 16554.  1379.
2 Sucursal Sur Monterrey             8 13317.  1665.
3 Sucursal Express Querétaro          10 11194.  1119.
4 Sucursal Plaza A CDMX                9 10945.  1216.
5 Sucursal Norte Guadalajara          15 10049.   670.
6 Sucursal Este CDMX                 11 9983.   908.
7 Sucursal Plaza B Guadalajara          11 9928.   903.
8 Sucursal Centro CDMX                 5 5020.  1004.
9 Sucursal Premium CDMX                 5 4916.   983.
10 Sucursal Oeste Puebla                5 4488.   898.
```

Así se ve en la consola: útil para ti, pero no para un director. Ahora con **kable()**. Usamos **format = "pipe"** para que veas en el libro la tabla Markdown que genera; dentro de un **.Rmd** no hace falta ponerlo, porque knitr elige el formato adecuado para HTML, PDF o Word.

```
1 kable(ventas_dic,
2       format = "pipe", # Markdown (para verla aquí)
```

```

3   col.names = c("Tienda", "Ciudad", "Transacciones",
4                 "Ventas ($)", "Ticket ($)", # encabezados legibles
5   digits = 0, # sin decimales
6   format.args = list(big.mark = ","), # separador de miles
7   align = c("l", "l", "r", "r", "r"), # texto izq., números der.
8   caption = "Ventas de diciembre de 2023 por tienda")

```

Table: Ventas de diciembre de 2023 por tienda

| Tienda | Ciudad | Transacciones | Ventas (\$) | Ticket (\$) |
|------------------|-------------|---------------|-------------|-------------|
| Sucursal Outlet | Tijuana | 12 | 16,554 | 1,379 |
| Sucursal Sur | Monterrey | 8 | 13,317 | 1,665 |
| Sucursal Express | Querétaro | 10 | 11,194 | 1,119 |
| Sucursal Plaza A | CDMX | 9 | 10,945 | 1,216 |
| Sucursal Norte | Guadalajara | 15 | 10,049 | 670 |
| Sucursal Este | CDMX | 11 | 9,983 | 908 |
| Sucursal Plaza B | Guadalajara | 11 | 9,928 | 903 |
| Sucursal Centro | CDMX | 5 | 5,020 | 1,004 |
| Sucursal Premium | CDMX | 5 | 4,916 | 983 |
| Sucursal Oeste | Puebla | 5 | 4,488 | 898 |

Cada argumento resuelve un problema de presentación:

- `col.names` cambia los nombres técnicos (`nombre_tienda`) por nombres para humanos.
- `digits` redondea todas las columnas numéricas y `format.args = list(big.mark = ",")` agrega separador de miles.
- `align` alinea: "l" izquierda para texto, "r" derecha para números (así se comparan las cifras por su magnitud), "c" centro.
- `caption` pone el título numerado ("Tabla 1: ...").

Los dos puntos en la segunda línea de la tabla Markdown (`|---:|`) son los que indican a pandoc la alineación a la derecha.

8.4.1 Formato de negocio antes de la tabla

`kable()` no sabe poner el signo de pesos ni porcentajes. Cuando quieras ese formato, conviértelo tú antes con las funciones de **scales** (las mismas que usaste en los ejes de ggplot2 en el Módulo 3). Ojo: al formatear, la columna se vuelve texto, así que ordena *antes* de formatear.

```

1  tabla_dic <- ventas_dic %>%
2    mutate(participacion = ventas / sum(ventas), # % del total
3           ventas        = dollar(ventas, accuracy = 1),

```

```

4      ticket      = dollar(ticket, accuracy = 0.01),
5      participacion = percent(participacion, accuracy = 0.1)) %>%
6      select(-ciudad) %>%
7      slice_head(n = 5)                                # top 5
8
9      kable(tabla_dic, format = "pipe", align = c("l", "r", "r", "r", "r"),
10           col.names = c("Tienda", "Transacciones", "Ventas", "Ticket",
11                          "% del total"))

```

| Tienda | Transacciones | Ventas | Ticket | % del total |
|------------------|---------------|----------|------------|-------------|
| Sucursal Outlet | 12 | \$16,554 | \$1,379.48 | 17.2% |
| Sucursal Sur | 8 | \$13,317 | \$1,664.67 | 13.8% |
| Sucursal Express | 10 | \$11,194 | \$1,119.44 | 11.6% |
| Sucursal Plaza A | 9 | \$10,945 | \$1,216.07 | 11.4% |
| Sucursal Norte | 15 | \$10,049 | \$669.96 | 10.4% |

La tabla dice que en diciembre la Sucursal Outlet fue la que más vendió (\$16,554, el 17.2 % de la empresa) y que las cinco primeras tiendas tienen tickets promedio muy distintos: la Sucursal Sur logró \$13,317 con solo 8 transacciones (ticket de \$1,664.67), mientras la Sucursal Norte necesitó 15 transacciones para vender \$10,049. Esa es la clase de observación que luego puedes convertir en texto dinámico.

8.4.2 Más allá de kable: kableExtra y gt

Para tablas con colores, filas resaltadas, encabezados agrupados o notas al pie, existen dos paquetes muy usados (no los necesitas para este módulo):

```

1  # kableExtra: añade estilo a kable() (HTML y PDF)
2  install.packages("kableExtra")
3  library(kableExtra)
4  kable(tabla_dic) %>%
5    kable_styling(bootstrap_options = c("striped", "hover")) %>%
6    row_spec(1, bold = TRUE, background = "#fde6dc") # resalta la 1a fila
7
8  # gt: "gramática de tablas" de Posit, muy flexible
9  install.packages("gt")
10 library(gt)
11 ventas_dic %>%
12   gt() %>%
13   fmt_currency(columns = c(ventas, ticket)) %>%
14   tab_header(title = "Ventas de diciembre de 2023")

```

 Hazlo tú

Haz una tabla con `kable()` de las ventas totales de 2023 por `metodo_pago` (de `transacciones.csv`), con las ventas en formato de pesos, el porcentaje del total y el título “Ventas por método de pago”.

8.5 Texto dinámico: frases que se escriben solas

Un reporte ejecutivo no es solo tablas: es *narrativa*. “Las ventas crecieron 19.7 % respecto a noviembre”. Si esa frase se escribe a mano, el mes siguiente puede quedarse con el número viejo, o peor, decir “crecieron” cuando en realidad cayeron. El **texto dinámico** es texto que R construye a partir de los datos: el número *y* la palabra (“crecieron”/“cayeron”) salen del cálculo.

8.5.1 Formatear números para leerlos en una frase

```
1 x <- 96393.537
2 dollar(x)                                # pesos, con separador de miles
3 dollar(x, accuracy = 1)                  # sin centavos
4 dollar(x, scale = 1e-3, suffix = " mil", accuracy = 0.1)
5 comma(1234567)                          # separador de miles
6 percent(0.19731, accuracy = 0.1)
7 format(x, big.mark = ",", nsmall = 2)    # con base R
8 format(as.Date("2023-12-15"), "%d de %B de %Y") # fecha en texto
```

```
[1] "$96,393.54"
[1] "$96,394"
[1] "$96.4 mil"
[1] "1,234,567"
[1] "19.7%"
[1] "96,393.54"
[1] "15 de diciembre de 2023"
```

La última línea usa el idioma de tu computadora para el nombre del mes. Si tu Windows está en inglés, diría **December**. Para que el reporte salga siempre en español, en el caso práctico usaremos un vector propio con los nombres de los meses.

8.5.2 Frases que cambian según los datos

Ejemplo práctico: La frase de crecimiento mensual

Queremos una función que reciba las ventas del mes actual y del anterior y devuelva una frase correcta en cualquier caso: si crecieron, si cayeron, si se quedaron igual o si no hay dato del mes anterior.

```

1 frase_variacion <- function(actual, anterior, periodo = "el mes anterior") {
2   # Caso 1: no hay con qué comparar
3   if (is.na(anterior) || anterior == 0) {
4     return(paste0("Las ventas fueron de ", dollar(actual, accuracy = 1),
5                  "; no hay datos para comparar con ", periodo, "."))
6   }
7   var <- actual / anterior - 1 # variación relativa
8   # Caso 2, 3 y 4: la palabra depende del signo
9   verbo <- if (abs(var) < 0.005) {
10    "se mantuvieron sin cambio"
11  } else if (var > 0) {
12    paste("crecieron", percent(var, accuracy = 0.1))
13  } else {
14    paste("cayeron", percent(abs(var), accuracy = 0.1))
15  }
16  paste0("Las ventas ", verbo, " en comparación con ", periodo, " (de ",
17        dollar(anterior, accuracy = 1), " a ",
18        dollar(actual, accuracy = 1), ").")
19 }
20
21 # Probamos los cuatro casos
22 frase_variacion(12000, 10000)
23 frase_variacion(8000, 10000)
24 frase_variacion(10020, 10000)
25 frase_variacion(5000, NA)

```

```

[1] "Las ventas crecieron 20.0% en comparación con el mes anterior (de $10,000 a $12
↪ ,000)."
```

```

[1] "Las ventas cayeron 20.0% en comparación con el mes anterior (de $10,000 a $8,000)
↪ ."
```

```

[1] "Las ventas se mantuvieron sin cambio en comparación con el mes anterior (de $10
↪ ,000 a $10,020)."
```

```

[1] "Las ventas fueron de $5,000; no hay datos para comparar con el mes anterior."
```

Fíjate en el tercer caso: una variación de 0.2 % se reporta como “sin cambio”, porque decir “crecieron 0.2 %” exagera algo que es ruido. Estas reglas de redacción son decisiones de negocio que conviene acordar con quien lee el reporte.

Ahora apliquemos la función a los datos reales: ventas totales de la empresa en diciembre contra

noviembre.

```
1 ventas_mes <- transacciones %>%
2   group_by(mes) %>%
3   summarise(ventas = sum(total_transaccion), .groups = "drop")
4
5 actual <- ventas_mes$ventas[ventas_mes$mes == "2023-12"]
6 anterior <- ventas_mes$ventas[ventas_mes$mes == "2023-11"]
7 frase_variacion(actual, anterior, periodo = "noviembre")
```

```
[1] "Las ventas crecieron 19.7% en comparación con noviembre (de $80,507 a $96,394)."
```

En el .Rmd, esta frase se inserta con código en línea:

```
1 ## Resumen
2
3 `r frase_variacion(actual, anterior, periodo = "noviembre")`
```

8.5.3 Una frase para cada tienda

Como la función trabaja con dos números, podemos aplicarla a las diez tiendas con `map2_chr()` de **purrr** (que recorre dos vectores a la vez y devuelve texto):

```
1 comparacion <- transacciones %>%
2   filter(mes %in% c("2023-11", "2023-12")) %>%
3   group_by(tienda_id, mes) %>%
4   summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
5   pivot_wider(names_from = mes, values_from = ventas) %>%
6   rename(nov = `2023-11`, dic = `2023-12`) %>%
7   left_join(tiendas %>% select(tienda_id, nombre_tienda),
8             by = "tienda_id") %>%
9   mutate(frase = map2_chr(dic, nov, frase_variacion,
10                          periodo = "noviembre"))
11
12 # Imprimimos las frases de las 3 primeras tiendas
13 cat(paste0(comparacion$nombre_tienda[1:3], ": ",
14            comparacion$frase[1:3]), sep = "\n")
```

```
Sucursal Centro: Las ventas cayeron 52.6% en comparación con noviembre (de $10,593 a
↪ $5,020).
Sucursal Norte: Las ventas crecieron 25.8% en comparación con noviembre (de $7,990 a
↪ $10,049).
Sucursal Sur: Las ventas crecieron 97.4% en comparación con noviembre (de $6,746 a
↪ $13,317).
```

Tres tiendas, tres historias distintas, cero frases escritas a mano. La Sucursal Centro cayó a menos de la mitad y la Sucursal Sur casi duplicó sus ventas. Pero cuidado con la interpretación: cada tienda tiene solo entre 3 y 16 transacciones al mes en estos datos, así que variaciones tan grandes pueden deberse al azar. Un buen reporte automático también advierte eso (lo haremos en las conclusiones del caso práctico).

8.5.4 Conclusiones automáticas con reglas

Un paso más allá de la frase es la **conclusión automática**: una regla de negocio que decide qué mensaje dar. Por ejemplo: si la variación es mayor a 10 %, “crecimiento sólido”; si cae más de 10 %, “alerta”. Con `case_when()` lo aplicamos a todas las tiendas:

```
1 alertas <- comparacion %>%
2   mutate(variacion = dic / nov - 1,
3          estado = case_when(
4            variacion > 0.10 ~ "Crecimiento sólido",
5            variacion >= 0   ~ "Estable",
6            variacion > -0.10 ~ "Ligera caída",
7            TRUE             ~ "Alerta"
8          )) %>%
9   arrange(variacion) %>%
10  select(nombre_tienda, variacion, estado)
11
12 alertas %>%
13   mutate(variacion = percent(variacion, accuracy = 0.1)) %>%
14   kable(format = "pipe", col.names = c("Tienda", "Variación", "Estado"))
```

| Tienda | Variación | Estado |
|------------------|-----------|--------------------|
| Sucursal Centro | -52.6% | Alerta |
| Sucursal Oeste | -41.6% | Alerta |
| Sucursal Premium | -37.3% | Alerta |
| Sucursal Outlet | -13.5% | Alerta |
| Sucursal Norte | 25.8% | Crecimiento sólido |
| Sucursal Express | 34.2% | Crecimiento sólido |
| Sucursal Plaza B | 84.2% | Crecimiento sólido |
| Sucursal Sur | 97.4% | Crecimiento sólido |
| Sucursal Plaza A | 170.1% | Crecimiento sólido |
| Sucursal Este | 264.4% | Crecimiento sólido |

Y con `results = "asis"`, una lista de Markdown para el director que solo menciona las tiendas en alerta:

```
1 en_alerta <- alertas %>% filter(estado == "Alerta")
2
```



```

3 if (nrow(en_alerta) == 0) {
4   cat("- Ninguna tienda cayó más de 10 % este mes.\n")
5 } else {
6   cat(paste0("- **", en_alerta$nombre_tienda, "**: cayó ",
7             percent(abs(en_alerta$variacion), accuracy = 0.1),
8             " contra noviembre."), sep = "\n")
9 }

```

```

- **Sucursal Centro**: cayó 52.6% contra noviembre.
- **Sucursal Oeste**: cayó 41.6% contra noviembre.
- **Sucursal Premium**: cayó 37.3% contra noviembre.
- **Sucursal Outlet**: cayó 13.5% contra noviembre.

```

En la empresa: El reporte que avisa solo

En muchas empresas el reporte mensual es un documento de 30 páginas que nadie lee completo. Un reporte con conclusiones automáticas pone arriba, en tres líneas, lo que requiere atención (“3 tiendas en alerta”) y deja el detalle para quien lo necesite. El analista deja de “escribir el reporte” y pasa a *diseñar las reglas* con las que se escribe, que es un trabajo mucho más valioso.

Hazlo tú

Modifica `frase_variacion()` para que, cuando el crecimiento sea mayor a 50 %, agregue al final “Verifica que no haya ventas extraordinarias.”. Pruébala con `frase_variacion(16000, 10000)`.

8.6 Formatos de salida: HTML, PDF y Word

El mismo `.Rmd` puede producir tres formatos distintos. Solo cambia el campo `output` del encabezado.

8.6.1 HTML

```

1 output:
2   html_document:
3     toc: true           # índice (table of contents)
4     toc_float: true     # el índice flota a la izquierda
5     theme: flatly       # tema visual (cerulean, cosmo, journal...)
6     code_folding: hide  # el código se oculta con un botón "Code"
7     number_sections: true

```

El HTML es el formato más flexible: admite gráficas interactivas (como las de **plotly** del Módulo 6), índice flotante y el botón `code_folding`, que es ideal cuando el lector *a veces* quiere ver el código: está oculto, pero a un clic.

8.6.2 PDF

```
1 output:
2   pdf_document:
3     toc: true
4     fig_caption: true
5     extra_dependencies: ["float"]    # paquete LaTeX adicional
6     geometry: margin=2cm            # márgenes de la página
7     lang: es-MX                     # idioma (guiones, "Figura")
```

Para producir PDF, R Markdown necesita una instalación de **LaTeX**, el sistema de composición tipográfica con el que se hacen libros y artículos (este libro, por ejemplo). Si no tienes LaTeX, la forma más sencilla es **TinyTeX**, una distribución pequeña que se instala desde R:

```
1 install.packages("tinytex")
2 tinytex::install_tinytex()    # descarga e instala LaTeX (una sola vez)
3 tinytex::is_tinytex()        # TRUE si quedó instalado
```

La primera vez que tejas un PDF, TinyTeX descargará automáticamente los paquetes de LaTeX que falten; por eso esa primera vez tarda más y necesita Internet.

8.6.3 Word

```
1 output:
2   word_document:
3     toc: true
4     fig_caption: true
5     reference_docx: plantilla_corporativa.docx
```

Word es el formato preferido cuando alguien más va a *editar* el reporte (agregar comentarios, ajustar un párrafo). La opción `reference_docx` es muy poderosa: le das un documento de Word cuyo estilo copiará tu reporte (fuentes, colores de títulos, márgenes, encabezado con el logo de la empresa). Para crear esa plantilla:

1. Teje una vez cualquier `.Rmd` a Word.
2. Abre el `.docx` generado y modifica los *estilos* (no el texto): clic derecho en “Título 1” → **Modificar**, y así con “Normal”, “Título 2”, etc.
3. Guárdalo junto al `.Rmd` con un nombre como `plantilla_corporativa.docx` y agrega la opción `reference_docx`.

8.6.4 Varios formatos en un mismo archivo

Puedes declarar los tres formatos en el encabezado (es lo que hace nuestro reporte ejecutivo). El botón **Knit** muestra entonces un menú para elegir, y desde R puedes generar uno o todos:

```
1 library(rmarkdown)
2 render("reportes/02-reporte-ejecutivo.Rmd",
3       output_format = "pdf_document") # solo PDF
4 render("reportes/02-reporte-ejecutivo.Rmd",
5       output_format = "all")          # HTML, PDF y Word
```

Tabla 8.4. ¿Qué formato de salida elegir?

| | HTML | PDF | Word |
|----------------|---|---|--|
| Requiere | Nada extra | LaTeX (TinyTeX) | Nada extra (Word para abrirlo) |
| Ideal para | Reportes en pantalla, intranet, correo | Documentos formales, impresión, archivo | Documentos que otros van a editar |
| Interactividad | Sí (plotly, tablas DT, pestañas) | No | No |
| Aspecto | Controlado con <code>theme</code> y CSS | Muy profesional, controlado con LaTeX | Controlado con <code>reference_docx</code> |
| Riesgo | Algunos correos bloquean HTML adjunto | Errores de LaTeX al instalar | Se puede modificar el contenido |

Consejo

Una opción que dependa del formato se puede decidir desde el propio código. `knitr::pandoc_to()` devuelve el formato que se está generando ("html", "latex", "docx"). Nuestro reporte ejecutivo la usa para no pedir `fig.align` en Word, que no la admite: `es_word <- knitr::pandoc_to("docx")`.

8.7 Reportes parametrizados

Hasta ahora nuestro reporte habla de “la empresa”. Pero el gerente de la Sucursal Norte quiere *su* reporte, y la directora quiere el de diciembre, no el de noviembre. Copiar el `.Rmd` diez veces y cambiar el número de tienda en cada copia sería volver al trabajo manual. La solución son los **parámetros**.

Concepto clave: Reporte parametrizado

Un **reporte parametrizado** es una plantilla `.Rmd` que declara en su encabezado unos *parámetros* (por ejemplo, `tienda` y `mes`) con un valor por defecto. Dentro del código se usan como `params$tienda` y `params$mes`. Al generar el reporte puedes darles otro valor, y obtienes un documento distinto desde la misma plantilla.

8.7.1 Declarar los parámetros en el YAML

```

1 params:
2   tienda:
3     label: "Tienda (1 a 10)"
4     value: 1
5     input: select
6     choices: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
7   mes:
8     label: "Mes del reporte (AAAA-MM)"
9     value: "2023-12"
10    input: text
11  ruta_datos:
12    label: "Carpeta de datos"
13    value: "datasets"
14    input: text

```

La forma mínima es **tienda: 1**. La forma extendida (con **label**, **value**, **input**) sirve para el botón **Knit > Knit with Parameters...**: RStudio abre un pequeño formulario con una lista desplegable para la tienda y una caja de texto para el mes. Así, alguien que no programa puede generar el reporte de su tienda eligiendo en un menú.

8.7.2 Usar los parámetros en el código

Dentro del `.Rmd` existe automáticamente una lista llamada **params**. Se usa como cualquier lista de R:

```

1 ```{r datos}
2 id_tienda <- as.integer(params$tienda)
3 mes_actual <- params$mes
4 ventas_mes <- ventas %>%
5   filter(tienda_id == id_tienda, mes == mes_actual)
6 ```
7
8 # Reporte de la tienda `r id_tienda` (`r mes_actual`)

```

Un truco muy útil para *desarrollar* la plantilla: en la consola crea tú una lista **params** con los valores que quieras probar, y ejecuta los bloques uno por uno. El código se comporta igual que al tejer. Hagamos eso con la validación que usa nuestro reporte ejecutivo:

```

1 # En la consola simulamos los parámetros que recibiría el .Rmd
2 params <- list(tienda = 3, mes = "2023-12")
3
4 id_tienda <- as.integer(params$tienda)
5 mes_actual <- params$mes
6

```

```

7 # Validaciones: mejor un error claro que un reporte vacío
8 info_tienda <- tiendas %>% filter(tienda_id == id_tienda)
9 if (nrow(info_tienda) == 0) stop("La tienda ", id_tienda, " no existe.")
10 if (!mes_actual %in% transacciones$mes) {
11   stop("No hay datos del mes ", mes_actual)
12 }
13
14 # Mes anterior: el día 1 del mes menos un día cae en el mes previo
15 mes_previo <- format(as.Date(paste0(mes_actual, "-01")) - 1, "%Y-%m")
16
17 info_tienda$nombre_tienda
18 mes_previo

```

```

[1] "Sucursal Sur"
[1] "2023-11"

```

El cálculo del mes anterior merece una explicación: "2023-12-01" menos un día es "2023-11-30", y su formato "%Y-%m" es "2023-11". El truco funciona también en enero (el 1 de enero menos un día es diciembre del año anterior), algo en lo que suelen fallar las fórmulas hechas a mano.

▲ Cuidado

Dentro de un documento que se está tejiendo, `params` es de solo lectura: `params$tienda <- 5` produce el error “no se puede cambiar el valor de un vínculo bloqueado para ‘params’”. Si necesitas transformar un parámetro, guárdalo en otra variable, como hicimos con `id_tienda`.

8.7.3 Generar el reporte con otros parámetros desde R

La función `rmarkdown::render()` recibe los parámetros en una lista. Todo lo que no indiques toma el valor por defecto del YAML:

```

1 library(rmarkdown)
2 # Reporte de la Sucursal Sur (tienda 3), diciembre de 2023, en PDF
3 render("reportes/02-reporte-ejecutivo.Rmd",
4       output_format = "pdf_document",
5       output_file   = "reporte_T03_2023-12.pdf",
6       output_dir    = "resultados/reportes",
7       params        = list(tienda = 3, mes = "2023-12"),
8       knit_root_dir = getwd())

```

Los argumentos de `render()` que usarás siempre son:

- `input`: la plantilla `.Rmd`.
- `output_format`: "html_document", "pdf_document", "word_document" o "all".
- `output_file` y `output_dir`: nombre y carpeta del archivo generado (si no los das, se guarda

junto al `.Rmd` con el mismo nombre, y cada reporte *sobrescribe* al anterior).

- `params`: la lista de parámetros.
- `knit_root_dir`: el directorio de trabajo en el que se ejecutará el código del `.Rmd` (lo explicamos enseguida).
- `envir`: el entorno donde se crean los objetos; `new.env()` garantiza que cada reporte empiece limpio.
- `quiet = TRUE`: no imprime el avance de knitr.

8.7.4 ¿Desde qué carpeta se ejecuta el `.Rmd`?

Este detalle causa más dolores de cabeza que cualquier otro, así que vale la pena entenderlo bien. Cuando tejes, **el directorio de trabajo es la carpeta donde está el `.Rmd`**, sin importar cuál sea el directorio de trabajo de tu consola o de tu proyecto. Con nuestra estructura:

- Tu consola está en `curso-bi-rstudio/`: ahí `datasets/transacciones.csv` existe.
- El `.Rmd` está en `curso-bi-rstudio/reportes/`: desde ahí, `datasets/transacciones.csv` *no* existe (habría que escribir `../datasets/transacciones.csv`).

Hay tres soluciones, y nuestro reporte ejecutivo combina las dos primeras para funcionar en cualquier caso:

1. `knit_root_dir` en `render()`: le dice a knitr que ejecute el código desde otra carpeta. Con `knit_root_dir = getwd()` (o la ruta del proyecto) el código corre en la carpeta del curso y `datasets/` se encuentra. En RStudio puedes lograr lo mismo desde el botón Knit: **Knit > Knit Directory > Project Directory**.
2. Un parámetro con la ruta y una búsqueda robusta: el parámetro `ruta_datos` indica la carpeta de datos y el código la busca en el directorio actual y, si no, en el superior:

```

1  ```{r datos}
2  candidatas <- c(params$ruta_datos, file.path(".", params$ruta_datos))
3  carpeta_datos <- candidatas[dir.exists(candidatas)][1]
4  if (is.na(carpeta_datos)) {
5    stop("No encontré la carpeta de datos '", params$ruta_datos,
6         "'. Directorio actual: ", getwd(),
7         "'. Usa knit_root_dir en render() o el parámetro ruta_datos.")
8  }
9  transacciones <- read_csv(file.path(carpeta_datos, "transacciones.csv"),
10                           show_col_types = FALSE)
11  ```

```

3. **Paquete here**: `here::here("datasets", "transacciones.csv")` construye la ruta desde la raíz del proyecto de RStudio, esté donde esté el `.Rmd`. Es una excelente opción si trabajas con proyectos (`.Rproj`).

Así, el reporte funciona si presionas Knit en RStudio (se ejecuta en `reportes/` y encuentra `../datasets`), si lo generas con `render(..., knit_root_dir = getwd())` (encuentra `datasets`), y si alguien tiene los datos en otra carpeta (le pasa `params = list(ruta_datos = "D:/datos")`). Y si no los encuentra, el mensaje de error dice exactamente qué pasó y cómo arreglarlo.

Hazlo tú

Genera desde la consola el reporte ejecutivo de la Sucursal Premium (tienda 9) para octubre de 2023 en Word. Luego ábrelo con **Knit > Knit with Parameters...** y genera el de la tienda 4 de junio en HTML.

8.8 Automatización: un reporte por tienda, sin tocar nada

Con una plantilla parametrizada, generar los diez reportes es cuestión de un bucle. El script `generar_reportes.R` hace cuatro cosas: construye los nombres de archivo, llama a `render()` una vez por tienda, atrapa los errores para que una tienda con problemas no detenga a las demás, y deja un registro de lo generado. Veamos cada pieza por separado y luego el script completo.

8.8.1 Nombres de archivo con fecha

Un buen nombre de archivo dice qué es, de qué periodo y cuándo se generó, y se ordena bien alfabéticamente. Usaremos el formato `AAAA-MM_Tnn_nombre-de-tienda_AAAA-MM-DD.html`. Como los nombres de las tiendas tienen espacios (y podrían tener acentos), los “limpiamos”:

```
1 # "Sucursal Plaza A" -> "sucursal-plaza-a" (sin acentos ni espacios)
2 limpiar_nombre <- function(x) {
3   x <- chartr("áéíóúüñÁÉÍÓÚÜÑ", "aeiouunAEIOUUN", x) # quita acentos
4   x <- gsub("[^A-Za-z0-9]+", "-", x) # lo que no sea letra -> guion
5   tolower(gsub("^-|-$", "", x)) # sin guiones al inicio/final
6 }
7 limpiar_nombre(c("Sucursal Plaza A", "Tienda Querétaro Ñuñoa"))
8
9 # En el script real la fecha es Sys.Date(); aquí la fijamos para que
10 # el ejemplo dé siempre el mismo resultado
11 fecha_ejecucion <- as.Date("2024-01-05")
12 mes_reporte <- "2023-12"
13
14 archivos <- sprintf("%s_T%02d_%s_%s.html", mes_reporte,
15                      tiendas$tienda_id,
16                      limpiar_nombre(tiendas$nombre_tienda),
17                      format(fecha_ejecucion, "%Y-%m-%d"))
18 head(archivos, 4)
```

```
[1] "sucursal-plaza-a"      "tienda-queretaro-nunoa"
[1] "2023-12_T01_sucursal-centro_2024-01-05.html"
[2] "2023-12_T02_sucursal-norte_2024-01-05.html"
[3] "2023-12_T03_sucursal-sur_2024-01-05.html"
[4] "2023-12_T04_sucursal-este_2024-01-05.html"
```

`sprintf()` arma el texto con un molde: `%s` inserta un texto y `%02d` un número entero con dos dígitos (`1` → `01`), para que la tienda 10 quede después de la 9 al ordenar.

8.8.2 La tabla de trabajos: qué reportes generar

Cuando los reportes dependen de dos parámetros (tienda y mes, o tienda y formato), conviene armar primero una **tabla de trabajos** con todas las combinaciones y después recorrerla. `expand_grid()` de **tidyr** crea todas las combinaciones:

```
1 trabajos <- expand_grid(tienda_id = tiendas$tienda_id,
2                       mes = c("2023-11", "2023-12")) %>%
3   left_join(tiendas %>% select(tienda_id, nombre_tienda),
4             by = "tienda_id") %>%
5   mutate(archivo = sprintf("%s_T%02d_%s.html", mes, tienda_id,
6                                limpiar_nombre(nombre_tienda)))
7
8 nrow(trabajos)
9 head(trabajos, 3)
10
11 # La lista de parámetros de la primera fila, lista para render()
12 parametros <- list(tienda = trabajos$tienda_id[1],
13                   mes = trabajos$mes[1])
14 str(parametros)
```

```
[1] 20
# A tibble: 3 x 4
  tienda_id mes      nombre_tienda  archivo
  <dbl> <chr>    <chr>          <chr>
1         1 2023-11 Sucursal Centro 2023-11_T01_sucursal-centro.html
2         1 2023-12 Sucursal Centro 2023-12_T01_sucursal-centro.html
3         2 2023-11 Sucursal Norte 2023-11_T02_sucursal-norte.html
List of 2
 $ tienda: num 1
 $ mes   : chr "2023-11"
```


8.8.3 Errores controlados con tryCatch

Si al generar el reporte de la tienda 7 ocurre un error (por ejemplo, no tiene ventas ese mes), sin protección el bucle se detiene y las tiendas 8, 9 y 10 se quedan sin reporte. `tryCatch()` ejecuta un código y, si falla, en vez de detenerse ejecuta la función que le indique en `error`. Para practicar sin generar archivos, simulemos `render()` con una función que valide los parámetros igual que el `.Rmd`:

```
1 # Simula render(): falla igual que el .Rmd cuando algo no cuadra
2 generar_uno <- function(tienda, mes) {
3   if (!tienda %in% tiendas$tienda_id) {
4     stop("La tienda ", tienda, " no existe.")
5   }
6   if (!mes %in% transacciones$mes) stop("No hay datos del mes ", mes)
7   sprintf("reporte_T%02d_%s.html", tienda, mes)
8 }
9
10 pendientes <- tibble(tienda = c(1, 2, 15, 3),
11                      mes = c("2023-12", "2023-12", "2023-12", "2024-01"))
12 registro <- vector("list", nrow(pendientes))
13
14 for (fila in seq_len(nrow(pendientes))) {
15   resultado <- tryCatch({
16     archivo <- generar_uno(pendientes$tienda[fila], pendientes$mes[fila])
17     paste("OK:", archivo)
18   }, error = function(e) {
19     paste("ERROR:", conditionMessage(e)) # el mensaje del error
20   })
21   registro[[fila]] <- tibble(tienda = pendientes$tienda[fila],
22                             mes = pendientes$mes[fila],
23                             estado = if (startsWith(resultado, "OK"))
24                               "OK" else "ERROR",
25                             detalle = resultado)
26 }
27 registro <- bind_rows(registro)
28 registro
```

```
# A tibble: 4 x 4
  tienda mes      estado detalle
  <dbl> <chr>    <chr> <chr>
1     1 2023-12 OK      OK: reporte_T01_2023-12.html
2     2 2023-12 OK      OK: reporte_T02_2023-12.html
3    15 2023-12 ERROR  ERROR: La tienda 15 no existe.
4     3 2024-01 ERROR  ERROR: No hay datos del mes 2024-01
```

El bucle terminó las cuatro vueltas. Dos reportes salieron bien y dos fallaron, pero en lugar de detenerse, cada error quedó anotado con su causa: la tienda 15 no existe y enero de 2024 no tiene datos. Esa tabla es el **registro** (*log*): al día siguiente, en vez de preguntarte “¿se habrán generado

los reportes?”, lo abres y lo sabes.

8.8.4 El script completo

Este es `generar_reportes.R`. Como llama a `render()` y escribe archivos, lo mostramos como referencia; en la sección de caso práctico verás su ejecución real.

reportes/generar_reportes.R (parte 1: configuración)

```

1  suppressPackageStartupMessages({
2    library(rmarkdown)
3    library(readr)
4    library(dplyr)
5  })
6
7  # ---- 1. Configuración -----
8  # Argumentos opcionales de la terminal: mes (AAAA-MM) y formato
9  argumentos <- commandArgs(trailingOnly = TRUE)
10 mes_reporte <- if (length(argumentos) >= 1) argumentos[1] else "2023-12"
11 formato     <- if (length(argumentos) >= 2) argumentos[2] else "html"
12 # En producción usarías el mes anterior a hoy, por ejemplo:
13 # mes_reporte <- format(lubridate::floor_date(Sys.Date(), "month") - 1,
14 #                       "%Y-%m")
15
16 # Formatos disponibles: nombre corto -> formato de R Markdown y extensión
17 formatos <- list(
18   html = c("html_document", "html"),
19   pdf  = c("pdf_document", "pdf"),
20   word = c("word_document", "docx")
21 )
22 if (!formato %in% names(formatos)) {
23   stop("Formato no válido: ", formato, ". Usa html, pdf o word.")
24 }
25
26 # Carpeta del curso: la actual o la superior (si ejecutas desde reportes/)
27 raiz <- if (dir.exists("datasets")) "." else
28         if (dir.exists("../datasets")) ".."
29 if (is.null(raiz)) {
30   stop("No encuentro datasets/. Ejecuta desde la carpeta del curso.")
31 }
32 raiz <- normalizePath(raiz)
33
34 plantilla      <- file.path(raiz, "reportes", "02-reporte-ejecutivo.Rmd")
35 carpeta_salida <- file.path(raiz, "resultados", "reportes")
36 dir.create(carpeta_salida, recursive = TRUE, showWarnings = FALSE)
37 archivo_log    <- file.path(carpeta_salida, "registro_reportes.csv")
38 fecha_hoy     <- format(Sys.Date(), "%Y-%m-%d")

```

La primera parte lee dos argumentos opcionales con `commandArgs(trailingOnly = TRUE)`: así el

mismo script sirve para cualquier mes y formato cuando se ejecuta desde la terminal ([Rscript reportes/generar_reportes.R 2023-12 pdf](#)). También localiza la carpeta del curso y crea la carpeta de salida.

reportes/generar_reportes.R (parte 2: el bucle)

```

1  # ---- 2. Lista de tiendas -----
2  tiendas <- read_csv(file.path(raiz, "datasets", "tiendas.csv"),
3                      show_col_types = FALSE)
4  # (aquí va la función limpiar_nombre() que ya viste)
5
6  # ---- 3. Bucle: un reporte por tienda -----
7  cat("Generando", nrow(tiendas), "reportes del mes", mes_reporte,
8      "en formato", formato, "\n")
9  registro <- vector("list", nrow(tiendas))
10
11 for (i in seq_len(nrow(tiendas))) {
12   id      <- tiendas$tienda_id[i]
13   nombre  <- tiendas$nombre_tienda[i]
14   archivo <- sprintf("%s_T%02d_%s_%s.%s", mes_reporte, id,
15                       limpiar_nombre(nombre), fecha_hoy,
16                       formatos[[formato]][2])
17   inicio <- Sys.time()
18
19   # tryCatch: si UNA tienda falla, se registra el error y seguimos
20   resultado <- tryCatch({
21     render(
22       input      = plantilla,
23       output_format = formatos[[formato]][1],
24       output_file  = archivo,
25       output_dir   = carpeta_salida,
26       params       = list(tienda = id, mes = mes_reporte),
27       knit_root_dir = raiz,      # el código del .Rmd corre en la raíz
28       envir        = new.env(), # entorno limpio para cada reporte
29       quiet        = TRUE
30     )
31     "OK"
32   }, error = function(e) {
33     paste("ERROR:", conditionMessage(e))
34   })
35
36   segundos <- round(as.numeric(difftime(Sys.time(), inicio,
37                                         units = "secs")), 1)
38   registro[[i]] <- tibble(
39     fecha_hora = format(Sys.time(), "%Y-%m-%d %H:%M:%S"),
40     tienda_id  = id,
41     tienda     = nombre,
42     mes        = mes_reporte,
43     archivo    = archivo,
44     estado     = if (resultado == "OK") "OK" else "ERROR",

```

```

45     detalle     = resultado,
46     segundos    = segundos
47   )
48   cat(sprintf(" [%2d/%d] %-18s %-5s %s\n", i, nrow(tiendas), nombre,
49           registro[[i]]$estado, archivo))
50 }
51
52 # ---- 4. Registro (log) -----
53 registro <- bind_rows(registro)
54 # append = TRUE agrega al historial; los encabezados solo la primera vez
55 write_csv(registro, archivo_log, append = file.exists(archivo_log))
56
57 n_ok <- sum(registro$estado == "OK")
58 cat("\nListo:", n_ok, "reportes generados,", nrow(registro) - n_ok,
59     "con error. Carpeta:", carpeta_salida, "\n")
60 cat("Registro guardado en:", archivo_log, "\n")

```

Tres detalles importantes del bucle:

- `knit_root_dir = raiz` hace que el `.Rmd` encuentre `datasets/` (sección 8.7.4).
- `envir = new.env()` da a cada reporte un entorno vacío. Sin esto, los objetos del reporte de la tienda 1 seguirían existiendo al generar el de la tienda 2, y un error de programación (usar una variable que no se recalculó) podría pasar desapercibido y *mezclar datos de dos tiendas*: justo el error de copiar y pegar que queríamos eliminar.
- El registro se guarda en CSV con `append = TRUE`: cada ejecución agrega sus filas, así que con el tiempo tienes el historial completo de qué se generó y cuándo.

8.8.5 Programar la ejecución

Un script que genera todo con un comando ya es un gran avance. El último paso es que se ejecute *solo*, por ejemplo el día 1 de cada mes a las 7:00. Para eso se usa el programador de tareas del sistema operativo, que llama a **Rscript**, el programa que ejecuta un script de R desde la terminal sin abrir RStudio.

Linux y Mac: cron

cron es el programador de tareas de Linux y macOS. Se edita con el comando `crontab -e` en la terminal, y cada línea es una tarea con cinco campos de tiempo (minuto, hora, día del mes, mes, día de la semana) seguidos del comando:

```

1 # min hora dia mes dia_semana comando
2 # Día 1 de cada mes a las 7:00: reportes del mes anterior en PDF
3 # (todo el comando va en UNA sola línea del crontab)

```

```
4 0 7 1 * * cd /home/andrea/curso-bi-rstudio && /usr/bin/Rscript reportes/
    generar_reportes.R $(date -d "last month" +%Y-%m) pdf >> resultados/cron.log
    2>&1
```

- `0 7 1 * *`: minuto 0, hora 7, día 1, cualquier mes, cualquier día de la semana. (`0 8 * * 1` sería “cada lunes a las 8:00”.)
- `cd ...&&`: primero se cambia a la carpeta del curso, para que las rutas relativas funcionen.
- `$(date -d "last month" +%Y-%m)` calcula el mes anterior (en cron el símbolo `%` se escribe `\%`). En Mac, usa `date -v-1m +%Y-%m`. Otra opción es calcular el mes dentro del script (la línea comentada en la parte 1).
- `>> resultados/cron.log 2>&1` guarda todo lo que imprime el script, incluidos los errores, en un archivo.

Windows: Programador de tareas

En Windows se usa el **Programador de tareas** (*Task Scheduler*):

1. Abre el menú Inicio y busca “Programador de tareas”.
2. Clic en **Crear tarea básica...**, ponle nombre (“Reportes mensuales”) y elige **Mensualmente**, día 1, 7:00.
3. En **Acción** elige **Iniciar un programa**.
4. En **Programa o script** escribe la ruta completa de `Rscript.exe`, que está en la carpeta `bin` de tu instalación de R (por ejemplo, dentro de `C:\Program Files\R\R-4.3.3`).
5. En **Agregar argumentos** escribe `reportes\generar_reportes.R` y en **Iniciar en** la carpeta del curso (por ejemplo `C:\Users\andrea\curso-bi-rstudio`).
6. Finaliza. Para probarla, clic derecho sobre la tarea → **Ejecutar**.

Si prefieres hacerlo desde R, el paquete **taskscheduleR** crea la tarea por ti (y RStudio le agrega un complemento con formulario en **Addins**). En Linux/Mac, el equivalente es **cronR**:

```
1 install.packages("taskscheduleR")
2 library(taskscheduleR)
3 taskscheduler_create(
4   taskname = "reportes_mensuales",
5   rscript  = file.path("C:/Users/andrea/curso-bi-rstudio",
6                        "reportes/generar_reportes.R"),
7   schedule = "MONTHLY",
8   days     = 1,
9   starttime = "07:00"
10 )
11 taskscheduler_ls()           # lista las tareas programadas
12 taskscheduler_delete("reportes_mensuales") # para borrarla
```

▲ Cuidado

Una tarea programada se ejecuta *sin ti*: nadie verá los errores en pantalla. Por eso es indispensable el registro (`registro_reportes.csv` y `cron.log`) y que el script use rutas que funcionen desde la carpeta indicada. Además, la computadora debe estar encendida a esa hora; en empresas, estas tareas suelen vivir en un servidor.

8.8.6 Enviar los reportes por correo

El paquete **blastula** permite redactar y enviar correos desde R, con el reporte como adjunto. Las credenciales (usuario y contraseña del correo) **nunca** se escriben en el script: se guardan una sola vez en un archivo cifrado o en el llavero del sistema.

```

1  install.packages("blastula")
2  library(blastula)
3
4  # UNA sola vez, de forma interactiva: guarda las credenciales cifradas
5  # en el llavero del sistema (te pide la contraseña, no queda en el código)
6  create_smtp_creds_key(id = "correo_bi", user = "reportes@techretail.mx",
7                        host = "smtp.office365.com", port = 587,
8                        use_ssl = TRUE)
9
10 # En generar_reportes.R, después de generar cada archivo:
11 correo <- compose_email(
12   body = md(paste0(
13     "Hola, adjunto el reporte ejecutivo de **", nombre, "** del mes ",
14     mes_reporte, ".\n\nÁrea de Business Intelligence"))
15 ) %>%
16   add_attachment(file = file.path(carpeta_salida, archivo))
17
18 smtp_send(correo,
19           to = "gerente.centro@techretail.mx",
20           from = "reportes@techretail.mx",
21           subject = paste("Reporte de ventas", mes_reporte, "-", nombre),
22           credentials = creds_key(id = "correo_bi"))

```

Para enviar a cada gerente su reporte, agrega a `tiendas.csv` (o a una tabla aparte) una columna con el correo del gerente y úsala dentro del bucle. Otra alternativa segura para las credenciales son las variables de entorno: una línea `CORREO_PASS=...` en el archivo `.Renviron` de tu usuario (que no se comparte ni se sube a Git) y `Sys.getenv("CORREO_PASS")` en el script.

📌 Consejo

Antes de automatizar el envío, deja que el sistema genere los reportes durante uno o dos meses mientras *tú* los revisas y los envías a mano. Cuando confíes en ellos, activa el envío automático. Y agrega una regla de seguridad: si el registro tiene algún **ERROR**, que el script te envíe un correo a ti en vez de a los gerentes.

8.9 Reportes en Excel con writexl

Muchos usuarios de negocio no quieren un PDF: quieren “el Excel” para filtrar, hacer sus propias tablas dinámicas o pegarlo en su presupuesto. En vez de pelear contra eso, dales un Excel bien hecho, generado automáticamente. El paquete **writexl** escribe archivos **.xlsx** sin necesitar Excel ni Java, y si le das una *lista* de data frames, crea una hoja por cada elemento.

Ejemplo práctico: Libro de Excel de ventas 2023

Crear **resultados/reporte_ventas.xlsx** con cuatro hojas: un resumen de KPIs, las ventas mensuales con su variación, las ventas por tienda y el detalle de todas las transacciones con el nombre de la tienda y del producto. Después, releerlo para comprobar que quedó bien.

```

1 library(writexl) # escribir .xlsx
2 library(readxl)  # leer .xlsx (para verificar)
3
4 # Hoja "Detalle": transacciones con nombres legibles
5 detalle <- transacciones %>%
6   left_join(tiendas %>% select(tienda_id, nombre_tienda),
7             by = "tienda_id") %>%
8   left_join(productos %>% select(producto_id, nombre_producto, categoria),
9             by = "producto_id") %>%
10  select(transaccion_id, fecha, mes, nombre_tienda, nombre_producto,
11         categoria, cantidad, precio_venta, total_transaccion,
12         metodo_pago) %>%
13  arrange(fecha, transaccion_id)
14
15 # Hoja "Resumen": un KPI por fila
16 resumen <- tibble(
17   indicador = c("Ventas totales", "Transacciones", "Unidades vendidas",
18               "Ticket promedio"),
19   valor = c(round(sum(detalle$total_transaccion), 2), nrow(detalle),
20             sum(detalle$cantidad),
21             round(mean(detalle$total_transaccion), 2))
22 )
23
24 # Hoja "Mensual": ventas por mes y variación contra el mes anterior
25 mensual <- detalle %>%
26   group_by(mes) %>%
27   summarise(ventas = round(sum(total_transaccion), 2),
28             transacciones = n(), .groups = "drop") %>%
29   mutate(variacion_pct = round((ventas / lag(ventas) - 1) * 100, 1))
30
31 # Hoja "Por tienda"
32 por_tienda <- detalle %>%
33   group_by(nombre_tienda) %>%
34   summarise(ventas = round(sum(total_transaccion), 2),

```

```

35     transacciones = n(),
36     ticket_promedio = round(mean(total_transaccion), 2),
37     .groups = "drop") %>%
38   arrange(desc(ventas))
39
40 resumen

```

```

# A tibble: 4 x 2
  indicador      valor
  <chr>         <dbl>
1 Ventas totales 959993.
2 Transacciones  1000
3 Unidades vendidas 2969
4 Ticket promedio  960.

```

Ahora escribimos las cuatro hojas de una vez. Los nombres de la lista son los nombres de las pestañas (pueden llevar espacios si los pones entre acentos graves):

```

1  dir.create("resultados", showWarnings = FALSE)
2
3  write_xlsx(
4    list(Resumen      = resumen,
5          Mensual      = mensual,
6          `Por tienda` = por_tienda,
7          Detalle      = detalle),
8    path = "resultados/reporte_ventas.xlsx"
9  )
10
11 # Verificación: ¿qué hojas tiene y cuántas filas hay en cada una?
12 hojas <- excel_sheets("resultados/reporte_ventas.xlsx")
13 hojas
14 sapply(hojas, function(h) {
15   nrow(read_excel("resultados/reporte_ventas.xlsx", sheet = h))
16 })

```

```

[1] "Resumen"    "Mensual"    "Por tienda" "Detalle"
     Resumen    Mensual Por tienda    Detalle
         4         12         10         1000

```

Las cuatro hojas están ahí y el número de filas cuadra: 4 KPIs, 12 meses, 10 tiendas y 1,000 transacciones. Releer lo que escribiste es una verificación barata que te ahorra enviar un archivo vacío. Comprobemos también que las fechas y números no se convirtieron en texto:

```

1  read_excel("resultados/reporte_ventas.xlsx", sheet = "Mensual") %>%
2    head(4)

```



```
# A tibble: 4 x 4
  mes      ventas transacciones variacion_pct
<chr>   <dbl>         <dbl>         <dbl>
1 2023-01 82560.           98           NA
2 2023-02 65899.           79        -20.2
3 2023-03 90931.           91           38
4 2023-04 57038.           71        -37.3
```

Los números llegan como `<dbl>`, así que en Excel se podrán sumar y graficar. La variación de enero es `NA` (no hay mes anterior dentro de los datos) y en Excel aparecerá como celda vacía.

El archivo `codigo/reportes/reporte_excel.R` contiene la versión completa de este ejemplo, con una hoja adicional de los 15 productos más vendidos. Para ejecutarlo, copia el archivo a tu carpeta `reportes/` y usa `source("reportes/reporte_excel.R")`.

8.9.1 Formato avanzado con openxlsx

writexl es rápido y sencillo, pero no da formato (colores, ancho de columnas, formato de moneda). Si lo necesitas, usa **openxlsx**:

```
1 install.packages("openxlsx")
2 library(openxlsx)
3 libro <- createWorkbook()
4 addWorksheet(libro, "Por tienda")
5 writeData(libro, "Por tienda", por_tienda,
6           headerStyle = createStyle(textDecoration = "bold",
7                                     fgFill = "#173F69",
8                                     fontColour = "white"))
9 addStyle(libro, "Por tienda", createStyle(numFmt = "$#,##0.00"),
10          rows = 2:11, cols = c(2, 4), gridExpand = TRUE)
11 setColWidths(libro, "Por tienda", cols = 1:4, widths = "auto")
12 freezePane(libro, "Por tienda", firstRow = TRUE) # fija el encabezado
13 saveWorkbook(libro, "resultados/reporte_formato.xlsx", overwrite = TRUE)
```

Hazlo tú

Agrega al libro de Excel una hoja `"Por categoría"` con las ventas y unidades por `categoría` de producto, ordenada de mayor a menor, y comprueba con `excel_sheets()` que ahora hay cinco hojas.

8.10 Quarto: el sucesor de R Markdown

Quarto es la nueva generación de R Markdown, creada por la misma empresa (Posit, antes RStudio). Funciona con R, pero también con Python y Julia, y viene incluido en las versiones recientes de RStudio. La buena noticia: **casi todo lo que aprendiste en este módulo sirve igual**. El

Markdown es el mismo, knitr sigue ejecutando el código R y pandoc sigue convirtiendo el documento. Cambian tres cosas:

1. La extensión: `.qmd` en lugar de `.Rmd`.
2. En el YAML, `output: html_document` pasa a ser `format: html` (y `pdf`, `docx`).
3. Las opciones de los bloques se escriben preferentemente *dentro* del bloque, en líneas que empiezan con `#|`, y con guiones en lugar de puntos (`fig-cap` en vez de `fig.cap`).

reportes/reporte-quarto.qmd

```

1  ---
2  title: "Reporte de ventas"
3  format:
4    html:
5      toc: true
6      docx: default
7  params:
8    tienda: 1
9  execute:
10    echo: false
11    warning: false
12  ---
13
14  ```{r}
15  #| label: cargar-datos
16  library(dplyr)
17  library(readr)
18  ventas <- read_csv("../datasets/transacciones.csv",
19                    show_col_types = FALSE)
20  ventas_tienda <- filter(ventas, tienda_id == params$tienda)
21  ```
22
23  La tienda `r params$tienda` vendió
24  `$`r round(sum(ventas_tienda$total_transaccion))` en 2023.
25
26  ```{r}
27  #| label: fig-mensual
28  #| fig-cap: "Ventas mensuales"
29  #| fig-width: 7
30  ventas_tienda %>%
31    count(mes, wt = total_transaccion) %>%
32    with(barplot(n, names.arg = substr(mes, 6, 7)))
33  ```

```

Las opciones globales ya no van en un bloque `setup:` se ponen en el YAML, bajo `execute`. Para generarlo usas el botón **Render** (el equivalente de Knit) o la terminal:

```

1 quarto render reportes/reporte-quarto.qmd          # todos los formatos
2 quarto render reportes/reporte-quarto.qmd --to docx # solo Word
3 quarto render reportes/reporte-quarto.qmd -P tienda:3 # con parámetro

```

Desde R, el paquete **quarto** ofrece `quarto_render()`, equivalente a `rmarkdown::render()`:

```

1 install.packages("quarto")
2 quarto::quarto_render("reportes/reporte-quarto.qmd",
3                       output_format = "html",
4                       execute_params = list(tienda = 3))

```

Consejo

¿R Markdown o Quarto? Si empiezas un proyecto nuevo, Quarto es la apuesta a futuro, sobre todo si tu equipo también usa Python o quieres hacer presentaciones y sitios web. Pero R Markdown no va a desaparecer: sigue mantenido, hay miles de reportes en producción escritos con él y Quarto puede generar la mayoría de los `.Rmd` sin cambios. Lo que aprendas con uno lo aplicas al otro.

8.11 Buenas prácticas para reportes automáticos

Un reporte automático que falla en silencio es peor que un reporte manual: genera confianza en números equivocados. Estas prácticas lo evitan.

8.11.1 Reproducibilidad

- **Rutas relativas**, nunca `setwd("C:/Users/andrea/...")` dentro del reporte: en otra computadora esa ruta no existe. Usa rutas relativas a la carpeta del proyecto, `knit_root_dir` o `here`.
- **El `.Rmd` debe ser autocontenido**: cada objeto que usa lo crea él mismo. Knit se ejecuta en una sesión limpia y *no ve* los objetos de tu consola (sección 8.13).
- `set.seed()` antes de cualquier cálculo aleatorio (muestras, simulaciones, k-means del Módulo 7), para que dos ejecuciones den el mismo reporte.
- **Versiones de paquetes**: una actualización de un paquete puede cambiar un resultado o romper el código. El paquete **renv** guarda las versiones exactas que usa tu proyecto en un archivo `renv.lock`, y en otra computadora `renv::restore()` instala esas mismas versiones.

```

1 install.packages("renv")
2 renv::init()          # crea una biblioteca de paquetes propia del proyecto
3 renv::snapshot()      # guarda las versiones actuales en renv.lock
4 renv::restore()       # (en otra PC) instala exactamente esas versiones

```

8.11.2 Organización

- **Separa datos, código y salidas:** `datasets/` (entrada, no se modifica a mano), `reportes/` (plantillas y scripts) y `resultados/` (todo lo que se puede borrar y regenerar).
- **Nombres de archivo** sin espacios ni acentos, con fechas en formato AAAA-MM-DD (se ordenan solos) y con lo que identifica al reporte: `2023-12_T03_sucursal-sur_2024-01-05.pdf`.
- **Nombra tus chunks** (`cargar-datos`, `grafica-tendencia`): cuando algo falla, el mensaje de error dice el nombre del bloque y encuentras el problema en segundos.
- **Pon la lógica pesada en funciones o scripts** y deja en el `.Rmd` la presentación. Si una función se usa en varios reportes, guárdala en un `funciones.R` y cárgala con `source()`.

8.11.3 Calidad: revisa antes de enviar

- **Valida los datos de entrada** dentro del reporte y detente con un error claro (`stop()`) si algo no cuadra: mes sin datos, tienda inexistente, columnas faltantes.
- **Cuadra los totales:** la suma de las tiendas debe dar el total de la empresa; compara un par de cifras con el sistema fuente.
- **Lee el reporte generado**, al menos al principio y cada vez que cambies la plantilla: ¿las frases tienen sentido para una tienda que cayó y para una que creció? ¿se ve bien en PDF y en Word?
- **Advierte las limitaciones:** si hay pocos datos, dilo en el propio reporte; si detectas un problema de calidad (como los productos con costo mayor al precio de catálogo que viste en módulos anteriores), menciónalo.

8.12 Caso práctico integral: reporte ejecutivo mensual automático

En la empresa: El encargo

La directora comercial de TechRetail México pide: “Quiero que cada gerente reciba el primer día hábil del mes un reporte de su tienda con los números del mes anterior, cómo va contra el mes previo, cómo se compara con las demás tiendas, qué productos vendió más y una conclusión clara de qué hacer. En PDF, para imprimirlo en la junta; y quiero poder generar el de cualquier tienda y mes cuando lo necesite”.

La solución tiene tres piezas que ya conoces: la plantilla parametrizada `02-reporte-ejecutivo.Rmd`, el script `generar_reportes.R` y la tarea programada. Recorramos la plantilla por partes.

8.12.1 Parte 1: encabezado y configuración

reportes/02-reporte-ejecutivo.Rmd (encabezado)

```

1 ---
2 title: "Reporte ejecutivo de ventas"
3 subtitle: "Tienda `r params$tienda` | Periodo `r params$mes`"
4 author: "Área de Business Intelligence - TechRetail México"
5 date: "Generado el `r format(Sys.Date(), '%d/%m/%Y')`"
6 lang: es-MX
7 geometry: margin=2cm
8 header-includes:
9   - \AtBeginDocument{\renewcommand{\tablename}{Tabla}}
10 params:
11   tienda:
12     label: "Tienda (1 a 10)"
13     value: 1
14     input: select
15     choices: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
16   mes:
17     label: "Mes del reporte (AAAA-MM)"
18     value: "2023-12"
19     input: text
20   ruta_datos:
21     label: "Carpeta de datos"
22     value: "datasets"
23     input: text
24 output:
25   html_document:
26     toc: true
27     toc_float: true
28     theme: flatly
29   pdf_document:
30     fig_caption: true
31     extra_dependencies: ["float"]
32   word_document:
33     fig_caption: true
34 ---

```

El subtítulo usa los parámetros, así que cada reporte dice de qué tienda y periodo es. **lang: es-MX** pone en español las palabras automáticas (“Figura”) y **header-includes** agrega una instrucción de LaTeX para que las tablas del PDF digan “Tabla” y no “Cuadro”. El paquete **float** de **extra_dependencies** permite fijar cada figura donde aparece en el texto (opción **fig.pos = "H"**).

Bloque configuracion

```

1 ```{r configuracion, include=FALSE}
2 # ---- Opciones globales: un reporte ejecutivo NO muestra código ----
3 es_word <- knitr::pandoc_to("docx") # TRUE si la salida es Word

```

```

4 knitr::opts_chunk$set(
5   echo = FALSE, message = FALSE, warning = FALSE,
6   fig.width = 7, fig.height = 3.4, out.width = "90%",
7   fig.pos = "H",                      # PDF: la figura va donde está
8   fig.align = if (es_word) "default" else "center"
9 )
10 library(dplyr)
11 library(readr)
12 library(ggplot2)
13 library(scales)
14 library(knitr)
15
16 # Colores corporativos
17 color_principal <- "#2a78d6"
18 color_resalte   <- "#eb6834"
19 color_gris      <- "#b5b3ad"
20 tema_reporte <- theme_minimal(base_size = 11) +
21   theme(panel.grid.minor = element_blank(),
22         plot.title = element_text(face = "bold"))
23
24 # Nombres de los meses en español (no dependen del idioma de la PC)
25 meses_es <- c("enero", "febrero", "marzo", "abril", "mayo", "junio",
26              "julio", "agosto", "septiembre", "octubre", "noviembre",
27              "diciembre")
28 nombre_mes <- function(m) {
29   paste(meses_es[as.integer(substr(m, 6, 7))], "de", substr(m, 1, 4))
30 }
31 # Formatos de negocio ("s/d" = sin dato cuando el valor es NA)
32 dinero <- function(x) ifelse(is.na(x), "s/d", dollar(x, accuracy = 1))
33 dinero2 <- function(x) ifelse(is.na(x), "s/d", dollar(x, accuracy = 0.01))
34 pct <- function(x) ifelse(is.na(x), "s/d", percent(x, accuracy = 0.1))
35 ```

```

8.12.2 Parte 2: datos, validación y KPIs

El bloque **datos** busca la carpeta de datos (como viste en la sección 8.7.4), lee y une transacciones y productos, valida los parámetros con **stop()** (tienda inexistente, mes sin datos), calcula el mes anterior con el truco del día 1 menos un día y filtra la tienda en el mes actual y el anterior (**ventas_mes**, **ventas_previo**). Es el mismo código que probaste en la consola en la sección 8.7.

El bloque **kpis** calcula los indicadores con una función que se aplica igual al mes actual y al anterior (así no hay dos cálculos que puedan desincronizarse), la posición de la tienda en el ranking del mes y la frase de variación. Antes de ponerlo en la plantilla, probemos la función en la consola con la tienda 1 en diciembre, el mismo caso que verás en la figura del reporte:

```

1 calcular_kpis <- function(datos) {
2   tibble(

```

```

3     ventas      = sum(datos$total_transaccion),
4     transacciones = nrow(datos),
5     unidades     = sum(datos$cantidad),
6     ticket       = if (nrow(datos) > 0) ventas / transacciones
7                   else NA_real_
8   )
9 }
10
11 tienda_1 <- transacciones %>% filter(tienda_id == 1)
12 kpi      <- calcular_kpis(tienda_1 %>% filter(mes == "2023-12"))
13 kpi_ant  <- calcular_kpis(tienda_1 %>% filter(mes == "2023-11"))
14 bind_rows(diciembre = kpi, noviembre = kpi_ant, .id = "periodo")
15
16 # Ranking de las tiendas en diciembre
17 ranking <- ventas_dic %>% mutate(posicion = row_number())
18 ranking %>% filter(nombre_tienda == "Sucursal Centro") %>%
19   select(nombre_tienda, ventas, posicion)

```

```

# A tibble: 2 x 5
  periodo  ventas transacciones unidades ticket
<chr>      <dbl>         <int>      <dbl> <dbl>
1 diciembre 5020.             5         16 1004.
2 noviembre 10593.            9         24 1177.
# A tibble: 1 x 3
  nombre_tienda  ventas posicion
<chr>          <dbl>    <int>
1 Sucursal Centro 5020.         8

```

Fíjate en el detalle de `calcular_kpis()`: dentro de `tibble()` la columna `ticket` usa las columnas `ventas` y `transacciones` que se acaban de crear, y si no hay ventas devuelve `NA` en lugar de dividir entre cero. La Sucursal Centro vendió \$5,020 en 5 transacciones y quedó en el lugar 8 de 10. Estos son los números que deben aparecer en el reporte: *así se revisa un reporte automático*, comparando un caso contra un cálculo independiente.

Bloque kpis (extracto: la frase de variación)

```

1 texto_variacion <- if (!hay_previo) {
2   "no se puede comparar porque no hay ventas registradas el mes anterior"
3 } else if (var_ventas >= 0) {
4   paste0("crecieron **", pct(var_ventas), "** respecto a ",
5         nombre_mes(mes_previo), " (", dinero(kpi_ant$ventas), ")")
6 } else {
7   paste0("cayeron **", pct(abs(var_ventas)), "** respecto a ",
8         nombre_mes(mes_previo), " (", dinero(kpi_ant$ventas), ")")
9 }

```

8.12.3 Parte 3: el texto del resumen y la tabla de KPIs

Con todo calculado, el resumen es un párrafo normal de Markdown salpicado de código en línea:

```
1 # `r info_tienda$nombre_tienda` (`r info_tienda$ciudad`)
2
3 ## Resumen del mes
4
5 En **`r nombre_mes(mes_actual)`**, la **`r info_tienda$nombre_tienda`**
6 vendió **`r dinero(kpi$ventas)`** en `r kpi$transacciones` transacciones,
7 con un ticket promedio de **`r dinero2(kpi$ticket)`**. Las ventas
8 `r texto_variacion`. La tienda ocupó el lugar **`r posicion` de
9 `r n_tiendas`** en ventas del mes y aportó el **`r pct(participa)`** de las
10 ventas de la empresa.
```

Debajo, el bloque `tabla-kpis` arma un tibble con cuatro filas (ventas, transacciones, unidades y ticket), las columnas “Mes actual”, “Mes anterior” y “Variación” ya formateadas con `dinero()` y `pct()`, y lo muestra con `kable()` y el título `paste("Indicadores clave de", nombre_mes(mes_actual))`.

8.12.4 Parte 4: las gráficas

El reporte tiene tres gráficas que siguen las reglas del Módulo 3: la tendencia del año con el mes del reporte resaltado en naranja y el resto en gris, el ranking de tiendas con la tienda del reporte resaltada, y las ventas por categoría en un solo color. Los títulos de las figuras (`fig.cap`) son variables de R, de modo que también dicen el mes correcto:

Bloque grafica-tendencia

```
1 ```{r grafica-tendencia, fig.cap = titulo_tendencia}
2 mensual <- ventas_tienda %>%
3   filter(mes <= mes_actual) %>%
4   group_by(mes) %>%
5   summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
6   mutate(es_actual = mes == mes_actual,
7          # "2023-12" -> "dic" (y respeta el orden cronológico)
8          etiqueta = substr(meses_es[as.integer(substr(mes, 6, 7))], 1, 3),
9          etiqueta = factor(etiqueta, levels = etiqueta))
10
11 ggplot(mensual, aes(x = etiqueta, y = ventas, fill = es_actual)) +
12   geom_col(width = 0.7) +
13   scale_fill_manual(values = c(`FALSE` = color_gris,
14                                `TRUE` = color_resalte),
15                     guide = "none") +
16   scale_y_continuous(labels = label_dollar(scale = 1e-3, suffix = " mil")) +
17   labs(x = NULL, y = "Ventas") +
18   tema_reporte
19 ```
```


El filtro `mes <= mes_actual` hace que el reporte de junio muestre solo enero a junio: un reporte de un mes pasado no debe “ver el futuro”. Las otras dos gráficas siguen el mismo patrón (consúltalas en el archivo completo).

8.12.5 Parte 5: conclusiones automáticas

El último bloque aplica las reglas de negocio de la sección 8.5 y escribe una lista de Markdown con `results = "asis"`:

Bloque conclusiones (extracto)

```

1  ```{r conclusiones, results = "asis"}
2  conclusiones <- character(0)
3
4  # 1. Crecimiento contra el mes anterior
5  if (!hay_previo) {
6    conclusiones <- c(conclusiones,
7      "No hay ventas del mes anterior para comparar el crecimiento.")
8  } else if (var_ventas > 0.10) {
9    conclusiones <- c(conclusiones, paste0(
10     "**Crecimiento sólido:** las ventas subieron ", pct(var_ventas),
11     ". Identifica qué funcionó para replicarlo en otras tiendas.))
12  } else if (var_ventas >= 0) {
13    # ... "Estabilidad" ...
14  } else if (var_ventas > -0.10) {
15    # ... "Ligera caída" ...
16  } else {
17    conclusiones <- c(conclusiones, paste0(
18     "**Alerta:** las ventas cayeron ", pct(abs(var_ventas)),
19     ". Revisa inventario, personal y promociones con el gerente.))
20  }
21  # 2. Posición (top 3 / 3 últimas)    3. Ticket contra la empresa
22  # 4. Pocos datos: advertir que las variaciones pueden ser azar
23  if (kpi$transacciones < 10) {
24    conclusiones <- c(conclusiones, paste0(
25     "Precaución: el mes solo tiene ", kpi$transacciones,
26     " transacciones; interpreta las variaciones con cuidado.))
27  }
28  # 5. Calidad de datos: productos con costo mayor al precio de catálogo
29
30  # Imprimir como lista de Markdown (por eso results = "asis")
31  cat(paste0("- ", conclusiones), sep = "\n")
32  ```

```

La regla 4 es un ejemplo de honestidad analítica: con menos de 10 transacciones en el mes, una caída del 50 % puede ser simple azar, y el reporte lo dice. La regla 5 revisa un problema de calidad de datos que ya conoces: hay productos cuyo costo es mayor que su precio de catálogo.

8.12.6 El resultado

Generamos el reporte de la Sucursal Centro de diciembre de 2023 en PDF con:

```
1 rmarkdown::render("reportes/02-reporte-ejecutivo.Rmd",
2                   output_format = "pdf_document",
3                   output_file   = "reporte.pdf",
4                   output_dir    = "resultados",
5                   params         = list(tienda = 1, mes = "2023-12"),
6                   knit_root_dir = getwd())
```

La figura 8.3 muestra las dos primeras páginas del PDF tal como salieron, sin ningún retoque. Todo lo que ves (el nombre de la tienda, los montos, la frase “cayeron 52.6 %”, la tabla, la barra naranja de diciembre) salió del código.

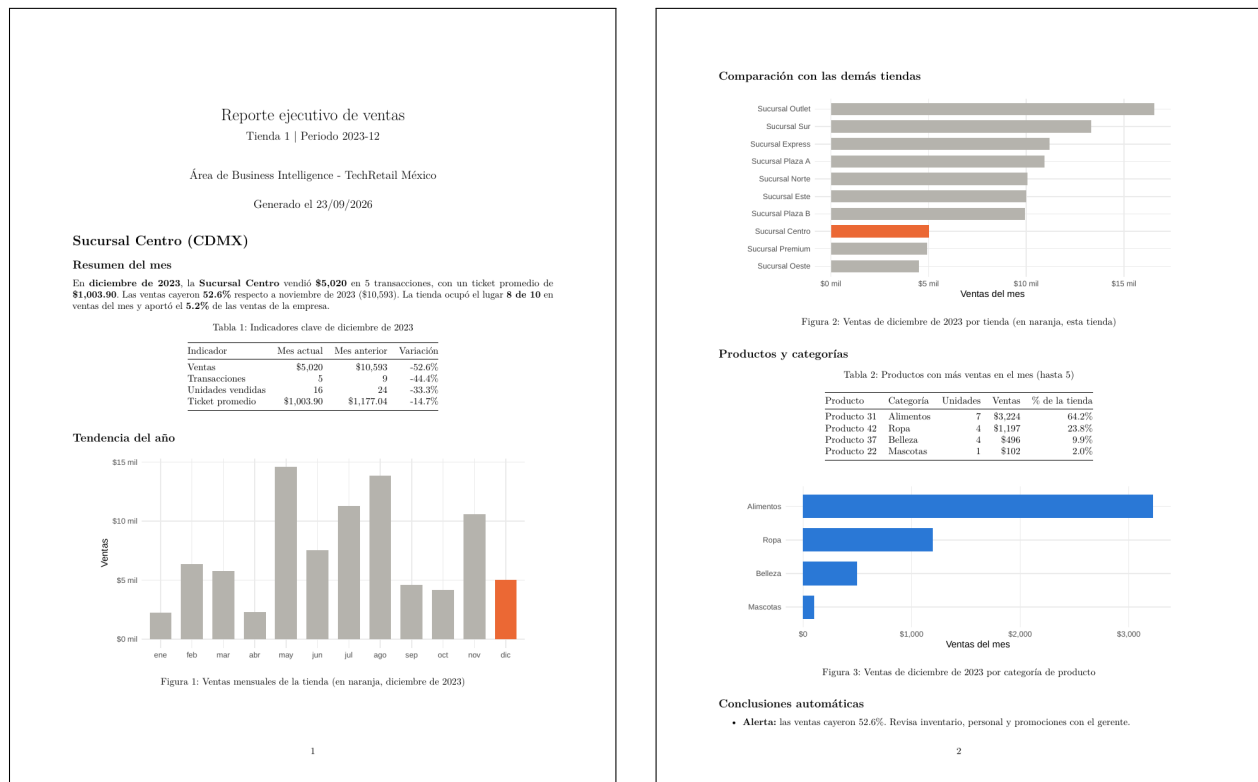


Figura 8.3. Páginas 1 y 2 del reporte ejecutivo en PDF generado para la Sucursal Centro, diciembre de 2023.

La tercera página contiene el final de las conclusiones automáticas. Para esta tienda, el reporte escribió:

- 1 - Alerta: las ventas cayeron 52.6%. Revisa inventario, personal y
- 2 promociones con el gerente.
- 3 - La tienda está entre las 3 con menores ventas (lugar 8 de 10).
- 4 - El ticket promedio (\$1,003.90) es menor que el de la empresa

```

5 ($1,059.27): hay espacio para venta cruzada.
6 - Precaución: el mes solo tiene 5 transacciones; interpreta las
7 variaciones con cuidado.

```

El mismo archivo produce la versión HTML (con índice flotante) y la de Word (editable), sin cambiar una línea: basta con `output_format = "all"`.

8.12.7 Generar los diez reportes

Con la carpeta del curso como directorio de trabajo, en la terminal (o en la pestaña **Terminal** de RStudio):

```
1 Rscript reportes/generar_reportes.R 2023-12 pdf
```

Salida real del script

```

1 Generando 10 reportes del mes 2023-12 en formato pdf
2 [ 1/10] Sucursal Centro OK 2023-12_T01_sucursal-centro_2026-09-23.pdf
3 [ 2/10] Sucursal Norte OK 2023-12_T02_sucursal-norte_2026-09-23.pdf
4 [ 3/10] Sucursal Sur OK 2023-12_T03_sucursal-sur_2026-09-23.pdf
5 ...
6 [10/10] Sucursal Express OK 2023-12_T10_sucursal-express_2026-09-23.pdf
7
8 Listo: 10 reportes generados, 0 con error. Carpeta: .../resultados/reportes
9 Registro guardado en: .../resultados/reportes/registro_reportes.csv

```

En unos 20 segundos, los diez PDF están en `resultados/reportes/`. Si ejecutas el script para un mes sin datos, por ejemplo `Rscript reportes/generar_reportes.R 2024-01`, ningún reporte se genera, pero el script no se detiene en la primera tienda: recorre las diez, y el registro explica la causa:

resultados/reportes/registro_reportes.csv (extracto)

```

1 fecha_hora,tienda_id,tienda,mes,archivo,estado,detalle,segundos
2 2026-09-23 22:02:00,1,Sucursal Centro,2023-12,2023-12_T01_sucursal-centro_2026-09-23.
  pdf,OK,OK,2.2
3 ...
4 2026-09-23 22:02:28,1,Sucursal Centro,2024-01,2024-01_T01_sucursal-centro_2026-09-23.
  html,ERROR,ERROR: No hay datos del mes 2024-01,0.4

```

Hazlo tú

Genera los reportes de junio de 2023 en Word para todas las tiendas. Abre el de la tienda que tuvo el mayor crecimiento y comprueba que su conclusión dice “Crecimiento sólido”. Luego revisa `registro_reportes.csv`: ¿cuántas filas tiene ahora y por qué?

8.13 Errores comunes

Estos son los errores que casi todo el mundo encuentra en sus primeros reportes. Los mensajes son los que muestra R (con las rutas acortadas).

▲ Cuidado

Archivo de datos no encontrado al hacer Knit. El código funciona en la consola, pero al tejer aparece:

```
1 Quitting from lines 7-9 [cargar] (reporte.Rmd)
2 Error:
3 ! 'datasets/ventas_retail.csv' does not exist in current working
4   directory ('.../curso-bi-rstudio/reportes').
```

Causa: el .Rmd se ejecuta en su propia carpeta (reportes/), no en la del proyecto. **Solución:** usa `knit_root_dir` en `render()` o **Knit > Knit Directory > Project Directory**, una ruta `../datasets/`, el paquete **here** o la búsqueda robusta con el parámetro `ruta_datos` (sección 8.7.4). El mensaje te dice cuál es el directorio de trabajo real: léelo.

▲ Cuidado

Objeto no encontrado (pero en la consola sí existe).

```
1 Quitting from lines 7-8 [resumen] (reporte.Rmd)
2 Error:
3 ! objeto 'ventas' no encontrado
```

Causa: Knit ejecuta el documento en una *sesión nueva y limpia*. El objeto `ventas` existe en tu consola porque lo creaste ahí, pero el .Rmd nunca lo crea. **Solución:** el .Rmd debe leer sus datos y cargar sus paquetes (`library()`) él mismo. Prueba: reinicia R (**Session > Restart R**) y ejecuta todos los bloques en orden con **Run > Run All**; si falla ahí, fallará al tejer. Lo mismo ocurre con `could not find function "%>%"`: falta un `library(dplyr)` dentro del documento.

▲ Cuidado

Falta LaTeX para generar PDF.

```
1 ! sh: 1: pdflatex: not found
2
3 Error: LaTeX failed to compile reporte.tex. See
4 https://yihui.org/tinytex/r/#debugging for debugging tips.
```

En Windows el mensaje suele ser *“No LaTeX installation detected (LaTeX is required to create PDF output)”*. **Solución:** `tinytex::install_tinytex()` y reinicia RStudio. Si ya tienes TinyTeX

y el error menciona un archivo `.sty` faltante, ejecuta de nuevo el render con Internet para que se descargue.

▲ Cuidado

Nombre de chunk duplicado.

```
1 Error in parse_block(g[-1], g[1], params.src, markdown_mode) :
2   Duplicate chunk label 'grafica', which has been used for the chunk:
3   plot(1:10)
```

Causa: dos bloques tienen el mismo nombre (pasa mucho al copiar y pegar un bloque). **Solución:** cada nombre debe ser único; renombra el segundo (`grafica-tiendas`).

▲ Cuidado

Error de YAML por sangría.

```
1 Error in yaml::yaml.load(..., eval.expr = TRUE) :
2   Scanner error: mapping values are not allowed in this context at
3   line 5, column 11
```

El encabezado que lo produjo tenía un espacio de más antes de `theme`:

```
1 output:
2   html_document:
3     toc: true
4     theme: flatly
```

Solución: alinea las opciones del mismo nivel exactamente con la misma sangría (múltiplos de dos espacios, sin tabuladores). El mensaje indica la línea y columna *dentro del YAML*. Otros errores de YAML frecuentes: olvidar los dos puntos, o escribir un título con dos puntos sin comillas (`title: Ventas: diciembre`).

▲ Cuidado

Modificar un parámetro. `params$tienda <- 5` dentro del documento produce “*no se puede cambiar el valor de un vínculo bloqueado para ‘params’*”. Copia el parámetro a otra variable.

Tabla 8.5. Otros problemas frecuentes y su solución

| Síntoma | Solución |
|--|---|
| El reporte sale con mensajes de carga de paquetes y advertencias | <code>message = FALSE, warning = FALSE</code> en <code>opts_chunk\$set()</code> |
| Cada reporte del bucle sobrescribe al anterior | Da un <code>output_file</code> distinto en cada vuelta |
| La tabla sale como texto de consola (con <code>##</code>) | Usa <code>kable()</code> en lugar de imprimir el data frame |
| El texto de <code>cat()</code> aparece en un recuadro gris | Agrega <code>results = "asis"</code> al bloque |
| Los meses salen en inglés | Usa un vector propio con los nombres en español |
| Los números de un reporte “se parecen” a los de otra tienda | Usa <code>envir = new.env()</code> en <code>render()</code> |

Resumen del módulo

- Un **reporte reproducible** se genera por completo desde los datos y el código: sin copiar y pegar, sin errores de transcripción, y se actualiza con un clic.
- Un `.Rmd` tiene **encabezado YAML**, **texto en Markdown**, **chunks** de código y **código en línea**. **knitr** ejecuta el código y **pandoc** lo convierte a HTML, PDF (vía LaTeX) o Word.
- Las **opciones de chunk** controlan qué se ve: `echo`, `include`, `message`, `fig.cap`, `results = "asis"`; se fijan para todo el documento con `knitr::opts_chunk$set()`.
- `kable()` convierte data frames en tablas de documento; **scales** da formato de negocio a números y porcentajes.
- El **texto dinámico** y las **conclusiones automáticas** (`if/else`, `case_when()`) hacen que la narrativa cambie con los datos.
- Los **parámetros** (`params`) convierten un `.Rmd` en una plantilla; `render()` con `params`, `output_file` y `knit_root_dir` genera cada versión.
- Un script con un bucle, `tryCatch()` y un registro genera todos los reportes; **cron** o el **Programador de tareas** lo ejecutan solos, y **blastula** los envía por correo.
- **writexl** crea libros de Excel con varias hojas a partir de una lista de data frames. **Quarto** (`.qmd`) es el sucesor de R Markdown y usa casi la misma sintaxis.

Tabla 8.6. Funciones y opciones clave del Módulo 8

| Función u opción | Para qué sirve |
|--|--|
| <code>rmarkdown::render()</code> | Genera el reporte desde código (lo que hace el botón Knit) |
| <code>params = list(...)</code> | Valores de los parámetros del reporte |
| <code>knit_root_dir, output_dir, output_file</code> | Carpeta de ejecución, carpeta y nombre de salida |
| <code>knitr::opts_chunk\$set()</code> | Opciones por defecto de todos los chunks |
| <code>echo, include, eval, message, warning</code> | Qué se muestra y qué se ejecuta |
| <code>fig.cap, fig.width, fig.height, out.width</code> | Título y tamaño de las figuras |
| <code>results = "asis"</code> | El texto impreso se trata como Markdown |
| <code>knitr::kable()</code> | Tabla con formato de documento |
| <code>knitr::knit()</code> | Teje un documento (o un texto) a Markdown |
| <code>knitr::pandoc_to()</code> | Formato de salida que se está generando |
| <code>dollar(), percent(), comma()</code> | Formato de números en texto y tablas |
| <code>tryCatch()</code> | Atrapa errores para que el bucle continúe |
| <code>sprintf()</code> | Arma nombres de archivo con un molde |
| <code>expand_grid()</code> | Todas las combinaciones de parámetros |
| <code>commandArgs()</code> | Lee los argumentos de Rscript |
| <code>writexl::write_xlsx()</code> | Excel con una hoja por data frame |
| <code>readxl::excel_sheets()</code> | Nombres de las hojas de un Excel |
| <code>tinytex::install_tinytex()</code> | Instala LaTeX para generar PDF |

8.14 Ejercicios

Para los ejercicios que piden un archivo `.Rmd`, guárdalo en la carpeta **reportes/** del curso y généralo con **Knit** o con `rmarkdown::render()`. Los demás se resuelven con código R en la consola, con la carpeta del curso como directorio de trabajo.

Ejercicio 8.1: Tu primer reporte de clientes

Nivel: **Básico**

Crea **reportes/ej1-clientes.Rmd** con salida HTML que: (a) tenga título, tu nombre y la fecha automática; (b) lea **clientes.csv** en un bloque que no se muestre en el reporte; (c) diga en una frase, con código en línea, cuántos clientes hay y cuántos son premium; (d) muestre una tabla con `kable()` del número de clientes por **ciudad**, ordenada de mayor a menor.

 Solución en la página 694

Ejercicio 8.2: Tabla ejecutiva de categorías

Nivel: **Básico**

Con **transacciones.csv** y **productos.csv**, construye una tabla con `kable(format = "pipe")` de las ventas de 2023 por **categoría**: ventas en pesos sin centavos, unidades con separador de miles y porcentaje del total con un decimal, ordenada de mayor a menor y con el título “Ventas 2023 por categoría”. ¿Qué categoría vende más y qué porcentaje representa?

 Solución en la página 696

Ejercicio 8.3: Frase de cumplimiento de metaNivel: **Intermedio**

Escribe una función `frase_meta(ventas, meta)` que devuelva: “Se superó la meta en X % (ventas de \$A contra meta de \$B).” si `ventas >= meta`; “Se alcanzó el X % de la meta...” si se alcanzó al menos el 90 %; y “Alerta: solo se alcanzó el X % de la meta...” en otro caso. Pruébala con los tres casos y aplícala a las ventas trimestrales de 2023 (de `transacciones.csv`) con una meta de \$220,000 por trimestre.

 Solución en la página 697**Ejercicio 8.4: Opciones de chunk para un reporte ejecutivo**Nivel: **Intermedio**

Crea `reportes/ej4-opciones.Rmd` con un bloque de configuración global que oculte código, mensajes y advertencias. Incluye: un bloque invisible que calcule las ventas por mes de `ventas_retail.csv`; una gráfica de líneas con `fig.cap`, 7×3 pulgadas y `out.width = "80%"`; y un bloque con `results = "asis"` que escriba como lista de Markdown los tres meses con más ventas. Génalo en HTML y en Word.

 Solución en la página 698**Ejercicio 8.5: Reporte parametrizado por categoría**Nivel: **Intermedio**

Crea `reportes/ej5-categoria.Rmd` con un parámetro `categoria` (valor por defecto “**Electrónica**”) que muestre, para esa categoría: ventas totales de 2023 (en una frase), una tabla de sus 5 productos más vendidos y una gráfica de ventas por mes. Después genera, con `render()`, los reportes de “**Ropa**” y “**Hogar**” con nombres de archivo distintos en `resultados/`.

 Solución en la página 699**Ejercicio 8.6: Plan de generación por trimestre**Nivel: **Avanzado**

Construye con `expand_grid()` la tabla de trabajos para generar un reporte por tienda y por trimestre (Q1 a Q4). Agrega una columna `archivo` con el formato `2023-Q1_T01_sucursal-centro.pdf` y una columna `n_transacciones` con cuántas transacciones tiene cada combinación (según `transacciones.csv`). ¿Cuántos reportes hay que generar? ¿Qué combinación tiene menos transacciones? Marca con `generar = FALSE` las que tengan menos de 20.

 Solución en la página 702**Ejercicio 8.7: Un Excel con una hoja por tienda**Nivel: **Avanzado**

Genera `resultados/ventas_por_tienda.xlsx` con una hoja "Resumen" (ventas y transacciones por tienda) seguida de una hoja por tienda con el detalle de sus transacciones, cuyo nombre sea el de la tienda sin la palabra "Sucursal" (por ejemplo, "Centro"). Pista: `split()` divide un data frame en una lista. Verifica con `excel_sheets()` que hay 11 hojas y que la suma de filas de las hojas de tiendas es 1,000.

 Solución en la página 703

Ejercicio 8.8: Sistema de alertas con registro

Nivel: **Reto**

Escribe una función `revisar_tienda(id, mes)` que calcule la variación de ventas de una tienda contra el mes anterior y devuelva un tibble de una fila con la tienda, el mes, la variación y un estado ("Crecimiento sólido", "Estable", "Ligera caída", "Alerta"), y que lance un error con `stop()` si la tienda o el mes no existen, o si el mes anterior no tiene datos. Recorre todas las tiendas para "2023-06", más las combinaciones inválidas `(11, "2023-06")` y `(1, "2023-01")`, atrapando los errores con `tryCatch()`. Muestra el registro y escribe con `cat()` el texto Markdown de alertas que iría en el reporte.

 Solución en la página 704

PARTE IV

Proyecto integrador

Proyecto final: TechRetail México

Al terminar este módulo podrás...

- Plantear un proyecto de Business Intelligence completo a partir de un reto de negocio real: preguntas, entregables, cronograma y criterios de evaluación.
- Construir un proceso ETL auditable: cargar varias fuentes, detectar problemas de calidad, documentar cada decisión de limpieza y armar un modelo en estrella listo para analizar.
- Calcular los KPIs de la empresa y convertir cada tabla o gráfica en un hallazgo de negocio con números reales.
- Aplicar segmentación de clientes y pronóstico de ventas siendo honesto sobre lo que los datos permiten (y lo que no).
- Diseñar el tablero ejecutivo, redactar el reporte para la dirección y preparar la presentación de resultados.
- Organizar el proyecto de forma reproducible y presentarlo en tu portafolio para buscar trabajo en BI.

Llegaste al final del curso. Ya sabes importar y transformar datos (Módulo 2), graficarlos con criterio (Módulo 3), explorarlos con estadística (Módulo 4), guardarlos en una base de datos (Módulo 5), construir tableros interactivos (Módulo 6), entrenar modelos (Módulo 7) y automatizar reportes (Módulo 8). Cada módulo resolvió una pieza. En la vida real nadie te pide “una pieza”: te piden *resolver un problema de negocio*, y tú decides qué herramientas usar, en qué orden y cómo comunicar lo que encontraste.

Este módulo simula exactamente eso. Vas a trabajar como consultor de BI para **TechRetail México**, una cadena de diez sucursales que tiene un año de datos y muchas preguntas. No vamos a re-enseñar técnicas: cada vez que uses algo que ya aprendiste, te remitiremos al módulo donde se explicó y nos concentraremos en lo que distingue a un analista junior de uno senior: el *razonamiento de negocio*, las *decisiones* (y su justificación) y la *comunicación* de resultados.

Una advertencia honesta antes de empezar: los datos del curso son simulados. Eso significa que algunos patrones que esperarías en una tienda real (más ventas en diciembre, fines de semana más fuertes, clientes premium que gastan más) pueden no aparecer, o aparecer por azar. Lejos de ser un problema, es un excelente entrenamiento: aprenderás a decir “los datos no muestran diferencia” con la misma seguridad con la que dirías “encontramos una oportunidad de un millón de pesos”. Un buen analista no fuerza conclusiones.

Concepto clave: Proyecto integrador de BI

Un **proyecto integrador** recorre el ciclo completo de la inteligencia de negocios: *pregunta de negocio* → *datos* (ETL) → *análisis* (KPIs, exploración, modelos) → *producto* (tablero, reporte) → *decisión* (presentación y recomendaciones). El valor no está en ninguna etapa aislada, sino en que la cadena completa lleve de un dato crudo a una acción concreta de la dirección.

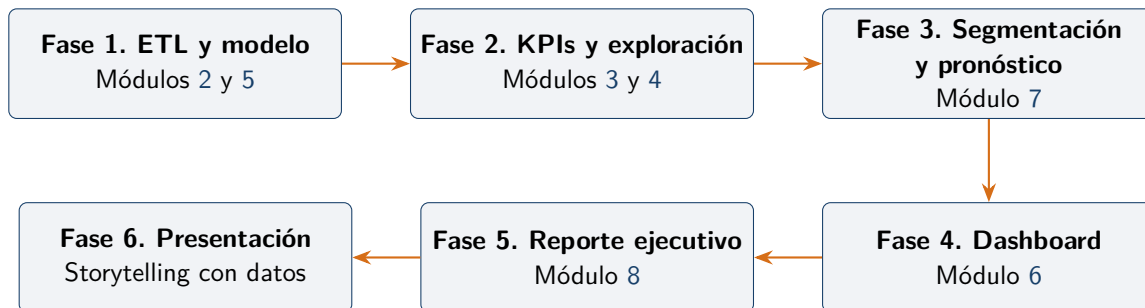


Figura 9.1. Las seis fases del proyecto y los módulos del libro que usa cada una.

9.1 El caso: TechRetail México

TechRetail México es una cadena minorista con diez sucursales en seis ciudades del país. Vende productos de diez categorías (desde electrónica y juguetes hasta alimentos y artículos para mascotas) y registra cada venta en un sistema de punto de venta. La dirección tiene los datos de 2023 pero nunca los ha analizado de forma integral: cada gerente de tienda lleva su propio Excel, y las juntas de resultados terminan en discusiones sobre “qué número es el bueno”.

Antes de escribir una sola línea de análisis, prepara el entorno del proyecto. Fíjate en la **fecha de corte**: es la fecha “de hoy” para el proyecto. El script original del curso usaba `Sys.Date()`, lo que hace que los resultados cambien cada día que lo ejecutas (la antigüedad de una tienda o los días desde la última compra de un cliente serían distintos mañana). En un proyecto profesional eso es inaceptable: quien revise tu trabajo dentro de seis meses debe obtener exactamente los mismos números.

Configuración del proyecto

```

1 # ---- Paquetes (todos se explicaron en módulos anteriores) ----
2 library(tidyverse) # dplyr, ggplot2, readr, tidyr, purrr, lubridate
3 library(scales)    # formatos de negocio: dollar(), percent(), comma()
4 library(DBI)       # interfaz a bases de datos (Módulo 5)
5 library(RSQLite)   # motor SQLite en un archivo local
6 library(cluster)   # silueta para evaluar k-means (Módulo 7)
7 library(forecast)   # modelos de pronóstico (Módulo 7)
8
9 # ---- Fecha de corte FIJA (nunca Sys.Date() en un proyecto) ----
10 fecha_corte <- as.Date("2024-01-01")
11

```

```

12 # ---- Carpetas de salida del proyecto ----
13 dir.create("resultados", showWarnings = FALSE)
14 dir.create("datos", showWarnings = FALSE)
15
16 # ---- Colores corporativos y tema (los mismos del Módulo 3) ----
17 color_principal <- "#2a78d6" # una sola serie
18 color_resalte <- "#eb6834" # lo que queremos destacar
19 color_gris <- "#b5b3ad" # el resto
20 paleta_libro <- c("#2a78d6", "#eb6834", "#1baf7a", "#eda100",
21                  "#e87ba4", "#008300", "#4a3aa7", "#e34948")
22
23 tema_libro <- function(base_size = 11) {
24   theme_minimal(base_size = base_size) +
25     theme(
26       plot.title = element_text(face = "bold", color = "#0b0b0b"),
27       plot.subtitle = element_text(color = "#52514e"),
28       plot.caption = element_text(color = "#52514e", size = rel(0.8)),
29       axis.title = element_text(color = "#52514e"),
30       axis.text = element_text(color = "#52514e"),
31       panel.grid.minor = element_blank(),
32       panel.grid.major = element_line(color = "#e6e5e1", linewidth = 0.3),
33       legend.position = "top",
34       legend.title = element_text(color = "#52514e")
35     )
36 }
37
38 # Formato de pesos sin decimales para textos y tablas
39 pesos <- function(x) dollar(x, accuracy = 1)

```

Al cargar **tidyverse** y **forecast** verás mensajes de bienvenida o avisos como **Registered S3 method overwritten**; son normales y no indican ningún error.

9.1.1 La cadena en números

Describe a la empresa con sus propios datos, no con adjetivos. El archivo `tiendas.csv` es el catálogo de sucursales.

```

1 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
2
3 # Antigüedad de cada sucursal calculada a la fecha de corte
4 tiendas %>%
5   mutate(anios = floor(time_length(
6     interval(fecha_apertura, fecha_corte), "years"))) %>%
7   select(tienda = nombre_tienda, ciudad, zona, tipo,
8         m2 = tamano_m2, empleados, anios) %>%
9   arrange(desc(m2))

```

```
# A tibble: 10 x 7
  tienda      ciudad zona tipo      m2 empleados anios
  <chr>      <chr>  <chr> <chr>    <dbl>    <dbl> <dbl>
1 Sucursal Premium CDMX     Sur  Premium    800      40    14
2 Sucursal Plaza A CDMX     Oeste Principal   700      35     7
3 Sucursal Sur Monterrey Norte Estándar    600      30    12
4 Sucursal Oeste Puebla  Centro Estándar    550      22    10
5 Sucursal Centro CDMX     Centro Principal   500      25    13
6 Sucursal Plaza B Guadalajara Norte Estándar    480      21     9
7 Sucursal Norte Guadalajara Norte Estándar    450      20    11
8 Sucursal Este CDMX     Este Estándar    400      18     8
9 Sucursal Outlet Tijuana Norte Outlet     350      15     5
10 Sucursal Express Querétaro Centro Express    300      12     3
```

```
1 # Resumen de la red de sucursales
2 tiendas %>%
3   summarise(sucursales = n(),
4             ciudades   = n_distinct(ciudad),
5             m2_totales = sum(tamano_m2),
6             empleados  = sum(empleados),
7             m2_por_empleado = round(sum(tamano_m2) / sum(empleados), 1))
8
9 tiendas %>% count(ciudad, sort = TRUE)
```

```
# A tibble: 1 x 5
  sucursales ciudades m2_totales empleados m2_por_empleado
  <int>    <int>    <dbl>    <dbl>    <dbl>
1      10      6    5130      238      21.6

# A tibble: 6 x 2
  ciudad      n
  <chr>    <int>
1 CDMX      4
2 Guadalajara 2
3 Monterrey  1
4 Puebla     1
5 Querétaro  1
6 Tijuana    1
```

¿Qué nos dice esto? TechRetail tiene 10 sucursales en 6 ciudades, con 5,130 m² de piso de venta y 238 empleados. La Ciudad de México concentra 4 sucursales, incluidas la más grande (Sucursal Premium, 800 m² y 40 empleados, la más antigua: 14 años) y la Sucursal Plaza A. En el otro extremo están la Sucursal Express de Querétaro (300 m², abierta en 2020) y la Sucursal Outlet de Tijuana. Hay cinco formatos (Principal, Estándar, Outlet, Premium y Express), lo cual será importante: no es justo comparar las ventas absolutas de una tienda de 800 m² con las de una de 300.

▲ Cuidado

Observa que la “Sucursal Sur” está en Monterrey y pertenece a la zona *Norte*. Los nombres heredados no siempre describen la realidad. En el análisis usaremos siempre la columna **zona**, nunca el nombre de la tienda, para agrupar geográficamente.

9.1.2 El reto de la dirección

En la empresa: El correo de la directora general

“Necesitamos dejar de discutir números y empezar a tomar decisiones. Quiero, en seis semanas: (1) una sola versión de la verdad de nuestras ventas de 2023, (2) entender qué tiendas, productos y clientes nos hacen ganar dinero, (3) un tablero que los gerentes consulten cada lunes y (4) una recomendación clara de en qué enfocarnos en 2024. Preséntalo al comité directivo en 20 minutos.”

Traducido a BI: la dirección pide **calidad y gobierno de datos** (una sola versión de la verdad), **análisis descriptivo y diagnóstico** (qué pasó y por qué), un **producto de datos recurrente** (tablero) y **recomendaciones accionables** (presentación y reporte).

9.1.3 Las 20 preguntas de negocio

El primer entregable de cualquier consultoría es la lista de preguntas que vas a contestar, acordada con el cliente. Evita el “análisis de todo”: si una tabla no responde una pregunta, sobra. La Tabla 9.1 agrupa las 20 preguntas en cuatro áreas e indica dónde se responde cada una en este módulo.

Tabla 9.1. Las 20 preguntas de negocio del proyecto y dónde se responden.

| # | Pregunta | Dónde se responde |
|--------------------|---|-------------------|
| Ventas | | |
| 1 | ¿Cuánto vendimos en 2023, en cuántas transacciones y con qué ticket promedio? | Sec. 9.3.1 |
| 2 | ¿Cómo evolucionaron las ventas mes a mes? ¿Estamos creciendo? | Sec. 9.3.2 |
| 3 | ¿Qué categorías generan más ingresos y más margen? | Sec. 9.3.3 |
| 4 | ¿Qué método de pago prefieren los clientes y cambia el ticket? | Sec. 9.3.5 |
| 5 | ¿Qué ventas podemos esperar en las próximas semanas? | Sec. 9.4.2 |
| Clientes | | |
| 6 | ¿Cuántos clientes activos tenemos y cuántos no compraron? | Sec. 9.3.1 |
| 7 | ¿Los clientes premium gastan más que los demás? | Sec. 9.3.9 |
| 8 | ¿Qué segmentos de clientes existen según su comportamiento? | Sec. 9.4.1 |
| 9 | ¿Cuántos clientes vuelven a comprar y cada cuánto? | Reto 9.1 |
| 10 | ¿Se comportan distinto los clientes según el mes en que llegaron? | Reto 9.4 |
| Productos | | |
| 11 | ¿Pocos productos concentran la mayoría de las ventas (Pareto)? | Sec. 9.3.7 |
| 12 | ¿Qué productos tienen margen negativo en el catálogo? | Sec. 9.2.3 |
| 13 | ¿Vendemos al precio de catálogo? | Sec. 9.2.3 |
| 14 | ¿Se venden productos marcados como inactivos? | Sec. 9.2.3 |
| 15 | ¿Qué productos necesitan reabastecerse pronto? | Reto 9.3 |
| Operaciones | | |
| 16 | ¿Qué sucursales y zonas venden más? | Sec. 9.3.4 |
| 17 | ¿Qué sucursales son más productivas por m ² y por empleado? | Reto 9.2 |
| 18 | ¿Qué días y horas concentran las ventas? | Sec. 9.3.6 |
| 19 | ¿Qué vendedores venden más y se relaciona con su salario? | Sec. 9.3.8 |
| 20 | ¿Hay transacciones atípicas que debamos revisar? | Reto 9.5 |

**Consejo**

Escribe las preguntas *antes* de abrir los datos y pídele a la dirección que las priorice. Si al final del proyecto alguien pregunta “¿y esto para qué sirve?”, debes poder señalar qué pregunta responde cada gráfica.

9.1.4 Entregables y cronograma

El proyecto se organiza en seis fases con un entregable concreto cada una (Tabla 9.2). Las horas son una estimación para alguien que termina este libro; en una empresa real, la Fase 1 suele consumir más tiempo del que todos esperan.

Tabla 9.2. Cronograma del proyecto por fases.

| Fase | Actividades principales | Entregable | Semana | Horas |
|-----------------------|--|---|--------|-------|
| 1. ETL y modelo | Cargar fuentes, auditar calidad, limpiar, modelo en estrella | Datos limpios (RDS y SQLite) y bitácora de decisiones | 1 | 5–6 |
| 2. KPIs y exploración | KPIs globales, tendencias, cortes por tienda, producto, cliente y vendedor | Diccionario de KPIs y cuaderno de hallazgos | 2 | 5–6 |
| 3. Modelos | Segmentación RFM con k-means y pronóstico de ventas | Segmentos con acciones y pronóstico con intervalos | 3 | 4–5 |
| 4. Dashboard | Diseño y construcción del tablero en Shiny | App funcionando con filtros | 4 | 4–5 |
| 5. Reporte | Reporte ejecutivo automatizado en R Markdown | PDF o HTML de 1–2 páginas | 5 | 2–3 |
| 6. Presentación | Guion, diapositivas y ensayo | Presentación de 8–10 diapositivas | 6 | 2–3 |

9.1.5 Rúbrica de evaluación

Si usas este proyecto para un curso, un bootcamp o tu portafolio, evalúalo con criterios explícitos (Tabla 9.3). Observa que el código vale menos de la mitad: en BI, un análisis técnicamente perfecto que nadie entiende no sirve.

Tabla 9.3. Rúbrica de evaluación del proyecto final.

| Criterio | Peso | Excelente | Insuficiente |
|---------------------------|------|--|---|
| Calidad de datos | 15 % | Auditoría con evidencia, decisiones documentadas y justificadas | Se limpian datos “a ojo” o se borran filas sin explicar |
| Código y reproducibilidad | 20 % | Scripts numerados, comentados, fecha de corte fija; corre de principio a fin | Rutas absolutas, <code>Sys.Date()</code> , pasos manuales |
| Análisis y KPIs | 25 % | Cada gráfica responde una pregunta y trae un hallazgo con números | Muchas gráficas sin conclusión |
| Modelos | 10 % | Modelo justificado, validado y con limitaciones explícitas | Modelo sin validar o con conclusiones exageradas |
| Tablero y reporte | 15 % | Tablero claro con KPIs y filtros; reporte breve y automatizado | Tablero saturado; reporte copiado a mano |
| Comunicación | 15 % | Historia clara, recomendaciones accionables con impacto | Lista de datos sin recomendación |

Hazlo tú

Agrega tres preguntas propias a la Tabla 9.1 (por ejemplo, sobre marcas o proveedores). Para cada una anota qué archivo y qué columnas necesitarías para responderla.

9.2 Fase 1: ETL y modelo de datos

La primera fase construye la “única versión de la verdad” que pidió la directora. Seguimos las tres letras del **ETL**: *extraer* (leer las fuentes), *transformar* (auditar, limpiar, enriquecer) y *cargar* (guardar el resultado en un formato listo para analizar). Las funciones de **dplyr** y **readr** se explicaron en el Módulo 2 y las bases de datos en el Módulo 5; aquí nos enfocamos en las decisiones.

9.2.1 Extraer: cargar las fuentes

```

1 # Tabla de hechos candidata principal: una fila por transacción
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4 # Segunda fuente de ventas (una venta registrada por día)
5 ventas_retail <- read_csv("datasets/ventas_retail.csv",
6                           show_col_types = FALSE)
7 # Catálogos (futuras dimensiones)
8 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
9 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
10 empleados <- read_csv("datasets/empleados.csv", show_col_types = FALSE)
11 # tiendas ya se cargó en la sección anterior
12
13 glimpse(transacciones)

```

```

Rows: 1,000
Columns: 14
$ transaccion_id    <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ~
$ fecha             <date> 2023-01-13, 2023-01-09, 2023-02-20, 2023-~
$ hora              <time> 13:15:00, 10:45:00, 10:30:00, 10:45:00, 1~
$ cliente_id        <dbl> 155, 155, 75, 51, 173, 135, 90, 168, 13, 9~
$ producto_id       <dbl> 3, 47, 6, 17, 28, 46, 30, 48, 21, 12, 49, ~
$ cantidad          <dbl> 4, 4, 3, 4, 1, 2, 4, 2, 2, 5, 2, 3, 4, 2, ~
$ precio_venta      <dbl> 303.97, 58.07, 275.46, 105.28, 205.47, 534~
$ metodo_pago       <chr> "Tarjeta Crédito", "Tarjeta Débito", "Tarj~
$ tienda_id         <dbl> 10, 2, 7, 6, 7, 7, 8, 8, 5, 2, 7, 6, 8, 2, ~
$ vendedor_id       <dbl> 4, 22, 14, 5, 14, 22, 23, 5, 3, 14, 5, 9, ~
$ total_transaccion <dbl> 1215.88, 232.28, 826.38, 421.12, 205.47, 1~
$ mes               <chr> "2023-01", "2023-01", "2023-02", "2023-05"~
$ trimestre         <chr> "Q1", "Q1", "Q1", "Q2", "Q1", "Q1", "Q2", ~
$ dia_semana        <chr> "viernes", "lunes", "lunes", "viernes", "m~

```

Fíjate en un detalle: en el archivo la hora viene como texto sin cero inicial ("8:00", "13:15"), pero **readr** la reconoció como hora (<time>). Eso nos permitirá extraer la hora del día con **hour()** sin conversiones manuales.

9.2.2 ¿Qué tabla usar como hechos?

Tenemos *dos* archivos de ventas. Antes de sumar nada, compáralos: si fueran el mismo registro visto de dos formas, sumarlos duplicaría las ventas; si son registros distintos, necesitamos saber cuál es el oficial.

```

1  # Perfil rápido de cada fuente de ventas
2  perfil_fuente <- function(datos, nombre, col_total) {
3    tibble(fuente    = nombre,
4           filas     = nrow(datos),
5           columnas  = ncol(datos),
6           desde     = min(datos$fecha),
7           hasta     = max(datos$fecha),
8           ventas    = sum(datos[[col_total]]))
9  }
10
11 bind_rows(
12   perfil_fuente(transacciones, "transacciones", "total_transaccion"),
13   perfil_fuente(ventas_retail, "ventas_retail", "total")
14 )
15
16 # ¿Qué información tiene cada fuente que la otra no?
17 setdiff(names(transacciones), names(ventas_retail))
18 setdiff(names(ventas_retail), names(transacciones))
19
20 # ¿Coinciden registros (misma fecha, cliente y producto)?
21 inner_join(transacciones, ventas_retail,
22            by = c("fecha", "cliente_id", "producto_id")) %>%
23   nrow()

```

```

# A tibble: 2 x 6
  fuente      filas columnas desde      hasta      ventas
  <chr>      <int>   <int> <date>   <date>   <dbl>
1 transacciones 1000     14 2023-01-01 2023-12-31 959993.
2 ventas_retail  365     14 2023-01-01 2023-12-31 513917.
[1] "transaccion_id"      "hora"      "precio_venta"
[4] "metodo_pago"         "total_transaccion"
[1] "precio_unitario"     "descuento_pct"  "subtotal"
[4] "descuento_monto"     "total"
[1] 0

```

Las dos fuentes cubren todo 2023, pero no son el mismo registro: ninguna combinación de fecha, cliente y producto coincide entre ellas, y los totales son muy distintos (\$959,993 contra \$513,917). **Decisión:** usamos `transacciones.csv` como tabla de hechos porque (1) es más granular (1,000 transacciones contra 365 registros, uno por día), (2) trae información que la otra no tiene y que la dirección pidió (hora, método de pago) y (3) su total es consistente con su detalle (lo comprobaremos en la auditoría). `ventas_retail.csv` queda como fuente secundaria: es la única que registra descuentos, así que la

usaremos solo para preguntas sobre promociones (Reto 9.6). **Nunca sumaremos ambas fuentes.**

▲ Cuidado

Mezclar fuentes de ventas es el error más caro de un proyecto de BI: el tablero muestra un total que nadie reconoce y el proyecto pierde credibilidad en la primera junta. Si dos sistemas registran ventas, documenta cuál es el oficial para cada pregunta y dilo explícitamente en el reporte.

9.2.3 Auditoría de calidad de datos

Una **auditoría de calidad** revisa sistemáticamente cada tabla antes de usarla: tamaño, faltantes, duplicados, rangos, integridad de llaves y reglas de negocio. Conviértela en una función para aplicarla igual a todas las tablas (y reutilizarla cada mes cuando lleguen datos nuevos).

```

1  # Auditoría básica de una tabla: tamaño, faltantes y duplicados
2  auditar <- function(datos, nombre, llave) {
3    tibble(
4      tabla      = nombre,
5      filas      = nrow(datos),
6      columnas   = ncol(datos),
7      celdas_na  = sum(is.na(datos)),          # valores faltantes
8      filas_dup  = sum(duplicated(datos)),      # filas idénticas
9      llaves_dup = sum(duplicated(datos[[llave]])) # llave repetida
10   )
11 }
12
13 auditoria <- bind_rows(
14   auditar(transacciones, "transacciones", "transaccion_id"),
15   auditar(clientes,      "clientes",      "cliente_id"),
16   auditar(productos,     "productos",     "producto_id"),
17   auditar(tiendas,       "tiendas",       "tienda_id"),
18   auditar(empleados,     "empleados",     "empleado_id")
19 )
20 auditoria

```

```

# A tibble: 5 x 6
  tabla      filas columnas celdas_na filas_dup llaves_dup
  <chr>      <int>   <int>   <int>   <int>   <int>
1 transacciones 1000     14      0      0      0
2 clientes       200     10      0      0      0
3 productos      50     13      0      0      0
4 tiendas        10      8      0      0      0
5 empleados     100     12      0      0      0

```

Buenas noticias en lo estructural: ninguna tabla tiene valores faltantes, filas duplicadas ni llaves

repetidas. Eso no significa que los datos sean correctos; significa que están *completos*. Sigue con los rangos y la consistencia interna de la tabla de hechos.

```

1 # Rangos de las columnas clave de la tabla de hechos
2 rango <- function(x) str_c(min(x), " a ", max(x))
3 transacciones %>%
4   summarise(fecha      = rango(fecha),
5             hora       = rango(hms::as_hms(hora)),
6             cantidad   = rango(cantidad),
7             precio     = rango(precio_venta),
8             total      = rango(total_transaccion)) %>%
9   pivot_longer(everything(), names_to = "columna",
10               values_to = "rango")
11
12 # ¿El total cuadra con cantidad x precio? (tolerancia de 1 centavo)
13 transacciones %>%
14   summarise(totales_incorrectos =
15             sum(abs(total_transaccion - cantidad * precio_venta) > 0.01))

```

```

# A tibble: 5 x 2
  columna rango
  <chr>    <chr>
1 fecha    2023-01-01 a 2023-12-31
2 hora     28800 a 74700
3 cantidad 1 a 5
4 precio   20.14 a 598.81
5 total    22.65 a 2992.35
# A tibble: 1 x 1
  totales_incorrectos
  <int>
1                   0

```

Las fechas cubren el año completo, las horas van de 8:00 a 20:45 (horario de tienda razonable), no hay cantidades ni precios negativos y los 1,000 totales cuadran con su detalle. Ahora revisa la **integridad referencial**: toda llave de la tabla de hechos debe existir en su catálogo. Para eso `anti_join()` es ideal: devuelve las filas de la primera tabla que *no* encuentran pareja en la segunda (Módulo 2).

```

1 # Llaves huérfanas: transacciones cuyo id no existe en el catálogo
2 tibble(
3   llave = c("cliente_id", "producto_id", "tienda_id", "vendedor_id"),
4   huérfanas = c(
5     nrow(anti_join(transacciones, clientes, by = "cliente_id")),
6     nrow(anti_join(transacciones, productos, by = "producto_id")),
7     nrow(anti_join(transacciones, tiendas, by = "tienda_id")),
8     nrow(anti_join(transacciones, empleados,

```

```

9         by = c("vendedor_id" = "empleado_id"))
10     )
11 )
12
13 # En sentido contrario: elementos del catálogo sin ninguna venta
14 clientes_sin_compra <- anti_join(clientes, transacciones,
15                                 by = "cliente_id")
16 clientes_sin_compra %>% select(cliente_id, nombre, fecha_registro)
17 nrow(anti_join(productos, transacciones, by = "producto_id"))

```

```

# A tibble: 4 x 2
  llave      huérfanas
  <chr>      <int>
1 cliente_id      0
2 producto_id      0
3 tienda_id       0
4 vendedor_id      0
# A tibble: 2 x 3
  cliente_id nombre      fecha_registro
  <dbl> <chr>      <date>
1      171 Rosa Jiménez  2022-08-27
2      199 Ricardo Hernández 2023-09-16
[1] 0

```

No hay llaves huérfanas: cada transacción apunta a un cliente, producto, tienda y vendedor que existen. En sentido contrario, 2 de los 200 clientes registrados no compraron en 2023 (los usaremos para distinguir “clientes registrados” de “clientes activos”) y los 50 productos tuvieron al menos una venta.

La parte más valiosa de la auditoría son las **reglas de negocio**: condiciones que los datos deberían cumplir según cómo funciona la empresa. Aquí es donde aparecen los hallazgos interesantes.

```

1 # Unimos a cada transacción los datos del producto y del cliente
2 revision <- transacciones %>%
3   left_join(productos, by = "producto_id") %>%
4   left_join(select(clientes, cliente_id, fecha_registro),
5             by = "cliente_id")
6
7 vendedores_ids <- unique(transacciones$vendedor_id)
8
9 hallazgos <- tibble(
10   regla = c(
11     "Productos con margen de catálogo negativo",
12     "Transacciones con precio menor al costo",
13     "Transacciones a +/-20% del precio de catálogo",
14     "Productos inactivos que sí se vendieron",
15     "Transacciones de productos inactivos",

```

```

16   "Transacciones anteriores al registro del cliente",
17   "Clientes con compras antes de registrarse",
18   "Vendedores que no son del depto. de Ventas"),
19   casos = c(
20     sum(productos$margen_pct < 0),
21     sum(revision$precio_venta < revision$costo),
22     sum(abs(revision$precio_venta / revision$precio_catalogo - 1)
23         <= 0.20),
24     n_distinct(revision$producto_id[!revision$activo]),
25     sum(!revision$activo),
26     sum(revision$fecha < revision$fecha_registro),
27     n_distinct(revision$cliente_id[revision$fecha <
28                 revision$fecha_registro]),
29     empleados %>%
30       filter(empleado_id %in% vendedores_ids,
31             departamento != "Ventas") %>% nrow()
32   )
33 hallazgos

```

```

# A tibble: 8 x 2
  regla                                casos
  <chr>                                <int>
1 Productos con margen de catálogo negativo      10
2 Transacciones con precio menor al costo      252
3 Transacciones a +/-20% del precio de catálogo  149
4 Productos inactivos que sí se vendieron         7
5 Transacciones de productos inactivos          137
6 Transacciones anteriores al registro del cliente 114
7 Clientes con compras antes de registrarse       34
8 Vendedores que no son del depto. de Ventas      21

```

Cada renglón es una conversación con el área de negocio. Revisemos los más importantes con detalle.

```

1  # Detalle: productos cuyo costo supera al precio de catálogo
2  productos %>%
3    filter(margen_pct < 0) %>%
4    select(producto_id, categoria, precio_catalogo, costo, margen_pct,
5           activo) %>%
6    arrange(margen_pct)
7
8  # ¿Qué tan lejos está el precio de venta del de catálogo?
9  revision %>%
10    mutate(razon_precio = precio_venta / precio_catalogo) %>%
11    summarise(minimo = min(razon_precio),
12              mediana = median(razon_precio),
13              maximo = max(razon_precio))
14

```

```

15 # ¿A qué departamento pertenecen los vendedores según Recursos Humanos?
16 empleados %>%
17   filter(empleado_id %in% vendedores_ids) %>%
18   count(departamento, sort = TRUE)

```

```

# A tibble: 10 x 6
  producto_id categoria    precio_catalogo costo margen_pct activo
    <dbl> <chr>          <dbl> <dbl>    <dbl> <lgl>
1         5 Alimentos      25.5 355.    -1292. TRUE
2        46 Automotriz      64.5 178.    -175. FALSE
3        32 Hogar        143. 210.    -46.7 TRUE
4        26 Electrónica    115. 159.    -38.3 TRUE
5        29 Belleza       298. 392.    -31.6 TRUE
6         9 Hogar        250. 328.    -31.2 TRUE
7         2 Belleza       313. 399.    -27.7 TRUE
8        12 Mascotas      289. 369.    -27.4 TRUE
9        40 Electrónica     50.1 58.2    -16.0 TRUE
10       13 Ropa         348. 393.    -12.8 FALSE

# A tibble: 1 x 3
  minimo mediana maximo
    <dbl>   <dbl>   <dbl>
1 0.0288 0.750 23.4

# A tibble: 7 x 2
  departamento    n
    <chr>      <int>
1 Atención al Cliente 10
2 RRHH                4
3 Ventas               4
4 Marketing            3
5 Finanzas             2
6 IT                   1
7 Operaciones          1

```

Esto es lo que encontró la auditoría, traducido a lenguaje de negocio:

- **Márgenes negativos en el catálogo.** 10 de los 50 productos cuestan más de lo que dice su precio de lista. El caso extremo es el producto 5 (Alimentos): precio de \$25.47 y costo de \$354.63, un margen de $-1,292\%$. Casi seguro es un error de captura (un costo por caja registrado como costo unitario, por ejemplo), pero debe confirmarlo Compras.
- **El precio de venta no respeta el catálogo.** Solo 15% (149) de las transacciones se vendió a menos de 20% de distancia del precio de lista; hay ventas a 3% del precio de catálogo y otras a 23 veces ese precio. O el catálogo está desactualizado, o el punto de venta permite capturar cualquier precio. En cualquier caso, *el margen que calculemos con el costo del catálogo será una estimación*, y en 252 transacciones (una de cada cuatro) el precio cobrado quedó por debajo del costo.
- **Productos inactivos con ventas.** Los 7 productos marcados como inactivos suman 137

transacciones en 2023. Lo más probable es que la bandera **activo** describa el estado *actual* (productos discontinuados después), no el de todo el año.

- **Compras antes del registro.** 114 transacciones de 34 clientes ocurrieron antes de su fecha de registro. La fecha de registro no es confiable para medir antigüedad; usaremos la fecha de la primera compra.
- **Los “vendedores” no son de Ventas.** De los 25 **vendedor_id**, solo 4 corresponden a empleados del departamento de Ventas según Recursos Humanos. O la llave **vendedor_id** no es el **empleado_id**, o la tabla de empleados está desactualizada. Además, la columna **sucursal** de empleados usa nombres (“Matriz”, “Sucursal Norte”...) que no corresponden a las 10 tiendas, así que no podemos asignar empleados a tiendas con esa tabla.

En la empresa: La auditoría también es un entregable

En una consultoría real, la lista de hallazgos de calidad suele ser lo primero que se presenta, y muchas veces es lo más valorado: la dirección descubre que su catálogo tiene costos imposibles o que el punto de venta deja cobrar cualquier precio. Presenta cada hallazgo con su evidencia (el número de casos), su impacto y un responsable de corregirlo.

9.2.4 Decisiones de limpieza

Limpiar no es “borrar lo raro”. Cada decisión se documenta con su justificación en una **bitácora de decisiones** (Tabla 9.4), que acompaña al proyecto y se entrega a la dirección.

Tabla 9.4. Bitácora de decisiones de limpieza.

| Problema | Decisión | Justificación |
|---|---|---|
| Dos fuentes de ventas distintas | Hechos = transacciones.csv ; ventas_retail.csv solo para descuentos | Más granular, con hora y método de pago; sumar ambas duplicaría ventas |
| 10 productos con margen negativo | Se conservan y se marcan con margen_negativo | Son ventas reales; el error está en el costo y lo debe corregir Compras |
| Precio de venta distinto al de catálogo | El ingreso se calcula con el precio cobrado; el margen se reporta como “estimado” | El precio cobrado es lo que entró a caja; el costo del catálogo es dudoso |
| Ventas por debajo del costo | Se conservan con la bandera bajo_costo | Pueden ser liquidaciones o errores; se reportan para revisión |
| Productos inactivos con ventas | Se conservan con la bandera producto_inactivo | La bandera describe el estado actual, no el de 2023 |
| Compras antes del registro | No se usa fecha_registro para antigüedad; se usa la primera compra | La fecha de registro no es confiable |
| Vendedores fuera del depto. de Ventas | Se conserva vendedor_id ; el análisis de salario se marca como preliminar | La llave no está validada con Recursos Humanos |
| Datos personales de clientes | No se copian email ni teléfono al modelo | Minimizar datos personales: el análisis no los necesita |

 Consejo

Observa que ninguna decisión borra filas. Borrar transacciones cambiaría el total de ventas y abriría la discusión de “qué número es el bueno” que la directora quiere terminar. *Marcar* con banderas (columnas lógicas) te permite excluir casos en un análisis específico sin alterar la verdad contable.

9.2.5 Transformar: modelo en estrella

Organizamos los datos en un **modelo en estrella**: una **tabla de hechos** con las medidas (cantidad, total, costo, margen) y las llaves, rodeada de **tablas de dimensiones** que describen el contexto (quién compró, qué, dónde, cuándo y quién vendió). Es el mismo diseño que usan Power BI, Tableau y los data warehouses corporativos (Módulo 5). Empezamos por las dimensiones.

```

1 # ---- Dimensión cliente (sin datos personales) ----
2 dim_cliente <- clientes %>%
3   select(cliente_id, nombre, edad, genero, ciudad, segmento_edad = segmento,
4          es_premium, fecha_registro) %>%
5   mutate(con_compras = cliente_id %in% transacciones$cliente_id)
6
7 # ---- Dimensión producto (con banderas de calidad) ----
8 dim_producto <- productos %>%
9   select(producto_id, nombre_producto, categoria, subcategoria, marca,
10          proveedor, precio_catalogo, costo, margen_pct, activo,
11          stock_actual, stock_minimo, estado_stock) %>%
12   mutate(margen_negativo = margen_pct < 0)
13
14 # ---- Dimensión tienda (antigüedad a la fecha de corte) ----
15 dim_tienda <- tiendas %>%
16   rename(num_empleados = empleados) %>%
17   mutate(anios_operacion = floor(time_length(
18     interval(fecha_apertura, fecha_corte), "years")))
19
20 # ---- Dimensión vendedor (solo los 25 que aparecen en ventas) ----
21 dim_vendedor <- empleados %>%
22   filter(empleador_id %in% vendedores_ids) %>%
23   select(vendedor_id = empleado_id, nombre_vendedor = nombre,
24          departamento, puesto, salario_mensual, antigüedad_anos) %>%
25   mutate(depto_ventas = departamento == "Ventas")
26
27 # ---- Dimensión fecha (calendario completo de 2023) ----
28 dias_es <- c("lunes", "martes", "miércoles", "jueves", "viernes",
29             "sábado", "domingo")
30 dim_fecha <- tibble(fecha = seq(as.Date("2023-01-01"),
31                                as.Date("2023-12-31"), by = "day")) %>%
32   mutate(anio      = year(fecha),
33          mes       = month(fecha),

```

```

34     trimestre = quarter(fecha),
35     semana_iso = isoweek(fecha),
36     dia_semana = factor(dias_es[wday(fecha, week_start = 1)],
37                         levels = dias_es),
38     fin_semana = wday(fecha, week_start = 1) >= 6)
39
40 map_int(list(dim_cliente, dim_producto, dim_tienda, dim_vendedor,
41             dim_fecha), nrow)

```

```
[1] 200 50 10 25 365
```

Construimos el día de la semana con un vector propio (`dias_es`) en lugar de `wday(label = TRUE)`, porque este último depende del idioma de tu computadora: en una laptop configurada en inglés obtendrías *Monday* y tu tablero mezclaría idiomas. Ahora la tabla de hechos:

```

1 hechos_ventas <- transacciones %>%
2   select(transaccion_id, fecha, hora, cliente_id, producto_id,
3         tienda_id, vendedor_id, metodo_pago, cantidad, precio_venta,
4         total = total_transaccion) %>%
5   # Costo unitario vigente y banderas que vienen de los catálogos
6   left_join(select(productos, producto_id, costo_unitario = costo,
7                   activo), by = "producto_id") %>%
8   left_join(select(clientes, cliente_id, fecha_registro),
9             by = "cliente_id") %>%
10  mutate(
11    hora_dia      = hour(hora),                # 8, 9, ..., 20
12    costo_total   = cantidad * costo_unitario,
13    margen_estimado = total - costo_total,
14    bajo_costo    = precio_venta < costo_unitario,
15    producto_inactivo = !activo,
16    antes_registro = fecha < fecha_registro
17  ) %>%
18  select(-activo, -fecha_registro)
19
20 # Control: un join mal hecho puede duplicar filas sin avisar
21 stopifnot(nrow(hechos_ventas) == nrow(transacciones))
22 stopifnot(sum(hechos_ventas$total) == sum(transacciones$total_transaccion))
23
24 hechos_ventas %>%
25   summarise(filas = n(), ventas = sum(total),
26             margen_estimado = sum(margen_estimado),
27             bajo_costo = sum(bajo_costo),
28             inactivos = sum(producto_inactivo),
29             antes_registro = sum(antes_registro))

```

```
# A tibble: 1 x 6
  filas ventas margen_estimado bajo_costo inactivos antes_registro
<int> <dbl> <dbl> <int> <int> <int>
1 1000 959993. 444637. 252 137 114
```

Los dos `stopifnot()` son **controles de conciliación**: si algún `left_join()` duplicara filas (porque una llave estuviera repetida en el catálogo), el script se detendría en lugar de reportar ventas infladas. La tabla de hechos conserva las 1,000 transacciones y los \$959,993 de ventas, y las banderas coinciden con la auditoría. La Figura 9.2 muestra el modelo resultante.

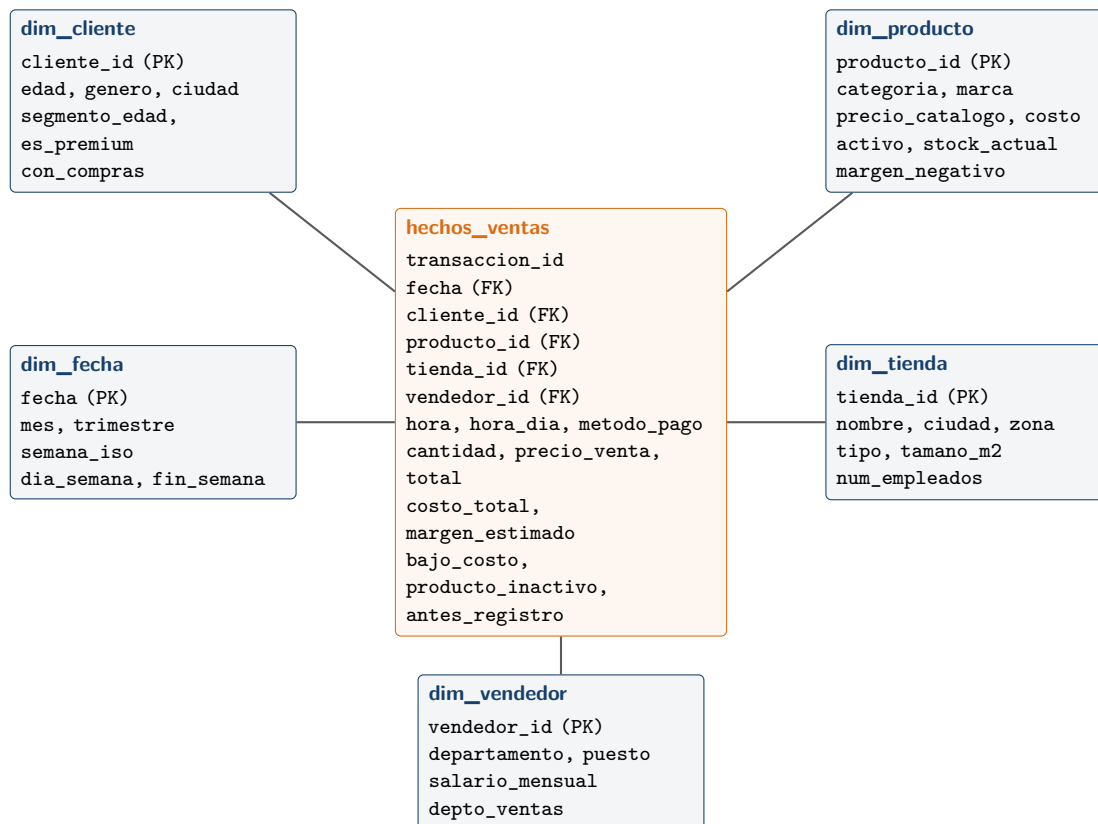


Figura 9.2. Modelo en estrella de TechRetail: una tabla de hechos (naranja) y cinco dimensiones (azul).

9.2.6 Cargar: guardar el modelo

El resultado de la Fase 1 se guarda en dos formatos. Los archivos **RDS** conservan los tipos de R (fechas, horas, factores) y son la entrada de las fases siguientes y del tablero. La base **SQLite** permite que otras personas o herramientas (Power BI, Excel, Python) consulten los mismos datos con SQL.

```
1 # ---- RDS: una lista con todas las tablas del modelo ----
2 modelo <- list(hechos_ventas = hechos_ventas,
3               dim_cliente   = dim_cliente,
4               dim_producto  = dim_producto,
```

```

5         dim_tienda    = dim_tienda,
6         dim_vendedor  = dim_vendedor,
7         dim_fecha     = dim_fecha)
8 saveRDS(modelo, "resultados/modelo_techretail.rds")
9
10 # ---- SQLite: fechas y horas como texto para no perder el formato ----
11 con <- dbConnect(RSQLite::SQLite(), "datos/techretail.sqlite")
12 walk2(modelo, names(modelo), function(tabla, nombre) {
13   tabla <- tabla %>%
14     mutate(across(where(is.Date), as.character),
15            across(where(hms::is_hms), as.character),
16            across(where(is.factor), as.character))
17   dbWriteTable(con, nombre, tabla, overwrite = TRUE)
18 })
19 dbListTables(con)
20
21 # Comprobación con SQL: ventas por zona directamente en la base
22 dbGetQuery(con, "
23   SELECT t.zona, COUNT(*) AS transacciones,
24          ROUND(SUM(h.total), 0) AS ventas
25   FROM hechos_ventas h
26   JOIN dim_tienda t ON h.tienda_id = t.tienda_id
27   GROUP BY t.zona
28   ORDER BY ventas DESC")
29 dbDisconnect(con)

```

```

[1] "dim_cliente"    "dim_fecha"      "dim_producto"   "dim_tienda"
[5] "dim_vendedor"   "hechos_ventas"
      zona transacciones ventas
1 Norte              399 388941
2 Centro              290 275878
3 Oeste              109 105216
4 Sur                 108 101538
5 Este                 94  88420

```

SQLite guarda las fechas como números si no las conviertes, por eso las pasamos a texto antes de escribir (Módulo 5). La consulta de comprobación es una buena costumbre: demuestra que la base quedó bien cargada y te da el primer resultado de negocio, la zona Norte (4 sucursales) concentra la mayor parte de las ventas.

Hazlo tú

Agrega a `dim_tienda` una columna `region` que agrupe las ciudades en “Centro” (CDMX, Puebla, Querétaro) y “Norte-Occidente” (Guadalajara, Monterrey, Tijuana), vuelve a guardar el RDS y escribe en SQL la consulta de ventas por región.

9.3 Fase 2: KPIs y análisis exploratorio

Con el modelo listo, respondemos las preguntas descriptivas. La regla de esta fase es simple: **cada tabla o gráfica termina en una frase de negocio con números**. Las técnicas de agrupación son las del Módulo 2; las gráficas siguen los principios del Módulo 3, y las pruebas estadísticas, los del Módulo 4.

Primero armamos una **vista analítica**: la tabla de hechos con los atributos de las dimensiones que vamos a usar. Es el equivalente a una “tabla plana” de Excel, pero construida desde el modelo, así que todos los análisis parten de la misma fuente.

```

1 ventas <- hechos_ventas %>%
2   left_join(select(dim_producto, producto_id, nombre_producto,
3                     categoria), by = "producto_id") %>%
4   left_join(select(dim_tienda, tienda_id, nombre_tienda, zona, tipo,
5                     tamano_m2), by = "tienda_id") %>%
6   left_join(select(dim_cliente, cliente_id, es_premium),
7             by = "cliente_id") %>%
8   left_join(select(dim_fecha, fecha, mes, dia_semana), by = "fecha")
9
10 stopifnot(nrow(ventas) == nrow(hechos_ventas)) # conciliación

```

9.3.1 KPIs globales

Un **KPI** (indicador clave de desempeño) es una métrica que la dirección sigue para saber si va bien. Calcula los globales de 2023 y preséntalos ya formateados, como aparecerían en la primera diapositiva.

```

1 kpis <- ventas %>%
2   summarise(ventas_totales = sum(total),
3             transacciones  = n(),
4             ticket_promedio = mean(total),
5             unidades       = sum(cantidad),
6             clientes       = n_distinct(cliente_id),
7             margen         = sum(margen_estimado))
8
9 tabla_kpis <- tibble(
10   kpi = c("Ventas totales", "Transacciones", "Ticket promedio",
11           "Unidades vendidas", "Unidades por transacción",
12           "Clientes activos", "Base de clientes activa",
13           "Venta anual por cliente activo", "Margen bruto estimado",
14           "Margen estimado (%)"),
15   valor = c(pesos(kpis$ventas_totales),
16             comma(kpis$transacciones),
17             dollar(kpis$ticket_promedio, accuracy = 0.01),
18             comma(kpis$unidades),

```

```

19     number(kpis$unidades / kpis$transacciones, accuracy = 0.01),
20     comma(kpis$clientes),
21     percent(kpis$clientes / nrow(dim_cliente), accuracy = 0.1),
22     pesos(kpis$ventas_totales / kpis$clientes),
23     pesos(kpis$margen),
24     percent(kpis$margen / kpis$ventas_totales, accuracy = 0.1))
25 )
26 tabla_kpis

```

```

# A tibble: 10 x 2
  kpi                                valor
  <chr>                             <chr>
1 Ventas totales                    $959,993
2 Transacciones                     1,000
3 Ticket promedio                   $959.99
4 Unidades vendidas                 2,969
5 Unidades por transacción          2.97
6 Clientes activos                  198
7 Base de clientes activa            99.0%
8 Venta anual por cliente activo $4,848
9 Margen bruto estimado              $444,637
10 Margen estimado (%)               46.3%

```

Hallazgo: en 2023 TechRetail vendió \$959,993 en 1,000 transacciones, con un ticket promedio de \$959.99 y casi 3 unidades por compra. 198 de los 200 clientes registrados compraron al menos una vez (99%), y cada cliente activo dejó en promedio \$4,848 en el año. El margen bruto estimado es de 46.3 %, pero recuerda la auditoría: con el costo del catálogo en duda, ese número debe presentarse como *estimado*.

La Tabla 9.5 es el **diccionario de KPIs**: la definición exacta de cada indicador. Parece burocracia, pero evita la mitad de las discusiones de las juntas (“¿ticket promedio por transacción o por cliente?”).

Tabla 9.5. Diccionario de KPIs del proyecto.

| KPI | Definición | Fuente |
|---------------------------|--|-----------------------|
| Ventas totales | Suma de total (precio cobrado × cantidad) | hechos |
| Ticket promedio | Ventas totales / número de transacciones | hechos |
| Clientes activos | Clientes distintos con al menos una compra en el periodo | hechos |
| Base activa | Clientes activos / clientes registrados | hechos + dim_cliente |
| Margen estimado | Ventas – cantidad × costo unitario del catálogo | hechos + dim_producto |
| Crecimiento mensual | Ventas del mes / ventas del mes anterior – 1 | hechos + dim_fecha |
| Ventas por m ² | Ventas de la sucursal / tamano_m2 | hechos + dim_tienda |

9.3.2 Evolución mensual

```

1 meses_es <- c("ene", "feb", "mar", "abr", "may", "jun",
2               "jul", "ago", "sep", "oct", "nov", "dic")
3
4 ventas_mes <- ventas %>%
5   group_by(mes) %>%
6   summarise(ventas = sum(total), transacciones = n(),
7             ticket = mean(total), .groups = "drop") %>%
8   mutate(crecimiento = ventas / lag(ventas) - 1,           # vs mes previo
9          nombre_mes = factor(meses_es[mes], levels = meses_es))
10
11 ventas_mes %>%
12   transmute(nombre_mes, ventas = round(ventas), transacciones,
13             ticket = round(ticket), crecimiento = percent(crecimiento, 0.1))

```

```

# A tibble: 12 x 5
  nombre_mes ventas transacciones ticket crecimiento
  <fct>         <dbl>         <int>  <dbl> <chr>
1 ene         82560             98    842 <NA>
2 feb         65899             79    834 -20.2%
3 mar         90931             91    999 38.0%
4 abr         57038             71    803 -37.3%
5 may         93910             88   1067 64.6%
6 jun         88509             84   1054 -5.8%
7 jul         86456             83   1042 -2.3%
8 ago         78929             75   1052 -8.7%
9 sep         68856             86    801 -12.8%
10 oct         70004             74    946  1.7%
11 nov         80507             80   1006 15.0%
12 dic         96394             91   1059 19.7%

```

```

1 promedio_mes <- mean(ventas_mes$ventas)
2 extremos <- ventas_mes %>%
3   filter(ventas %in% range(ventas)) %>%
4   mutate(ajuste = if_else(ventas == max(ventas), -1.1, 2)) # arriba/abajo
5
6 ggplot(ventas_mes, aes(x = nombre_mes, y = ventas, group = 1)) +
7   geom_hline(yintercept = promedio_mes, linetype = "dashed",
8             color = color_gris, linewidth = 0.6) +
9   geom_line(color = color_principal, linewidth = 0.8) +
10  geom_point(color = color_principal, size = 2) +
11  geom_point(data = extremos, color = color_resalte, size = 3) +
12  geom_text(data = extremos, aes(label = pesos(ventas), vjust = ajuste),
13           color = color_resalte, size = 3.5) +
14  scale_y_continuous(labels = dollar, limits = c(0, 110000)) +

```



```

15 labs(title = "Ventas mensuales de TechRetail, 2023",
16       subtitle = str_c("Mucha variación y sin tendencia; línea punteada = ",
17                         "promedio mensual (", pesos(promedio_mes), ")"),
18       x = NULL, y = "Ventas") +
19 tema_libro()

```

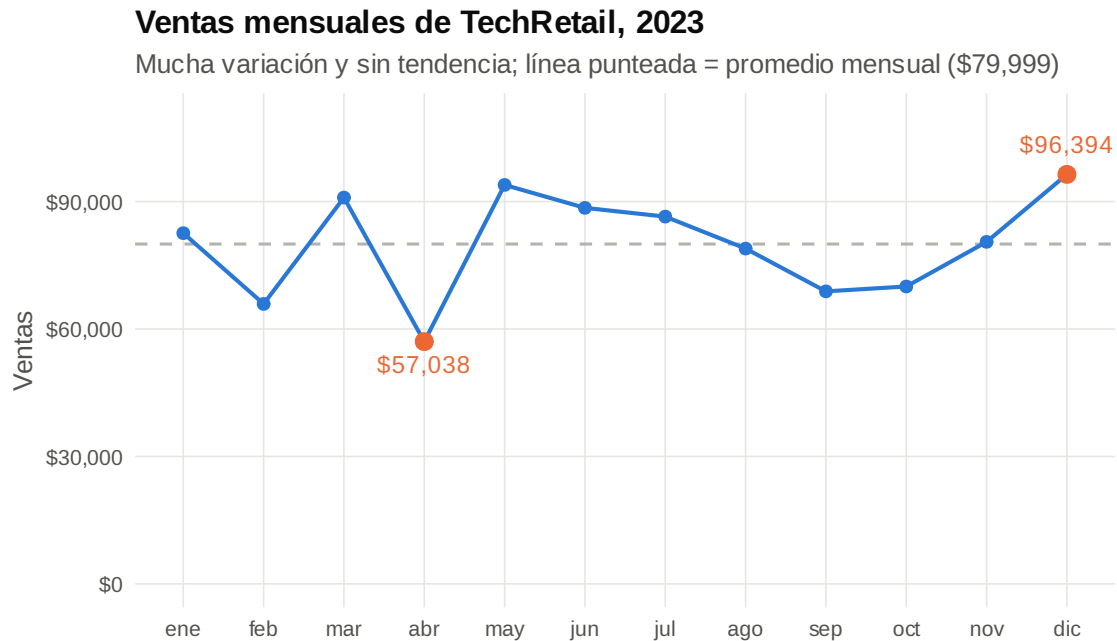


Figura 9.3. Ventas mensuales con el mejor y el peor mes resaltados.

Hallazgo: diciembre fue el mejor mes (\$96,394) y abril el peor (\$57,038). El crecimiento mensual oscila entre -37.3% (abril) y $+64.6\%$ (mayo): los saltos son grandes pero no forman una tendencia; la línea sube y baja alrededor de un promedio de \$80,000 al mes. Para la dirección, esto significa que *no hay evidencia de crecimiento ni de caída en 2023* y que comparar un mes contra el anterior genera falsas alarmas. Es mejor comparar contra el promedio o contra el mismo mes del año anterior (cuando exista).

▲ Cuidado

Con un solo año no puedes afirmar que “diciembre es temporada alta”. Un diciembre fuerte en un año puede ser azar; la estacionalidad se confirma cuando el patrón se repite al menos dos o tres años (Sección 9.4.2).

9.3.3 Categorías: ventas contra margen

```

1 ventas_categoria <- ventas %>%
2   group_by(categoria) %>%
3   summarise(ventas = sum(total), margen = sum(margen_estimado),
4             .groups = "drop") %>%
5   mutate(participacion = ventas / sum(ventas),
6          pct_margen     = margen / ventas) %>%
7   arrange(desc(ventas))
8
9 ventas_categoria %>%
10  mutate(across(c(ventas, margen), round),
11         across(c(participacion, pct_margen), ~ percent(.x, 0.1)))

```

```

# A tibble: 10 x 5
  categoria  ventas margen participacion pct_margen
  <chr>      <dbl> <dbl> <chr>      <chr>
1 Belleza    182143  56054 19.0%      30.8%
2 Electrónica 132672  73067 13.8%      55.1%
3 Juguetes   125093  57714 13.0%      46.1%
4 Alimentos  121543  73870 12.7%      60.8%
5 Automotriz 104085  55538 10.8%      53.4%
6 Deportes   86171  48301 9.0%       56.1%
7 Hogar      63407  13079 6.6%       20.6%
8 Mascotas   57668  35423 6.0%       61.4%
9 Libros     57162  25757 6.0%       45.1%
10 Ropa       30048   5835 3.1%       19.4%

```

```

1 ventas_categoria %>%
2   mutate(grupo = if_else(pct_margen < 0.25, "Margen estimado < 25%",
3                                     "Margen estimado de 25% o más"),
4          etiqueta = if_else(pct_margen < 0.25,
5                               str_c("margen ", percent(pct_margen, 1)), "")) %>%
6   ggplot(aes(x = ventas, y = fct_reorder(categoria, ventas),
7          fill = grupo)) +
8   geom_col(width = 0.7) +
9   geom_text(aes(label = etiqueta), hjust = -0.1, size = 3.3,
10            color = color_resalte) +
11   scale_fill_manual(values = c("Margen estimado < 25%" = color_resalte,
12                                "Margen estimado de 25% o más" = color_gris)) +
13   scale_x_continuous(labels = dollar, expand = expansion(c(0, 0.18))) +
14   labs(title = "Ventas por categoría, 2023",
15        subtitle = "Hogar y Ropa dejan el menor margen estimado",
16        x = "Ventas", y = NULL, fill = NULL) +
17   tema_libro()

```

Ventas por categoría, 2023

Hogar y Ropa dejan el menor margen estimado

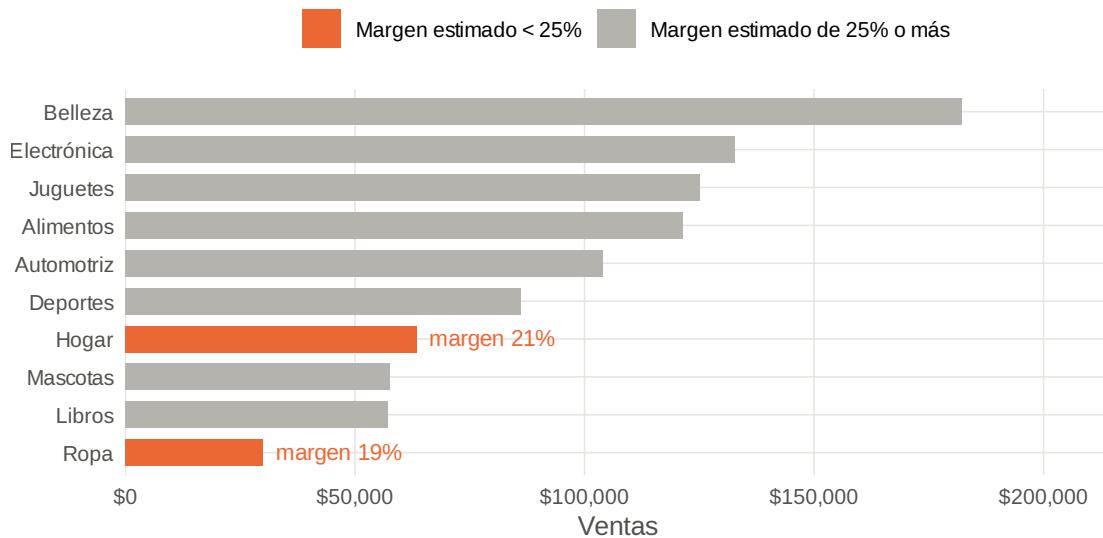


Figura 9.4. Ventas por categoría; en naranja, las de margen estimado menor a 25 %.

Hallazgo: Belleza es la categoría que más vende (19 % de las ventas), pero su margen estimado es de solo 30.8 % por dos productos con costo mayor al precio de catálogo. Hogar y Ropa tienen los márgenes más bajos (alrededor de 20 %). En el otro extremo, Mascotas y Alimentos rinden más de 60 % de margen. **Implicación:** crecer en Belleza sin corregir costos o precios sería vender más para ganar proporcionalmente menos; la primera acción es revisar los productos con margen negativo de la auditoría.

9.3.4 Sucursales y zonas

```

1 ventas_tienda <- ventas %>%
2   group_by(nombre_tienda, zona, tipo, tamano_m2) %>%
3   summarise(ventas = sum(total), transacciones = n(),
4             ticket = mean(total), .groups = "drop") %>%
5   arrange(desc(ventas))
6
7 ventas_tienda %>% mutate(across(c(ventas, ticket), round))
8
9 # Por zona: total y promedio por sucursal (las zonas tienen
10 # distinto número de tiendas)
11 ventas_tienda %>%
12   group_by(zona) %>%
13   summarise(tiendas = n(),
14             ventas_por_tienda = round(mean(ventas)), # antes de sumar
15             ventas_zona = round(sum(ventas)), .groups = "drop") %>%
16   arrange(desc(ventas_por_tienda))

```

```

# A tibble: 10 x 7
  nombre_tienda zona tipo tamano_m2 ventas transacciones ticket
  <chr>         <chr> <chr>    <dbl>  <dbl>         <int>  <dbl>
1 Sucursal Outlet Norte Outl~    350 114377         106  1079
2 Sucursal Sur Norte Está~    600 105305         109   966
3 Sucursal Plaza A Oeste Prin~    700 105216         109   965
4 Sucursal Express Centro Expr~    300 102791         108   952
5 Sucursal Premium Sur Prem~    800 101538         108   940
6 Sucursal Norte Norte Está~    450 89024          99   899
7 Sucursal Este Este Está~    400 88420          94   941
8 Sucursal Centro Centro Prin~    500 88414          87  1016
9 Sucursal Oeste Centro Está~    550 84672          95   891
10 Sucursal Plaza B Norte Está~    480 80236          85   944
# A tibble: 5 x 4
  zona tiendas ventas_por_tienda ventas_zona
  <chr>   <int>         <dbl>         <dbl>
1 Oeste     1      105216      105216
2 Sur       1      101538      101538
3 Norte     4       97235      388941
4 Centro     3       91959      275878
5 Este      1       88420       88420

```

```

1 ventas_tienda %>%
2   mutate(tienda = str_c(nombre_tienda, " (", tamano_m2, " m²)",
3                         formato = if_else(tamano_m2 < 400, "Menos de 400 m²",
4                                           "400 m² o más")) %>%
5   ggplot(aes(x = ventas, y = fct_reorder(tienda, ventas),
6     fill = formato)) +
7   geom_col(width = 0.7) +
8   scale_fill_manual(values = c("Menos de 400 m²" = color_resalte,

```

```

9           "400 m² o más" = color_gris)) +
10   scale_x_continuous(labels = dollar, expand = expansion(c(0, 0.1))) +
11   labs(title = "Ventas por sucursal, 2023",
12        subtitle = "Las dos tiendas más chicas están entre las 4 que más venden",
13        x = "Ventas", y = NULL, fill = "Tamaño") +
14   tema_libro()

```

Ventas por sucursal, 2023

Las dos tiendas más chicas están entre las 4 que más venden

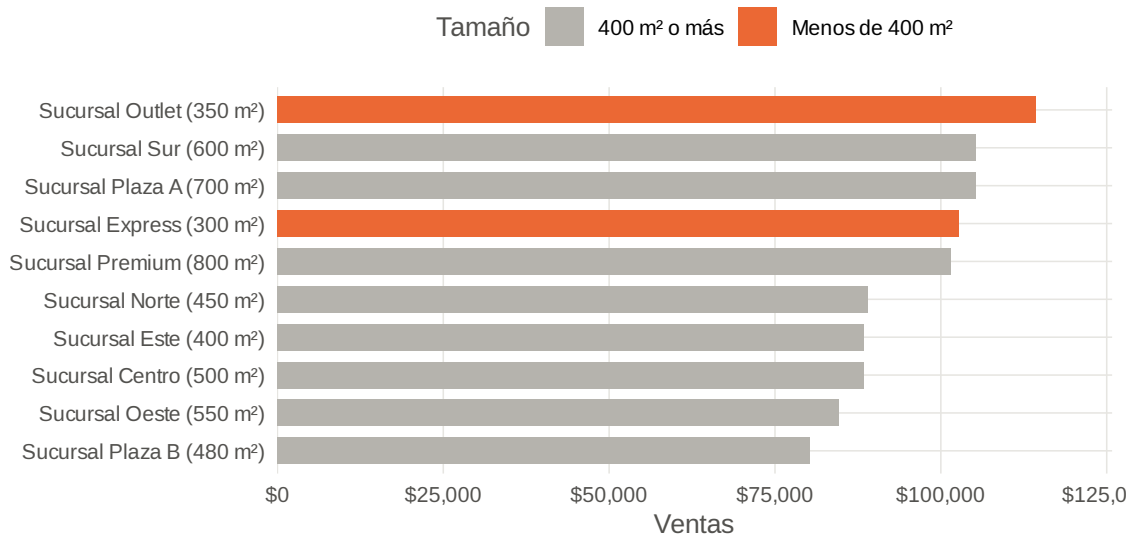


Figura 9.5. Ventas por sucursal; en naranja, los formatos de menos de 400 m².

Hallazgo: la sucursal que más vende es la Outlet de Tijuana (\$114,377), una de las más pequeñas (350 m²), y la Express de Querétaro, la más chica de todas, es cuarta. La Premium, con 800 m² y 40 empleados, queda en quinto lugar. Entre la mejor y la peor sucursal (Plaza B, \$80,236) hay una diferencia de 43 %. Por zona, la Norte vende más en total (\$388,941) solo porque tiene 4 tiendas; por sucursal, la zona Oeste (una tienda) es la más alta. **Implicación:** el tamaño no explica las ventas; la pregunta correcta es la productividad por m² y por empleado (Reto 9.2), clave si la dirección evalúa abrir más formatos chicos.

9.3.5 Método de pago

```

1  ventas %>%
2    group_by(metodo_pago) %>%
3    summarise(transacciones = n(), ventas = sum(total),
4              ticket = mean(total), .groups = "drop") %>%
5    mutate(pct_ventas = percent(ventas / sum(ventas), 0.1),
6           across(c(ventas, ticket), round)) %>%

```

```

7   arrange(desc(ventas))
8
9   # ¿El ticket difiere según el método de pago? (Módulo 4)
10  kruskal.test(total ~ metodo_pago, data = ventas)$p.value

```

```

# A tibble: 4 x 5
  metodo_pago    transacciones ventas ticket pct_ventas
  <chr>          <int>   <dbl>   <dbl> <chr>
1 Tarjeta Crédito      350 351171   1003 36.6%
2 Efectivo             315 303735    964 31.6%
3 Tarjeta Débito       233 220855    948 23.0%
4 Transferencia        102  84232    826  8.8%
[1] 0.2027074

```

Hallazgo: la tarjeta de crédito es el método más usado (350 transacciones, 36.6 % de las ventas) y tiene el ticket más alto (\$1,003); la transferencia es el menos usado (8.8 %) y su ticket es el más bajo (\$826). Sin embargo, la prueba de Kruskal-Wallis da un valor p de 0.20: la diferencia de tickets *no es estadísticamente significativa*. **Implicación:** no justifica, por sí sola, promociones para empujar el pago con crédito; sí vale la pena revisar las comisiones bancarias, porque 61 % de las ventas se paga con tarjeta.

9.3.6 Días y horas

```

1  # Ventas por día de la semana (dia_semana ya es un factor ordenado)
2  ventas %>%
3    group_by(dia_semana) %>%
4    summarise(transacciones = n(), ventas = round(sum(total)),
5              .groups = "drop")
6
7  # Mapa de calor día x hora
8  calor <- ventas %>%
9    count(dia_semana, hora_dia, wt = total, name = "ventas")
10
11 ggplot(calor, aes(x = factor(hora_dia), y = fct_rev(dia_semana),
12                    fill = ventas)) +
13   geom_tile(color = "white", linewidth = 0.6) +
14   scale_fill_gradient(low = "#e3eefb", high = "#0f4c92",
15                      labels = dollar) +
16   labs(title = "Ventas por día de la semana y hora, 2023",
17        subtitle = "Sin horas pico claras: las celdas oscuras están dispersas",
18        x = "Hora del día", y = NULL, fill = "Ventas") +
19   tema_libro() +
20   theme(panel.grid.major = element_blank(),
21         legend.key.width = unit(1.5, "cm"))

```

```
# A tibble: 7 x 3
  dia_semana transacciones ventas
  <fct>         <int>   <dbl>
1 lunes           136 128878
2 martes          129 119364
3 miércoles       155 141329
4 jueves          146 145280
5 viernes         158 161266
6 sábado          130 115172
7 domingo         146 148703
```

Ventas por día de la semana y hora, 2023

Sin horas pico claras: las celdas oscuras están dispersas

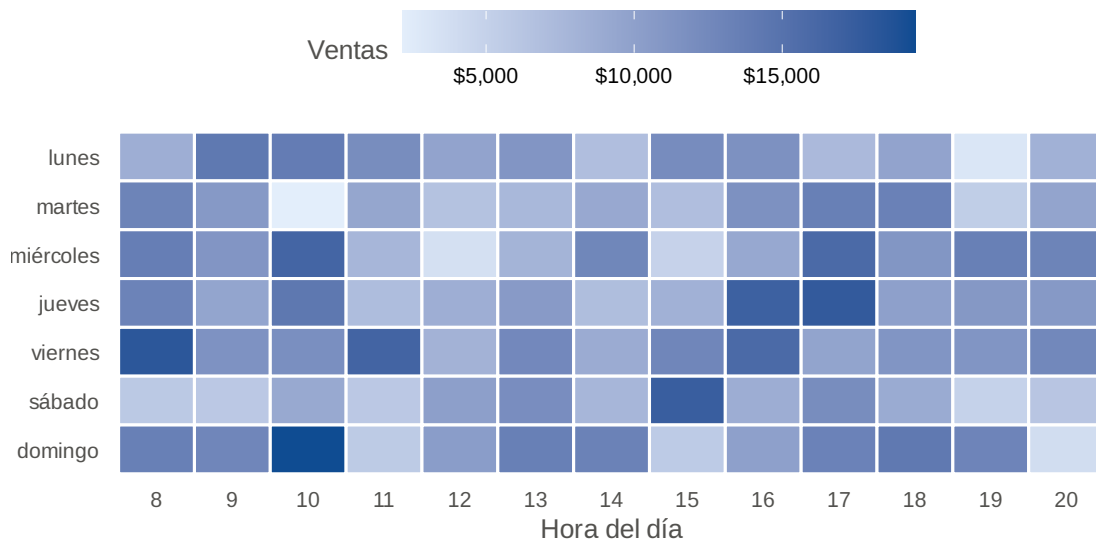


Figura 9.6. Mapa de calor de ventas por día y hora (escala secuencial de un solo tono).

Hallazgo: el viernes es el día con más ventas (\$161,266) y, contra lo que dicta la intuición del retail, el sábado es el de menos (\$115,172). El mapa de calor no muestra una franja horaria dominante: las celdas más oscuras aparecen dispersas. Ojo con la escala: 1,000 transacciones repartidas en 91 celdas (7 días × 13 horas) son unas 11 por celda, así que una sola venta grande pinta una celda de oscuro. **Implicación:** antes de mover turnos de personal con base en este mapa, conviene acumular más meses de datos; con estos números no hay una hora pico que justifique reforzar el personal.

9.3.7 Pareto de productos

El **principio de Pareto** (80/20) dice que en muchos negocios cerca del 20 % de los productos genera el 80 % de las ventas. ¿Se cumple en TechRetail?

```

1 pareto <- ventas %>%
2   group_by(producto_id, nombre_producto, categoria) %>%
3   summarise(ventas = sum(total), .groups = "drop") %>%
4   arrange(desc(ventas)) %>%
5   mutate(rango      = row_number(),
6          pct_productos = rango / n(),
7          pct_acumulado = cumsum(ventas) / sum(ventas))
8
9 pareto %>%
10  summarise(
11    ventas_top20pct = percent(max(pct_acumulado[pct_productos <= 0.2]),
12                             0.1),
13    productos_al_80 = min(rango[pct_acumulado >= 0.8]),
14    mayor_vs_menor  = round(max(ventas) / min(ventas), 1))

```

```

# A tibble: 1 x 3
  ventas_top20pct productos_al_80 mayor_vs_menor
  <chr>          <int>          <dbl>
1 26.4%          37          3.3

```

```

1 ggplot(pareto, aes(x = pct_productos, y = pct_acumulado)) +
2   geom_abline(slope = 1, intercept = 0, linetype = "dashed",
3              color = color_gris, linewidth = 0.6) +
4   geom_hline(yintercept = 0.8, linetype = "dotted", color = color_gris) +
5   geom_vline(xintercept = 0.2, linetype = "dotted", color = color_gris) +
6   geom_line(color = color_principal, linewidth = 0.8) +
7   annotate("point", x = 0.2, y = 0.264, color = color_resalte, size = 3) +
8   annotate("text", x = 0.24, y = 0.12, hjust = 0, size = 3.4,
9           color = color_resalte,
10          label = "El 20% de los productos genera 26% de las ventas") +
11  annotate("text", x = 0.62, y = 0.52, hjust = 0, size = 3,
12          color = "#52514e", label = "Reparto perfectamente igual") +
13  scale_x_continuous(labels = percent) +
14  scale_y_continuous(labels = percent) +
15  labs(title = "Curva de Pareto de productos, 2023",
16       subtitle = "Las ventas están repartidas: no hay un grupo de productos
17   ↪ estrella",
18       x = "Productos (ordenados de mayor a menor venta)",
19       y = "Ventas acumuladas") +
20  tema_libro()

```

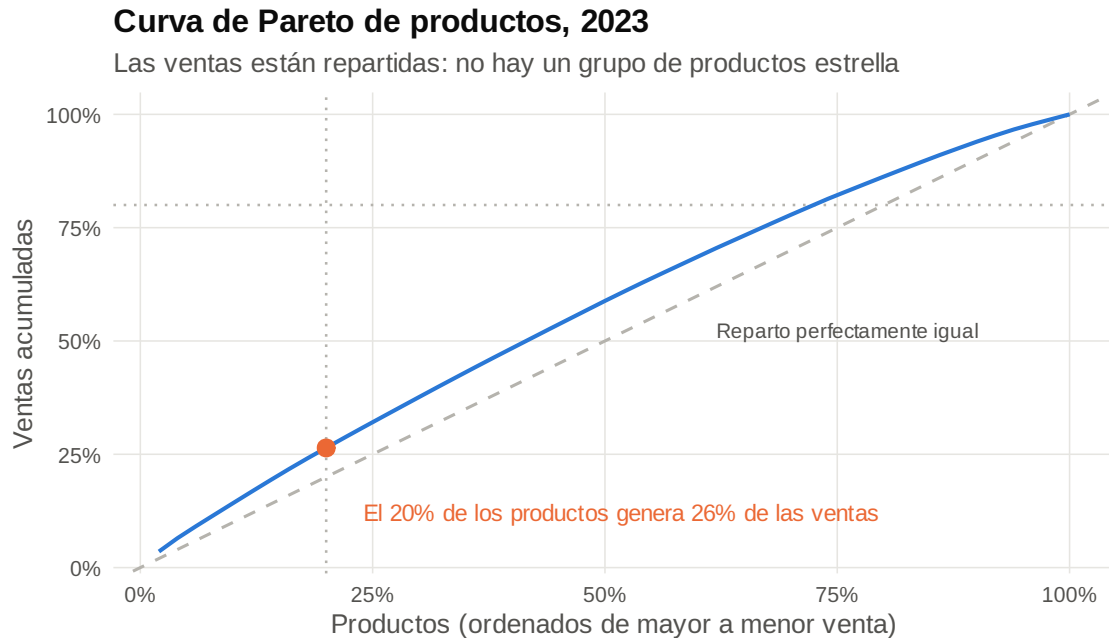



Figura 9.7. Curva de Pareto: la curva azul está cerca de la diagonal de reparto igual.

Hallazgo: los 10 productos más vendidos (20 % del catálogo) generan solo 26.4 % de las ventas, y se necesitan 37 productos (74 % del catálogo) para llegar al 80 %. El producto que más vende factura 3.3 veces lo que el que menos vende. **Implicación:** TechRetail *no* depende de unos cuantos productos estrella; una estrategia de “concentrar el inventario en el top 10” dejaría fuera casi tres cuartas partes de las ventas. Es un resultado “negativo” igual de valioso que uno positivo: descarta una estrategia.

9.3.8 Desempeño de vendedores

```

1 ventas_vendedor <- ventas %>%
2   group_by(vendedor_id) %>%
3   summarise(ventas = sum(total), transacciones = n(),
4             .groups = "drop") %>%
5   left_join(dim_vendedor, by = "vendedor_id") %>%
6   arrange(desc(ventas))
7
8 ventas_vendedor %>%
9   select(vendedor_id, departamento, puesto, ventas, transacciones,
10         salario_mensual) %>%
11   mutate(ventas = round(ventas)) %>%
12   slice(c(1:3, 23:25))           # los 3 mejores y los 3 últimos
13
14 # ¿Venden más quienes ganan más? (correlación, Módulo 4)
15 prueba_salario <- cor.test(ventas_vendedor$ventas,
16                             ventas_vendedor$salario_mensual)

```

```

17 c(r = round(unname(prueba_salario$estimate), 3),
18   p_valor = round(prueba_salario$p.value, 3))

```

```

# A tibble: 6 x 6
  vendedor_id departamento puesto ventas transacciones salario_mensual
    <dbl> <chr>      <chr>   <dbl>         <int>         <dbl>
1         7 RRHH      Coord~   50470          45         14488.
2         2 Atención al~ Asist~   50168          39         15293.
3        11 Atención al~ Coord~   50017          48         30743.
4        14 Ventas      Geren~   26430          33         38782.
5         1 Marketing  Anali~   24281          25         17291.
6        10 IT         Espec~   21678          24         11205.

r p_valor
-0.138  0.510

```

```

1 ventas_vendedor %>%
2   mutate(grupo = if_else(depto_ventas, "Depto. de Ventas",
3                             "Otro departamento")) %>%
4   ggplot(aes(x = salario_mensual, y = ventas, color = grupo)) +
5   geom_point(size = 2.8) +
6   scale_color_manual(values = c("Depto. de Ventas" = color_resalte,
7                                 "Otro departamento" = color_gris)) +
8   scale_x_continuous(labels = dollar) +
9   scale_y_continuous(labels = dollar) +
10  labs(title = "Ventas anuales contra salario mensual por vendedor",
11        subtitle = "Sin relación (r = -0.14, p = 0.51); dato de RR. HH. por validar",
12        x = "Salario mensual", y = "Ventas 2023", color = NULL) +
13  tema_libro()

```

Ventas anuales contra salario mensual por vendedor

Sin relación ($r = -0.14$, $p = 0.51$); dato de RR. HH. por validar

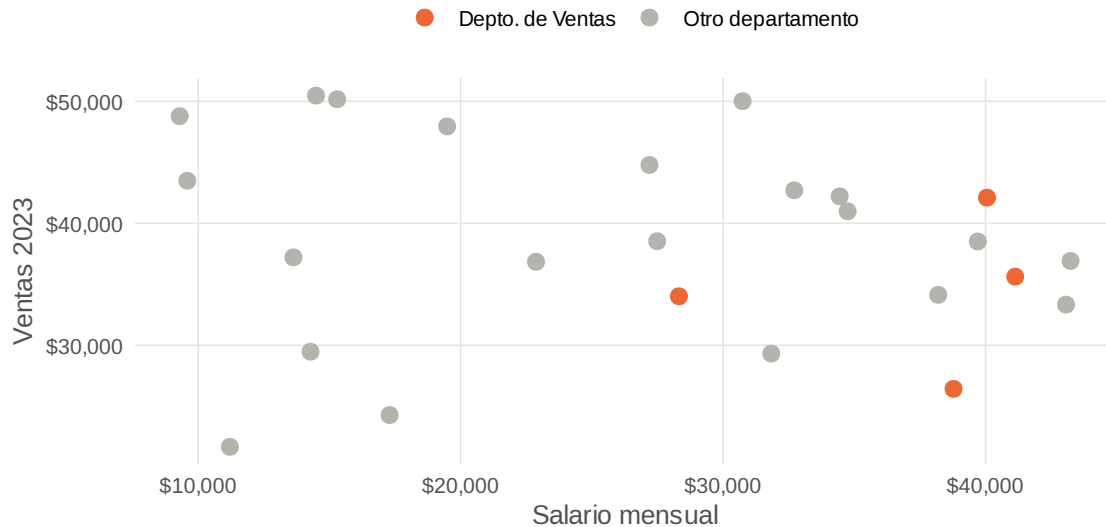


Figura 9.8. Ventas contra salario de los 25 vendedores.

Hallazgo: el mejor vendedor (id 7) vendió \$50,470 y el último (id 10) \$21,678, 2.3 veces menos. La correlación entre ventas y salario es prácticamente nula ($r = -0.14$, $p = 0.51$): ganar más no se asocia con vender más. **Implicación y cautela:** esto sugiere revisar el esquema de compensación (por ejemplo, comisiones sobre venta), pero recuerda que la auditoría mostró que 21 de los 25 “vendedores” figuran en otros departamentos. Antes de presentar este hallazgo, valida la llave con Recursos Humanos; si la llave está mal, la gráfica compara ventas de una persona con el salario de otra.

9.3.9 Clientes premium contra no premium

El programa premium tiene un costo (beneficios, descuentos). Si funciona, los clientes premium deberían gastar más. Es una pregunta de **prueba de hipótesis** (Módulo 4).

```
1 gasto_cliente <- ventas %>%
2   group_by(cliente_id, es_premium) %>%
3   summarise(gasto = sum(total), compras = n(), .groups = "drop") %>%
4   mutate(grupo = if_else(es_premium, "Premium", "No premium"))
5
6 gasto_cliente %>%
7   group_by(grupo) %>%
8   summarise(clientes = n(), gasto_promedio = round(mean(gasto)),
9             gasto_mediano = round(median(gasto)),
10            compras_promedio = round(mean(compras), 2), .groups = "drop")
11
12 # Prueba t de Welch y, como respaldo, Wilcoxon (no paramétrica)
```

```

13 t.test(gasto ~ grupo, data = gasto_cliente)
14 wilcox.test(gasto ~ grupo, data = gasto_cliente)$p.value

```

```

# A tibble: 2 x 5
  grupo      clientes gasto_promedio gasto_mediano compras_promedio
  <chr>      <int>      <dbl>      <dbl>      <dbl>
1 No premium    126      4929      4525        5.18
2 Premium        72      4708      4517        4.82

      Welch Two Sample t-test

data:  gasto by grupo
t = 0.56874, df = 160.59, p-value = 0.5703
alternative hypothesis: true difference in means between group No premium and group
  ↪ Premium is not equal to 0
95 percent confidence interval:
 -544.5031  984.9854
sample estimates:
mean in group No premium    mean in group Premium
          4928.536              4708.294

[1] 0.8882566

```

```

1 ggplot(gasto_cliente, aes(x = grupo, y = gasto)) +
2   geom_boxplot(fill = color_principal, alpha = 0.25, width = 0.5,
3     color = color_principal, outlier.shape = NA) +
4   geom_jitter(width = 0.15, alpha = 0.5, size = 1.5,
5     color = color_principal) +
6   stat_summary(fun = mean, geom = "point", shape = 18, size = 4,
7     color = color_resalte) +
8   scale_y_continuous(labels = dollar) +
9   labs(title = "Gasto anual por cliente: premium contra no premium",
10     subtitle = "Distribuciones casi idénticas (rombo naranja = promedio)",
11     x = NULL, y = "Gasto en 2023") +
12   tema_libro()

```

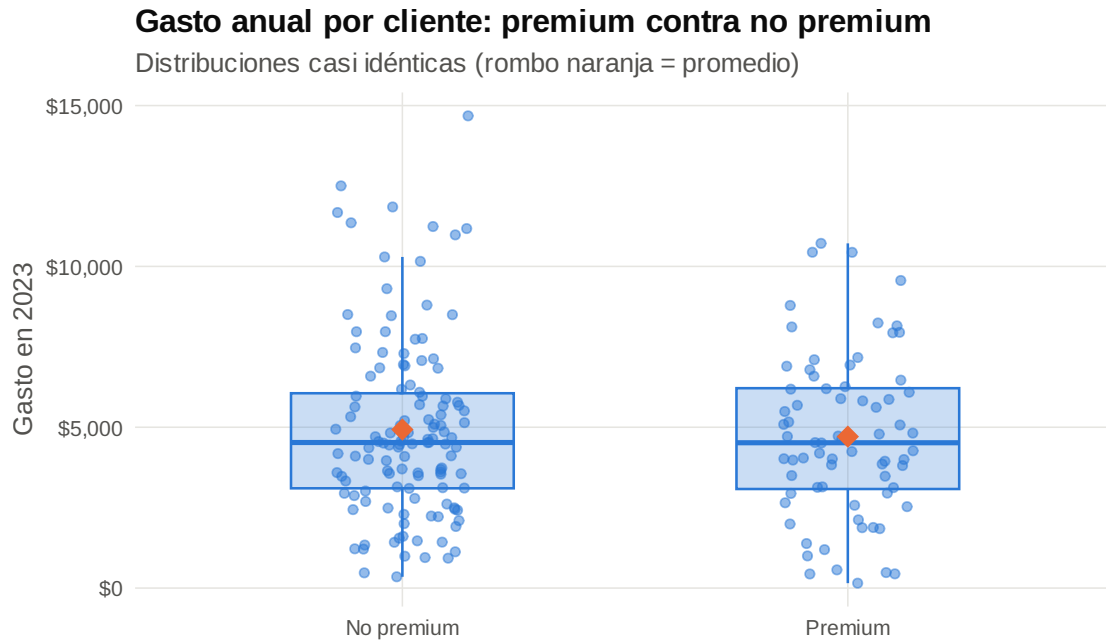


Figura 9.9. Gasto anual por cliente según el programa premium.

Hallazgo: los clientes premium gastaron en promedio \$4,708 contra \$4,929 de los no premium (sus medianas casi coinciden: \$4,517 y \$4,525) y compraron con la misma frecuencia (4.8 y 5.2 compras). La prueba t da $p = 0.57$ y la de Wilcoxon $p = 0.89$: *no hay diferencia significativa*. **Implicación:** el programa premium, tal como está, no se traduce en más gasto. Es una recomendación directa para la dirección: rediseñar los beneficios (atarlos a metas de compra) o medir si el programa aporta otra cosa (por ejemplo, menor abandono) antes de seguir financiándolo.

Hazlo tú

Repite la comparación premium contra no premium, pero con el *ticket promedio* por transacción en lugar del gasto anual. ¿Cambia la conclusión? Después compara el gasto por `segmento_edad` con `kruskal.test()`.

9.4 Fase 3: segmentación y predicción

Las fases anteriores describen el pasado. Esta fase agrega dos modelos que la dirección suele pedir: *¿qué tipos de clientes tenemos?* y *¿cuánto vamos a vender?* Ambas técnicas se explicaron a fondo en el Módulo 7; aquí nos concentramos en decidir, validar y comunicar.

9.4.1 Segmentación RFM con k-means

El análisis **RFM** describe a cada cliente con tres números: *recencia* (días desde su última compra hasta la fecha de corte), *frecuencia* (número de compras) y *monto* (gasto total). Aquí se ve por qué importaba fijar `fecha_corte`: con `Sys.Date()` la recencia cambiaría cada día y los segmentos

también.

```
1 rfm <- hechos_ventas %>%
2   group_by(cliente_id) %>%
3   summarise(recencia = as.numeric(fecha_corte - max(fecha)),
4             frecuencia = n(),
5             monto = sum(total), .groups = "drop")
6
7 summary(select(rfm, -cliente_id))
```

| | recencia | frecuencia | monto |
|----------|----------|----------------|-----------------|
| Min. | : 1.00 | Min. : 1.000 | Min. : 152.6 |
| 1st Qu.: | 21.00 | 1st Qu.: 3.000 | 1st Qu.: 3101.4 |
| Median : | 48.00 | Median : 5.000 | Median : 4521.8 |
| Mean : | 69.19 | Mean : 5.051 | Mean : 4848.4 |
| 3rd Qu.: | 98.75 | 3rd Qu.: 6.000 | 3rd Qu.: 6183.0 |
| Max. : | 314.00 | Max. : 13.000 | Max. : 14680.1 |

La mitad de los clientes compró por última vez hace 48 días o menos y la mediana es de 5 compras en el año; el gasto va de \$153 a \$14,680. Las tres variables tienen escalas muy distintas (días, compras y pesos), así que las estandarizamos antes de k-means. Para elegir el número de grupos revisamos dos criterios del Módulo 7: la suma de cuadrados dentro de los grupos (método del codo) y la silueta promedio.

```
1 rfm_escalado <- scale(select(rfm, recencia, frecuencia, monto))
2
3 set.seed(123)
4 evaluacion_k <- map_dfr(2:7, function(k) {
5   modelo_k <- kmeans(rfm_escalado, centers = k, nstart = 25)
6   tibble(k = k,
7         suma_cuadrados = round(modelo_k$tot.withinss, 1),
8         silueta = round(mean(silhouette(modelo_k$cluster,
9                                       dist(rfm_escalado))[, 3]), 3))
10 })
11 evaluacion_k
```

```
# A tibble: 6 x 3
  k suma_cuadrados silueta
<int>      <dbl>   <dbl>
1     2         344.  0.345
2     3         239.  0.337
3     4         175.  0.328
4     5         146.  0.325
5     6         127.  0.292
6     7         114.  0.279
```

La silueta es parecida entre 2 y 5 grupos (no hay un ganador claro) y el codo se suaviza a partir de 4. Cuando la estadística no decide, decide el negocio: **dos** grupos (“buenos” y “malos”) no le dicen nada nuevo a Marketing, mientras que **cuatro** permiten diseñar cuatro acciones distintas. Elegimos $k = 4$ y lo decimos explícitamente en el reporte.

```

1  set.seed(123)
2  km_rfm <- kmeans(rfm_escalado, centers = 4, nstart = 25)
3
4  perfil <- rfm %>%
5    mutate(cluster = km_rfm$cluster) %>%
6    group_by(cluster) %>%
7    summarise(clientes = n(), recencia = mean(recencia),
8              frecuencia = mean(frecuencia), monto = mean(monto),
9              .groups = "drop")
10
11 # Nombres de negocio a partir del perfil (no del número de cluster,
12 # que puede cambiar entre versiones de R)
13 perfil <- perfil %>%
14   mutate(segmento = case_when(
15     monto == max(monto) ~ "Campeones",
16     recencia == max(recencia) ~ "En riesgo",
17     monto == max(monto[!monto == max(monto) &
18       !recencia == max(recencia)]) ~ "Leales",
19     TRUE ~ "Ocasionales"))
20
21 rfm <- rfm %>%
22   mutate(cluster = km_rfm$cluster) %>%
23   left_join(select(perfil, cluster, segmento), by = "cluster")
24
25 perfil %>%
26   mutate(pct_clientes = percent(clientes / sum(clientes), 1),
27          pct_ventas = percent(clientes * monto / sum(clientes * monto),
28                                1),
29          across(c(recencia, frecuencia), ~ round(.x, 1)),
30          monto = round(monto)) %>%
31   select(segmento, clientes, pct_clientes, recencia, frecuencia, monto,
32          pct_ventas) %>%
33   arrange(desc(monto))

```

```

# A tibble: 4 x 7
  segmento  clientes pct_clientes recencia frecuencia monto pct_ventas
  <chr>      <int> <chr>          <dbl>      <dbl> <dbl> <chr>
1 Campeones    22 11%          34.5        9.2 10159 23%
2 Leales       87 44%          41.8        5.9  5577 51%
3 Ocasional~   60 30%          60.5        3.4  2879 18%
4 En riesgo    29 15%          196.        2.8  2708  8%

```

```

1 ggplot(rfm, aes(x = recencia, y = monto, color = segmento)) +
2   geom_point(size = 2, alpha = 0.8) +
3   scale_color_manual(values = c("Campeones" = paleta_libro[3],
4                                 "Leales" = paleta_libro[1],
5                                 "Ocasionales" = paleta_libro[4],
6                                 "En riesgo" = paleta_libro[2])) +
7   scale_y_continuous(labels = dollar) +
8   labs(title = "Segmentos de clientes según RFM (k-means, k = 4)",
9         subtitle = "A la derecha, clientes que llevan meses sin comprar",
10        x = "Recencia (días desde la última compra)",
11        y = "Gasto en 2023", color = NULL) +
12   tema_libro()

```

Segmentos de clientes según RFM (k-means, k = 4)

A la derecha, clientes que llevan meses sin comprar

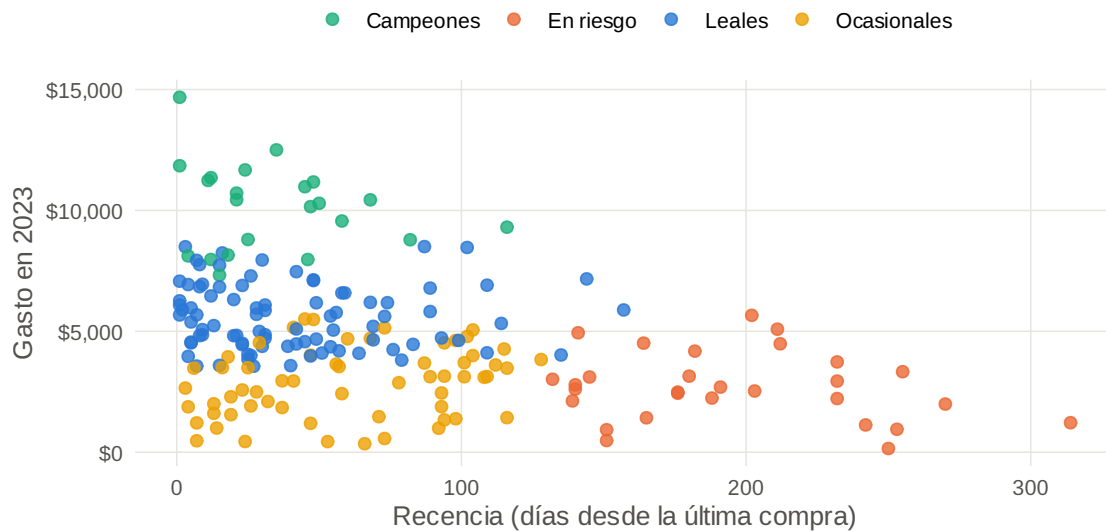


Figura 9.10. Clientes por recencia y gasto, coloreados por segmento.

El perfil convierte los grupos en acciones concretas (Tabla 9.6). Observa que cada acción tiene un indicador para medir si funcionó: sin eso, la segmentación se queda en una gráfica bonita.

Tabla 9.6. Segmentos RFM y acciones propuestas.

| Segmento | Acción propuesta | Indicador de éxito |
|-------------|---|--|
| Campeones | Programa de lealtad real (este sí, a diferencia del premium actual), acceso anticipado a lanzamientos | Retención del segmento y gasto por cliente |
| Leales | Venta cruzada con categorías de alto margen (Mascotas, Alimentos) | Categorías por cliente y margen |
| Ocasionales | Cupón de segunda compra con vigencia de 30 días | Tasa de recompra a 30 días |
| En riesgo | Campaña de reactivación antes de que pasen a inactivos | % reactivado en 60 días |

Hallazgo: los 22 *Campeones* (11 % de los clientes) compran en promedio 9.2 veces al año, gastan \$10,159 y generan 23 % de las ventas; junto con los 87 *Leales* suman 74 % de las ventas. En el otro extremo, 29 clientes *En riesgo* (15 %) llevan en promedio 196 días sin comprar: compraron en la primera mitad del año y no volvieron. **Implicación:** retener a 109 clientes (Campeones y Leales) protege tres cuartas partes del negocio, y el segmento En riesgo es una lista concreta, con nombre y apellido, para una campaña de reactivación.

Hazlo tú

Cruza el segmento RFM con `es_premium` usando `count()`. ¿Cuántos Campeones son premium? Si el programa premium no coincide con los mejores clientes, tienes un argumento más para rediseñarlo.

9.4.2 Pronóstico de ventas: lo que un año permite (y lo que no)

La directora quiere saber cuánto venderán en las próximas semanas. Antes de elegir un modelo, sé honesto sobre los datos: tenemos *un solo año*. Con 12 meses no se puede estimar la **estacionalidad anual** (“enero siempre baja, diciembre siempre sube”), porque para saber que un patrón se *repite* necesitas verlo al menos dos veces. R te lo dice sin rodeos:

```
1 ventas_mes_ts <- ts(ventas_mes$ventas, start = c(2023, 1),
2                     frequency = 12)
3 # Intentar descomponer tendencia + estacionalidad con 12 meses
4 tryCatch(stl(ventas_mes_ts, s.window = "periodic"),
5          error = function(e) conditionMessage(e))
```

```
[1] "series is not periodic or has less than two periods"
```

El mensaje dice que la serie “no es periódica o tiene menos de dos periodos”. Por eso trabajamos con **semanas** (más observaciones) y comparamos modelos sencillos, validando con las últimas 8 semanas como conjunto de prueba (Módulo 7).

```

1 # Semanas completas de lunes a domingo (el 1 de enero de 2023 fue
2 # domingo: lo dejamos fuera para no tener una semana de un solo día)
3 ventas_semana <- hechos_ventas %>%
4   filter(fecha >= as.Date("2023-01-02")) %>%
5   mutate(semana = floor_date(fecha, "week", week_start = 1)) %>%
6   count(semana, wt = total, name = "ventas")
7
8 nrow(ventas_semana)
9 round(c(promedio = mean(ventas_semana$ventas),
10         desv_est = sd(ventas_semana$ventas),
11         cv = sd(ventas_semana$ventas) / mean(ventas_semana$ventas)), 2)
12
13 # ¿Hay autocorrelación? (si no, la serie se comporta como ruido)
14 Box.test(ventas_semana$ventas, lag = 10, type = "Ljung-Box")$p.value

```

```

[1] 52
promedio desv_est      cv
18444.94  4591.29      0.25
[1] 0.5985765

```

Tenemos 52 semanas con ventas promedio de \$18,445 y un coeficiente de variación de 25%: una semana típica se aleja una cuarta parte del promedio. La prueba de Ljung-Box da $p = 0.60$, es decir, no hay evidencia de que la venta de una semana ayude a predecir la siguiente. En términos de negocio: la serie se comporta como *ruido alrededor de un nivel constante*. Con esa sospecha, comparemos tres modelos:

```

1 serie <- ts(ventas_semana$ventas)
2 entrenamiento <- window(serie, end = 44)
3 prueba <- window(serie, start = 45)
4
5 modelos <- list(
6   promedio = meanf(entrenamiento, h = 8), # el promedio histórico
7   ingenuo = naive(entrenamiento, h = 8), # "igual que la última semana"
8   ets = forecast(ets(entrenamiento), h = 8)
9 )
10 map_dfr(modelos, ~ as_tibble(t(accuracy(.x, prueba)["Test set",
11                                   c("RMSE", "MAE", "MAPE")])),
12         .id = "modelo") %>%
13   mutate(across(-modelo, ~ round(.x, 1)))

```

```

# A tibble: 3 x 4
  modelo    RMSE    MAE  MAPE
  <chr>    <dbl> <dbl> <dbl>
1 promedio 3606. 3318. 17.5
2 ingenuo  8140. 7370. 43.1

```

```
3 ets      3606. 3318.  17.5
```

El modelo ingenuo (“la próxima semana venderemos lo mismo que esta”) es el peor: su error porcentual medio (MAPE) es de 43 %. El promedio histórico y ETS empatan con un MAPE de 17.5 %, y no es casualidad: al ver una serie sin tendencia ni patrón, ETS elige un modelo de nivel constante con suavizamiento casi nulo, que es... el promedio. Cuando un modelo sofisticado no le gana al promedio, **reporta el modelo sencillo**: es igual de preciso y cualquiera en la junta lo entiende.

```
1 # Pronóstico final con todas las semanas
2 modelo_final <- ets(serie)
3 pronostico <- forecast(modelo_final, h = 8, level = c(80, 95))
4 modelo_final$method
5
6 tabla_pronostico <- tibble(
7   semana    = max(ventas_semana$semana) + weeks(1:8),
8   estimado  = as.numeric(pronostico$mean),
9   inf_80    = as.numeric(pronostico$lower[, "80%"]),
10  sup_80     = as.numeric(pronostico$upper[, "80%"]),
11  inf_95     = as.numeric(pronostico$lower[, "95%"]),
12  sup_95     = as.numeric(pronostico$upper[, "95%"]))
13
14 tabla_pronostico %>%
15   mutate(across(-semana, round)) %>%
16   slice(1:3)
```

```
[1] "ETS(M,N,N)"
# A tibble: 3 x 6
  semana      estimado inf_80 sup_80 inf_95 sup_95
<date>      <dbl> <dbl> <dbl> <dbl> <dbl>
1 2024-01-01  18315 12370 24261  9222 27408
2 2024-01-08  18315 12370 24261  9222 27408
3 2024-01-15  18315 12370 24261  9222 27408
```

```
1 ggplot() +
2   geom_ribbon(data = tabla_pronostico,
3             aes(x = semana, ymin = inf_95, ymax = sup_95),
4             fill = color_resalte, alpha = 0.15) +
5   geom_ribbon(data = tabla_pronostico,
6             aes(x = semana, ymin = inf_80, ymax = sup_80),
7             fill = color_resalte, alpha = 0.25) +
8   geom_line(data = ventas_semana, aes(x = semana, y = ventas),
9            color = color_principal, linewidth = 0.8) +
10  geom_line(data = tabla_pronostico, aes(x = semana, y = estimado),
11          color = color_resalte, linewidth = 0.8) +
```

```

12  annotate("text", x = as.Date("2024-02-19"), y = 29000, size = 3.2,
13         hjust = 1, vjust = 0, color = color_resalte,
14         label = "Pronóstico e intervalos de 80% y 95%") +
15  scale_y_continuous(labels = dollar, limits = c(0, NA)) +
16  scale_x_date(date_labels = "%b", date_breaks = "2 months") +
17  labs(title = "Ventas semanales y pronóstico a 8 semanas",
18       subtitle = "Sin tendencia ni patrón: el pronóstico es un nivel constante",
19       x = NULL, y = "Ventas por semana") +
20  tema_libro()

```

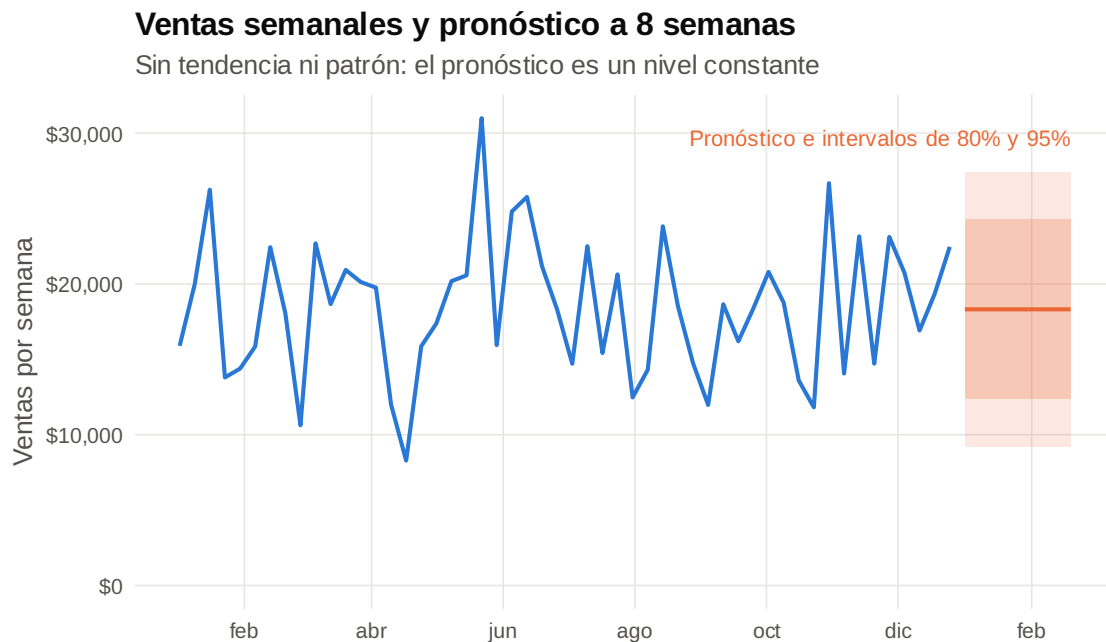


Figura 9.11. Ventas semanales de 2023 (azul) y pronóstico ETS con intervalos de 80 % y 95 % (naranja).

Hallazgo: el modelo elegido es ETS(M,N,N), un nivel constante sin tendencia ni estacionalidad. Pronostica \$18,315 por semana, con un intervalo de 80 % entre \$12,370 y \$24,261 y de 95 % entre \$9,222 y \$27,408. **Implicación:** para planear, la dirección puede usar unos \$18,300 semanales (unos \$80,000 al mes), pero debe prepararse para semanas un tercio por encima o por debajo. Lo que *no* debe hacer es usar este pronóstico para decidir el inventario de temporada navideña: con un año de datos el modelo no sabe que existe una temporada.

En la empresa: Qué datos harían falta para un mejor pronóstico

Cuando el modelo no alcanza, el entregable honesto es *qué necesitas* para mejorarlo: (1) al menos dos o tres años de historia para estimar estacionalidad anual; (2) un calendario de promociones y días festivos (Buen Fin, Navidad, regreso a clases) como variables explicativas; (3) datos de inventario, para distinguir “no se vendió porque no había” de “no se vendió porque nadie lo quiso”; y (4) pronósticos por tienda o categoría, donde los patrones suelen ser más claros que en el total. Presentar esta lista es mucho más profesional que presentar un modelo complicado

que no le gana al promedio.

▲ Cuidado

Un intervalo de confianza amplio no es un error del modelo: es información. Decir “venderemos \$18,300 por semana” sin intervalo hace creer a la dirección que la cifra es precisa; decir “entre \$12,400 y \$24,300 en 8 de cada 10 semanas” le permite planear inventario y flujo de caja con el riesgo a la vista.

9.5 Fase 4: el tablero ejecutivo

Con los hallazgos en la mano, diseña el tablero que los gerentes consultarán cada lunes. El orden importa: **primero se diseña en papel, después se programa**. Un tablero ejecutivo sigue la jerarquía de lectura en “Z”: arriba los KPIs (la respuesta a “¿cómo vamos?”), luego los filtros y después las gráficas que explican los números (Figura 9.12).

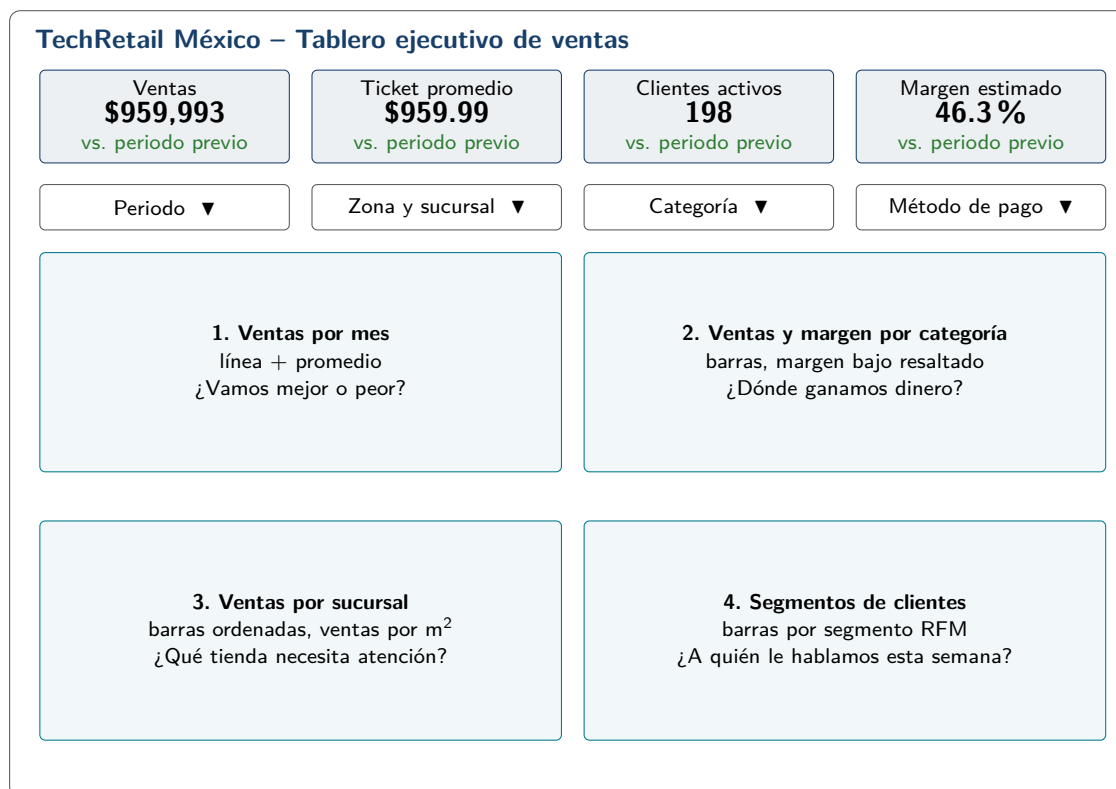


Figura 9.12. Maqueta del tablero: KPIs arriba, filtros en una fila y cuatro gráficas, cada una con la pregunta que responde.

Tabla 9.7. Cada elemento del tablero y la pregunta que responde.

| Elemento | Pregunta de la dirección | Hallazgo que lo motivó |
|---------------------|--|------------------------|
| KPIs (4 tarjetas) | ¿Cómo vamos en el periodo? | Preguntas 1 y 6 |
| Filtros | ¿Y en mi zona, mi tienda o mi categoría? | Gerentes de tienda |
| Ventas por mes | ¿Hay tendencia o solo variación? | Sec. 9.3.2 |
| Categorías y margen | ¿Vendemos lo que deja margen? | Sec. 9.3.3 |
| Ventas por sucursal | ¿Qué tienda rinde más por m ² ? | Sec. 9.3.4 |
| Segmentos RFM | ¿Cuántos clientes están en riesgo? | Sec. 9.4.1 |

No necesitas empezar desde cero. En el Módulo 6 construiste la app `latex/codigo/apps/03-dashboard-ejecutivo/app.R`, que ya tiene fila de KPIs, filtros globales, pestañas y descargas, con toda la lógica en `R/funciones_dashboard.R`. Para adaptarla al proyecto haz tres cambios:

1. **Leer el modelo de la Fase 1** en lugar de los CSV: así el tablero y el reporte muestran exactamente los mismos números.
2. **Agregar el filtro de zona** y la tarjeta de margen estimado.
3. **Agregar la pestaña de segmentos** con el resultado RFM guardado en la Fase 3.

Adaptación de funciones_dashboard.R al proyecto

```

1 # ---- Cambio 1: cargar el modelo en estrella de la Fase 1 ----
2 cargar_datos_dashboard <- function(ruta = "resultados/modelo_techretail.rds") {
3   modelo <- readRDS(ruta)
4   modelo$hechos_ventas %>%
5     left_join(select(modelo$dim_producto, producto_id, nombre_producto,
6                     categoria), by = "producto_id") %>%
7     left_join(transmute(modelo$dim_tienda, tienda_id, zona, ciudad,
8                       tienda = sub("Sucursal ", "", nombre_tienda)),
9               by = "tienda_id") %>%
10    left_join(select(modelo$dim_cliente, cliente_id,
11                    nombre_cliente = nombre, segmento = segmento_edad,
12                    es_premium), by = "cliente_id") %>%
13    mutate(mes = floor_date(fecha, "month")) %>%
14    rename(total_transaccion = total, costo = costo_unitario)
15 }
16
17 # ---- Cambio 2: filtro de zona (en app.R, dentro de dashboardSidebar) ----
18 selectizeInput("zonas", "Zonas:", choices = sort(unique(datos$zona)),
19               multiple = TRUE, options = list(placeholder = "Todas"))
20
21 # ... y en filtrar_datos(), un argumento más:
22 filtrar_datos <- function(datos, fechas, tiendas = NULL,
23                           categorias = NULL, zonas = NULL) {
24   datos <- filter(datos, fecha >= fechas[1], fecha <= fechas[2])
25   if (length(zonas) > 0)   datos <- filter(datos, zona %in% zonas)
26   if (length(tiendas) > 0) datos <- filter(datos, tienda %in% tiendas)

```

```

27   if (length(categorias) > 0) datos <- filter(datos, categoria %in% categorias)
28   datos
29 }
30
31 # ---- Cambio 3: segmentos RFM guardados en la Fase 3 ----
32 segmentos_rfm <- readRDS("resultados/segmentos_rfm.rds")
33 output$graf_rfm <- renderPlot({
34   datos_validos() %>%
35     distinct(cliente_id) %>%
36     inner_join(segmentos_rfm, by = "cliente_id") %>%
37     count(segmento) %>%
38     ggplot(aes(x = n, y = reorder(segmento, n))) +
39     geom_col(fill = color_principal) +
40     labs(x = "Clientes", y = NULL) +
41     tema_dashboard()
42 }, res = 96)

```

Para que el cambio 3 funcione, la Fase 3 debe guardar los segmentos. Es una línea, pero es la que conecta el modelo con el producto:

```

1 saveRDS(select(rfm, cliente_id, recencia, frecuencia, monto, segmento),
2         "resultados/segmentos_rfm.rds")
3 saveRDS(tabla_pronostico, "resultados/pronostico_semanal.rds")
4 file.exists(c("resultados/segmentos_rfm.rds",
5              "resultados/pronostico_semanal.rds"))

```

```
[1] TRUE TRUE
```

Consejo

Un tablero ejecutivo tiene *pocos* elementos. Si alguien pide “una gráfica más”, pregunta qué decisión tomará con ella. Los análisis de detalle (vendedores, horas, Pareto) viven mejor en pestañas secundarias o en el reporte, no en la portada del tablero.

9.6 Fase 5: el reporte ejecutivo

La dirección no abrirá tu script ni navegará el tablero durante una hora. Leerá, si acaso, una o dos páginas. El **reporte ejecutivo** sigue una estructura fija:

1. **Situación** (2–3 líneas): qué se analizó y el resultado principal en un número.
2. **Hallazgos** (3–5 viñetas): cada uno con su cifra y su implicación.
3. **Recomendaciones** (3 máximo): acción, responsable e impacto esperado.
4. **Próximos pasos**: qué sigue y qué datos faltan.

En el Módulo 8 automatizaste reportes con R Markdown; la plantilla `latex/codigo/reportes/02-reporte-ejecutivo.Rmd` ya genera un reporte por tienda y mes con parámetros. Para el proyecto la adaptarías para leer `resultados/modelo_techretail.rds` y agregar las secciones de hallazgos. Lo más valioso que aprendiste ahí es a **redactar las frases con código**: si los datos cambian, el texto se actualiza solo y nunca queda un número viejo en el reporte.

```

1 # Insumos del resumen: todos salen de objetos calculados antes
2 mejor_mes <- ventas_mes %>% slice_max(ventas, n = 1)
3 peor_mes  <- ventas_mes %>% slice_min(ventas, n = 1)
4 top_cat   <- ventas_categoria %>% slice_max(ventas, n = 1)
5 top_tienda <- ventas_tienda %>% slice_max(ventas, n = 1)
6 p_premium <- t.test(gasto ~ grupo, data = gasto_cliente)$p.value
7 en_riesgo <- perfil %>% filter(segmento == "En riesgo")
8 meses_largos <- c("enero", "febrero", "marzo", "abril", "mayo", "junio",
9                  "julio", "agosto", "septiembre", "octubre",
10                 "noviembre", "diciembre")
11
12 resumen_ejecutivo <- c(
13   str_glue("SITUACIÓN. En 2023 TechRetail vendió {pesos(kpis$ventas_totales)} ",
14           "en {comma(kpis$transacciones)} transacciones (ticket promedio ",
15           "de {pesos(kpis$ticket_promedio)}) a {kpis$clientes} clientes, ",
16           "con un margen bruto estimado de ",
17           "{percent(kpis$margen / kpis$ventas_totales, 0.1)}."),
18   str_glue("VENTAS. Sin tendencia anual: el mejor mes fue ",
19           "{meses_largos[mejor_mes$mes]} ({pesos(mejor_mes$ventas)}) y el ",
20           "peor {meses_largos[peor_mes$mes]} ({pesos(peor_mes$ventas)})"),
21   str_glue("PRODUCTOS. {top_cat$categoria} lidera con ",
22           "{percent(top_cat$participacion, 0.1)} de las ventas, pero con ",
23           "margen de {percent(top_cat$pct_margen, 0.1)}; ",
24           "{sum(productos$margen_pct < 0)} productos tienen costo mayor ",
25           "al precio de catálogo."),
26   str_glue("TIENDAS. La que más vende es {top_tienda$nombre_tienda} ",
27           "({pesos(top_tienda$ventas)}) con solo {top_tienda$tamano_m2} m²."),
28   str_glue("CLIENTES. Los clientes premium no gastan más que el resto ",
29           "(p = {round(p_premium, 2)}); {en_riesgo$clientes} clientes ",
30           "están en riesgo (en promedio {round(en_riesgo$recencia)} días ",
31           "sin comprar).")
32 )
33 cat(strwrap(resumen_ejecutivo, width = 68, exdent = 2), sep = "\n")

```

SITUACIÓN. En 2023 TechRetail vendió \$959,993 en 1,000 transacciones (ticket promedio de \$960) a 198 clientes, con un margen bruto estimado de 46.3%.

VENTAS. Sin tendencia anual: el mejor mes fue diciembre (\$96,394) y el peor abril (\$57,038).

PRODUCTOS. Belleza lidera con 19.0% de las ventas, pero con margen de 30.8%; 10 productos tienen costo mayor al precio de catálogo.

TIENDAS. La que más vende es Sucursal Outlet (\$114,377) con solo

350 m².

CLIENTES. Los clientes premium no gastan más que el resto ($p = 0.57$); 29 clientes están en riesgo (en promedio 195 días sin comprar).

Con esas frases, el reporte de una página para la dirección queda así (las recomendaciones y próximos pasos sí son redacción tuya, porque requieren criterio):

En la empresa: Reporte ejecutivo – TechRetail México, resultados 2023

Situación. En 2023 TechRetail vendió \$959,993 en 1,000 transacciones a 198 clientes, con ticket promedio de \$960 y margen bruto estimado de 46.3 %. Las ventas no muestran tendencia: oscilaron entre \$57,038 (abril) y \$96,394 (diciembre).

Hallazgos.

- **Margen en riesgo.** 10 de 50 productos tienen costo mayor al precio de catálogo y 252 ventas (25 %) se cobraron por debajo del costo.
- **Categoría líder con margen bajo.** Belleza aporta 19.0 % de las ventas con 30.8 % de margen; Mascotas y Alimentos superan el 60 %.
- **Formatos chicos productivos.** La Outlet (350 m²) es la sucursal que más vende (\$114,377); la Premium (800 m²) es quinta.
- **Clientes.** Los premium no gastan más ($p = 0.57$); 109 clientes (Campeones y Leales) generan 74 % de las ventas y 29 están en riesgo.

Recomendaciones.

1. *Corregir costos y precios (Compras y Sistemas, 30 días).* Revisar los 10 productos con margen negativo y bloquear en el punto de venta precios fuera de un rango del catálogo. Es la acción con mayor impacto directo en margen.
2. *Rediseñar el programa premium (Marketing, 60 días).* Hoy no genera más gasto; atar beneficios a metas de compra y medirlo con un grupo de control.
3. *Reactivar clientes en riesgo (Marketing, 30 días).* Campaña dirigida al segmento “En riesgo” con un indicador de reactivación.

Próximos pasos. Validar la llave de vendedores con Recursos Humanos, acumular historia para pronosticar con estacionalidad y publicar el tablero semanal para los gerentes.

Hazlo tú

Agrega al vector `resumen_ejecutivo` una frase sobre la sucursal con *menos* ventas y otra sobre el método de pago más usado. Usa `slice_min()` y `str_glue()` como en el código anterior.

9.7 Fase 6: presentar los resultados

Tienes 20 minutos con el comité directivo. Es el momento en que un proyecto de seis semanas se gana o se pierde. La técnica que mejor funciona es el **storytelling con datos**: contar los resultados como una historia de tres actos.

Concepto clave: Contexto, conflicto y resolución

- **Contexto:** dónde estamos. “En 2023 vendimos \$960 mil en 10 sucursales, con un margen estimado de 46 %.”
- **Conflicto:** qué problema u oportunidad encontramos. “Pero uno de cada cuatro tickets se cobró por debajo del costo, nuestra categoría líder tiene el margen más bajo y el programa premium no genera más gasto.”
- **Resolución:** qué proponemos y qué ganaremos. “Tres acciones en 60 días: corregir precios, rediseñar el premium y reactivar a los clientes en riesgo.”

La Tabla 9.8 propone una estructura de 9 diapositivas. Nota que la recomendación aparece en la segunda: los directivos quieren saber *desde el inicio* qué les vas a pedir.

Tabla 9.8. Estructura de la presentación al comité directivo.

| # | Diapositiva | Contenido |
|---|-----------------------|--|
| 1 | Título | “Resultados 2023 y tres decisiones para 2024” |
| 2 | Resumen ejecutivo | Situación en un número y las 3 recomendaciones |
| 3 | Contexto | KPIs globales (4 tarjetas, como el tablero) |
| 4 | Ventas en el tiempo | Figura 9.3: sin tendencia, mucha variación |
| 5 | Conflicto 1: margen | Auditoría: márgenes negativos y ventas bajo costo; Figura 9.4 |
| 6 | Conflicto 2: clientes | Premium sin efecto (Figura 9.9) y clientes en riesgo |
| 7 | Tiendas | Los formatos chicos venden tanto como los grandes (Figura 9.5) |
| 8 | Resolución | Tres acciones, responsable, plazo e indicador |
| 9 | Próximos pasos | Datos que faltan y calendario del tablero |

Cada diapositiva lleva un **título de acción**: una frase con la conclusión, no un tema. “Ventas por categoría” es un tema; “Belleza vende más pero deja menos margen” es una conclusión. Si el comité solo lee los títulos, debe entender toda la historia.

▲ Cuidado

Errores comunes al presentar resultados:

- **Mostrar el proceso en lugar del resultado.** A la dirección no le importa cuántos `left_join()` hiciste; le importa qué decidir. El método va en un anexo.
- **Saturar las diapositivas.** Una idea por diapositiva y una gráfica por idea.
- **Exagerar la certeza.** Si el margen es estimado o el pronóstico tiene un intervalo amplio, dilo. Tu credibilidad vale más que un hallazgo espectacular.

- **Esconder los resultados “negativos”.** “El premium no funciona” o “no hay productos estrella” son hallazgos que ahorran dinero.
- **No terminar con una petición concreta.** Cierra con qué necesitas que aprueben hoy.

Hazlo tú

Escribe los títulos de acción de las 9 diapositivas de la Tabla 9.8 usando tus propios números. Léelos en voz alta en orden: ¿cuentan la historia completa sin ver las gráficas?

9.8 El script maestro y la reproducibilidad

A lo largo del módulo ejecutamos todo en un solo documento, pero un proyecto real se organiza en carpetas y scripts numerados, uno por fase. Así cualquier persona (incluido tú dentro de seis meses) puede reconstruir todos los resultados con un solo comando.

Estructura del proyecto

```

1 techretail-bi/
2 |-- techretail-bi.Rproj      # proyecto de RStudio (fija el directorio)
3 |-- README.md              # qué es, cómo ejecutarlo, hallazgos
4 |-- datasets/              # datos crudos: NUNCA se modifican
5 |   |-- transacciones.csv
6 |   |-- clientes.csv productos.csv tiendas.csv empleados.csv
7 |   `-- ventas_retail.csv
8 |-- R/
9 |   |-- 00_ejecutar_todo.R   # script maestro
10 |   |-- 01_etl.R            # Fase 1: auditoría, limpieza, modelo
11 |   |-- 02_kpis_eda.R       # Fase 2: KPIs y figuras
12 |   |-- 03_modelos.R        # Fase 3: RFM y pronóstico
13 |   `-- funciones.R         # auditar(), tema_libro(), pesos()...
14 |-- dashboard/
15 |   |-- app.R               # Fase 4 (adaptada del Módulo 6)
16 |   `-- R/funciones_dashboard.R
17 |-- reportes/
18 |   `-- reporte_ejecutivo.Rmd # Fase 5 (adaptado del Módulo 8)
19 |-- datos/
20 |   `-- techretail.sqlite    # modelo en estrella para SQL
21 `-- resultados/
22     |-- modelo_techretail.rds
23     |-- segmentos_rfm.rds
24     |-- pronostico_semanal.rds
25     |-- figuras/            # PNG/PDF de cada gráfica
26     `-- bitacora_decisiones.csv

```

El script maestro ejecuta las fases en orden, mide cuánto tarda cada una y se detiene si alguna falla:

R/00_ejecutar_todo.R

```

1  # =====
2  # 00_ejecutar_todo.R - Reconstruye TODO el proyecto TechRetail
3  # Ejecuta con el directorio de trabajo en la raíz del proyecto
4  # (abre techretail-bi.Rproj y corre: source("R/00_ejecutar_todo.R"))
5  # =====
6  inicio <- Sys.time()
7
8  # Un solo lugar para los parámetros del proyecto
9  fecha_corte <- as.Date("2024-01-01") # fija: resultados reproducibles
10 set.seed(123)                        # k-means y cualquier muestreo
11
12 fases <- c("R/funciones.R",          # funciones compartidas
13           "R/01_etl.R",              # -> resultados/modelo_techretail.rds
14           "R/02_kpis_eda.R",         # -> resultados/figuras/
15           "R/03_modelos.R")          # -> segmentos y pronóstico
16
17 for (fase in fases) {
18   cat("\n==>", fase, "\n")
19   t0 <- Sys.time()
20   source(fase, encoding = "UTF-8", echo = FALSE)
21   cat("    listo en", round(difftime(Sys.time(), t0, units = "secs"), 1),
22       "segundos\n")
23 }
24
25 # Fase 5: el reporte se genera con los resultados recién calculados
26 rmarkdown::render("reportes/reporte_ejecutivo.Rmd",
27                  output_dir = "resultados",
28                  knit_root_dir = getwd())
29
30 # Registro de la sesión: versiones de R y paquetes usados
31 writeLines(capture.output(sessionInfo()), "resultados/session_info.txt")
32 cat("\nProyecto completo en",
33     round(difftime(Sys.time(), inicio, units = "mins"), 1), "minutos\n")
34
35 # Fase 4: el tablero se abre aparte (queda esperando en el navegador)
36 # shiny::runApp("dashboard")

```

 Consejo

Reglas de reproducibilidad que aplicaste en este proyecto: (1) rutas relativas, nunca `setwd("C:/Users/...")`; (2) fecha de corte fija, nunca `Sys.Date()` en los cálculos; (3) `set.seed()` antes de cualquier proceso aleatorio; (4) los datos crudos no se editan, todo cambio queda en código; (5) controles `stopifnot()` que detienen el proceso si algo no cuadra; (6) `sessionInfo()` guardado junto a los resultados. Si quieres ir más lejos, el paquete **renv** congela las versiones exactas de los paquetes del proyecto.

9.9 Errores comunes en un proyecto de BI

Tabla 9.9. Errores frecuentes en proyectos integradores y cómo evitarlos.

| Error | Síntoma | Cómo evitarlo |
|---|--|--|
| Join que duplica filas | Las ventas totales del tablero no cuadran con contabilidad | <code>stopifnot(nrow(...)==nrow(...))</code> después de cada join |
| Usar <code>Sys.Date()</code> | Los números cambian cada vez que corres el script | Fecha de corte fija en un solo lugar |
| Sumar dos fuentes de ventas | Ventas infladas o “dos verdades” | Documentar qué fuente es oficial para cada pregunta |
| Borrar filas “raras” | Totales que nadie reconoce | Marcar con banderas y decidir por análisis |
| Promediar promedios | Ticket por zona incorrecto | Calcular desde la tabla de hechos: suma / conteo |
| Comparar totales de grupos de distinto tamaño | “La zona Norte es la mejor” (tiene 4 tiendas) | Normalizar: por tienda, por m ² , por empleado |
| Afirmar estacionalidad con un año | Promesas que no se cumplen | Pedir 2–3 años o decir la limitación |
| Días de la semana dependientes del idioma | El tablero mezcla “Monday” y “lunes” | Vectores propios de nombres (<code>dias_es</code> , <code>meses_es</code>) |

9.10 Checklist de entrega y portafolio

Antes de entregar (a tu jefe, a tu profesor o a un reclutador), revisa la Tabla 9.10.

Tabla 9.10. Checklist final de entrega.

| Área | Punto a revisar | ✓ |
|---------------|--|--------------------------|
| Datos | Los datos crudos están intactos y el ETL corre desde cero | ☐ |
| | La auditoría de calidad y la bitácora de decisiones están incluidas | ☐ |
| | Hay controles de conciliación (filas y totales) | ☐ |
| Código | <code>00_ejecutar_todo.R</code> reconstruye todo sin pasos manuales | ☐ |
| | Rutas relativas, fecha de corte fija y <code>set.seed()</code> | ☐ |
| | Código comentado y funciones para lo que se repite | ☐ |
| Análisis | Las 20 preguntas tienen respuesta (o una razón para no tenerla) | ☐ |
| | Cada gráfica tiene título de acción y formato de negocio en los ejes | ☐ |
| | Los modelos se validaron y sus limitaciones están escritas | ☐ |
| Productos | El tablero abre, filtra y muestra los mismos números que el reporte | ☐ |
| | El reporte ejecutivo cabe en 1–2 páginas y se genera con código | ☐ |
| | La presentación tiene 8–10 diapositivas y cierra con una petición | ☐ |
| Documentación | README con objetivo, estructura, cómo ejecutar y hallazgos | ☐ |
| | Sin datos personales (correos, teléfonos) en los archivos publicados | ☐ |

9.10.1 Cómo presentar el proyecto en tu portafolio

Un proyecto integrador bien presentado es tu mejor carta para conseguir trabajo en BI, sobre todo si no tienes experiencia formal. Los reclutadores dedican menos de un minuto a cada portafolio, así que facilita la lectura:

- **GitHub.** Sube el proyecto como repositorio público con la estructura de la Sección 9.8. El `README.md` es tu portada: empieza con el problema de negocio en dos líneas, una captura del tablero, los 3-4 hallazgos principales con sus números y la instrucción para ejecutarlo. Agrega un `.gitignore` para no subir archivos pesados o temporales.
- **Tablero en línea.** Publica la app en shinyapps.io (Módulo 6) y enlázala desde el README: que el reclutador pueda *usar* tu trabajo sin instalar nada.
- **Reporte en HTML.** Publica el reporte ejecutivo con GitHub Pages o RPubS.
- **LinkedIn.** Escribe una publicación con la historia (contexto, conflicto, resolución), una gráfica y el enlace al repositorio. En la sección de proyectos, describe el resultado, no la herramienta: “Detecté que 25 % de las ventas se cobraba por debajo del costo” dice más que “Usé dplyr y ggplot2”.
- **Sé transparente.** Aclara que los datos son simulados. Los reclutadores lo valoran; lo que evalúan es tu razonamiento.

En la empresa: Cómo hablar del proyecto en una entrevista

Prepara una versión de dos minutos con el método **STAR**: *Situación* (cadena de 10 tiendas sin una sola versión de la verdad), *Tarea* (construir el sistema de BI y recomendar prioridades), *Acción* (ETL auditado, modelo en estrella, KPIs, RFM, pronóstico, tablero y reporte automatizado) y *Resultado* (tres recomendaciones con impacto en margen y clientes). Ten a la mano un hallazgo de calidad de datos: casi siempre te preguntarán “¿qué problema encontraste en los datos y cómo lo resolviste?”.

Resumen del módulo

En este proyecto final integraste todo el libro en un caso de negocio:

- Convertiste un correo de la dirección en 20 preguntas, entregables, cronograma y rúbrica.
- Construiste un ETL auditable: elegiste la fuente oficial de ventas, detectaste márgenes negativos, precios fuera de catálogo, productos inactivos con ventas y llaves dudosas, y documentaste cada decisión sin borrar datos.
- Armaste un modelo en estrella, lo guardaste en RDS y SQLite y lo protegiste con controles de conciliación.
- Calculaste KPIs y convertiste cada tabla en un hallazgo con números, incluidos los resultados “negativos” (sin tendencia, sin productos estrella, premium sin efecto).
- Segmentaste clientes con RFM y k-means y pronosticaste ventas semanales, reconociendo que un año no basta para estimar estacionalidad.

- Diseñaste el tablero, redactaste el reporte con frases generadas por código y estructuraste la presentación como una historia.

| Fase | Técnicas y funciones clave | Módulo |
|------------|---|--------|
| ETL | <code>read_csv()</code> , <code>anti_join()</code> , <code>left_join()</code> , <code>stopifnot()</code> | 2 |
| Modelo | <code>saveRDS()</code> , <code>dbWriteTable()</code> , <code>dbGetQuery()</code> | 5 |
| KPIs y EDA | <code>summarise()</code> , <code>lag()</code> , <code>cumsum()</code> , <code>ggplot()</code> | 3, 4 |
| Pruebas | <code>t.test()</code> , <code>wilcox.test()</code> , <code>kruskal.test()</code> , <code>cor.test()</code> | 4 |
| Modelos | <code>scale()</code> , <code>kmeans()</code> , <code>silhouette()</code> , <code>ets()</code> , <code>forecast()</code> | 7 |
| Productos | <code>shinydashboard</code> , <code>str_glue()</code> , <code>rmarkdown::render()</code> | 6, 8 |

9.11 Retos de extensión

Estos retos amplían el proyecto con preguntas que la dirección haría en la siguiente junta. Cada solución del Apéndice A es autocontenida: carga sus paquetes y datos desde `datasets/`.

Ejercicio 9.1: Tasa de recompra y días entre compras

Nivel: **Intermedio**

Con `transacciones.csv`, calcula: (a) qué porcentaje de los clientes activos compró dos veces o más en 2023; (b) la mediana de días entre compras consecutivas de un mismo cliente; y (c) cuántos clientes compraron dos veces o más en un plazo de 30 días. Interpreta qué implica para una campaña de “segunda compra”.

 Solución en la página 706

Ejercicio 9.2: Productividad por m² y por empleado

Nivel: **Intermedio**

Calcula para cada sucursal las ventas por m² y por empleado, y su ranking en cada medida. Haz una gráfica de barras de ventas por m² resaltando la mejor sucursal. ¿Qué sucursal cambia más de posición entre ventas totales y ventas por m²?

 Solución en la página 707

Ejercicio 9.3: Alerta de stock bajo

Nivel: **Intermedio**

Con `productos.csv` y las unidades vendidas en 2023, calcula para cada producto *activo* la venta promedio diaria y los días de cobertura (`stock_actual` / venta diaria). Genera una lista de alerta con los productos que tienen menos de 30 días de cobertura o están por debajo del `stock_minimo`, ordenada por urgencia.

 Solución en la página 708

Ejercicio 9.4: Cohortes por mes de primera compraNivel: **Avanzado**

Como la fecha de registro no es confiable, define la cohorte de cada cliente por el mes de su *primera compra*. Construye la matriz de cohortes: para cada cohorte, qué porcentaje de sus clientes volvió a comprar 1, 2 y 3 meses después. Muestra las cohortes del primer trimestre y comenta el resultado.

 Solución en la página 710**Ejercicio 9.5: Transacciones atípicas**Nivel: **Avanzado**

Detecta transacciones atípicas con dos reglas: (a) total fuera del rango intercuartílico ampliado (regla de Tukey, $1.5 \times \text{IQR}$); (b) precio de venta que se aleja más de 3 veces del precio mediano cobrado por ese mismo producto (hacia arriba o hacia abajo). ¿Cuántas marca cada regla? Arma la lista para que Auditoría las revise, con tienda y vendedor.

 Solución en la página 711**Ejercicio 9.6: Simulación de una campaña de descuento**Nivel: **Avanzado**

Marketing propone un 10 % de descuento a todos durante un mes. Con `ventas_retail.csv` (la fuente que registra descuentos), compara la cantidad promedio vendida con y sin descuento. Después, con los datos de `transacciones.csv` de diciembre, simula (1,000 repeticiones con `set.seed(123)`) el ingreso del mes si el descuento aumenta las unidades entre 0 % y 15 % (incremento uniforme aleatorio). ¿En qué porcentaje de las simulaciones la campaña pierde ingreso? ¿Qué incremento de unidades se necesita para no perder?

 Solución en la página 713**Ejercicio 9.7: Ranking mensual de vendedores con funciones de ventana**Nivel: **Avanzado**

Calcula el ranking de los 25 vendedores en cada mes con `min_rank()` y, con funciones de ventana, su venta acumulada en el año y el cambio de lugar respecto al mes anterior (`lag()`). ¿Qué vendedores estuvieron más meses en el top 5? ¿Quién tuvo la mayor subida de un mes a otro?

 Solución en la página 714**Ejercicio 9.8: Paquete de Excel para cada gerente de tienda**Nivel: **Reto**

Escribe una función `paquete_gerente(id)` que genere, en `resultados/gerentes/`, un archivo de Excel por sucursal con tres hojas: *Resumen* (KPIs de la tienda contra el promedio de la cadena), *Mensual* (ventas por mes y crecimiento) y *Top productos* (10 productos más vendidos

en esa tienda). Genera los 10 archivos con `walk()` y comprueba cuántos se crearon.

🔗 Solución en la página 715

Soluciones de los ejercicios

Aquí están las soluciones completas de todos los ejercicios del libro, ordenadas por módulo. Cada caja indica la página donde está el enunciado.



Consejo

Intenta cada ejercicio por tu cuenta antes de mirar la solución, aunque tardes. Cuando la consultes, compara tu código con el de la solución: casi siempre hay más de una forma correcta de resolverlo. Si tu resultado coincide con el de la solución, tu enfoque es válido aunque el código sea distinto.

Todas las soluciones son **autocontenidas**: cada una carga sus paquetes y sus datos, así que puedes copiarlas a un script nuevo y ejecutarlas tal cual con el directorio de trabajo en la carpeta del curso (la que contiene `datasets/`). También las encuentras en la carpeta `latex/codigo/` con el nombre `sol-NN.R`.

A.1 Módulo 1. Fundamentos de R y RStudio

Solución del ejercicio 1.1

Guardamos cada dato en una variable para que el cálculo sea fácil de revisar y de modificar. El cupón se aplica al subtotal *antes* del IVA, así que el IVA también baja.

Enunciado en la página 73

```
1 # ---- Datos de la compra -----
2 precio_audifonos <- 899
3 precio_teclado <- 1299
4 precio_usb <- 189
5 tasa_iva <- 0.16
6 cupon <- 0.10
7
8 # ---- Cálculos -----
9 subtotal <- 2 * precio_audifonos + 1 * precio_teclado + 3 * precio_usb
10 iva <- subtotal * tasa_iva
11 total <- subtotal + iva
12 subtotal_cupon <- subtotal * (1 - cupon)
13 total_cupon <- subtotal_cupon * (1 + tasa_iva)
14 ahorro <- total - total_cupon
```

```

15
16 subtotal
17 iva
18 total
19 total_cupon
20 ahorro
21 scales::dollar(total_cupon, accuracy = 0.01)

```

```

[1] 3664
[1] 586.24
[1] 4250.24
[1] 3825.216
[1] 425.024
[1] "$3,825.22"

```

El subtotal es de \$3,664 y con IVA el cliente pagaría \$4,250.24. Con el cupón paga \$3,825.22: ahorra \$425.02, que es justo el 10% del total con IVA, porque el descuento reduce también la base del impuesto.

Solución del ejercicio 1.2

El punto de equilibrio responde: ¿cuánto hay que vender para que la utilidad bruta alcance a cubrir los gastos fijos? Como cada peso vendido deja **margen_bruto** pesos de utilidad bruta, basta dividir los gastos fijos entre ese margen (expresado como proporción).

Enunciado en la página 73

```

1 ingresos <- 612400
2 costo_ventas <- 379700
3 gastos_fijos <- 158000
4 meta <- 580000
5
6 utilidad_bruta <- ingresos - costo_ventas
7 utilidad_operacion <- utilidad_bruta - gastos_fijos
8 margen_bruto <- utilidad_bruta / ingresos      # Proporción (0 a 1)
9 margen_operacion <- utilidad_operacion / ingresos
10
11 utilidad_bruta
12 utilidad_operacion
13 round(margen_bruto * 100, 1)
14 round(margen_operacion * 100, 1)
15 round(ingresos / meta * 100, 1)                # Cumplimiento %
16
17 punto_equilibrio <- gastos_fijos / margen_bruto
18 round(punto_equilibrio, 2)
19 round(ingresos - punto_equilibrio, 2)          # Colchón en pesos
20 round((ingresos / punto_equilibrio - 1) * 100, 1) # Colchón en %

```

```
[1] 232700
[1] 74700
[1] 38
[1] 12.2
[1] 105.6
[1] 415810.9
[1] 196589.1
[1] 47.3
```

La sucursal tuvo un margen bruto de 38 % y un margen de operación de 12.2 %, y superó su meta (105.6 %). Necesita vender unos \$415,811 al mes para no perder dinero; vendió unos \$196,589 más que eso, es decir, está 47.3 % por encima de su punto de equilibrio: tiene un colchón cómodo si las ventas bajan.

Solución del ejercicio 1.3

Para los montos quitamos el signo de pesos y las comas con `gsub()` antes de convertir. Para las fechas indicamos el formato "`%d/%m/%Y`".

Enunciado en la página 73

```
1 montos_texto <- c("$1,250.50", "$980.00", "$12,400.75", "$3,105.20")
2 fechas_texto <- c("05/01/2023", "19/01/2023", "02/02/2023", "28/02/2023")
3
4 # (a) Montos a número
5 montos <- as.numeric(gsub("$,", "", montos_texto))
6 montos
7 sum(montos)
8
9 # (b) Fechas a Date
10 fechas <- as.Date(fechas_texto, format = "%d/%m/%Y")
11 fechas
12 class(fechas)
13
14 # (c) Vencimiento a 30 días en formato dd/mm/aaaa
15 vencimientos <- fechas + 30
16 format(vencimientos, "%d/%m/%Y")
17
18 # (d) Día de la semana del vencimiento
19 weekdays(vencimientos)
```

```
[1] 1250.50  980.00 12400.75  3105.20
[1] 17736.45
[1] "2023-01-05" "2023-01-19" "2023-02-02" "2023-02-28"
[1] "Date"
[1] "04/02/2023" "18/02/2023" "04/03/2023" "30/03/2023"
[1] "sábado" "sábado" "sábado" "jueves"
```

Las cuatro facturas suman \$17,736.45. Observa la última: vence el 30 de marzo porque febrero de 2023 tuvo 28 días; R lleva el calendario por ti. Tres de las cuatro facturas vencen en sábado, algo que conviene saber si tu área de cobranza no trabaja fines de semana.

Solución del ejercicio 1.4

Creamos el vector con nombres y aplicamos las funciones de resumen. Para el crecimiento dividimos cada diferencia entre el mes anterior.

Enunciado en la página 73

```

1 meses <- c("ene", "feb", "mar", "abr", "may", "jun",
2           "jul", "ago", "sep", "oct", "nov", "dic")
3 ventas_mes <- c(420, 385, 455, 402, 478, 510, 495, 530, 468, 501, 560, 640)
4 names(ventas_mes) <- meses
5
6 sum(ventas_mes)                # Total anual (miles)
7 round(mean(ventas_mes), 1)     # Promedio mensual
8 names(which.max(ventas_mes))   # Mejor mes
9 names(which.min(ventas_mes))   # Peor mes
10 sum(ventas_mes > 500)          # Meses sobre 500 mil
11 names(ventas_mes)[ventas_mes > 500]
12
13 crecimiento <- diff(ventas_mes) / head(ventas_mes, -1) * 100
14 round(crecimiento, 1)
15
16 acumulado <- cumsum(ventas_mes)
17 acumulado
18 names(which(acumulado > 3000)[1]) # Primer mes que supera 3 millones

```

```

[1] 5844
[1] 487
[1] "dic"
[1] "feb"
[1] 5
[1] "jun" "ago" "oct" "nov" "dic"
  feb  mar  abr  may  jun  jul  ago  sep  oct  nov  dic
-8.3 18.2 -11.6 18.9  6.7 -2.9  7.1 -11.7  7.1 11.8 14.3
ene feb mar abr may jun jul ago sep oct nov dic
420  805 1260 1662 2140 2650 3145 3675 4143 4644 5204 5844
[1] "jul"

```

La tienda vendió 5.84 millones en el año, con un promedio de 487 mil por mes. Diciembre fue el mejor mes (640 mil) y febrero el peor (385 mil). Cinco meses superaron los 500 mil, todos en el segundo semestre salvo junio. Los mayores crecimientos mensuales fueron en mayo (+18.9%) y marzo (+18.2%), y diciembre cerró fuerte (+14.3%), típico de la temporada navideña. El acumulado superó los 3 millones en julio.

Solución del ejercicio 1.5

La clave es usar `na.rm = TRUE` en los cálculos y `is.na()` para localizar los faltantes.

Enunciado en la página 73

```

1  ventas_dias <- c(8200, 7650, NA, 9100, 10400, NA, 8800, 12100, NA, 9650)
2
3  # (a) Días sin dato
4  sum(is.na(ventas_dias))
5  which(is.na(ventas_dias))
6
7  # (b) Promedio y total con los días disponibles
8  promedio_obs <- mean(ventas_dias, na.rm = TRUE)
9  round(promedio_obs, 2)
10 sum(ventas_dias, na.rm = TRUE)
11
12 # (c) Días con dato que superaron la meta de 9,000
13 sobre_meta <- sum(ventas_dias > 9000, na.rm = TRUE)
14 sobre_meta
15 round(sobre_meta / sum(!is.na(ventas_dias)) * 100, 1)
16
17 # (d) Imputar con el promedio y comparar
18 ventas_imputadas <- ventas_dias
19 ventas_imputadas[is.na(ventas_imputadas)] <- promedio_obs
20 round(sum(ventas_imputadas), 2)
21 sum(ifelse(is.na(ventas_dias), 0, ventas_dias)) # Si imputáramos con 0

```

```

[1] 3
[1] 3 6 9
[1] 9414.29
[1] 65900
[1] 4
[1] 57.1
[1] 94142.86
[1] 65900

```

Faltan los datos de los días 3, 6 y 9. Con los 7 días disponibles la venta promedio es de \$9,414.29 y 4 de esos 7 días (57.1 %) superaron la meta. Si rellenamos los faltantes con el promedio, el total estimado de los 10 días es de \$94,142.86. Rellenar con cero daría \$65,900, como si la tienda no hubiera vendido nada esos días: subestimaría la venta en casi 30 %. Un **NA** significa “no sabemos”, no “cero”. Nota que `sum(ventas_dias > 9000, na.rm = TRUE)` necesita el `na.rm`: la comparación con un **NA** da **NA**.

Solución del ejercicio 1.6

Combinamos `trimws()` y `toupper()` para limpiar, y `sprintf()` para los mensajes y los códigos.

Enunciado en la página 74

```

1 nombres <- c(" ana LÓPEZ", "Carlos pérez ", "MARÍA gonzález")
2 ciudades <- c("cdmx", "Monterrey ", " CDMX")
3 compras <- c(15230.5, 8900, 42100.75)
4
5 # (a) Limpieza
6 nombres_limpios <- toupper(trimws(nombres))
7 ciudades_limpias <- toupper(trimws(ciudades))
8 nombres_limpios
9 ciudades_limpias
10
11 # (b) Mensaje por cliente
12 sprintf("%s (%s) compró %s", nombres_limpios, ciudades_limpias,
13         scales::dollar(compras, accuracy = 0.01))
14
15 # (c) Códigos de cliente
16 codigos <- sprintf("CLI-%03d", seq_along(nombres))
17 codigos
18
19 # (d) Clientes de CDMX
20 sum(ciudades_limpias == "CDMX")

```

```

[1] "ANA LÓPEZ"      "CARLOS PÉREZ"    "MARÍA GONZÁLEZ"
[1] "CDMX"          "MONTERREY"      "CDMX"
[1] "ANA LÓPEZ (CDMX) compró $15,230.50"
[2] "CARLOS PÉREZ (MONTERREY) compró $8,900.00"
[3] "MARÍA GONZÁLEZ (CDMX) compró $42,100.75"
[1] "CLI-001" "CLI-002" "CLI-003"
[1] 2

```

`toupper()` respeta los acentos (*Pérez* pasa a *PÉREZ*). Antes de limpiar habríamos contado un solo cliente en “CDMX”; después de limpiar son dos. `%03d` rellena con ceros a tres dígitos y `seq_along()` numera tantos clientes como haya.

Solución del ejercicio 1.7

Creamos el data frame, agregamos las columnas calculadas con `ifelse()` y resumimos con `aggregate()`.

Enunciado en la página 74

```

1 empleados <- data.frame(
2   nombre = c("Ana", "Bruno", "Carla", "Diego", "Elena", "Fernando",
3             "Gabriela", "Héctor"),
4   departamento = c("Ventas", "Ventas", "Operaciones", "TI", "Ventas",
5                   "Operaciones", "TI", "Ventas"),
6   salario = c(18500, 22000, 16800, 35000, 19500, 17200, 41000, 24500),
7   antigüedad = c(3, 6, 2, 4, 1, 8, 7, 5),
8   desempeno = c(4, 5, 3, 4, 2, 4, 5, 3)
9 )
10
11 # (a) Bono: 10% del salario anual si el desempeño es 4 o más
12 empleados$bono <- ifelse(empleados$desempeno >= 4,
13                          empleados$salario * 12 * 0.10, 0)
14 empleados
15
16 # (b) Ventas ordenado por salario (mayor a menor)
17 ventas_dep <- empleados[empleados$departamento == "Ventas", ]
18 ventas_dep[order(ventas_dep$salario, decreasing = TRUE),
19            c("nombre", "salario", "desempeno")]
20
21 # (c) Salario promedio y bonos totales por departamento
22 aggregate(salario ~ departamento, data = empleados, FUN = mean)
23 aggregate(bono ~ departamento, data = empleados, FUN = sum)
24
25 # (d) Costo anual de nómina por departamento
26 empleados$costo_anual <- empleados$salario * 12 + empleados$bono
27 costo_dep <- tapply(empleados$costo_anual, empleados$departamento, sum)
28 costo_dep
29 names(which.max(costo_dep))

```

| | nombre | departamento | salario | antigüedad | desempeno | bono |
|---|----------|--------------|---------|------------|-----------|-------|
| 1 | Ana | Ventas | 18500 | 3 | 4 | 22200 |
| 2 | Bruno | Ventas | 22000 | 6 | 5 | 26400 |
| 3 | Carla | Operaciones | 16800 | 2 | 3 | 0 |
| 4 | Diego | TI | 35000 | 4 | 4 | 42000 |
| 5 | Elena | Ventas | 19500 | 1 | 2 | 0 |
| 6 | Fernando | Operaciones | 17200 | 8 | 4 | 20640 |
| 7 | Gabriela | TI | 41000 | 7 | 5 | 49200 |
| 8 | Héctor | Ventas | 24500 | 5 | 3 | 0 |

| | nombre | salario | desempeno |
|---|--------|---------|-----------|
| 8 | Héctor | 24500 | 3 |
| 2 | Bruno | 22000 | 5 |
| 5 | Elena | 19500 | 2 |
| 1 | Ana | 18500 | 4 |

| | departamento | salario |
|---|--------------|---------|
| 1 | Operaciones | 17000 |
| 2 | TI | 38000 |
| 3 | Ventas | 21125 |


```

departamento bono
1 Operaciones 20640
2          TI 91200
3          Ventas 48600
Operaciones          TI          Ventas
      428640      1003200      1062600
[1] "Ventas"

```

TI tiene el salario promedio más alto (\$38,000) y reparte la mayor bolsa de bonos (\$91,200), pero el departamento con mayor costo anual de nómina es Ventas (\$1,062,600), simplemente porque tiene cuatro personas contra dos de TI. Para comparar departamentos de distinto tamaño conviene mirar tanto el total como el promedio por persona.

Solución del ejercicio 1.8

Primero calculamos el cumplimiento de forma vectorizada; después lo usamos en el `ifelse()`, en el bucle y en el `while`.

Enunciado en la página 74

```

1 vendedores <- c("Luis", "Marta", "Nora", "Oscar", "Paula", "Raul")
2 ventas_trim <- c(185, 240, 132, 310, 198, 260)
3 meta <- 200
4
5 # (a) Semáforo
6 cumplimiento <- ventas_trim / meta * 100
7 semaforo <- ifelse(cumplimiento >= 100, "Verde",
8                   ifelse(cumplimiento >= 85, "Amarillo", "Rojo"))
9 semaforo
10
11 # (b) Reporte con for
12 for (k in seq_along(vendedores)) {
13   cat(sprintf("%-6s %4d mil %6.1f%% %s\n", vendedores[k],
14       ventas_trim[k], cumplimiento[k], semaforo[k]))
15 }
16
17 # (c) Trimestres para que el de menor venta alcance la meta
18 venta <- min(ventas_trim)
19 trimestres <- 0
20 while (venta < meta) {
21   venta <- venta * 1.08
22   trimestres <- trimestres + 1
23 }
24 vendedores[which.min(ventas_trim)]
25 trimestres
26 round(venta, 1)

```

```

[1] "Amarillo" "Verde"      "Rojo"      "Verde"      "Amarillo" "Verde"
Luis      185 mil    92.5% Amarillo
Marta     240 mil   120.0% Verde
Nora      132 mil    66.0% Rojo
Oscar     310 mil   155.0% Verde
Paula     198 mil    99.0% Amarillo
Raul      260 mil   130.0% Verde
[1] "Nora"
[1] 6
[1] 209.5

```

Tres vendedores están en verde, dos en amarillo (Luis y Paula, cerca de la meta) y Nora en rojo con 66 % de cumplimiento. Aun creciendo 8 % cada trimestre, Nora necesitaría 6 trimestres (año y medio) para llegar a la meta: es un caso para un plan de acompañamiento, no solo para esperar.

Solución del ejercicio 1.9

Las validaciones van al inicio de cada función, antes de los cálculos. El punto de equilibrio en unidades es $\text{costos_fijos} / (\text{precio} - \text{costo_variable})$, redondeado hacia arriba porque no se venden fracciones de unidad.

Enunciado en la página 75

```

1  # Precio final: aplica descuento (proporción) y luego IVA
2  precio_final <- function(precio, descuento = 0, iva = 0.16) {
3    if (any(precio < 0)) stop("El precio no puede ser negativo")
4    if (descuento < 0 || descuento > 1) {
5      stop("El descuento debe estar entre 0 y 1 (0.15 = 15%)")
6    }
7    round(precio * (1 - descuento) * (1 + iva), 2)
8  }
9
10 # Unidades mínimas para no perder dinero
11 punto_equilibrio <- function(costos_fijos, precio, costo_variable) {
12   if (precio <= costo_variable) {
13     stop("El precio debe ser mayor que el costo variable")
14   }
15   ceiling(costos_fijos / (precio - costo_variable))
16 }
17
18 # (c) Uso
19 catalogo <- c(laptop = 18999, mouse = 349, monitor = 5499, silla = 3899)
20 precio_final(catalogo, descuento = 0.20)
21 precio_final(1000) # Solo IVA
22 punto_equilibrio(85000, precio = 450, costo_variable = 280)
23
24 # (d) ¿Qué pasa con un descuento inválido? tryCatch() atrapa el error
25 tryCatch(precio_final(1000, descuento = 1.5),

```

```
26      error = function(e) conditionMessage(e))
```

```
      laptop    mouse  monitor    silla
17631.07    323.87  5103.07  3618.27
[1] 1160
[1] 500
[1] "El descuento debe estar entre 0 y 1 (0.15 = 15%)"
```

El negocio necesita vender 500 unidades para cubrir sus \$85,000 de costos fijos: cada unidad deja \$170 de contribución y $500 \times 170 = 85,000$. `tryCatch()` ejecuta el código y, si ocurre un error, en lugar de detener el script llama a la función de `error`; aquí solo devolvemos el mensaje. Sin `tryCatch()`, R mostraría:

```
1 precio_final(1000, descuento = 1.5)
```

```
Error in precio_final(1000, descuento = 1.5) :
  El descuento debe estar entre 0 y 1 (0.15 = 15%)
```

Solución del ejercicio 1.10

Leemos el catálogo, buscamos los márgenes negativos (un problema de calidad de datos o de precios) y exportamos dos hojas.

Enunciado en la página 75

```
1 library(readr)
2 library(writexl)
3
4 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
5
6 # (a) Tamaño
7 dim(productos)
8
9 # (b) Margen negativo
10 negativos <- productos[productos$margen_pct < 0, ]
11 nrow(negativos)
12 peores <- negativos[order(negativos$margen_pct),
13                      c("nombre_producto", "categoria", "precio_catalogo",
14                        "costo", "margen_pct")]
15 head(peores, 5)
16
17 # (c) Margen promedio por categoría
18 margen_cat <- aggregate(margen_pct ~ categoria, data = productos,
19                          FUN = mean)
```

```

20 margen_cat$margen_pct <- round(margen_cat$margen_pct, 1)
21 margen_cat[order(margen_cat$margen_pct, decreasing = TRUE), ]
22
23 # (d) Problemas de inventario
24 table(productos$estado_stock)
25 sum(productos$estado_stock %in% c("Stock Bajo", "Sin Stock"))
26
27 # (e) Exportar
28 dir.create("resultados", showWarnings = FALSE)
29 write_xlsx(list(Margen_negativo = as.data.frame(peores),
30               Por_categoria = margen_cat),
31             "resultados/calidad_catalogo.xlsx")

```

```

[1] 50 13
[1] 10
# A tibble: 5 x 5
  nombre_producto categoria  precio_catalogo costo margen_pct
  <chr>           <chr>          <dbl> <dbl>    <dbl>
1 Producto 05     Alimentos        25.5  355.   -1292.
2 Producto 46     Automotriz        64.5  178.   -175.
3 Producto 32     Hogar          143.   210.   -46.7
4 Producto 26     Electrónica      115.   159.   -38.3
5 Producto 29     Belleza         298.   392.   -31.6
  categoria margen_pct
4 Deportes      73.2
8 Libros        73.0
9 Mascotas      44.5
5 Electrónica   43.3
7 Juguetes      39.5
3 Belleza       38.6
10 Ropa          35.7
2 Automotriz    16.8
6 Hogar         -6.3
1 Alimentos    -161.6

Stock Bajo  Stock OK
          4         46
[1] 4

```

Conclusiones para Compras: 10 de los 50 productos (20%) se venden por debajo de su costo; en el peor caso el costo es unas 14 veces el precio, lo que sugiere errores de captura o precios desactualizados que hay que revisar antes de cualquier análisis de rentabilidad. Además, 4 productos tienen stock bajo y ninguno está agotado. (Los datos del curso son simulados, por eso aparecen casos tan extremos; en una empresa real serían una alerta inmediata.)

Solución del ejercicio 1.11

Resumimos con `aggregate()` y `tapply()`, y usamos `%in%` para agrupar las dos tarjetas.

Enunciado en la página 75

```

1 library(readr)
2
3 trans <- read_csv("datasets/transacciones.csv", show_col_types = FALSE)
4
5 # (a) Resumen por método de pago
6 resumen_pago <- aggregate(total_transaccion ~ metodo_pago, data = trans,
7                             FUN = sum)
8 resumen_pago$transacciones <- as.numeric(table(trans$metodo_pago)[
9     resumen_pago$metodo_pago])
10 resumen_pago$ticket_promedio <- round(resumen_pago$total_transaccion /
11                                         resumen_pago$transacciones, 2)
12 resumen_pago
13
14 # (b) Porcentaje del monto pagado con tarjeta
15 con_tarjeta <- trans$metodo_pago %in% c("Tarjeta Crédito",
16                                         "Tarjeta Débito")
17 round(sum(trans$total_transaccion[con_tarjeta]) /
18       sum(trans$total_transaccion) * 100, 1)
19
20 # (c) Mejor mes y día con más transacciones
21 por_mes <- tapply(trans$total_transaccion, trans$mes, sum)
22 names(which.max(por_mes))
23 round(max(por_mes), 2)
24 sort(table(trans$dia_semana), decreasing = TRUE)[1:3]
25
26 # (d) Exportar y comprobar
27 dir.create("resultados", showWarnings = FALSE)
28 write_csv(resumen_pago, "resultados/resumen_metodo_pago.csv")
29 saveRDS(resumen_pago, "resultados/resumen_metodo_pago.rds")
30 identical(resumen_pago, readRDS("resultados/resumen_metodo_pago.rds"))

```

```

      metodo_pago total_transaccion transacciones ticket_promedio
1      Efectivo      303734.65           315          964.24
2 Tarjeta Crédito      351170.69           350         1003.34
3 Tarjeta Débito      220855.30           233          947.88
4 Transferencia       84232.05           102          825.80
[1] 59.6
[1] "2023-12"
[1] 96393.61

viernes miércoles    domingo
    158         155         146

```

```
[1] TRUE
```

La tarjeta de crédito es el método más usado y el de mayor monto, y las dos tarjetas juntas concentran el 59.6 % del dinero cobrado: la disponibilidad de terminales es crítica. El ticket promedio casi no cambia entre métodos. Como `resumen_pago` es un data frame de R base, el RDS lo recupera idéntico.

Solución del ejercicio 1.12

Dividimos el problema en piezas: una función que resume una tienda, un bucle que la aplica a las diez y, al final, el ranking y la exportación. Para traer el nombre, la ciudad y el tamaño de la tienda usamos `match()`, que encuentra en qué fila de `tiendas` está cada `tienda_id`.

Enunciado en la página 75

```
1 library(readr)
2 library(writexl)
3
4 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
5 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
6
7 # Resume las ventas de una tienda en un data frame de una fila
8 resumen_tienda <- function(id, ventas, tiendas) {
9   if (!id %in% tiendas$tienda_id) stop("La tienda no existe: ", id)
10  v <- ventas[ventas$tienda_id == id, ]
11  fila_tienda <- match(id, tiendas$tienda_id)
12  venta_mes <- tapply(v$total, v$mes, sum)
13  data.frame(
14    tienda = tiendas$nombre_tienda[fila_tienda],
15    ciudad = tiendas$ciudad[fila_tienda],
16    num_ventas = nrow(v),
17    venta_total = round(sum(v$total), 2),
18    ticket_promedio = round(mean(v$total), 2),
19    mejor_mes = names(which.max(venta_mes)),
20    venta_m2 = round(sum(v$total) / tiendas$tamano_m2[fila_tienda], 2)
21  )
22 }
23
24 resumen_tienda(1, ventas, tiendas)           # Prueba con una tienda
25
26 # Aplicar a las 10 tiendas y unir las filas
27 lista_resmenes <- lapply(tiendas$tienda_id, resumen_tienda,
28                           ventas = ventas, tiendas = tiendas)
29 ranking <- do.call(rbind, lista_resmenes)
30 ranking <- ranking[order(ranking$venta_total, decreasing = TRUE), ]
31 rownames(ranking) <- NULL
32
33 # Ranking impreso
34 cat("RANKING DE TIENDAS 2023\n")
```

```

35 for (k in seq_len(nrow(ranking))) {
36   cat(sprintf("%2d. %-17s %11s %3d ventas %8s por m2\n", k,
37             ranking$tienda[k], scales::dollar(ranking$venta_total[k]),
38             ranking$num_ventas[k], scales::dollar(ranking$venta_m2[k],
39             accuracy = 0.01)))
40 }
41
42 # Exportar
43 dir.create("resultados", showWarnings = FALSE)
44 write_xlsx(ranking, "resultados/ranking_tiendas.xlsx")
45 ranking$tienda[which.max(ranking$venta_m2)]

```

```

      tienda ciudad num_ventas venta_total ticket_promedio
1 Sucursal Centro CDMX          32    43898.54         1371.83
mejor_mes venta_m2
1 2023-01      87.8
RANKING DE TIENDAS 2023
 1. Sucursal Outlet    $70,154.72  47 ventas    $200.44 por m2
 2. Sucursal Premium  $61,369.12  46 ventas     $76.71 por m2
 3. Sucursal Plaza B   $55,290.32  37 ventas    $115.19 por m2
 4. Sucursal Norte     $54,846.35  38 ventas    $121.88 por m2
 5. Sucursal Sur       $54,065.76  37 ventas     $90.11 por m2
 6. Sucursal Plaza A   $54,050.18  34 ventas     $77.21 por m2
 7. Sucursal Oeste     $53,611.23  37 ventas     $97.47 por m2
 8. Sucursal Centro    $43,898.54  32 ventas     $87.80 por m2
 9. Sucursal Express   $42,171.47  34 ventas    $140.57 por m2
10. Sucursal Este     $24,458.92  23 ventas     $61.15 por m2
[1] "Sucursal Outlet"

```

¿Qué nos dice esto? La Sucursal Outlet es la que más vende en total (\$70,155) y, con solo 350 m², también la más productiva por metro cuadrado (\$200.44/m²). En cambio, la Sucursal Premium, la más grande (800 m²), vende menos de la mitad por metro cuadrado. La Sucursal Este es la de menor venta total, con solo 23 ventas en el año. Recuerda que en estos datos cada registro es una venta diaria asignada al azar a una tienda, así que las diferencias entre tiendas son pequeñas; con datos reales, este mismo script te daría el ranking verdadero cada mes.

`lapply()` aplica la función a cada `tienda_id` y devuelve una lista de data frames de una fila; `do.call(rbind, lista)` los apila en una sola tabla. Es el patrón de R base para “hacer lo mismo por cada grupo y juntar los resultados”; en el Módulo 2 lo harás con `group_by()` y `summarise()`.

A.2 Módulo 2. Manipulación de datos con dplyr y tidyr

Solución del ejercicio 2.1

Filtramos con dos condiciones, elegimos columnas y ordenamos; después resumimos el resultado con `summarise()`.

Enunciado en la página 159

```
1 library(tidyverse)
```

```
1 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
2
3 tienda3_desc <- ventas %>%
4   filter(tienda_id == 3, descuento_pct >= 10) %>% # ambas condiciones
5   select(fecha, producto_id, descuento_pct, total) %>%
6   arrange(desc(total))
7 tienda3_desc
8
9 tienda3_desc %>%
10  summarise(ventas = n(), total = sum(total))
```

```
# A tibble: 4 x 4
  fecha      producto_id descuento_pct total
  <date>      <dbl>      <dbl> <dbl>
1 2023-11-05         27         10 1439.
2 2023-03-02         26         20 1227.
3 2023-12-20         17         10 1098.
4 2023-07-08          6         10  742.
# A tibble: 1 x 2
  ventas total
  <int> <dbl>
1     4 4506.
```

La tienda 3 hizo 4 ventas con descuento de 10% o más, que suman \$4,506; la mayor fue de \$1,439 el 5 de noviembre. Observa que en `summarise(total = sum(total))` el nombre nuevo puede coincidir con el de la columna: dentro de la función, `total` todavía es la columna original.

Solución del ejercicio 2.2

Combinamos tres condiciones en `filter()` (el producto activo y el stock bajo el mínimo) y calculamos el faltante con `mutate()`.

Enunciado en la página 159


```

1 library(tidyverse)
2 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
3
4 resurtir <- productos %>%
5   filter(activo, stock_actual < stock_minimo) %>%
6   mutate(faltante = stock_minimo - stock_actual) %>%
7   select(nombre_producto, proveedor, stock_actual, stock_minimo,
8         faltante) %>%
9   arrange(desc(faltante))
10 resurtir
11 resurtir %>% count(proveedor, wt = faltante, sort = TRUE)

```

```

# A tibble: 3 x 5
  nombre_producto proveedor  stock_actual stock_minimo faltante
  <chr>           <chr>      <dbl>         <dbl>     <dbl>
1 Producto 03     Proveedor 3      22          26         4
2 Producto 33     Proveedor 1      42          45         3
3 Producto 06     Proveedor 1      36          38         2
# A tibble: 2 x 2
  proveedor      n
  <chr>         <dbl>
1 Proveedor 1     5
2 Proveedor 3     4

```

Solo 3 productos activos están bajo su mínimo. Por unidades faltantes hay que llamar primero al Proveedor 1 (5 unidades en dos productos), aunque el producto con el mayor faltante individual (Producto 03, 4 unidades) es del Proveedor 3. `filter(activo, ...)` funciona porque `activo` ya es lógica.

Solución del ejercicio 2.3

`case_when()` evalúa las reglas en orden, así que basta con dar el límite superior de cada rango. El porcentaje se calcula después de `count()`.

Enunciado en la página 159

```

1 library(tidyverse)
2 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
3
4 clientes %>%
5   mutate(rango_edad = case_when(
6     edad < 30 ~ "18-29",
7     edad < 45 ~ "30-44",
8     edad < 60 ~ "45-59",
9     .default = "60+"
10   )) %>%
11   count(rango_edad) %>%

```

```
12 mutate(pct = round(n / sum(n) * 100, 1))
```

```
# A tibble: 4 x 3
  rango_edad      n  pct
  <chr>      <int> <dbl>
1 18-29         33 16.5
2 30-44         52  26
3 45-59         51 25.5
4 60+          64  32
```

El grupo más grande es el de 60 años o más (64 clientes, 32 %) y el más chico el de 18 a 29 (33 clientes, 16.5 %): una base de clientes madura.

Solución del ejercicio 2.4

El promedio de una columna lógica es la proporción de **TRUE**: con `mean(es_premium)` obtenemos directamente el porcentaje de premium.

Enunciado en la página 159

```
1 library(tidyverse)
2 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
3
4 perfil_ciudad <- clientes %>%
5   group_by(ciudad) %>%
6   summarise(clientes = n(),
7             pct_premium = round(mean(es_premium) * 100, 1),
8             edad_prom = round(mean(edad), 1),
9             .groups = "drop") %>%
10  arrange(desc(clientes))
11 perfil_ciudad
12 perfil_ciudad %>% slice_max(pct_premium, n = 1)
```

```
# A tibble: 10 x 4
  ciudad      clientes pct_premium edad_prom
  <chr>      <int>      <dbl>      <dbl>
1 Puebla         28        42.9        51.3
2 CDMX           27        48.1        56.2
3 Guadalajara    22        27.3        49.2
4 Cancún         20         30         45
5 Toluca          20         30         51
6 León           18        16.7        39.1
7 Mérida          17        23.5        47.6
8 Querétaro       17        35.3        53.7
9 Tijuana         17        41.2        37.8
10 Monterrey      14        71.4        51.1
```

```
# A tibble: 1 x 4
  ciudad    clientes pct_premium edad_prom
  <chr>      <int>    <dbl>    <dbl>
1 Monterrey    14      71.4     51.1
```

Puebla y la CDMX tienen más clientes, pero Monterrey tiene la mayor proporción de premium: 71.4% de sus 14 clientes. Es una plaza chica pero de alto valor.

Solución del ejercicio 2.5

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 159

```
1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4
5 transacciones %>%
6   mutate(dia = wday(fecha, label = TRUE, abbr = FALSE)) %>%
7   group_by(dia) %>%
8   summarise(transacciones = n(),
9             ingresos = sum(total_transaccion),
10             ticket_promedio = round(mean(total_transaccion), 2),
11             .groups = "drop") %>%
12   mutate(participacion_pct = round(ingresos / sum(ingresos) * 100, 1))
```

```
# A tibble: 7 x 5
  dia      transacciones ingresos ticket_promedio participacion_pct
  <ord>      <int>    <dbl>    <dbl>          <dbl>
1 domingo    146  148703.    1019.         15.5
2 lunes     136  128878.     948.         13.4
3 martes    129  119364.     925.         12.4
4 miércoles  155  141329.     912.         14.7
5 jueves    146  145280.     995.         15.1
6 viernes    158  161266.    1021.         16.8
7 sábado    130  115172.     886.          12
```

El viernes es el día con más transacciones (158) e ingresos (\$161,266, 16.8% del total), seguido del domingo; el sábado es el más flojo (12%). Conviene reforzar el personal los viernes y domingos.

Solución del ejercicio 2.6

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 160

```

1 library(tidyverse)
2 empleados <- read_csv("datasets/empleados.csv", show_col_types = FALSE)
3 fecha_corte <- as.Date("2024-01-01")
4
5 empleados_antig <- empleados %>%
6   mutate(
7     antig_calc = time_length(interval(fecha_ingreso, fecha_corte),
8                                     "years"),
9     diferencia = abs(antig_calc - antiguedad_anos),
10    grupo = case_when(
11      antig_calc < 3 ~ "1. Menos de 3 años",
12      antig_calc <= 6 ~ "2. 3 a 6 años",
13      .default      = "3. Más de 6 años"
14    )
15  )
16 max(empleados_antig$diferencia)           # diferencia máxima
17
18 empleados_antig %>%
19   group_by(grupo) %>%
20   summarise(empleados = n(),
21             salario_prom = round(mean(salario_mensual), 2),
22             .groups = "drop")

```

```

[1] 0.05245902
# A tibble: 3 x 3
  grupo      empleados salario_prom
<chr>      <int>      <dbl>
1 1. Menos de 3 años      41      27354.
2 2. 3 a 6 años          34      27821.
3 3. Más de 6 años       25      27228.

```

La diferencia máxima entre ambos cálculos es de 0.05 años (unos 19 días): la columna original dividía los días entre 365, mientras que `time_length()` considera los años bisiestos. El salario promedio casi no cambia con la antigüedad (entre \$27,228 y \$27,821): en estos datos simulados la antigüedad no se refleja en el sueldo, algo que en una empresa real merecería una revisión de política salarial.

Solución del ejercicio 2.7

Solo necesitamos del catálogo la categoría y el costo, así que los seleccionamos dentro del *join*.

Enunciado en la página 160

```

1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
5

```

```

6 margen_cat <- transacciones %>%
7   left_join(select(productos, producto_id, categoria, costo),
8             by = "producto_id") %>%
9   mutate(utilidad = (precio_venta - costo) * cantidad) %>%
10  group_by(categoria) %>%
11  summarise(con_perdida = sum(utilidad < 0), # antes de resumir
12            ingresos = sum(total_transaccion),
13            utilidad = sum(utilidad),
14            .groups = "drop") %>%
15  mutate(margen_pct = round(utilidad / ingresos * 100, 1)) %>%
16  arrange(desc(utilidad))
17 margen_cat

```

```

# A tibble: 10 x 5
  categoria    con_perdida ingresos utilidad margen_pct
  <chr>          <int>     <dbl>   <dbl>     <dbl>
1 Alimentos         13  121543.  73870.     60.8
2 Electrónica        27  132672.  73067.     55.1
3 Juguetes           30  125093.  57714.     46.1
4 Belleza            66  182143.  56054.     30.8
5 Automotriz         26  104085.  55538.     53.4
6 Deportes           20   86171.  48301.     56.1
7 Mascotas           10   57668.  35423.     61.4
8 Libros             21   57162.  25757.     45.1
9 Hogar              24   63407.  13079.     20.6
10 Ropa              15   30048.   5835.     19.4

```

Alimentos es la categoría que más utilidad deja (\$73,870), aunque Belleza tiene los mayores ingresos (\$182,143). Belleza es también la más problemática: 66 transacciones con utilidad negativa y un margen de 30.8 %. Ropa y Hogar tienen los márgenes más bajos (19–21 %). Cuidado con el orden dentro de `summarise()`: si calculas `utilidad = sum(utilidad)` antes de `sum(utilidad < 0)`, la segunda usaría la suma ya calculada y el conteo saldría 0.

Solución del ejercicio 2.8

Trabajamos con listas de clientes *únicos* de cada tabla (`distinct()`) para que los conteos sean de clientes y no de ventas.

Enunciado en la página 160

```

1 library(tidyverse)
2 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
3 transacciones <- read_csv("datasets/transacciones.csv",
4                           show_col_types = FALSE)
5 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
6
7 cli_ventas <- ventas %>% distinct(cliente_id)

```

```

8 cli_trans <- transacciones %>% distinct(cliente_id)
9
10 # (a) En ambas tablas
11 cli_ventas %>% semi_join(cli_trans, by = "cliente_id") %>% nrow()
12 # (b) En transacciones pero no en ventas
13 cli_trans %>% anti_join(cli_ventas, by = "cliente_id") %>% nrow()
14 # (c) Registrados que no aparecen en ninguna
15 compradores <- bind_rows(cli_ventas, cli_trans) %>% distinct()
16 clientes %>% anti_join(compradores, by = "cliente_id") %>%
17   select(cliente_id, nombre, fecha_registro)

```

```

[1] 170
[1] 28
# A tibble: 0 x 3
#   i 3 variables: cliente_id <dbl>, nombre <chr>,
#   fecha_registro <date>

```

170 clientes compraron en ambas tablas y 28 aparecen solo en transacciones. Ningún cliente registrado quedó sin comprar en alguna de las dos: el resultado de (c) es una tabla vacía (0 filas), lo cual también es una respuesta válida.

Solución del ejercicio 2.9

Calculamos el porcentaje en formato largo (agrupado por tienda) y al final pivotamos a ancho.

Enunciado en la página 160

```

1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
5
6 pagos_tienda <- transacciones %>%
7   left_join(select(tiendas, tienda_id, nombre_tienda), by = "tienda_id") %>%
8   group_by(nombre_tienda, metodo_pago) %>%
9   summarise(ingresos = sum(total_transaccion), .groups = "drop_last") %>%
10  mutate(pct = round(ingresos / sum(ingresos) * 100, 1)) %>%
11  ungroup() %>%
12  select(-ingresos) %>%
13  pivot_wider(names_from = metodo_pago, values_from = pct,
14             values_fill = 0) %>%
15  arrange(desc(Efectivo))
16 pagos_tienda

```

```

# A tibble: 10 x 5
  nombre_tienda    Efectivo `Tarjeta Crédito` `Tarjeta Débito`

```

| | <chr> | <dbl> | <dbl> | <dbl> |
|----|------------------|-------|-------|-------|
| 1 | Sucursal Oeste | 45.5 | 35.6 | 10.7 |
| 2 | Sucursal Este | 37.5 | 27 | 25.6 |
| 3 | Sucursal Norte | 37.1 | 43.5 | 14.2 |
| 4 | Sucursal Plaza A | 36.2 | 21.5 | 31.4 |
| 5 | Sucursal Plaza B | 33.5 | 42.8 | 18.1 |
| 6 | Sucursal Sur | 29.3 | 36.4 | 29.6 |
| 7 | Sucursal Outlet | 29.1 | 29.5 | 28 |
| 8 | Sucursal Centro | 28.9 | 44 | 21.2 |
| 9 | Sucursal Premium | 24.6 | 40.8 | 25.2 |
| 10 | Sucursal Express | 18.8 | 47.6 | 20.8 |

i 1 more variable: Transferencia <dbl>

La Sucursal Oeste es la más dependiente del efectivo: 45.5% de sus ingresos. En el otro extremo, la Sucursal Express cobra solo 18.8% en efectivo y 47.6% con tarjeta de crédito. Como los porcentajes se calcularon con el resultado agrupado por tienda (`.groups = "drop_last"`), cada fila suma 100.

Solución del ejercicio 2.10

Primero completamos los meses faltantes; si no, `lag()` compararía contra el mes equivocado. `group_by(tienda_id)` hace que `lag()` y `cumsum()` reinicien en cada tienda.

Enunciado en la página 160

```
1 library(tidyverse)
2 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
3
4 tienda_mes <- ventas %>%
5   group_by(tienda_id, mes) %>%
6   summarise(ingresos = sum(total), .groups = "drop") %>%
7   complete(tienda_id, mes, fill = list(ingresos = 0)) %>%
8   arrange(tienda_id, mes) %>%
9   group_by(tienda_id) %>%
10  mutate(cambio_pesos = ingresos - lag(ingresos),
11         acumulado    = cumsum(ingresos)) %>%
12  ungroup()
13 nrow(tienda_mes)
14
15 tienda_mes %>%
16   slice_min(cambio_pesos, n = 3) %>%           # las 3 mayores caídas
17   mutate(across(where(is.numeric), ~ round(.x, 2)))
```

```
[1] 120
# A tibble: 3 x 5
  tienda_id mes      ingresos cambio_pesos acumulado
  <dbl> <chr>      <dbl>      <dbl>      <dbl>
1      8 2023-08      781.      -12373.    50452
```

| | | | | | |
|---|---|---------|-------|--------|--------|
| 2 | 8 | 2023-03 | 1222. | -9457. | 17226. |
| 3 | 2 | 2023-05 | 3698. | -7626. | 25458. |

La tabla completa tiene 120 filas (10 tiendas $\times$ 12 meses) gracias a `complete()`. La mayor caída fue la de la tienda 8 en agosto: \$12,373 menos que en julio, al vender solo \$781 en el mes.

Solución del ejercicio 2.11

Seguimos el mismo orden que en la sección 2.10: duplicados, texto, números y fechas, y al final reglas de negocio.

Enunciado en la página 161

```

1 library(tidyverse)
2 lista_proveedor <- tibble(
3   id      = c(1, 2, 2, 3, 4, 5, 6),
4   categoria = c("electronica", "Electrónica ", "Electrónica ",
5                 "HOGAR", "hogar", "Ropa", " ropa"),
6   precio   = c("$1,250.00", "$899.50", "$899.50", "450", "$0",
7                 "$320.75", "-15"),
8   fecha     = c("2023-05-01", "02/05/2023", "02/05/2023", "2023-05-03",
9                 "04/05/2023", "2023-05-05", "06/05/2023")
10 )
11
12 lista_limpia <- lista_proveedor %>%
13   distinct() %>%                                # 1. duplicados
14   mutate(
15     categoria = str_to_title(str_squish(categoria)), # 2. texto
16     categoria = recode(categoria, "Electronica" = "Electrónica"),
17     precio     = parse_number(precio),               # 3. número
18     fecha      = as_date(parse_date_time(fecha,      # 4. fecha
19                                           orders = c("ymd", "dmy"))),
20     sku        = str_c("SKU-", str_pad(id, 4, pad = "0"))
21   ) %>%
22   filter(precio > 0)                               # 5. regla de negocio
23 lista_limpia
24
25 lista_limpia %>%
26   group_by(categoria) %>%
27   summarise(productos = n(), precio_prom = mean(precio),
28             .groups = "drop")

```

```

# A tibble: 4 x 5
   id categoria  precio fecha      sku
<dbl> <chr>      <dbl> <date>   <chr>
1     1 Electrónica 1250 2023-05-01 SKU-0001
2     2 Electrónica  900 2023-05-02 SKU-0002

```



```

3      3 Hogar          450 2023-05-03 SKU-0003
4      5 Ropa           321. 2023-05-05 SKU-0005
# A tibble: 3 x 3
  categoria  productos precio_prom
  <chr>      <int>      <dbl>
1 Electrónica    2      1075.
2 Hogar          1       450
3 Ropa           1       321.

```

De 7 registros quedaron 4: se eliminaron el duplicado del id 2, el precio en cero (id 4) y el negativo (id 6). Las categorías quedaron unificadas y las fechas en un solo formato.

Solución del ejercicio 2.12

Construimos una tabla maestra, la validamos con `stopifnot()`, calculamos las tres hojas y las exportamos con `writexl`.

Enunciado en la página 161

```

1 library(tidyverse)
2 library(writexl)
3 transacciones <- read_csv("datasets/transacciones.csv",
4                           show_col_types = FALSE)
5 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
6 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
7
8 maestra_t <- transacciones %>%
9   left_join(select(productos, producto_id, nombre_producto, categoria),
10            by = "producto_id") %>%
11   left_join(select(tiendas, tienda_id, nombre_tienda), by = "tienda_id")
12 stopifnot(nrow(maestra_t) == nrow(transacciones)) # control
13
14 resumen <- maestra_t %>%
15   group_by(nombre_tienda) %>%
16   summarise(ingresos = sum(total_transaccion), transacciones = n(),
17            ticket_promedio = round(mean(total_transaccion), 2),
18            clientes_unicos = n_distinct(cliente_id), .groups = "drop") %>%
19   mutate(ranking = min_rank(desc(ingresos))) %>%
20   arrange(ranking)
21
22 trimestres <- maestra_t %>%
23   group_by(nombre_tienda, trimestre) %>%
24   summarise(ingresos = round(sum(total_transaccion), .groups = "drop") %>%
25     pivot_wider(names_from = trimestre, values_from = ingresos,
26                 values_fill = 0)
27
28 top_producto <- maestra_t %>%
29   group_by(nombre_tienda, nombre_producto, categoria) %>%

```

```

30  summarise(ingresos = sum(total_transaccion), .groups = "drop") %>%
31  group_by(nombre_tienda) %>%
32  slice_max(ingresos, n = 1, with_ties = FALSE) %>%
33  ungroup()
34
35  dir.create("resultados", showWarnings = FALSE)
36  write_xlsx(list("Resumen" = resumen, "Trimestres" = trimestres,
37                "Top producto" = top_producto),
38            "resultados/reporte_tiendas.xlsx")
39  readxl::excel_sheets("resultados/reporte_tiendas.xlsx")
40  resumen %>% select(ranking, nombre_tienda, ingresos, ticket_promedio)
41  top_producto %>% head(3)

```

```

[1] "Resumen"      "Trimestres"   "Top producto"
# A tibble: 10 x 4
  ranking nombre_tienda    ingresos ticket_promedio
  <int> <chr>                <dbl>         <dbl>
1     1 1 Sucursal Outlet    114377.        1079.
2     2 2 Sucursal Sur      105305.         966.
3     3 3 Sucursal Plaza A 105216.         965.
4     4 4 Sucursal Express 102791.         952.
5     5 5 Sucursal Premium 101538.         940.
6     6 6 Sucursal Norte    89024.         899.
7     7 7 Sucursal Este     88420.         941.
8     8 8 Sucursal Centro   88414.        1016.
9     9 9 Sucursal Oeste   84672.         891.
10    10 10 Sucursal Plaza B 80236.         944.
# A tibble: 3 x 4
  nombre_tienda nombre_producto categoria    ingresos
  <chr>         <chr>         <chr>         <dbl>
1 Sucursal Centro Producto 40    Electrónica    5611.
2 Sucursal Este  Producto 47    Automotriz    6652.
3 Sucursal Express Producto 27    Juguetes      6072.

```

El archivo tiene las tres hojas. La Sucursal Outlet vuelve a ser la número uno (\$114,377 en transacciones), igual que en **ventas**. Usamos **with_ties = FALSE** en **slice_max()** para garantizar un solo producto por tienda aunque hubiera empates.

A.3 Módulo 3. Visualización de datos con ggplot2

Solución del ejercicio 3.1

Contamos los tickets por tienda con `count()` y graficamos con `geom_col()` en horizontal, ordenando con `fct_reorder()`.

Enunciado en la página 217

```

1 library(tidyverse)
2 library(scales)
3 color_principal <- "#2a78d6"
4
5 transacciones <- read_csv("datasets/transacciones.csv",
6                           show_col_types = FALSE)
7 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
8
9 tickets_tienda <- transacciones %>%
10   left_join(tiendas, by = "tienda_id") %>%
11   mutate(tienda = str_remove(nombre_tienda, "Sucursal ")) %>%
12   group_by(tienda) %>%
13   summarise(tickets = n(),
14             ingresos = sum(total_transaccion),
15             .groups = "drop") %>%
16   arrange(desc(tickets))
17 head(tickets_tienda, 4)
18
19 s1 <- ggplot(tickets_tienda,
20             aes(x = tickets, y = fct_reorder(tienda, tickets))) +
21   geom_col(fill = color_principal, width = 0.7) +
22   scale_x_continuous(expand = expansion(mult = c(0, 0.05))) +
23   labs(title = "Sur y Plaza A atendieron más tickets en 2023",
24        x = "Número de tickets", y = NULL,
25        caption = "Fuente: transacciones.csv") +
26   theme_minimal() +
27   theme(panel.grid.minor = element_blank(),
28         panel.grid.major.y = element_blank())
29 s1

```

```

# A tibble: 4 x 3
  tienda tickets ingresos
  <chr>    <int>    <dbl>
1 Plaza A     109  105216.
2 Sur         109  105305.
3 Express     108  102791.
4 Premium     108  101538.

```

Interpretación. Sur y Plaza A empatan con 109 tickets, pero la tienda con más *ingresos* es Outlet (106 tickets): vende un poco menos de tickets pero de mayor importe. Contar tickets e ingresos responde preguntas distintas (tráfico contra valor).

Solución del ejercicio 3.2

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 218

```

1 library(tidyverse)
2 color_principal <- "#2a78d6"
3 color_resalte   <- "#eb6834"
4
5 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
6 mediana_edad <- median(clientes$edad)
7 mediana_edad
8
9 s2 <- ggplot(clientes, aes(x = edad)) +
10   geom_histogram(binwidth = 5, boundary = 0,
11     fill = color_principal, color = "white") +
12   geom_vline(xintercept = mediana_edad, linetype = "dashed",
13     color = color_resalte, linewidth = 0.8) +
14   annotate("text", x = mediana_edad, y = Inf, vjust = 1.5,
15     hjust = -0.1, label = paste("Mediana:", mediana_edad),
16     color = color_resalte, size = 3.5) +
17   labs(title = "Edad de los clientes",
18     subtitle = "Barras de 5 años", x = "Edad", y = "Clientes") +
19   theme_minimal() +
20   theme(panel.grid.minor = element_blank())
21 s2

```

```
[1] 48.5
```

Interpretación. La mediana es de 48.5 años y las barras tienen alturas parecidas entre los 18 y los 75 años: la distribución es más o menos *pareja* (uniforme), sin una edad dominante. Es lo esperable porque las edades se simularon al azar en ese rango; en una empresa real casi siempre verías una concentración en algún grupo de edad.

Solución del ejercicio 3.3

Los cuatro errores:

1. `fill = "#2a78d6"` dentro de `aes()`: el color se trata como un dato, sale un color por defecto y una leyenda rara. Va fuera de `aes()`, en la geometría.
2. El `+` está al inicio de la línea: R termina la instrucción en `ggplot(...)` y falla con “Cannot use `+` with a single argument”.

3. `geom_bar()` con `y`: debe ser `geom_col()` porque los ingresos ya están calculados.
4. `%>%` entre capas y `labels = percent` en un eje de pesos: debe ser `+` y `labels = dollar`. (Además conviene agregar `.groups = "drop"`, aunque aquí no es obligatorio porque se agrupa por una sola columna.)

Enunciado en la página 218

```

1 library(tidyverse)
2 library(scales)
3 transacciones <- read_csv("datasets/transacciones.csv",
4                           show_col_types = FALSE)
5
6 ingresos_pago <- transacciones %>%
7   group_by(metodo_pago) %>%
8   summarise(ingresos = sum(total_transaccion), .groups = "drop")
9
10 s3 <- ggplot(ingresos_pago, aes(x = metodo_pago, y = ingresos)) +
11   geom_col(fill = "#2a78d6") +
12   scale_y_continuous(labels = dollar)
13 s3

```

La versión corregida se ejecuta sin errores y muestra cuatro columnas azules con el eje en pesos. Para dejarla lista para la dirección faltaría ordenarla, ponerla horizontal y agregar títulos (lista de verificación de la sección 3.12).

Solución del ejercicio 3.4

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 218

```

1 library(tidyverse)
2 color_principal <- "#2a78d6"
3 color_resalte   <- "#eb6834"
4 color_gris      <- "#b5b3ad"
5
6 transacciones <- read_csv("datasets/transacciones.csv",
7                           show_col_types = FALSE)
8
9 tickets_mes <- transacciones %>%
10   mutate(mes = floor_date(fecha, unit = "month")) %>%
11   count(mes, name = "tickets")
12 promedio <- mean(tickets_mes$tickets)
13 mes_min <- tickets_mes %>% slice_min(tickets, n = 1)
14 promedio
15 mes_min
16

```

```

17 s4 <- ggplot(tickets_mes, aes(x = mes, y = tickets)) +
18   geom_hline(yintercept = promedio, color = color_gris,
19             linetype = "dashed", linewidth = 0.6) +
20   geom_line(color = color_principal, linewidth = 0.8) +
21   geom_point(color = color_principal, size = 2) +
22   annotate("text", x = mes_min$mes, y = mes_min$tickets,
23            label = paste("Mínimo:", mes_min$tickets, "tickets"),
24            vjust = 1.8, color = color_resalte, size = 3.5) +
25   scale_x_date(date_labels = "%b", date_breaks = "1 month") +
26   labs(title = "Abril tuvo el menor número de tickets del año",
27        subtitle = "Tickets por mes, 2023 (línea punteada = promedio)",
28        x = NULL, y = "Tickets") +
29   theme_minimal() +
30   theme(panel.grid.minor = element_blank())
31 s4

```

```

[1] 83.33333
# A tibble: 1 x 2
  mes      tickets
<date>    <int>
1 2023-04-01      71

```

Interpretación. El promedio es de 83.3 tickets por mes y abril queda muy por debajo con 71. Como abril también fue el peor mes en ingresos, la caída de ese mes se explica en buena parte por menos tráfico (menos tickets), no solo por tickets más pequeños.

Solución del ejercicio 3.5

`wday(fecha, label = TRUE, week_start = 1)` devuelve el día como factor ordenado de lunes a domingo; así el eje queda en el orden correcto sin hacer nada más.

Enunciado en la página 218

```

1 library(tidyverse)
2 library(scales)
3 color_resalte <- "#eb6834"
4 color_gris    <- "#b5b3ad"
5
6 transacciones <- read_csv("datasets/transacciones.csv",
7                           show_col_types = FALSE)
8
9 ingresos_dia <- transacciones %>%
10   mutate(dia = wday(fecha, label = TRUE, week_start = 1)) %>%
11   group_by(dia) %>%
12   summarise(ingresos = sum(total_transaccion), .groups = "drop") %>%
13   mutate(destacar = if_else(ingresos == max(ingresos), "Máximo",
14                             "Resto"))

```

```

15 ingresos_dia
16
17 s_dia <- ggplot(ingresos_dia, aes(x = dia, y = ingresos,
18                                   fill = destacar)) +
19   geom_col(width = 0.7) +
20   geom_text(data = filter(ingresos_dia, destacar == "Máximo"),
21             aes(label = dollar(ingresos, accuracy = 1)),
22             vjust = -0.5, size = 3.5, color = color_resalte) +
23   scale_fill_manual(values = c(Máximo = color_resalte,
24                                 Resto = color_gris),
25                     guide = "none") +
26   scale_y_continuous(labels = dollar,
27                       expand = expansion(mult = c(0, 0.1))) +
28   labs(title = "El viernes es el día que más vende",
29         subtitle = "Ingresos por día de la semana, 2023",
30         x = NULL, y = "Ingresos (MXN)") +
31   theme_minimal() +
32   theme(panel.grid.minor = element_blank(),
33         panel.grid.major.x = element_blank())
34 s_dia

```

```

# A tibble: 7 x 3
  dia   ingresos destacar
<ord>   <dbl> <chr>
1 lun    128878. Resto
2 mar    119364. Resto
3 mié    141329. Resto
4 jue    145280. Resto
5 vie    161266. Máximo
6 sáb    115172. Resto
7 dom    148703. Resto

```

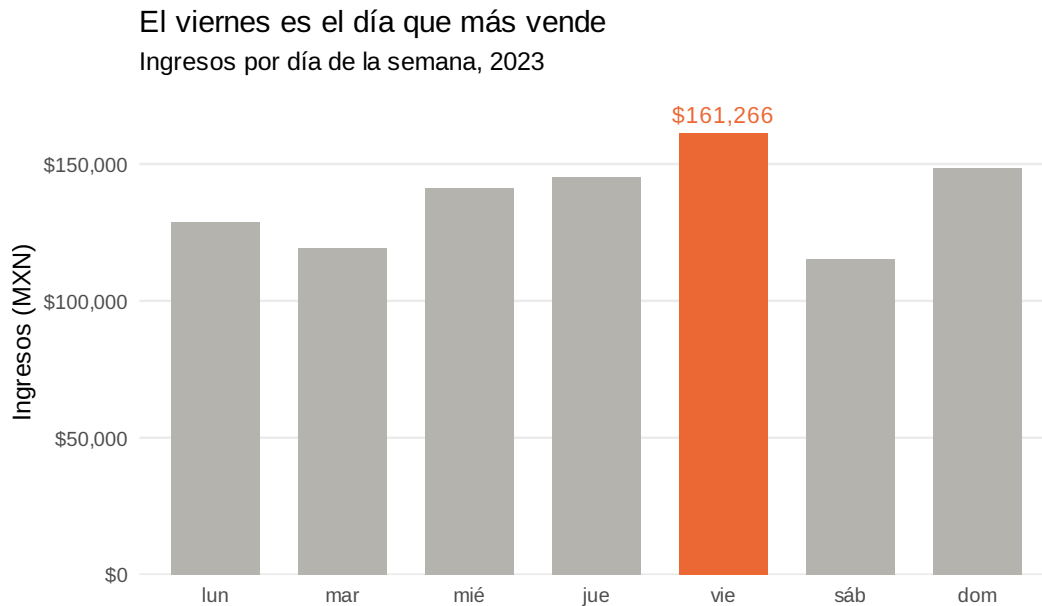


Figura A.1. Solución del ejercicio m3-5.

Interpretación. El viernes suma \$161,266, contra \$115,172 del sábado, el día más bajo. Pero recuerda el mapa de calor (figura 3.16): el viernes no es el mejor día *todos* los meses. La diferencia anual existe, pero no es tan consistente como para basar en ella una estrategia sin más análisis.

Solución del ejercicio 3.6

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 218

```

1 library(tidyverse)
2 library(scales)
3 empleados <- read_csv("datasets/empleados.csv", show_col_types = FALSE)
4
5 empleados %>%
6   group_by(departamento) %>%
7   summarise(
8     mediana = median(salario_mensual),
9     ric      = IQR(salario_mensual), # rango intercuartílico
10    n        = n(),
11    .groups  = "drop") %>%
12   arrange(desc(mediana))
13
14 s6 <- ggplot(empleados,
15   aes(x = salario_mensual,
16       y = fct_reorder(departamento, salario_mensual,
17                       .fun = median))) +
18   geom_boxplot(fill = "#cde2fb", color = "#2a78d6", width = 0.6) +
19   scale_x_continuous(labels = dollar) +

```



```

19 labs(title = "IT y Operaciones tienen los salarios medianos más altos",
20       x = "Salario mensual (MXN)", y = NULL) +
21 theme_minimal() +
22 theme(panel.grid.minor = element_blank(),
23       panel.grid.major.y = element_blank())
24 s6

```

```

# A tibble: 7 x 4
  departamento      mediana    ric     n
  <chr>           <dbl>  <dbl> <int>
1 IT              35044. 20725.     9
2 Operaciones     34945. 17114.    10
3 Ventas          32102. 22089.    16
4 Finanzas        31123. 14796.     9
5 Atención al Cliente 30663. 17242.    29
6 Marketing       24193. 11419.    12
7 RRHH            18291. 13712.    15

```

Interpretación. IT y Operaciones tienen las medianas más altas (unos \$35,000) y RRHH la más baja (\$18,291). La mayor dispersión la tiene Ventas: su rango intercuartílico (**ric**, el ancho de la caja) es de \$22,089. Cuidado: algunos departamentos tienen solo 9 o 10 personas, así que un boxplot con tan pocos datos es orientativo; siempre acompáñalo del número de observaciones.

Solución del ejercicio 3.7

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 219

```

1 library(tidyverse)
2 library(scales)
3 color_principal <- "#2a78d6"
4 color_resalte   <- "#eb6834"
5 color_gris      <- "#b5b3ad"
6
7 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
8
9 productos_margen <- productos %>%
10   mutate(situacion = if_else(costo > precio_catalogo,
11                             "Costo mayor al precio",
12                             "Margen positivo"))
13 count(productos_margen, situacion)
14
15 s_costo <- ggplot(productos_margen,
16                  aes(x = costo, y = precio_catalogo,
17                     color = situacion)) +
18   geom_abline(slope = 1, intercept = 0, linetype = "dashed",

```

```

19     color = color_gris) +
20   geom_point(size = 2.5, alpha = 0.8) +
21   annotate("text", x = 380, y = 390, label = "precio = costo",
22          angle = 20, size = 3, color = "#52514e", vjust = -0.5) +
23   scale_color_manual(values = c("Margen positivo" = color_principal,
24                                "Costo mayor al precio" = color_resalte)) +
25   scale_x_continuous(labels = dollar) +
26   scale_y_continuous(labels = dollar) +
27   labs(title = "10 de 50 productos cuestan más de lo que se venden",
28        subtitle = "Cada punto es un producto del catálogo",
29        x = "Costo", y = "Precio de catálogo", color = NULL) +
30   theme_minimal() +
31   theme(panel.grid.minor = element_blank(),
32         legend.position = "top")
33 s_costo

```

```

# A tibble: 2 x 2
  situacion      n
  <chr>        <int>
1 Costo mayor al precio    10
2 Margen positivo         40

```

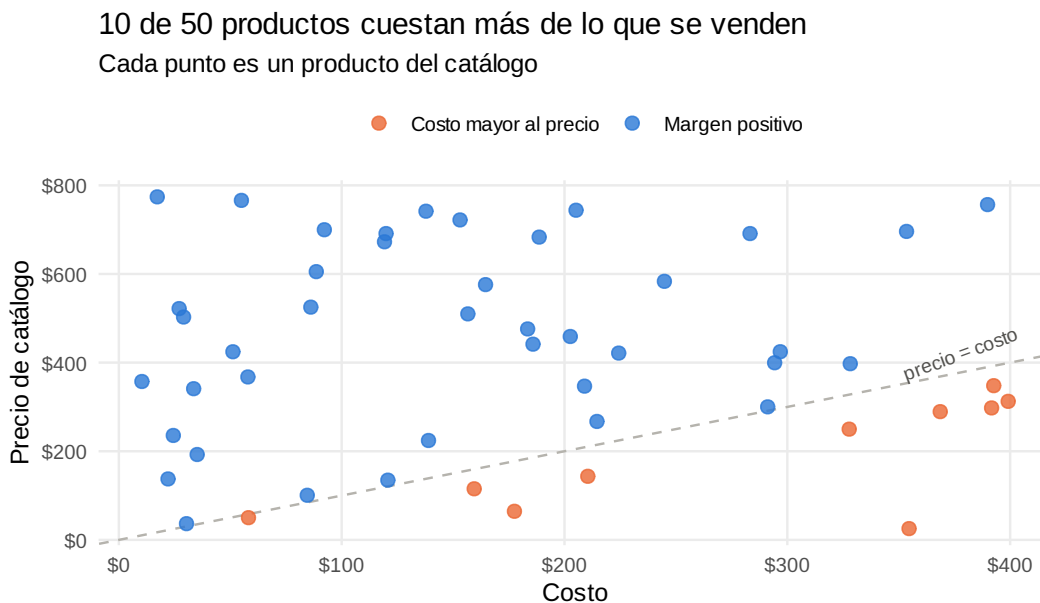


Figura A.2. Solución del ejercicio m3-7: los puntos naranjas, debajo de la diagonal, tienen margen negativo.

Interpretación. Todo punto por debajo de la diagonal tiene un costo mayor a su precio: 10 de los 50 productos (20 %). Es un *hallazgo de calidad de datos* o de negocio que hay que escalar: o el costo está mal capturado, o se están vendiendo productos con pérdida. Al área de compras le pedirías revisar primero estos diez productos.

Solución del ejercicio 3.8

La clave es dibujar dos capas con datos distintos: primero las siete tiendas en gris (con `group` para que cada una sea su propia línea y sin `color` en `aes()`, así no entran a la leyenda) y encima las tres líderes con `color = tienda`.

Enunciado en la página 219

```

1 library(tidyverse)
2 library(scales)
3 color_gris <- "#b5b3ad"
4 paleta_libro <- c("#2a78d6", "#eb6834", "#1baf7a", "#eda100",
5                 "#e87ba4", "#008300", "#4a3aa7", "#e34948")
6
7 transacciones <- read_csv("datasets/transacciones.csv",
8                          show_col_types = FALSE)
9 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
10
11 acumulado <- transacciones %>%
12   left_join(tiendas, by = "tienda_id") %>%
13   mutate(tienda = str_remove(nombre_tienda, "Sucursal "),
14          mes = floor_date(fecha, unit = "month")) %>%
15   group_by(tienda, mes) %>%
16   summarise(ingresos = sum(total_transaccion), .groups = "drop") %>%
17   group_by(tienda) %>%
18   arrange(mes, .by_group = TRUE) %>%
19   mutate(acumulado = cumsum(ingresos)) %>%
20   ungroup()
21
22 # las 3 con más ingresos acumulados en el último mes
23 top3 <- acumulado %>%
24   filter(mes == max(mes)) %>%
25   slice_max(acumulado, n = 3) %>%
26   pull(tienda)
27 top3
28
29 lideres <- acumulado %>%
30   filter(tienda %in% top3) %>%
31   mutate(tienda = factor(tienda, levels = top3)) # orden de la leyenda
32 resto <- acumulado %>% filter(!tienda %in% top3)
33
34 s_top3 <- ggplot(mapping = aes(x = mes, y = acumulado)) +
35   geom_line(data = resto, aes(group = tienda),
36            color = color_gris, linewidth = 0.5) +
37   geom_line(data = lideres, aes(color = tienda), linewidth = 0.9) +
38   scale_color_manual(values = paleta_libro[1:3]) +
39   scale_x_date(date_labels = "%b", date_breaks = "1 month") +
40   scale_y_continuous(labels = dollar) +
41   labs(title = "Las tres tiendas líderes al cierre de 2023",

```

```

42     subtitle = "Ingresos acumulados; en gris, las otras 7 tiendas",
43     x = NULL, y = "Ingresos acumulados (MXN)", color = NULL) +
44     theme_minimal() +
45     theme(panel.grid.minor = element_blank(),
46           legend.position = "top")
47   s_top3

```

```
[1] "Outlet" "Sur" "Plaza A"
```

Las tres tiendas líderes al cierre de 2023
Ingresos acumulados; en gris, las otras 7 tiendas

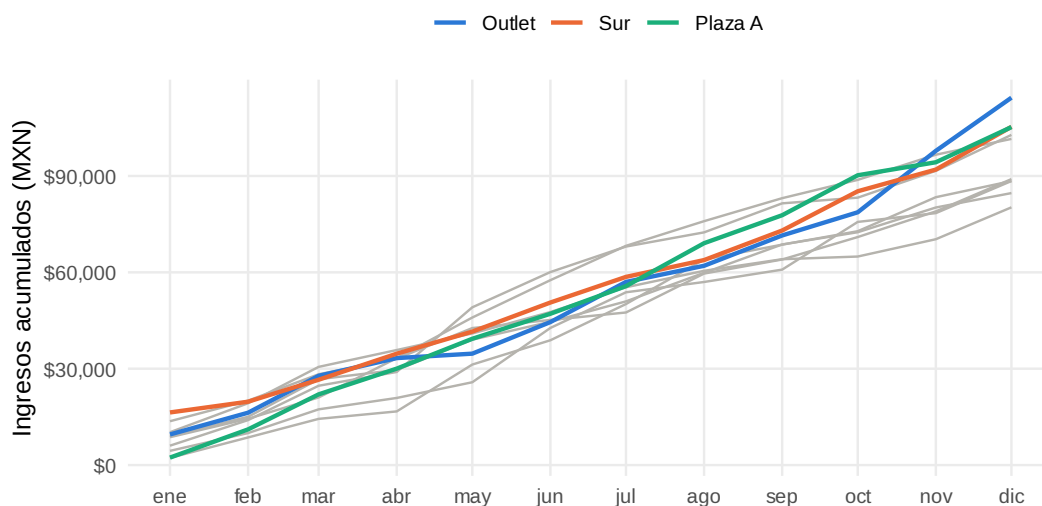


Figura A.3. Solución del ejercicio m3-8.

Interpretación. Outlet, Sur y Plaza A terminan arriba. Sur y Plaza A cierran prácticamente empatadas (\$105,305 y \$105,216). El primer lugar cambió de manos varias veces durante el año y Outlet lo tomó en noviembre. Como `top3` se calcula con código, la gráfica se actualiza sola si cambian los datos.

Solución del ejercicio 3.9

Dentro de una función, `{{ categoria }}` (se lee “*curly-curly*”) le dice a **ggplot2** y a **dplyr** que use la columna cuyo nombre recibió el argumento.

Enunciado en la página 219

```

1 library(tidyverse)
2 library(scales)
3 color_principal <- "#2a78d6"
4

```

```

5 tema_empresa <- function(base_size = 11) {
6   theme_minimal(base_size = base_size) +
7     theme(plot.title = element_text(face = "bold", color = "#0b0b0b"),
8           plot.title.position = "plot",
9           axis.text = element_text(color = "#52514e"),
10          panel.grid.minor = element_blank(),
11          panel.grid.major.y = element_blank(),
12          plot.margin = margin(10, 15, 10, 10))
13 }
14
15 grafica_ranking <- function(datos, categoria, valor, titulo) {
16   ggplot(datos, aes(x = {{ valor }}),
17          y = fct_reorder({{ categoria }}, {{ valor }})) +
18     geom_col(fill = color_principal, width = 0.7) +
19     scale_x_continuous(labels = dollar,
20                       expand = expansion(mult = c(0, 0.05))) +
21     labs(title = titulo, x = NULL, y = NULL) +
22     tema_empresa()
23 }
24
25 transacciones <- read_csv("datasets/transacciones.csv",
26                           show_col_types = FALSE)
27 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
28
29 por_pago <- transacciones %>%
30   group_by(metodo_pago) %>%
31   summarise(ingresos = sum(total_transaccion), .groups = "drop")
32
33 por_tienda <- transacciones %>%
34   left_join(tiendas, by = "tienda_id") %>%
35   group_by(nombre_tienda) %>%
36   summarise(ingresos = sum(total_transaccion), .groups = "drop")
37
38 g_pago <- grafica_ranking(por_pago, metodo_pago, ingresos,
39                          "Tarjeta de crédito, el método más usado")
40 g_tienda <- grafica_ranking(por_tienda, nombre_tienda, ingresos,
41                            "Sucursal Outlet lidera en ingresos")
42
43 dir.create("resultados", showWarnings = FALSE)
44 ggsave("resultados/ranking_pago.png", g_pago, width = 7, height = 4,
45        dpi = 300, bg = "white")
46 ggsave("resultados/ranking_tienda.png", g_tienda, width = 7, height = 4,
47        dpi = 300, bg = "white")
48 file.exists(c("resultados/ranking_pago.png",
49              "resultados/ranking_tienda.png"))

```

```
[1] TRUE TRUE
```

Interpretación. Con una sola función produces gráficas idénticas en estilo para cualquier tabla con una columna de categoría y otra de valor. Es el primer paso hacia un “kit” de gráficas del área de BI.

Solución del ejercicio 3.10

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 219

```

1 library(tidyverse)
2 library(scales)
3 color_principal <- "#2a78d6"
4 color_resalte   <- "#eb6834"
5 color_gris      <- "#b5b3ad"
6
7 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
8
9 # (a) KPIs en consola
10 cat("Clientes totales:", comma(nrow(clientes)), "\n")
11 cat("% premium:", percent(mean(clientes$es_premium), accuracy = 0.1),
12     "\n")
13 cat("Edad mediana:", median(clientes$edad), "años\n")
14
15 # (b) clientes por ciudad, resaltando la mayor
16 clientes_ciudad <- clientes %>%
17   count(ciudad, name = "clientes") %>%
18   mutate(destacar = if_else(clientes == max(clientes), "Mayor",
19                             "Resto"))
20
21 s_ciudades <- ggplot(clientes_ciudad,
22                      aes(x = clientes,
23                          y = fct_reorder(ciudad, clientes),
24                          fill = destacar)) +
25   geom_col(width = 0.7) +
26   geom_text(aes(label = clientes), hjust = -0.3, size = 3.2,
27             color = "#52514e") +
28   scale_fill_manual(values = c(Mayor = color_resalte,
29                                Resto = color_gris),
30                     guide = "none") +
31   scale_x_continuous(expand = expansion(mult = c(0, 0.1))) +
32   labs(title = "Puebla encabeza por poco: 28 clientes contra 27 de CDMX",
33        subtitle = "Clientes registrados por ciudad",
34        x = "Clientes", y = NULL,
35        caption = "Fuente: clientes.csv") +
36   theme_minimal() +
37   theme(panel.grid.minor = element_blank(),
38         panel.grid.major.y = element_blank())
39 s_ciudades

```

Clientes totales: 200
 % premium: 36.5%
 Edad mediana: 48.5 años

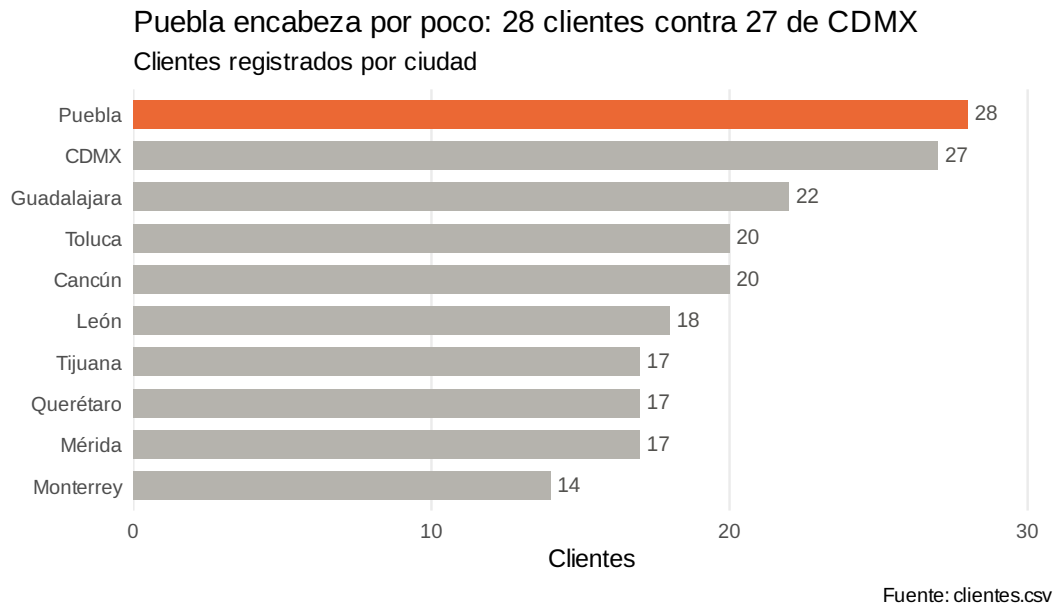


Figura A.4. Solución del ejercicio m3-10, inciso (b).

Para el inciso (c), el porcentaje de premium por segmento es el promedio de la columna lógica `es_premium` (`TRUE` cuenta como 1). Ordenamos los segmentos por edad, no por valor, porque tienen un orden natural.

```
1 premium_segmento <- clientes %>%
2   mutate(segmento = factor(segmento,
3                             levels = c("Joven", "Adulto", "Maduro",
4                                           "Senior"))) %>%
5   group_by(segmento) %>%
6   summarise(clientes = n(), pct_premium = mean(es_premium),
7             .groups = "drop")
8 premium_segmento
9
10 s_premium <- ggplot(premium_segmento,
11                     aes(x = segmento, y = pct_premium)) +
12   geom_col(fill = color_principal, width = 0.6) +
13   geom_text(aes(label = percent(pct_premium, accuracy = 1)),
14             vjust = -0.5, size = 3.5, color = "#52514e") +
15   scale_y_continuous(labels = percent, limits = c(0, 0.5),
16                     expand = expansion(mult = c(0, 0.05))) +
17   labs(title = "El segmento Maduro tiene más clientes premium",
18        subtitle = "% de clientes premium por segmento de edad",
```

```

19     x = NULL, y = NULL, caption = "Fuente: clientes.csv") +
20     theme_minimal() +
21     theme(panel.grid.minor = element_blank(),
22           panel.grid.major.x = element_blank())
23
24 dir.create("resultados", showWarnings = FALSE)
25 ggsave("resultados/clientes_ciudad.png", s_ciudades, width = 7,
26        height = 4, dpi = 300, bg = "white")
27 ggsave("resultados/premium_segmento.png", s_premium, width = 7,
28        height = 4, dpi = 300, bg = "white")

```

```

# A tibble: 4 x 3
  segmento clientes pct_premium
  <fct>      <int>      <dbl>
1 Joven        21      0.333
2 Adulto       48      0.312
3 Maduro       70      0.414
4 Senior       61      0.361

```

Conclusiones para marketing.

1. Tenemos 200 clientes, 36.5 % premium, con edad mediana de 48.5 años: la base es madura.
2. Puebla (28) y CDMX (27) concentran el 27.5 % de los clientes; Monterrey es la ciudad con menos (14), una posible oportunidad de crecimiento.
3. El segmento Maduro (41–60 años) es el más grande (70 clientes) y el de mayor proporción premium (41 %); los Jóvenes son pocos (21), así que su 33 % premium se basa en muy pocos casos.

A.4 Módulo 4. Análisis exploratorio y estadística

Solución del ejercicio 4.1

Cargamos los datos y calculamos cada medida sobre `precio_venta`.

Enunciado en la página 289

```

1 library(tidyverse)
2 options(cli.unicode = FALSE)
3 transacciones <- read_csv("datasets/transacciones.csv",
4                           show_col_types = FALSE)

```

```

1 precio <- transacciones$precio_venta
2

```



```

3 c(media    = mean(precio),
4     mediana  = median(precio),
5     desv_est = sd(precio),
6     cv       = sd(precio) / mean(precio),
7     iqr      = IQR(precio))
8 quantile(precio, probs = c(0.10, 0.90))

```

| | media | mediana | desv_est | cv | iqr |
|-----|-------------|-------------|-------------|-----------|-------------|
| | 318.3110300 | 324.9200000 | 171.5302293 | 0.5388762 | 303.8950000 |
| 10% | | 90% | | | |
| | 76.705 | 545.047 | | | |

La media (\$318.31) y la mediana (\$324.92) son casi iguales: la distribución del precio es simétrica (recuerda que su asimetría era -0.08). El coeficiente de variación es 0.54, menor que el del ticket (0.79): el precio varía menos en términos relativos porque el ticket multiplica dos fuentes de variación (precio y cantidad). El 80 % central de los precios está entre \$77 y \$545.

Solución del ejercicio 4.2

Definimos la función `moda()` y luego armamos la tabla por día.

Enunciado en la página 289

```

1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4
5 moda <- function(x) {
6   frecuencias <- table(x)
7   names(frecuencias)[which.max(frecuencias)]
8 }

```

```

1 moda(transacciones$dia_semana)
2
3 transacciones %>%
4   group_by(dia_semana) %>%
5   summarise(transacciones = n(),
6             ventas = sum(total_transaccion),
7             .groups = "drop") %>%
8   mutate(pct_transacciones = transacciones / sum(transacciones)) %>%
9   arrange(desc(transacciones))

```

```

[1] "viernes"
# A tibble: 7 x 4
  dia_semana transacciones  ventas pct_transacciones

```

| | <chr> | <int> | <dbl> | <dbl> |
|---|-----------|-------|---------|-------|
| 1 | viernes | 158 | 161266. | 0.158 |
| 2 | miércoles | 155 | 141329. | 0.155 |
| 3 | domingo | 146 | 148703. | 0.146 |
| 4 | jueves | 146 | 145280. | 0.146 |
| 5 | lunes | 136 | 128878. | 0.136 |
| 6 | sábado | 130 | 115172. | 0.13 |
| 7 | martes | 129 | 119364. | 0.129 |

El viernes es el día con más transacciones (moda) y también el de más ventas (\$161,266). Pero revisa el resto de la columna **ventas**: el domingo es segundo en ventas y solo tercero en transacciones; el orden no coincide exactamente, porque el ticket promedio cambia de un día a otro. Para programar personal importa el número de transacciones; para metas de venta, el monto.

Solución del ejercicio 4.3

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 290

```
1 library(tidyverse)
2 empleados <- read_csv("datasets/empleados.csv", show_col_types = FALSE)
```

```
1 empleados %>%
2   group_by(departamento) %>%
3   summarise(empleados = n(),
4             media      = mean(salario_mensual),
5             mediana     = median(salario_mensual),
6             diferencia = media - mediana,
7             .groups = "drop") %>%
8   arrange(desc(abs(diferencia)))
```

```
# A tibble: 7 x 5
  departamento      empleados  media mediana diferencia
  <chr>            <int>   <dbl>   <dbl>      <dbl>
1 RRHH              15 22956.  18291.    4665.
2 Operaciones       10 30380.  34945.   -4566.
3 IT                 9 30842.  35044.   -4202.
4 Ventas            16 29032.  32102.   -3071.
5 Finanzas           9 28116.  31123.   -3007.
6 Atención al Cliente 29 27919.  30663.   -2745.
7 Marketing         12 24602.  24193.    410.
```

En RRHH la media supera a la mediana por unos \$4,665: hay algunos salarios altos que la jalan hacia arriba, mientras que la mitad de los 15 empleados gana menos de \$18,291. Para anunciar el “salario típico” de RRHH conviene la mediana. En IT y Operaciones pasa lo contrario (media menor

que la mediana): hay salarios bajos que la jalan hacia abajo. Nota que algunos departamentos tienen muy pocos empleados (9 en Finanzas e IT), así que sus medidas son poco estables.

Solución del ejercicio 4.4

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 290

```
1 library(tidyverse)
2 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
```

```
1 q1 <- quantile(ventas$total, 0.25)
2 q3 <- quantile(ventas$total, 0.75)
3 limite_superior <- unname(q3 + 1.5 * (q3 - q1))
4 limite_superior
5
6 ventas_marcadas <- ventas %>%
7   mutate(atipico_iqr = total < q1 - 1.5 * (q3 - q1) |
8           total > limite_superior,
9           z = (total - mean(total)) / sd(total),
10            atipico_z = abs(z) > 3)
11
12 sum(ventas_marcadas$atipico_iqr) # regla IQR
13 sum(ventas_marcadas$atipico_z)  # regla z
14
15 ventas_marcadas %>%
16   filter(atipico_iqr) %>%
17   select(fecha, cantidad, precio_unitario, descuento_pct, total, z)
18
19 # Asimetría
20 mean((ventas$total - mean(ventas$total))^3) / sd(ventas$total)^3
```

```
[1] 4751.18
[1] 3
[1] 3
# A tibble: 3 x 6
  fecha      cantidad precio_unitario descuento_pct total    z
  <date>      <dbl>      <dbl>          <dbl> <dbl> <dbl>
1 2023-04-21      10        492.            0 4925.  3.13
2 2023-07-24      10        495.            0 4950.  3.15
3 2023-09-04      10        487.            0 4875.  3.08
[1] 0.8517708
```

Ambas reglas marcan las mismas 3 ventas: están por encima de \$4,751 (límite del IQR) y su z apenas supera 3. Las tres son ventas de 10 unidades a precios altos (más de \$480) sin descuento: son ventas

grandes *legítimas*, no errores, así que se conservan. La asimetría de 0.85 confirma una cola moderada a la derecha, como la del ticket de `transacciones.csv`.

Solución del ejercicio 4.5

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 290

```
1 library(tidyverse)
2 library(scales)
3 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
```

```
1 meta_trimestral <- 130000
2
3 kpi_trimestral <- ventas %>%
4   group_by(trimestre) %>%
5   summarise(ventas = sum(total), .groups = "drop") %>%
6   arrange(trimestre) %>%
7   mutate(crec_qoq = ventas / lag(ventas) - 1,
8          acumulado = cumsum(ventas),
9          indice_100 = ventas / first(ventas) * 100,
10         vs_meta = ventas / meta_trimestral - 1)
11 kpi_trimestral
12
13 sum(kpi_trimestral$vs_meta >= 0)           # trimestres en meta
14 sum(kpi_trimestral$ventas) / (4 * meta_trimestral) - 1 # vs meta anual
```

```
# A tibble: 4 x 6
  trimestre ventas crec_qoq acumulado indice_100 vs_meta
  <chr>      <dbl>   <dbl>    <dbl>    <dbl>   <dbl>
1 Q1        126274.    NA      126274.    100  -0.0287
2 Q2        120717.  -0.0440  246991.    95.6 -0.0714
3 Q3        134427.   0.114   381418.   106.  0.0341
4 Q4        132498.  -0.0144  513917.   105.  0.0192
[1] 2
[1] -0.01169883
```

Solo el tercer y el cuarto trimestre superaron la meta de \$130,000 (Q3 por 3.4% y Q4 por 1.9%). El segundo trimestre cayó 4.4% contra el primero, el tercero creció 11.4% y el cuarto bajó 1.4%. En el año, las ventas quedaron 1.2% por debajo de la meta anual de \$520,000: la recuperación del segundo semestre no alcanzó para compensar el primero.

Solución del ejercicio 4.6

H_0 : el número promedio de compras por cliente es igual en premium y no premium. H_1 : es distinto. Usamos $\alpha = 0.05$.

Enunciado en la página 290

```
1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)
```

```
1 frecuencia <- transacciones %>%
2   count(cliente_id, name = "compras") %>%           # compras por cliente
3   left_join(clientes, by = "cliente_id")
4
5 frecuencia %>%
6   group_by(es_premium) %>%
7   summarise(clientes = n(), compras_prom = mean(compras),
8             .groups = "drop")
9
10 t.test(compras ~ es_premium, data = frecuencia)
```

```
# A tibble: 2 x 3
  es_premium clientes compras_prom
  <lgl>      <int>      <dbl>
1 FALSE      126      5.18
2 TRUE       72      4.82

Welch Two Sample t-test

data:  compras by es_premium
t = 1.063, df = 146.53, p-value = 0.2895
alternative hypothesis: true difference in means between group FALSE and group TRUE is
  ⇨ not equal to 0
95 percent confidence interval:
 -0.3119807  1.0381711
sample estimates:
mean in group FALSE mean in group TRUE
    5.182540         4.819444
```

Los clientes premium compraron en promedio 4.8 veces en el año y los no premium 5.2: los premium incluso compran un poco *menos* (0.36 compras), aunque la diferencia es pequeña. Con $p = 0.29$ no rechazamos H_0 . Conclusión para la directora: *con los datos de 2023 no hay evidencia de que los clientes premium compren con más frecuencia ni gasten más por ticket que los demás; el programa premium no está cambiando el comportamiento de compra*. Solo se incluyen los 198 clientes con al

menos una compra.

Solución del ejercicio 4.7

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 290

```
1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)

1 abc_clientes <- transacciones %>%
2   group_by(cliente_id) %>%
3   summarise(gasto = sum(total_transaccion), .groups = "drop") %>%
4   arrange(desc(gasto)) %>%
5   mutate(pct_acumulado = cumsum(gasto) / sum(gasto),
6          pct_clientes = row_number() / n(),
7          clase = case_when(pct_acumulado <= 0.80 ~ "A",
8                            pct_acumulado <= 0.95 ~ "B",
9                            TRUE ~ "C"))
10
11 abc_clientes %>%
12   group_by(clase) %>%
13   summarise(clientes = n(),
14             pct_clientes = n() / nrow(abc_clientes),
15             pct_ingresos = sum(gasto) / sum(abc_clientes$gasto),
16             .groups = "drop")
17
18 # Participación del 20% de clientes que más gasta
19 abc_clientes %>%
20   filter(pct_clientes <= 0.20) %>%
21   summarise(clientes = n(), pct_ingresos = max(pct_acumulado))
```

```
# A tibble: 3 x 4
  clase clientes pct_clientes pct_ingresos
<chr>   <int>      <dbl>      <dbl>
1 A         119      0.601      0.798
2 B          44      0.222      0.151
3 C          35      0.177      0.0511
# A tibble: 1 x 2
  clientes pct_ingresos
  <int>      <dbl>
1      39      0.367
```

Los 119 clientes de la clase A (el 60.1 % de la clientela activa) juntan el 79.8 % de los ingresos, y el 20 % de clientes que más gasta (39 clientes) aporta el 36.7 %. Los ingresos están más concentrados en

clientes que en productos (donde el 20 % superior aportaba el 26.4 %), pero siguen lejos de la regla 80/20. Para la empresa significa que un programa de lealtad no puede enfocarse en unos cuantos clientes: la base de clientes valiosos es amplia.

Solución del ejercicio 4.8

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 291

```

1 # 1. Normal: ventas semanales
2 pnorm(20000, mean = 18445, sd = 4591) -
3   pnorm(15000, mean = 18445, sd = 4591)      # P(15,000 < X < 20,000)
4 qnorm(0.95, mean = 18445, sd = 4591)        # se supera 5% de las veces
5
6 # 2. Binomial: devoluciones de 150 ventas con p = 0.03
7 1 - pbinom(8, size = 150, prob = 0.03)      # P(más de 8)
8 dbinom(0, size = 150, prob = 0.03)        # P(ninguna)
9
10 # 3. Poisson: llamadas en 10 minutos con lambda = 4
11 1 - ppois(7, lambda = 4)                  # P(8 o más)

```

```

[1] 0.4060719
[1] 25996.52
[1] 0.03780957
[1] 0.01036956
[1] 0.05113362

```

1. Hay un 40.6 % de probabilidad de que una semana venda entre \$15,000 y \$20,000. Solo el 5 % de las semanas supera \$25,997.
2. Con 150 ventas se esperan $150 \times 0.03 = 4.5$ devoluciones. Tener más de 8 ocurre el 3.8 % de los días; no tener ninguna, solo el 1.0 %.
3. Recibir 8 o más llamadas en 10 minutos tiene una probabilidad de 5.1 %: pasa en uno de cada 20 intervalos, útil para dimensionar el personal del centro de atención.

Observa los detalles de “más de” y “o más”: $P(X > 8) = 1 - P(X \leq 8)$, pero $P(X \geq 8) = 1 - P(X \leq 7)$.

Solución del ejercicio 4.9

1. Prueba A/B. H_0 : las dos tasas de conversión son iguales.

Enunciado en la página 291

```

1 compras <- c(A = 180, B = 225)
2 visitas <- c(A = 3000, B = 3000)

```

```

3 compras / visitas          # tasas de conversión
4 prop.test(x = compras, n = visitas)

```

```

      A      B
0.060 0.075

      2-sample test for equality of proportions with continuity
      correction

data:  compras out of visitas
X-squared = 5.1263, df = 1, p-value = 0.02357
alternative hypothesis: two.sided
95 percent confidence interval:
 -0.028024006 -0.001975994
sample estimates:
prop 1 prop 2
 0.060  0.075

```

El diseño A convierte 6.0% y el B 7.5%. Con $p = 0.024 < 0.05$ rechazamos H_0 : el diseño B convierte más. La diferencia real está entre 0.2 y 2.8 puntos porcentuales a favor de B (R la reporta como A menos B, por eso los signos negativos).

2. ANOVA de salarios por departamento.

```

1 library(tidyverse)
2 empleados <- read_csv("datasets/empleados.csv", show_col_types = FALSE)

```

```

1 modelo_salarios <- aov(salario_mensual ~ departamento, data = empleados)
2 summary(modelo_salarios)

```

```

              Df    Sum Sq   Mean Sq F value Pr(>F)
departamento  6 6.399e+08 106656199   0.876  0.516
Residuals    93 1.133e+10 121778750

```

El p-valor es 0.516: no hay evidencia de que el salario promedio difiera entre departamentos. Como el ANOVA no es significativo, **no tiene sentido** aplicar `TukeyHSD()` buscando qué par es distinto: sería buscar diferencias donde la prueba global dice que no las hay (una forma de p-hacking). Además, los datos de salarios son simulados al azar, así que el resultado es el esperado.

Solución del ejercicio 4.10

Este es el código completo, con su salida y la interpretación.

Enunciado en la página 291


```

1 library(tidyverse)
2 set.seed(123)
3 semanas <- tibble(
4   precio = runif(52, min = 80, max = 120)
5 ) %>%
6   mutate(unidades = 500 - 3 * precio + rnorm(52, sd = 15))

```

1. Regresión.

```

1 modelo_precio <- lm(unidades ~ precio, data = semanas)
2 coef(modelo_precio)
3 confint(modelo_precio)
4 summary(modelo_precio)$r.squared

```

```

(Intercept)      precio
  498.510218    -2.973617
           2.5 %    97.5 %
(Intercept) 465.24655 531.773889
precio      -3.30276  -2.644474
[1] 0.8681726

```

La pendiente estimada es -2.97 : por cada peso que sube el precio, se venden en promedio 2.97 unidades menos a la semana. Su intervalo del 95 % (de -3.30 a -2.64) contiene el valor real de -3 , así que la regresión recuperó la relación simulada. El R^2 de 0.87 indica que el precio explica el 87 % de la variación de las unidades.

2 y 3. Predicciones.

```

1 predict(modelo_precio, newdata = tibble(precio = 100),
2        interval = "prediction")
3 predict(modelo_precio, newdata = tibble(precio = 200))
4 range(semanas$precio)      # rango de precios observado

```

```

      fit      lwr      upr
1 201.1485 173.0502 229.2469
      1
-96.21315
[1] 80.98455 119.77079

```

Con un precio de \$100 esperamos vender unas 201 unidades en la semana, con un intervalo de predicción de 173 a 229. Con \$200 el modelo predice -96 unidades, algo imposible: los precios observados van de \$80 a \$120 y fuera de ese rango no sabemos si la relación sigue siendo lineal. Es un ejemplo claro de por qué no se debe extrapolar.

4. Residuos.

```

1 diagnostico <- tibble(ajustado = fitted(modelo_precio),
2                       residuo = residuals(modelo_precio))
3 summary(diagnostico$residuo)
4 # Correlación entre residuos y ajustados (debe ser ~0)
5 round(cor(diagnostico$ajustado, diagnostico$residuo), 4)
6
7 ggplot(diagnostico, aes(x = ajustado, y = residuo)) +
8   geom_hline(yintercept = 0, color = "#eb6834") +
9   geom_point(color = "#2a78d6") +
10  labs(x = "Unidades ajustadas", y = "Residuo") +
11  theme_minimal()

```

```

      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-35.405  -8.198  -1.030    0.000  10.582   31.869
[1] 0

```

Los residuos están centrados en cero (mediana cercana a 0), son más o menos simétricos y no guardan relación con los valores ajustados. En la gráfica deberías ver una nube sin patrón: el modelo lineal es adecuado dentro del rango de precios observado.

A.5 Módulo 5. Bases de datos, SQL y ETL

Solución del ejercicio 5.1

Creamos la base en memoria, cargamos el CSV con `dbWriteTable()` y exploramos. Primero los paquetes (en todas las soluciones de este módulo usamos los mismos):

Enunciado en la página 357

```

1 library(tidyverse)
2 library(DBI)
3 library(RSQLite)
4 library(dbplyr)

```

```

1 con <- dbConnect(RSQLite::SQLite(), ":memory:")
2
3 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
4 dbWriteTable(con, "productos", productos)
5
6 dbListTables(con)
7 dbListFields(con, "productos")
8
9 dbGetQuery(con, "

```

```

10  SELECT categoria,
11         COUNT(*) AS productos,
12         ROUND(AVG(precio_catalogo), 2) AS precio_promedio
13  FROM productos
14  GROUP BY categoria
15  ORDER BY productos DESC, categoria
16  ")
17
18  dbDisconnect(con)

```

```

[1] "productos"
[1] "producto_id"      "nombre_producto" "categoria"
[4] "subcategoria"     "marca"           "precio_catalogo"
[7] "costo"            "stock_actual"    "stock_minimo"
[10] "proveedor"        "activo"          "margen_pct"
[13] "estado_stock"

  categoria productos precio_promedio
1    Belleza         9         474.54
2 Electrónica         7         413.72
3   Alimentos         6         354.92
4   Juguetes          6         425.89
5 Automotriz          5         365.35
6   Deportes          4         452.79
7     Libros           4         500.17
8   Mascotas           4         388.89
9     Hogar            3         361.51
10    Ropa             2         357.82

```

La tabla tiene 13 columnas. Belleza es la categoría con más productos (9) y Ropa la que menos (2). Libros tiene el precio de catálogo promedio más alto (\$500.17), aunque con solo 4 productos.

Solución del ejercicio 5.2

El filtro combina dos condiciones con **AND**; el faltante es una columna calculada con alias, que podemos usar en el **ORDER BY**.

Enunciado en la página 357

```

1  con <- dbConnect(RSQLite::SQLite(), ":memory:")
2  dbWriteTable(con, "productos",
3              read_csv("datasets/productos.csv", show_col_types = FALSE))
4
5  alerta <- dbGetQuery(con, "
6      SELECT nombre_producto, categoria, proveedor, stock_actual,
7             stock_minimo, stock_minimo - stock_actual AS faltante
8      FROM productos
9      WHERE activo = 1                -- TRUE se guardó como 1

```

```

10     AND stock_actual < stock_minimo
11     ORDER BY faltante DESC
12 ")
13 alerta
14
15 # ¿Qué proveedor aparece más?
16 dbGetQuery(con, "
17     SELECT proveedor, COUNT(*) AS productos_en_alerta
18     FROM productos
19     WHERE activo = 1 AND stock_actual < stock_minimo
20     GROUP BY proveedor
21     ORDER BY productos_en_alerta DESC
22 ")
23 dbDisconnect(con)

```

| | nombre_producto | categoria | proveedor | stock_actual | stock_minimo |
|---|-----------------|------------|-------------|--------------|--------------|
| 1 | Producto 03 | Automotriz | Proveedor 3 | 22 | 26 |
| 2 | Producto 33 | Alimentos | Proveedor 1 | 42 | 45 |
| 3 | Producto 06 | Juguetes | Proveedor 1 | 36 | 38 |

| | faltante |
|---|----------|
| 1 | 4 |
| 2 | 3 |
| 3 | 2 |

| | proveedor | productos_en_alerta |
|---|-------------|---------------------|
| 1 | Proveedor 1 | 2 |
| 2 | Proveedor 3 | 1 |

Solo tres productos activos están por debajo de su mínimo, todos por pocas piezas (el Producto 03 es el más urgente, con 4 faltantes). El Proveedor 1 surte dos de ellos: es con quien conviene hablar primero.

Solución del ejercicio 5.3

HAVING filtra los grupos después de agregarlos.

Enunciado en la página 358

```

1 con <- dbConnect(RSQLite::SQLite(), ":memory:")
2 dbWriteTable(con, "empleados",
3     read_csv("datasets/empleados.csv", show_col_types = FALSE) %>%
4     mutate(fecha_ingreso = format(fecha_ingreso, "%Y-%m-%d")))
5
6 dbGetQuery(con, "
7     SELECT departamento,
8         COUNT(*) AS empleados,
9         ROUND(AVG(salario_mensual), 2) AS salario_promedio,
10        MAX(salario_mensual) AS salario_maximo

```

```

11 FROM empleados
12 GROUP BY departamento
13 HAVING COUNT(*) >= 15
14 ORDER BY salario_promedio DESC
15 ")
16 dbDisconnect(con)

```

| | departamento | empleados | salario_promedio | salario_maximo |
|---|---------------------|-----------|------------------|----------------|
| 1 | Ventas | 16 | 29031.76 | 43777.92 |
| 2 | Atención al Cliente | 29 | 27918.75 | 44438.77 |
| 3 | RRHH | 15 | 22955.96 | 42797.09 |

Solo tres departamentos tienen 15 empleados o más. Ventas paga el salario promedio más alto (\$29 031.76) y Atención al Cliente es el departamento más grande (29 personas).

Solución del ejercicio 5.4

Reutilizamos la idea de `leer_para_bd()` del capítulo para guardar las fechas como texto ISO. En la parte (b), la condición de diciembre va *dentro* del **ON** del **LEFT JOIN**: si la pusiéramos en el **WHERE**, eliminaríamos precisamente los productos sin pareja que buscamos.

Enunciado en la página 358

```

1 con <- dbConnect(RSQLite::SQLite(), ":memory:")
2 leer_para_bd <- function(archivo) {
3   read_csv(file.path("datasets", archivo), show_col_types = FALSE) %>%
4     mutate(across(where(is.Date), ~ format(.x, "%Y-%m-%d")),
5            across(where(hms::is_hms), as.character))
6 }
7 for (tabla in c("transacciones", "tiendas", "productos")) {
8   dbWriteTable(con, tabla, leer_para_bd(paste0(tabla, ".csv")))
9 }
10
11 # (a) Ventas por ciudad de la tienda
12 dbGetQuery(con, "
13   SELECT ti.ciudad,
14          COUNT(*) AS tickets,
15          ROUND(SUM(t.total_transaccion), 0) AS ventas,
16          ROUND(AVG(t.total_transaccion), 2) AS ticket_promedio
17   FROM transacciones AS t
18   INNER JOIN tiendas AS ti ON t.tienda_id = ti.tienda_id
19   GROUP BY ti.ciudad
20   ORDER BY ventas DESC
21 ")
22
23 # (b) Productos sin ventas en diciembre
24 dbGetQuery(con, "

```

```

25 SELECT p.producto_id, p.nombre_producto, p.categoria
26 FROM productos AS p
27 LEFT JOIN transacciones AS t
28     ON p.producto_id = t.producto_id
29     AND t.fecha BETWEEN '2023-12-01' AND '2023-12-31'
30 WHERE t.transaccion_id IS NULL
31 ORDER BY p.producto_id
32 ")
33 dbDisconnect(con)

```

| | ciudad | tickets | ventas | ticket_promedio |
|---|-------------|---------|--------|-----------------|
| 1 | CDMX | 398 | 383588 | 963.79 |
| 2 | Guadalajara | 184 | 169260 | 919.89 |
| 3 | Tijuana | 106 | 114377 | 1079.03 |
| 4 | Monterrey | 109 | 105305 | 966.10 |
| 5 | Querétaro | 108 | 102791 | 951.77 |
| 6 | Puebla | 95 | 84672 | 891.29 |

| | producto_id | nombre_producto | categoria |
|----|-------------|-----------------|-------------|
| 1 | 5 | Producto 05 | Alimentos |
| 2 | 10 | Producto 10 | Hogar |
| 3 | 13 | Producto 13 | Ropa |
| 4 | 18 | Producto 18 | Libros |
| 5 | 23 | Producto 23 | Belleza |
| 6 | 25 | Producto 25 | Juguetes |
| 7 | 38 | Producto 38 | Alimentos |
| 8 | 40 | Producto 40 | Electrónica |
| 9 | 43 | Producto 43 | Automotriz |
| 10 | 45 | Producto 45 | Electrónica |
| 11 | 49 | Producto 49 | Belleza |

CDMX concentra las ventas (\$383 588) porque ahí están cuatro de las diez tiendas, pero Tijuana, con una sola tienda (el Outlet), tiene el ticket promedio más alto (\$1 079.03). En diciembre, 11 de los 50 productos no tuvieron ninguna venta; si además tienen mucho inventario, son candidatos a promoción.

Solución del ejercicio 5.5

La tabla `ventas_mal` guarda el número de días desde 1970 y `strptime()` lo malinterpreta; la tabla `ventas` guarda texto ISO y todo funciona.

Enunciado en la página 358

```

1 con <- dbConnect(RSQLite::SQLite(), ":memory:")
2 ventas_r <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
3
4 dbWriteTable(con, "ventas_mal", ventas_r)
5 dbWriteTable(con, "ventas",

```

```

6      ventas_r %>% mutate(fecha = format(fecha, "%Y-%m-%d"))
7
8  dbGetQuery(con, "SELECT fecha, typeof(fecha) AS tipo,
9                    strftime('%m', fecha) AS mes
10                   FROM ventas_mal LIMIT 2")
11 dbGetQuery(con, "SELECT fecha, typeof(fecha) AS tipo,
12                   strftime('%m', fecha) AS mes
13                   FROM ventas LIMIT 2")
14
15 dbGetQuery(con, "
16   SELECT (CAST(strftime('%m', fecha) AS INTEGER) + 2) / 3 AS trimestre,
17          ROUND(SUM(total), 2) AS ventas,
18          ROUND(AVG(descuento_pct), 2) AS descuento_prom,
19          SUM(descuento_pct > 0) AS dias_con_descuento
20   FROM ventas
21   GROUP BY trimestre
22   ORDER BY trimestre
23 ")
24 dbDisconnect(con)

```

```

      fecha tipo mes
1 19358 real  11
2 19359 real  11
      fecha tipo mes
1 2023-01-01 text 01
2 2023-01-02 text 01
      trimestre  ventas descuento_prom dias_con_descuento
1             1 126273.9             4.33             45
2             2 120716.9             4.51             38
3             3 134427.4             5.43             49
4             4 132498.3             4.84             45

```

En la tabla mal cargada, la fecha es un número (19358 son los días desde 1970 del 1 de enero de 2023) y `strftime()` devuelve el mes equivocado (11) sin marcar error. En la tabla correcta, la fecha es texto y el mes es 01. El tercer trimestre fue el de más ventas (\$134 427.4) y también el de mayor descuento promedio (5.43 %).

Solución del ejercicio 5.6

Los marcadores `:cliente`, `:desde` y `:hasta` se llenan con la lista `params`. El valor nunca se interpreta como SQL.

Enunciado en la página 358

```

1 con <- dbConnect(RSQLite::SQLite(), ":memory:")
2 dbWriteTable(con, "transacciones",
3              read_csv("datasets/transacciones.csv",

```

```

4         show_col_types = FALSE) %>%
5         mutate(fecha = format(fecha, "%Y-%m-%d"),
6                hora = as.character(hora)))
7
8 ventas_cliente <- function(con, cliente, desde, hasta) {
9   dbGetQuery(con, "
10     SELECT COUNT(*)                                AS compras,
11           ROUND(SUM(total_transaccion), 2) AS gasto
12     FROM transacciones
13     WHERE cliente_id = :cliente
14           AND fecha BETWEEN :desde AND :hasta",
15     params = list(cliente = cliente,
16                   desde   = format(as.Date(desde)),
17                   hasta   = format(as.Date(hasta))))
18 }
19
20 ventas_cliente(con, 5, "2023-01-01", "2023-06-30")
21 ventas_cliente(con, 5, "2023-07-01", "2023-12-31")
22
23 # Intento de inyección: se busca un cliente_id igual al texto
24 ventas_cliente(con, "5 OR 1=1", "2023-01-01", "2023-12-31")
25
26 # Lo que habría pasado con paste0() (¡no lo hagas!)
27 dbGetQuery(con, paste0("SELECT COUNT(*) AS compras FROM transacciones ",
28                        "WHERE cliente_id = ", "5 OR 1=1"))
29 dbDisconnect(con)

```

```

      compras  gasto
1         4 4560.68
      compras  gasto
1         9 10119.42
      compras  gasto
1         0    NA
      compras
1      1000

```

El cliente 5 hizo 4 compras por \$4 560.68 en el primer semestre y 9 por \$10 119.42 en el segundo. Con el texto **"5 OR 1=1"** la consulta parametrizada no encuentra nada (0 compras; la suma de cero filas es **NA**), mientras que la versión con **paste0()** habría devuelto las 1 000 transacciones de todos los clientes.

Solución del ejercicio 5.7

Con **dbplyr** construimos la consulta con verbos de dplyr; el trimestre ya viene como columna en la tabla.

Enunciado en la página 358


```

1 library(dbplyr)
2 con <- dbConnect(RSQLite::SQLite(), ":memory:")
3 dbWriteTable(con, "transacciones",
4               read_csv("datasets/transacciones.csv",
5                         show_col_types = FALSE) %>%
6                         mutate(fecha = format(fecha, "%Y-%m-%d"),
7                                hora = as.character(hora)))
8
9 consulta <- tbl(con, "transacciones") %>%
10  group_by(metodo_pago, trimestre) %>%
11  summarise(tickets = n(),
12            ventas = sum(total_transaccion, na.rm = TRUE),
13            .groups = "drop") %>%
14  arrange(metodo_pago, trimestre)
15
16 show_query(consulta)
17 resultado_dbplyr <- collect(consulta)
18 head(resultado_dbplyr, 4)
19
20 resultado_sql <- dbGetQuery(con, "
21   SELECT metodo_pago, trimestre, COUNT(*) AS tickets,
22          SUM(total_transaccion) AS ventas
23   FROM transacciones
24   GROUP BY metodo_pago, trimestre
25   ORDER BY metodo_pago, trimestre")
26
27 all.equal(as.data.frame(resultado_dbplyr), resultado_sql)
28 dbDisconnect(con)

```

```

<SQL>
SELECT
  `metodo_pago`,
  `trimestre`,
  COUNT(*) AS `tickets`,
  SUM(`total_transaccion`) AS `ventas`
FROM `transacciones`
GROUP BY `metodo_pago`, `trimestre`
ORDER BY `metodo_pago`, `trimestre`
# A tibble: 4 x 4
  metodo_pago trimestre tickets ventas
  <chr>         <chr>      <int> <dbl>
1 Efectivo     Q1          90 80565.
2 Efectivo     Q2          73 71206.
3 Efectivo     Q3          73 73187.
4 Efectivo     Q4          79 78777.
[1] TRUE

```

El SQL generado es prácticamente idéntico al que escribirías a mano, y `all.equal()` devuelve

TRUE: ambos caminos dan el mismo resultado. Convertimos el tibble con `as.data.frame()` porque `dbGetQuery()` devuelve un data frame simple y `all.equal()` también compara la clase.

Solución del ejercicio 5.8

En (a) calculamos el gasto por cliente en una CTE y lo rankeamos dentro de cada ciudad. En (b), `SUM(ventas) OVER (ORDER BY ventas DESC)` da el acumulado y `SUM(ventas) OVER ()` (ventana vacía) el total de todas las filas.

Enunciado en la página 359

```

1 con <- dbConnect(RSQLite::SQLite(), ":memory:")
2 dbWriteTable(con, "clientes",
3               read_csv("datasets/clientes.csv", show_col_types = FALSE) %>%
4               mutate(fecha_registro = format(fecha_registro)))
5 dbWriteTable(con, "transacciones",
6               read_csv("datasets/transacciones.csv",
7                         show_col_types = FALSE) %>%
8               mutate(fecha = format(fecha), hora = as.character(hora)))
9
10 # (a) Los dos mejores clientes de cada ciudad
11 dbGetQuery(con, "
12   WITH gasto AS (
13     SELECT c.ciudad, c.nombre, SUM(t.total_transaccion) AS gasto
14     FROM transacciones AS t
15     JOIN clientes AS c ON t.cliente_id = c.cliente_id
16     GROUP BY c.cliente_id
17   ),
18   ranking AS (
19     SELECT ciudad, nombre, ROUND(gasto, 0) AS gasto,
20            RANK() OVER (PARTITION BY ciudad ORDER BY gasto DESC) AS lugar
21     FROM gasto
22   )
23   SELECT * FROM ranking
24   WHERE lugar <= 2 AND ciudad IN ('Monterrey', 'Guadalajara', 'CDMX')
25   ORDER BY ciudad, lugar
26 ")
27
28 # (b) Curva de Pareto de productos
29 pareto <- dbGetQuery(con, "
30   WITH ventas_producto AS (
31     SELECT producto_id, SUM(total_transaccion) AS ventas
32     FROM transacciones
33     GROUP BY producto_id
34   )
35   SELECT producto_id, ROUND(ventas, 0) AS ventas,
36          ROW_NUMBER() OVER (ORDER BY ventas DESC) AS posicion,
37          ROUND(100.0 * SUM(ventas) OVER (ORDER BY ventas DESC) /
38                SUM(ventas) OVER (), 1) AS pct_acumulado

```

```

39 FROM ventas_producto
40 ORDER BY ventas DESC
41 ")
42 head(pareto, 3)
43 pareto %>% filter(pct_acumulado >= 80) %>% head(1)
44 dbDisconnect(con)

```

| | ciudad | nombre | gasto | lugar |
|---|-------------|--------------------|-------|-------|
| 1 | CDMX | Carmen Cruz | 10438 | 1 |
| 2 | CDMX | Fernando Hernández | 8242 | 2 |
| 3 | Guadalajara | Carmen Jiménez | 11848 | 1 |
| 4 | Guadalajara | Alberto Reyes | 11356 | 2 |
| 5 | Monterrey | Marta Ramírez | 6261 | 1 |
| 6 | Monterrey | Isabel Hernández | 5888 | 2 |

| | producto_id | ventas | posicion | pct_acumulado |
|---|-------------|--------|----------|---------------|
| 1 | 6 | 34028 | 1 | 3.5 |
| 2 | 19 | 28368 | 2 | 6.5 |
| 3 | 31 | 25430 | 3 | 9.1 |

| | producto_id | ventas | posicion | pct_acumulado |
|---|-------------|--------|----------|---------------|
| 1 | 22 | 16949 | 37 | 81.4 |

En Guadalajara los dos mejores clientes gastaron más de \$11 000 cada uno, mientras que en Monterrey el mejor apenas llega a \$6 261. En la curva de Pareto, el 80 % de las ventas se alcanza hasta el producto en la posición 37: se necesitan 37 de 50 productos (74 %). Las ventas están muy repartidas, lejos del clásico “20 % de los productos genera el 80 % de las ventas”.

Solución del ejercicio 5.9

Primero el DDL con llaves y reglas; luego la carga con `dbAppendTable()` (que respeta la estructura), la vista y el índice. La fecha es la llave primaria de los hechos porque hay una venta por día.

Enunciado en la página 359

```

1 con <- dbConnect(RSQLite::SQLite(), ":memory:")
2 dbExecute(con, "PRAGMA foreign_keys = ON")
3 dbExecute(con, "CREATE TABLE dim_producto (
4   producto_id INTEGER PRIMARY KEY, nombre_producto TEXT, categoria TEXT)")
5 dbExecute(con, "CREATE TABLE dim_tienda (
6   tienda_id INTEGER PRIMARY KEY, nombre_tienda TEXT, tipo TEXT)")
7 dbExecute(con, "CREATE TABLE fact_ventas_diarias (
8   fecha TEXT PRIMARY KEY,
9   producto_id INTEGER NOT NULL REFERENCES dim_producto(producto_id),
10  tienda_id   INTEGER NOT NULL REFERENCES dim_tienda(tienda_id),
11  cantidad    INTEGER CHECK (cantidad > 0),
12  total REAL)")
13

```

```

14 dbAppendTable(con, "dim_producto",
15   read_csv("datasets/productos.csv", show_col_types = FALSE) %>%
16     transmute(producto_id = as.integer(producto_id), nombre_producto,
17       categoria))
18 dbAppendTable(con, "dim_tienda",
19   read_csv("datasets/tiendas.csv", show_col_types = FALSE) %>%
20     transmute(tienda_id = as.integer(tienda_id), nombre_tienda, tipo))
21 dbAppendTable(con, "fact_ventas_diarias",
22   read_csv("datasets/ventas_retail.csv", show_col_types = FALSE) %>%
23     transmute(fecha = format(fecha, "%Y-%m-%d"),
24       producto_id = as.integer(producto_id),
25       tienda_id = as.integer(tienda_id),
26       cantidad = as.integer(cantidad), total))
27
28 dbExecute(con, "CREATE VIEW vw_trimestre_tipo AS
29   SELECT (CAST(strftime('%m', f.fecha) AS INTEGER) + 2) / 3 AS trimestre,
30     t.tipo, COUNT(*) AS dias, ROUND(SUM(f.total), 0) AS ventas
31   FROM fact_ventas_diarias AS f
32   JOIN dim_tienda AS t ON f.tienda_id = t.tienda_id
33   GROUP BY trimestre, t.tipo")
34 dbGetQuery(con, "SELECT * FROM vw_trimestre_tipo
35   WHERE trimestre = 1 ORDER BY ventas DESC")
36
37 consulta <- "SELECT SUM(total) FROM fact_ventas_diarias WHERE tienda_id = 3"
38 dbGetQuery(con, paste("EXPLAIN QUERY PLAN", consulta))$detail
39 dbExecute(con, "CREATE INDEX idx_tienda ON fact_ventas_diarias(tienda_id)")
40 dbGetQuery(con, paste("EXPLAIN QUERY PLAN", consulta))$detail
41 dbDisconnect(con)

```

```

[1] 0
[1] 0
[1] 0
[1] 0
[1] 50
[1] 10
[1] 365
[1] 0
  trimestre      tipo dias ventas
1          1 Estándar  46  54838
2          1 Principal 14  30699
3          1  Outlet  14  17226
4          1 Premium  10  16706
5          1 Express   6   6804
[1] "SCAN fact_ventas_diarias"
[1] 0
[1] "SEARCH fact_ventas_diarias USING INDEX idx_tienda (tienda_id=?)"

```

Los [1] 0 son las instrucciones DDL y los [1] 50, 10 y 365 las filas cargadas por `dbAppendTable()`.

En el primer trimestre, las tiendas de tipo Estándar suman más ventas porque son seis. El plan pasa de **SCAN** (recorrer toda la tabla) a **SEARCH ... USING INDEX idx_tienda** tras crear el índice.

Solución del ejercicio 5.10

La transformación *no* filtra cantidades negativas a propósito: la regla **CHECK** de la base es la última línea de defensa y queremos ver cómo actúa. La carga usa una tabla temporal y un **INSERT** que solo toma las fechas que aún no existen. ¿Por qué no **INSERT OR IGNORE**? Porque en SQLite **OR IGNORE** ignora en silencio *cualquier* violación de restricción, no solo las llaves repetidas: también la de **CHECK**. El lote con cantidad negativa se “cargaría” con estado OK y 0 filas nuevas, sin ningún error. Si lo pruebas, lo verás.

Enunciado en la página 359

```

1 con <- dbConnect(RSQLite::SQLite(), ":memory:")
2 dbExecute(con, "CREATE TABLE fact_ventas_diarias (
3   fecha TEXT PRIMARY KEY, producto_id INTEGER, cliente_id INTEGER,
4   tienda_id INTEGER, cantidad INTEGER CHECK (cantidad > 0),
5   precio_unitario REAL, descuento_pct REAL, total REAL)")
6 dbExecute(con, "CREATE TABLE log_etl (
7   id INTEGER PRIMARY KEY AUTOINCREMENT, fecha_hora TEXT, lote TEXT,
8   leidas INTEGER, nuevas INTEGER, estado TEXT, mensaje TEXT)")
9
10 extraer_ventas <- function(ruta) read_csv(ruta, show_col_types = FALSE)
11
12 transformar_ventas <- function(datos) {
13   datos %>%
14     distinct(fecha, .keep_all = TRUE) %>%
15     transmute(fecha = format(as.Date(fecha), "%Y-%m-%d"),
16               producto_id = as.integer(producto_id),
17               cliente_id = as.integer(cliente_id),
18               tienda_id = as.integer(tienda_id),
19               cantidad = as.integer(cantidad), precio_unitario,
20               descuento_pct, total)
21 }
22
23 cargar_ventas <- function(con, datos) {
24   dbWriteTable(con, "stg", datos, temporary = TRUE, overwrite = TRUE)
25   # Solo las fechas que aún no existen (sin OR IGNORE: ver explicación)
26   n <- dbExecute(con, "INSERT INTO fact_ventas_diarias
27                     SELECT * FROM stg
28                     WHERE fecha NOT IN (SELECT fecha
29                                       FROM fact_ventas_diarias)")
30   dbExecute(con, "DROP TABLE stg")
31   n
32 }
33
34 ejecutar_etl <- function(con, ruta) {
35   leidas <- NA_integer_

```

```

36  en_transaccion <- FALSE
37  registrar <- function(nuevas, estado, mensaje = NA_character_) {
38    dbAppendTable(con, "log_etl", tibble(
39      fecha_hora = format(Sys.time()), lote = basename(ruta),
40      leidas = leidas, nuevas = nuevas, estado = estado,
41      mensaje = mensaje))
42  }
43  tryCatch({
44    datos <- extraer_ventas(ruta)
45    leidas <- nrow(datos)
46    limpios <- transformar_ventas(datos)
47    dbBegin(con); en_transaccion <- TRUE
48    nuevas <- cargar_ventas(con, limpios)
49    registrar(nuevas, "OK")
50    dbCommit(con)
51  }, error = function(e) {
52    if (en_transaccion) dbRollback(con)
53    registrar(0L, "ERROR", conditionMessage(e))
54  })
55  invisible(NULL)
56 }
57
58 # Simular los lotes en una carpeta temporal
59 carpeta <- file.path(tempdir(), "lotes")
60 dir.create(carpeta, showWarnings = FALSE)
61 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
62 lote <- function(desde, hasta, nombre) {
63   ruta <- file.path(carpeta, nombre)
64   ventas %>% filter(fecha >= as.Date(desde), fecha <= as.Date(hasta)) %>%
65     write_csv(ruta)
66   ruta
67 }
68 l1 <- lote("2023-01-01", "2023-06-30", "lote_ene_jun.csv")
69 l2 <- lote("2023-04-01", "2023-09-30", "lote_abr_sep.csv")
70 l3 <- lote("2023-10-01", "2023-12-31", "lote_oct_dic.csv")
71 l_malo <- file.path(carpeta, "lote_malo.csv")
72 tibble(fecha = as.Date("2024-01-01"), producto_id = 1, cliente_id = 1,
73        tienda_id = 1, cantidad = -2, precio_unitario = 100,
74        descuento_pct = 0, total = -200) %>% write_csv(l_malo)
75
76 for (ruta in c(l1, l2, l3, l3, l_malo)) ejecutar_etl(con, ruta)
77
78 dbGetQuery(con, "SELECT id, lote, leidas, nuevas, estado FROM log_etl")
79 dbGetQuery(con, "SELECT mensaje FROM log_etl WHERE estado = 'ERROR'")
80
81 # Validación final contra el CSV
82 dbGetQuery(con, "SELECT COUNT(*) AS filas, ROUND(SUM(total), 2) AS total
83               FROM fact_ventas_diarias")
84 round(sum(ventas$total), 2)

```

```
85 dbDisconnect(con)
```

```
[1] 0
[1] 0
  id          lote leidas nuevas estado
1  1 lote_ene_jun.csv   181    181    OK
2  2 lote_abr_sep.csv   183     92    OK
3  3 lote_oct_dic.csv    92     92    OK
4  4 lote_oct_dic.csv    92      0    OK
5  5  lote_malo.csv      1      0 ERROR
      mensaje
1 CHECK constraint failed: cantidad > 0
  filas  total
1   365 513916.6
[1] 513916.6
```

La bitácora cuenta la historia: el segundo lote leyó 183 días pero solo cargó 92 (abril-junio ya estaban), el lote repetido no cargó nada y el lote malo terminó en **ERROR** porque la regla **CHECK** rechazó la cantidad negativa. La tabla final tiene exactamente 365 filas y el mismo total que el CSV: el ETL es idempotente, atómico y trazable.

A.6 Módulo 6. Dashboards interactivos con Shiny

Solución del ejercicio 6.1

Agregamos dos entradas a la **ui** y ampliamos el cálculo del **renderText()**. El IVA se aplica después del descuento.

Enunciado en la página 414

```
1 library(shiny)
2
3 ui <- fluidPage(
4   titlePanel("Calculadora de precio con descuento"),
5   numericInput("precio", "Precio de lista ($):", value = 500,
6     min = 0, step = 10),
7   numericInput("cantidad", "Cantidad de piezas:", value = 1, min = 1),
8   sliderInput("descuento", "Descuento (%):", min = 0, max = 50,
9     value = 10, step = 5),
10  checkboxInput("iva", "Incluir IVA (16%)", value = FALSE),
11  h3(textOutput("total"))
12 )
13
14 server <- function(input, output, session) {
15   output$total <- renderText({
```

```

16   req(input$precio, input$cantidad)      # sin valores, en blanco
17   subtotal <- input$precio * input$cantidad * (1 - input$descuento / 100)
18   total <- if (input$iva) subtotal * 1.16 else subtotal
19   paste0("Total a pagar: ", scales::dollar(total))
20 })
21 }
22
23 testServer(server, {
24   session$setInputs(precio = 500, cantidad = 3, descuento = 10,
25                     iva = FALSE)
26   print(output$total)
27   session$setInputs(iva = TRUE)
28   print(output$total)
29 })

```

```

[1] "Total a pagar: $1,350"
[1] "Total a pagar: $1,566"

```

Sin IVA, 3 piezas de \$500 con 10 % de descuento cuestan \$1,350; con IVA, \$1,566, como pedía el enunciado. Para abrir la app:

```
1 shinyApp(ui, server)
```

Solución del ejercicio 6.2

Hay tres cambios en `app.R` de la app 02: (1) un argumento nuevo en `filtrar_ventas()`, (2) la entrada en la barra lateral y (3) pasar la entrada al filtrar. Primero probamos la función modificada, de forma autocontenida:

Enunciado en la página 414

```

1 library(tidyverse)
2 library(scales)
3 ventas <- read_csv("datasets/transacciones.csv", show_col_types = FALSE)
4
5 # (1) filtrar_ventas() con el argumento nuevo 'pagos' (resto igual)
6 filtrar_pagos <- function(datos, pagos = NULL) {
7   if (length(pagos) > 0) {                # vacío = todos los métodos
8     datos <- filter(datos, metodo_pago %in% pagos)
9   }
10  datos
11 }
12 solo_efectivo <- filtrar_pagos(ventas, pagos = "Efectivo")
13 nrow(solo_efectivo)
14 dollar(sum(solo_efectivo$total_transaccion), accuracy = 1)

```



```
[1] 315
[1] "$303,735"
```

Con solo “Efectivo” quedan 315 transacciones por \$303,735. Los cambios en la app (la app completa modificada se probó arrancándola y con `testServer()`):

```
1 # (1) En filtrar_ventas(): nuevo argumento y nuevo if
2 filtrar_ventas <- function(datos, tienda = "Todas", categorias = NULL,
3                             fechas = range(datos$fecha), pagos = NULL) {
4   # ... filtros de fecha, tienda y categoría como antes ...
5   if (length(pagos) > 0) {
6     resultado <- filter(resultado, metodo_pago %in% pagos)
7   }
8   resultado
9 }
10
11 # (2) En sidebarPanel(), después del selectizeInput de categorías:
12 checkboxGroupInput("pagos", "Método de pago:",
13                    choices = sort(unique(ventas$metodo_pago)),
14                    selected = sort(unique(ventas$metodo_pago))),
15
16 # (3) En el server:
17 datos_filtrados <- reactive({
18   req(input$fechas)
19   filtrar_ventas(ventas, input$tienda, input$categorias, input$fechas,
20                 input$pagos)
21 })
```

Si el usuario desmarca todas las casillas, `input$pagos` vale `NULL` y, por nuestra convención, se muestran todos los métodos.

Solución del ejercicio 6.3

Primero la lógica. El margen bruto de cada transacción es lo cobrado menos el costo de las piezas vendidas (`costo * cantidad`):

Enunciado en la página 414

```
1 library(tidyverse)
2 library(scales)
3 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
4 datos <- read_csv("datasets/transacciones.csv", show_col_types = FALSE) %>%
5   left_join(select(productos, producto_id, categoria, costo),
6             by = "producto_id")
7
8 calcular_margen <- function(datos) {
9   datos %>%
```

```

10     summarise(ventas = sum(total_transaccion),
11               margen = sum(total_transaccion - costo * cantidad),
12               pct_margen = margen / ventas)
13   }
14
15   calcular_margen(datos)
16   datos %>%
17     group_by(categoria) %>%
18     calcular_margen() %>%
19     arrange(pct_margen) %>%
20     head(3)

```

```

# A tibble: 1 x 3
  ventas margen pct_margen
  <dbl> <dbl>    <dbl>
1 959993. 444637.    0.463
# A tibble: 3 x 4
  categoria ventas margen pct_margen
  <chr>    <dbl> <dbl>    <dbl>
1 Ropa      30048.  5835.    0.194
2 Hogar     63407. 13079.    0.206
3 Belleza  182143. 56054.    0.308

```

En total la empresa obtiene un margen bruto de 46.3 % (\$444,637). Las categorías con menor margen son Ropa (19.4 %), Hogar (20.6 %) y Belleza (30.8 %): Belleza es la tercera más baja porque varios de sus productos tienen un costo superior a su precio de catálogo (el hallazgo de calidad de datos que ya conoces), aunque sus precios de venta reales lo compensan en parte. Justo para eso sirve el color condicional. En la app, agregamos `calcular_margen()` a `R/funciones_dashboard.R` y estas piezas a `app.R`:

```

1  # ui: nueva fila dentro de tabItem(tabName = "resumen")
2  fluidRow(
3    valueBoxOutput("kpi_margen", width = 6),
4    valueBoxOutput("kpi_pct_margen", width = 6)
5  )
6
7  # server
8  margen <- reactive(calcular_margen(datos_filtrados()))
9
10 output$kpi_margen <- renderValueBox({
11   m <- margen()$margen
12   valueBox(formato_pesos(m), "Margen bruto",
13            icon = icon("sack-dollar"),
14            color = if (isTRUE(m >= 0)) "green" else "red")
15 })
16 output$kpi_pct_margen <- renderValueBox({
17   valueBox(formato_pct(margen()$pct_margen), "% de margen",

```

```

18         icon = icon("percent"), color = "blue")
19     })

```

`isTRUE()` evita un error cuando el filtro no deja ventas (el margen sería 0 y el porcentaje `NaN`). Para ver la caja en rojo, elige la tienda Norte y la categoría Ropa: en 2023 esa combinación perdió \$2,520.

Solución del ejercicio 6.4

El truco es que *todas* las salidas dependan de un `eventReactive()` ligado al botón, y no directamente de los filtros.

Enunciado en la página 414

```

1  library(shiny)
2  library(tidyverse)
3  library(scales)
4  tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
5  ventas <- read_csv("datasets/transacciones.csv", show_col_types = FALSE) %>%
6    left_join(select(tiendas, tienda_id, nombre_tienda), by = "tienda_id")
7
8  ui <- fluidPage(
9    titlePanel("Ventas con botón Aplicar"),
10    sidebarLayout(
11      sidebarPanel(
12        selectizeInput("tiendas", "Tiendas:",
13                      choices = sort(unique(ventas$nombre_tienda)),
14                      multiple = TRUE, options = list(placeholder = "Todas")),
15        dateRangeInput("fechas", "Periodo:", start = "2023-01-01",
16                      end = "2023-12-31", language = "es",
17                      separator = " a "),
18        actionButton("aplicar", "Aplicar", icon = icon("filter"))
19      ),
20    mainPanel(h3(textOutput("ventas")), h4(textOutput("transacciones")))
21  )
22 )
23
24 server <- function(input, output, session) {
25   # ignoreNULL = FALSE: calcula también al abrir la app (con el valor 0
26   # del botón), para no empezar con la pantalla vacía
27   datos <- eventReactive(input$aplicar, {
28     d <- filter(ventas, fecha >= input$fechas[1], fecha <= input$fechas[2])
29     if (length(input$tiendas) > 0) {
30       d <- filter(d, nombre_tienda %in% input$tiendas)
31     }
32     d
33   }, ignoreNULL = FALSE)
34
35   output$ventas <- renderText(

```

```

36   paste("Ventas:", dollar(sum(datos())$total_transaccion), accuracy = 1)))
37   output$transacciones <- renderText(
38     paste("Transacciones:", nrow(datos())))
39 }
40
41 testServer(server, {
42   session$setInputs(tiendas = NULL, aplicar = 0,
43     fechas = as.Date(c("2023-01-01", "2023-12-31")))
44   print(output$ventas)
45   session$setInputs(tiendas = "Sucursal Norte")      # sin pulsar
46   print(output$ventas)
47   session$setInputs(aplicar = 1)                     # pulsa Aplicar
48   print(output$ventas)
49   print(output$transacciones)
50 })

```

```

[1] "Ventas: $959,993"
[1] "Ventas: $959,993"
[1] "Ventas: $89,024"
[1] "Transacciones: 99"

```

La prueba lo demuestra: al elegir “Sucursal Norte” sin pulsar, las ventas siguen en \$959,993; al pulsar “Aplicar” cambian a las de la Sucursal Norte (\$89,024 en 99 transacciones).

```
1 shinyApp(ui, server)
```

Solución del ejercicio 6.5

Primero la función de la gráfica. Con `wday(fecha, label = TRUE, week_start = 1)` obtenemos el día como factor ordenado de lunes a domingo (en español, porque trabajamos con locale `es_MX`):

Enunciado en la página 414

```

1 library(tidyverse)
2 library(lubridate)
3 library(scales)
4 datos <- read_csv("datasets/transacciones.csv", show_col_types = FALSE)
5
6 ventas_por_dia <- function(datos) {
7   datos %>%
8     mutate(dia = wday(fecha, label = TRUE, abbr = FALSE,
9       week_start = 1)) %>%
10    group_by(dia) %>%
11    summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
12    mutate(es_mejor = ventas == max(ventas))

```

```

13 }
14
15 grafica_dias <- function(datos) {
16   ggplot(ventas_por_dia(datos), aes(x = dia, y = ventas, fill = es_mejor)) +
17     geom_col(width = 0.7) +
18     scale_fill_manual(values = c(`TRUE` = "#eb6834", `FALSE` = "#b5b3ad"),
19                       guide = "none") +
20     scale_y_continuous(labels = label_dollar(),
21                       expand = expansion(mult = c(0, 0.05))) +
22     labs(x = NULL, y = "Ventas") +
23     theme_minimal(base_size = 12) +
24     theme(panel.grid.minor = element_blank())
25 }
26
27 ventas_por_dia(datos)
28 p_dias <- grafica_dias(datos)

```

```

# A tibble: 7 x 3
  dia      ventas es_mejor
<ord>    <dbl> <lgl>
1 lunes    128878 FALSE
2 martes   119364 FALSE
3 miércoles 141329 FALSE
4 jueves   145280 FALSE
5 viernes   161266 TRUE
6 sábado    115172 FALSE
7 domingo   148703 FALSE

```

En todo 2023 el mejor día es el viernes (\$161,266) y el más flojo el sábado (\$115,172). Como la función recibe los datos, en la app basta con pasarle `datos_validos()` para que respete los filtros globales. Cambios en la app 03:

```

1 # En sidebarMenu(), después de "Clientes":
2 menuItem("Días", tabName = "dias", icon = icon("calendar-week")),
3
4 # En tabItems(), una pestaña nueva:
5 tabItem(tabName = "dias",
6         fluidRow(
7           box(title = "Ventas por día de la semana", width = 12,
8               status = "primary", plotOutput("graf_dias", height = 380))
9         )
10 ),
11
12 # En el server:
13 output$graf_dias <- renderPlot({
14   grafica_dias(datos_validos())
15 }, res = 96)

```

La función `grafica_dias()` (con `ventas_por_dia()`) va en `R/funciones_dashboard.R`, junto a las demás gráficas.

Solución del ejercicio 6.6

Para que la descarga contenga “exactamente la tabla que se ve”, guardamos la tabla en un reactivo que usan *tanto* la tabla como los dos botones. Así no hay forma de que diverjan. Probemos primero, autocontenido, lo que escribirá cada botón:

Enunciado en la página 415

```

1 library(tidyverse)
2 library(scales)
3 library(writexl)
4 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
5 ventas <- read_csv("datasets/transacciones.csv", show_col_types = FALSE) %>%
6   left_join(select(productos, producto_id, nombre_producto, categoria),
7             by = "producto_id")
8
9 top_productos <- function(datos, n = 10) {
10   datos %>%
11     group_by(Producto = nombre_producto, `Categoría` = categoria) %>%
12     summarise(Unidades = sum(cantidad),
13              Ventas = sum(total_transaccion), .groups = "drop") %>%
14     slice_max(Ventas, n = n, with_ties = FALSE) %>%
15     mutate(Ventas = dollar(Ventas))
16 }
17
18 dir.create("resultados", showWarnings = FALSE)
19 nombre <- paste0("top_productos_", Sys.Date())
20 top5 <- top_productos(ventas, n = 5)
21 write_excel_csv(top5, file.path("resultados", paste0(nombre, ".csv")))
22 write_xlsx(top5, file.path("resultados", paste0(nombre, ".xlsx")))
23 read_csv(file.path("resultados", paste0(nombre, ".csv")),
24           show_col_types = FALSE)

```

```

# A tibble: 5 x 4
  Producto Categoría Unidades Ventas
  <chr>      <chr>      <dbl> <chr>
1 Producto 06 Juguetes      82 $34,028.04
2 Producto 19 Deportes      85 $28,367.95
3 Producto 31 Alimentos      74 $25,429.70
4 Producto 47 Automotriz     82 $24,527.21
5 Producto 46 Automotriz     74 $24,470.29

```

Los cambios en la app 02:

```

1  # ui: la pestaña "Top productos" ahora tiene los botones arriba
2  tabPanel("Top productos",
3    br(),
4    downloadButton("bajar_csv", "Descargar CSV"),
5    downloadButton("bajar_excel", "Descargar Excel"),
6    br(), br(),
7    tableOutput("tabla_top"))
8
9  # server: un reactivo con la tabla visible
10 tabla_visible <- reactive(top_productos(datos_filtrados(), n = input$n_top))
11
12 output$tabla_top <- renderTable(tabla_visible(), align = "llrr",
13                                digits = 0)
14
15 output$bajar_csv <- downloadHandler(
16   filename = function() paste0("top_productos_", Sys.Date(), ".csv"),
17   content = function(file) write_excel_csv(tabla_visible(), file)
18 )
19 output$bajar_excel <- downloadHandler(
20   filename = function() paste0("top_productos_", Sys.Date(), ".xlsx"),
21   content = function(file) writexl::write_xlsx(tabla_visible(), file)
22 )

```

Para probarlo con `testServer()` sobre la app modificada, basta con leer la salida del botón, que devuelve la ruta del archivo generado:

```

1  testServer(shinyAppDir("mis-apps/02-mi-explorador"), {
2    session$setInputs(tienda = "Todas", categorias = NULL, n_top = 5,
3                      fechas = as.Date(c("2023-01-01", "2023-12-31")),
4                      metrica = "ventas")
5    read_csv(output$bajar_csv, show_col_types = FALSE) # 5 filas
6  })

```

Solución del ejercicio 6.7

El módulo recibe los datos como *reactivo* (sin paréntesis al pasarlo) y el nombre de la columna como texto. Dentro, cada id pasa por `ns()`, así que las dos copias no chocan aunque ambas tengan un deslizador llamado "n".

Enunciado en la página 415

```

1  library(shiny)
2  library(shinydashboard)
3  library(tidyverse)
4  library(scales)
5  tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)

```

```

6 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
7 ventas <- read_csv("datasets/transacciones.csv", show_col_types = FALSE) %>%
8   left_join(select(tiendas, tienda_id, ciudad), by = "tienda_id") %>%
9   left_join(select(productos, producto_id, nombre_producto),
10             by = "producto_id")
11
12 # ---- Módulo -----
13 ranking_ui <- function(id, titulo) {
14   ns <- NS(id)
15   box(title = titulo, width = 6, status = "primary",
16       sliderInput(ns("n"), "Mostrar top:", min = 3, max = 10, value = 5),
17       plotOutput(ns("grafica"), height = 300))
18 }
19
20 ranking_server <- function(id, datos, columna) {
21   moduleServer(id, function(input, output, session) {
22     resumen <- reactive({
23       datos() %>%
24         group_by(nombre = .data[[columna]]) %>%
25         summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
26         slice_max(ventas, n = input$n, with_ties = FALSE)
27     })
28     output$grafica <- renderPlot({
29       ggplot(resumen(), aes(x = ventas, y = reorder(nombre, ventas))) +
30       geom_col(fill = "#2a78d6", width = 0.7) +
31       scale_x_continuous(labels = label_dollar()) +
32       labs(x = "Ventas", y = NULL) +
33       theme_minimal(base_size = 12)
34     }, res = 96)
35     resumen # el módulo devuelve su tabla (reactivo)
36   })
37 }
38
39 # ---- App que usa el módulo dos veces -----
40 ui <- dashboardPage(
41   dashboardHeader(title = "Rankings"),
42   dashboardSidebar(disable = TRUE),
43   dashboardBody(fluidRow(
44     ranking_ui("productos", "Top productos"),
45     ranking_ui("ciudades", "Top ciudades")
46   ))
47 )
48 server <- function(input, output, session) {
49   datos <- reactive(ventas)
50   ranking_server("productos", datos, "nombre_producto")
51   ranking_server("ciudades", datos, "ciudad")
52 }
53
54 # Cada copia del módulo se prueba por separado con su propio 'n'

```



```

55 testServer(ranking_server,
56           args = list(datos = reactive(ventas), columna = "ciudad"), {
57   session$setInputs(n = 3)
58   print(resumen())
59 })
60 testServer(ranking_server,
61           args = list(datos = reactive(ventas),
62                       columna = "nombre_producto"), {
63   session$setInputs(n = 4)
64   print(nrow(resumen()))
65 })

```

```

# A tibble: 3 x 2
  nombre      ventas
  <chr>      <dbl>
1 CDMX      383588.
2 Guadalajara 169260.
3 Tijuana    114377.
[1] 4

```

La copia de ciudades con `n = 3` devuelve las tres ciudades con más ventas (la CDMX, con cuatro tiendas, domina con \$383,588) y la de productos con `n = 4` devuelve 4 filas: cada módulo responde a su propio deslizador. En la `ui` completa los ids reales son `productos-n` y `ciudades-n`.

```

1 shinyApp(ui, server)

```

Solución del ejercicio 6.8

Seguimos la receta del módulo: datos fuera del `server`, funciones de lógica probadas por separado y un servidor corto.

Enunciado en la página 415

```

1 library(shiny)
2 library(shinydashboard)
3 library(tidyverse)
4 library(scales)
5 library(DT)
6 library(writexl)
7
8 empleados <- read_csv("datasets/empleados.csv", show_col_types = FALSE)
9
10 # ---- Lógica -----
11 filtrar_empleados <- function(datos, deptos = NULL, sucursales = NULL) {
12   if (length(deptos) > 0) datos <- filter(datos, departamento %in% deptos)
13   if (length(sucursales) > 0) {

```

```

14   datos <- filter(datos, sucursal %in% sucursales)
15 }
16 datos
17 }
18
19 kpis_rrhh <- function(datos) {
20   datos %>%
21     summarise(plantilla = n(),
22               salario_prom = mean(salario_mensual),
23               antigüedad_prom = mean(antigüedad_anos),
24               pct_posgrado = mean(nivel_educacion %in%
25                                   c("Maestría", "Doctorado")))
26 }
27
28 grafica_salario_puesto <- function(datos) {
29   datos %>%
30     group_by(puesto) %>%
31     summarise(salario = mean(salario_mensual), .groups = "drop") %>%
32     ggplot(aes(x = salario, y = reorder(puesto, salario))) +
33     geom_col(fill = "#2a78d6", width = 0.7) +
34     scale_x_continuous(labels = label_dollar(),
35                        expand = expansion(mult = c(0, 0.05))) +
36     labs(x = "Salario mensual promedio", y = NULL) +
37     theme_minimal(base_size = 12) +
38     theme(panel.grid.minor = element_blank())
39 }
40
41 tabla_empleados <- function(datos) {
42   datos %>%
43     select(Nombre = nombre, Departamento = departamento, Puesto = puesto,
44           Sucursal = sucursal, Salario = salario_mensual,
45           `Antigüedad` = antigüedad_anos) %>%
46     arrange(desc(Salario))
47 }
48
49 kpis_rrhh(empleados)
50 kpis_rrhh(filtrar_empleados(empleados, deptos = "Ventas"))

```

```

# A tibble: 1 x 4
  plantilla salario_prom antigüedad_prom pct_posgrado
  <int>         <dbl>         <dbl>         <dbl>
1     100     27481.         3.96         0.35
# A tibble: 1 x 4
  plantilla salario_prom antigüedad_prom pct_posgrado
  <int>         <dbl>         <dbl>         <dbl>
1      16     29032.         4.14         0.312

```

La empresa tiene 100 empleados con salario mensual promedio de \$27,481 y casi 4 años de antigüedad

promedio; el 35 % tiene maestría o doctorado. El departamento de Ventas tiene 16 personas, con salario promedio de \$29,032. Ahora la interfaz y el servidor:

```

1 ui <- dashboardPage(
2   dashboardHeader(title = "Recursos Humanos"),
3   dashboardSidebar(
4     selectizeInput("deptos", "Departamentos:",
5                     choices = sort(unique(empleados$departamento)),
6                     multiple = TRUE, options = list(placeholder = "Todos")),
7     selectizeInput("sucursales", "Sucursales:",
8                     choices = sort(unique(empleados$sucursal)),
9                     multiple = TRUE, options = list(placeholder = "Todas"))
10  ),
11  dashboardBody(
12    fluidRow(
13      valueBoxOutput("plantilla", width = 3),
14      valueBoxOutput("salario", width = 3),
15      valueBoxOutput("antigüedad", width = 3),
16      valueBoxOutput("posgrado", width = 3)
17    ),
18    fluidRow(
19      box(title = "Salario promedio por puesto", width = 5,
20          status = "primary", plotOutput("graf_puesto", height = 320)),
21      box(title = "Empleados", width = 7, status = "primary",
22          downloadButton("bajar", "Descargar Excel"), br(), br(),
23          DTOutput("tabla"))
24    )
25  )
26 )
27
28 server <- function(input, output, session) {
29   datos <- reactive(filtrar_empleados(empleados, input$deptos,
30                                       input$sucursales))
31   kpis <- reactive(kpis_rrhh(datos()))
32
33   output$plantilla <- renderValueBox(
34     valueBox(kpis()$plantilla, "Plantilla", icon = icon("users")))
35   output$salario <- renderValueBox(
36     valueBox(dollar(kpis()$salario_prom, accuracy = 1),
37              "Salario mensual promedio", icon = icon("money-bill")))
38   output$antigüedad <- renderValueBox(
39     valueBox(paste(round(kpis()$antigüedad_prom, 1), "años"),
40              "Antigüedad promedio", icon = icon("clock")))
41   output$posgrado <- renderValueBox(
42     valueBox(percent(kpis()$pct_posgrado, accuracy = 1),
43              "Con maestría o doctorado", icon = icon("graduation-cap")))
44
45   output$graf_puesto <- renderPlot({
46     validate(need(nrow(datos()) > 0, "No hay empleados con esos filtros."))

```

```

47   grafica_salario_puesto(datos())
48 }, res = 96)
49
50 output$tabla <- renderDT(
51   datatable(tabla_empleados(datos()), rownames = FALSE,
52             options = list(pageLength = 8)) %>%
53   formatCurrency("Salario", currency = "$", digits = 0))
54
55 output$bajar <- downloadHandler(
56   filename = function() paste0("empleados_", Sys.Date(), ".xlsx"),
57   content = function(file) write_xlsx(tabla_empleados(datos()), file)
58 )
59 }
60
61 testServer(server, {
62   session$setInputs(deptos = c("IT", "Finanzas"), sucursales = NULL)
63   cat("Plantilla IT + Finanzas:", kpis()$plantilla, "\n")
64   cat("Filas en el Excel:",
65       nrow(readxl::read_excel(output$bajar)), "\n")
66 })

```

```

Plantilla IT + Finanzas: 18
Filas en el Excel: 18

```

La prueba confirma que los filtros múltiples funcionan y que el Excel descargado contiene exactamente a los empleados filtrados. Para abrir el dashboard:

```

1 shinyApp(ui, server)

```

A.7 Módulo 7. Machine learning para BI

Solución del ejercicio 7.1

Parte conceptual.

- Estimar el ticket promedio de un cliente nuevo: **regresión** (la respuesta es un número). Regresión lineal o random forest.
- Detectar si un correo es una queja: **clasificación** (queja / no queja). Regresión logística o random forest sobre variables del texto (palabras clave).
- Agrupar sucursales por su mezcla de ventas: **clustering**. k-means con las variables escaladas.
- Demanda semanal de pan de muerto: **serie de tiempo** con estacionalidad anual muy marcada. Ingenio estacional como línea base, ETS o ARIMA.

- (e) Qué recomendar a quien compra una cafetera: **reglas de asociación** (análisis de canasta) ordenadas por lift.

Parte práctica. Ajustamos el modelo y extraemos el R^2 ajustado:

Enunciado en la página 496

```
1 library(tidyverse)
2 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
3
4 modelo <- lm(total ~ descuento_pct + dia_semana, data = ventas)
5 round(summary(modelo)$r.squared, 4)
6 round(summary(modelo)$adj.r.squared, 4)
```

```
[1] 0.004
[1] -0.0155
```

El R^2 es de apenas 0.004 y el R^2 ajustado es negativo (-0.0155): el descuento y el día de la semana no explican el total de la venta. No hay señal, porque en el generador de datos el descuento y el día se eligieron al azar, independientemente del monto.

Solución del ejercicio 7.2

Escribimos cada fórmula directamente con vectores:

Enunciado en la página 497

```
1 real <- c(120, 95, 150, 80, 110, 130)
2 pronostico <- c(110, 100, 140, 90, 115, 120)
3 error <- real - pronostico
4
5 mae <- mean(abs(error))
6 rmse <- sqrt(mean(error^2))
7 mape <- mean(abs(error) / real) * 100
8 r2 <- 1 - sum(error^2) / sum((real - mean(real))^2)
9
10 round(c(MAE = mae, RMSE = rmse, MAPE = mape, R2 = r2), 3)
```

```
MAE RMSE MAPE R2
8.333 8.660 7.500 0.856
```

En promedio, el gerente se equivoca por 8.3 mil pesos por sucursal (MAE), que equivale a 7.5 % del valor real (MAPE). El RMSE (8.66) es apenas mayor que el MAE porque ningún error es mucho más grande que los demás (todos son de 5 o 10 mil pesos). El R^2 de 0.856 indica que el pronóstico captura el 85.6 % de las diferencias entre sucursales.

Solución del ejercicio 7.3

Los datos: 50 fraudes reales, de los que se detectaron 40 (VP = 40, FN = 10); 950 legítimas, de las que 60 se marcaron como fraude (FP = 60, VN = 890).

Enunciado en la página 497

```
1 matriz <- matrix(c(890, 60, 10, 40), nrow = 2,
2                     dimnames = list(Prediccion = c("Legítima", "Fraude"),
3                                         Real = c("Legítima", "Fraude")))
4 matriz
5
6 vp <- 40; fn <- 10; fp <- 60; vn <- 890
7 precision <- vp / (vp + fp)
8 sensibilidad <- vp / (vp + fn)
9 round(c(exactitud = (vp + vn) / (vp + vn + fp + fn),
10         precision = precision,
11         sensibilidad = sensibilidad,
12         especificidad = vn / (vn + fp),
13         F1 = 2 * precision * sensibilidad / (precision + sensibilidad)), 3)
14
15 costo_errores <- fn * 5000 + fp * 100
16 costo_errores
```

| | Real | | | | |
|------------|----------|--------|-----------|--------------|---------------|
| Prediccion | Legítima | Fraude | | | |
| Legítima | 890 | 10 | | | |
| Fraude | 60 | 40 | | | |
| exactitud | | | precision | sensibilidad | especificidad |
| | 0.930 | | 0.400 | 0.800 | 0.937 |
| F1 | | | | | 0.533 |

[1] 56000

La exactitud (93%) parece alta, pero la precisión es de solo 40%: de cada 10 alertas, 6 son transacciones legítimas. La sensibilidad de 80% indica que se detecta a 4 de cada 5 fraudes. El costo de los errores es de \$56,000: \$50,000 por los 10 fraudes no detectados y \$6,000 por las 60 revisiones innecesarias. Con estos costos, conviene bajar el umbral aunque aumenten las falsas alarmas: cada fraude detectado adicional ahorra lo mismo que 50 revisiones.

Solución del ejercicio 7.4

Calculamos el RFM, las calificaciones con `ntile()` (recordando que en recencia menos es mejor) y los niveles con `case_when()`:

Enunciado en la página 497

```
1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
```

```

3           show_col_types = FALSE)
4 fecha_corte <- as.Date("2024-01-01")
5
6 niveles <- transacciones %>%
7   group_by(cliente_id) %>%
8   summarise(recencia = as.numeric(fecha_corte - max(fecha)),
9             frecuencia = n(),
10            monto      = sum(total_transaccion), .groups = "drop") %>%
11   mutate(r = ntile(desc(recencia), 5),
12          f = ntile(frecuencia, 5),
13          m = ntile(monto, 5),
14          puntaje = r + f + m,
15          nivel = case_when(puntaje >= 13 ~ "Oro",
16                            puntaje >= 9  ~ "Plata",
17                            TRUE         ~ "Bronce"),
18          nivel = factor(nivel, levels = c("Oro", "Plata", "Bronce")))
19
20 niveles %>%
21   group_by(nivel) %>%
22   summarise(clientes = n(),
23            monto_prom = round(mean(monto)),
24            ingresos = sum(monto), .groups = "drop") %>%
25   mutate(pct_ingresos = round(100 * ingresos / sum(ingresos), 1)) %>%
26   select(-ingresos)

```

```

# A tibble: 3 x 4
  nivel  clientes monto_prom pct_ingresos
<fct>   <int>     <dbl>     <dbl>
1 Oro      39      8439       34.3
2 Plata    70      5444       39.7
3 Bronce   89      2807       26

```

Los 39 clientes Oro (20% de los 198 compradores) generan el 34.3% de los ingresos, con un monto promedio tres veces mayor que el de los Bronce (\$8,439 contra \$2,807). Los Plata son el grupo con mayor peso en ingresos (39.7%): son el candidato natural para programas que los conviertan en Oro.

Solución del ejercicio 7.5

Construimos las variables por cliente, escalamos y calculamos la silueta promedio para k de 2 a 6:

Enunciado en la página 497

```

1 library(tidyverse)
2 library(cluster)
3 transacciones <- read_csv("datasets/transacciones.csv",
4                           show_col_types = FALSE)

```

```

5
6 perfil_cliente <- transacciones %>%
7   group_by(cliente_id) %>%
8   summarise(monto_total = sum(total_transaccion),
9             ticket_promedio = mean(total_transaccion),
10            n_tiendas = n_distinct(tienda_id),
11            n_productos = n_distinct(producto_id), .groups = "drop")
12
13 datos_esc <- scale(perfil_cliente %>% select(-cliente_id))
14
15 siluetas <- map_dbl(2:6, function(k) {
16   set.seed(123)
17   km <- kmeans(datos_esc, centers = k, nstart = 25)
18   mean(silhouette(km$cluster, dist(datos_esc))[, "sil_width"])
19 })
20 tibble(k = 2:6, silueta = round(siluetas, 3))
21 k_elegido <- (2:6)[which.max(siluetas)]
22 k_elegido

```

```

# A tibble: 5 x 2
      k silueta
  <int>   <dbl>
1     2  0.322
2     3  0.325
3     4  0.356
4     5  0.326
5     6  0.331
[1] 4

```

La silueta más alta es para $k = 4$ (0.356), aunque todas están entre 0.32 y 0.36: la estructura de grupos es débil (datos aleatorios). El código guarda el k elegido en `k_elegido` para no escribirlo a mano.

```

1 set.seed(123)
2 km <- kmeans(datos_esc, centers = k_elegido, nstart = 25)
3
4 perfil <- perfil_cliente %>%
5   mutate(cluster = km$cluster) %>%
6   group_by(cluster) %>%
7   summarise(clientes = n(),
8             monto_total = round(mean(monto_total)),
9             ticket = round(mean(ticket_promedio)),
10            tiendas = round(mean(n_tiendas), 1),
11            productos = round(mean(n_productos), 1), .groups = "drop")
12
13 # Nombres según el perfil (reglas de negocio, nunca por número de grupo)
14 extremos <- range(perfil$monto_total)

```



```

15 perfil %>%
16   mutate(segmento = case_when(
17     monto_total == extremos[2] ~ "Alto valor",
18     monto_total == extremos[1] ~ "Valor mínimo",
19     ticket == max(ticket[!monto_total %in% extremos]) ~
20       "Ticket alto ocasional",
21     TRUE ~ "Valor medio")) %>%
22   arrange(desc(monto_total)) %>%
23   select(segmento, everything(), -cluster)

```

```

# A tibble: 4 x 6
  segmento      clientes monto_total ticket tiendas productos
  <chr>          <int>      <dbl>  <dbl>   <dbl>      <dbl>
1 Alto valor         22      10159   1123     6.1        8.5
2 Valor medio        96       4999    873     4.7        5.6
3 Ticket alto ocasional 39       4756   1547     2.7         3
4 Valor mínimo        41       1734    597     2.6        2.8

```

Los nombres salen de reglas aplicadas al perfil: el mayor monto es “Alto valor” (22 clientes que compran en 6 tiendas distintas en promedio), el menor es “Valor mínimo”, y entre los dos restantes, el de mayor ticket es “Ticket alto ocasional” (39 clientes que compran pocas veces, en pocas tiendas, pero con el ticket más alto: \$1,547). Este último grupo no se ve en un RFM clásico y sugiere una estrategia propia: aumentar su frecuencia con recordatorios, porque cada visita vale mucho.

Solución del ejercicio 7.6

Reproducimos la simulación y la división del texto, y comparamos tres modelos:

Enunciado en la página 497

```

1 library(tidyverse)
2 library(caret)
3
4 set.seed(123)
5 n_clientes <- 2000
6 clientes_churn <- tibble(
7   cliente_id      = sprintf("C%04d", 1:n_clientes),
8   antiguedad_meses = sample(1:72, n_clientes, replace = TRUE),
9   plan             = sample(c("Básico", "Estándar", "Premium"),
10                             n_clientes, replace = TRUE,
11                             prob = c(0.5, 0.3, 0.2)),
12   uso_mensual      = rpois(n_clientes, 8),
13   quejas_6m        = rpois(n_clientes, 0.8),
14   pagos_tardios     = rpois(n_clientes, 0.4),
15   genero           = sample(c("F", "M"), n_clientes, replace = TRUE),
16   edad             = sample(18:70, n_clientes, replace = TRUE)
17 ) %>%

```

```

18 mutate(
19   efecto_plan = case_when(plan == "Básico" ~ 0,
20                           plan == "Estándar" ~ -0.6,
21                           TRUE ~ -1.3),
22   riesgo = 2.0 - 0.05 * antiguedad_meses +
23     1.0 * (antiguedad_meses <= 6) + 1.0 * quejas_6m -
24     0.35 * uso_mensual + 0.8 * pagos_tardios + efecto_plan,
25   churn = if_else(runif(n_clientes) < plogis(riesgo), "Si", "No"),
26   churn = factor(churn, levels = c("No", "Si")),
27   plan = factor(plan, levels = c("Básico", "Estándar", "Premium")),
28   cliente_nuevo = as.integer(antiguedad_meses <= 6) # variable nueva
29 ) %>%
30 select(-efecto_plan, -riesgo)
31
32 set.seed(123)
33 filas_ent <- createDataPartition(clientes_churn$churn, p = 0.7,
34                                 list = FALSE)
35 ent <- clientes_churn[filas_ent, ]
36 prueba <- clientes_churn[-filas_ent, ]
37
38 calcular_auc <- function(prob, real, positivo = "Si") {
39   p_pos <- prob[real == positivo]
40   p_neg <- prob[real != positivo]
41   mean(outer(p_pos, p_neg, ">") + 0.5 * outer(p_pos, p_neg, "=="))
42 }
43
44 f_original <- churn ~ antiguedad_meses + plan + uso_mensual + quejas_6m +
45   pagos_tardios + genero + edad
46 f_nuevo <- update(f_original, . ~ . + cliente_nuevo)
47 f_justo <- update(f_nuevo, . ~ . - genero - edad)
48
49 modelos <- list(Original = f_original, `Con cliente_nuevo` = f_nuevo,
50               `Sin genero ni edad` = f_justo) %>%
51   map(~ glm(.x, data = ent, family = binomial))
52
53 map_dbl(modelos, ~ calcular_auc(predict(.x, prueba, type = "response"),
54                                 prueba$churn)) %>%
55   round(3)
56
57 round(exp(coef(modelos[["Con cliente_nuevo"]])["cliente_nuevo"]), 2)

```

| | Original | Con cliente_nuevo | Sin genero ni edad |
|---------------|----------|-------------------|--------------------|
| | 0.860 | 0.862 | 0.861 |
| cliente_nuevo | | | |
| | | 2.36 | |

Agregar `cliente_nuevo` sube el AUC de 0.860 a 0.862 y su razón de momios es 2.36: estar en los primeros 6 meses multiplica por 2.36 los momios de abandono, además del efecto de la antigüedad.

La mejora en AUC es pequeña porque la antigüedad lineal ya capturaba parte del efecto, pero la variable tiene sentido de negocio (reforzar el *onboarding*). Quitar género y edad casi no cambia el AUC (0.861): el modelo más justo y más simple predice igual de bien, así que es el que conviene usar.

Solución del ejercicio 7.7

Reproducimos datos, división y modelo logístico, y recalculamos los costos con el nuevo precio del descuento:

Enunciado en la página 498

```

1 library(tidyverse)
2 library(caret)
3
4 set.seed(123)
5 n_clientes <- 2000
6 clientes_churn <- tibble(
7   cliente_id      = sprintf("C%04d", 1:n_clientes),
8   antigüedad_meses = sample(1:72, n_clientes, replace = TRUE),
9   plan            = sample(c("Básico", "Estándar", "Premium"),
10                          n_clientes, replace = TRUE,
11                          prob = c(0.5, 0.3, 0.2)),
12   uso_mensual     = rpois(n_clientes, 8),
13   quejas_6m       = rpois(n_clientes, 0.8),
14   pagos_tardios    = rpois(n_clientes, 0.4),
15   genero          = sample(c("F", "M"), n_clientes, replace = TRUE),
16   edad            = sample(18:70, n_clientes, replace = TRUE)
17 ) %>%
18 mutate(
19   efecto_plan = case_when(plan == "Básico" ~ 0,
20                          plan == "Estándar" ~ -0.6,
21                          TRUE ~ -1.3),
22   riesgo = 2.0 - 0.05 * antigüedad_meses +
23     1.0 * (antigüedad_meses <= 6) + 1.0 * quejas_6m -
24     0.35 * uso_mensual + 0.8 * pagos_tardios + efecto_plan,
25   churn = if_else(runif(n_clientes) < plogis(riesgo), "Si", "No"),
26   churn = factor(churn, levels = c("No", "Si")),
27   plan = factor(plan, levels = c("Básico", "Estándar", "Premium"))
28 ) %>%
29 select(-efecto_plan, -riesgo)
30
31 set.seed(123)
32 filas_ent <- createDataPartition(clientes_churn$churn, p = 0.7,
33                                 list = FALSE)
34 ent <- clientes_churn[filas_ent, ]
35 prueba <- clientes_churn[-filas_ent, ]
36
37 modelo_log <- glm(churn ~ antigüedad_meses + plan + uso_mensual +
38                  quejas_6m + pagos_tardios + genero + edad,

```

```

39       data = ent, family = binomial)
40 prob <- predict(modelo_log, prueba, type = "response")
41 real <- prueba$churn == "Si"
42
43 costo_descuento <- 800
44 costo_vp <- costo_descuento + 0.5 * 3000
45 costo_fp <- costo_descuento
46 costo_fn <- 3000
47
48 costos <- map_dfr(seq(0.05, 0.95, by = 0.05), function(u) {
49   pred <- prob >= u
50   tibble(umbral = u, llamadas = sum(pred),
51          costo_total = sum(pred & real) * costo_vp +
52                        sum(pred & !real) * costo_fp + sum(!pred & real) * costo_fn)
53 })
54 costos %>% arrange(costo_total) %>% head(3)

```

```

# A tibble: 3 x 3
  umbral llamadas costo_total
  <dbl>    <int>    <dbl>
1  0.6      74    381700
2  0.65     61    381800
3  0.7      52    382100

```

Con un descuento de \$800, el umbral óptimo sube de 0.15 a 0.6 y solo se llamaría a 74 clientes de prueba (costo total de \$381,700). La razón: cada llamada ahora es más cara y el ahorro al retener a un cliente (la mitad de \$3,000 menos \$800) es mucho menor, así que solo vale la pena llamar a quien tiene una probabilidad alta de irse. Observa que los tres mejores umbrales tienen costos casi idénticos: la curva de costos es plana en esa zona.

Solución del ejercicio 7.8

Reproducimos la serie mensual, la agregamos a trimestres con `aggregate()` y comparamos los modelos:

Enunciado en la página 498

```

1 library(tidyverse)
2 library(forecast)
3
4 set.seed(123)
5 meses <- seq(as.Date("2021-01-01"), as.Date("2025-12-01"), by = "month")
6 n_meses <- length(meses)
7 factor_mes <- c(0.88, 0.85, 0.95, 0.97, 1.04, 0.96,
8                0.97, 1.00, 0.93, 0.98, 1.15, 1.32)
9 factor_mes <- factor_mes / mean(factor_mes)
10 unidades <- round((1500 + 15 * (1:n_meses)) * factor_mes[month(meses)] *

```

```

11           (1 + rnorm(n_meses, 0, 0.03)))
12 serie <- ts(unidades, start = c(2021, 1), frequency = 12)
13
14 # Suma por trimestre: frecuencia 4
15 serie_trim <- aggregate(serie, nfrequency = 4, FUN = sum)
16 serie_trim
17
18 ent <- window(serie_trim, end = c(2024, 4))
19 prueba <- window(serie_trim, start = c(2025, 1))
20
21 pronosticos <- list(
22   `Ingenuo estacional` = snaive(ent, h = 4),
23   ETS = forecast(ets(ent), h = 4),
24   ARIMA = forecast(auto.arima(ent), h = 4)
25 )
26 comparacion <- map_dfr(pronosticos, function(p) {
27   acc <- accuracy(p, prueba)["Test set", ]
28   tibble(MAE = acc["MAE"], RMSE = acc["RMSE"], MAPE = acc["MAPE"])
29 }, .id = "modelo") %>%
30   mutate(across(where(is.numeric), ~ round(.x, 1))) %>%
31   arrange(MAPE)
32 comparacion

```

```

      Qtr1 Qtr2 Qtr3 Qtr4
2021 4139 4765 4626 5822
2022 4579 5229 5176 6240
2023 4999 5757 5797 7156
2024 5557 6200 6399 7468
2025 6089 6903 6774 8305
# A tibble: 3 x 4
  modelo      MAE  RMSE  MAPE
  <chr>      <dbl> <dbl> <dbl>
1 ETS        96.2  105.   1.4
2 ARIMA      163.  196.   2.2
3 Ingenuo estacional 612.  636.   8.6

```

`aggregate(serie, nfrequency = 4, FUN = sum)` suma los tres meses de cada trimestre. En trimestres, ETS es el mejor modelo con un MAPE de 1.4 %, seguido de ARIMA (2.2 %); el ingenuo estacional se equivoca 8.6 % porque no considera el crecimiento. Reentrenamos el ganador con toda la serie:

```

1 # Reentrenamos el ganador (menor MAPE) con toda la serie trimestral
2 ganador <- comparacion$modelo[1]
3 modelo_final <- switch(ganador,
4   ETS = ets(serie_trim),
5   ARIMA = auto.arima(serie_trim),
6   snaive(serie_trim, h = 4))

```

```

7 ganador
8 forecast(modelo_final, h = 4)

```

```

[1] "ETS"
      Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
2026 Q1      6504.441 6355.576 6653.305 6276.772 6732.109
2026 Q2      7369.522 7200.758 7538.287 7111.419 7627.625
2026 Q3      7303.435 7136.086 7470.783 7047.497 7559.372
2026 Q4      8855.932 8652.896 9058.969 8545.414 9166.450

```

El código elige el modelo automáticamente con `switch()` según el ganador. Para 2026 se esperan 6,504 unidades en el primer trimestre y 8,856 en el cuarto (entre 8,653 y 9,059 con 80 % de confianza). Con datos trimestrales hay menos observaciones (20), pero los errores relativos son menores porque al sumar tres meses el ruido mensual se compensa.

Solución del ejercicio 7.9

Reproducimos los tickets, marcamos por ticket si tiene cada producto y contamos:

Enunciado en la página 498

```

1 library(tidyverse)
2
3 set.seed(123)
4 n_tickets <- 3000
5 tickets_ancha <- tibble(
6   ticket_id = 1:n_tickets,
7   Laptop    = rbinom(n_tickets, 1, 0.08),
8   Monitor   = rbinom(n_tickets, 1, 0.07),
9   Impresora  = rbinom(n_tickets, 1, 0.05),
10  Audifonos  = rbinom(n_tickets, 1, 0.15),
11  USB        = rbinom(n_tickets, 1, 0.18),
12  Papel      = rbinom(n_tickets, 1, 0.20),
13  Teclado    = rbinom(n_tickets, 1, 0.08)
14 ) %>%
15 mutate(
16   Mouse = if_else(Laptop == 1, rbinom(n(), 1, 0.60),
17                     rbinom(n(), 1, 0.08)),
18   Funda = if_else(Laptop == 1, rbinom(n(), 1, 0.40),
19                     rbinom(n(), 1, 0.02)),
20   `Cable HDMI` = if_else(Monitor == 1, rbinom(n(), 1, 0.50),
21                           rbinom(n(), 1, 0.04)),
22   Tinta = if_else(Impresora == 1, rbinom(n(), 1, 0.70),
23                   rbinom(n(), 1, 0.06))
24 )
25 tickets <- tickets_ancha %>%
26   pivot_longer(-ticket_id, names_to = "producto", values_to = "compro") %>%

```

```

27   filter(compro == 1) %>%
28   select(-compro)
29
30   # Una fila por ticket con TRUE/FALSE para los productos de interés
31   por_ticket <- tickets %>%
32     group_by(ticket_id) %>%
33     summarise(laptop = "Laptop" %in% producto,
34               mouse  = "Mouse"  %in% producto,
35               funda  = "Funda"  %in% producto, .groups = "drop")
36   n_total <- nrow(por_ticket)
37
38   soporte_funda <- mean(por_ticket$funda)
39   tres <- por_ticket %>%
40     reframe(regla = c("Laptop+Mouse -> Funda", "Laptop -> Funda"),
41             tickets_x = c(sum(laptop & mouse), sum(laptop)),
42             tickets_xy = c(sum(laptop & mouse & funda),
43                             sum(laptop & funda)))
44   tres %>%
45     mutate(soporte = round(tickets_xy / n_total, 3),
46            confianza = round(tickets_xy / tickets_x, 3),
47            lift      = round(tickets_xy / tickets_x / soporte_funda, 2))

```

```

# A tibble: 2 x 6
  regla                tickets_x tickets_xy soporte confianza lift
  <chr>                <int>      <int>    <dbl>    <dbl> <dbl>
1 Laptop+Mouse -> Funda    136         51  0.026    0.375  5.14
2 Laptop -> Funda         225         82  0.042    0.364  5

```

Saber que el cliente ya lleva mouse casi no cambia la probabilidad de que lleve funda: la confianza pasa de 36.4% a 37.5% y el lift de 5.0 a 5.14. Tiene sentido, porque en la simulación la funda depende solo de la laptop. Una regla de tres productos solo aporta si su confianza es claramente mayor que la de la regla más simple; si no, quédate con la simple, que además tiene más soporte (4.2% contra 2.6% de los tickets). Nota: usamos `reframe()` en lugar de `summarise()` porque el resultado tiene dos filas.

Solución del ejercicio 7.10

Organizamos el reto en funciones: una para simular clientes (la usamos para el histórico y para el lote nuevo), el entrenamiento con **caret**, la elección del modelo y la función de puntuación.

Enunciado en la página 498

```

1 library(tidyverse)
2 library(caret)
3 library(randomForest)
4
5 simular_clientes <- function(n, prefijo = "C") {

```

```

6  tibble(
7    cliente_id      = sprintf("%s%05d", prefijo, 1:n),
8    antiguedad_meses = sample(1:72, n, replace = TRUE),
9    plan             = sample(c("Básico", "Estándar", "Premium"), n,
10                           replace = TRUE, prob = c(0.5, 0.3, 0.2)),
11    uso_mensual      = rpois(n, 8),
12    quejas_6m        = rpois(n, 0.8),
13    pagos_tardios     = rpois(n, 0.4),
14    genero           = sample(c("F", "M"), n, replace = TRUE),
15    edad             = sample(18:70, n, replace = TRUE)
16  ) %>%
17  mutate(
18    efecto_plan = case_when(plan == "Básico" ~ 0,
19                           plan == "Estándar" ~ -0.6,
20                           TRUE ~ -1.3),
21    riesgo = 2.0 - 0.05 * antiguedad_meses +
22            1.0 * (antiguedad_meses <= 6) + 1.0 * quejas_6m -
23            0.35 * uso_mensual + 0.8 * pagos_tardios + efecto_plan,
24    churn = if_else(runif(n) < plogis(riesgo), "Si", "No"),
25    churn = factor(churn, levels = c("Si", "No")), # "Si" primero: caret
26    plan = factor(plan, levels = c("Básico", "Estándar", "Premium")),
27    cuota_mensual = case_when(plan == "Básico" ~ 199,
28                              plan == "Estándar" ~ 349,
29                              TRUE ~ 599)
30  ) %>%
31  select(-efecto_plan, -riesgo)
32 }
33
34 set.seed(2024)
35 historico <- simular_clientes(3000)
36 set.seed(2024)
37 filas_ent <- createDataPartition(historico$churn, p = 0.7, list = FALSE)
38 ent <- historico[filas_ent, ]
39 prueba <- historico[-filas_ent, ]
40
41 # Sin género ni edad (sección de ética)
42 formula_churn <- churn ~ antiguedad_meses + plan + uso_mensual +
43   quejas_6m + pagos_tardios
44 control <- trainControl(method = "cv", number = 5, classProbs = TRUE,
45                          summaryFunction = twoClassSummary)
46 set.seed(2024)
47 m_log <- train(formula_churn, data = ent, method = "glm",
48               family = binomial, trControl = control, metric = "ROC")
49 set.seed(2024)
50 m_rf <- train(formula_churn, data = ent, method = "rf", ntree = 200,
51               trControl = control, metric = "ROC",
52               tuneGrid = data.frame(mtry = 2:3))
53
54 calcular_auc <- function(prob, real, positivo = "Si") {

```



```

55 p_pos <- prob[real == positivo]
56 p_neg <- prob[real != positivo]
57 mean(outer(p_pos, p_neg, ">") + 0.5 * outer(p_pos, p_neg, "=="))
58 }
59 auc_prueba <- c(
60   Logistica = calcular_auc(predict(m_log, prueba, type = "prob")[, "Si"],
61                               prueba$churn),
62   RandomForest = calcular_auc(predict(m_rf, prueba, type = "prob")[, "Si"],
63                                 prueba$churn))
64 round(auc_prueba, 3)
65
66 mejor <- list(Logistica = m_log, RandomForest = m_rf)[[
67   names(which.max(auc_prueba))]]
68 dir.create("resultados", showWarnings = FALSE)
69 saveRDS(mejor, "resultados/modelo_churn_reto.rds")

```

| Logistica | RandomForest |
|-----------|--------------|
| 0.873 | 0.843 |

La regresión logística gana en prueba (AUC de 0.873 contra 0.843 del random forest), así que es el modelo que se guarda en `resultados/modelo_churn_reto.rds`. El código elige al ganador con `which.max()`: si mañana gana el bosque, el resto del sistema sigue funcionando igual, porque ambos modelos de `caret` se usan con `predict(..., type = "prob")`.

```

1 puntuar <- function(nuevos, modelo, top = 100) {
2   nuevos %>%
3     mutate(prob_churn = predict(modelo, newdata = nuevos,
4                                 type = "prob")[, "Si"],
5            perdida_esperada = prob_churn * cuota_mensual * 12) %>%
6     arrange(desc(perdida_esperada)) %>%
7     slice_head(n = top) %>%
8     mutate(prioridad = row_number()) %>%
9     select(prioridad, cliente_id, plan, quejas_6m, prob_churn,
10            perdida_esperada)
11 }
12
13 set.seed(99)
14 lote_nuevo <- simular_clientes(1000, prefijo = "N") %>% select(-churn)
15
16 modelo <- readRDS("resultados/modelo_churn_reto.rds")
17 top100 <- puntuar(lote_nuevo, modelo)
18 write_csv(top100, "resultados/top100_churn.csv")
19
20 top100 %>%
21   mutate(across(c(prob_churn, perdida_esperada), ~ round(.x, 2))) %>%
22   head(5)
23 nrow(read_csv("resultados/top100_churn.csv", show_col_types = FALSE))

```

```
# A tibble: 5 x 6
  prioridad cliente_id plan      quejas_6m prob_churn perdida_esperada
    <int>   <chr>      <fct>      <int>      <dbl>      <dbl>
1         1 N00330    Premium         3         0.94         6765.
2         2 N00043    Premium         3         0.89         6399.
3         3 N00814    Premium         2         0.87         6268.
4         4 N00456    Premium         2         0.75         5368.
5         5 N00904    Premium         1         0.67         4787.
[1] 100
```

La función carga el modelo guardado (como se haría en producción), calcula la pérdida esperada de cada cliente del lote y exporta los 100 más urgentes; releer el CSV confirma que tiene 100 filas. Los primeros lugares son clientes Premium con varias quejas y probabilidades de 0.67 a 0.94. Para ponerlo en producción faltaría programar el script (Módulo 8) y monitorear la deriva de las variables cada semana.

A.8 Módulo 8. Reportes automáticos

Solución del ejercicio 8.1

El bloque que lee los datos lleva `include = FALSE` para que no se vea; la fecha es código en línea en el YAML, y la frase usa código en línea con `nrow()` y `sum()` sobre la columna lógica `es_premium` (cada `TRUE` cuenta como 1). Como el archivo se guarda en `reportes/`, los datos se buscan también en `../datasets`.

Enunciado en la página 547

reportes/ej1-clientes.Rmd

```
1 ---
2 title: "Reporte de clientes"
3 author: "Tu nombre"
4 date: "`r format(Sys.Date(), '%d/%m/%Y')`"
5 output: html_document
6 ---
7
8 ```{r datos, include = FALSE}
9 library(dplyr)
10 library(readr)
11 library(knitr)
12 carpeta <- if (dir.exists("datasets")) "datasets" else "../datasets"
13 clientes <- read_csv(file.path(carpeta, "clientes.csv"),
14                      show_col_types = FALSE)
15 ```
16
17 ## Resumen
```

```

18
19 La base tiene **`r nrow(clientes)` clientes**, de los cuales
20 **`r sum(clientes$es_premium)`** son premium
21 (`r round(mean(clientes$es_premium) * 100, 1)` %).
22
23 ## Clientes por ciudad
24
25 ```{r tabla-ciudades, echo = FALSE}
26 clientes %>%
27   count(ciudad, sort = TRUE) %>%
28   kable(col.names = c("Ciudad", "Clientes"),
29         caption = "Clientes por ciudad")
30 ```

```

Lo generamos con **Knit** (o con `rmarkdown::render("reportes/ej1-clientes.Rmd")`). Para comprobar qué dirá el reporte, ejecutamos los mismos cálculos en la consola:

```

1 library(tidyverse)
2 library(knitr)
3 clientes <- read_csv("datasets/clientes.csv", show_col_types = FALSE)

```

```

1 nrow(clientes)
2 sum(clientes$es_premium)
3 round(mean(clientes$es_premium) * 100, 1)
4 clientes %>%
5   count(ciudad, sort = TRUE) %>%
6   kable(format = "pipe", col.names = c("Ciudad", "Clientes"))

```

```

[1] 200
[1] 73
[1] 36.5

```

| Ciudad | Clientes |
|-------------|----------|
| Puebla | 28 |
| CDMX | 27 |
| Guadalajara | 22 |
| Cancún | 20 |
| Toluca | 20 |
| León | 18 |
| Mérida | 17 |
| Querétaro | 17 |
| Tijuana | 17 |
| Monterrey | 14 |

El reporte generado dice que hay 200 clientes, 73 premium (36.5 %), y la tabla empieza con Puebla (28 clientes) y CDMX (27).

Solución del ejercicio 8.2

Unimos las transacciones con la categoría del producto, resumimos, ordenamos *antes* de formatear (porque al formatear las columnas se vuelven texto) y damos formato de negocio con **scales**.

Enunciado en la página 547

```

1 library(tidyverse)
2 library(knitr)
3 library(scales)
4
5 transacciones <- read_csv("datasets/transacciones.csv",
6                           show_col_types = FALSE)
7 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
8
9 por_categoria <- transacciones %>%
10   left_join(productos %>% select(producto_id, categoria),
11             by = "producto_id") %>%
12   group_by(categoria) %>%
13   summarise(ventas = sum(total_transaccion),
14             unidades = sum(cantidad), .groups = "drop") %>%
15   arrange(desc(ventas)) %>% # ordenar con números
16   mutate(participacion = percent(ventas / sum(ventas), accuracy = 0.1),
17          ventas = dollar(ventas, accuracy = 1),
18          unidades = comma(unidades))
19
20 kable(por_categoria, format = "pipe",
21       col.names = c("Categoría", "Ventas", "Unidades", "% del total"),
22       align = c("l", "r", "r", "r"),
23       caption = "Ventas 2023 por categoría")

```

Table: Ventas 2023 por categoría

| Categoría | Ventas | Unidades | % del total |
|-------------|-----------|----------|-------------|
| Belleza | \$182,143 | 551 | 19.0% |
| Electrónica | \$132,672 | 403 | 13.8% |
| Juguetes | \$125,093 | 356 | 13.0% |
| Alimentos | \$121,543 | 356 | 12.7% |
| Automotriz | \$104,085 | 325 | 10.8% |
| Deportes | \$86,171 | 280 | 9.0% |
| Hogar | \$63,407 | 187 | 6.6% |
| Mascotas | \$57,668 | 201 | 6.0% |
| Libros | \$57,162 | 204 | 6.0% |
| Ropa | \$30,048 | 106 | 3.1% |

Belleza es la categoría que más vende: \$182,143, el 19.0 % del total, seguida de Electrónica (13.8 %). Ropa queda al final con solo el 3.1 %. Observa la alineación: los números a la derecha permiten comparar magnitudes de un vistazo.

Solución del ejercicio 8.3

La función calcula primero el cumplimiento (`ventas / meta`) y con `if/else if/else` elige la frase. Para aplicarla a cada trimestre usamos `map_chr()`, que llama a la función con cada valor de ventas.

Enunciado en la página 548

```

1 library(tidyverse)
2 library(scales)
3
4 frase_meta <- function(ventas, meta) {
5   cumplimiento <- ventas / meta
6   montos <- paste0("(ventas de ", dollar(ventas, accuracy = 1),
7                     " contra meta de ", dollar(meta, accuracy = 1), ").")
8   if (ventas >= meta) {
9     paste("Se superó la meta en",
10          percent(cumplimiento - 1, accuracy = 0.1), montos)
11   } else if (cumplimiento >= 0.9) {
12     paste("Se alcanzó el", percent(cumplimiento, accuracy = 0.1),
13          "de la meta", montos)
14   } else {
15     paste("Alerta: solo se alcanzó el",
16          percent(cumplimiento, accuracy = 0.1), "de la meta", montos)
17   }
18 }
19
20 # Los tres casos
21 frase_meta(110000, 100000)
22 frase_meta(95000, 100000)
23 frase_meta(70000, 100000)
24
25 # Aplicación a los trimestres de 2023
26 transacciones <- read_csv("datasets/transacciones.csv",
27                           show_col_types = FALSE)
28 trimestres <- transacciones %>%
29   group_by(trimestre) %>%
30   summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
31   mutate(frase = map_chr(ventas, frase_meta, meta = 220000))
32 cat(paste0(trimestres$trimestre, ": ", trimestres$frase), sep = "\n")

```

```

[1] "Se superó la meta en 10.0% (ventas de $110,000 contra meta de $100,000)."
[1] "Se alcanzó el 95.0% de la meta (ventas de $95,000 contra meta de $100,000)."
[1] "Alerta: solo se alcanzó el 70.0% de la meta (ventas de $70,000 contra meta de

```

↪ \$100,000)."

Q1: Se superó la meta en 8.8% (ventas de \$239,390 contra meta de \$220,000).
 Q2: Se superó la meta en 8.8% (ventas de \$239,457 contra meta de \$220,000).
 Q3: Se superó la meta en 6.5% (ventas de \$234,241 contra meta de \$220,000).
 Q4: Se superó la meta en 12.2% (ventas de \$246,905 contra meta de \$220,000).

Los cuatro trimestres superaron la meta de \$220,000; el mejor fue Q4, con 12.2 % por encima. Si cambias la meta a \$240,000, Q1, Q2 y Q3 pasan a “Se alcanzó el ...” sin tocar el texto: la frase depende solo de los datos.

Solución del ejercicio 8.4

El bloque `configuracion` fija las opciones globales; el bloque de cálculo lleva `include = FALSE`; la gráfica define su tamaño y su pie; y el bloque final usa `results = "asis"` para que el texto de `cat()` se convierta en una lista.

Enunciado en la página 548

reportes/ej4-opciones.Rmd

```

1  ---
2  title: "Ventas mensuales 2023"
3  output:
4    html_document: default
5    word_document: default
6  ---
7
8  ```{r configuracion, include = FALSE}
9  knitr::opts_chunk$set(echo = FALSE, message = FALSE, warning = FALSE)
10 library(dplyr)
11 library(readr)
12 library(ggplot2)
13 library(scales)
14 ```
15
16 ```{r calculos, include = FALSE}
17 carpeta <- if (dir.exists("datasets")) "datasets" else "../datasets"
18 ventas <- read_csv(file.path(carpeta, "ventas_retail.csv"),
19                   show_col_types = FALSE)
20 por_mes <- ventas %>%
21   group_by(mes) %>%
22   summarise(ventas = sum(total), .groups = "drop")
23 ```
24
25 ## Tendencia
26
27 ```{r grafica, fig.cap = "Ventas por mes, 2023", fig.width = 7,
28   fig.height = 3, out.width = "80%"}
29 ggplot(por_mes, aes(x = mes, y = ventas, group = 1)) +

```

```

30   geom_line(color = "#2a78d6", linewidth = 0.8) +
31   scale_y_continuous(labels = dollar) +
32   labs(x = NULL, y = "Ventas") +
33   theme_minimal() +
34   theme(panel.grid.minor = element_blank(),
35         axis.text.x = element_text(angle = 45, hjust = 1))
36   ```
37
38   ## Los tres mejores meses
39
40   ```{r top-meses, results = "asis"}
41   top3 <- por_mes %>% arrange(desc(ventas)) %>% slice_head(n = 3)
42   cat(paste0("- **", top3$mes, "***: ", dollar(top3$ventas, accuracy = 1)),
43       sep = "\n")
44   ```

```

Se genera en los dos formatos con:

```
1 rmarkdown::render("reportes/ej4-opciones.Rmd", output_format = "all")
```

La lista que escribe el último bloque es exactamente la que produce este código en la consola:

```

1 library(tidyverse)
2 library(scales)
3 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
4 por_mes <- ventas %>%
5   group_by(mes) %>%
6   summarise(ventas = sum(total), .groups = "drop")
7 top3 <- por_mes %>% arrange(desc(ventas)) %>% slice_head(n = 3)
8 cat(paste0("- **", top3$mes, "***: ", dollar(top3$ventas, accuracy = 1)),
9     sep = "\n")

```

```

- **2023-07**: $54,056
- **2023-11**: $50,398
- **2023-05**: $48,108

```

En el HTML y en el Word esas líneas aparecen como una lista con viñetas y los meses en negritas, sin recuadro gris ni **##**.

Solución del ejercicio 8.5

El parámetro se declara en el YAML y se usa como `params$categoria` en el filtro, en el título y en las frases.

Enunciado en la página 548

reportes/ej5-categoria.Rmd

```

1  ---
2  title: "Reporte de la categoría `r params$categoria`"
3  output: html_document
4  params:
5    categoria: "Electrónica"
6  ---
7
8  ```{r configuracion, include = FALSE}
9  knitr::opts_chunk$set(echo = FALSE, message = FALSE, warning = FALSE)
10 library(dplyr)
11 library(readr)
12 library(ggplot2)
13 library(scales)
14 library(knitr)
15 carpeta <- if (dir.exists("datasets")) "datasets" else "../datasets"
16 transacciones <- read_csv(file.path(carpeta, "transacciones.csv"),
17                           show_col_types = FALSE)
18 productos <- read_csv(file.path(carpeta, "productos.csv"),
19                       show_col_types = FALSE)
20 ventas_cat <- transacciones %>%
21   left_join(productos, by = "producto_id") %>%
22   filter(categoria == params$categoria)
23 if (nrow(ventas_cat) == 0) stop("Sin ventas de ", params$categoria)
24 ```
25
26 En 2023, la categoría **`r params$categoria`** vendió
27 **`r dollar(sum(ventas_cat$total_transaccion), accuracy = 1)`** en
28 `r nrow(ventas_cat)` transacciones.
29
30 ```{r top-productos}
31 ventas_cat %>%
32   group_by(nombre_producto) %>%
33   summarise(unidades = sum(cantidad),
34             ventas = sum(total_transaccion), .groups = "drop") %>%
35   arrange(desc(ventas)) %>%
36   slice_head(n = 5) %>%
37   mutate(ventas = dollar(ventas, accuracy = 1)) %>%
38   kable(col.names = c("Producto", "Unidades", "Ventas"),
39         align = c("l", "r", "r"),
40         caption = "Productos más vendidos")
41 ```
42
43 ```{r grafica, fig.cap = "Ventas por mes", fig.height = 3}
44 ventas_cat %>%
45   group_by(mes) %>%
46   summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
47   ggplot(aes(x = mes, y = ventas)) +

```



```

48   geom_col(fill = "#2a78d6", width = 0.7) +
49   scale_y_continuous(labels = dollar) +
50   labs(x = NULL, y = "Ventas") +
51   theme_minimal() +
52   theme(panel.grid.minor = element_blank(),
53         axis.text.x = element_text(angle = 45, hjust = 1))
54   ...

```

Para generar dos versiones, se recorre un vector de categorías y se da un `output_file` distinto a cada una (si no, la segunda sobrescribe a la primera):

```

1  for (cat_actual in c("Ropa", "Hogar")) {
2    rmarkdown::render(
3      "reportes/ej5-categoria.Rmd",
4      params      = list(categoria = cat_actual),
5      output_file  = paste0("categoria_", tolower(cat_actual), ".html"),
6      output_dir   = "resultados",
7      knit_root_dir = getwd(),
8      envir        = new.env()
9    )
10 }

```

Las frases de cada reporte coinciden con este cálculo:

```

1  library(tidyverse)
2  library(scales)
3  transacciones <- read_csv("datasets/transacciones.csv",
4                           show_col_types = FALSE)
5  productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
6  transacciones %>%
7    left_join(productos, by = "producto_id") %>%
8    filter(categoria %in% c("Ropa", "Hogar")) %>%
9    group_by(categoria) %>%
10    summarise(ventas = dollar(sum(total_transaccion), accuracy = 1),
11             transacciones = n(), .groups = "drop")

```

```

# A tibble: 2 x 3
  categoria ventas transacciones
  <chr>      <chr>          <int>
1 Hogar    $63,407             64
2 Ropa     $30,048             38

```

Se obtienen `resultados/categoria_ropa.html` y `resultados/categoria_hogar.html`, cada uno con su título, su frase, su tabla y su gráfica.

Solución del ejercicio 8.6

`expand_grid()` crea las 40 combinaciones; con `count()` contamos las transacciones por tienda y trimestre y las unimos con `left_join()`. Si alguna combinación no tuviera transacciones, quedaría `NA`, que convertimos en 0 con `replace_na()`.

Enunciado en la página 548

```

1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
5
6 limpiar_nombre <- function(x) {
7   x <- chartr("áéíóúüñÁÉÍÓÚÜÑ", "aeiouunAEIOUUN", x)
8   x <- gsub("[^A-Za-z0-9]+", "-", x)
9   tolower(gsub("^-|-$", "", x))
10 }
11
12 conteo <- transacciones %>% count(tienda_id, trimestre,
13                                  name = "n_transacciones")
14
15 trabajos <- expand_grid(tienda_id = tiendas$tienda_id,
16                         trimestre = paste0("Q", 1:4)) %>%
17   left_join(tiendas %>% select(tienda_id, nombre_tienda),
18             by = "tienda_id") %>%
19   left_join(conteo, by = c("tienda_id", "trimestre")) %>%
20   mutate(n_transacciones = replace_na(n_transacciones, 0),
21          archivo = sprintf("2023-%s_T%02d_%s.pdf", trimestre, tienda_id,
22                           limpiar_nombre(nombre_tienda)),
23          generar = n_transacciones >= 20)
24
25 nrow(trabajos)
26 trabajos %>% arrange(n_transacciones) %>%
27   select(archivo, n_transacciones, generar) %>%
28   head(4)
29 sum(trabajos$generar)

```

```

[1] 40
# A tibble: 4 x 3
  archivo                                n_transacciones generar
  <chr>                                <int> <lgl>
1 2023-Q2_T07_sucursal-plaza-b.pdf          16 FALSE
2 2023-Q4_T05_sucursal-oeste.pdf           18 FALSE
3 2023-Q4_T07_sucursal-plaza-b.pdf          18 FALSE
4 2023-Q4_T01_sucursal-centro.pdf           19 FALSE
[1] 34

```

Hay que generar 40 reportes (10 tiendas $\times$ 4 trimestres). La combinación con menos transacciones es la Sucursal Plaza B en el segundo trimestre (16). Seis combinaciones quedan por debajo de 20 y se marcan con `generar = FALSE`, así que el bucle generaría 34 reportes; en el script de automatización bastaría con recorrer `filter(trabajos, generar)`.

Solución del ejercicio 8.7

`split()` divide el detalle en una lista con un data frame por tienda; los nombres de esa lista serán los nombres de las hojas. Con `c()` ponemos la hoja `Resumen` al principio.

Enunciado en la página 548

```

1 library(tidyverse)
2 library(writexl)
3 library(readxl)
4
5 transacciones <- read_csv("datasets/transacciones.csv",
6                           show_col_types = FALSE)
7 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
8
9 detalle <- transacciones %>%
10   left_join(tiendas %>% select(tienda_id, nombre_tienda),
11             by = "tienda_id") %>%
12   mutate(hoja = str_remove(nombre_tienda, "Sucursal ")) %>%
13   arrange(tienda_id, fecha)
14
15 resumen <- detalle %>%
16   group_by(hoja) %>%
17   summarise(ventas = round(sum(total_transaccion), 2),
18             transacciones = n(), .groups = "drop") %>%
19   arrange(desc(ventas))
20
21 # Una hoja por tienda (sin la columna auxiliar "hoja")
22 por_tienda <- split(detalle %>% select(-hoja), detalle$hoja)
23
24 dir.create("resultados", showWarnings = FALSE)
25 archivo <- "resultados/ventas_por_tienda.xlsx"
26 write_xlsx(c(list(Resumen = resumen), por_tienda), path = archivo)
27
28 # Verificación
29 hojas <- excel_sheets(archivo)
30 length(hojas)
31 hojas
32 filas <- sapply(hojas[-1], function(h) nrow(read_excel(archivo, sheet = h)))
33 sum(filas)

```

```

[1] 11
[1] "Resumen" "Centro"  "Este"    "Express" "Norte"   "Oeste"

```

```
[7] "Outlet" "Plaza A" "Plaza B" "Premium" "Sur"
[1] 1000
```

El libro tiene 11 hojas (el resumen y diez tiendas) y las hojas de tiendas suman las 1,000 transacciones: no se perdió ni se duplicó ninguna. Observa que `split()` ordena las hojas alfabéticamente (Centro, Este, Express...); si quisieras otro orden, reordena la lista antes de escribirla.

Solución del ejercicio 8.8

La función valida sus argumentos con `stop()`, igual que la plantilla del reporte ejecutivo. El bucle envuelve cada llamada en `tryCatch()`: las filas exitosas se guardan en una lista de resultados y los errores en el registro, así que ninguna combinación inválida detiene el proceso.

Enunciado en la página 549

```
1 library(tidyverse)
2 library(scales)
3
4 transacciones <- read_csv("datasets/transacciones.csv",
5                           show_col_types = FALSE)
6 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
7
8 revisar_tienda <- function(id, mes) {
9   if (!id %in% tiendas$tienda_id) stop("La tienda ", id, " no existe.")
10  if (!mes %in% transacciones$mes) stop("No hay datos del mes ", mes)
11  mes_previo <- format(as.Date(paste0(mes, "-01")) - 1, "%Y-%m")
12  if (!mes_previo %in% transacciones$mes) {
13    stop("No hay datos del mes anterior (", mes_previo, ")")
14  }
15  v <- transacciones %>% filter(tienda_id == id)
16  actual <- sum(v$total_transaccion[v$mes == mes])
17  anterior <- sum(v$total_transaccion[v$mes == mes_previo])
18  variacion <- actual / anterior - 1
19  tibble(tienda_id = id,
20         tienda = tiendas$nombre_tienda[tiendas$tienda_id == id],
21         mes = mes, variacion = variacion,
22         estado = case_when(variacion > 0.10 ~ "Crecimiento sólido",
23                             variacion >= 0 ~ "Estable",
24                             variacion > -0.10 ~ "Ligera caída",
25                             TRUE ~ "Alerta"))
26 }
27
28 pendientes <- tibble(id = c(tiendas$tienda_id, 11, 1),
29                      mes = c(rep("2023-06", 10), "2023-06", "2023-01"))
30 resultados <- list()
31 registro <- list()
32
33 for (fila in seq_len(nrow(pendientes))) {
```

```

34 id <- pendientes$id[filas]
35 mes <- pendientes$mes[filas]
36 estado <- tryCatch({
37   resultados[[length(resultados) + 1]] <- revisar_tienda(id, mes)
38   "OK"
39 }, error = function(e) paste("ERROR:", conditionMessage(e)))
40 registro[[filas]] <- tibble(id = id, mes = mes, estado = estado)
41 }
42
43 registro <- bind_rows(registro)
44 registro %>% count(estado)
45 registro %>% filter(estado != "OK")
46
47 resultados <- bind_rows(resultados)
48 resultados %>%
49   mutate(variacion = percent(variacion, accuracy = 0.1)) %>%
50   select(tienda, variacion, estado)
51
52 # Texto Markdown para el reporte (iría en un bloque results = "asis")
53 alertas <- resultados %>% filter(estado == "Alerta")
54 cat(paste0("- **", alertas$tienda, "**: las ventas cayeron ",
55   percent(abs(alertas$variacion), accuracy = 0.1),
56   " respecto a mayo."), sep = "\n")

```

```

# A tibble: 3 x 2
  estado      n
  <chr>    <int>
1 ERROR: La tienda 11 no existe.      1
2 ERROR: No hay datos del mes anterior (2022-12) 1
3 OK                                  10
# A tibble: 2 x 3
  id mes      estado
  <dbl> <chr>   <chr>
1 11 2023-06 ERROR: La tienda 11 no existe.
2 1 2023-01 ERROR: No hay datos del mes anterior (2022-12)
# A tibble: 10 x 3
  tienda      variacion estado
  <chr>    <chr>    <chr>
1 Sucursal Centro -48.1%  Alerta
2 Sucursal Norte -73.5%  Alerta
3 Sucursal Sur 31.8%  Crecimiento sólido
4 Sucursal Este 245.5% Crecimiento sólido
5 Sucursal Oeste -41.0%  Alerta
6 Sucursal Plaza A -16.3%  Alerta
7 Sucursal Plaza B 29.1%  Crecimiento sólido
8 Sucursal Outlet 612.9% Crecimiento sólido
9 Sucursal Premium -8.0%  Ligera caída
10 Sucursal Express -45.5%  Alerta

```

- **Sucursal Centro**: las ventas cayeron 48.1% respecto a mayo.
- **Sucursal Norte**: las ventas cayeron 73.5% respecto a mayo.
- **Sucursal Oeste**: las ventas cayeron 41.0% respecto a mayo.
- **Sucursal Plaza A**: las ventas cayeron 16.3% respecto a mayo.
- **Sucursal Express**: las ventas cayeron 45.5% respecto a mayo.

De las 12 combinaciones, 10 se revisaron y 2 quedaron en el registro con su causa: la tienda 11 no existe y enero de 2023 no tiene mes anterior en los datos. En junio, cinco tiendas están en alerta y la Sucursal Outlet creció 612.9 %: una variación tan grande con tan pocas transacciones merece revisar los datos antes de celebrarla. El texto final está listo para insertarse en un reporte con `results = "asis"`.

A.9 Módulo 9. Proyecto final: retos de extensión

Solución del ejercicio 9.1

La recompra se mide sobre los clientes *activos* (los que compraron al menos una vez). Para los días entre compras ordenamos las compras de cada cliente por fecha y restamos la fecha anterior con `lag()`.

Enunciado en la página 603

```
1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
```

Al cargar **tidyverse** aparece su mensaje de bienvenida (no lo copiamos aquí). Ahora los tres cálculos:

```
1 # (a) Porcentaje de clientes activos con 2 o más compras
2 compras_cliente <- transacciones %>% count(cliente_id, name = "compras")
3 compras_cliente %>%
4   summarise(activos = n(),
5             recompran = sum(compras >= 2),
6             tasa_recompra = round(100 * mean(compras >= 2), 1))
7
8 # (b) Días entre compras consecutivas del mismo cliente
9 intervalos <- transacciones %>%
10  arrange(cliente_id, fecha) %>%
11  group_by(cliente_id) %>%
12  mutate(dias_desde_anterior = as.numeric(fecha - lag(fecha))) %>%
13  ungroup() %>%
14  filter(!is.na(dias_desde_anterior))
15
16 quantile(intervalos$dias_desde_anterior, c(0.25, 0.5, 0.75))
17
```

```

18 # (c) Clientes con al menos una recompra en 30 días o menos
19 intervalos %>%
20   group_by(cliente_id) %>%
21   summarise(recompra_30 = any(dias_desde_anterior <= 30)) %>%
22   summarise(clientes = sum(recompra_30))

```

```

# A tibble: 1 x 3
  activos recompran tasa_recompra
    <int>    <int>      <dbl>
1    198    192        97
25% 50% 75%
  19  41  82
# A tibble: 1 x 1
  clientes
    <int>
1    140

```

Interpretación: 192 de los 198 clientes activos (97%) compraron dos veces o más, así que “conseguir la segunda compra” no es el problema de TechRetail. La mediana entre compras es de 41 días (la mitad de las recompras ocurre entre 19 y 82 días) y 140 clientes volvieron alguna vez en 30 días o menos. Una campaña de segunda compra aportaría poco; es más útil una campaña que *acorte* el intervalo (por ejemplo, un cupón válido 30 días) y medir si la mediana baja de 41 días.

Solución del ejercicio 9.2

Unimos las ventas por sucursal con `tiendas.csv`, dividimos entre m^2 y empleados y usamos `min_rank(desc())` para los rankings.

Enunciado en la página 603

```

1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
5
6 productividad <- transacciones %>%
7   group_by(tienda_id) %>%
8   summarise(ventas = sum(total_transaccion), .groups = "drop") %>%
9   left_join(tiendas, by = "tienda_id") %>%
10  mutate(ventas_m2      = ventas / tamano_m2,
11         ventas_empleado = ventas / empleados,
12         rank_ventas     = min_rank(desc(ventas)),
13         rank_m2         = min_rank(desc(ventas_m2)),
14         rank_empleado   = min_rank(desc(ventas_empleado)),
15         cambio          = rank_ventas - rank_m2) %>%
16  arrange(rank_m2)
17

```

```

18 productividad %>%
19   transmute(tienda = str_remove(nombre_tienda, "Sucursal "),
20             m2 = tamano_m2, ventas = round(ventas),
21             por_m2 = round(ventas_m2), por_empleado = round(ventas_empleado),
22             rank_ventas, rank_m2, cambio)

```

```

# A tibble: 10 x 8
  tienda      m2 ventas por_m2 por_empleado rank_ventas rank_m2 cambio
  <chr>    <dbl> <dbl> <dbl>      <dbl>      <int>    <int> <int>
1 Express  300 102791   343      8566         4         1     3
2 Outlet   350 114377   327      7625         1         2    -1
3 Este     400  88420   221      4912         7         3     4
4 Norte    450  89024   198      4451         6         4     2
5 Centro   500  88414   177      3537         8         5     3
6 Sur      600 105305   176      3510         2         6    -4
7 Plaza B  480  80236   167      3821        10         7     3
8 Oeste    550  84672   154      3849         9         8     1
9 Plaza A  700 105216   150      3006         3         9    -6
10 Premium 800 101538   127      2538         5        10    -5

```

```

1 color_resalte <- "#eb6834"; color_gris <- "#b5b3ad"
2 productividad %>%
3   mutate(mejor = rank_m2 == 1) %>%
4   ggplot(aes(x = ventas_m2, y = fct_reorder(nombre_tienda, ventas_m2),
5       fill = mejor)) +
6   geom_col(width = 0.7, show.legend = FALSE) +
7   scale_fill_manual(values = c(`TRUE` = color_resalte,
8       `FALSE` = color_gris)) +
9   scale_x_continuous(labels = scales::dollar) +
10  labs(title = "Ventas por metro cuadrado, 2023",
11       x = "Ventas por m²", y = NULL) +
12  theme_minimal() +
13  theme(panel.grid.minor = element_blank())

```

Interpretación: la Express (300 m²) es la más productiva por metro cuadrado (\$343) y por empleado (\$8,566), seguida de la Outlet. La Plaza A es la que más cae: tercera en ventas totales y novena por m² (6 lugares), y la Premium queda última por m² (\$127). Si la dirección piensa en expansión, los formatos chicos producen casi el triple por metro cuadrado que los grandes; antes de decidir habría que agregar el costo de renta por m², que no está en los datos.

Solución del ejercicio 9.3

La venta diaria promedio es el total de unidades de 2023 entre 365 días. La cobertura dice cuántos días alcanza el stock actual a ese ritmo.

Enunciado en la página 603


```

1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                             show_col_types = FALSE)
4 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
5
6 alerta <- transacciones %>%
7   group_by(producto_id) %>%
8   summarise(unidades_2023 = sum(cantidad), .groups = "drop") %>%
9   right_join(productos, by = "producto_id") %>%
10  filter(activo) %>%
11  mutate(unidades_2023 = replace_na(unidades_2023, 0),
12         venta_diaria = unidades_2023 / 365,
13         dias_cobertura = stock_actual / venta_diaria,
14         bajo_minimo = stock_actual < stock_minimo,
15         alerta = dias_cobertura < 30 | bajo_minimo) %>%
16  filter(alerta) %>%
17  arrange(dias_cobertura) %>%
18  transmute(producto_id, stock = stock_actual, minimo = stock_minimo,
19            diaria = round(venta_diaria, 2),
20            cobertura = round(dias_cobertura), bajo_minimo)
21
22 alerta
23
24 # Contexto: distribución de la cobertura de todos los activos
25 productos %>%
26   filter(activo) %>%
27   left_join(count(transacciones, producto_id, wt = cantidad,
28                  name = "unidades"), by = "producto_id") %>%
29   mutate(dias_cobertura = stock_actual / (unidades / 365)) %>%
30   pull(dias_cobertura) %>%
31   quantile(c(0, 0.25, 0.5)) %>%
32   round()

```

```

# A tibble: 3 x 6
  producto_id stock minimo diaria cobertura bajo_minimo
      <dbl> <dbl>   <dbl> <dbl>      <dbl> <lgl>
1           3    22    26  0.18      125 TRUE
2           6    36    38  0.22      160 TRUE
3          33    42    45  0.21      199 TRUE
0%  25%  50%
125  927 1717

```

Interpretación: ningún producto activo tiene menos de 30 días de cobertura: el que menos tiene alcanza para 125 días. Las 3 alertas se deben a que el stock está por debajo del mínimo definido (productos 3, 6 y 33), aunque al ritmo de venta actual (unas 0.2 unidades diarias) les alcanza para más de cuatro meses. El hallazgo importante es el contrario: la mediana de cobertura es de 1,717 días (más de cuatro años de inventario). Con 1,000 transacciones al año, TechRetail parece

tener *sobreinventario*, y los `stock_minimo` están desalineados con la demanda real. Recomendación: recalcular mínimos a partir de la venta diaria (por ejemplo, 30 días de cobertura) y revisar si el stock se registra en unidades o en cajas.

Solución del ejercicio 9.4

La cohorte es el mes de la primera compra. Para cada compra calculamos cuántos meses pasaron desde ese mes (`meses_desde`) y contamos clientes distintos por cohorte y mes.

Enunciado en la página 604

```

1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                             show_col_types = FALSE)
4
5 compras <- transacciones %>%
6   mutate(mes_compra = floor_date(fecha, "month")) %>%
7   group_by(cliente_id) %>%
8   mutate(cohorte = min(mes_compra)) %>%
9   ungroup() %>%
10  mutate(meses_desde = (year(mes_compra) - year(cohorte)) * 12 +
11          month(mes_compra) - month(cohorte))
12
13 # Tamaño de cada cohorte
14 compras %>% distinct(cliente_id, cohorte) %>% count(cohorte) %>%
15   mutate(cohorte = format(cohorte, "%Y-%m"))
16
17 matriz <- compras %>%
18   distinct(cliente_id, cohorte, meses_desde) %>%
19   count(cohorte, meses_desde, name = "clientes") %>%
20   group_by(cohorte) %>%
21   mutate(pct = round(100 * clientes / clientes[meses_desde == 0])) %>%
22   ungroup()
23
24 # Primer trimestre: % de la cohorte que compró 1, 2 y 3 meses después
25 matriz %>%
26   filter(cohorte <= as.Date("2023-03-01"), meses_desde <= 3) %>%
27   mutate(cohorte = format(cohorte, "%Y-%m")) %>%
28   select(-clientes) %>%
29   pivot_wider(names_from = meses_desde, values_from = pct,
30               names_prefix = "mes_")

```

```

# A tibble: 10 x 2
  cohorte      n
  <chr>    <int>
1 2023-01     80
2 2023-02     36
3 2023-03     29

```

```

4 2023-04      16
5 2023-05      14
6 2023-06       3
7 2023-07       9
8 2023-08       6
9 2023-09       3
10 2023-12      2
# A tibble: 3 x 5
  cohorte mes_0 mes_1 mes_2 mes_3
  <chr>    <dbl> <dbl> <dbl> <dbl>
1 2023-01   100    39    38    30
2 2023-02   100    36    19    31
3 2023-03   100    34    38    28

```

Interpretación: 80 de los 198 clientes activos hicieron su primera compra en enero, y las cohortes se achican mes a mes porque con compras aleatorias casi todos los clientes aparecen pronto (solo 2 debutan en diciembre). En las cohortes del primer trimestre, entre 34 % y 39 % vuelve a comprar al mes siguiente y alrededor de 30 % al tercer mes, sin una caída clara. Ojo al interpretar: las cohortes tardías tienen muy pocos clientes (3 en junio o septiembre) y sus porcentajes serían puro ruido; por eso mostramos solo las del primer trimestre. En una empresa real las cohortes se definen con fecha de alta confiable y varios años de historia.

Solución del ejercicio 9.5

La regla de Tukey marca como atípico lo que queda fuera de $[Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$. La segunda regla compara el precio de cada transacción con la mediana del mismo producto.

Enunciado en la página 604

```

1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4
5 # Regla (a): Tukey sobre el total de la transacción
6 q <- quantile(transacciones$total_transaccion, c(0.25, 0.75))
7 iqr <- unname(q[2] - q[1])
8 limites <- c(inferior = q[[1]] - 1.5 * iqr, superior = q[[2]] + 1.5 * iqr)
9 round(limites, 2)
10 sum(transacciones$total_transaccion < limites["inferior"] |
11      transacciones$total_transaccion > limites["superior"])
12
13 # Regla (b): precio muy alejado de la mediana del mismo producto
14 revision <- transacciones %>%
15   group_by(producto_id) %>%
16   mutate(precio_mediano = median(precio_venta),
17          razon = precio_venta / precio_mediano) %>%
18   ungroup() %>%
19   mutate(atipico_precio = razon > 3 | razon < 1 / 3)

```

```

20
21 sum(revision$atipico_precio)
22
23 # Lista para Auditoría: los casos más extremos primero
24 revision %>%
25   filter(atipico_precio) %>%
26   mutate(distancia = abs(log(razon))) %>%
27   arrange(desc(distancia)) %>%
28   transmute(id = transaccion_id, tienda = tienda_id,
29             vendedor = vendedor_id, producto = producto_id,
30             precio = precio_venta, mediana = precio_mediano,
31             razon = round(razon, 2)) %>%
32   head(6)
33
34 # ¿Se concentran en algún vendedor?
35 revision %>% filter(atipico_precio) %>%
36   count(vendedor_id, sort = TRUE) %>% head(3)

```

```

inferior superior
-1365.16 3184.89
[1] 0
[1] 148
# A tibble: 6 x 7
  id tienda vendedor producto precio mediana razon
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 106     9      12     45  20.1  434.  0.05
2 930     6       6     43  24.2  393.  0.06
3 185     3      22     16  22.7  299.  0.08
4 909     1      20     42  22.6  293.  0.08
5 745     4      22     24  20.2  260.  0.08
6 925     2      19      7  26.4  335.  0.08
# A tibble: 3 x 2
  vendedor_id    n
  <dbl> <int>
1      23     9
2      24     9
3       5     8

```

Interpretación: la regla de Tukey no marca *ninguna* transacción: el total máximo (\$2,992) está por debajo del límite superior (\$3,185) y el límite inferior es negativo. Esto pasa porque la cantidad (1 a 5) y el precio (20 a 600) están acotados. En cambio, comparar el precio con el de su propio producto marca 148 transacciones (15%); las más extremas se cobraron a entre 5% y 8% del precio mediano del producto, lo cual confirma el hallazgo de la auditoría: el problema no son los montos, sino precios inconsistentes para un mismo producto. Los vendedores 23 y 24 concentran 9 casos cada uno; antes de sacar conclusiones sobre personas, compáralo con su número total de transacciones (43 y 44), porque con 148 casos repartidos en 25 vendedores, 9 por vendedor está dentro de lo esperable.

Solución del ejercicio 9.6

Primero medimos si los descuentos se asocian con más unidades en `ventas_retail.csv`; después simulamos diciembre con un descuento de 10% y un aumento aleatorio de unidades.

Enunciado en la página 604

```

1 library(tidyverse)
2 ventas_retail <- read_csv("datasets/ventas_retail.csv",
3                           show_col_types = FALSE)
4 transacciones <- read_csv("datasets/transacciones.csv",
5                           show_col_types = FALSE)
6
7 # 1. ¿Con descuento se venden más unidades?
8 ventas_retail %>%
9   mutate(con_descuento = descuento_pct > 0) %>%
10  group_by(con_descuento) %>%
11  summarise(ventas = n(), cantidad_promedio = round(mean(cantidad), 2),
12            .groups = "drop")
13 t.test(cantidad ~ descuento_pct > 0, data = ventas_retail)$p.value
14
15 # 2. Simulación de diciembre con 10% de descuento
16 ingreso_base <- transacciones %>%
17   filter(month(fecha) == 12) %>%
18   summarise(total = sum(total_transaccion)) %>%
19   pull(total)
20
21 set.seed(123)
22 simulacion <- tibble(aumento_unidades = runif(1000, 0, 0.15)) %>%
23   mutate(ingreso = ingreso_base * (1 + aumento_unidades) * 0.90,
24          diferencia = ingreso - ingreso_base)
25
26 simulacion %>%
27   summarise(ingreso_base = round(ingreso_base),
28             ingreso_promedio = round(mean(ingreso)),
29             pct_pierde = round(100 * mean(diferencia < 0), 1),
30             peor_caso = round(min(diferencia)),
31             mejor_caso = round(max(diferencia)))
32
33 # 3. Aumento de unidades necesario para no perder: (1 + x) * 0.9 = 1
34 round(100 * (1 / 0.90 - 1), 1)

```

```

# A tibble: 2 x 3
  con_descuento ventas cantidad_promedio
  <lgl>          <int>          <dbl>
1 FALSE          188          5.61
2 TRUE           177          5.67
[1] 0.8404565

```

```
# A tibble: 1 x 5
  ingreso_base ingreso_promedio pct_pierde peor_caso mejor_caso
      <dbl>         <dbl>      <dbl>    <dbl>    <dbl>
1      96394         93225        74.5    -9633     3366
[1] 11.1
```

Interpretación: en `ventas_retail.csv` las ventas con descuento no tienen más unidades (5.67 contra 5.61, $p = 0.84$): no hay evidencia de que los descuentos muevan volumen. Con un descuento de 10 % se necesita vender 11.1 % más unidades solo para igualar el ingreso. Si el aumento está entre 0 % y 15 %, la campaña pierde ingreso en 74.5 % de las simulaciones; en promedio el ingreso de diciembre bajaría de \$96,394 a \$93,225, con una pérdida de hasta \$9,633 y una ganancia máxima de apenas \$3,366. Y esto es ingreso: con márgenes estimados de 46 %, el efecto en utilidad sería peor. Recomendación: no lanzar un descuento general; probarlo primero en unas tiendas con un grupo de control.

Solución del ejercicio 9.7

Las **funciones de ventana** (`min_rank()`, `lag()`, `cumsum()`) calculan un valor por fila usando otras filas del mismo grupo. Primero completamos la tabla vendedor-mes con `complete()`, para que un mes sin ventas cuente como cero.

Enunciado en la página 604

```
1 library(tidyverse)
2 transacciones <- read_csv("datasets/transacciones.csv",
3                           show_col_types = FALSE)
4
5 ranking <- transacciones %>%
6   mutate(mes = month(fecha)) %>%
7   count(vendedor_id, mes, wt = total_transaccion, name = "ventas") %>%
8   complete(vendedor_id, mes = 1:12, fill = list(ventas = 0)) %>%
9   group_by(mes) %>%
10  mutate(lugar = min_rank(desc(ventas))) %>%           # ranking del mes
11  group_by(vendedor_id) %>%
12  arrange(mes, .by_group = TRUE) %>%
13  mutate(acumulado = cumsum(ventas),                  # venta acumulada
14         cambio_lugar = lag(lugar) - lugar) %>%      # + = subió
15  ungroup()
16
17 # Vendedores con más meses en el top 5
18 ranking %>%
19   group_by(vendedor_id) %>%
20   summarise(meses_top5 = sum(lugar <= 5),
21            ventas_anuales = round(max(acumulado)), .groups = "drop") %>%
22   arrange(desc(meses_top5), desc(ventas_anuales)) %>%
23   head(5)
24
25 # Mayor subida de un mes a otro
```

```

26 ranking %>%
27   slice_max(cambio_lugar, n = 1) %>%
28   select(vendedor_id, mes, lugar, cambio_lugar, ventas) %>%
29   mutate(ventas = round(ventas))

```

```

# A tibble: 5 x 3
  vendedor_id meses_top5 ventas_anuales
      <dbl>      <int>      <dbl>
1         7         6      50470
2        11         4      50017
3         3         4      48784
4         6         4      47946
5        15         4      42709
# A tibble: 3 x 5
  vendedor_id  mes lugar cambio_lugar ventas
      <dbl> <dbl> <int>      <int> <dbl>
1         1     10     3         21  5357
2         2      5     3         21  8119
3        17      3     2         21  9971

```

Interpretación: el vendedor 7, que fue el primero del año (\$50,470), estuvo en el top 5 en 6 de los 12 meses; ningún otro supera 4 meses. La mayor subida fue de 21 lugares y la lograron tres vendedores (el 1 en octubre, el 2 en mayo y el 17 en marzo): pasar del lugar 23 o 24 al top 3 de un mes a otro indica que el ranking mensual es muy volátil con pocas transacciones por vendedor (unas 3 o 4 al mes). Para evaluar desempeño conviene un ranking trimestral o un promedio móvil, no el mes aislado.

Solución del ejercicio 9.8

La función arma las tres hojas como una lista con nombre y la escribe con `write_xlsx()` de **writexl** (cada elemento de la lista es una hoja). `walk()` la aplica a las 10 tiendas.

Enunciado en la página 604

```

1 library(tidyverse)
2 library(writexl)
3 transacciones <- read_csv("datasets/transacciones.csv",
4                           show_col_types = FALSE)
5 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
6 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
7
8 dir.create("resultados/gerentes", recursive = TRUE, showWarnings = FALSE)
9
10 # KPIs de un conjunto de transacciones
11 kpis_de <- function(datos) {
12   datos %>% summarise(ventas = round(sum(total_transaccion)),
13                         transacciones = n(),

```

```

14         ticket = round(mean(total_transaccion), 2),
15         clientes = n_distinct(cliente_id))
16     }
17
18     # Promedio de la cadena por tienda (para comparar)
19     cadena <- transacciones %>%
20       group_by(tienda_id) %>%
21       group_modify(~ kpis_de(.x)) %>%
22       ungroup() %>%
23       summarise(across(-tienda_id, mean)) %>%
24       mutate(ventas = round(ventas), ticket = round(ticket, 2),
25             clientes = round(clientes, 1))
26
27     paquete_gerente <- function(id) {
28       info <- filter(tiendas, tienda_id == id)
29       datos <- filter(transacciones, tienda_id == id)
30
31       resumen <- bind_rows(
32         kpis_de(datos) %>% mutate(nivel = info$nombre_tienda),
33         cadena %>% mutate(nivel = "Promedio cadena")) %>%
34       relocate(nivel)
35
36       mensual <- datos %>%
37         group_by(mes) %>%
38         summarise(ventas = round(sum(total_transaccion)),
39                 transacciones = n(), .groups = "drop") %>%
40         mutate(crecimiento_pct = round(100 * (ventas / lag(ventas) - 1), 1))
41
42       top <- datos %>%
43         group_by(producto_id) %>%
44         summarise(unidades = sum(cantidad),
45                 ventas = round(sum(total_transaccion), .groups = "drop") %>%
46         left_join(select(productos, producto_id, nombre_producto, categoria),
47                   by = "producto_id") %>%
48         slice_max(ventas, n = 10, with_ties = FALSE)
49
50       archivo <- file.path("resultados/gerentes",
51                           str_c("tienda_", sprintf("%02d", id), "_",
52                               str_replace_all(info$nombre_tienda, " ", "_"),
53                               ".xlsx"))
54       write_xlsx(list(Resumen = resumen, Mensual = mensual,
55                     `Top productos` = top), archivo)
56     }
57
58     walk(tiendas$tienda_id, paquete_gerente)
59
60     archivos <- list.files("resultados/gerentes", pattern = "xlsx$")
61     length(archivos)
62     head(archivos, 3)

```



```

63
64 # Comprobación: leer la hoja Resumen de la tienda 8
65 readxl::read_excel(file.path("resultados/gerentes", archivos[8]),
66                     sheet = "Resumen")

```

```

[1] 10
[1] "tienda_01_Sucursal_Centro.xlsx" "tienda_02_Sucursal_Norte.xlsx"
[3] "tienda_03_Sucursal_Sur.xlsx"
# A tibble: 2 x 5
  nivel      ventas transacciones ticket clientes
  <chr>      <dbl>          <dbl>  <dbl>    <dbl>
1 Sucursal Outlet 114377          106  1079.     82
2 Promedio cadena  95999          100   959.    79.5

```

Interpretación: se generaron los 10 archivos, uno por sucursal, con un nombre que se ordena solo, porque empieza con *tienda_01*, *tienda_02*, etc. La hoja *Resumen* de la Sucursal Outlet muestra que vendió \$114,377 contra un promedio de \$95,999 por tienda, con un ticket de \$1,079 contra \$959. Ese formato de “mi tienda contra la cadena” es justo lo que un gerente necesita. Para automatizarlo cada mes, combina esta función con la programación de tareas del Módulo 8. Si quieres dar formato (colores, anchos de columna) usa **openxlsx**, que no está instalado en este entorno.

Referencia rápida

Este apéndice es tu **chuleta** (en inglés, *cheat sheet*): una colección de tablas para consultar en segundos la función, el atajo o la fórmula que necesitas mientras trabajas. No sustituye a los módulos (ahí se explica el porqué de cada cosa); está pensado para tenerlo abierto junto a RStudio cuando ya sabes *qué* quieres hacer y solo necesitas recordar *cómo*.

Cada tabla sigue la misma lógica: **función o sintaxis** → **qué hace** → **ejemplo corto**. Los ejemplos usan las columnas reales de los datos del curso (`ventas_retail.csv`, `productos.csv`, `tiendas.csv`, `clientes.csv...`) para que puedas copiarlos casi sin cambios.

Tabla B.1. Mapa del apéndice: dónde buscar cada cosa.

| Sección | Úsala cuando... |
|-------------------------------|---|
| B.1 Atajos de RStudio | quieras trabajar más rápido con el teclado. |
| B.2 R base esencial | dudes de un operador, un tipo de dato, un NA o un bucle. |
| B.3 Importar y exportar | tengas que leer o guardar CSV, Excel, RDS o bases de datos. |
| B.4 dplyr | necesites filtrar, agrupar, resumir, clasificar o unir tablas. |
| B.5 tidyr, lubridate, stringr | haya que reestructurar tablas, trabajar fechas o limpiar texto. |
| B.6 ggplot2 | elijas el tipo de gráfico o el formato de los ejes. |
| B.7 SQL, dplyr y Excel | traduzcas una consulta o una fórmula de Excel a R. |
| B.8 Estadística y KPIs | elijas una prueba o calcules un indicador de negocio. |
| B.9 Machine learning | elijas algoritmo y métrica para un problema de negocio. |
| B.10 Shiny y R Markdown | armes un dashboard o un reporte automático. |
| B.11 Errores frecuentes | R te muestre un mensaje de error que no entiendes. |
| B.12 Glosario | te encuentres un término de BI que no conoces. |
| B.13 Recursos | quieras seguir aprendiendo. |

Los pocos bloques de código de este apéndice se ejecutan de corrido, como un solo script, desde la carpeta del curso. Este es el arranque que todos asumen:

Arranque de cualquier análisis del curso

```

1 # Paquetes y datos que usan los ejemplos de este apéndice
2 library(tidyverse) # dplyr, ggplot2, tidyr, readr, stringr, lubridate
3 library(scales)    # formatos de negocio: dólar, coma, porcentaje
4
5 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
6 productos <- read_csv("datasets/productos.csv", show_col_types = FALSE)
7 tiendas <- read_csv("datasets/tiendas.csv", show_col_types = FALSE)
```

B.1 Atajos de teclado de RStudio

En Mac, **Ctrl** suele cambiarse por **Cmd** y **Alt** por **Option**. La lista completa está en **Help > Keyboard Shortcuts Help** (o con **Alt + Shift + K**), y puedes cambiar cualquier atajo en **Tools > Modify Keyboard Shortcuts**.

Tabla B.2. Atajos de teclado más útiles de RStudio.

| Acción | Windows / Linux | Mac |
|--|--|--|
| Ejecutar código | | |
| Ejecutar línea o selección | Ctrl + Enter | Cmd + Return |
| Ejecutar todo el script (con eco) | Ctrl + Shift + Enter | Cmd + Shift + Return |
| Ejecutar todo el script (source) | Ctrl + Shift + S | Cmd + Shift + S |
| Ejecutar desde el inicio hasta la línea actual | Ctrl + Alt + B | Cmd + Option + B |
| Detener la ejecución | Esc | Esc |
| Reiniciar R (sesión limpia) | Ctrl + Shift + F10 | Cmd + Shift + O |
| Escribir código | | |
| Insertar <- | Alt + - | Option + - |
| Insertar pipe %>% (o >) | Ctrl + Shift + M | Cmd + Shift + M |
| Autocompletar | Tab | Tab |
| Comentar / descomentar líneas | Ctrl + Shift + C | Cmd + Shift + C |
| Reindentar líneas | Ctrl + I | Cmd + I |
| Reformatear código | Ctrl + Shift + A | Cmd + Shift + A |
| Insertar sección # Título -- | Ctrl + Shift + R | Cmd + Shift + R |
| Mover línea arriba / abajo | Alt + ↑ / ↓ | Option + ↑ / ↓ |
| Duplicar línea hacia abajo | Alt + Shift + ↓ | Cmd + Option + ↓ |
| Borrar línea | Ctrl + D | Cmd + D |
| Cursor múltiple (varias líneas) | Ctrl + Alt + ↑ / ↓ | Ctrl + Option + ↑ / ↓ |
| Renombrar variable en su ámbito | Ctrl + Alt + Shift + M | Cmd + Option + Shift + M |
| Convertir selección en función | Ctrl + Alt + X | Cmd + Option + X |
| Navegar y buscar | | |
| Ir al editor / a la consola | Ctrl + 1 / Ctrl + 2 | Ctrl + 1 / Ctrl + 2 |
| Buscar y reemplazar en el archivo | Ctrl + F | Cmd + F |
| Buscar en todos los archivos | Ctrl + Shift + F | Cmd + Shift + F |
| Ir a archivo o función | Ctrl + . | Ctrl + . |
| Ver índice del documento (secciones) | Ctrl + Shift + O | Cmd + Shift + O |
| Ir a la definición de una función | F2 | F2 |
| Ayuda de la función bajo el cursor | F1 | F1 |
| Paleta de comandos | Ctrl + Shift + P | Cmd + Shift + P |
| Plegar todo el código | Alt + O | Cmd + Option + O |
| Consola y archivos | | |
| Comando anterior en la consola | ↑ | ↑ |

| Acción | Windows / Linux | Mac |
|---------------------------------------|---|---|
| Historial filtrado por lo escrito | Ctrl + ↑ | Cmd + ↑ |
| Limpiar la consola | Ctrl + L | Ctrl + L |
| Nuevo script | Ctrl + Shift + N | Cmd + Shift + N |
| Guardar | Ctrl + S | Cmd + S |
| R Markdown | | |
| Insertar bloque (<i>chunk</i>) de R | Ctrl + Alt + I | Cmd + Option + I |
| Ejecutar el bloque actual | Ctrl + Shift + Enter | Cmd + Shift + Return |
| Ejecutar todos los bloques anteriores | Ctrl + Alt + P | Cmd + Option + P |
| Generar el documento (<i>Knit</i>) | Ctrl + Shift + K | Cmd + Shift + K |

Consejo

Si solo vas a memorizar cinco atajos, que sean estos: ejecutar línea (**Ctrl** + **Enter**), asignación (**Alt** + **=**), pipe (**Ctrl** + **Shift** + **M**), comentar (**Ctrl** + **Shift** + **C**) y reiniciar R (**Ctrl** + **Shift** + **F10**). Reiniciar a menudo es la mejor forma de comprobar que tu script funciona desde cero.

B.2 R base esencial

B.2.1 Operadores y asignación

Tabla B.3. Operadores de R.

| Sintaxis | Qué hace | Ejemplo |
|--|---|---|
| <code><-</code> | Asigna un valor a un nombre (estilo recomendado). | <code>meta <- 50000</code> |
| <code>=</code> | Asigna argumentos dentro de funciones. | <code>round(x, digits = 2)</code> |
| <code>+ - * /</code> | Suma, resta, multiplicación, división. | <code>total * 0.16</code> |
| <code>^</code> | Potencia. | <code>(1 + 0.05)^12</code> |
| <code>/% % y %%</code> | División entera y residuo. | <code>17 %/% 5 da 3; 17 %% 5 da 2</code> |
| <code>== != > >= < <=</code> | Comparaciones (devuelven TRUE/FALSE). | <code>tienda_id == 3</code> |
| <code>& !</code> | Y, O, NO (vectorizados, elemento a elemento). | <code>total > 1000 & descuento_pct == 0</code> |
| <code>&& </code> | Y, O de un solo valor (para if). | <code>if (n > 0 && !is.na(x))</code> |
| <code>%in%</code> | ¿Está el valor en el conjunto? | <code>ciudad %in% c("CDMX", "Puebla")</code> |
| <code>:</code> | Secuencia de enteros. | <code>1:12</code> |
| <code>%>% y ></code> | Pipe: pasa el resultado a la siguiente función. | <code>ventas %>% head()</code> |
| <code>::</code> | Usa una función de un paquete sin cargarlo. | <code>readxl::read_excel(...)</code> |
| <code>#</code> | Comentario (R lo ignora). | <code># Calcular el IVA</code> |
| <code>? y ??</code> | Ayuda de una función / búsqueda en la ayuda. | <code>?mutate, ??regression</code> |

B.2.2 Tipos de datos y conversión

Tabla B.4. Tipos de datos, cómo comprobarlos y cómo convertirlos.

| Tipo (clase) | Para qué sirve | Comprobar / convertir |
|---|---|--|
| <code>numeric</code> (double) | Montos, precios, promedios. | <code>is.numeric(x)</code> , <code>as.numeric("12.5")</code> da 12.5 |
| <code>integer</code> | Conteos enteros (5L). | <code>as.integer(3.9)</code> da 3 (trunca, no redondea) |
| <code>character</code> | Texto: nombres, ciudades, IDs con ceros. | <code>is.character(x)</code> , <code>as.character(101)</code> |
| <code>logical</code> | <code>TRUE/FALSE</code> : banderas como <code>es_premium</code> . | <code>as.logical("TRUE")</code> ; <code>sum(es_premium)</code> cuenta los TRUE |
| <code>factor</code> | Categorías con orden o niveles fijos. | <code>factor(segmento, levels = c("Joven", "Adulto"))</code> |
| <code>Date</code> | Fechas sin hora. | <code>as.Date("2023-03-15")</code> ; <code>as.Date("15/03/2023", format = "%d/%m/%Y")</code> |
| <code>POSIXct</code> | Fecha con hora. | <code>as.POSIXct("2023-03-15 14:30")</code> , <code>ymd_hms()</code> |
| <code>data.frame</code> / <code>tibble</code> | Tabla: columnas de igual longitud. | <code>tibble(tienda = 1:3, ventas = c(10, 20, 30))</code> |
| <code>list</code> | Contenedor de objetos distintos. | <code>list(modelo = m, fecha = Sys.Date())</code> |
| Funciones para inspeccionar | | |
| <code>class()</code> , <code>typeof()</code> | Clase y tipo interno de un objeto. | <code>class(ventas\$fecha)</code> da "Date" |
| <code>str()</code> , <code>glimpse()</code> | Estructura: columnas, tipos y primeros valores. | <code>glimpse(ventas)</code> |
| <code>parse_number()</code> | Extrae el número de un texto sucio (<code>readr</code>). | <code>parse_number("\$1,250.50")</code> da 1250.5 |
| <code>as.numeric()</code>
<code>as.character(f)</code> | Convierte un factor numérico a número real. | Sin <code>as.character()</code> obtendrías los códigos internos 1, 2, 3... |

B.2.3 Vectores, indexación y valores faltantes

Tabla B.5. Crear vectores, seleccionar elementos y manejar NA.

| Sintaxis | Qué hace | Ejemplo |
|---|---|---|
| <code>c()</code> | Combina valores en un vector. | <code>precios <- c(120, 85, 240)</code> |
| <code>seq()</code> , <code>rep()</code> | Secuencias y repeticiones. | <code>seq(0, 100, by = 25)</code> ; <code>rep("A", 3)</code> |
| <code>length()</code> | Número de elementos. | <code>length(precios)</code> |
| <code>x[i]</code> | Elemento(s) por posición. | <code>precios[2]</code> , <code>precios[c(1, 3)]</code> ,
<code>precios[-1]</code> |
| <code>x[condición]</code> | Elementos que cumplen una condición. | <code>precios[precios > 100]</code> |
| <code>which()</code> , <code>which.max()</code> | Posiciones que cumplen / posición del máximo. | <code>which(precios > 100)</code> |
| <code>df[filas, columnas]</code> | Indexar un data frame. | <code>ventas[1:5, c("fecha", "total")]</code> |
| <code>df\$columna</code> | Extraer una columna como vector. | <code>mean(ventas\$total)</code> |
| <code>lista[[1]]</code> , <code>lista\$nombre</code> | Extraer un elemento de una lista. | <code>modelo\$coefficients</code> |
| <code>head()</code> , <code>tail()</code> | Primeras / últimas filas. | <code>head(ventas, 10)</code> |
| <code>sort()</code> , <code>order()</code> , <code>rev()</code> | Ordenar valores / posiciones de orden / invertir. | <code>sort(precios, decreasing = TRUE)</code> |
| <code>unique()</code> , <code>duplicated()</code> | Valores únicos / ¿está repetido? | <code>sum(duplicated(clientes\$email))</code> |
| Valores faltantes (NA) | | |

| Sintaxis | Qué hace | Ejemplo |
|---|---|---|
| <code>NA</code> | Dato faltante; “contagia” cálculos. | <code>mean(c(1, NA))</code> da <code>NA</code> |
| <code>is.na()</code> | ¿Es faltante? (nunca uses <code>x == NA</code>). | <code>sum(is.na(ventas\$total))</code> |
| <code>na.rm = TRUE</code> | Ignora los NA en funciones de resumen. | <code>sum(total, na.rm = TRUE)</code> |
| <code>complete.cases()</code> | Filas sin ningún NA. | <code>ventas[complete.cases(ventas),]</code> |
| <code>colSums(is.na(df))</code> | NA por columna (diagnóstico rápido). | <code>colSums(is.na(clientes))</code> |
| <code>NULL</code> , <code>NaN</code> , <code>Inf</code> | Vacío, “no es número” (0/0), infinito (1/0). | Aparecen al dividir entre cero: revisa denominadores. |

El comportamiento de `NA` es la fuente número uno de sorpresas, así que vale la pena verlo en acción:

```

1 precios <- c(120, 85, NA, 240, 99)      # vector con un dato faltante
2 mean(precios)                          # el NA "contagia" el resultado
3 mean(precios, na.rm = TRUE)            # ignora los NA
4 sum(is.na(precios))                    # ¿cuántos faltan?
5 precios[precios > 100 & !is.na(precios)] # filtrar sin arrastrar NA

```

```

[1] NA
[1] 136
[1] 1
[1] 120 240

```

El promedio sin `na.rm = TRUE` es `NA`; con él, R promedia los cuatro valores conocidos (136). En un reporte, un `NA` en el total es señal de que falta tratar datos faltantes, no de que las ventas sean cero.

B.2.4 Funciones de resumen

Tabla B.6. Funciones de resumen numérico y de conteo.

| Función | Qué hace | Ejemplo |
|--|---|---|
| <code>sum()</code> , <code>mean()</code> , <code>median()</code> | Suma, promedio, mediana. | <code>median(ventas\$total)</code> |
| <code>min()</code> , <code>max()</code> , <code>range()</code> | Mínimo, máximo, ambos. | <code>range(ventas\$fecha)</code> |
| <code>sd()</code> , <code>var()</code> | Desviación estándar, varianza. | <code>sd(ventas\$total)</code> |
| <code>quantile()</code> , <code>IQR()</code> | Percentiles y rango intercuartílico. | <code>quantile(total, c(0.25, 0.9))</code> |
| <code>summary()</code> | Resumen de 6 números (o de un modelo). | <code>summary(ventas\$total)</code> |
| <code>cumsum()</code> , <code>cumprod()</code> | Acumulados. | <code>cumsum(ventas_mes)</code> |
| <code>diff()</code> | Diferencia entre valores consecutivos. | <code>diff(c(5, 8, 17))</code> da 3 9 |
| <code>round()</code> , <code>signif()</code> | Redondeo a decimales / cifras significativas. | <code>round(1234.567, 1)</code> |
| <code>ceiling()</code> , <code>floor()</code> | Redondeo hacia arriba / abajo. | <code>ceiling(4.1)</code> da 5 |
| <code>table()</code> , <code>prop.table()</code> | Frecuencias y proporciones. | <code>prop.table(table(clientes\$genero))</code> |
| <code>cor()</code> | Correlación entre variables numéricas. | <code>cor(ventas\$cantidad, ventas\$total)</code> |
| <code>skimr::skim()</code> | Resumen completo de un data frame. | <code>skimr::skim(productos)</code> |

B.2.5 Texto y fechas en R base

Tabla B.7. Funciones base para texto y fechas (sus equivalentes tidyverse están en la sección B.5).

| Función | Qué hace | Ejemplo → resultado |
|---|--------------------------------------|--|
| <code>paste()</code> , <code>paste0()</code> | Une textos (con / sin espacio). | <code>paste0("Q", 1:4) → "Q1" "Q2" "Q3" "Q4"</code> |
| <code>nchar()</code> | Número de caracteres. | <code>nchar("Mérida") → 6</code> |
| <code>toupper()</code> , <code>tolower()</code> | Mayúsculas / minúsculas. | <code>toupper("puebla") → "PUEBLA"</code> |
| <code>substr()</code> | Recorta por posición. | <code>substr("Guadalajara", 1, 3) → "Gua"</code> |
| <code>trimws()</code> | Quita espacios al inicio y final. | <code>trimws(" hola ") → "hola"</code> |
| <code>grepl()</code> | ¿Contiene el patrón? | <code>grepl("Tarjeta", metodo_pago)</code> |
| <code>gsub()</code> | Reemplaza todas las coincidencias. | <code>gsub(",", "", "1,250") → "1250"</code> |
| <code>strsplit()</code> | Divide un texto. | <code>strsplit("a,b,c", ",")</code> |
| <code>sprintf()</code> | Formato tipo C. | <code>sprintf("%.2f", 3.14159) → "3.14"</code> |
| <code>format()</code> | Números con separador de miles. | <code>format(1234567.891, big.mark = ",", nsmall = 2) → "1,234,567.89"</code> |
| <code>Sys.Date()</code> , <code>Sys.time()</code> | Fecha / fecha y hora actuales. | <code>Sys.Date() - as.Date("2023-01-01")</code> |
| <code>format(fecha, "...")</code> | Fecha como texto. | <code>format(as.Date("2023-03-15"), "%d de %B de %Y") → "15 de marzo de 2023"</code> |
| <code>weekdays()</code> , <code>months()</code> | Nombre del día / mes (según idioma). | <code>weekdays(as.Date("2023-03-15")) → "miércoles"</code> |
| <code>difftime()</code> | Diferencia entre fechas. | <code>difftime(fin, inicio, units = "days")</code> |
| <code>seq()</code> con fechas | Secuencia de fechas. | <code>seq(as.Date("2023-01-01"), by = "month", length.out = 3)</code> |

Códigos de formato de fecha más usados: **%Y** año con 4 dígitos, **%y** con 2, **%m** mes (01–12), **%B** nombre del mes, **%b** mes abreviado, **%d** día, **%A** día de la semana, **%H: %M** hora y minuto.

B.2.6 Control de flujo, funciones y familia apply

Tabla B.8. Estructuras de control, funciones propias e iteración.

| Sintaxis | Qué hace | Ejemplo |
|---|---|--|
| <code>if (cond) {} else {}</code> | Decide con UN valor lógico. | <code>if (ventas > meta) "Cumplió" else "No cumplió"</code> |
| <code>ifelse(cond, a, b)</code> | Versión vectorizada (una respuesta por elemento). | <code>ifelse(total > 1000, "alta", "normal")</code> |
| <code>for (i in x) {}</code> | Repite para cada elemento. | <code>for (t in 1:10) print(t)</code> |
| <code>while (cond) {}</code> | Repite mientras se cumpla. | <code>while (stock > 0) stock <- stock - 1</code> |
| <code>switch()</code> | Elige según un valor. | <code>switch("b", a = "uno", b = "dos") → "dos"</code> |
| <code>function(args) {}</code>
<code>return()</code> | Crea una función propia.
Devuelve un valor y sale de la función. | <code>iva <- function(x, tasa = 0.16) x * tasa</code>
<code>if (is.na(x)) return(0)</code> |
| <code>stop()</code> , <code>warning()</code>
<code>tryCatch()</code> | Lanza un error / una advertencia.
Captura errores sin detener el script. | <code>if (n == 0) stop("Sin datos")</code>
<code>tryCatch(log(-1), warning = function(w) NA)</code> |

| Sintaxis | Qué hace | Ejemplo |
|----------------------------------|---|---|
| <code>sapply(x, f)</code> | Aplica <code>f</code> a cada elemento; simplifica a vector. | <code>sapply(ventas[, c("cantidad", "total")], mean)</code> |
| <code>lapply(x, f)</code> | Igual, pero devuelve una lista. | <code>lapply(archivos, read_csv)</code> |
| <code>vapply(x, f, tipo)</code> | Como <code>sapply</code> con tipo de salida garantizado. | <code>vapply(lista, mean, numeric(1))</code> |
| <code>apply(m, 1 o 2, f)</code> | Por filas (1) o columnas (2) de una matriz. | <code>apply(matriz, 2, sum)</code> |
| <code>tapply(x, grupo, f)</code> | Resume <code>x</code> por grupo. | <code>tapply(ventas\$total, ventas\$tienda_id, sum)</code> |
| <code>purrr::map_dbl()</code> | Versión tidyverse de <code>sapply</code> para números. | <code>map_dbl(lista, mean)</code> |

Una función típica de negocio combina casi todo lo anterior:

```

1  # Clasifica un monto de venta; umbral tiene un valor por omisión
2  clasificar_venta <- function(monto, umbral = 1000) {
3    if (is.na(monto)) return("sin dato")    # protegerse de NA
4    if (monto >= umbral) "alta" else "normal"
5  }
6  clasificar_venta(1500)                    # usa umbral = 1000
7  clasificar_venta(300, umbral = 200)      # cambia el umbral
8  sapply(c(1500, 300, NA), clasificar_venta) # aplicar a cada elemento
9  for (trimestre in c("Q1", "Q2")) {      # bucle sencillo
10   cat("Procesando", trimestre, "\n")
11 }

```

```

[1] "alta"
[1] "alta"
[1] "alta"      "normal"    "sin dato"
Procesando Q1
Procesando Q2

```

B.2.7 Archivos, carpetas y sesión

Tabla B.9. Trabajar con archivos, carpetas y la sesión de R.

| Función | Qué hace | Ejemplo |
|----------------------------|---|--|
| <code>getwd()</code> | Muestra la carpeta de trabajo. | <code>getwd()</code> |
| <code>setwd()</code> | Cambia la carpeta (mejor usa proyectos de RStudio). | <code>setwd("C:/Users/Ana/curso")</code> |
| <code>list.files()</code> | Lista archivos. | <code>list.files("datasets", pattern = "csv\$")</code> |
| <code>file.exists()</code> | ¿Existe el archivo? | <code>file.exists("datasets/tiendas.csv")</code> |
| <code>file.path()</code> | Arma rutas portables. | <code>file.path("resultados", "reporte.csv")</code> |

| Función | Qué hace | Ejemplo |
|---------------------------------------|--|---|
| <code>dir.create()</code> | Crea una carpeta. | <code>dir.create("resultados", showWarnings = FALSE)</code> |
| <code>source()</code> | Ejecuta otro script. | <code>source("datasets/generar_datasets.R")</code> |
| <code>ls()</code> , <code>rm()</code> | Lista / borra objetos del entorno. | <code>rm(list = ls())</code> |
| <code>install.packages()</code> | Instala un paquete (una sola vez). | <code>install.packages("writexl")</code> |
| <code>library()</code> | Carga un paquete (en cada sesión). | <code>library(dplyr)</code> |
| <code>sessionInfo()</code> | Versiones de R y paquetes (para reportar problemas). | <code>sessionInfo()</code> |
| <code>Sys.setenv()</code> | Variables de entorno. | <code>Sys.setenv(LANGUAGE = "en")</code> (mensajes en inglés) |

B.3 Importar y exportar datos

Tabla B.10. Leer y escribir archivos: CSV, Excel, RDS y otros formatos.

| Función (paquete) | Qué hace | Ejemplo |
|---|---|--|
| <code>read_csv()</code> (readr) | Lee CSV separado por comas. | <code>read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)</code> |
| <code>read_csv2()</code> (readr) | CSV con ; y coma decimal (Excel en español). | <code>read_csv2("datos/ventas_excel.csv")</code> |
| <code>read_delim()</code> (readr)
Argumentos útiles | Cualquier separador.
Controlar la lectura. | <code>read_delim("datos.txt", delim = " ")</code>
<code>locale = locale(encoding = "latin1"), skip = 2, na = c("", "NA", "-"), col_types = cols(tienda_id = col_character())</code> |
| <code>write_csv()</code> (readr) | Escribe CSV (UTF-8). | <code>write_csv(resumen, "resultados/resumen.csv")</code> |
| <code>write_excel_csv()</code> | CSV que Excel abre con acentos correctos. | <code>write_excel_csv(resumen, "resultados/resumen.csv")</code> |
| <code>read.csv()</code> (base) | Lector base (más lento; data.frame). | <code>read.csv("datasets/tiendas.csv")</code> |
| <code>excel_sheets()</code> (readxl) | Nombres de las hojas de un Excel. | <code>excel_sheets("datasets/datos_empresa.xlsx")</code> |
| <code>read_excel()</code> (readxl) | Lee una hoja de Excel. | <code>read_excel("datasets/datos_empresa.xlsx", sheet = "Productos")</code> |
| Argumentos útiles
<code>write_xlsx()</code> (writexl) | Rango, filas a saltar, tipos.
Escribe Excel; una lista = varias hojas. | <code>range = "A1:F20", skip = 3, na = "N/D"</code>
<code>write_xlsx(list(Ventas = v, Tiendas = t), "resultados/reporte.xlsx")</code> |
| <code>saveRDS()</code> , <code>readRDS()</code> | Guarda / lee UN objeto de R tal cual (tipos, factores, modelos). | <code>saveRDS(modelo, "resultados/modelo.rds")</code> |
| <code>save()</code> , <code>load()</code> | Varios objetos en un .RData. | <code>save(ventas, tiendas, file = "datos/base.RData")</code> |
| <code>haven::read_sav()</code> y similares | SPSS (<code>read_sav</code>), Stata (<code>read_dta</code>), SAS (<code>read_sas</code>). | <code>haven::read_sav("encuesta.sav")</code> |
| <code>jsonlite::fromJSON()</code> | JSON (APIs web). | <code>fromJSON("https://api.ejemplo.com/ventas")</code> |
| <code>data.table::fread()</code> | Lectura muy rápida de archivos grandes. | <code>fread("datos/transacciones_2023.csv")</code> |

Tabla B.11. Bases de datos con **DBI** (y **dbplyr**).

| Función | Qué hace | Ejemplo |
|--------------------------------------|---|--|
| <code>dbConnect()</code> | Abre la conexión. | <code>con <- dbConnect(RSQLite::SQLite(), "datos/empresa.sqlite")</code> |
| <code>dbListTables()</code> | Tablas disponibles. | <code>dbListTables(con)</code> |
| <code>dbListFields()</code> | Columnas de una tabla. | <code>dbListFields(con, "ventas")</code> |
| <code>dbWriteTable()</code> | Sube un data frame como tabla. | <code>dbWriteTable(con, "ventas", ventas, overwrite = TRUE)</code> |
| <code>dbReadTable()</code> | Descarga una tabla completa. | <code>dbReadTable(con, "tiendas")</code> |
| <code>dbGetQuery()</code> | Ejecuta un SELECT y trae el resultado. | <code>dbGetQuery(con, "SELECT * FROM ventas LIMIT 5")</code> |
| <code>dbExecute()</code> | Ejecuta INSERT, UPDATE, DELETE, CREATE. | <code>dbExecute(con, "DELETE FROM ventas WHERE total < 0")</code> |
| <code>params = list()</code> | Consulta parametrizada (evita inyección SQL). | <code>dbGetQuery(con, "SELECT * FROM ventas WHERE tienda_id = ?", params = list(3))</code> |
| <code>dbDisconnect()</code> | Cierra la conexión (¡siempre!). | <code>dbDisconnect(con)</code> |
| <code>tbl()</code> (dbplyr) | Referencia perezosa a una tabla remota. | <code>tbl(con, "ventas") %>% count(tienda_id)</code> |
| <code>show_query()</code> | Muestra el SQL que genera dplyr. | <code>consulta %>% show_query()</code> |
| <code>collect()</code> | Trae el resultado a R. | <code>consulta %>% collect()</code> |
| Otros motores | Solo cambia el controlador. | <code>RPostgres::Postgres()</code> , <code>RMariaDB::MariaDB()</code> , <code>odbc::odbc()</code> (SQL Server) |

▲ Cuidado

Nunca escribas contraseñas dentro del script. Guárdalas en el archivo `.Renviron` (por ejemplo, `BD_CLAVE=...`) y léelas con `Sys.getenv("BD_CLAVE")`. Así puedes compartir tu código sin exponer credenciales.

B.4 dplyr: manipulación de datos

Todos los ejemplos parten de `ventas %>% ...`: el resultado de cada verbo es una nueva tabla que puedes encadenar con el siguiente.

Tabla B.12. Verbos principales de **dplyr**.

| Verbo | Qué hace | Ejemplo (tras <code>ventas %>%</code>) |
|-------------------------|---|---|
| <code>filter()</code> | Conserva filas que cumplen condiciones (coma = Y).
Con <code>%in%</code> , <code> </code> o <code>between()</code> . | <code>filter(total > 1000, tienda_id == 3)</code>

<code>filter(dia_semana %in% c("sábado", "domingo")); filter(between(total, 500, 1000))</code> |
| <code>select()</code> | Elige, ordena o quita columnas.
Con ayudantes de selección. | <code>select(fecha, tienda_id, total); select(-mes)</code>
<code>select(starts_with("desc"), ends_with("_id"), contains("precio"), where(is.numeric), everything())</code> |
| <code>rename()</code> | Cambia nombres (nuevo = viejo). | <code>rename(importe = total)</code> |
| <code>relocate()</code> | Mueve columnas. | <code>relocate(total, .after = fecha)</code> |

| Verbo | Qué hace | Ejemplo (tras <code>ventas %>%</code>) |
|---|---|--|
| <code>mutate()</code> | Crea o modifica columnas. | <code>mutate(iva = total * 0.16, total_con_iva = total + iva)</code> |
| <code>arrange()</code> | Ordena filas (<code>desc()</code> = descendente). | <code>arrange(tienda_id, desc(total))</code> |
| <code>summarise()</code> | Resume muchas filas en una. | <code>summarise(ventas = sum(total), ticket = mean(total))</code> |
| <code>group_by()</code> | Agrupar para calcular por grupo. | <code>group_by(tienda_id) %>% summarise(ventas = sum(total), .groups = "drop")</code> |
| <code>.by =</code> | Agrupar solo para esa operación (<code>dplyr ≥ 1.1</code>). | <code>summarise(ventas = sum(total), .by = trimestre)</code> |
| <code>ungroup()</code> | Quita la agrupación. | <code>group_by(tienda_id) %>% mutate(p = total / sum(total)) %>% ungroup()</code> |
| <code>count()</code> | Cuenta filas por grupo. | <code>count(tienda_id, sort = TRUE)</code> |
| <code>add_count()</code> | Añade el conteo como columna. | <code>add_count(cliente_id, name = "compras_cliente")</code> |
| <code>distinct()</code> | Filas únicas. | <code>distinct(cliente_id, tienda_id)</code> |
| <code>pull()</code> | Extrae una columna como vector. | <code>pull(total)</code> |
| <code>slice_max()</code> / <code>slice_min()</code> | Filas con valores más altos / bajos. | <code>slice_max(total, n = 5)</code> |
| <code>slice_head()</code> / <code>slice_tail()</code> | Primeras / últimas filas. | <code>slice_head(n = 10)</code> |
| <code>slice_sample()</code> | Muestra aleatoria. | <code>slice_sample(n = 50)</code> (usa <code>set.seed()</code> antes) |
| <code>slice()</code> | Filas por posición. | <code>slice(1:3)</code> |
| <code>glimpse()</code> | Vistazo a columnas y tipos. | <code>glimpse()</code> |
| <code>bind_rows()</code> / <code>bind_cols()</code> | Apila tablas / pega columnas. | <code>bind_rows(ventas_2022, ventas_2023, .id = "origen")</code> |

Tabla B.13. Funciones auxiliares que se usan dentro de `mutate()`, `summarise()` y `filter()`.

| Función | Qué hace | Ejemplo |
|--|--|---|
| <code>n()</code> , <code>n_distinct()</code> | Número de filas / de valores distintos. | <code>summarise(tickets = n(), clientes = n_distinct(cliente_id))</code> |
| <code>if_else()</code> | Condición sí/no (estricta con tipos). | <code>mutate(tipo = if_else(descuento_pct > 0, "Con desc.", "Sin desc."))</code> |
| <code>case_when()</code> | Varias condiciones en orden; <code>.default</code> para el resto. | <code>mutate(nivel = case_when(total >= 3000 ~ "Alto", total >= 1000 ~ "Medio", .default = "Bajo"))</code> |
| <code>coalesce()</code> | Sustituye NA por otro valor. | <code>mutate(descuento_pct = coalesce(descuento_pct, 0))</code> |
| <code>na_if()</code> | Convierte un valor en NA. | <code>mutate(edad = na_if(edad, 0))</code> |
| <code>across()</code> | Aplica una función a varias columnas.
Con selección por tipo y nombres. | <code>summarise(across(c(subtotal, total), sum))</code>

<code>mutate(across(where(is.character), str_trim));</code>
<code>summarise(across(where(is.numeric), mean, .names = "prom_{.col}"))</code> |
| <code>lag()</code> , <code>lead()</code> | Valor anterior / siguiente. | <code>mutate(crec = ventas / lag(ventas) - 1)</code> |
| <code>cumsum()</code> , <code>cummean()</code> | Acumulado / promedio acumulado. | <code>mutate(acumulado = cumsum(ventas))</code> |
| <code>row_number()</code> | Número de fila (ranking sin empates). | <code>mutate(pos = row_number(desc(total)))</code> |

| Función | Qué hace | Ejemplo |
|---|---|---|
| <code>min_rank()</code> , <code>dense_rank()</code> | Ranking con empates (con / sin saltos). | <code>mutate(rank = min_rank(desc(ventas)))</code> |
| <code>percent_rank()</code> , <code>ntile()</code> | Percentil / grupos de igual tamaño. | <code>mutate(cuartil = ntile(total, 4))</code> |
| <code>first()</code> , <code>last()</code> , <code>nth()</code>
<code>between()</code> | Primer, último o n-ésimo valor.
¿Está en el rango (inclusive)? | <code>summarise(primer_compra = first(venta))</code>
<code>filter(between(venta, as.Date("2023-06-01"), as.Date("2023-06-30")))</code> |
| <code>cut()</code> | Convierte números en intervalos. | <code>mutate(rango = cut(edad, breaks = c(0, 25, 40, 60, 100)))</code> |

Tabla B.14. Uniones (*joins*) de tablas.

| Función | Qué conserva | Ejemplo |
|--|---|---|
| <code>left_join()</code> | Todas las filas de la izquierda + lo que coincida (NA si no hay). El más usado: “agregar datos del catálogo”. | <code>ventas %>% left_join(tiendas, by = "tienda_id")</code> |
| <code>inner_join()</code> | Solo filas con coincidencia en ambas. | <code>inner_join(ventas, productos, by = "producto_id")</code> |
| <code>right_join()</code> | Todas las de la derecha. | <code>right_join(ventas, tiendas, by = "tienda_id")</code> |
| <code>full_join()</code>
<code>semi_join()</code> | Todas las filas de ambas.
Filas de la izquierda que SÍ tienen pareja (sin añadir columnas). | <code>full_join(metas, reales, by = "mes")</code>
<code>clientes %>% semi_join(ventas, by = "cliente_id")</code> (clientes que compraron) |
| <code>anti_join()</code> | Filas de la izquierda que NO tienen pareja. | <code>clientes %>% anti_join(ventas, by = "cliente_id")</code> (nunca compraron) |
| Llaves distintas | Columnas con nombre diferente. | <code>by = c("id_tienda" = "tienda_id")</code> o <code>by = join_by(id_tienda == tienda_id)</code> |
| Varias llaves | Unir por más de una columna. | <code>by = c("tienda_id", "mes")</code> |

▲ Cuidado

Después de cada `left_join()` compara `nrow()` antes y después. Si el número de filas crece, la tabla de la derecha tiene llaves repetidas y estarías duplicando ventas. dplyr lo avisa con “*Detected an unexpected many-to-many relationship*” (sección B.11).

Esta es la “plantilla” de análisis que más vas a repetir: filtrar, unir, agrupar, resumir, ordenar y dar formato.

Plantilla: ventas con descuento por ciudad

```

1 ventas %>%
2   filter(descuento_pct > 0) %>%           # 1. filas relevantes
3   left_join(tiendas, by = "tienda_id") %>% # 2. agregar la ciudad
4   group_by(ciudad) %>%                   # 3. agrupar
5   summarise(ventas = sum(total),         # 4. resumir
6             tickets = n(),
7             ticket_prom = mean(total),
8             .groups = "drop") %>%
9   arrange(desc(ventas)) %>%             # 5. ordenar

```

```

10 mutate(participacion = percent(ventas / sum(ventas),
11                                accuracy = 0.1)) # 6. formato

```

```

# A tibble: 6 x 5
  ciudad      ventas tickets ticket_prom participacion
  <chr>      <dbl>   <int>      <dbl>   <chr>
1 CDMX      88746.     66      1345.  37.1%
2 Guadalajara 59122.     42      1408.  24.7%
3 Tijuana   35240.     25      1410.  14.7%
4 Puebla   21767.     14      1555.   9.1%
5 Querétaro 19365.     17      1139.   8.1%
6 Monterrey 15027.     13      1156.   6.3%

```

CDMX concentra el 37.1 % de las ventas con descuento, pero es porque tiene más tiendas y más tickets (66); el ticket promedio más alto lo tiene Puebla (1,555). Leer las dos columnas juntas evita conclusiones apresuradas.

B.5 tidyr, lubridate y stringr

Tabla B.15. tidyr: reestructurar y completar tablas.

| Función | Qué hace | Ejemplo |
|---|--|---|
| <code>pivot_wider()</code> | De largo a ancho (tabla dinámica: categorías como columnas). | <code>pivot_wider(names_from = trimestre, values_from = ventas, values_fill = 0)</code> |
| <code>pivot_longer()</code> | De ancho a largo (formato <i>tidy</i> , ideal para ggplot). | <code>pivot_longer(cols = Q1:Q4, names_to = "trimestre", values_to = "ventas")</code> |
| <code>separate_wider_delim()</code> | Divide una columna en varias. | <code>separate_wider_delim(mes, delim = "-", names = c("anio", "num_mes"))</code> |
| <code>unite()</code> | Une columnas en una. | <code>unite("clave", tienda_id, mes, sep = "_")</code> |
| <code>drop_na()</code> | Quita filas con NA (en todas o en ciertas columnas). | <code>drop_na(total)</code> |
| <code>replace_na()</code> | Sustituye NA por valores. | <code>replace_na(list(descuento_pct = 0))</code> |
| <code>fill()</code> | Rellena NA con el valor anterior (o siguiente). | <code>fill(region, .direction = "down")</code> |
| <code>complete()</code> | Agrega combinaciones faltantes (p. ej., meses sin venta). | <code>complete(tienda_id, mes, fill = list(ventas = 0))</code> |
| <code>expand_grid()</code> | Todas las combinaciones de vectores. | <code>expand_grid(tienda = 1:10, mes = 1:12)</code> |
| <code>nest()</code> / <code>unnest()</code> | Anida sub-tablas por grupo / las expande. | <code>group_by(tienda_id) %>% nest()</code> |

Tabla B.16. **lubridate**: fechas sin dolor.

| Función | Qué hace | Ejemplo → resultado |
|---|---|--|
| <code>ymd()</code> , <code>dmy()</code> , <code>mdy()</code> | Texto a fecha según el orden año-mes-día. | <code>dmy("15/03/2023")</code> → "2023-03-15" |
| <code>ymd_hms()</code> | Texto a fecha y hora. | <code>ymd_hms("2023-03-15 14:30:00")</code> |
| <code>make_date()</code> | Fecha a partir de números. | <code>make_date(2023, 3, 15)</code> |
| <code>today()</code> , <code>now()</code> | Fecha / fecha-hora actual. | <code>today()</code> - <code>ymd("2023-01-01")</code> |
| <code>year()</code> , <code>month()</code> , <code>day()</code> | Extraen componentes. | <code>month(fecha, label = TRUE, abbr = FALSE)</code> → enero |
| <code>wday()</code> | Día de la semana. | <code>wday(fecha, label = TRUE); wday(fecha, week_start = 1)</code> (lunes = 1) |
| <code>quarter()</code> , <code>week()</code> , <code>isoweek()</code> | Trimestre y semana del año. | <code>quarter(ymd("2023-05-10"))</code> → 2 |
| <code>floor_date()</code> | Redondea hacia abajo al periodo. | <code>floor_date(fecha, "month")</code> (primer día del mes) |
| <code>ceiling_date()</code> , <code>rollback()</code> | Inicio del periodo siguiente / último día del mes anterior. | <code>rollback(ymd("2023-05-10"))</code> → "2023-04-30" |
| <code>%m+%</code> con <code>months()</code> | Suma meses sin fechas inválidas. | <code>ymd("2023-01-31") %m+% months(1)</code> → "2023-02-28" |
| <code>days()</code> , <code>weeks()</code> , <code>years()</code> | Periodos para sumar o restar. | <code>fecha + days(30)</code> |
| <code>interval()</code> , <code>time_length()</code> | Duración entre dos fechas en la unidad elegida. | <code>time_length(interval(fecha_registro, today()), "year")</code> (antigüedad) |

Tabla B.17. **stringr**: limpieza y manejo de texto (todas empiezan con `str_`).

| Función | Qué hace | Ejemplo → resultado |
|---|---|---|
| <code>str_detect()</code> | ¿Contiene el patrón? (para filter). | <code>filter(str_detect(metodo_pago, "Tarjeta"))</code> |
| <code>str_starts()</code> , <code>str_ends()</code> | ¿Empieza / termina con? | <code>str_ends(email, "@email.com")</code> |
| <code>str_replace_all()</code> | Reemplaza todas las coincidencias. | <code>str_replace_all("Tarjeta Crédito", " ", "_")</code> → "Tarjeta_Crédito" |
| <code>str_remove_all()</code> | Elimina coincidencias. | <code>str_remove_all("\$1,250.50", "[\$,]")</code> → "1250.50" |
| <code>str_to_upper()</code> , <code>_lower()</code> , <code>_title()</code> | Mayúsculas, minúsculas, tipo título. | <code>str_to_title("sucursal centro")</code> → "Sucursal Centro" |
| <code>str_trim()</code> , <code>str_squish()</code> | Quita espacios de los extremos / también los repetidos. | <code>str_squish(" Sucursal Centro ")</code> → "Sucursal Centro" |
| <code>str_sub()</code> | Recorta por posición (negativo = desde el final). | <code>str_sub("Producto 07", -2)</code> → "07" |
| <code>str_extract()</code> | Extrae la parte que coincide. | <code>str_extract("Producto 07", "\\d+")</code> → "07" |
| <code>str_split()</code> | Divide en partes. | <code>str_split("CDMX-Norte", "-")</code> |
| <code>str_c()</code> | Concatena. | <code>str_c("Q", 1:2, collapse = ", ")</code> → "Q1, Q2" |
| <code>str_glue()</code> | Plantilla con <code>{variables}</code> . | <code>str_glue("Tienda {3}: {dollar(1500)}")</code> → Tienda 3: \$1,500 |
| <code>str_pad()</code> | Rellena a un ancho fijo. | <code>str_pad(7, width = 3, pad = "0")</code> → "007" |
| <code>str_length()</code> , <code>str_count()</code> | Longitud / número de coincidencias. | <code>str_count("banana", "a")</code> → 3 |

Patrones (expresiones regulares) frecuentes: `^` inicio, `$` final, `.` cualquier carácter, `\\d` dígito, `[aeiou]` cualquiera de esos, `+` una o más veces, `|` "o". Para buscar un punto literal escribe `\\. .`

```

1 fechas <- ymd(c("2023-01-15", "2023-07-04"))
2 month(fechas, label = TRUE, abbr = FALSE) # nombre del mes
3 wday(fechas, label = TRUE)                # día de la semana
4 floor_date(fechas, "month")                # primer día del mes
5 fechas %m+% months(1)                     # un mes después
6 str_to_title("sucursal centro norte")
7 str_detect(c("Tarjeta Crédito", "Efectivo"), "Tarjeta")

```

```

[1] enero julio
12 Levels: enero < febrero < marzo < abril < mayo < ... < diciembre
[1] dom mar
Levels: dom < lun < mar < mié < jue < vie < sáb
[1] "2023-01-01" "2023-07-01"
[1] "2023-02-15" "2023-08-04"
[1] "Sucursal Centro Norte"
[1] TRUE FALSE

```

Observa que `month()` y `wday()` con `label = TRUE` devuelven *factores ordenados*: por eso en ggplot los meses salen en orden de calendario y no alfabético. Los nombres aparecen en español porque la sesión usa el idioma del sistema.

B.6 ggplot2: visualización

B.6.1 Estructura de un gráfico

Todo gráfico de ggplot2 se arma por capas unidas con `+` (no con `%>%`):

```

1 ggplot(datos, aes(x = ..., y = ..., color = ..., fill = ...)) + # datos + mapeo
2   geom_...() + # 1 o más capas geométricas
3   scale_..._...() + # escalas: formato de ejes y colores
4   facet_wrap(~ variable) + # (opcional) paneles por grupo
5   labs(title = , subtitle = , x = , y = , caption = ) + # textos
6   theme_minimal() + theme(...) # apariencia

```

Esta plantilla completa (datos, geom, escala con formato, títulos, tema y guardado) sirve de punto de partida para casi cualquier gráfica de tendencia:

Plantilla de gráfico de tendencia y exportación

```

1 color_principal <- "#2a78d6" # color corporativo
2 p <- ventas %>%
3   mutate(mes = floor_date(fecha, "month")) %>%
4   group_by(mes) %>%
5   summarise(ventas = sum(total), .groups = "drop") %>%

```

```

6  ggplot(aes(x = mes, y = ventas)) +
7  geom_line(color = color_principal, linewidth = 0.8) +
8  geom_point(color = color_principal) +
9  scale_y_continuous(labels = label_dollar(), limits = c(0, NA)) +
10 scale_x_date(date_labels = "%b", date_breaks = "1 month") +
11 labs(title = "Ventas mensuales 2023", x = NULL, y = "Ventas",
12       caption = "Fuente: ventas_retail.csv") +
13 theme_minimal() +
14 theme(panel.grid.minor = element_blank())
15 dir.create("resultados", showWarnings = FALSE)
16 ggsave("resultados/ventas_mensuales.png", p,
17       width = 8, height = 4.5, dpi = 300)      # PNG listo para PowerPoint

```

B.6.2 ¿Qué gráfico uso? Objetivo → geom

Tabla B.18. Elegir el tipo de gráfico según la pregunta de negocio.

| Objetivo | Geom | Ejemplo |
|--|---|---|
| Comparar montos entre categorías | <code>geom_col()</code> (barras con valor ya calculado) | <code>ggplot(res, aes(x = reorder(ciudad, ventas), y = ventas)) + geom_col() + coord_flip()</code> |
| Contar registros por categoría | <code>geom_bar()</code> (cuenta sola) | <code>ggplot(clientes, aes(x = segmento)) + geom_bar()</code> |
| Tendencia en el tiempo | <code>geom_line()</code> + <code>geom_point()</code> | <code>ggplot(mensual, aes(mes, ventas)) + geom_line(linewidth = 0.8)</code> |
| Varias series en el tiempo | <code>geom_line(aes(color = grupo))</code> | <code>aes(mes, ventas, color = ciudad)</code> (máximo 5–8 series; si no, facetas) |
| Distribución de una variable | <code>geom_histogram()</code> , <code>geom_density()</code> | <code>ggplot(ventas, aes(total)) + geom_histogram(bins = 30)</code> |
| Comparar distribuciones entre grupos | <code>geom_boxplot()</code> , <code>geom_violin()</code> | <code>ggplot(ventas, aes(factor(tienda_id), total)) + geom_boxplot()</code> |
| Relación entre dos numéricas | <code>geom_point()</code> + <code>geom_smooth()</code> | <code>ggplot(productos, aes(costo, precio_catalogo)) + geom_point() + geom_smooth(method = "lm")</code> |
| Composición (partes de un total) | <code>geom_col(position = "fill")</code> | <code>geom_col(position = "fill") + scale_y_continuous(labels = percent)</code> |
| Pocas partes de un total (máx. 5) | <code>geom_col()</code> + <code>coord_polar("y")</code> | Solo si las diferencias son grandes; si no, usa barras. |
| Mapa de calor (día × hora, mes × tienda) | <code>geom_tile()</code> | <code>ggplot(t, aes(mes, tienda, fill = ventas)) + geom_tile()</code> |
| Ranking con valor destacado | <code>geom_col()</code> + <code>geom_text()</code> | <code>geom_text(aes(label = dollar(ventas)), hjust = -0.1)</code> |
| Meta o referencia | <code>geom_hline()</code> , <code>geom_vline()</code> | <code>geom_hline(yintercept = meta, linetype = "dashed")</code> |
| Rango o incertidumbre | <code>geom_ribbon()</code> , <code>geom_errorbar()</code> | <code>geom_ribbon(aes(ymin = li, ymax = ls), alpha = 0.2)</code> (pronósticos) |
| Anotar un evento | <code>annotate()</code> | <code>annotate("text", x = fecha, y = 5000, label = "Buen Fin")</code> |

B.6.3 Escalas, formatos, temas y facetas

Tabla B.19. Escalas, formato de ejes (**scales**), temas, facetas y exportación.

| Función | Qué hace | Ejemplo |
|---|--|--|
| <code>label_dollar()</code> | Formato moneda. | <code>scale_y_continuous(labels = label_dollar())</code> |
| <code>label_comma()</code> | Separador de miles. | <code>scale_x_continuous(labels = label_comma())</code> |
| <code>label_percent()</code> | Porcentaje (0.125 → 12.5 %). | <code>labels = label_percent(accuracy = 0.1)</code> |
| <code>label_number(scale_cut = cut_short_scale())</code> | Abrevia miles (K) y millones (M). | <code>labels = label_number(scale_cut = cut_short_scale(), prefix = "\$")</code> |
| <code>scale_x_date()</code> | Ejes de fecha. | <code>scale_x_date(date_labels = "%b %Y", date_breaks = "2 months")</code> |
| <code>limits, expand</code> | Rango del eje (incluir el cero en barras). | <code>scale_y_continuous(limits = c(0, NA))</code> |
| <code>scale_fill_manual()</code> ,
<code>scale_color_manual()</code> | Colores propios. | <code>scale_fill_manual(values = paleta_libro)</code> |
| <code>scale_fill_gradient()</code> | Secuencial (un tono de claro a oscuro). | <code>scale_fill_gradient(low = "#dbe8f7", high = "#2a78d6")</code> |
| <code>scale_fill_gradient2()</code> | Divergente (dos tonos, centro neutro). | <code>scale_fill_gradient2(low = "#eb6834", mid = "grey90", high = "#2a78d6", midpoint = 0)</code> |
| <code>labs()</code> | Títulos, ejes, leyenda, fuente. | <code>labs(title = "...", x = NULL, color = "Ciudad", caption = "Fuente: ...")</code> |
| <code>theme_minimal()</code> ,
<code>theme_classic()</code> | Temas base limpios. | <code>theme_minimal(base_size = 12)</code> |
| <code>theme()</code> | Ajuste fino de cualquier elemento. | <code>theme(legend.position = "top", panel.grid.minor = element_blank())</code> |
| <code>coord_flip()</code> | Gira ejes (barras horizontales). | Útil con nombres largos de categorías. |
| <code>facet_wrap()</code> | Un panel por categoría. | <code>facet_wrap(~ ciudad, ncol = 3)</code> |
| <code>facet_grid()</code> | Rejilla filas × columnas. | <code>facet_grid(tipo ~ trimestre)</code> |
| <code>ggsave()</code> | Guarda el gráfico (PNG, PDF, SVG). | <code>ggsave("resultados/g.png", p, width = 8, height = 4.5, dpi = 300)</code> |
| <code>plotly::ggplotly()</code> | Convierte a gráfico interactivo. | <code>ggplotly(p)</code> |

Las funciones de **scales** también sirven fuera de los gráficos, por ejemplo para dar formato a tablas de un reporte:

```

1 dollar(1250.5)                # moneda
2 comma(12500)                  # miles
3 percent(0.1253, accuracy = 0.1) # porcentaje con 1 decimal
4 label_number(scale_cut = cut_short_scale(), prefix = "$")(c(12500, 2500000))

```

```

[1] "$1,250.50"
[1] "12,500"
[1] "12.5%"
[1] "$12K" "$2M"

```

 Consejo

Reglas de oro de la visualización del curso (Módulo 3): una sola serie, un solo color; resalta lo importante con un color de acento y el resto en gris; nunca uses doble eje Y; barras siempre desde cero; ejes con formato de negocio; y un título que diga la conclusión (“Las ventas caen en marzo”) y no solo el tema (“Ventas por mes”).

B.7 Equivalencias SQL, dplyr y Excel

Si vienes de Excel o de SQL, esta tabla es tu diccionario. Los ejemplos de dplyr parten de `ventas %>%`.

Tabla B.20. Diccionario SQL ↔ dplyr ↔ Excel.

| Tarea | SQL | dplyr | Excel |
|---------------------------------------|---|--|---|
| Elegir columnas | <code>SELECT fecha, total FROM ventas</code> | <code>select(fecha, total)</code> | Ocultar / copiar columnas |
| Renombrar | <code>SELECT total AS importe</code> | <code>rename(importe = total)</code> | Cambiar encabezado |
| Filtrar filas | <code>WHERE total > 1000 AND tienda_id = 3</code> | <code>filter(total > 1000, tienda_id == 3)</code> | Autofiltro; <code>FILTRAR()</code> |
| Lista de valores | <code>WHERE ciudad IN ('CDMX', 'Puebla')</code> | <code>filter(ciudad %in% c("CDMX", "Puebla"))</code> | Autofiltro con varias casillas |
| Rango | <code>WHERE total BETWEEN 500 AND 1000</code> | <code>filter(between(total, 500, 1000))</code> | Filtro personalizado |
| Texto que contiene | <code>WHERE metodo_pago LIKE '%Tarjeta%'</code> | <code>filter(str_detect(metodo_pago, "Tarjeta"))</code> | Filtro “Contiene” |
| Nulos | <code>WHERE total IS NULL</code> | <code>filter(is.na(total))</code> | Filtro “(Vacías)” |
| Columna calculada | <code>SELECT total * 0.16 AS iva</code> | <code>mutate(iva = total * 0.16)</code> | Fórmula <code>=L2*0.16</code> arrastrada |
| Condicional | <code>CASE WHEN total >= 1000 THEN 'Alta' ELSE 'Normal' END</code> | <code>if_else(total >= 1000, "Alta", "Normal"); case_when()</code> | <code>SI(), SI.CONJUNTO()</code> |
| Sustituir nulos | <code>COALESCE(descuento, 0)</code> | <code>coalesce(descuento, 0)</code> | <code>SI.ERROR(), SI(ESBLANCO())</code> |
| Ordenar | <code>ORDER BY total DESC</code> | <code>arrange(desc(total))</code> | Ordenar de mayor a menor |
| Primeras n filas | <code>LIMIT 10</code> | <code>slice_head(n = 10)</code> | — |
| Top n por valor | <code>ORDER BY total DESC LIMIT 5</code> | <code>slice_max(total, n = 5)</code> | Filtro “Diez mejores” |
| Únicos | <code>SELECT DISTINCT cliente_id</code> | <code>distinct(cliente_id)</code> | Quitar duplicados; <code>UNICOS()</code> |
| Totales generales | <code>SELECT SUM(total), AVG(total), COUNT(*)</code> | <code>summarise(sum(total), mean(total), n())</code> | <code>SUMA(), PROMEDIO(), CONTAR()</code> |
| Contar distintos | <code>COUNT(DISTINCT cliente_id)</code> | <code>n_distinct(cliente_id)</code> | Tabla dinámica con “Recuento distinto” |
| Totales por grupo | <code>SELECT tienda_id, SUM(total) ... GROUP BY tienda_id</code> | <code>group_by(tienda_id) %>% summarise(ventas = sum(total))</code> | Tabla dinámica; <code>SUMAR.SI()</code> |
| Suma condicional con varios criterios | <code>SUM(total) ... WHERE ... GROUP BY tienda_id, mes</code> | <code>filter(...) %>% summarise(sum(total), .by = c(tienda_id, mes))</code> | <code>SUMAR.SI.CONJUNTO()</code> |
| Conteo condicional | <code>SUM(CASE WHEN total > 1000 THEN 1 ELSE 0 END)</code> | <code>summarise(n_altas = sum(total > 1000))</code> | <code>CONTAR.SI()</code> |

| Tarea | SQL | dplyr | Excel |
|-----------------------------|---|--|--|
| Promedio condicional | <code>AVG(total) ... GROUP BY ...</code> | <code>summarise(prom = mean(total),
.by = ciudad)</code> | <code>PROMEDIO.SI()</code> |
| Filtrar grupos | <code>GROUP BY tienda_id HAVING
COUNT(*) >= 20</code> | <code>group_by(tienda_id)%>%
filter(n() >= 20) o
summarise() + filter()</code> | Filtro sobre la tabla dinámica |
| Traer datos de otra tabla | <code>LEFT JOIN tiendas t ON
v.tienda_id = t.tienda_id</code> | <code>left_join(tiendas, by =
"tienda_id")</code> | <code>BUSCARV()</code> , <code>BUSCARX()</code> ,
<code>INDICE+COINCIDIR</code> |
| Solo coincidencias | <code>INNER JOIN</code> | <code>inner_join()</code> | — |
| Registros sin pareja | <code>LEFT JOIN ... WHERE t.id IS
NULL</code> | <code>anti_join()</code> | <code>BUSCARV</code> que da #N/A |
| Apilar tablas | <code>UNION ALL</code> | <code>bind_rows()</code> | Copiar y pegar debajo |
| Pivotear | <code>SUM(CASE WHEN trimestre =
'Q1' THEN total END) AS Q1,
...</code> | <code>pivot_wider(names_from =
trimestre, values_from =
ventas)</code> | Tabla dinámica (campo en Columnas) |
| Despivotear | <code>UNION ALL</code> por columna | <code>pivot_longer(Q1:Q4, names_to =
"trim", values_to =
"ventas")</code> | Power Query: “Anular dinamización” |
| Funciones de ventana | | | |
| Número de fila por grupo | <code>ROW_NUMBER() OVER
(PARTITION BY tienda_id
ORDER BY total DESC)</code> | <code>group_by(tienda_id)%>%
mutate(pos =
row_number(desc(total)))</code> | <code>JERARQUIA()</code> con criterio |
| Ranking con empates | <code>RANK(), DENSE_RANK() OVER
(...)</code> | <code>min_rank(), dense_rank()</code> | <code>JERARQUIA.EQV()</code> |
| Valor anterior | <code>LAG(ventas) OVER (ORDER BY
mes)</code> | <code>mutate(anterior =
lag(ventas))</code> | Referencia a la celda de arriba |
| Acumulado | <code>SUM(ventas) OVER (ORDER BY
mes)</code> | <code>mutate(acum = cumsum(ventas))</code> | <code>=SUMA(\$B\$2:B2)</code> |
| % del total del grupo | <code>total / SUM(total) OVER
(PARTITION BY tienda_id)</code> | <code>group_by(tienda_id)%>%
mutate(p = total /
sum(total))</code> | “Mostrar valores como % del total” |
| Promedio móvil | <code>AVG(v) OVER (ORDER BY mes
ROWS 2 PRECEDING)</code> | <code>zoo::rollmean(v, 3, fill = NA,
align = "right")</code> | <code>PROMEDIO()</code> de 3 celdas arrastrado |
| Cuartiles | <code>NTILE(4) OVER (ORDER BY
total)</code> | <code>mutate(cuartil = ntile(total,
4))</code> | <code>CUARTIL()</code> |
| Texto y fechas | | | |
| Unir texto | <code>CONCAT(nombre, ' ',
apellido) o </code> | <code>str_c(nombre, " ", apellido)</code> | <code>CONCAT()</code> , <code>&</code> |
| Mayúsculas / espacios | <code>UPPER(), TRIM()</code> | <code>str_to_upper(), str_trim()</code> | <code>MAYUSC()</code> , <code>ESPACIOS()</code> |
| Subcadena | <code>SUBSTR(x, 1, 3)</code> | <code>str_sub(x, 1, 3)</code> | <code>IZQUIERDA()</code> , <code>EXTRAE()</code> |
| Año / mes de una fecha | <code>EXTRACT(YEAR FROM fecha); en
SQLite strftime('%Y',
fecha)</code> | <code>year(fecha), month(fecha)</code> | <code>AÑO()</code> , <code>MES()</code> |
| Diferencia de fechas | <code>fecha2 - fecha1 (según
motor)</code> | <code>as.numeric(fecha2 - fecha1)</code> | <code>=B2-A2, SIFECHA()</code> |

La misma consulta, en SQL y en dplyr, da el mismo resultado. Esta es la consulta en SQL:

```
SELECT tienda_id, COUNT(*) AS n, ROUND(SUM(total), 2) AS ventas
FROM ventas
```

```
WHERE descuento_pct > 0
GROUP BY tienda_id
HAVING COUNT(*) >= 20
ORDER BY ventas DESC
```

Y así se ejecuta desde R sobre una base SQLite en memoria, junto a su equivalente en dplyr:

```
1 library(DBI)
2 con <- dbConnect(RSQLite::SQLite(), ":memory:") # base temporal
3 dbWriteTable(con, "ventas", select(ventas, tienda_id, descuento_pct, total))
4 dbGetQuery(con, "
5   SELECT tienda_id, COUNT(*) AS n, ROUND(SUM(total), 2) AS ventas
6   FROM ventas
7   WHERE descuento_pct > 0
8   GROUP BY tienda_id
9   HAVING COUNT(*) >= 20
10  ORDER BY ventas DESC")
11 ventas %>% # lo mismo en dplyr
12   filter(descuento_pct > 0) %>%
13   group_by(tienda_id) %>%
14   summarise(n = n(), ventas = round(sum(total), 2), .groups = "drop") %>%
15   filter(n >= 20) %>% # el HAVING
16   arrange(desc(ventas))
17 dbDisconnect(con)
```

```
  tienda_id  n  ventas
1         8 25 35240.49
2         2 25 34557.58
3         9 22 30632.71
4         6 21 27960.98
# A tibble: 4 x 3
  tienda_id      n ventas
  <dbl> <int> <dbl>
1         8     25 35240.
2         2     25 34558.
3         9     22 30633.
4         6     21 27961.
```

Las cifras coinciden; solo cambia la presentación (el tibble muestra menos decimales, pero guarda los mismos valores). Nota que en dplyr el **HAVING** es simplemente otro **filter()** después del **summarise()**.

B.8 Estadística y KPIs

B.8.1 ¿Qué prueba uso?

Tabla B.21. Elegir la prueba estadística según la pregunta de negocio.

| Pregunta de negocio | Prueba | Función de R |
|---|-----------------------------------|---|
| ¿El ticket promedio es distinto entre fin de semana y entre semana? (2 grupos, variable numérica) | t de Student (Welch) | <code>t.test(total ~ fin_semana, data = ventas)</code> |
| ¿El ticket promedio es distinto de la meta de \$1,500? | t de una muestra | <code>t.test(ventas\$total, mu = 1500)</code> |
| ¿Cambió la venta por tienda antes y después de la campaña? (mismas unidades) | t pareada | <code>t.test(despues, antes, paired = TRUE)</code> |
| ¿La versión B del correo convierte más que la A? (2 proporciones, prueba A/B) | Prueba de proporciones | <code>prop.test(x = c(conv_a, conv_b), n = c(env_a, env_b))</code> |
| ¿La tasa de devolución supera el 5 %? | Proporción vs. valor | <code>prop.test(x = devol, n = total, p = 0.05, alternative = "greater")</code> |
| ¿El ticket promedio difiere entre 3 o más ciudades? | ANOVA de un factor | <code>summary(aov(total ~ ciudad, data = d)); después TukeyHSD(aov(...))</code> |
| Igual que las anteriores, pero con datos muy asimétricos o pocos datos | No paramétricas | <code>wilcox.test(total ~ grupo)</code> (2 grupos);
<code>kruskal.test(total ~ ciudad)</code> (3+) |
| ¿El método de pago depende del segmento del cliente? (2 categóricas) | Chi-cuadrada de independencia | <code>chisq.test(table(d\$segmento, d\$metodo_pago))</code> |
| ¿Se relacionan el precio y las unidades vendidas? (2 numéricas) | Correlación de Pearson / Spearman | <code>cor.test(d\$precio, d\$unidades); method = "spearman"</code> si hay atípicos |
| ¿Cuánto aumentan las ventas por cada \$1 de publicidad? (explicar / predecir numérica) | Regresión lineal | <code>m <- lm(ventas ~ publicidad + precio, data = d); summary(m)</code> |
| ¿Qué factores aumentan la probabilidad de que un cliente se vaya? (respuesta sí/no) | Regresión logística | <code>glm(churn ~ antigüedad + compras, family = binomial, data = d)</code> |
| ¿Los datos son aproximadamente normales? | Shapiro–Wilk (y un histograma) | <code>shapiro.test(ventas\$total)</code> |

Concepto clave: Cómo leer el p-valor

El **p-valor** es la probabilidad de observar una diferencia (o una relación) tan grande como la de tus datos *si en realidad no hubiera ninguna*. Regla práctica con un nivel de significancia de 0.05:

- **p < 0.05:** la diferencia es *estadísticamente significativa*; es poco probable que sea solo azar.
- **p ≥ 0.05:** no hay evidencia suficiente de diferencia (lo que *no* demuestra que sean iguales).

Significativo no quiere decir importante: con miles de datos, una diferencia de \$2 puede ser significativa y aun así no importar para el negocio. Reporta siempre el tamaño del efecto (la diferencia en pesos o en puntos porcentuales) y su intervalo de confianza, no solo el p-valor.

```

1 # ¿Gastan más los clientes en fin de semana?
2 ventas <- ventas %>%
3   mutate(fin_semana = dia_semana %in% c("sábado", "domingo"))
4 prueba <- t.test(total ~ fin_semana, data = ventas)
5 prueba$estimate      # promedio de cada grupo (FALSE = entre semana)
6 prueba$p.value       # p-valor

```

```

mean in group FALSE mean in group TRUE
          1428.730          1356.635
[1] 0.572521

```

El ticket de fin de semana (1,356.6) es unos \$72 menor que entre semana (1,428.7), pero el p-valor de 0.57 es mucho mayor que 0.05: con estos datos no hay evidencia de que el día influya en el ticket. No conviene lanzar una promoción “de fin de semana” justificada en esa diferencia.

B.8.2 Fórmulas de KPIs de negocio

Tabla B.22. KPIs comunes: fórmula y código **dplyr**.

| KPI | Fórmula | Código dplyr |
|----------------------------------|---|---|
| Crecimiento vs. periodo anterior | $\frac{\text{actual} - \text{anterior}}{\text{anterior}}$ | <code>mutate(crec = ventas / lag(ventas) - 1)</code> (ordena por fecha antes) |
| Crecimiento anual (YoY) | Igual, contra el mismo mes del año anterior | <code>mutate(yoy = ventas / lag(ventas, 12) - 1)</code> (datos mensuales completos) |
| Margen bruto (%) | $\frac{\text{ingresos} - \text{costo de ventas}}{\text{ingresos}}$ | <code>summarise(margen = (sum(total) - sum(cantidad * costo)) / sum(total))</code> |
| Ticket promedio | $\frac{\text{ventas totales}}{\text{número de transacciones}}$ | <code>summarise(ticket = sum(total) / n())</code> |
| Unidades por ticket | $\frac{\text{unidades vendidas}}{\text{transacciones}}$ | <code>summarise(upt = sum(cantidad) / n())</code> |
| Participación (<i>share</i>) | $\frac{\text{ventas del grupo}}{\text{ventas totales}}$ | <code>mutate(share = ventas / sum(ventas))</code> |
| Tasa de descuento | $\frac{\text{descuentos}}{\text{venta bruta}}$ | <code>summarise(tasa_desc = sum(descuento_monto) / sum(subtotal))</code> |
| Cumplimiento de meta | $\frac{\text{real}}{\text{meta}}$ | <code>mutate(cumpl = ventas / meta)</code> |
| Ventas por m ² | $\frac{\text{ventas}}{\text{superficie}}$ | <code>left_join(tiendas) %>% mutate(v_m2 = ventas / tamano_m2)</code> |
| Tasa de conversión | $\frac{\text{compradores (o pedidos)}}{\text{visitas (o leads)}}$ | <code>summarise(conv = sum(compro) / n())</code> |
| Retención | $\frac{\text{clientes al final} - \text{clientes nuevos}}{\text{clientes al inicio}}$ | <code>mutate(ret = (clientes_fin - nuevos) / clientes_inicio)</code> |
| Churn (abandono) | $\frac{\text{clientes perdidos}}{\text{clientes al inicio}} = 1 - \text{retención}$ | <code>mutate(churn = perdidos / clientes_inicio)</code> |
| CLV simple | $\text{ticket} \times \text{compras al año} \times \text{margen} \times \text{años de vida} (\approx 1/\text{churn anual})$ | <code>mutate(clv = ticket * frec_anual * margen * (1 / churn))</code> |

| KPI | Fórmula | Código dplyr |
|----------------------------|---|---|
| Costo de adquisición (CAC) | $\frac{\text{gasto en marketing y ventas}}{\text{clientes nuevos}}$ | <code>mutate(cac = gasto_mkt / nuevos)</code> |
| ROI | $\frac{\text{ganancia} - \text{inversión}}{\text{inversión}}$ | <code>mutate(roi = (ganancia - inversion) / inversion)</code> |
| Rotación de inventario | $\frac{\text{costo de ventas}}{\text{inventario promedio (a costo)}}$ | <code>mutate(rotacion = costo_ventas / inv_promedio)</code> |
| Días de inventario | $\frac{365}{\text{rotación inventario promedio}} = \frac{\text{costo de ventas}}{\text{stock actual}} \times 365$ | <code>mutate(dias_inv = 365 / rotacion)</code> |
| Cobertura de stock | $\frac{\text{stock actual}}{\text{venta diaria promedio}}$ | <code>mutate(dias_cobertura = stock_actual / venta_diaria)</code> |

Así se ven algunos de estos KPIs calculados con los datos del curso:

KPIs mensuales: ventas, tickets, ticket promedio y crecimiento

```
1 kpi_mes <- ventas %>%
2   mutate(mes = floor_date(fecha, "month")) %>%
3   group_by(mes) %>%
4   summarise(ventas = sum(total), tickets = n(), .groups = "drop") %>%
5   mutate(ticket_prom = ventas / tickets,          # ticket promedio
6          crec_mom = ventas / lag(ventas) - 1)     # vs. mes anterior
7 kpi_mes %>%
8   mutate(crec_mom = percent(crec_mom, accuracy = 0.1)) %>%
9   head(4)
```

```
# A tibble: 4 x 5
  mes      ventas tickets ticket_prom crec_mom
<date>    <dbl>   <int>    <dbl> <chr>
1 2023-01-01 46884.     31    1512. <NA>
2 2023-02-01 42656.     28    1523. -9.0%
3 2023-03-01 36733.     31    1185. -13.9%
4 2023-04-01 34738.     30    1158. -5.4%
```

Enero no tiene crecimiento (no hay mes anterior: por eso **NA**). En marzo las ventas cayeron 13.9% con casi los mismos tickets (31), así que la caída se explica por un ticket promedio más bajo, no por menos clientes.

Margen bruto global

```
1 ventas %>%
2   left_join(select(productos, producto_id, costo), by = "producto_id") %>%
3   summarise(ingresos = sum(total),
4            costo_ventas = sum(cantidad * costo),
5            margen_bruto_pct = (ingresos - costo_ventas) / ingresos)
```

```
# A tibble: 1 x 3
  ingresos costo_ventas margen_bruto_pct
  <dbl>      <dbl>      <dbl>
1  513917.    363346.    0.293
```

El margen bruto del año es de 29.3 %. Recuerda que en estos datos simulados el precio de cada venta no coincide con el precio de catálogo y hay productos con costo mayor al precio: antes de presentar un margen, revisa siempre la calidad de los datos de costo.

B.9 Machine learning

Tabla B.23. Del problema de negocio al algoritmo, la función de R y la métrica.

| Problema de negocio | Tipo | Algoritmo | Función de R | Métrica |
|--|-------------------------------|--|---|-----------------------------------|
| Pronosticar las ventas de los próximos meses | Series de tiempo | ARIMA, suavizamiento exponencial (ETS) | <code>forecast::auto.arima()</code> , <code>ets()</code> , <code>forecast(m, h = 6)</code> | MAPE, RMSE |
| Estimar el monto de compra o el precio de una casa | Regresión | Regresión lineal, bosque aleatorio | <code>lm()</code> , <code>randomForest()</code> | RMSE, MAE, R^2 |
| Explicar qué mueve las ventas (precio, promoción, temporada) | Regresión explicativa | Regresión lineal múltiple | <code>lm(ventas ~ precio + promo + mes)</code> | R^2 , p-valores de coeficientes |
| Predecir qué clientes se darán de baja (churn) | Clasificación binaria | Logística, árbol, bosque aleatorio | <code>glm(family = binomial)</code> , <code>rpart::rpart()</code> , <code>randomForest()</code> | AUC, recall, F1 |
| Calificar prospectos (<i>lead scoring</i>) | Clasificación binaria | Regresión logística | <code>glm() + predict(type = "response")</code> | AUC, lift |
| Detectar fraude (casos muy raros) | Clasificación desbalanceada | Bosque aleatorio con remuestreo | <code>caret::train(method = "rf")</code> con <code>sampling = "down"</code> | Precisión, recall (no exactitud) |
| Segmentar clientes para campañas | Agrupamiento (no supervisado) | k-medias, jerárquico | <code>kmeans()</code> , <code>hclust()</code> , <code>factoextra::fviz_cluster()</code> | Silüeta, codo (WSS) |
| Productos que se compran juntos | Reglas de asociación | Apriori | <code>arules::apriori()</code> | Soporte, confianza, lift |
| Detectar ventas atípicas | Detección de anomalías | Regla del IQR, puntaje z | <code>boxplot.stats(x)\$out</code> , <code>scale()</code> | Revisión manual |
| Ajustar y comparar modelos con validación cruzada | Cualquiera supervisado | Varios | <code>caret::train(..., trControl = trainControl(method = "cv", number = 5))</code> | La del problema |

Tabla B.24. Métricas de evaluación: fórmula y cuándo usarlas. VP, VN, FP, FN = verdaderos/falsos positivos/negativos; y = real, $\hat{y}$ = predicho.

| Métrica | Fórmula | Cuándo usarla |
|-------------------------------|---|--|
| Regresión y pronóstico | | |
| MAE | $\frac{1}{n} \sum y_i - \hat{y}_i $ | Error promedio en las unidades del negocio (pesos, piezas). Fácil de explicar. |
| RMSE | $\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$ | Penaliza más los errores grandes. Úsala si un error grande es muy costoso (p. ej., quedarte sin inventario). |
| MAPE | $\frac{100\%}{n} \sum \left \frac{y_i - \hat{y}_i}{y_i} \right $ | Error en % para comparar series de distinta escala. No sirve si hay valores reales iguales o cercanos a 0. |
| R ² | $1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$ | Proporción de la variación explicada (0 a 1). Útil para modelos explicativos; no mide el error de pronóstico. |
| Clasificación | | |
| Exactitud | $\frac{VP+VN}{VP+VN+FP+FN}$ | % de aciertos. Solo si las clases están balanceadas: con 5 % de churn, decir “nadie se va” da 95 % de exactitud. |
| Precisión | $\frac{VP}{VP+FP}$ | De los que marqué como positivos, ¿cuántos lo eran? Importa cuando cada falso positivo cuesta (p. ej., bloquear una tarjeta legítima). |
| Recall (sensibilidad) | $\frac{VP}{VP+FN}$ | De los positivos reales, ¿cuántos encontré? Importa cuando perder un caso es caro (fraude no detectado, cliente que se va sin aviso). |
| F1 | $2 \cdot \frac{\text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}$ | Equilibrio entre precisión y recall en un solo número; buena para clases desbalanceadas. |
| AUC (ROC) | Área bajo la curva ROC (0.5 = azar, 1 = perfecto) | Capacidad de ordenar positivos por encima de negativos, sin fijar un umbral. Ideal para comparar modelos de puntuación. |

Todas estas métricas se calculan con pocas líneas de R base. El paquete **pROC** (se instala junto con **caret**) calcula el AUC:

Métricas de regresión y de clasificación a mano

```

1 # --- Regresión / pronóstico ---
2 real      <- c(120, 95, 130, 150, 110)
3 pronostico <- c(115, 100, 128, 140, 118)
4 error <- real - pronostico
5 tibble(MAE = mean(abs(error)),
6        RMSE = sqrt(mean(error^2)),
7        MAPE = mean(abs(error / real)))
8 # --- Clasificación (1 = el cliente se fue) ---
9 obs  <- factor(c(1, 0, 1, 1, 0, 0, 1, 0, 0, 0), levels = c(1, 0))
10 pred <- factor(c(1, 0, 0, 1, 0, 1, 1, 0, 0, 0), levels = c(1, 0))
11 mc <- table(Predicho = pred, Real = obs)      # matriz de confusión
12 mc
13 vp <- mc["1", "1"]; fp <- mc["1", "0"]

```

```

14 fn <- mc["0", "1"]; vn <- mc["0", "0"]
15 exactitud <- (vp + vn) / sum(mc)
16 precision <- vp / (vp + fp)
17 recall <- vp / (vp + fn)
18 f1 <- 2 * precision * recall / (precision + recall)
19 round(c(exactitud = exactitud, precision = precision,
20         recall = recall, f1 = f1), 3)
21 # --- AUC a partir de probabilidades ---
22 prob <- c(0.81, 0.30, 0.45, 0.66, 0.12, 0.58, 0.90, 0.25, 0.40, 0.08)
23 curva <- pROC::roc(response = obs, predictor = prob,
24                   levels = c("0", "1"), direction = "<")
25 pROC::auc(curva)

```

```

# A tibble: 1 x 3
  MAE  RMSE  MAPE
  <dbl> <dbl> <dbl>
1     6  6.60 0.0498

  Real
Predicho 1 0
          1 3 1
          0 1 5
exactitud precision  recall      f1
      0.80      0.75      0.75    0.75
Area under the curve: 0.9583

```

El pronóstico se equivoca en promedio 6 unidades (MAPE de 5%). El clasificador acierta 8 de 10 casos, encuentra 3 de los 4 clientes que se fueron (recall 0.75) y su AUC de 0.96 indica que ordena muy bien a los clientes por riesgo.

B.10 Shiny y R Markdown

B.10.1 Shiny: entradas, salidas y reactividad

Tabla B.25. Controles de entrada (*inputs*) de Shiny.

| Función | Control | Ejemplo |
|-------------------------------|--|--|
| <code>selectInput()</code> | Lista desplegable (una u varias opciones). | <code>selectInput("ciudad", "Ciudad", choices = ciudades, multiple = TRUE)</code> |
| <code>sliderInput()</code> | Deslizador numérico o de rango. | <code>sliderInput("desc", "Descuento", min = 0, max = 20, value = c(0, 20))</code> |
| <code>dateRangeInput()</code> | Rango de fechas. | <code>dateRangeInput("fechas", "Periodo", start = "2023-01-01", end = "2023-12-31")</code> |
| <code>dateInput()</code> | Una fecha. | <code>dateInput("corte", "Fecha de corte")</code> |
| <code>numericInput()</code> | Número. | <code>numericInput("meta", "Meta mensual", value = 40000)</code> |
| <code>textInput()</code> | Texto libre. | <code>textInput("buscar", "Buscar producto")</code> |

| Función | Control | Ejemplo |
|-----------------------------------|---------------------------|---|
| <code>checkboxInput()</code> | Casilla sí/no. | <code>checkboxInput("premium", "Solo premium", FALSE)</code> |
| <code>checkboxGroupInput()</code> | Varias casillas. | <code>checkboxGroupInput("pago", "Método", choices = metodos)</code> |
| <code>radioButtons()</code> | Una opción entre varias. | <code>radioButtons("metrica", "Métrica", c("Ventas", "Tickets"))</code> |
| <code>actionButton()</code> | Botón (dispara acciones). | <code>actionButton("actualizar", "Actualizar")</code> |
| <code>fileInput()</code> | Subir un archivo. | <code>fileInput("archivo", "Sube tu CSV", accept = ".csv")</code> |

Tabla B.26. Pares salida (en **ui**) ↔ render (en **server**).

| En la ui | En el server | Muestra |
|--|---|--|
| <code>plotOutput("g")</code> | <code>output\$g <- renderPlot({...})</code> | Gráfico de ggplot2 o base. |
| <code>plotly::plotlyOutput("g")</code> | <code>renderPlotly({ggplotly(p)})</code> | Gráfico interactivo. |
| <code>tableOutput("t")</code> | <code>renderTable({...})</code> | Tabla simple. |
| <code>DT::DTOutput("t")</code> | <code>DT::renderDT({...})</code> | Tabla con búsqueda, orden y paginación. |
| <code>textOutput("x")</code> | <code>renderText({...})</code> | Texto (un KPI, un mensaje). |
| <code>verbatimTextOutput("x")</code> | <code>renderPrint({...})</code> | Salida de consola (p. ej., <code>summary(modelo)</code>). |
| <code>uiOutput("x")</code> | <code>renderUI({...})</code> | Controles creados dinámicamente. |
| <code>valueBoxOutput("x")</code> | <code>renderValueBox({...})</code> | Tarjeta de KPI (shinydashboard). |
| <code>downloadButton("d")</code> | <code>downloadHandler(filename, content)</code> | Botón para descargar CSV o Excel. |

Tabla B.27. Funciones reactivas y de diseño de Shiny.

| Función | Qué hace | Ejemplo |
|--|---|--|
| <code>reactive()</code> | Cálculo reutilizable que se actualiza solo cuando cambian sus entradas. Se usa como función: <code>datos()</code> . | <code>datos <- reactive({ventas%>% filter(ciudad%in% input\$ciudad)})</code> |
| <code>eventReactive()</code> | Como <code>reactive()</code> , pero solo se recalcula al pulsar un botón. | <code>eventReactive(input\$actualizar, {...})</code> |
| <code>observeEvent()</code> | Ejecuta una acción (efecto) cuando ocurre un evento. | <code>observeEvent(input\$guardar, {write_csv(...)})</code> |
| <code>observe()</code> | Acción que se repite cuando cambian sus entradas. | <code>observe({print(input\$ciudad)})</code> |
| <code>reactiveVal()</code> | Valor que puedes modificar desde el server. | <code>contador <- reactiveVal(0)</code> |
| <code>req()</code> | Detiene en silencio si falta una entrada. | <code>req(input\$archivo)</code> |
| <code>validate(need())</code> | Muestra un mensaje amable en lugar de un error. | <code>validate(need(nrow(datos()) > 0, "Sin datos"))</code> |
| <code>isolate()</code> | Lee una entrada sin reaccionar a ella. | <code>isolate(input\$meta)</code> |
| <code>update*Input()</code> | Cambia opciones de un control. | <code>updateSelectInput(session, "tienda", choices = t)</code> |
| <code>fluidPage()</code> ,
<code>sidebarLayout()</code> | Página con panel lateral de filtros y panel principal. | <code>sidebarLayout(sidebarPanel(...), mainPanel(...))</code> |
| <code>fluidRow()</code> , <code>column()</code> | Rejilla de 12 columnas. | <code>fluidRow(column(4, ...), column(8, ...))</code> |

| Función | Qué hace | Ejemplo |
|---|--|---|
| <code>tabsetPanel()</code> ,
<code>navbarPage()</code>
<code>dashboardPage()</code> | Pestañas / menú superior.

Estructura de shinydashboard . | <code>tabsetPanel(tabPanel("Ventas", ...))</code>

<code>dashboardPage(dashboardHeader(),
dashboardSidebar(), dashboardBody())</code> |
| <code>box()</code> , <code>valueBox()</code> ,
<code>menuItem()</code> | Piezas de shinydashboard . | <code>box(plotOutput("g"), width = 8)</code> |

El esqueleto mínimo de una app (se ejecuta en RStudio con el botón **Run App**):

Esqueleto de app Shiny

```

1 library(shiny)
2 library(tidyverse)
3 ventas <- read_csv("datasets/ventas_retail.csv", show_col_types = FALSE)
4
5 ui <- fluidPage(
6   titlePanel("Ventas por tienda"),
7   sidebarLayout(
8     sidebarPanel(selectInput("tienda", "Tienda", sort(unique(ventas$tienda_id)))),
9     mainPanel(textOutput("kpi"), plotOutput("grafico"))
10  )
11 )
12
13 server <- function(input, output, session) {
14   datos <- reactive(filter(ventas, tienda_id == input$tienda))
15   output$kpi <- renderText(paste("Ventas:", scales::dollar(sum(datos())$total)))
16   output$grafico <- renderPlot(
17     ggplot(datos(), aes(fecha, total)) + geom_line(linewidth = 0.8)
18   )
19 }
20
21 shinyApp(ui, server)

```

B.10.2 R Markdown: opciones de bloques y formatos

Tabla B.28. Opciones de los bloques (*chunks*) de código en R Markdown.

| Opción | Qué hace | Uso típico |
|--|--|---|
| <code>echo = FALSE</code> | Oculto el código, muestra el resultado. | Reportes para directivos. |
| <code>eval = FALSE</code> | Muestra el código sin ejecutarlo. | Ejemplos, código de referencia. |
| <code>include = FALSE</code> | Ejecuta pero no muestra nada. | Bloque de carga de paquetes y datos. |
| <code>message = FALSE</code> ,
<code>warning = FALSE</code> | Oculto mensajes y advertencias. | Evitar el ruido de <code>library()</code> . |
| <code>error = TRUE</code> | Continúa aunque el bloque falle. | Material didáctico. |
| <code>results = "hide" /</code>
<code>"asis"</code> | Oculto la salida / la inserta como texto tal cual. | Tablas generadas como Markdown. |

| Opción | Qué hace | Uso típico |
|---------------------------------------|--|---|
| <code>fig.width, fig.height</code> | Tamaño de la figura en pulgadas. | <code>fig.width = 8, fig.height = 4.5</code> |
| <code>fig.cap, fig.align</code> | Pie y alineación de figura. | <code>fig.align = "center"</code> |
| <code>out.width</code> | Ancho en el documento final. | <code>out.width = "100%"</code> |
| <code>dpi</code> | Resolución de la imagen. | <code>dpi = 300</code> para impresión. |
| <code>cache = TRUE</code> | Guarda resultados para no recalcular. | Modelos lentos. |
| <code>knitr::opts_chunk\$set()</code> | Opciones por omisión para todo el documento. | <code>knitr::opts_chunk\$set(echo = FALSE, message = FALSE)</code> |
| <code>'r expresión'</code> | Código en línea dentro del texto. | "Las ventas fueron de <code>'r</code> dollar(total) <code>'</code> ". |

Formatos de salida más comunes en el encabezado YAML:

- `html_document`: interactivo; admite `toc`, `toc_float` y `code_folding`.
- `pdf_document`: requiere LaTeX (p. ej., `tinytex`).
- `word_document`: editable por otras áreas.
- `powerpoint_presentation` e `ioslides_presentation`: presentaciones.
- `flexdashboard::flex_dashboard`: tablero estático.

Un reporte parametrizado se ve así:

Encabezado YAML de un reporte parametrizado

```

1 ---
2 title: "Reporte mensual de ventas"
3 author: "Área de BI"
4 date: "`r Sys.Date()`"
5 output:
6   html_document:
7     toc: true
8     toc_float: true
9     code_folding: hide
10  params:
11    tienda: 3
12    mes: "2023-06"
13 ---

```

Dentro del documento usas `params$tienda`. Para generar un reporte por tienda, llama a `render()` (del paquete `rmarkdown`) dentro de un bucle:

Un reporte por tienda

```

1 for (t in 1:10) {
2   rmarkdown::render("reporte.Rmd", params = list(tienda = t),
3                     output_file = paste0("reporte_tienda_", t, ".html"))
4 }

```

B.11 Errores frecuentes de R

R muestra sus mensajes en el idioma de tu sistema. Los mensajes de R base suelen estar traducidos al español; los de **dplyr**, **ggplot2** y otros paquetes del tidyverse casi siempre aparecen en inglés, a veces mezclados (por ejemplo, **Caused by error: ! objeto 'b' no encontrado**). La tabla muestra el mensaje exacto en inglés y, debajo, en cursiva, la versión en español cuando R la traduce. Los nombres como **'x'** o **'b'** cambian según tu código.

Consejo

Para buscar un error en internet, conviene tenerlo en inglés (hay muchas más respuestas). Ejecuta `Sys.setenv(LANGUAGE = "en")` y vuelve a correr la línea: el mensaje saldrá en inglés. Busca la parte del mensaje que no depende de tus nombres de variables. En la consola, después de un error del tidyverse, `rlang::last_trace()` muestra dónde ocurrió.

Tabla B.29. Errores y advertencias más frecuentes: mensaje, causa y solución.

| Mensaje (inglés / <i>español</i>) | Causa típica | Solución |
|---|--|--|
| Objetos, funciones y paquetes | | |
| object 'total_ventas' not found
<i>objeto 'total_ventas' no encontrado</i> | Error de escritura (mayúsculas, acentos), objeto aún no creado, o <code>filter()</code> de stats porque no cargaste dplyr. | Revisa el nombre con <code>ls()</code> o <code>names(df)</code> ; ejecuta el script desde el inicio; carga los paquetes. |
| could not find function "%>%"
<i>no se pudo encontrar la función "%>%"</i> | No cargaste el paquete (aquí dplyr o tidyverse), o escribiste mal la función (library). | <code>library(tidyverse)</code> al inicio del script; revisa la ortografía. |
| there is no package called 'paquete' | El paquete no está instalado (o está mal escrito). | <code>install.packages("paquete")</code> una vez; después <code>library(paquete)</code> . |
| unused argument (digitos = 2) | Nombre de argumento inexistente. | Consulta <code>?round</code> : el argumento se llama digits . |
| argument "x" is missing, with no default
<i>el argumento "x" está ausente, sin valor por omisión</i> | Llamaste a una función sin un argumento obligatorio. | Pásale el argumento o define un valor por omisión: <code>function(x = 0)</code> . |
| object of type 'closure' is not subsettable
<i>objeto de tipo 'closure' no es subconjunto</i> | Usaste <code>\$</code> o <code>[]</code> sobre una función: p. ej., <code>df\$total</code> cuando <code>df</code> no existe (y R encuentra la función <code>df()</code>). | Crea o carga el data frame antes; evita nombres de funciones (<code>df</code> , <code>data</code> , <code>c</code>). |
| no applicable method for 'mutate' applied to an object of class "character" | Le pasaste texto (p. ej., el nombre del archivo) en lugar del data frame. | Primero lee los datos: <code>ventas <- read_csv(...)</code> y luego <code>mutate(ventas, ...)</code> . |
| Sintaxis | | |
| unexpected symbol in "ventas total"
<i>unexpected symbol en "ventas total"</i> | Falta una coma, un operador o unas comillas entre dos nombres. | Revisa la línea señalada: <code>ventas\$total, c(a, b)...</code> |
| unexpected ')' in "sum(1, 2))"
<i>inesperado ')' en "sum(1, 2))"</i> | Paréntesis (o llaves) de más o de menos. | RStudio resalta la pareja de cada paréntesis; usa <code>Ctrl+I</code> para reindentar y verlo. |

| Mensaje (inglés / <i>español</i>) | Causa típica | Solución |
|---|---|--|
| unexpected end of input | Falta cerrar un paréntesis, llave o comilla. | Busca el (, { o " sin cerrar. Si la consola muestra +, pulsa Esc . |
| Tipos de datos | | |
| non-numeric argument to binary operator
<i>argumento no-numérico para operador binario</i> | Operación aritmética con texto: la columna se leyó como character (por "\$1,250" o comas). | parse_number() o as.numeric() ; revisa con glimpse() . |
| invalid 'type' (character) of argument
<i>'type' (character) de argumento no válido</i> | sum() sobre texto. | Convierte la columna a número. |
| argument is not numeric or logical: returning NA (advertencia) | mean() sobre texto. | Igual que el anterior. |
| NAs introduced by coercion (advertencia)
<i>NAs introducidos por coerción</i> | as.numeric() encontró valores que no son números ("diez", "N/D"). | Localízalos: x[is.na(as.numeric(x))] ; límpialos o usa parse_number() . |
| longer object length is not a multiple of shorter object length (advertencia)
<i>longitud de objeto mayor no es múltiplo de la longitud de uno menor</i> | Operaste vectores de distinta longitud: R "recicla" el corto. | Verifica length() de ambos; suele indicar un error de lógica. |
| Can't combine '..1\$x' <double> and '..2\$x' <character>. | bind_rows() , case_when() o if_else() con tipos distintos. | Iguala tipos: as.character() , o usa NA_character_ en lugar de NA . |
| Condiciones e indexación | | |
| missing value where TRUE/FALSE needed
<i>valor ausente donde TRUE/FALSE es necesario</i> | El if recibió un NA . | Protege con if (!is.na(x) && x > 5) . |
| the condition has length > 1 | if con un vector (desde R 4.2 es error). | Para columnas usa if_else() o case_when() ; para un valor, any() o all() . |
| argument is of length zero
<i>argumento tiene longitud cero</i> | El if recibió un vector vacío (p. ej., un filtro sin resultados). | Comprueba length(x) > 0 antes. |
| subscript out of bounds
<i>subíndice fuera de los límites</i> | Pediste un elemento que no existe (lista[[5]] con 3 elementos). | Revisa length() o names() . |
| \$ operator is invalid for atomic vectors | Usaste \$ sobre un vector. | Usa x["nombre"] , o verifica que el objeto sea un data frame. |
| undefined columns selected | df[, "col"] con un nombre que no existe. | Revisa names(df) . |
| arguments imply differing number of rows: 3, 2 | data.frame() con columnas de distinta longitud. | Todas las columnas deben medir lo mismo. |
| replacement has 2 rows, data has 3 | Asignaste a una columna un vector de longitud distinta. | Calcula la columna sobre el mismo data frame (con mutate()). |

dplyr y tidyr

| Mensaje (inglés / <i>español</i>) | Causa típica | Solución |
|--|---|--|
| Can't subset columns that don't exist.
x Column 'b' doesn't exist.
In argument: 'c = b * 2'.
Caused by error: ! object 'b' not found
We detected a named input.
This usually means that you've used '=' instead of '=='.
'b' must be size 3 or 1, not 2. | <code>select()</code> con un nombre mal escrito.

<code>mutate()</code> usa una columna inexistente o que se creó en otro paso sin guardar.

<code>filter(tienda_id = 3)</code> . | <code>names(df)</code> ; cuidado con acentos y mayúsculas.

Revisa el nombre y que asignaste el resultado previo con <code><-</code> .

Usa <code>==</code> para comparar:
<code>filter(tienda_id == 3)</code> . |
| 'n' must be a vector, not a function.
Must only be used inside data-masking verbs like 'mutate()', 'filter()', and 'group_by()'.
'summarise()' has grouped output by 'a'. You can override using the '.groups' argument. (mensaje) | En <code>mutate()</code> creaste una columna con una longitud distinta a la del data frame.
Escribiste <code>n</code> en lugar de <code>n()</code> . | Usa un valor único o un vector del mismo largo.

<code>summarise(n = n())</code> . |
| Can't join 'x\$id' with 'y\$id' due to incompatible types.
Join columns in 'y' must be present in the data.
Detected an unexpected many-to-many relationship between 'x' and 'y'. (advertencia) | Usaste <code>n()</code> fuera de un verbo de dplyr.

Agrupaste por dos o más columnas: el resultado sigue agrupado. | Úsalo dentro de <code>summarise()</code> o <code>mutate()</code> .

Añade <code>.groups = "drop"</code> . |
| Values from 'v' are not uniquely identified; output will contain list-cols. (advertencia) | La llave es número en una tabla y texto en la otra.
La columna del <code>by</code> no existe en la tabla derecha.
Llaves repetidas en ambas tablas: se duplican filas. | Convierte antes: <code>mutate(id = as.character(id))</code> .
Usa <code>by = c("id_tienda" = "tienda_id")</code> .
Revisa duplicados con <code>count(tabla, llave) %>% filter(n > 1)</code> . |
| ggplot2 | | |
| 'mapping' must be created by 'aes()'.
Did you use '%>%' or ' >' instead of '+'? | <code>pivot_wider()</code> con varias filas por celda. | Resume antes o usa <code>values_fn = sum</code> . |
| Cannot use '+' with a single argument
Did you accidentally put '+' on a new line? | Uniste capas con el pipe.

El <code>+</code> quedó al inicio de la línea siguiente. | Las capas se unen con <code>+</code> .

Deja el <code>+</code> al <i>final</i> de cada línea. |
| 'stat_count()' must only have an x or y aesthetic.
Discrete value supplied to a continuous scale
Aesthetics must be either length 1 or the same as the data (3) | <code>geom_bar()</code> con <code>y</code> .

Escala continua para una variable de texto o factor.
En <code>aes()</code> pusiste un vector externo de otra longitud. | Con valores ya calculados usa <code>geom_col()</code> .
Usa <code>scale_x_discrete()</code> o convierte la variable.
Mapea columnas del data frame, no vectores sueltos. |
| Archivos, fechas y modelos | | |
| 'ventas.csv' does not exist in current working directory ('...'). | <code>read_csv()</code> no encuentra el archivo: ruta o carpeta de trabajo incorrecta. | Abre el proyecto <code>.Rproj</code> ; revisa <code>getwd()</code> y <code>list.files()</code> . |

| Mensaje (inglés / <i>español</i>) | Causa típica | Solución |
|---|--|--|
| cannot open the connection + cannot open file 'ventas.csv': No such file or directory
<i>no se puede abrir la conexión + no fue posible abrir el archivo</i> | Lo mismo con <code>read.csv()</code> o <code>readRDS()</code> . | Igual que el anterior. |
| Sheet 'Ventas2024' not found | Nombre de hoja de Excel incorrecto. | Lista las hojas con <code>excel_sheets()</code> . |
| cannot change working directory
<i>no es posible cambiar el directorio de trabajo</i> | <code>setwd()</code> a una carpeta que no existe (o con \ en Windows). | Usa / en las rutas, o mejor, proyectos de RStudio. |
| All formats failed to parse. No formats found. (advertencia) | <code>ymd()</code> recibió fechas en otro orden o inválidas. | Usa la función del orden correcto (<code>dmy()</code> , <code>mdy()</code>). |
| factor factor(cyl) has new level 5 | <code>predict()</code> con una categoría que el modelo no vio. | Agrupar categorías raras o reentrenar con todas. |
| missing values in object | <code>randomForest()</code> (y otros) no aceptan NA. | <code>drop_na()</code> o imputa antes de entrenar. |

B.12 Glosario de BI y analítica

API (*Application Programming Interface*)

Puerta de acceso para que un programa pida datos a otro sistema (por ejemplo, un CRM o una tienda en línea), casi siempre en formato JSON.

Churn (tasa de abandono)

Porcentaje de clientes que dejan de comprar o cancelan en un periodo. Es el complemento de la retención.

Clave foránea

Columna que apunta a la clave primaria de otra tabla (`tienda_id` en ventas apunta a la tabla de tiendas). Es la base de los *joins*.

Clave primaria

Columna (o combinación) que identifica de forma única cada fila de una tabla, como `cliente_id`.

CLV / LTV (*Customer Lifetime Value*)

Valor (ingreso o margen) que se espera obtener de un cliente durante toda su relación con la empresa. Sirve para decidir cuánto invertir en adquirirlo o retenerlo.

Cohorte

Grupo de clientes que comparten un evento inicial en el mismo periodo (p. ej., “clientes registrados en enero”). El análisis de cohortes sigue su comportamiento en el tiempo.

Confianza

En reglas de asociación, probabilidad de comprar B dado que se compró A: $P(B | A)$.

Conjunto de entrenamiento y de prueba

Partición de los datos: el modelo aprende con el primero y se evalúa con el segundo, que no ha

visto.

Cross-selling (venta cruzada)

Ofrecer productos complementarios a lo que el cliente ya compra (funda para el celular).

Dashboard (tablero)

Pantalla que reúne los KPIs clave en gráficos y tablas para monitorear el negocio de un vistazo.

Data lake (lago de datos)

Repositorio que guarda grandes volúmenes de datos en su formato original (estructurados o no) para procesarlos después.

Data mart

Subconjunto de un data warehouse orientado a un área (ventas, finanzas, RR. HH.).

Data warehouse (almacén de datos)

Base de datos central, integrada e histórica, diseñada para análisis y reportes, no para operar el día a día.

Dimensión

Tabla con atributos descriptivos para analizar los hechos: tiempo, producto, tienda, cliente.

Drill-down / roll-up

Pasar de un nivel agregado a uno más detallado (año → mes → día) o al revés.

ELT

Extract, Load, Transform: primero se cargan los datos crudos en el almacén y ahí se transforman. Común en plataformas en la nube.

Esquema estrella

Modelo de data warehouse con una tabla de hechos en el centro unida a varias tablas de dimensiones.

Estacionalidad

Patrón que se repite en periodos fijos (diciembre alto, enero bajo; fines de semana).

ETL

Extract, Transform, Load: extraer datos de las fuentes, limpiarlos y transformarlos, y cargarlos en el destino. Con R: leer, **dplyr**, escribir.

Feature (variable predictora)

Columna que el modelo usa como entrada para predecir (antigüedad, número de compras, ticket promedio).

Forecast (pronóstico)

Estimación de valores futuros de una serie (ventas, demanda), idealmente con un intervalo de incertidumbre.

Granularidad

Nivel de detalle de una tabla: una fila por transacción, por día o por tienda-mes. Antes de unir o sumar, confirma la granularidad.

Intervalo de confianza

Rango de valores plausibles para un parámetro (p. ej., ticket promedio entre \$1,300 y \$1,500 con 95 % de confianza).

Join (unión)

Combinar dos tablas usando columnas llave comunes.

KPI (*Key Performance Indicator*)

Indicador clave de desempeño: métrica ligada a un objetivo del negocio, con meta y responsable.

Lift

En reglas de asociación, cuántas veces más probable es comprar B cuando se compra A que en general ($\text{lift} > 1$ = asociación positiva). En modelos, cuánto mejor que el azar concentra a los positivos.

Margen bruto

Ingresos menos costo de lo vendido, a menudo expresado como porcentaje de los ingresos.

Matriz de confusión

Tabla que cruza clases reales y predichas; de ella salen exactitud, precisión y recall.

OLAP

Online Analytical Processing: sistemas y consultas para analizar datos agregados desde varias dimensiones (cubos, tablas dinámicas).

OLTP

Online Transaction Processing: sistemas que registran operaciones del día a día (punto de venta, ERP). Optimizados para escribir, no para analizar.

Outlier (valor atípico)

Observación muy alejada del resto; puede ser un error de captura o un caso real importante.

Overfitting (sobreajuste)

Cuando un modelo memoriza los datos de entrenamiento y falla con datos nuevos.

p-valor

Probabilidad de observar un resultado tan extremo como el obtenido si no hubiera efecto real. Valores pequeños (< 0.05) indican evidencia contra la hipótesis nula.

Pipeline

Cadena de pasos automatizados (extraer, limpiar, modelar, reportar). En R, también la cadena de funciones unidas con `%>%`.

Prueba A/B

Experimento que compara dos versiones (de un correo, una página, un precio) asignadas al azar para medir cuál funciona mejor.

Reproducibilidad

Capacidad de obtener los mismos resultados al volver a ejecutar el análisis: scripts en lugar de clics, semillas (`set.seed()`), datos y versiones documentados.

Retención

Porcentaje de clientes que siguen comprando de un periodo al siguiente.

RFM

Segmentación por *Recency* (hace cuánto compró), *Frequency* (cuántas veces) y *Monetary* (cuánto gastó).

Segmentación

División de clientes (o productos) en grupos homogéneos para tratarlos de forma diferenciada.

Serie de tiempo

Secuencia de valores medidos a intervalos regulares (ventas diarias, mensuales).

Soporte

En reglas de asociación, proporción de tickets que contienen un conjunto de productos.

SQL

Structured Query Language: lenguaje estándar para consultar y administrar bases de datos relacionales.

Tabla de hechos

Tabla central de un esquema estrella con los eventos medibles (ventas, unidades, montos) y las llaves a las dimensiones.

Tasa de conversión

Proporción de visitantes o prospectos que realizan la acción deseada (comprar, registrarse).

Ticket promedio

Monto promedio por transacción: ventas totales entre número de tickets.

Tidy data (datos ordenados)

Formato en el que cada variable es una columna, cada observación una fila y cada valor una celda. Es el formato que esperan dplyr y ggplot2.

Up-selling

Ofrecer una versión superior o más cara de lo que el cliente piensa comprar.

Validación cruzada

Técnica que divide los datos en k partes, entrena k veces dejando una parte fuera para evaluar, y promedia el desempeño. Da una estimación más estable que una sola partición.

Variable objetivo

Lo que el modelo quiere predecir (churn sí/no, ventas del mes). También se llama variable respuesta o dependiente.

B.13 Recursos para seguir aprendiendo

Todos los recursos de esta lista son gratuitos en línea, salvo que se indique lo contrario.

Tabla B.30. Libros, documentación y comunidades recomendados.

| Recurso | Para qué | Dirección |
|---|---|--|
| Libros | | |
| <i>R for Data Science</i> (2. ^a ed.), H. Wickham, M. Çetinkaya-Rundel y G. Grolemund | La referencia del tidyverse: importar, transformar, visualizar, comunicar. | https://r4ds.hadley.nz |
| <i>R para Ciencia de Datos</i> (traducción) | Versión en español del libro anterior. | https://es.r4ds.hadley.nz |
| <i>R Graphics Cookbook</i> , W. Chang | Recetas para resolver cualquier gráfico con ggplot2. | https://r-graphics.org |
| <i>ggplot2: Elegant Graphics for Data Analysis</i> , H. Wickham | La gramática de gráficos a fondo. | https://ggplot2-book.org |
| <i>Mastering Shiny</i> , H. Wickham | Aplicaciones Shiny, de lo básico a producción. | https://mastering-shiny.org |
| <i>Forecasting: Principles and Practice</i> , R. J. Hyndman y G. Athanasopoulos | Pronósticos: 3. ^a ed. con fable ; 2. ^a ed. con forecast (el paquete del curso). | https://otexts.com/fpp3
https://otexts.com/fpp2 |
| <i>Tidy Modeling with R</i> , M. Kuhn y J. Silge | Machine learning moderno con tidymodels . | https://www.tmwr.org |
| <i>R Markdown: The Definitive Guide</i> , Y. Xie y otros | Reportes, documentos y presentaciones con R Markdown. | https://bookdown.org/yihui/r-markdown |
| Documentación y chuletas | | |
| Cheatsheets de Posit | Hojas de referencia de dplyr, ggplot2, tidyr, lubridate, Shiny, R Markdown... (varias traducidas al español). | https://posit.co/resources/cheatsheets/ |
| Sitio del tidyverse | Documentación y ejemplos de cada paquete. | https://www.tidyverse.org |
| Referencia de dplyr y ggplot2 | Todas las funciones con ejemplos. | https://dplyr.tidyverse.org
https://ggplot2.tidyverse.org |
| Quarto | Sucesor de R Markdown para documentos y sitios. | https://quarto.org |
| CRAN Task Views | Paquetes organizados por tema (series de tiempo, ML...). | https://cran.r-project.org/web/views/ |
| Comunidades | | |
| R-Ladies | Organización mundial que promueve la diversidad en R; hay capítulos en muchas ciudades de habla hispana. | https://rladies.org |
| Comunidad R Hispano | Asociación de usuarios de R en español. | https://r-es.org |
| LatinR | Conferencia latinoamericana de R. | https://latin-r.com |
| Stack Overflow en español | Preguntas y respuestas de programación en español. | https://es.stackoverflow.com |
| Stack Overflow (etiqueta R) | La mayor base de preguntas resueltas sobre R (en inglés). | https://stackoverflow.com/questions/tagged/r |
| Posit Community | Foro oficial de RStudio / Posit. | https://forum.posit.co |

En la empresa: Rutas de carrera en BI y analítica

Lo que aprendiste en este libro es la base de varios puestos, que suelen recorrerse en este orden (aunque no hay un único camino):

- **Analista de datos / analista de BI:** SQL, Excel, una herramienta de visualización y un lenguaje como R; construye reportes y dashboards y responde preguntas del negocio.
- **Ingeniero de analítica o de datos:** automatiza la extracción y transformación de datos (ETL/ELT), modela el data warehouse y cuida la calidad de los datos.
- **Científico de datos:** estadística y machine learning para pronosticar, segmentar y optimizar decisiones.
- **Líder o gerente de analítica:** define KPIs, prioriza proyectos y traduce los hallazgos en decisiones.

En cuanto a certificaciones, existen tres familias útiles: las de *proveedores de herramientas de BI* (por ejemplo, Microsoft Power BI o Tableau), las de *plataformas de datos en la nube* (AWS, Google Cloud, Microsoft Azure) y los programas de análisis de datos y ciencia de datos de universidades y plataformas de cursos en línea. Consulta el sitio oficial de cada proveedor para conocer requisitos y contenidos vigentes. Aun así, lo que más pesa en una entrevista suele ser un **portafolio**: publica en GitHub proyectos como el del Módulo 9, con código limpio, un reporte en R Markdown y un dashboard en Shiny.

Consejo

La mejor manera de no olvidar lo aprendido es usarlo cada semana: toma un reporte que hoy haces a mano en Excel y automatízalo con un script de R. Cuando funcione, conviértelo en un R Markdown parametrizado. Cuando lo pidan otras áreas, conviértelo en un dashboard. Esta referencia rápida estará aquí cada vez que necesites recordar un detalle.